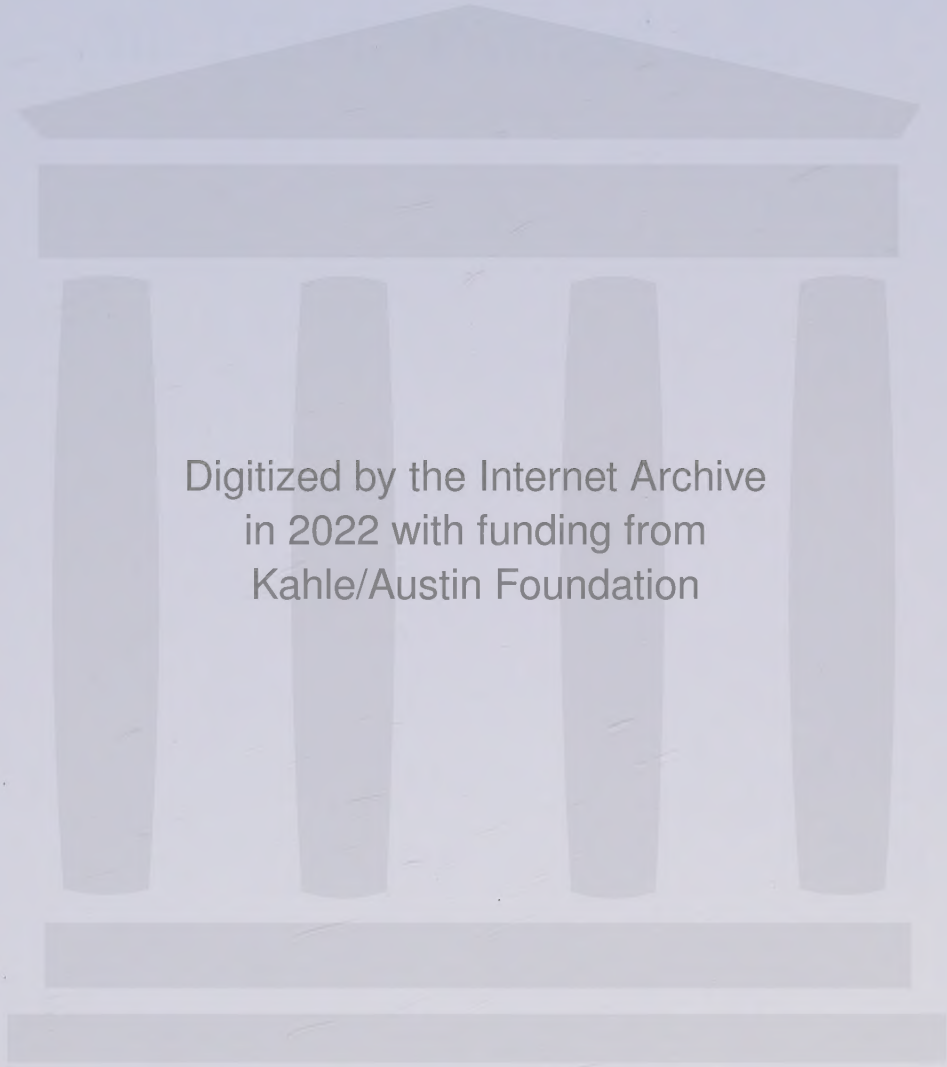
SIXTH EDITION

DATA COMMUNICATIONS & NETWORKING

WITH TCP/IP PROTOCOL SUITE

Behrouz A. Forouzan

Mc Graw Hill



Digitized by the Internet Archive
in 2022 with funding from
Kahle/Austin Foundation

https://archive.org/details/datacommunicatio0000foro_i7v4

Data Communications and Networking with TCP/IP Protocol Suite

Data Communications and Networking with TCP/IP Protocol Suite

SIXTH EDITION

Behrouz A. Forouzan

Mc Graw Hill

DATA COMMUNICATIONS AND NETWORKING WITH TCP/IP PROTOCOL SUITE, SIXTH EDITION

Published by McGraw Hill LLC, 1325 Avenue of the Americas, New York, NY 10121. Copyright ©2022 by McGraw Hill LLC. All rights reserved. Printed in the United States of America. Previous editions ©2013, 2007, and 2004. No part of this publication may be reproduced or distributed in any form or by any means, or stored in a database or retrieval system, without the prior written consent of McGraw Hill LLC, including, but not limited to, in any network or other electronic storage or transmission, or broadcast for distance learning.

Some ancillaries, including electronic and print components, may not be available to customers outside the United States.

This book is printed on acid-free paper.

1 2 3 4 5 6 7 8 9 LCR 26 25 24 23 22 21

ISBN 978-0-07-802209-8 (bound edition)
MHID 0-07-802209-6 (bound edition)
ISBN 978-1-260-72310-6 (loose-leaf edition)
MHID 1-260-72310-0 (loose-leaf edition)

Portfolio Manager: *Beth Bettcher*
Product Developer: *Theresa Collins*
Senior Marketing Manager: *Shannon O'Donnell*
Content Project Manager: *Jane Mohr*
Buyer: *Susan K. Culbertson*
Designer: *David W. Hash*
Content Licensing Specialist: *Lorraine Buczek*
Cover Image: *Ingram Publishing/SuperStock*
Compositor: *MPS Limited*

All credits appearing on page or at the end of the book are considered to be an extension of the copyright page.

Library of Congress Cataloging-in-Publication Data

Names: Forouzan, Behrouz A., author.
Title: Data communications and networking with TCP/IP protocol suite / Behrouz A. Forouzan.
Other titles: Data communications and networking
Description: Sixth edition. | New York, NY : McGraw Hill, 2022 | Series: McGraw-Hill Forouzan networking series | Revised edition of: Data communications and networking. | Includes bibliographical references and index.
Identifiers: LCCN 2020052158 | ISBN 9780078022098 (hardcover; acid-free paper) | ISBN 9781260723106 (spiral bound; acid-free paper) | ISBN 9781260407976 (epub)
Subjects: LCSH: Data transmission systems. | Computer networks. | TCP/IP (Computer network protocol)
Classification: LCC TK5105 .F6617 2021 | DDC 004.6—dc23
LC record available at https://lccn.loc.gov/2020052158

The Internet addresses listed in the text were accurate at the time of publication. The inclusion of a website does not indicate an endorsement by the authors or McGraw Hill LLC, and McGraw Hill LLC does not guarantee the accuracy of the information presented at these sites.

mheducation.com/highered

To my beloved daughter.

BRIEF CONTENTS

Preface xix

Trademark xxv

Chapter 1 *Introduction* 1

Chapter 2 *Physical Layer* 29

Chapter 3 *Data-Link Layer* 63

Chapter 4 *Local Area Networks: LANs* 115

Chapter 5 *Wide Area Networks: WANs* 149

Chapter 6 *Connecting Devices and Virtual LANs* 187

Chapter 7 *Network Layer: Data Transfer* 203

Chapter 8 *Network Layer: Routing of Packets* 285

Chapter 9 *Transport Layer* 341

Chapter 10 *Application Layer* 437

Chapter 11 *Multimedia* 543

Chapter 12 *Network Management* 605

Chapter 13 *Cryptography and Network Security* 637

Appendices

Appendix A *Unicode* 713

Appendix B *Positional Numbering System* 719

Appendix C *HTML, CSS, XML, and XSL* 727

Appendix D *A Touch of Probability* 737

Appendix E *Checksum* 743

Appendix F *Acronyms* 751

Glossary 761

References 805

Index 811

CONTENTS

Preface xix

Trademark xxv

Chapter 1 *Introduction* 1

1.1 DATA COMMUNICATIONS 2
- 1.1.1 Components 2
- 1.1.2 Message 3
- 1.1.3 Data Flow 4

1.2 NETWORKS 5
- 1.2.1 Network Criteria 5
- 1.2.2 Physical Structures 5

1.3 NETWORK TYPES 8
- 1.3.1 Local Area Network 8
- 1.3.2 Wide Area Network (WAN) 8
- 1.3.3 The Internet 10
- 1.3.4 Accessing the Internet 12

1.4 PROTOCOL LAYERING 13
- 1.4.1 Scenarios 13
- 1.4.2 Principles of Protocol Layering 16
- 1.4.3 Logical Connections 16

1.5 TCP/IP PROTOCOL SUITE 17
- 1.5.1 Layered Architecture 17
- 1.5.2 Brief Description of Layers 18
- 1.5.3 Description of Each Layer 20

1.6 THE OSI MODEL 21
- 1.6.1 OSI versus TCP/IP 21
- 1.6.2 Lack of OSI Model's Success 22

1.7 END-OF-CHAPTER MATERIALS 23
- 1.7.1 Recommended Reading 23
- 1.7.2 Key Terms 23
- 1.7.3 Summary 23

1.8 PRACTICE SET 24
- 1.8.1 Quizzes 24
- 1.8.2 Questions 24
- 1.8.3 Problems 26

Chapter 2 *Physical Layer* *29*

2.1 SIGNALS 31
2.1.1 Analog Signals 31
2.1.2 Digital Signals 33
2.2 SIGNAL IMPAIRMENT 35
2.2.1 Attenuation and Amplification 35
2.2.2 Distortion 35
2.2.3 Data Rate Limits 36
2.2.4 Performance 38
2.3 DIGITAL TRANSMISSION 40
2.3.1 Digital-to-Digital Conversion 40
2.3.2 Analog-to-Digital Conversion 41
2.4 ANALOG TRANSMISSION 42
2.4.1 Digital-to-Analog Conversion 42
2.4.2 Analog-to-Analog Conversion 45
2.5 MULTIPLEXING 47
2.5.1 Frequency-Division Multiplexing 48
2.5.2 Time-Division Multiplexing 48
2.6 TRANSMISSION MEDIA 49
2.6.1 Guided Media 50
2.6.2 Unguided Media: Wireless 53
2.7 END-OF-CHAPTER MATERIALS 55
2.7.1 Recommended Reading 55
2.7.2 Key Terms 55
2.7.3 Summary 55
2.8 PRACTICE SET 56
2.8.1 Quizzes 56
2.8.2 Questions 56
2.8.3 Problems 58

Chapter 3 *Data-Link Layer* *63*

3.1 INTRODUCTION 64
3.1.1 Nodes and Links 65
3.1.2 Two Types of Links 65
3.1.3 Two Sublayers 66
3.2 DATA-LINK CONTROL 66
3.2.1 Framing 66
3.2.2 Error Control 70
3.2.3 Two DLC Protocols 80
3.3 MEDIA ACCESS PROTOCOLS 88
3.3.1 Random Access 88
3.3.2 Controlled Access 101

3.4 LINK-LAYER ADDRESSING 104
3.4.1 Three Types of Addresses 106
3.4.2 Address Resolution Protocol (ARP) 107
3.5 END-OF-CHAPTER MATERIALS 107
3.5.1 Recommended Reading 107
3.5.2 Key Terms 107
3.5.3 Summary 108
3.6 PRACTICE SET 108
3.6.1 Quizzes 108
3.6.2 Questions 109
3.6.3 Problems 110

Chapter 4 *Local Area Networks: LANs 115*

4.1 ETHERNET 116
4.1.1 Standard Ethernet (10 Mbps) 117
4.1.2 Fast Ethernet (100 Mbps) 121
4.1.3 Gigabit Ethernet (1000 Mbps) 123
4.1.4 10 Gigabit Ethernet 126
4.2 WIFI, IEEE 802.11 PROJECT 126
4.2.1 Architecture 127
4.2.2 MAC Sublayer 128
4.2.3 Addressing Mechanism 133
4.2.4 Physical Layer 135
4.3 BLUETOOTH 138
4.3.1 Architecture 138
4.3.2 Bluetooth Layers 140
4.4 END-OF-CHAPTER MATERIALS 145
4.4.1 Recommended Reading 145
4.4.2 Key Terms 145
4.4.3 Summary 146
4.5 PRACTICE SET 146
4.5.1 Quizzes 146
4.5.2 Questions 146
4.5.3 Problems 147

Chapter 5 *Wide Area Networks: WANs 149*

5.1 TELEPHONE NETWORKS 150
5.1.1 Major Components 150
5.1.2 LATAs 151
5.1.3 Signaling 152
5.1.4 Services Provided by Telephone Networks 155
5.1.5 Dial-Up Service 156
5.1.6 Digital Subscriber Line (DSL) 158

5.2 CABLE NETWORKS 159
5.2.1 Traditional Cable Networks 160
5.2.2 Hybrid Fiber-Coaxial (HFC) Network 160
5.2.3 Cable TV for Data Transfer 161
5.3 CELLULAR TELEPHONY 162
5.3.1 Operation 163
5.3.2 First Generation (1G) 165
5.3.3 Second Generation (2G) 166
5.3.4 Third Generation (3G) 173
5.3.5 Fourth Generation (4G) 174
5.4 SATELLITE NETWORK 175
5.4.1 Operation 175
5.4.2 GEO Satellites 178
5.4.3 MEO Satellites 178
5.4.4 LEO Satellites 181
5.5 END–OF–CHAPTER MATERIALS 182
5.5.1 Recommended Reading 182
5.5.2 Key Terms 182
5.5.3 Summary 183
5.6 PRACTICE SET 184
5.6.1 Quizzes 184
5.6.2 Questions 184
5.6.3 Problems 185

Chapter 6 *Connecting Devices and Virtual LANs 187*

6.1 CONNECTING DEVICES 188
6.1.1 Hubs 188
6.1.2 Link-Layer Switches 189
6.1.3 Routers 195
6.2 VIRTUAL LANS 196
6.2.1 Membership 198
6.2.2 Configuration 198
6.2.3 Communication among Switches 199
6.2.4 Advantages 199
6.3 END-OF-CHAPTER MATERIALS 200
6.3.1 Recommended Reading 200
6.3.2 Key Terms 200
6.3.3 Summary 200
6.4 PRACTICE SET 201
6.4.1 Quizzes 201
6.4.2 Questions 201
6.4.3 Problems 201

Chapter 7 *Network Layer: Data Transfer* 203

7.1 SERVICES 205
- 7.1.1 Packetizing 205
- 7.1.2 Routing 205
- 7.1.3 Error Control 205
- 7.1.4 Flow Control 205
- 7.1.5 Congestion Control 206
- 7.1.6 Quality of Service 206
- 7.1.7 Security 206

7.2 PACKET SWITCHING 206
- 7.2.1 Datagram Approach: Connectionless Service 207
- 7.2.2 Virtual-Circuit Approach: Connection-Oriented Service 207

7.3 PERFORMANCE 207
- 7.3.1 Delay 208
- 7.3.2 Throughput 209
- 7.3.3 Packet Loss 210

7.4 INTERNET PROTOCOL VERSION 4 210
- 7.4.1 IPv4 Addressing 210
- 7.4.2 Main and Auxiliary Protocols 219
- 7.4.3 Options 229
- 7.4.4 ICMPv4 231
- 7.4.5 Mobile IP 237
- 7.4.6 Forwarding of IP Packets 247

7.5 NEXT GENERATION IP (IPV6) 256
- 7.5.1 IPv6 Addressing 257
- 7.5.2 The IPv6 Protocol 264
- 7.5.3 The ICMPv6 Protocol 269

7.6 TRANSITION FROM IPV4 TO IPV6 273

7.7 END-OF-CHAPTER MATERIALS 275
- 7.7.1 Recommended Reading 275
- 7.7.2 Key Terms 275
- 7.7.3 Summary 276

7.8 PRACTICE SET 276
- 7.8.1 Quizzes 276
- 7.8.2 Questions 277
- 7.8.3 Problems 279

Chapter 8 *Network Layer: Routing of Packets* 285

8.1 INTRODUCTION 286
- 8.1.1 General Idea 286
- 8.1.2 Least-Cost Routing 286

8.2 ROUTING ALGORITHMS 288
8.2.1 Distance-Vector Routing 288
8.2.2 Link-State Routing 294
8.2.3 Path-Vector Routing 297
8.3 UNICAST ROUTING PROTOCOLS 301
8.3.1 Internet Structure 301
8.3.2 Routing Information Protocol (RIP) 303
8.3.3 Open Shortest Path First (OSPF) 308
8.3.4 Border Gateway Protocol Version 4 (BGP4) 313
8.4 MULTICAST ROUTING 322
8.4.1 Unicasting 322
8.4.2 Multicasting 323
8.4.3 Distance Vector Multicast Routing Protocol 324
8.4.4 Multicast Open Shortest Path First 327
8.4.5 Protocol Independent Multicast (PIM) 327
8.5 IGMP 331
8.5.1 Messages 331
8.5.2 Propagation of Membership Information 332
8.5.3 Encapsulation 333
8.6 END-OF-CHAPTER MATERIALS 333
8.6.1 Recommended Reading 333
8.6.2 Key Terms 333
8.6.3 Summary 334
8.7 PRACTICE SET 335
8.7.1 Quizzes 335
8.7.2 Questions 335
8.7.3 Problems 337

Chapter 9 *Transport Layer 341*

9.1 TRANSPORT-LAYER SERVICES 342
9.1.1 Process-to-Process Communication 342
9.1.2 Addressing: Port Numbers 343
9.1.3 Encapsulation and Decapsulation 345
9.1.4 Multiplexing and Demultiplexing 346
9.1.5 Flow Control 346
9.1.6 Error Control 349
9.1.7 Combination of Flow and Error Control 350
9.1.8 Congestion Control 352
9.1.9 Connectionless and Connection-Oriented Protocols 352
9.2 TRANSPORT-LAYER PROTOCOLS 356
9.2.1 Services 356
9.2.2 Port Numbers 357

9.3 USER DATAGRAM PROTOCOL (UDP) 358
9.3.1 UDP Services 359
9.3.2 UDP Applications 362
9.4 TRANSMISSION CONTROL PROTOCOL 363
9.4.1 TCP Services 364
9.4.2 TCP Features 367
9.4.3 Segment 368
9.4.4 A TCP Connection 371
9.4.5 State Transition Diagram 378
9.4.6 Windows in TCP 380
9.4.7 Flow Control 383
9.4.8 Error Control 389
9.4.9 TCP Congestion Control 398
9.4.10 TCP Timers 408
9.4.11 Options 412
9.5 SCTP 412
9.5.1 SCTP Services 412
9.5.2 SCTP Features 414
9.5.3 Packet Format 416
9.5.4 An SCTP Association 418
9.5.5 Flow Control 421
9.5.6 Error Control 423
9.6 END-OF-CHAPTER MATERIALS 427
9.6.1 Recommended Reading 427
9.6.2 Key Terms 427
9.6.3 Summary 428
9.7 PRACTICE SET 429
9.7.1 Quizzes 429
9.7.2 Questions 429
9.7.3 Problems 432

Chapter 10 *Application Layer 437*

10.1 INTRODUCTION 438
10.1.1 Providing Services 439
10.1.2 Application-Layer Paradigms 440
10.2 CLIENT/SERVER PARADIGM 443
10.2.1 Application Programming Interface 443
10.2.2 Using Services of the Transport Layer 447
10.3 STANDARD APPLICATIONS 448
10.3.1 World Wide Web and HTTP 449
10.3.2 FTP 464
10.3.3 Electronic Mail 468
10.3.4 TELNET 481

10.3.5 Secure Shell (SSH) 484
10.3.6 Domain Name System (DNS) 486
10.4 PEER-TO-PEER PARADIGM 498
10.4.1 P2P Networks 498
10.4.2 Distributed Hash Table (DHT) 500
10.4.3 Chord 503
10.4.4 Pastry 510
10.4.5 Kademlia 515
10.4.6 A Popular P2P Network: BitTorrent 518
10.5 SOCKET INTERFACE PROGRAMMING 521
10.5.1 Data Structure for Socket 521
10.5.2 Header Files 522
10.5.3 Iterative Communication Using UDP 522
10.5.4 Communication Using TCP 528
10.6 END-OF-CHAPTER MATERIALS 535
10.6.1 Recommended Reading 535
10.6.2 Key Terms 536
10.6.3 Summary 536
10.7 PRACTICE SET 537
10.7.1 Quizzes 537
10.7.2 Questions 537
10.7.3 Problems 539

Chapter 11 *Multimedia 543*

11.1 COMPRESSION 544
11.1.1 Lossless Compression 544
11.1.2 Lossy Compression 554
11.2 MULTIMEDIA DATA 560
11.2.1 Text 560
11.2.2 Image 560
11.2.3 Video 564
11.2.4 Audio 566
11.3 MULTIMEDIA IN THE INTERNET 568
11.3.1 Streaming Stored Audio/Video 568
11.3.2 Streaming Live Audio/Video 571
11.3.3 Real-Time Interactive Audio/Video 572
11.4 REAL-TIME INTERACTIVE PROTOCOLS 577
11.4.1 Rationale for New Protocols 578
11.4.2 RTP 581
11.4.3 RTCP 583
11.4.4 Session Initialization Protocol (SIP) 587
11.4.5 H.323 594

11.5 END-OF-CHAPTER MATERIALS 597
11.5.1 Recommended Reading 597
11.5.2 Key Terms 597
11.5.3 Summary 597
11.6 PRACTICE SET 598
11.6.1 Quizzes 598
11.6.2 Questions 598
11.6.3 Problems 600

Chapter 12 *Network Management 605*

12.1 INTRODUCTION 606
12.1.1 Configuration Management 606
12.1.2 Fault Management 608
12.1.3 Performance Management 609
12.1.4 Security Management 609
12.1.5 Accounting Management 610
12.2 SNMP 610
12.2.1 Managers and Agents 611
12.2.2 Management Components 611
12.2.3 An Overview 613
12.2.4 SMI 614
12.2.5 MIB 618
12.2.6 SNMP Operation 622
12.3 ASN.1 627
12.3.1 Language Basics 628
12.3.2 Data Types 629
12.3.3 Encoding 632
12.4 END-OF-CHAPTER MATERIALS 632
12.4.1 Recommended Reading 632
12.4.2 Key Terms 632
12.4.3 Summary 632
12.5 PRACTICE SET 633
12.5.1 Quizzes 633
12.5.2 Questions 633
12.5.3 Problems 634

Chapter 13 *Cryptography and Network Security 637*

13.1 INTRODUCTION 638
13.1.1 Security Goals 638
13.1.2 Attacks 639
13.1.3 Services and Techniques 641
13.2 CONFIDENTIALITY 641
13.2.1 Symmetric-Key Ciphers 641
13.2.2 Asymmetric-Key Ciphers 653

13.3 OTHER ASPECTS OF SECURITY 658
13.3.1 Message Integrity 658
13.3.2 Message Authentication 659
13.3.3 Digital Signature 660
13.3.4 Entity Authentication 666
13.3.5 Key Management 668
13.4 NETWORK-LAYER SECURITY 674
13.4.1 Two Modes 675
13.4.2 Two Security Protocols 676
13.4.3 Services Provided by IPSec 680
13.4.4 Security Association 680
13.4.5 Internet Key Exchange (IKE) 684
13.4.6 Virtual Private Network (VPN) 684
13.5 TRANSPORT-LAYER SECURITY 685
13.5.1 SSL Architecture 686
13.5.2 Four Protocols 689
13.6 APPLICATION-LAYER SECURITY 691
13.6.1 E-mail Security 691
13.6.2 Pretty Good Privacy (PGP) 693
13.6.3 S/MIME 698
13.7 FIREWALLS 702
13.7.1 Packet-Filter Firewall 703
13.7.2 Proxy Firewall 704
13.8 END-OF-CHAPTER MATERIALS 705
13.8.1 Recommended Reading 705
13.8.2 Key Terms 705
13.8.3 Summary 706
13.9 PRACTICE SET 707
13.9.1 Quizzes 707
13.9.2 Questions 707
13.9.3 Problems 709

Appendices

Appendix A *Unicode 713*
Appendix B *Positional Numbering System 719*
Appendix C *HTML, CSS, XML, and XSL 727*
Appendix D *A Touch of Probability 737*
Appendix E *Checksum 743*
Appendix F *Acronyms 751*

Glossary 761
References 805
Index 811

PREFACE

Welcome to the sixth edition of *Data Communications and Networking with TCP/IP Protocol Suite*. We are living in an information age, and information is distributed faster that ever using the Internet, which works based on the topics discussed in this book.

Features

Although the main goal of this book is to teach the principles of networking, it is designed to teach these principles using the following features:

TCP/IP Protocol Suite

This book is designed to teach the principles of networking by using the TCP/IP protocol suite. Teaching these principles using protocol layering is beneficial because these principles are repeated and better understood in relation to each layer. For example, *addressing* is an issue that is applied to several layers of the TCP/IP protocol suite. Another example is *framing and packetizing*, which is repeated in several layers, but each layer treats the principle differently.

Bottom-Up Approach

This book uses a bottom-up approach. Each layer in the TCP/IP protocol suite is built on the services provided by the layer below. We learn how bits are moving at the physical layer (first layer) before learning how some programs exchange messages at the application layer (fifth layer).

Organization

The book is made up of 13 chapters, six appendices, a list of references, and a glossary.

Chapter 1: Introduction

This chapter is an introduction to *Data Communications and Networking with TCP/IP Protocol Suite*. It defines the concept of protocol layering and gives a brief description of the TCP/IP protocol suite and the OSI model.

Chapter 2: Physical Layer

This chapter describes the first layer of the TCP/IP protocol suite: the physical layer. It explains the relationship between data and signals and describes both analog and digital signals. It also discusses multiplexing to benefit from the available bandwidth. Finally, it goes below the physical layer and discusses the transmission media.

Chapter 3: Data-Link Layer

This chapter discusses the data-link layer, the second layer in the TCP/IP protocol suite. It shows that the data-link layer is made up of two sublayers: medial link control and data link control. It also discusses link-layer addressing.

Chapter 4: Local Area Networks: LANs

This chapter discusses the local area networks (LANs) that use only the first two layers of the TCP/IP protocol suite. It describes both wired LANs (Ethernet) and wireless LANs (WiFi and Bluetooth).

Chapter 5: Wide Area Networks: WANs

This chapter discusses the wide area networks (WANs) that also use only the first two layers of the TCP/IP protocol suite. It describes several WANs, including the telephone network, cable network, cellular telephony, and satellite networks.

Chapter 6: Connecting Devices and Virtual LANs

This chapter discusses the connecting devices such as hubs, link-layer switches, and routers. It also describes virtual LANs.

Chapter 7: Network Layer: Data Transfer

This chapter discuses the first duty of the network layer: data transfer. It explains the service in this duty such as packetizing, routing, error control, flow control, congestion control, and quality of services. It then describes the concept of packet switching. It also describe network-layer performance. The main goal is to introduce the two versions of the network layer in the Internet: IPv4 and IPv6.

Chapter 8: Network Layer: Routing Packets

This chapter discusses the second duty of the network layer: routing of packets. It discusses unicast routing protocols such as distance vector routing, link-state routing, and path-vector routing. It also discuss multicast routing and protocols.

Chapter 9: Transport Layer

This chapter discusses the transport layer. It first describes the services expected from a transfer-layer protocol. It then describes a simple transport layer protocol UDP. Finally, it describes a more sophisticated protocol TCP. Finally, it describes SCTP, a transport-layer protocol that uses association.

Chapter 10: Application Layer

This chapter discusses the application layer, the highest level in the TCP/IP protocol suite. It shows how this layer uses client/server programs. It then introduces some applications such as the Web, file transfer, and e-mail. Finally, the chapter discusses some peer-to-peer applications. It finally shows how application programs can be created using the C-language.

Chapter 11: Multimedia

This chapter discusses multimedia. It shows how compression is used in multimedia. It then defines the elements of multimedia such as text, image, video, and audio. It then describes how multimedia is used in the Internet.

Chapter 12: Network Management

This chapter introduces network management and discusses five general areas used in network management. It also defines the Simple Network Management Protocol (SNMP) that is used in the Internet, which is based on Simple Management Information (SMI).

Chapter 13: Cryptography and Network Security

This chapter briefly discusses the concept of security goals including confidentiality, integrity, and availability. It then describes how these goals can be achieved using message integrity, message authentication, digital signature, and entity authentication. The chapter then describes how these goals can be achieved using security in the transport layer and application layer.

Appendix A

This appendix discusses Unicode, the coding system used in communication.

Appendix B

This appendix discusses the positional numbering system and how the system uses numbers in different bases.

Appendix C

This appendix discusses mark-up languages such as HTML, CSS, XML, and XSL, which are used in data communications and networking.

Appendix D

This appendix gives a touch of probability that can be useful in understanding some networking protocols.

Appendix E

This appendix discusses checksum.

Appendix F

This appendix gives the list of acronyms used in the book for quick reference.

References

The book contains a list of references for further reading.

Glossary

The Glossary provides definitions for all key terms from the text and other important terminology.

Pedagogy

Several pedagogical features of this text are designed to make it particularly easy for students to understand data communications and networking.

Visual Approach

The book presents highly technical subject matter without complex formulas by using a balance of text and figures. More than 500 figures accompanying the text provide a visual and intuitive opportunity for understanding the material. Figures are particularly important in explaining networking concepts. For many students, these concepts are more easily grasped visually than verbally.

Highlighted Points

The book repeats important concepts in boxes for quick reference and immediate attention.

Examples and Applications

Whenever appropriate, examples illustrate the concepts introduced in the text. Also, some real-life applications provided throughout each chapter help motivate students.

End-of-Chapter Materials

Each chapter ends with a set of materials that includes the following:

Key Terms

The new terms used in each chapter are listed at the end of the chapter, and their definitions are included in the glossary.

Summary

Each chapter ends with a summary of the material covered by that chapter. The summary glues the important materials together to be seen in one shot.

Recommended Reading

This section gives a brief list of references relative to the chapter. The references can be used to quickly find the corresponding literature in the reference section at the end of the book.

Practice Set

Each chapter includes a practice set designed to reinforce salient concepts and encourage students to apply them. It consists of three parts: quizzes, questions, and problems.

Quizzes

Quizzes, which are posted on the book website, provide quick concept checking. Students can take these quizzes to check their understanding of the materials. Students receive feedback regarding their responses immediately.

Questions

This section contains simple questions about the concepts discussed in the book. Answers to the odd-numbered questions are posted on the book website to be checked by the student. There are more than 630 end-of-chapter questions.

Problems

This section contains more difficult problems that need a deeper understanding of the materials discussed in the chapter. I strongly recommend that students try to solve all of these problems. Answers to the odd-numbered problems are also posted on the book website to be checked by the student. There are more than 600 end-of-chapter problems.

Audience

This book is written for both an academic and a professional audience. It can be used as a self-study guide for interested professionals. As a textbook, it can be used for a one-semester or one-quarter course. It is designed for the last year of undergraduate study or the first year of graduate study. Although some problems at the end of the chapters require some knowledge of probability, only general mathematical knowledge taught in the first year of college is needed to study the text.

Instruction Resources

The book contains complete instruction resources that can be downloaded from the book web site **www.mhhe.com/forouzan6e**. They include:

Presentations

The site includes a set of colorful and animated PowerPoint presentations for teaching the course.

Solution to Practice Sets

Solutions to all questions and problems are provided at the book website for the use of professors who teach the course.

Student Resources

The book contains complete student resources that can be downloaded from the book site **www.mhhe.com/forouzan6e**. They include:

Quizzes

There are quizzes at the end of each chapter that can be taken by the students. Students are encouraged to take the quizzes to test their general understanding of the materials presented in the corresponding chapter.

Solutions to Odd-Numbered Practice Set Questions and Problems

Solutions to all odd-number questions and problems are provided at the book website for the use of students.

Website

The McGraw-Hill Website contains much additional material. Available at **www.mhhe.com/forouzan6e**. As students read through *Data Communications and Networking with TCP/IP Protocol Suite,* they can go online to take self-grading quizzes. They can also access lecture materials such as PowerPoint slides and get additional review from animated figures from the book. Selected solutions are also available over the Web. The solutions to odd-numbered problems are provided to students, and instructors can use a password to access the complete set of solutions.

Acknowledgments

It is obvious that the development of a book of this scope needs the support of many people. I would like to acknowledge the contributions from peer reviewers to the development of the book. These reviewers are:

Azad Azadmanesh, University of Nebraska–Omaha
Maurice Dosso, Mt. Sierra College
John Doyle, Indiana University
Meng Han, Kennesaw State University
Tamer Omar, Cal Poly Pomona
Pat Smith, Oklahoma Christian University
Lawrence Teitelman, Queens College, City University of New York
Zhanyang Zhang, City University of New York

Special thanks go to the staff of McGraw-Hill. Beth Bettcher, the portfolio manager, proved how a proficient publisher can make the impossible, possible. Beth Baugh, the product developer, gave help whenever I needed it. Jane Mohr, the project manager, guided us through the production process with enormous enthusiasm. I also thank Sandeep Rawat, the full-service project manager, and David Hash, the cover designer.

Behrouz A. Forouzan
Los Angeles, CA
January 2021

TRADEMARK

Throughout the text we have used several trademarks. Rather than insert a trademark symbol with each mention of the trademark name, we acknowledge the trademarks here and state that they are used with no intention of infringing upon them. Other product names, trademarks, and registered trademarks are the property of their respective owners.

CHAPTER 1

Introduction

Data communications and networking have changed the way we do business and the way we live. The largest computer network, the Internet, has billions of users in the world who use wired and wireless transmission media to connect small and large computers.

Data communications and networking are not only used in business and personal communication but have found many political and social applications. People are able how to communicate with others all over the world to express their social and political opinions and problems. Communities are not isolated any more.

But before we ask how quickly we can get hooked up, we need to know how networks operate, what types of technologies are available, and which design best fills which set of needs.

This chapter paves the way for the rest of the book. It is divided into six sections.

- The first section introduces data communications and defines its components and the types of data exchanged.
- The second section introduces networks and defines their criteria and structures.
- The third section discusses different types of networks: LANs, WANs, and internetworks (internets). It also introduces the Internet, the largest internet in the world.
- The fourth section introduces protocol layering and its principles.
- The fifth section introduces the TCP/IP protocol suite and gives a brief description of each layer.
- The sixth section gives a brief historical description of the OSI model and compares it with the TCP/IP protocol suite.

1.1 DATA COMMUNICATIONS

When we communicate, we are sharing information or data. This sharing can be local or remote. Local communication usually occurs face to face, while remote communication takes place over a distance. The word ***data*** refers to information presented in whatever form is agreed upon by the parties creating and using it.

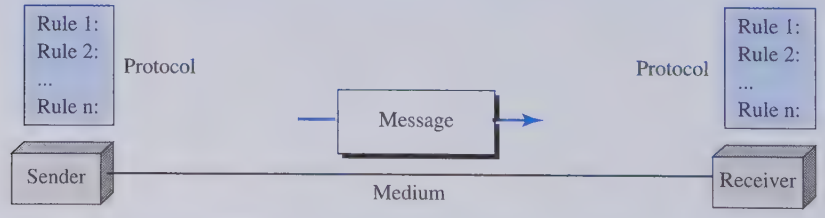
Data communications is the exchange of data between two devices via some form of transmission medium such as a wire cable. For data communications to occur, the communicating devices must be part of a communications system made up of a combination of hardware (physical equipment) and software (programs). The effectiveness of a data communications system depends on four fundamental characteristics: delivery, accuracy, timeliness, and jitter.

1. **Delivery.** The system must deliver data to the correct destination. Data must be received by the intended device or user and only by that device or user.
2. **Accuracy.** The system must deliver the data accurately. Data that have been altered in transmission and left uncorrected are unusable.
3. **Timeliness.** The system must deliver data in a timely manner. Data delivered late are useless. In the case of video and audio, timely delivery means delivering data as they are produced, in the same order that they are produced, and without significant delay. This kind of delivery is called *real-time* transmission.
4. **Jitter.** Jitter refers to the variation in the packet arrival time. It is the uneven delay in the delivery of audio or video packets. For example, let us assume that video packets are sent every 30 ms. If some of the packets arrive with a 30-ms delay and others with a 40-ms delay, the video will have an uneven quality.

1.1.1 Components

A data communications system has five components (see Figure 1.1).

Figure 1.1 *Five components of a data communications system*



1. **Message.** The **message** is the information (data) to be communicated. Popular forms of information include text, numbers, pictures, audio, and video.
2. **Sender.** The **sender** is the device that sends the data message. It can be a computer, a telephone handset, a video camera, and so on.

3. **Receiver.** The **receiver** is the device that receives the message. It can be a computer, workstation, telephone handset, television, and so on.
4. **Transmission medium.** The **transmission medium** is the physical path by which a message travels from sender to receiver. Some examples of transmission media include twisted-pair wire, coaxial cable, fiber-optic cable, and radio waves.
5. **Protocol.** A **protocol** is a set of rules that govern data communications. It represents an agreement between the communicating devices. Without a protocol, two devices may be connected but not able to communicate, just as a person speaking French cannot be understood by a person who speaks only Japanese.

1.1.2 Message

Information today comes in different forms such as text, numbers, images, audio, and video.

Text

In data communications, text is represented as a bit pattern, a sequence of bits (0s or 1s). Different sets of bit patterns have been designed to represent text symbols. Each set is called a **code**, and the process of representing symbols is coding. Today, the prevalent coding system is **Unicode**, which uses 32 bits to represent a symbol or character used in any language in the world (see Appendix A).

Numbers

Numbers are also represented by bit patterns. However, a code such as Unicode is not used to represent numbers; a number is directly converted to a binary number to simplify mathematical operations (see Appendix B).

Images

Images are also represented by bit patterns. In its simplest form, an image is composed of a matrix of pixels (picture elements), where each pixel is a small dot. The number of pixels depends on the *resolution*. For example, an image can be divided into 1000 pixels or 10,000 pixels. In the second case, there is a better representation of the image (better resolution), but more memory is needed to store the image.

After an image is divided into pixels, each pixel is assigned a bit pattern. The size and the value of the pattern depend on the image. For an image made up of only black and white dots (e.g., a chessboard), a 1-bit pattern is enough to represent a pixel.

If an image is not made up of pure white and pure black pixels, you can increase the size of the bit pattern to include the gray scale. For example, to show four levels of gray scale, you can use 2-bit patterns. A black pixel can be represented by 00, a dark gray pixel by 01, a light gray pixel by 10, and a white pixel by 11.

There are several methods to represent color images. One method is called **RGB**, so called because each color is made up of a combination of three primary colors: *r*ed, *g*reen, and *b*lue. The intensity of each color is measured, and a bit pattern is assigned to it. Another method is called **YCM**, in which a color is made up of a combination of three other primary colors: *y*ellow, cyan, and magenta.

Audio

Audio refers to the recording or broadcasting of sound or music. Audio is by nature different from text, numbers, or images. It is continuous, not discrete. Even when we use a microphone to change voice or music to an electric signal, we create a continuous signal. Later in the book we learn how to change sound or music to a digital or an analog signal.

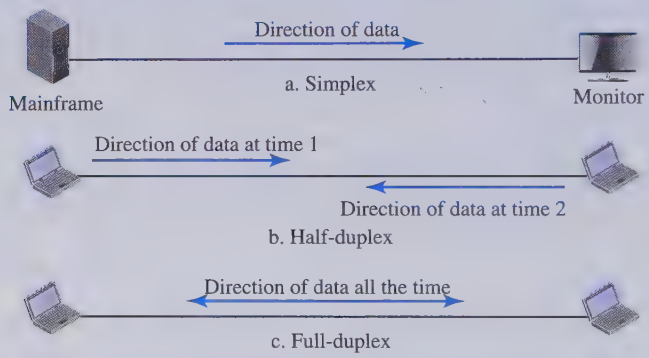
Video

Video refers to the recording or broadcasting of a picture or movie. Video can either be produced as a continuous entity (e.g., by a TV camera), or it can be a combination of images, each a discrete entity, arranged to convey the idea of motion.

1.1.3 Data Flow

Communication between two devices can be simplex, half-duplex, or full-duplex as shown in Figure 1.2.

Figure 1.2 *Data flow (simplex, half-duplex, and full-duplex)*



Simplex

In **simplex mode**, the communication is unidirectional, as on a one-way street. Only one of the two devices on a link can transmit; the other can only receive (see Figure 1.2a). Keyboards and traditional monitors are examples of simplex devices. The keyboard can only introduce input; the monitor can only accept output.

Half-Duplex

In **half-duplex mode**, each station can both transmit and receive, but not at the same time. When one device is sending, the other can only receive, and vice versa (see Figure 1.2b). Walkie-talkies and CB (citizens band) radios are both half-duplex systems.

Full-Duplex

In **full-duplex mode**, both stations can transmit and receive simultaneously (see Figure 1.2c). The full-duplex mode is like a two-way street with traffic flowing in both

directions at the same time. In full-duplex mode, signals going in one direction share the capacity of the link with signals going in the other direction. One common example of full-duplex communication is the telephone network. When two people are communicating by a telephone line, both can talk and listen at the same time.

1.2 NETWORKS

A **network** is the interconnection of a set of devices capable of communication. In this definition, a device can be a **host**, such as a large computer, desktop, laptop, workstation, cellular phone, or security system. A device in this definition can also be a **connecting device** such as a router that connects the network to other networks, a switch that connects devices together, or a modem (modulator-demodulator) that changes the form of data.

1.2.1 Network Criteria

A network must be able to meet a certain number of criteria. The most important of these are ***performance***, ***reliability***, and ***security***.

Performance

Performance can be measured in many ways, including transit time and response time. Transit time is the amount of time required for a message to travel from one device to another. Response time is the elapsed time between an inquiry and a response. The performance of a network depends on a number of factors, including the number of users, the type of transmission medium, the capabilities of the connected hardware, and the efficiency of the software.

Reliability

In addition to accuracy of delivery, network **reliability** is measured by the frequency of failure, the time it takes a link to recover from a failure, and the network's robustness in a catastrophe.

Security

Network **security** issues include protecting data from unauthorized access, protecting data from damage and development, and implementing policies and procedures for recovery from breaches and data losses.

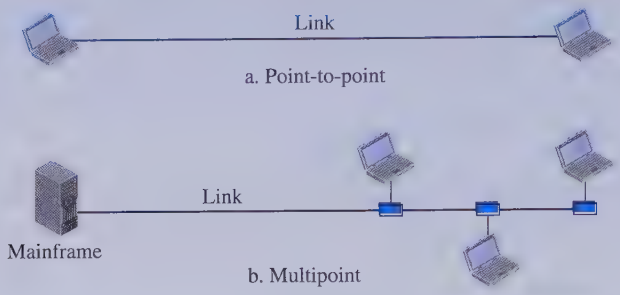
1.2.2 Physical Structures

Before discussing networks, we need to define some network attributes.

Type of Connection

A network is two or more devices connected through links. A link is a communications pathway that transfers data from one device to another. For communication to occur, two devices must be connected in some way to the same link at the same time. There are two possible types of connections: *point-to-point* and *multipoint* (see Figure 1.3 on next page).

Figure 1.3 *Types of connections: point-to-point and multipoint*



Point-to-Point

A **point-to-point connection** provides a dedicated link between two devices. The entire capacity of the link is reserved for transmission between those two devices.

Multipoint

A **multipoint** (also called **multidrop**) **connection** is one in which more than two devices share a single link. In a multipoint environment, the capacity of the channel is shared, either spatially or temporally. If several devices can use the link simultaneously, it is a *spatially shared* connection. If users must take turns, it is a *timeshared* connection.

Physical Topology

The term ***physical topology*** refers to the way in which a network is laid out physically. Two or more devices connect to a link; two or more links form a topology. The topology of a network is the geometric representation of the relationship of all the links and linking devices (usually called **nodes**) to one another. There are four basic topologies possible: *mesh*, *star*, *bus*, and *ring*.

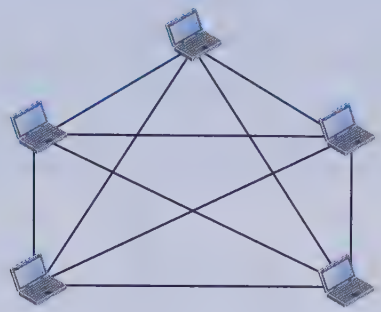
Mesh Topology

In a **mesh topology**, every device has a dedicated point-to-point link to every other device. The term *dedicated* means that the link carries traffic only between the two devices it connects. To find the number of physical links in a fully connected mesh network with n nodes, we first consider that each node must be connected to every other node. Node 1 must be connected to $n - 1$ nodes, node 2 must be connected to $n - 1$ nodes, and finally node n must be connected to $n - 1$ nodes. We need $n(n - 1)$ physical links. To accommodate that many links, every device on the network must have $n - 1$ input/output (I/O) ports (see Figure 1.4 on next page) to be connected to the other $n - 1$ stations.

Star Topology

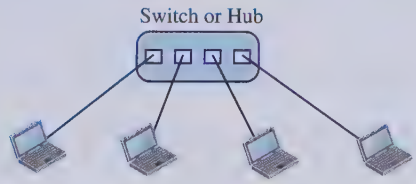
In a **star topology**, each device has a dedicated point-to-point link only to a central controller, usually called a **hub**. The devices are not directly linked to one another. Unlike a mesh topology, a star topology does not allow direct traffic between devices. The controller acts as an exchange: If one device wants to send data to another, it sends

Figure 1.4 *A fully connected mesh topology (five devices)*



the data to the controller, which then relays the data to the other connected device (see Figure 1.5).

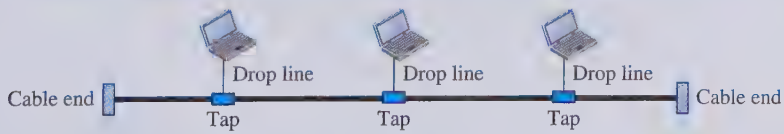
Figure 1.5 *A star topology connecting four stations*



Bus Topology

The preceding topology examples all describe point-to-point connections. A **bus topology**, on the other hand, is multipoint. One long cable acts as a **backbone** to link all the devices in a network (see Figure 1.6).

Figure 1.6 *A bus topology connecting three stations*

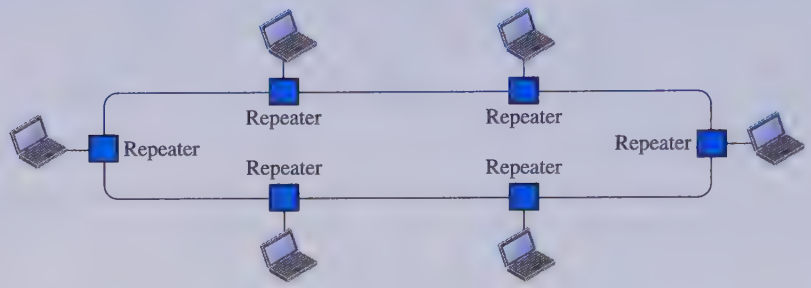


Nodes are connected to the bus cable by drop lines and taps. A *drop line* is a connection running between the device and the main cable. A *tap* is a connector that either splices into the main cable or punctures the sheathing of a cable to create a contact with the metallic core.

Ring Topology

In a **ring topology**, each device has a dedicated point-to-point connection with only the two devices on either side of it. A signal is passed along the ring in one direction, from device to device, until it reaches its destination. Each device in the ring incorporates a repeater, which regenerates the bits and passes them along (see Figure 1.7).

Figure 1.7 *A ring topology connecting six stations*



1.3 NETWORK TYPES

Now we discuss different types of networks: LANs and WANs.

1.3.1 Local Area Network

A **local area network (LAN)** is usually privately owned and connects some hosts in a single office, building, or campus.

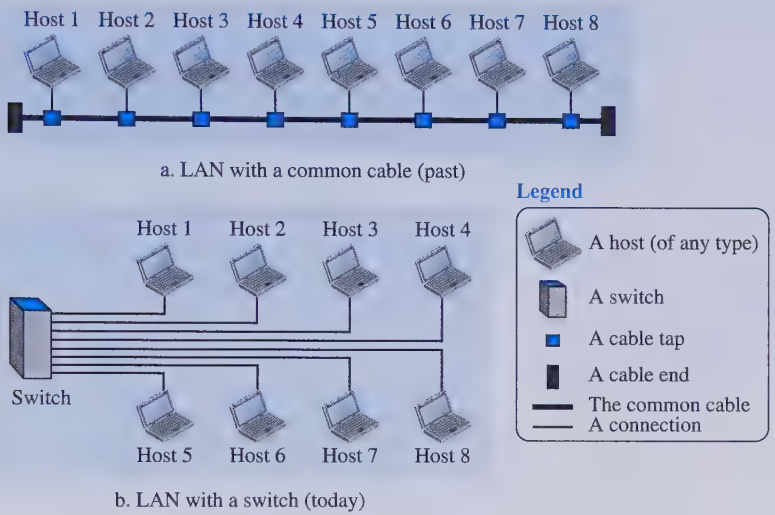
Each host in a LAN has an identifier, which is an address that uniquely defines the host in the LAN. A packet sent by a host to another host carries both the source host's and the destination host's addresses.

When LANs were used in isolation (which is rare today), they were designed to allow resources to be shared between the hosts. As we will see shortly, LANs today are connected to each other and to WANs (discussed next) to create communication at a wider level (see Figure 1.8 on next page).

1.3.2 Wide Area Network (WAN)

A **wide area network (WAN)** is also an interconnection of devices capable of communication. However, there are some differences between a LAN and a WAN. A LAN is normally limited in size, spanning an office, a building, or a campus; a WAN has a wider geographical span, spanning a town, a state, a country, or even the world. A LAN interconnects hosts; a WAN interconnects connecting devices such as switches, routers, or modems. We see two distinct examples of WANs today: point-to-point WANs and switched WANs.

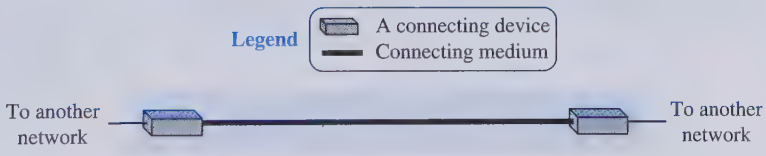
Figure 1.8 *An isolated LAN in the past and today*



Point-to-Point WAN

A point-to-point WAN is a network that connects two communicating devices through a transmission medium (cable or air). Figure 1.9 shows an example of a point-to-point WAN.

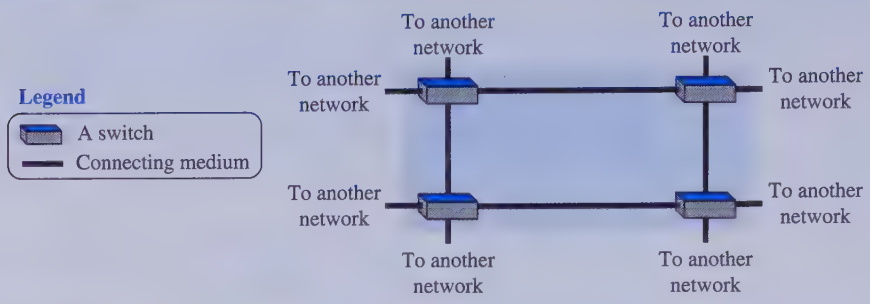
Figure 1.9 *A point-to-point WAN*



Switched WAN

A switched WAN is a network with more than two ends. It is used in the backbone of a global communications network today. Figure 1.10 shows an example of a switched WAN.

Figure 1.10 *A switched WAN*



Internetwork

Today, it is very rare to see a LAN or a WAN in isolation; they are connected to one another. When two or more networks are connected, they make an **internetwork**, or **internet**. As an example, assume that an organization has two offices, one on the east coast and the other on the west coast. Each office has a LAN that allows all employees in the office to communicate with each other. To make the communication between employees at different offices possible, the management leases a point-to-point dedicated WAN from a service provider, such as a telephone company, and connects the two LANs. Now the company has an internetwork, or a private internet (with lowercase *i*). Communication between offices is now possible. Figure 1.11 shows this internet.

Figure 1.11 *An internetwork made of two LANs and one point-to-point WAN*

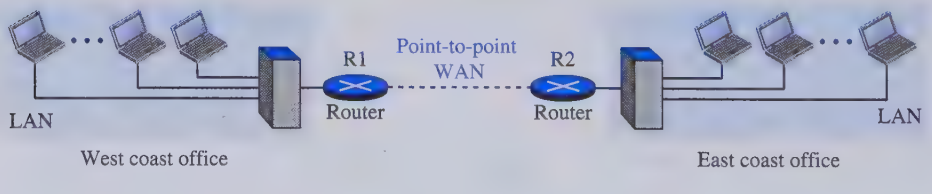


Figure 1.12 shows another internet with several LANs and WANs connected. One of the WANs is a switched WAN with four switches.

1.3.3 The Internet

As we discussed before, an *internet* (note the lowercase *i*) is two or more networks that can communicate with each other. The most notable internet is called the **Internet** (uppercase *I*) and is composed of thousands of interconnected networks. Figure 1.13 shows a conceptual (not geographical) view of the Internet.

Figure 1.12 *A heterogeneous internetwork made of four WANs and two LANs*

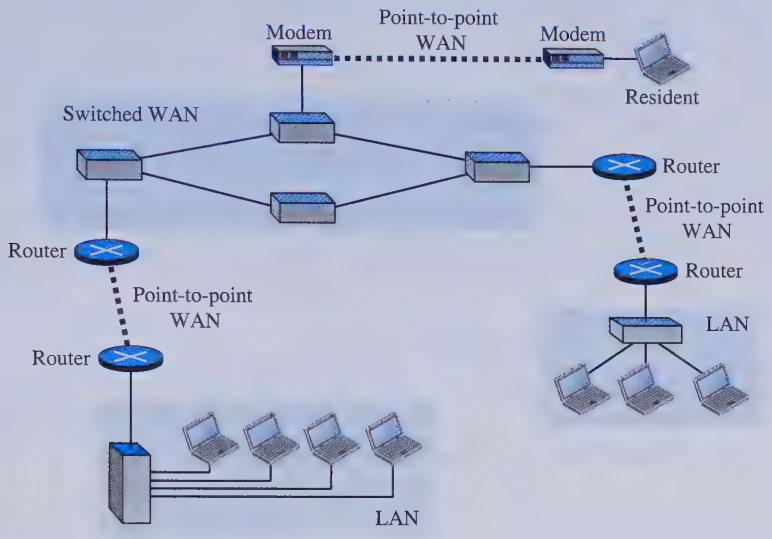
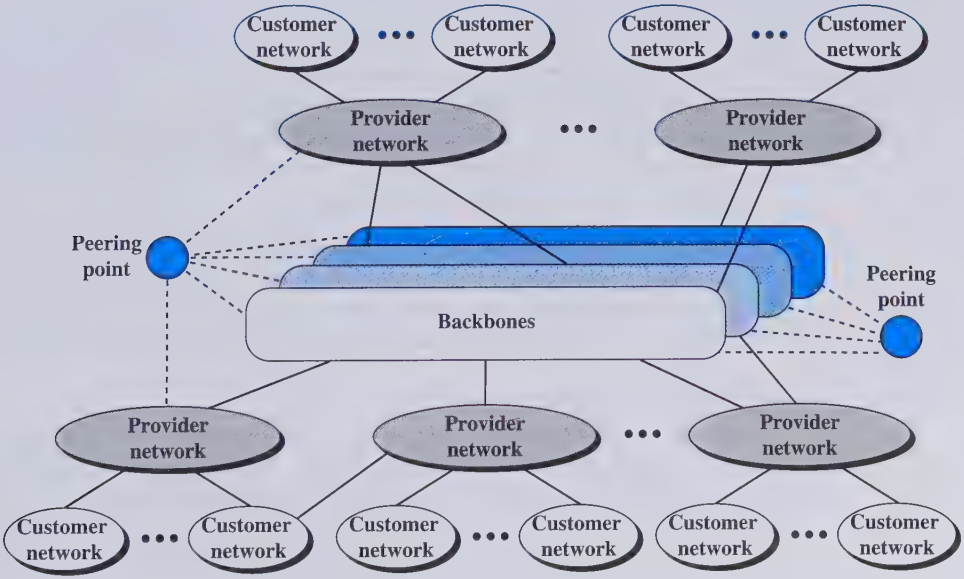


Figure 1.13 *The Internet today*



The figure shows the Internet as several backbones, provider networks, and customer networks. At the top level, the *backbones* are large networks owned by some communication companies. The backbone networks are connected through some complex switching systems, called *peering points*. At the second level, there are smaller networks, called *provider networks*, that use the services of the backbones for a fee. The provider networks are connected to backbones and sometimes to other provider networks. The *customer networks* are networks at the edge of the Internet that actually use the services provided by the Internet. They pay fees to provider networks for receiving services.

Backbones and provider networks are also called **Internet Service Providers (ISPs)**. The backbones are often referred to as international ISPs; the provider networks are often referred to as national or regional ISPs.

1.3.4 Accessing the Internet

The Internet today is an internetwork that allows any user to become part of it. The user, however, needs to be physically connected to an ISP. The physical connection is normally done through a point-to-point WAN (such as a telephone network, a cable network, a wireless network, or other types of networks).

Using Telephone Networks

Today most residences and small businesses have telephone service, which means they are connected to a telephone network. Because most telephone networks have already connected themselves to the Internet, one option for residences and small businesses to connect to the Internet is to change the voice line between the residence or business and the telephone center to a point-to-point WAN. This can be done in two ways.

- ***Dial-up service.*** The first solution is to add a modem that converts data to voice to the telephone line. The software installed on the computer dials the ISP and imitates making a telephone connection. Unfortunately, the dial-up service is very slow, and when the line is used for an Internet connection, it cannot be used for a telephone (voice) connection. It is only useful for small residences and businesses with occasional connection to the Internet.
- ***DSL Service.*** Since the advent of the Internet, some telephone companies have upgraded their telephone lines to provide higher-speed Internet services to residences or small businesses. The digital subscriber line (DSL) service also allows the line to be used simultaneously for voice and data communications.

Using Cable Networks

More and more residents over the last two decades have begun using cable TV services instead of antennas to receive TV broadcasting. The cable companies have been upgrading their cable networks and connecting to the Internet. A residence or a small business can be connected to the Internet by using this service. It provides a higher-speed connection, but the speed varies depending on the number of neighbors that use the same cable.

Using Wireless Networks

Wireless connectivity has recently become increasingly popular. A household or a small business can use a combination of wireless and wired connections to access the Internet. With the growing wireless WAN access, a household or a small business can be connected to the Internet through a wireless WAN.

Direct Connection to the Internet

A large organization or a large corporation can itself become a local ISP and be connected to the Internet. This can be done if the organization or the corporation leases a high-speed WAN from a carrier provider and connects itself to a regional ISP. For example, a large university with several campuses can create an internetwork and then connect the internetwork to the Internet.

1.4 PROTOCOL LAYERING

We defined the term *protocol* before. In data communications and networking, a protocol defines the rules that both the sender and receiver and all intermediate devices need to follow to be able to communicate effectively. When communication is simple, we may need only one simple protocol; when the communication is complex, we may need to divide the task between different layers, in which case we need a protocol at each layer, or **protocol layering**.

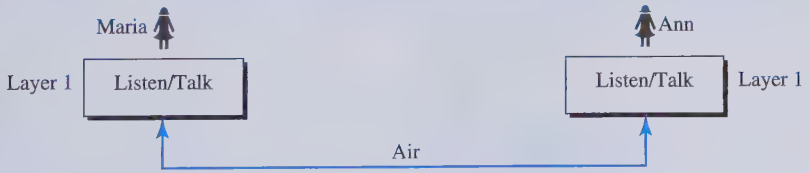
1.4.1 Scenarios

Let us develop two simple scenarios to better understand the need for protocol layering.

First Scenario

In the first scenario, communication is so simple that it can occur in only one layer. Assume Maria and Ann are neighbors with a lot of common ideas. Communication between Maria and Ann takes place in one layer, face to face, in the same language, as shown in Figure 1.14.

Figure 1.14 *A single-layer protocol*



Even in this simple scenario, we can see that a set of rules needs to be followed. First, Maria and Ann know that they should greet each other when they meet. Second, they know that they should confine their vocabulary to the level of their friendship.

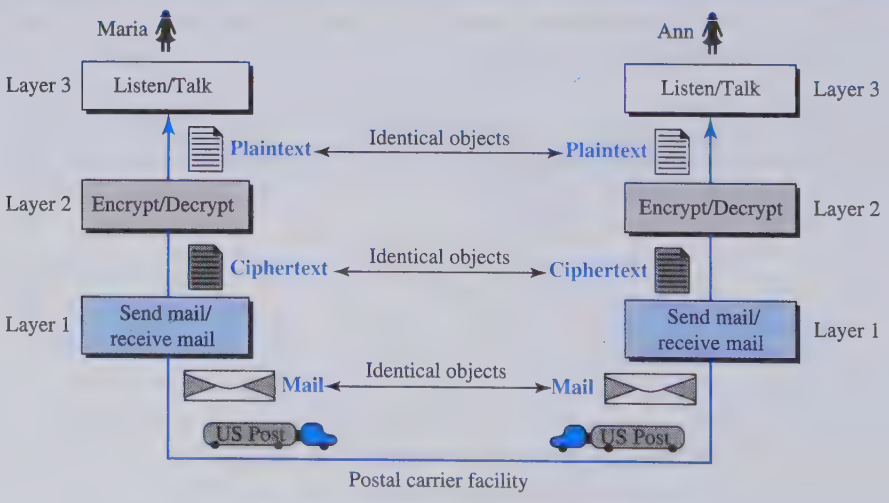
Third, each party knows that she should refrain from speaking when the other party is speaking. Fourth, each party knows that the conversation should be a dialog, not a monolog: Both should have the opportunity to talk about the issue. Fifth, they should exchange some nice words when they leave.

We can see that the protocol used by Maria and Ann is different from the communication between a professor and the students in a lecture hall. The communication in the second case is mostly monolog; the professor talks most of the time unless a student has a question, a situation in which the protocol dictates that she should raise her hand and wait for permission to speak. In this case, the communication is normally very formal and limited to the subject being taught.

Second Scenario

In the second scenario, we assume that Ann is offered a higher-level position in her company but needs to move to another branch located in a city very far from Maria. The two friends still want to continue their communication and exchange ideas because they have come up with an innovative project to start a new business when they both retire. They decide to continue their conversations using regular mail through the post office. However, they do not want their ideas to be revealed to other people if the letters are intercepted. They agree on an encryption/decryption technique. The sender of the letter encrypts it to make it unreadable by an intruder; the receiver of the letter decrypts it to get the original letter. We discuss the encryption/decryption methods later in the book, but for the moment we assume that Maria and Ann use one technique to make it hard to decrypt the letter if one does not have the key for doing so. Now we can say that the communication between Maria and Ann takes place in three layers, as shown in Figure 1.15. We assume that Ann and Maria each have three machines (or robots) that can perform the task at each layer.

Figure 1.15 *A three-layer protocol*



Let us assume that Maria sends the first letter to Ann. Maria talks to the machine at the third layer as though the machine is Ann and is listening to her. The third-layer machine listens to what Maria says and creates the plaintext (a letter in English), which is passed to the second-layer machine. The second-layer machine takes the plaintext, encrypts it, and creates the ciphertext, which is passed to the first-layer machine. The first-layer machine, presumably a robot, takes the ciphertext, puts it in an envelope, adds the sender and receiver addresses, and mails it.

At Ann's side, the first-layer machine picks up the letter from Ann's mailbox, recognizing the letter from Maria by the sender address. The machine takes out the ciphertext from the envelope and delivers it to the second-layer machine. The second-layer machine decrypts the message, creates the plaintext, and passes the plaintext to the third-layer machine. The third-layer machine takes the plaintext and reads it as though Maria is speaking.

Protocol layering enables us to divide a complex task into several smaller and simpler tasks. For example, in Figure 1.15, we could have used only one machine to do the job of all three machines. However, if Maria and Ann decide that the encryption/decryption done by the machine is not enough to protect their secrecy, they have to change the whole machine. In the present situation, they need to change only the second-layer machine; the other two can remain the same. This is referred to as *modularity*. Modularity in this case means independent layers. A layer (module) can be defined as a black box with inputs and outputs, without concern about how inputs are changed to outputs. If two machines provide the same outputs when given the same inputs, they can replace each other. For example, Ann and Maria can buy the second-layer machine from two different manufacturers. As long as the two machines create the same ciphertext from the same plaintext and vice versa, they do the job.

One of the advantages of protocol layering is that it allows us to separate the services from the implementation. A layer needs to be able to receive a set of services from the lower layer and to give the services to the upper layer; we don't care about how the layer is implemented. For example, Maria may decide not to buy the machine (robot) for the first layer; she can do the job herself. As long as Maria can do the tasks provided by the first layer, in both directions, the communications system works.

Another advantage of protocol layering, which cannot be seen in our simple examples, but reveals itself when we discuss protocol layering in the Internet, is that communication does not always use only two end systems; there are intermediate systems that need only some layers, but not all layers. If we did not use protocol layering, we would have to make each intermediate system as complex as the end systems, which makes the whole system more expensive.

Is there any disadvantage to protocol layering? One can argue that having a single layer makes the job easier. There is no need for each layer to provide a service to the upper layer and give service to the lower layer. For example, Ann and Maria could find or build one machine that could do all three tasks. However, as mentioned above, if one day they found that their code was broken, each would have to replace the whole machine with a new one instead of just changing the machine in the second layer.

1.4.2 Principles of Protocol Layering

Let us discuss the two principles of protocol layering.

First Principle

The first principle dictates that if we want bidirectional communication, we need to make each layer so that it is able to perform two opposite tasks, one in each direction. For example, the third-layer task is to listen (in one direction) and *talk* (in the other direction). The second layer needs to be able to encrypt and decrypt. The first layer needs to send and receive mail.

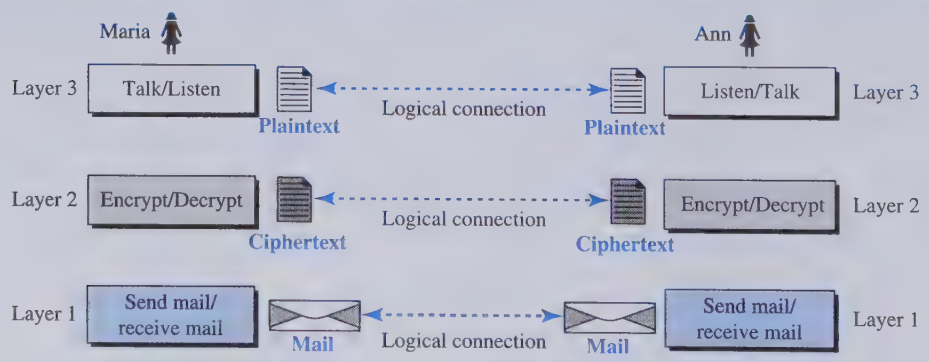
Second Principle

The second important principle that we need to follow in protocol layering is that the two objects under each layer at both sites should be identical. For example, the object under the third layer at both sites should be a plaintext letter. The object under the second layer at both sites should be a ciphertext letter. The object under the first layer at both sites should be a piece of mail.

1.4.3 Logical Connections

After following the above two principles, we can think about logical connections between each layer as shown in Figure 1.16. This means that we have layer-to-layer communication. Maria and Ann can think that there is a logical (imaginary) connection at each layer through which they can send the object created from that layer. We will see that the concept of logical connection will help us better understand the task of layering we encounter in data communications and networking.

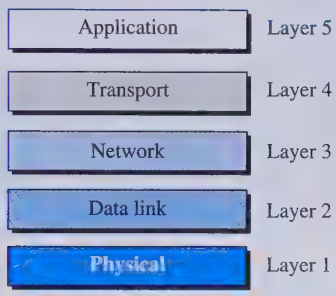
Figure 1.16 *Logical connections between peer layers*



1.5 TCP/IP PROTOCOL SUITE

Now that we know about the concept of protocol layering and the logical connections between layers in our second scenario, we can introduce the **Transmission Control Protocol/Internet Protocol (TCP/IP).** TCP/IP is a protocol suite (a set of protocols organized in different layers) used in the Internet today. It is a hierarchical protocol made up of interactive modules, each of which provides a specific functionality. The term *hierarchical* means that each upper-level protocol is supported by the services provided by one or more lower-level protocols. The **TCP/IP protocol suite** is defined as five layers as shown in Figure 1.17.

Figure 1.17 *Layers in the TCP/IP protocol suite*



1.5.1 Layered Architecture

To show how the layers in the TCP/IP protocol suite are involved in communication between two hosts, we assume that we want to use the suite in a small internet made up of three LANs (links), each with a link-layer switch. We also assume that the links are connected by one router, as shown in Figure 1.18 (on next page).

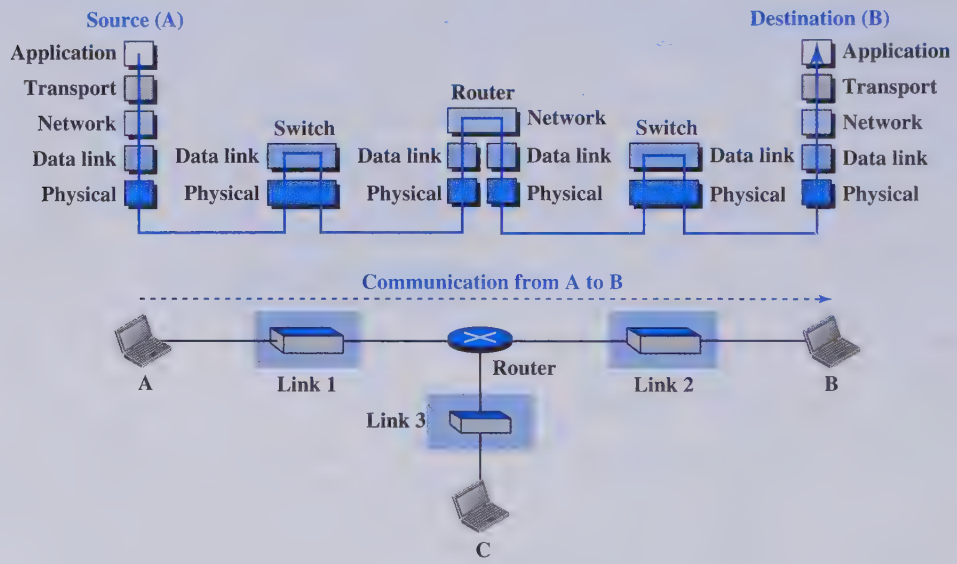
Let us assume that computer A communicates with computer B. As Figure 1.18 shows, we have five communicating devices in this communication: source host (computer A), the link-layer switch in link 1, the router, the link-layer switch in link 2, and the destination host (computer B). Each device is involved with a set of layers depending on the role of the device in the internet. The two hosts are involved in all five layers. The source host needs to create a message in the application layer and send it down the layers so that it is physically sent to the destination host. The destination host needs to receive the communication at the physical layer and then deliver it through the other layers to the application layer.

The router is involved only in three layers; there is no transport or application layer in a router as long as the router is used only for routing. Although a router is always involved in one network layer, it is involved in n combinations of link and physical layers in which n is the number of links the router is connected to. The reason is that each link

may use its own data-link or physical protocol. For example, in Figure 1.18, the router is involved in three links, but the message sent from source computer A to destination computer B is involved in two links. Each link may be using different link-layer and physical-layer protocols; the router needs to receive a packet from link 1 based on one pair of protocols and deliver it to link 2 based on another pair of protocols.

A link-layer switch in a link, however, is involved only in two layers, data-link and physical protocols. Although each switch in Figure 1.18 has two different connections, the connections are in the same link, which uses only one set of protocols. This means that, unlike a router, a link-layer switch is involved only in one data-link and one physical layer.

Figure 1.18 *Communication through an internet*

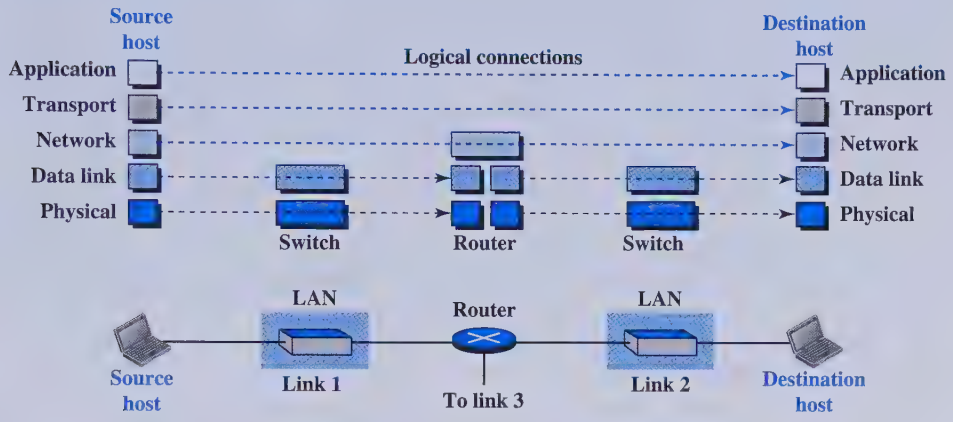


1.5.2 Brief Description of Layers

We now briefly discuss the functions and duties of layers in the TCP/IP protocol suite. Each layer is discussed in detail in a separate chapter of the book. To better understand the duties of each layer, we need to think about the logical connections between the layers. Figure 1.19 shows the logical connections in our simple internet.

Using logical connections makes it easier for us to think about the duty of each layer. As Figure 1.19 shows, the duty of the application, transport, and network layers is end-to-end. However, the duty of the data-link and physical layers is hop-to-hop, in which a hop is a host or router. In other words, the domain of duty of the top three layers is the internet, and the domain of duty of the two lower layers is the link.

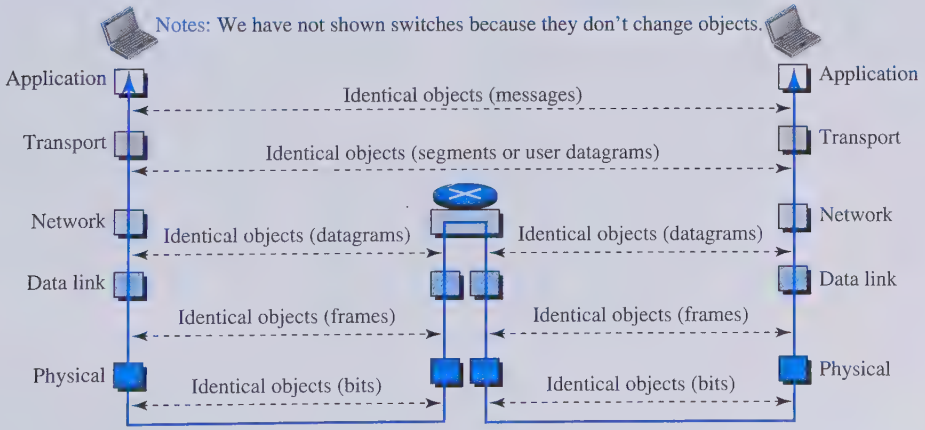
Figure 1.19 *Logical connections between layers of the TCP/IP protocol suite*



Another way of thinking about the logical connections is to think about the data unit created from each layer. In the top three layers, the data unit (packets) should not be changed by any router or link-layer switch. In the bottom two layers, the packet created by the host is changed only by the routers, not by the link-layer switches.

Figure 1.20 shows the second principle discussed previously for protocol layering. We show the identical objects below each layer related to each device.

Figure 1.20 *Identical objects in the TCP/IP protocol suite*



Note that, although the logical connection at the network layer is between the two hosts, we can only say that identical objects exist between two hops in this case because a router may fragment the packet at the network layer and send more packets than

received. (See Chapter 4 for a discussion of fragmentation.) Note that the link between two hops does not change the object.

1.5.3 Description of Each Layer

After understanding the concept of logical communication, we are ready to briefly discuss the duty of each layer.

Physical Layer

We can say that the physical layer is responsible for carrying individual bits in a frame across the link. The physical layer is the lowest level in the TCP/IP protocol suite. The communication between two devices at the physical layer is still a logical communication because there is another hidden layer, the transmission media, under the physical layer. We discuss the physical layer in Chapter 2.

Data-Link Layer

We have seen that an *internet* is made up of several links (LANs and WANs) connected by routers. When the next link to travel is determined by the router, the data-link layer is responsible for taking the datagram and moving it across the link. We discuss the data-link layer in Chapters 3, 4, 5, and 6.

Network Layer

The network layer is responsible for creating a connection between the source computer and the destination computer. The communication at the network layer is host-to-host. However, because there can be several routers from the source to the destination, the routers in the path are responsible for choosing the best route for each packet. We discuss the network layer in Chapters 7 and 8.

Transport Layer

The logical connection at the transport layer is also end-to-end. The transport layer at the source host gets the message from the application layer; encapsulates it in a transport-layer packet (called a *segment* or a *user datagram* in different protocols); and sends it, through the logical (imaginary) connection, to the transport layer at the destination host. In other words, the transport layer is responsible for giving services to the application layer: to get a message from an application program running on the source host and deliver it to the corresponding application program on the destination host. We need to say that we discuss the transport layer in Chapter 9.

Application Layer

The logical connection between the two application layers is end-to-end. The two application layers exchange *messages* between each other as though there were a bridge between the two layers. However, we should know that the communication is done through all the layers. Communication at the application layer is between two *processes* (two programs running at this layer). To communicate, a process sends a request to the other process and receives a response. Process-to-process communication is the duty of the application layer. We discuss the application layer in Chapter 10.

1.6 THE OSI MODEL

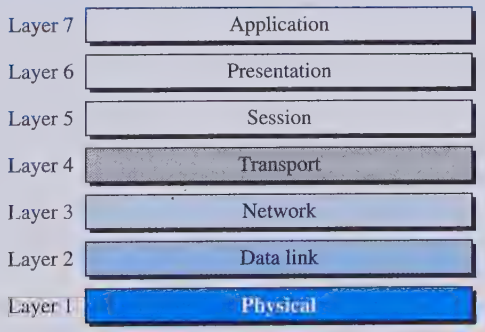
Although, when speaking of the Internet, everyone talks about the TCP/IP protocol suite, it is not the only suite of protocols defined. Established in 1947, the **International Organization for Standardization (ISO)** is a multinational body dedicated to worldwide agreement on international standards. Almost three-fourths of the countries in the world are represented in the ISO. An ISO standard that covers all aspects of network communications is the **Open Systems Interconnection (OSI)** model. It was first introduced in the late 1970s.

ISO is the organization; OSI is the model.

An *open system* is a set of protocols that allows any two different systems to communicate regardless of their underlying architecture. The purpose of the OSI model is to show how to facilitate communication between different systems without requiring changes to the logic of the underlying hardware and software. The OSI model is not a protocol; it is a model for understanding and designing a network architecture that is flexible, robust, and interoperable. The OSI model was intended to be the basis for the creation of the protocols in the OSI stack.

The OSI model is a layered framework for the design of network systems that allows communication between all types of computer systems. It consists of seven separate but related layers, each of which defines a part of the process of moving information across a network (see Figure 1.21).

Figure 1.21 *The OSI model*

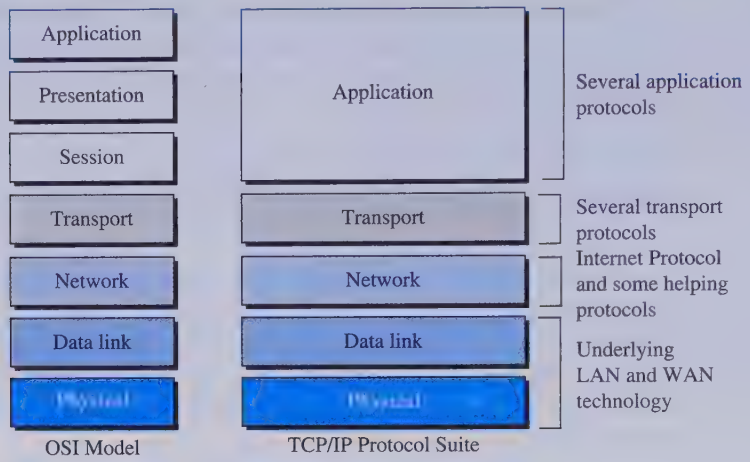


1.6.1 OSI versus TCP/IP

When we compare the two models, we find that two layers, session and presentation, are missing from the TCP/IP protocol suite. These two layers were not added to the TCP/IP protocol suite after the publication of the OSI model. The application layer in the suite

is usually considered to be the combination of three layers in the OSI model, as shown in Figure 1.22.

Figure 1.22 *TCP/IP and OSI model*



Two reasons were mentioned for this decision. First, TCP/IP has more than one transport-layer protocol. Some of the functionalities of the session layer are available in some of the transport-layer protocols. Second, the application layer is not only one piece of software. Many applications can be developed at this layer. If some of the functionalities mentioned in the session and presentation layers are needed for a particular application, they can be included in the development of that piece of software.

1.6.2 Lack of OSI Model's Success

The OSI model appeared after the TCP/IP protocol suite. Most experts were at first excited and thought that the TCP/IP protocol would be fully replaced by the OSI model. This did not happen for several reasons, but we describe only three, which are agreed upon by all experts in the field. First, OSI was completed when TCP/IP was fully in place and a lot of time and money had been spent on the suite; changing it would cost a lot. Second, some layers in the OSI model were never fully defined. For example, although the services provided by the presentation and the session layers were listed in the document, actual protocols for these two layers were not fully defined, nor were they fully described, and the corresponding software was not fully developed. Third, when OSI was implemented by an organization in a different application, it did not show a high enough level of performance to entice the Internet authority to switch from the TCP/IP protocol suite to the OSI model.

1.7 END-OF-CHAPTER MATERIALS

1.7.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books, papers, and Requests for Comments (RFCs). The items enclosed in brackets refer to the reference list at the end of the book.

Books and Papers

Several books and papers give a thorough coverage about the materials discussed in this chapter: [Seg 98], [Lei et al. 98], [Kle 04], [Cer 89], and [Jen et al. 86].

Requests for Comments

Two RFCs in particular discuss the TCP/IP suite: RFC 791 (IP) and RFC 817 (TCP). In future chapters we list different RFCs related to each protocol in each layer.

1.7.2 Key Terms

audio
backbone
bus topology
code
connecting device
data
data communications
full-duplex mode
half-duplex mode
host
hub
image
International Organization for Standardization (ISO)
internet
Internet
Internet Service Provider (ISP)
internetwork
local area network (LAN)
mesh topology
message
multipoint or multidrop connection
network
node
Open System Interconnection (OSI)
performance
physical topology
point-to-point connection
protocol
protocol layering
receiver
reliability
RGB
ring topology
security
sender
simplex mode
star topology
TCP/IP protocol suite
Transmission Control Protocol/ Internet Protocol (TCP/IP)
transmission medium
Unicode
video
wide area network (WAN)
YCM

1.7.3 Summary

Data communications are the transfer of data from one device to another via some form of transmission medium. A data communications system must transmit data to the correct destination in an accurate and timely manner. The five components that make up a data communications system are the message, sender, receiver, medium, and

protocol. Text, numbers, images, audio, and video are different forms of information. Data flow between two devices can occur in one of three ways: simplex, half-duplex, or full-duplex.

A network is a set of communication devices connected by media links. In a point-to-point connection, two and only two devices are connected by a dedicated link. In a multipoint connection, three or more devices share a link. Topology refers to the physical or logical arrangement of a network. Devices may be arranged in a mesh, star, bus, or ring topology.

A network can be categorized as a local area network or a wide area network. A LAN is a data communication system within a building, plant, or campus, or between nearby buildings. A WAN is a data communication system spanning states, countries, or the whole world. An internet is a network of networks. The Internet is a collection of many separate networks.

TCP/IP is a hierarchical protocol suite made up of five layers: physical, data-link, network, transport, and application. The physical layer coordinates the functions required to transmit a bit stream over a physical medium. The data-link layer is responsible for delivering data units from one station to the next without errors. The network layer is responsible for the source-to-destination delivery of a packet across multiple network links. The transport layer is responsible for the process-to-process delivery of the entire message. The application layer enables the users to access the network.

Four levels of addresses are used in an internet following the TCP/IP protocols: physical (link) addresses, logical (IP) addresses, port addresses, and specific addresses. The physical address, also known as the link address, is the address of a node as defined by its LAN or WAN. The IP address uniquely defines a host on the Internet. The port address identifies a process on a host. A specific address is a user-friendly address.

Another model that defines protocol layering is the Open Systems Interconnection (OSI) model. Two layers in the OSI model, session and presentation, are missing from the TCP/IP protocol suite. These two layers were not added to the TCP/IP protocol suite after the publication of the OSI model. The application layer in the suite is usually considered to be the combination of three layers in the OSI model. The OSI model did not replace the TCP/IP protocol suite because it was completed when TCP/IP was fully in place and because some layers in the OSI model were never fully defined.

1.8 PRACTICE SET

1.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that the students take the quizzes to check their understanding of the materials before continuing with the practice set.

1.8.2 Questions

Q1-1. Identify the five components of a data communications system.

Q1-2. What are the three criteria necessary for an effective and efficient network?

Q1-3. What are the advantages of a multipoint connection over a point-to-point connection?

Q1-4. What are the two types of line configuration?

Q1-5. Categorize the four basic topologies in terms of line configuration.

Q1-6. What is the difference between half-duplex and full-duplex transmission modes?

Q1-7. Name the four basic network topologies, and cite an advantage of each type.

Q1-8. For n devices in a network, what is the number of cable links required for a mesh, ring, bus, and star topology?

Q1-9. What are some of the factors that determine whether a communications system is a LAN or WAN?

Q1-10. What is an internet? What is the Internet?

Q1-11. Why are protocols needed?

Q1-12. In a LAN with a link-layer switch (Figure 1.8b), host 1 wants to send a message to host 3. Because communication is through the link-layer switch, does the switch need to have an address? Explain.

Q1-13. How many point-to-point WANs are needed to connect n LANs if each LAN should be able to directly communicate with any other LAN?

Q1-14. When a resident uses a dial-up or DLS service to connect to the Internet, what is the role of the telephone company?

Q1-15. What is the first principle we discussed in this chapter for protocol layering that needs to be followed to make the communication bidirectional?

Q1-16. Which layers of the TCP/IP protocol suite are involved in a link-layer switch?

Q1-17. A router connects three links (networks). How many of each of the following layers can the router be involved with?

a. physical layer **b.** data-link layer **c.** network layer

Q1-18. In the TCP/IP protocol suite, what are the identical objects at the sender and the receiver sites when we think about the logical connection at the application layer?

Q1-19. A host communicates with another host using the TCP/IP protocol suite. What is the unit of data sent or received at each of the following layers?

a. application layer **b.** network layer **c.** data-link layer

Q1-20. Which of the following data units is encapsulated in a frame?

a. a user datagram **b.** a datagram **c.** a segment

Q1-21. Which of the following data units has an application-layer message plus the header from layer 4?

a. a frame **b.** a user datagram **c.** a bit

Q1-22. List some application-layer protocols mentioned in this chapter.

Q1-23. If a port number is 16 bits (2 bytes), what is the minimum header size at the transport layer of the TCP/IP protocol suite?

Q1-24. What are the types of addresses (identifiers) used in each of the following layers?

a. application layer **b.** network layer **c.** data-link layer

Q1-25. Assume we want to connect two isolated hosts together to let each host communicate with the other. Do we need a link-layer switch between the two? Explain.

Q1-26. If there is a single path between the source host and the destination host, do we need a router between the two hosts?

1.8.3 Problems

P1-1. What is the maximum number of characters or symbols that can be represented by Unicode?

P1-2. A color image uses 16 bits to represent a pixel. What is the maximum number of different colors that can be represented?

P1-3. Assume six devices are arranged in a mesh topology. How many cables are needed? How many ports are needed for each device?

P1-4. For each of the following four networks, discuss the consequences if a connection fails.

- **a.** Five devices arranged in a mesh topology
- **b.** Five devices arranged in a star topology (not counting the hub)
- **c.** Five devices arranged in a bus topology
- **d.** Five devices arranged in a ring topology

P1-5. In the ring topology in Figure 1.7, what happens if one of the stations is unplugged?

P1-6. In the bus topology in Figure 1.6, what happens if one of the stations is unplugged?

P1-7. When a party makes a local telephone call to another party, is this a point-to-point or multipoint connection? Explain your answer.

P1-8. Compare the telephone network and the Internet. What are the similarities? What are the differences?

P1-9. Answer the following questions about Figure 1.15 when the communication is from Maria to Ann:

- **a.** What is the service provided by layer 2 to layer 3 at Maria's site?
- **b.** What is the service provided by layer 2 to layer 3 at Ann's site?

P1-10. Assume that the number of hosts connected to the Internet at year 2010 is 500 million. If the number of hosts increases only 20 percent per year, what is the number of hosts in year 2020?

P1-11. Assume a system uses five protocol layers. If the application program creates a message of 100 bytes and each layer (including the fifth and the first) adds a header of 10 bytes to the data unit, what is the efficiency (the ratio of application-layer bytes to the number of bytes transmitted) of the system?

P1-12. Match the following to one or more layers of the TCP/IP protocol suite:

- **a.** route determination
- **b.** connection to transmission media
- **c.** providing services for the end user

P1-13. Match the following to one or more layers of the TCP/IP protocol suite:

- **a.** creating user datagrams
- **b.** responsibility for handling frames between adjacent nodes

P1-14. Assume that a private internet requires that the messages at the application layer be encrypted and decrypted for security purposes. If we need to add some

information about the encryption/decryption process (such as the algorithms used in the process), does it mean that we are adding one layer to the TCP/IP protocol suite? Redraw the TCP/IP layers (Figure 1.17b) if you think so.

P1-15. Protocol layering can be found in many aspects of our lives such as air travelling. Imagine you make a round trip to spend some time on vacation at a resort. You need to go through some processes at your city airport before flying. You also need to go through some processes when you arrive at the resort airport. Show the protocol layering for the round trip using some layers such as baggage checking/claiming, boarding/unboarding, takeoff/landing.

P1-16. The presentation of data is becoming more and more important in today's Internet. Some people argue that the TCP/IP protocol suite needs to add a new layer to take care of the presentation of data (see Appendix C). If this new layer is added in the future, where should its position be in the suite? Redraw Figure 1.17 to include this layer.

CHAPTER 2

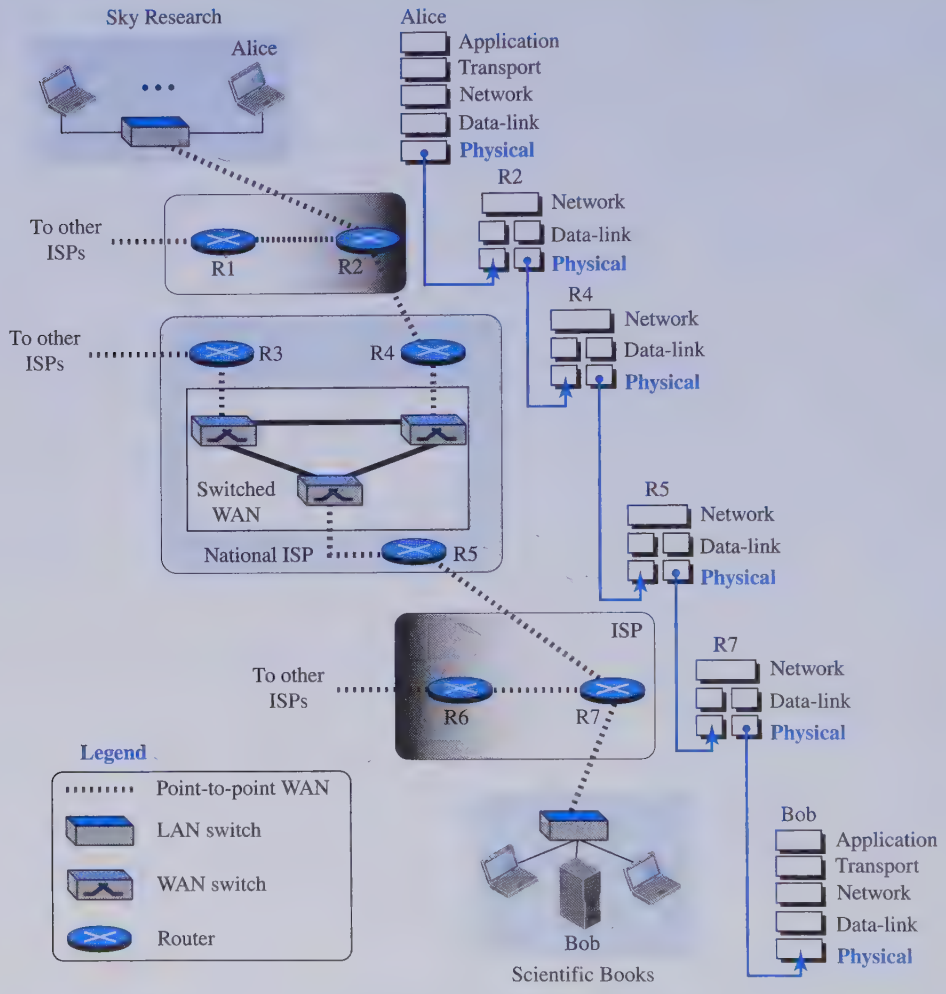
Physical Layer

We start the discussion of the TCP/IP protocol suite with the first layer of the suite: the physical layer. This chapter is divided into six sections.

- In the first section, we discuss signals. We show how data and signals can be both analog and digital. We then discuss analog and digital signals and their respective characteristics.
- In the second section, we discuss transmission. We first discuss transmission impairments (attenuation, distortion, and noise). We then discuss data rate limits. Finally, we discuss the performance of a network.
- In the third section, we concentrate on digital transmission. We first show how to convert digital data to digital signals. We then show how to convert analog data to digital signals.
- In the fourth section, we concentrate on analog transmission. We first show how to convert digital data to analog signals. We then show how to convert analog data to analog signals.
- In the fifth section, we discuss multiplexing to benefit more from the bandwidth available. We first discuss frequency-division multiplexing, and then we discuss time-division multiplexing.
- In the sixth section, we go below the physical layer and discuss the transmission media that are used in data communication. We introduce both guided (wires and cables) and unguided media (air).

Figure 2.1 shows a scenario in which a scientist, Alice, working at Sky Research, needs to order a book for her research from Bob, the manager of Scientific Books, an online bookstore.

Figure 2.1 *Communication at the physical layer*

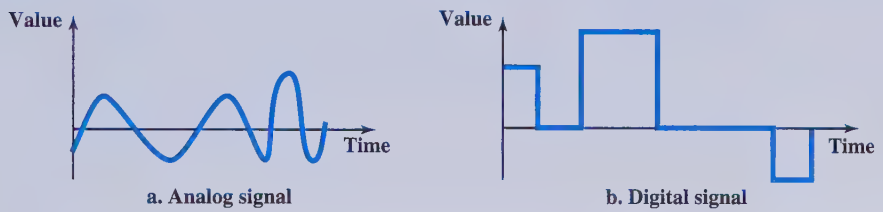


We assume that there are only three Internet service providers (ISPs) between Alice and Bob. An ISP is a company that provides Internet services to an area. In our scenario, Alice is connected to her ISP, and Bob is connected to his ISP. The two local ISPs are connected through a national ISP. Figure 2.1 shows seven routers, R1 to R7, and some switches. A router is a device that routes data when there is more than one possible route. A switch is a device that connects some devices when there is no need for routing. We discuss routers and switches in later chapters. For the moment, we are interested in a physical route between Alice and Bob that contains only four routers: R2, R4, R5, and R7.

2.1 Signals

What are really exchanged between Alice and Bob are *data* (information). However, what goes through the network connecting Alice to Bob at the physical layer are *signals*. When Alice sends a message to Bob, the message is changed to electrical signals; when Bob receives the message, the signals are changed back to the message. The reverse situation occurs when Bob sends a message to Alice. The signals can be *analog* or *digital*. An analog signal takes many values; a digital signal takes a limited number of values as shown in Figure 2.2.

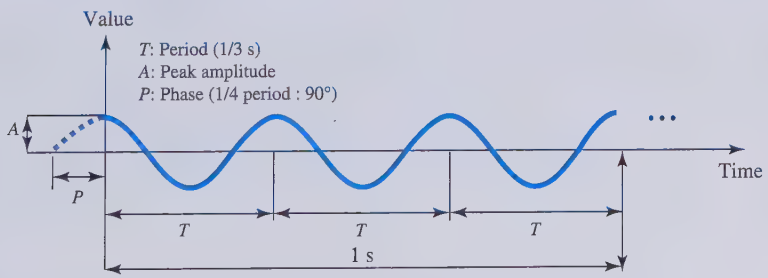
Figure 2.2 *Comparison of analog and digital signals*



2.1.1 Analog Signals

Let us first concentrate on analog signals. An analog signal can take one of two forms: *periodic* or *aperiodic* (*nonperiodic*). In data communications, we commonly use periodic analog signals. They can be classified as simple or composite. A simple periodic analog signal, a **sine wave**, cannot be decomposed into simpler signals. Figure 2.3 shows a sine wave.

Figure 2.3 *A sine wavey*



The sine wave is the most fundamental form of a periodic analog signal. When we visualize it as a simple oscillating curve, its change over the course of a cycle is smooth and consistent. A sine wave can be represented by three parameters: *period*, *peak amplitude*, and *phase*. These three parameters fully describe a sine wave. The frequency is not an independent parameter: It is the inverse of the period.

Peak Amplitude

The ***peak amplitude*** of a signal is the absolute value of its highest intensity. For electrical signals, peak amplitude is normally measured in volts.

Period and Frequency

The ***period*** (T) refers to the amount of time, in seconds, a signal needs to complete one cycle. The ***frequency*** (f), measured in hertz (Hz), refers to the number of periods in 1 s. Note that period and frequency are just one characteristic defined in two ways. Period and frequency are inverses of each other, in other words ($f = 1/T$).

Example 2.1

The voltage of a battery is constant (for example, 1.5 V). However, this can be considered to be periodic with a frequency of 0 (and a period of infinity).

Example 2.2

The electrical voltage in our homes in the United States is periodic with a peak value between 110 to 120 V. Its frequency is 60 Hz.

Phase

The term **phase** describes the position of the waveform relative to time 0. If we think of the wave as something that can be shifted backward or forward along the time axis, phase describes the amount of that shift. It indicates the status of the first cycle. Phase is measured in degrees or radians (360° is 2π rad).

Wavelength

The term **wavelength** is another characteristic of a signal traveling through a transmission medium. The wavelength is the distance a simple signal can travel in one period. The wavelength binds the period or the frequency of a simple sine wave to the propagation speed in the medium.

While the frequency of a signal is independent of the medium, the wavelength depends on both the frequency and the medium. The wavelength can be calculated if one is given the propagation speed of the medium and the frequency of the signal. If we represent wavelength by λ, propagation speed by c, and frequency by f, we get

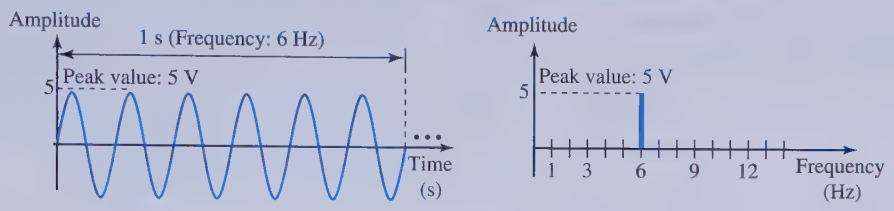
$$\lambda = c / f = c \times T$$

Time and Frequency Domains

A sine wave is comprehensively defined by its amplitude, frequency, and phase. We have been showing a sine wave by using what is called a **time-domain plot**, which shows changes in signal amplitude with respect to time. To show the relationship between amplitude and frequency, we can use a **frequency-domain plot**. Figure 2.4 shows a signal in both the time and frequency domains.

In the frequency domain, a sine wave is represented by one spike. The position of the spike shows the frequency; its height shows the peak amplitude.

Figure 2.4 *The time-domain and frequency-domain plots of a sine wave*



a. Time domain

b. Frequency domain

Composite Signals

So far, we have focused on simple sine waves. Simple sine waves have many applications in daily life, such as sending energy from one place to another. However, if we had only one single sine wave to convey a conversation over the phone, it would make no sense and carry no information. We would just hear a buzz. We need to send a composite signal to communicate data. A **composite signal** is made up of many simple sine waves.

Bandwidth

The range of frequencies contained in a composite signal is its **bandwidth**. The bandwidth is the difference between the lowest and highest frequencies in the signal. For example, if a composite signal contains frequencies between 1000 and 5000, its bandwidth is 5000 – 1000, or 4000.

The bandwidth of a composite signal is the difference between the highest and the lowest frequencies contained in that signal.

2.1.2 Digital Signals

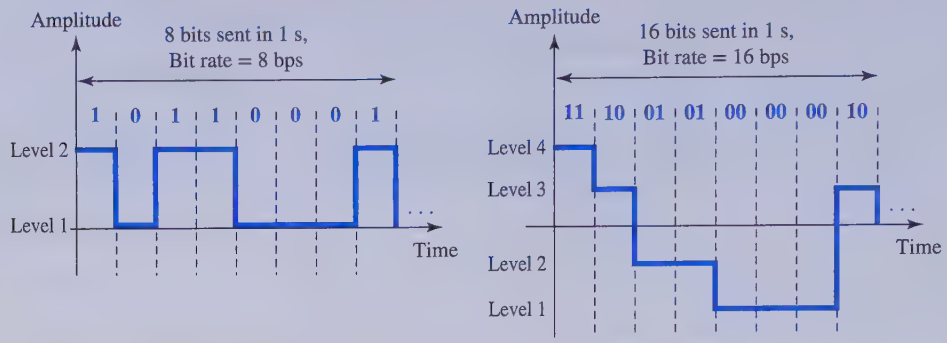
In addition to being represented by an analog signal, information can also be represented by a digital signal. For example, a value 1 can be encoded as a positive voltage and a value 0 as zero voltage. A digital signal can have more than two levels. In this case, we can send more than 1 bit for each level. Figure 2.5 shows two signals, one with two levels and the other with four.

We send 1 bit per level in Figure 2.5a and 2 bits per level in Figure 2.5b. In general, if a signal has L levels, each level needs $\log_2 L$ bits.

Bit Rate

Most digital signals are nonperiodic, and thus period and frequency are not appropriate characteristics of digital signals. Another term—*bit rate* (instead of *frequency*)—is used to describe digital signals. The **bit rate** is the number of bits sent in 1 s, expressed in bits per second (bps). The bit rate can be represented as kbps (kilo bits per second, where kilo means one thousand) or Mbps (mega bits per second, where mega means one million).

Figure 2.5 *Two digital signals: one with two signal levels and the other with four signal levels*



a. A digital signal with two levels

b. A digital signal with four levels

Example 2.3

Assume we need to download text documents at the rate of 100 pages per minute. What is the required bit rate of the channel? A page is an average of 24 lines with 80 characters in each line. If we assume that one character requires 8 bits, the bit rate is

$$100 \times 24 \times 80 \times 8 = 1{,}536{,}000 \text{ bps} = 1.536 \text{ Mbps}$$

Bit Length

We discussed the concept of the wavelength for an analog signal: the distance one cycle occupies on the transmission medium. We can define something similar for a digital signal: the bit length. The **bit length** is the distance 1 bit occupies on the transmission medium.

$$\text{Bit length} = 1 / (\text{bit rate})$$

Example 2.4

The length of the bit in Example 2.3 is

$$1/1{,}536{,}000 = 0.000000651 \text{ s} = 0.651 \ \mu\text{s}$$

Transmission of Digital Signals

The previous discussion asserts that a digital signal is a composite analog signal with frequencies between zero and infinity. For the remainder of the discussion, let us consider the case of a nonperiodic digital signal, similar to the ones we encounter in data communications. The fundamental question is, how can we send a digital signal from point A to point B? We can transmit a digital signal by using one of two different approaches: *baseband transmission* or *broadband transmission*. **Baseband transmission** means sending a digital signal over a channel without changing it to an analog signal. Broadband transmission or modulation means changing the digital signal to an analog signal for transmission.

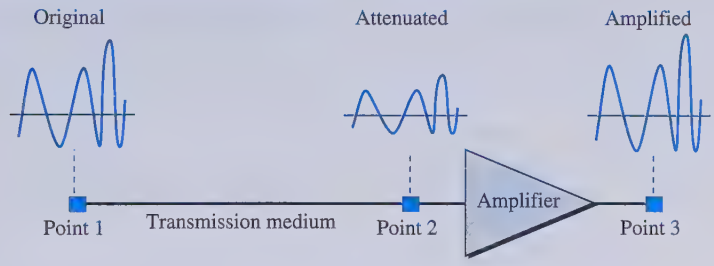
2.2 Signal Impairment

Signals travel through transmission media, which are not perfect. The imperfection causes signal impairment. This means that the signal at the beginning of the medium is not the same as the signal at the end of the medium. What is sent is not what is received. Three causes of impairment are *attenuation*, *distortion*, and *noise*.

2.2.1 Attenuation and Amplification

Attenuation means a loss of energy. When a signal, simple or composite, travels through a medium, it loses some of its energy in overcoming the resistance of the medium. To compensate for this loss, we need **amplification**. Figure 2.6 shows the effect of attenuation and amplification.

Figure 2.6 *Attenuation and amplification*



To show that a signal has lost or gained strength, engineers use the unit of the decibel. The **decibel (dB)** measures the relative strengths of two signals or one signal at two different points. Note that the decibel is negative if a signal is attenuated and positive if a signal is amplified. Variables P_1 and P_2 are the powers of a signal at points 1 and 2, respectively.

$$\mathbf{dB = 10 \log_{10} (P_2/P_1)}$$

Example 2.5

Suppose a signal travels through a transmission medium and its power is reduced to one-half. This means that $P_2 = 0.5P_1$. In this case, the attenuation (loss of power) can be calculated as

$$\mathbf{10 \log_{10} P_2 / P_1 = 10 \log_{10} (0.5P_1) / P_1 = 10 \log_{10} 0.5 = 10 \times (-0.3) = -3\ dB}$$

A loss of 3 dB (−3 dB) is equivalent to losing one-half the power.

2.2.2 Distortion

Distortion means that the signal changes its form or shape. Distortion can occur in a composite signal made up of different frequencies. Each signal component has its own propagation speed through a medium and, therefore, its own delay in arriving at the

final destination. Differences in delay may create a difference in phase if the delay is not exactly the same as the period duration.

Noise

Noise is another cause of impairment. Several types of noise, such as *thermal noise*, *induced noise*, *crosstalk*, and *impulse noise*, may corrupt the signal. **Thermal noise** is the random motion of electrons in a wire, which creates an extra signal not originally sent by the transmitter. **Induced noise** comes from sources such as motors and appliances. These devices act as a sending antenna, and the transmission medium acts as the receiving antenna. **Crosstalk** is the effect of one wire on the other. One wire acts as a sending antenna and the other as the receiving antenna. **Impulse noise** is a spike (a signal with high energy and very short duration) that comes from power lines, lightning, and so on.

Signal-to-Noise Ratio (SNR)

To find the theoretical bit-rate limit, we need to know the ratio of the signal power to the noise power. The **signal-to-noise ratio** is defined as

$$\textbf{SNR} = \textbf{(average signal power)} \,/\, \textbf{(average noise power)}$$

We need to consider the average signal power and the average noise power because these may change with time. Because SNR is the ratio of two powers, it is often described in decibel units, SNR_{dB}, as

$$\textbf{SNRdB} = 10 \log_{10} \textbf{SNR}$$

2.2.3 Data Rate Limits

A very important consideration in data communications is how fast we can send data, in bits per second, over a channel. Data rate depends on three factors:

1. The bandwidth available
2. The level of the signals we use
3. The quality of the channel (the level of noise)

Two theoretical formulas were developed to calculate the data rate: one by Nyquist for a noiseless channel, another by Shannon for a noisy channel.

Noiseless Channel: Nyquist Bit Rate

For a noiseless channel, the **Nyquist bit rate** formula defines the theoretical maximum bit rate.

$$\textbf{BitRate} = 2 \times B \times \log_2 L$$

In this formula, B is the bandwidth of the channel, L is the number of signal levels used to represent data, and BitRate is the number of bits per second.

According to the formula, we might think that, given a specific bandwidth, we can have any bit rate we want by increasing the number of signal levels. Although the idea is theoretically correct, practically there is a limit. When we increase the number of signal levels, we impose a burden on the receiver. If the number of levels in a signal is just 2,

the receiver can easily distinguish between a 0 and a 1. If the level of a signal is 64, the receiver must be very sophisticated to distinguish between 64 different levels. In other words, increasing the levels of a signal reduces the reliability of the system.

Example 2.6

We need to send 265 kbps over a noiseless (ideal) channel with a bandwidth of 20 kHz. How many signal levels do we need? We can use the Nyquist formula as shown:

$$265{,}000 = 2 \times 20{,}000 \times \log_2 L \quad \rightarrow \quad \log_2 L = 6.625 \qquad L = 2^{6.625} = 98.7 \text{ levels}$$

Because this result is not a power of 2, we need to either increase the number of levels or reduce the bit rate. If we have 128 levels, the bit rate is 280 kbps. If we have 64 levels, the bit rate is 240 kbps.

Noisy Channel: Shannon Capacity

In reality, we cannot have a noiseless channel; the channel is always noisy. Claude Shannon introduced a formula, called the **Shannon capacity**, to determine the theoretical highest data rate for a noisy channel:

$$C = B \times \log_2 (1 + \text{SNR})$$

In this formula, B is the bandwidth of the channel, SNR is the signal-to-noise ratio, and C is the capacity of the channel in bits per second. Note that in the Shannon formula there is no indication of the signal level, which means that no matter how many levels we have, we cannot achieve a data rate higher than the capacity of the channel. In other words, the formula defines a characteristic of the channel, not the method of transmission. The capacity defines the upper boundary of the channel bit rate.

Example 2.7

Consider an extremely noisy channel in which the value of the signal-to-noise ratio is almost zero. In other words, the noise is so strong that the signal is faint. For this channel the capacity C is calculated as

$$C = B \log_2 (1 + \text{SNR}) = B \log_2 (1 + 0) = B \log_2 1 = B \times 0 = 0$$

This means that the capacity of this channel is zero regardless of the bandwidth. In other words, the data are so corrupted in this channel that they are useless when received.

Example 2.8

We can calculate the theoretical highest bit rate of a regular telephone line. A telephone line normally has a bandwidth of 3000 Hz (300 to 3300 Hz) assigned for data communications. The signal-to-noise ratio is usually 3162. For this channel the capacity is calculated as

$$C = B \log_2 (1 + \text{SNR}) = 3000 \log_2 (1 + 3162) = 34{,}881 \text{ bps}$$

This means that the highest bit rate for a telephone line is 34.881 kbps. If we want to send data faster than this, we can either increase the bandwidth of the line or improve the signal-to-noise ratio.

Using Both Limits

In practice, we need to use both methods to find the limits and signal levels. Let us show this with an example.

Example 2.9

We have a channel with a 1-MHz bandwidth. The SNR for this channel is 63. What are the appropriate bit rate and signal level?

Solution

First, we use the Shannon formula to find the upper limit.

$$C = B \log_2 (1 + \text{SNR}) = 10^6 \log_2 (1 + 63) = 10^6 \log_2 64 = 6 \text{ Mbps}$$

The Shannon formula gives us 6 Mbps, the upper limit. For better performance we choose something lower, 4 Mbps, for example. Then we use the Nyquist formula to find the number of signal levels.

$$4 \text{ Mbps} = 2 \times 1 \text{ MHz} \times \log_2 L \rightarrow \log_2 L = 2 \rightarrow L = 4$$

The Shannon capacity gives us the upper limit;
the Nyquist formula tells us how many signal levels we need

2.2.4 Performance

Up to now, we have discussed the tools of transmitting data (signals) over a network and how the data behave. One important issue in networking is the performance of the network—how good is it?

Bandwidth

One characteristic that measures network performance is bandwidth. However, the term can be used in two different contexts with two different measuring values: bandwidth in hertz and bandwidth in bits per second. Bandwidth in hertz is the range of frequencies involved. Bandwidth in bits per second can be defined as the number of bits a channel can pass.

Example 2.10

The bandwidth of a subscriber line is 4 kHz for voice or data. The bandwidth of this line for data transmission can be up to 56 kbps, using a sophisticated device to change the digital signal to analog. If the telephone company improves the quality of the line and increases the bandwidth to 8 kHz, we can send 112 kbps.

Example 2.11

One can say the bandwidth of a fast Ethernet network (discussed in future chapters) is 100 Mbps. This means that this network can send 100 Mbps.

Throughput

The **throughput** is a measure of how fast we can actually send data through a network. Although, at first glance, bandwidth in bits per second and throughput seem the same, they are different. A link may have a bandwidth of B bps, but we can send only T bps through this link, with T always less than B. In other words, the bandwidth is a potential measurement of a link; the throughput is an actual measurement of how fast we can send

data. For example, we may have a link with a bandwidth of 1 Mbps, but the devices connected to the end of the link may handle only 200 kbps. This means that we cannot send more than 200 kbps through this link.

Imagine a highway designed to transmit 1000 cars per minute from one point to another. However, if there is congestion on the road, this figure may be reduced to 100 cars per minute. The bandwidth is 1000 cars per minute; the throughput is 100 cars per minute.

Latency (Delay)

The latency or delay defines how long it takes for an entire message to completely arrive at the destination from the time the first bit is sent out from the source. We say that normally there are four types of delay: propagation delay, transmission delay, queuing delay, and processing delay. The latency or total delay is

Latency = propagation delay + transmission delay + queuing delay + processing delay

Bandwidth-Delay Product

Bandwidth and delay are two performance metrics of a link. However, what is very important in data communications is the product of the two, the bandwidth-delay product. Let us elaborate on this issue, using two hypothetical cases as examples.

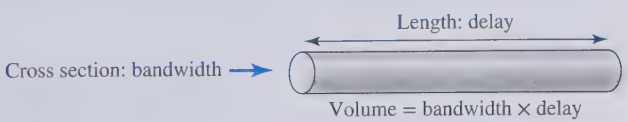
- **Case 1.** Let us assume that we have a link with a bandwidth of 1 bps. We also assume that the delay of the link is 5 s (also unrealistic). We want to see what the bandwidth-delay product means in this case. We can see that the bandwidth-delay product (1×5) is the maximum number of bits that can fill the link. There can be no more than 5 bits at any one time on the link.
- **Case 2.** Now assume we have a bandwidth of 5 bps with a delay of 5 s. We can see that there can be a maximum of $5 \times 5 = 25$ bits on the line. The reason is that, at each second, there are 5 bits on the line.

The bandwidth-delay product defines the number of bits that can fill the link.

Example 2.12

We can think about the link between two points as a pipe. The cross section of the pipe represents the bandwidth, and the length of the pipe represents the delay. We can say the volume of the pipe defines the bandwidth-delay product, as shown in Figure 2.7.

Figure 2.7 *Bandwidth-delay product*



Jitter

Another performance issue that is related to delay is **jitter**. We can roughly say that jitter is a problem if different packets of data encounter different delays and the application using the data at the receiver site is time-sensitive (audio and video data, for example). If the delay for the first packet is 20 ms, for the second is 45 ms, and for the third is 40 ms, then the real-time application that uses the packets endures jitter.

2.3 Digital Transmission

A computer network is designed to send information from one point to another. This information needs to be converted to either a digital signal or an analog signal for transmission. In this section, we discuss the first choice; in Section 2.4, we discuss the second choice.

In digital transmission, if data are digital, we need to use **digital-to-digital conversion** techniques. If data are analog, we need to use **analog-to-digital conversion**.

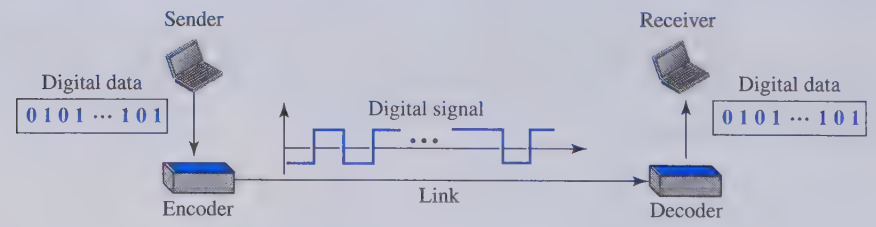
2.3.1 Digital-to-Digital Conversion

If we use digital transmission and our data are already digital, we need digital-to-digital conversion. The conversion involves three techniques: *line coding*, *block coding*, and *scrambling*. Line coding is always needed; block coding and scrambling may or may not be needed.

Line Coding

Line coding is the process of converting digital data to digital signals. We assume that data, in the form of text, numbers, graphical images, audio, or video, are stored in computer memorys as sequences of bits (see Figure 2.8).

Figure 2.8 *Line coding and decoding*



Line coding converts a sequence of bits to a digital signal. At the sender, digital data are encoded into a digital signal; at the receiver, the digital data are re-created by decoding the digital signal.

Block Coding

We need redundancy to ensure synchronization and to provide some kind of inherent error detecting. Block coding can give us this redundancy and improve the performance of line coding. In general, **block coding** changes a block of m bits into a block of n bits, where n is larger than m. Block coding is referred to as an mB/nB encoding technique.

Block coding normally involves three steps: division, substitution, and combination. In the division step, a sequence of bits is divided into groups of m bits. For example, in 4B/5B encoding, the original bit sequence is divided into 4-bit groups. The heart of block coding is the substitution step. In this step, we substitute an m-bit group for an n-bit group. For example, in the case of 4B/5B encoding, we substitute a 4-bit code for a 5-bit group. Finally, the n-bit groups are combined to form a stream.

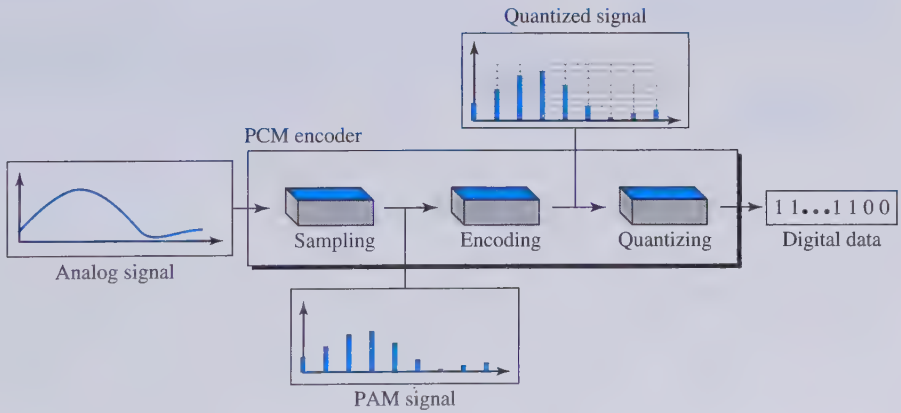
2.3.2 Analog-to-Digital Conversion

Sometimes we have an analog signal such as one created by a microphone or camera. The tendency today is to change an analog signal to digital data because the digital signal is less susceptible to noise. In this section we describe two techniques, pulse code modulation and delta modulation. After the digital data are created (digitization), we can use one of the techniques described in Section 2.3.1, to convert the digital data to a digital signal.

Pulse Code Modulation (PCM)

The most common technique used to change an analog signal to digital data (**digitization**) is called **pulse code modulation (PCM)**. A PCM encoder has three processes, as shown in Figure 2.9.

Figure 2.9 *Components of a PCM encoder*



The three processes are:

1. The analog signal is sampled every T s.
2. The sampled signal is quantized, which means every sample is considered as a pulse.
3. The quantized values (pulses) are encoded as streams of bits.

Example 2.13

We want to digitize the human voice. What is the bit rate, assuming 8 bits per sample?

Solution

The human voice normally contains frequencies from 0 to 4000 Hz. So the sampling rate and bit rate are calculated as

$$\text{Sampling rate} = 4000 \times 2 = 8000 \text{ samples/s}$$
$$\text{Bit rate} = 8000 \times 8 = 64{,}000 \text{ bps} = 64 \text{ kbps}$$

PCM Bandwidth

It can be proved that the minimum bandwidth of the digital signal is

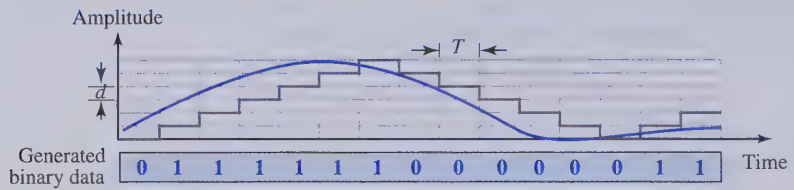
$$B_{\text{min}} = n_b \times B_{\text{analog}}$$

This means the minimum bandwidth of the digital signal is n_b times greater than the bandwidth of the analog signal. This is the price we pay for digitization.

Delta Modulation (DM)

PCM is a very complex technique. Other techniques have been developed to reduce its complexity. The simplest is **delta modulation (DM)**. PCM finds the value of the signal amplitude for each sample; DM finds the change from the previous sample. Figure 2.10 shows the process. Note that there are no code words here; bits are sent one after the other.

Figure 2.10 *The process of delta modulation*



2.4 Analog Transmission

Although digital transmission is desirable, it needs a low-pass channel (a channel that starts from 0); analog transmission is the only choice if we have a bandpass channel (a channel that does not start from zero). Converting digital data to a bandpass analog signal is traditionally called *digital-to-analog conversion*. Converting a low-pass analog signal to a bandpass analog signal is traditionally called *analog-to-analog conversion.* In this section, we discuss these two types of conversions.

2.4.1 Digital-to-Analog Conversion

Digital-to-analog conversion is the process of changing one of the characteristics of an analog signal based on the information in digital data. A sine wave is defined by three characteristics: amplitude, frequency, and phase. Any of the three characteristics can be altered in this way, giving us at least three mechanisms for modulating digital data into an analog signal: **amplitude shift keying (ASK)**, **frequency shift keying (FSK)**, and **phase shift keying (PSK)**. In addition, there is a fourth (and better) mechanism that combines changing both the amplitude and phase, called **quadrature amplitude modulation (QAM)**. QAM is the most efficient of these options and is the mechanism commonly used today.

Amplitude Shift Keying

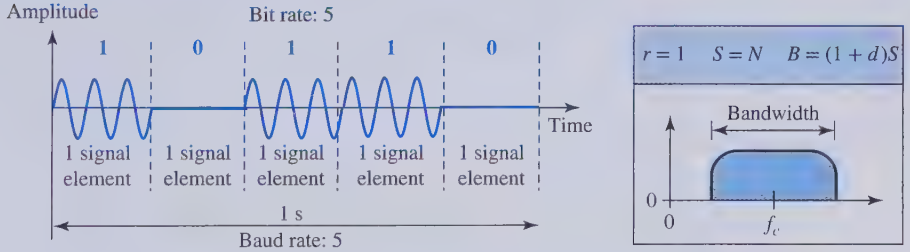
In amplitude shift keying, the amplitude of the carrier signal is varied to create signal elements. Both frequency and phase remain constant while the amplitude changes.

Binary ASK (BASK)

ASK is normally implemented using only two levels. This is referred to as *binary amplitude shift keying* or *on-off keying* (OOK). The peak amplitude of one signal level is 0; the other is the same as the amplitude of the carrier frequency. Figure 2.11 gives a conceptual view of binary ASK.

Figure 2.11 also shows the bandwidth for ASK. Although the carrier signal is only one simple sine wave, the process of modulation produces a nonperiodic composite signal. This signal, as was discussed before, has a continuous set of frequencies. As we expect, the bandwidth is proportional to the signal rate (baud). However, there is normally another factor involved, called d, which depends on the modulation and filtering process. The value of d is between 0 and 1. This means that the bandwidth can be expressed as shown, where S is the signal rate and B is the bandwidth.

Figure 2.11 *Binary amplitude shift keying*



The formula $B = (1 + d)S$ shows that the required bandwidth has a minimum value of S and a maximum value of $2S$. The most important point here is the location of the bandwidth. The middle of the bandwidth is where f_c, the carrier frequency, is located. This means if we have a bandpass channel available, we can choose our f_c so that the modulated signal occupies that bandwidth. This is, in fact, the most important advantage of digital-to-analog conversion. We can shift the resulting bandwidth to match what is available.

Multilevel ASK

The above discussion uses only two amplitude levels. We can have multilevel ASK in which there are more than two levels. We can use 4, 8, 16, or more different amplitudes for the signal and modulate the data using 2, 3, 4, or more bits at a time. In these cases, $r = 2$, $r = 3$, $r = 4$, and so on. Although this is not implemented with pure ASK, it is implemented with QAM (as we will see later).

Frequency Shift Keying (FSK)

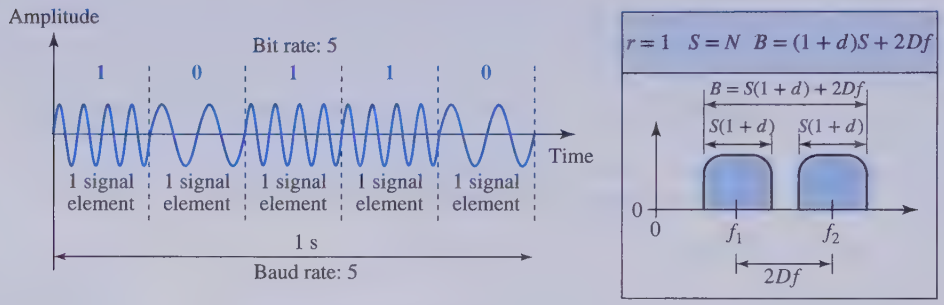
In frequency shift keying, the frequency of the carrier signal is varied to represent data. The frequency of the modulated signal is constant for the duration of one signal element,

but changes for the next signal element if the data element changes. Both the peak amplitude and phase remain constant for all signal elements.

Binary FSK (BFSK)

One way to think about binary FSK (BFSK) is to consider two carrier frequencies. In Figure 2.12, we have selected two carrier frequencies, f_1 and f_2. We use the first carrier frequency if the data element is 0; we use the second if the data element is 1. However, note that this example is unrealistic and used only for demonstration purposes. Normally the carrier frequencies are very high, and the difference between them is very small.

Figure 2.12 *Binary frequency shift keying*



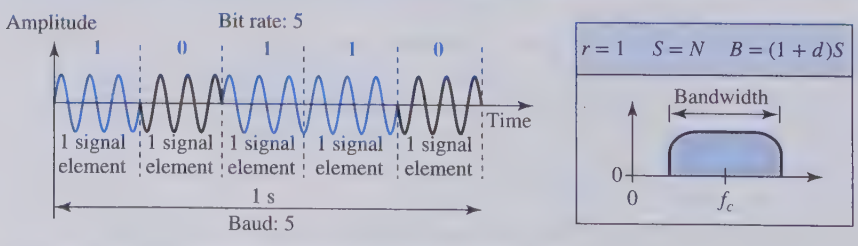
Phase Shift Keying

In phase shift keying, the phase of the carrier is varied to represent two or more different signal elements. Both peak amplitude and frequency remain constant as the phase changes. Today, PSK is more common than ASK or FSK. However, we will see shortly that QAM, which combines ASK and PSK, is the dominant method of digital-to-analog modulation.

Binary PSK (BPSK)

The simplest PSK is binary BPSK, in which we have only two signal elements, one with a phase of 0°, and the other with a phase of 180°. Figure 2.13 gives a conceptual view of PSK. Binary PSK is as simple as binary ASK with one big advantage—it is less susceptible to noise.

Figure 2.13 *Binary phase shift keying*



2.4.2 Analog-to-Analog Conversion

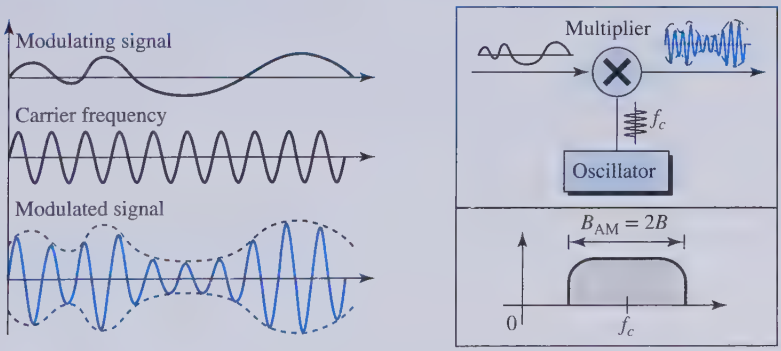
Analog-to-analog conversion, or analog modulation, is the representation of analog information by an analog signal. One may ask why we need to modulate an analog signal; it is already analog. Modulation is needed if the medium is bandpass in nature or if only a bandpass channel is available to us. An example is radio. The government assigns a narrow bandwidth to each radio station. The analog signal produced by each station is a low-pass signal, all in the same range. To be able to listen to different stations, the low-pass signals need to be shifted, each to a different range.

Analog-to-analog conversion can be accomplished in three ways: **amplitude modulation (AM)**, **frequency modulation (FM)**, and **phase modulation (PM)**. FM and PM are usually categorized together.

Amplitude Modulation

In AM transmission, the carrier signal is modulated so that its amplitude varies with the changing amplitudes of the modulating signal. The frequency and phase of the carrier remain the same; only the amplitude changes to follow variations in the information. Figure 2.14 shows how this concept works. The modulating signal is the envelope of the carrier.

Figure 2.14 *Amplitude modulation*



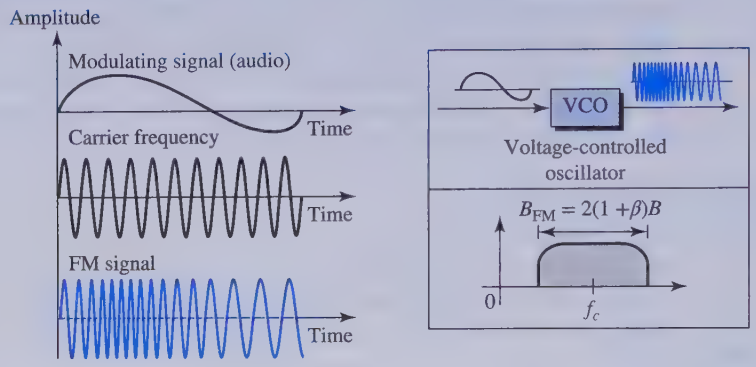
As Figure 2.14 shows, AM is normally implemented by using a simple multiplier because the amplitude of the carrier signal needs to be changed according to the amplitude of the modulating signal. Figure 2.14 also shows the bandwidth of an AM signal. The modulation creates a bandwidth that is twice the bandwidth of the modulating signal and covers a range centered on the carrier frequency. However, the signal components above and below the carrier frequency carry exactly the same information. For this reason, some implementations discard one-half of the signals and cut the bandwidth in half.

Frequency Modulation

In FM transmission, the frequency of the carrier signal is modulated to follow the changing voltage level (amplitude) of the modulating signal. The peak amplitude and phase

of the carrier signal remain constant, but as the amplitude of the information signal changes, the frequency of the carrier changes correspondingly. Figure 2.15 shows the relationships of the modulating signal, the carrier signal, and the resultant FM signal.

Figure 2.15 *Frequency modulation*



As Figure 2.15 shows, FM is normally implemented by using a voltage-controlled oscillator as with FSK. The frequency of the oscillator changes according to the input voltage, which is the amplitude of the modulating signal. Figure 2.15 also shows the bandwidth of an FM signal. The actual bandwidth is difficult to determine exactly, but it can be shown empirically as $B_{FM} = 2(1 + \beta)B$ where β is a factor that depends on modulation technique with a common value of 4.

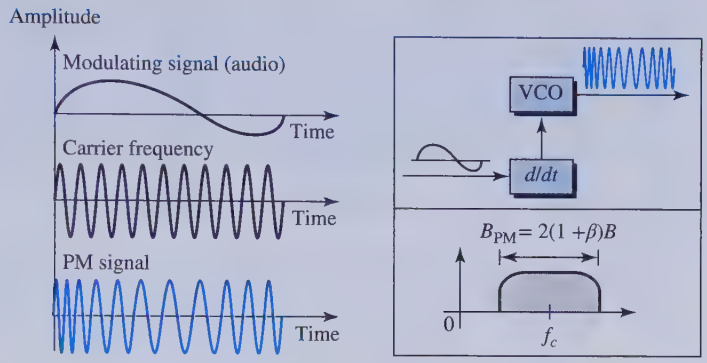
Phase Modulation

In PM transmission, the phase of the carrier signal is modulated to follow the changing voltage level (amplitude) of the modulating signal. The peak amplitude and frequency of the carrier signal remain constant, but as the amplitude of the information signal changes, the phase of the carrier changes correspondingly. It can be proven mathematically that PM is the same as FM with one difference. In FM, the instantaneous change in the carrier frequency is proportional to the amplitude of the modulating signal; in PM the instantaneous change in the carrier frequency is proportional to the derivative of the amplitude of the modulating signal. Figure 2.16 shows the relationships of the modulating signal, the carrier signal, and the resultant PM signal.

As Figure 2.16 shows, PM is normally implemented by using a voltage-controlled oscillator along with a derivative. The frequency of the oscillator changes according to the derivative of the input voltage, which is the amplitude of the modulating signal.

Figure 2.16 also shows the bandwidth of a PM signal. The actual bandwidth is difficult to determine exactly, but it can be shown empirically that it is several times that of the analog signal. Although the formula shows the same bandwidth for FM and PM, the value of β is lower in the case of PM (around 1 for narrowband and 3 for wideband).

Figure 2.16 *Phase modulation*

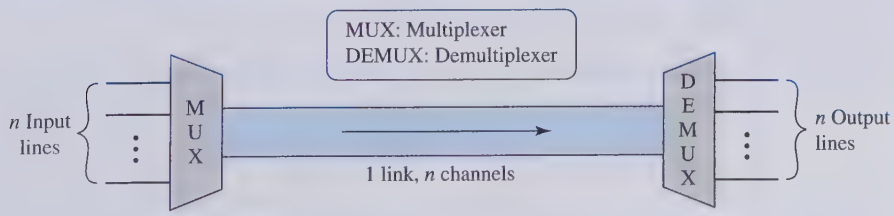


2.5 MULTIPLEXING

Whenever the bandwidth of a medium linking two devices is greater than the bandwidth needs of the devices, the link can be shared. **Multiplexing** is the set of techniques that allows the simultaneous transmission of multiple signals across a single data link. As data and telecommunications use increases, so does traffic. We can accommodate this increase by continuing to add individual links each time a new channel is needed, or we can install higher-bandwidth links and use each to carry multiple signals. Today's technology includes high-bandwidth media such as optical fiber and terrestrial and satellite microwaves. Each has a bandwidth far in excess of that needed for the average transmission signal. If the bandwidth of a link is greater than the bandwidth needs of the devices connected to it, the bandwidth is wasted. An efficient system maximizes the utilization of all resources; bandwidth is one of the most precious resources we have in data communications.

In a multiplexed system, *n* lines share the bandwidth of one link. Figure 2.17 shows the basic format of a multiplexed system. The lines on the left direct their transmission streams to a *multiplexer,* which combines them into a single stream (many-to-one). At

Figure 2.17 *Dividing a link into channels*



the receiving end, that stream is fed into a *demultiplexer*, which separates the stream back into its component transmissions (one-to-many) and directs them to their corresponding lines. In Figure 2.17, the word *link* refers to the physical path. The word *channel* refers to the portion of a link that carries a transmission between a given pair of lines. One link can have many (n) channels.

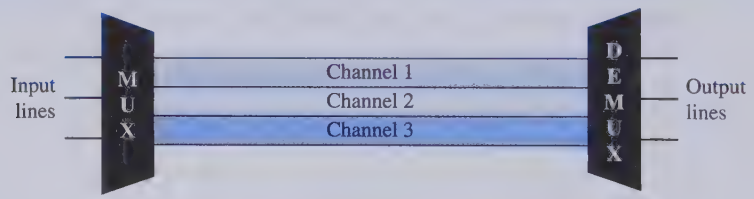
There are three basic multiplexing techniques: frequency-division multiplexing (FDM), wavelength-division multiplexing (WDM), and time-division multiplexing (TDM). The first two are techniques designed for analog signals; the third, for digital signals.

2.5.1 Frequency-Division Multiplexing

Frequency-division multiplexing (FDM) is an analog technique that can be applied when the bandwidth of a link (in hertz) is greater than the combined bandwidths of the signals to be transmitted. In FDM, signals generated by each sending device modulate different carrier frequencies. These modulated signals are then combined into a single composite signal that can be transported by the link. Carrier frequencies are separated by sufficient bandwidth to accommodate the modulated signal. These bandwidth ranges can be thought of as channels through which the various signals travel. Channels can be separated by strips of unused bandwidth—**guard bands**—to prevent signals from overlapping. In addition, carrier frequencies must not interfere with the original data frequencies.

Figure 2.18 gives a conceptual view of FDM. In this illustration, the transmission path is divided into three parts, each representing a channel that carries one transmission.

Figure 2.18 *Frequency-division multiplexing*

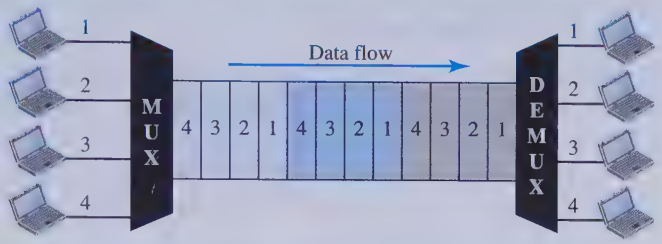


We consider FDM to be an analog multiplexing technique; however, this does not mean that FDM cannot be used to combine sources sending digital signals. Digital signals can be converted to analog signals before FDM is used to multiplex them.

2.5.2 Time-Division Multiplexing

Time-division multiplexing (TDM) is a digital process that allows several connections to share the high bandwidth of a link. Instead of sharing a portion of the bandwidth as in FDM, time is shared. Each connection occupies a portion of time in the link. Figure 2.19 gives a conceptual view of TDM. Note that the same link is used as in FDM; here, however, the link is shown sectioned by time rather than by frequency. In the figure, portions of signals 1, 2, 3, and 4 occupy the link sequentially.

Figure 2.19 *TDM*



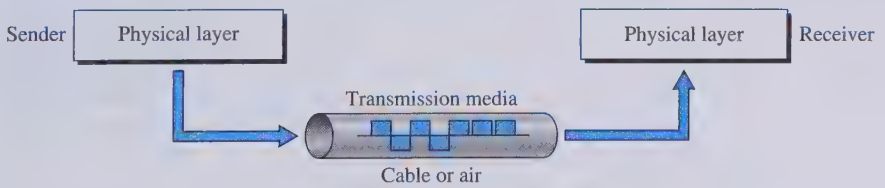
Note that in Figure 2.19 we are concerned with only multiplexing, not switching. This means that all the data in a message from source 1 always go to one specific destination, be it 1, 2, 3, or 4. The delivery is fixed and unvarying, unlike switching.

We also need to remember that TDM is, in principle, a digital multiplexing technique. Digital data from different sources are combined into one timeshared link. However, this does not mean that the sources cannot produce analog data; analog data can be sampled, changed to digital data, and then multiplexed by using TDM.

2.6 Transmission Media

We discussed many issues related to the physical layer in this chapter. In this section, we discuss **transmission media**. Transmission media are actually located below the physical layer and are directly controlled by the physical layer. Figure 2.20 shows the position of transmission media in relation to the physical layer. Because we discussed the physical layer in this chapter, we now also briefly discuss the transmission media that carries signals for the physical layer.

Figure 2.20 *Transmission media and physical layer*



A transmission medium can be broadly defined as anything that can carry information from a source to a destination. For example, the transmission medium for two people having a dinner conversation is the air. The air can also be used to convey the message in a smoke signal or semaphore.

In data communications the definition of the information and the transmission medium is more specific. The transmission medium is usually free space, metallic cable, or fiber-optic cable. The information is usually a signal that is the result of a conversion of data to signal.

In telecommunications, transmission media can be divided into two broad categories: guided and unguided.

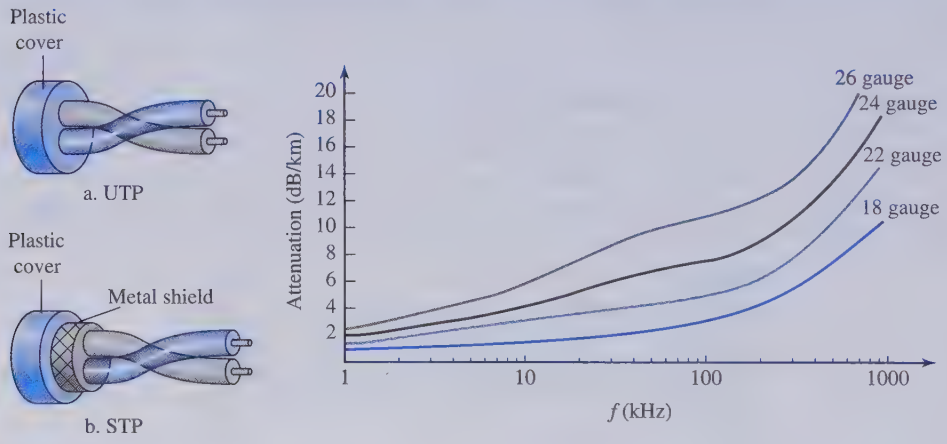
2.6.1 Guided Media

Guided media, which are those that provide a conduit from one device to another, include **twisted-pair cable**, **coaxial cable**, and **fiber-optic cable**. A signal traveling along any of these media is directed and contained by the physical limits of the medium. Twisted-pair and coaxial cable use metallic (copper) conductors that accept and transport signals in the form of electric current. Fiber-optic cable accepts and transports signals in the form of light.

Twisted-Pair Cable

A twisted pair consists of two conductors (normally copper), each with its own plastic insulation, twisted together, as shown in Figure 2.21.

Figure 2.21 *Twisted-pair cable*



One of the wires is used to carry signals to the receiver, and the other is used only as a ground reference. The receiver uses the difference between the two.

In addition to the signal from the sender, interference (noise) and crosstalk may affect both wires and create unwanted signals. If the two wires are parallel, the effect of these unwanted signals is not the same in both wires because they are at different locations relative to the noise or crosstalk sources. This results in a difference at the receiver. By twisting the pairs, a balance is maintained. The most common twisted-pair cable used in communications is referred to as **unshielded twisted pair (UTP)**. There is also a version of twisted-pair cable called **shielded twisted pair (STP)**. STP cable has a metal foil or braided-mesh covering that encases each pair of insulated conductors. Although metal casing improves the quality of cable by preventing the penetration of noise or crosstalk, it is bulkier and more expensive.

Performance

One way to measure the performance of twisted-pair cable is to compare attenuation versus frequency and distance. A twisted-pair cable can pass a wide range of frequencies. However, Figure 2.21 also shows that with increasing frequency, the attenuation, measured in decibels per kilometer (dB/km), sharply increases with frequencies above 100 kHz. Note that gauge is a measure of the thickness of the wire (inversely).

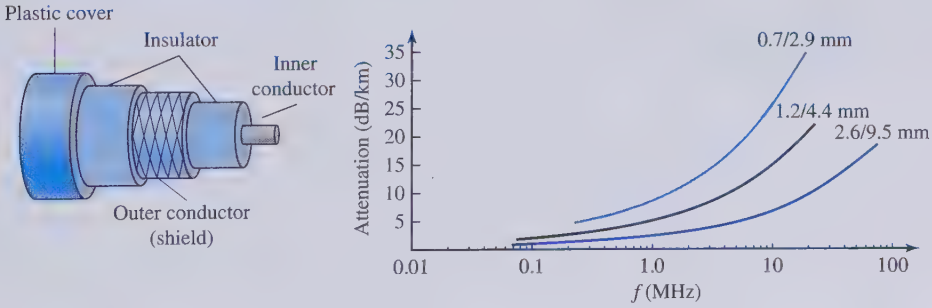
Applications

Twisted-pair cables are used in telephone lines to provide voice and data channels. The local loop—the line that connects subscribers to the central telephone office—commonly consists of unshielded twisted-pair cables. The digital subscriber lines (DSLs) that are used by the telephone companies to provide high-data-rate connections also use the high-bandwidth capability of unshielded twisted-pair cables. We will see some application of twisted-pair cable when we discuss LANs.

Coaxial Cable

Coaxial cable (or *coax*) carries signals of higher frequency ranges than those in twisted-pair cable, in part because the two media are constructed quite differently. Instead of having two wires, coax has a central core conductor of solid or stranded wire (usually copper) enclosed in an insulating sheath, which is, in turn, encased in an outer conductor of metal foil, braid, or a combination of the two. The outer metallic wrapping serves both as a shield against noise and as the second conductor, which completes the circuit. This outer conductor is also enclosed in an insulating sheath, and the whole cable is protected by a plastic cover (see Figure 2.22).

Figure 2.22 *Coaxial cable*



Performance

As we did with twisted-pair cables, we can measure the performance of a coaxial cable. We notice in Figure 2.22 that the attenuation is much higher in coaxial cable than in twisted-pair cable. In other words, although coaxial cable has a much higher bandwidth, the signal weakens rapidly and requires the frequent use of repeaters.

Applications

Coaxial cable was widely used in analog telephone networks where a single coaxial network could carry 10,000 voice signals. Later it was used in digital telephone networks where a single coaxial cable could carry digital data up to 600 Mbps. However, coaxial cable in telephone networks has largely been replaced today with fiber-optic cable.

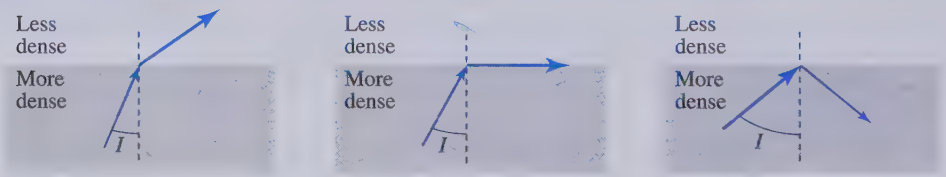
Cable TV networks also use coaxial cable. In the traditional cable TV network, the entire network used coaxial cable. Later, however, cable TV providers replaced most of the media with fiber-optic cable; hybrid networks use coaxial cable only at the network boundaries, near the consumer premises.

Fiber-Optic Cable

A fiber-optic cable is made of glass or plastic and transmits signals in the form of light. To understand optical fiber, we first need to explore several aspects of the nature of light.

Light travels in a straight line as long as it is moving through a single uniform substance. If a ray of light traveling through one substance suddenly enters another substance (of a different density), the ray changes direction. Figure 2.23 shows how a ray of light changes direction when going from a more dense to a less dense substance.

Figure 2.23 *Bending of light ray*



$I <$ critical angle, refraction

$I =$ critical angle, refraction

$I >$ critical angle, refraction

As the figure shows, if the **angle of incidence** I (the angle the ray makes with the line perpendicular to the interface between the two substances) is less than the **critical angle**, the ray **refracts** and moves closer to the surface. If the angle of incidence is equal to the critical angle, the light bends along the interface. If the angle is greater than the critical angle, the ray **reflects** (makes a turn) and travels again in the denser substance. Note that the critical angle is a property of the substance, and its value differs from one substance to another.

Optical fibers use reflection to guide light through a channel. A glass or plastic **core** is surrounded by a **cladding** of less dense glass or plastic. The difference in density of the two materials must be such that a beam of light moving through the core is reflected off the cladding instead of being refracted into it. See Figure 2.24.

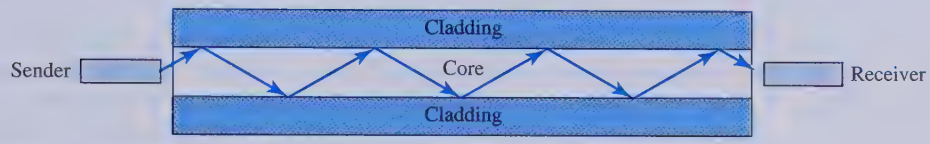
Performance

The attenuation in an optical fiber is much less than in a twisted-pair cable.

Applications

Fiber-optic cable is often found in backbone networks because its wide bandwidth is cost effective. Some cable TV companies use a combination of optical fiber and coaxial cable, thus creating a hybrid network. Optical fiber provides the backbone structure, while coaxial cable provides the connection to the user premises. This is a cost-effective configuration because the narrow bandwidth requirement at the user end does not justify the use of optical fiber. We will see that some local area networks (LANs) use fiber-optic cable.

Figure 2.24 *Optical fiber*

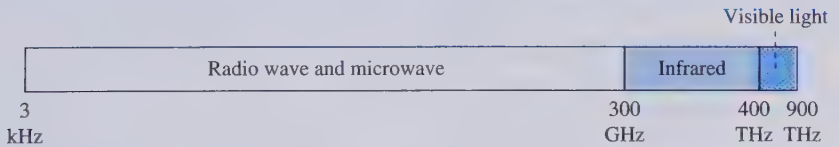


2.6.2 Unguided Media: Wireless

Unguided media transport electromagnetic waves without using a physical conductor. This type of communication is often referred to as *wireless communication.* Signals are normally broadcast through free space and thus are available to anyone who has a device capable of receiving them.

Figure 2.25 shows the part of the **electromagnetic spectrum**, ranging from 3 kHz to 900 THz, used for wireless communication.

Figure 2.25 *Electromagnetic spectrum for wireless communication*



Radio Waves

Although there is no clear-cut demarcation between radio waves and microwaves, electromagnetic waves ranging in frequencies between 3 kHz and 1 GHz are normally called **radio waves**; waves ranging in frequencies between 1 and 300 GHz are called **microwaves**. However, the behavior of the waves, rather than the frequencies, is a better criterion for classification. Radio waves, for the most part, are omnidirectional. When an antenna transmits radio waves, they are propagated in all directions. This means that the sending and receiving antennas do not have to be aligned. A sending antenna sends waves that can be received by any receiving antenna. The omnidirectional property has a disadvantage, too. The radio waves transmitted by one antenna are susceptible to interference by another antenna that may send signals using the same frequency or band.

Radio waves, particularly those waves that propagate in the sky mode, can travel long distances. This makes radio waves a good candidate for long-distance broadcasting such as AM radio.

Radio waves, particularly those of low and medium frequencies, can penetrate walls. This characteristic can be both an advantage and a disadvantage. It is an advantage because, for example, an AM radio can receive signals inside a building. It is a disadvantage because we cannot isolate a communication to just inside or outside a building. The radio wave band is relatively narrow, just under 1 GHz, compared to the microwave band. When this band is divided into subbands, the subbands are also narrow, leading to a low data rate for digital communications.

Microwaves

Electromagnetic waves having frequencies between 1 and 300 GHz are called microwaves. Microwaves are unidirectional. When an antenna transmits microwaves, they can be narrowly focused. This means that the sending and receiving antennas need to be aligned. The unidirectional property has an obvious advantage. A pair of antennas can be aligned without interfering with another pair of aligned antennas. The following describes some characteristics of microwave propagation:

- Microwave propagation is in line of sight. Because the towers with the mounted antennas need to be in direct sight of each other, towers that are far apart need to be very tall. The curvature of the Earth as well as other blocking obstacles do not allow two short towers to communicate by using microwaves. Repeaters are often needed for long-distance communication.
- Very-high-frequency microwaves cannot penetrate walls. This characteristic can be a disadvantage if receivers are inside buildings.
- The microwave band is relatively wide, almost 299 GHz. Therefore, wider subbands can be assigned and a high data rate is possible.
- Use of certain portions of the band requires permission from authorities.

Applications

Microwaves, due to their unidirectional properties, are very useful when unicast (one-to-one) communication is needed between the sender and the receiver. They are used in cellular phones, satellite networks, and wireless LANs.

Infrared

Infrared waves, with frequencies from 300 GHz to 400 THz (wavelengths from 1 mm to 770 nm), can be used for short-range communication. Infrared waves, having high frequencies, cannot penetrate walls. This advantageous characteristic prevents interference between one system and another; a short-range communication system in one room cannot be affected by another system in the next room. When we use our infrared remote control, we do not interfere with the use of the remote by our neighbors. However, this same characteristic makes infrared signals useless for long-range communication. In addition, we cannot use infrared waves outside a building because the sun's rays contain infrared waves that can interfere with the communication.

2.7 END-OF-CHAPTER MATERIALS

2.7.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items in brackets refer to the reference list at the end of the text. Several books address the materials discussed in this chapter: [Pea 92], [Cou 01], [Ber 96], [Hsu 03], [Spi 74], [Sta 04], [Tan 03], [GW 04], [SSS 05], [BEL 01], and [Max 99].

2.7.2 Key Terms

amplification
amplitude modulation (AM)
amplitude shift keying (ASK)
analog-to-analog conversion
analog-to-digital conversion
angle of incidence
attenuation
bandwidth
baseband transmission
bit length
bit rate
block coding
cladding
coaxial cable
composite signal
critical angle
crosstalk
decibel (dB)
delta modulation (DM)
digital-to-analog conversion
digital-to-digital conversion
digitization
distortion
electromagnetic spectrum
fiber-optic cable
frequency
frequency-division multiplexing (FDM)
frequency-domain plot
frequency modulation (FM)
frequency shift keying (FSK)
guard bands
guided media
impulse noise
induced noise
infrared waves
jitters
line coding
microwaves
multiplexing
noise
Nyquest bit rate
peak amplitude
period
phase
phase modulation (PM)
phase shift keying (PSK)
pulse code modulation (PCM)
quadrature amplitude modulation (QAM)
radio waves
reflects
refracts
Shannon capacity
shielded twisted pair (STP)
signal-to-noise ratio (SNR)
thermal noise
time-division multiplexing (TDM)
time-domain plot
transmission medium
twisted-pair cable
unguided media
unshielded twisted-pair (UTP)
wavelength

2.7.3 Summary

Data must be transformed to electromagnetic signals to be transmitted. Analog signals can have an infinite number of values in a range; digital signals can have only a limited number of values.

Signals lose some of their energy when traveling through a transmission medium because of the imperfection of the medium. This affects the data rate and the shape of the signal. The Nyquest bit rate and Shannon capacity deal with these imperfections.

To transmit data using digital transmission requires either digital-to-digital conversion (changing digital data to a digital signal) or analog-to-digital conversion (changing analog data to an analog signal).

To transmit data using analog transmission requires either digital-to-analog conversion (changing digital data to an analog signal) or analog-to-analog conversion (changing analog data to an analog signal).

Bandwidth utilization is the use of available bandwidth to achieve specific goals. Efficiency can be achieved by using multiplexing. We discuss two types of multiplexing: frequency-division multiplexing and time-division multiplexing.

Transmission media lies below the physical layer. A guided medium provides a physical conduit from one device to another. Twisted-pair cable, coaxial cable, and fiber-optic cable are the most popular types of guided media. Unguided media (free space) transport electromagnetic waves without the use of a physical conductor.

2.8 PRACTICE SET

2.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

2.8.2 Questions

Q2-1. What is the relationship between period and frequency?

Q2-2. What does the amplitude of a signal measure? What does the frequency of a signal measure? What does the phase of a signal measure?

Q2-3. How can a composite signal be decomposed into its individual frequencies?

Q2-4. Name three types of transmission impairment.

Q2-5. Distinguish between baseband transmission and broadband transmission.

Q2-6. Distinguish between a low-pass channel and a bandpass channel.

Q2-7. What does the Nyquist theorem have to do with communications?

Q2-8. What does the Shannon capacity have to do with communications?

Q2-9. Why do optical signals used in fiber-optic cables have a very short wave length?

Q2-10. Can we say if a signal is periodic or nonperiodic by just looking at its frequency-domain plot? How?

Q2-11. Is the frequency-domain plot of a voice signal discrete or continuous?

Q2-12. Is the frequency-domain plot of an alarm system discrete or continuous?

Q2-13. We send a voice signal from a microphone to a recorder. Is this baseband or broadband transmission?

Q2-14. How can we find the period of a sine wave when its frequency is given?

Q2-15. Which of the following measures the value of a signal at any time?

a. amplitude **b.** frequency **c.** phase

Q2-16. Can we say whether a signal is periodic or nonperiodic by just looking at its time-domain plot? How?

Q2-17. Which of the following are causes of transmission impairment?

a. attenuation **b.** modulation **c.** noise

Q2-18. Which of the following is the characteristic of the low-pass channel?

a. A channel with a bandwidth that starts from zero.

b. A channel with a bandwidth that does not start from zero.

Q2-19. We send a digital signal from one station on a LAN to another station. Is this baseband or broadband transmission?

Q2-20. Which of the following is the definition of a baseband transmission?

a. Sending a digital or an analog signal without modulation using a low-pass channel.

b. Modulating a digital or an analog signal using a bandpass channel.

Q2-21. How can a periodic composite signal be decomposed into its individual frequencies?

Q2-22. Which of the following defines the theoretical maximum bit rate of a noiseless channel?

a. Nyquist theorem **b.** Shannon capacity

Q2-23. Which of the following techniques are examples of digital-to-digital conversion?

a. line coding

b. block coding

c. amplitude modulation

Q2-24. Which characteristics of an analog signal are changed to represent the digital signal in each of the following digital-to-analog conversion mechanisms?

a. ASK **b.** PSK

Q2-25. Which of the two digital-to-analog conversion techniques is more susceptible to noise?

a. ASK **b.** FSK

Q2-26. Which characteristics of an analog signal are changed to represent the low-pass analog signal in each of the following analog-to-analog conversions?

a. FM **b.** PM

Q2-27. Which of the three multiplexing techniques are used to combine analog signals?

Q2-28. Define synchronous TDM, and compare it with statistical TDM.

Q2-29. What is the position of the transmission media in the TCP/IP protocol suite?

Q2-30. Name the two major categories of transmission media.

Q2-31. What are the three major classes of guided media?

Q2-32. What is the purpose of cladding in an optical fiber?

Q2-33. Describe how omnidirectional waves are propagated.

Q2-34. List three techniques of digital-to-digital conversion.

Q2-35. Distinguish between a signal element and a data element.

Q2-36. Which characteristics of an analog signal are changed to represent the digital signal in each of the following digital-to-analog conversions?

a. ASK **b.** FSK **c.** PSK **d.** QAM

Q2-37. Which of the four digital-to-analog conversion techniques (ASK, FSK, PSK, or QAM) is the most susceptible to noise?

Q2-38. Define analog-to-analog conversion.

Q2-39. Which characteristics of an analog signal are changed to represent the lowpass analog signal in each of the following analog-to-analog conversions?

a. AM **b.** FM **c.** PM

Q2-40. Which of the three analog-to-analog conversion techniques (AM, FM, or PM) is the most susceptible to noise? Defend your answer.

Q2-41. Describe the goals of multiplexing.

Q2-42. List the three main multiplexing techniques mentioned in this chapter.

Q2-43. Which of the three multiplexing techniques is (are) used to combine analog signals? Which of the three multiplexing techniques is (are) used to combine digital signals?

Q2-44. Name the two major categories of transmission media.

Q2-45. How do guided media differ from unguided media?

Q2-46. What are the three major classes of guided media?

Q2-47. What is the significance of the twisting in twisted-pair cable?

Q2-48. What is refraction? What is reflection?

Q2-49. Name the advantages of fiber-optic cable over twisted-pair and coaxial cable.

Q2-50. What is the difference between omnidirectional waves and unidirectional waves?

2.8.3 Problems

P2-1. Calculate the corresponding periods of the following frequencies.

a. 24 Hz **b.** 8 MHz **c.** 140 kHz

P2-2. Calculate the corresponding frequencies of the following periods.

a. 5 s **b.** 12 μs **c.** 220 ns

P2-3. Give the phase shift for each of the following.

a. A sine wave with maximum amplitude at time zero

b. A sine wave with maximum amplitude after 1/4 cycle

c. A sine wave with zero amplitude after 3/4 cycle and increasing

P2-4. What is the bandwidth of a signal that can be decomposed into five sine waves with frequencies of 0, 20, 50, 100, and 200 Hz? All peak amplitudes are the same. Draw the bandwidth.

P2-5. A periodic composite signal with a bandwidth of 2000 Hz is composed of two sine waves. The first one has a frequency of 100 Hz with a maximum amplitude of 20 V; the second one has a maximum amplitude of 5 V. Draw the bandwidth.

P2-6. Which signal has a wider bandwidth, a sine wave with a frequency of 100 Hz or a sine wave with a frequency of 200 Hz?

P2-7. Give the bit rate for each of the following signals.

a. A signal in which 1 bit lasts 0.001 s

b. A signal in which 1 bit lasts 2 ms

c. A signal in which 10 bits last 20 μs

P2-8. A device is sending out data at the rate of 1000 bps.
- **a.** How long does it take to send out 10 bits?
- **b.** How long does it take to send out a single character (8 bits)?
- **c.** How long does it take to send a file of 100,000 characters?

P2-9. A periodic composite signal contains frequencies from 10 to 30 kHz, each with an amplitude of 10 V. Draw the frequency spectrum.

P2-10. A nonperiodic composite signal contains frequencies from 10 to 30 kHz. The peak amplitude is 10 V for the lowest and the highest signals and is 30 V for the 20 kHz signal. Assuming that the amplitudes change gradually from the minimum to the maximum, draw the frequency spectrum.

P2-11. A TV channel has a bandwidth of 6 MHz. If we send a digital signal using one channel, what are the data rates if we use one harmonic, three harmonics, and five harmonics?

P2-12. A signal travels from point A to point B. At point A, the signal power is 100 W. At point B, the power is 90 W. What is the attenuation in decibels?

P2-13. The attenuation of a signal is –10 dB. What is the final signal power if it was originally 5 W?

P2-14. A signal has passed through three cascaded amplifiers, each with a 4-dB gain. What is the total gain? How much is the signal amplified?

P2-15. If the bandwidth of the channel is 5 kbps, how long does it take to send a frame of 100,000 bits out of this device?

P2-16. The light of the sun takes approximately 8 min to reach the Earth. What is the distance between the sun and the Earth?

P2-17. A signal has a wavelength of 1 μm in air. How far can the front of the wave travel during 1000 periods?

P2-18. A line has a signal-to-noise ratio of 1000 and a bandwidth of 4000 kHz. What is the maximum data rate supported by this line?

P2-19. A file contains 2 million bytes. How long does it take to download this file using a 56-kbps channel? a 1-Mbps channel?

P2-20. A computer monitor has a resolution of 1200 by 1000 pixels. If each pixel uses 1024 colors, how many bits are needed to send the complete contents of a screen?

P2-21. A signal with 200-mW power passes through 10 devices, each with an average noise of 2 μW. What is the SNR? What is the SNR_{dB}?

P2-22. If the peak voltage value of a signal is 20 times the peak voltage value of the noise, what is the SNR? What is the SNR_{dB}?

P2-23. What is the theoretical capacity of a channel in each of the following cases:
- **a.** Bandwidth: 20 kHz $SNR_{dB} = 40$
- **b.** Bandwidth: 200 kHz $SNR_{dB} = 4$
- **c.** Bandwidth: 1 MHz $SNR_{dB} = 20$

P2-24. We need to upgrade a channel to a higher bandwidth.
- **a.** How is the rate improved if we double the bandwidth?
- **b.** How is the rate improved if we double the SNR?

P2-25. We have a channel with 4-kHz bandwidth. If we want to send data at 100 kbps, what is the minimum SNR_{dB}? What is the SNR?

P2-26. What is the transmission time of a packet sent by a station if the length of the packet is 1 million bytes and the bandwidth of the channel is 200 kbps?

P2-27. What is the length of a bit in a channel with a propagation speed of 2×10^8 m/s for the following channel bandwidths?

a. 1 Mbps **b.** 10 Mbps **c.** 100 Mbps

P2-28. How many bits can fit on a link with a 2-ms delay for the following link bandwidths?

a. 1 Mbps **b.** 10 Mbps **c.** 100 Mbps

P2-29. The input stream to a 4B/5B block encoder is

0100 0000 0000 0000 0000 0001

a. What is the output stream?

b. What is the length of the longest consecutive sequence of 0s in the input?

c. What is the length of the longest consecutive sequence of 0s in the output?

P2-30. What is the Nyquist sampling rate for each of the following signals?

a. A low-pass signal with a bandwidth of 200 kHz

b. A bandpass signal with a bandwidth of 200 kHz if the lowest frequency is

c. 100 kHz

P2-31. We have sampled a low-pass signal with a bandwidth of 200 kHz using 1024 levels of quantization.

a. Calculate the bit rate of the digitized signal.

b. Calculate the SNR_{dB} for this signal.

c. Calculate the PCM bandwidth of this signal.

P2-32. What is the maximum data rate of a channel with a bandwidth of 200 kHz if we use four levels of digital signaling?

P2-33. An analog signal has a bandwidth of 20 kHz. If we sample this signal and send it through a 30 kbps channel, what is the SNR_{dB}?

P2-34. We want to transmit 1000 characters with each character encoded as 8 bits. Calculate the baud rate for the given bit rate and type of modulation.

a. 2000 bps, FSK **c.** 6000 bps, QPSK

b. 4000 bps, ASK **d.** 36,000 bps, 64-QAM

P2-35. Calculate the bit rate for the given baud rate and type of modulation.

a. 1000 baud, FSK **c.** 1000 baud, BPSK

b. 1000 baud, ASK **d.** 1000 baud, 16-QAM

P2-36. Give the number of bits per baud for the following techniques.

a. ASK with four different amplitudes

b. FSK with eight different frequencies

c. PSK with four different phases

d. QAM with a constellation of 128 points.

P2-37. How many bits per baud can we send in each of the following cases if the signal constellation has one of the following number of points?

a. 2 **b.** 4 **c.** 16 **d.** 1024

P2-38. What is the required bandwidth for the following cases if we need to send 4000 bps? Let $d = 1$.

a. ASK **c.** QPSK

b. FSK with $2\Delta f = 4$ kHz **d.** 16-QAM

P2-39. The telephone line has a 4-kHz bandwidth. What is the maximum number of bits we can send using each of the following techniques? Let $d = 0$.

a. ASK **c.** 16-QAM

b. QPSK **d.** 64-QAM

P2-40. A cable company uses one of the cable TV channels (with a bandwidth of 6 MHz) to provide digital communication for each resident. What is the available data rate for each resident if the company uses a 64-QAM technique?

P2-41. Assume that a voice channel occupies a bandwidth of 4 kHz. We need to multiplex 10 voice channels with guard bands of 500 Hz using FDM. Calculate the required bandwidth.

P2-42. Ten sources, six with a bit rate of 200 kbps and four with a bit rate of 400 kbps, are to be combined using multilevel TDM with no synchronizing bits. Answer the following questions about the final stage of the multiplexing.

a. What is the size of a frame in bits?

b. What is the frame rate?

c. What is the duration of a frame?

d. What is the data rate?

P2-43. Find the bandwidth for the following situations if we need to modulate a 5-kHz voice.

a. AM **b.** FM ($\beta = 5$) **c.** PM ($\beta = 1$)

P2-44. Find the total number of channels in the corresponding band allocated by the Federal Communications Commission (FCC).

a. AM **b.** FM

P2-45. A light signal is traveling through a fiber. What is the delay in the signal if the length of the fiber-optic cable is 10 m, 100 m, and 1 km (assume a propagation speed of 2×10^8 m)?

P2-46. A beam of light moves from one medium to another medium with less density. The critical angle is 60°. Do we have refraction or reflection for each of the following incident angles? Show the bending of the light ray in each case.

a. 40° **b.** 60° **c.** 80°

P2-47. Which signal has a wider bandwidth, a sine wave with a frequency of 100 Hz or a sine wave with a frequency of 200 Hz?

P2-48. Calculate the baud for the given bit rate and type of modulation.

a. 2000 bps, FSK

b. 4000 bps, ASK

c. 36,000 bps, 64-QAM

P2-49. Calculate the bit rate for the given baud and type of modulation.

a. 1000 baud, FSK

b. 1000 baud, ASK

c. 1000 baud, 16-QAM

P2-50. What is the number of bits per baud for the following techniques?

a. FSK with eight frequencies

b. QAM with a 128-point constellation

P2-51. How many bits per baud can we send in each of the following cases if the signal constellation has the given number of points?

a. 2 **b.** 4 **c.** 16 **d.** 1024

P2-52. What is the required bandwidth for the following cases if we need to send 4000 bps? Let $d = 1$.

a. ASK

b. FSK ($2\,\Delta f = 4$ kHz)

c. 16-QAM

P2-53. As an example of the Nyquist theorem, let us sample a simple sine wave at three sampling rates: $f_s = 4f$ (2 times the Nyquist rate), $f_s = 2f$ (Nyquist rate), and $f_s = f$ (one-half the Nyquist rate). Show how we can recover the wave.

P2-54. Assume that a voice channel occupies a bandwidth of 4 kHz. We need to multiplex 10 voice channels with guard bands of 500 Hz using FDM. Calculate the required bandwidth.

CHAPTER 3

Data-Link Layer

In Chapter 2, we discussed the first layer of the TCP/IP protocol suite, the physical layer. In this chapter, we discuss the second layer, the data-link layer. These two layers are the foundation on which local area networks (LANs) and wide area networks (WANs) are built.

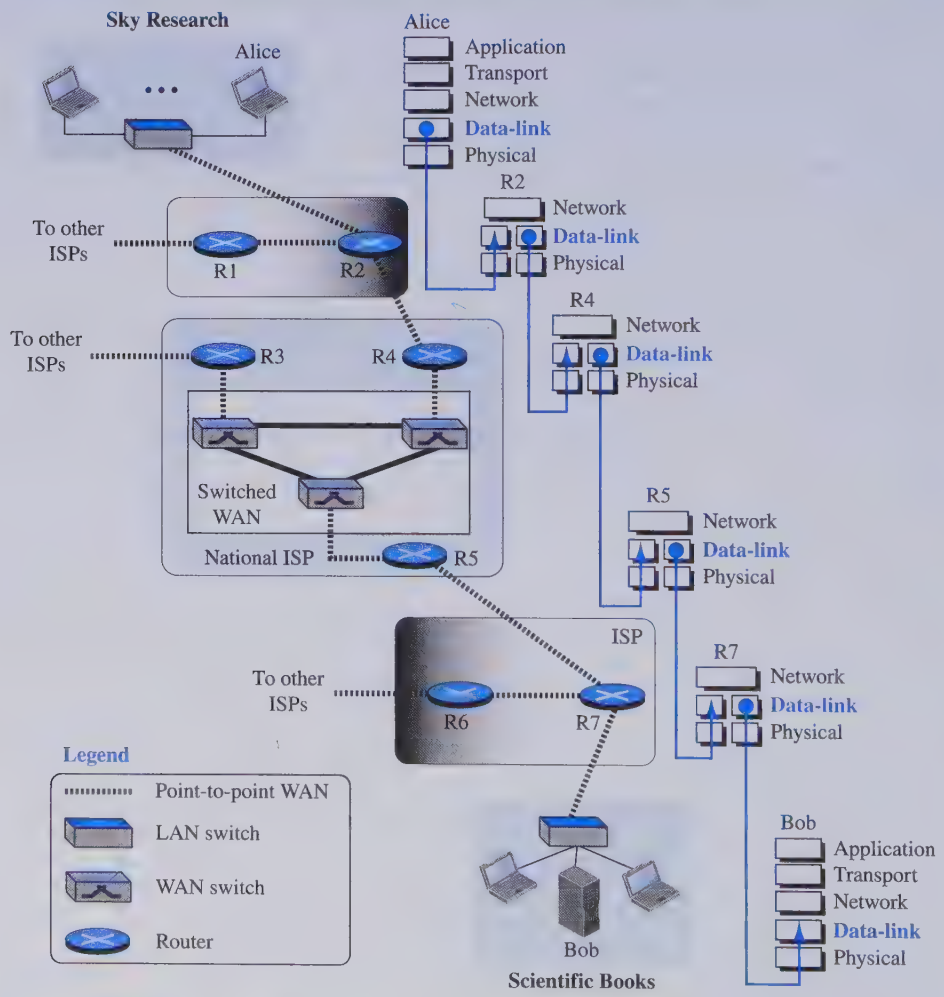
This chapter is divided into four sections.

- In the first section, we define the location of the data-link layer with respect to other layers and introduce the terms *node* and *link*. We also give the name of the sublayers used in the data-link layer: data-link control (DLC) and media access control (MAC).
- In the second section, we discuss the sublayer DLC and discuss framing and error control.
- In the third section, we discuss the MAC sublayer and define the type of protocols used in this sublayer including random-access protocols, controlled-access protocols, and channelization.
- In the fourth section, we introduce link-layer addressing and mention how a link-layer address can be found using the ARP protocol.

3.1 INTRODUCTION

The Internet is a combination of networks glued together by combining devices (routers and switches). If a packet is to travel from a host to another host, it needs to pass through these networks. Figure 3.1 shows communication between Alice and Bob, using the same scenario we followed in Chapter 2, but we are now interested in communication at the data-link layer. Communication at the data-link layer is made up of five separate logical connections between the data-link layers in the path.

Figure 3.1 *Communication at the data-link layer*

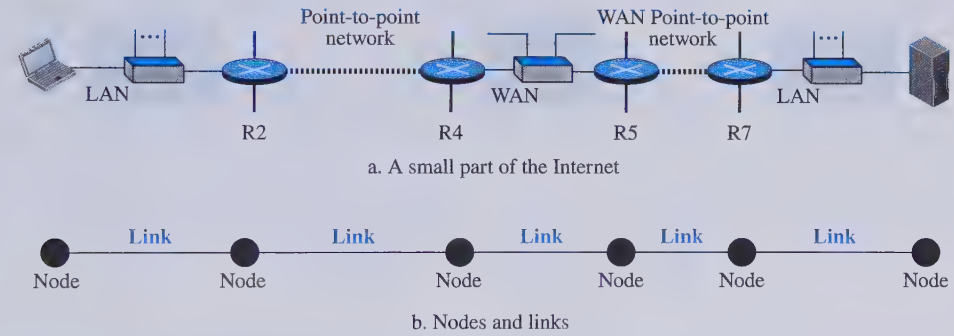


The data-link layer at Alice's computer communicates with the data-link layer at router R2. The data-link layer at router R2 communicates with the data-link layer at router R4, and so on. Finally, the data-link layer at router R7 communicates with the data-link layer at Bob's computer. Communication between Alice's computer and Bob's computer involves one data-link layer; communication at routers involve two data-link layers.

3.1.1 Nodes and Links

Although communication at the application, transport, and network layers is end-to-end, communication at the data-link layer is node-to-node. As we have learned in the previous chapters, a data unit from one point in the Internet needs to pass through many networks (LANs and WANs) to reach another point. Theses LANs and WANs are connected by routers. It is customary to refer to the two end hosts and the routers as **nodes** and the networks in between as **links.** The following is a simple representation of links and nodes when the path of the data unit is only six nodes (Figure 3.2).

Figure 3.2 *Nodes and Links*



The first node is the source host; the last node is the destination host. The other four nodes are routers. The first, third, and fifth links represent the three LANs; the second and fourth links represent the two WANs.

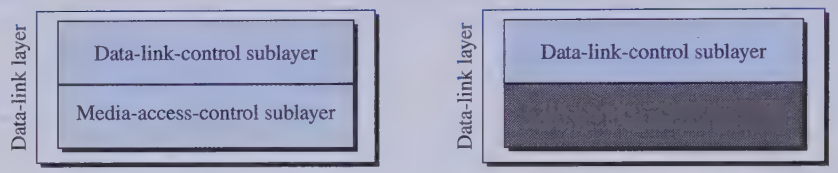
3.1.2 Two Types of Links

Although two nodes are physically connected by a transmission medium such as cable or air, we need to remember that the data-link layer controls how the medium is used. We can have a data-link layer that uses the whole capacity of the medium; we can also have a data-link layer that uses only part of the capacity of the link. In other words, we can have a *point-to-point link* or a *broadcast link*. In a point-to-point link, the link is dedicated to the two devices; in a broadcast link, the link is shared between several pairs of devices. For example, when two friends use the traditional home phones to chat, they are using a point-to-point link; when the same two friends use their cellular phones, they are using a broadcast link (the air is shared among many cell phone users).

3.1.3 Two Sublayers

To better understand the functionality of and the services provided by the link layer, we can divide the data-link layer into two sublayers: *data-link control (DLC)* and *media access control (MAC)*. This is not unusual because, as we will see in Chapter 4, LAN protocols actually use the same strategy. The data-link-control sublayer deals with all issues common to both point-to-point and broadcast links; the MAC sublayer deals only with issues specific to broadcast links. In other words, we separate theses two types of links at the data-link layer as shown in Figure 3.3.

Figure 3.3 *Dividing the data-link layer into two sublayers*



a. Data-link layer of a broadcast link

b. Data-link layer of a point-to-point link

In this chapter, we first discuss the data-link-control sublayer that is common to both types of links. We then discuss the media-access-control sublayer that is used only in the broadcast link.

3.2 DATA-LINK CONTROL

Data-link control (DLC) deals with procedures for communication between two adjacent nodes—node-to-node communication—no matter whether the link is dedicated or broadcast. Its functions include *framing* and *error control.* In this section, we first discuss framing, or how to organize the bits that are carried by the physical layer. We then discuss flow and error control. Techniques for error detection are discussed at the end of this section.

3.2.1 Framing

Data transmission in the physical layer means moving bits in the form of a signal from the source to the destination. The physical layer provides bit synchronization to ensure that the sender and receiver use the same bit durations and timing. We discussed the physical layer in Chapter 2.

The data-link layer, on the other hand, needs to pack bits into frames, so that each frame is distinguishable from another. Our postal system practices a type of framing. The simple act of inserting a letter into an envelope separates one piece of information from another; the envelope serves as the delimiter. In addition, each envelope defines the sender and receiver addresses, which is necessary since the postal system is a many-to-many carrier facility.

Framing in the data-link layer separates a message from one source to a destination by adding a sender address and a destination address. The destination address defines where the packet is to go; the sender address helps the recipient acknowledge the receipt.

Although the whole message could be packed in one frame, that is not normally done. One reason is that a frame can be very large, making flow and error control very inefficient. When a message is carried in one very large frame, even a single-bit error would require the retransmission of the whole frame. When a message is divided into smaller frames, a single-bit error affects only that small frame.

Frame Size

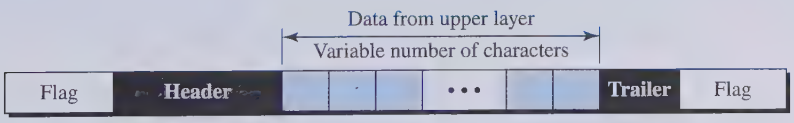
Frames can be of fixed or variable size. In *fixed-size framing*, there is no need to define the boundaries of the frames; the size itself can be used as a delimiter. An example of this type of framing is the ATM WAN, which uses frames of fixed size called *cells*.

Our main discussion in this chapter concerns *variable-size framing*, prevalent in local-area networks. In variable-size framing, we need a way to define the end of one frame and the beginning of the next. Historically, two approaches have been used for this purpose: a character-oriented approach and a bit-oriented approach.

Character-Oriented Framing

In *character-oriented (or byte-oriented) framing*, data to be carried are 8-bit characters from a coding system such as ASCII (see Appendix A). The header, which normally carries the source and destination addresses and other control information, and the trailer, which carries error detection redundant bits, are also multiples of 8 bits. To separate one frame from the next, an 8-bit (1-byte) **flag** is added at the beginning and the end of a frame. The flag, composed of protocol-dependent special characters, signals the start or end of a frame. Figure 3.4 shows the format of a frame in a character-oriented protocol.

Figure 3.4 *A frame in a character-oriented protocol*

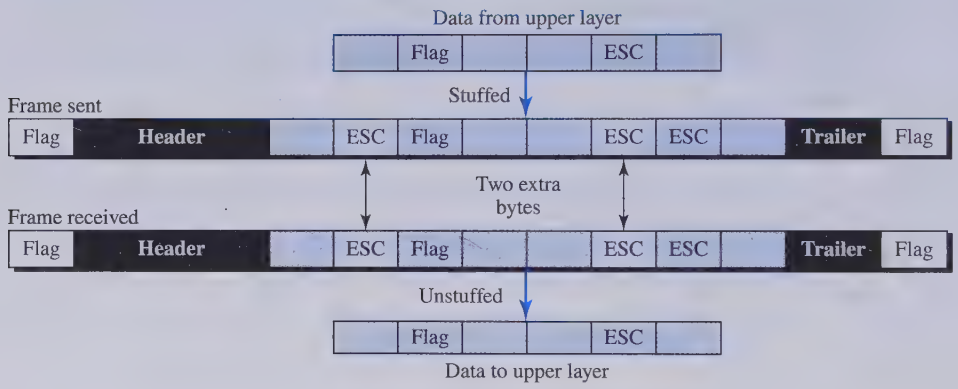


Character-oriented framing was popular when only text was exchanged by the data-link layers. The flag could be selected to be any character not used for text communication. Now, however, we send other types of information such as graphs, audio, and video; any pattern used for the flag could also be part of the information. If this happens, the receiver, when it encounters this pattern in the middle of the data, thinks it has reached the end of the frame. To fix this problem, a **byte-stuffing** strategy was added to character-oriented framing. In byte stuffing (or character stuffing), a special byte is added to the data section of the frame when there is a character with the same pattern as the flag. The data section is stuffed with an extra byte. This byte is usually called the *escape*

character (ESC) and has a predefined bit pattern. Whenever the receiver encounters the ESC character, it removes it from the data section and treats the next character as data, not as a delimiting flag.

Byte stuffing by the escape character allows the presence of the flag in the data section of the frame, but it creates another problem. What happens if the text contains one or more escape characters followed by a byte with the same pattern as the flag? The receiver removes the escape character, but keeps the next byte, which is incorrectly interpreted as the end of the frame. To solve this problem, the escape characters that are part of the text must also be marked by another escape character. In other words, if the escape character is part of the text, an extra one is added to show that the second one is part of the text. Figure 3.5 shows the situation.

Figure 3.5 *Byte stuffing and unstuffing*



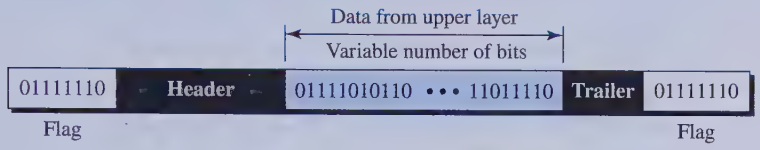
Byte stuffing is the process of adding one extra byte whenever there is a flag or escape character in the text.

Character-oriented protocols present another problem in data communications. The universal coding systems in use today, such as Unicode, have 16-bit and 32-bit characters that conflict with 8-bit characters. We can say that, in general, the tendency is moving toward the bit-oriented protocols that we will discuss next.

Bit-Oriented Framing

In *bit-oriented framing*, the data section of a frame is a sequence of bits to be interpreted by the upper layer as text, graphic, audio, video, and so on. However, in addition to headers (and possible trailers), we still need a delimiter to separate one frame from the other. Most protocols use a special 8-bit pattern flag, 01111110, as the delimiter to define the beginning and end of the frame, as shown in Figure 3.6.

Figure 3.6 *A frame in a bit-oriented protocol*

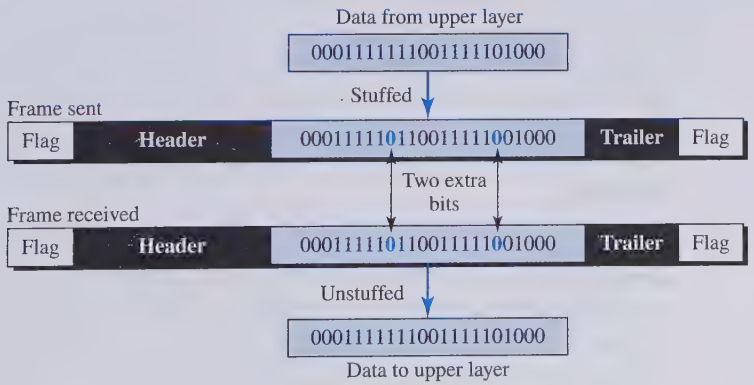


This flag can create the same type of problem we saw in the character-oriented protocols. That is, if the flag pattern appears in the data, we need to somehow inform the receiver that this is not the end of the frame. We do this by stuffing one single bit (instead of one byte) to prevent the pattern from looking like a flag. The strategy is called **bit stuffing**. In bit stuffing, if a 0 and five consecutive 1 bits are encountered, an extra 0 is added. This extra stuffed bit is eventually removed from the data by the receiver. Note that the extra bit is added after one 0 followed by five 1s regardless of the value of the next bit. This guarantees that the flag field sequence does not inadvertently appear in the frame.

Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

Figure 3.7 shows bit stuffing at the sender and bit removal at the receiver. Note that even if we have a 0 after five 1s, we still stuff a 0. The 0 will be removed by the receiver.

Figure 3.7 *Bit stuffing and unstuffing*



This means that if the flaglike pattern 01111110 appears in the data, it will change to 011111010 (stuffed) and is not mistaken for a flag by the receiver. The real flag 01111110 is not stuffed by the sender and is recognized by the receiver.

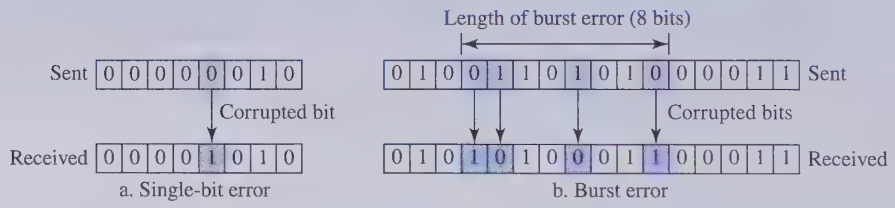
3.2.2 Error Control

Error control is both error detection and error correction. It allows the receiver to inform the sender of any frames lost or damaged in transmission and coordinates the retransmission of those frames by the sender. In the data-link layer, the term ***error control*** refers primarily to methods of error detection and retransmission (error correction is done using retransmission of the corrupted frame).

Types of Errors

Whenever bits flow from one point to another, they are subject to unpredictable changes because of **interference**. This interference can change the shape of the signal. The term **single-bit error** means that only 1 bit of a given data unit (such as a byte, character, or packet) is changed from 1 to 0 or from 0 to 1. The term **burst error** means that two or more bits in the data unit have changed from 1 to 0 or from 0 to 1. Figure 3.8 shows the effect of a single-bit error and burst error, respectively, on a data unit.

Figure 3.8 *Single-bit error and burst error*



A burst error is more likely to occur than a single-bit error because the duration of the noise signal is normally longer than the duration of one bit, which means that when noise affects data, it affects a set of bits. The number of bits affected depends on the data rate and duration of noise. For example, if we are sending data at 1 kbps, a noise of 1/100 s can affect 10 bits; if we are sending data at 1 Mbps, the same noise can affect 10,000 bits.

Redundancy

The central concept in detecting or correcting errors is *redundancy*. To be able to detect or correct errors, we need to send some extra bits with our data. These redundant bits are added by the sender and removed by the receiver. Their presence allows the receiver to detect or correct corrupted bits.

Detection versus Correction

The correction of errors is more difficult than the detection. In *error detection*, we are looking only to see if any error has occurred. The answer is a simple yes or no. We are not even interested in the number of corrupted bits. A single-bit error is the same for us as a burst error. In *error correction*, we need to know the exact number of bits that are corrupted and, more importantly, their location in the message. The number of the errors and the size of the message are important factors. If we need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations; if we need to correct two errors in a data unit of the same size, we need to consider 28 (permutation of 8 by 2) possibilities. You can imagine the receiver's difficulty in finding 10 errors in a data unit of 1000 bits. We concentrate on error detection.

Coding

Redundancy is achieved through various coding schemes. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits. The receiver checks the relationships between the two sets of bits to detect errors. The ratio of redundant bits to the data bits and the robustness of the process are important factors in any coding scheme.

We can divide coding schemes into two broad categories: *block coding* and *convolution coding*. In this book, we concentrate on block coding; convolution coding is more complex and beyond the scope of this book.

Block Coding

In block coding, we divide our message into blocks, each consisting of k bits, called **datawords**. We add r redundant bits to each block to make the length $n = k + r$. The resulting n-bit blocks are called **codewords**. How the extra r bits are chosen or calculated is something we will discuss later. For the moment, it is important to know that we have a set of datawords, each of size k, and a set of codewords, each of size of n. With k bits, we can create a combination of 2^k datawords; with n bits, we can create a combination of 2^n codewords. Since $n > k$, the number of possible codewords is larger than the number of possible datawords. The block-coding process is one-to-one; the same dataword is always encoded as the same codeword. This means that we have $2^n - 2^k$ codewords that are not used. We call these codewords invalid or illegal. The trick in error detection is the existence of these invalid codes, as we discuss next. If the receiver receives an invalid codeword, this indicates that the data were corrupted during transmission.

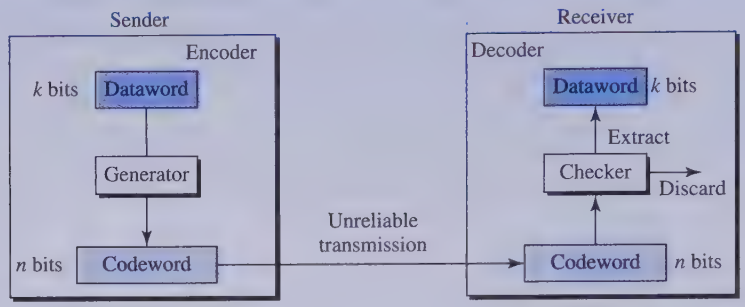
Error Detection

How can errors be detected by using block coding? If the following two conditions are met, the receiver can detect a change in the original codeword.

1. The receiver has (or can find) a list of valid codewords.
2. The original codeword has changed to an invalid one.

Figure 3.9 shows the role of block coding in error detection.

Figure 3.9 *Process of error detection in block coding*



The sender creates codewords out of datawords by using a generator that applies the rules and procedures of encoding (discussed later). Each codeword sent to the receiver may change during transmission. If the received codeword is the same as one of the valid codewords, the word is accepted; the corresponding dataword is extracted for use. If the received codeword is not valid, it is discarded. However, if the codeword is corrupted during transmission but the received word still matches a valid codeword, the error remains undetected.

Example 3.1

Let us assume that $k = 2$ and $n = 3$. Table 3.1 shows the list of datawords and codewords. Later, we will see how to derive a codeword from a dataword.

Table 3.1 *A code for error detection in Example 3.1*

Datawords	*Codewords*	*Datawords*	*Codewords*
00	000	10	101
01	011	11	110

Assume the sender encodes the dataword 01 as 011 and sends it to the receiver. Consider the following cases:

1. The receiver receives 011. It is a valid codeword. The receiver extracts the dataword 01 from it.
2. The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted). This is not a valid codeword and is discarded.
3. The codeword is corrupted during transmission, and 000 is received (the right 2 bits are corrupted). This is a valid codeword. The receiver incorrectly extracts the dataword 00. Two corrupted bits have made the error undetectable.

An error-detecting code can detect only the types of errors for which it is designed; other types of errors may remain undetected.

Hamming Distance

One of the central concepts in coding for error control is the idea of the Hamming distance. The **Hamming distance** between two words (of the same size) is the number of differences between the corresponding bits. We show the Hamming distance between two words x and y as $d(x, y)$. We may wonder why the Hamming distance is important for error detection. The reason is that the Hamming distance between the received codeword and the sent codeword is the number of bits that are corrupted during transmission. For example, if the codeword 00000 is sent and 01101 is received, 3 bits are in error and the Hamming distance between the two is $d(00000, 01101) = 3$. In other words, if the Hamming distance between the sent and the received codeword is not zero, the codeword has been corrupted during transmission.

The Hamming distance can easily be found if we apply the XOR operation ($\oplus$) on the two words and count the number of 1s in the result. Note that the Hamming distance is a value greater than or equal to zero.

The Hamming distance between two words is the number of differences between corresponding bits.

Example 3.2

Let us find the Hamming distance between two pairs of words.

1. The Hamming distance d (000, 011) is 2 because (000 $\oplus$ 011) is 011 (two 1s).
2. The Hamming distance d (10101, 11110) is 3 because (10101 $\oplus$ 11110) is 01011 (three 1s).

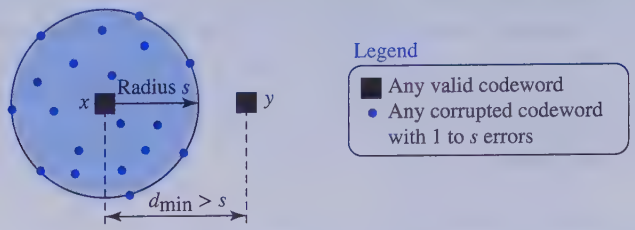
Minimum Hamming Distance for Error Detection

In a set of codewords, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of codewords. Now let us find the minimum Hamming distance in a code if we want to be able to detect up to s errors. If s errors occur during transmission, the Hamming distance between the sent codeword and received codeword is s. If our system is to detect up to s errors, the minimum distance between the valid codes must be $(s + 1)$ so that the received codeword does not match a valid codeword. In other words, if the minimum distance between all valid codewords is $(s + 1)$, the received codeword cannot be erroneously mistaken for another codeword. The error will be detected. We need to clarify a point here: Although a code with $d_{min} = s + 1$ may be able to detect more than s errors in some special cases, only s or fewer errors are guaranteed to be detected.

To guarantee the detection of up to s errors in all cases, the minimum Hamming distance in a block code must be $d_{min} = s + 1$.

We can look at this criteria geometrically. Let us assume that the sent codeword x is at the center of a circle with radius s. All received codewords that are created by 0 to s errors are points inside the circle or on the perimeter of the circle. All other valid codewords must be outside the circle, as shown in Figure 3.10. This means that d_{min} must be an integer greater than s or $d_{min} = s + 1$.

Figure 3.10 *Geometric concept explaining d_{min} in error detection*



Example 3.3

The minimum Hamming distance for our first code scheme (Table 3.1) is 2. This code guarantees detection of only a single error. For example, if the third codeword (101) is sent and one error occurs, the received codeword does not match any valid codeword. If two errors occur, however, the received codeword may match a valid codeword and the errors are not detected.

Example 3.4

A code scheme has a Hamming distance $d_{min} = 4$. This code guarantees the detection of up to three errors ($d = s + 1$ or $s = 3$).

Linear Block Codes

Almost all block codes used today belong to a subset of block codes called *linear block codes*. The use of nonlinear block codes for error detection and correction is not as widespread because their structure makes theoretical analysis and implementation difficult. We therefore concentrate on linear block codes. The formal definition of linear block codes requires the knowledge of abstract algebra (particularly Galois fields), which is beyond the scope of this book. We therefore give an informal definition. For our purposes, a linear block code is a code in which the exclusive OR (addition modulo-2) of two valid codewords creates another valid codeword.

Example 3.5

The code in Table 3.1 is a linear block code because the result of XORing any codeword with any other codeword is a valid codeword. For example, the XORing of the second and third codewords creates the fourth one.

Minimum Distance for Linear Block Codes

The minimum Hamming distance for a linear block code is simple to find. It is the number of 1s in the nonzero valid codeword with the smallest number of 1s.

Example 3.6

In our first code (Table 3.1), the numbers of 1s in the nonzero codewords are 2, 2, and 2. So the minimum Hamming distance is $d_{min} = 2$.

Parity-Check Code

Perhaps the most familiar error-detecting code is the **parity-check code**. This code is a linear block code. In this code, a k-bit dataword is changed to an n-bit codeword where $n = k + 1$. The extra bit, called the *parity bit*, is selected to make the total number of 1s in the codeword even. The minimum Hamming distance for this category is $d_{min} = 2$, which means that the code is a single-bit error-detecting code. Our first code (Table 3.1) is a parity-check code ($k = 2$ and $n = 3$). The code in Table 3.2 is also a parity-check code with $k = 4$ and $n = 5$.

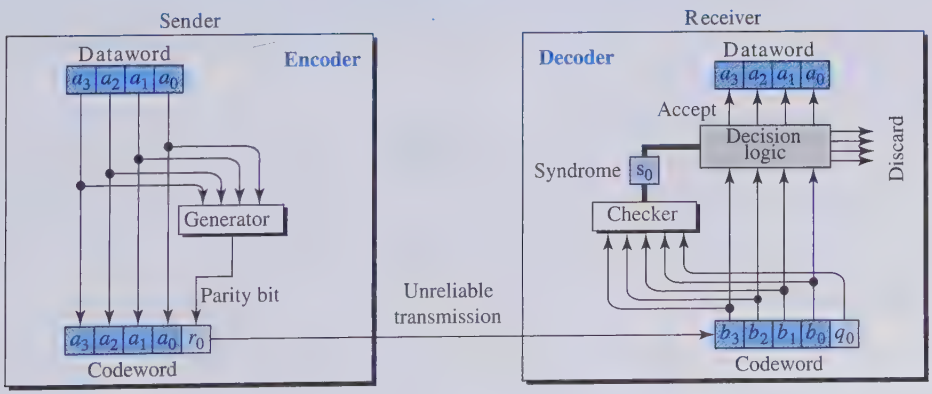
Table 3.2 *Simple parity-check code C(5, 4)*

Datawords	**Codewords**	*Datawords*	**Codewords**
0000	**00000**	1000	**10001**
0001	**00011**	1001	**10010**
0010	**00101**	1010	**10100**
0011	**00110**	1011	**10111**
0100	**01001**	1100	**11000**
0101	**01010**	1101	**11011**
0110	**01100**	1110	**11101**
0111	**01111**	1111	**11110**

Figure 3.11 shows a possible structure of an encoder (at the sender) and a decoder (at the receiver). The encoder uses a generator that takes a copy of a 4-bit dataword (a_0, a_1, a_2, and a_3) and generates a parity bit r_0. The dataword bits and the parity bit create the 5-bit codeword. The parity bit that is added makes the number of 1s in the codeword even. This is normally done by adding the 4 bits of the dataword (modulo-2); the result is the parity bit. In other words,

$$r_0 = a_3 + a_2 + a_1 + a_0 \qquad \textbf{(modulo-2)}$$

Figure 3.11 *Encoder and decoder for simple parity-check code*



If the number of 1s is even, the result is 0; if the number of 1s is odd, the result is 1. In both cases, the total number of 1s in the codeword is even.

The sender sends the codeword, which may be corrupted during transmission. The receiver receives a 5-bit word. The checker at the receiver does the same thing as the generator in the sender with one exception: The addition is done over all 5 bits. The result, which is called the **syndrome**, is just 1 bit. The syndrome is 0 when the number of 1s in the received codeword is even; otherwise, it is 1.

$$s_0 = b_3 + b_2 + b_1 + b_0 + q_0 \qquad \text{(modulo-2)}$$

The syndrome is passed to the decision logic analyzer. If the syndrome is 0, there is no detectable error in the received codeword; the data portion of the received codeword is accepted as the dataword. If the syndrome is 1, the data portion of the received codeword is discarded. The dataword is not created.

Example 3.7

Let us look at some transmission scenarios. Assume the sender sends the dataword 1011. The codeword created from this dataword is 10111, which is sent to the receiver. We examine five cases:

1. No error occurs; the received codeword is 10111. The syndrome is 0. The dataword 1011 is created.
2. One single-bit error changes a_1. The received codeword is 10011. The syndrome is 1. No dataword is created.
3. One single-bit error changes r_0. The received codeword is 10110. The syndrome is 1. No dataword is created. Note that although none of the dataword bits are corrupted, no dataword is created because the code is not sophisticated enough to show the position of the corrupted bit.
4. An error changes r_0, and a second error changes a_3. The received codeword is 00110. The syndrome is 0. The dataword 0011 is created at the receiver. Note that here the dataword is wrongly created due to the syndrome value. The simple parity-check decoder cannot detect an even number of errors. The errors cancel each other out and give the syndrome a value of 0.
5. Three bits—a_3, a_2, and a_1—are changed by errors. The received codeword is 01011. The syndrome is 1. The dataword is not created. This shows that the simple parity check, guaranteed to detect one single error, can also find an odd number of errors.

A parity-check code can detect an odd number of errors.

Cyclic Codes

Cyclic codes are special linear block codes with one extra property. In a **cyclic code**, if a codeword is cyclically shifted (rotated), the result is another codeword. For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword. In this case, if we call the bits in the first word a_0 to a_6, and the bits in the second word b_0 to b_6, we can shift the bits by using the following:

$$b_1 = a_0 \qquad b_2 = a_1 \qquad b_3 = a_2 \qquad b_4 = a_3 \qquad b_5 = a_4 \qquad b_6 = a_5 \qquad b_0 = a_6$$

In the rightmost equation, the last bit of the first word is wrapped around and becomes the first bit of the second word.

Cyclic Redundancy Check

We can create cyclic codes to correct errors. However, the theoretical background required is beyond the scope of this book. In this section, we simply discuss a subset of cyclic codes called the **cyclic redundancy check (CRC)** that is used in networks such as LANs and WANs.

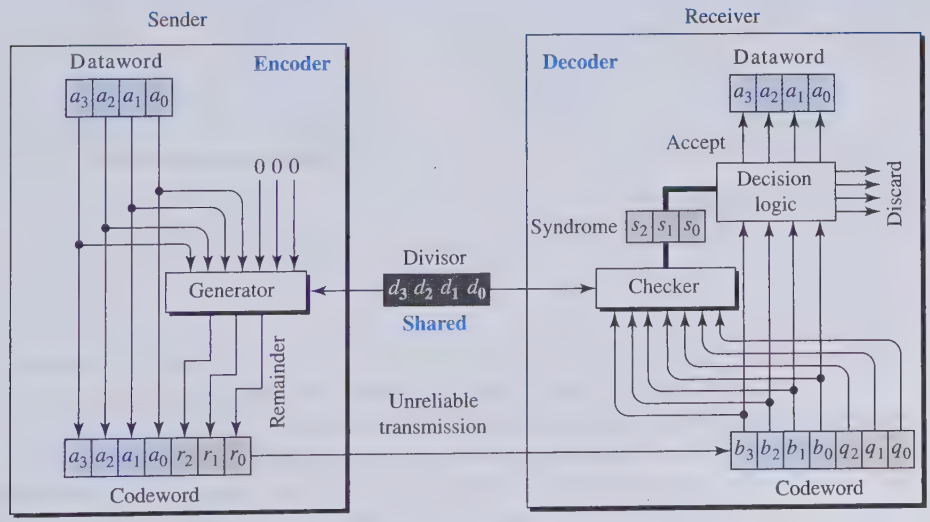
Table 3.3 shows an example of a CRC code. We can see both the linear and cyclic properties of this code.

Table 3.3 *A CRC code with C(7, 4)*

Dataword	*Codeword*	*Dataword*	*Codeword*
0000	0000000	1000	1000101
0001	0001011	1001	1001110
0010	0010110	1010	1010011
0011	0011101	1011	1011000
0100	0100111	1100	1100010
0101	0101100	1101	1101001
0110	0110001	1110	1110100
0111	0111010	1111	1111111

Figure 3.12 shows one possible design for the encoder and decoder. In the encoder, the dataword has k bits (4 here); the codeword has n bits (7 here). The size of the dataword is augmented by adding $n - k$ (3 here) 0s to the right-hand side of the word. The

Figure 3.12 *CRC encoder and decoder*

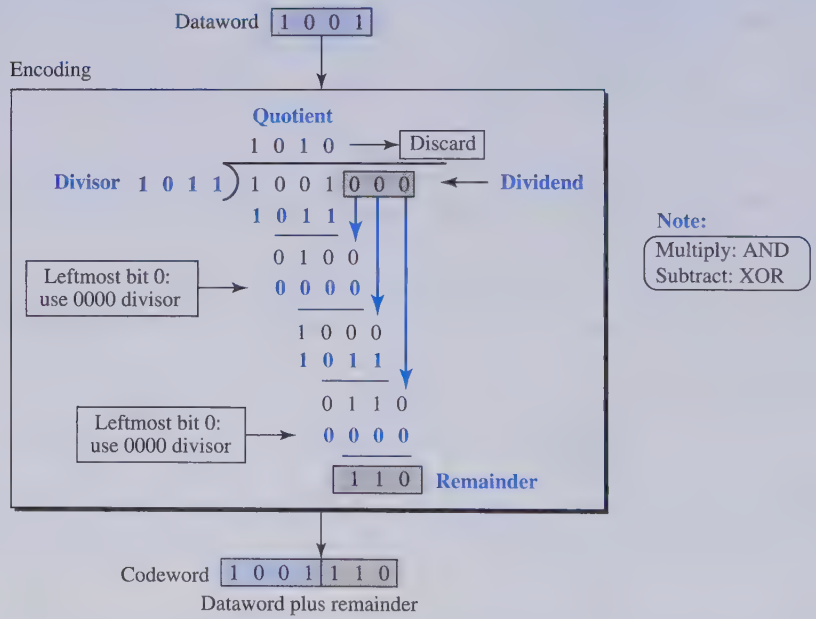


n-bit result is fed into the generator. The generator uses a divisor of size $n - k + 1$ (4 here), predefined and agreed upon. The generator divides the augmented dataword by the divisor (modulo-2 division). The quotient of the division is discarded; the remainder $(r_2 r_1 r_0)$ is appended to the dataword to create the codeword.

The decoder receives the codeword (possibly corrupted in transition). A copy of all n bits is fed to the checker, which is a replica of the generator. The remainder produced by the checker is a syndrome of $n - k$ (3 here) bits, which is fed to the decision logic analyzer. The analyzer has a simple function. If the syndrome bits are all 0s, the 4 leftmost bits of the codeword are accepted as the dataword (interpreted as no error); otherwise, the 4 bits are discarded (error).

Encoder Let us take a closer look at the encoder. The encoder takes a dataword and augments it with $n - k$ number of 0s. It then divides the augmented dataword by the divisor, as shown in Figure 3.13.

Figure 3.13 *Division in CRC encoder*



The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. However, addition and subtraction in this case are the same; we use the XOR operation to do both.

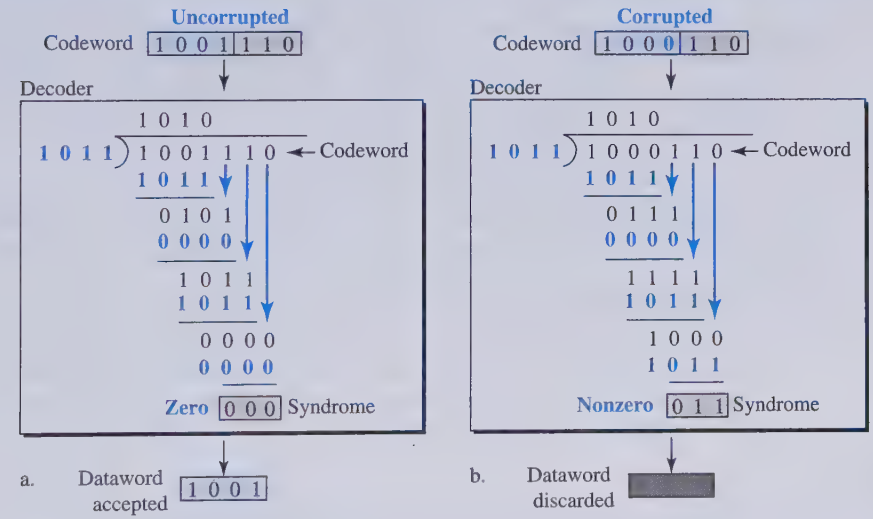
As in decimal division, the process is done step by step. In each step, a copy of the divisor is XORed with the 4 bits of the dividend. The result of the XOR operation (remainder) is 3 bits (in this case), which is used for the next step after 1 extra

bit is pulled down to make it 4 bits long. There is one important point we need to remember in this type of division. If the leftmost bit of the dividend (or the part used in each step) is 0, the step cannot use the regular divisor; we need to use an all-0s divisor.

When there are no bits left to pull down, we have a result. The 3-bit remainder forms the check bits (r_2, r_1, and r_0). They are appended to the dataword to create the codeword.

Decoder The codeword can change during transmission. The decoder does the same division process as the encoder. The remainder of the division is the syndrome. If the syndrome is all 0s, there is no error with a high probability; the dataword is separated from the received codeword and accepted. Otherwise, everything is discarded. Figure 3.14 shows two cases: Figure 3.14a shows the value of the syndrome when no error has occurred; the syndrome is 000. Figure 3.14b shows the case in which there is a single error. The syndrome is not all 0s (it is 011).

Figure 3.14 *Division in the CRC decoder for two cases*



Divisor You may be wondering how the divisor 1011 is chosen. This depends on the expectation we have from the code. Some of the standard divisors used in networking are shown in Table 3.4. The number in the name of the divisor (for example, CRC-32) refers to the degree of the polynomial (the highest power) representing the divisor. The number of bits is always one more than the degree of the polynomial. For example, CRC-8 has 9 bits and CRC-32 has 33 bits.

Table 3.4 *Standard polynomials*

Name	*Binary*	*Application*
CRC-8	100000111	ATM header
CRC-10	11000110101	ATM AAL
CRC-16	10001000000100001	HDLC
CRC-32	100000100110000010001110110110111	LANs

Requirement

We can mathematically prove that a bit pattern needs to have at least two properties to be considered a generator (divisor):

1. The pattern should have at least 2 bits.
2. The rightmost and leftmost bits should both be 1s.

Performance

The following shows the performance of CRC.

- **Single errors.** All qualified generators (see above) can detect any single-bit error.
- **Odd number of errors.** All qualified generators can detect any odd number of errors if the generator can be evenly divided by $(11)_2$ using binary division in modulo-2 arithmetic; otherwise, only some odd number errors will be detected.
- **Burst errors.** If we assume the length of the burst error is L bits and r is the length of the remainder (r is the length of the generator minus 1; it is also the value of the highest power in the polynomial representing the generator):
  - All burst errors of the size $L \leq r$ are detected.
  - All burst errors of the size $L = r + 1$ are detected with probability $1 - (0.5)^{r-1}$.
  - All burst errors of the size $L > r + 1$ are detected with probability $1 - (0.5)^{r}$.

Advantages of Cyclic Codes

Cyclic codes can easily be implemented in hardware and software. They are especially fast when implemented in hardware. This has made cyclic codes a good candidate for many networks. The book website shows how division can be done by a shift register that is included in the hardware of the node.

Checksum

Checksum is an error-detecting technique that can be applied to a message of any length. In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer. We discuss it when we discuss the network layer.

3.2.3 Two DLC Protocols

Having finished presenting all issues related to the DLC sublayer, we now discuss two DLC protocols that actually implement those concepts. The first, High-Level Data-Link Control, is the base of many protocols that have been designed for LANs. The second, Point-to-Point Protocol, is derived from HDLC and is used for point-to-point links.

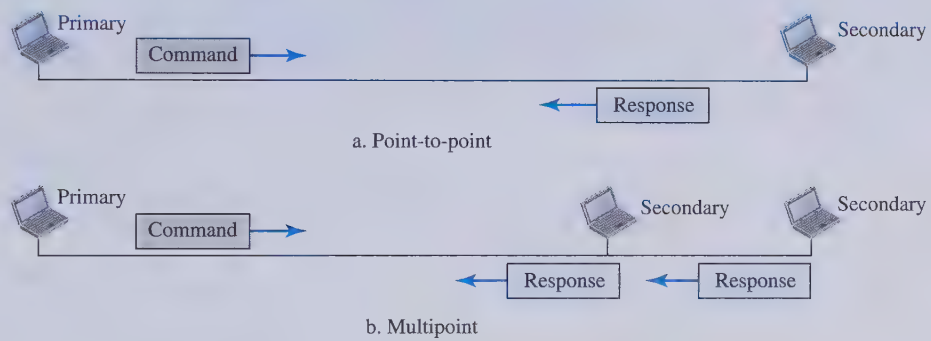
High-Level Data-Link Control

High-level Data-Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links.

Configurations and Transfer Modes

HDLC provides two common transfer modes that can be used in different configurations: *normal response mode (NRM)* and *asynchronous balanced mode (ABM)*. In NRM, the station configuration is unbalanced. We have one primary station and multiple secondary stations. A *primary station* can send commands; a *secondary station* can only respond. The NRM is used for both point-to-point and multipoint links, as shown in Figure 3.15.

Figure 3.15 *Normal response mode*



In ABM, the configuration is balanced. The link is point-to-point, and each station can function as a primary and a secondary (acting as peers), as shown in Figure 3.16. This is the common mode today.

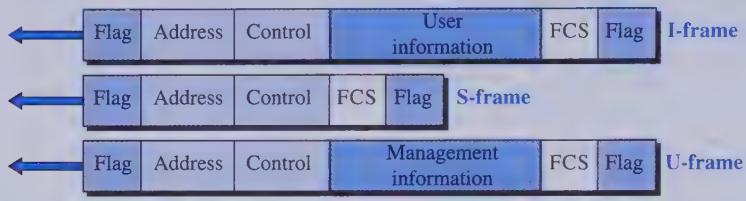
Figure 3.16 *Asynchronous balanced mode*

Frames

To provide the flexibility necessary to support all the options possible in the modes and configurations just described, HDLC defines three types of frames: *information frames (I-frames)*, *supervisory frames (S-frames)*, and *unnumbered frames (U-frames)*. Each type of frame serves as an envelope for the transmission of a different type of message.

I-frames are used to transport user data and control information relating to user data (piggybacking). S-frames are used only to transport control information. U-frames are reserved for system management. Information carried by U-frames is intended for managing the link itself. Each frame in HDLC may contain up to six fields, as shown in Figure 3.17: a beginning flag field, an address field, a control field, an information field, a frame check sequence (FCS) field, and an ending flag field. In multiple-frame transmissions, the ending flag of one frame can serve as the beginning flag of the next frame.

Figure 3.17 *HDLC frames*

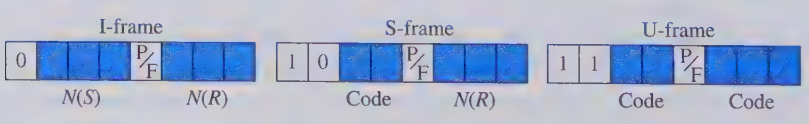


Let us now discuss the fields and their use in different frame types.

- **Flag field.** This field contains synchronization pattern 01111110, which identifies both the beginning and the end of a frame.
- **Address field.** This field contains the address of the secondary station. If a primary station created the frame, it contains a *to* address. If a secondary station creates the frame, it contains a *from* address. The address field can be 1 byte or several bytes long, depending on the needs of the network.
- **Control field.** The control field is 1 or 2 bytes used for flow and error control. The interpretation of bits is discussed later.
- **Information field.** The information field contains the user's data from the network layer or management information. Its length can vary from one network to another.
- **FCS field.** The frame check sequence (FCS) is the HDLC error-detection field. It can contain either a 2- or 4-byte CRC.

The control field determines the type of frame and defines its functionality. So let us discuss the format of this field in detail. The format is specific for the type of frame, as shown in Figure 3.18.

Figure 3.18 *Control field format for the different frame types*



Control Field for I-Frames I-frames are designed to carry user data from the network layer. In addition, they can include flow- and error-control information (piggybacking). The subfields in the control field are used to define these functions. The first bit defines the type. If the first bit of the control field is 0, this means the frame is an I-frame. The next 3 bits, called $N(S)$, define the sequence number of the frame. Note that with 3 bits, we can define a sequence number between 0 and 7. The last 3 bits, called $N(R)$, correspond to the acknowledgment number when piggybacking is used. The single bit between $N(S)$ and $N(R)$ is called the P/F bit. The P/F field is a single bit with a dual purpose. It has meaning only when it is set (bit = 1) and can mean poll or final. It means *poll* when the frame is sent by a primary station to a secondary station (when the address field contains the address of the receiver). It means *final* when the frame is sent by a secondary station to a primary station (when the address field contains the address of the sender).

Control Field for S-Frames Supervisory frames are used for flow and error control whenever piggybacking is either impossible or inappropriate. S-frames do not have information fields. If the first 2 bits of the control field are 10, this means the frame is an S-frame. The last 3 bits, called $N(R)$, correspond to the acknowledgment number (ACK) or negative acknowledgment number (NAK) depending on the type of S-frame. The 2 bits called *code* are used to define the type of S-frame itself. With 2 bits, we can have four types of S-frames:

- **Receive ready (RR).** If the value of the code subfield is 00, it is an RR S-frame. This kind of frame acknowledges the receipt of a safe and sound frame or group of frames. In this case, the value of the $N(R)$ field defines the acknowledgment number.
- **Receive not ready (RNR).** If the value of the code subfield is 10, it is an RNR S-frame. This kind of frame is an RR frame with additional functions. It acknowledges the receipt of a frame or group of frames, and it announces that the receiver is busy and cannot receive more frames. It acts as a kind of congestion-control mechanism by asking the sender to slow down. The value of $N(R)$ is the acknowledgment number.
- **Reject (REJ).** If the value of the code subfield is 01, it is an REJ S-frame. The frame is rejected.
- **Selective reject (SREJ).** If the value of the code subfield is 11, it is an SREJ S-frame. This is a selective reject frame. The receiver can keep or delete the frame.

Control Field for U-Frames Unnumbered frames are used to exchange session management and control information between connected devices. Unlike S-frames, U-frames contain an information field, but one used for system management information, not user data. As with S-frames, however, much of the information carried by U-frames is contained in codes included in the control field. U-frame codes are divided into two sections: a 2-bit prefix before the P/F bit and a 3-bit suffix after the P/F bit. Together, these two segments (5 bits) can be used to create up to 32 different types of U-frames.

Point-to-Point Protocol One of the most common protocols for point-to-point access is the **Point-to-Point Protocol (PPP)**. Today, millions of Internet users who need to connect their home computers to the server of an Internet service provider use PPP. The majority of these users have a traditional **modem**; they are connected to the Internet

through a telephone line, which provides the services of the physical layer. But to control and manage the transfer of data, there is a need for point-to-point access at the data-link layer. PPP is by far the most common.

Services

The designers of PPP have included several services to make it suitable for a point-to-point protocol, but have ignored some traditional services to make it simple.

Services Provided by PPP PPP defines the format of the frame to be exchanged between devices. It also defines how two devices can negotiate the establishment of the link and the exchange of data. PPP is designed to accept payloads from several network layers [not only Internet Protocol (IP)]. Authentication is also provided in the protocol, but it is optional. The new version of PPP, called *Multilink PPP*, provides connections over multiple links. One interesting feature of PPP is that it provides network address configuration. This is particularly useful when a home user needs a temporary network address to connect to the Internet.

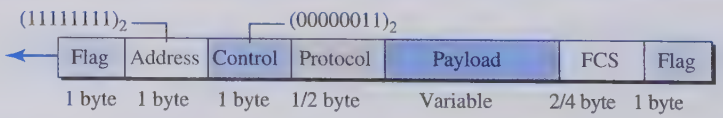
Services Not Provided by PPP PPP does not provide flow control. A sender can send several frames one after another with no concern about overwhelming the receiver. PPP has a very simple mechanism for error control. A CRC field is used to detect errors. If the frame is corrupted, it is silently discarded; the upper-layer protocol needs to take care of the problem. Lack of error control and sequence numbering may cause a packet to be received out of order. PPP does not provide a sophisticated addressing mechanism to handle frames in a multipoint configuration.

Framing

PPP uses a character-oriented (or byte-oriented) frame. Figure 3.19 shows the format of a PPP frame. The description of each field follows:

- **Flag.** A PPP frame starts and ends with a 1-byte flag with the bit pattern 01111110.
- **Address.** The address field in this protocol is a constant value and set to 11111111 (broadcast address).
- **Control.** This field is set to the constant value 00000011 (imitating unnumbered frames in HDLC). As we will discuss later, PPP does not provide any flow control. Error control is also limited to error detection.
- **Protocol.** The protocol field defines what is being carried in the data field: either user data or other information. This field is by default 2 bytes long, but the two parties can agree to use only 1 byte.

Figure 3.19 *PPP frame format*



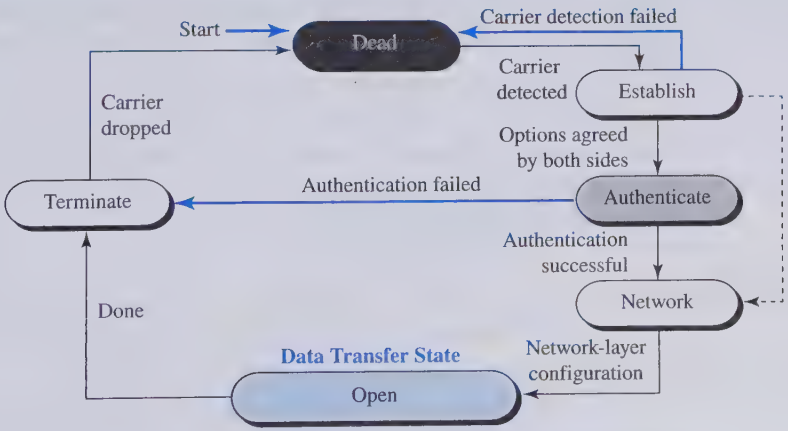
- **Payload field.** This field carries either the user data or other information that we will discuss shortly. The data field is a sequence of bytes with the default of a maximum of 1500 bytes, but this can be changed during negotiation. The data field is byte-stuffed if the flag byte pattern appears in this field. Because there is no field defining the size of the data field, padding is needed if the size is less than the maximum default value or the maximum negotiated value.
- **FCS.** The frame check sequence (FCS) is simply a 2-byte or 4-byte standard CRC.

Byte Stuffing Because PPP is a byte-oriented protocol, the flag in PPP is a byte that needs to be escaped whenever it appears in the data section of the frame. The escape byte is 01111101, which means that every time the flaglike pattern appears in the data, this extra byte is stuffed to tell the receiver that the next byte is not a flag. Obviously, the escape byte itself should be stuffed with another escape byte.

Transition Phases

A PPP connection goes through phases that can be shown in a *transition phase* diagram (see Figure 3.20). The transition diagram starts with the *dead* state. In this state, there is no active carrier (at the physical layer) and the line is quiet. When one of the two nodes starts the communication, the connection goes into the *establish* state. In this state, options are negotiated between the two parties. If the two ends agree with authentication, the system goes to the *authenticate* state; otherwise, the system goes to the *network* state. The Link Control Protocol packets, discussed shortly, are used for this purpose. Several packets may be exchanged here. Data transfer takes place in the *open* state. When a connection reaches this state, the exchange of data packets can be started. The connection remains in this state until one of the endpoints wants to terminate the connection. In this case, the system goes to the *terminate* state. The system remains in this state until the carrier (physical-layer signal) is dropped, which moves the system to the *dead* state again.

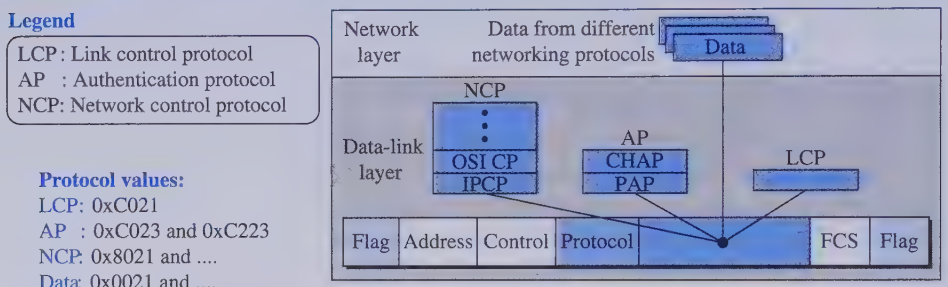
Figure 3.20 *Transition phases*



Multiplexing

Although PPP is a link-layer protocol, it uses another set of protocols to establish the link, authenticate the parties involved, and carry the network-layer data. Three sets of protocols are defined to make PPP powerful: the Link Control Protocol (LCP), two Authentication Protocols (APs), and several Network Control Protocols (NCPs). At any moment, a PPP packet can carry data from one of these protocols in its data field, as shown in Figure 3.21. Note that there is one LCP, two APs, and several NCPs. Data may also come from several different network layers.

Figure 3.21 *Multiplexing in PPP*



Link Control Protocol The **Link Control Protocol (LCP)** is responsible for establishing, maintaining, configuring, and terminating links. It also provides negotiation mechanisms to set options between the two endpoints. Both endpoints of the link must reach an agreement about the options before the link can be established.

Authentication Protocols Authentication plays a very important role in PPP because PPP is designed for use over dial-up links where verification of user identity is necessary. *Authentication* means validating the identity of a user who needs to access a set of resources. PPP has created two protocols for authentication: Password Authentication Protocol and Challenge Handshake Authentication Protocol. Note that these protocols are used during the authentication phase.

- **PAP.** The *Password Authentication Protocol (PAP)* is a simple authentication procedure with a two-step process:
  - **a.** The user who wants to access a system sends an authentication identification (usually the user name) and a password.
  - **b.** The system checks the validity of the identification and password and either accepts or denies connection.
- **CHAP.** The *Challenge Handshake Authentication Protocol (CHAP)* is a three-way handshaking authentication protocol that provides greater security than PAP. In this method, the password is kept secret; it is never sent online.
  - **a.** The system sends the user a challenge packet containing a challenge value, usually a few bytes.

b. The user applies a predefined function that takes the challenge value and the user's own password and creates a result. The user sends the result in the response packet to the system.

c. The system does the same. It applies the same function to the password of the user (known to the system) and the challenge value to create a result. If the result created is the same as the result sent in the response packet, access is granted; otherwise, it is denied. CHAP is more secure than PAP, especially if the system continuously changes the challenge value. Even if the intruder learns the challenge value and the result, the password is still secret.

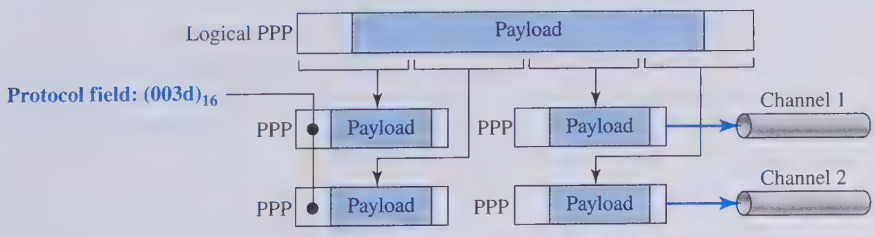
Network Control Protocols PPP is a multiple-network-layer protocol. It can carry a network-layer data packet from protocols defined by the Internet, OSI, Xerox, DECnet, AppleTalk, Novel, and so on. To do this, PPP has defined a specific Network Control Protocol for each network protocol. Xerox CP does the same for the Xerox protocol data packets, and so on. Note that none of the NCP packets carry network-layer data; they just configure the link at the network layer for the incoming data. One NCP protocol is the **Internet Protocol Control Protocol (IPCP)**. This protocol configures the link used to carry IP data packets in the Internet. IPCP is especially of interest to us.

Data from the Network Layer After the network-layer configuration is completed by one of the NCP protocols, users can exchange data packets from the network layer. Here again, there are different protocol fields for different network layers. For example, if PPP is carrying data from the IP network layer, the field value is $(0021)_{16}$. If PPP is carrying data from the OSI network layer, the protocol field value is $(0023)_{16}$, and so on.

Multilink PPP

PPP was originally designed for a single-channel point-to-point physical link. The availability of multiple channels in a single point-to-point link motivated the development of Multilink PPP. In this case, a logical PPP frame is divided into several actual PPP frames. A segment of the logical frame is carried in the payload of an actual PPP frame, as shown in Figure 3.22. To show that the actual PPP frame is carrying a fragment of a logical PPP frame, the protocol field is set to $(003d)_{16}$. This new development adds complexity. For example, a sequence number needs to be added to the actual PPP frame to show a fragment's position in the logical frame.

Figure 3.22 *Multilink PPP*



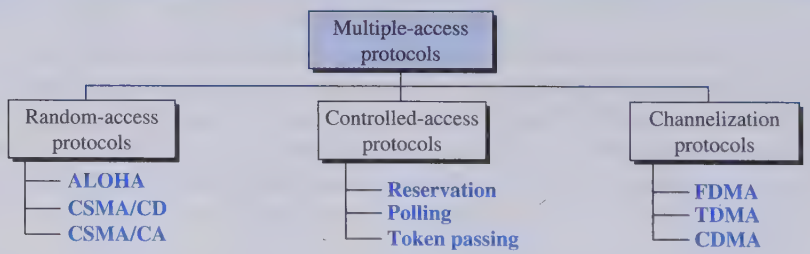
3.3 MEDIA ACCESS PROTOCOLS

We said that the data-link layer is divided into two sublayers: data-link control (DLC) and media access control. We discussed DLC in Section 3.2; we talk about **media access control (MAC)** in this section. When we are using a dedicated link, such as a dial-up telephone line, we need only a data-link-control protocol, such as the Point-to-Point Protocol (PPP), that manages the data transfer between the two ends. On the other hand, if we are sharing the media, wire or air, with other users, we need to have a protocol to first manage the sharing process and then to do the data transfer. For example, if we use our cellular phone to connect to another cellular phone, the channel (the band allocated to the vendor company) is not dedicated. A person a few feet away from us may be using the same band to talk to her friend.

When nodes or stations are connected and use a common link, called a multipoint or broadcast link, we need a multiple-access protocol to coordinate access to the link. The problem of controlling access to the medium is similar to the rules of speaking in an assembly. The procedures guarantee that the right to speak is upheld and ensure that two people do not speak at the same time, do not interrupt each other, do not monopolize the discussion, and so on. The situation is similar for multipoint networks. We need to be sure that each node gets access to the link. The first goal is to prevent any collision between nodes. If somehow a collision does occur, the second goal is to handle the collision.

Many protocols have been devised to handle access to a shared link. We categorize them into three groups. Protocols belonging to each group are shown in Figure 3.23.

Figure 3.23 *Taxonomy of multiple-access protocols discussed in this chapter*



3.3.1 Random Access

In **random-access** or **contention methods**, no station is superior to another station and none is assigned control over another. At each instance, a station that has data to send uses a procedure defined by the protocol to make a decision on whether or not to send. This decision depends on the state of the medium (idle or busy). In other words, each station can transmit when it desires on the condition that it follows the predefined procedure, including the testing of the state of the medium.

Two features give this method its name. First, there is no scheduled time for a station to transmit. Transmission is random among the stations. That is why these methods

are called *random access*. Second, no rules specify which station should send next. Stations compete with one another to access the medium. That is why these methods are also called *contention* methods.

In a random-access method, each station has the right to the medium without being controlled by any other station. However, if more than one station tries to send, there is an access conflict—*collision*—and the frames will be either destroyed or modified. To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:

- When can the station access the medium?
- What can the station do if the medium is busy?
- How can the station determine the success or failure of the transmission?
- What can the station do if there is an access conflict?

The random-access methods we study in this chapter have evolved from a very interesting protocol known as *ALOHA*, which used a very simple procedure called *multiple access (MA)*. The method was improved with the addition of a procedure that forces the station to sense the medium before transmitting. This was called *carrier sense multiple access (CSMA)*. This method later evolved into two parallel methods: *carrier sense multiple access with collision detection (CSMA/CD)*, which tells the station what to do when a collision is detected, and *carrier sense multiple access with collision avoidance (CSMA/CA)*, which tries to avoid the collision.

ALOHA

ALOHA, the earliest random-access method, was developed at the University of Hawaii in early 1970. It was designed for a radio (wireless) LAN, but it can be used on any shared medium.

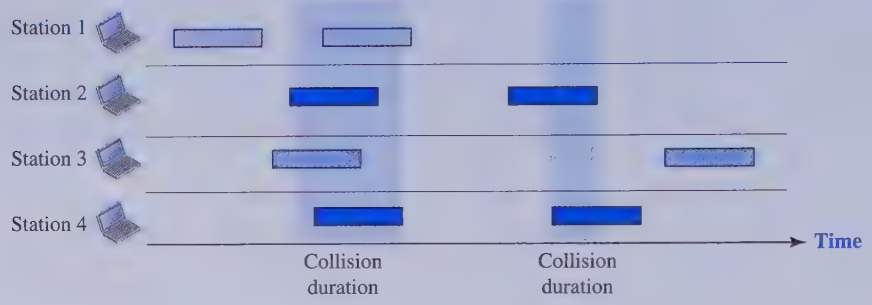
It is obvious that there are potential collisions in this arrangement. The medium is shared between the stations. When a station sends data, another station may attempt to do so at the same time. The data from the two stations collide and become garbled.

Pure ALOHA

The original ALOHA protocol is called **pure ALOHA**. This is a simple but elegant protocol. The idea is that each station sends a frame whenever it has a frame to send (multiple access). However, because there is only one channel to share, there is the possibility of collision between frames from different stations. Figure 3.24 shows an example of frame collisions in pure ALOHA.

There are four stations (unrealistic assumption) that contend with one another for access to the shared channel. Figure 3.24 shows that each station sends two frames; There are a total of eight frames on the shared medium. Some of these frames collide because multiple frames are in contention for the shared channel. Only two frames survive: one frame from station 1 and one frame from station 3. We need to mention that even if 1 bit of a frame coexists on the channel with 1 bit from another frame, there is a collision and both will be destroyed. It is obvious that we need to resend the frames that have been destroyed during transmission.

Figure 3.24 *Frames in a pure ALOHA network*

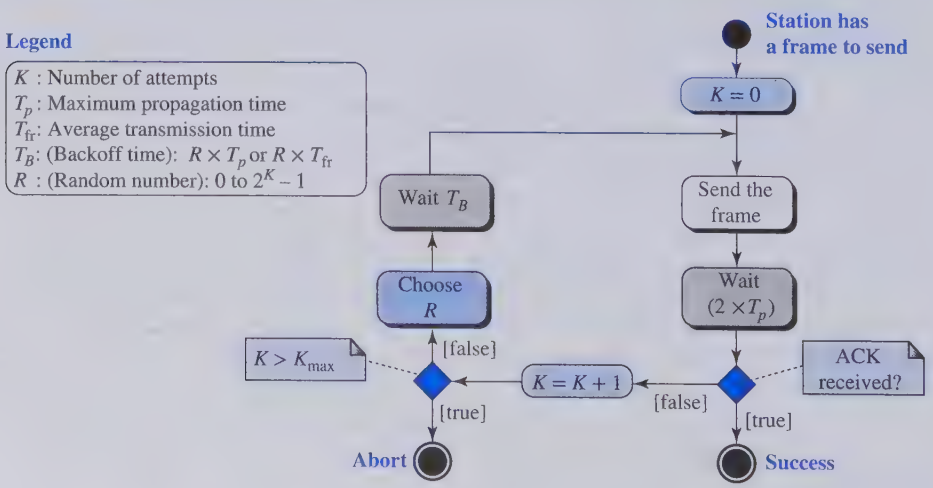


The pure ALOHA protocol relies on acknowledgments from the receiver. When a station sends a frame, it expects the receiver to send an acknowledgment. If the acknowledgment does not arrive after a time-out period, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.

A collision involves two or more stations. If all these stations try to resend their frames after the time-out, the frames will collide again. Pure ALOHA dictates that when the time-out period passes, each station waits a random amount of time before resending its frame. The randomness will help avoid more collisions. We call this time the *backoff time* T_B.

Pure ALOHA has a second method to prevent congesting the channel with retransmitted frames. After a maximum number of retransmission attempts K_{max}, a station must give up and try later. Figure 3.25 shows the procedure for pure ALOHA based on the above strategy.

Figure 3.25 *Procedure for pure ALOHA protocol*



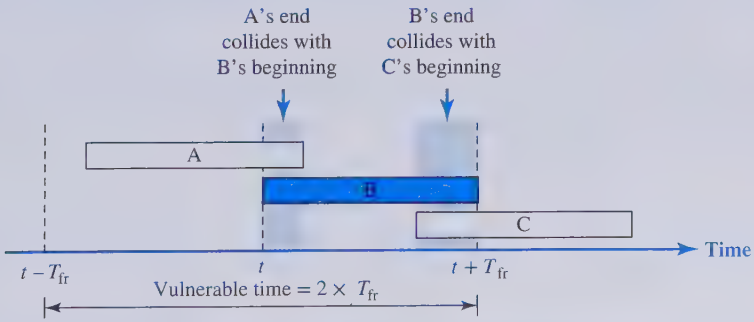
The time-out period is equal to the maximum possible round-trip propagation delay, which is twice the amount of time required to send a frame between the two most widely separated stations ($2 \times T_p$). The backoff time T_B is a random value that normally depends on K (the number of attempted unsuccessful transmissions). The formula for T_B depends on the implementation. One common formula is the *binary exponential backoff*. In this method, for each retransmission, a multiplier $R = 0$ to $2^K - 1$ is randomly chosen and multiplied by T_p (maximum propagation time) or T_{fr} (the average time required to send out a frame) to find T_B. Note that in this procedure, the range of the random numbers increases after each collision. The value of K_{max} is usually chosen as 15.

Example 3.8

The stations on a wireless ALOHA network are a maximum of 600 km apart. If we assume that signals propagate at 3×10^8 m/s, we find $T_p = (600 \times 10^3) / (3 \times 10^8) = 2$ ms. For $K = 2$, the range of R is $\{0, 1, 2, 3\}$. This means that T_B can be 0, 2, 4, or 6 ms, based on the outcome of the random variable R.

Vulnerable time Let us find the length of time, the *vulnerable time*, in which there is a possibility of collision. We assume that the stations send fixed-length frames with each frame taking T_{fr} seconds to send. Figure 3.26 shows the vulnerable time for station B.

Figure 3.26 *Vulnerable time for pure ALOHA protocol*



Station B starts to send a frame at time t. Now imagine station A has started to send its frame after $t - T_{fr}$. This leads to a collision between the frames from station B and station A. On the other hand, suppose that station C starts to send a frame before time $t + T_{fr}$. Here, there is also a collision between frames from station B and station C.

Looking at Figure 3.26, we see that the vulnerable time, during which a collision may occur in pure ALOHA, is 2 times the frame transmission time.

$$\textbf{Pure ALOHA vulnerable time} = 2 \times T_{fr}$$

Example 3.9

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. What is the requirement to make this frame collision-free?

Solution

The average frame transmission time T_{fr} is 200 bits/200 kbps or 1 ms. The vulnerable time is 2×1 ms = 2 ms. This means no station should send later than 1 ms before this station starts transmission and no station should start sending during the period (1 ms) that this station is sending.

Throughput Let us call G the average number of frames generated by the system during one frame transmission time. Then it can be proven that the average number of successfully transmitted frames for pure ALOHA is $S = G \times e^{-2G}$. The maximum throughput S_{max} is 0.184, for $G = 1/2$. (We can find it by setting the derivative of S with respect to G to 0.) In other words, if one-half a frame is generated during one frame transmission time (one frame during two frame transmission times), then 18.4 percent of these frames reach their destination successfully. We expect $G = 1/2$ to produce the maximum throughput because the vulnerable time is 2 times the frame transmission time. Therefore, if a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

The throughput for pure ALOHA is $S = G \times e^{-2G}$.
The maximum throughput $S_{max} = 1/(2e) = 0.184$ when $G = 1/2$.

Example 3.10

A pure ALOHA network transmits 200-bit frames on a shared channel of 200 kbps. Find the throughput for each of the following frames per second produced by the system.

a. 1000
b. 500
c. 250

Solution

The frame transmission time is 200/200 kbps or 1 ms.

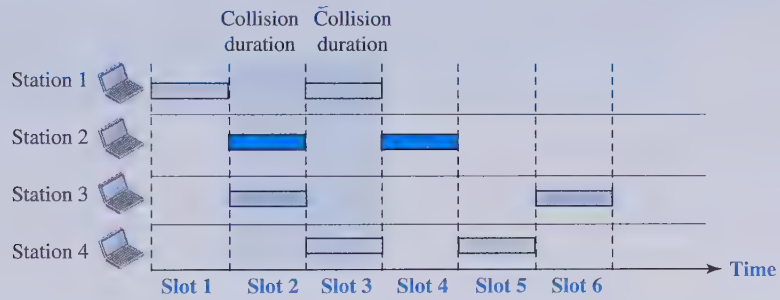
a. If the system creates 1000 frames per second, or 1 frame per millisecond, then $G = 1$. In this case, $S = G \times e^{-2G} = 0.135$ (13.5 percent). This means that the throughput is $1000 \times 0.135 = 135$ frames. Only 135 frames out of 1000 will probably survive.
b. If the system creates 500 frames per second, or 1/2 frame per millisecond, then $G = 1/2$. In this case, $S = G \times e^{-2G} = 0.184$ (18.4 percent). This means that the throughput is $500 \times 0.184 = 92$ and that only 92 frames out of 500 will probably survive. Note that this is the *maximum* throughput case, percentage-wise.
c. If the system creates 250 frames per second, or 1/4 frame per millisecond, then $G = 1/4$. In this case, $S = G \times e^{-2G} = 0.152$ (15.2 percent). This means that the throughput is $250 \times 0.152 = 38$. Only 38 frames out of 250 will probably survive.

Slotted ALOHA

Pure ALOHA has a vulnerable time of $2 \times T_{fr}$. This is so because there is no rule that defines when the station can send. A station may send soon after another station has started or just before another station has finished. Slotted ALOHA was invented to improve the efficiency of pure ALOHA.

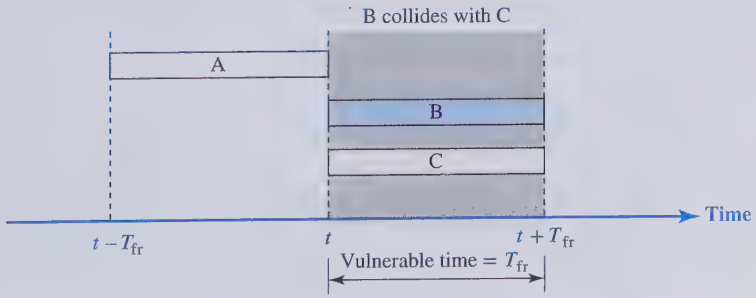
In **slotted ALOHA** we divide the time into slots of T_{fr} seconds and force the station to send only at the beginning of the time slot. Figure 3.27 shows an example of frame collisions in slotted ALOHA.

Figure 3.27 *Frames in a slotted ALOHA network*



Because a station is allowed to send only at the beginning of the synchronized time slot, if a station misses this moment, it must wait until the beginning of the next time slot. This means that the station that started at the beginning of this slot has already finished sending its frame. Of course, there is still the possibility of collision if two stations try to send at the beginning of the same time slot. However, the vulnerable time is now reduced to one-half, equal to T_{fr}. Figure 3.28 shows this situation.

Figure 3.28 *Vulnerable time for slotted ALOHA protocol*



Throughput It can be proven that the average number of successful transmissions for slotted ALOHA is $S = G \times e^{-G}$. The maximum throughput S_{max} is 0.368, when $G = 1$. In other words, if one frame is generated during one frame transmission time, then 36.8 percent of these frames reach their destination successfully. We expect $G = 1$ to produce maximum throughput because the vulnerable time is equal to the frame transmission time.

$$\textbf{Slotted ALOHA vulnerable time} = T_{fr}$$

Therefore, if a station generates only one frame in this vulnerable time (and no other station generates a frame during this time), the frame will reach its destination successfully.

The throughput for slotted ALOHA is $S = G \times e^{-G}$.
The maximum throughput $S_{max} = 0.368$ when $G = 1$.

Example 3.11

A slotted ALOHA network transmits 200-bit frames using a shared channel with a 200-kbps bandwidth. Find the throughput for each of the following frames per second produced by the system (all stations together).

a. 1000
b. 500
c. 250

Solution

This situation is similar to Exercise 3.10 except that the network is using slotted ALOHA instead of pure ALOHA. The frame transmission time is 200/200 kbps or 1 ms.

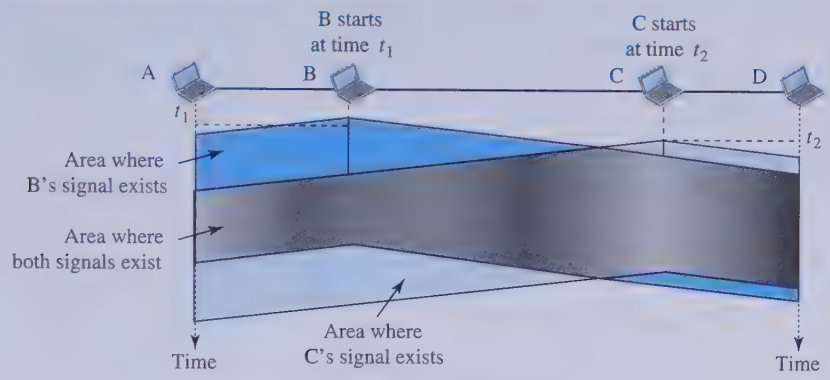
a. In this case G is 1. So $S = G \times e^{-G} = 0.368$ (36.8 percent). This means that the throughput is $1000 \times 0.0368 = 368$ frames. Only 368 out of 1000 frames will probably survive. Note that this is the maximum throughput case, percentage-wise.
b. Here G is 1/2. In this case $S = G \times e^{-G} = 0.303$ (30.3 percent). This means that the throughput is $500 \times 0.0303 = 151$. Only 151 frames out of 500 will probably survive.
c. Now G is 1/4. In this case $S = G \times e^{-G} = 0.195$ (19.5 percent). This means that the throughput is $250 \times 0.195 = 49$. Only 49 frames out of 250 will probably survive.

Carrier Sense Multiple Access (CSMA)

To minimize the chance of collision and, therefore, increase the performance, the CSMA method was developed. The chance of collision can be reduced if a station senses the medium before trying to use it. **Carrier sense multiple access (CSMA)** requires that each station first listen to the medium (or check the state of the medium) before sending. In other words, CSMA is based on the principle "sense before transmit" or "listen before talk."

CSMA can reduce the possibility of collision, but it cannot eliminate it. The reason for this is shown in Figure 3.29, a space and time model of a CSMA network. Stations are connected to a shared channel (usually a dedicated medium).

Figure 3.29 *Space and time model of a collision in CSMA*



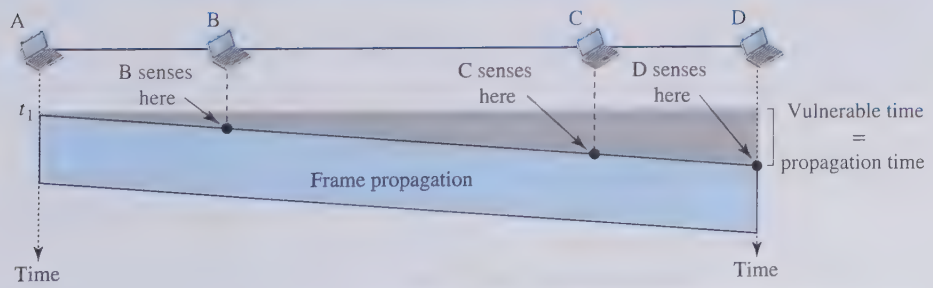
The possibility of collision still exists because of propagation delay; when a station sends a frame, it still takes time (although very short) for the first bit to reach every station and for every station to sense it. In other words, a station may sense the medium and find it idle, only because the first bit sent by another station has not yet been received.

At time t_1, station B senses the medium and finds it idle, so it sends a frame. At time t_2 ($t_2 > t_1$), station C senses the medium and finds it idle because, at this time, the first bits from station B have not reached station C. Station C also sends a frame. The two signals collide, and both frames are destroyed.

Vulnerable Time

The vulnerable time for CSMA is the *propagation time* T_p. This is the time needed for a signal to propagate from one end of the medium to the other. When a station sends a frame and any other station tries to send a frame during this time, a collision will result. But if the first bit of the frame reaches the end of the medium, every station will already have heard the bit and will refrain from sending. Figure 3.30 shows the worst case. The leftmost station A sends a frame at time t_1, which reaches the rightmost station D at time $t_1 + T_p$. The gray area shows the vulnerable area in time and space.

Figure 3.30 *Vulnerable time in CSMA*



Persistence Methods

What should a station do if the channel is busy? What should a station do if the channel is idle? Three methods have been devised to answer these questions: **1-persistent method**, **nonpersistent method**, and ***p*-persistent method**. Figure 3.31 shows the behavior of these three persistence methods when a station finds a channel busy.

Figure 3.31 *Behavior of three persistence methods*

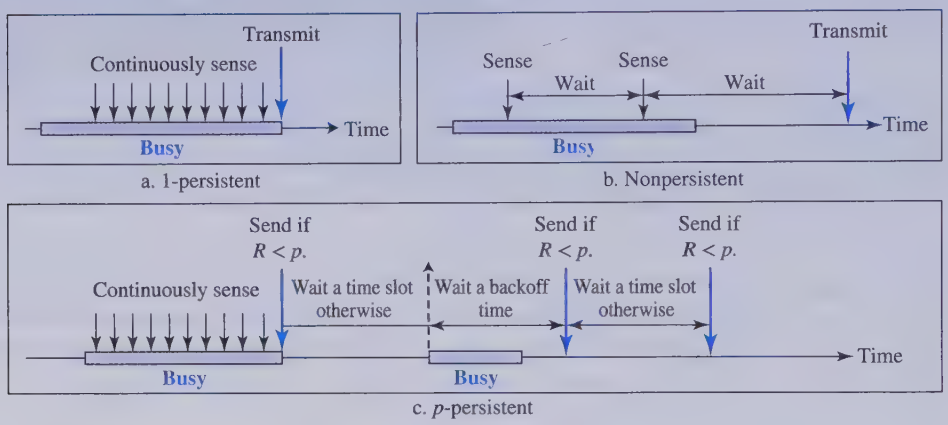


Figure 3.32 shows the flow diagrams for these methods.

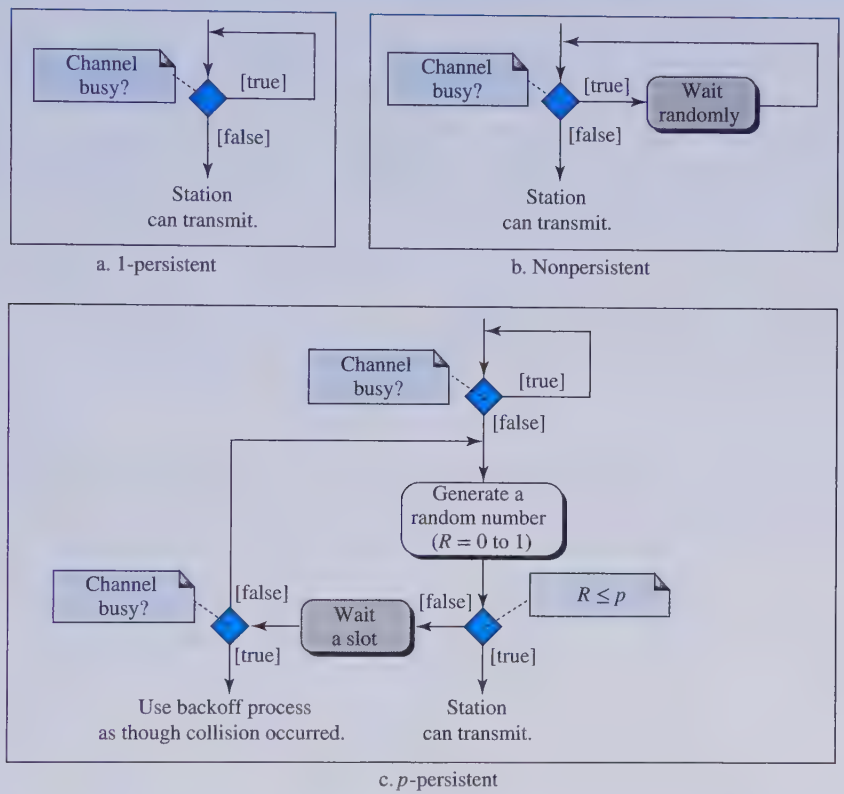
1-Persistent The *1-persistent method* is simple and straightforward. In this method, after the station finds the line idle, it sends its frame immediately (with probability 1). This method has the highest chance of collision because two or more stations may find the line idle and send their frames immediately. We will see later that Ethernet uses this method.

Nonpersistent In the *nonpersistent method*, a station that has a frame to send senses the line. If the line is idle, it sends immediately. If the line is not idle, it waits a random amount of time and then senses the line again. The nonpersistent approach reduces the chance of collision because it is unlikely that two or more stations will wait the same amount of time and retry to send simultaneously. However, this method reduces the efficiency of the network because the medium remains idle when there may be stations with frames to send.

p*-*Persistent The p-*persistent method* is used if the channel has time slots with a slot duration equal to or greater than the maximum propagation time. The *p*-persistent approach combines the advantages of the other two strategies. It reduces the chance of collision and improves efficiency. In this method, after the station finds the line idle, it follows these steps:

1. With probability *p*, the station sends its frame.

Figure 3.32 *Flow diagram for three persistence methods*



2. With probability $q = 1 - p$, the station waits for the beginning of the next time slot and checks the line again.
   a. If the line is idle, it goes to step 1.
   b. If the line is busy, it acts as though a collision has occurred and uses the backoff procedure.

CSMA/CD

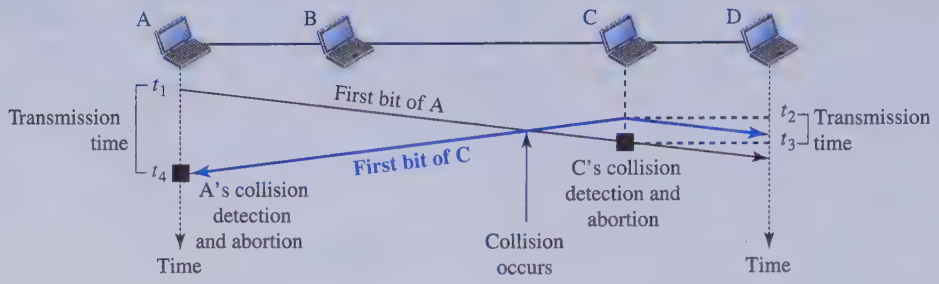
The CSMA method does not specify the procedure following a collision. **Carrier sense multiple access with collision detection (CSMA/CD)** augments the algorithm to handle the collision.

In this method, a station monitors the medium after it sends a frame to see if the transmission was successful. If so, the station is finished. If, however, there is a collision, the frame is sent again.

To better understand CSMA/CD, let us look at the first bits transmitted by the two stations involved in the collision. Although each station continues to send bits in the

frame until it detects the collision, we show what happens as the first bits collide. In Figure 3.33, stations A and C are involved in the collision.

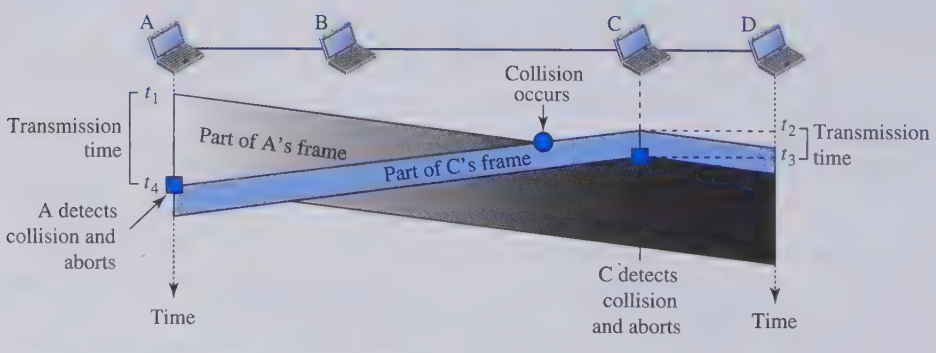
Figure 3.33 *Collision of the first bits in CSMA/CD*



At time t_1, station A has executed its persistence procedure and starts sending the bits of its frame. At time t_2, station C has not yet sensed the first bit sent by A. Station C executes its persistence procedure and starts sending the bits in its frame, which propagate both to the left and to the right. The collision occurs sometime after time t_2. Station C detects a collision at time t_3 when it receives the first bit of A's frame. Station C immediately (or after a short time, but we assume immediately) aborts transmission. Station A detects a collision at time t_4 when it receives the first bit of C's frame; it also immediately aborts transmission. Looking at Figure 3.33, we see that A transmits for the duration $t_4 - t_1$; C transmits for the duration $t_3 - t_2$.

Now that we know the time durations for the two transmissions, we can show a more complete graph in Figure 3.34.

Figure 3.34 *Collision and abortion in CSMA/CD*



Minimum Frame Size

For CSMA/CD to work, we need a restriction on the frame size. Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort the transmission. This is so because the station, once the entire frame is sent, does not keep a copy of the frame and does not monitor the line for collision detection. Therefore, the frame transmission time T_{fr} must be at least 2 times the maximum propagation time T_p. To understand the reason, let us think about the worst-case scenario. If the two stations involved in a collision are the maximum distance apart, the signal from the first takes time T_p to reach the second, and the effect of the collision takes another time T_p to reach the first. So the requirement is that the first station must still be transmitting after $2T_p$.

Example 3.12

A network using CSMA/CD has a bandwidth of 10 Mbps. If the maximum propagation time (including the delays in the devices and ignoring the time needed to send a jamming signal, as we see later) is 25.6 μs, what is the minimum size of the frame?

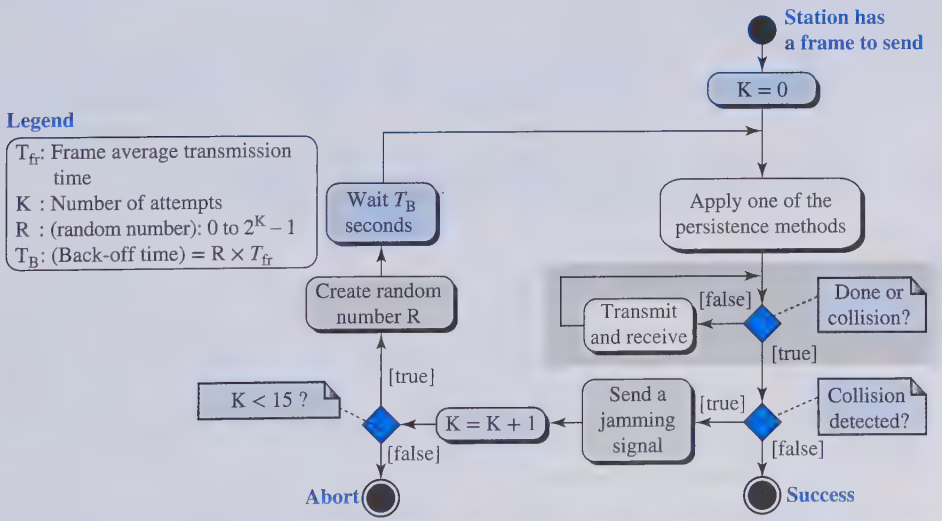
Solution

The minimum frame transmission time is $T_{fr} = 2 \times T_p = 51.2$ μs. This means, in the worst case, a station needs to transmit for a period of 51.2 μs to detect the collision. The minimum size of the frame is 10 Mbps × 51.2 μs = 512 bits or 64 bytes. This is actually the minimum size of the frame for Standard Ethernet, as we will see in Chapter 4.

Procedure

Now let us look at the flow diagram for CSMA/CD in Figure 3.35. It is similar to the one for the ALOHA protocol, but there are differences.

Figure 3.35 *Flow diagram for the CSMA/CD*



The first difference is the addition of the persistence process. We need to sense the channel before we start sending the frame by using one of the persistence processes we discussed previously (nonpersistent, 1-persistent, or *p*-persistent). The corresponding box can be replaced by one of the persistence processes shown in Figure 3.32.

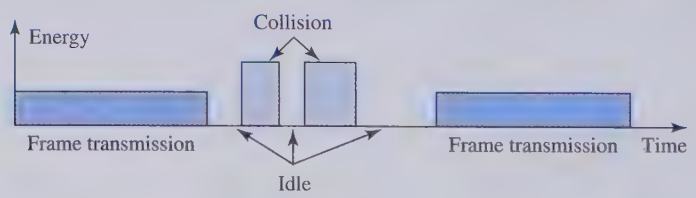
The second difference is the frame transmission. In ALOHA, we first transmit the entire frame and then wait for an acknowledgment. In CSMA/CD, transmission and collision detection is a continuous process. We do not send the entire frame and then look for a collision. The station transmits and receives continuously and simultaneously (using two different ports or a bidirectional port). We use a loop to show that transmission is a continuous process. We constantly monitor to detect one of two conditions: Either transmission is finished, or a collision is detected. Either event stops transmission. When we come out of the loop, if a collision has not been detected, it means that transmission is complete; the entire frame is transmitted. Otherwise, a collision has occurred.

The third difference is the sending of a short *jamming signal* to make sure that all other stations become aware of the collision.

Energy Level

We can say that the level of energy in a channel can have three values: zero, normal, and abnormal. At the zero level, the channel is idle. At the normal level, a station has successfully captured the channel and is sending its frame. At the abnormal level, there is a collision and the level of the energy is twice the normal level. A station that has a frame to send or is sending a frame needs to monitor the energy level to determine if the channel is idle, busy, or in collision mode. Figure 3.36 shows the situation.

Figure 3.36 *Energy level during transmission, idleness, or collision*



Throughput

The throughput of CSMA/CD is greater than that of pure or slotted ALOHA. The maximum throughput occurs at a different value of G and is based on the persistence method and the value of p in the p-persistent approach. For the 1-persistent method, the maximum throughput is around 50 percent when $G = 1$. For the nonpersistent method, the maximum throughput can go up to 90 percent when G is between 3 and 8.

Traditional Ethernet

One of the LAN protocols that used CSMA/CD is the traditional Ethernet with the data rate of 10 Mbps. We discuss the Ethernet LANs in Chapter 4. but it is good to know

that the traditional Ethernet was a broadcast LAN that used 1-persistence method to control access to the common media. Later versions of the Ethernet try to move from CSMA/CD access methods for the reason that we discuss in Chapter 4 when we discuss wired LANs.

CSMA/CA

A variation of the CSMA method is **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)**, which is used in wireless LANs. We'll postpone the discussion until Chapter 4, which deals with that topic.

3.3.2 Controlled Access

In **controlled access**, the stations consult one another to find which station has the right to send. A station cannot send unless it has been authorized by other stations. We discuss three controlled-access methods: reservation, polling, and token passing.

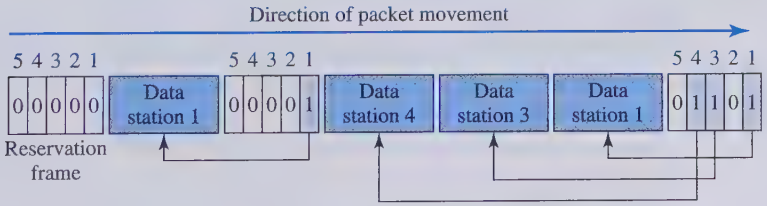
Reservation

In the **reservation** method, a station needs to make a reservation before sending data. Time is divided into intervals. In each interval, a reservation frame precedes the data frames sent in that interval.

If there are N stations in the system, there are exactly N reservation minislots in the reservation frame. Each minislot belongs to a station. When a station needs to send a data frame, it makes a reservation in its own minislot. The stations that have made reservations can send their data frames after the reservation frame.

Figure 3.37 shows a situation with five stations and a five-minislot reservation frame. In the first interval, only stations 1, 3, and 4 have made reservations. In the second interval, only station 1 has made a reservation.

Figure 3.37 *Reservation access method*

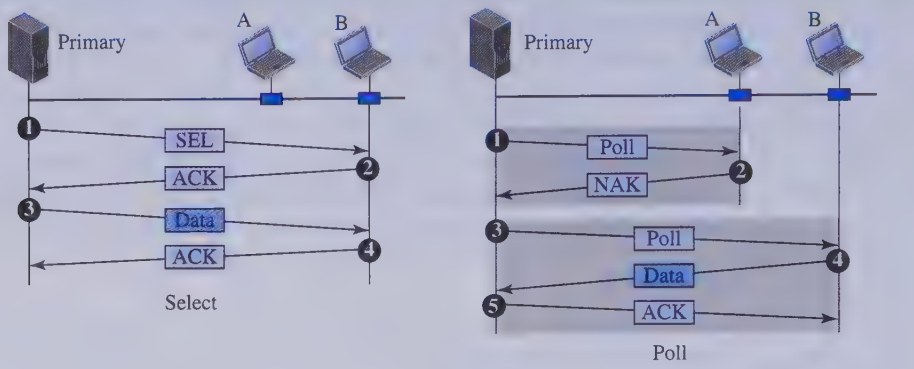


Polling

Polling works with topologies in which one device is designated as a *primary station* and the other devices are *secondary stations.* All data exchanges must be made through the primary device even when the ultimate destination is a secondary device. The primary device controls the link; the secondary devices follow its instructions. It is up to the primary device to determine which device is allowed to use the channel at a given time.

The primary device, therefore, is always the initiator of a session (see Figure 3.38). This method uses poll and select functions to prevent collisions. However, the drawback is if the primary station fails, the system goes down.

Figure 3.38 *Select and poll functions in the polling-access method*



Select

The *select* function is used whenever the primary device has something to send. Remember that the primary controls the link. If the primary is neither sending nor receiving data, it knows the link is available. If it has something to send, the primary device sends it. What it does not know, however, is whether the target device is prepared to receive. So the primary must alert the secondary to the upcoming transmission and wait for an acknowledgment of the secondary's ready status. Before sending data, the primary creates and transmits a select (SEL) frame, one field of which includes the address of the intended secondary.

Poll

The *poll* function is used by the primary device to solicit transmissions from the secondary devices. When the primary is ready to receive data, it must ask (poll) each device in turn if it has anything to send. When the first secondary is approached, it responds either with a NAK frame if it has nothing to send or with data (in the form of a data frame) if it does. If the response is negative (a NAK frame), then the primary polls the next secondary in the same manner until it finds one with data to send. When the response is positive (a data frame), the primary reads the frame and returns an acknowledgment (ACK frame) verifying its receipt.

Token Passing

In the **token-passing** method, the stations in a network are organized in a logical ring. In other words, for each station, there is a *predecessor* and a *successor*. The predecessor is the station that is logically before the station in the ring; the successor is the station that is after the station in the ring. The current station is the one that is accessing the

channel now. The right to this access has been passed from the predecessor to the current station. The right will be passed to the successor when the current station has no more data to send.

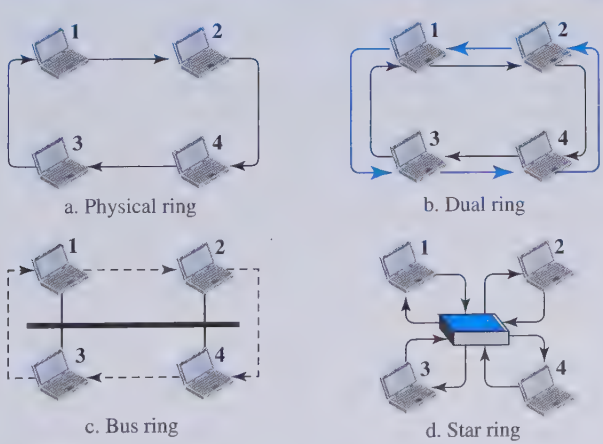
But how is the right to access the channel passed from one station to another? In this method, a special packet called a *token* circulates through the ring. The possession of the token gives the station the right to access the channel and send its data. When a station has some data to send, it waits until it receives the token from its predecessor. It then holds the token and sends its data. When the station has no more data to send, it releases the token, passing it to the next logical station in the ring. The station cannot send data until it receives the token again in the next round. In this process, when a station receives the token and has no data to send, it just passes the data to the next station.

Token management is needed for this access method. Stations must be limited in the time they can have possession of the token. The token must be monitored to ensure it has not been lost or destroyed. For example, if a station that is holding the token fails, the token will disappear from the network. Another function of token management is to assign priorities to the stations and to the types of data being transmitted. And finally, token management is needed to make low-priority stations release the token to high-priority stations.

Logical Ring

In a token-passing network, stations do not have to be physically connected in a ring; the ring can be a logical one. Figure 3.39 shows four different physical topologies that can create a logical ring.

Figure 3.39 *Logical ring and physical topology in token-passing access method*



In the physical ring topology, when a station sends the token to its successor, the token cannot be seen by other stations; the successor is the next one in line. This means that the token does not need to have the address of the next successor. The problem with

this topology is that if one of the links—the medium between two adjacent stations—fails, the whole system fails.

The dual-ring topology uses a second (auxiliary) ring that operates in the reverse direction compared with the main ring. The second ring is for emergencies only (such as a spare tire for a car). If one of the links in the main ring fails, the system automatically combines the two rings to form a temporary ring. After the failed link is restored, the auxiliary ring becomes idle again. Note that for this topology to work, each station needs to have two transmitter ports and two receiver ports. The high-speed Token Ring networks called FDDI (Fiber Distributed Data Interface) and CDDI (Copper Distributed Data Interface) use this topology.

In the bus ring topology, also called a token bus, the stations are connected to a single cable called a bus. They, however, make a logical ring, because each station knows the address of its successor (and also predecessor for token management purposes). When a station has finished sending its data, it releases the token and inserts the address of its successor in the token. Only the station with the address matching the destination address of the token gets the token to access the shared media. The Token Bus LAN, standardized by IEEE, uses this topology.

In a star ring topology, the physical topology is a star. There is a hub, however, that acts as the connector. The wiring inside the hub makes the ring; the stations are connected to this ring through the two wire connections. This topology makes the network less prone to failure because if a link goes down, it will be bypassed by the hub and the rest of the stations can operate. Also adding and removing stations from the ring is easier. This topology is still used in the Token Ring LAN designed by IBM.

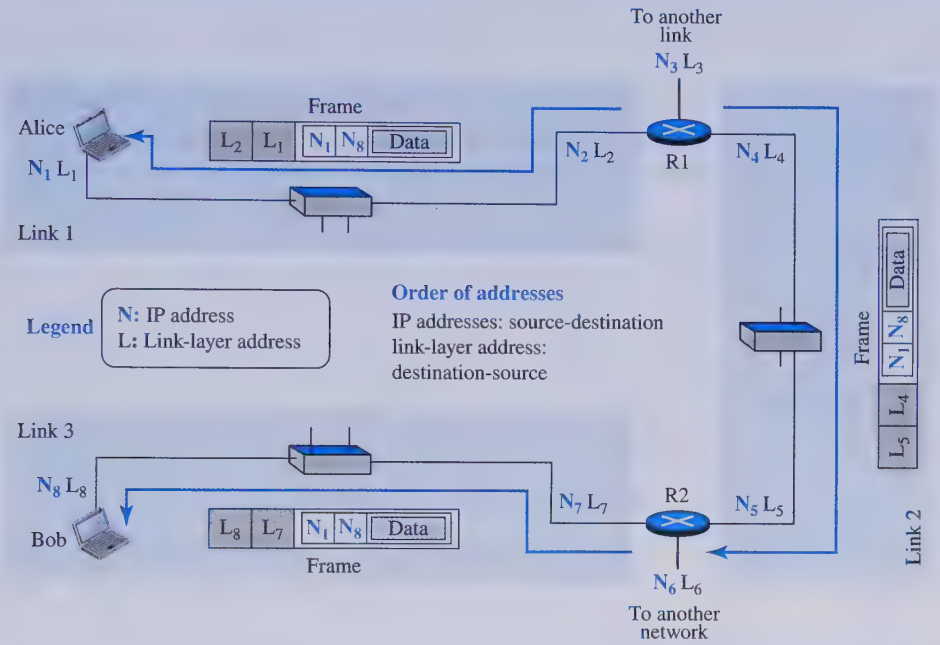
3.4 LINK-LAYER ADDRESSING

The next issue we need to discuss about the data-link layer is the link-layer addresses. In Chapter 7, we will discuss IP addresses as the identifiers at the network layer that define the exact points in the Internet where the source and destination hosts are connected. However, in a connectionless internetwork such as the Internet we cannot make a datagram reach its destination using only IP addresses. The reason is that each datagram in the Internet, from the same source host to the same destination host, may take a different path. The source and destination IP addresses define the two ends but cannot define which links the datagram should pass through.

The above discussion shows that we need another addressing mechanism in a connectionless internetwork: the link-layer addresses of the two nodes. A *link-layer address* is sometimes called a *link address*, sometimes a *physical address*, and sometimes a *MAC address*. We use these terms interchangeably in this book.

Because a link is controlled at the data-link layer, the addresses need to belong to the data-link layer. When a datagram passes from the network layer to the data-link layer, the datagram will be encapsulated in a frame and two data-link addresses are added to the frame header. These two addresses are changed every time the frame moves from one link to another. Figure 3.40 demonstrates the concept in a small internet.

Figure 3.40 *IP addresses and link-layer addresses in a small internet*



In Figure 3.40, we have three links and two routers. We also have shown only two hosts: Alice (source) and Bob (destination). For each host, we have shown two addresses, the IP addresses (N) and the link-layer addresses (L). Note that a router has as many pairs of addresses as the number of links the router is connected to. We have shown three frames, one in each link. Each frame carries the same datagram with the same source and destination addresses (N1 and N8), but the link-layer addresses of the frame change from link to link. In link 1, the link-layer addresses are L_1 and L_2. In link 2, they are L_4 and L_5. In link 3, they are L_7 and L_8. Note that the IP addresses and the link-layer addresses are not in the same order. For IP addresses, the source address comes before the destination address; for link-layer addresses, the destination address comes before the source. The datagrams and frames are designed in this way, and we follow the design. We may raise several questions:

- If the IP address of a router does not appear in any datagram sent from a source to a destination, why do we need to assign an IP address to the router? The answer is that in some protocols a router may act as a sender or receiver of a datagram. For example, in the routing protocols we will discuss in Chapter 8, a router is a sender or a receiver of a message. The communications in these protocols are between routers.
- Why do we need more than one IP address in a router? Do we need one for each interface? The answer is that an interface is a connection of a router to a link. We will see that an IP address defines a point in the Internet at which a device is connected.

A router with n interfaces is connected to the Internet at n points. This is like the situation where a house is at the corner of a street with two gates; each gate has the address related to the corresponding street.

- How are the source and destination IP addresses in a packet determined? The answer is that the host should know its own IP address, which becomes the source IP address in the packet. As we will discuss in Chapter 10, the application layer uses the services of the DNS to find the destination address of the packet and passes it to the network layer to be inserted in the packet.
- How are the source and destination link-layer addresses determined for each link? Again, each hop (router or host) should know its own link-layer address, as we discuss later in Section 3.4.1. The destination link-layer address is determined by using the Address Resolution Protocol, which we discuss in Section 3.4.2.
- What is the size of link-layer addresses? The answer is that it depends on the protocol used by the link. Although we have only one IP protocol for the whole Internet, we may be using different data-link protocols in different links. This means that we can define the size of the address when we discuss different link-layer protocols.

3.4.1 Three Types of Addresses

Some link-layer protocols define three types of addresses: unicast, multicast, and broadcast.

Unicast Address

Each host or each interface of a router is assigned a unicast address. Unicasting means one-to-one communication. A frame with a unicast address destination is destined only for one entity in the link.

Example 3.13

As we will see in Chapter 4, the unicast link-layer addresses in the most common LAN, the Ethernet, are 48 bits (6 bytes) that are presented as 12 hexadecimal digits separated by colons; for example, the following is a link-layer address of a computer.

A3:34:45:11:92:F1

Multicast Address

Some link-layer protocols define multicast addresses. Multicasting means one-to-many communication. However, the jurisdiction is local (inside the link).

Example 3.14

As we will see in Chapter 4, the multicast link-layer addresses in the most common LAN, the Ethernet, are 48 bits (6 bytes) that are presented as 12 hexadecimal digits separated by colons. The second digit, however needs to be an even number in hexadecimal. The following shows a multicast address:

A2:34:45:11:92:F1

Broadcast Address

Some link-layer protocols define a broadcast address. Broadcasting means one-to-all communication. A frame with a destination broadcast address is sent to all entities in the link.

Example 3.15

As we will see in Chapter 4, the broadcast link-layer addresses in the most common LAN, the Ethernet, are 48 bits that are all 1s and presented as 12 hexadecimal digits separated by colons. The following shows a broadcast address:

FF:FF:FF:FF:FF:FF

3.4.2 Address Resolution Protocol (ARP)

Any time a node has an IP datagram to send to another node in a link, it has the IP address of the receiving node. The source host knows the IP address of the default router. Each router except the last one in the path gets the IP address of the next router by using its forwarding table. The last router knows the IP address of the destination host. However, the IP address of the next node is not helpful in moving a frame through a link; we need the link-layer address of the next node. This is the time when the **Address Resolution Protocol (ARP)** becomes helpful. The ARP protocol is one of the auxiliary protocols defined in the network layer.

3.5 END-OF-CHAPTER MATERIALS

3.5.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and Requests for Comments (RFCs). The items in brackets refer to the reference list at the end of the text.

Books

Several excellent books discuss link-layer issues. Among them we recommend [Tan 03], [For 03], [KMK 04], and [Sta 04].

Requests for Comments

A discussion of the use of the checksum in the Internet can be found in RFC 1141.

3.5.2 Key Terms

1-persistant method
Address Resolution Protocol (ARP)
ALOHA
bit stuffing
burst error
byte stuffing

carrier sense multiple access (CSMA)
carrier sense multiple access with collision avoidance (CSMA/CA)
carrier sense multiple access with collision detection (CSMA/CD)
channelization
codeword
contention method
controlled access
cyclic redundancy check (CRC)
data-link control (DLC)
dataword
flag
Hamming distance
interference
Internet Protocol Control Protocol (IPCP)
links
Link Control Protocol (LCP)
media access control (MAC)
nodes
nonpersistent method
parity-check code
Point-to-Point Protocol (PPP)
p-persistent method
polling
pure ALOHA
random-access method
reservation method
single-bit error
slotted ALOHA
syndrome
token-passing method

3.5.3 Summary

We can consider the data-link layer as two sublayers. The upper sublayer is responsible for data-link control, and the lower sublayer is responsible for resolving access to the shared media. Data-link control (DLC) deals with the design and procedures for communication between two adjacent nodes: node-to-node communication. This sublayer is responsible for framing and error control. Error control deals with data corruption during transmission. We discussed two link-layer protocols in this chapter: HDLC and PPP. High-level Data Link Control (HDLC) is a bit-oriented protocol for communication over point-to-point and multipoint links. However, the most common protocol for point-to-point access is the Point-to-Point Protocol (PPP), which is a byte-oriented protocol.

Many formal protocols have been devised to handle access to a shared link. We categorize them into three groups: random-access protocols, controlled-access protocols, and channelization protocols. In random-access or contention methods, no station is superior to another station and none is assigned the control over another. In controlled access, the stations consult one another to find which station has the right to send. Channelization is a multiple-access method in which the available bandwidth of a link is shared in time, frequency, or through code, between different stations.

At the data-link layer, we use link-layer addressing. The system normally finds the link-layer address of the next node using the Address Resolution Protocol.

3.6 PRACTICE SET

3.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

3.6.2 Questions

Q3-1. Distinguish between communication at the physical layer and communication at the data-link layer.

Q3-2. Distinguish between a point-to-point link and a broadcast link.

Q3-3. Explain why flags are needed when we use variable-size frames.

Q3-4. Explain why we cannot bit-stuff in character-oriented framing to change a flag byte appearing in the text.

Q3-5. How does a single-bit error differ from a burst error?

Q3-6. What is the definition of a linear block code?

Q3-7. In a block code, a dataword is 20 bits and the corresponding codeword is 25 bits. What are the values of k, r, and n according to the definitions in the text? How many redundant bits are added to each dataword?

Q3-8. In a codeword, we add two redundant bits to each 8-bit data word. Find the following:

a. number of valid codewords **b.** number of invalid codewords

Q3-9. What is the minimum Hamming distance?

Q3-10. If we want to be able to detect 2-bit errors, what should be the minimum Hamming distance?

Q3-11. A category of error-detecting (and correcting) code, called the Hamming code, is a code in which $\boldsymbol{d}_{min} = 3$. This code can detect up to two errors (or correct one single error). In this code, the values of n, k, and r are related as: $n = 2^r - 1$ and $k = n - r$. Find the number of bits in the dataword and the codewords if r is 3.

Q3-12. In CRC, if the dataword is 5 bits and the codeword is 8 bits, how many 0s need to be added to the dataword to make the dividend? What is the size of the remainder? What is the size of the divisor?

Q3-13. In CRC, which of the following generators (divisors) guarantees the detection of a single bit error?

a. 101 **b.** 100 **c.** 1

Q3-14. In CRC, which of the following generators (divisors) guarantees the detection of an odd number of errors?

a. 10111 **b.** 101101 **c.** 111

Q3-15. In CRC, we have chosen the generator 1100101. What is the probability of detecting a burst error of each of the following lengths?

a. 5 **b.** 7 **c.** 10

Q3-16. Assume we are sending data items of 16-bit length. If two data items are swapped during transmission, can the traditional checksum detect this error? Explain.

Q3-17. Can the value of a traditional checksum be all 0s (in binary)? Defend your answer.

Q3-18. Explain why there is only one address field (instead of two) in an HDLC frame.

Q3-19. Which of the following is a random-access protocol?

a. CSMA/CD **b.** Polling **c.** TDMA

Q3-20. Stations in a pure Aloha network send frames of size 1000 bits at the rate of 1 Mbps. What is the vulnerable time for this network?

Q3-21. In a pure Aloha network with $G = 1/2$, how is the throughput affected in each of the following cases?
a. G is increased to 1. **b.** G is decreased to 1/4.

Q3-22. Assume the propagation delay in a broadcast network is 5 μs and the frame transmission time is 10 μs.
a. How long does it take for the first bit to reach the destination?
b. How long does it take for the last bit to reach the destination after the first bit has arrived?
c. How long is the network involved with this frame (vulnerable to collision)?

Q3-23. Assume the propagation delay in a broadcast network is 3 μs and the frame transmission time is 5 μs. Can the collision be detected no matter where it occurs?

Q3-24. Can two hosts in two different networks have the same link-layer address?

Q3-25. Explain why collision is an issue in random-access protocols but not in controlled access or channelizing protocols.

3.6.3 Problems

P3-1. Byte-stuff the following frame payload in which E is the escape byte, F is the flag byte, and D is a data byte other than an escape or a flag character.

D	E	D	D	F	D	D	E	E	D	F	D

P3-2. Bit-stuff the following frame payload:

000111111100111110100011111111110000111

P3-3. What is the maximum effect of a 2-ms burst of noise on data transmitted at the following rates?
a. 1500 bps **b.** 12 kbps **c.** 100 kbps **d.** 100 Mbps

P3-4. Exclusive-OR (XOR) is one of the most used operations in the calculation of codewords. Apply the exclusive-OR operation on the following pair of patterns. Interpret the results.
a. (10001) ⊕ (10001) **b.** (11100) ⊕ (00000) **c.** (10011) ⊕ (11111)

P3-5. Prove that the code represented by the following codewords is not linear. You need to find only one case that violates the linearity.

{(00000), (01011), (10111), (11111)}

P3-6. What is the Hamming distance for each of the following codewords?
a. d (10000, 00000) **c.** d (00000, 11111)
b. d (10101, 10000) **d.** d (00000, 00000)

P3-7. Although it can be formally proved that the code in Table 3.3 is both linear and cyclic, use only two tests to partially prove the fact:
a. Test the cyclic property on codeword 0101100.
b. Test the linear property on codewords 0010110 and 1111111.

P3-8. Referring to the CRC-8 in Table 3.4, answer the following questions:
- **a.** Does it detect a single error? Defend your answer.
- **b.** Does it detect a burst error of size 6? Defend your answer.
- **c.** What is the probability of detecting a burst error of size 9?
- **d.** What is the probability of detecting a burst error of size 15?

P3-9. Assuming even parity, find the parity bit for each of the following data units.
a. 1001011 **b.** 0001100 **c.** 1000000 **d.** 1110111

P3-10. Given the dataword 101001111 and the divisor 10111, show the generation of the CRC codeword at the sender site (using binary division).

P3-11. To formulate the performance of a multiple-access network, we need a mathematical model. When the number of stations in a network is very large, the Poisson distribution, $p[x] = (e^{-\lambda} \times \lambda^x)/(x!)$, is used. In this formula, $p[x]$ is the probability of generating x number of frames in a period of time and λ is the average number of generated frames during the same period of time. Using the Poisson distribution:
- **a.** Find the probability that a pure Aloha network generates x number of frames during the vulnerable time. Note that the vulnerable time for this network is 2 times the frame transmission time (T_{fr}).
- **b.** Find the probability that a slotted Aloha network generates x number of frames during the vulnerable time. Note that the vulnerable time for this network is equal to the frame transmission time (T_{fr}).

P3-12. A multiple-access network with a large number of stations can be analyzed using the Poisson distribution. When there is a limited number of stations in a network, we need to use another approach for this analysis. In a network with N stations, we assume that each station has a frame to send during the frame transmission time (T_{fr}) with probability p. In such a network, a station is successful in sending its frame if the station has a frame to send during the vulnerable time and no other station has a frame to send during this period of time.
- **a.** Find the probability that a station in a pure Aloha network can successfully send a frame during the vulnerable time.
- **b.** Find the probability that a station in a slotted Aloha network can successfully send a frame during the vulnerable time.

P3-13. There are only three active stations in a slotted Aloha network: A, B, and C. Each station generates a frame in a time slot with the corresponding probabilities $p_A = 0.2$, $p_B = 0.3$, and $p_C = 0.4$, respectively.
- **a.** What is the throughput of each station?
- **b.** What is the throughput of the network?

P3-14. There are only three active stations in a slotted Aloha network: A, B, and C. Each station generates a frame in a time slot with the corresponding probabilities $p_A = 0.2$, $p_B = 0.3$, and $p_C = 0.4$, respectively.
- **a.** What is the probability that any station can send a frame in the first slot?
- **b.** What is the probability that station A can successfully send a frame for the first time in the second slot?
- **c.** What is the probability that station C can successfully send a frame for the first time in the third slot?

P3-15. A slotted Aloha network is working with maximum throughput.
- **a.** What is the probability that a slot is empty?
- **b.** How many slots, n, on average, should pass before getting an empty slot?

P3-16. One of the useful parameters in a LAN is the number of bits that can fit in 1 meter of the medium ($n_{b/m}$). Find the value of $n_{b/m}$ if the data rate is 100 Mbps and the medium propagation speed is 2×10^8 m/s.

P3-17. Another useful parameter in a LAN is the bit length of the medium (L_b), which defines the number of bits that the medium can hold at any time. Find the bit length of a LAN if the data rate is 100 Mbps and the medium length in meters (L_m) for a communication between two stations is 200 m. Assume the propagation speed in the medium is 2×10^8 m/s.

P3-18. We have defined the parameter a as the number of frames that can fit the medium between two stations as $a = (T_p)/(T_{fr})$. Another way to define this parameter is $a = L_b/F_b$, in which L_b is the bit length of the medium and F_b is the frame length of the medium. Show that the two definitions are equivalent.

P3-19. In a bus CSMD network with a data rate of 10 Mbps, a collision occurs 20 μs after the first bit of the frame leaves the sending station. What should the length of the frame be so that the sender can detect the collision?

P3-20. Assume that there are only two stations, A and B, in a bus CSMA/CD network. The distance between the two stations is 2000 m and the propagation speed is 2×10^8 m/s. If station A starts transmitting at time t_1:
- **a.** Does the protocol allow station B to start transmitting at time $t_1 + 8$ μs? If the answer is yes, what will happen?
- **b.** Does the protocol allow station B to start transmitting at time $t_1 + 11$ μs? If the answer is yes, what will happen?

P3-21. There are only two stations, A and B, in a bus 1-persistence CSMA/CD network with $T_p = 25.6$ μs and $T_{fr} = 51.2$ μs. Station A has a frame to send to station B. The frame is unsuccessful 2 times and succeeds on the third try. Draw a time-line diagram for this problem. Assume that the R is 1 and 2, respectively, and ignore the time for sending a jamming signal.

P3-22. To understand why we need to have a minimum frame size $T_{fr} = 2 \times T_p$ in a CDMA/CD network, assume we have a bus network with only two stations, A and B, in which $T_{fr} = 40$ μs and $T_p = 25$ μs. Station A starts sending a frame at time $t = 0.0$ μs, and station B starts sending a frame at $t = 23.0$ μs. Answer the following questions:
- **a.** Do frames collide?
- **b.** If the answer to part a is yes, does station A detect collision?
- **c.** If the answer to part a is yes, does station B detect collision?

P3-23. In a bus 1-persistence CSMA/CD with $T_p = 50$ μs and $T_{fr} = 120$ μs, there are two stations A and B. Both stations start sending frames to each other at the same time. Because the frames collide, each station tries to retransmit. Station A comes out with $R = 0$ and station B with $R = 1$. Ignore any other delay

including the delay for sending jamming signals. Do the frames collide again? Draw a time-line diagram to prove your claim. Does the generation of a random number help avoid collision in this case?

P3-24. We have a pure ALOHA network with a data rate of 10 Mbps. What is the maximum number of 1000-bit frames that can be successfully sent by this network?

P3-25. In a CDMA/CD network with a data rate of 10 Mbps, the minimum frame size is found to be 512 bits for the correct operation of the collision detection process. What should the minimum frame size be if we keep the size of the network constant, but we increase the data rate to each of the following?

a. 100 Mbps **b.** 1 Gbps **c.** 10 Gbps

CHAPTER 4

Local Area Networks: LANs

After discussing the physical and data-link layers in Chapters 2 and 3, it is time to discuss the networks that use these two layers. The literature refers to those networks with limited jurisdiction as LANs (local area networks) and those with broad jurisdiction as WANs (wide area networks). We discuss LANs in this chapter and WANs in Chapter 5.

LANs can be further divided into two broad categories: wired LANs and wireless LANs. During the last two decades, several wired-LAN technologies and several wireless LAN technologies appeared in the market, but only a few survived. We discuss only one wired LAN, Ethernet. We discuss two wireless LANs, WiFi and Bluetooth.

This chapter is divided into three sections.

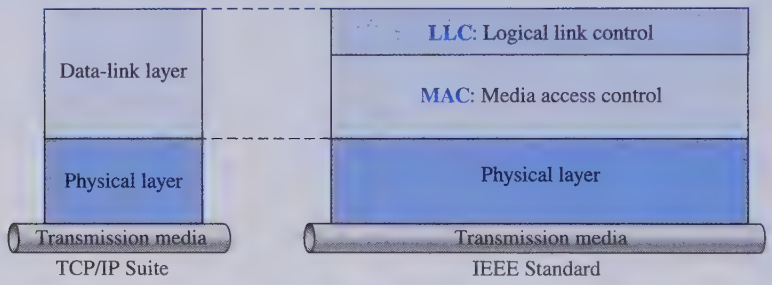
- The first section discusses the Ethernet protocol, a wired LAN. It gives the ideas behind the technology and explains all four versions of Ethernet: Standard Ethernet (with 10 Mbps), Fast Ethernet (with 100 Mbps), Gigabit Ethernet (with 1000 Mbps), and 10 Gigabit Ethernet with (10,000 Mbps).
- The second section discusses a wireless LAN called *WiFi* (Wireless Fidelity), based on the IEEE 802.11 standard, that is designed to provide services to wireless devices.
- The third section discusses another wireless LAN called *Bluetooth* that is used to provide services to different types of devices.

4.1 ETHERNET

In Chapter 1, we learned that a local area network (LAN) is a computer network that is designed for a limited geographic area such as a building or campus. Although a LAN can be used as an isolated network to connect computers in an organization for the sole purpose of sharing resources, most LANs today are also linked to a wide area network (WAN) or the Internet.

In the 1980s and 1990s, several different types of wired LANs were used. The Institute of Electrical and Electronics Engineers (IEEE) has subdivided the data-link layer into two sublayers: **logical link control (LLC)** and **media access control (MAC)**. Note that logical link control is similar to data-link control (DLC), which we discussed in Chapter 3. The IEEE has also created several physical-layer standards for different LAN protocols. All these wired LANs use a media access method to solve the problem of sharing the media. The relationship of the IEEE 802 standard to the TCP/IP protocol suite is shown in Figure 4.1.

Figure 4.1 *IEEE standard for wired LANs*

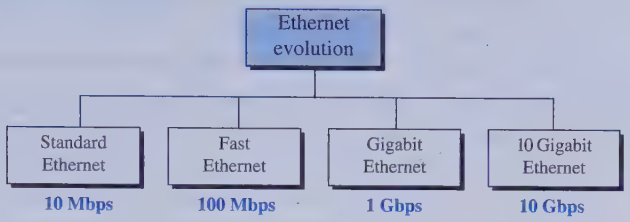


Almost every wired LAN technology except Ethernet has disappeared from the marketplace because Ethernet was able to update itself to meet the needs of the time. This means that we confine our discussion of wired LANs to the discussion of Ethernet.

Before we discuss the Ethernet protocol and all its generations, we need to briefly discuss the IEEE standard that we often encounter in text or real life. In 1985, the Computer Society of the IEEE started a project, called **Project 802**, to set standards to enable intercommunication among equipment from a variety of manufacturers. Project 802 does not seek to replace any part of the OSI model or TCP/IP protocol suite. Instead, it is a way of specifying functions of the physical layer and the data-link layer of major LAN protocols.

The Ethernet LAN was developed in the 1970s by Robert Metcalfe and David Boggs. Since then, it has gone through four generations: **Standard Ethernet** (10 Mbps), **Fast Ethernet** (100 Mbps), **Gigabit Ethernet** (1 Gbps), and **10 Gigabit Ethernet** (10 Gbps), as shown in Figure 4.2. We briefly discuss all these generations.

Figure 4.2 *Ethernet evolution through four generations*



4.1.1 Standard Ethernet (10 Mbps)

We refer to the original Ethernet technology with the data rate of 10 Mbps as the Standard Ethernet. Although most implementations have moved to other technologies in the Ethernet evolution, there are some features of the Standard Ethernet that have not changed during the evolution. We discuss this standard version to pave the way for understanding the other three technologies.

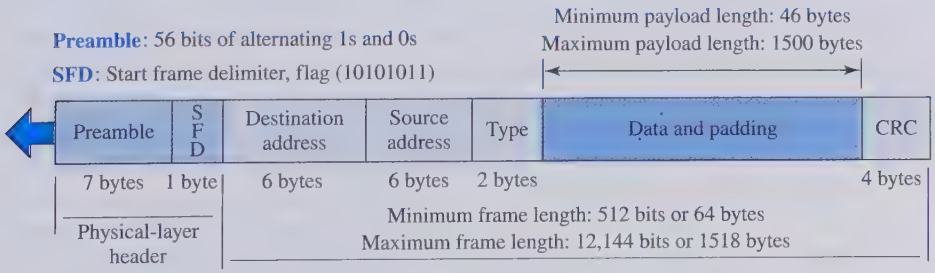
Connectionless and Unreliable Service

Ethernet provides a connectionless service, which means each frame sent is independent of the previous or next frame. Ethernet has no connection establishment or connection termination phases. The sender sends a frame whenever it has it; the receiver may or may not be ready for it. The sender may overwhelm the receiver with frames, which may result in dropped frames. If a frame drops, the sender data-link layer will not know about it unless an upper-layer protocol takes care of it. Ethernet is also unreliable. If a frame is corrupted during transmission and the receiver finds out about the corruption, the receiver drops the frame silently. It is the duty of high-level protocols to find out about it.

Frame Format

The Ethernet frame contains seven fields, as shown in Figure 4.3.

Figure 4.3 *Ethernet frame*



- **Preamble.** This field contains 7 bytes (56 bits) of alternating 0s and 1s that alert the receiving system to the coming frame and enable it to synchronize its clock if it's out of synchronization. The pattern provides only an alert and a timing pulse. The 56-bit pattern allows the stations to miss some bits at the beginning of the frame. The *preamble* is actually added at the physical layer and is not (formally) part of the frame.
- **Start frame delimiter (SFD).** This field (1 byte: 10101011) signals the beginning of the frame. The SFD warns the station or stations that this is the last chance for synchronization. The last 2 bits are $(11)_2$ and alert the receiver that the next field is the destination address. This field is actually a flag that defines the beginning of the frame. We need to remember that an Ethernet frame is a variable-length frame. It needs a flag to define the beginning of the frame. The SFD field is also added at the physical layer.
- **Destination address (DA).** This field is 6 bytes (48 bits) and contains the link-layer address of the destination station or stations to receive the packet. We will discuss addressing shortly. When the receiver sees its own link-layer address, a multicast address for a group that the receiver is a member of, or a broadcast address, it decapsulates the data from the frame and passes the data to the upper-layer protocol defined by the value of the type field.
- **Source address (SA).** This field is also 6 bytes and contains the link-layer address of the sender of the packet.
- **Type.** This field defines the upper-layer protocol whose packet is encapsulated in the frame. This protocol can be IP, ARP, Open Shortest Path First (OSPF), and so on as we will see in the next chapters.
- **Data.** This field carries data encapsulated from the upper-layer protocols. It is a minimum of 46 and a maximum of 1500 bytes. If the data coming from the upper layer is more than 1500 bytes, it should be fragmented and encapsulated in more than one frame. If it is less than 46 bytes, it needs to be padded with extra 0s. A padded data frame is delivered to the upper-layer protocol as it is (without removing the padding), which means that it is the responsibility of the upper layer to remove or add the padding. The upper-layer protocol needs to know the length of its data.
- **CRC.** The last field contains error-detection information, in this case a CRC-32. The CRC is calculated over the addresses, types, and data field. If the receiver calculates the CRC and finds that it is not zero (corruption in transmission), it discards the frame.

Frame Length

Ethernet imposes restrictions on both the minimum and maximum lengths of a frame. The minimum length restriction is required for the correct operation of CSMA/CD, as we discussed in Chapter 3. An Ethernet frame needs to have a minimum length of 512 bits or 64 bytes. Part of this length is the header and the trailer. If we count 18 bytes of header and trailer (6 bytes of source address, 6 bytes of destination address, 2 bytes of length or type, and 4 bytes of CRC), then the minimum length of data from the upper layer is $64 - 18 = 46$ bytes. If the upper-layer packet is less than 46 bytes, padding is added to make up the difference.

The standard defines the maximum length of a frame (without preamble and SFD field) as 1518 bytes. If we subtract the 18 bytes of header and trailer, the maximum

length of the payload is 1500 bytes. There are two historical reasons for the maximum length restriction. First, memory was very expensive when Ethernet was designed; a maximum length restriction helped to reduce the size of the buffer. Second, the maximum length restriction prevents one station from monopolizing the shared medium, blocking other stations that have data to send.

Minimum frame length: 64 bytes	**Minimum data length: 46 bytes**
Maximum frame length: 1518 bytes	**Maximum data length: 1500 bytes**

Addressing

Each station on an Ethernet network (such as a PC, workstation, or printer) has its own **network interface card (NIC)**. The NIC fits inside the station and provides the station with a link-layer address. The Ethernet address is 6 bytes (48 bits), normally written in **hexadecimal notation**, with a colon between the bytes. For example, the following shows an Ethernet MAC address:

47:20:1B:2E:08:EE

Transmission of Address Bits

The way the addresses are sent out online is different from the way they are written in hexadecimal notation. The transmission is left to right, byte by byte; however, for each byte, the least significant bit is sent first and the most significant bit is sent last. This means that the bit that defines an address as unicast or multicast arrives first at the receiver. This helps the receiver to immediately know if the packet is unicast or multicast.

Example 4.1

This example shows how the address 47:20:1B:2E:08:EE is sent out online.

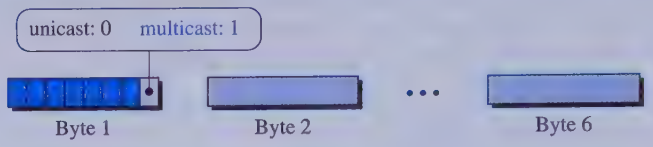
Hexadecimal	47	20	1B	2E	08	EE
Binary	01000111	00100000	00011011	00101110	00001000	11101110
Transmitted ←	11100010	00000100	11011000	01110100	00010000	01110111

Unicast, Multicast, and Broadcast Addresses

A source address is always a *unicast address*—the frame comes from only one station. The destination address, however, can be *unicast*, *multicast*, or *broadcast*. Figure 4.4 shows how to distinguish a unicast address from a multicast address. If the least significant bit of the first byte in a destination address is 0, the address is *unicast*; otherwise, it is *multicast*. Note that with the way the bits are transmitted, the unicast/multicast bit is the first bit that is transmitted or received.

The *broadcast* address is a special case of the multicast address: The recipients are all the stations on the LAN. A broadcast destination address is forty-eight 1s.

Figure 4.4 *Unicast and multicast addresses*



Example 4.2

Define the type of the following destination addresses:

a. **4A:30:10:21:10:1A**
b. **47:20:1B:2E:08:EE**
c. **FF:FF:FF:FF:FF:FF**

Solution

To find the type of the address, we need to look at the second hexadecimal digit from the left. If it is even, the address is unicast. If it is odd, the address is multicast. If all digits are Fs, the address is broadcast. Therefore, we have the following:

a. This is a unicast address because A in binary is 1010 (even).
b. This is a multicast address because 7 in binary is 0111 (odd).
c. This is a broadcast address because all digits are Fs in hexadecimal.

Implementation

The Standard Ethernet defined several implementations, but only four of them became popular during the 1980s. Table 4.1 shows a summary of Standard Ethernet implementations.

Table 4.1 *Summary of Standard Ethernet implementations*

Implementation	*Medium*	*Medium Length(m)*	*Encoding*
10Base5	Thick coax	500	Line coding
10Base2	Thin coax	185	Line coding
10Base-T	2 UTP	100	Line coding
10Base-F	2 Fiber	2000	Line coding

In the nomenclature 10BaseX, the number defines the data rate (10 Mbps), the term *Base* means baseband (digital) signal, and X approximately defines either the maximum size of the cable in 100 m (for example, 5 for 500 or 2 for 185 m) or the type of the cable [T for unshielded twisted-pair (UTP) cable and F for fiber-optic cable]. The Standard Ethernet uses a baseband signal, which means that the bits are changed to a digital signal and directly sent on the line.

Encoding and Decoding

All standard implementations use digital signaling (baseband) at 10 Mbps. At the sender, data are converted to a digital signal using the line coding scheme; at the receiver, the received signal is interpreted as coded and decoded into data.

4.1.2 Fast Ethernet (100 Mbps)

In the 1990s, some LAN technologies with transmission rates higher than 10 Mbps such as Fiber Distributed Data Interface (FDDI) and Fibre Channel, appeared on the market. If the Standard Ethernet wanted to survive, it had to compete with these technologies. Ethernet made a big jump by increasing the transmission rate to 100 Mbps, and the new generation was called the Fast Ethernet. The designers of the Fast Ethernet needed to make it compatible with the Standard Ethernet. The MAC sublayer was left unchanged, which meant the frame format and the maximum and minimum size could also remain unchanged. By increasing the transmission rate, features of the Standard Ethernet that depend on the transmission rate, access method and implementation, had to be reconsidered. The goals of Fast Ethernet can be summarized as follows:

1. Upgrade the data rate to 100 Mbps.
2. Make it compatible with Standard Ethernet.
3. Keep the same 48-bit address.
4. Keep the same frame format.

Access Method

We remember that the proper operation of the CSMA/CD depends on the transmission rate, the minimum size of the frame, and the maximum network length. If we want to keep the minimum size of the frame, the maximum length of the network should be changed. In other words, if the minimum frame size is still 512 bits, and it is transmitted 10 times faster, the collision needs to be detected 10 times sooner, which means the maximum length of the network should be 10 times shorter (the propagation speed does not change). So the Fast Ethernet came with two solutions (it can work with either choice):

1. The first solution was to totally drop the bus topology and use a passive hub and star topology but make the maximum size of the network 250 m instead of 2500 m as in the Standard Ethernet. This approach is kept for compatibility with the Standard Ethernet.
2. The second solution is to use a **switch** with buffer to store frames and full-duplex connection to each host to make the transmission medium private for each host. In this case, there is no need for CSMA/CD because the hosts are not competing with each other. The link-layer switch receives a frame from a source host and stores it in the buffer (queue) waiting for processing. It then checks the destination address and sends the frame out of the corresponding interface. Because the connection to the switch is full-duplex, the destination address can even send a frame to another station at the same time that it is receiving a frame. In other words, the shared medium is changed to many point-to-point media, and there is no need for contention.

Autonegotiation

A new feature that was added to Fast Ethernet is called **autonegotiation**. It allows a station or a hub a range of capabilities. Autonegotiation allows two devices to negotiate the mode or data rate of operation. It was designed particularly to allow incompatible devices to connect to one another. For example, a device with a maximum data rate of

10 Mbps can communicate with a device with a 100-Mbps data rate (but which can work at a lower rate). Autonegotiation was designed particularly for the following purposes:

- To allow incompatible devices to connect to one another. For example, a device with a maximum capacity of 10 Mbps can communicate with a device with a 100-Mbps capacity (but can work at a lower rate).
- To allow one device to have multiple capabilities.
- To allow a station to check a hub's capabilities.

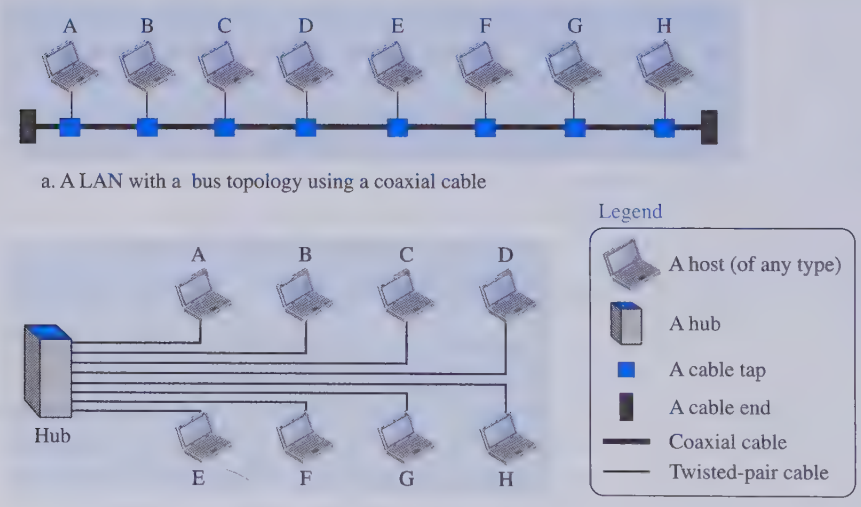
Physical Layer

To be able to handle a 100-Mbps data rate, several changes need to be made at the physical layer.

Topology

Fast Ethernet is designed to connect two or more stations together. If there are only two stations, they can be connected point-to-point. Three or more stations need to be connected in a star topology with a hub or a switch at the center (see Figure 4.5).

Figure 4.5 *Topology of Fast Ethernet*



a. A LAN with a bus topology using a coaxial cable

b. A LAN with a star topology using a hub

Encoding

Fast Ethernet uses an encoding system called Manchester with 200-Mbaud bandwidth, which makes it unsuitable for a medium such as twisted-pair cable. For this reason, the Fast Ethernet designers sought some alternative encoding/decoding scheme. However, it was found that no scheme would perform equally well for all three implementations. Therefore, three different encoding schemes were chosen (see Figure 4.5): 100Base-TX, 100base-FX, and 100base-T4.

100Base-TX uses two pairs of twisted-pair cable (either category 5 UTP or STP). For this implementation, this scheme was selected because it has good bandwidth performance. However, because this scheme is not a self-synchronous line coding scheme, 4B/5B block coding is used to provide bit synchronization by preventing the occurrence of a long sequence of 0s and 1s. This creates a data rate of 125 Mbps, which is fed into this scheme for encoding.

100Base-FX uses two pairs of fiber-optic cables. Optical fiber can easily handle high bandwidth requirements by using simple encoding schemes.

Summary

Fast Ethernet implementation at the physical layer can be categorized as either two-wire or four-wire. The two-wire implementation can be either shielded twisted-pair (STP), which is called 100Base-TX, or fiber-optic cable, which is called 100Base-FX. The four-wire implementation is designed only for unshielded twisted-pair (UTP) cable, which is called 100Base-T4. Table 4.2 is a summary of the Fast Ethernet implementations.

Table 4.2 *Summary of Fast Ethernet implementations*

Implementation	*Medium*	*Medium Length(m)*	*Wires*	*Encoding*
100Base-TX	STP	100	2	4B/5B + MLT-3
100Base-FX	Fiber	185	2	4B/5B + NRZ-I
100Base-T4	UTP	100	4	Two 8B/6T

4.1.3 Gigabit Ethernet (1000 Mbps)

The need for an even higher data rate resulted in the design of the Gigabit Ethernet Protocol (1000 Mbps). The IEEE committee calls the Standard 802.3z. The goals of the Gigabit Ethernet were to upgrade the data rate to 1 Gbps, but keep the address length, the frame format, and the maximum and minimum frame lengths the same. The goals of the Gigabit Ethernet design can be summarized as follows:

1. Upgrade the data rate to 1 Gbps.
2. Make it compatible with Standard or Fast Ethernet.
3. Use the same 48-bit address.
4. Use the same frame format.
5. Keep the same minimum and maximum frame lengths.
6. Support autonegotiation as defined in Fast Ethernet.

MAC Sublayer

A main consideration in the evolution of Ethernet was to keep the MAC sublayer untouched. However, to achieve a data rate of 1 Gbps, this was no longer possible. Gigabit Ethernet has two distinctive approaches for medium access: half-duplex and full-duplex. Almost all implementations of Gigabit Ethernet follow the full-duplex

approach, so the half-duplex mode is not often used. In full-duplex mode, there is a central switch connected to all computers or other switches. In this mode, for each input port, each switch has buffers in which data are stored until they are transmitted. Because the switch uses the destination address of the frame and sends a frame out of the port connected to that particular destination, there is no collision. This means that CSMA/CD is not used. Lack of collision implies that the maximum length of the cable is determined by the signal attenuation in the cable, not by the collision detection process.

Full-Duplex Mode

In full-duplex mode, there is a central switch connected to all computers or other switches. In this mode, each switch has buffers for each input port in which data are stored until they are transmitted. There is no collision in this mode, as we discussed before. This means that CSMA/CD is not used. Lack of collision implies that the maximum length of the cable is determined by the signal attenuation in the cable, not by the collision detection process.

In the full-duplex mode of Gigabit Ethernet, there is no collision; the maximum length of the cable is determined by the signal attenuation in the cable.

Half-Duplex Mode

Gigabit Ethernet can also be used in half-duplex mode, although it is rare. In this case, a switch can be replaced by a hub, which acts as the common cable in which a collision might occur. The half-duplex approach uses CSMA/CD. However, as we saw before, the maximum length of the network in this approach is totally dependent on the minimum frame size. Three methods have been defined: traditional, carrier extension, and frame bursting.

Traditional

In the traditional approach, we keep the minimum length of the frame as in traditional Ethernet (512 bits). However, because the length of a bit is 1/100 shorter in Gigabit Ethernet than in 10-Mbps Ethernet, the slot time for Gigabit Ethernet is 512 bits × 1/1000 μs, which is equal to 0.512 μs. The reduced slot time means that collision is detected 100 times earlier. This means that the maximum length of the network is 25 m. This length may be suitable if all the stations are in one room, but it may not even be long enough to connect the computers in one single office.

Carrier Extension

To allow for a longer network, we increase the minimum frame length. The **carrier extension** approach defines the minimum length of a frame as 512 bytes (4096 bits). This means that the minimum length is 8 times longer. This method forces a station to add extension bits (padding) to any frame that is less than 4096 bits. In this way, the maximum length of the network can be increased 8 times to a length of 200 m. This allows a length of 100 m from the hub to the station.

Frame Bursting

Carrier extension is very inefficient if we have a series of short frames to send; each frame carries redundant data. To improve efficiency, **frame bursting** was proposed. Instead of adding an extension to each frame, multiple frames are sent. However, to make these multiple frames look like one frame, padding is added between the frames (the same as that used for the carrier extension method) so that the channel is not idle. In other words, the method deceives other stations into thinking that a very large frame has been transmitted.

Physical Layer

The physical layer in Gigabit Ethernet is more complicated than that in Standard or Fast Ethernet. We briefly discuss some features of this layer.

Topology

Gigabit Ethernet is designed to connect two or more stations. If there are only two stations, they can be connected point-to-point. Three or more stations need to be connected in a star topology with a hub or a switch at the center. Another possible configuration is to connect several star topologies or let a star topology be part of another.

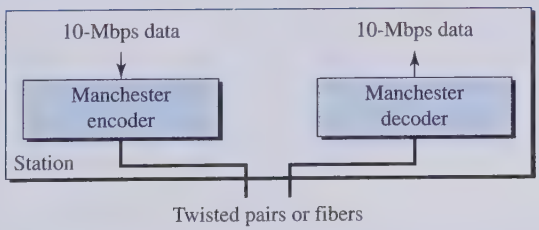
Implementation

Gigabit Ethernet can be categorized as either a two-wire or a four-wire implementation. The two-wire implementations use fiber-optic cable [**1000Base-SX** (short-wave,) or **1000Base-LX** (long-wave)] or STP (**1000Base-CX**). The four-wire version uses category 5 twisted-pair cable (**1000Base-T**). In other words, we have four implementations. 1000Base-T was designed in response to those users who had already installed this wiring for other purposes such as Fast Ethernet or telephone services.

Encoding

Figure 4.6 shows the encoding/decoding schemes for the four implementations. Gigabit Ethernet cannot use the Manchester encoding scheme because it involves a very high bandwidth (2 Gbaud). The two-wire implementations use an NRZ scheme, but NRZ does not self-synchronize properly. To synchronize bits, particularly at this high data rate, other encoding system such as 8B/10B is used which is beyond the scope of this book.

Figure 4.6 *Encoding in Gigabit Ethernet implementations*



Implementation Summary

Table 4.3 is a summary of the Gigabit Ethernet implementations. S-W and L-W mean short wave and long wave, respectively.

Table 4.3 *Summary of Gigabit Ethernet implementations*

Implementation	*Medium*	*Medium Length(m)*	*Wires*	*Encoding*
1000Base-SX	Fiber S-W	550	2	8B/10B + NRZ
1000Base-LX	Fiber L-W	5000	2	8B/10B + NRZ
1000Base-CX	STP	25	2	8B/10B + NRZ
1000Base-T4	UTP	100	4	4D-PAM5

4.1.4 10 Gigabit Ethernet

In recent years, there has been another look at Ethernet for use in metropolitan areas. The idea is to extend the technology, the data rate, and the coverage distance so that Ethernet can be used as a LAN and a metropolitan area network(MAN). The IEEE committee created 10 Gigabit Ethernet and called it Standard 802.3ae. The goals of the 10 Gigabit Ethernet design can be summarized as upgrading the data rate to 10 Gbps; keeping the same frame size and format; and allowing the interconnection of LANs, MANs, and WANs possible. This data rate is possible only with fiber-optic technology at this time. The standard defines two types of physical layers: LAN PHY and WAN PHY. The first is designed to support existing LANs; the second actually defines a WAN with links connected through SONET OC-192 (discussed in Chapter 5).

Implementation

10 Gigabit Ethernet operates only in full-duplex mode, which means there is no need for contention; CSMA/CD is not used in 10 Gigabit Ethernet. Four implementations are the most common: **10GBase-SR**, **10GBase-LR**, **10GBase-EW**, and **10GBase-X4**. Table 4.4 shows a summary of these 10-Gigabit Ethernet implementations.

Table 4.4 *Summary of 10 Gigabit Ethernet implementations*

Implementation	*Medium*	*Medium Length*	*Number of wires*	*Encoding*
10GBase-SR	Fiber 850 nm	300 m	2	64B66B
10GBase-LR	Fiber 1310 nm	10 km	2	64B66B
10GBase-EW	Fiber 1350 nm	40 km	2	SONET
10GBase-X4	Fiber 1310 nm	300 m to 10 km	2	8B10B

4.2 WIFI, IEEE 802.11 PROJECT

IEEE has defined the specifications for a wireless LAN, called IEEE 802.11, which covers the physical and data-link layers. It is sometimes called *wireless Ethernet*. In some countries, including the United States, the public uses the term *WiFi* (short for

wireless fidelity) as a synonym for *wireless LAN*. WiFi, however, is a wireless LAN that is certified by the WiFi Alliance, a global, nonprofit industry association of more than 300 member companies devoted to promoting the growth of wireless LANs.

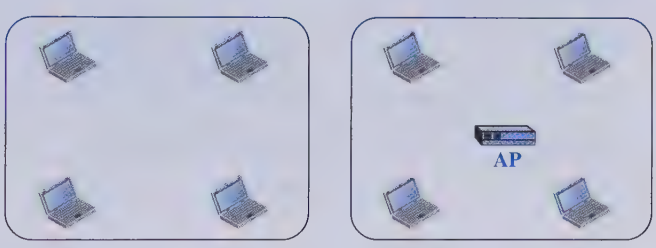
4.2.1 Architecture

The IEEE standard defines two kinds of services: the basic service set (BSS) and the extended service set (ESS).

Basic Service Set

IEEE 802.11 defines the **basic service set (BSS)** as the building blocks of a wireless LAN. A basic service set is made up of stationary or mobile wireless stations and an optional central base station, known as the access point (AP). Figure 4.7 shows two sets in this standard.

Figure 4.7 *Basic service sets (BSSs)*



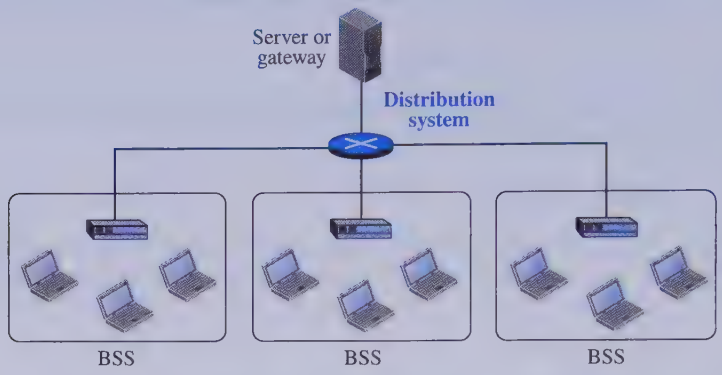
The BSS without an AP is a stand-alone network and cannot send data to other BSSs. It is called an *ad hoc architecture*. In this architecture, stations can form a network without the need of an AP; they can locate one another and agree to be part of a BSS. A BSS with an AP is sometimes referred to as an *infrastructure* BSS.

Extended Service Set

An **extended service set (ESS)** is made up of two or more BSSs with APs. In this case, the BSSs are connected through a *distribution system*, which is a wired or a wireless network. The distribution system connects the APs in the BSSs. IEEE 802.11 does not restrict the distribution system; it can be any IEEE LAN such as an Ethernet. Note that the extended service set uses two types of stations: mobile and stationary. The mobile stations are normal stations inside a BSS. The stationary stations are AP stations that are part of a wired LAN. Figure 4.8 shows an ESS.

When BSSs are connected, the stations within reach of one another can communicate without the use of an AP. However, communication between a station in a BSS and the outside BSS occurs via the AP.

Figure 4.8 *Extended service set (ESS)*



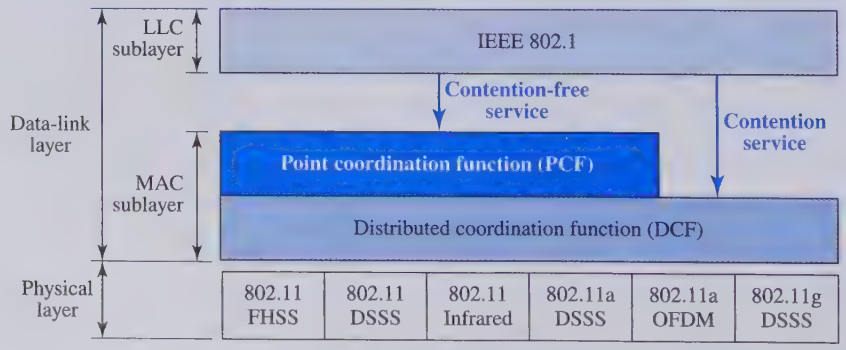
Station Types

IEEE 802.11 defines three types of stations based on their mobility in a wireless LAN: *no-transition*, *BSS-transition*, and *ESS-transition* mobility. A station with no-transition mobility is either stationary (not moving) or moving only inside a BSS. A station with BSS-transition mobility can move from one BSS to another, but the movement is confined inside one ESS. A station with ESS-transition mobility can move from one ESS to another. However, IEEE 802.11 does not guarantee that communication is continuous during the move.

4.2.2 MAC Sublayer

IEEE 802.11 defines two MAC sublayers: the distributed coordination function (DCF) and the point coordination function (PCF). Figure 4.9 shows the relationship among the two MAC sublayers, the LLC sublayer, and the physical layer. We discuss the physical-layer implementations in Section 4.2.4. and will now concentrate on the MAC sublayer.

Figure 4.9 *MAC layers in the IEEE 802.11 standard*



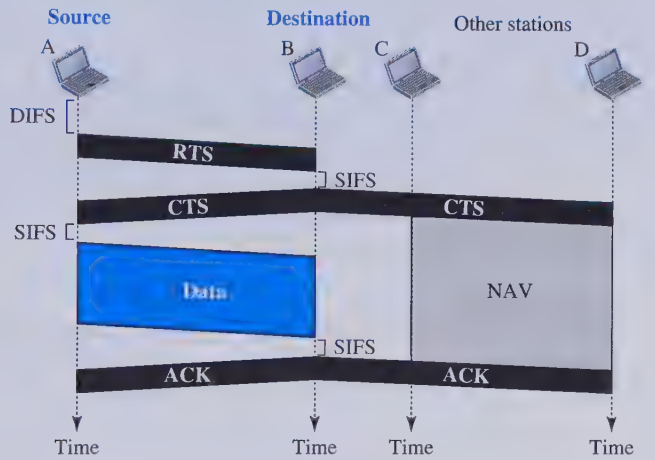
Distributed Coordination Function

One of the two protocols defined by the IEEE at the MAC sublayer is called the **distributed coordination function (DCF)**. DCF uses CSMA/CA as the access method (see Chapter 3).

Frame Exchange Time Line

Figure 4.10 shows the exchange of data and control frames in time.

Figure 4.10 *CSMA/CA and NAV*



1. Before sending a frame, the source station senses the medium by checking the energy level at the carrier frequency.
   a. The channel uses a persistence strategy with backoff until the channel is idle.
   b. After the station is found to be idle, the station waits for a period of time called the **DCF interframe space (DIFS)**; then the station sends a control frame called the *request to send (RTS)*.
2. After receiving the RTS and waiting a period of time called the **short interframe space (SIFS)**, the destination station sends a control frame, called the *clear to send (CTS)*, to the source station. This control frame indicates that the destination station is ready to receive data.
3. The source station sends data after waiting an amount of time equal to the SIFS.
4. The destination station, after waiting an amount of time equal to the SIFS, sends an acknowledgment to show that the frame has been received. Acknowledgment is needed in this protocol because the station does not have any means to check for the successful arrival of its data at the destination. On the other hand, the lack of collision in CSMA/CD is a kind of indication to the source that data have arrived.

Network Allocation Vector

How do other stations defer sending their data if one station acquires access? In other words, how is the *collision avoidance* aspect of this protocol accomplished? The key is a feature called NAV.

When a station sends an RTS frame, it includes the duration of time that it needs to occupy the channel. The stations that are affected by this transmission create a timer called a **network allocation vector (NAV)** that shows how much time must pass before these stations are allowed to check the channel for idleness. Each time a station accesses the system and sends an RTS frame, other stations start their NAV. In other words, each station, before sensing the physical medium to see if it is idle, first checks its NAV to see if it has expired. Figure 4.10 shows the idea of a NAV.

Collision During Handshaking

What happens if there is a collision during the time when RTS or CTS control frames are in transition, often called the handshaking period? Two or more stations may try to send RTS frames at the same time. These control frames may collide. However, because there is no mechanism for collision detection, the sender assumes there has been a collision if it has not received a CTS frame from the receiver. The backoff strategy is employed, and the sender tries again.

Hidden-Station Problem

The solution to the hidden-station problem is the use of the handshake frames (RTS and CTS). Figure 4.10 also shows that the RTS message from B reaches A, but not C. However, because both B and C are within the range of A, the CTS message, which contains the duration of data transmission from B to A, reaches C. Station C knows that some hidden station is using the channel and refrains from transmitting until that duration is over.

Point Coordination Function (PCF)

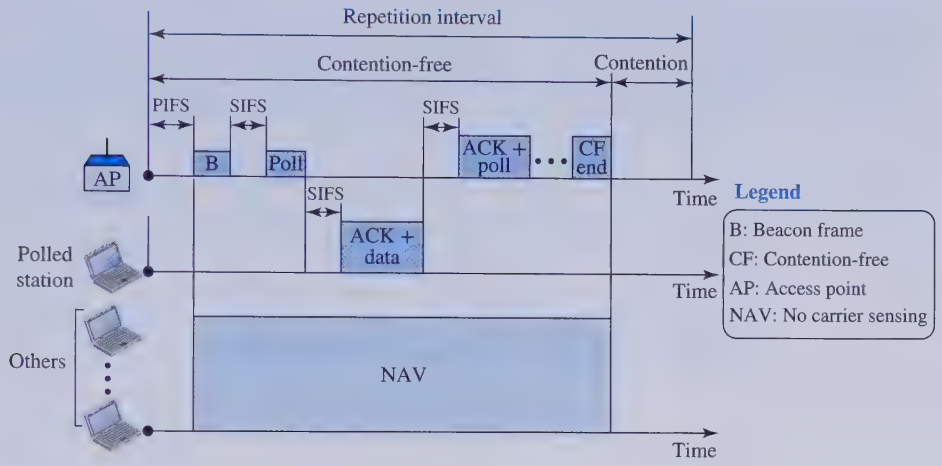
The **point coordination function (PCF)** is an optional access method that can be implemented in an infrastructure network (not in an ad hoc network). It is implemented on top of the DCF and is used mostly for time-sensitive transmission.

PCF has a centralized, contention-free polling access method. The access point performs polling for stations that are capable of being polled. The stations are polled one after another, sending any data they have to the access point.

To give priority to PCF over DCF, another interframe space, has been defined: point coordination function interframe space [PCF IFS (PIFS)]. PIFS is shorter than the DIFS. This means that if, at the same time, a station wants to use only DCF and an access point wants to use PCF, the access point has priority.

Because of the priority of PCF over DCF, stations that only use DCF may not gain access to the medium. To prevent this, a repetition interval has been designed to cover both contention-free PCF and contention-based DCF traffic. The *repetition interval*, which is repeated continuously, starts with a special control frame, called a **beacon frame**. When the stations hear the beacon frame, they start their NAV for the duration of the contention-free period of the repetition interval. Figure 4.11 shows an example of a repetition interval.

Figure 4.11 *Example of repetition interval*



During the repetition interval, the point controller (PC) can send a poll frame, receive data, send an ACK, receive an ACK, or do any combination of these (802.11 uses piggybacking). At the end of the contention-free (CF) period, the PC sends a CF end frame to allow the contention-based stations to use the medium.

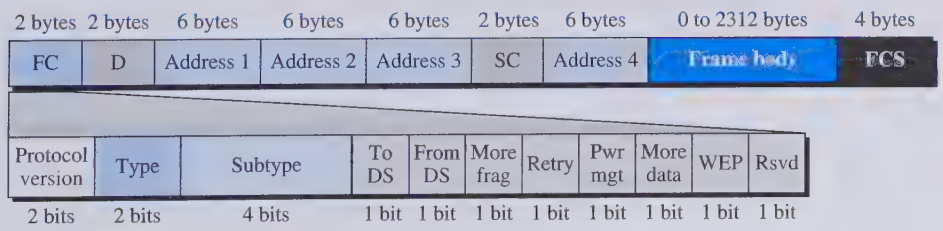
Fragmentation

The wireless environment is very noisy, so frames are often corrupted. A corrupt frame has to be retransmitted. The protocol, therefore, recommends fragmentation—the division of a large frame into smaller ones. It is more efficient to resend a small frame than a large one.

Frame Format

The MAC layer frame consists of nine fields, as shown in Figure 4.12.

Figure 4.12 *Frame format*



- ***Frame control (FC).*** The FC field is 2 bytes long and defines the type of frame and some control information. Table 4.5 describes the subfields. We will discuss each frame type next.

Table 4.5 *Subfields in FC field*

Field	*Explanation*
Version	Current version is 0
Type	Type of information: management (00), control (01), or data (10)
Subtype	Subtype of each type (see Table 4.6)
To DS	Defined later
From DS	Defined later
More flag	When set to 1, means more fragments
Retry	When set to 1, means retransmitted frame
Pwr mgt	When set to 1, means station is in power management mode
More data	When set to 1, means station has more data to send
WEP	Wired equivalent privacy (encryption implemented)
Rsvd	Reserved

- **D.** This field defines the duration of the transmission that is used to set the value of NAV. In one control frame, it defines the ID of the frame.
- **Addresses.** There are four address fields, each 6 bytes long. The meaning of each address field depends on the value of the *To DS* and *From DS* subfields.
- **Sequence control.** This field, often called the SC field, defines a 16-bit value. The first 4 bits define the fragment number; the last 12 bits define the sequence number, which is the same in all fragments.
- **Frame body.** This field, which can be between 0 and 2312 bytes, contains information based on the type and the subtype defined in the FC field.
- **FCS.** The FCS field is 4 bytes long and contains a CRC-32 error-detection sequence.

Frame Types

A wireless LAN defined by IEEE 802.11 has three categories of frames: management frames, control frames, and data frames.

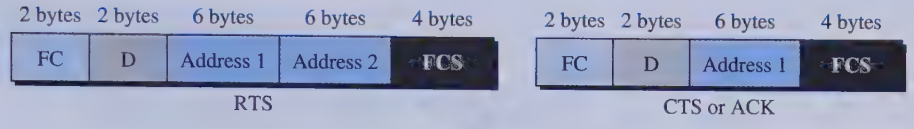
Management Frames

Management frames are used for the initial communication between stations and access points.

Control Frames

Control frames are used for accessing the channel and acknowledging frames. Figure 4.13 shows the format.

Figure 4.13 *Control frames*



For control frames the value of the type field is 01; the values of the subtype fields for frames we have discussed are shown in Table 4.6.

Table 4.6 *Values of subfields in control frames*

Subtype	*Meaning*
1011	Request to send (RTS)
1100	Clear to send (CTS)
1101	Acknowledgment (ACK)

Data Frames

Data frames are used for carrying data and control information.

4.2.3 Addressing Mechanism

IEEE 802.11 specifies four addressing mechanism cases, defined by the value of the two flags in the FC field, *To DS* and *From DS*. Each flag can be either 0 or 1, resulting in four different situations. The interpretation of the four addresses (address 1 to address 4) in the MAC frame depends on the value of these flags, as shown in Table 4.7.

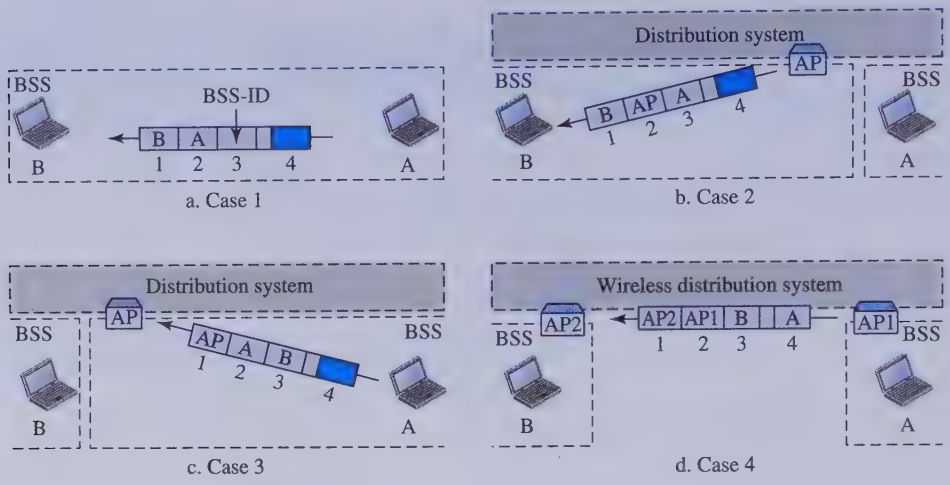
Table 4.7 *Addresses*

To DS	*From DS*	*Address 1*	*Address 2*	*Address 3*	*Address 4*
0	0	Destination	Source	BSS ID	N/A
0	1	Destination	Sending AP	Source	N/A
1	0	Receiving AP	Source	Destination	N/A
1	1	Receiving AP	Sending AP	Destination	Source

Note that address 1 is always the address of the next device that the frame will visit. Address 2 is always the address of the previous device that the frame has left. Address 3 is the address of the final destination station if it is not defined by address 1 or the original source station if is not defined by address 2. Address 4 is the original source when the distribution system is also wireless.

- **Case 1:00.** In this case, *To DS* = 0 and *From DS* = 0. This means that the frame is not going to a distribution system (*To DS* = 0) and is not coming from a distribution system (*From DS* = 0). The frame is going from one station in a BSS to another without passing through the distribution system. The addresses are shown in Figure 4.14.

Figure 4.14 *Addressing mechanisms*

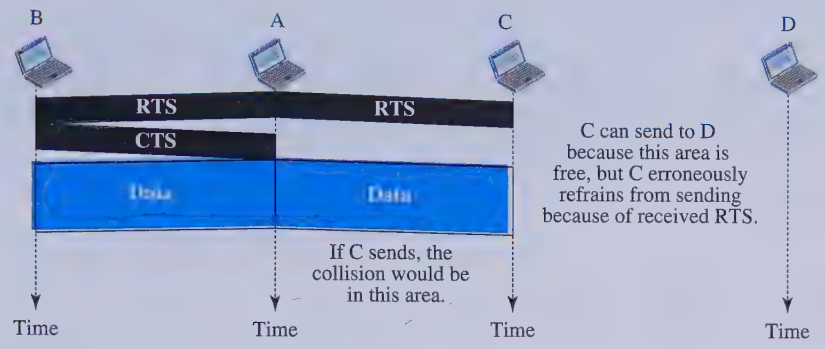


- **Case 2:01.** In this case, *To DS* = 0 and *From DS* = 1. This means that the frame is coming from a distribution system (*From DS* = 1). The frame is coming from an AP and going to a station. The addresses are as shown in Figure 4.14. Note that address 3 contains the original sender of the frame (in another BSS).
- **Case 3:10.** In this case, *To DS* = 1 and *From DS* = 0. This means that the frame is going to a distribution system (*To DS* = 1). The frame is going from a station to an AP. The ACK is sent to the original station. The addresses are as shown in Figure 4.14. Note that address 3 contains the final destination of the frame in the distribution system.
- **Case 4:11.** In this case, *To DS* = 1 and *From DS* = 1. This is the case in which the distribution system is also wireless. The frame is going from one AP to another AP in a wireless distribution system. Here, we need four addresses to define the original sender, the final destination, and two intermediate APs. Figure 4.14 shows the situation.

Exposed-Station Problem

We discussed how to solve the hidden-station problem. A similar problem is called the *exposed-station problem*. In this problem a station refrains from using a channel when it is, in fact, available. In Figure 4.15, station A is transmitting to station B. Station C has some data to send to station D, which can be sent without interfering with the transmission from A to B. However, station C is exposed to transmission from A; it hears what A is sending and thus refrains from sending. In other words, C is too conservative and wastes the capacity of the channel. The handshaking messages RTS and CTS

Figure 4.15 *Exposed-station problem*



cannot help in this case. Station C hears the RTS from A and refrains from sending, even though the communication between C and D cannot cause a collision in the zone between A and C; station C cannot know that station A's transmission does not affect the zone between C and D.

4.2.4 Physical Layer

We discuss five specifications, as shown in Table 4.8. All implementations, except the infrared, operate in the *industrial, scientific, and medical (ISM)* band, which defines three unlicensed bands in the three ranges 902–928 MHz, 2.400–4.835 GHz, and 5.725–5.850 GHz.

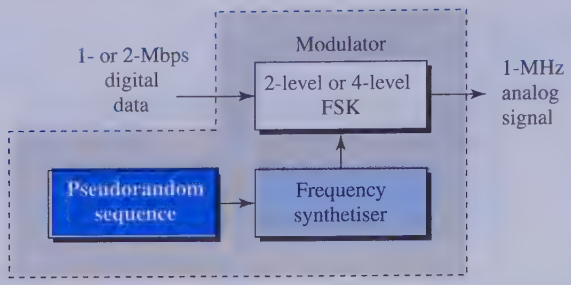
Table 4.8 *Specifications*

IEEE	*Technique*	*Band*	*Modulation*	*Rate (Mbps)*
802.11	FHSS	2.400–4.835 GHz	FSK	1 and 2
	DSSS	2.400–4.835 GHz	PSK	1 and 2
	None	Infrared	PPM	1 and 2
802.11a	OFDM	5.725–5.850 GHz	PSK or QAM	6 to 54
802.11b	DSSS	2.400–4.835 GHz	PSK	5.5 and 11
802.11g	OFDM	2.400–4.835 GHz	Different	22 and 54
802.11n	OFDM	5.725–5.850 GHz	Different	600

IEEE 802.11 FHSS

IEEE 802.11 FHSS uses the frequency-hopping spread spectrum (FHSS) method. FHSS uses the 2.400–4.835 GHz ISM band. The band is divided into 79 subbands of 1 MHz (and some guard bands). A pseudorandom number generator selects the hopping

Figure 4.16 *Physical layer of IEEE 802.11 FHSS*

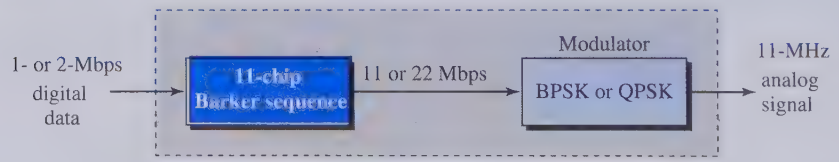


sequence. The modulation technique in this specification is either two-level FSK or four-level FSK with 1 or 2 bits/baud, which results in a data rate of 1 or 2 Mbps, as shown in Figure 4.16.

IEEE 802.11 DSSS

IEEE 802.11 **DSSS** uses the direct sequence spread spectrum (DSSS) method. DSSS uses the 2.400–4.835 GHz ISM band. The modulation technique in this specification is PSK at 1 Mbaud/s. The system allows 1 or 2 bits/baud (BPSK or QPSK), which results in a data rate of 1 or 2 Mbps, as shown in Figure 4.17.

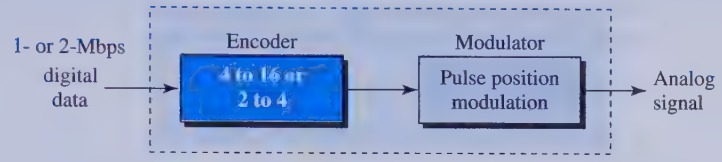
Figure 4.17 *Physical layer of IEEE 802.11 DSSS*



IEEE 802.11 Infrared

IEEE 802.11 infrared uses infrared light in the range of 800 to 950 nm. The modulation technique is called **pulse position modulation (PPM)**. For a 1-Mbps data rate, a 4-bit sequence is first mapped into a 16-bit sequence in which only 1 bit is set to 1 and the rest are set to 0. For a 2-Mbps data rate, a 2-bit sequence is first mapped into a 4-bit sequence in which only 1 bit is set to 1 and the rest are set to 0. The mapped sequences are then converted to optical signals; the presence of light specifies 1, and the absence of light specifies 0. See Figure 4.18.

Figure 4.18 *Physical layer of IEEE 802.11 infrared*



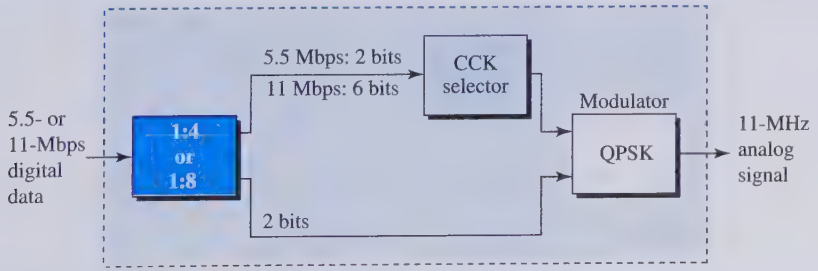
IEEE 802.11a OFDM

IEEE 802.11a OFDM describes the **orthogonal frequency-division multiplexing (OFDM)** method for signal generation in a 5.725–5.850 GHz ISM band. OFDM is similar to FDM with one major difference: All the subbands are used by one source at a given time. Sources contend with one another at the data-link layer for access. The band is divided into 52 subbands, with 48 subbands for sending 48 groups of bits at a time and 4 subbands for control information. Dividing the band into subbands diminishes the effects of interference. If the subbands are used randomly, security can also be increased. OFDM uses PSK and QAM for modulation. The common data rates are 18 Mbps (PSK) and 54 Mbps (QAM).

IEEE 802.11b HR-DSSS

IEEE 802.11b DSSS describes the **high-rate direct-sequence spread spectrum (HR-DSSS)** method for signal generation in the 2.400–4.835 GHz ISM band. HR-DSSS is similar to DSSS except for the encoding method, which is called **complementary code keying (CCK)**. CCK encodes 4 or 8 bits to one CCK symbol. To be backward compatible with DSSS, HR-DSSS defines four data rates: 1, 2, 5.5, and 11 Mbps. The first two use the same modulation techniques as DSSS. The 5.5-Mbps version uses BPSK and transmits at 1.375 Mbaud/s with 4-bit CCK encoding. The 11-Mbps version uses QPSK and transmits at 1.375 Mbps with 8-bit CCK encoding. Figure 4.19 shows the modulation technique for this standard.

Figure 4.19 *Physical layer of IEEE 802.11b*



IEEE 802.11g

IEEE 802.11g, a new specification, defines forward error correction and OFDM using the 2.400–4.835 GHz ISM band. The modulation technique achieves a 22- or 54-Mbps data rate. It is backward compatible with 802.11b, but the modulation technique is OFDM.

IEEE 802.11n

An upgrade to the 802.11 project is called 802.11n (the next generation of wireless LAN). The goal is to increase the throughput of 802.11 wireless LANs. The new standard emphasizes not only the higher bit rate but also eliminating some unnecessary overhead. The standard uses what is called **MIMO (multiple-input multiple-output)** to overcome the noise problem in wireless LANs. The idea is that if we can send multiple output signals and receive multiple input signals, we are in a better position to eliminate noise. Some implementations of this project have reached up to a 600-Mbps data rate.

4.3 BLUETOOTH

Although many wireless LANs were developed during the last decades, the one that is the most common is Bluetooth. **Bluetooth** is a wireless LAN technology designed to connect devices of different functions such as telephones, notebooks, computers (desktop and laptop), cameras, printers, and even coffee makers when they are at a short distance from each other. A Bluetooth LAN is an ad hoc network, which means that the network is formed spontaneously; the devices, sometimes called gadgets, find each other and make a network called a piconet. A Bluetooth LAN can even be connected to the Internet if one of the gadgets has this capability. A Bluetooth LAN, by nature, cannot be large. If there are many gadgets that try to connect, there is chaos.

Bluetooth technology has several applications. Peripheral devices such as a wireless mouse or keyboard can communicate with the computer through this technology. Monitoring devices can communicate with sensor devices in a small health care center. Home security devices can use this technology to connect different sensors to the main security controller. Conference attendees can synchronize their laptop computers at a conference.

Bluetooth was originally started as a project by the Ericsson Company. It is named for Harald Blaatand, the king of Denmark (940–981) who united Denmark and Norway. *Blaatand* translates to *Bluetooth* in English.

Today, Bluetooth technology is the implementation of a protocol defined by the IEEE 802.15 standard. The standard defines a wireless personal area network (WPAN) operable in an area the size of a room or a hall.

4.3.1 Architecture

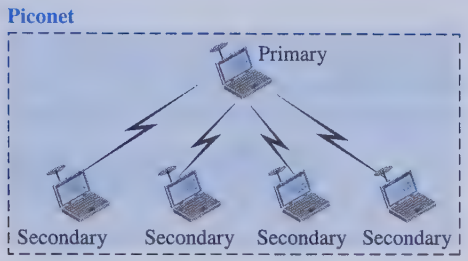
Bluetooth defines two types of networks: piconet and scatternet.

Piconets

A Bluetooth network is called a **piconet**, or a small net. A piconet can have up to eight stations, one of which is called the *primary*; the rest are called *secondaries*. All the

secondary stations synchronize their clocks and hopping sequence with the primary. Note that a piconet can have only one primary station. The communication between the primary and secondary stations can be one-to-one or one-to-many. Figure 4.20 shows a piconet.

Figure 4.20 *Piconet*

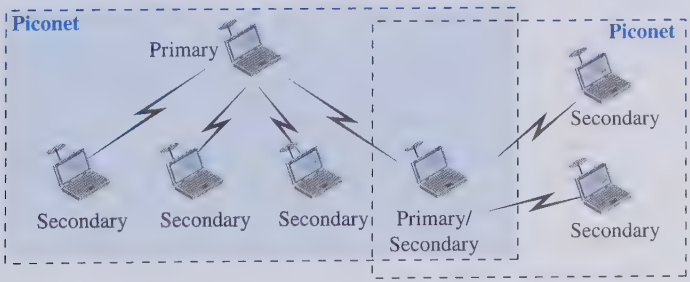


Although a piconet can have a maximum of seven secondaries, additional secondaries can be in the *parked state*. A secondary in a parked state is synchronized with the primary but cannot take part in communication until it is moved from the parked state to the active state. Because only eight stations can be active in a piconet, activating a station from the parked state means that an active station must go to the parked state.

Scatternet

Piconets can be combined to form what is called a **scatternet**. A secondary station in one piconet can be the primary in another piconet. This station can receive messages from the primary in the first piconet (as a secondary) and, acting as a primary, deliver them to secondaries in the second piconet. A station can be a member of two piconets. Figure 4.21 illustrates a scatternet.

Figure 4.21 *Scatternet*



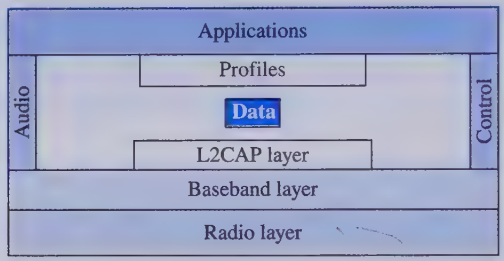
Bluetooth Devices

A Bluetooth device has a built-in short-range radio transmitter. The current data rate is 1 Mbps with a 2.4-GHz bandwidth. This means that there is a possibility of interference between the IEEE 802.11b wireless LANs and Bluetooth LANs.

4.3.2 Bluetooth Layers

Bluetooth uses several layers that do not exactly match those of the Internet model we have defined in this book. Figure 4.22 shows these layers.

Figure 4.22 *Bluetooth layers*



L2CAP

The **Logical Link Control and Adaptation Protocol (L2CAP)** (L2 here means LL) is roughly equivalent to the LLC sublayer in LANs. It is used for data exchange on an ACL link; SCO channels do not use L2CAP. Figure 4.23 shows the format of the data packet at this level.

Figure 4.23 *L2CAP data packet format*

The 16-bit length field defines the size of the data, in bytes, coming from the upper layers. Data can be up to 65,535 bytes. The channel ID (CID) defines a unique identifier for the virtual channel created at this level.

The L2CAP has specific duties: multiplexing, segmentation and reassembly, quality of service (QoS), and group management.

Multiplexing

The L2CAP can do multiplexing. At the sender site, it accepts data from one of the upper-layer protocols, frames them, and delivers them to the baseband layer. At the

receiver site, it accepts a frame from the baseband layer, extracts the data, and delivers them to the appropriate protocol layer. It creates a kind of virtual channel that we will discuss in later chapters on higher-level protocols.

Segmentation and Reassembly

The maximum size of the payload field in the baseband layer is 2774 bits, or 343 bytes. This includes 4 bytes to define the packet and packet length. Therefore, the size of the packet that can arrive from an upper layer can only be 339 bytes. However, application layers sometimes need to send a data packet that can be up to 65,535 bytes (an Internet packet, for example). The L2CAP divides these large packets into segments and adds extra information to define the location of the segments in the original packet. The L2CAP segments the packet at the source and reassembles the segments at the destination.

Group Management

Another functionality of L2CAP is to allow devices to create a type of logical addressing between themselves. This is similar to multicasting. For example, two or three secondary devices can be part of a multicast group to receive data from the primary.

Baseband Layer

The baseband layer is roughly equivalent to the MAC sublayer in LANs. The access method is TDMA. The primary and secondary stations communicate with each other using time slots. The length of a time slot is exactly the same as the dwell time, 625 μs. This means that during the time that one frequency is used, a primary sends a frame to a secondary, or a secondary sends a frame to the primary. Note that the communication is only between the primary and a secondary; secondaries cannot communicate directly with one another.

TDMA

Bluetooth uses a form of TDMA that is called **TDD-TDMA (time-division duplex TDMA)**. TDD-TDMA is a kind of half-duplex communication in which the sender and receiver send and receive data, but not at the same time (half-duplex); however, the communication for each direction uses different hops. This is similar to walkie-talkies using different carrier frequencies.

- **Single-secondary communication.** If the piconet has only one secondary, the TDMA operation is very simple. The time is divided into slots of 625 μs. The primary uses even-numbered slots (0, 2, 4, . . .); the secondary uses odd-numbered slots (1, 3, 5, . . .). TDD-TDMA allows the primary and the secondary to communicate in half-duplex mode. In slot 0, the primary sends and the secondary receives; in slot 1, the secondary sends and the primary receives. The cycle is repeated. Figure 4.24 shows the concept.
- **Multiple-secondary communication.** The process is a little more involved if there is more than one secondary in the piconet. Again, the primary uses the even-numbered slots, but a secondary sends in the next odd-numbered slot if the packet in the previous slot was addressed to it. All secondaries listen on even-numbered slots, but only one secondary sends in any odd-numbered slot. Figure 4.25 shows a scenario.

Figure 4.24 *Single-secondary communication*

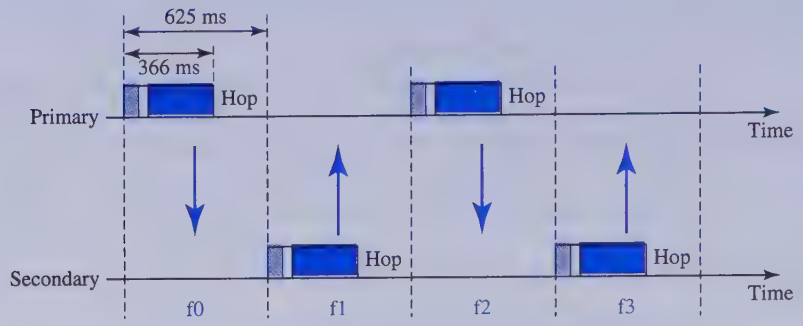
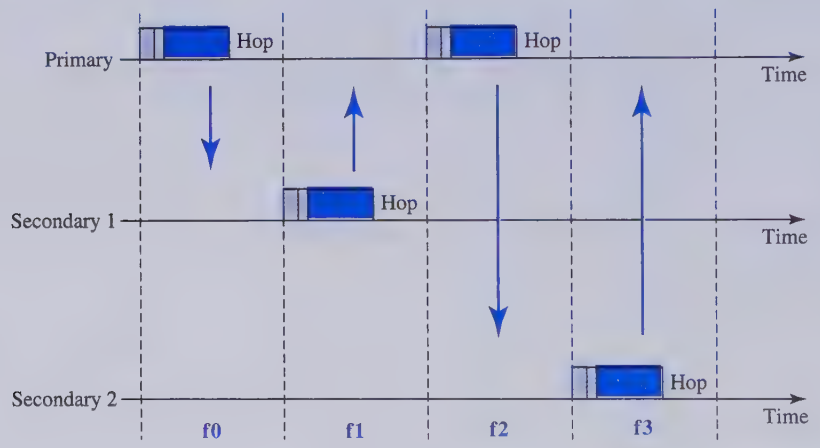


Figure 4.25 *Multiple-secondary communication*



Let us elaborate on Figure 4.25.

1. In slot 0, the primary sends a frame to secondary 1.
2. In slot 1, only secondary 1 sends a frame to the primary because the previous frame was addressed to secondary 1; other secondaries are silent.
3. In slot 2, the primary sends a frame to secondary 2.
4. In slot 3, only secondary 2 sends a frame to the primary because the previous frame was addressed to secondary 2; other secondaries are silent.
5. The cycle continues.

We can say that this access method is similar to a poll/select operation with reservations. When the primary selects a secondary, it also polls it. The next time slot is reserved for the polled station to send its frame. If the polled secondary has no frame to send, the channel is silent.

Links

Two types of links can be created between a primary and a secondary: SCO links and ACL links.

- **SCO.** A **synchronous connection-oriented (SCO)** link is used when avoiding latency (delay in data delivery) is more important than integrity (error-free delivery). In an SCO link, a physical link is created between the primary and a secondary by reserving specific slots at regular intervals. The basic unit of connection is two slots, one for each direction. If a packet is damaged, it is never retransmitted. SCO is used for real-time audio where avoiding delay is all-important. A secondary can create up to three SCO links with the primary, sending digitized audio (PCM) at 64 kbps in each link.
- **ACL.** An **asynchronous connectionless link (ACL)** is used when data integrity is more important than avoiding latency. In this type of link, if a payload encapsulated in the frame is corrupted, it is retransmitted. A secondary returns an ACL frame in the available odd-numbered slot if the previous slot has been addressed to it. ACL can use one, three, or more slots and can achieve a maximum data rate of 721 kbps.

Frame Format

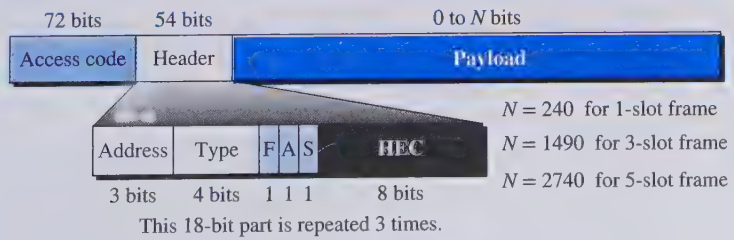
A frame in the baseband layer can be one of three types: one-slot, three-slot, or five-slot. A slot, as we said before, is 625 μs. However, in a one-slot frame exchange, 259 μs is needed for hopping and control mechanisms. This means that a one-slot frame can last only 625−259, or 366 μs. With a 1-MHz bandwidth and 1 bit/Hz, the size of a one-slot frame is 366 bits.

A three-slot frame occupies three slots. However, since 259 μs is used for hopping, the length of the frame is $3 \times 625 - 259 = 1616$ μs or 1616 bits. A device that uses a three-slot frame remains at the same hop (at the same carrier frequency) for three slots. Even though only one hop number is used, three hop numbers are consumed. That means the hop number for each frame is equal to the first slot of the frame.

A five-slot frame also uses 259 bits for hopping, which means that the length of the frame is $5 \times 625 - 259 = 2866$ bits.

Figure 4.26 shows the format of the three frame types.

Figure 4.26 *Frame format types*



The following describes each field:

- **Access code.** This 72-bit field normally contains synchronization bits and the identifier of the primary to distinguish the frame of one piconet from another.
- **Header.** This 54-bit field is a repeated 18-bit pattern. Each pattern has the following subfields:
  - **Address.** The 3-bit address subfield can define up to seven secondaries (1 to 7). If the address is zero, it is used for broadcast communication from the primary to all secondaries.
  - **Type.** The 4-bit type subfield defines the type of data coming from the upper layers. We discuss these types later.
  - **F.** This 1-bit subfield is for flow control. When set (1), it indicates that the device is unable to receive more frames (buffer is full).
  - **A.** This 1-bit subfield is for acknowledgment. Bluetooth uses Stop-and-Wait ARQ; 1 bit is sufficient for acknowledgment.
  - **S.** This 1-bit subfield holds a sequence number. Bluetooth uses Stop-and-Wait ARQ; 1 bit is sufficient for sequence numbering.
  - **HEC.** The 8-bit header error-correction subfield is a checksum to detect errors in each 18-bit header section. The header has three identical 18-bit sections. The receiver compares these three sections, bit by bit. If each of the corresponding bits is the same, the bit is accepted; if not, the majority opinion rules. This is a form of forward error correction (for the header only). This double error control is needed because the nature of the communication, via air, is very noisy. Note that there is no retransmission in this sublayer.
- **Payload.** This subfield can be 0 to 2740 bits long. It contains data or control information coming from the upper layers.

Radio Layer

The radio layer is roughly equivalent to the physical layer of the Internet model. Bluetooth devices are low-power and have a range of 10 m.

Band

Bluetooth uses a 2.4-GHz ISM band divided into 79 channels of 1 MHz each.

FHSS

Bluetooth uses the **frequency-hopping spread spectrum (FHSS)** method in the physical layer to avoid interference from other devices or other networks. Bluetooth hops 1600 times per second, which means that each device changes its modulation frequency 1600 times per second. A device uses a frequency for only 625 μs (1/1600 s) before it hops to another frequency; the dwell time is 625 μs.

Modulation

To transform bits to a signal, Bluetooth uses a sophisticated version of FSK, called GFSK (FSK with Gaussian bandwidth filtering; a discussion of this topic is beyond the scope of this book). GFSK has a carrier frequency. Bit 1 is represented by a frequency deviation above the carrier; bit 0 is represented by a frequency deviation below the

carrier. The frequencies, in megahertz, are defined according to the following formula for each channel:

$$f_c = 2402 + n \text{ MHz} \qquad n = 0, 1, 2, 3, \ldots, 78$$

For example, the first channel uses carrier frequency 2402 MHz (2.402 GHz), and the second channel uses carrier frequency 2403 MHz (2.403 GHz).

4.4 END-OF-CHAPTER MATERIALS

4.4.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items in brackets refer to the reference list at the end of the text.

Books

Several books address the materials discussed in this chapter. Among them we recommend [Ham 80], [Zar 02], [Ror 96], [Tan 03], [GW 04], [For 03], [KMK 04], [Sta 04], [Kes 02], [PD 03], [Kei 02], [Spu 00], [KCK 98], [Sau 98], [Izz 00], [Per 00], and [WV 00].

4.4.2 Key Terms

10 Gigabit Ethernet
1000Base-CX
1000Base-LX
1000Base-SX
1000Base-T
100Base-FX
100Base-T4
100Base-TX
10Base2
10Base5
10Base-F
10Base-T
10GBase-EW
10GBase-LR
10GBase-SR
10GBase-X4
asynchronous connectionless link (ACL)
autonegotiation
basic service set (BSS)
beacon frame
Bluetooth
carrier extension
complementary code keying (CCK)
DCF interframe space (DIFS)
distributed coordination function (DCF)
DSSS
extended service set (ESS)
Fast Ethernet
frame bursting
frequency-hopping spread spectrum (FHSS)
Gigabit Ethernet
hexadecimal notation
high-rate direct-sequence spread spectrum (HR-DSSS)
logical link control (LLC)
Logical Link Control and Adaptation Protocol (L2CAP)
media access control (MAC)
multiple-input multiple-output (MIMO)
network allocation vector (NAV)
network interface card (NIC)
orthogonal frequency-division multiplexing (OFDM)
piconet
point coordination function (PCF)
Project 802
pulse position modulation (PPM)
scatternet
short interframe space (SIFS)
Standard Ethernet
switch
synchronous connection-oriented (SCO)
time-division duplex TDMA (TDD-TDMA)

4.4.3 Summary

Local area networks (LANs) can be wired (Ethernet) or wireless (WiFi and Bluetooth). Ethernet is the most widely used local area network protocol. There are four versions of Ethernet. Standard Ethernet (10 Mbps) is the original Ethernet technology. The next generation is called Fast Ethernet (100 Mbps) and has autonegotiation, which allows two devices to negotiate the mode or data rate of operation. The next evolution is Gigabit Ethernet (1000 Mbps), with access methods that include half-duplex mode and full-duplex mode (most popular method). The latest standard is 10 Gigabit Ethernet (10 Gbps), which uses fiber-optic cables in full-duplex mode.

IEEE 802.11 has defined a wireless Ethernet call WiFi, which covers the physical and data link layer.

Bluetooth is a wireless LAN technology designed to connect devices of different functions when they are at a short distance from each other. There are two types of Bluetooth architecture: piconet and scatternet networks. Bluetooth has several layers including Logical Link Control and Adaptation Protocol (L2CAP), baseband, and radio.

4.5 PRACTICE SET

4.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

4.5.2 Questions

Q4-1. Compare the data rates for Standard Ethernet, Fast Ethernet, Gigabit Ethernet, and 10 Gigabit Ethernet.

Q4-2. What are the common Standard Ethernet implementations?

Q4-3. What are the common Fast Ethernet implementations?

Q4-4. What are the common Gigabit Ethernet implementations?

Q4-5. What are the common 10 Gigabit implementations?

Q4-6. How is the preamble field different from the SFD field?

Q4-7. What is the difference between unicast, multicast, and broadcast addresses?

Q4-8. What is the reason that Bluetooth is normally called a wireless personal area network (WPAN) instead of a wireless local area network (WLAN)?

Q4-9. Compare a piconet and a scatternet in the Bluetooth architecture.

Q4-10. Can a piconet have more than eight stations? Explain.

Q4-11. What is the actual bandwidth used for communication in a Bluetooth network?

Q4-12. What is the role of the *radio* layer in Bluetooth?

Q4-13. Fill in the blanks: The 83.5-MHz bandwidth in Bluetooth is divided into_____ channels, each of _____ MHz.

Q4-14. What is the spread spectrum technique used by Bluetooth?

Q4-15. What is the modulation technique in the radio layer of Bluetooth? In other words, how are digital data (bits) changed to analog signals (radio waves)?

Q4-16. What MAC protocol is used in the baseband layer of Bluetooth?

Q4-17. What is the role of the *L2CAP* layer in Bluetooth?

4.5.3 Problems

P4-1. What is the hexadecimal equivalent of the following Ethernet address?

01011010 00010001 01010101 00011000 10101010 00001111

P4-2. How does the Ethernet address 1A:2B:3C:4D:5E:6F appear on the line in binary?

P4-3. If an Ethernet destination address is 07:01:02:03:04:05, what is the type of the address (unicast, multicast, or broadcast)?

P4-4. Suppose the length of a 10Base5 cable is 2500 m. If the speed of propagation in a thick coaxial cable is 200,000,000 m/s, how long does it take for a bit to travel from the beginning to the end of the network? Assume there is a 10-μs delay in the equipment.

P4-5. Suppose you are to design a LAN for a company that has 100 employees, each with a desktop computer attached to the LAN. Give the data rate of the LAN for the following typical uses of the LAN:

a. Each employee needs to retrieve a file of average size of 10 Mbytes in a second. An employee may do this on average 10 times during the 8-hour working time.

b. Each employee needs to access the Internet at 250 kbps. This can happen for 10 employees simultaneously.

c. Each employee may receive 10 e-mails per hour with an average size of 100 kbytes. Half of the employees may receive e-mails simultaneously.

P4-6. In a Standard Ethernet LAN, the average size of a frame is 1000 bytes. If a noise of 2 ms occurs on the LAN, how many frames are destroyed?

P4-7. In a LAN based on IEEE 802.11, give the value of the address 1 field in each of the following situations (left bit defines *To DS* and right bit defines *From DS*).

a. 00 **b.** 01 **c.** 10 **d.** 11

P4-8. In a LAN based on IEEE 802.11, give the value of the address 2 field in each of the following situations (left bit defines *To DS* and right bit defines *From DS*).

a. 00 **b.** 01 **c.** 10 **d.** 11

P4-9. In a LAN based on IEEE 802.11, give the value of the address 3 field in each of the following situations (left bit defines *To DS* and right bit defines *From DS*).

a. 00 **b.** 01 **c.** 10 **d.** 11

P4-10. In a LAN based on IEEE 802.11, give the value of the address 4 field in each of the following situations (left bit defines *To DS* and right bit defines *From DS*).

a. 00 **b.** 01 **c.** 10 **d.** 11

P4-11. In a BSS with no AP (ad hoc network), we have five stations: A, B, C, D, and E. Station A needs to send a message to station B. Answer the following questions for the situation where the network is using the DCF protocol:

a. What are the values of the *To DS* and *From DS* bits in the frames exchanged?

b. Which station sends the RTS frame, and what is (are) the value(s) of the address field(s) in this frame?

c. Which station sends the CTS frame, and what is (are) the value(s) of the address field(s) in this frame?

d. Which station sends the data frame, and what is (are) the value(s) of the address field(s) in this frame?

e. Which station sends the ACK frame, and what is (are) the value(s) of the address field(s) in this frame?

P4-12. Assume that a frame moves from a wired network using the 802.3 protocol to a wireless network using the 802.11 protocol. Show how the field values in the 802.11 frame are filled with the values of the 802.3 frame. Assume that the transformation occurs at the AP that is on the boundary between the two networks.

P4-13. Assume a frame moves from a wireless network using the 802.11 protocol to a wired network using the 802.3 protocol. Show how the field values in the 802.3 frame are filled with the values of the 802.11 frame. Assume that the transformation occurs at the AP that is on the boundary between the two -networks.

P4-14. A BSS ID (BSSID) is a 48-bit address assigned to a BSS in an 802.11 network. Do some research and find what the use of the BSSID is and how BSSIDs are assigned in ad hoc and infrastructure networks.

P4-15. Do some research and find out how flow and error control are accomplished in an 802.11 network using the DCF MAC sublayer.

CHAPTER 5

Wide Area Networks: WANs

After discussing the wired and wireless local area networks (LANs) in Chapter 4, it is time to discuss the wired and wireless wide area networks (WANs). This chapter is divided into four sections.

- In the first section, we discuss the telephone network, a wired WAN that was originally designed to handle voice but today handles any type of data.
- In the second section, we discuss the cable network, another wired WAN that was originally designed to provide voice and picture but today handles any type of data.
- In the third section, we discuss cellular telephone, a wireless WAN that provides any type of communication.
- In the fourth section, we discuss the satellite network, a wireless WAN that provides any type of communication.

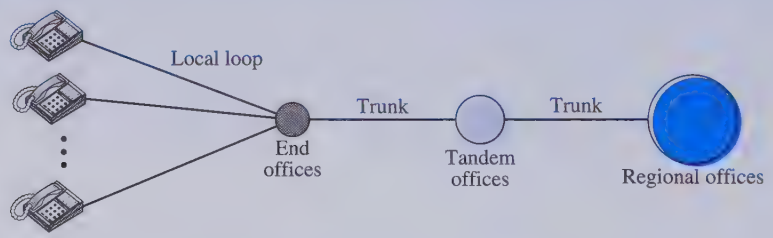
5.1 TELEPHONE NETWORKS

The telephone network had its beginnings in the late 1800s. The entire network, which is referred to as the **plain old telephone system (POTS)**, was originally an analog system using analog signals to transmit voice. With the advent of the computer era, the network, in the 1980s, began to carry data in addition to voice. During the last decade, the telephone network has undergone many technical changes. The network is now digital as well as analog.

5.1.1 Major Components

The telephone network, as shown in Figure 5.1, is made up of three major components: local loops, trunks, and switching offices. The telephone network has several levels of switching offices such as **end offices**, **tandem offices**, and **regional offices**.

Figure 5.1 *A telephone system*



Local Loops

One component of the telephone network is the **local loop**, a twisted-pair cable that connects the subscriber telephone to the nearest end office or local central office. The local loop, when used for voice, has a bandwidth of 4000 Hz (4 kHz). It is interesting to examine the telephone number associated with each local loop. The first three digits of a local telephone number define the office, and the next four digits define the local loop number.

Trunks

Trunks are transmission media that handle the communication between offices. A trunk normally handles hundreds or thousands of connections through multiplexing. Transmission is usually through optical fibers or satellite links.

Switching Offices

To avoid having a permanent physical link between any two subscribers, the telephone company has switches located in a **switching office**. A switch connects several local loops or trunks and allows a connection between different subscribers.

5.1.2 LATAs

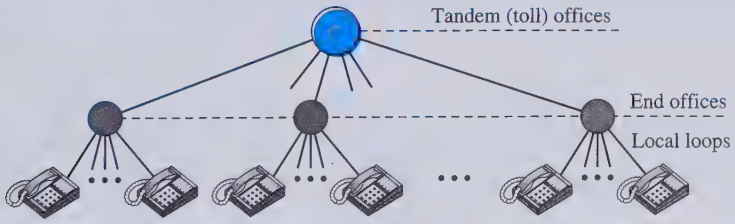
After 1984, the United States was divided into more than 200 **local access transport areas (LATAs)**. The number of LATAs has increased since then. A LATA can be a small or large metropolitan area. A small state may have one single LATA; a large state may have several LATAs. A LATA boundary may overlap the boundary of a state; part of a LATA can be in one state, part in another state.

Intra-LATA Services

The services offered by the **common carriers** (telephone companies) inside a LATA are called *intra-LATA* services. The carrier that handles these services is called a **local exchange carrier (LEC)**. Before the Telecommunications Act of 1996, intra-LATA services were granted to one single carrier. This was a monopoly. After 1996, more than one carrier could provide services inside a LATA. The carrier that provided services before 1996 owns the cabling system (local loops) and is called the **incumbent local exchange carrier (ILEC)**. The new carriers that can provide services are called **competitive local exchange carriers (CLECs)**. To avoid the costs of new cabling, it was agreed that the ILECs would continue to provide the main services, and the CLECs would provide other services such as mobile telephone service, and toll calls inside a LATA. Figure 5.2 shows a LATA and switching offices.

Intra-LATA services are provided by local exchange carriers. Since 1996, there are two types of LECs: incumbent local exchange carriers and competitive local exchange carriers.

Figure 5.2 *Switching offices in a LATA*



Communication inside a LATA is handled by end switches and tandem switches. A call that can be completed by using only end offices is considered toll-free. A call that has to go through a tandem office (intra-LATA toll office) is charged.

Inter-LATA Services

The services between LATAs are handled by **interexchange carriers (IXCs)**. These carriers, sometimes called **long-distance companies**, provide communication services between two customers in different LATAs. After the Telecommunications Act of 1996, these services can be provided by any carrier, including those involved in intra-LATA

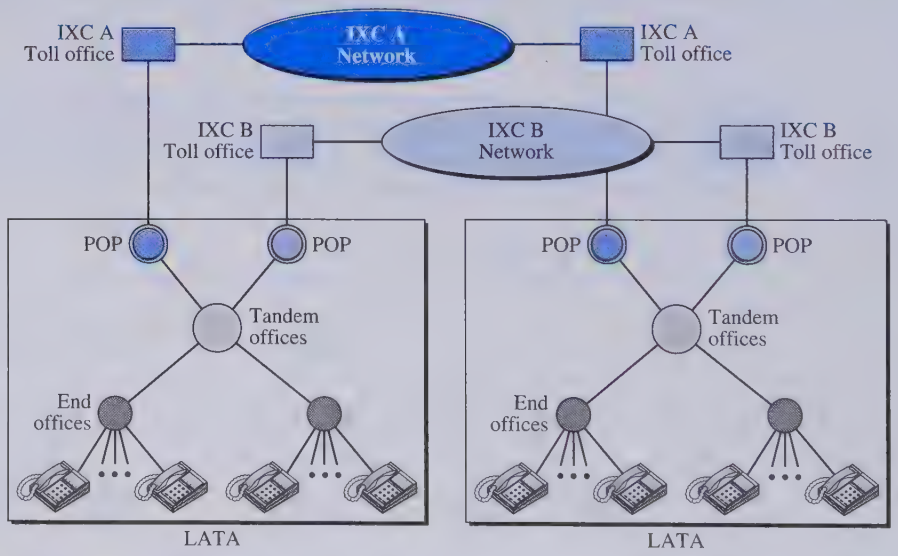
services. The field is wide open. Carriers providing inter-LATA services include AT&T, Sprint, and Verizon.

The IXCs are long-distance carriers that provide general data communications services including telephone service. A telephone call going through an IXC is normally digitized, with the carriers using several types of networks to provide service.

Points of Presence

As we discussed, intra-LATA services can be provided by several LECs (one ILEC and possibly more than one CLEC). We also said that inter-LATA services can be provided by several IXCs. How do these carriers interact with one another? The answer is that they interact via a switching office called a **point of presence (POP)**. Each IXC that wants to provide inter-LATA services in a LATA must have a POP in that LATA. The LECs that provide services inside the LATA must provide connections so that every subscriber can have access to all POPs. Figure 5.3 illustrates the concept.

Figure 5.3 *Points of presence (POPs)*



A subscriber who needs to make a connection with another subscriber is connected first to an end switch and then, either directly or through a tandem switch, to a POP. The call now goes from the POP of an IXC (the one the subscriber has chosen) in the source LATA to the POP of the same IXC in the destination LATA. The call is passed through the toll office of the IXC and is carried through the network provided by the IXC.

5.1.3 Signaling

The telephone network, at its beginning, used a circuit-switched network with dedicated links (multiplexing had not yet been invented) to transfer voice communication.

A circuit-switched network needs the setup and teardown phases to establish and terminate paths between the two communicating parties. In the beginning, this task was performed by human operators. The operator room was a center to which all subscribers were connected. A subscriber who wished to talk to another subscriber picked up the receiver (off-hook) and rang the operator. The operator, after listening to the caller and getting the identifier of the called party, connected the two by using a wire with two plugs inserted into the corresponding two jacks. A dedicated circuit was created in this way. One of the parties, after the conversation ended, informed the operator to disconnect the circuit. This type of signaling is called **in-band signaling** because the same circuit can be used for both signaling and voice communication.

Later, the signaling system became automatic. Rotary telephones were invented that sent a digital signal defining each digit in a multidigit telephone number. The switches in the telephone companies used the digital signals to create a connection between the caller and the called parties. Both in-band and **out-of-band signaling** were used. In in-band signaling, the 4-kHz voice channel was also used to provide signaling. In out-of-band signaling, a portion of the voice channel bandwidth was used for signaling; the voice bandwidth and the signaling bandwidth were separate.

As telephone networks evolved into a complex network, the functionality of the signaling system increased. The signaling system was required to perform other tasks such as

1. Providing dial tone, ring tone, and busy tone
2. Transferring telephone numbers between offices
3. Maintaining and monitoring the call
4. Keeping billing information
5. Maintaining and monitoring the status of the telephone network equipment
6. Providing other functions such as caller ID, voice mail, and so on

These complex tasks resulted in the provision of a separate network for signaling. This means that a telephone network today can be thought of as two networks: a signaling network and a data transfer network.

The tasks of data transfer and signaling are separated in modern telephone networks: Data transfer is done by one network, signaling by another.

However, we need to emphasize a point here. Although the two networks are separate, this does not mean that there are separate physical links everywhere; the two networks may use separate channels of the same link in parts of the system.

Data Transfer Network

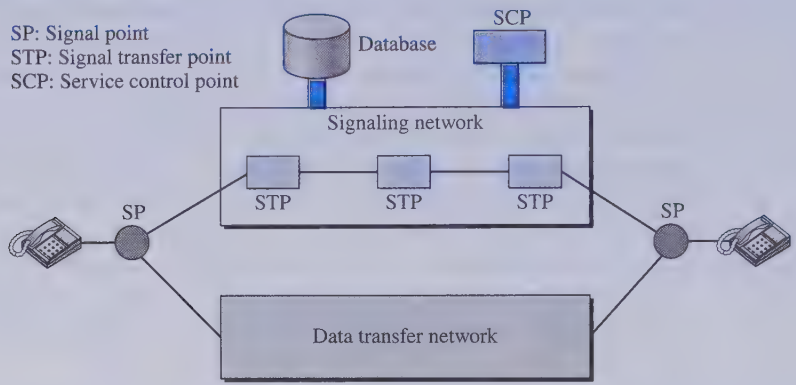
The data transfer network that can carry multimedia information today is, for the most part, a circuit-switched network, although it can also be a packet-switched network. This network follows the same type of protocols and model as other networks discussed in this book.

Signaling Network

The signaling network, which is our main concern in this section, is a packet-switched network involving the layers similar to those in the OSI model or Internet model,

discussed in Chapter 1. The nature of signaling makes it more suited to a packet-switching network with different layers. For example, the information needed to convey a telephone address can easily be encapsulated in a packet with all the error-control and addressing information. Figure 5.4 shows a simplified situation of a telephone network in which the two networks are separated.

Figure 5.4 *Data transfer and signaling networks*



The user telephone or computer is connected to the **signal points (SPs)**. The link between the telephone set and SP is common for the two networks. The signaling network uses nodes called **signal transport ports (STPs)** that receive and forward signaling messages. The signaling network also includes a **service control point (SCP)** that controls the whole operation of the network. Other systems such as a database center may be included to provide stored information about the entire signaling network.

Signaling System Seven (SS7)

The protocol that is used in the signaling network is called **Signaling System Seven (SS7)**. It is very similar to the five-layer Internet model we saw in Chapter 1 but the layers have different names, as shown in Figure 5.5.

Physical Layer: MTP Level 1

The physical layer in SS7 is called **message transport part (MTP) level 1** and uses physical layer specifications.

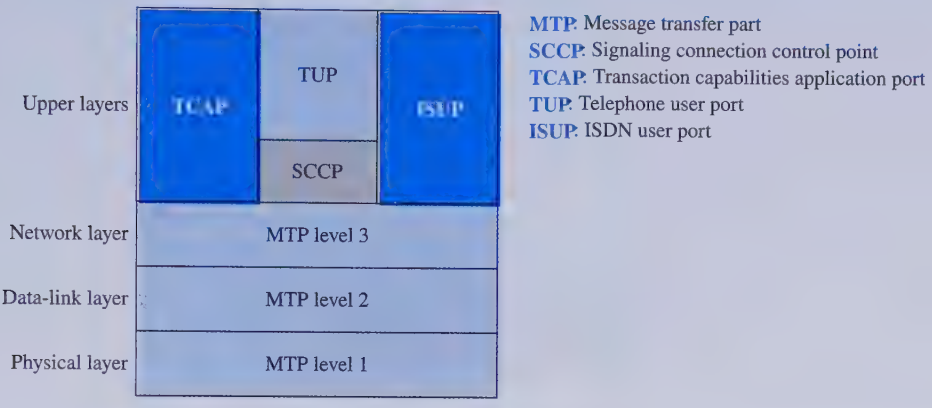
Data-Link Layer: MTP Level 2

The MTP level 2 layer provides typical data-link-layer services such as packetizing, using source and destination address in the packet header, and CRC for error checking.

Network Layer: MTP Level 3

The MTP level 3 layer provides end-to-end connectivity by using the datagram approach to switching. Routers and switches route the signal packets from the source to the destination.

Figure 5.5 *Layers in SS7*



Transport Layer: SCCP

The **signaling connection control point (SCCP)** is used for special services such as 800-call processing.

Upper Layers: TUP, TCAP, and ISUP

There are three protocols at the upper layers. **Telephone user port (TUP)** is responsible for setting up voice calls. It receives the dialed digits and routes the calls. **Transaction capabilities application port (TCAP)** provides remote calls that let an application program on a computer invoke a procedure on another computer. **ISDN user port (ISUP)** can replace TUP to provide services similar to those of an ISDN network.

5.1.4 Services Provided by Telephone Networks

Telephone companies provide two types of services: analog and digital.

Analog Services

In the beginning, telephone companies provided their subscribers with analog services. These services still continue today. We can categorize these services as either **analog switched services** or **analog leased services**.

Analog Switched Services

This is the familiar dial-up service most often encountered when a home telephone is used. The signal on a local loop is analog, and the bandwidth is usually between 0 and 4000 Hz. A local call service is normally provided for a flat monthly rate, although in some LATAs, the carrier charges for each call or a set of calls. The rationale for a non-flat-rate charge is to provide cheaper service for those customers who do not make many calls. A toll call can be intra-LATA or inter-LATA. If the LATA is geographically large, a call may go through a tandem office (toll office) and the subscriber will pay a fee for the call. The inter-LATA calls are long-distance calls and are charged as such.

Another service is called **800 service**. If a subscriber (normally an organization) needs to provide free connections for other subscribers (normally customers), it can request the 800 service. In this case, the call is free for the caller, but it is paid by the callee. An organization uses this service to encourage customers to call. The rate is less expensive than that for a normal long-distance call.

The **wide area telephone service (WATS)** is the opposite of the 800 service. The latter are inbound calls paid by the organization; the former are outbound calls paid by the organization. This service is a less expensive alternative to regular toll calls; charges are based on the number of calls. The service can be specified as outbound calls to the same state, to several states, or to the whole country, with rates charged accordingly.

The **900 services** are like the 800 service, in that they are inbound calls to a subscriber. However, unlike the 800 service, the call is paid by the caller and is normally much more expensive than a normal long-distance call. The reason is that the carrier charges *two* fees: the first is the long-distance toll, and the second is the fee paid to the callee for each call.

Analog Leased Service

An analog leased service offers customers the opportunity to lease a line, sometimes called a *dedicated line*, that is permanently connected to another customer. Although the connection still passes through the switches in the telephone network, subscribers experience it as a single line because the switch is always closed; no dialing is needed.

Digital Services

Recently telephone companies began offering **digital services** to their subscribers. Digital services are less sensitive than analog services to noise and other forms of interference. The two most common digital services are switched/56 service and **digital data service (DDS)**.

Switched/56 Service

Switched/56 service is the digital version of an analog switched line. It is a switched digital service that allows data rates of up to 56 kbps. To communicate through this service, both parties must subscribe. A caller with normal telephone service cannot connect to a telephone or computer with switched/56 service even if the caller is using a modem. On the whole, digital and analog services represent two completely different domains for the telephone companies. Because the line in a switched/56 service is already digital, subscribers do not need modems to transmit digital data. However, they do need another device called a **digital service unit (DSU)**.

Digital Data Service

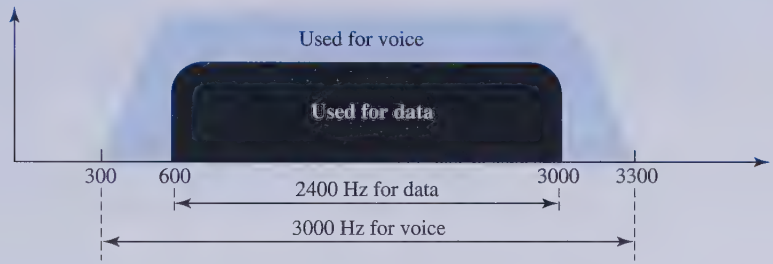
Digital data service (DDS) is the digital version of an analog leased line; it is a digital leased line with a maximum data rate of 64 kbps.

5.1.5 Dial-Up Service

Traditional telephone lines can carry frequencies between 300 and 3300 Hz, giving them a bandwidth of 3000 Hz. This entire range is used for transmitting voice, where a great deal of interference and distortion can be accepted without loss of intelligibility. As we have seen, however, data signals require a higher degree of accuracy to ensure integrity.

For safety's sake, therefore, the edges of this range are not used for data communications. In general, we can say that the signal bandwidth must be smaller than the cable bandwidth. The effective bandwidth of a telephone line being used for data transmission is 2400 Hz, covering the range from 600 to 3000 Hz. Note that today some telephone lines are capable of handling greater bandwidth than traditional lines. However, modem design is still based on traditional capabilities (see Figure 5.6)

Figure 5.6 *Telephone line bandwidth*

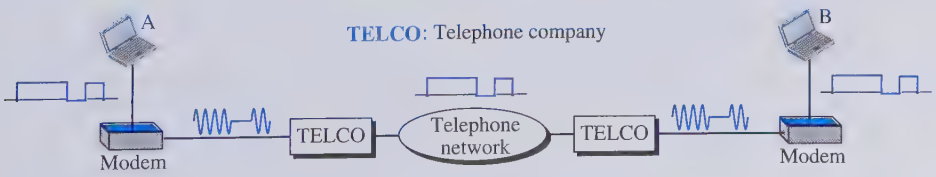


The term **modem** is a composite word that refers to the two functional entities that make up the device: a signal *mo*dulator and a signal *demo*dulator. A **modulator** creates a bandpass analog signal from binary data. A **demodulator** recovers the binary data from the modulated signal.

***Modem* stands for modulator/demodulator.**

Figure 5.7 shows the relationship of modems to a communications link. The computer on the left sends a digital signal to the modulator portion of the modem; the data are sent as an analog signal on the telephone lines. The modem on the right receives the analog signal, demodulates it through its demodulator, and delivers data to the computer on the right. The communication can be bidirectional, which means the computer on the right can simultaneously send data to the computer on the left, using the same modulation/demodulation processes.

Figure 5.7 *Modulation/demodulation*



56K modems

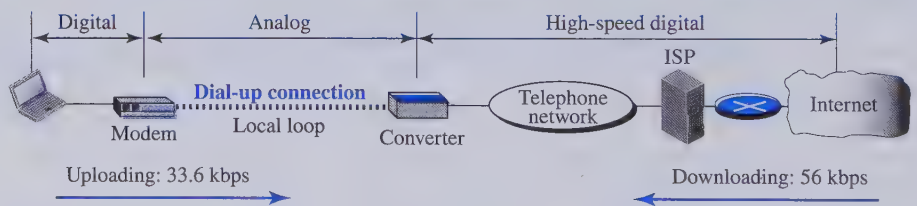
Traditional modems have a data rate limitation of 33.6 kbps, as determined by the Shannon capacity (see Chapter 2). However, modern modems with a bit rate of 56,000 bps are available; these are called **56K modems**. These modems may be used only if one party is using digital signaling (such as through an Internet provider). They are asymmetric in that the downloading rate [flow of data from the Internet service provider to the personal computer (PC)] is a maximum of 56 kbps, while the uploading rate (flow of data from the PC to the Internet provider) can be a maximum of 33.6 kbps. Do these modems violate the Shannon capacity principle? No, in the downstream direction, the SNR ratio is higher because there is no quantization error (see Figure 5.8).

In **uploading**, the analog signal must still be sampled at the switching station. In this direction, noise is introduced into the signal, which reduces the SNR ratio and limits the rate to 33.6 kbps.

However, there is no sampling in the **downloading**. The signal is not affected by quantization noise and is not subject to the Shannon capacity limitation. The maximum data rate in the uploading direction is still 33.6 kbps, but the data rate in the downloading direction is now 56 kbps.

One may wonder how we arrive at the 56-kbps figure. The telephone companies sample 8000 times per second with 8 bits per sample. One of the bits in each sample is used for control purposes, which means each sample is 7 bits. The rate is therefore 8000×7, or 56,000 bps or 56 kbps.

Figure 5.8 *Dial-up network to provide Internet access*



5.1.6 Digital Subscriber Line (DSL)

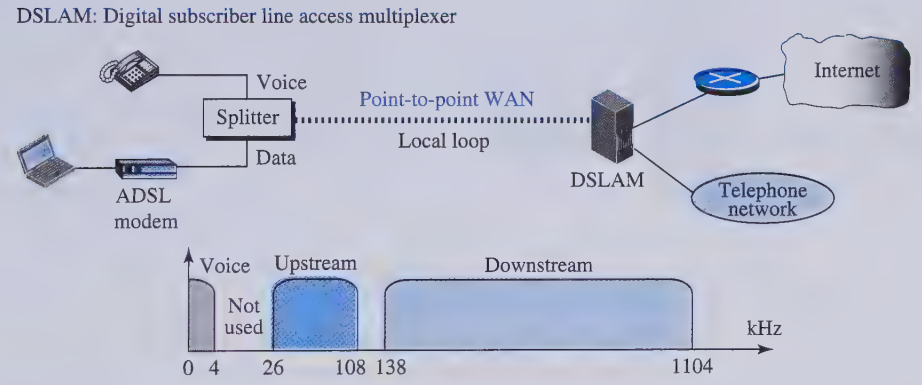
After traditional modems reached their peak data rate, telephone companies developed another technology, DSL, to provide higher-speed access to the Internet. **Digital subscriber line (DSL)** technology is one of the most promising for supporting high-speed digital communications over the existing telephone. DSL technology is a set of technologies, each differing in the first letter (ADSL, VDSL, HDSL, and SDSL). The set is often referred to as *x*DSL, where *x* can be replaced by A, V, H, or S. We will only discuss the first technology in the set. **Asymmetric DSL (ADSL)**, like a 56K modem, provides higher speed (bit rate) in the downstream direction (from the Internet to the resident) than in the upstream direction (from the resident to the Internet). That is the reason it is called asymmetric. Unlike the asymmetry in 56K modems, the designers of ADSL specifically divided the available bandwidth of the local loop unevenly for the

residential customer. The service is not suitable for business customers who need a large bandwidth in both directions.

Using Existing Local Loops

One interesting point is that ADSL uses the existing telephone lines (local loop). But how does ADSL reach a data rate that was never achieved with traditional modems? The answer is that the twisted-pair cable used in telephone lines is actually capable of handling bandwidths up to 1.1 MHz, but the filter installed at the end office of the telephone company where each local loop terminates limits the bandwidth to 4 kHz (sufficient for voice communication). If the filter is removed, however, the entire 1.1 MHz is available for data and voice communications. Typically, an available bandwidth of 1.104 MHz is divided into a voice channel, an upstream channel, and a downstream channel, as shown in Figure 5.9. Note that the ASDL uses a digital subscriber line access multiplexer (DSLAM) that separates data from voice in the destination.

Figure 5.9 *ASDL point-to-point network*



ADSL allows the subscriber to use the voice channel and the data channel at the same time. The rate for the upstream can reach 1.44 Mbps. However, the data rate is normally below 500 kbps because of the high-level noise in this channel. The downstream data rate can reach 13.4 Mbps. However, the data rate is normally below 8 Mbps because of noise in this channel. A very interesting point is that the telephone company in this case serves as the ISP, so services such as e-mail or Internet access are provided by the telephone company itself.

5.2 CABLE NETWORKS

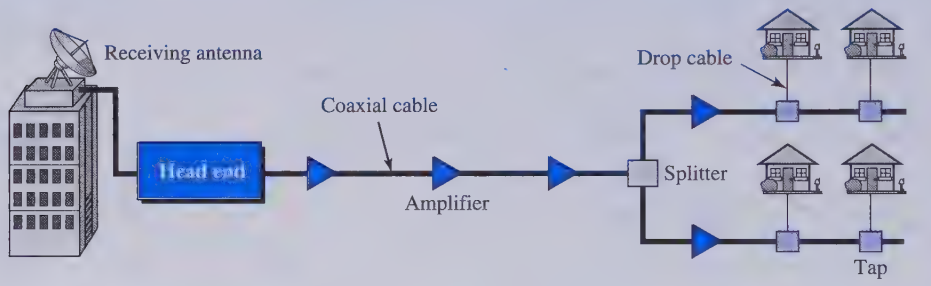
Cable networks were originally created to provide access to TV programs for those subscribers who had no reception because of natural obstructions such as mountains. Later the cable networks became popular with people who just wanted a better signal. In addition, cable networks enabled access to remote broadcasting stations via microwave

connections. Cable TV also found a good market in Internet access provision, using some of the channels originally designed for video. After discussing the basic structure of cable networks, we discuss how cable modems can provide a high-speed connection to the Internet.

5.2.1 Traditional Cable Networks

Cable TV started to distribute broadcast video signals to locations with poor or no reception in the late 1940s. It was called **community antenna TV (CATV)** because an antenna at the top of a tall hill or building received the signals from the TV stations and distributed them, via coaxial cables, to the community. Figure 5.10 shows a schematic diagram of a traditional **cable TV network**.

Figure 5.10 *Traditional cable TV network*



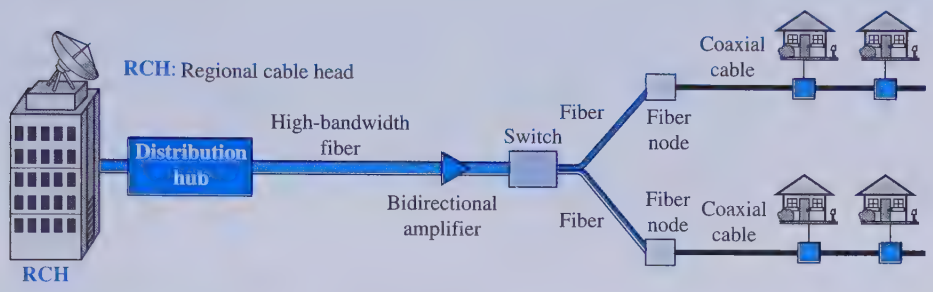
The cable TV office, called the **head end**, receives video signals from broadcasting stations and feeds the signals into coaxial cables. The signals became weaker and weaker with distance, so amplifiers were installed throughout the network to renew the signals. There could be up to 35 amplifiers between the head end and the subscriber premises. At the other end, splitters split the cable, and taps and drop cables make the connections to the subscriber premises.

The traditional cable TV system used coaxial cable end-to-end. Because of attenuation of the signals and the use of a large number of amplifiers, communication in the traditional network was unidirectional (one-way). Video signals were transmitted downstream, from the head end to the subscriber premises.

5.2.2 Hybrid Fiber-Coaxial (HFC) Network

The second generation of cable networks is called a **hybrid fiber-coaxial (HFC) network**. The network uses a combination of fiber-optic and coaxial cable. The transmission medium from the cable TV office to a box, called the **fiber node**, is optical fiber; from the fiber node through the neighborhood and into the house is still coaxial cable. Figure 5.11 shows a schematic diagram of an HFC network.

Figure 5.11 *Hybrid fiber-coaxial (HFC) network*



The **regional cable head (RCH)** normally serves up to 400,000 subscribers. The RCHs feed the **distribution hubs**, each of which serves up to 40,000 subscribers. The distribution hub plays an important role in the new infrastructure. Modulation and distribution of signals are done here; the signals are then fed to the fiber nodes through fiber-optic cables. The fiber node splits the analog signals so that the same signal is sent to each coaxial cable. Each coaxial cable serves up to 1000 subscribers. The use of fiber-optic cable reduces the need for amplifiers down to eight or less.

One reason for moving from traditional to hybrid infrastructure is to make the cable network bidirectional (two-way).

5.2.3 Cable TV for Data Transfer

Cable companies are now competing with telephone companies for the residential customer who wants high-speed data transfer. DSL technology provides high-data-rate connections for residential subscribers over the local loop. However, DSL uses the existing unshielded twisted-pair cable, which is very susceptible to interference. This imposes an upper limit on the data rate. A solution is the use of the cable TV network. In this section, we briefly discuss this technology.

Even in an HFC system, the last part of the network, from the fiber node to the subscriber premises, is still a coaxial cable. This coaxial cable has a bandwidth that ranges from 5 to 750 MHz (approximately). To provide Internet access, the cable company has divided this bandwidth into three bands: **video**, **downstream data**, and **upstream databands**, as shown in Figure 5.12.

Figure 5.12 *Division of coaxial cable band by CATV*

The *video band* occupies frequencies from 54 to 550 MHz. Because each TV channel occupies 6 MHz, this can accommodate more than 80 channels. The downstream data (from the Internet to the subscriber premises) occupies the upper band, from 550 to 750 MHz. This band is also divided into 6-MHz channels.

The upstream data (from the subscriber premises to the Internet) occupies the lower band, from 5 to 42 MHz. This band is also divided into 6-MHz channels. There are 2 bits/baud in QPSK. The standard specifies 1 Hz for each baud; this means that, theoretically, upstream data can be sent at 12 Mbps (2 bits/Hz × 6 MHz). However, the data rate is usually less than 12 Mbps.

Sharing

Both upstream and downstream bands are shared by the subscribers. The upstream data bandwidth is 37 MHz. This means that there are only six 6-MHz channels available in the upstream direction. A subscriber needs to use one channel to send data in the upstream direction. The question is, "How can six channels be shared in an area with 1000, 2000, or even 100,000 subscribers? " The solution is timesharing. The band is divided into channels; these channels must be shared between subscribers in the same neighborhood. The cable provider allocates one channel, statically or dynamically, for a group of subscribers. If one subscriber wants to send data, she or he contends for the channel with others who want access; the subscriber must wait until the channel is available.

We have a similar situation in the downstream direction. The downstream band has 33 channels of 6 MHz. A cable provider probably has more than 33 subscribers; therefore, each channel must be shared between a group of subscribers. However, the situation is different for the downstream direction; here we have a multicasting situation. If there are data for any of the subscribers in the group, the data are sent to that channel. Each subscriber is sent the data. But because each subscriber also has an address registered with the provider; the cable modem for the group matches the address carried with the data to the address assigned by the provider. If the address matches, the data are kept; otherwise, they are discarded.

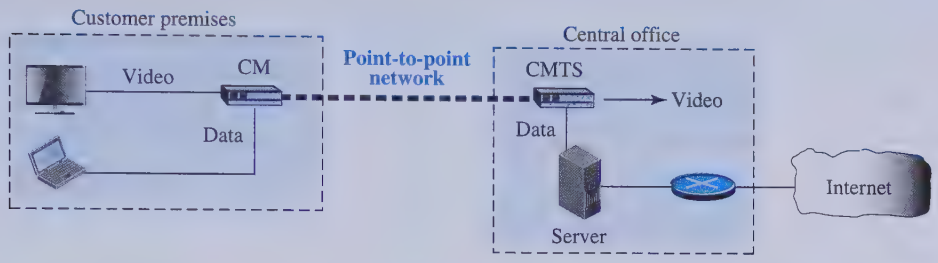
CM and CMTS

To use a cable network for data transmission, we need two key devices: a **cable modem (CM)** and a **cable modem transmission system (CMTS)**. The cable modem is installed on the subscriber premises. The cable modem transmission system is installed inside the cable company. It receives data from the Internet and sends them to the subscriber. The CMTS also receives data from the subscriber and passes them to the Internet. It is similar to an ADSL modem. Figure 5.13 shows the location of these two devices. Like DSL technology, the cable company needs to become an ISP and provide Internet services to the subscriber. At the subscriber premises, the CM separates the video from data and sends them to the television set or the computer.

5.3 CELLULAR TELEPHONY

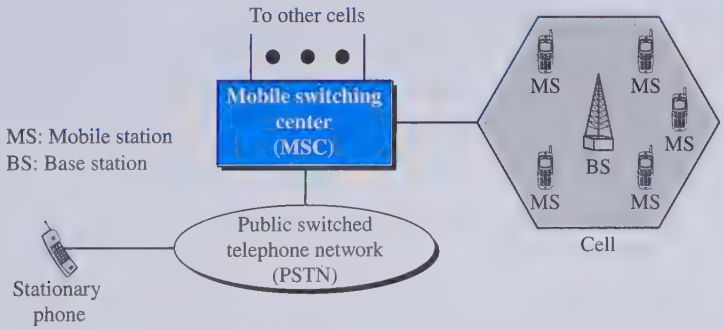
Cellular telephony is designed to provide communications between two moving units, called *mobile stations (MSs)*, or between one mobile unit and one stationary unit, often called a *land unit*. A service provider must be able to locate and track a caller, assign a channel to the call, and transfer the channel from base station to base station as the caller moves out of range.

Figure 5.13 *Cable modem transmission system (CMTS)*



To make this tracking possible, each cellular service area is divided into small regions called *cells*. Each cell contains an antenna and is controlled by a solar- or AC-powered network station, called the *base station (BS)*. Each base station, in turn, is controlled by a switching office, called a **mobile switching center (MSC)**. The MSC coordinates communication between all the base stations and the telephone central office. It is a computerized center that is responsible for connecting calls, recording call information, and billing (see Figure 5.14).

Figure 5.14 *Cellular system*



Cell size is not fixed and can be increased or decreased depending on the population of the area. The typical radius of a cell is 1 to 12 mi. High-density areas require more, geographically smaller cells to meet traffic demands than do low-density areas. Once determined, cell size is optimized to prevent the interference of adjacent cell signals. The transmission power of each cell is kept low to prevent its signal from interfering with those of other cells.

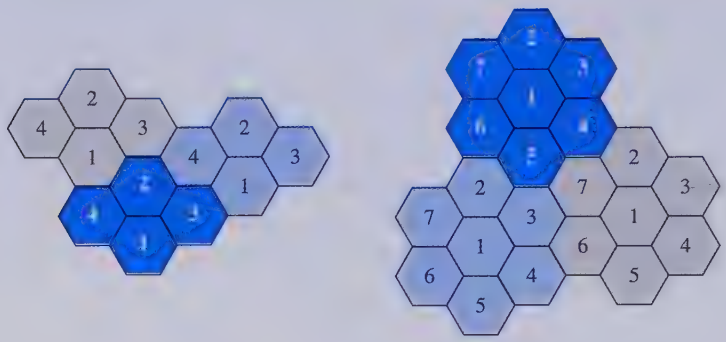
5.3.1 Operation

Let us first briefly discuss the operation of the cellular telephony.

Frequency-Reuse Principle

In general, neighboring cells cannot use the same set of frequencies for communication because it may create interference for the users located near the cell boundaries. However, the set of frequencies available is limited, and frequencies need to be reused. A frequency-reuse pattern is a configuration of N cells, N being the **reuse factor**, in which each cell uses a unique set of frequencies. When the pattern is repeated, the frequencies can be reused. There are several different patterns. Figure 5.15 shows two of them.

Figure 5.15 *Frequenc–reuse patterns*



a. Reuse factor of 4 b. Reuse factor of 7

The numbers in the cells define the pattern. The cells with the same number in a pattern can use the same set of frequencies. We call these cells the *reusing cells*. As Figure 5.15 shows, in a pattern with reuse factor 4, only one cell separates the cells using the same set of frequencies. In the pattern with reuse factor 7, two cells separate the reusing cells.

Transmitting

To place a call from a mobile station, the caller enters a code of 7 or 10 digits (a phone number) and presses the send button. The mobile station then scans the band, seeking a setup channel with a strong signal, and sends the data (phone number) to the closest base station using that channel. The base station relays the data to the MSC. The MSC sends the data on to the telephone central office. If the called party is available, a connection is made and the result is relayed back to the MSC. At this point, the MSC assigns an unused voice channel to the call, and a connection is established. The mobile station automatically adjusts its tuning to the new channel, and communication can begin.

Receiving

When a mobile phone is called, the telephone central office sends the number to the MSC. The MSC searches for the location of the mobile station by sending query signals to each cell in a process called *paging*. Once the mobile station is found, the MSC transmits a ringing signal and, when the mobile station answers, assigns a voice channel to the call, allowing voice communication to begin.

Handoff

It may happen that, during a conversation, the mobile station moves from one cell to another. When it does, the signal may become weak. To solve this problem, the MSC monitors the level of the signal every few seconds. If the strength of the signal diminishes, the MSC seeks a new cell that can better accommodate the communication. The MSC then changes the channel carrying the call (hands the signal off from the old channel to a new one).

Hard Handoff

Early systems used a hard **handoff**. In a hard handoff, a mobile station only communicates with one base station. When the MS moves from one cell to another, communication must first be broken with the previous base station before communication can be established with the new one. This may create a rough transition.

Soft Handoff

New systems use a soft handoff. In this case, a mobile station can communicate with two base stations at the same time. This means that, during handoff, a mobile station may continue with the new base station before breaking off from the old one.

Roaming

One feature of cellular telephony is called **roaming**. Roaming means, in principle, that a user can have access to communication or can be reached where there is coverage. A service provider usually has limited coverage. Neighboring service providers can provide extended coverage through a roaming contract. The situation is similar to snail mail between countries. The charge for delivery of a letter between two countries can be divided upon agreement by the two countries.

5.3.2 First Generation (1G)

Cellular telephony is now in its fourth generation. The first generation was designed for voice communication using analog signals. We discuss one first-generation mobile system used in North America, AMPS.

AMPS

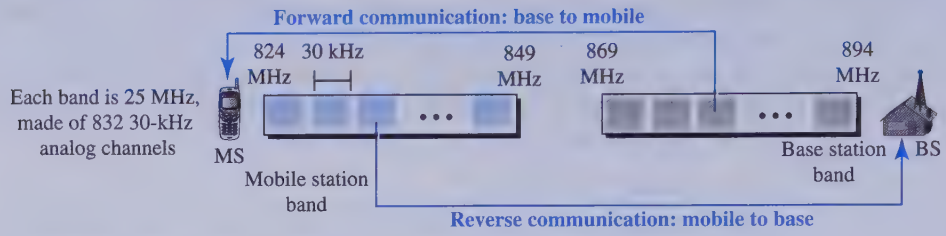
Advanced Mobile Phone System (AMPS) is one of the leading analog cellular systems in North America. It uses FDMA (see Chapter 2) to separate channels in a link.

AMPS is an analog cellular phone system using FDMA.

Bands

AMPS operates in the ISM 800-MHz band. The system uses two separate analog channels, one for forward (base station to mobile station) communication and one for reverse (mobile station to base station) communication. The band between 824 and 849 MHz carries reverse communication; the band between 869 and 894 MHz carries forward communication (see Figure 5.16).

Figure 5.16 *Cellular bands for AMPS*

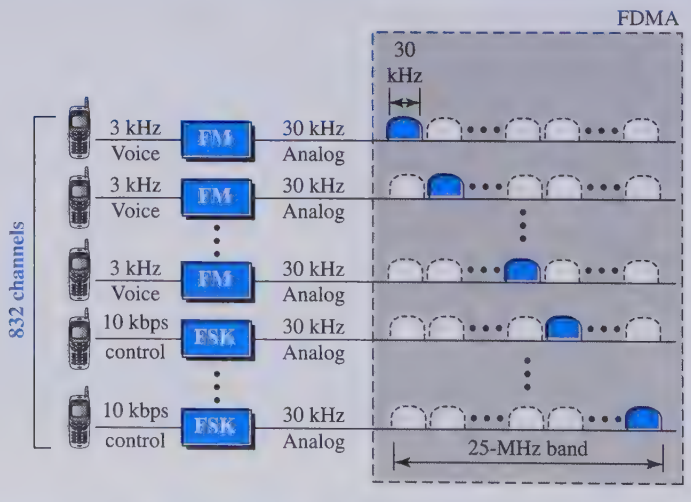


Each band is divided into 832 channels. However, two providers can share an area, which means 416 channels in each cell for each provider. Out of these 416, twenty-one channels are used for control, which leaves 395 channels. AMPS has a frequency reuse factor of 7; this means only one-seventh of these 395 traffic channels are actually available in a cell.

Transmission

AMPS uses FM and FSK for modulation. Figure 5.17 shows the transmission in the reverse direction. Voice channels are modulated using FM, and control channels use FSK to create 30-kHz analog signals. AMPS uses FDMA to divide each 25-MHz band into 30-kHz channels.

Figure 5.17 *AMPS reverse communication band*



5.3.3 Second Generation (2G)

To provide higher-quality (less noise-prone) mobile voice communications, the second generation of the cellular phone network was developed. While the first generation was designed

for analog voice communication, the second generation was mainly designed for digitized voice. Three major systems evolved in the second generation: D-AMPS, GSM, and IS-95.

D-AMPS

The product of the evolution of the analog AMPS into a digital system is **digital AMPS (D-AMPS)**. D-AMPS was designed to be backward-compatible with AMPS. This means that in a cell, one telephone can use AMPS and another D-AMPS. D-AMPS was first defined by IS-54 (Interim Standard 54) and later revised by IS-136.

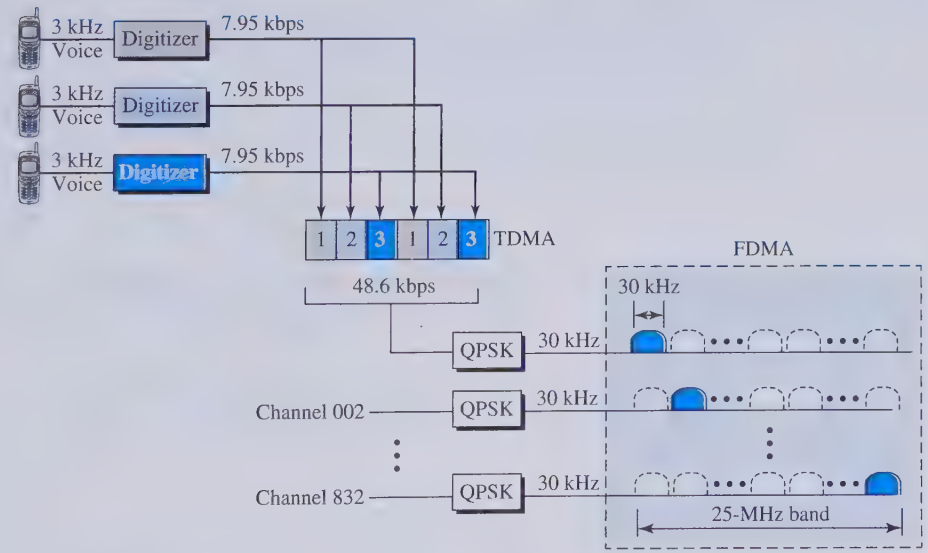
Band

D-AMPS uses the same bands and channels as AMPS.

Transmission

Each voice channel is digitized using a very complex PCM and compression technique. A voice channel is digitized to 7.95 kbps. Three 7.95-kbps digital voice channels are combined using **time-division multiple access (TDMA)**. The result is 48.6 kbps of digital data; much of this is overhead. As Figure 5.18 shows, the system sends 25 frames per second, with 1944 bits per frame. Each frame lasts 40 ms (1/25) and is divided into six slots shared by three digital channels; each channel is allotted two slots.

Figure 5.18 *D-AMPS*



Each slot holds 324 bits. However, only 159 bits come from the digitized voice; 64 bits are for control and 101 bits are for error correction. In other words, each channel drops 159 bits of data into each of the two channels assigned to it. The system adds 64 control bits and 101 error-correcting bits.

The resulting 48.6 kbps of digital data modulates a carrier using QPSK; the result is a 30-kHz analog signal. Finally, the 30-kHz analog signals share a 25-MHz band (FDMA). D-AMPS has a frequency-reuse factor of 7.

D-AMPS, or IS-136, is a digital cellular phone system using TDMA and FDMA.

GSM

The **Global System for Mobile Communication (GSM)** is a European standard that was developed to provide a common second-generation technology for all Europe. The aim was to replace a number of incompatible first-generation technologies.

Bands

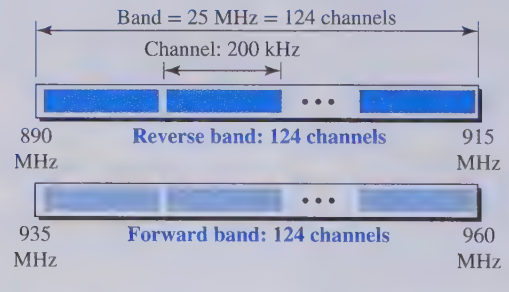
GSM uses two bands for duplex communication. Each band is 25 MHz in width, shifted toward 900 MHz, as shown in Figure 5.19. Each band is divided into 124 channels of 200 kHz separated by guard bands.

Transmission

Figure 5.20 shows a GSM system. Each voice channel is digitized and compressed to a 13-kbps digital signal. Each slot carries 156.25 bits. Eight slots share a frame (TDMA). Twenty-six frames also share a multiframe (TDMA). We can calculate the bit rate of each channel as follows.

$$\text{Channel data rate} = (1/120\ \text{ms}) \times 26 \times 8 \times 156.25 = 270.8\ \text{kbps}$$

Figure 5.19 *GSM bands*



Each 270.8-kbps digital channel modulates a carrier using GMSK (a form of FSK used mainly in European systems); the result is a 200-kHz analog signal. Finally 124 analog channels of 200 kHz are combined using FDMA. The result is a 25-MHz band. Figure 5.21 shows the user data and overhead in a multiframe.

The reader may have noticed the large amount of overhead in TDMA. The user data are only 65 bits per slot. The system adds extra bits for error correction to make it 114 bits per slot. To do this, control bits are added to bring it up to 156.25 bits per slot.

Figure 5.20 *GSM*

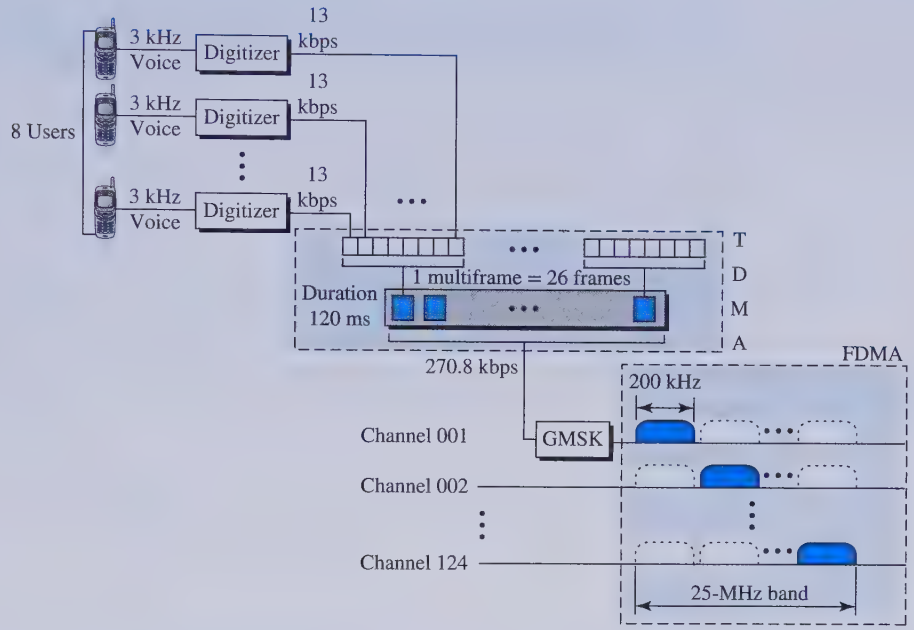
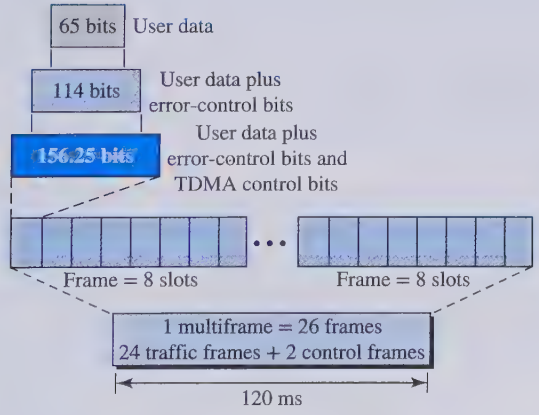


Figure 5.21 *Multiframe components*



Eight slots are encapsulated in a frame. Twenty-four traffic frames and two additional control frames make a multiframe. A multiframe has a duration of 120 ms. However, the architecture does define superframes and hyperframes that do not add any overhead; we will not discuss them here.

Reuse Factor

Because of the complex error-correction mechanism, GSM allows a reuse factor as low as 3.

GSM is a digital cellular phone system using TDMA and FDMA.

IS-95

One of the dominant second-generation standards in North America is **Interim Stand 95 (IS-95)**. It is based on CDMA and DSSS.

Bands and Channels

IS-95 uses two bands for duplex communication. The bands can be the traditional ISM 800-MHz band or the ISM 1900-MHz band. Each band is divided into 20 channels of 1.228 MHz separated by guard bands. Each service provider is allotted 10 channels. IS-95 can be used in parallel with AMPS. Each IS-95 channel is equivalent to 41 AMPS channels (41×30 kHz = 1.23 MHz).

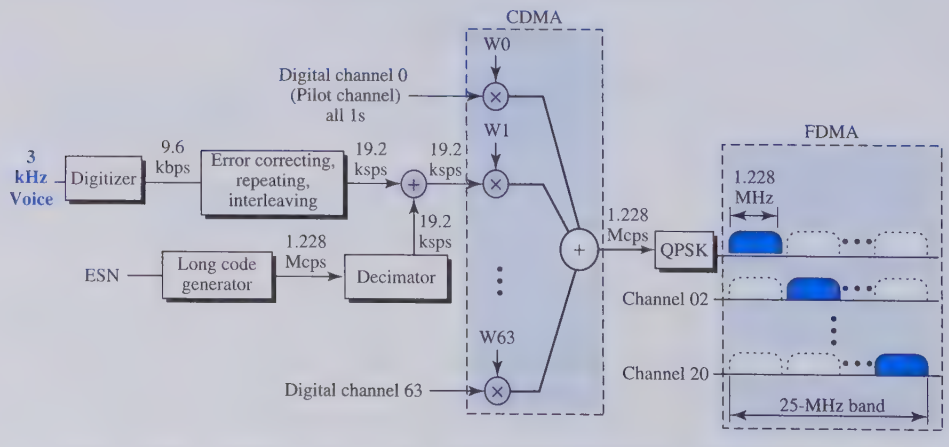
Synchronization

All base channels need to be synchronized to use CDMA. To provide synchronization, bases use the services of the Global Positioning System (GPS), a satellite system that we discuss in Section 5.4.3.

Forward Transmission

IS-95 has two different transmission techniques: one for use in the forward (base to mobile) direction and another for use in the reverse (mobile to base) direction. In the forward direction, communications between the base and all mobiles are synchronized; the base sends synchronized data to all mobiles. Figure 5.22 shows a simplified diagram for the forward direction.

Figure 5.22 *IS-95 forward transmission*



Each voice channel is digitized, producing data at a basic rate of 9.6 kbps. After adding error-correcting and repeating bits, and interleaving, the result is a signal of 19.2 kilo signals per second (ksps). This output is now scrambled using a 19.2-ksps signal. The scrambling signal is produced from a long code generator that uses the electronic serial number (ESN) of the mobile station and generates 2^{42} pseudorandom chips, each chip having 42 bits. Note that the chips are generated pseudorandomly, not randomly, because the pattern repeats itself. The output of the long code generator is fed to a decimator, which chooses 1 bit out of 64 bits. The output of the decimator is used for scrambling. The scrambling is used to create privacy; the ESN is unique for each station.

The result of the scrambler is combined using CDMA. For each traffic channel, one 64×64 row chip is selected. The result is a signal of 1.228 megachips per second (Mcps).

$$\textbf{19.2 ksps} \times \textbf{64 cps} = \textbf{1.228 Mcps}$$

The signal is fed into a QPSK modulator to produce a signal of 1.228 MHz. The resulting bandwidth is shifted appropriately, using FDMA. An analog channel creates 64 digital channels, of which 55 channels are traffic channels (carrying digitized voice). Nine channels are used for control and synchronization:

a. Channel 0 is a pilot channel. This channel sends a continuous stream of 1s to mobile stations. The stream provides bit synchronization, serves as a phase reference for demodulation, and allows the mobile station to compare the signal strength of neighboring bases for handoff decisions.

b. Channel 32 gives information about the system to the mobile station.

c. Channels 1 to 7 are used for paging, to send messages to one or more mobile stations.

d. Channels 8 to 31 and 33 to 63 are traffic channels carrying digitized voice from the base station to the corresponding mobile station.

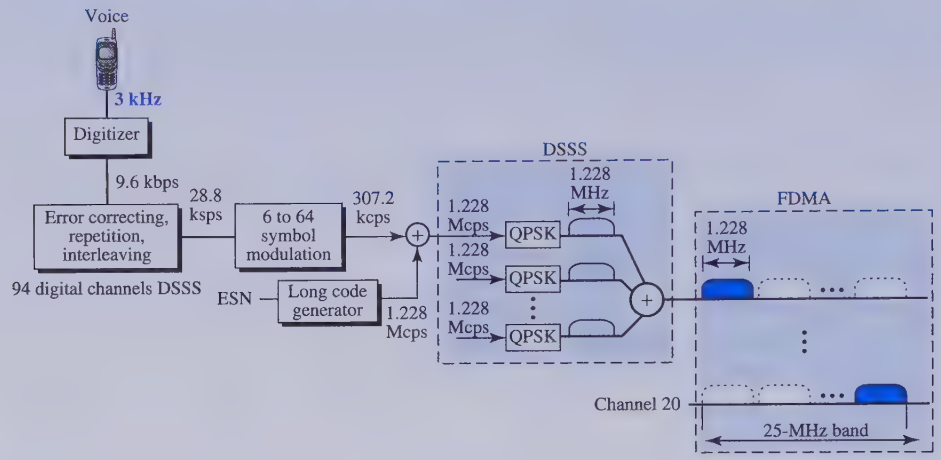
Reverse Transmission

The use of CDMA in the forward direction is possible because the pilot channel sends a continuous sequence of 1s to synchronize transmission. The synchronization is not used in the reverse direction because we need an entity to do that, which is not feasible. Instead of CDMA, the reverse channels use **direct sequence spread spectrum (DSSS)**, which is a technique that replaces each bit with n different bits according to a predefined pattern. Figure 5.23 shows a simplified diagram for reverse transmission.

Each voice channel is digitized, producing data at a rate of 9.6 kbps. However, after adding error-correcting and repeating bits, plus interleaving, the result is a signal of 28.8 ksps. The output is now passed through a 6/64 symbol modulator. The symbols are divided into six-symbol chunks, and each chunk is interpreted as a binary number (from 0 to 63). The binary number is used as the index to a 64×64 matrix for selection of a row of chips. Note that this procedure is not CDMA; each bit is not multiplied by the chips in a row. Each six-symbol chunk is replaced by a 64-chip code. This is done to provide a kind of orthogonality; it differentiates the streams of chips from the different mobile stations. The result creates a signal of 307.2 kcps or $(28.8/6) \times 64$.

Spreading is the next step; each chip is spread into 4. Again the ESN of the mobile station creates a long code of 42 bits at a rate of 1.228 Mcps, which is 4 times 307.2.

Figure 5.23 *IS-95 reverse transmission*



After spreading, each signal is modulated using QPSK, which is slightly different from the one used in the forward direction; we do not go into details here. Note that there is no multiple-access mechanism here; all reverse channels send their analog signal into the air, but the correct chips will be received by the base station due to spreading.

Although we can create $2^{42} - 1$ digital channels in the reverse direction (because of the long code generator), normally 94 channels are used; 62 are traffic channels, and 32 are channels used to gain access to the base station.

IS-95 is a digital cellular phone system using CDMA/DSSS and FDMA.

Two Data Rate Sets

IS-95 defines two data rate sets, with four different rates in each set. The first set defines 9600, 4800, 2400, and 1200 bps. If, for example, the selected rate is 1200 bps, each bit is repeated 8 times to provide a rate of 9600 bps. The second set defines 14,400, 7200, 3600, and 1800 bps. This is possible by reducing the number of bits used for error correction. The bit rates in a set are related to the activity of the channel. If the channel is silent, only 1200 bits can be transferred, which improves the spreading by repeating each bit 8 times.

Frequency-Reuse Factor

In an IS-95 system, the frequency-reuse factor is normally 1 because the interference from neighboring cells cannot affect CDMA or DSSS transmission.

Soft Handoff

Every base station continuously broadcasts signals using its pilot channel. This means a mobile station can detect the pilot signal from its cell and neighboring cells. This enables a mobile station to do a soft handoff in contrast to a hard handoff.

5.3.4 Third Generation (3G)

The third generation of cellular telephony refers to a combination of technologies that provide both digital data and voice communication. Using a small portable device, a person is able to talk to anyone else in the world with a voice quality similar to that of the existing fixed telephone network. A person can download and watch a movie, download and listen to music, surf the Internet or play games, have a video conference, and do much more. One of the interesting characteristics of a third-generation system is that the portable device is always connected; you do not need to dial a number to connect to the Internet.

The third-generation concept started in 1992, when ITU issued a blueprint called the **Internet Mobile Communication 2000 (IMT-2000)**. The blueprint defines some criteria for third-generation technology as outlined here:

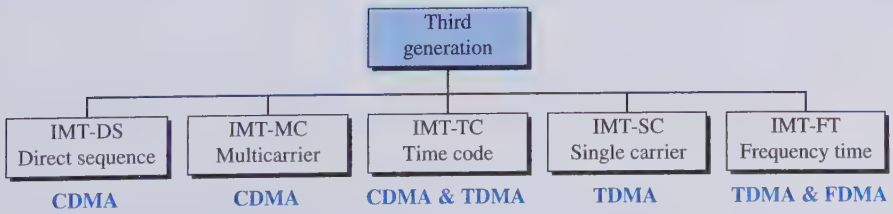
- **a.** Voice quality comparable to that of the existing public telephone network
- **b.** Data rate of 144 kbps for access in a moving vehicle (car), 384 kbps for access as the user walks (pedestrians), and 2 Mbps for the stationary user (office or home)
- **c.** Support for packet-switched and circuit-switched data services
- **d.** A band of 2 GHz
- **e.** Bandwidths of 2 MHz
- **f.** Interface to the Internet

The main goal of third-generation cellular telephony is to provide universal personal communication.

IMT-2000 Radio Interface

Figure 5.24 shows the radio interfaces (wireless standards) adopted by IMT-2000. All five are developed from second-generation technologies. The first two evolve from CDMA technology. The third evolves from a combination of CDMA and TDMA. The fourth evolves from TDMA, and the last evolves from both FDMA and TDMA.

Figure 5.24 *IMT-2000 radio interfaces*



IMT-DS
This approach uses a version of CDMA called wideband CDMA (W-CDMA) that uses a 5-MHz bandwidth. It was developed in Europe and is compatible with the CDMA used in IS-95.

IMT-MC
This approach was developed in North America and is known as CDMA 2000. It is an evolution of CDMA technology used in IS-95 channels. It combines the new wideband (15-MHz) spread spectrum with the narrowband (1.25-MHz) CDMA of IS-95. It is backward-compatible with IS-95. It allows communication on multiple 1.25-MHz channels (1, 3, 6, 9, 12 times), up to 15 MHz. The use of the wider channels allows it to reach the 2-Mbps data rate defined for the third generation.

IMT-TC
This standard uses a combination of W-CDMA and TDMA. The standard tries to reach the IMT-2000 goals by adding TDMA multiplexing to W-CDMA.

IMT-SC
This standard uses only TDMA.

IMT-FT
This standard uses a combination of FDMA and TDMA.

5.3.5 Fourth Generation (4G)

The fourth generation of cellular telephony is a complete evolution in wireless communications. Some of the objectives defined by the 4G working group are as follows:

a. A spectrally efficient system
b. High network capacity
c. Data rate of 100 Mbit/s for access in a moving car and 1 Gbit/s for stationary users
d. Data rate of at least 100 Mbit/s between any two points in the world
e. Smooth handoff across heterogeneous networks
f. Seamless connectivity and global roaming across multiple networks
g. Interoperability with existing wireless standards
h. All IP, packet-switched, networks

The fourth generation is only packet-based (unlike 3G) and supports Internet Protocol version 6 (discussed in Chapter 7). This provides better multicast, security, and route optimization capabilities.

Access Scheme

To increase efficiency, capacity, and scalability, new access techniques are being considered for 4G. For example, **orthogonal FDMA** (**OFDMA**) and **interleaved FDMA** (**IFDMA**) are being considered, respectively, for the downlink and uplink of the next-generation **Universal Mobile Telecommunications System** (**UMTS**). Similarly, **multicarrier code division multiple access** (**MC-CDMA**) is proposed for the IEEE 802.20 standard.

Modulation

More efficient quadrature amplitude modulation (64-QAM) is being proposed for use with the new version of the standard.

Radio System

The fourth generation uses a **Software Defined Radio** (**SDR**) system. Unlike a common radio that uses hardware, the components of an SDR are pieces of software and thus flexible. The SDR can change its program to shift its frequencies to mitigate frequency interference.

Antenna

The **multiple-input multiple-output** (**MIMO**) and **multiuser MIMO** (**MU-MIMO**) **antenna** system, a branch of intelligent antenna, is proposed for 4G. Using this antenna system together with special multiplexing, 4G allows independent streams to be transmitted simultaneously from all the antennas to increase the data rate into multiple folds. MIMO also allows the transmitter and receiver coordinates to move to an open frequency when interference occurs.

Applications

At the present rates of 15–30 Mbit/s, 4G is capable of providing users with streaming high-definition television. At rates of 100 Mbit/s, the content of a DVD can be downloaded within about 5 min for offline access.

5.4 SATELLITE NETWORK

A *satellite network* is a combination of nodes, some of which are satellites, that provides communication from one point on the Earth to another. A node in the network can be a satellite, an Earth station, or an end-user terminal or telephone. Although a natural satellite, such as the moon, can be used as a relaying node in the network, the use of artificial satellites is preferred because we can install electronic equipment on the satellite to regenerate the signal that has lost its energy during travel. Another restriction on using natural satellites is their distances from the Earth, which create a long delay in communication.

Satellite networks are like cellular networks in that they divide the planet into cells. Satellites can provide transmission capability to and from any location on Earth, no matter how remote. This advantage makes high-quality communication available to undeveloped parts of the world without requiring a huge investment in ground-based infrastructure.

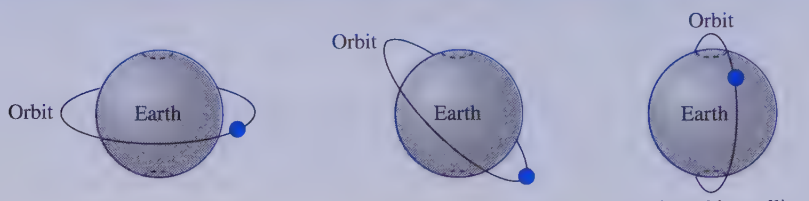
5.4.1 Operation

Let us first discuss some general issues related to the operation of satellites.

Orbits

An artificial satellite needs to have an *orbit*, the path in which it travels around Earth. The orbit can be equatorial, inclined, or polar, as shown in Figure 5.25.

Figure 5.25 *Satellite orbits*



a. Equatorial-orbit satellite b. Inclined-orbit satellite c. Polar-orbit satellite

The period of a satellite, the time required for a satellite to make a complete trip around the Earth, is determined by Kepler's law, which defines the period as a function of the distance of the satellite from the center of the Earth.

Example 5.1

What is the period of the moon, according to Kepler's law?

$$\textbf{Period} = C \times \textbf{distance}^{1.5}$$

Here C is a constant approximately equal to 1/100. The period is in seconds, and the distance is in kilometers.

Solution

The moon is located approximately 384,000 km above the Earth. The radius of the Earth is 6378 km. Applying the formula, we get the following.

$$\textbf{Period} = (1/100) \times (384{,}000 + 6378)^{1.5} = 2{,}439{,}090 \text{ s} = 1 \text{ month}$$

Example 5.2

According to Kepler's law, what is the period of a satellite that is located at an orbit approximately 35,786 km above the Earth?

Solution

Applying the formula, we get the following.

$$\textbf{Period} = (1/100) \times (35{,}786 + 6378)^{1.5} = 86{,}579 \text{ s} = 24 \text{ h}$$

This means that a satellite located at 35,786 km above the earth has a period of 24 h, which is the same as the rotation period of the Earth. A satellite like this is said to be *stationary* to the Earth. The orbit, as we will see, is called a *geostationary orbit*.

Footprint

Satellites process microwaves with bidirectional antennas (line-of-sight). Therefore, the signal from a satellite is normally aimed at a specific area called the footprint. The signal power at the center of the footprint is maximum. The power decreases as we move out from the footprint center. The boundary of the footprint is the location where the power level is at a predefined threshold.

Frequency Bands for Satellite Communication

The frequencies reserved for satellite microwave communication are in the gigahertz (GHz) range. Each satellite sends and receives over two different bands. Transmission from the Earth to the satellite is called the *uplink*. Transmission from the satellite to the Earth is called the *downlink*. Table 5.1 gives the band names and frequencies for each range.

Table 5.1 *Satellite frequency bands*

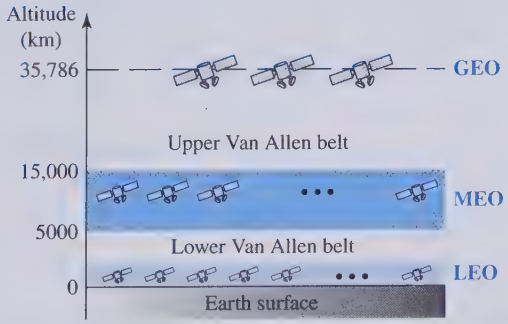
Band	*Downlink (GHz)*	*Uplink (GHz)*	*Bandwidth (MHz)*
L	1.5	1.6	15
S	1.9	2.2	70
C	4.0	6.0	500
Ku	11.0	14.0	500
Ka	20.0	30.0	3500

Three Categories of Satellites

Based on the location of the orbit, satellites can be divided into three categories: **geostationary Earth orbit (GEO)**, **low-Earth-orbit (LEO)**, and **medium-Earth-orbit (MEO)**.

Figure 5.26 shows the satellite altitudes with respect to the surface of the Earth. There is only one orbit, at an altitude of 35,786 km for the GEO satellite. MEO satellites are located at altitudes between 5000 and 15,000 km. LEO satellites are normally below an altitude of 2000 km.

Figure 5.26 *Satellite orbit altitudes*



One reason for having different orbits is the existence of two Van Allen belts. A Van Allen belt is a layer that contains charged particles. A satellite orbiting in one of these two belts would be totally destroyed by the energetic charged particles. The MEO orbits are located between these two belts.

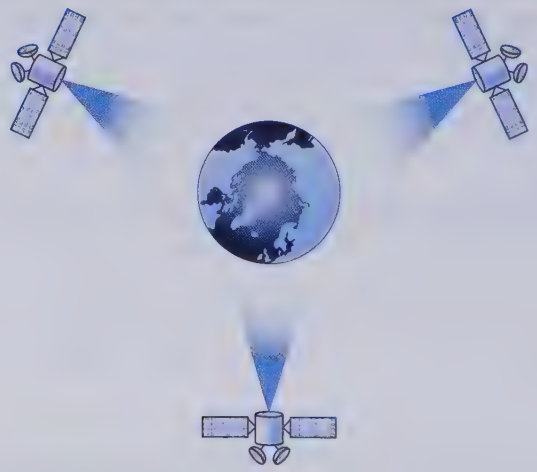
5.4.2 GEO Satellites

Line-of-sight propagation requires that the sending and receiving antennas be locked onto each other's location at all times (one antenna must have the other in sight). For this reason, a satellite that moves faster or slower than the Earth's rotation is useful only for short periods. To ensure constant communication, the satellite must move at the same speed as the Earth so that it seems to remain fixed above a certain spot. Such satellites are called *geostationary.*

Because orbital speed is based on the distance from the planet, only one orbit can be geostationary. This orbit occurs at the equatorial plane and is approximately 22,000 mi above the surface of the Earth.

But one geostationary satellite cannot cover the whole Earth. One satellite in orbit has line-of-sight contact with a vast number of stations, but the curvature of the Earth still keeps much of the planet out of sight. It takes a minimum of three satellites equidistant from each other in geostationary Earth orbit (GEO) to provide full global transmission. Figure 5.27 shows three satellites, each 120° from each other in geosynchronous orbit around the equator. The view is from the North Pole.

Figure 5.27 *Satellites in geostationary orbit*



5.4.3 MEO Satellites

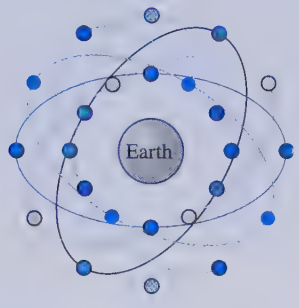
Medium-Earth-orbit (MEO) satellites are positioned between the two Van Allen belts. A satellite at this orbit takes approximately 6 to 8 h to circle the Earth.

Global Positioning System

One example of a MEO satellite system is the **Global Positioning System (GPS)**, contracted and operated by the U.S. Department of Defense, orbiting at an altitude about 18,000 km (11,000 mi) above the Earth. The system consists of 24 satellites and is used

for land, sea, and air navigation to provide time and location for vehicles and ships. GPS uses 24 satellites in six orbits, as shown in Figure 5.28. The orbits and the locations of the satellites in each orbit are designed in such a way that, at any time, four satellites are visible from any point on Earth. A GPS receiver has an almanac that tells the current position of each satellite.

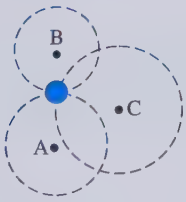
Figure 5.28 *Orbits for Global Positioning System (GPS) satellites*



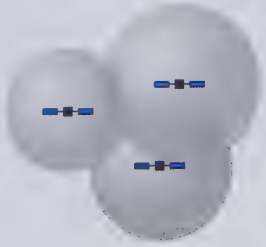
Trilateration

GPS is based on a principle called trilateration. The terms trilateration and triangulation are often used interchangeably, but they have different meanings. We use the word **trilateration**, which means using three distances, instead of **triangulation**, which refer to three different angles. three angles. On a plane, if we know our distance from three points, we know exactly where we are. Let us say that we are 10 mi away from point A, 12 mi away from point B, and 15 mi away from point C. If we draw three circles with the centers at A, B, and C, we must be somewhere on circle A, somewhere on circle B, and somewhere on circle C. These three circles meet at one single point (if our distances are correct); this is our position. Figure 5.29a shows the concept.

Figure 5.29 *Trilateration on a plane*



a. Two-dimensional trilateration



b. Three-dimensional trilateration

In three-dimensional space, the situation is different. Three spheres do not necessarily all meet at one point, as shown in Figure 5.29b. We need at least four spheres to find our exact position in space (longitude, latitude, and altitude). However, if we have additional facts about our location (for example, we know that we are not inside the ocean or somewhere in space), three spheres are enough, because one of the two points, where the spheres meet, is so improbable that the other can be selected without a doubt.

Measuring the distance

The trilateration principle can find our location on the Earth if we know our distance from three satellites and know the position of each satellite. The position of each satellite can be calculated by a GPS receiver (using the predetermined path of the satellites). The GPS receiver, then, needs to find its distance from at least three GPS satellites (center of the spheres). Measuring the distance is done using a principle called one-way **ranging**. For the moment, let us assume that all GPS satellites and the receiver on the Earth are synchronized. Each of 24 satellites synchronously transmits a complex signal, each satellite's signal having a unique pattern. The computer on the receiver measures the delay between the signals from the satellites and its copy of the signals to determine the distances to the satellites.

Synchronization

The preceding discussion was based on the assumption that the satellites' clocks are synchronized with each other and with the receiver's clock. Satellites use atomic clocks, which are precise and can function synchronously with each other. The receiver's clock, however, is a normal quartz clock (an atomic clock costs more than $50,000), and there is no way to synchronize it with the satellite clocks. There is an unknown offset between the satellite clocks and the receiver clock that introduces a corresponding offset in the distance calculation. Because of this offset, the measured distance is called a *pseudorange*.

GPS uses an elegant solution to the clock offset problem, by recognizing that the offset's value is the same for all satellites being used. The calculation of position becomes finding four unknowns: the x_r, y_r, z_r coordinates of the receiver, and the common clock offset dt. For finding these four unknown values, we need at least four equations. This means that we need to measure pseudoranges from four satellites instead of three. If we call the four measured pseudoranges PR_1, PR_2, PR_3, and PR_4 and the coordinates of each satellite x_i, y_i, and z_i (for $i = 1$ to 4), we can find the four previously mentioned unknown values using the following four equations (the four unknown values are shown in color).

$$\mathbf{PR}_1 = [(x_1 - x_r)^2 + (y_1 - y_r)^2 + (z_1 - z_r)^2]^{1/2} + c \times dt$$
$$\mathbf{PR}_2 = [(x_2 - x_r)^2 + (y_2 - y_r)^2 + (z_2 - z_r)^2]^{1/2} + c \times dt$$
$$\mathbf{PR}_3 = [(x_3 - x_r)^2 + (y_3 - y_r)^2 + (z_3 - z_r)^2]^{1/2} + c \times dt$$
$$\mathbf{PR}_4 = [(x_4 - x_r)^2 + (y_4 - y_r)^2 + (z_4 - z_r)^2]^{1/2} + c \times dt$$

The coordinates used in the preceding formulas are in an Earth-Centered Earth-Fixed (ECEF) reference frame, which means that the origin of the coordinate space is at the center of the Earth and the coordinate space rotates with the Earth. This implies that the ECEF coordinates of a fixed point on the surface of the Earth do not change.

Application

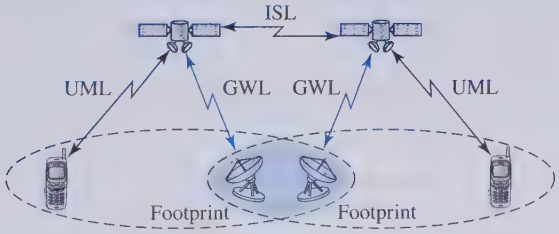
GPS is used by military forces. For example, thousands of portable GPS receivers were used during the Persian Gulf war by foot soldiers, vehicles, and helicopters. Another use of GPS is in navigation. The driver of a car can find the location of the car and then consult a database in the memory of the automobile to be directed to the destination. In other words, GPS gives the location of the car, and the database uses this information to find a path to the destination. A very interesting application is clock synchronization. As we mentioned previously, the IS-95 cellular telephone system uses GPS to create time synchronization between the base stations.

5.4.4 LEO Satellites

Low-Earth-orbit (LEO) satellites have polar orbits. The altitude is between 500 and 2000 km, with a rotation period of 90 to 120 min. The satellite has a speed of 20,000 to 25,000 km/h. A LEO system usually has a cellular type of access, similar to the cellular telephone system. The footprint normally has a diameter of 8000 km. Because LEO satellites are close to the Earth, the round-trip time propagation delay is normally less than 20 ms, which is acceptable for audio communication.

A LEO system is made up of a constellation of satellites that work together as a network; each satellite acts as a switch. Satellites that are close to each other are connected through intersatellite links (ISLs). A mobile system communicates with the satellite through a user mobile link (UML). A satellite can also communicate with an Earth station (gateway) through a gateway link (GWL). Figure 5.30 shows a typical LEO satellite network.

Figure 5.30 *LEO satellite system*



LEO satellites can be divided into three categories: little LEOs, big LEOs, and broadband LEOs. The little LEOs operate under 1 GHz. They are mostly used for low-data-rate messaging. The big LEOs operate between 1 and 3 GHz. **Globalstar** is one of the examples of a big LEO satellite system. It uses 48 satellites in six polar orbits with each orbit hosting 8 satellites. The orbits are located at an altitude of almost 1400 km. Iridium systems are also examples of big LEOs. The **Iridium** system has 66 satellites divided into six orbits, with 11 satellites in each orbit. The orbits are at an altitude of 750 km. The satellites in each orbit are separated from one another by approximately

32° of latitude. The broadband LEOs provide communication similar to fiber-optic networks. The first broadband LEO system was Teledesic. **Teledesic** is a system of satellites that provides fiber-optic-like communication (broadband channels, low error rate, and low delay). Its main purpose is to provide broadband Internet access for users all over the world. It is sometimes called "Internet in the sky." The project was started in 1990 by Craig McCaw and Bill Gates; later, other investors joined the consortium. The project is scheduled to be fully functional in the near future.

5.5 END–OF–CHAPTER MATERIALS

5.5.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items in brackets refer to the reference list at the end of the text.

Books

Several books cover materials discussed in this chapter, including [Sch 03], [Gas 02]. [For 03], [Sta 04], [Sta 02], [Kei 02], [Jam 03], [AZ 03], [Tan 03], [Cou 01], [Com 06], [GW 04], and [PD 03].

5.5.2 Key Terms

56K modem
800 service
900 service
Advanced Mobile Phone System (AMPS)
analog leased service
analog switched service
asymmetric DSL (ADSL)
cable modem (CM)
cable modem transmission system (CMTS)
cellular telephony
common carrier
community antenna TV (CATV)
competitive local exchange carrier (CLEC)
demodulator
digital AMPS (D-AMPS)
digital data service (DDS)
digital service
digital service unit (DSU)
digital subscriber line (DSL)
direct sequence spread spectrum (DSSS)
distribution hub
downloading
downstream data band
end office
fiber node
footprint
geostationary Earth orbit (GEO)
Global Positioning System (GPS)
Global System for Mobile Communication (GSM)
Globalstar
handoff
head end
hybrid fiber-coaxial (HFC) network
in-band signaling
incumbent local exchange carrier (ILEC)
interexchange carrier (IXC)
Interim Standard 95 (IS-95)
Interleaved FDMA (IFDMA)
Internet Mobile Communication 2000 (IMT-2000)
Iridium
ISDN user port (ISUP)
local access transport area (LATA)
local exchange carrier (LEC)
local loop
long-distance company
low-Earth-orbit (LEO)
medium-Earth-orbit (MEO)
message transport port (MTP) level

mobile switching center (MSC)
modem
modulator
multicarrier CDMA (MC-CDMA)
multiple-input multiple-output (MIMO) antenna
multiuser MIMO (MU-MIMO) antenna
orthogonal FDMA (OFDMA)
out-of-band signaling
plain old telephone system (POTS)
point of presence (POP)
ranging
regional cable head (RCH)
regional office
reuse factor
roaming
server control point (SCP)
signal point (SP)
signal transport port (STP)
signaling connection control point (SCCP)
Signaling System Seven (SS7)
Software Defined Radio (SDR)
switched/56 service
switching office
tandem office
Teledesic
telephone user port (TUP)
time-division multiple access (TDMA)
transaction capabilities application port (TCAP)
triangulation
trilateration
trunk
Universal Mobile Telecommunication System (UMTS)
uploading
upstream data band
video band
wide area telephone service (WATS)

5.5.3 Summary

The telephone network was originally an analog system. During the last decade, the telephone network has undergone many technical changes. The network is now digital as well as analog. The telephone network is made up of three major components: local loops, trunks, and switching offices. Telephone companies provide two types of services: analog and digital. We can categorize analog services as either analog switched services or analog leased services. The two most common digital services are switched/56 service and digital data service (DDS). Data transfer using the telephone local loop was traditionally done using a dial-up modem. The term *modem* is a composite word that refers to the two functional entities that make up the device: a signal modulator and a signal demodulator. Most popular modems available are based on 56K modems. Telephone companies developed another technology, digital subscriber line (DSL), to provide higher-speed access to the Internet. DSL technology is a set of technologies, but we discussed only the common one, ADSL.

Community antenna TV (CATV) was originally designed to provide video services for the community. The traditional cable TV system used coaxial cable end-to-end. The second generation of cable networks is called a hybrid fiber-coaxial (HFC) network. The network uses a combination of fiber-optic and coaxial cable. Cable companies are now competing with telephone companies for the residential customer who wants high-speed access to the Internet. To use a cable network for data transmission, we need two key devices: a cable modem (CM) and a cable modem transmission system (CMTS).

Cellular telephony provides communication between two devices. One or both may be mobile. A cellular service area is divided into cells. Advanced Mobile Phone System (AMPS) is a first-generation cellular phone system. Digital AMPS (D-AMPS) is

a second-generation cellular phone system that is a digital version of AMPS. Global System for Mobile Communication (GSM) is a second-generation cellular phone system used in Europe. Interim Standard 95 (IS-95) is a second-generation cellular phone system based on CDMA and DSSS. The third-generation cellular phone system provides universal personal communication. The fourth generation is the new generation of cellular phones that are becoming popular.

A satellite network uses satellites to provide communication between any points on Earth. A geostationary Earth orbit (GEO) is at the equatorial plane and revolves in phase with Earth's rotation. Global Positioning System (GPS) satellites are medium-Earth-orbit (MEO) satellites that provide time and location information for vehicles and ships. Iridium satellites are low-Earth-orbit (LEO) satellites that provide direct universal voice and data communications for handheld terminals. Teledesic satellites are low-Earth-orbit satellites that will provide universal broadband Internet access.

5.6 PRACTICE SET

5.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

5.6.2 Questions

Q5-1. What are the three major components of a telephone network?

Q5-2. What is LATA? What are intra-LATA and inter-LATA services?

Q5-3. Describe the SS7 service and its relation to the telephone network.

Q5-4. What are the two major services provided by telephone companies in the United States?

Q5-5. What is dial-up modem technology? List some of the common modem standards discussed in this chapter, and give their data rates.

Q5-6. What is DSL technology? What are the services provided by the telephone companies using DSL?

Q5-7. Compare and contrast a traditional cable network with a hybrid fiber-coaxial network.

Q5-8. How is data transfer achieved using CATV channels?

Q5-9. Distinguish between CM and CMTS.

Q5-10. Why is multiplexing more efficient if all the data units are the same size?

Q5-11. What is the relationship between TPs, VPs, and VCs?

Q5-12. What is the relationship between a base station and a mobile switching center?

Q5-13. What are the functions of a mobile switching center?

Q5-14. Which is better, a low reuse factor or a high reuse factor? Explain your answer.

Q5-15. What is AMPS?

Q5-16. What is the relationship between D-AMPS and AMPS?

Q5-17. What is GSM?
Q5-18. What is the function of the CDMA in IS-95?
Q5-19. What are the three types of orbits?
Q5-20. Which type of orbit does a GEO satellite have? Explain your answer.
Q5-21. What is a footprint?
Q5-22. What is the relationship between the Van Allen belts and satellites?
Q5-23. Compare an uplink with a downlink.
Q5-24. What is the purpose of GPS?
Q5-25. What is the main difference between Iridium and Globalstar?

5.6.3 Problems

P5-1. When we have an overseas telephone conversation, we sometimes experience a delay. Can you explain the reason?
P5-2. Draw a barchart to compare the different downloading data rates of common modems.
P5-3. Draw a barchart to compare the different downloading data rates of common DSL technology implementations (use minimum data rates).
P5-4. Calculate the minimum time required to download one million bytes of information using a 56K modem.
P5-5. What type of topology is used when customers in an area use DSL modems for data transfer purposes? Explain.
P5-6. What type of topology is used when customers in an area use cable modems for data transfer purposes? Explain.
P5-7. What type of topology is used when customers in an area use DSL modems for data transfer purposes? Explain.
P5-8. Draw a cell pattern with a frequency-reuse factor of 5.
P5-9. Draw a cell pattern with a frequency-reuse factor of 3.
P5-10. What is the maximum number of callers in each cell in AMPS?
P5-11. What is the maximum number of simultaneous calls in each cell in an IS-136 (D-AMPS) system, assuming no analog control channels?
P5-12. What is the maximum number of simultaneous calls in each cell in a GSM assuming no analog control channels?
P5-13. What is the maximum number of callers in each cell in an IS-95 system?
P5-14. Find the efficiency of AMPS in terms of simultaneous calls per megahertz of bandwidth. In other words, find the number of calls that can be used in 1-MHz bandwidth allocation.
P5-15. Guess the relationship between a 3-kHz voice channel and a 30-kHz modulated channel in a system using AMPS.
P5-16. How many slots are sent each second in a channel using D-AMPS? How many slots are sent by each user in 1 s?
P5-17. Use Kepler's formula to check the accuracy of a given period and altitude for a GPS satellite.
P5-18. Use Kepler's formula to check the accuracy of a given period and altitude for an Iridium satellite.

P5-19. Use Kepler's formula to check the accuracy of a given period and altitude for a Globalstar satellite.

P5-20. Find the efficiency of the AMPS protocol in terms of simultaneous calls per megahertz of bandwidth. In other words, find the number of calls that can be made in a 1-MHz bandwidth allocation.

P5-21. Find the efficiency of the D-AMPS protocol in terms of simultaneous calls per megahertz of bandwidth. In other words, find the number of calls that can be made in a 1-MHz bandwidth allocation.

P5-22. Find the efficiency of the GSM protocol in terms of simultaneous calls per megahertz of bandwidth. In other words, find the number of calls that can be made in a 1-MHz bandwidth allocation.

P5-23. Find the efficiency of the IS-95 protocol in terms of simultaneous calls per megahertz of bandwidth. In other words, find the number of calls that can be made in a 1-MHz bandwidth allocation.

CHAPTER 6

Connecting Devices and Virtual LANs

Devices in the Internet are connected together using connecting devices. Connecting devices can operate in different layers of the Internet model. After discussing some connecting devices, we show how they are used to create virtual local area networks (VLANs).

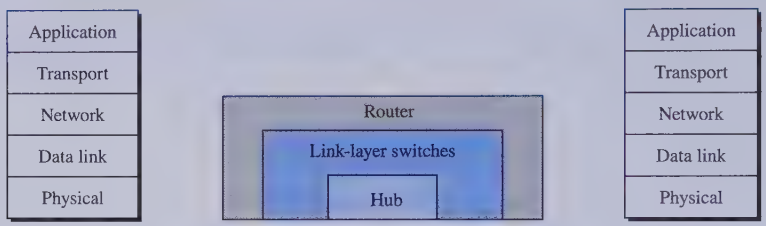
This chapter is divided into two sections.

- The first section discusses connecting devices. It first describes hubs and their features. Then it discusses link-layer switches (or simply switches as they are called) and shows how they can create loops if they are connected LANs with broadcast domain. This section also briefly discusses routers that are being used in the network layer discussed in Chapters 7 and 8.
- The second section discusses virtual LANs or VLANs. It first shows how membership in a VLAN can be defined. Then it discusses the VLAN configuration. Next it shows how switches can communicate in a VLAN. Finally, the section mentions the advantages of a VLAN.

6.1 CONNECTING DEVICES

Hosts and networks do not normally operate in isolation. We use **connecting devices** to connect hosts together to make a network or to connect networks together to make an internet. Connecting devices can operate in different layers of the TCP/IP protocol suite. We discuss three kinds of *connecting devices*: hubs, link-layer switches, and routers. Hubs today operate in the first layer of the Internet model. Link-layer switches operate in the first two layers. Routers operate in the first three layers (Figure 6.1).

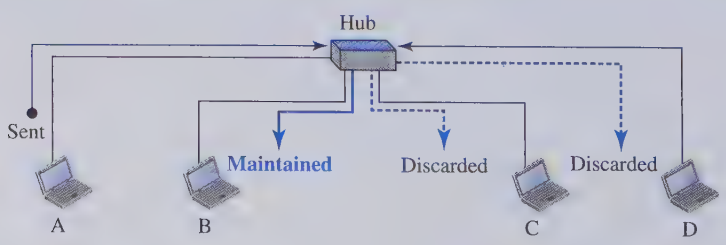
Figure 6.1 *Three categories of connecting devices*



6.1.1 Hubs

A **hub** is a device that operates only in the physical layer. Signals that carry information within a network can travel a fixed distance before attenuation endangers the integrity of the data. A **repeater** receives a signal and, before it becomes too weak or corrupted, *regenerates* and *retimes* the original bit pattern. The repeater then sends the refreshed signal. In the past, when Ethernet LANs were using bus topology, a repeater was used to connect two segments of a LAN to overcome the length restriction of the coaxial cable. Today, however, Ethernet LANs use star topology. In a star topology, a repeater is a multiport device, often called a *hub*, that can be used to serve as the connecting point and at the same time function as a repeater. Figure 6.2 shows that when a packet from station A to station B arrives at the hub, the signal representing the frame is

Figure 6.2 *A hub*



regenerated to remove any possible corrupting noise, but the hub forwards the packet from all outgoing ports except the one from which the signal has been received. In other words, the frame is broadcast. All stations in the LAN receive the frame, but only station B keeps it. The rest of the stations discard it. Figure 6.2 shows the role of a repeater or a hub in a switched LAN.

Figure 6.2 definitely shows that a hub does not have a filtering capability; it does not have the intelligence to find from which port the frame should be sent out.

A repeater has no filtering capability.

A hub or a repeater is a physical-layer device. These devices do not have a link-layer address, and they do not check the link-layer address of the received frame. They just regenerate the corrupted bits and send them out from every port.

6.1.2 Link-Layer Switches

A **link-layer switch** operates in both the physical and the data-link layers. As a physical-layer device, it regenerates the signal it receives. As a link-layer device, the link-layer switch can check the MAC addresses (source and destination) contained in the frame.

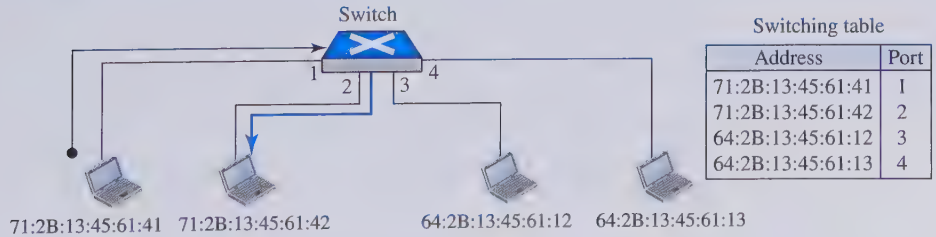
Filtering

One may ask what is the difference in functionality between a link-layer switch and a hub. A link-layer switch has **filtering** capability. It can check the destination link-layer address of a frame and can decide from which outgoing port the frame should be sent.

A link-layer switch has a table used in filtering decisions.

Let us give an example. In Figure 6.3, we have a LAN with four stations that are connected to a link-layer switch. If a frame destined for station 71:2B:13:45:61:42 arrives at port 1, the link-layer switch consults its table to find the departing port.

Figure 6.3 *Link-layer switch*



According to its table, frames for 71:2B:13:45:61:42 should be sent out only through port 2; therefore, there is no need to forward the frame through other ports.

A link-layer switch does not change the link-layer (MAC) addresses in a frame.

Transparent Switches

A **transparent switch** is a switch in which the stations are completely unaware of the switch's existence. If a switch is added or deleted from the system, reconfiguration of the stations is unnecessary. According to the IEEE 802.1d specification, a system equipped with transparent switches must meet three criteria:

1. Frames must be forwarded from one station to another.
2. The forwarding table is automatically made by learning frame movements in the network.
3. Loops in the system must be prevented.

Forwarding

A transparent switch must correctly forward the frames, as discussed in Section 6.1.1.

Learning

The earliest switches had switching tables that were static. The system administrator would manually enter each table entry during switch setup. Although the process was simple, it was not practical. If a station was added or deleted, the table had to be modified manually. The same was true if a station's MAC address changed, which is not a rare event. For example, putting in a new network card means a new MAC address.

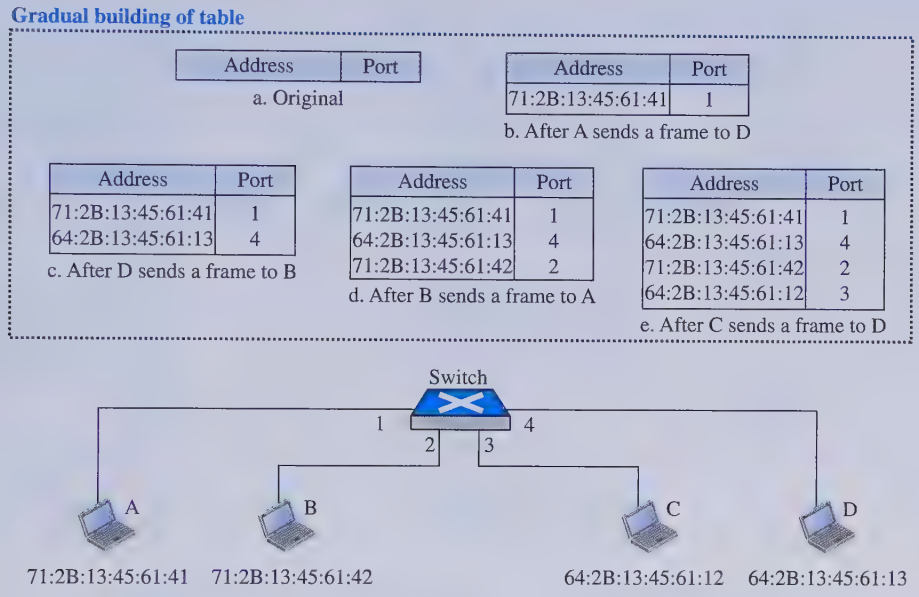
A better solution to the static table is a dynamic table that maps addresses to ports (interfaces) automatically. To make a table dynamic, we need a switch that gradually learns from the frame movements. To do this, the switch inspects both the destination and the source addresses. The destination address is used for the forwarding decision (table lookup); the source address is used for adding entries to the table and for updating purposes. Let us elaborate on this process using Figure 6.4.

1. When station A sends a frame to station D, the switch does not have an entry for either D or A. The frame goes out from all three ports; the frame floods the network. However, by looking at the source address, the switch learns that station A must be connected to port 1. This means that frames destined for A, in the future, must be sent out through port 1. The switch adds this entry to its table. The table has its first entry now.
2. When station D sends a frame to station B, the switch has no entry for B, so it floods the network again. However, it adds one more entry to the table related to station D.
3. The learning process continues until the table has information about every port. However, note that the learning process may take a long time. For example, if a station does not send out a frame (a rare situation), the station will never have an entry in the table.

Loop Problem

Transparent switches work fine as long as there are no redundant switches in the system. Systems administrators, however, like to have redundant switches (more than one switch between a pair of LANs) to make the system more reliable. If a switch fails, another

Figure 6.4 *Learning switch*

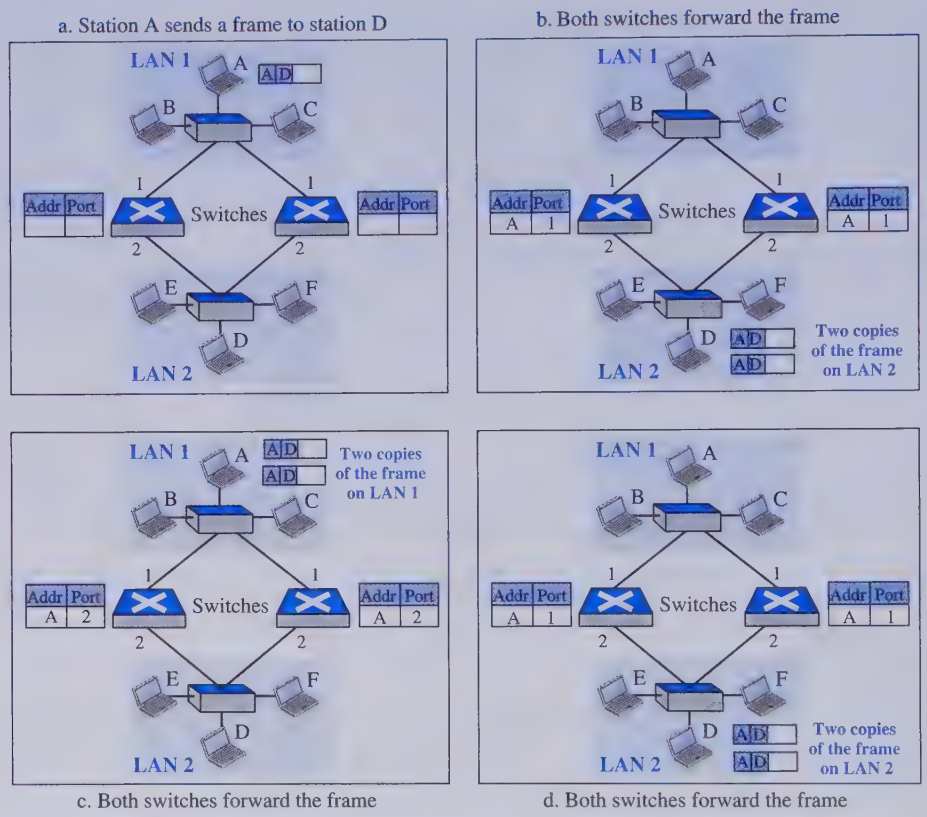


switch takes over until the failed one is repaired or replaced. Redundancy can create loops in the system, which is very undesirable. Loops can be created only when two or more broadcasting LANs (those using hubs, for example) are connected by more than one switch.

Figure 6.5 shows a very simple example of a loop created in a system with two LANs connected by two switches.

1. Station A sends a frame to station D. The tables of both switches are empty. Both forward the frame and update their tables based on the source address A.
2. Now there are two copies of the frame on LAN 2. The copy sent out by the left switch is received by the right switch, which does not have any information about the destination address D; it forwards the frame. The copy sent out by the right switch is received by the left switch and is sent out for lack of information about D. Note that each frame is handled separately because switches, as two nodes on a broadcast network sharing the medium, use an access method such as CSMA/CD. The tables of both switches are updated, but still there is no information for destination D.
3. Now there are two copies of the frame on LAN 1. Step 2 is repeated, and both copies are sent to LAN 2.
4. The process continues on and on. Note that switches are also repeaters and regenerate frames. So in each iteration, there are newly generated fresh copies of the frames.

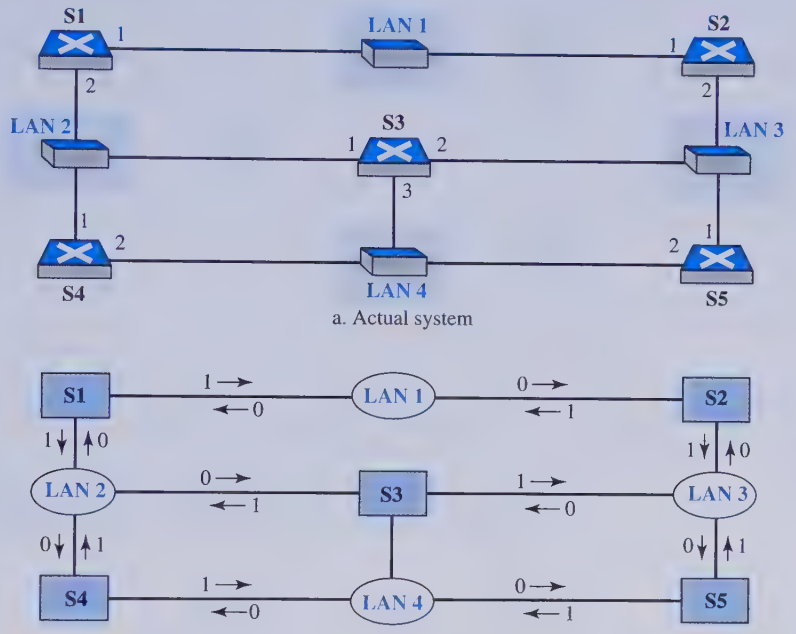
Figure 6.5 *Loop problem in a learning switch*



Spanning Tree Algorithm

To solve the looping problem, the IEEE specification requires that switches use the spanning tree algorithm to create a loopless topology. In graph theory, a **spanning tree** is a graph in which there is no loop. In a switched LAN, this means creating a topology in which each LAN can be reached from any other LAN through one path. We cannot change the physical topology of the system because of physical connections between cables and switches, but we can create a logical topology that overlays the physical one. Figure 6.6 shows a system with four LANs and five switches. We have shown the physical system and its representation in graph theory. Although some textbooks represent the LANs as nodes and the switches as the connecting arcs, we have shown both LANs and switches as nodes. The connecting arcs show the connection of a LAN to a switch and vice versa. To find the spanning tree, we need to assign a cost (metric) to each arc. The interpretation of the cost is left up to the systems administrator. We have chosen the minimum hops.

Figure 6.6 *A system of connected LANs and its graph representation*



The process to find the spanning tree involves four steps:

1. Every switch has a built-in ID (normally the serial number, which is unique). Each switch broadcasts this ID so that all switches know which one has the smallest ID. The switch with the smallest ID is selected as the *root* switch (root of the tree). We assume that switch S1 has the smallest ID. It is, therefore, selected as the root switch.
2. The algorithm tries to find the shortest path (a path with the shortest cost) from the root switch to every other switch or LAN. The shortest path can be found by examining the total cost from the root switch to the destination. Figure 6.7 shows the shortest paths. We have used the Dijkstra algorithm described in Chapter 8.
3. The combination of the shortest paths creates the shortest tree, which is also shown in Figure 6.7.
4. Based on the spanning tree, we mark the ports that are part of it, the **forwarding ports**, which forward a frame that the switch receives. We also mark those ports that are not part of the spanning tree, the **blocking ports**, which block the frames received by the switch. Figure 6.8 shows the logical systems of LANs with forwarding points (solid lines) and blocking ports (broken lines).

Note that there is only one single path from any LAN to any other LAN in the spanning tree system. This means there is only one single path from one LAN to any other LAN. No loops are created. You can prove to yourself that there is only one path from

Figure 6.7 *Finding the shortest paths and the spanning tree in a system of switches*

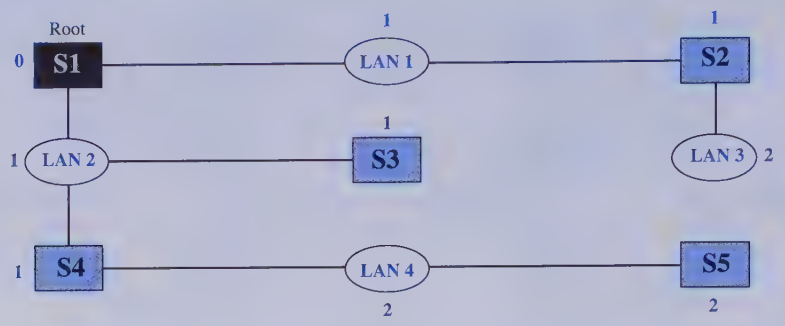
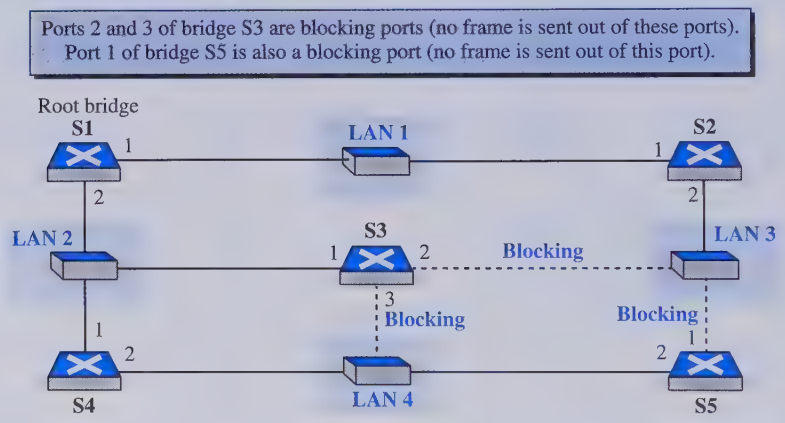


Figure 6.8 *Forwarding and blocking ports after using spanning tree algorithm*



LAN 1 to LAN 2, LAN 3, or LAN 4. Similarly, there is only one path from LAN 2 to LAN 1, LAN 3, and LAN 4. The same is true for LAN 3 and LAN 4.

We have described the spanning tree algorithm as though it required manual entries. This is not true. Each switch is equipped with a software package that carries out this process dynamically.

Advantages of Switches

A link-layer switch has several advantages over a hub. We discuss only two of them here.

Collision Elimination

As we mentioned earlier in this chapter, a link-layer switch eliminates the collision. This means increasing the average bandwidth available to a host in the network. In a switched LAN, there is no need for carrier sensing and collision detection; each host can transmit at any time.

Connecting Heterogenous Devices

A link-layer switch can connect devices that use different protocols at the physical layer (data rates) and different transmission media. As long as the format of the frame at the data-link layer does not change, a switch can receive a frame from a device that uses twisted-pair cable and sends data at 10 Mbps and deliver the frame to another device that uses fiber-optic cable and can receive data at 100 Mbps.

6.1.3 Routers

We will discuss routers in Chapter 8 when we discuss the network layer. In this section, we mention routers to compare them with a two-layer switch and a hub. A **router** is a three-layer device; it operates in the physical, data-link, and network layers. As a physical-layer device, it regenerates the signal it receives. As a link-layer device, the router checks the physical addresses (source and destination) contained in the packet. As a network-layer device, a router checks the network-layer addresses.

A router is a three-layer (physical, data-link, and network) device.

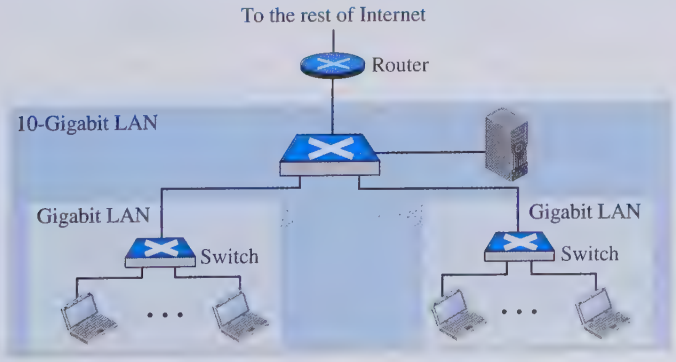
A router can connect networks. In other words, a router is an internet-working device; it connects independent networks to form an internetwork. According to this definition, two networks connected by a router become an internetwork or an internet.

There are three major differences between a router and a repeater or a switch.

1. A router has a link-layer and network-layer address for each of its interfaces.
2. A router acts only on those packets in which the link-layer destination address matches the address of the interface at which the packet arrives.
3. A router changes the link-layer address of the packet (both source and destination) when it forwards the packet.

Let us give an example. In Figure 6.9, assume an organization has two separate buildings with a Gigabit Ethernet LAN installed in each building. The organization uses switches in each LAN. The two LANs can be connected to form a larger LAN using 10-Gigabit

Figure 6.9 *Routing example*



Ethernet technology that speeds up the connection to the Ethernet and the connection to the organization server. A router then can connect the whole system to the Internet.

A router will change the MAC address it receives because the MAC addresses have only local jurisdictions.

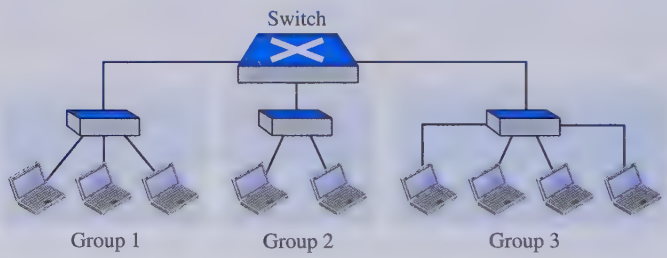
A router changes the link-layer addresses in a packet.

6.2 VIRTUAL LANS

A station is considered part of a LAN if it physically belongs to that LAN. The criterion of membership is geographic. What happens if we need a virtual connection between two stations belonging to two different physical LANs? We can roughly define a **virtual local area network (VLAN)** as a local area network configured by software, not by physical wiring.

Let us use an example to elaborate on this definition. Figure 6.10 shows a switched LAN in an engineering firm in which 10 stations are grouped into three LANs that are connected by a switch.

Figure 6.10 *A switch connecting three LANs*

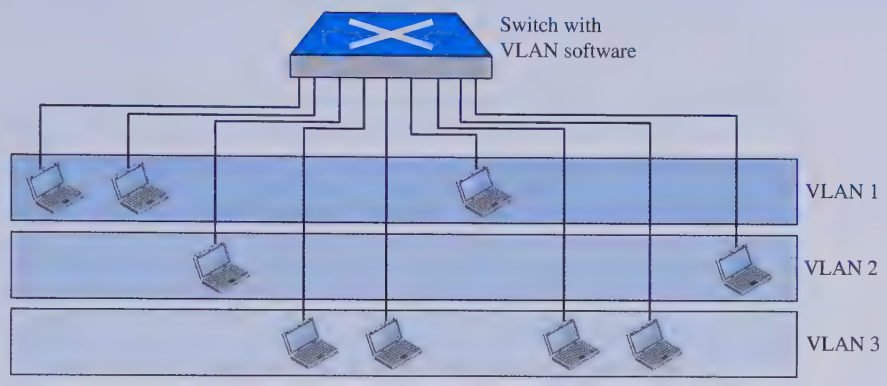


The first three engineers work together as the first group, the next two engineers work together as the second group, and the last four engineers work together as the third group. The LAN is configured to allow this arrangement.

But what would happen if the administrators needed to move two engineers from the first group to the third group, to speed up the project being done by the third group? The LAN configuration would need to be changed. The network of technicians must rewire. The problem is repeated if, in another week, the two engineers move back to their previous group. In a switched LAN, changes in the work group mean physical changes in the network configuration.

Figure 6.11 shows the same switched LAN divided into VLANs. The whole idea of VLAN technology is to divide a LAN into logical, instead of physical, segments. A LAN can be divided into several logical LANs, called VLANs. Each VLAN is a work group in the organization. If a person moves from one group to another, there is no need to change the physical configuration. The group membership in VLANs

Figure 6.11 *A switch using VLAN software*

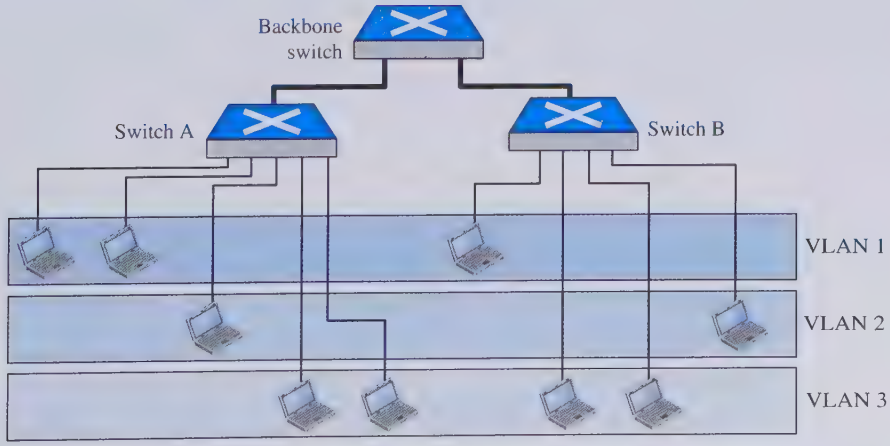


is defined by software, not hardware. Any station can be logically moved to another VLAN. All members belonging to a VLAN can receive broadcast messages sent to that particular VLAN. This means if a station moves from VLAN 1 to VLAN 2, it receives broadcast messages sent to VLAN 2, but no longer receives broadcast messages sent to VLAN 1.

It is obvious that the problem in our previous example can easily be solved by using VLANs. Moving engineers from one group to another through software is easier than changing the configuration of the physical network.

VLAN technology even allows the grouping of stations connected to different switches in a VLAN. Figure 6.12 shows a backbone local area network with two switches and three VLANs. Stations from switches A and B belong to each VLAN.

Figure 6.12 *Two switches in a backbone using VLAN software*



This is a good configuration for a company with two separate buildings. Each building can have its own switched LAN connected by a backbone. People in the first building and people in the second building can be in the same work group even though they are connected to different physical LANs.

From these three examples, we can see that a VLAN defines broadcast domains. VLANs group stations belonging to one or more physical LANs into broadcast domains. The stations in a VLAN communicate with one another as though they belonged to a physical segment.

6.2.1 Membership

What characteristic can be used to group stations in a VLAN? Vendors use different characteristics such as interface number, MAC addresses or a combination of two or more of these.

Interface Numbers

Some VLAN vendors use switch interface numbers as a membership characteristic. For example, the administrator can define that stations connecting to ports 1, 2, 3, and 7 belong to VLAN 1; stations connecting to ports 4, 10, and 12 belong to VLAN 2; and so on.

MAC Addresses

Some VLAN vendors use the 48-bit MAC address as a membership characteristic. For example, the administrator can stipulate that stations having MAC addresses E2:13:42:A1:23:34 and F2:A1:23:BC:D3:41 belong to VLAN 1.

Combination

Recently, the software available from some vendors allows all these characteristics to be combined. The administrator can choose one or more characteristics when installing the software. In addition, the software can be reconfigured to change the settings.

6.2.2 Configuration

How are the stations grouped into different VLANs? Stations are configured in one of three ways: manual, automatic, and semiautomatic.

Manual Configuration

In a manual configuration, the network administrator uses the VLAN software to manually assign the stations into different VLANs at setup. Later migration from one VLAN to another is also done manually. Note that this is not a physical configuration; it is a logical configuration. The term *manually* here means that the administrator types the port numbers, the IP addresses, or other characteristics, using the VLAN software.

Automatic Configuration

In an automatic configuration, the stations are automatically connected or disconnected from a VLAN using criteria defined by the administrator. For example, the

administrator can define the project number as the criterion for being a member of a group. When a user changes projects, he or she automatically migrates to a new VLAN.

Semiautomatic Configuration

A semiautomatic configuration is somewhere between a manual configuration and an automatic configuration. Usually, the initializing is done manually, with migrations done automatically.

6.2.3 Communication among Switches

In a multiswitched backbone, each switch must know not only which station belongs to which VLAN, but also the membership of stations connected to other switches. For example, in Figure 6.12, switch A must know the membership status of stations connected to switch B, and switch B must know the same about switch A. Three methods have been devised for this purpose: table maintenance, frame tagging, and time-division multiplexing.

Table Maintenance

In this method, when a station sends a broadcast frame to its group members, the switch creates an entry in a table and records station membership. The switches send their tables to one another periodically for updating.

Frame Tagging

In this method, when a frame is traveling between switches, an extra header is added to the MAC frame to define the destination VLAN. The frame tag is used by the receiving switches to determine the VLANs to be receiving the broadcast message.

Time-Division Multiplexing (TDM)

In this method, the connection (trunk) between switches is divided into timeshared channels (such as TDM). For example, if the total number of VLANs in a backbone is five, each trunk is divided into five channels. The traffic destined for VLAN 1 travels in channel 1, the traffic destined for VLAN 2 travels in channel 2, and so on. The receiving switch determines the destination VLAN by checking the channel from which the frame arrived.

IEEE Standard

In 1996, the IEEE 802.1 subcommittee passed a standard called 802.1Q that defines the format for frame tagging. The standard also defines the format to be used in multiswitched backbones and enables the use of multivendor equipment in VLANs. IEEE 802.1Q has opened the way for further standardization in other issues related to VLANs. Most vendors have already accepted the standard.

6.2.4 Advantages

There are several advantages to using VLANs.

Cost and Time Reduction

VLANs can reduce the migration cost of stations going from one group to another. Physical reconfiguration takes time and is costly. Instead of physically moving one station to another segment or even to another switch, it is much easier and quicker to move it by using software.

Creating Virtual Work Groups

VLANs can be used to create virtual work groups. For example, in a campus environment, professors working on the same project can send broadcast messages to one another without the necessity of belonging to the same department. This can reduce traffic if the multicasting capability of IP was previously used.

Security

VLANs provide an extra measure of security. People belonging to the same group can send broadcast messages with the guaranteed assurance that users in other groups will not receive these messages.

6.3 END-OF-CHAPTER MATERIALS

6.3.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items in brackets refer to the reference list at the end of the text.

Books

Several books discuss link-layer issues. Among them we recommend [Ham 80], [Zar 02], [Ror 96], [Tan 03], [GW 02], [For 03], [KMK 04], [Sta 04], [Kes 02], [PD 03], [Kei 02], [Spu 00], [KCK 98], [Sau 98], [Izz 00], [Per 00], and [WV 00].

6.3.2 Key Terms

blocking port
connecting device
filtering
forwarding port
hub
link-layer switch
repeater
router
spanning tree
transparent switch
virtual local area network (VLAN)

6.3.3 Summary

A repeater is a connecting device that operates in the physical layer of the Internet model. A repeater regenerates a signal, connects segments of a LAN, and has no filtering capability. A link-layer switch is a connecting device that operates in the physical and data-link layers of the Internet model. A transparent switch can forward and filter

frames and automatically build its forwarding table. A switch can use the spanning tree algorithm to create a loopless topology.

A virtual local area network (VLAN) is configured by software, not by physical wiring. Membership in a VLAN can be based on MAC addresses, IP addresses, IP multicast addresses, or a combination of these features. VLANs are cost and time efficient, can reduce network traffic, and provide an extra measure of security.

6.4 PRACTICE SET

6.4.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

6.4.2 Questions

Q6-1. What do we mean when we say that a switch can filter traffic? Why is filtering important?

Q6-2. What is a transparent switch?

Q6-3. How is a hub related to a repeater?

Q6-4. What is the difference between a forwarding port and a blocking port?

Q6-5. How does a VLAN save a company time and money?

Q6-6. How does a VLAN provide extra security for a network?

Q6-7. How does a VLAN reduce network traffic?

Q6-8. What is the basis for membership in a VLAN?

Q6-9. What do we mean when we say that a link-layer switch can filter traffic? Why is filtering important?

Q6-10. Which one has more overhead, a switch or a router? Explain your answer.

Q6-11. Which one has more overhead, a hub or a switch? Explain your answer.

6.4.3 Problems

P6-1. A switch uses a filtering table; a router uses a routing table. Can you explain the difference?

P6-2. Repeat the steps in Figure 6.5 if host F in LAN 2 sends a frame to host B in LAN 1.

P6-3. Repeat the steps in Figure 6.5 if host B in LAN 1 sends a frame to host C in the same LAN.

P6-4. In Figure 6.5, do we have a loop problem if we change the hub in one of the LANs with a link-layer switch?

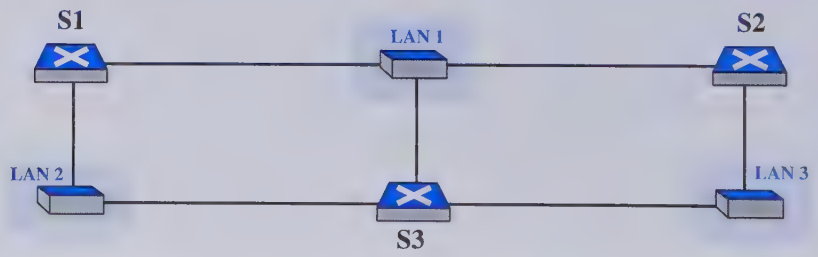
P6-5. In Figure 6.5, do we have a loop problem if we change each hub in the LANs with a link-layer switch?

P6-6. Find the spanning tree and the logical connection between the switches in Figure 6.13.

Figure 6.13 *Problem P6-6.*

P6-7. Find the spanning tree and the logical connection between the switches in Figure 6.14.

Figure 6.14 *Problem P6-7.*



CHAPTER 7

Network Layer: Data Transfer

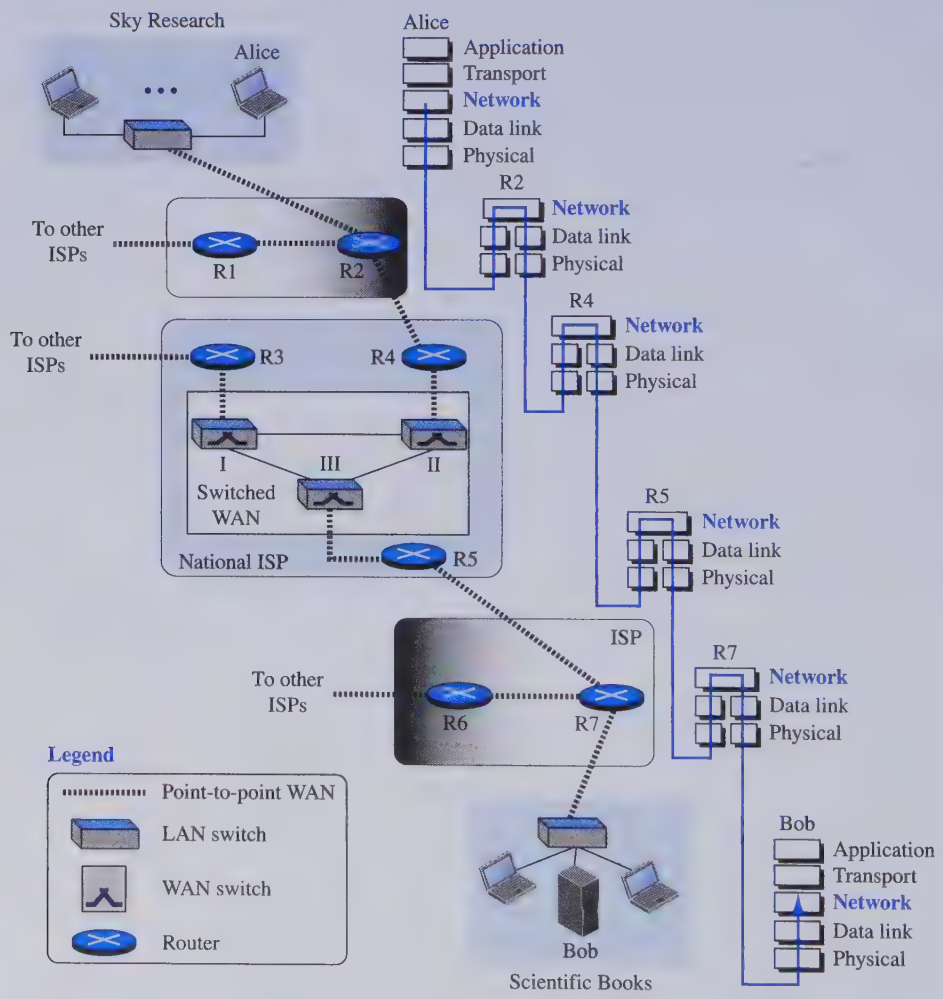
The network layer in the TCP/IP protocol suite is responsible for the host-to-host delivery of messages. It provides services to the transport layer and receives services from the data-link layer. In this chapter, we introduce the general concepts and issues in the network layer. This chapter also discusses the addressing mechanism uses in the network layer.

This chapter is divided into six sections.

- The first section introduces the network layer by defining the services provided by this layer, including packetizing, routing, error control, flow control, congestion control, quality of service, and security.
- The second section discusses packet switching, including the data-gram approach and circuit-switching approach.
- The third section discusses network-layer performance, including delay, throughput, and packet loss.
- The fourth section discusses Internet Protocol version 4 (IPv4), including addressing, packet format, options, ICMPv4, and mobile IP.
- The fifth section discusses Internet Protocol version 6 (IPv6), including addressing, packet format, and options.
- The sixth section briefly discusses the transition from IPv4 to IPv6.

Figure 7.1 shows the communication between Alice and Bob at the network layer. At the source host, the network layer encapsulates data received from the transport layer in a network layer packet. At the destination host, the network layer decapsulates data from the network layer and delivers it to the transport layer. The routers do not do any encapsulation or decapsulation unless in some special cases when the packet needs to be fragmented.

Figure 7.1 *Communication at the network layer*



7.1 SERVICES

We briefly discuss the services provided at the network layer.

7.1.1 Packetizing

The first duty of the network layer is definitely **packetizing**: encapsulating the payload (data received from the upper layer) in a network-layer packet at the source and decapsulating the payload from the network-layer packet at the destination. In other words, one duty of the network layer is to carry a payload from the source to the destination without changing it or using it. The network layer is doing the service of a carrier such as the postal office, which is responsible for delivery of packages from a sender to a receiver without changing or using the contents.

The source host receives the payload from an upper-layer protocol, adds a header that contains the source and destination addresses and some other information that is required by the network-layer protocol (as discussed later), and delivers the packet to the data-link layer.

The destination host receives the network-layer packet from its data-link layer, decapsulates the packet, and delivers the payload to the corresponding upper-layer protocol. If the packet is fragmented at the source or at routers along the path, the network layer is responsible for waiting until all fragments arrive, reassembling them, and delivering them to the upper-layer protocol.

7.1.2 Routing

The network layer is responsible for routing a network-layer packet from its source to the destination. A physical network is a combination of networks (LANs and WANs) and routers that connect them. This means that there is more than one route from the source to the destination. The network layer is responsible for finding the best one among these possible routes. The network layer needs to have some specific strategies for defining the best route.

7.1.3 Error Control

In Chapter 3, we discussed error detection and correction. Although error control also can be implemented in the network layer, the designers of the network layer in the Internet ignored this issue for the data being carried by the network layer. One reason for this decision is the fact that the packet in the network layer may be fragmented at each router, which makes error checking at this layer inefficient.

The designers of the network layer, however, have added a checksum field to the datagram to control any corruption in the header, but not the whole datagram. This checksum may prevent any changes or corruptions in the header of the datagram between two hops and from end to end.

7.1.4 Flow Control

Flow control regulates the amount of data a source can send without overwhelming the receiver. If the upper layer at the source computer produces data faster than the upper layer at the destination computer can consume it, the receiver will be overwhelmed

with data. To control the flow of data, the receiver needs to send some feedback to the sender to inform the latter that it is overwhelmed with data.

7.1.5 Congestion Control

Another issue in a network-layer protocol is congestion control. Congestion in the network layer is a situation in which too many datagrams are present in an area of the Internet. Congestion may occur if the number of datagrams sent by source computers is beyond the capacity of the network or routers. In this situation, some routers may drop some of the datagrams. However, as more datagrams are dropped, the situation may become worse because, due to the error-control mechanism at the upper layers, the sender may send duplicates of the lost packets. If the congestion continues, sometimes a situation may reach a point where the system collapses and no datagrams are delivered.

7.1.6 Quality of Service

As the Internet has allowed new applications such as multimedia communication (in particular real-time communication of audio and video), the quality of service (QoS) of the communication has become more and more important. The Internet has thrived by providing better quality of service to support these applications. However, to keep the network layer untouched, these provisions are mostly implemented in the upper layer.

7.1.7 Security

Another issue related to communication at the network layer is security. Security was not a concern when the Internet was originally designed because it was used by a small number of users at universities for research activities; other people had no access to the Internet. The network layer was designed with no security provision. Today, however, security is a big concern. To provide security for a connectionless network layer, we need to have another virtual level that changes the connectionless service to a connection-oriented service. This virtual layer, called IPSec, is discussed in Chapter 13.

7.2 PACKET SWITCHING

From the discussion of routing and forwarding in Section 7.1, we infer that a kind of *switching* occurs at the network layer. A router, in fact, is a switch that creates a connection between an input port and an output port (or a set of output ports), just as an electrical switch connects the input to the output to let electricity flow.

Although in data communications switching techniques are divided into two broad categories, circuit switching and packet switching, only packet switching is used at the network layer because the unit of data at this layer is a packet.

At the network layer, a message from the upper layer is divided into manageable packets and each packet is sent through the network. The source of the message sends the packets one by one; the destination of the message receives the packets one by one. The destination waits for all packets belonging to the same message to arrive before delivering the message to the upper layer. The connecting devices in a packet-switched

network still need to decide how to route the packets to the final destination. Today, a packet-switched network can use two different approaches to route the packets: the *datagram approach* and the *virtual-circuit approach*. We discuss both approaches next.

7.2.1 Datagram Approach: Connectionless Service

When the Internet started, to make it simple, the network layer was designed to provide a connectionless service in which the network-layer protocol treats each packet independently, with each packet having no relationship to any other packet. The idea was that the network layer is only responsible for delivery of packets from the source to the destination. In this approach, the packets in a message may or may not travel the same path to their destination.

When the network layer provides a connectionless service, each packet traveling in the Internet is an independent entity; there is no relationship between packets belonging to the same message. The switches in this type of network are called *routers*. A packet belonging to a message may be followed by a packet belonging to the same message or to a different message. A packet may be followed by a packet coming from the same source or from a different source.

Each packet is routed based on the information contained in its header: source and destination addresses. The destination address defines where it should go; the source address defines where it comes from. The router in this case routes the packet based only on the destination address. The source address may be used to send an error message to the source if the packet is discarded.

7.2.2 Virtual-Circuit Approach: Connection-Oriented Service

In a connection-oriented service (also called a *virtual-circuit approach*), there is a relationship between all packets belonging to a message. Before all datagrams in a message can be sent, a virtual connection should be set up to define the path for the datagrams. After connection setup, the datagrams can all follow the same path. In this type of service, not only must the packet contain the source and destination addresses, it must also contain a **flow label**, a virtual-circuit identifier that defines the virtual path the packet should follow. Although it looks as though the use of the label may make the source and destination addresses unnecessary during the data transfer phase, parts of the Internet at the network layer still keep these addresses. One reason is that part of the packet path may still be using the connectionless service. Another reason is that the protocol at the network layer is designed with these addresses, and it may take a while before they can be changed.

7.3 PERFORMANCE

The upper-layer protocols that use the service of the network layer expect to receive an ideal service, but the network layer is not perfect. The performance of a network can be measured in terms of *delay*, *throughput*, and *packet loss*. Congestion control is an issue that can improve the performance.

7.3.1 Delay

All of us expect an instantaneous response from a network, but a packet, from its source to its destination, encounters delays. The delays in a network can be divided into four types: transmission delay, propagation delay, processing delay, and queuing delay. Let us first discuss each of these delay types and then show how to calculate a packet delay from the source to the destination.

Transmission Delay

A source host or a router cannot send a packet instantaneously. A sender needs to put the bits in a packet on the line one by one. If the first bit of the packet is put on the line at time t_1 and the last bit is put on the line at time t_2, transmission delay of the packet is $(t_2 - t_1)$. Definitely, the transmission delay is longer for a longer packet and shorter if the sender can transmit faster. In other words, the transmission delay is

$$\textbf{Delay}_{tr} = \textbf{(packet length) / (transmission rate)}$$

Propagation Delay

Propagation delay is the time it takes for a bit to travel from point A to point B in the transmission media. The propagation delay for a packet-switched network depends on the propagation delay of each network (LAN or WAN). The propagation delay depends on the propagation speed of the media, which is 3×10^8 m/s in a vacuum and normally much less in a wired medium; it also depends on the distance of the link. In other words, propagation delay is

$$\textbf{Delay}_{pg} = \textbf{(distance) / (propagation speed)}$$

Processing Delay

The processing delay is the time required for a router or a destination host to receive a packet from its input port, remove the header, perform an error-detection procedure, and deliver the packet to the output port (in the case of a router) or deliver the packet to the upper-layer protocol (in the case of the destination host). The processing delay may be different for each packet, but normally is calculated as an average.

$$\textbf{Delay}_{pr} = \textbf{time required to process a packet in a router or a destination host}$$

Queuing Delay

Queuing delay can normally happen in a router. As we discuss in Chapter 8, a router has an input queue connected to each of its input ports to store packets waiting to be processed; the router also has an output queue connected to each of its output ports to store packets waiting to be transmitted. The queuing delay for a packet in a router is measured as the time a packet waits in the input queue and output queue of a router. We can compare the situation with a busy airport. Some planes may need to wait to get the landing band (input delay); some planes may need to wait to get the departure band (output delay).

$$\textbf{Delay}_{qu} = \textbf{time a packet waits in input and output queues in a router}$$

Total Delay

Assuming equal delays for the sender, routers, and receiver, the total delay (source-to-destination delay) a packet encounters can be calculated if we know the number of routers, n, in the whole path.

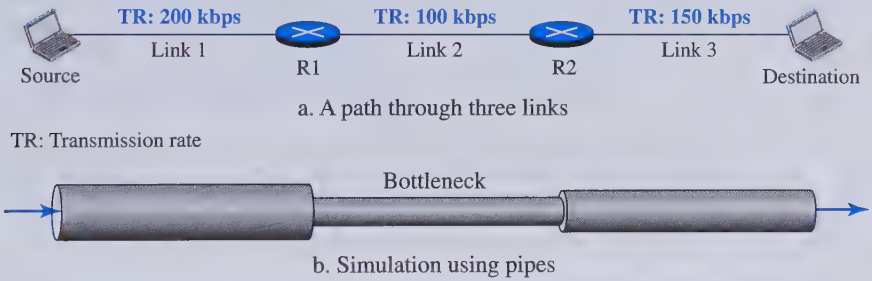
$$\textbf{Total delay} = (n + 1)\,(\textbf{delay}_{tr} + \textbf{delay}_{pg} + \textbf{delay}_{pr}) + (n)\,(\textbf{delay}_{qu})$$

Note that if we have n routers, we have $(n + 1)$ links. Therefore, we have $(n + 1)$ transmission delays related to n routers and the source, $(n + 1)$ propagation delays related to $(n + 1)$ links, $(n + 1)$ processing delays related to n routers and the destination, and only n queuing delays related to n routers.

7.3.2 Throughput

Throughput at any point in a network is defined as the number of bits passing through the point in a second, which is actually the transmission rate of data at that point. In a path from source to destination, a packet may pass through several links (networks), each with a different transmission rate. How, then, can we determine the throughput of the whole path? To see the situation, assume that we have three links, each with a different transmission rate, as shown in Figure 7.2.

Figure 7.2 *Throughput in a path with three links in a series*



a. A path through three links

b. Simulation using pipes

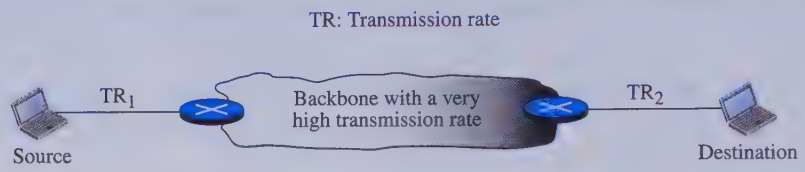
In Figure 7.2, the data can flow at the rate of 200 kbps in link 1. However, when the data arrives at router R1, it cannot pass at this rate. Data need to be queued at the router and sent at 100 kbps. When data arrive at router R2, they could be sent at the rate of 150 kbps, but there is not enough data to be sent. In other words, the average rate of the data flow in link 3 is also 100 kbps. We can conclude that the average data rate for this path is 100 kbps, the minimum of the three different data rates. Figure 7.2 also shows that we can simulate the behavior of each link with pipes of different sizes; the average throughput is determined by the bottleneck, the pipe with the smallest diameter. In general, in a path with n links in series, we have

$$\textbf{Throughput} = \textbf{minimum}\,\{\textbf{TR}_1, \textbf{TR}_2, \ldots, \textbf{TR}_n\}$$

Although the situation in Figure 7.2 shows how to calculate the throughput when the data are passed through several links, the actual situation in the Internet is that the

data normally pass through two access networks and the Internet backbone, as shown in Figure 7.3.

Figure 7.3 *A path through the Internet backbone*



7.3.3 Packet Loss

Another issue that severely affects the performance of communication is the number of packets lost during transmission. When a router receives a packet while processing another packet, the received packet needs to be stored in the input buffer waiting for its turn. A router, however, has an input buffer with a limited size. A time may come when the buffer is full and the next packet needs to be dropped. The effect of packet loss on the Internet network layer is that the packet needs to be re-sent, which in turn may create overflow and cause more packet loss. A lot of theoretical studies have been done in queuing theory to prevent the overflow of queues and prevent packet loss.

7.4 INTERNET PROTOCOL VERSION 4

The network layer in the Internet has gone through several versions, but only two versions have survived: IP Version 4 (IPv4) and IP Version 6 (IPv6). Although IPv4 is almost depleted, we discuss it because there are still some areas that use this version and also because it is the foundation for IPv6.

7.4.1 IPv4 Addressing

The identifier used in the IP layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the Internet address or IP address. An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.

IPv4 addresses are unique in the sense that each address defines one, and only one, connection to the Internet. If a device has two connections to the Internet, via two networks, it has two IPv4 addresses. IPv4 addresses are universal in the sense that the addressing system must be accepted by any host that wants to be connected to the Internet.

Address Space

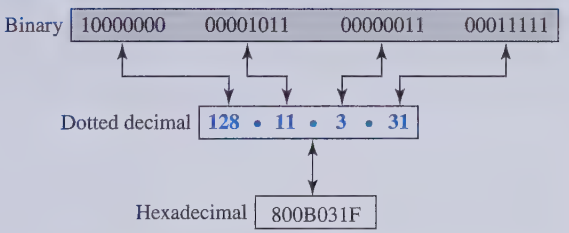
A protocol like IPv4 that defines addresses has an address space. An address space is the total number of addresses used by the protocol. If a protocol uses b bits to define an

address, the address space is 2^b because each bit can have two different values (0 or 1). IPv4 uses 32-bit addresses, which means that the address space is 2^{32} or 4,294,967,296 (more than 4 billion). If there were no restrictions, more than 4 billion devices could be connected to the Internet.

Notation

There are three common notations to show an IPv4 address: binary notation (base 2), dotted-decimal notation (base 256), and hexadecimal notation (base 16). In *binary notation*, an IPv4 address is displayed as 32 bits. To make the address more readable, one or more spaces are usually inserted between each octet (8 bits). Each octet is often referred to as a byte. To make the IPv4 address more compact and easier to read, it is usually written in decimal form with a decimal point (dot) separating the bytes. This format is referred to as *dotted-decimal notation*. Note that because each byte (octet) is only 8 bits, each number in the dotted-decimal notation is between 0 and 255. We sometimes see an IPv4 address in hexadecimal notation. Each hexadecimal digit is equivalent to 4 bits. This means that a 32-bit address has eight hexadecimal digits. This notation is often used in network programming. Figure 7.4 shows an IP address in the three discussed notations.

Figure 7.4 *Three different notations in IPv4 addressing*



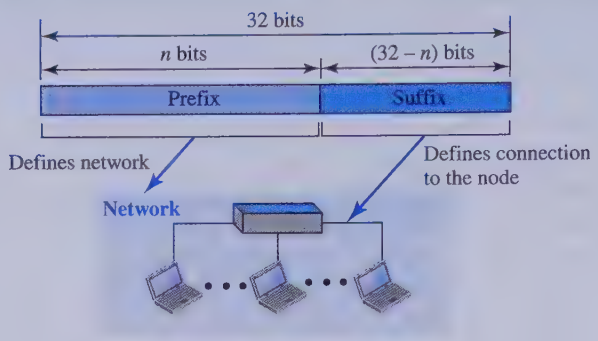
Hierarchy in Addressing

In any communication network that involves delivery, such as a telephone network or a postal network, the addressing system is hierarchical. In a postal network, the postal address (mailing address) includes the country, state, city, street, house number, and the name of the mail recipient. Similarly, a telephone number is divided into the country code, area code, local exchange, and the connection.

A 32-bit IPv4 address is also hierarchical but is divided only into two parts. The first part of the address, called the *prefix*, defines the network; the second part of the address, called the *suffix*, defines the node (connection of a device to the Internet). Figure 7.5 shows the prefix and suffix of a 32-bit IPv4 address. The prefix length is n bits, and the suffix length is $(32 - n)$ bits.

A prefix can be fixed length or variable length. The network identifier in the IPv4 was first designed as a fixed-length prefix. This scheme, which is now obsolete, is referred to as classful addressing. The new scheme, which is referred to as classless

Figure 7.5 *Hierarchy in addressing*

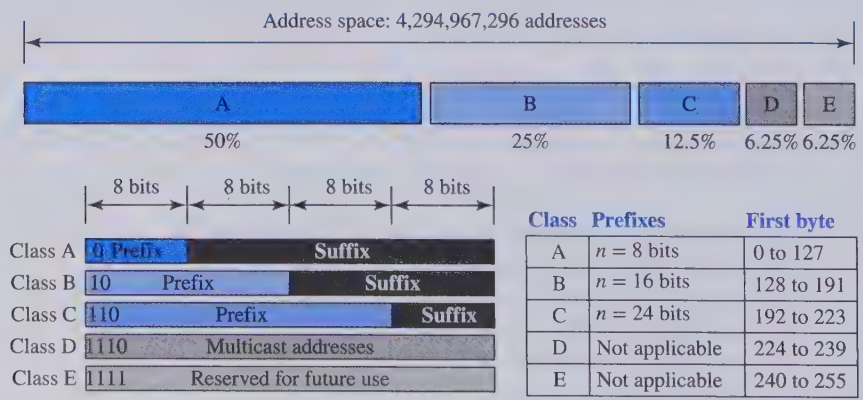


addressing, uses a variable-length network prefix. First, we briefly discuss classful addressing; then we concentrate on classless addressing.

Classful Addressing

When the Internet started, an IPv4 address was designed with a fixed-length prefix, but to accommodate both small and large networks, three fixed-length prefixes were designed instead of one ($n = 8$, $n = 16$, and $n = 24$). The whole address space was divided into five classes (classes A, B, C, D, and E), as shown in Figure 7.6. This scheme is referred to as **classful addressing**. Although classful addressing belongs to the past, it helps us to understand classless addressing.

Figure 7.6 *Occupation of the address space in classful addressing*



Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

In class A, the network length is 8 bits, but because the first bit, which is 0, defines the class, we can have only 7 bits as the network identifier. This means there are only $2^7 = 128$ networks in the world that can have a class A address.

In class B, the network length is 16 bits, but because the first 2 bits, which are $(10)_2$, define the class, we can have only 14 bits as the network identifier. This means there are only $2^{14} = 16{,}384$ networks in the world that can have a class B address.

All addresses that start with $(110)_2$ belong to class C. In class C, the network length is 24 bits, but because 3 bits define the class, we can have only 21 bits as the network identifier. This means there are $2^{21} = 2{,}097{,}152$ networks in the world that can have a class C address.

Class D is not divided into prefix and suffix. It is used for multicast addresses. All addresses that start with 1111 in binary belong to class E. As in class D, class E is not divided into prefix and suffix and is used as reserve.

Address Depletion

The reason that classful addressing has become obsolete is address depletion. Because the addresses were not distributed properly, the Internet was faced with the problem of the addresses being rapidly used up, resulting in no more addresses being available for organizations and individuals that needed to have an Internet connection. To understand the problem, let us think about class A. This class can be assigned to only 128 organizations in the world, but each organization would need to have one single network (seen by the rest of the world) with 16,777,216 nodes (computers in this single network). Because there were only a few organizations that are this large, most of the addresses in this class were wasted (unused). Class B addresses were designed for midsize organizations, but many of the addresses in this class also remained unused. Class C addresses have a completely different design flaw. The number of addresses that can be used in each network (256) was so small that most companies were not comfortable using a block in this address. Class E addresses were almost never used, wasting the whole class.

Classless Addressing

With the growth of the Internet, it was clear that a larger address space was needed as a long-term solution. The larger address space, however, requires that the length of IP addresses also be increased, which means the format of the IP packets needs to be changed. Although the long-range solution has already been devised and is called IPv6 (discussed in Section 7.5), a short-term solution was also devised to use the same address space but to change the distribution of addresses to provide a fair share to each organization. The short-term solution still uses IPv4 addresses, but it is called *classless addressing*. In other words, the class privilege was removed from the distribution to compensate for the address depletion.

There was another motivation for classless addressing. During the 1990s, Internet Service Providers (ISPs) came into prominence. An ISP is an organization that provides Internet access and services for individuals, small businesses, and midsize organizations that do not want to create an Internet site and become involved in providing Internet services (such as electronic mail) for their employees. An ISP is granted a large range of addresses and then subdivides the addresses (in groups of 1, 2, 4, 8, 16, and so on), giving a range of addresses to a household or a small business. The customers are connected via a dial-up modem, DSL, or cable modem to the ISP. However, each customer needs some IPv4 addresses.

In 1996, the Internet authorities announced a new architecture called **classless addressing**. In classless addressing, variable-length blocks are used that belong to no classes. We can have a block of 1 address, 2 addresses, 4 addresses, 128 addresses, and so on.

In classless addressing, the whole address space is divided into variable-length blocks. The prefix in an address defines the block (network); the suffix defines the node (device). Theoretically, we can have a block of $2^0, 2^1, 2^2, \ldots, 2^{32}$ addresses. One of the restrictions is that the number of addresses in a block needs to be a power of 2. An organization can be granted one block of addresses. Figure 7.7 shows the division of the whole address space into nonoverlapping blocks.

Figure 7.7 *Variable-length blocks in classless addressing*

Prefix Length: Slash Notation

The first question that we need to answer in classless addressing is how to find the prefix length if an address is given. Because the prefix length is not inherent in the address, we need to separately give the length of the prefix. In this case, the prefix length, n, is added to the address, separated by a slash. The notation is informally referred to as *slash notation* and formally as **classless interdomain routing (CIDR**, pronounced cider) strategy. An address in classless addressing can then be represented as shown in Figure 7.8.

Figure 7.8 *Slash notation (CIDR)*

In other words, an address in classless addressing does not, per se, define the block or network to which the address belongs; we need to give the prefix length also.

Extracting Information from an Address

Given any address in the block, we normally like to know three pieces of information about the block to which the address belongs: the number of addresses, the first address in the block, and the last address. Because the value of prefix length, n, is given, we can easily find these three pieces of information.

1. To find the first address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 0s.
2. To find the last address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 1s.

Example 7.1

A classless address is given as 167.199.170.82/27. We can find the desired three pieces of information as follows. The number of addresses in the network is $2^{32-n} = 2^5 = 32$ addresses. The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01010010
First address: 167.199.170.64/27	10100111	11000111	10101010	01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

Address Mask

Another way to find the first and last addresses in the block is to use the address mask. The address mask is a 32-bit number in which the n leftmost bits are set to 1s and the rest of the bits $(32 - n)$ are set to 0s. A computer can easily find the address mask because it is the complement of $(2^{32-n} - 1)$. The reason for defining a mask in this way is that it can be used by a computer program to extract the information in a block, using the three bit-wise operations NOT, AND, and OR.

1. The number of addresses in the block N = **NOT** (Mask) + 1.
2. The first address in the block = (Any address in the block) **AND** (Mask).
3. The last address in the block = (Any address in the block) **OR** [(**NOT** (Mask)].

Example 7.2

We repeat Example 7.1 using the mask. The mask in dotted-decimal notation is 256.256.256.224. The AND, OR, and NOT operations can be applied to individual bytes using calculators and applets at the book website.

Number of addresses in the block:	N = **NOT** (mask) + 1 = 0.0.0.31 + 1 = 32 addresses
First address:	First = (address) **AND** (mask) = 167.199.170. 82
Last address:	Last = (address) **OR** (**NOT** mask) = 167.199.170. 255

Example 7.3

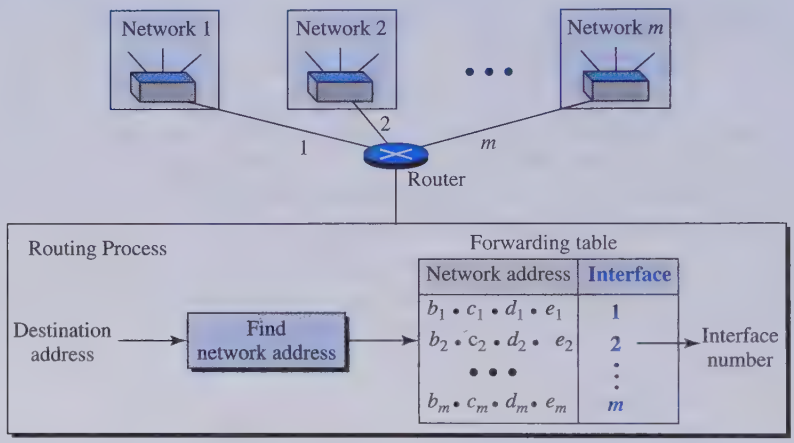
In classless addressing, an address cannot per se define the block the address belongs to. For example, the address 230.8.24.56 can belong to many blocks. Some of them are shown here with the value of the prefix associated with that block.

Prefix length:16	→	Block:	230.8.0.0	to	230.8.255.255
Prefix length:20	→	Block:	230.8.16.0	to	230.8.31.255
Prefix length:26	→	Block:	230.8.24.0	to	230.8.24.63
Prefix length:27	→	Block:	230.8.24.32	to	230.8.24.63
Prefix length:29	→	Block:	230.8.24.56	to	230.8.24.63
Prefix length:31	→	Block:	230.8.24.56	to	230.8.24.57

Network Address

The preceding examples show that, given any address, we can find all information about the block. The first address, the **network address**, is particularly important because it is used in routing a packet to its destination network. For the moment, let us assume that an internet is made up of m networks and a router with m interfaces. When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent and from which interface the packet should be sent out. When the packet arrives at the network, it reaches its destination host using link-layer addressing, which was discussed in Chapter 3 (Section 3.4). Figure 7.9 shows the idea.

Figure 7.9 *Network address*



After the network address has been found, the router consults its forwarding table to find the corresponding interface from which the packet should be sent out. The network address is actually the identifier of the network; each network is identified by its network address.

Block Allocation

The next issue in classless addressing is block allocation. How are the blocks allocated? The ultimate responsibility of block allocation is given to a global authority called the Internet Corporation for Assigned Names and Numbers (ICANN). However, ICANN does not normally allocate addresses to individual Internet users. It assigns a large block of addresses to an ISP (or a larger organization that is considered an ISP in this case). For the proper operation of the CIDR, two restrictions need to be applied to the allocated block.

1. The number of requested addresses, N, needs to be a power of 2. The reason is that $N = 2^{32-n}$ or $n = 32 - \log_2 N$. If N is not a power of 2, we cannot have an integer value for n.
2. The requested block needs to be allocated where there are a contiguous number of available addresses in the address space. However, there is a restriction on choosing

the first address in the block. The first address needs to be divisible by the number of addresses in the block. The reason is that the first address needs to be the prefix followed by $(32 - n)$ number of 0s. The decimal value of the first address is then

$$\text{First address} = (\text{prefix in decimal}) \times 2^{32-n} = (\text{prefix in decimal}) \times N$$

Example 7.4

An ISP has requested a block of 1000 addresses. Because 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/22, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Subnetting

More levels of hierarchy can be created using subnetting. An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a subnetwork (or subnet). Note that nothing stops the organization from creating more levels. A subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

Designing Subnets The subnetworks in a network should be carefully designed to enable the routing of packets. We assume the total number of addresses granted to the organization is N, the prefix length is n, the assigned number of addresses to each subnetwork is N_{sub}, and the prefix length for each subnetwork is n_{sub}. Then the following steps need to be carefully followed to guarantee the proper operation of the subnetworks.

- The number of addresses in each subnetwork should be a power of 2.
- The prefix length for each subnetwork should be found using the following formula:

$$n_{sub} = 32 - \log_2 N_{sub}$$

- The starting address in each subnetwork should be divisible by the number of addresses in that subnetwork. This can be achieved if we first assign addresses to larger subnetworks.

Finding Information about Each Subnetwork After designing the subnetworks, the information about each subnetwork, such as first and last address, can be found using the process we described to find the information about each network in the Internet.

Example 7.5

An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have three subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Solution

There are $2^{32-24} = 256$ addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as

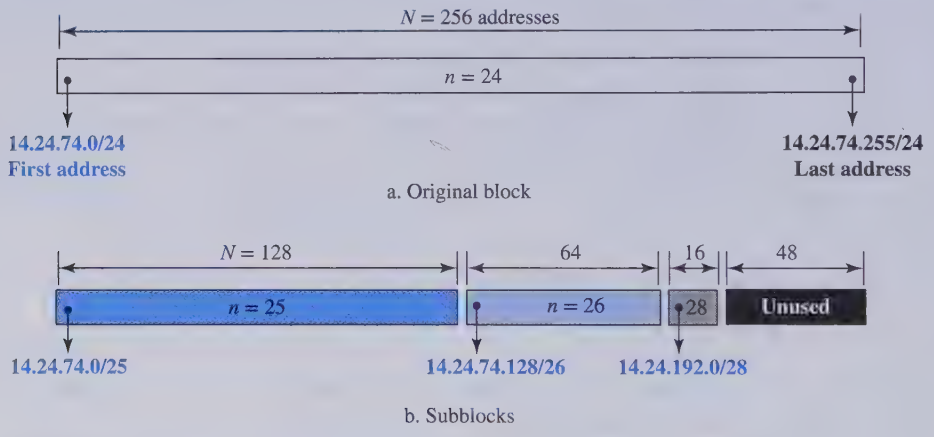
$n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/25; the last address is 14.24.74.127/25.

b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as $n_2 = 32 - \log_2 64 = 26$. The first address in this block is 14.24.74.128/26; the last address is 14.24.74.191/26.

c. The number of addresses in the smallest subblock, which requires 10 addresses, is not a power of 2 either. We allocate 16 addresses. The subnet mask for this subnet can be found as $n_3 = 32 - \log_2 16 = 28$. The first address in this block is 14.24.74.192/28; the last address is 14.24.74.207/28.

If we add all addresses in the previous subblocks, the result is 208 addresses, which means 48 addresses are left in reserve. The first address in this range is 14.24.74.208. The last address is 14.24.74.255. We don't know about the prefix length yet. Figure 7.10 shows the configuration of blocks. It shows the first address in each block.

Figure 7.10 *Solution to Example 7.5*



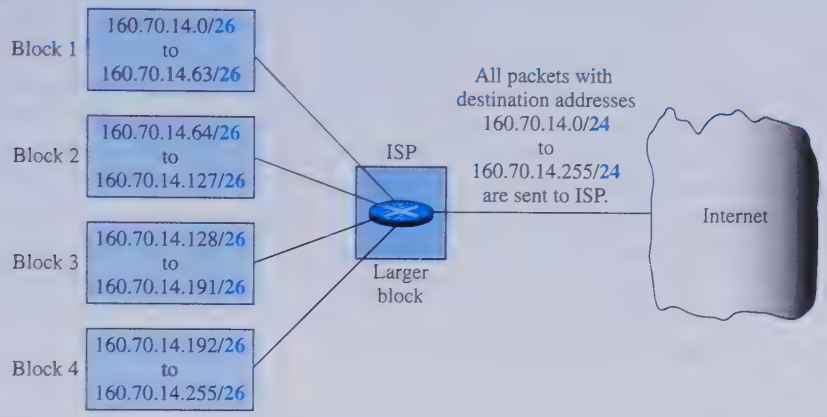
Address Aggregation

One of the advantages of the CIDR strategy is **address aggregation** (sometimes called *address summarization* or *route summarization*). When blocks of addresses are combined to create a larger block, routing can be done based on the prefix of the larger block. ICANN assigns a large block of addresses to an ISP. Each ISP in turn divides its assigned block into smaller subblocks and grants the subblocks to its customers.

Example 7.6

Figure 7.11 shows how four small blocks of addresses are assigned to four organizations by an ISP. The ISP combines these four blocks into one single block and advertises the larger block to the rest of the world. Any packet destined for this larger block should be sent to this ISP. It is the responsibility of the ISP to forward the packet to the appropriate organization. This is similar to the routing we find in a postal network. All packages coming from outside a country are sent first to the capital and then distributed to the corresponding destination.

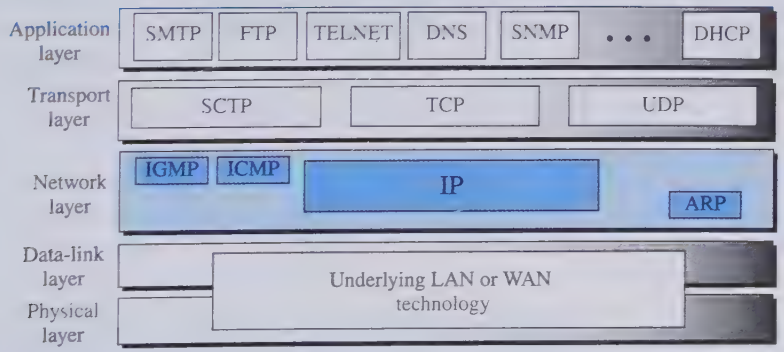
Figure 7.11 *Example of address aggregation*



7.4.2 Main and Auxiliary Protocols

The network layer in version 4 can be thought of as one main protocol and three auxiliary protocols. The main protocol, Internet Protocol version 4 (IPv4), is responsible for packetizing, forwarding, and delivery of a packet at the network layer. The Internet Control Message Protocol version 4 (ICMPv4) helps IPv4 to handle some errors that may occur in the network-layer delivery. The Internet Group Management Protocol (IGMP) is used to help IPv4 in multicasting. The Address Resolution Protocol (ARP) is used to glue the network and data-link layers in mapping network-layer addresses to link-layer addresses. Figure 7.12 shows the positions of these four protocols in the TCP/IP protocol suite.

Figure 7.12 *Position of IP and other network-layer protocols in TCP/IP protocol suite*



We discuss IPv4, ICMPv4, and ARP in this chapter. We will discuss IGMP when we talk about multicasting in Chapter 8.

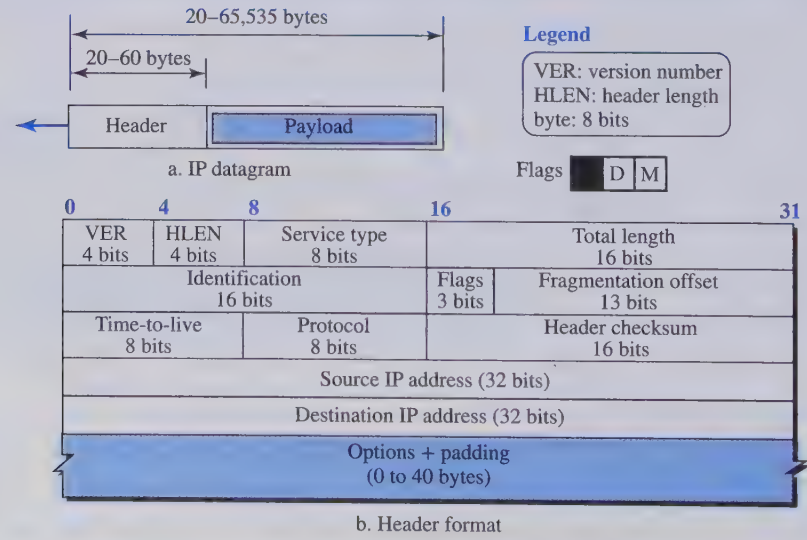
IPv4 is an unreliable datagram protocol—a best-effort delivery service. The term *best effort* means that IPv4 packets can be corrupted, be lost, arrive out of order, be delayed or create congestion for the network. If reliability is important, IPv4 must be paired with a reliable transport-layer protocol such as TCP. An example of a common best-effort delivery service is the post office. The post office does its best to deliver the regular mail but does not always succeed. The post office itself does not keep track of every letter and cannot notify a sender of loss or damage of one. If an unregistered letter is lost or damaged in the mail, the would-be recipient will not receive the correspondence and the sender will need to re-create it.

IPv4 is also a connectionless protocol that uses the datagram approach. This means that each datagram is handled independently, and each datagram can follow a different route to the destination. This implies that datagrams sent by the same source to the same destination could arrive out of order. Again, IPv4 relies on a higher-level protocol to take care of all these problems.

Datagram Format

In this section, we begin by discussing the first service provided by IPv4, packetizing. We show how IPv4 defines the format of a packet in which the data coming from the upper layer or other protocols are encapsulated. Packets used by the IP are called *datagrams*. Figure 7.13 shows the IPv4 datagram format. A datagram is a variable-length packet consisting of two parts: the header and payload (data). The header is 20 to 60 bytes in length and contains information essential to routing and delivery. The first 20 bytes are essential and together are called the main header. The next 40 bytes include options and padding that may or may not be present. It is customary in TCP/IP to show the header in 4-byte sections.

Figure 7.13 *IP datagram*



Discussing the meaning and rationale for the existence of each field is essential to understanding the operation of IPv4; a brief description of each field is in order.

- **Version number.** The 4-bit version number (VER) field defines the version of the IPv4 protocol, which, obviously, has the value of 4.
- **Header length.** The 4-bit header length (HLEN) field defines the total length of the datagram header in 4-byte words. The IPv4 datagram has a variable-length header. When a device receives a datagram, it needs to know when the header stops and the data, which are encapsulated in the packet, start. However, to make the value of the header length (number of bytes) fit in a 4-bit header length, the total length of the header is calculated as 4-byte words. The total length is divided by 4, and the value is inserted in the field. The receiver needs to multiply the value of this field by 4 to find the total length.
- **Service type.** In the original design of the IP header, this field was referred to as type of service (TOS), which defined how the datagram should be handled. In the late 1990s, the Internet Engineering Task Force (IETF) redefined the field to provide *differentiated services* (DiffServ), which divide applications into different classes according to their priority. The use of a 4-byte word for the header length is also logical because the IP header is always needed to be aligned in 4-byte boundaries.
- **Total length.** This 16-bit field defines the total length (header plus data) of the IP datagram in bytes. A 16-bit number can define a total length of up to 65,535 (when all bits are 1s). However, the size of the datagram is normally much less than this. This field helps the receiving device know when the packet has completely arrived. To find the length of the data coming from the upper layer, subtract the header length from the total length. The header length can be found by multiplying the value in the HLEN field by 4.

$$\text{Length of data} = \text{total length} - (\text{HLEN}) \times 4$$

Though a size of 65,535 bytes might seem large, the size of the IPv4 datagram may increase in the near future as the underlying technologies allow even more throughput (greater bandwidth).

One may ask why we need this field anyway. When a machine (router or host) receives a frame, it drops the header and the trailer, leaving the datagram. Why include an extra field that is not needed? The answer is that in many cases we really do not need the value in this field. However, there are occasions in which the datagram is not the only thing encapsulated in a frame; it may be that padding has been added. For example, the Ethernet protocol has a minimum and maximum restriction on the size of data that can be encapsulated in a frame (46 to 1500 bytes). If the size of an IPv4 datagram is less than 46 bytes, some padding will be added to meet this requirement. In this case, when a machine decapsulates the datagram, it needs to check the total length field to determine how much is really data and how much is padding.

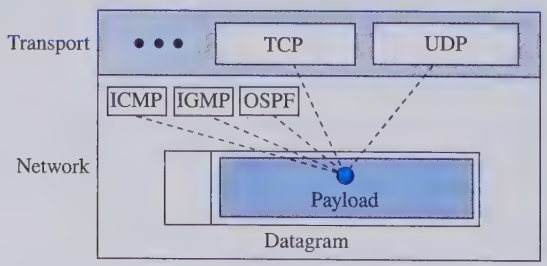
- **Identification, flags, and fragmentation offset.** These three fields are related to the fragmentation of the IP datagram when the size of the datagram is larger than

the underlying network can carry. We discuss the contents and importance of these fields when we talk about fragmentation later in this section.

- **Time-to-live.** Because of some malfunctioning of routing protocols (discussed in the next bullet point) a datagram may be circulating in the Internet, visiting some networks over and over without reaching the destination. This may create extra traffic in the Internet. The time-to-live (TTL) field is used to control the maximum number of hops (routers) visited by the datagram. When a source host sends the datagram, it stores a number in this field. This value is approximately 2 times the maximum number of routes between any two hosts. Each router that processes the datagram decrements this number by one. If this value, after being decremented, is zero, the router discards the datagram.

- **Protocol.** In TCP/IP, the data section of a packet, called the *payload*, carries the whole packet from another protocol. A datagram, for example, can carry a packet belonging to any transport-layer protocol such as UDP or TCP. A datagram can also carry a packet from other protocols that directly use the service of the IP, such as some routing protocols or some auxiliary protocols. The Internet authority has given any protocol that uses the service of the IP a unique 8-bit number that is inserted in the protocol field. When the payload is encapsulated in a datagram at the source IP, the corresponding protocol number is inserted in this field; when the datagram arrives at the destination, the value of this field helps to define to which protocol the payload should be delivered. In other words, this field provides multiplexing at the source and demultiplexing at the destination, as shown in Figure 7.14. Note that the protocol fields at the network layer play the same role as the port numbers at the transport layer (Chapter 9). However, we need two port numbers in a transport-layer packet because the port numbers at the source and destination are different, but we need only one protocol field because this value is the same for each protocol no matter whether it is located at the source or the destination.

Figure 7.14 *Multiplexing and demultiplexing using the value of the protocol field*



Some protocol values

ICMP	01
IGMP	02
TCP	06
UDP	17
OSPF	89

- **Header checksum.** IP is not a reliable protocol; it does not check whether the payload carried by a datagram is corrupted during the transmission. IP puts the burden of error checking of the payload on the protocol that owns the payload, such as UDP or TCP. The datagram header, however, is added by IP, and its error checking is the

responsibility of IP. Errors in the IP header can be a disaster. For example, if the destination IP address is corrupted, the packet can be delivered to the wrong host. If the protocol field is corrupted, the payload may be delivered to the wrong protocol. If the fields related to the fragmentation are corrupted, the datagram cannot be reassembled correctly at the destination, and so on. For these reasons, IP a header checksum field to check the header, but not the payload. We need to remember that, because the value of some fields, such as TTL, which are related to fragmentation and options, may change from router to router, the checksum needs to be recalculated at each router, as we show in Example 7.11.

- **Source and destination addresses.** These 32-bit source and destination address fields define the IP address of the source and destination, respectively. The source host should know its IP address. The destination IP address is either known by the protocol that uses the service of IP or is provided by the DNS as described in Chapter 10. Note that the value of these fields must remain unchanged during the time the IP datagram travels from the source host to the destination host. IP addresses were discussed earlier in this chapter.
- **Options.** A datagram header can have up to 40 bytes of options. Options can be used for network testing and debugging. Although options are not a required part of the IP header, option processing is required of the IP software. This means that all implementations must be able to handle options if they are present in the header. The existence of options in a header creates some burden on the datagram handling; some options can be changed by routers, which forces each router to recalculate the header checksum. There are 1-byte and multibyte options that we will briefly discuss later in the chapter.
- **Payload.** Payload, or data, is the main reason for creating a datagram. Payload is the packet coming from other protocols that use the service of IP. Comparing a datagram to a postal package, payload is the content of the package; the header is only the information written on the package.

Example 7.7

An IPv4 packet has arrived with the first 8 bits as $(01000010)_2$. The receiver discards the packet. Why?

Solution

There is an error in this packet. The 4 leftmost bits $(0100)_2$ show the version, which is correct. The next 4 bits $(0010)_2$ show an invalid header length ($2 \times 4 = 8$). The minimum number of bytes in the header must be 20. The packet has been corrupted in transmission.

Example 7.8

In an IPv4 packet, the value of HLEN is $(1000)_2$. How many bytes of options are being carried by this packet?

Solution

The HLEN value is 8, which means the total number of bytes in the header is 8×4, or 32 bytes. The first 20 bytes are the **base header**, the next 12 bytes are the options.

Example 7.9

In an IPv4 packet, the value of HLEN is 5, and the value of the total length field is $(0028)_{16}$. How many bytes of data are being carried by this packet?

Solution

The HLEN value is 5, which means the total number of bytes in the header is 5×4, or 20 bytes (no options). The total length is $(0028)_{16}$ or 40 bytes, which means the packet is carrying 20 bytes of data $(40 - 20)$.

Example 7.10

An IPv4 packet has arrived with the first few hexadecimal digits as shown.

$$(45000028000100000102 \ldots)_{16}$$

How many hops can this packet travel before being dropped? To which upper-layer protocol do the data belong?

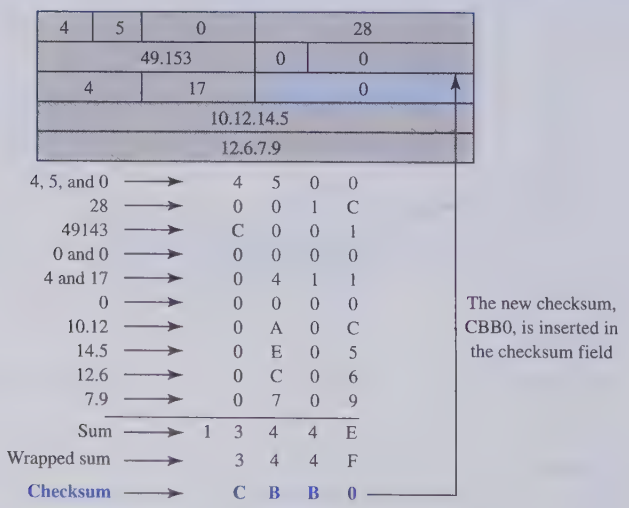
Solution

To find the time-to-live field, we skip 8 bytes (16 hexadecimal digits). The time-to-live field is the ninth byte, which is $(01)_{16}$. This means the packet can travel only one hop. The protocol field is the next byte $(02)_{16}$, which means that the upper-layer protocol is IGMP.

Example 7.11

Figure 7.15 shows an example of a checksum calculation for an IPv4 header without options. The header is divided into 16-bit sections. All the sections are added, and the sum is complemented after wrapping the leftmost digit. The result is inserted in the checksum field.

Figure 7.15 *Example of checksum calculation in IPv4*



Note that the calculation of wrapped sum and checksum can also be done as follows in hexadecimal:

```
Wrapped Sum = Sum mod FFFF
Checksum = FFFF – Wrapped Sum
```

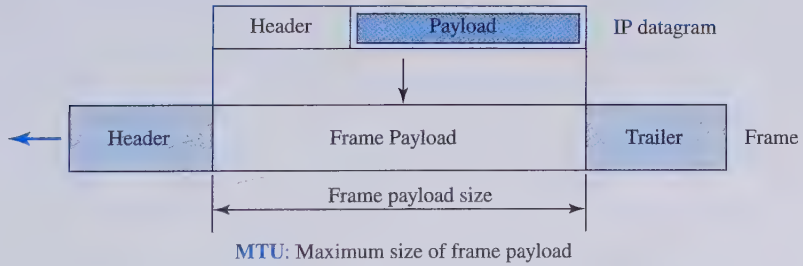
Fragmentation

A datagram can travel through different networks. Each router decapsulates the IP datagram from the frame it receives, processes it, and then encapsulates it in another frame. The format and size of the received frame depend on the protocol used by the physical network through which the frame has just traveled. The format and size of the sent frame depend on the protocol used by the physical network through which the frame is going to travel. For example, if a router connects a LAN to a WAN, it receives a frame in the LAN format and sends a frame in the WAN format.

Maximum Transfer Unit (MTU)

Each link-layer protocol has its own frame format. One of the features of each format is the maximum size of the payload that can be encapsulated. In other words, when a datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size, which is defined by the restrictions imposed by the hardware and software used in the network (see Figure 7.16).

Figure 7.16 *Maximum transfer unit (MTU)*



The value of the maximum transfer unit (MTU) differs from one physical network protocol to another. For example, the value for a LAN is normally 1500 bytes, but for a WAN it can be larger or smaller.

To make the IP protocol independent of the physical network, the designers decided to make the maximum length of the IP datagram equal to 65,535 bytes. This makes transmission more efficient if one day we use a link-layer protocol with an MTU of this size. However, for other physical networks, we must divide the datagram to make it possible for it to pass through these networks. This is called **fragmentation**.

When a datagram is fragmented, each fragment has its own header with most of the fields repeated, but some have been changed. A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU. In other words, a datagram may be fragmented several times before it reaches the final destination.

A datagram can be fragmented by the source host or any router in the path. The *reassembly* of the datagram, however, is done only by the destination host, because each fragment becomes an independent datagram. Whereas the fragmented datagram can travel through different routes, and we can never control or guarantee which route a fragmented datagram may take, all the fragments belonging to the same datagram should finally arrive at the destination host. So it is logical to do the reassembly at the final destination. An even stronger objection for reassembling packets during the transmission is the loss of efficiency it incurs.

When we talk about fragmentation, we mean that the payload of the IP datagram is fragmented. However, most parts of the header, with the exception of some options, must be copied by all fragments. The host or router that fragments a datagram must change the values of three fields: flags, fragmentation offset, and total length. The rest of the fields must be copied. Of course, the value of the checksum must be recalculated regardless of fragmentation.

Fields Related to Fragmentation

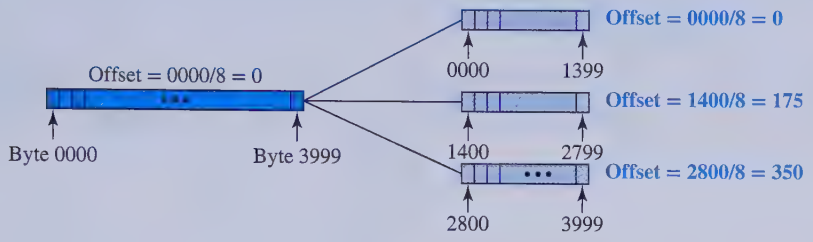
We mentioned earlier that three fields in an IP datagram are related to fragmentation: *identification*, *flags*, and *fragmentation offset*. Let us explain these fields now.

The 16-bit *identification field* identifies a datagram originating from the source host. The combination of the identification and source IP address must uniquely define a datagram as it leaves the source host. To guarantee uniqueness, the IP protocol uses a counter to label the datagrams. The counter is initialized to a positive number. When the IP protocol sends a datagram, it copies the current value of the counter to the identification field and increments the counter by one. As long as the counter is kept in the main memory, uniqueness is guaranteed. When a datagram is fragmented, the value in the identification field is copied into all fragments. In other words, all fragments have the same identification number, which is also the same as the original datagram. The identification number helps the destination in reassembling the datagram. It knows that all fragments having the same identification value should be assembled into one datagram.

The 3-bit *flags field* defines three flags. The leftmost bit is reserved (not used). The second bit (D bit) is called the *do not fragment* bit. If its value is 1, the machine must not fragment the datagram. If it cannot pass the datagram through any available physical network, it discards the datagram and sends an ICMP error message to the source host (discussed in Section 7.4.4). If its value is 0, the datagram can be fragmented if necessary. The third bit (M bit) is called the *more fragment bit*. If its value is 1, it means the datagram is not the last fragment; there are more fragments after this one. If its value is 0, it means this is the last or only fragment.

The 13-bit *fragmentation offset field* shows the relative position of this fragment with respect to the whole datagram. It is the offset of the data in the original datagram measured in units of 8 bytes. Figure 7.17 shows a datagram with a data size of 4000 bytes fragmented into three fragments. The bytes in the original datagram are numbered 0 to 3999. The first fragment carries bytes 0 to 1399. The offset for this datagram is $0/8 = 0$. The second fragment carries bytes 1400 to 2799; the offset value for this fragment is $1400/8 = 175$. Finally, the third fragment carries bytes 2800 to 3999. The offset value for this fragment is $2800/8 = 350$.

Figure 7.17 *Fragmentation example*



Remember that the value of the offset is measured in units of 8 bytes. This is done because the length of the offset field is only 13 bits long and cannot represent a sequence of bytes greater than 8191. This forces hosts or routers that fragment datagrams to choose the size of each fragment so that the first byte number is divisible by 8.

Figure 7.18 shows an expanded view of the fragments in Figure 7.17. The original packet starts at the client; the fragments are reassembled at the server. The value of the identification field is the same in all fragments, as is the value of the flags field with the more fragment bit (M bit) set for all fragments except the last. Also, the value of the offset field for each fragment is shown. Note that although the fragments arrived out of order at the destination, they can be correctly reassembled.

Figure 7.18 also shows what happens if a fragment itself is fragmented. In this case the value of the offset field is always relative to the original datagram. For example, in the figure, the second fragment is itself fragmented later into two fragments of 800 bytes and 600 bytes, but the offset shows the relative position of the fragments to the original data.

It is obvious that even if each fragment follows a different path and arrives out of order, the final destination host can reassemble the original datagram from the fragments received (if none of them is lost) using the following strategy:

a. The first fragment has an offset field value of zero.
b. Divide the length of the first fragment by 8. The second fragment has an offset value equal to that result.
c. Divide the total length of the first and second fragment by 8. The third fragment has an offset value equal to that result.
d. Continue the process. The last fragment has its M bit set to 0.
e. Continue the process. The last fragment has an M bit value of 0.

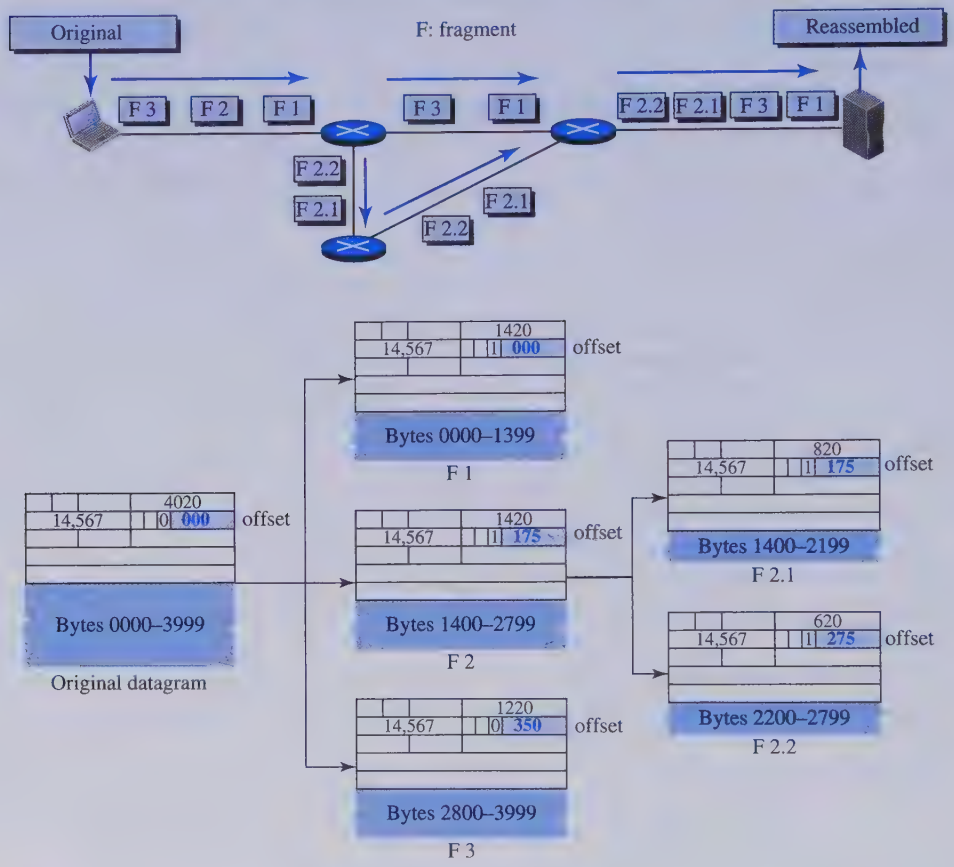
Example 7.12

A packet has arrived with an M bit value of 0. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

Solution

If the M bit is 0, it means that there are no more fragments; the fragment is the last one. However, we cannot say if the original packet was fragmented or not. A nonfragmented packet is considered the last fragment.

Figure 7.18 *Detailed fragmentation example*



Example 7.13

A packet has arrived with an M bit value of 1. Is this the first fragment, the last fragment, or a middle fragment? Do we know if the packet was fragmented?

Solution

If the M bit is 1, it means that there is at least one more fragment. This fragment can be the first one or a middle one, but not the last one. We don't know if it is the first one or a middle one; we need more information (the value of the fragmentation offset).

Example 7.14

A packet has arrived with an M bit value of 1 and a fragmentation offset value of 0. Is this the first fragment, the last fragment, or a middle fragment?

Solution

Because the M bit is 1, it is either the first fragment or a middle one. Because the offset value is 0, it is the first fragment.

Example 7.15

A packet has arrived in which the offset value is 100. What is the number of the first byte? Do we know the number of the last byte?

Solution

To find the number of the first byte, we multiply the offset value by 8. This means that the first byte number is 800. We cannot determine the number of the last byte unless we know the length of the data.

Example 7.16

A packet has arrived in which the offset value is 100, the value of HLEN is 5, and the value of the total length field is 100. What are the numbers of the first byte and the last byte?

Solution

The first byte number is $100 \times 8 = 800$. The total length is 100 bytes, and the header length is 20 bytes (5×4), which means that there are 80 bytes in this datagram. If the first byte number is 800, the last byte number must be 879.

7.4.3 Options

The header of the IPv4 datagram is made of two parts: a fixed part and a variable part. The fixed part is 20 bytes long and was discussed in Figure 7.13. The variable part comprises the options that can be a maximum of 40 bytes (in multiples of 4 bytes) to preserve the boundary of the header.

Options, as the name implies, are not required for a datagram. They can be used for network testing and debugging. Although options are not a required part of the IPv4 header, option processing is required of the IPv4 software. This means that all implementations must be able to handle options if they are present in the header. Options are divided into two broad categories: single-byte options and multiple-byte options.

Single-Byte Options

There are two single-byte options.

No Operation

A **no-operation option** is a 1-byte option used as a filler between options.

End of Option

An **end-of-option option** is a 1-byte option used for padding at the end of the option field. It, however, can only be used as the last option.

Multiple-Byte Options

There are four multiple-byte options.

Record Route

A **record route option** is used to record the Internet routers that handle the datagram. It can list up to nine router addresses. It can be used for debugging and management purposes.

Strict Source Route

A **strict source route option** is used by the source to predetermine a route for the datagram as it travels through the Internet. Dictation of a route by the source can be useful for several purposes. The sender can choose a route with a specific type of service, such as minimum delay or maximum throughput. Alternatively, it may choose a route that is safer or more reliable for the sender's purpose. For example, a sender can choose a route so that its datagram does not travel through a competitor's network.

If a datagram specifies a strict source route, all the routers defined in the option must be visited by the datagram. A router must not be visited if its IPv4 address is not listed in the datagram. If the datagram visits a router that is not on the list, the datagram is discarded and an error message is issued. If the datagram arrives at the destination and some of the entries were not visited, it will also be discarded and an error message issued.

Loose Source Route

A **loose source route option** is similar to the strict source route, but it is less rigid. Each router in the list must be visited, but the datagram can visit other routers as well.

Timestamp

A **timestamp option** is used to record the time of datagram processing by a router. The time is expressed in milliseconds from midnight, Universal Time or Greenwich Mean Time. Knowing the time a datagram is processed can help users and managers track the behavior of the routers in the Internet. We can estimate the time it takes for a datagram to go from one router to another. We say *estimate* because, although all routers may use Universal Time, their local clocks may not be synchronized.

Security of IPv4 Datagrams

The IPv4 protocol, as well as the whole Internet, was started when the Internet users trusted each other. No security was provided for the IPv4 protocol. Today, however, the situation is different; the Internet is not secure anymore. Although we will discuss network security in general and IP security in particular in Chapter 13, here we give a brief idea about the security issues in IP protocol and the solution. There are three security issues that are particularly applicable to the IP protocol: packet sniffing, packet modification, and IP spoofing.

Packet Sniffing

An intruder may intercept an IP packet and make a copy of it. Packet sniffing is a passive attack, in which the attacker does not change the contents of the packet. This type of attack is very difficult to detect because the sender and the receiver may never know that the packet has been copied. Although packet sniffing cannot be stopped, **encryption** of the packet can make the attacker's effort useless. The attacker may still sniff the packet, but the content is not detectable.

Packet Modification

The second type of attack is to modify the packet. The attacker intercepts the packet, changes its contents, and sends the new packet to the receiver. The receiver believes that the packet is coming from the original sender. This type of attack can be detected using a data integrity mechanism. The receiver, before opening and using the contents of the

message, can use this mechanism to make sure that the packet has not been changed during the transmission. We discuss packet integrity in Chapter 13.

IP Spoofing

An attacker can masquerade as somebody else and create an IP packet that carries the source address of another computer. An attacker can send an IP packet to a bank pretending that it is coming from one of the customers. This type of attack can be prevented using an origin authentication mechanism (see Chapter 13).

IPSec

The IP packets today can be protected from the previously mentioned attacks using a protocol called IPSec (IP Security). This protocol, which is used in conjunction with the IP protocol, creates a connection-oriented service between two entities in which they can exchange IP packets without worrying about the three attacks discussed previously. We will discuss IPSec in detail in Chapter 13; here it is enough to mention that IPSec provides the following four services:

- **Defining algorithms and keys.** The two entities that want to create a secure channel between themselves can agree on some available algorithms and keys to be used for security purposes.
- **Packet encryption.** The packets exchanged between two parties can be encrypted for privacy using one of the encryption algorithms and a shared key agreed upon in the first step. This makes the packet sniffing attack useless.
- **Data integrity.** Data integrity guarantees that the packet is not modified during the transmission. If the received packet does not pass the data integrity test, it is discarded. This prevents the second attack, packet modification, as was described previously.
- **Origin authentication.** IPSec can authenticate the origin of the packet to be sure that the packet is not created by an imposter. This can prevent IP spoofing attacks as was described previously.

7.4.4 ICMPv4

The IPv4 has no error-reporting or error-correcting mechanism. What happens if something goes wrong? What happens if a router must discard a datagram because it cannot find a route to the final destination or because the time-to-live field has a zero value? What happens if the final destination host must discard the received fragments of a datagram because it has not received all fragments within a predetermined time limit? These are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host.

The IP protocol also lacks a mechanism for host and management queries. A host sometimes needs to determine if a router or another host is alive. And sometimes a network manager needs information from another host or router.

The **Internet Control Message Protocol version 4 (ICMPv4)** has been designed to compensate for the above two deficiencies. It is a companion to the IP protocol. ICMP itself is a network-layer protocol. However, its messages are not passed directly to the data-link layer as would be expected. Instead, the messages are first encapsulated inside

IP datagrams before going to the lower layer. When an IP datagram encapsulates an ICMP message, the value of the protocol field in the IP datagram is set to 1 to indicate that the IP payroll is an ICMP message.

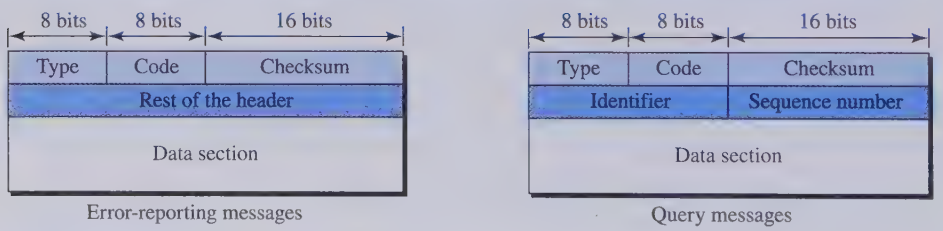
MESSAGES

ICMP messages are divided into two broad categories: **error-reporting messages** and **query messages**. The error-reporting messages report problems that a router or a host (destination) may encounter when it processes an IP packet. The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host. For example, nodes can discover their neighbors. Also, hosts can discover and learn about routers on their network and routers can help a node redirect its messages.

An ICMP message has an 8-byte header and a variable-size data section. Although the general format of the header is different for each message type, the first 4 bytes are common to all. As Figure 7.19 shows, the first field, ICMP type, defines the type of the message. The code field specifies the reason for the particular message type. The last common field is the checksum field (to be discussed later in this section). The rest of the header is specific for each message type.

The data section in error messages carries information for finding the original packet that had the error. In query messages, the data section carries extra information based on the type of the query.

Figure 7.19 *General format of ICMP messages*



Type and code values

Error-reporting messages	Query messages
03: Destination unreachable (codes 0 to 15)	08 and 00: Echo request and reply (only code 0)
04: Source quench (only code 0)	13 and 14: Timestamp request and reply (only code 0)
05: Redirection (codes 0 to 3)	
11: Time exceeded (codes 0 and 1)	
12: Parameter problem (codes 0 and 1)	

Error-Reporting Messages

Because IP is an unreliable protocol, one of the main responsibilities of ICMP is to report some errors that may occur during the processing of the IP datagram. ICMP does not correct errors; it simply reports them. Error correction is left to the higher-level protocols. Error messages are always sent to the original source because the only information available in the datagram about the route is the source and destination IP addresses. ICMP uses

the source IP address to send the error message to the source (originator) of the datagram. To make the error-reporting process simple, ICMP follows some rules in reporting messages. First, no error message will be generated for a datagram having a multicast address or special address (such as *this host* or *loopback*). Second, no ICMP error message will be generated in response to a datagram carrying an ICMP error message. Third, no ICMP error message will be generated for a fragmented datagram that is not the first fragment.

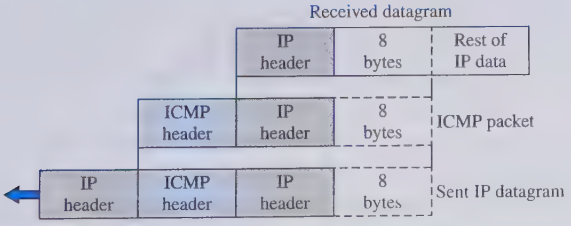
Note that all error messages contain a data section that includes the IP header of the original datagram plus the first 8 bytes of data in that datagram. The original datagram header is added to give the original source, which receives the error message, information about the datagram itself. The 8 bytes of data are included because the first 8 bytes provide information about the port numbers (UDP and TCP) and sequence number (TCP). This information is needed so the source can inform the protocols (TCP or UDP) about the error.

The following are important points about ICMP error messages:

- No ICMP error message will be generated in response to a datagram carrying an ICMP error message.
- No ICMP error message will be generated for a fragmented datagram that is not the first fragment.
- No ICMP error message will be generated for a datagram having a multicast address.
- No ICMP error message will be generated for a datagram having a special address such as 127.0.0.0 or 0.0.0.0.

Note that all error messages contain a data section that includes the IP header of the original datagram plus the first 8 bytes of data in that datagram. The original datagram header is added to give the original source, which receives the error message, information about the datagram itself. The 8 bytes of data are included because, as we will see in Chapter 9 on UDP and TCP protocols, the first 8 bytes provide information about the port numbers (UDP and TCP) and sequence number (TCP). This information is needed so the source can inform the protocols (TCP or UDP) about the error. ICMP forms an error packet, which is then encapsulated in an IP datagram (see Figure 7.20).

Figure 7.20 *Contents of the data field for the error messages*



Destination Unreachable

The most widely used error message is the destination unreachable (type 3) message. This message uses different codes (0 to 15) to define the type of error message and the reason why a datagram has not reached its final destination. For example, code 0 tells the source that a host is unreachable. This may happen, for example, when we use the HTTP protocol to access a web page, but the server is down. The message "destination host is not reachable" is created and sent back to the source.

Source Quench

Another error message is called the source quench (type 4) message, which informs the sender that the network has encountered congestion and the datagram has been dropped. The source needs to slow down the sending of more datagrams. In other words, ICMP adds a kind of congestion-control mechanism to the IP protocol by using this type of message.

Redirection Message

The redirection message (type 5) is used when the source uses a wrong router to send out its message. The router redirects the message to the appropriate router but informs the source that it needs to change its default router in the future. The IP address of the default router is sent in the message.

We discussed the purpose of the *time-to-live* (TTL) field in the IP datagram and explained that it prevents a datagram from being aimlessly circulated in the Internet. When the TTL value becomes 0, the datagram is dropped by the visiting router and a *time-exceeded* message (type 11) with code 0 is sent to the source to inform it about the situation. The time-exceeded message (with code 1) can also be sent when not all fragments of a datagram arrive within a predefined period of time.

Parameter Problem

A parameter problem message (type 12) can be sent when either there is a problem in the header of a datagram (code 0) or some options are missing or cannot be interpreted (code 1).

Query Messages

Interestingly, query messages in ICMP can be used independently without relation to an IP datagram. Of course, a query message needs to be encapsulated in a datagram, as a carrier. Query messages are used to probe or test the liveliness of hosts or routers in the internet, find the one-way or the round-trip time for an IP datagram between two devices, or even find out whether the clocks in two devices are synchronized. Naturally, query messages come in pairs: request and reply.

The *echo-request* (type 8) and the *echo-reply* (type 0) pair of messages are used by a host or a router to test the liveliness of another host or router. A host or router sends an echo-request message to another host or router; if the latter is alive, it responds with an echo-reply message. We shortly see the applications of this pair in two debugging tools: *ping* and *traceroute*.

The *timestamp request* (type 13) and the *timestamp reply* (type 14) pair of messages are used to find the round-trip time between two devices or to check whether the clocks

in two devices are synchronized. The timestamp request message sends a 32-bit number, which defines the time the message is sent. The timestamp reply resends that number but also includes two new 32-bit numbers representing the time the request was received and the time the response was sent. If all timestamps represent Universal Time, the sender can calculate the one-way and round-trip time.

Debugging Tools

There are several tools that can be used in the Internet for debugging. We can determine the viability of a host or router. We can trace the route of a packet. We introduce two tools that use ICMP for debugging: *ping* and *traceroute*.

Ping

We can use the *ping* program to find if a host is alive and responding. We use *ping* here to see how it uses ICMP packets. The source host sends ICMP echo-request messages; the destination, if alive, responds with ICMP echo-reply messages. The *ping* program sets the identifier field in the echo-request and echo-reply message and starts the sequence number from 0; this number is incremented by 1 each time a new message is sent. Note that *ping* can calculate the round-trip time. It inserts the sending time in the data section of the message. When the packet arrives, it subtracts the arrival time from the departure time to get the round-trip time (RTT).

Example 7.17

The following shows how we send a *ping* message to the auniversity.edu site. We set the identifier field in the echo-request and echo-reply message and start the sequence number from 0; this number is incremented by one each time a new message is sent. Note that *ping* can calculate the round-trip time. It inserts the sending time in the data section of the message. When the packet arrives, it subtracts the arrival time from the departure time to get the *round-trip time* (rtt).

```
$ ping auniversity.edu
PING auniversity.edu (152.181.8.3)  56 (84)  bytes of data.
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=0   ttl=62   time=1.91 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=1   ttl=62   time=2.04 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=2   ttl=62   time=1.90 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=3   ttl=62   time=1.97 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=4   ttl=62   time=1.93 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=5   ttl=62   time=2.00 ms
--- auniversity.edu statistics ---
6 packets transmitted, 6 received, 0% packet loss
rtt min/avg/max = 1.90/1.95/2.04 ms
```

Traceroute or Tracert

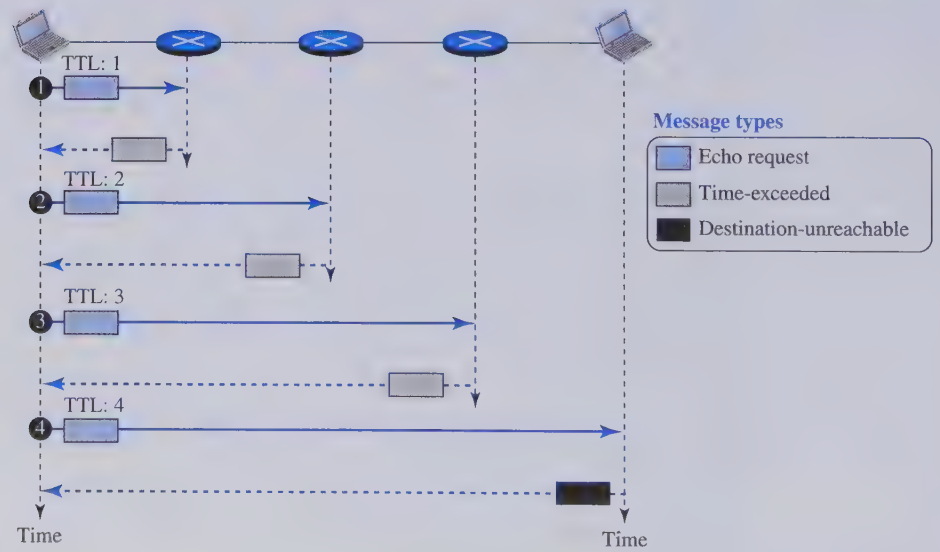
The ***traceroute*** program in UNIX or ***tracert*** in Windows can be used to trace the path of a packet from a source to the destination. It can find the IP addresses of all the routers that are visited along the path. The program is usually set to check for the maximum of

30 hops (routers) to be visited. The number of hops in the Internet is normally less than this. Because these two programs behave different in Unix and Windows, we explain them separately.

Traceroute

The traceroute program is different from the ping program. The ping program gets help from two query messages; the traceroute program gets help from two error-reporting messages: time exceeded and destination unreachable. The traceroute is an application-layer program, but only the client program is needed, because, as we can see, the client program never reaches the application layer in the destination host. In other words, there is no *traceroute* server program. The *traceroute* application program is encapsulated in a UDP user datagram, but *traceroute* intentionally uses a port number that is not available at the destination. If there are n routers in the path, the *traceroute* program sends $(n + 1)$ messages. The first n messages are discarded by the n routers, one by each router; the last message is discarded by the destination host. The *traceroute* client program uses the $(n + 1)$ ICMP error-reporting messages received to find the path between the routers. We will show shortly that the *traceroute* program does not need to know the value of n; it is found automatically. In Figure 7.21, the value of n is 3.

Figure 7.21 *Use of ICMPv4 in traceroute*



The first *traceroute* message is sent with the time-to-live (TTL) value set to 1; the message is discarded at the first router and a time-exceeded ICMP error message is sent, from which the *traceroute* program can find the IP address of the first router

(the source IP address of the error message) and the router name (in the data section of the message). The second *traceroute* message is sent with TTL set to 2, which can find the IP address and the name of the second router. Similarly, the third message can find the information about router 3. The fourth message, however, reaches the destination host. This host is also dropped, but for another reason. The destination host cannot find the port number specified in the UDP user datagram. This time ICMP sends a different message, the destination-unreachable message with code 3 to show the port number is not found. After receiving this different ICMP message, the *traceroute* program knows that the final destination is reached. It uses the information in the received message to find the IP address and the name of the final destination.

The *traceroute* program also sets a timer to find the round-trip time for each router and the destination. Most *traceroute* programs send three messages to each device, with the same TTL value, to be able to find a better estimate for the round-trip time. The following shows an example of a *traceroute* program, which uses three probes for each device and gets three RTTs.

```
$ traceroute printers.com
traceroute to printers.com (13.1.69.93), 30 hops max, 38 byte packets
1  route.front.edu    (153.18.31.254)   0.622 ms   0.891 ms   0.875 ms
2  ceneric.net        (137.164.32.140)  3.069 ms   2.875 ms   2.930 ms
3  satire.net         (132.16.132.20)   3.071 ms   2.876 ms   2.929 ms
4  alpha.printers.com (13.1.69.93)      5.922 ms   5.048 ms   4.922 ms
```

Tracert

The *tracert* program in windows behaves differently. The *tracert* messages are encapsulated directly in IP datagrams. The *tracert* like *traceroute* sends echo-request messages. However, when the last echo request reaches the destination host, an echo-reply message is issued.

ICMP Checksum

In ICMP the checksum is calculated over the entire message (header and data).

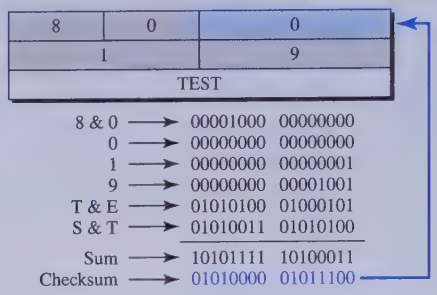
Example 7.18

Figure 7.22 shows an example of checksum calculation for a simple echo-request message. We randomly chose the identifier to be 1 and the sequence number to be 9. The message is divided into 16-bit (2-byte) words. The words are added, and the sum is complemented. Now the sender can put this value in the checksum field.

7.4.5 Mobile IP

In this section, we discuss mobile IP. As mobile and personal computers such as notebooks become increasingly popular, we need to think about mobile IP, the extension of IP that allows mobile computers to be connected to the Internet at any location where the connection is possible.

Figure 7.22 *Example of checksum calculation*



Addressing

The main problem that must be solved in providing mobile communication using IP is addressing.

Stationary Hosts

The original IP addressing was based on the assumption that a host is stationary, attached to one specific network. A router uses an IP address to route an IP datagram. As we will learn in chapter 8, an IP address has two parts: a prefix and a suffix. The prefix associates a host with a network. For example, the IP address 10.3.4.24/8 defines a host attached to the network 10.0.0.0/8. This implies that a host in the Internet does not have an address that it can carry with itself from one place to another. The address is valid only when the host is attached to the network. If the network changes, the address is no longer valid. Routers use this association to route a packet; they use the prefix to deliver the packet to the network to which the host is attached. This scheme works perfectly with **stationary hosts**.

The IP addresses are designed to work with stationary hosts because part of the address defines the network to which the host is attached.

Mobile Hosts

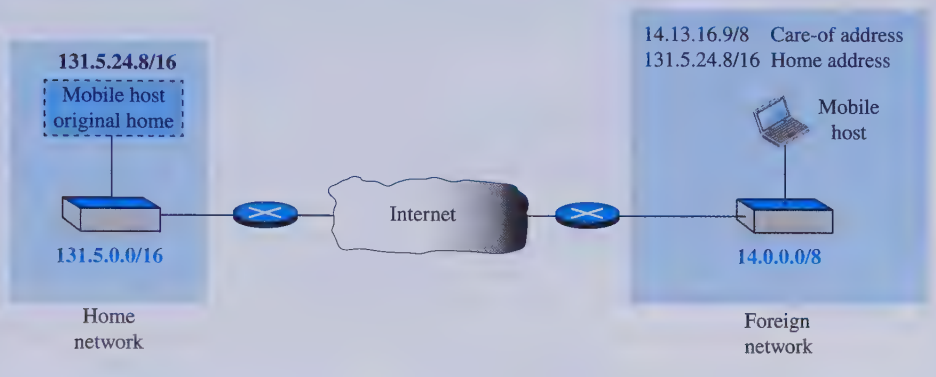
When a host moves from one network to another, the IP addressing structure needs to be modified. Several solutions have been proposed.

Changing the Address One simple solution is to let the **mobile host** change its address as it goes to the new network. This does not happen today because it creates many problems.

Two Addresses The approach that is more feasible is the use of two addresses. The host has its original address, called the **home address**, and a temporary address, called the **care-of address**. The home address is permanent; it associates the host to its **home network**, the network that is the permanent home of the host. The care-of address is

temporary. When a host moves from one network to another, the care-of address changes; it is associated with the **foreign network**, the network to which the host moves. Figure 7.23 shows the concept.

Figure 7.23 *Home address and care-of address*



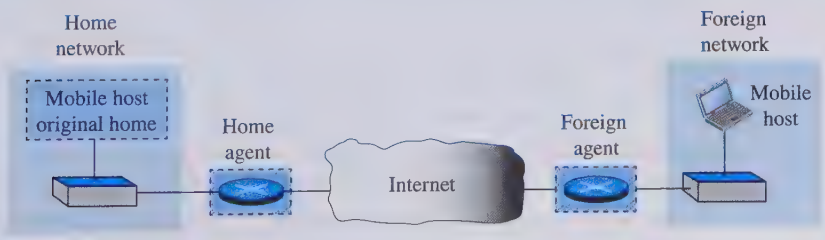
Mobile IP has two addresses for a mobile host: one home address and one care-of address. The home address is permanent; the care-of address changes as the mobile host moves from one network to another.

When a mobile host visits a foreign network, it receives its care-of address during the agent discovery and registration phase, described shortly.

Agents

To make the change of address transparent to the rest of the Internet requires a **home agent** and a **foreign agent**. Figure 7.24 shows the position of a home agent relative to the home network and a foreign agent relative to the foreign network.

Figure 7.24 *Home agent and foreign agent*



We have shown the home and the foreign agents as routers, but we need to emphasize that their specific function as an agent is performed in the application layer. In other words, they are both routers and hosts.

Home Agent

The home agent is usually a router attached to the home network of the mobile host. The home agent acts on behalf of the mobile host when a remote host sends a packet to the mobile host. The home agent receives the packet and sends it to the foreign agent.

Foreign Agent

The foreign agent is usually a router attached to the foreign network. The foreign agent receives and delivers packets sent by the home agent to the mobile host.

The mobile host can also act as a foreign agent. In other words, the mobile host and the foreign agent can be the same. However, to do this, a mobile host must be able to receive a care-of address by itself, which can be done through the use of DHCP. In addition, the mobile host needs the necessary software to allow it to communicate with the home agent and to have two addresses: its home address and its care-of address. This dual addressing must be transparent to the application programs.

When the mobile host acts as a foreign agent, the care-of address is called a **collocated care-of address**.

When the mobile host and the foreign agent are the same, the care-of address is called a collocated care-of address.

The advantage of using a collocated care-of address is that the mobile host can move to any network without worrying about the availability of a foreign agent. The disadvantage is that the mobile host needs extra software to act as its own foreign agent.

Three Phases

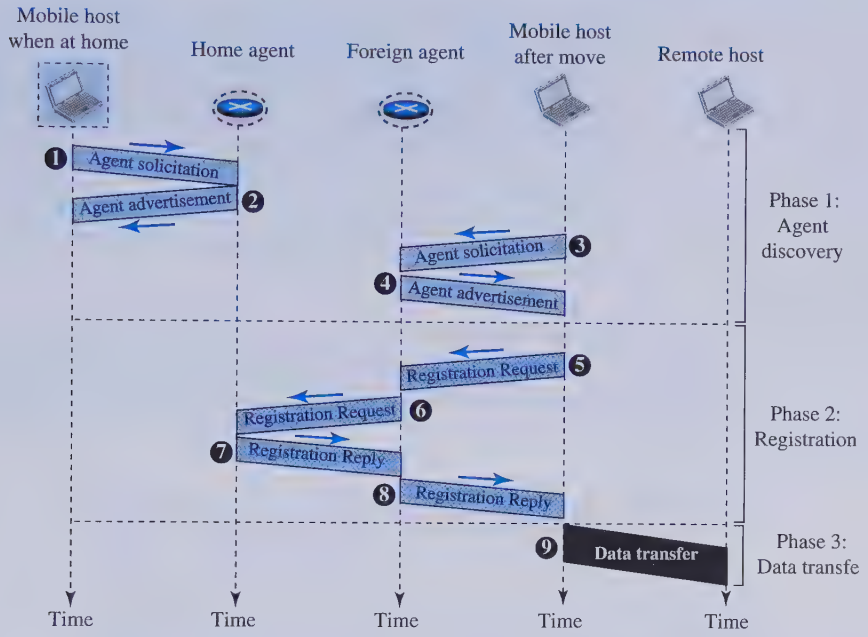
To communicate with a remote host, a mobile host goes through three phases: agent discovery, registration, and data transfer, as shown in Figure 7.25.

The first phase, agent discovery, involves the mobile host, the foreign agent, and the home agent. The second phase, registration, also involves the mobile host and the two agents. Finally, in the third phase, the remote host is also involved. We discuss each phase separately.

Agent Discovery

The first phase in mobile communication, *agent discovery*, consists of two subphases. A mobile host must discover (learn the address of) a home agent before it leaves its home network. A mobile host must also discover a foreign agent after it has moved to a foreign network. This discovery consists of learning the care-of address as well as the foreign agent's address. The discovery involves two types of messages: advertisement and solicitation.

Figure 7.25 *Remote host and mobile host communication*



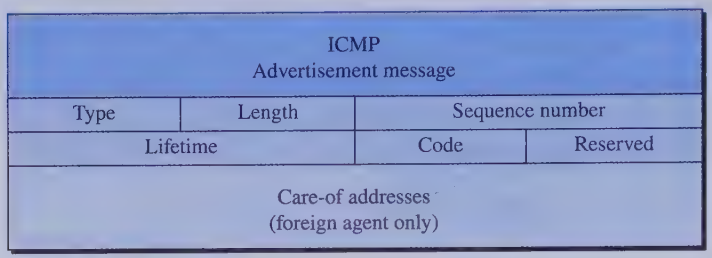
Agent Advertisement

When a router advertises its presence on a network using an ICMP router advertisement, it can append an *agent advertisement* to the packet if it acts as an agent. Figure 7.26 shows how an agent advertisement is piggybacked to the router advertisement packet.

The field descriptions are as follows:

- **Type.** The 8-bit type field is set to 16.
- **Length.** The 8-bit length field defines the total length of the extension message (not the length of the ICMP advertisement message).
- **Sequence number.** The 16-bit sequence number field holds the message number. The recipient can use the sequence number to determine if a message is lost.
- **Lifetime.** The lifetime field defines the number of seconds during which the agent will accept requests. If the value is a string of 1s, the lifetime is infinite.
- **Code.** The code field is an 8-bit flag in which each bit is set (1) or unset (0). The meanings of the bits are shown in Table 7.1.
- **Care-of addresses.** This field contains a list of addresses available for use as care-of addresses. The mobile host can choose one of these addresses. The selection of this care-of address is announced in the registration request. Note that this field is used only by a foreign agent.

Figure 7.26 *Agent advertisement*



Mobile IP does not use a new packet type for agent advertisement; it uses the router advertisement packet of ICMP and appends an agent advertisement message.

Table 7.1 *Code Bits*

Bit	*Meaning*
0	Registration required. No collocated care-of address.
1	Agent is busy and does not accept registration at this moment.
2	Agent acts as a home agent.
3	Agent acts as a foreign agent.
4	Agent uses minimal encapsulation.
5	Agent uses generic routing encapsulation (GRE).
6	Agent supports header compression.
7	Unused (0).

Agent Solicitation

When a mobile host has moved to a new network and has not received agent advertisements, it can initiate an *agent solicitation.* It can use the ICMP solicitation message to inform an agent that it needs assistance.

Mobile IP does not use a new packet type for agent solicitation; it uses the router solicitation packet of ICMP.

Registration

The second phase in mobile communication is *registration.* After a mobile host has moved to a foreign network and discovered the foreign agent, it must register. There are four aspects of registration:

1. The mobile host must register itself with the foreign agent.

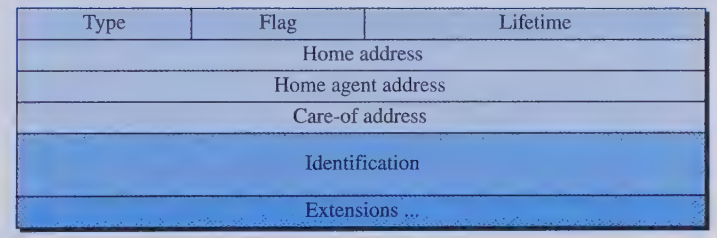
2. The mobile host must register itself with its home agent. This is normally done by the foreign agent on behalf of the mobile host.
3. The mobile host must renew registration if it has expired.
4. The mobile host must cancel its registration (deregistration) when it returns home.

Request and Reply

To register with the foreign agent and the home agent, the mobile host uses a *registration request* and a registration reply, as shown in Figure 7.25.

Registration Request A registration request is sent from the mobile host to the foreign agent to register its care-of address and also to announce its home address and home agent address. The foreign agent, after receiving and registering the request, relays the message to the home agent. Note that the home agent now knows the address of the foreign agent because the IP packet that is used for relaying has the IP address of the foreign agent as the source address. Figure 7.27 shows the format of the registration request.

Figure 7.27 *Registration request format*



The field descriptions are as follows:

- **Type.** The 8-bit type field defines the type of the message. For a request message the value of this field is 1.
- **Flag.** The 8-bit flag field defines forwarding information. The value of each bit can be set or unset. The meaning of each bit is given in Table 7.2.

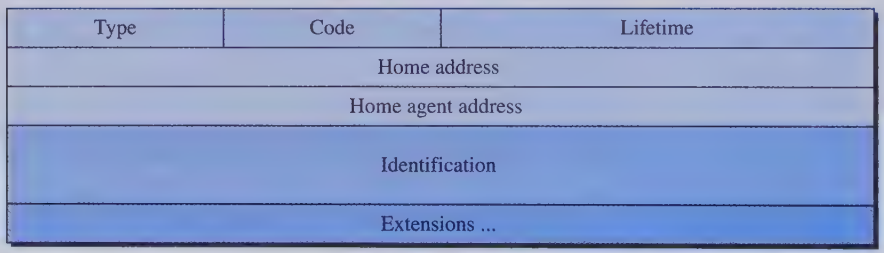
Table 7.2 *Registration request flag field bits*

Bit	*Meaning*
0	Mobile host requests that home agent retain its prior care-of address.
1	Mobile host requests that home agent tunnel any broadcast message.
2	Mobile host is using collocated care-of address.
3	Mobile host requests that home agent use minimal encapsulation.
4	Mobile host requests generic routing encapsulation (GRE).
5	Mobile host requests header compression.
6 –7	Reserved bits.

- **Lifetime.** This field defines the number of seconds the registration is valid. If the field is a string of 0s, the request message is asking for deregistration. If the field is a string of 1s, the lifetime is infinite.
- **Home address.** This field contains the permanent (first) address of the mobile host.
- **Home agent address.** This field contains the address of the home agent.
- **Care-of address.** This field is the temporary (second) address of the mobile host.
- **Identification.** This field contains a 64-bit number that is inserted into the request by the mobile host and repeated in the reply message. It matches a request with a reply.
- **Extensions.** Variable-length extensions are used for authentication. They allow a home agent to authenticate the mobile agent. We discuss authentication in Chapter 13.

Registration Reply A registration reply is sent from the home agent to the foreign agent and then relayed to the mobile host. The reply confirms or denies the registration request. Figure 7.28 shows the format of the registration reply.

Figure 7.28 *Registration reply format*



The fields are similar to those of the registration request with the following exceptions. The value of the type field is 3. The code field replaces the flag field and shows the result of the registration request (acceptance or denial). The care-of address field is not needed.

Encapsulation

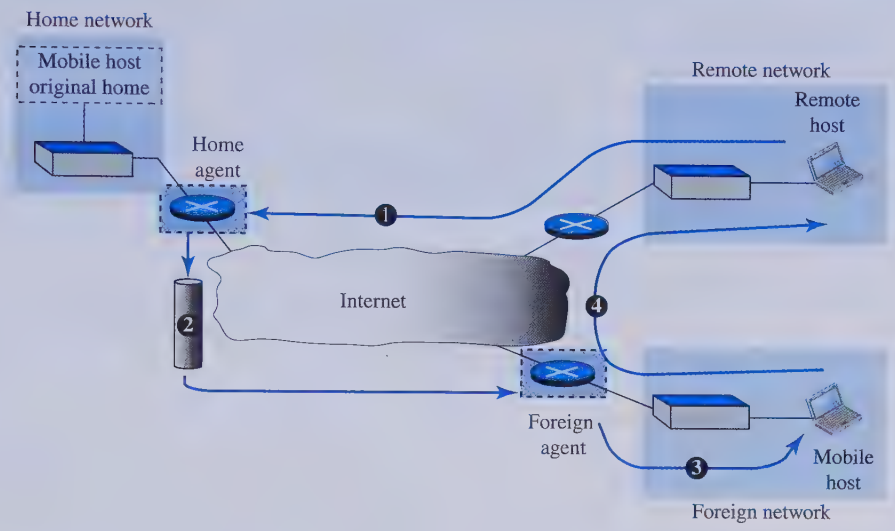
Registration messages are encapsulated in a UDP user datagram. An agent uses the well-known port 434; a mobile host uses an ephemeral port.

A registration request or reply is sent by UDP using the well-known port 434.

Data Transfer

After agent discovery and registration, a mobile host can communicate with a remote host. Figure 7.29 shows the idea.

Figure 7.29 *Data transfer*



From Remote Host to Home Agent

When a remote host wants to send a packet to the mobile host, it uses its address as the source address and the home address of the mobile host as the destination address. In other words, the remote host sends a packet as though the mobile host is at its home network. The packet, however, is intercepted by the home agent, which pretends it is the mobile host. Path 1 of Figure 7.29 shows this step.

From Home Agent to Foreign Agent

After receiving the packet, the home agent sends the packet to the foreign agent. The home agent encapsulates the whole IP packet inside another IP packet using its address as the source and the foreign agent's address as the destination. Path 2 of Figure 7.29 shows this step.

From Foreign Agent to Mobile Host

When the foreign agent receives the packet, it removes the original packet. However, because the destination address is the home address of the mobile host, the foreign agent consults a registry table to find the care-of address of the mobile host. (Otherwise, the packet would just be sent back to the home network.) The packet is then sent to the care-of address. Path 3 of Figure 7.29 shows this step.

From Mobile Host to Remote Host

When a mobile host wants to send a packet to a remote host (for example, a response to the packet it has received), it sends as it does normally. The mobile host prepares a packet with its home address as the source, and the address of the remote host as the destination. Although the packet comes from the foreign network, it has the home address of the mobile host. Path 4 of Figure 7.29 shows this step.

Transparency

In this data transfer process, the remote host is unaware of any movement by the mobile host. The remote host sends packets using the home address of the mobile host as the destination address; it receives packets that have the home address of the mobile host as the source address. The movement is totally transparent. The rest of the Internet is not aware of the mobility of the moving host.

The movement of the mobile host is transparent to the rest of the Internet.

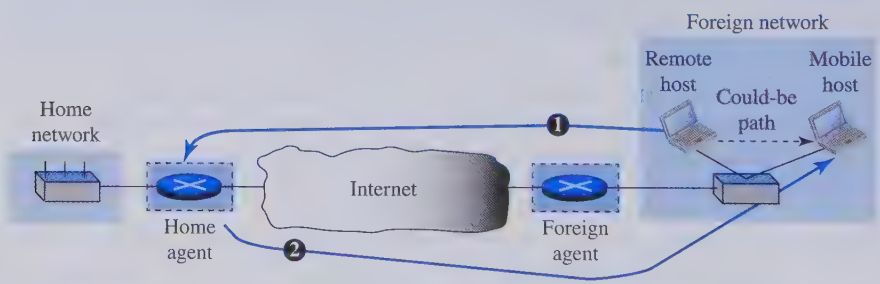
Inefficiency in Mobile IP

Communication involving mobile IP can be inefficient. The inefficiency can be severe or moderate. The severe case is called *double crossing* or *2X*. The moderate case is called *triangle routing* or *dog-leg routing*.

Double Crossing

Double crossing occurs when a remote host communicates with a mobile host that has moved to the same network (or site) as the remote host (see Figure 7.30) When the mobile host sends a packet to the remote host, there is no inefficiency; the communication is local. However, when the remote host sends a packet to the mobile host, the packet crosses the Internet twice. Because a computer usually communicates with other local computers (principle of locality), the inefficiency from double crossing is significant.

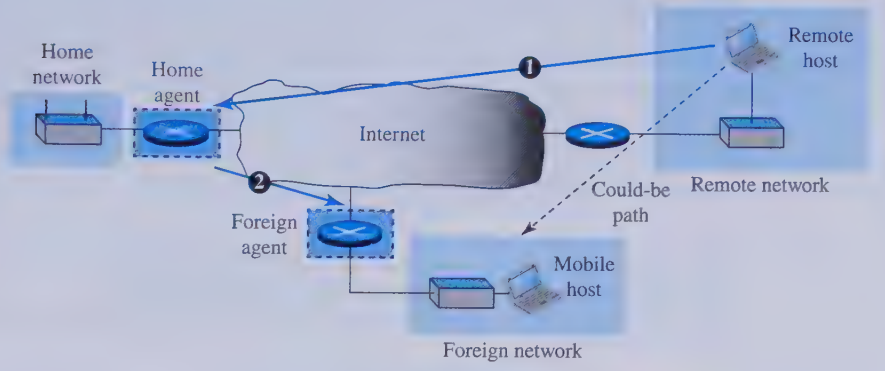
Figure 7.30 *Double crossing*



Triangle Routing

Triangle routing, the less severe case, occurs when the remote host communicates with a mobile host that is not attached to the same network (or site) as the mobile host. When the mobile host sends a packet to the remote host, there is no inefficiency. However, when the remote host sends a packet to the mobile host, the packet goes from the remote host to the home agent and then to the mobile host. The packet travels the two sides of a triangle, instead of just one side (see Figure 7.31).

Figure 7.31 *Triangle routing*



Solution

One solution to inefficiency is for the remote host to bind the care-of address to the home address of a mobile host. For example, when a home agent receives the first packet for a mobile host, it forwards the packet to the foreign agent; it could also send an *update binding packet* to the remote host so that future packets to this host could be sent to the care-of address. The remote host can keep this information in a cache.

The problem with this strategy is that the cache entry becomes outdated once the mobile host moves. In this case the home agent needs to send a *warning packet* to the remote host to inform it of the change.

7.4.6 Forwarding of IP Packets

Forwarding means to deliver the packet to the next hop (which can be the final destination or the intermediate connecting device). Although the IP was originally designed as a connectionless protocol, today the tendency is to change it to a connection-oriented protocol. We discuss both cases.

When IP is used as a connectionless protocol, forwarding is based on the destination address of the IP datagram; when the IP is used as a connection-oriented protocol, forwarding is based on the label attached to an IP datagram.

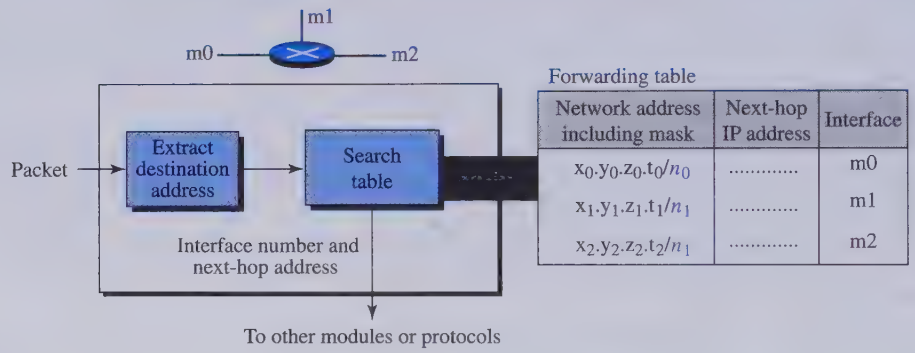
Forwarding Based on the Destination Address

We first discuss forwarding based on the destination address. This is a traditional approach, which is prevalent today. In this case, forwarding requires a host or a router to have a forwarding table. When a host has a packet to send or when a router has received a packet to be forwarded, it looks at this table to find the next hop to deliver the packet to.

In classless addressing, the whole address space is one entity; there are no classes. This means that forwarding requires one row of information for each block involved.

The table needs to be searched based on the network address (first address in the block). Unfortunately, the destination address in the packet gives no clue about the network address. To solve the problem, we need to include the mask (/*n*) in the table. In other words, a classless forwarding table needs to include four pieces of information: the mask, the network address, the interface number, and the IP address of the next router. However, we often see in the literature that the first two pieces are combined. For example, if *n* is 26 and the network address is 180.70.65.192, then one can combine the two as one piece of information: 180.70.65.192/26. Figure 7.32 shows a simple forwarding module and forwarding table for a router with only three interfaces.

Figure 7.32 *Simplified forwarding module in classless address*



The job of the forwarding module is to search the table, row by row. In each row, the *n* leftmost bits of the destination address (prefix) are kept and the rest of the bits (suffix) are set to 0s. If the resulting address (which we call the *network address*), matches with the address in the first column, the information in the next two columns is extracted; otherwise the search continues. Normally, the last row has a default value in the first column (not shown in Figure 7.32), which indicates all destination addresses that did not match the previous rows.

Sometimes, the literature explicitly shows the value of the *n* leftmost bits that should be matched with the *n* leftmost bits of the destination address. The concept is the same, but the presentation is different. For example, instead of giving the address-mask combination of 180.70.65.192/26, we can give the value of the 26 leftmost bits:

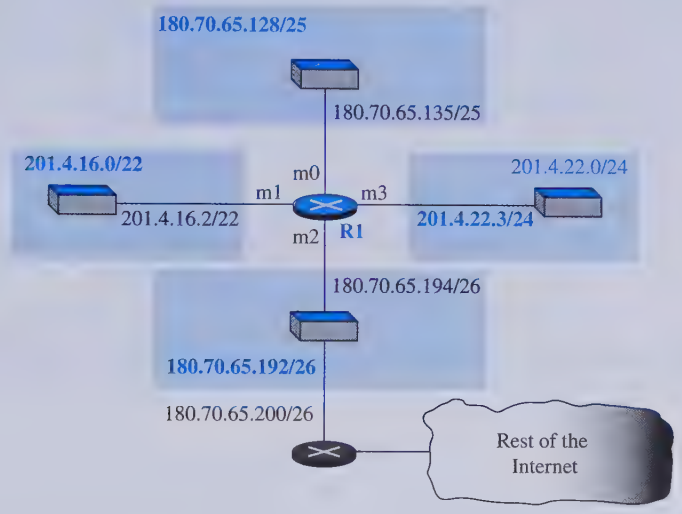
```
10110100  01000110  01000001  11
```

Note that we still need to use an algorithm to find the prefix and compare it with the bit pattern. In other words, the algorithm is still needed, but the presentation is different. We use this format in our forwarding tables in the exercises when we use smaller address spaces just for practice.

Example 7.19

Make a forwarding table for router R1 using the configuration in Figure 7.33.

Figure 7.33 *Configuration for Example 7.19*



Solution

Table 7.3 shows the corresponding table.

Table 7.3 *Forwarding table for router R1 in Figure 7.33*

Network address/mask	*Next hop*	*Interface*
180.70.65.192/**26**	—	m2
180.70.65.128/**25**	—	m0
201.4.22.0/**24**	—	m3
201.4.16.0/**22**	—	m1
Default	180.70.65.200	m2

Example 7.20

Instead of Table 7.3, we can use Table 7.4, in which the network address/mask is given in bits.

Table 7.4 *Forwarding table for router R1 in Figure 7.33 using prefix bits*

Leftmost bits in the destination address	*Next hop*	*Interface*
10110100 01000110 01000001 11	—	m2
10110100 01000110 01000001 1	—	m0
11001001 00000100 00011100	—	m3
11001001 00000100 000100	—	m1
Default	180.70.65.200	m2

When a packet arrives whose leftmost 26 bits in the destination address match the bits in the first row, the packet is sent out from interface m2. When a packet arrives whose leftmost 25 bits in the address match the bits in the second row, the packet is sent out from interface m0. And so on. Table 7.4 clearly shows that the first row has the longest prefix and the fourth row has the shortest prefix. The longer prefix means a smaller range of addresses; the shorter prefix means a larger range of addresses.

Example 7.21

Show the forwarding process if a packet arrives at R1 in Figure 7.33 with the destination address 180.70.65.140.

Solution

The router performs the following steps:

1. The first mask (/26) is applied to the destination address. The result is 180.70.65.128, which does not match the corresponding network address.
2. The second mask (/25) is applied to the destination address. The result is 180.70.65.128, which matches the corresponding network address. The next-hop address and the interface number m0 are extracted for forwarding the packet.

Address Aggregation

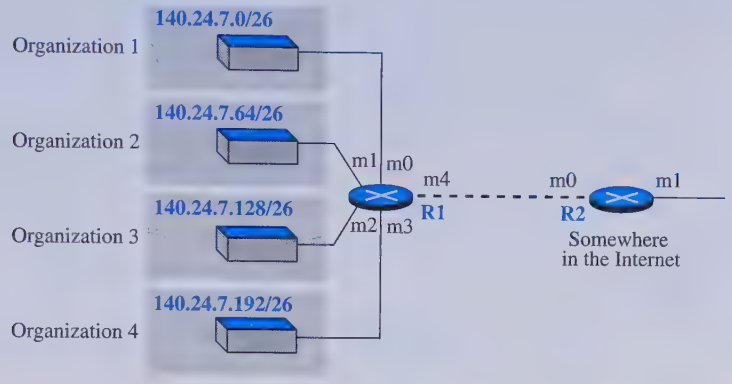
When we use classful addressing, there is only one entry in the forwarding table for each site outside the organization. The entry defines the site even if that site is subnetted. When a packet arrives at the router, the router checks the corresponding entry and forwards the packet accordingly. When we use classless addressing, it is likely that the number of forwarding table entries will increase. This is because the intent of classless addressing is to divide up the whole address space into manageable blocks. The increased size of the table results in an increase in the amount of time needed to search the table. To alleviate the problem, the idea of address aggregation was designed. In Figure 7.34 we have two routers.

R1 is connected to networks of four organizations that each use 64 addresses. R2 is somewhere far from R1. R1 has a longer forwarding table because each packet must be correctly routed to the appropriate organization. R2, on the other hand, can have a very small forwarding table. For R2, any packet with destination 140.24.7.0 to 140.24.7.255 is sent out from interface m0 regardless of the organization number. This is called address aggregation because the blocks of addresses for four organizations are aggregated into one larger block. R2 would have a longer forwarding table if each organization had addresses that could not be aggregated into one block.

Longest Mask Matching

What happens if one of the organizations in Figure 7.34 is not geographically close to the other three? For example, if organization 4 cannot be connected to router R1 for some reason, can we still use the idea of address aggregation and still assign block 140.24.7.192/26 to organization 4? The answer is yes, because routing in classless addressing uses another principle, *longest mask matching*. This principle states that the forwarding table is sorted from the longest mask to the shortest mask. In other words, if there are three masks, /27, /26, and /24, then mask /27 must be the first entry and /24 must be the last. Let us see if this principle solves the situation in which organization 4 is separated from the other three organizations. Figure 7.35 shows the situation.

Figure 7.34 *Address aggregation*



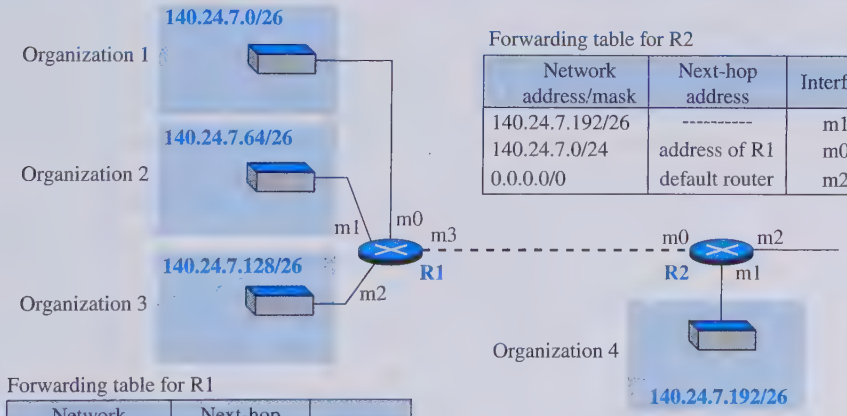
Forwarding table for R1

Network address/mask	Next-hop address	Interface
140.24.7.0/26	----------	m0
140.24.7.64/26	----------	m1
140.24.7.128/26	----------	m2
140.24.7.192/26	----------	m3
0.0.0.0/0	address of R2	m4

Forwarding table for R2

Network address/mask	Next-hop address	Interface
140.24.7.0/24	----------	m0
0.0.0.0/0	default router	m1

Figure 7.35 *Longest mask matching*



Forwarding table for R2

Network address/mask	Next-hop address	Interface
140.24.7.192/26	----------	m1
140.24.7.0/24	address of R1	m0
0.0.0.0/0	default router	m2

Forwarding table for R1

Network address/mask	Next-hop address	Interface
140.24.7.0/26	----------	m0
140.24.7.64/26	----------	m1
140.24.7.128/26	----------	m2
0.0.0.0/0	default router	m3

Suppose a packet arrives at router R2 for organization 4 with destination address 140.24.7.200. The first mask at router R2 is applied, which gives the network address 140.24.7.192. The packet is routed correctly from interface m1 and reaches organization 4. If, however, the forwarding table was not stored with the longest prefix first, applying the /24 mask would result in the incorrect routing of the packet to router R1.

Hierarchical Routing

To solve the problem of gigantic forwarding tables, we can create a sense of hierarchy in the forwarding tables. In Chapter 1, we mentioned that the Internet today has a sense of hierarchy. We said that the Internet is divided into backbone and national ISPs. National ISPs are divided into regional ISPs, and regional ISPs are divided into local ISPs. If the forwarding table has a sense of hierarchy like the Internet architecture, the forwarding table can decrease in size.

Let us take the case of a local ISP. It can be assigned a single, but large, block of addresses with a certain prefix length. The local ISP can divide this block into smaller blocks of different sizes and assign these to individual users and organizations, both large and small. If the block assigned to the local ISP starts with a.b.c.d/n, the ISP can create blocks starting with e.f.g.h/m, where m may vary for each customer and is greater than n.

How does this reduce the size of the forwarding table? The rest of the Internet does not have to be aware of this division. All customers of the local ISP are defined as a.b.c.d/n to the rest of the Internet. Every packet destined for one of the addresses in this large block is routed to the local ISP. There is only one entry in every router in the world for all these customers. They all belong to the same group. Of course, inside the local ISP, the router must recognize the subblocks and route the packet to the destined customer. If one of the customers is a large organization, it also can create another level of hierarchy by subnetting and dividing its subblock into smaller subblocks (or sub-subblocks). In classless routing, the levels of hierarchy are unlimited as long as we follow the rules of classless addressing.

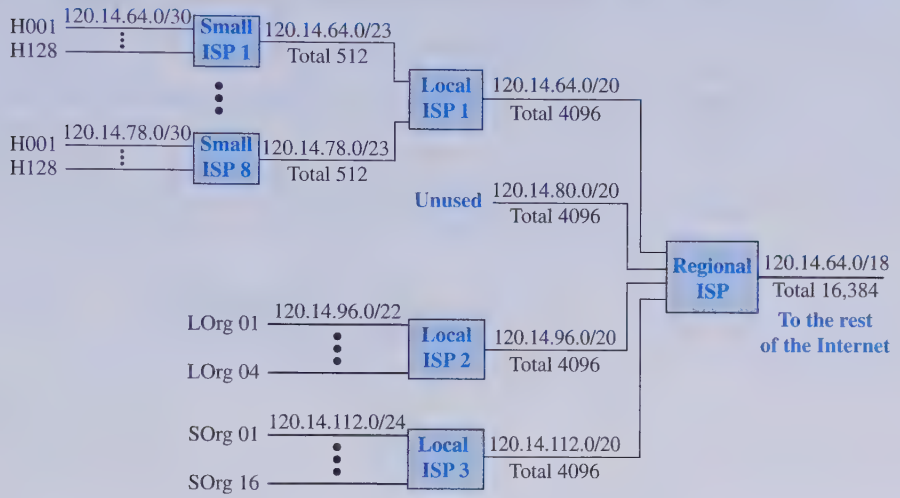
Example 7.22

As an example of hierarchical routing, let us consider Figure 7.36. A regional ISP is granted 16,384 addresses starting from 120.14.64.0. The regional ISP has decided to divide this block into four subblocks, each with 4096 addresses. Three of these subblocks are assigned to three local ISPs, the second subblock is reserved for future use. Note that the mask for each block is /20 because the original block with mask /18 is divided into four blocks.

The first local ISP has divided its assigned subblock into eight smaller blocks and assigned each to a small ISP. Each small ISP provides services to 128 households (H001 to H128), each using four addresses. Note that the mask for each small ISP is now /23 because the block is further divided into eight blocks. Each household has a mask of /30, because a household has only four addresses ($2^{32-30} = 4$). The second local ISP has divided its block into four blocks and has assigned the addresses to four large organizations (LOrg01 to LOrg04). Note that each large organization has 1024 addresses and the mask is /22.

The third local ISP has divided its block into 16 blocks and assigned each block to a small organization (SOrg01 to SOrg16). Each small organization has 256 addresses, and the mask is /24. There is a sense of hierarchy in this configuration. All routers in the Internet send a packet with destination address 120.14.64.0 to 120.14.127.255 to the regional ISP. The regional ISP sends every packet with destination address 120.14.64.0 to 120.14.79.255 to Local ISP 1. Local ISP 1 sends every packet with destination address 120.14.64.0 to 120.14.64.3 to H001.

Figure 7.36 *Hierarchical routing with ISPs*



Geographical Routing

To decrease the size of the forwarding table even further, we need to extend hierarchical routing to include geographical routing. We must divide the entire address space into a few large blocks. We assign a block to America, a block to Europe, a block to Asia, a block to Africa, and so on. The routers of ISPs outside of Europe will have only one entry for packets to Europe in their forwarding tables. The routers of ISPs outside of America will have only one entry for packets to America in their forwarding tables, and so on.

Forwarding Table Search Algorithms

In classless addressing, there is no network information in the destination address. The simplest, but not the most efficient, search method is called the longest prefix match. The forwarding table can be divided into buckets, one for each prefix. The router first tries the longest prefix. If the destination address is found in this bucket, the search is complete. If the address is not found, the next prefix is searched, and so on. It is obvious that this type of search takes a long time.

One solution is to change the data structure used for searching. Instead of a list, other data structures (such as a tree or a binary tree) can be used. One candidate is a trie (a special kind of tree). However, this discussion is beyond the scope of this book.

Forwarding Based on Label

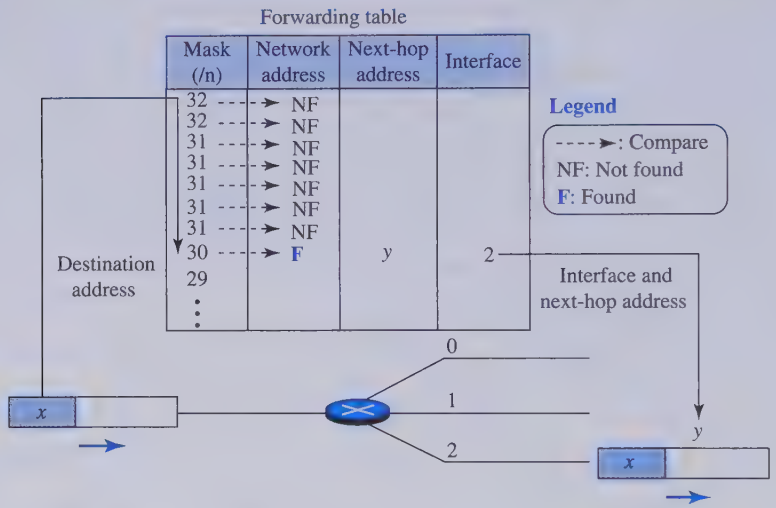
In the 1980s, an effort started to somehow change IP to behave like a connection-oriented protocol in which the routing is replaced by switching. As we discussed earlier in Section 7.2.1, in a connectionless network (datagram approach), a router forwards a packet based on the destination address in the header of the packet. On the other hand, in a connection-oriented network (virtual-circuit approach), a switch forwards a packet

based on the label attached to the packet. Routing is normally based on searching the contents of a table; switching can be done by accessing a table using an index. In other words, routing involves searching; switching involves accessing.

Example 7.23

Figure 7.37 shows a simple example of searching in a forwarding table using the longest mask algorithm. Although there are some more efficient algorithms today, the principle is the same.

Figure 7.37 *Example 7.23: Forwarding based on destination address*



When the forwarding algorithm gets the destination address of the packet, it needs to delve into the mask column. For each entry, it needs to apply the mask to find the destination network address. It then needs to check the network addresses in the table until it finds the match. The router then extracts the next-hop address and the interface number to be delivered to the data-link layer.

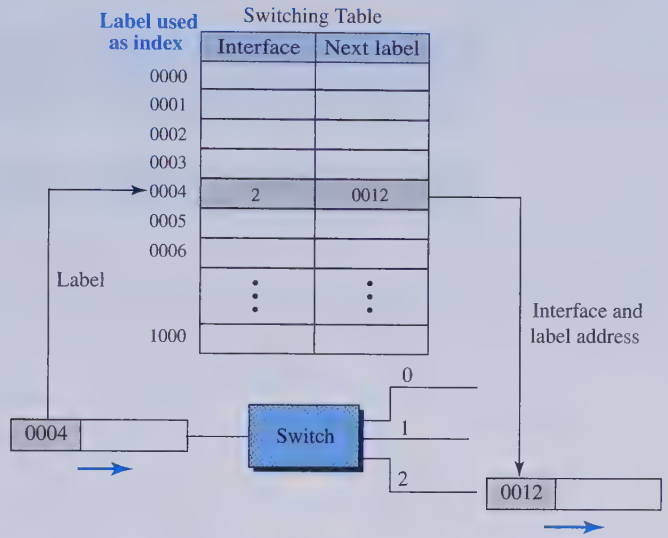
Example 7.24

Figure 7.38 shows a simple example of using a label to access a switching table. Because the labels are used as the index to the table, finding the information in the table is immediate.

Multi-Protocol Label Switching (MPLS)

During the 1980s, several vendors created routers that implement switching technology. Later the IETF approved a standard called Multi-Protocol Label Switching (MPLS). In this standard, some conventional routers in the Internet can be replaced by MPLS routers, which each can behave like a router and a switch. When behaving like a router, MPLS can forward the packet based on the destination address; when behaving like a switch, it can forward a packet based on the label.

Figure 7.38 *Example 7.24: Forwarding based on label*



A New Header

To simulate connection-oriented switching using a protocol like IP, the first thing that is needed is to add a field to the packet that carries the label. The IPv4 packet format does not allow this extension (although this field is provided in the IPv6 packet format, as we will see in Section 7.5). The solution is to encapsulate the IPv4 packet in an MPLS packet (as though MPLS is a layer between the data-link layer and the network layer). The whole IP packet is encapsulated as the payload in an MPLS packet, and an MPLS header is added. Figure 7.39 shows the encapsulation.

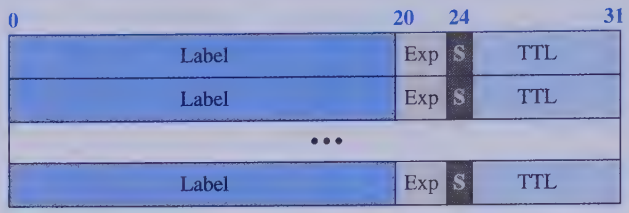
Figure 7.39 *MPLS header added to an IP packet*

The MPLS header is actually a stack of subheaders that is used for multilevel hierarchical switching, as we will discuss shortly. Figure 7.40 shows the format of an MPLS header in which each subheader is 32 bits (4 bytes) long.

The following is a brief description of each field:

- **Label.** This 20-bit field defines the label that is used to index the forwarding table in the router.
- **Exp.** This 3-bit field is reserved for experimental purposes.

Figure 7.40 *MPLS header made of a stack of labels*



- **S.** The 1-bit stack field defines the situation of the subheader in the stack. When the bit is 1, it means that the header is the last one in the stack.
- **TTL.** This 8-bit field is similar to the TTL field in the IP datagram. Each visited router decrements the value of this field. When it reaches zero, the packet is discarded to prevent looping.

Hierarchical Switching

A stack of labels in MPLS allows hierarchical switching. This is similar to conventional hierarchical routing. For example, a packet with two labels can use the top label to forward the packet through switches outside an organization; the bottom label can be used to route the packet inside the organization to reach the destination subnet.

7.5 NEXT GENERATION IP (IPV6)

The address depletion of IPv4 and other shortcomings of this protocol prompted a new version of IP protocol in the early 1990s. The new version, which is called **Internet Protocol version 6 (IPv6)** or **IP new generation (IPng)**, was a proposal to augment the address space of IPv4 and at the same time redesign the format of the IP packet and revise some auxiliary protocols such as ICMP. It is interesting to know that IPv5 was a proposal, based on the OSI model, that never materialized.

The main changes needed in the new protocol were as follows: larger address space, better header format, new options, allowance for extension, support for resource allocation, and support for more security. The implementation of theses changes made it necessary to create a new version of the ICMP protocol, ICMPv6.

This section has four subsections:

- The first subsection discusses the addressing mechanism in the new generation of the Internet. It first describes the representation and address space. It then shows the allocation in the address space. Finally, it explains autoconfiguration and renumbering, which makes it easy for a host to move from one network to another.
- The second subsection discusses IPv6 protocol. First the new packet format is described. Then it shows how the idea of an extension header can replace the options in version 4.

- The third subsection discusses ICMPv6. It describes how the new protocol replaces several auxiliary protocols in version 4. The subsection also divides the messages in this protocol into four categories and describes them.
- The fourth subsection briefly shows how a smooth transition can be made from the current version to the new one. It explains three strategies that need to be followed for this smooth transition.

7.5.1 IPv6 Addressing

The main reason for migration from IPv4 to IPv6 is the small size of the address space in IPv4. In this section, we show how the huge address space of IPv6 prevents address depletion in the future. We also discuss how the new addressing responds to some problems in the IPv4 addressing mechanism. An IPv6 address is 128 bits or 16 bytes (octets) long, 4 times the address length in IPv4.

Representation

A computer normally stores the address in binary, but it is clear that 128 bits cannot easily be handled by humans. Several notations have been proposed to represent IPv6 addresses when they are handled by humans. The following shows two of these notations: binary and colon hexadecimal.

Binary (128 bits)	**1111111011110101011 ... 1111111100000000**
Colon hexadecimal	**FEF6:BA98:7654:3210:ADEF:BBFF:2922:FF00**

Binary notation is used when the addresses are stored in a computer. The colon **hexadecimal notation** (or colon hex for short) divides the address into eight sections, each made of four hexadecimal digits separated by colons.

Abbreviation

Although an IPv6 address, even in hexadecimal format, is very long, many of the digits are zeros. In this case, we can abbreviate the address. The leading zeros of a section can be omitted. Using this form of abbreviation, 0074 can be written as 74, 000F as F, and 0000 as 0. Note that 3210 cannot be abbreviated. Further abbreviation, often called **zero compression**, can be applied to colon hex notation if there are consecutive sections consisting of zeros only. We can remove all the zeros altogether and replace them with a double semicolon.

FDEC:0:0:0:0:BBFF:0:FFFF → **FDEC::BBFF:0:FFFF**

Note that this type of abbreviation is allowed only once per address. If there is more than one run of zero sections, only one of them can be compressed.

Mixed Notation

Sometimes we see a mixed representation of an IPv6 address: colon hex and dotted-decimal notation. This is appropriate during the transition period in which an IPv4 address is embedded in an IPv6 address (as the rightmost 32 bits). We can use the colon hex

notation for the leftmost six sections and 4-byte dotted-decimal notation instead of the rightmost two sections. However, this happens when all or most of the leftmost sections of the IPv6 address are zeros. For example, the address (::130.24.24.18) is a legitimate address in IPv6, in which the zero compression shows that all 96 leftmost bits of the address are zeros.

CIDR Notation

As we will see shortly, IPv6 uses hierarchical addressing. For this reason, IPv6 allows slash or CIDR notation. For example, the following shows how we can define a prefix of 60 bits using CIDR. We will later show how an IPv6 address is divided into a prefix and a suffix.

FDEC::BBFF:0:FFFF/60

Address Space

The address space of IPv6 contains 2^{128} addresses. This address space is 2^{96} times the IPv4 address—definitely no address depletion. As shown, the size of the space is

340, 282, 366, 920, 938, 463, 374, 607, 431, 768, 211, 456

To give some idea about the number of addresses, we assume that only 1/64 (almost 2 percent) of the addresses in the space can be assigned to the people on the planet Earth and the rest are reserved for special purposes. We also assume that the number of people on the Earth is soon to be 2^{34} (more than 16 billion). Each person can have 2^{88} addresses to use. Address depletion in this version is impossible.

Three Address Types

In IPv6, a destination address can belong to one of three categories: unicast, anycast, and multicast.

Unicast Address A unicast address defines a single interface (computer or router). The packet sent to a unicast address will be routed to the intended recipient.

Anycast Address An **anycast address** defines a group of computers that all share a single address. A packet with an anycast address is delivered to only one member of the group, the most reachable one. An anycast communication is used, for example, when there are several servers that can respond to an inquiry. The request is sent to the one that is most reachable. The hardware and software generate only one copy of the request; the copy reaches only one of the servers. IPv6 does not designate a block for anycasting; the addresses are assigned from the unicast block.

Multicast Address A multicast address also defines a group of computers. However, there is a difference between anycasting and multicasting. In anycasting, only one copy of the packet is sent to one of the members of the group; in multicasting each member of the group receives a copy. As we will see shortly, IPv6 has designated a block for multicasting from which the same address is assigned to the members of

the group. It is interesting that IPv6 does not define broadcasting, even in a limited version. IPv6 considers broadcasting as a special case of multicasting.

Address Space Allocation

Like the address space of IPv4, the address space of IPv6 is divided into several blocks of varying size and each block is allocated for a special purpose. Most of the blocks are still unassigned and have been set aside for future use. Table 7.5 shows only the assigned blocks. In this table, the last column shows the fraction each block occupies in the whole address space.

Table 7.5 *Prefixes for assigned IPv6 addresses*

Block prefix	*CIDR*	*Block assignment*	*Fraction*
0000 0000	0000::/8	Special addresses	1/256
001	**2000::/3**	**Global unicast**	**1/8**
1111 110	FC00::/7	Unique local unicast	1/128
1111 1110 10	FE80::/10	Link local addresses	1/1024
1111 1111	FF00::/8	Multicast addresses	1/256

Global Unicast Addresses

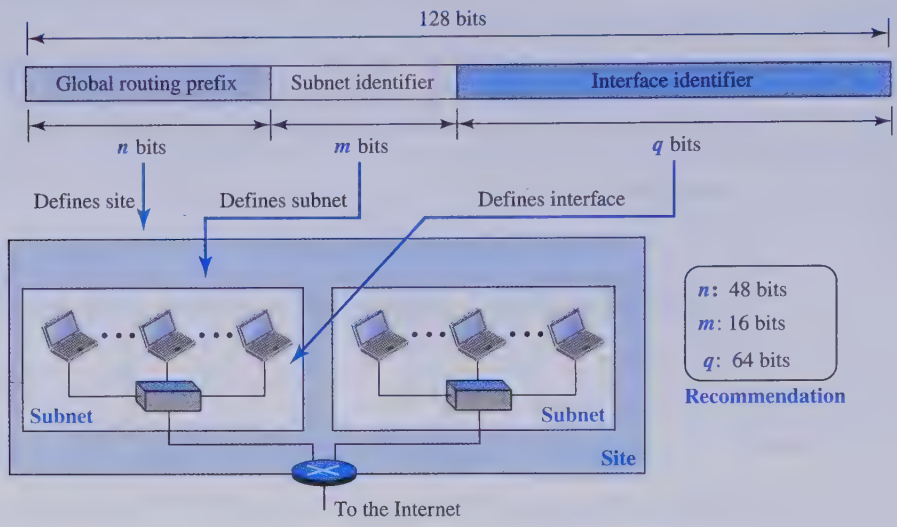
The block in the address space that is used for unicast (one-to-one) communication between two hosts in the Internet is called the *global unicast address block*. CIDR for the block is 2000::**/3**, which means that the 3 leftmost bits are the same for all addresses in this block (001). The size of this block is 2^{125} bits, which is more than enough for Internet expansion for many years to come. An address in this block is divided into three parts: *global routing prefix* (*n* bits), *subnet identifier* (*m* bits), and *interface identifier* (*q* bits), as shown in Figure 7.41. The figure also shows the recommended length for each part.

The global routing prefix is used to route the packet through the Internet to the organization site, such as the ISP that owns the block. Because the first 3 bits in this part are fixed (001), the rest of the 45 bits can be defined for up to 2^{45} sites (a private organization or an ISP). The global routers in the Internet route a packet to its destination site based on the value of *n*. The next *m* bits (16 bits based on recommendation) define a subnet in an organization. This means that an organization can have up to $2^{16} = 65{,}536$ subnets, which is more than enough.

The last *q* bits (64 bits based on recommendation) define the interface identifier. The interface identifier is similar to the hostid in IPv4 addressing although the term interface identifier is a better choice because, as we discussed earlier, the host identifier actually defines the interface, not the host. If the host is moved from one interface to another, its IP address needs to be changed.

In IPv4 addressing, there is not a specific relation between the hostid (at the IP level) and link-layer address (at the data-link layer) because the link-layer address is normally much longer than the hostid. The IPv6 addressing allows this relationship. A link-layer address whose length is less than 64 bits can be embedded as the whole

Figure 7.41 *Global unicast address*

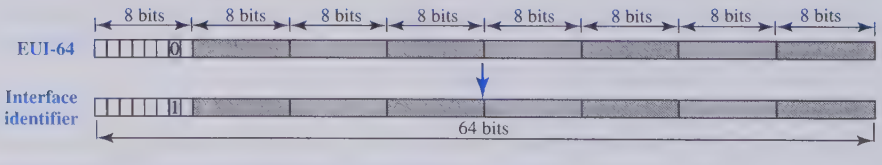


or part of the interface identifier, eliminating the mapping process. Two common link-layer addressing schemes can be considered for this purpose: the 64-bit extended unique identifier (EUI-64) defined by IEEE and the 48-bit link-layer address defined by Ethernet.

Mapping EUI-64

To map a 64-bit physical address, the global/local bit of this format needs to be changed from 0 to 1 (local to global) to define an interface address, as shown in Figure 7.42.

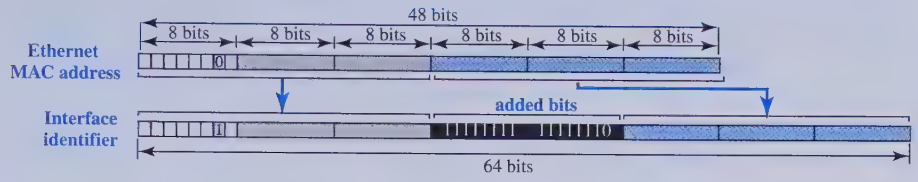
Figure 7.42 *Mapping for EUI-64*



Mapping Ethernet MAC Address

Mapping a 48-bit Ethernet address into a 64-bit interface identifier is more involved. We need to change the local/global bit to 1 and insert an additional 16 bits. The additional 16 bits are defined as 15 ones followed by one zero, or $FFFE_{16}$. Figure 7.43 shows the mapping.

Figure 7.43 *Mapping for Ethernet MAC*



Example 7.25

An organization is assigned the block 2000:1456:2474/48. What is the CIDR notation for the blocks in the first and second subnets in this organization?

Solution

Theoretically, the first and second subnets should use the block with subnet identifier 0001_{16} and 0002_{16}. This means that the blocks are 2000:1456:2474:0000/64 and 2000:1456:2474:0001/64.

Example 7.26

Find the interface identifier if the physical address in the EUI is $(\mathbf{F5\text{-}A9\text{-}23\text{-}EF\text{-}07\text{-}14\text{-}7A\text{-}D2})_{16}$ using the format we defined for Ethernet addresses.

Solution

We only need to change the seventh bit of the first octet from 0 to 1 and change the format to colon hex notation. The result is **F7A9:23EF:0714:7AD2**.

Example 7.27

Find the interface identifier if the Ethernet physical address is $(\mathbf{F5\text{-}A9\text{-}23\text{-}14\text{-}7A\text{-}D2})_{16}$ using the format we defined for Ethernet addresses.

Solution

We only need to change the seventh bit of the first octet from 0 to 1, insert two octet $FFFE_{16}$, and change the format to colon hex notation. The result is `F7A9:23FF:FE14:7AD2` in colon hex.

Example 7.28

An organization is assigned the block 2000:1456:2474/48. What is the IPv6 address of an interface in the third subnet if the IEEE physical address of the computer is $(\mathbf{F5\text{-}A9\text{-}23\text{-}14\text{-}7A\text{-}D2})_{16}$?

Solution

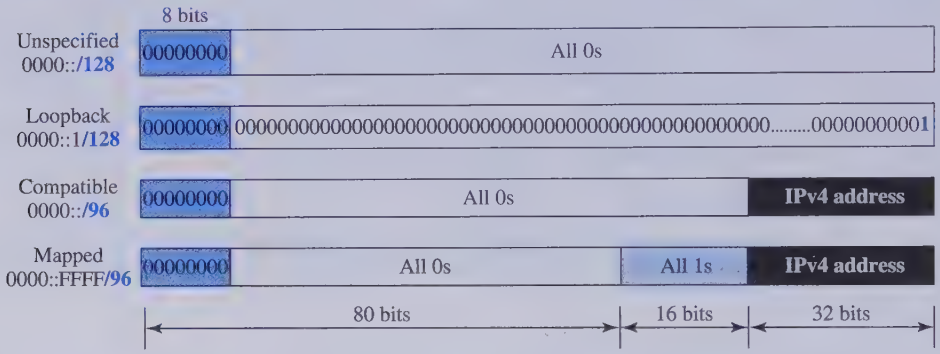
The interface identifier for this interface is `F7A9:23FF:FE14:7AD2` (see solution to Example 7.27). If we add this identifier to the global prefix and the subnet identifier, we get:

```
2000:1456:2474:0003:F7A9:23FF:FE14:7AD2/128
```

Special Addresses

After discussing the global unicast block, let us discuss the characteristics and purposes of assigned and reserved blocks in the first row of Table 7.5. Addresses that use the prefix (**0000::/8**) are reserved, but part of this block is used to define some special addresses. Figure 7.44 shows the assigned addresses in this block.

Figure 7.44 *Special addresses*



The unspecified address is a subblock containing only one single address, which is used during bootstrap when a host does not know its own address and wants to send an inquiry to find it.

The loopback address also consists of one single address. We discussed loopback addresses for IPv4 before. In IPv4 the block is made up of a range of addresses; in IPv6, the block has only a single address in it.

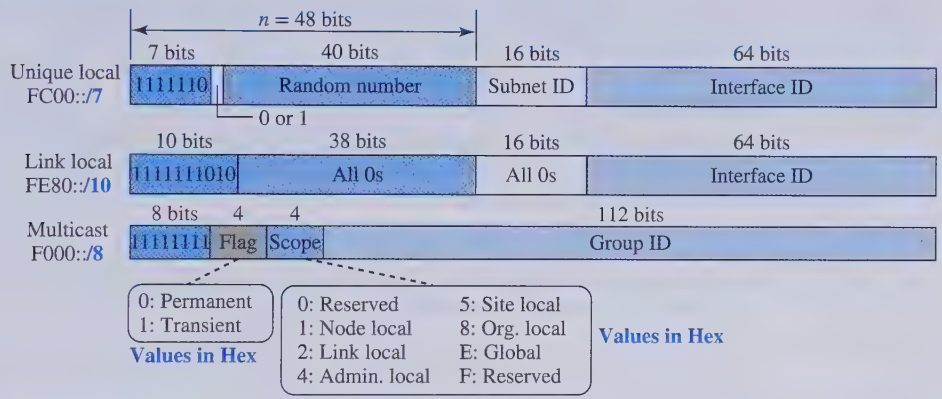
As we will see later in Section 7.6, during the transition from IPv4 to IPv6, hosts can use their IPv4 addresses embedded in IPv6 addresses. Two formats have been designed for this purpose: compatible and mapped. A **compatible address** is an address of 96 zero bits followed by 32 bits of the IPv4 address. It is used when a computer using IPv6 wants to send a message to another computer using IPv6. A **mapped address** is used when a computer already migrated to version 6 wants to send an address to a computer still using version 4. A very interesting point about mapped and compatible addresses is that they are designed such that, when calculating the checksum, one can use either the embedded address or the total address because extra 0s or 1s in multiples of 16 do not have any effect in the checksum calculation. This is important for UDP and TCP, which use a pseudoheader to calculate the checksum because the checksum calculation is not affected if the address of the packet is changed from IPv6 to IPv4 by a router.

Other Assigned Blocks

IPv6 uses two large blocks for private addressing and one large block for multicasting, as shown in Figure 7.45. A subblock in a **unique local unicast block** can be privately created and used by a site. The packet carrying this type of address as the destination

address is not expected to be routed. This type of address has the identifier 1111 110; the next bit can be 0 or 1 to define how the address is selected (locally or by an authority). The next 40 bits are selected by the site using a randomly generated number of length 40 bits. This means that the total of 48 bits defines a subblock that looks like a global unicast address. The 40-bit random number makes the probability of duplication of the address extremely small. Note the similarity between the format of these addresses and the global unicast. The second block, designed for private addresses, is the **link local block**. A subblock in this block can be used as a private address in a network.

Figure 7.45 *Unique local unicast block*



This type of address has the block identifier 1111111010. The next 54 bits are set to zero. The last 64 bits can be changed to define the interface for each computer. Note the similarity between the format of these addresses and the global unicast address.

We discussed multicast addresses of IPv4 earlier in Section 7.5.1. Multicast addresses are used to define a group of hosts instead of just one. In IPv6, a large block of addresses are assigned for multicasting. All these addresses use the prefix 11111111. The second field is a flag that defines the group address as either permanent or transient. A permanent group address is defined by the Internet authorities and can be accessed at all times. A transient group address, on the other hand, is used only temporarily. Systems engaged in a teleconference, for example, can use a transient group address. The third field defines the scope of the group address. Many different scopes have been defined, as shown in Figure 7.45.

Autoconfiguration

One of the interesting features of IPv6 addressing is the **autoconfiguration** of hosts. As we discussed in IPv4, the host and routers are originally configured manually by the network manager. However, the Dynamic Host Configuration Protocol (DHCP) can be used to allocate an IPv4 address to a host that joins the network. In IPv6, the DHCP protocol can still be used to allocate an IPv6 address to a host, but a host can also configure itself.

When a host in IPv6 joins a network, it can configure itself using the following process:

1. The host first creates a **link local address** for itself. This is by taking the 10-bit link local prefix (1111 1110 10), adding 54 zeros, and adding the 64-bit interface identifier, which any host knows how to generate from its interface card. The result is a 128-bit link local address.
2. The host then tests to see if this link local address is unique and not used by other hosts.
3. If the uniqueness of the link local address is passed, the host stores this address as its link local address (for private communication), but it still needs a global unicast address. The host then sends a message to a local router. If there is a router running on the network, the host receives a message that includes the global unicast prefix and the subnet prefix that the host needs to add to its interface identifier to generate its global unicast address.

Example 7.29

Assume a host with Ethernet address $(\mathbf{F5\text{-}A9\text{-}23\text{-}11\text{-}9B\text{-}E2})_{16}$ has joined the network. What would be its global unicast address if the global unicast prefix of the organization is 3A21:1216:2165 and the subnet identifier is A245:1232?

Solution

The host first creates its interface identifier as **F7A9:23FF:FE11:9BE2** using the Ethernet address read from its card. The host then creates its link local address as

```
FE80::F7A9:23FF:FE11:9BE2
```

Assuming that this address is unique, the host sends a router message and receives the router message that announces the combination of global unicast prefix and the subnet identifier as 3A21:1216:2165:A245:1232. The host then appends its interface identifier to this prefix to find and store its global unicast address as

```
3A21:1216:2165:A245:1232:F7A9:23FF:FE11:9BE2
```

Renumbering

To allow sites to change the service provider, **renumbering** of the address prefix (n) was built into IPv6 addressing. As we discussed before, each site is given a prefix by the service provider to which it is connected. If the site changes the provider, the address prefix needs to be changed. A router to which the site is connected can advertise a new prefix and let the site use the old prefix for a short time before disabling it. In other words, during the transition period, a site has two prefixes. The main problem in using the renumbering mechanism is the support of the DNS, which needs to propagate the new addressing associated with a domain name. A new protocol for DNS, called Next Generation DNS, is under study to provide support for this mechanism.

7.5.2 The IPv6 Protocol

The change of the IPv6 address size requires the change in the IPv4 packet format. The designer of IPv6 decided to implement other shortcomings now that a change is

inevitable. The following shows other changes implemented in the protocol in addition to changing address size and format.

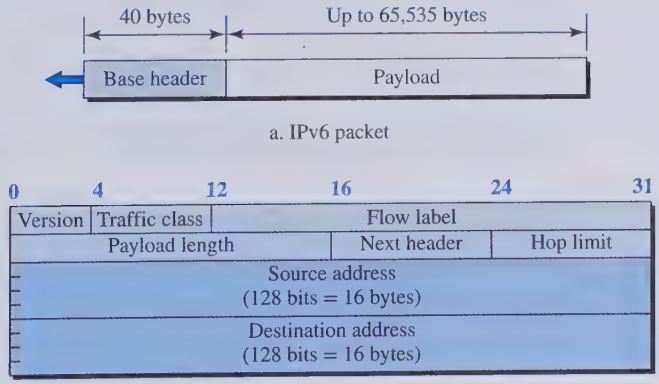
- **Better header format.** IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.
- **New options.** IPv6 has new options to allow for additional functionalities.
- **Allowance for extension.** IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.
- **Support for resource allocation.** In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.
- **Support for more security.** The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

Packet Format

The IPv6 packet is shown in Figure 7.46. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas the payload can be up to 65,535 bytes of information. The description of fields follows.

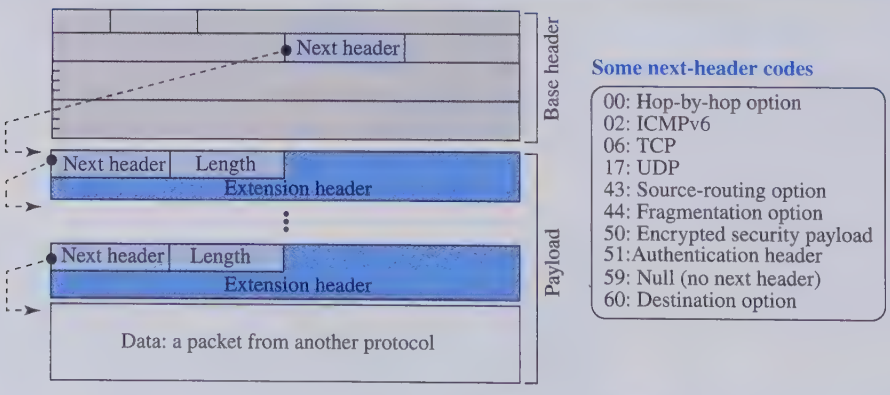
- **Version.** The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.
- **Traffic class.** The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the *type-of-service* field in IPv4.

Figure 7.46 *IPv6 datagram*



- **Flow label.** The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data. We will discuss this field later in the section.
- **Payload length.** The 2-byte payload length field defines the length of the IP datagram excluding the header. Note that IPv4 defines two fields related to the length: header length and total length. In IPv6, the length of the base header is fixed (40 bytes); only the length of the payload needs to be defined.
- **Next header.** The **next header** is an 8-bit field defining the type of first extension header (if present) or the type of the data that follows the base header in the datagram. This field is similar to the protocol field in IPv4, but we talk more about it when we discuss the payload.
- **Hop limit.** The 8-bit **hop limit** field serves the same purpose as the TTL field in IPv4.
- **Source and destination address.** The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.
- **Payload.** Compared to IPv4, the payload field in IPv6 has a different format and meaning, as shown in Figure 7.47. The payload in IPv6 means a combination of zero or more extension headers (options) followed by the data from other protocols (UDP, TCP, and so on). In IPv6, options, which are part of the header in IPv4, are designed as extension headers. The payload can have as many extension headers as required by the situation. Each extension header has two mandatory fields, next header and the length, followed by information related to the particular option. Note that each next header field value (code) defines the type of the next header

Figure 7.47 *Payload in an IPv6 datagram*



(hop-by-hop option, source-routing option, . . .); the last next header field defines the protocol (UDP, TCP, . . .) that is carried by the datagram.

Concept of Flow and Priority in IPv6

The IP was originally designed as a connectionless protocol. However, the tendency is to use the IP as a connection-oriented protocol. The MPLS technology described earlier allows us to encapsulate an IPv4 packet in an MPLS header using a label field. In version 6, the flow label has been directly added to the format of the IPv6 datagram to allow us to use IPv6 as a connection-oriented protocol.

To a router, a flow is a sequence of packets that share the same characteristics, such as traveling the same path, using the same resources, and having the same kind of security. A router that supports the handling of flow labels has a flow label table. The table has an entry for each active flow label; each entry defines the services required by the corresponding flow label. When the router receives a packet, it consults its flow label table to find the corresponding entry for the flow label value defined in the packet. It then provides the packet with the services mentioned in the entry. However, note that the flow label itself does not provide the information for the entries of the flow label table; the information is provided by other means, such as the hop-by-hop options or other protocols.

In its simplest form, a flow label can be used to speed up the processing of a packet by a router. When a router receives a packet, instead of consulting the forwarding table and going through a routing algorithm to define the address of the next hop, it can easily look in a flow label table for the next hop.

In its more sophisticated form, a flow label can be used to support the transmission of real-time audio and video. Real-time audio or video, particularly in digital form, requires resources such as high bandwidth, large buffers, and long processing time. A process can make a reservation for these resources beforehand to guarantee that real-time data will not be delayed due to a lack of resources.

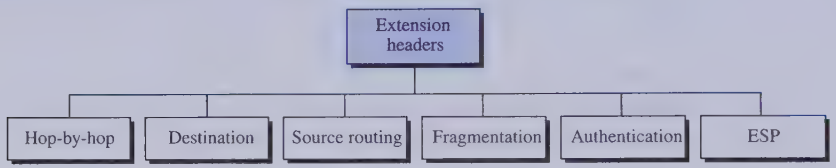
Fragmentation and Reassembly

There is still fragmentation and reassembly of datagrams in the IPv6 protocol, but there is a major difference in this respect. IPv6 datagrams can be fragmented only by the source, not by the routers; the reassembly takes place at the destination. The fragmentation of packets at routers is not allowed to speed up the processing of packets in the router. The fragmentation of a packet in a router needs a lot of processing. The packet needs to be fragmented; all fields related to the fragmentation need to be recalculated. In IPv6, the source can check the size of the packet and make the decision to fragment the packet or not. When a router receives the packet, it can check the size of the packet and drop it if the size is larger than allowed by the MTU of the network ahead. The router then sends a packet-too-big ICMPv6 error message (discussed later in Section 7.5.3) to inform the source.

Extension Header

An IPv6 packet is made up of a base header and some extension headers. The length of the base header is fixed at 40 bytes. However, to give more functionality to the IP datagram, the base header can be followed by up to six **extension headers**. Many of these headers are options in IPv4. Six types of extension headers have been defined. These are hop-by-hop option, source routing, fragmentation, authentication, encrypted security payload, and destination option (see Figure 7.48).

Figure 7.48 *Extension header types*



Hop-by-Hop Option

The **hop-by-hop option** is used when the source needs to pass information to all routers visited by the datagram. For example, perhaps routers must be informed about certain management, debugging, or control functions. Or, if the length of the datagram is more than the usual 65,535 bytes, routers must have this information. So far, only three hop-by-hop options have been defined: **Pad1**, **PadN**, and **jumbo payload**.

- **Pad1.** This option is 1 byte long and is designed for alignment purposes. Some options need to start at a specific bit of the 32-bit word. If an option falls short of this requirement by exactly 1 byte, the rest will be filled by 0s.
- **PadN.** PadN is similar in concept to Pad1. The difference is that PadN is used when 2 or more bytes are needed for alignment.
- **Jumbo payload.** Recall that the length of the payload in the IP datagram can be a maximum of 65,535 bytes. However, if for any reason a longer payload is required, we can use the jumbo payload option to define this longer length.

Destinati]on Option

The **destination option** is used when the source needs to pass information to the destination only. Intermediate routers are not permitted access to this information. The format of the destination option is the same as the hop-by-hop option. So far, only the Pad1 and PadN options have been defined.

Source Routing

The source routing extension header combines the concepts of the strict source route and the loose source route options of IPv4.

Fragmentation

The concept of **fragmentation** in IPv6 is the same as that in IPv4. However, the place where fragmentation occurs differs. In IPv4, the source or a router is required to fragment if the size of the datagram is larger than the MTU of the network over which the datagram travels. In IPv6, only the original source can fragment. A source must use a **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.

If the source does not use a Path MTU Discovery technique, it fragments the datagram to a size of 1280 bytes or smaller. This is the minimum size of MTU required for each network connected to the Internet.

Authentication

The **authentication** extension header has a dual purpose: It validates the message sender and ensures the integrity of data. The former is needed so the receiver can be sure that a message is from the genuine sender and not from an imposter. The latter is needed to check that the data are not altered in transition by some hacker. We discuss more about authentication in Chapter 13.

Encrypted Security Payload

The **encrypted security payload (ESP)** is an extension that provides confidentiality and guards against eavesdropping. Again we discuss more about providing confidentiality for IP packets in Chapter 13.

Comparison of Options between IPv4 and IPv6

The following shows a quick comparison between the options used in IPv4 and the options used in IPv6 (as extension headers).

- The no-operation and end-of-option options in IPv4 are replaced by Pad1 and PadN options in IPv6.
- The record route option is not implemented in IPv6 because it was not used.
- The timestamp option is not implemented because it was not used.
- The source route option is called the source route extension header in IPv6.
- The fragmentation fields in the base header section of IPv4 have moved to the fragmentation extension header in IPv6.
- The authentication extension header is new in IPv6.
- The encrypted security payload extension header is new in IPv6.

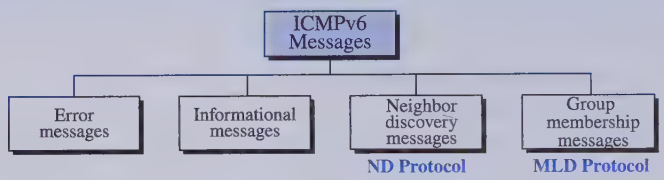
7.5.3 The ICMPv6 Protocol

Another protocol that has been modified in version 6 of the TCP/IP protocol suite is ICMP. This new version, Internet Control Message Protocol version 6 (ICMPv6), follows the same strategy and purposes of version 4. ICMPv6, however, is more complicated than ICMPv4: Some protocols that were independent in version 4 are now part of ICMPv6, and some new messages have been added to make it more useful. Figure 7.49 compares the network layer of version 4 to that of version 6.

Figure 7.49 *Comparison of network layer in version 4 and version 6*

We briefly describe the ICMPv6 in this section. We can divide the messages in ICMPv6 into four groups: error-reporting messages, informational messages, neighbor-discovery messages, and group-membership messages as shown in Figure 7.50.

Figure 7.50 *Categories of ICMPv6 messages*



Error-Reporting Messages

As we saw in our discussion of version 4, one of the main responsibilities of ICMPv6 is to report errors. Four types of errors are handled: destination unreachable, packet too big, time exceeded, and parameter problems. Note that the source-quenched message, which is used to control congestion in version 4, is eliminated in this version because the priority and flow label fields in IPv6 are supposed to take care of congestion. The redirection message has moved from the error-reporting category to the neighbor-discovery category, so we discuss it as part of the neighbor-discovery messages.

ICMPv6 forms an error packet, which is then encapsulated in an IPv6 datagram. This is delivered to the original source of the failed datagram.

Destination-Unreachable Message

The concept of the destination unreachable message is the same as described for ICMPv4. When a router cannot forward a datagram or a host cannot deliver the content of the datagram to the upper-layer protocol, the router or the host discards the datagram and sends a *destination-unreachable* error message to the source host.

Packet-Too-Big Message

This is a new type of message added to version 6. Because IPv6 does not fragment at the router, if a router receives a datagram that is larger than the maximum transmission unit (MTU) size of the network through which the datagram should pass, two things happen.

First, the router discards the datagram. Second, an ICMP error packet—a **packet-too-big message**—is sent to the source.

Time-Exceeded Message

A *time-exceeded* error message is generated in two cases: when the *time-to-live* value becomes zero and when not all fragments of a datagram have arrived in the time limit. The format of the *time-exceeded* message in version 6 is similar to the one in version 4. The only difference is that the type value has changed to 3.

Parameter-Problem Message

Any ambiguity in the header of the datagram can create serious problems as the datagram travels through the Internet. If a router or the destination host discovers any ambiguous or missing value in any field, it discards the datagram and sends a parameter-problem message to the source. The message in ICMPv6 is similar to its version 4 counterpart.

Informational Messages

Two of the ICMPv6 messages can be categorized as informational messages: echo-request and echo-reply messages. The echo-request and echo-response messages are designed to check if two devices in the Internet can communicate with each other. A host or router can send an echo-request message to another host; the receiving computer or router can reply using the echo-response message.

Echo-Request Message

The idea and format of the echo-request message is the same as the one in version 4.

Echo-Reply Message

The idea and format of the echo-reply message is the same as the one in version 4.

Neighbor-Discovery Messages

Several messages in the ICMPv6 have been redefined in ICMPv6 to handle the issue of neighbor discovery. Some new messages have also been added to provide extension. The most important issue is the definition of two new protocols that clearly define the functionality of these group messages: the **Neighbor-Discovery (ND) protocol** and the **Inverse-Neighbor-Discovery (IND) protocol**. These two protocols are used by nodes (hosts or routers) on the same link (network) for three main purposes:

1. Hosts use the ND protocol to find routers in the neighborhood that will forward packets for them.
2. Nodes use the ND protocol to find the link-layer addresses of neighbors (nodes attached to the same network).
3. Nodes use the IND protocol to find the IPv6 addresses of the neighbor.

Router-Solicitation Message

The idea behind the *router-solicitation* message is the same as in version 4. A host uses the router-solicitation message to find a router in the network that can forward an IPv6 datagram for the host. The only option that is so far defined for this message is the

inclusion of physical (data-link layer) address of the host to make the response easier for the router.

Router-Advertisement Message

The *router-advertisement* message is sent by a router in response to a router-solicitation message.

Neighbor-Solicitation Message

As previously mentioned, the network layer in version 4 contains an independent protocol called Address Resolution Protocol (ARP). In version 6, this protocol is eliminated, and its duties are included in ICMPv6. The neighbor solicitation message has the same duty as the ARP request message. This message is sent when a host or router has a message to send to a neighbor. The sender knows the IP address of the receiver, but needs the data-link address of the receiver. The data-link address is needed for the IP datagram to be encapsulated in a frame. The only option announces the sender data-link address for the convenience of the receiver. The receiver can use the sender data-link address to use a unicast response.

Neighbor-Advertisement Message

The **neighbor-advertisement message** is sent in response to the neighbor-solicitation message.

Redirection Message

The purpose of the redirection message is the same as described for version 4. However, the format of the packet now accommodates the size of the IP address in version 6. Also, an option is added to let the host know the physical address of the target router.

Inverse-Neighbor-Solicitation Message

The **inverse-neighbor-solicitation message** is sent by a node that knows the link-layer address of a neighbor, but not the neighbor's IP address. The message is encapsulated in an IPv6 datagram using an all-node multicast address. The sender must send the following two pieces of information in the option field: its link-layer address and the link-layer address of the target node. The sender can also include its IP address and the MTU value for the link.

Inverse-Neighbor-Advertisement Message

The **inverse-neighbor-advertisement message** is sent in response to the inverse-neighbor-discovery message. The sender of this message must include the link-layer address of the sender and the link-layer address of the target node in the option section.

Group Membership Messages

The management of multicast delivery handling in IPv4 is given to the IGMPv3 protocol. In IPv6, this responsibility is given to the **Multicast Listener Delivery protocol**. MLDv1 is the counterpart to IGMPv2; MLDv2 is the counterpart to IGMPv3. The material discussed in this section is taken from RFC 3810. The idea is the same as we discussed in IGMPv3, but the sizes and formats of the messages have been changed to fit the larger multicast address size in IPv6. Like IGMPv3, MLDv2 has two types of messages:

membership-query message and *membership-report message*. The first type can be divided into three subtypes: *general*, *group-specific*, and *group-and-source specific*.

Membership-Query Message

A membership-query message is sent by a router to find active group members in the network. The fields are almost the same as the ones in IGMPv3 except that the size of the multicast address and the source address has been changed from 32 to 128 bits. Another noticeable change in the field size is in the *maximum response code* field, in which the size has been changed from 8 to 16 bits. Also note that the format of the first 8 bytes matches the format for other ICMPv6 packets because MLDv2 is considered to be part of ICMPv6.

Membership-Report Message

The format of the membership report in MLDv2 is exactly the same as the one in IGMPv3 except that the sizes of the fields are changed because of the address size. In particular, the record type is the same as the one defined for IGMPv3 (types 1 to 6).

7.6 TRANSITION FROM IPV4 TO IPV6

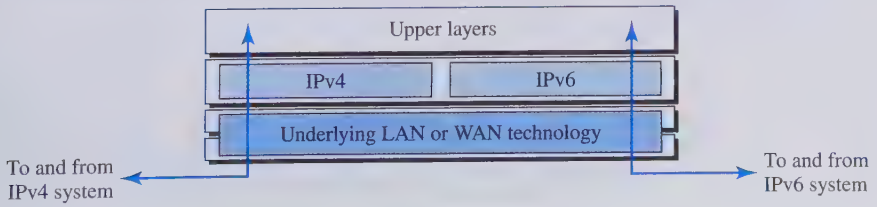
Although we have a new version of the IP, how can we make the transition to stop using IPv4 and start using IPv6? The first solution that comes to mind is to define a transition day on which every host or router should stop using the old version and start using the new version. However, this is not practical; because of the huge number of systems on the Internet, the transition from IPv4 to IPv6 cannot happen suddenly. It will take a considerable amount of time before every system in the Internet can move from IPv4 to IPv6. The transition must be smooth to prevent any problems between the IPv4 and IPv6 systems.

Three strategies have been devised for the transition: dual stack, tunneling, and header translation. One or all of these three strategies can be implemented during the transition period.

Dual Stack

It is recommended that all hosts, before migrating completely to version 6, have a **dual stack** of protocols during the transition. In other words, a station must run IPv4 and IPv6 simultaneously until all the Internet uses IPv6. See Figure 7.51 for the layout of a dual-stack configuration.

Figure 7.51 *Dual stack strategy*

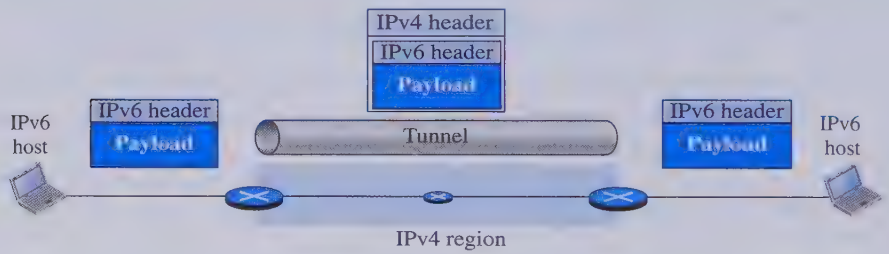


To determine which version to use when sending a packet to a destination, the source host queries the DNS. If the DNS returns an IPv4 address, the source host sends an IPv4 packet. If the DNS returns an IPv6 address, the source host sends an IPv6 packet.

Tunneling

Tunneling is a strategy used when two computers using IPv6 want to communicate with each other and the packet must pass through a region that uses IPv4. To pass through this region, the packet must have an IPv4 address. So the IPv6 packet is encapsulated in an IPv4 packet when it enters the region, and it leaves its capsule when it exits the region. It seems as if the IPv6 packet goes through a tunnel at one end and emerges at the other end. To make it clear that the IPv4 packet is carrying an IPv6 packet as data, the protocol value is set to 41. Tunneling is shown in Figure 7.52.

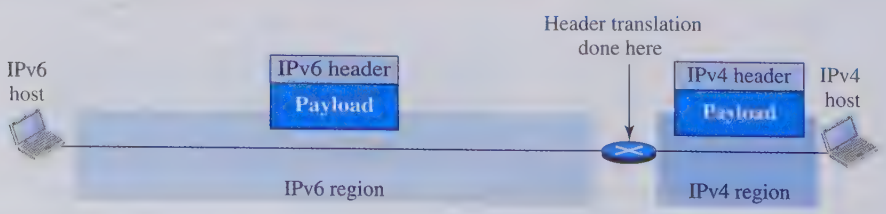
Figure 7.52 *Tunneling strategy*



Header Translation

Header translation is necessary when the majority of the Internet has moved to IPv6 but some systems still use IPv4. The sender wants to use IPv6, but the receiver does not understand IPv6. Tunneling does not work in this situation because the packet must be in the IPv4 format to be understood by the receiver. In this case, the header format must be totally changed through header translation. The header of the IPv6 packet is converted to an IPv4 header (see Figure 7.53)

Figure 7.53 *Header translation strategy*



7.7 END-OF-CHAPTER MATERIALS

7.7.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and Requests for Comments (RFCs). The items in brackets refer to the reference list at the end of the text.

Books

Several books give thorough coverage of materials discussed in this chapter. We recommend [Com 06], [Tan 03], [Koz 05], [Ste 95], [GW 04], [Per 00], [Kes 02], [Moy 98], [W & Z 01], and [Los 04].

Requests for Comments

IP is discussed in RFCs 791, 815, 894, 1122, 2474, and 2475. ICMP is discussed in RFCs 792, 950, 956, 957, 1016, 1122, 1256, 1305, and 1987.

7.7.2 Key Terms

address aggregation
anycast address
authentication
autoconfiguration
base header
care-of address
classful addressing
classless addressing
classless interdomain routing (CIDR)
collocated care-of address
colon hexadecimal notation
compatible address
destination option
double crossing
dual stack
encrypted security payload (ESP)
encryption
end-of-option option
error-reporting messages
extension header
flow label
foreign agent
foreign network
fragmentation
header translation
home address
home agent
home network
hop limit
hop-by-hop option
Internet Control Message Protocol version 4 (ICMPv4)
Internet Protocol version 6 (IPv6)
inverse-neighbor-advertisement message
Inverse-Neighbor-Discovery (IND) protocol
inverse-neighbor-solicitation message
IP new generation (IPng)
jumbo payload
link local address
link local block
loose source route option
mapped address
mobile host
Multicast Listener Delivery protocol
neighbor-advertisement message
Neighbor-Discovery (ND) protocol
network address
next header
no-operation option
packetizing
packet-too-big message
Pad1
Pad2
Path MTU Discovery technique
query messages
record route option
renumbering
stationary host

strict source route option
timestamp option
triangle routing
tunneling
unique local unicast block
zero compression

7.7.3 Summary

IPv4 is an unreliable connectionless protocol responsible for source-to-destination delivery. Packets in the IP layer are called datagrams. An IPv4 datagram is made up of a header, of size 20 to 60 bytes, and a payload. The total size of an IPv4 datagram can be up to 65,535 bytes. An IPv4 datagram can be fragmented, one or more times, during its path from the source to destination; reassembly of the fragments, however, should be done at the destination. The checksum for a datagram is calculated only for the header.

The Internet Control Message Protocol (ICMP) supports the unreliable and connectionless Internet Protocol (IP). ICMP messages are encapsulated in IP datagrams. There are two categories of ICMP messages: error-reporting and query messages. The error-reporting messages report problems that a router or a host (destination) may encounter when it processes an IP packet. The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host. Mobile IP, designed for mobile communication, is an enhanced version of IP. A mobile host has a home address on its home network and a care-of address on its foreign network. When the mobile host is on a foreign network, a home agent relays messages (for the mobile host) to a foreign agent. A foreign agent sends relayed messages to a mobile host.

IPv6 has a 128-bit address space. Addresses are presented using hexadecimal colon notation with abbreviation methods available. In IPv6, a destination address can belong to one of the three categories: unicast, anycast, and multicast. The address space of IPv6 is divided into several blocks of varying size and each block is allocated for a special purpose. The most important block is the one with prefix 001, which is used for global unicast addressing. Two interesting features of IPv6 addressing are autoconfiguration and numbering.

An IPv6 datagram is composed of a base header and a payload. A payload consists of optional extension headers and data from an upper layer. Extension headers add functionality to the IPv6 datagram.

ICMPv6, like ICMPv4, is message oriented; it uses messages to report errors, to get information, probe a neighbor, or manage multicast communication. However, a few other protocols are added to ICMPv6 to define the functionality and interpretation of the messages.

Three strategies used to handle the transition from version 4 to version 6 are dual stack, tunneling, and header translation.

7.8 PRACTICE SET

7.8.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

7.8.2 Questions

Q7-1. Why does the network-layer protocol need to provide packetizing service to the transport layer? Why can't the transport layer send out the segments without encapsulating them in datagrams?

Q7-2. Why is routing the responsibility of the network layer? In other words, why can't the routing be done at the transport layer or the data-link layer?

Q7-3. Distinguish between the process of routing a packet from the source to the destination and the process of forwarding a packet at each router.

Q7-4. What is the piece of information in a packet upon which the forwarding decision is made in each of the following approaches to switching?

a. Datagram approach **b.** Virtual-circuit approach

Q7-5. If a label in a connection-oriented service is 8 bits, how many virtual circuits can be established at the same time?

Q7-6. List the three phases in the virtual-circuit approach to switching.

Q7-7. Do we have any of the following services at the network layer of TCP/IP? If not, why?

a. Flow control **b.** Error control **c.** Congestion control

Q7-8. List four types of delays in a packet-switch network.

Q7-9. In Figure 7.2, assume that the link between R1 and R2 is upgraded to 170 kbps and the link between the source host and R1 is now downgraded to 140 kbps. What is the throughput between the source and destination after these changes? Which link is the bottleneck now?

Q7-10. In classless addressing, we know the first and last addresses in the block. Can we find the prefix length? If the answer is yes, show the process.

Q7-11. In classless addressing, we know the first address and the number of addresses in the block. Can we find the prefix length? If the answer is yes, show the process.

Q7-12. In classless addressing, can two different blocks have the same prefix length? Explain.

Q7-13. Can the value of the header length field in an IPv4 packet be less than 5? When is it exactly 5?

Q7-14. A host is sending 100 datagrams to another host. If the identification number of the first datagram is 1024, what is the identification number of the last?

Q7-15. An IP fragment has arrived with an offset value of 100. How many bytes of data were originally sent by the source before the data in this fragment?

Q7-16. List the three auxiliary protocols at the network layer of the TCP/IP suite that are designed to help the IPv4 protocol.

Q7-17. In an IPv4 datagram, the value of the header length (HLEN) field is $(6)_{16}$. How many bytes of options have been added to the packet?

Q7-18. Can the value of the TTL in a datagram be any of the following? Explain your answer.

a. 23 **b.** 0 **c.** 1 **d.** 301

Q7-19. Compare and contrast the protocol field at the network layer with the port numbers at the transport layer. What is their common purpose? Why do we

need two port-number fields but only one protocol field? Why is the size of the protocol field only half the size of each port number?

Q7-20. Which fields in the datagram are responsible for gluing together all fragments belonging to an original datagram?

Q7-21. Can the value of the offset field in a datagram be any of the following? Explain your answer.

a. 8 **b.** 31 **c.** 73 **d.** 56

Q7-22. Assume a destination computer receives several packets from a source. How can it be sure that the fragments belonging to a datagram are not mixed with the fragments belonging to another datagram?

Q7-23. Explain why the Internet does not create a report message to report the error in an IP datagram that carries an ICMP message.

Q7-24. What are the source and destination IP addresses in a datagram that carries the ICMP message reported by a router?

Q7-25. Explain why the registration request and reply are not directly encapsulated in an IP datagram. Why is there a need for the UDP user datagram?

Q7-26. Is registration required if the mobile host acts as a foreign agent? Explain your answer.

Q7-27. Discuss how the ICMP router solicitation message can also be used for agent solicitation. Why are there no extra fields?

Q7-28. Which protocol is the carrier of the agent advertisement and solicitation messages?

Q7-29. Explain the advantages of IPv6 when compared to IPv4.

Q7-30. Explain the use of the flow field in IPv6. What is the potential application of this field?

Q7-31. Distinguish between compatible and mapped addresses, and explain their applications.

Q7-32. List three protocols in the IPv4 network layer that are combined into a single protocol in IPv6.

Q7-33. What is the purpose of including the IP header and the first 8 bytes of datagram data in the error-reporting ICMP messages?

Q7-34. If you are assigned an IPv6 address by your ISP for your personal computer at home, what should be the first (leftmost) 3 bits of this address?

Q7-35. Find the size of the global unicast block from Table 7.5.

Q7-36. Find the size of the special address block from Table 7.5.

Q7-37. Find the size of the unique local unicast block from Table 7.5.

Q7-38. Find the size of the multicast block from Table 7.5.

Q7-39. Explain the benefit of autoconfiguration.

Q7-40. Explain the benefit of renumbering.

Q7-41. Which field in the IPv6 packet is responsible for multiplexing and demultiplexing?

Q7-42. Assume a datagram carries no option; do we still need a value for the next header field in Figure 7.47?

Q7-43. Which message in version 6 replaces the ARP request message in version 4? Which replaces the ARP reply message?

Q7-44. Which messages in version 6 replace the IGMPv6 messages in version 4?

Q7-45. In which transition strategy, do we need to encapsulate IPv6 packets in the IPv4 packets?

Q7-46. In which transition strategy, do we need to have both IPv4 and IPv6 in the path?

7.8.3 Problems

P7-1. What is the size of the address space in each of the following systems?

a. A system in which each address is only 16 bits

b. A system in which each address is made up of six hexadecimal digits

c. A system in which each address is made up of four octal digits

P7-2. Rewrite the following IP addresses using binary notation.

a. 110.11.5.88

b. 12.74.16.18

c. 201.24.44.32

P7-3. Rewrite the following IP addresses using dotted-decimal notation.

a. 01011110 10110000 01110101 00010101

b. 10001001 10001110 11010000 00110001

c. 01010111 10000100 00110111 00001111

P7-4. Find the class of the following classful IP addresses.

a. 130.34.54.12 **b.** 200.34.2.1 **c.** 245.34.2.8

P7-5. Find the class of the following classful IP addresses.

a. 01110111 11110011 10000111 11011101

b. 11101111 11000000 11110000 00011101

c. 11011111 10110000 00011111 01011101

P7-6. In classless addressing, show the whole address space as a single block using the CIDR notation.

P7-7. In classless addressing, what is the size of the block (N) if the value of the prefix length (n) is one of the following?

a. $n = 0$ **b.** $n = 14$ **c.** $n = 32$

P7-8. In classless addressing, what is the value of prefix length (n) if the size of the block (N) is one of the following?

a. $N = 1$ **b.** $N = 1024$ **c.** $N = 2^{32}$

P7-9. Change each of the following prefix lengths to a mask in dotted-decimal notation.

a. $n = 0$ **b.** $n = 14$ **c.** $n = 30$

P7-10. Change each of the following masks to a prefix length.

a. 255.224.0.0

b. 255.240.0.0

c. 255.255.255.128

P7-11. Which of the following cannot be a mask in CIDR?

a. 255.225.0.0

b. 255.192.0.0

c. 255.255.255.6

P7-12. Each of the following addresses belongs to a block. Find the first and last address in each block.

a. 14.12.72.8/24

b. 200.107.16.17/18

c. 70.110.19.17/16

P7-13. Show the *n* leftmost bits of the following network-addresses/masks that can be used in a forwarding table.

a. 170.40.11.0/24

b. 110.40.240.0/22

c. 70.14.0.0./18

P7-14. Explain how DHCP can be used when the size of the block assigned to an organization is less than the number of hosts in the organization.

P7-15. Assume we have an internet with an 8-bit address space. The addresses are equally divided between four networks (N_0 to N_3). The internetwork communication is done through a router with four interfaces (m0 to m3). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.

P7-16. Assume we have an internet with a 12-bit address space. The addresses are equally divided between eight networks (N_0 to N_7). The internetwork communication is done through a router with eight interfaces (m0 to m7). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.

P7-17. Assume we have an internet with a 9-bit address space. The addresses are divided between three networks (N_0 to N_2), with 64, 192, and 256 addresses, respectively. The internetwork communication is done through a router with three interfaces (m0 to m2). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.

P7-18. Combine the following three blocks of addresses into a single block:

a. 16.27.24.0/**26**

b. 16.27.24.64/**26**

c. 16.27.24.128/**25**

P7-19. A large organization with a large block address (12.44.184.0/**21**) is split into one medium-size company using the block address (12.44.184.0/**22**) and two small organizations. If the first small company uses the block (12.44.188.0/**23**), what is the remaining block that can be used by the second small company? Explain how the datagrams destined for the two small companies can be correctly routed to these companies if their address blocks are still part of the original company.

P7-20. An ISP is granted the block 16.12.64.0/**20**. The ISP needs to allocate addresses for eight organizations, each with 256 addresses.

a. Find the number and range of addresses in the ISP block.

b. Find the range of addresses for each organization and the range of unallocated addresses.

c. Show the outline of the address distribution and the forwarding table.

P7-21. An ISP is granted the block 80.70.56.0/21. The ISP needs to allocate addresses for two organizations each with 500 addresses, two organizations each with 250 addresses, and three organizations each with 50 addresses.

a. Find the number and range of addresses in the ISP block.

b. Find the range of addresses for each organization and the range of unallocated addresses.

c. Show the outline of the address distribution and the forwarding table.

P7-22. An organization is granted the block 130.56.0.0/16. The administrator wants to create 1024 subnets. Find the following:

a. Number of addresses in each subnet

b. Subnet prefix

c. First and last addresses in the first subnet

d. First and last addresses in the last subnet

P7-23. Can router R1 in Figure 7.35 receive a packet with destination address 140.24.7.194? What will happen to the packet if this occurs?

P7-24. Assume router R2 in Figure 7.35 receives a packet with destination address 140.24.7.42. How is the packet routed to its final destination?

P7-25. In an IPv4 datagram, the value of the total-length field is $(00A0)_{16}$ and the value of the header-length (HLEN) is $(5)_{16}$. How many bytes of payload are being carried by the datagram? What is the efficiency (ratio of the payload length to the total length) of this datagram?

P7-26. An IP datagram has arrived with the following partial information in the header (in hexadecimal):

```
45000054 00030000 2006...
```

a. What is the header size?

b. Are there any options in the packet?

c. What is the size of the data?

d. Is the packet fragmented?

e. How many more routers can the packet travel to?

f. What is the protocol number of the payload being carried by the packet?

P7-27. Referring to Figure 7.15, show the following:

a. How the wrapped sum can be calculated from the sum using modular arithmetic

b. How checksum can be calculated from the wrapped sum using modular arithmetic

P7-28. In Figure 7.15, show how the sum, wrapped sum, and checksum can be calculated when each word (16 bits) is created instead of waiting for the whole packet to be created.

P7-29. In Figure 7.15, show how the sum, wrapped sum, and checksum can be calculated when the words are given in decimal numbers (the way the words are stored in a computer memory).

P7-30. Which fields of the IPv4 main header may change from router to router?

P7-31. Determine if a datagram with the following information is a first fragment, a middle fragment. a last fragment, or the only fragment (no fragmentation):

a. M bit is set to 1 and the value of the offset field is zero.

b. M bit is set to 1 and the value of the offset field is nonzero.

P7-32. A packet has arrived in which the offset value is 300 and the payload size is 100 bytes. What are the numbers of the first and the last byte?

P7-33. Redo the checksum in Figure 7.22 using hexadecimal values.

P7-34. Redo the checksum in Figure 7.22 using decimal values and modular arithmetic.

P7-35. Briefly describe how we can defeat the following security attacks:

a. Packet sniffing

b. Packet modification

c. IP spoofing

P7-36. Redraw Figure 7.29 for the case where the mobile host acts as a foreign agent.

P7-37. Given the following information, show the contents of the IP datagram header sent from the remote host to the home agent.

Mobile host home address: 130.45.6.7/16
Mobile host care-of address: 14.56.8.9/8
Remote host address: 200.4.7.14/24
Home agent address: 130.45.10.20/16
Foreign agent address: 14.67.34.6/8

P7-38. Create a home agent advertisement message using 1456 as the sequence number and a lifetime of 3 h. Select your own values for the bits in the code field. Calculate and insert the value for the length field.

P7-39. Create a foreign agent advertisement message using 1672 as the sequence number and a lifetime of 4 h. Select your own values for the bits in the code field. Use at least three care-of addresses of your choice. Calculate and insert the value for the length field.

P7-40. Compare and contrast the IPv4 header with the IPv6 header. Create a table to compare each field.

P7-41. Show the unabbreviated colon hex notation for the following IPv6 addresses:

a. An address with 64 zeros followed by 32 two-bit (01)s

b. An address with 64 zeros followed by 32 two-bit (10)s

c. An address with 64 two-bit (01)s

d. An address with 32 four-bit (0111)s

P7-42. Show abbreviations for the following addresses:

a. 0000 : FFFF : FFFF : 0000:0000 : 0000 : 0000 :0000

b. 1234 : 2346 : 3456: 0000:0000 : 0000 : 0000 :FFFF

c. 0000 : 0001 : 0000: 0000:0000 : FFFF : 1200 :1000

d. 0000 : 0000 :0000 : 0000 : FFFF: FFFF: 24.123.12.6

P7-43. Decompress the following addresses, and show the complete unabbreviated IPv6 address.

a. :: 2222 **b.** 1111:: **c.** B : A :CC :: 1234:A

P7-44. Show the original (unabbreviated) form of the following IPv6 addresses.

a. ::2 **b.** 0:23::0 **c.** 0:A::3

P7-45. Give the corresponding block or subblock associated with each of the following IPv6 addresses based on Table 7.5.

a. FE80::12/**10** **b.** FD23::/**7** **c.** 32::/**3**

P7-46. An organization is assigned the block 2000:1234:1423/48. What is the CIDR for the blocks in the first and second subnets in this organization?

P7-47. Find the interface identifier if the physical address of the EUI is $(F5\text{-}A9\text{-}23\text{-}AA\text{-}07\text{-}14\text{-}7A\text{-}23)_{16}$ using the format we defined for Ethernet addresses.

P7-48. Find the interface identifier if the Ethernet physical address is $(F5\text{-}A9\text{-}23\text{-}12\text{-}7A\text{-}B2)_{16}$ using the format we defined for Ethernet addresses.

P7-49. An organization is assigned the block 2000:1110:1287/48. What is the IPv6 address of an interface in the third subnet if the IEEE physical address of the computer is $(F5\text{-}A9\text{-}23\text{-}14\text{-}7A\text{-}D2)_{16}$?

P7-50. Using the CIDR notation, show the IPv6 address compatible to the IPv4 address 129.6.12.34.

P7-51. Using the CIDR notation, show the IPv6 address mapped to the IPv4 address 129.6.12.34.

P7-52. Using the CIDR notation, show the IPv6 loopback address.

P7-53. Using the CIDR notation, show the link local address in which the node identifier is 0::123/48.

P7-54. Using the CIDR notation, show the site local address in which the node identifier is 0::123/48.

P7-55. An IPv6 packet consists of the base header and a TCP segment. The length of data is 320 bytes. Show the packet, and enter a value for each field.

P7-56. An IPv6 packet consists of a base header and a TCP segment. The length of data is 128,000 bytes (jumbo payload). Show the packet, and enter a value for each field.

P7-57. Which ICMP messages contain part of the IP datagram? Why is this needed?

P7-58. Make a table to compare and contrast error-reporting messages in ICMPv6 with error-reporting messages in ICMPv4.

P7-59. Make a table to compare and contrast informational messages in ICMPv6 with informational messages in ICMPv4.

P7-60. Make a table to compare and contrast neighbor-discovery messages in ICMPv6 with the corresponding messages in version 4.

P7-61. Make a table to compare and contrast inverse-neighbor-discovery messages in ICMPv6 with the corresponding messages in version 4.

P7-62. Make a table to compare and contrast group-membership messages in ICMPv6 with the corresponding messages in version 4.

CHAPTER 8

Network Layer: Routing of Packets

In an internet, the goal of the network layer is to deliver a datagram from its source to one or more destinations. If a datagram is destined for only one destination (one-to-one delivery), we have unicast routing. If the datagram is destined for several destinations (one-to-many delivery), we have multicast routing.

In previous chapters, we have shown that the routing can be possible if a router has a forwarding table to forward a packet to the appropriate next node on its way to the final destination or destinations. To make the forwarding tables of the router, the Internet needs routing protocols that will be active all the time in the background and update the forwarding tables.

This chapter is divided into four sections.

- The first section introduces the concept of unicast routing and describes the general ideas behind it. The section then describes the least-cost routing and the least-cost trees.
- The second section discusses the common unicast routing algorithm used in the Internet. The section first describes distance-vector routing. It then describes link-state routing. Finally, it explains path-vector routing.
- The third section explores unicast-routing protocols corresponding to the unicast-routing algorithms discussed in the second section. The section first describes RIP, a protocol that implements distance-vector routing algorithm. The section then describes OSPF, a protocol that implements the link-state routing algorithm. Finally, the section describes the BGP, a protocol that implements the path-vector routing algorithm.
- The fourth section explores multicast routing and multicast routing protocols.

8.1 INTRODUCTION

Unicast routing in the Internet, with a large number of routers and a huge number of hosts, can be done only by using hierarchical routing: routing in several steps using different routing algorithms. In this section, we first discuss the general concept of unicast routing in an *internet:* an internetwork made up of networks connected by routers. After the routing concepts and algorithms are understood, we show how we can apply them to the Internet using hierarchical routing.

8.1.1 General Idea

In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables. The source host does not need a forwarding table because it delivers its packet to the default router in its local network. The destination host does not need a forwarding table either because it receives the packet from its default router in its local network. This means that only the routers that glue together the networks in the internet need forwarding tables. With the preceding explanation, routing a packet from its source to its destination means routing the packet from a *source router* (the default router of the source host) to a *destination router* (the router connected to the destination network). Although a packet needs to visit the source and the destination routers, the question is what other routers the packet should visit. In other words, there are several routes that a packet can travel from the source to the destination; what must be determined is which route the packet should take.

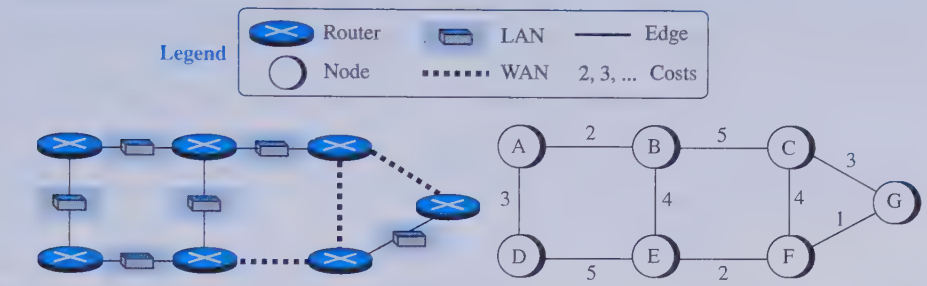
An Internet as a Graph

To find the best route, an internet can be modeled as a *graph*. A graph in computer science is a set of *nodes* and *edges* (lines) that connect the nodes. To model an internet as a graph, we can think of each router as a node and each network between a pair of routers as an edge. An internet is, in fact, modeled as a *weighted graph,* in which each edge is associated with a cost. If a weighted graph is used to represent a geographical area, the nodes can be cities and the edges can be roads connecting the cities; the weights, in this case, are distances between cities. In routing, however, the cost of an edge has a different interpretation in different routing protocols, which we discuss when we discuss that routing protocol. For the moment, we assume that there is a cost associated with each edge. If there is no edge between the nodes, the cost is infinity. Figure 8.1 shows how an internet can be modeled as a graph.

8.1.2 Least-Cost Routing

When an internet is modeled as a weighted graph, one of the ways to interpret the *best* route from the source router to the destination router is to find the *least cost* between the two. In other words, the source router chooses a route to the destination router in such a way that the total cost for the route is the least cost among all possible routes. In Figure 8.1, the best route between A and E is A-B-E, with the cost of 6. This means that each router needs to find the least-cost route between itself and all the other routers to be able to route a packet using this criteria.

Figure 8.1 *An internet and its graphical representation*

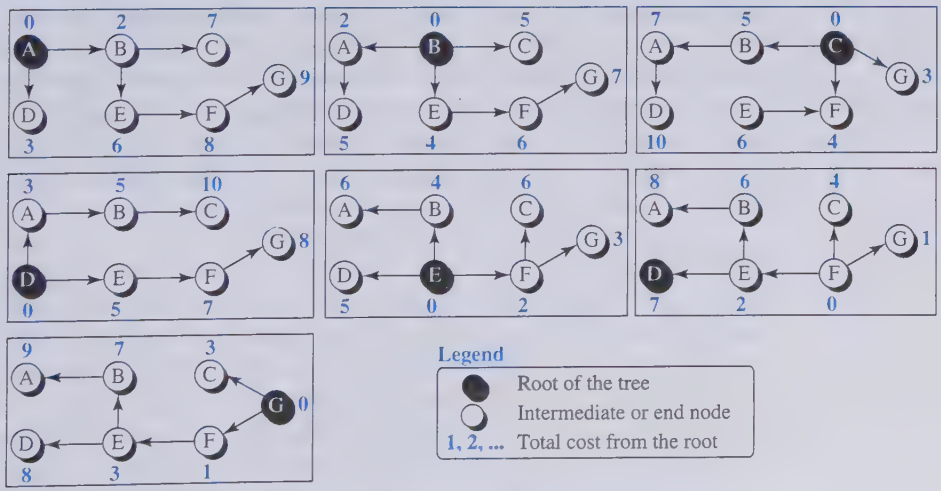


a. An internet b. The weighted graph

Least-Cost Trees

If there are N routers in an internet, there are $(N - 1)$ least-cost paths from each router to any other router. This means we need $N \times (N - 1)$ least-cost paths for the whole internet. If we have only 10 routers in an internet, we need 90 least-cost paths. A better way to see all these paths is to combine them into a **least-cost tree**. A least-cost tree is a tree with the source router as the root that spans the whole graph (visits all other nodes) and in which the path between the root and any other node is the shortest. In this way, we can have only one **shortest-path tree** for each node; we have N least-cost path trees for the whole internet. Figure 8.2 shows the seven least-cost trees for the internet in Figure 8.1.

Figure 8.2 *Least-cost trees for nodes in the internet of Figure 8.1*



The least-cost trees for a weighted graph can have several properties if they are created using consistent criteria.

1. The least-cost route from X to Y in X's tree is the inverse of the least cost route from Y to X in Y's tree; the cost in both directions is the same. For example, in Figure 8.2, the route from A to F in A's tree is (A → B → E → F), but the route from F to A in F's tree is (F → E → B → A), which is the inverse of the first route. The cost is 8 in each case.
2. Instead of traveling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree. For example, in Figure 8.2, we can go from A to G in A's tree using the route (A → B → E → F → G). We can also go from A to E in A's tree (A → B → E) and then continue in E's tree using the route (E → F → G). The combination of the two routes in the second case is the same route as in the first case. The cost in the first case is 9; the cost in the second case is also 9 (6 + 3).

8.2 ROUTING ALGORITHMS

After discussing the general idea behind least-cost trees and the forwarding tables that can be made from them, now we concentrate on the routing algorithms. Several routing algorithms have been designed in the past. The differences between these methods are in the way they interpret the least cost and the way they create the least-cost tree for each node. In this section, we discuss the common algorithm; later we show how a routing protocol in the Internet implements one of these algorithms.

8.2.1 Distance-Vector Routing

The **distance-vector (DV) routing** uses the goal we discussed in the introduction to find the best route. In distance-vector routing, the first thing each node creates is its own least-cost tree with the rudimentary information it has about its immediate neighbors. The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet. We can say that in distance-vector routing, a router continuously tells all its neighbors about what it knows about the whole internet (although the knowledge can be incomplete).

Before we show how incomplete least-cost trees can be combined to make complete ones, we need to discuss two important topics: the Bellman-Ford equation and the concept of distance vectors, which we cover next.

Bellman-Ford Equation

The heart of distance-vector routing is the famous **Bellman-Ford equation**. This equation is used to find the least cost (shortest distance) between a source node, x, and a destination node, y, through some intermediary nodes (**a, b, c,** . . .) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and the destination are given. The following shows the general case in which D_{ij} is the shortest distance and c_{ij} is the cost between nodes i and j.

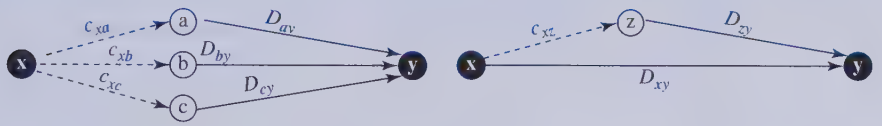
$$D_{xy} = \min \{(c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \ldots\}$$

In distance-vector routing, normally we want to update an existing least cost with a least cost through an intermediary node, such as z, if the latter is shorter. In this case, the equation becomes simpler:

$$D_{xy} = \min\{D_{xy}, (c_{xz} + D_{zy})\}$$

Figure 8.3 shows the idea graphically for both cases.

Figure 8.3 *Graphical idea behind Bellman-Ford equation*



a. General case with three intermediate nodes

b. Updating a path with a new route

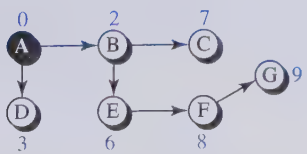
We can say that the Bellman-Ford equation enables us to build a new least-cost path from previously established least-cost paths. In Figure 8.3, we can think of $(a \rightarrow y)$, $(b \rightarrow y)$, and $(c \rightarrow y)$ as previously established least-cost paths and $(x \rightarrow y)$ as the new least-cost path. We can even think of this equation as the builder of a new least-cost tree from previously established least-cost trees if we use the equation repeatedly. In other words, the use of this equation in distance-vector routing is a witness that this method also uses least-cost trees, but this use may be in the background.

We will shortly show how we use the Bellman-Ford equation and the concept of distance vectors to build least-cost paths for each node in distance-vector routing, but first we need to discuss the concept of a distance vector.

Distance Vectors

The concept of a **distance vector** is the rationale for the name *-distance-vector routing*. A least-cost tree is a combination of least-cost paths from the root of the tree to all destinations. These paths are graphically glued together to form the tree. Distance-vector routing unglues these paths and creates a *distance vector*, a one-dimensional array to represent the tree. Figure 8.4 shows the tree for node A in the internet in Figure 8.1 and the corresponding distance vector.

Figure 8.4 *The distance vector corresponding to a tree*



a. Tree for node A

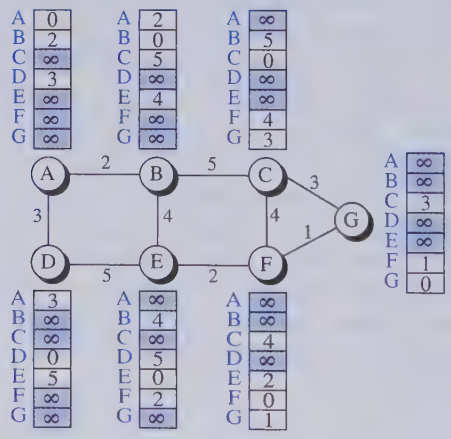
	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

Note that the *name* of the distance vector defines the root, the *indexes* define the destinations, and the *value* of each cell defines the least cost from the root to the destination. A distance vector does not give the path to the destinations as the least-cost tree does; it gives only the least costs to the destinations. Later we show how we can change a distance vector to a forwarding table, but we first need to find all distance vectors for an internet.

We know that a distance vector can represent least-cost paths in a least-cost tree, but the question is how each node in an internet originally creates the corresponding vector. Each node in an internet, when it is booted, creates a very rudimentary distance vector with the minimum information the node can obtain from its neighborhood. The node sends some greeting messages out of its interfaces and discovers the identity of the immediate neighbors and the distance between itself and each neighbor. It then makes a simple distance vector by inserting the discovered distances in the corresponding cells and leaves the value of other cells as infinity. Do these distance vectors represent least-cost paths? They do, considering the limited information a node has. When we know only one distance between two nodes, it is the least cost. Figure 8.5 shows all distance vectors for our internet. However, we need to mention that these vectors are made asynchronously, when the corresponding node has been booted; the existence of all of them in a figure does not mean synchronous creation of them.

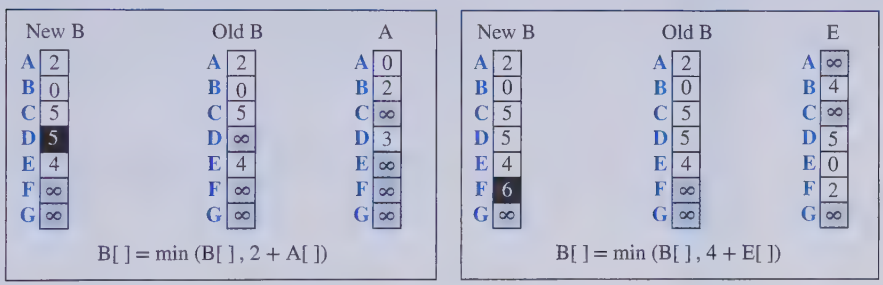
Figure 8.5 *The first distance vector for an internet*



These rudimentary vectors cannot help the internet to effectively forward a packet. For example, node A thinks that it is not connected to node G because the corresponding cell shows the least cost of infinity. To improve these vectors, the nodes in the internet need to help each other by exchanging information. After each node has created its

vector, it sends a copy of the vector to all its immediate neighbors. After a node receives a distance vector from a neighbor, it updates its distance vector using the Bellman-Ford equation (second case). However, we must understand that we need to update, not only one least cost, but N of them, in which N is the number of the nodes in the internet. If we are using a program, we can do this using a loop; if we are showing the concept on paper, we can show the whole vector instead of the N separate equations. In Figure 8.6, we show the whole vector instead of seven equations for each update. The figure shows two asynchronous events, happening one after another with some time in between. In the first event, node A has sent its vector to node B. Node B updates its vector using the cost $c_{BA} = 2$. In the second event, node E has sent its vector to node B. Node B updates its vector using the cost $c_{EA} = 4$.

Figure 8.6 *Updating distance vectors*



a. First event: B receives a copy of A's vector.

b. Second event: B receives a copy of E's vector.

Note:
X[ ]: the whole vector

After the first event, node B has one improvement in its vector; its least cost to node D has changed from infinity to 5 (via node A). After the second event, node B has one more improvement in its vector; its least cost to node F has changed from infinity to 6 (via node E). We hope that we have convinced the reader that exchanging vectors eventually stabilizes the system and allows all nodes to find the ultimate least cost between themselves and any other node. We need to remember that after updating a node, it immediately sends its updated vector to all neighbors. Even if its neighbors have received the previous vector, the updated one may help more.

Distance-Vector Routing Algorithm

Now we can give a simplified pseudocode for the distance-vector routing algorithm, as shown in Table 8.1. The algorithm is run by its node independently and asynchronously.

Table 8.1 *Distance-vector routing algorithm for a node*

```
1  Distance_Vector_Routing ( )
2  {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7        if (y is a neighbor)
8           D[y] = c[myself][y]
9        else
10          D[y] = ∞
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16       wait (for a vector D_w from a neighbor w or any change in the link)
17       for (y = 1 to N)
18       {
19          D[y] = min [D[y], (c[myself][w] + D_w[y])] // Bellman-Ford equation
20       }
21       if (any change in the vector)
22          send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 }
```

Lines 4 to 11 initialize the vector for the node. Lines 14 to 23 show how the vector can be updated after receiving a vector from the immediate neighbor. The *for* loop in lines 17 to 20 allows all entries (cells) in the vector to be updated after receiving a new vector. Note that the node sends its vector in line 12, after being initialized, and in line 22, after it is updated.

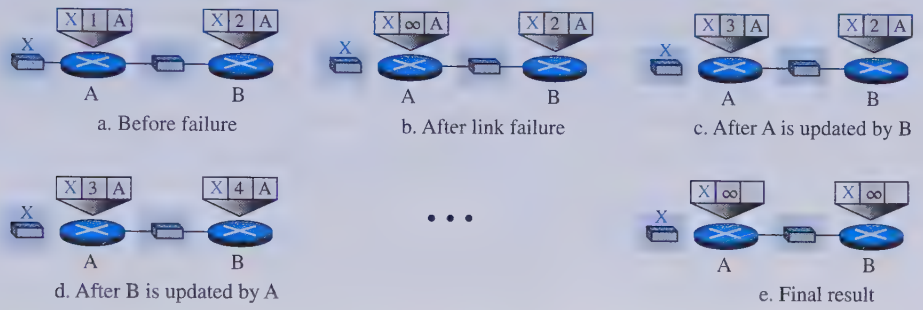
Count to Infinity

A problem with distance-vector routing is that any decrease in cost (good news) propagates quickly, but any increase in cost (bad news) will propagate slowly. For a routing protocol to work properly, if a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time. The problem is referred to as *count to infinity*. It sometimes takes several updates before the cost for a broken link is recorded as infinity by all routers.

Two-Node Loop

One example of count to infinity is the two-node loop problem. To understand the problem, let us look at the scenario depicted in Figure 8.7.

Figure 8.7 *Two-node instability*



The figure shows a system with three nodes. We have shown only the portions of the forwarding table needed for our discussion. At the beginning, both nodes A and B know how to reach node X. But suddenly, the link between A and X fails. Node A changes its table. If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its forwarding table to A before receiving A's forwarding table. Node A receives the update and, assuming that B has found a way to reach X, immediately updates its forwarding table. Now A sends its new update to B. Now B thinks that something has been changed around A and updates its forwarding table. The cost of reaching X increases gradually until it reaches infinity. At this moment, both A and B know that X cannot be reached. However, during this time the system is not stable. Node A thinks that the route to X is via B; node B thinks that the route to X is via A. If A receives a packet destined for X, the packet goes to B and then comes back to A. Similarly, if B receives a packet destined for X, it goes to A and comes back to B. Packets bounce between A and B, creating a two-node loop problem. A few solutions have been proposed for instability of this kind.

Split Horizon

One solution to instability is called *split horizon*. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface. If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows). Taking information from node A, modifying it, and sending it back to node A is what creates the confusion. In our scenario, node B eliminates the last line of its forwarding table before it sends it to A. In this case, node A keeps the value of infinity as the distance to X. Later, when node A sends its forwarding table to

B, node B also corrects its forwarding table. The system becomes stable after the first update: Both nodes A and B know that X is not reachable.

Poisoned Reverse

Using the split-horizon strategy has one drawback. Normally, the corresponding protocol uses a timer, and if there is no news about a route, the node deletes the route from its table. When node B in the previous scenario eliminates the route to X from its advertisement to A, node A cannot guess whether this is due to the split-horizon strategy (the source of information was A) or because B has not received any news about X recently. In the poisoned reverse strategy, B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning: "Do not use this value; what I know about this route comes from you."

Three-Node Instability

The two-node instability can be avoided using split horizon combined with poisoned reverse. However, if the instability is between three nodes, stability cannot be guaranteed.

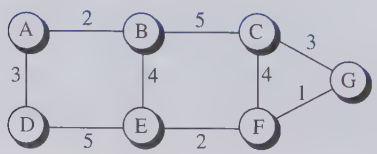
8.2.2 Link-State Routing

A routing algorithm that directly follows our discussion for creating least-cost trees and forwarding tables is **link-state (LS) routing**. This method uses the term *link state* to define the characteristic of a link (an edge) that represents a network in the internet. In this algorithm the cost associated with an edge defines the state of the link. Links with lower costs are preferred to links with higher costs; if the cost of a link is infinity, it means that the link does not exist or has been broken.

Link-State Database (LSDB)

To create a least-cost tree with this method, each node needs to have a complete *map* of the network, which means it needs to know the state of each link. The collection of states for all links is called the **link-state database** (**LSDB**). There is only one LSDB for the whole internet; each node needs to have a duplicate of it to be able to create the least-cost tree. Figure 8.8 shows an example of an LSDB for the graph in Figure 8.1. The LSDB can be represented as a two-dimensional array (matrix) in which the value of each cell defines the cost of the corresponding link.

Figure 8.8 *Example of a link-state database*



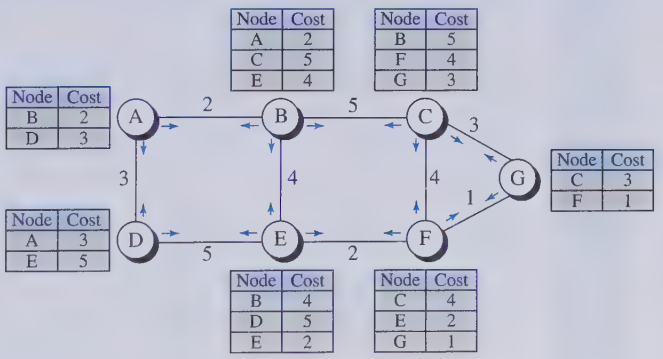
a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

Now the question is, "How can each node create this LSDB that contains information about the whole internet?" This can be done by a process called **flooding**. Each node can send some greeting messages to all its immediate neighbors (those nodes to which it is connected directly) to collect two pieces of information for each neighboring node: the identity of the node and the cost of the link. The combination of these two pieces of information is called the *LS packet* (LSP); the LSP is sent out of each interface, as shown in Figure 8.9 for our internet in Figure 8.1. When a node receives an LSP from one of its interfaces, it compares the LSP with the copy it may already have. If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP. If it is newer or the first one received, the node discards the old LSP (if there is one) and keeps the received one. It then sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the network (where a node has only one interface). We need to convince ourselves that, after receiving all new LSPs, each node creates the comprehensive LSDB as shown in Figure 8.9. This LSDB is the same for each node and shows the whole map of the internet. In other words, a node can make the whole map if it needs to, using this LSDB.

Figure 8.9 *LSPs created and sent out by each node to build the LSDB*



We can compare the link-state routing algorithm with the distance-vector routing algorithm. In the distance-vector routing algorithm, each router tells its neighbors what it knows about the whole internet; in the link-state routing algorithm, each router tells the whole internet what it knows about its neighbors.

Formation of Least-Cost Trees

To create a least-cost tree for itself, using the shared LSDB, each node needs to run the famous **Dijkstra's algorithm**. This iterative algorithm uses the following steps:

1. The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.

2. The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
3. The node repeats step 2 until all nodes are added to the tree.

We need to convince ourselves that these three steps finally create the least-cost tree. Table 8.2 shows a simplified version of Dijkstra's algorithm.

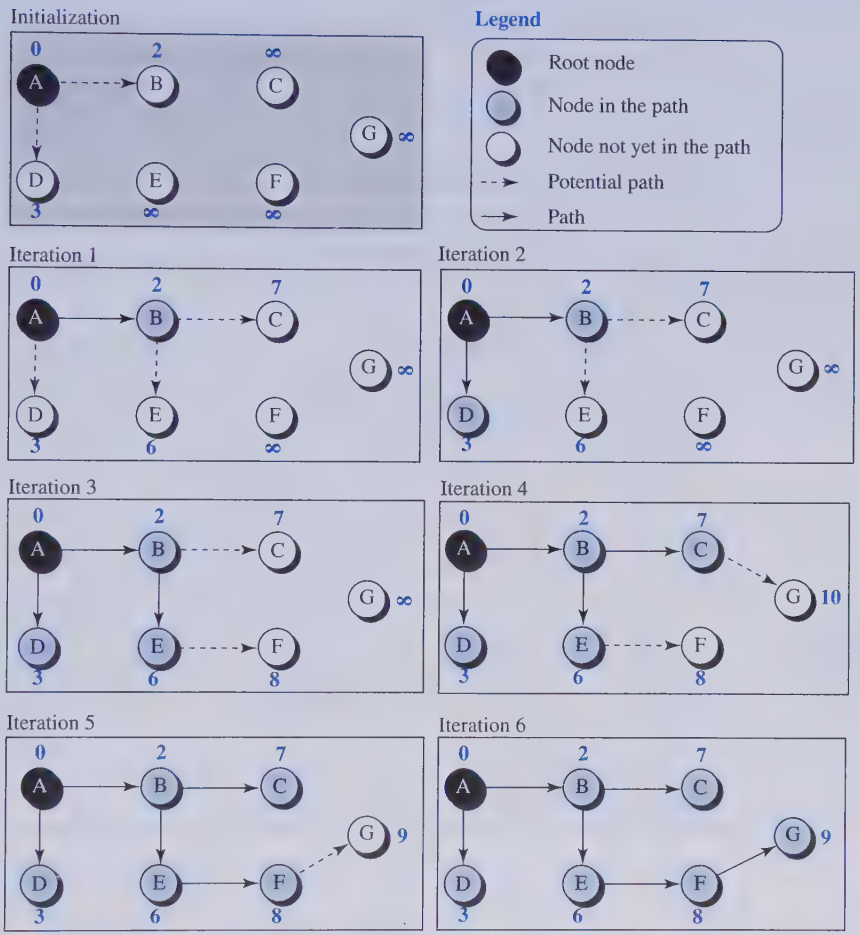
Table 8.2 *Dijkstra's algorithm*

```
1  Dijkstra's Algorithm ( )
2  {
3     // Initialization
4     Tree = {root}          // Tree is made only of the root
5     for (y = 1 to N)       // N is the number of nodes
6     {
7        if (y is the root)
8           D [y] = 0        // D [y] is shortest distance from root to node y
9        else if (y is a neighbor)
10          D [y] = c[root][y]      // c [x] [y] is cost between nodes x and y in LSDB
11       else
12          D [y] = ∞
13    }
14    // Calculation
15    repeat
16    {
17       find a node w, with D [w ] minimum among all nodes not in the Tree
18       Tree = Tree ∪ {w}      // Add w to tree
19       // Update distances for all neighbor of w
20       for (every node x, which is neighbor of w and not in the Tree)
21       {
22          D[x] = min{D[x], (D[w] + c[w][x])}
23       }
24    } until (all nodes included in the Tree)
25 }
```

Lines 4 to 13 implement step 1 in the algorithm. Lines 16 to 23 implement step 2 in the algorithm. Step 2 is repeated until all nodes are added to the tree.

Figure 8.10 shows the formation of the least-cost tree for the graph in Figure 8.8 using Dijkstra's algorithm. We need to go through an initialization step and six iterations to find the least-cost tree.

Figure 8.10 *Least-cost tree*



8.2.3 Path-Vector Routing

Both link-state and distance-vector routing are based on the least-cost goal. However, there are instances where this goal is not the priority. For example, assume that there are some routers in the internet that a sender wants to prevent its packets from going through. For example, a router may belong to an organization that does not provide enough security or that belongs to a commercial rival of the sender who might inspect the packets to obtain information. Least-cost routing does not prevent a packet from passing through an area when that area is in the least-cost path. In other words, the least-cost goal, applied by LS or DV routing, does not allow a sender to apply specific

policies to the route a packet may take. Aside from safety and security, there are occasions in which the mere goal of routing is reachability: to allow the packet to reach its destination more efficiently without assigning costs to the route.

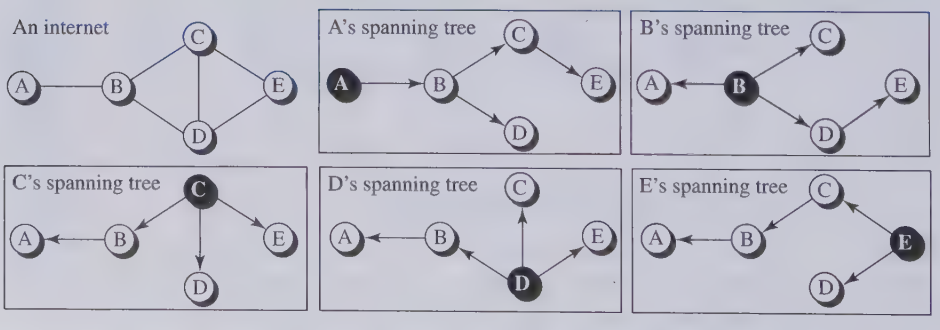
To respond to these demands, a third routing algorithm, called **path-vector (PV) routing**, has been devised. Path-vector routing does not have the drawbacks of LS or DV routing as described previously because it is not based on least-cost routing. The best route is determined by the source using the policy it imposes on the route. In other words, the source can control the path. Although path-vector routing is not actually used in an internet, and is mostly designed to route a packet between ISPs, we discuss the principle of this method in this section as though applied to an internet. In Section 8.3, we show how it is used in the Internet.

Spanning Trees

In path-vector routing, the path from a source to all destinations is also determined by the *best* spanning tree. The best spanning tree, however, is not the least-cost tree; it is the tree determined by the source when it imposes its own policy. If there is more than one route to a destination, the source can choose the route that meets its policy best. A source may apply several policies at the same time. One of the common policies uses the minimum number of nodes to be visited (something similar to least-cost). Another common policy is to avoid some nodes as the middle node in a route.

Figure 8.11 shows a small internet with only five nodes. Each source has created its own spanning tree that meets its policy. The policy imposed by all sources is to use the minimum number of nodes to reach a destination. The spanning tree selected by A and E is such that the communication does not pass through D as a middle node. Similarly, the spanning tree selected by B is such that the communication does not pass through C as a middle node.

Figure 8.11 *Spanning trees in path-vector routing*



Creation of Spanning Trees

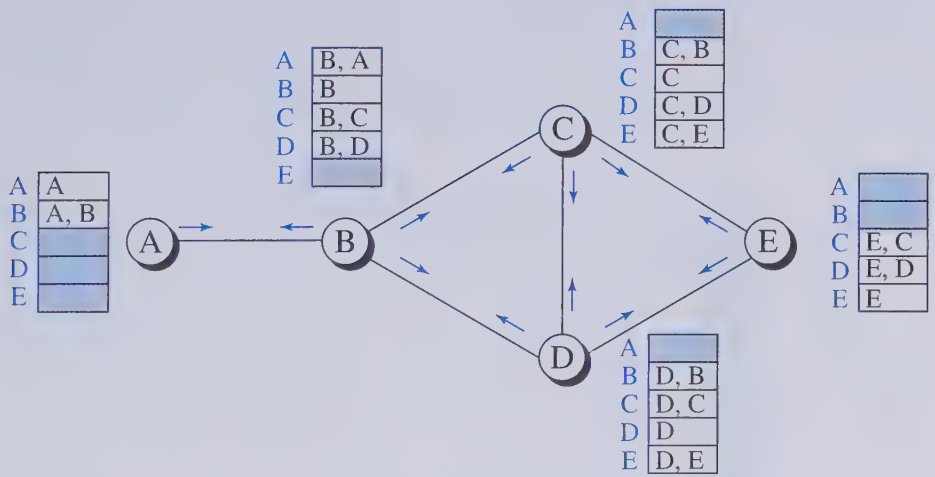
Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm. The spanning trees are made, gradually and asynchronously, by each node. When a node is booted, it creates a *path vector* based on the information it can

obtain about its immediate neighbor. A node sends greeting messages to its immediate neighbors to collect these pieces of information. Figure 8.12 shows all these path vectors for our internet in Figure 8.11. Note, however, that we do not mean that all these tables are created simultaneously; they are created when each node is booted. Figure 8.12 also shows how these path-vectors are sent to immediate neighbors after they have been created (arrows).

Each node, after the creation of the initial path vector, sends it to all its immediate neighbors. Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford equation, but applies its own policy instead of looking for the least cost. We can define this equation as

$$\text{Path}(x, y) = \text{best}\{\text{Path}(x, y), [(x + \text{Path}(\mathbf{v}, y)]\} \quad \text{for all } \mathbf{v}\text{'s in the internet}$$

Figure 8.12 *Path vectors made at booting time*

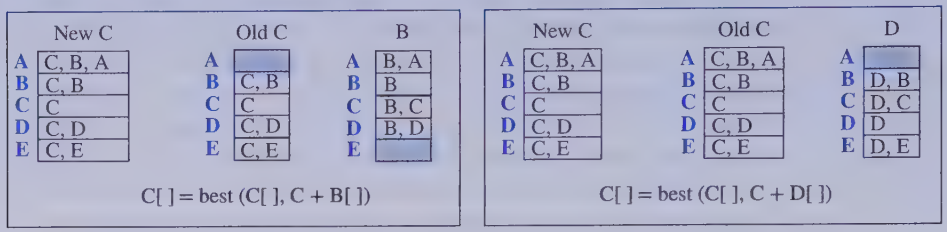


In this equation, the operator (+) means to add x to the beginning of the path. We also need to be cautious to avoid adding a node to an empty path because an empty path means one that does not exist.

The policy is defined by selecting the *best* of multiple paths. Path-vector routing also imposes one more condition on this equation: If path ($\mathbf{v}$, y) includes x, that path is discarded to avoid a loop in the path. In other words, x does not want to visit itself when it selects a path to y.

Figure 8.13 shows the path vector of node C after two events. In the first event, node C receives a copy of B's vector, which improves its vector: Now it knows how to reach node A. In the second event, node C receives a copy of D's vector, which does not change its vector. As a matter of fact, the vector for node C after the first event is stabilized and serves as its forwarding table.

Figure 8.13 *Updating path vectors*



Event 1: C receives a copy of B's vector

Event 2: C receives a copy of D's vector

Note:
X [ ]: vector X
Y: node Y

Path-Vector Algorithm

Based on the initialization process and the equation used in updating each forwarding table after receiving path vectors from neighbors, we can write a simplified version of the path vector algorithm as shown in Table 8.3.

Table 8.3 *Path-vector algorithm for a node*

```
1   Path_Vector_Routing ( )
2   {
3     // Initialization
4     for (y = 1 to N)
5     {
6       if (y is myself)
7         Path[y] = myself
8       else if (y is a neighbor)
9         Path[y] = myself + neighbor node
10      else
11        Path[y] = empty
12    }
13    Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14    // Update
15    repeat (forever)
16    {
17      wait (for a vector Path_w from a neighbor w)
18      for (y = 1 to N)
```

```
19        {
20            if (Path_w includes myself)
21                discard the path                    // Avoid any loop
22            else
23                Path[y] = best {Path[y], (myself + Path_w[y])}
24        }
25        If (there is a change in the vector)
26            Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27    }
28  } // End of Path Vector
```

Lines 4 to 12 show the initialization for the node. Lines 17 to 24 show how the node updates its vector after receiving a vector from the neighbor. The update process is repeated forever. We can see the similarities between this algorithm and the DV algorithm.

8.3 UNICAST ROUTING PROTOCOLS

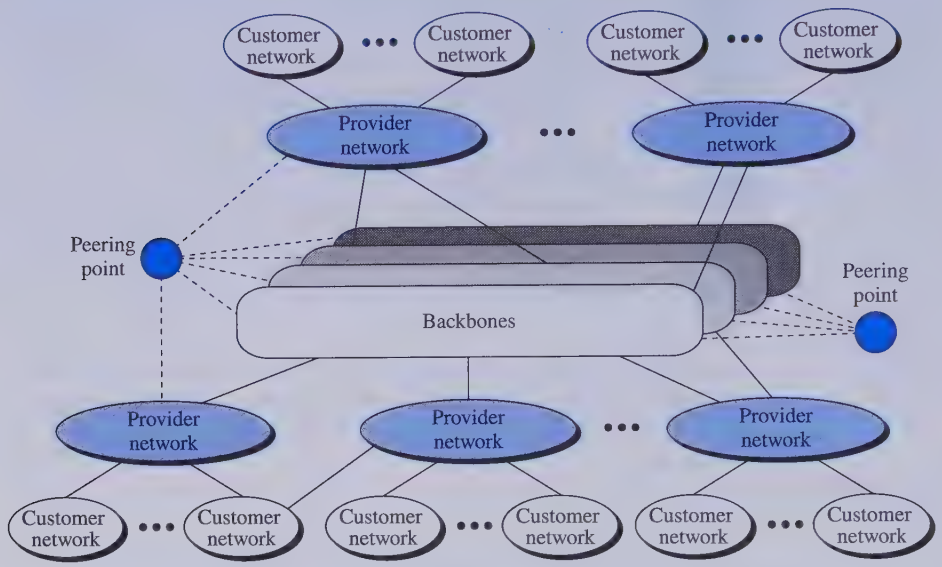
In Section 8.2, we discussed unicast routing algorithms; in this section, we discuss unicast routing protocols used in the Internet. Although three protocols we discuss here are based on the corresponding algorithms we discussed before, a protocol is more than an algorithm. A protocol needs to define its domain of operation, the messages exchanged, communication between routers, and interaction with protocols in other domains. After an introduction, we discuss three common protocols used in the Internet: Routing Information Protocol (RIP), based on the distance-vector algorithm, Open Shortest Path First (OSPF), based on the link-state algorithm; and Border Gateway Protocol (BGP), based on the path-vector algorithm.

8.3.1 Internet Structure

Before discussing unicast routing protocols, we need to understand the structure of today's Internet. The Internet has changed from a tree-like structure, with a single backbone, to a multi-backbone structure run by different private corporations today. Although it is difficult to give a general view of the Internet today, we can say that the Internet has a structure similar to what is shown in Figure 8.14.

There are several *backbones* run by private communication companies that provide global connectivity. These backbones are connected by some *peering points* that allow connectivity between backbones. At a lower level, there are some *provider networks* that use the backbones for global connectivity but provide services to Internet customers. Finally, there are some *consumer networks* that use the services provided by the provider networks. Any of these three entities (backbone, provider network, or customer network) can be called an Internet Service Provider (ISP). They provide services, but at different levels.

Figure 8.14 *Internet structure*



Hierarchical Routing

The Internet today is made up of a huge number of networks and routers that connect them. It is obvious that routing in the Internet cannot be done using one single protocol for two reasons: a scalability problem and an administrative issue. The *scalability problem* means that the size of the forwarding tables becomes huge, searching for a destination in a forwarding table becomes time consuming, and updating creates a huge amount of traffic. The *administrative issue* is related to the Internet structure described in Figure 8.14. As the figure shows, each ISP is run by an administrative authority. The administrator needs to have control in its system. The organization must be able to use as many subnets and routers as it needs, may desire that the routers be from a particular manufacturer, may wish to run a specific routing algorithm to meet the needs of the organization, and may want to impose some policy on the traffic passing through its ISP.

Hierarchical routing means considering each ISP as an **autonomous system (AS)**. Each AS can run a routing protocol that meets its needs, but the global Internet runs a global protocol to glue all ASs together. The routing protocol run in each AS is referred to as *intra-AS routing protocol*, *intradomain routing protocol*, or *interior gateway protocol (IGP)*; the global routing protocol is referred to as *inter-AS routing protocol*, *interdomain routing protocol*, or *exterior gateway protocol (EGP)*. We can have several intradomain routing protocols, and each AS is free to choose one, but it should be clear that we should have only one interdomain protocol that handles routing between these entities. Presently, the two common intradomain routing protocols are RIP and OSPF; the only interdomain routing protocol is BGP. The situation may change when we move to IPv6.

Autonomous Systems

As we said before, each ISP is an autonomous system when it comes to managing networks and routers under its control. Although we may have small, medium-size, and large ASs, each AS is given an autonomous number (ASN) by the Internet Corporation for Assigned Names and Numbers (ICANN). Each ASN is a 16-bit unsigned integer that uniquely defines an AS. The autonomous systems, however, are not categorized according to their size; they are categorized according to the way they are connected to other ASs. We have stub ASs, multihomed ASs, and transient ASs. The type affects the operation of the interdomain routing protocol in relation to that AS.

- **Stub AS.** A stub AS has only one connection to another AS. The data traffic can be either initiated or terminated in a stub AS; the data cannot pass through it. A good example of a stub AS is the costumer network, which is either the source or the sink of data.
- **Multihomed AS.** A multihomed AS can have more than one connection to other ASs, but it does not allow data traffic to pass through it. A good example of such an AS is some of the costumer ASs that may use the services of more than one provider network, but their policy does not allow data to be passed through them.
- **Transient AS.** A transient AS is connected to more than one other AS and also allows the traffic to pass through. The provider networks and the backbone are good examples of transient ASs.

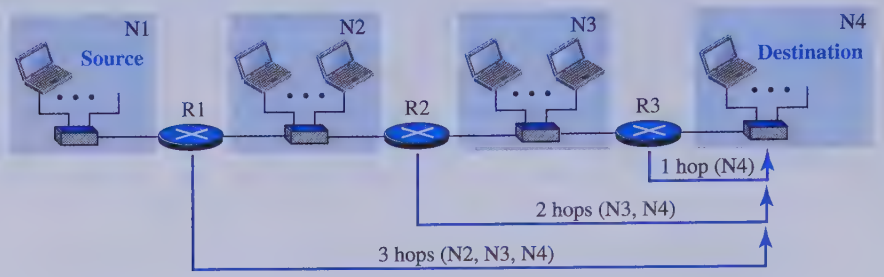
8.3.2 Routing Information Protocol (RIP)

The **Routing Information Protocol (RIP)** is one of the most widely used intradomain routing protocols based on the distance-vector routing algorithm we described earlier in Section 8.2.1. RIP was started as part of the Xerox Network System (XNS), but it was the Berkeley Software Distribution (BSD) version of UNIX that helped make the use of widespread RIP.

Hop Count

A router in this protocol basically implements the distance-vector routing algorithm shown in Table 8.1. However, the algorithm has been modified as we will now describe. First, because a router in an AS needs to know how to forward a packet to different networks (subnets) in an AS, RIP routers advertise the cost of reaching different networks instead of reaching other nodes in a theoretical graph. In other words, the cost is defined between a router and the network in which the destination host is located. Second, to make the implementation of the cost simpler (independent from performance factors of the routers and links, such as delay, and bandwidth), the cost is defined as the number of hops, which means the number of networks (subnets) a packet needs to travel through from the source router to the final destination host. Note that the network in which the source host is connected is not counted in this calculation because the source host does not use a forwarding table; the packet is delivered to the default router. Figure 8.15 shows the concept of hop count advertised by three routers from a source host to a destination host. In RIP, the maximum cost of a path can be 15, which means 16 is considered as

Figure 8.15 *Hop counts in RIP*



infinity (no connection). For this reason, RIP can be used only in autonomous systems in which the diameter of the AS is not more than 15 hops.

Forwarding Tables

Although the distance-vector algorithm we discussed in Section 8.2.1 is concerned with exchanging distance vectors between neighboring nodes, the routers in an autonomous system need to keep forwarding tables to forward packets to their destination networks. A forwarding table in RIP is a three-column table in which the first column is the address of the destination network, the second column is the address of the next router to which the packet should be forwarded, and the third column is the cost (the number of hops) to reach the destination network. Figure 8.16 shows the three forwarding tables for the routers in Figure 8.15. Note that the first and third columns together convey the same information as does a distance vector, but the cost shows the number of hops to the destination networks.

Figure 8.16 *Forwarding tables*

Forwarding table for R1

Destination network	Next router	Cost in hops
N1	——	1
N2	——	1
N3	R2	2
N4	R2	3

Forwarding table for R2

Destination network	Next router	Cost in hops
N1	R1	2
N2	——	1
N3	——	1
N4	R3	2

Forwarding table for R3

Destination network	Next router	Cost in hops
N1	R2	3
N2	R2	2
N3	——	1
N4	——	1

Although a forwarding table in RIP defines only the next router in the second column, it gives the information about the whole least-cost tree based on the second property of these trees. For example, R1 defines that the next router for the path to N4 is R2; R2 defines that the next router to N4 is R3; R3 defines that there is no next router for this path. The tree is then R1 → R2 → R3 → N4.

A question often asked about the forwarding table is what the use of the third column is. The third column is not needed for forwarding the packet, but it is needed for updating the forwarding table when there is a change in the route, as we will see shortly.

RIP Implementation

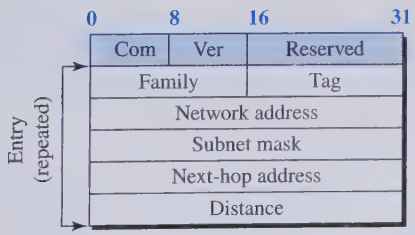
RIP is implemented as a process that uses the service of User Datagram Protocol (UDP) on the well-known port number 520. In BSD, RIP is a daemon process (a process running at the background), named *routed* (abbreviation for *route daemon* and pronounced *route-dee*). This means that, although RIP is a routing protocol to help IP route its datagrams through the AS, the RIP messages are encapsulated inside UDP user datagrams, which in turn are encapsulated inside IP datagrams. In other words, RIP runs at the application layer, but creates forwarding tables for IP at the network later.

RIP has gone through two versions: RIP-1 and RIP-2. The second version is backward-compatible with the first section; it allows the use of more information in the RIP messages that were set to 0 in the first version. We discuss only RIP-2 in this section.

RIP Messages

Two RIP processes, a client and a server, like any other processes, need to exchange messages. RIP-2 defines the format of the message, as shown in Figure 8.17. Part of the message, which we call an entry, can be repeated as needed in a message. Each entry carries the information related to one line in the forwarding table of the router that sends the message.

Figure 8.17 *RIP message format*



Fields

Com: Command, request (1), response (2)
Ver: Version, current version is 2
Family: Family of protocol, for TCP/IP value is 2
Tag: Information about autonomous system
Network address: Destination address
Subnet mask: Prefix length
Next-hop address: Address length
Distance: Number of hops to the destination

RIP has two types of messages: request and response. A request message is sent by a router that has just come up or by a router that has some time-out entries. A request message can ask about specific entries or all entries. A response (or update) message can be either solicited or unsolicited. A solicited response message is sent only in answer to a request message. It contains information about the destination specified in the corresponding request message. An unsolicited response message, on the other hand, is sent periodically, every 30 s or when there is a change in the forwarding table.

RIP Algorithm

RIP implements the same algorithm as the distance-vector routing algorithm we discussed in Section 8.2.1. However, some changes need to be made to the algorithm to enable a router to update its forwarding table:

- Instead of sending only distance vectors, a router needs to send the whole contents of its forwarding table in a response message.
- The receiver adds one hop to each cost and changes the next router field to the address of the sending router. We call each route in the modified forwarding table the *received route* and each route in the old forwarding table the *old route*. The received router selects the old routes as the new ones except in the following three cases:
  1. If the received route does not exist in the old forwarding table, it should be added to the route.
  2. If the cost of the received route is lower than the cost of the old one, the received route should be selected as the new one.
  3. If the cost of the received route is higher than the cost of the old one, but the value of the next router is the same in both routes, the received route should be selected as the new one. This is the case where the route was actually advertised by the same router in the past, but now the situation has been changed. For example, suppose a neighbor has previously advertised a route to a destination with cost 3, but now there is no path between this neighbor and that destination. The neighbor advertises this destination with cost value infinity (16 in RIP). The receiving router must not ignore this value even though its old route has a lower cost to the same destination.
- The new forwarding table needs to be sorted according to the destination route (mostly using the longest prefix first).

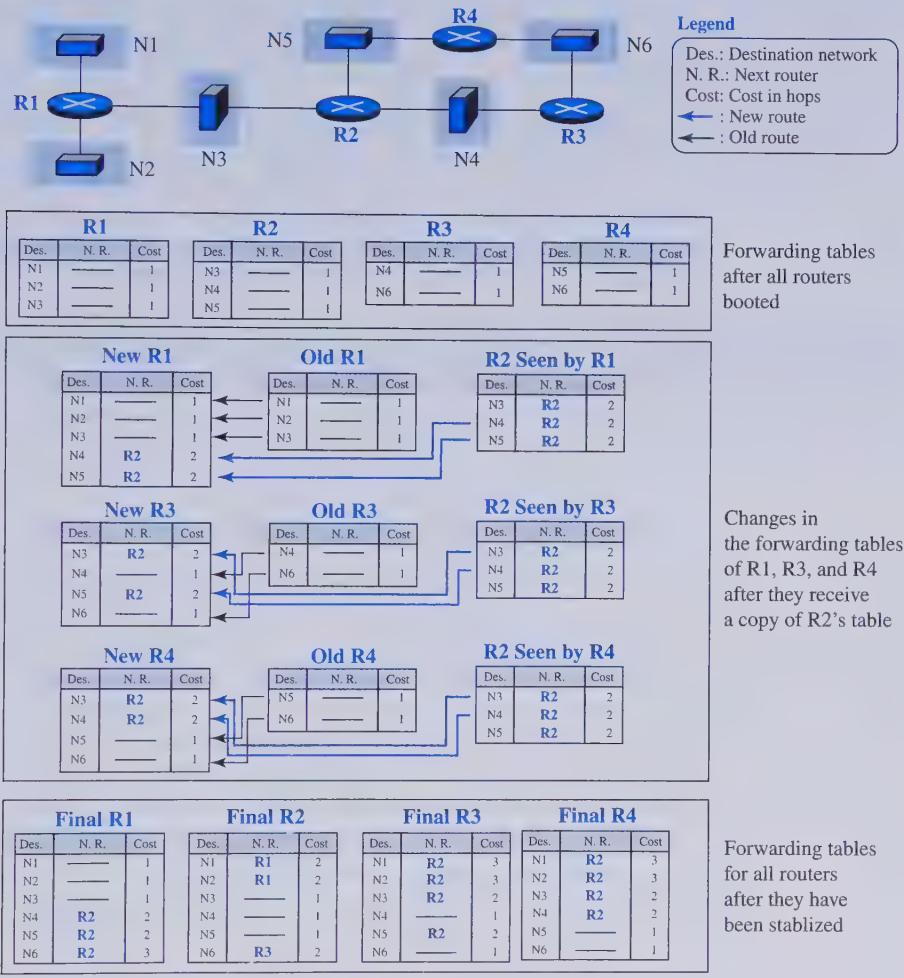
Example 8.1

Figure 8.18 shows a more realistic example of the operation of RIP in an autonomous system. First, the figure shows all forwarding tables after all routers have been booted. Then it shows changes in some tables when some update messages have been exchanged. Finally, it shows the stabilized forwarding tables when there is no more change.

Timers in RIP

RIP uses three timers to support its operation. The *periodic timer* controls the advertising of regular update messages. Each router has one periodic timer that is randomly set to a number between 25 and 35 s (to prevent all routers sending their messages at the same time and creating excess traffic). The timer counts down; when zero is reached, the update message is sent, and the timer is randomly set once again. The expiration timer governs the validity of a route. When a router receives update information for a route, the expiration timer is set to 180 s for that particular route. Every time a new update for the route is received, the timer is reset. If there is a problem on an internet and no update is received within the allotted 180 s, the route is considered expired and the hop count of the route is set to 16, which means the destination is

Figure 8.18 *Example of an autonomous system using RIP*



unreachable. Every route has its own expiration timer. The *garbage collection timer* is used to purge a route from the forwarding table. When the information about a route becomes invalid, the router does not immediately purge that route from its table. Instead, it continues to advertise the route with a metric value of 16. At the same time, a garbage collection timer is set to 120 s for that route. When the count reaches zero, the route is purged from the table. This timer allows neighbors to become aware of the invalidity of a route prior to purging.

Performance

Before ending this section, let us briefly discuss the performance of RIP:

- **Update messages.** The update messages in RIP have a very simple format and are sent only to neighbors; they are local. They do not normally create traffic because the routers try to avoid sending them at the same time.
- **Convergence of forwarding tables.** RIP uses the distance-vector algorithm, which can converge slowly if the domain is large, but, because RIP allows only 15 hops in a domain (16 is considered as infinity), there is normally no problem in convergence. The only problems that may slow down convergence are count-to-infinity and loops created in the domain; use of **poison-reverse** and **split-horizon strategies** added to the RIP extension may alleviate the situation.
- **Robustness.** As we said before, distance-vector routing is based on the concept that each router sends what it knows about the whole domain to its neighbors. This means that the calculation of the forwarding table depends on information received from immediate neighbors, which in turn receive their information from their own neighbors. If there is a failure or corruption in one router, the problem will be propagated to all routers and the forwarding in each router will be affected.

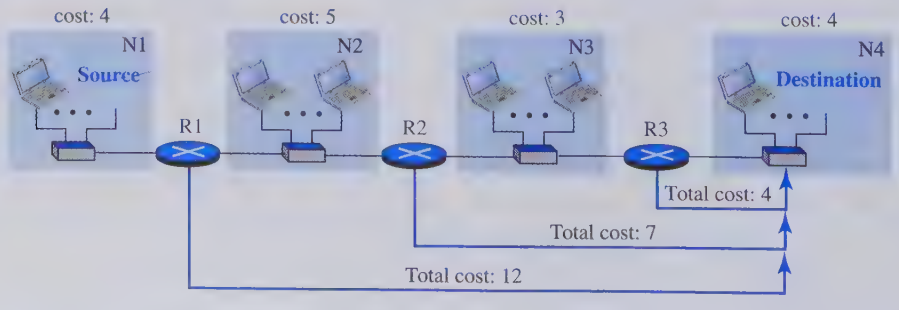
8.3.3 Open Shortest Path First (OSPF)

Open Shortest Path First (OSPF) is also an intradomain routing protocol like RIP, but it is based on the link-state routing protocol we described earlier in Section 8.2.2. OSPF is an *open* protocol, which means that the specification is a public document.

Metric

In OSPF, like RIP, the cost of reaching a destination from the host is calculated from the source router to the destination network. However, each link (network) can be assigned a weight based on the throughput, round-trip time, reliability, and so on. An administration can also decide to use the hop count as the cost. An interesting point about the cost in OSPF is that different service types (TOSs) can have different weights as the cost. Figure 8.19 shows the idea of the cost from a router to the destination host network. We can compare Figure 8.19 with Figure 8.15 for the RIP.

Figure 8.19 *Metric in OSPF*



Forwarding Tables

Each OSPF router can create a forwarding table after finding the shortest-path tree between itself and the destination using Dijkstra's algorithm, described earlier in Section 8.2.2. Figure 8.20 shows the forwarding tables for the simple AS in Figure 8.19. Comparing the forwarding tables for the OSPF and RIP in the same AS, we find that the only difference is the cost values. In other words, if we use the hop count for OSPF, the tables will be exactly the same. The reason for this consistency is that both protocols use the shortest-path trees to define the best route from a source to a destination.

Figure 8.20 *Forwarding tables in OSPF*

Forwarding table for R1

Destination network	Next router	Cost
N1	——	4
N2	——	5
N3	R2	8
N4	R2	12

Forwarding table for R2

Destination network	Next router	Cost
N1	R1	9
N2	——	5
N3	——	3
N4	R3	7

Forwarding table for R3

Destination network	Next router	Cost
N1	R2	12
N2	R2	8
N3	——	3
N4	——	4

Areas

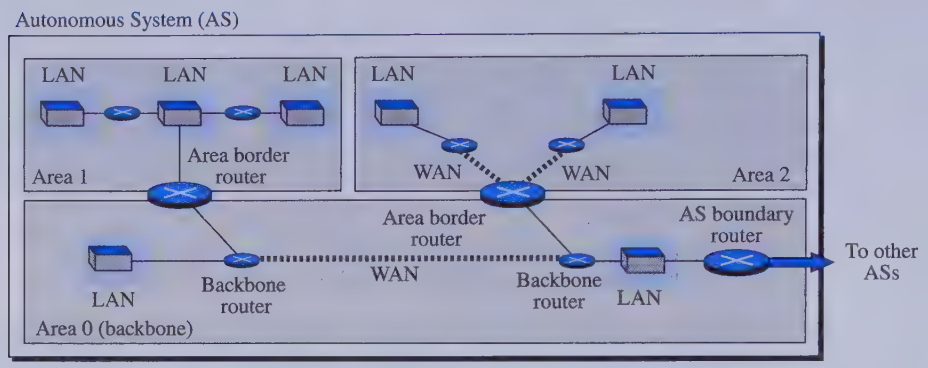
Compared with RIP, which is normally used in small ASs, OSPF was designed to be able to handle routing in a small or large autonomous system. However, the formation of shortest-path trees in OSPF requires that all routers flood the whole AS with their LSPs to create the global LSDB. Although this may not create a problem in a small AS, it may have created a huge volume of traffic in a large AS. To prevent this, the AS needs to be divided into small sections called *areas*. Each area acts as a small independent domain for flooding LSPs. In other words, OSPF uses another level of hierarchy in routing: The first level is the autonomous system, the second is the area.

However, each router in an area needs to know the information about the link states not only in its area but also in other areas. For this reason, one of the areas in the AS is designated as the *backbone area*, responsible for gluing the areas together. The routers in the backbone area are responsible for passing the information collected by each area to all other areas. In this way, a router in an area can receive all LSPs generated in other areas. For the purpose of communication, each area has an area identification. The area identification of the backbone is zero. Figure 8.21 shows an autonomous system and its areas.

Link-State Advertisement

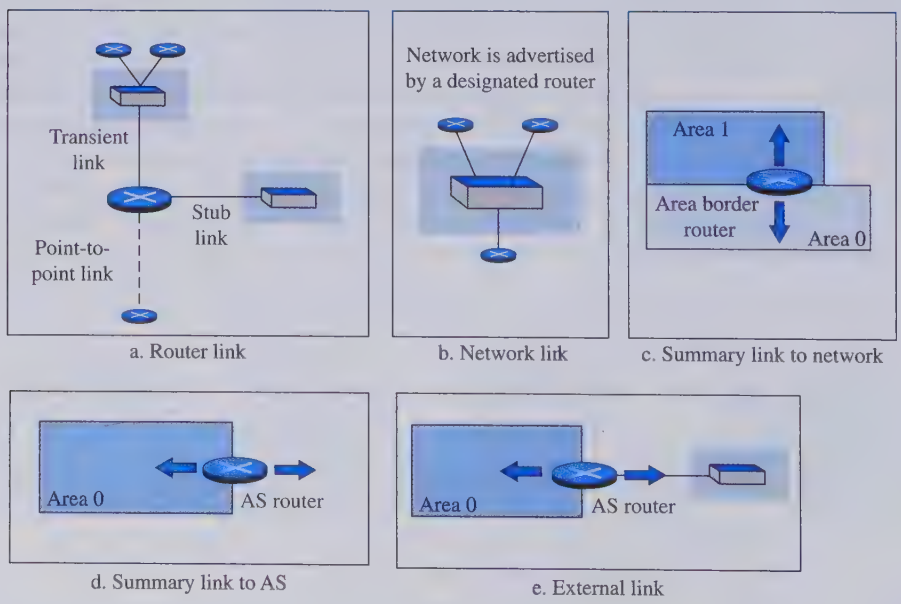
OSPF is based on the link-state routing algorithm, which requires that a router advertise the state of each link to all neighbors for the formation of the LSDB. When we discussed the link-state algorithm, we used the graph theory and assumed that each router is a node and each network between two routers is an edge. The situation is different in the real world, in which we need to advertise the existence of different entities as nodes,

Figure 8.21 *Areas in an autonomous system*



the different types of links that connect each node to its neighbors, and the different types of cost associated with each link. This means we need different types of advertisements, each capable of advertising different situations. We can have five types of link-state advertisements: *router link*, *network link*, *summary link to network*, *summary link to AS border router*, and *external link*. Figure 8.22 shows these five advertisements and their uses.

Figure 8.22 *Five different LSPs*



- **Router link.** A router link advertises the existence of a router as a node. In addition to giving the address of the announcing router, this type of advertisement can define one or more types of links that connect the advertising router to other entities. A *transient link* announces a link to a transient network, a network that is connected to the rest of the networks by one or more routers. This type of advertisement should define the address of the transient network and the cost of the link. A *stub link* advertises a link to a stub network, a network that is not a through network. Again, the advertisement should define the address of the network and the cost. A *point-to-point link* should define the address of the router at the end of the point-to-point line and the cost to get there.
- **Network link.** A network link advertises the network as a node. However, because a network cannot do announcements itself (it is a passive entity), one of the routers is assigned as the designated router and does the advertising. In addition to the address of the designated router, this type of LSP announces the IP address of all routers (including the designated router as a router and not as speaker of the network), but no cost is advertised because each router announces the cost to the network when it sends a router link advertisement.
- **Summary link to network.** This is done by an area border router; it advertises the summary of links collected by the backbone to an area or the summary of links collected by the area to the backbone. This type of information exchange is needed to glue the areas together.
- **Summary link to AS.** This is done by an AS router that advertises the summary links from other ASs to the backbone area of the current AS, information which later can be disseminated to the areas so that they will know about the networks in other ASs. The need for this type of information exchange is better understood when we discuss inter-AS routing (BGP).
- **External link.** This is also done by an AS router to announce the existence of a single network outside the AS to the backbone area to be disseminated into the areas.

OSPF Implementation

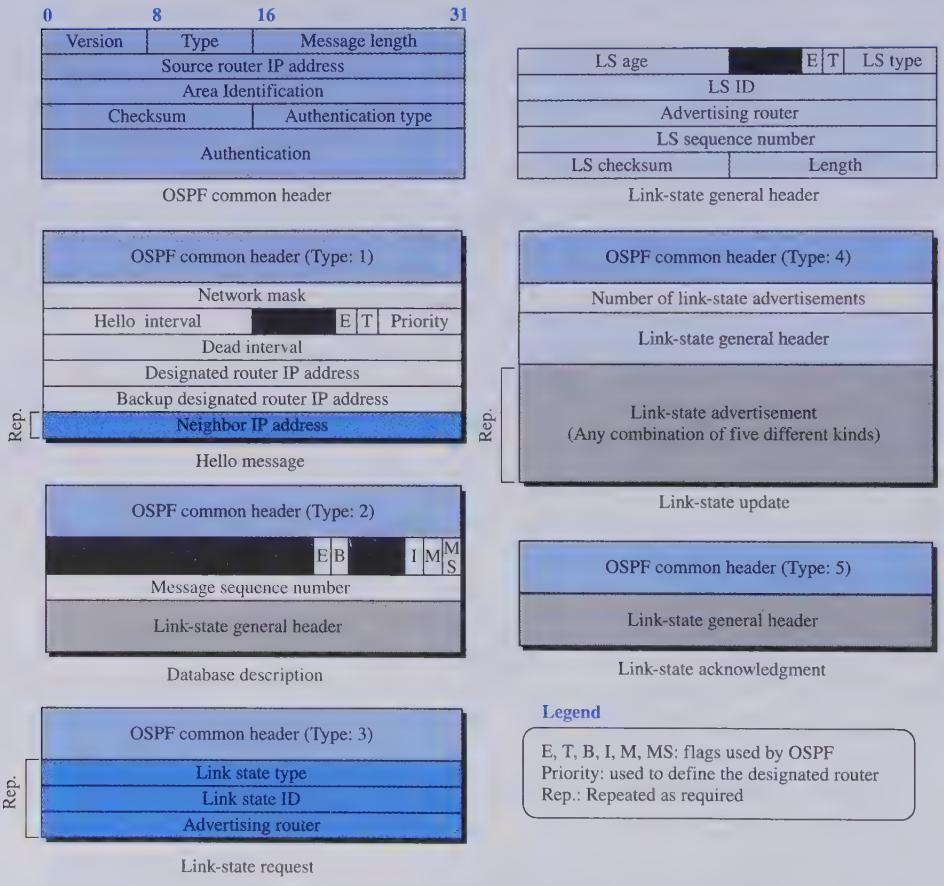
OSPF is implemented as a program in the network layer that uses the service of the IP for propagation. An IP datagram that carries a message from OSPF sets the value of the protocol field to 89. This means that, although OSPF is a routing protocol to help IP to route its datagrams inside an AS, the OSPF messages are encapsulated inside datagrams. OSPF has gone through two versions: version 1 and version 2. Most implementations use version 2.

OSPF Messages

OSPF is a very complex protocol; it uses five different types of messages. In Figure 8.23, we first show the format of the OSPF common header (which is used in all messages) and the link-state general header (which is used in some messages). We then give the outlines of five message types used in OSPF. The *hello* message (type 1) is used by a router to introduce itself to the neighbors and announces all neighbors that it already knows. The *database description* message (type 2) is normally sent in response to the

hello message to allow a newly joined router to acquire the full LSDB. The *link-state request* message (type 3) is sent by a router that needs information about a specific LS. The *link-state update message* (type 4) is the main OSPF message used for building the LSDB. This message, in fact, has five different versions (router link, network link, summary link to network, summary link to AS border router, and external link), as we discussed in Section 8.2.2. The *link-state acknowledgment* message (type 5) is used to create reliability in OSPF; each router that receives a link-state update message needs to acknowledge it.

Figure 8.23 *OSPF message formats*



Authentication

As Figure 8.23 shows, the OSPF common header has the provision for authentication of the message sender. As we will discuss in Chapter 13, this prevents a malicious entity from sending OSPF messages to a router and causing the router to become part of the routing system to which it actually does not belong.

OSPF Algorithm

OSPF implements the link-state routing algorithm we discussed in Section 8.2.2. However, some changes and augmentations need to be added to the algorithm:

- After each router has created the shortest-path tree, the algorithm needs to use it to create the corresponding routing algorithm.
- The algorithm needs to be augmented to handle sending and receiving all five types of messages.

Performance

Before ending this section, let us briefly discuss the performance of OSPF:

- **Update messages.** The link-state messages in OSPF have a somewhat complex format. They also are flooded to the whole area. If the area is large, these messages may create heavy traffic and use a lot of bandwidth.
- **Convergence of forwarding tables.** When the flooding of LSPs is completed, each router can create its own shortest-path tree and forwarding table; convergence is fairly quick. However, each router needs to run the Dijkstra's algorithm, which may take some time.
- **Robustness.** The OSPF protocol is more robust than RIP because, after receiving the completed LSDB, each router is independent and does not depend on other routers in the area. Corruption or failure in one router does not affect other routers as seriously as in RIP.

8.3.4 Border Gateway Protocol Version 4 (BGP4)

The Border Gateway Protocol version 4 (BGP4) is the only interdomain routing protocol used in the Internet today. BGP4 is based on the path-vector algorithm we described before, but it is tailored to provide information about the reachability of networks in the Internet.

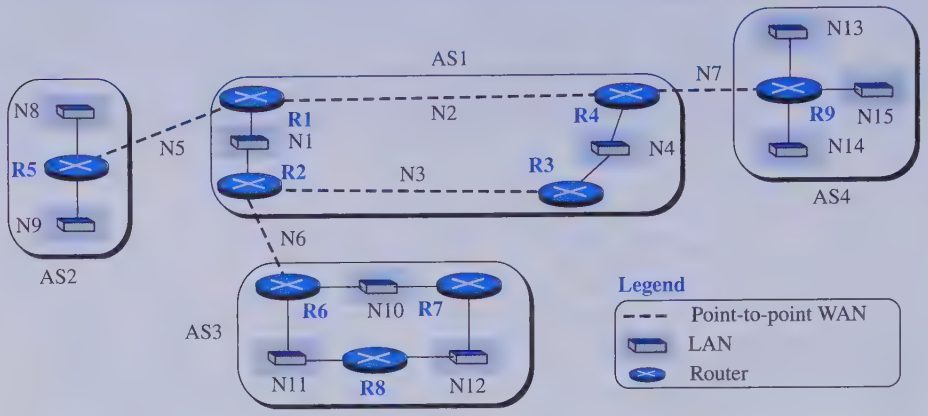
Introduction

Border Gateway Protocol (BGP), and in particular BGP4, is a complex protocol. In this section, we introduce the basics of BGP and its relationship with intradomain routing protocols (RIP or OSPF). Figure 8.24 shows an example of an internet with four autonomous systems. AS2, AS3, and AS4 are *stub* autonomous systems; AS1 is a *transient* one. In our example, data exchange between AS2, AS3, and AS4 should pass through AS1

Each autonomous system in this figure uses one of the two common intradomain protocols, RIP or OSPF. Each router in each AS knows how to reach a network that is in its own AS, but it does not know how to reach a network in another AS.

To enable each router to route a packet to any network in the internet, we first install a variation of BGP4, called *external BGP* (*eBGP*), on each *border router* (the one at the edge of each AS that is connected to a router at another AS). We then install the second variation of BGP, called *internal BGP* (*iBGP*), on all routers. This means that the border routers will be running three routing protocols (intradomain, eBGP, and iBGP), but

Figure 8.24 *A sample internet with four ASs*



other routers are running two protocols (intradomain and iBGP). We discuss the effect of each BGP variation separately.

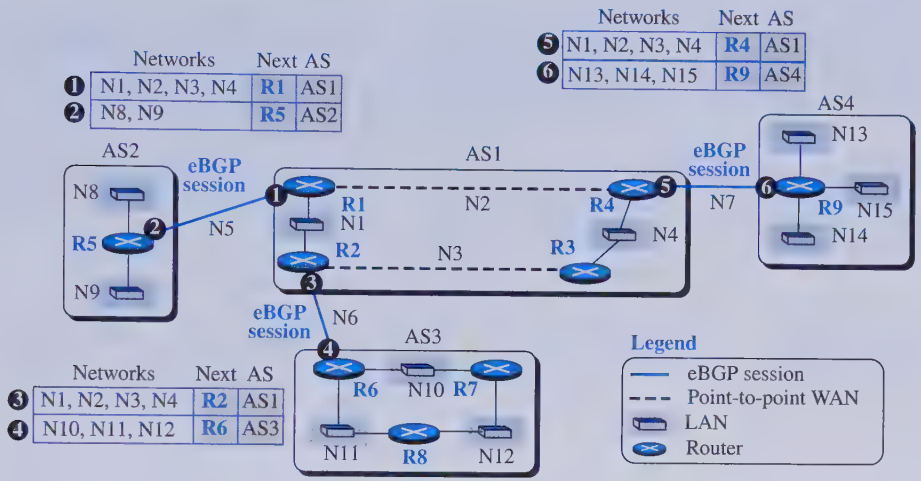
Operation of External BGP (eBGP)

We can say that the GP protocol is a kind of point-to-point protocol. When the software is installed on two routers, they try to create a TCP connection using the well-known port 179. In other words, a pair of client and server processes continuously communicate with each other to exchange messages. The two routers that run the BGP processes are called *BGP peers* or *BGP speakers*. We discuss different types of messages exchanged between two peers, but for the moment we are interested in only the update messages that announce reachability of networks in each AS.

The eBGP variation of BGP allows two physically connected border routers in two different ASs to form pairs of eBGP speakers and exchange messages. The routers that are eligible in our example in Figure 8.24 form three pairs: R1-R5, R2-R6, and R4-R9. The connection between these pairs is established over three physical WANs (N5, N6, and N7). However, there is a need for a logical TCP connection to be created over the physical connection to make the exchange of information possible. Each logical connection in BGP parlance is referred to as a *session*. This means that we need three sessions in our example, as shown in Figure 8.25.

Figure 8.25 also shows the simplified update messages sent by routers involved in the eBGP sessions. The circled number defines the sending router in each case. For example, message number 1 is sent by router R1 and tells router R5 that N1, N2, N3, and N4 can be reached through router R1. (R1 gets this information from the corresponding intradomain forwarding table.) Router R5 can now add these pieces of information at the end of its forwarding table. When R5 receives any packet destined for these four networks, it can use its forwarding table and find that the next router is R1.

Figure 8.25 *eBGP operation*



The reader may have noticed that the messages exchanged during three eBGP sessions help some routers know how to route packets to some networks in the internet, but the reachability information is not complete. There are two problems that need to be addressed:

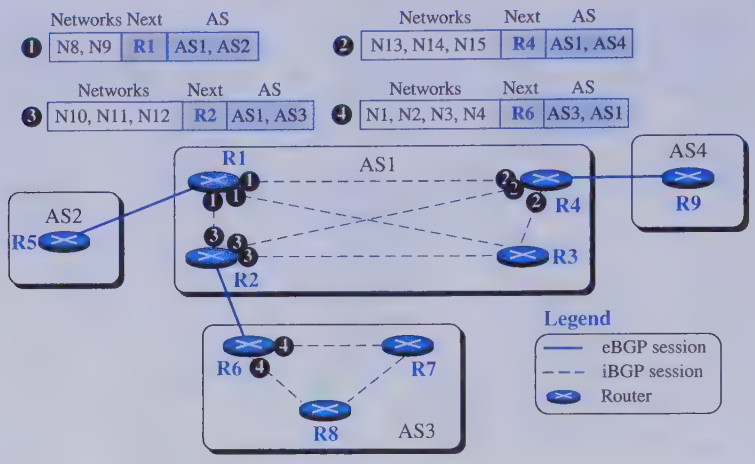
1. Some border routers do not know how to route a packet destined for non-neighbor ASs. For example, R5 does not know how to route packets destined for networks in AS3 and AS4. Routers R6 and R9 are in the same situation as R5: R6 does not know about networks in AS4, and R9 does not know about networks in AS3.
2. None of the nonborder routers know how to route a packet destined for any networks in other ASs.

To address these two problems, we need to allow all pairs of routers (border or nonborder) to run the second variation of the BGP protocol, iBGP.

Operation of Internal BGP (iBGP)

The iBGP protocol is similar to the eBGP protocol in that it uses the service of TCP on the well-known port 179, but it creates a session between any possible pair of routers inside an autonomous system. However, some points should be made clear. First, if an AS has only one router, there cannot be an iBGP session. For example, we cannot create an iBGP session inside AS2 or AS4 in our internet. Second, if there are n routers in an autonomous system, there should be $[n \times (n - 1) / 2]$ iBGP sessions in that autonomous system (a fully connected mesh) to prevent loops in the system. In other words, each router needs to advertise its own reachability to the peer in the session instead of flooding what it receives from another peer in another session. Figure 8.26 shows the combination of eBGP and iBGP sessions in our internet.

Figure 8.26 *Combination of eBGP and iBGP sessions in our internet*



Note that we have not shown the physical networks inside ASs because a session is made on an overlay network (TCP connection), possibly spanning more than one physical network as determined by the route dictated by the intradomain routing protocol. Also note that in this stage only four messages are exchanged. The first message (numbered 1) is sent by R1 announcing that networks N8 and N9 are reachable through the path AS1-AS2, but the next router is R1. This message is sent, through separate sessions, to R2, R3, and R4. Routers R2, R4, and R6 do the same thing but send different messages to different destinations. The interesting point is that, at this stage, R3, R7, and R8 create sessions with their peers, but they actually have no message to send.

The updating process does not stop here. For example, after R1 receives the update message from R2, it combines the reachability information about AS3 with the reachability information it already knows about AS1 and sends a new update message to R5. Now R5 knows how to reach networks in AS1 and AS3. The process continues when R1 receives the update message from R4. The point is that we need to make certain that at a point in time there are no changes in the previous updates and that all information is propagated through all ASs. At this time, each router combines the information received from eBGP and iBGP and creates what we may call a path table after applying the criteria for finding the best path. To demonstrate, we show the path tables in Figure 8.27 for the routers in Figure 8.24. For example, router R1 now knows that any packet destined for networks N8 or N9 should go through AS1 and AS2 and the next router to deliver the packet to is router R5. Similarly, router R4 knows that any packet destined for networks N10, N11, or N12 should go through AS1 and AS3 and the next router to deliver this packet to is router R1, and so on.

Figure 8.27 *Finalized BGP path tables*

Path table for R1

Networks	Next	Path
N8, N9	R5	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R2

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R6	AS1, AS3
N13, N14, N15	R1	AS1, AS4

Path table for R3

Networks	Next	Path
N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4

Path table for R4

Networks	Next	Path
N8, N9	R1	AS1, AS2
N10, N11, N12	R1	AS1, AS3
N13, N14, N15	R9	AS1, AS4

Path table for R5

Networks	Next	Path
N1, N2, N3, N4	R1	AS2, AS1
N10, N11, N12	R1	AS2, AS1, AS3
N13, N14, N15	R1	AS2, AS1, AS4

Path table for R6

Networks	Next	Path
N1, N2, N3, N4	R2	AS3, AS1
N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R2	AS3, AS1, AS4

Path table for R7

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R8

Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1
N8, N9	R6	AS3, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4

Path table for R9

Networks	Next	Path
N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R4	AS4, AS1, AS2
N10, N11, N12	R4	AS4, AS1, AS3

Injection of Information into Intradomain Routing

The role of an interdomain routing protocol such as BGP is to help the routers inside the AS to augment their routing information. In other words, the path tables collected and organized by BPG are not used, per se, for routing packets; they are injected into intradomain forwarding tables (RIP or OSPF) for routing packets. This can be done in several ways depending on the type of AS.

In the case of a stub AS, the only area border router adds a default entry at the end of its forwarding table and defines the next router to be the speaker router at the end of the eBGP connection. In Figure 8.24, router R5 in AS2 defines R1 as the default router for all networks other than N8 and N9. The situation is the same for router R9 in AS4 with the default router to be R4. In AS3, R6 set its default router to be R2, but R7 and R8 set their default routers to be R6. These settings are in accordance with the path tables we describe in Figure 8.27 for these routers. In other words, the path tables are injected into intradomain forwarding tables by adding only one default entry.

In the case of a transient AS, the situation is more complicated. Router R1 in AS1 needs to inject the whole contents of the path table for R1 in Figure 8.27 into its intradomain forwarding table. The situation is the same for R2, R3, and R4.

One issue to be resolved is the cost value. We know that RIP and OSPF use different metrics. One solution, which is very common, is to set the cost to the foreign networks at the same cost value as to reach the first AS in the path. For example, the cost for R5 to reach all networks in other ASs is the cost to reach N5. The cost for R1 to reach networks N10 to N12 is the cost to reach N6, and so on. The cost is taken from the intradomain forwarding tables (RIP or OSPF).

Figure 8.28 shows the interdomain forwarding tables. For simplicity, we assume that all ASs are using RIP as the intradomain routing protocol. The shaded areas are the augmentation injected by the BGP protocol; the default destinations are indicated as zero.

Figure 8.28 *Forwarding tables after injection from BGP*

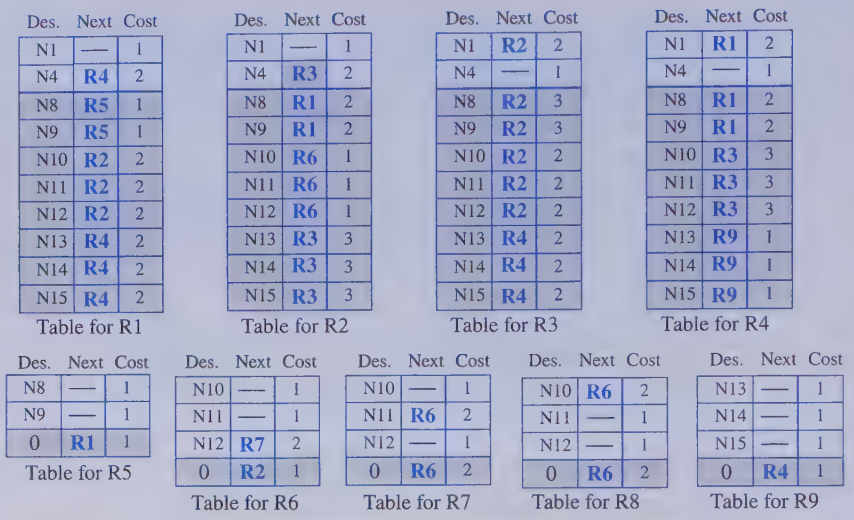


Table for R1

Des.	Next	Cost
N1	—	1
N4	R4	2
N8	R5	1
N9	R5	1
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R2

Des.	Next	Cost
N1	—	1
N4	R3	2
N8	R1	2
N9	R1	2
N10	R6	1
N11	R6	1
N12	R6	1
N13	R3	3
N14	R3	3
N15	R3	3

Table for R3

Des.	Next	Cost
N1	R2	2
N4	—	1
N8	R2	3
N9	R2	3
N10	R2	2
N11	R2	2
N12	R2	2
N13	R4	2
N14	R4	2
N15	R4	2

Table for R4

Des.	Next	Cost
N1	R1	2
N4	—	1
N8	R1	2
N9	R1	2
N10	R3	3
N11	R3	3
N12	R3	3
N13	R9	1
N14	R9	1
N15	R9	1

Table for R5

Des.	Next	Cost
N8	—	1
N9	—	1
0	R1	1

Table for R6

Des.	Next	Cost
N10	—	1
N11	—	1
N12	R7	2
0	R2	1

Table for R7

Des.	Next	Cost
N10	—	1
N11	R6	2
N12	—	1
0	R6	2

Table for R8

Des.	Next	Cost
N10	R6	2
N11	—	1
N12	—	1
0	R6	2

Table for R9

Des.	Next	Cost
N13	—	1
N14	—	1
N15	—	1
0	R4	1

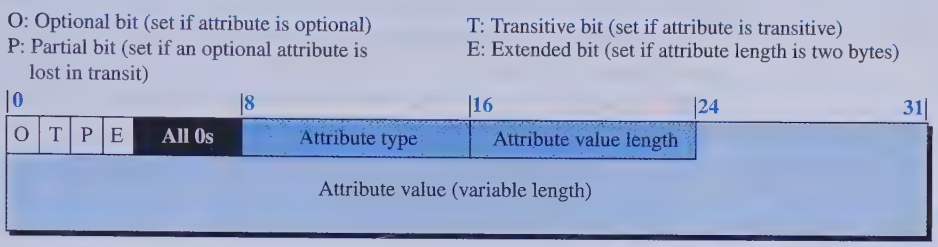
Address Aggregation

The reader may have realized that intradomain forwarding tables obtained with the help of the BGP4 protocols may become huge in the case of the global Internet because many destination networks may be included in a forwarding table. Fortunately, BGP4 uses the prefixes as destination identifiers and allows the aggregation of these prefixes. For example, prefixes 14.18.20.0/26, 14.18.20.64/26, 14.18.20.128/26, and 14.18.20.192/26 can be combined into 14.18.20.0/24 if all four subnets can be reached through one path. Even if one or two of the aggregated prefixes need a separate path, the longest prefix principle.

Path Attributes

In both intradomain routing protocols (RIP or OSPF), a destination is normally associated with two pieces of information: next hop and cost. The first one shows the address of the next router to deliver the packet; the second defines the cost to the final destination. Interdomain routing is more involved and naturally needs more information about how to reach the final destination. In BGP these pieces are called **path attributes**. BGP allows a destination to be associated with up to seven path attributes. Path attributes are divided into two broad categories: *well-known* and *optional*. A well-known attribute must be recognized by all routers; an optional attribute does not. A well-known attribute can be mandatory, which means that it must be present in any BGP update message, or discretionary, which means it does not have to be. An optional attribute can be either transitive, which means it can pass to the next AS, or intransitive, which means it cannot. All attributes are inserted after the corresponding destination prefix in an update message. The format for an attribute is shown in Figure 8.29.

Figure 8.29 *Format of a path attribute*



The first byte in each attribute defines the four attribute flags (as shown in Figure 8.29). The next byte defines the type of attributes assigned by ICANN (only seven types have been assigned, as explained next). The attribute value length defines the length of the attribute value field (not the length of the whole attributes section). The following gives a brief description of each attribute.

- **ORIGIN (type 1).** This is a well-known mandatory attribute, which defines the source of the routing information. This attribute can be defined by one of the three values: 1, 2, and 3. Value 1 means that the information about the path has been taken from an intradomain protocol (RIP or OSPF). Value 2 means that the information comes from BGP. Value 3 means that it comes from an unknown source.
- **AS-PATH (type 2).** This is a well-known mandatory attribute, which defines the list of autonomous systems through which the destination can be reached. We have used this attribute in our examples. The AS-PATH attribute, as we discussed in path-vector routing in Section 8.2.3, helps prevent a loop. Whenever an update message arrives at a router that lists the current AS as the path, the router drops that path. The AS-PATH can also be used in route selection.
- **NEXT-HOP (type 3).** This is a well-known mandatory attribute, which defines the next router to which the data packet should be forwarded. We have also used this attribute in our examples. As we have seen, this attribute helps to inject path information collected through the operations of eBGP and iBGP into the intradomain routing protocols such as RIP or OSPF.
- **MULT-EXIT-DISC (type 4).** The multiple-exit discriminator is an optional nontransitive attribute, which discriminates among multiple exit paths to a destination. The value of this attribute is normally defined by the metric in the corresponding intradomain protocol (an attribute value of a 4-byte unsigned integer). For example, if a router has multiple paths to the destination with different values related to these attributes, the one with the lowest value is selected. Note that this attribute is nontransitive, which means that it is not propagated from one AS to another.
- **LOCAL-PREF (type 5).** The local preference attribute is a well-known discretionary attribute. It is normally set by the administrator, based on an organization's policy. The routes the administrator prefers are given a higher local preference value

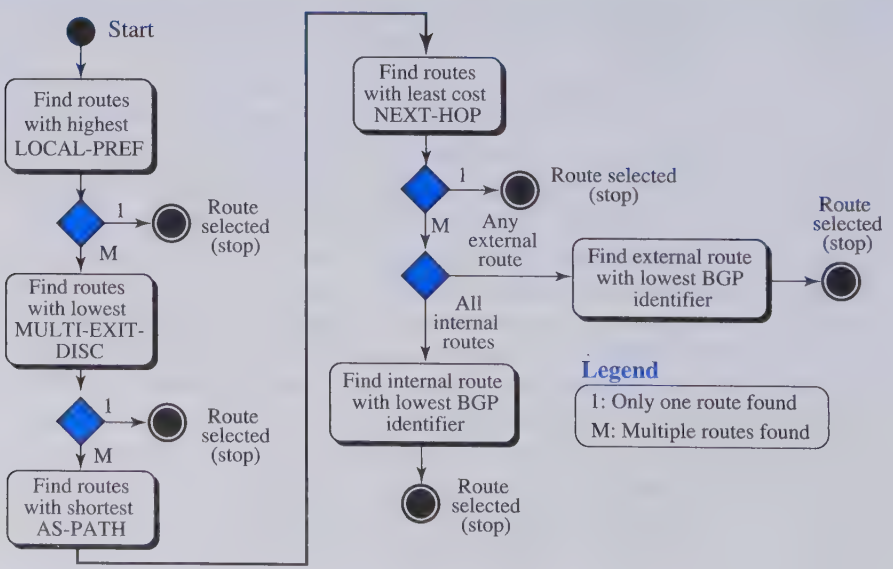
(an attribute value of a 4-byte unsigned integer). For example, in an internet with five ASs, the administrator of AS1 can set the local preference value of 400 to the path AS1-AS2-AS5, the value of 300 to AS1-AS3-AS5, and the value of 50 to AS1-AS4-AS5. This means that the administrator prefers the first path to the second one and prefers the second one to the third one. This may be a case where AS2 is the most secured and AS4 is the least secured AS for the administration of AS1. The last route should be selected if the other two are not available.

- **ATOMIC-AGGREGATE (type 6).** This is a well-known discretionary attribute, which defines the destination prefix as not aggregate; it only defines a single destination network. This attribute has no value field, which means the value of the length field is zero.
- **AGGREGATOR (type 7).** This is an optional transitive attribute, which emphasizes that the destination prefix is an aggregate. The attribute value gives the number of the last AS that did the aggregation followed by the IP address of the router that did so.

Route Selection

So far in this section, we have been silent about how a route is selected by a BGP router mostly because our simple example has one route to a destination. In the case where multiple routes are received to a destination, BGP needs to select one among them. The route selection process in BGP is not as easy as the ones in the intradomain routing protocol that is based on the shortest-path tree. A route in BGP has some attributes attached to it, and it may come from an eBGP session or an iBGP session. Figure 8.30 shows the flow diagram used by common implementations.

Figure 8.30 *Flow diagram for route selection*

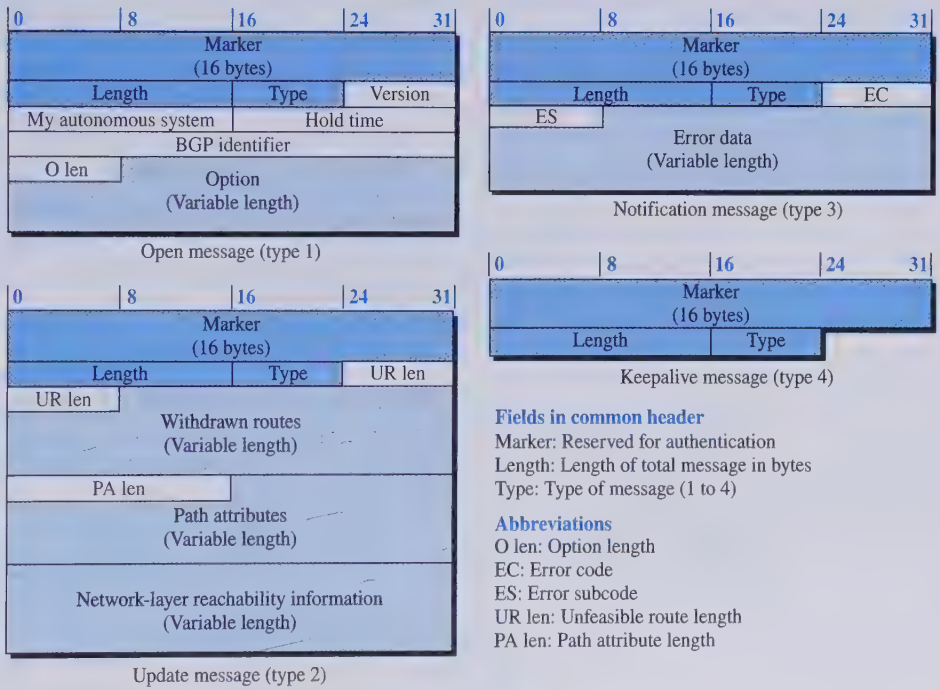


The router extracts the routes that meet the criteria in each step. If only one route is extracted, it is selected and the process stops; otherwise, the process continues with the next step. Note that the first choice is related to the LOCAL-PREF attribute, which reflects the policy imposed by the administration on the route.

Messages

BGP uses four types of messages for communication between the BGP speakers across the ASs and inside an AS: *open*, *update*, *keepalive*, and *notification* (see Figure 8.31). All BGP packets share the same common header.

Figure 8.31 *BGP messages*



- **Open message.** To create a neighborhood relationship, a router running BGP opens a TCP connection with a neighbor and sends an *open message*.
- **Update message.** The *update message* is the heart of the BGP protocol. It is used by a router to withdraw destinations that have been advertised previously, to announce a route to a new destination, or both. Note that BGP can withdraw several destinations that were advertised before, but it can only advertise one new destination (or multiple destinations with the same path attributes) in a single update message.

- **Keepalive message.** The BGP peers that are running exchange keep-alive messages regularly (before their hold time expires) to tell each other that they are alive.
- **Notification.** A notification message is sent by a router whenever an error condition is detected or a router wants to close the session.

Performance

BGP performance can be compared with RIP. BGP speakers exchange a lot of messages to create forwarding tables, but BGP is free from loops and count-to-infinity. The same weakness we mention for RIP about propagation of failure and corruptness also exists in BGP.

8.4 MULTICAST ROUTING

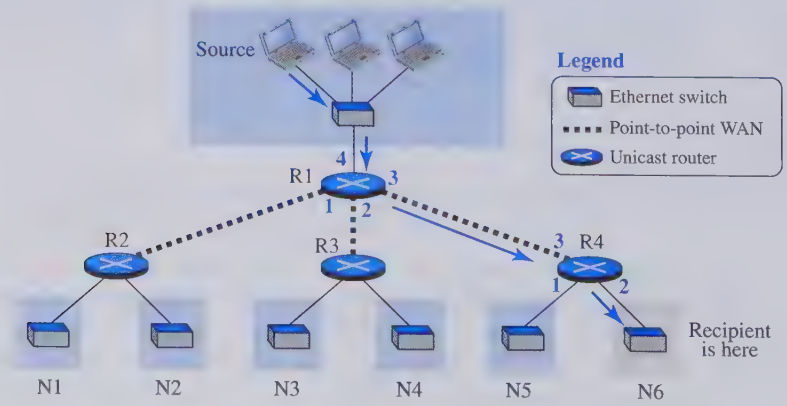
Communication in the Internet today is not only unicasting; multicasting communication is growing fast. In this section, we first discuss the general ideas behind unicasting, multicasting, and broadcasting. We then talk about some basic issues in multicast routing. Finally, we discuss multicasting routing protocols in the Internet.

From the previous chapters, we have learned that forwarding a datagram by a router is normally based on the prefix of the destination address in the datagram, which defines the network to which the destination host is connected. Understanding the above forwarding principle, we can now define unicasting, multicasting, and broadcasting. Let us clarify these terms as they relate to the Internet.

8.4.1 Unicasting

In unicasting, there is one source and one destination network. The relationship between the source and the destination network is one to one. Each router in the path of the datagram tries to forward the packet to one and only one of its interfaces. Figure 8.32 shows

Figure 8.32 *Unicasting*

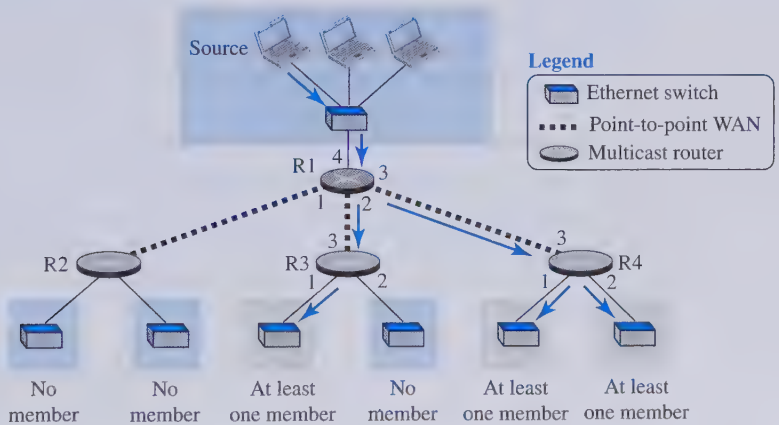


a small internet in which a unicast packet needs to be delivered from a source computer to a destination computer attached to N6. Router R1 is responsible for forwarding the packet only through interface 3; router R4 is responsible for forwarding the packet only through interface 2. When the packet arrives at N6, the delivery to the destination host is the responsibility of the network; either it is broadcast to all hosts, or the Ethernet switch delivers it only to the destination host.

8.4.2 Multicasting

In multicasting, there is one source and a group of destinations. The relationship is one to many. In this type of communication, the source address is a unicast address, but the destination address is a group address, a group of one or more destination networks in which there is at least one member of the group that is interested in receiving the multicast datagram. The group address defines the members of the group. Figure 8.33 shows the same small internet as in Figure 8.32, but the routers have been changed to multicast routers (or previous routers have been configured to do both types of jobs).

Figure 8.33 *Multicasting*



In multicasting, a multicast router may have to send out copies of the same datagram through more than one interface. In Figure 8.33, router R1 needs to send out the datagram through interfaces 2 and 3. Similarly, router R4 needs to send out the datagram through both its interfaces. Router R3, however, knows that there is no member belonging to this group in the area reached by interface 2; it only sends out the datagram through interface 1.

Multicast Applications

Multicasting has many applications today, such as access to distributed databases, information dissemination, teleconferencing, and distance learning.

- **Access to distributed databases.** Most of the large databases today are distributed. That is, the information is stored in more than one location, usually at the time of production. The user who needs to access the database does not know the location of the information. A user's request is multicast to all the database locations, and the location that has the information responds.
- **Information dissemination.** Businesses often need to send information to their customers. If the nature of the information is the same for each customer, it can be multicast. In this way, a business can send one message that can reach many customers. For example, a software update can be sent to all purchasers of a particular software package. In a similar manner, news can be easily disseminated through multicasting.
- **Teleconferencing.** Teleconferencing involves multicasting. The individuals attending a teleconference all need to receive the same information at the same time. Temporary or permanent groups can be formed for this purpose.
- **Distance learning.** One growing area in the use of multicasting is distance learning. Lessons taught by one professor can be received by a specific group of students. This is especially convenient for those students who find it difficult to attend classes on campus.

Multicast Addresses

We discussed multicast addresses for IPv4 and IPv6 in Chapter 7.

8.4.3 Distance Vector Multicast Routing Protocol

The **Distance Vector Multicast Routing Protocol** (**DVMRP**) is the extension of the Routing Information Protocol (RIP) that is used in unicast routing. It uses the source-based tree approach to multicasting. It is worth mentioning that each router in this protocol that receives a multicast packet to be forwarded implicitly creates a source-based multicast tree in three steps:

1. The router uses an algorithm called *reverse path forwarding* (RPF) to simulate creation of part of the optimal source-based tree between the source and itself.
2. The router uses an algorithm called *reverse path broadcasting* (RPB) to create a broadcast (spanning) tree whose root is the router itself and whose leaves are all networks in the internet.
3. The router uses an algorithm called *reverse path multicasting* (RPM) to create a multicast tree by cutting some branches of the tree that end in networks with no member in the group.

Reverse Path Forwarding (RPF)

The first algorithm, **reverse path forwarding** (**RPF**), forces the router to forward a multicast packet from one specific interface: the one which has come through the shortest path from the source to the router. How can a router know which interface is in this path if the router does not have a shortest-path tree rooted at the source? The router uses the first property of the shortest-path tree we discussed in unicast routing, which says

that the shortest path from A to B is also the shortest path from B to A. The router does not know the shortest path from the source to itself, but it can find which is the next router in the shortest path from itself to the source (reverse path). The router simply consults its unicast forwarding table, pretending that it wants to send a packet to the source; the forwarding table gives the next router and the interface the message that the packet should be sent out from in this reverse direction. The router uses this information to accept a multicast packet only if it arrives from this interface. This is needed to prevent looping. In multicasting, a packet may arrive at the same router that has forwarded it. If the router does not drop all arrived packets except the one, multiple copies of the packet will be circulating in the internet. Of course, the router may add a tag to the packet when it arrives the first time and discard packets that arrive with the same tag, but the RPF strategy is simpler.

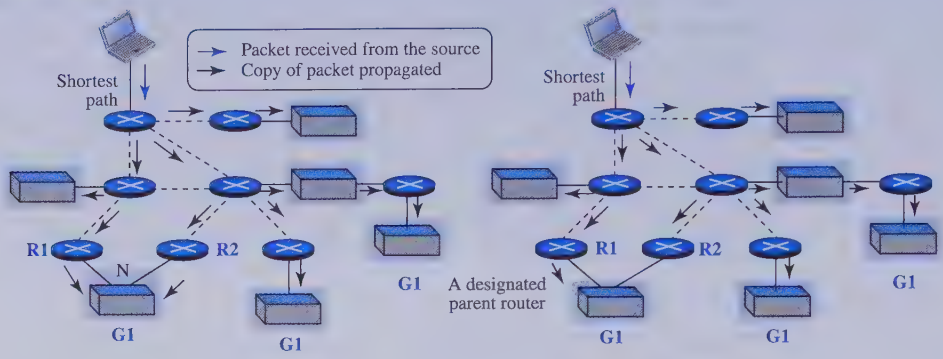
Reverse Path Broadcasting (RPB)

The RPF algorithm helps a router to forward only one copy received from a source and drop the rest. However, when we think about broadcasting in the second step, we need to remember that destinations are all the networks (LANs) in the internet. To be efficient, we need to prevent each network from receiving more than one copy of the packet. If a network is connected to more than one router, it may receive a copy of the packet from each router. RPF cannot help here, because a network does not have the intelligence to apply the RPF algorithm; we need to allow only one of the routers attached to a network to pass the packet to the network. One way to do so is to designate only one router as the *parent* of a network related to a specific source. When a router that is not the parent of the attached network receives a multicast packet, it simply drops the packet. There are several ways that the parent of the network related to a network can be selected; one way is to select the router that has the shortest path to the source (using the unicast forwarding table, again in the reverse direction). If there is a tie: in this case, the router with the smaller IP address can be selected. The reader may have noticed that RPB actually creates a broadcast tree from the graph that has been created by the RPF algorithm. RPB has cut those branches of the tree that cause cycles in the graph. If we use the shortest path criteria for choosing the parent router, we have actually created a shortest-path broadcast tree. In other words, after this step, we have a shortest-path tree with the source as the root and all networks (LANs) as the leaves. Every packet started from the source reaches all LANs in the internet traveling the shortest path. Figure 8.34 shows how RPB can avoid duplicate reception in a network by assigning a designated parent router, R1, for network N.

Reverse Path Multicasting (RPM)

As you may have noticed, RPB does not multicast the packet; it broadcasts it. This is not efficient. To increase efficiency, the multicast packet must reach only those networks that have active members for that particular group. This is called **reverse path multicasting (RPM)**. To change the broadcast shortest-path tree to a multicast shortest-path tree, each router needs to prune (make inactive) the interfaces that do not reach a network with active members corresponding to a particular source-group combination. This step can be done bottom-up, from the leaves to the root. At the

Figure 8.34 *RPF versus RPB*

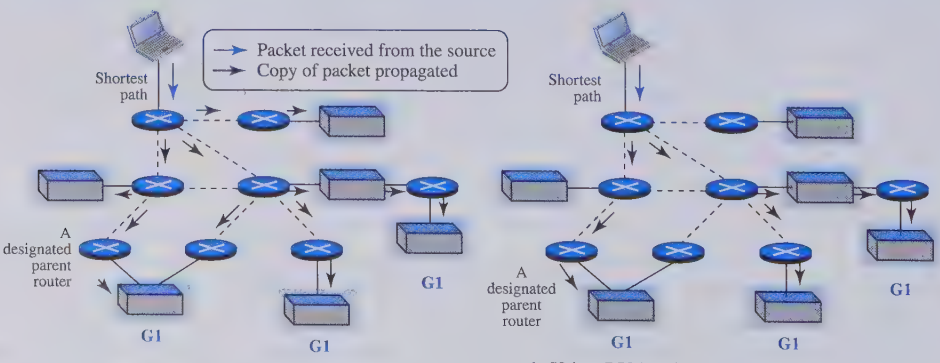


a. Using RPF, N receives two copies.

b. Using RPB, N receives only one copy.

leaf level, the routers connected to the network collect the membership information using the Internet Group Management Protocol (IGMP). The parent router of the network can then disseminate this information upward using the reverse shortest-path tree from the router to the source, the same way as the distance-vector messages are passed from one neighbor to another. When a router receives all these membership-related messages, it knows which interfaces need to be pruned. Of course, because these packets are disseminated periodically, if a new member is added to some networks, all routers are informed and can change the status of their interfaces accordingly. Joining and leaving is continuously applied. Figure 8.35 shows how pruning in RPM lets only networks with group members receive a copy of the packet unless they are in the path to a network with a member.

Figure 8.35 *RPB versus RPM*



a. Using RPB, all networks receive a copy.

b. Using RPM, only members receive a copy.

8.4.4 Multicast Open Shortest Path First

Multicast Open Shortest Path First (MOSPF) is the extension of the Open Shortest Path First (OSPF) protocol, which is used in unicast routing. It also uses the source-based tree approach to multicasting. If the internet is running a unicast link-state routing algorithm, the idea can be extended to provide a multicast link-state routing algorithm. Recall that in unicast link-state routing, each router in the internet has a link-state database (LSDB) that can be used to create a shortest-path tree. To extend unicasting to multicasting, each router needs to have another database, as with the case of unicast distance-vector routing, to show which interface has an active member in a particular group. Now a router goes through the following steps to forward a multicast packet received from source S and to be sent to destination G (a group of recipients):

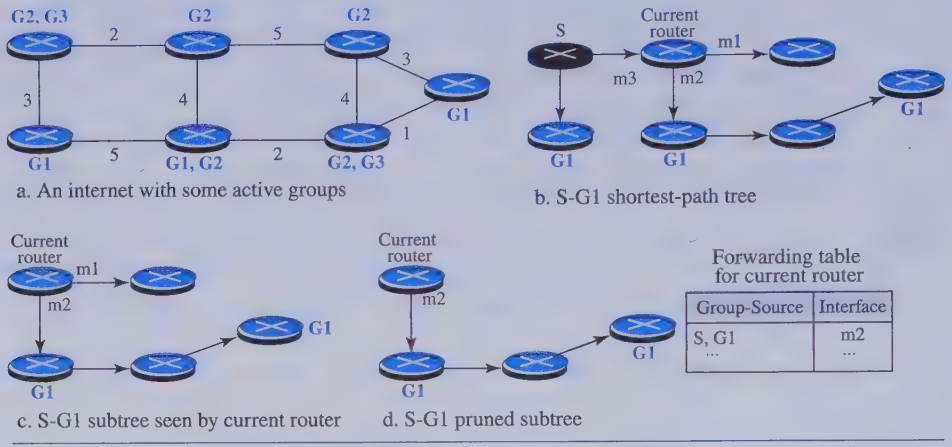
1. The router uses Dijkstra's algorithm to create a shortest-path tree with S as the root and all destinations in the internet as the leaves. Note that this shortest-path tree is different from the one the router normally uses for unicast forwarding, in which the root of the tree is the router itself. In this case, the root of the tree is the source of the packet defined in the source address of the packet. The router is capable of creating this tree because it has the LSDB, the whole topology of the internet; Dijkstra's algorithm can be used to create a tree with any root, no matter which router is using it. The point we need to remember is that the shortest-path tree created this way depends on the specific source. For each source, we need to create a different tree.
2. The router finds itself in the shortest-path tree created in the first step. In other words, the router creates a shortest-path subtree with itself as the root of the subtree.
3. The shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves. The router now uses a strategy similar to the one we describe in the case of DVMRP to prune the broadcast tree and to change it to a multicast tree. The IGMP is used to find the information at the leaf level. MOSPF has added a new type of link-state update packet that floods the membership to all routers. The router can use the information it receives in this way and prune the broadcast tree to make the multicast tree.
4. The router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree. We need to make certain that a copy of the multicast packet reaches all networks that have active members of the group and that it does not reach those networks that do not.

Figure 8.36 shows an example of using the steps to change a graph to a multicast tree. We have not shown the network for simplicity, but we added the groups to each router. The figure shows how a **source-based tree** is made with the source as the root and changed to a multicast subtree with the root at the current router.

8.4.5 Protocol Independent Multicast (PIM)

Protocol Independent Multicast (PIM) is the name given to a common protocol that needs a unicast routing protocol for its operation, but the unicast protocol can be either a distance-vector protocol or a link-state protocol. In other words, PIM needs to use the forwarding table of a unicast routing protocol to find the next router in a path to the

Figure 8.36 *Example of tree formation in MOSPF*



destination, but it does not matter how the forwarding table is created. PIM has another interesting feature. It can work in two different modes: dense and sparse. The term *dense* here means that the number of active members of a group in the internet is large; the probability that a router has a member in a group is high. This may happen, for example, in a popular teleconference that has a lot of members. The term *parse,* on the other hand, means that only a few routers in the internet have active members in the group; the probability that a router has a member of the group is low. This may happen, for example, in a very technical teleconference where a number of members are spread somewhere in the internet. When the protocol is working in the dense mode, it is referred to as PIM-DM; when it is working in the sparse mode, it is referred to as PIM-SM. We explain both protocols next.

Protocol Independent Multicast, Dense Mode (PIM-DM)

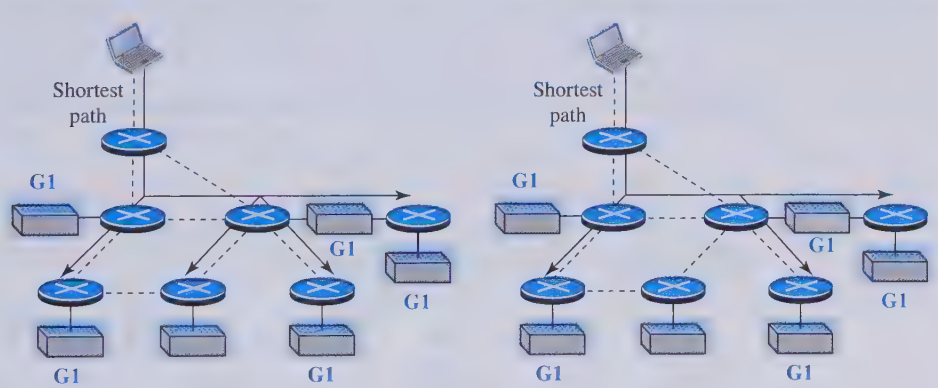
When the number of routers with attached members is large relative to the number of routers in the internet, PIM works in the dense mode and is called **PIM-DM**. In this mode, the protocol uses a source-based tree approach and is similar to DVMRP, but simpler. PIM-DM uses only two strategies described in DVMRP: RPF and RPM. But unlike DVMRP, forwarding of a packet is not suspended awaiting pruning of the first subtree. Let us explain the two steps used in PIM-DM to clear the matter.

1. A router that has received a multicast packet from the source S destined for the group G, first uses the RPF strategy to avoid receiving a duplicate of the packet. It consults the forwarding table of the underlying unicast protocol to find the next router if it wants to send a message to the source S (in the reverse direction). If the packet has not arrived from the next router in the reverse direction, it drops the packet and sends a prune message in that direction to prevent receiving future packets related to (S, G).

2. If the packet in the first step has arrived from the next router in the reverse direction, the receiving router forwards the packet from all its interfaces except the one from which the packet has arrived and the interface from which it has already received a prune message related to (S, G). Note that this is actually a broadcasting instead of a multicasting if the packet is the first packet from the source S to group G. However, each router down the stream that receives an unwanted packet sends a prune message to the router up the stream, and eventually the broadcasting is changed to multicasting. Note that DVMRP behaves differently: it requires that the prune messages (which are part of DV packets) arrive and the tree is pruned before sending any message through unpruned interfaces. PIM-DM does not care about this precaution because it assumes that most routers have an interest in the group (the idea of the dense mode).

Figure 8.37 shows the idea behind PIM-DM. The first packet is broadcast to all networks, which have or do not have members. After a prune message arrives from a router with no member, the second packet is only multicast.

Figure 8.37 *Idea behind PIM-DM*



a. First packet is broadcast.

b. Second packet is multicast.

Protocol Independent Multicast, Sparse Mode (PIM-SM)

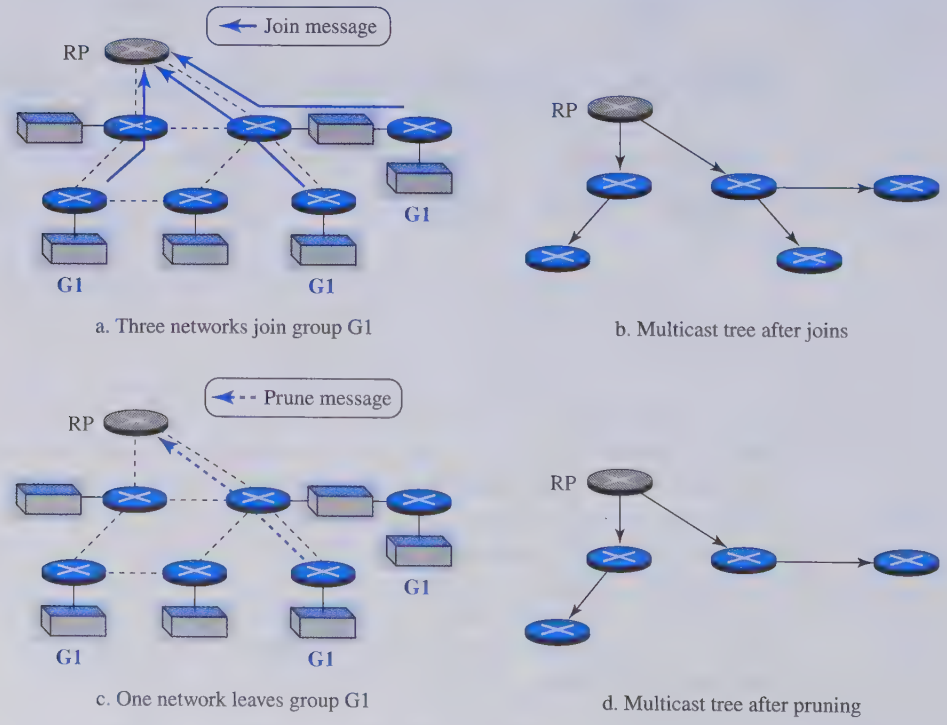
When the number of routers with attached members is small relative to the number of routers in the internet, PIM works in the sparse mode and is called **PIM-SM**. In this environment, the use of a protocol that broadcasts the packets until the tree is pruned is not justified; PIM-SM uses a **group-shared tree** approach to multicasting. The core router in PIM-SM is called the *rendezvous point (RP)*. Multicast communication is achieved in two steps. Any router that has a multicast packet to send to a group of destinations first encapsulates the multicast packet in a unicast packet (tunneling) and sends it to the RP. The RP then decapsulates the unicast packet and sends the multicast packet to its destination.

PIM-SM uses a complex algorithm to select one router among all routers in the internet as the RP for a specific group. This means that if we have *m* active groups, we need *m* RPs, although a router may serve more than one group. After the RP for each group is selected, each router creates a database and stores the group identifier and the IP address of the RP for tunneling multicast packets to it.

PIM-SM uses a spanning multicast tree rooted at the RP with leaves pointing to designated routers connected to each network with an active member. A very interesting point in PIM-SM is the formation of the multicast tree for a group. The idea is that each router helps to create the tree. The router should know the unique interface from which it should accept a multicast packet destined for a group (what was achieved by RPF in DVMRP). The router should also know the interface or interfaces from which it should send out a multicast packet destined for a group (what was achieved by RPM in DVMRP). To avoid delivering more than one copy of the same packet to a network through several routers (what was achieved by RPB in DVMRP), PIM-SM requires that designated routers only send PIM-SM messages, as we will see shortly.

To create a multicast tree rooted at the RP, PIM-SM uses *join* and *prune* messages. Figure 8.38 shows the operation of join and prune messages in PIM-SM. First, three networks join group G1 and form a multicast tree. Later, one of the networks leaves the group and the tree is pruned.

Figure 8.38 *Idea behind PIM-SM*



a. Three networks join group G1

b. Multicast tree after joins

c. One network leaves group G1

d. Multicast tree after pruning

The join message is used to add possible new branches to the tree; the prune message is used to cut branches that are not needed. When a designated router finds out that a network has a new member in the corresponding group (via IGMP), it sends a join message in a unicast packet destined for the RP. The packet travels through the unicast shortest-path tree to reach the RP. Any router in the path receives and forwards the packet, but at the same time, the router adds two pieces of information to its multicast forwarding table. The number of the interface through which the join message has arrived is marked (if not already) as one of the interfaces through which the multicast packet destined for the group should be sent out in the future. The router also adds a count to the number of join messages received here, as we discuss shortly. The number of the interface through which the join message was sent to the RP is marked (if not already) as the only interface through which the multicast packet destined for the same group should be received. In this way, the first join message sent by a designated router creates a path from the RP to one of the networks with group members.

To avoid sending multicast packets to networks with no members, PIM-SM uses the prune message. Each designated router that finds out (via IGMP) that there is no active member in its network, sends a prune message to the RP. When a router receives a prune message, it decrements the join count for the interface through which the message has arrived and forwards it to the next router. When the join count for an interface reaches zero, that interface is not part of the multicast tree anymore.

8.5 IGMP

The protocol that is used today for collecting information about group membership is the **Internet Group Management Protocol (IGMP)**. IGMP is a protocol defined at the network layer; it is one of the auxiliary protocols, like ICMP, that is considered part of the IP. IGMP messages, like ICMP messages, are encapsulated in an IP datagram.

8.5.1 Messages

There are only two types of messages in IGMP, version 3: query and report messages, as shown in Figure 8.39. A query message is periodically sent by a router to all hosts attached to it to ask them to report their interests about membership in groups. A report message is sent by a host as a response to a query message.

Figure 8.39 *IGMP operation*

Query Message

The query message is sent by a router to all hosts in each interface to collect information about their membership. There are three versions of a query message:

a. A *general* query message is sent about membership in any group. It is encapsulated in a datagram with the destination address 224.0.0.1 (all hosts and routers). Note that all routers attached to the same network receive this message to inform them that this message is already sent and that they should refrain from resending it.

b. A *group-specific* query message is sent from a router to ask about the membership related to a specific group. This is sent when a router does not receive a response about a specific group and wants to be sure that there is no active member of that group in the network. The group identifier (multicast address) is mentioned in the message. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.

c. A *source-and-group-specific* query message is sent from a router to ask about the membership related to a specific group when the message comes from a specific source or sources. Again the message is sent when the router does not hear about a specific group related to a specific host or hosts. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.

Report Message

A report message is sent by a host as a response to a query message. The message contains a list of records in which each record gives the identifier of the corresponding group (multicast address) and the addresses of all sources that the host is interested in receiving messages from (inclusion). The record can also mention the source addresses from which the host does not desire to receive a group message (exclusion). The message is encapsulated in a datagram with the multicast address 224.0.0.22 (multicast address assigned to IGMPv3). In IGMPv3, if a host needs to join a group, it waits until it receives a query message and then sends a report message. If a host needs to leave a group, it does not respond to a query message. If no other host responds to the corresponding message, the group is purged from the router database.

8.5.2 Propagation of Membership Information

After a router has collected membership information from the hosts and other routers at its own level in the tree, it can propagate it to the router located in a higher level of the tree. Finally, the router at the tree root can get the membership information to build the multicast tree. The process, however, is more complex than what we can explain in one paragraph. Interested readers can check the book website for the complete description of this protocol.

8.5.3 Encapsulation

The IGMP message is encapsulated in an IP datagram with the value of the protocol field set to 2 and the TTL field set to 1. The destination IP address of the datagram, however, depends on the type of the message, as shown in Table 8.4.

Table 8.4 *Destination IP Addresses*

Message Type	*IP Address*
General query	224.0.0.1
Other queries	Group address
Report	224.0.0.22

8.6 END-OF-CHAPTER MATERIALS

8.6.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and Requests for Comments (RFCs). The items in brackets refer to the reference list at the end of the text.

Books

Several books give thorough coverage of materials discussed in this chapter. We recommend [Com 06], [Tan 03], [Koz 05], [Ste 95], [GW 04], [Per 00], [Kes 02], [Moy 98], [W & Z 01], and [Los 04].

Requests for Comments

IPv4 addressing is discussed in RFCs 917, 927, 930, 932, 940, 950, 1122, and 1519. Forwarding is discussed in RFCs 1812, 1971, and 1980. MPLS is discussed in RFCs 3031, 3032, 3036, and 3212. IPv4 protocol is discussed in RFCs 791, 815, 894, 1122, 2474, and 2475. ICMP is discussed in RFCs 792, 950, 956, 957, 1016, 1122, 1256, 1305, and 1987. RIP is discussed in RFCs 1058 and 2453. OSPF is discussed in RFCs 1583 and 2328. BGP is discussed in RFCs 1654, 1771, 1773, 1997, 2439, 2918, and 3392. Multicasting is discussed in RFCs 1075, 1585, 2189, 2362, and 3376. IPv6 addressing is discussed in RFCs 2375, 2526, 3513, 3587, 3789, and 4291. IPv6 protocol is discussed in RFCs 2460, 2461, and 2462. ICMPv6 is discussed in RFCs 2461, 2894, 3122, 3810, 4443, and 4620.

8.6.2 Key Terms

autonomous system (AS)
Bellman-Ford equation
Border Gateway Protocol (BGP)
Dijkstra's algorithm
distance vector
distance-vector routing
Distance-Vector Multicast Routing Protocol (DVMRP)
flooding
Internet Group Management Protocol (IGMP)
least-cost tree

link-state database (LSDB)
link-state (LS) routing
Multicast Open Shortest Path First (MOSPF)
Open Shortest Path First (OSPF)
path attributes
path vector (PV) routing
poison-reverse strategy
Protocol Independent Multicast (PIM)
Protocol Independent Multicast, Dense Mode (PIM-DM)
Protocol Independent Multicast, Sparse Mode (PIM-SM)
reverse path forwarding (RPF)
reverse path multicasting (RPM)
Routing Information Protocol (RIP)
shortest-path tree
source-based tree
split-horizon strategy

8.6.3 Summary

In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables. Although there are several routes that a packet can travel from the source to the destination, the question is which should be the best. The interpretation of the term *best* depends on the cost and policy imposed on the trip.

Several routing algorithms, and the corresponding protocols, have been devised to find the best route among them, but only three have survived. In distance-vector routing, the first thing each node creates is its own least-cost tree with the rudimentary information it has about its immediate neighbors. The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet. In other words, in distance-vector routing, a router continuously tells all of its neighbors about what it knows about the whole internet. The protocol that implements distance-vector routing is called the Routing Information Protocol (RIP).

Another routing algorithm that has been used in the Internet is link-state routing. This method uses the term *link-state* to define the characteristic of a link (an edge) that represents a network in the internet. In this algorithm the cost associated with an edge defines the state of the link. In this algorithm, all routers flood the internet, with information related to their link states. When every router has the complete picture of the states, a link-state database can be created. The least-cost tree for each router and the corresponding forwarding table can be made from the link-state database. A protocol that implements link-state routing is called Opens Shortest Path First (OSPF).

Both link-state and distance-vector routing are based on the least-cost goal. However, there are instances where this goal is not the priority. Path-vector routing algorithm have been designed for this purpose. We can always insert policies in the forwarding table by preventing a packet from visiting a specific router. In path-vector routing, the best route from the source is the best path, the one that complies with the policy imposed. The protocol that implements path-vector routing is the Border Gateway Protocol (BGP).

Multicasting is the sending of the same message to more than one receiver simultaneously. It has many applications including distributed databases, information dissemination, teleconferencing, and distance learning.

In classless addressing, the block 224.0.0.0/4 is used for multicast addressing. This block is sometimes referred to as the multicast address space and is divided into several blocks (smaller blocks) for different purposes.

8.7 PRACTICE SET

8.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

8.7.2 Questions

Q8-1. In a graph, if we know that the shortest path from node A to node G is (A→B→E→G), what is the shortest path from node G to node A?

Q8-2. Assume the shortest path in a graph from node A to node H is A → B→ H. Also assume that the shortest path from node H to node N is H → G → N. What is the shortest path from node A to node N?

Q8-3. Explain why a router using link-state routing needs to receive the whole LSDB before creating and using its forwarding table. In other words, why can't the router create its forwarding table with a partially received LSDB?

Q8-4. Is the path-vector routing algorithm closer to the distance-vector routing algorithm or to the link-state routing algorithm? Explain.

Q8-5. List three types of autonomous systems (ASs) described in the text and make a comparison between them.

Q8-6. Explain the concept of hop count in RIP. Can you explain why no hop is counted between N1 and R1 in Figure 8.15?

Q8-7. Assume that we have an isolated AS running RIP. We can say that we have at least two different kinds of datagram traffic in this AS. The first kind carries the messages exchanged between hosts; the second carries messages belonging to RIP. What is the difference between the two kinds of traffic when we think about source and destination IP addresses? Does this show that routers also need IP addresses?

Q8-8. Router A sends two RIP messages to two immediate neighboring routers, B and C. Do the two datagrams carrying the messages have the same source IP addresses? Do the two datagrams have the same destination IP addresses?

Q8-9. At any moment, a RIP message may arrive at a router that runs RIP as the routing protocol. Does it mean that the RIP process should be running all the time?

Q8-10. Why do you think RIP uses UDP instead of TCP?

Q8-11. We say that OSPF is a hierarchical intradomain protocol, but RIP is not. What is the reason behind this statement?

Q8-12. In a very small AS using OSPF, is it more efficient to use only one single area (backbone) or several areas?

Q8-13. Why do you think we need only one update RIP message, but several OSPF update messages?

Q8-14. OSPF messages are exchanged between routers. Does this mean that we need to have OSPF processes run all the time to be able to receive an OSPF message when it arrives?

Q8-15. OSPF messages and ICMP messages are directly encapsulated in an IP datagram. If we intercept an IP datagram, how can we tell whether the payload belongs to OSPF or ICMP?

Q8-16. Explain what type of OSPF link state is advertised in each of the following cases:

a. a router needs to advertise the existence of another router at the end of a point-to-point link.

b. a router needs to advertise the existence of two stub networks and one transient network.

c. a designated router advertises a network as a node.

Q8-17. Can a router combine the advertisement of a link and a network in one single link-state update?

Q8-18. Explain why we can have different intradomain routing protocols in different ASs, but we need only one interdomain routing protocol in the whole Internet.

Q8-19. Can you explain why BGP uses the services of TCP instead of UDP?

Q8-20. Explain why policy routing can be implemented on an interdomain routing, but it cannot be implemented on an intradomain routing.

Q8-21. Distinguish between multicasting and multiple-unicasting.

Q8-22. When we send an e-mail to multiple recipients, are we are using multicasting or multiple unicasting? Give the reason for your answer.

Q8-23. Define which of the following addresses are multicast addresses.

a. 224.8.70.14 **b.** 226.17.3.53 **c.** 240.3.6.25

Q8-24. Can a host have more than one multicast address? Explain.

Q8-25. It is obvious that we need to have spanning trees for both unicasting and multicasting. How many leaves of the tree are involved in a transmission in each case?

a. a unicast transmission **b.** a multicast transmission

Q8-26. Assume we have 20 hosts in a small AS. There are only four groups in this AS. Find the number of spanning trees in each of the following approaches.

a. source-based tree **b.** group-shared tree

Q8-27. We say that a router in DVMRP creates a shortest-path tree *on demand*. What is the meaning of this statement? What is the advantage of creating shortest-path trees only on demand?

Q8-28. Does RPF actually create a shortest-path tree? Explain.

Q8-29. Does RPB actually create a shortest-path tree? Explain. What are the leaves of the tree?

Q8-30. Does RPM actually create a shortest-path tree? Explain. What are the leaves of the tree?

Q8-31. List three steps that a DVMRP router uses to create a source-based tree. Which phase is responsible for creating the part of the tree from the source to the current router? Which phase is responsible for creating a broadcast tree with the router as the root? Which phase is responsible for changing the broadcast tree to a multicast tree?

Q8-32. Explain why PIM is called *Protocol Independent Multicast*.

Q8-33. Which version of PIM uses the first and the third steps of DVMRP? What are these two steps?

Q8-34. Explain why broadcasting the first or the first few messages is not important in PIM-DM, but it is important in PIM-SM.

8.7.3 Problems

P8-1. Assume that the shortest distance between nodes ***a***, ***b***, ***c***, and ***d*** to node ***y*** and the costs from node ***x*** to nodes ***a***, ***b***, ***c***, and ***d*** are as follows:

$$\mathbf{D}_{ay} = 5 \qquad \mathbf{D}_{by} = 6 \qquad \mathbf{D}_{cy} = 4 \qquad \mathbf{D}_{dy} = 3$$

$$\mathbf{c}_{xa} = 2 \qquad \mathbf{c}_{xb} = 1 \qquad \mathbf{c}_{xc} = 3 \qquad \mathbf{c}_{xd} = 1$$

What is the shortest distance between node ***x*** and node ***y***, $\mathbf{D}_{xy}$, according to the Bellman-Ford equation?

P8-2. Assume a router using RIP has 10 entries in its forwarding table at time t_1. Six of these entries are still valid at time t_2. Four of these entries have been expired 70, 90, 110, and 210 s before time t_2. Find the number of periodic timers, expiration timers, and garbage collection timers running at time t_1 and time t_2.

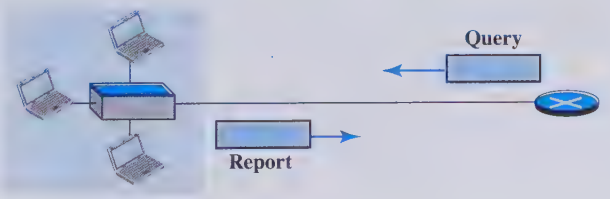
P8-3. When does an OSPF router send each of the following messages?

a. hello **b.** data description **c.** link-state request
d. link-state update **e.** link-state acknowledge

P8-4. To understand how the distance-vector algorithm in Table 8.1 works, let us apply it to a four-node internet as shown in Figure 8.40.

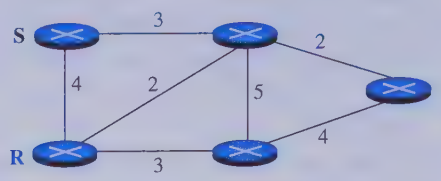
Assume that all nodes are initialized first. Also assume that the algorithm is applied, one at a time, to each node, respectively (A, B, C, D). Show that the process converges and all nodes will have their stable distance vectors.

Figure 8.40 *Problem P8-5*



P8-5. In distance-vector routing, good news (decrease in a link metric) will propagate fast. In other words, if a link distance decreases, all nodes quickly learn about it and update their vectors. In Figure 8.41, we assume that a four-node internet is stable, but suddenly the distance between nodes A and D, which is currently 6, is decreased to 1 (probably due to some improvement in the link quality). Show how this good news is propagated, and find the new distance vector for each node after stabilization.

Figure 8.41 *Problem P8-6*

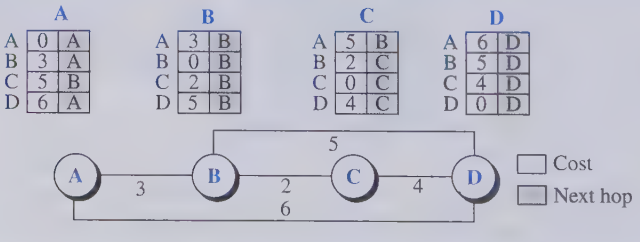


P8-6. In distance-vector routing, bad news (increase in a link metric) will propagate slowly. In other words, if a link distance increases, sometimes it takes a long time for all nodes to know the bad news. In Figure 8.40 (see problem P8-5), we assume that a four-node internet is stable, but suddenly the distance between nodes B and C, which is currently 2, is increased to infinity (link fails). Show how this bad news is propagated, and find the new distance vector for each node after stabilization. Assume that the implementation uses a periodic timer to trigger updates to neighbors (no more updates are triggered when there is change). Also assume that if a node receives a higher cost from the same previous neighbor, it uses the new cost because this means that the old advertisement is not valid anymore. To make the stabilization faster, the implementation also suspends a route when the next hop is not accessible.

P8-7. In computer science, when we encounter an algorithm, we often need to ask about the complexity of that algorithm (how many computations we need to do). To find the complexity of the distance vector's algorithm, find the number of operations a node needs to do when it receives a vector from a neighbor.

P8-8. Assume that the network in Figure 8.42 uses distance-vector routing with the forwarding table as shown for each node.

Figure 8.42 *Problem P8-9*



If each node periodically announces their vectors to the neighbor using the poisoned-reverse strategy, what is the distance vector advertised in each of the appropriate period?

a. from A to B **b.** from C to D **c.** from D to B **d.** from C to A

P8-9. Assume that the network in Figure 8.41 (see Problem P8-8) uses distance-vector routing with the forwarding table as shown for each node. If each node periodically announces their vectors to the neighbor using the split-horizon strategy, what is the distance vector advertised in the appropriate period?

a. from A to B **b.** from C to D **c.** from D to B **d.** from C to A

P8-10. Assume that the network in Figure 8.41 (see Problem P8-8) uses distance-vector routing with the forwarding table as shown for each node. If node E is added to the network with a link of cost 1 to node D, can you find the new forwarding tables for each node without using the distance-vector algorithm?

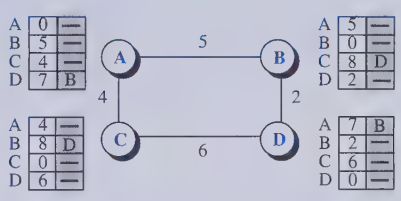
P8-11. Create the forwarding table for node A in Figure 8.10.

P8-12. Create the shortest-path tree and the forwarding table for node G in Figure 8.8.

P8-13. Create the shortest-path tree and the forwarding table for node B in Figure 8.8.

P8-14. Use Dijkstra's algorithm (Table 4.4) to find the shortest-path tree and the forwarding table for node A in Figure 8.43.

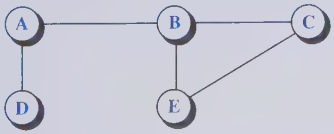
Figure 8.43 *Problem P8-15*



P8-15. In computer science, when we encounter an algorithm, we often need to ask about the complexity of that algorithm (how many computations we need to do). To find the complexity of Dijkstra's algorithm, find the number of searches we have do to find the shortest path for a single node when the number of nodes is n.

P8-16. Assume that A, B, C, D, and E in Figure 8.44 are autonomous systems (ASs). Find the path vector for each AS using the algorithm in Table 8.3. Assume that the best path in this case is the path that passes through the shorter list of ASs. Also assume that the algorithm first initializes each AS and then is applied, one at a time, to each node respectively (A, B, C, D, E). Show that the process converges and all ASs will have their stable path vectors.

Figure 8.44 *Problem P8-16*



P8-17. In Figure 8.24, assume that the intra-AS routing protocol used by AS1 is OSPF, but the one used by AS2 is RIP. Explain how R5 can find how to route a packet to N4.

P8-18. In Figure 8.24, assume that the intra-AS routing protocol used by AS4 and AS3 is RIP. Explain how R8 can find how to route a packet to N13.

P8-19. A multicast address for a group is 232.24.60.9. What is its 48-bit Ethernet address for a LAN using TCP/IP?

P8-20. Change the following IP multicast addresses to Ethernet multicast addresses. How many of them specify the same Ethernet address?

a. 224.18.72.8 **b.** 235.18.72.8 **c.** 237.18.6.88 **d.** 224.88.12.8

P8-21. A router using DVMRP receives a packet with source address 10.14.17.2 from interface 2. If the router forwards the packet, what are the contents of the entry related to this address in the unicast routing table?

P8-22. Router A sends a unicast RIP update packet to router B that says 134.23.0.0/16 is seven hops away. Network B sends an update packet to router A that says 13.23.0.0/16 is four hops away. If these two routers are connected to the same network, which one is the designated parent router?

P8-23. Assume that m is much less than n and that router R is connected to n networks in which only m of these networks are interested in receiving packets related to group G. How can router R manage to send a copy of the packet to only those networks interested in group G?

CHAPTER 9

Transport Layer

The transport layer in the TCP/IP suite is located between the application layer and the network layer. It provides services to the application layer and receives services from the network layer. The transport layer acts as a liaison between a client program and a server program, a process-to-process connection. The transport layer is the heart of the TCP/IP protocol suite; it is the end-to-end logical vehicle for transferring data from one point to another in the Internet.

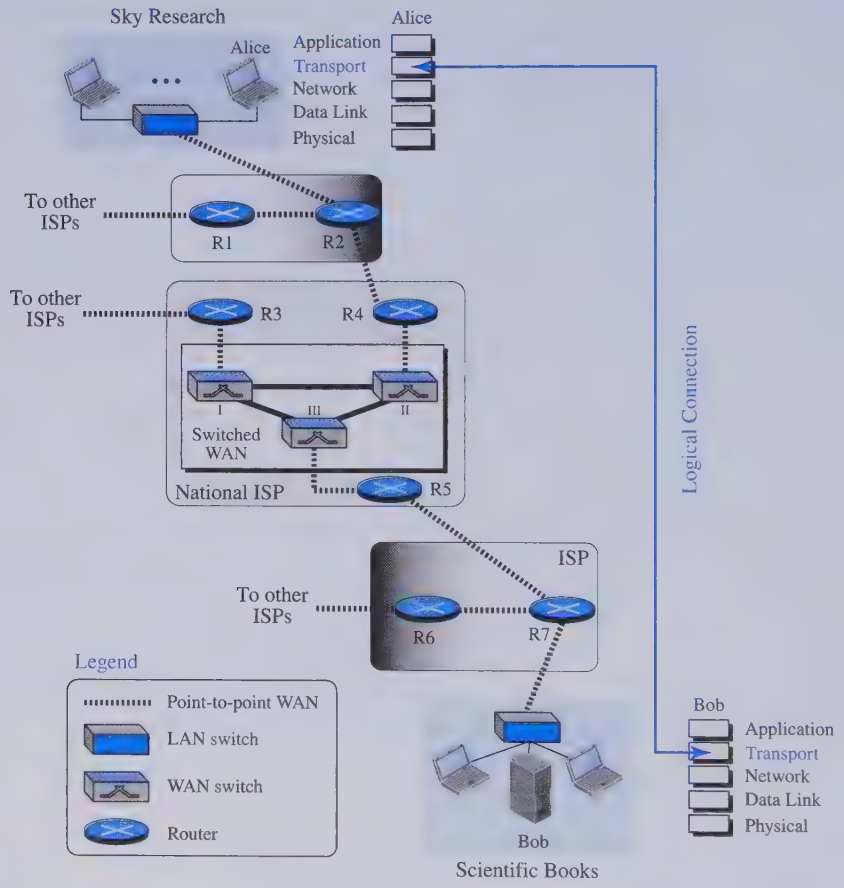
This chapter is divided into four sections.

- The first section introduces the idea behind a transport-layer protocol. We first discuss the general services we normally require from the transport layer, such as process-to-process communication; addressing; multiplexing and demultiplexing; error, flow, and congestion control.
- The second section concentrates on UDP. Although, UDP lacks many services we require from a transport-layer protocol, its simplicity is very attractive to some applications, as we show.
- The third section discusses TCP. It first lists TCP's services and features. It then shows how TCP provides a connection-oriented service using a transition diagram. Next, it shows how flow and error control are accomplished in TCP using abstract windows. Finally, congestion control in TCP (which was part of our discussion of the network layer in Chapter 7) is discussed.
- The fourth section discusses SCTP. The section first lists its services and features. It then shows how STCP creates an association. Finally, it shows how flow and error control are accomplished in SCTP using SACKs.

9.1 TRANSPORT-LAYER SERVICES

The transport layer is located between the network layer and the application layer. The transport layer is responsible for providing services to the application layer; it receives services from the network layer. Figure 9.1 shows the communication between Alice and Bob at the transport layer.

Figure 9.1 *Logical connection at the transport layer*



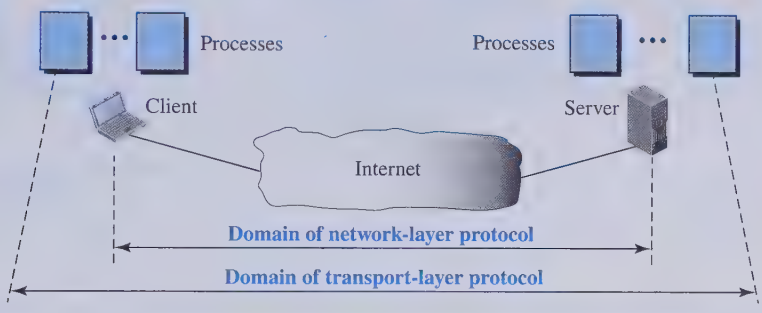
9.1.1 Process-to-Process Communication

The first duty of a transport-layer protocol is to provide **process-to-process communication**. A process is an application-layer entity (running program) that uses the services of the transport layer. Before we discuss how process-to-process communication can be

accomplished, we need to understand the difference between host-to-host communication and process-to-process communication.

The network layer is responsible for communication at the computer level (host-to-host communication). A network-layer protocol can deliver the message only to the destination computer. However, this is an incomplete delivery. The message still needs to be handed to the correct process. This is where a transport-layer protocol takes over. A transport-layer protocol is responsible for delivery of the message to the appropriate process. Figure 9.2 shows the domains of a network layer and a transport layer.

Figure 9.2 *Network layer versus transport layer*



9.1.2 Addressing: Port Numbers

Although, there are a few ways to achieve process-to-process communication, the most common is through the **client/server paradigm** (see Chapter 10). A process on the local host, called a *client*, needs services from a process usually on the remote host, called a *server.*

However, operating systems today support both multiuser and multiprogramming environments. A remote computer can run several server programs at the same time, just as several local computers can run one or more client programs at the same time. For communication, we must define the local host, local process, remote host, and remote process. The local host and the remote host are defined using IP addresses (discussed in Chapter 7). To define the processes, we need second identifiers, called **port numbers**. In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535 (16 bits).

The client program defines itself with a port number, called the **ephemeral port number.** The word *ephemeral* means "short-lived" and is used because the life of a client is normally short. An ephemeral port number is recommended to be greater than 1023 for some client/server programs to work properly.

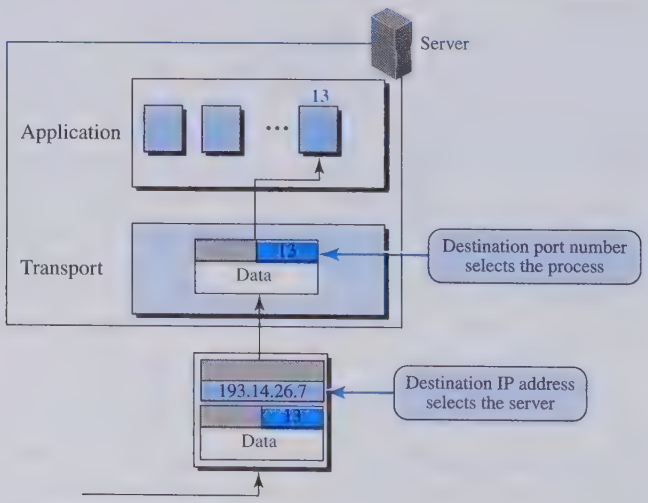
The server process must also define itself with a port number. This port number, however, cannot be chosen randomly. If the computer at the server site runs a server process and assigns a random number as the port number, the process at the client site that wants to access that server and use its services will not know the port number. TCP/IP has decided to use universal port numbers for servers; these are called **well-known**

port numbers. Every client process knows the well-known port number of the corresponding server process. For example, while the daytime client process, an application program, can use an ephemeral (temporary) port number, 52,000, to identify itself, the daytime server process must use the well-known (permanent) port number 13. Figure 9.3 shows this concept.

Figure 9.3 *Port numbers*

It should be clear by now that the IP addresses and port numbers play different roles in selecting the final destination of data. The destination IP address defines the host among the different hosts in the world. After the host has been selected, the port number defines one of the processes on this particular host (see Figure 9.4).

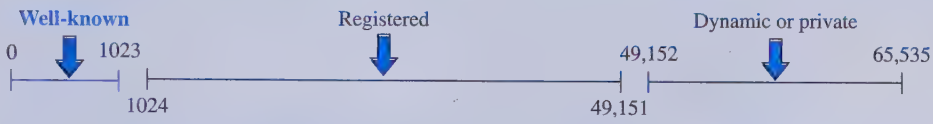
Figure 9.4 *IP addresses versus port numbers*



ICANN Ranges

The Internet Corporation for Assigned Names and Numbers (ICANN) has divided the port numbers into three ranges: well-known, registered, and dynamic (or private), as shown in Figure 9.5.

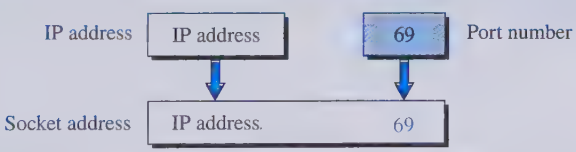
Figure 9.5 *ICANN ranges*



- **Well-known ports.** The ports ranging from 0 to 1023 are assigned and controlled by ICANN. These are the well-known ports.
- **Registered ports.** The ports ranging from 1024 to 49,151 are not assigned or controlled by ICANN. They can only be registered with ICANN to prevent duplication.
- **Dynamic ports.** The ports ranging from 49,152 to 65,535 are neither controlled nor socket addresses.

A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to make a connection. The combination of an IP address and a port number is called a **socket address**. The client socket address defines the client process uniquely, just as the server socket address defines the server process uniquely (see Figure 9.6).

Figure 9.6 *Socket address*



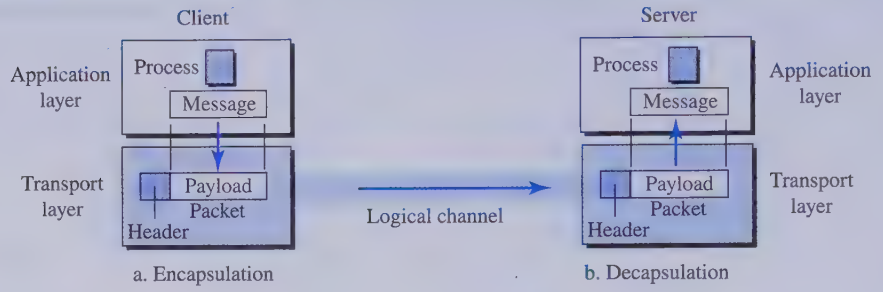
To use the services of the transport layer in the Internet, we need a pair of socket addresses: the client socket address and the server socket address. These four pieces of information are part of the network-layer packet header and the transport-layer packet header. The first header contains the IP addresses; the second header contains the port numbers.

9.1.3 Encapsulation and Decapsulation

To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages (Figure 9.7 on next page). Encapsulation happens at the sender site. When a process has a message to send, it passes the message to the transport layer along

with a pair of socket addresses and some other pieces of information, which depend on the transport-layer protocol. The transport layer receives the data and adds the transport-layer header.

Figure 9.7 *Encapsulation and decapsulation*



Decapsulation happens at the receiver site. When the message arrives at the destination transport layer, the header is dropped and the transport layer delivers the message to the process running at the application layer. The sender socket address is passed to the process in case it needs to respond to the message received.

9.1.4 Multiplexing and Demultiplexing

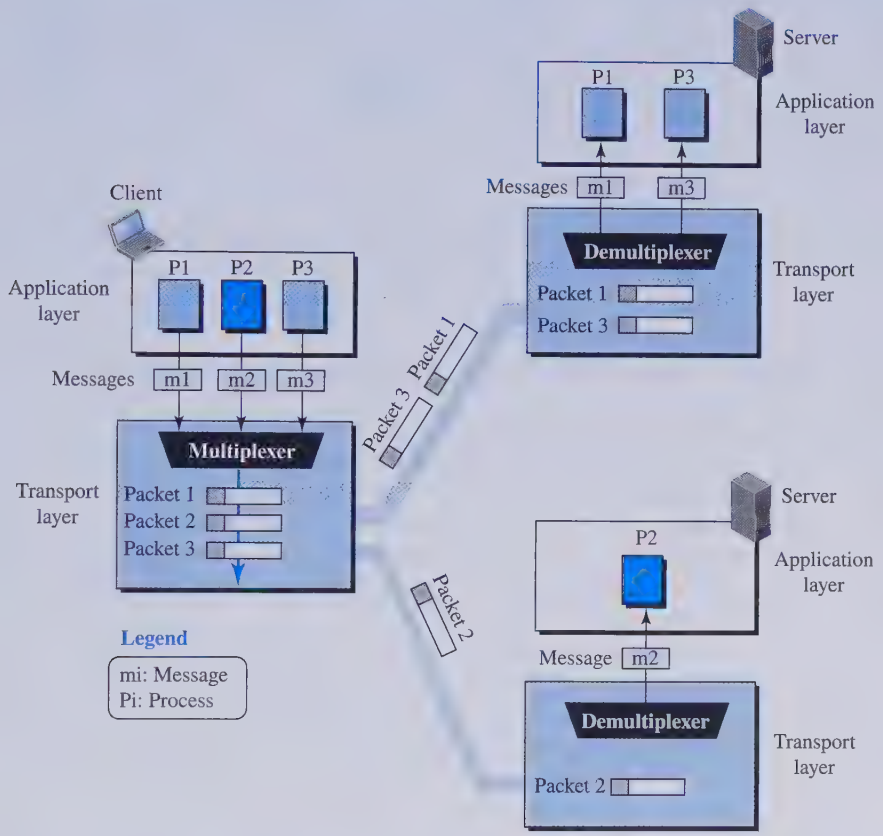
Whenever an entity accepts items from more than one source, this is referred to as **multiplexing** (many to one); whenever an entity delivers items to more than one source, this is referred to as **demultiplexing** (one to many). The transport layer at the source performs multiplexing; the transport layer at the destination performs demultiplexing (Figure 9.8).

Figure 9.8 shows communication between a client and two servers. Three client processes are running at the client site: P1, P2, and P3. The processes P1 and P3 need to send requests to the corresponding server process running in a server. The client process P2 needs to send a request to the corresponding server process running at another server. The transport layer at the client site accepts three messages from the three processes and creates three packets. It acts as a *multiplexer*. Packets 1 and 3 use the same logical channel to reach the transport layer of the first server. When they arrive at the server, the transport layer does the job of a *demultiplexer* and distributes the messages to two different processes. The transport layer at the second server receives packet 2 and delivers it to the corresponding process. Note that we still have demultiplexing although there is only one message.

9.1.5 Flow Control

Whenever an entity produces items and another entity consumes them, there should be a balance between production and consumption rates. If the items are produced faster

Figure 9.8 *Multiplexing and demultiplexing*

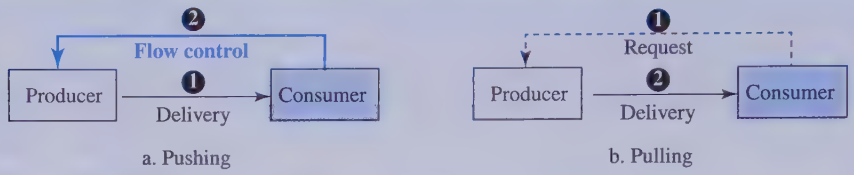


than they can be consumed, the consumer can be overwhelmed and may need to discard some items. If the items are produced more slowly than they can be consumed, the consumer must wait, and the system becomes less efficient. Flow control is related to the first issue. We need to prevent loss of the data items at the consumer site.

Pushing or Pulling

Delivery of items from a producer to a consumer can occur in one of two ways: *pushing* or *pulling*. If the sender delivers items whenever they are produced—without a prior request from the consumer—the delivery is referred to as *pushing*. If the producer delivers the items after the consumer has requested them, the delivery is referred to as *pulling*. Figure 9.9 shows these two types of deliveries.

Figure 9.9 *Pushing or pulling*



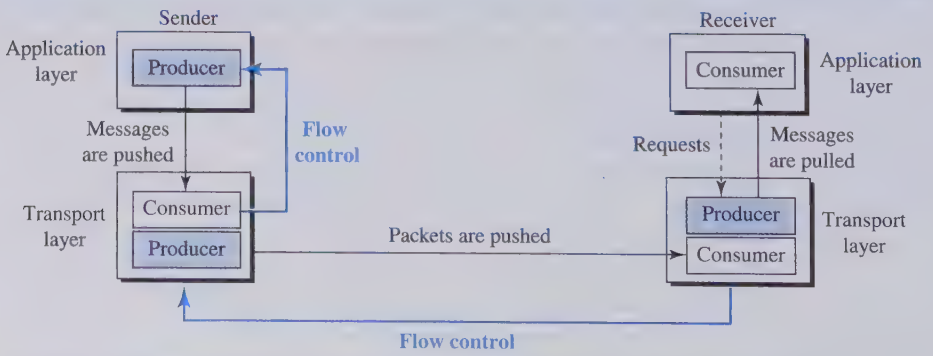
When the producer *pushes* the items, the consumer may be overwhelmed and there is a need for flow control, in the opposite direction, to prevent the items being discarded. In other words, the consumer needs to warn the producer to stop the delivery and to inform it when it is ready again to receive the items. When the consumer pulls the items, it requests them when it is ready. In this case, there is no need for flow control.

Handling Flow Control

In communication at the transport layer, we are dealing with four entities: sender process, sender transport layer, receiver transport layer, and receiver process. The sending process at the application layer is only a producer. It produces message chunks and pushes them to the transport layer. The sending transport layer has a double role: It is both a consumer and a producer. It consumes the messages pushed by the producer. It encapsulates the messages in packets and pushes them to the receiving transport layer. The receiving transport layer also has a double role: It is the consumer for the packets received from the sender and the producer that decapsulates the messages and delivers them to the application layer. The last delivery, however, is normally a pulling delivery; the transport layer waits until the application-layer process asks for messages.

Figure 9.10 shows that we need at least two cases of flow control: from the sending transport layer to the sending application layer and from the receiving transport layer to the sending transport layer.

Figure 9.10 *Flow control at the transport layer*



Buffers

Although flow control can be implemented in several ways, one of the solutions is normally to use two *buffers*: one at the sending transport layer and the other at the receiving transport layer. A buffer is a set of memory locations that can hold packets at the sender and receiver. The flow-control communication can occur by sending signals from the consumer to the producer.

When the buffer of the sending transport layer is full, it informs the application layer to stop passing chunks of messages; when there are some vacancies, it informs the application layer that it can pass message chunks again.

When the buffer of the receiving transport layer is full, it informs the sending transport layer to stop sending packets. When there are some vacancies, it informs the sending transport layer that it can send packets again.

Example 9.1

The preceding discussion requires that consumers communicate with the producers on two occasions: when the buffer is full and when there are vacancies. If the two parties use a buffer with only one slot, the communication can be easier. Assume that each transport layer uses one single memory location to hold a packet. When this single slot in the sending transport layer is empty, the sending transport layer sends a note to the application layer to send its next chunk; when this single slot in the receiving transport layer is empty, it sends an acknowledgment to the sending transport layer to send its next packet. However, this type of flow control, using a single-slot buffer at the sender and the receiver, is inefficient.

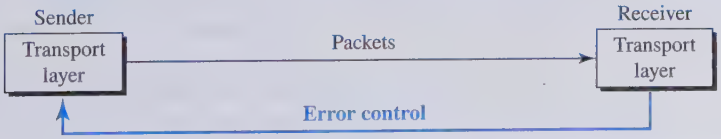
9.1.6 Error Control

In the Internet, because the underlying network layer (IP) is unreliable, we need to make the transport layer reliable if the application requires reliability. Reliability can be achieved to add error-control services to the transport layer. Error control at the transport layer is responsible for

1. Detecting and discarding corrupted packets
2. Keeping track of lost and discarded packets and resending them
3. Recognizing duplicate packets and discarding them
4. Buffering out-of-order packets until the missing packets arrive

Error control, unlike flow control, involves only the sending and receiving transport layers. We are assuming that the message chunks exchanged between the application and transport layers are error-free. Figure 9.11 shows the error control between the sending

Figure 9.11 *Error control at the transport layer*



and receiving transport layers. As with the case of flow control, the receiving transport layer manages error control, most of the time, by informing the sending transport layer about the problems.

Sequence Numbers

Error control requires that the sending transport layer knows which packet is to be resent and the receiving transport layer knows which packet is a duplicate, or which packet has arrived out of order. This can be done if the packets are numbered. We can add a field to the transport-layer packet to hold the **sequence number** of the packet. When a packet is corrupted or lost, the receiving transport layer can somehow inform the sending transport layer to resend that packet using the sequence number. The receiving transport layer can also detect duplicate packets if two received packets have the same sequence number. The out-of-order packets can be recognized by observing gaps in the sequence numbers.

Packets are numbered sequentially. However, because we need to include the sequence number of each packet in the header, we need to set a limit. If the header of the packet allows m bits for the sequence number, the sequence numbers range from 0 to $2^m - 1$. For example, if m is 4, the only sequence numbers are 0 through 15, inclusive. However, we can wrap around the sequence. So the sequence numbers in this case are:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, ...

In other words, the sequence numbers are modulo 2^m.

For error control, the sequence numbers are modulo 2^m, where m is the size of the sequence number field in bits.

Acknowledgment

We can use both positive and negative signals as error control, but we discuss only positive signals, which are more common at the transport layer. The receiver side can send an acknowledgment (ACK) for each of a collection of packets that have arrived safe and sound. The receiver can simply discard the corrupted packets. The sender can detect lost packets if it uses a timer. When a packet is sent, the sender starts a timer. If an ACK does not arrive before the timer expires, the sender resends the packet. Duplicate packets can be silently discarded by the receiver. Out-of-order packets can be either discarded (to be treated as lost packets by the sender) or stored until the missing ones arrive.

9.1.7 Combination of Flow and Error Control

We have discussed that flow control requires the use of two buffers, one at the sender site and the other at the receiver site. We have also discussed that error control requires the use of sequence and acknowledgment numbers by both sides. These two requirements can be combined if we use two numbered buffers: one at the sender and one at the receiver.

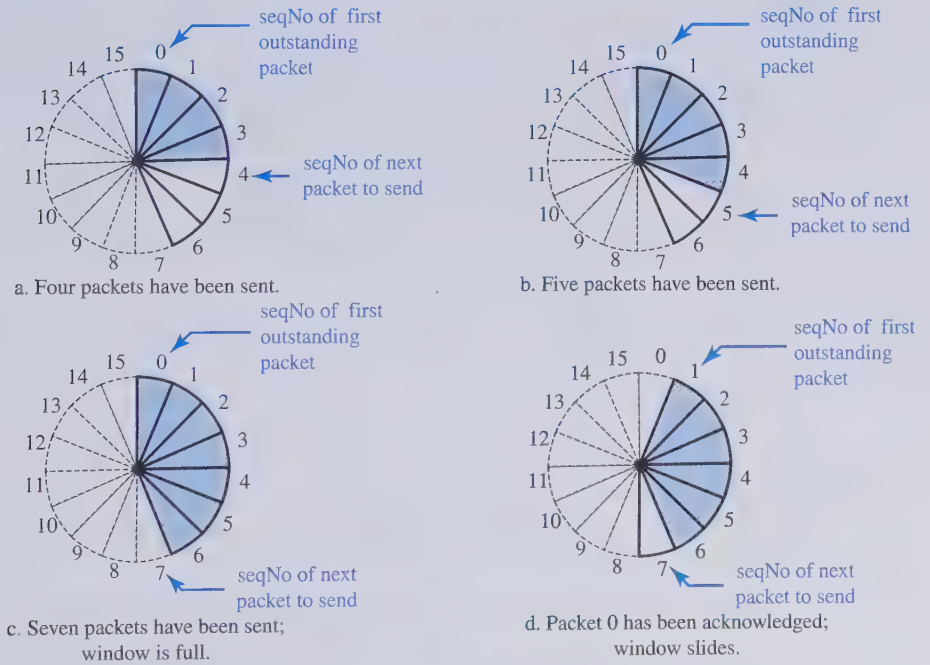
At the sender, when a packet is prepared to be sent, we use the number of the next free location, x, in the buffer as the sequence number of the packet. When the packet is sent, a copy is stored at memory location x, awaiting the acknowledgment from the other end. When an acknowledgment related to a sent packet arrives, the packet is purged and the memory location becomes free.

At the receiver, when a packet with sequence number y arrives, it is stored at the memory location y until the application layer is ready to receive it. An acknowledgment can be sent to announce the arrival of packet y.

Sliding Window

Because the sequence numbers used modulo 2^m, a circle can represent the sequence numbers from 0 to $2^m - 1$ (Figure 9.12). The buffer is represented as a set of slices, called the **sliding window**, that occupies part of the circle at any time. At the sender site, when a packet is sent, the corresponding slice is marked. When all the slices are marked, it means that the buffer is full and no further messages can be accepted from the application layer. When an acknowledgment arrives, the corresponding slice is unmarked. If some consecutive slices from the beginning of the window are unmarked, the window slides over the range of the corresponding sequence numbers to allow more free slices at the end of the window. Figure 9.12 shows the sliding window at the sender.

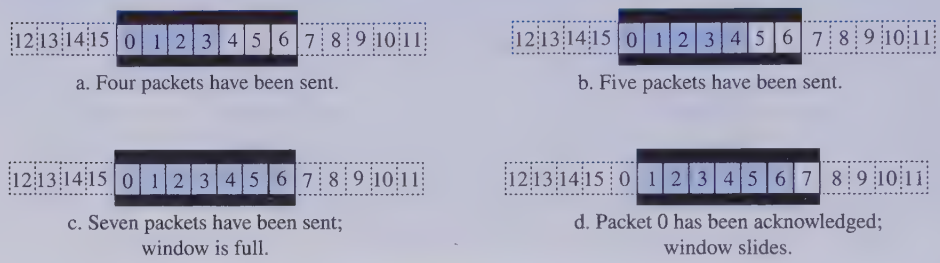
Figure 9.12 *Sliding window in a circular format*



The sequence numbers are in modulo 16 ($m = 4$), and the size of the window is 7. Note that the sliding window is just an abstraction: The actual situation uses computer variables to hold the sequence numbers of the next packet to be sent and the last packet sent.

Most protocols show the sliding window using linear representation. The idea is the same, but it normally takes less space on paper. Figure 9.13 shows this representation. Both representations tell us the same thing. If we take both sides of each diagram in Figure 9.13 and bend them up, we can make the same diagram as in Figure 9.12.

Figure 9.13 *Sliding window in linear format*



a. Four packets have been sent.

b. Five packets have been sent.

c. Seven packets have been sent; window is full.

d. Packet 0 has been acknowledged; window slides.

9.1.8 Congestion Control

An important issue in a packet-switched network, such as the Internet, is **congestion**. Congestion in a network may occur if the *load* on the network—the number of packets sent to the network—is greater than the *capacity* of the network—the number of packets a network can handle. **Congestion control** refers to the mechanisms and techniques that control the congestion and keep the load below the capacity.

We may ask why there is congestion in a network. Congestion happens in any system that involves waiting. For example, congestion can happen on a freeway because any abnormality in the flow, such as that caused by an accident during rush hour, can create a blockage.

Congestion in a network or internetwork occurs because routers and switches have queues—buffers that hold the packets before and after processing. A router, for example, has an input queue and an output queue for each interface. If a router cannot process the packets at the same rate at which they arrive, the queues become overloaded and congestion occurs. Congestion at the transport layer is actually the result of congestion at the network layer, which manifests itself at the transport layer. We discuss congestion control at the network layer and its causes in Chapter 7. Later in Section 9.3.1, we show how TCP, assuming that there is no congestion control at the network layer, implements its own congestion-control mechanism.

9.1.9 Connectionless and Connection-Oriented Protocols

A transport-layer protocol, like a network-layer protocol, can provide two types of services: connectionless and connection-oriented. The nature of these services at the

transport layer, however, is different from the ones at the network layer. At the network layer, a connectionless service may mean different paths for different datagrams belonging to the same message. At the transport layer, we are not concerned about the physical paths of packets (we assume a logical connection between two transport layers). Connectionless service at the transport layer means independency between packets; connection-oriented means dependency. Let us elaborate on these two services.

Connectionless Service

In a connectionless service, the source process (application program) needs to divide its message into chunks of data of the size acceptable by the transport layer and deliver them to the transport layer one by one. The transport layer treats each chunk as a single unit without any relation between the chunks. When a chunk arrives from the application layer, the transport layer encapsulates it in a packet and sends it. To show the independency of packets, assume that a client process has three chunks of messages to send to a server process. The chunks are handed over to the connectionless transport protocol in order. However, because there is no dependency between the packets at the transport layer, the packets may arrive out of order at the destination and will be delivered out of order to the server process (Figure 9.14).

Figure 9.14 *Connectionless service*

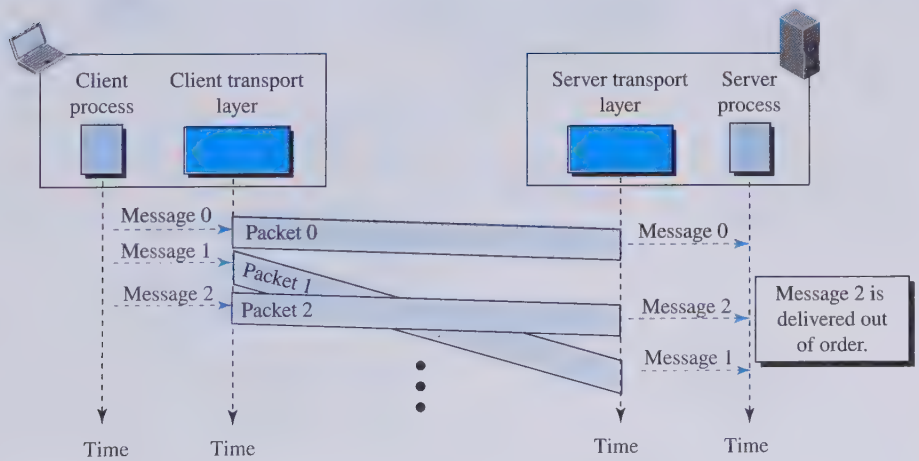


Figure 9.14 shows the movement of packets using a time line, but we have assumed that the delivery of the process to the transport layer and vice versa are instantaneous. The figure shows that at the client site, the three chunks of messages are delivered to the client transport layer in order (0, 1, and 2). Because of the extra delay in transportation of the second packet, the delivery of messages at the server is not in order (0, 2, 1). If these three chunks of data belong to the same message, the server process may have received a strange message.

The situation would be worse if one of the packets were lost. Because there is no numbering on the packets, the receiving transport layer has no idea that one of the messages has been lost. It just delivers two chunks of data to the server process.

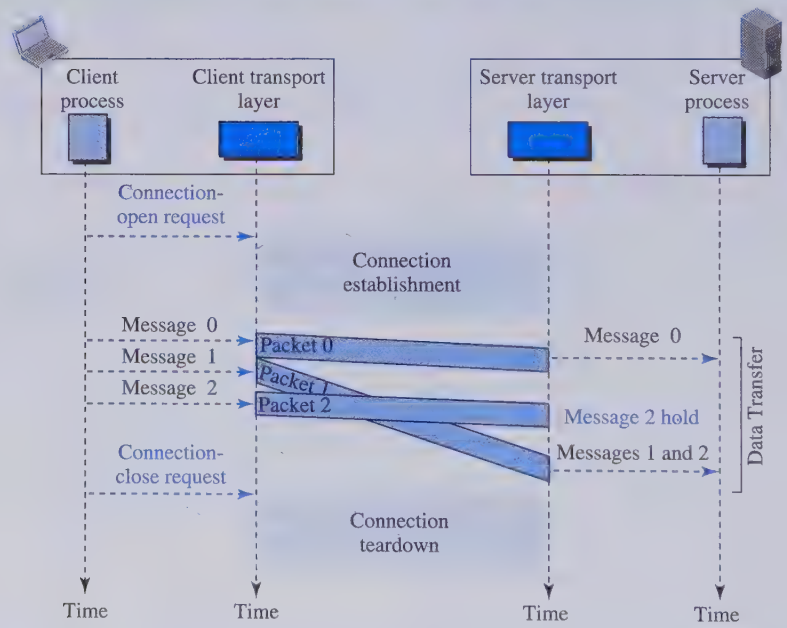
These two problems arise from the fact that the two transport layers do not coordinate with each other. The receiving transport layer does not know when the first packet will come nor when all the packets have arrived.

We can say that no flow control, error control, or congestion control can be effectively implemented in a connectionless service.

Connection-Oriented Service

In a connection-oriented service, the client and the server first need to establish a logical connection between themselves. The data exchange can only happen after the connection is established. After the data exchange, the connection needs to be torn down (Figure 9.15).

Figure 9.15 *Connection-oriented service*



As we mentioned before, the connection-oriented service at the transport layer is different from the same service at the network layer. In the network layer, connection-oriented service means a coordination between the two end hosts and all the routers in between. At the transport layer, connection-oriented service involves only the two hosts; the service is end-to-end. This means that we should be able to make a connection-oriented protocol at the transport layer over either a connectionless or

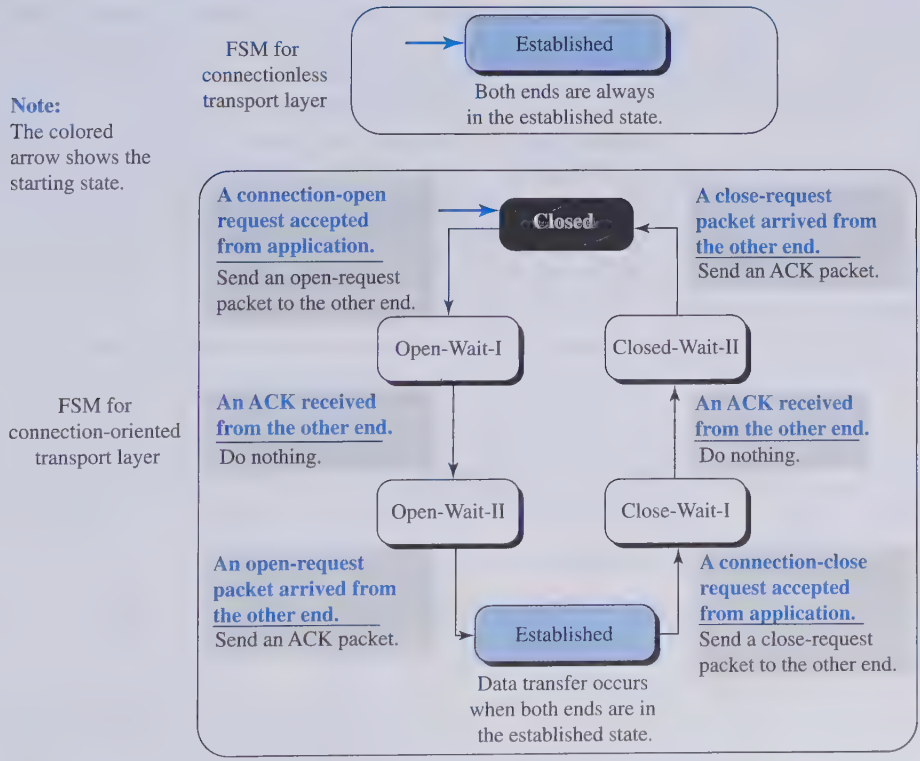
connection-oriented protocol at the network layer. Figure 9.15 shows the connection-establishment, data-transfer, and tear-down phases in a connection-oriented service at the transport layer.

We can implement flow control, error control, and congestion control in a connection-oriented protocol.

Finite State Machine

The behavior of a transport-layer protocol, both when it provides a connectionless protocol and when it provides a connection-oriented protocol, can be better shown as a **finite state machine (FSM)**. Figure 9.16 shows a representation of a transport layer using an FSM. Using this tool, each transport layer (sender or receiver) is taught as a machine with a finite number of states. The machine is always in one of the states until an *event* occurs. Each event is associated with two reactions: defining the list (possibly empty) of actions to be performed and determining the next state (which can be the same as the current state). One of the states must be defined as the initial state, the

Figure 9.16 *Connectionless and connection-oriented service represented as FSMs*



state in which the machine starts when it turns on. In Figure 9.16, we have used rounded-corner rectangles to show states, color text to show events, and regular black text to show actions. A horizontal line is used to separate the event from the actions, although later we replace the horizontal line with a slash. The arrow shows the movement to the next state.

We can think of a connectionless transport layer as an FSM with only one single state: the established state. The machine on each end (client and server) is always in the established state, ready to send and receive transport-layer packets.

An FSM in a connection-oriented transport layer, on the other hand, needs to go through three states before reaching the established state. The machine also needs to go through three states before closing the connection. The machine is in the *closed* state when there is no connection. It remains in this state until a request for opening the connection arrives from the local process; the machine sends an open-request packet to the remote transport layer and moves to the *open-wait-I* state. When an acknowledgment is received from the other end, the local FSM moves to the *open-wait-II* state. When the machine is in this state, a unidirectional connection has been established, but if a bidirectional connection is needed, the machine needs to wait in this state until the other end also requests a connection. When the request is received, the machine sends an acknowledgment and moves to the *established* state.

Data and data acknowledgment can be exchanged between the two ends when they are both in the established state. However, we need to remember that the established state, both in connectionless and connection-oriented transport layers, represents a set of data transfer states, which we discuss in Section 9.2.

To tear down a connection, the application layer sends a close-request message to its local transport layer. The transport layer sends a close-request packet to the other end and moves to the *close-wait-I* state. When an acknowledgment is received from the other end, the machine moves to the *close-wait-II* state and waits for the close-request packet from the other end. When this packet arrives, the machine sends an acknowledgment and moves to the *closed* state.

9.2 TRANSPORT-LAYER PROTOCOLS

After discussing the general principle behind the transport layer in Section 9.1, we concentrate on the transport protocols in the Internet in this section. Figure 9.17 shows the position of these three protocols in the TCP/IP protocol suite: UDP, TCP, and SCTP.

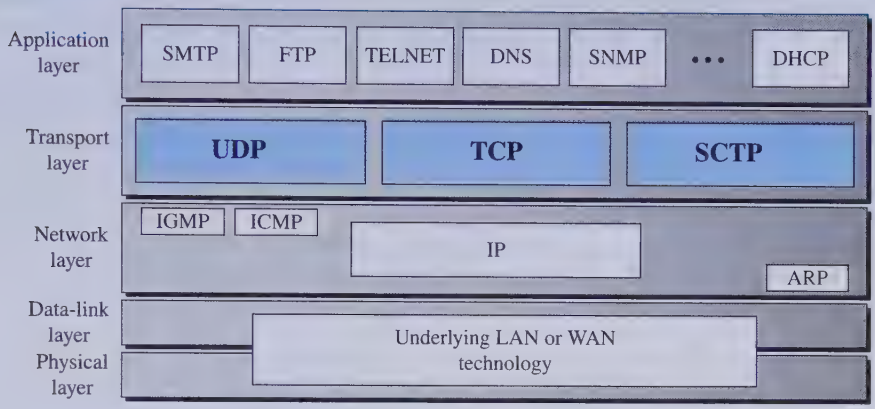
9.2.1 Services

Each protocol provides a different type of service and should be used appropriately.

UDP

UDP is an unreliable connectionless transport-layer protocol used for its simplicity and efficiency in applications where error control can be provided by the application-layer process.

Figure 9.17 *Position of transport-layer protocols in the TCP/IP protocol suite*



TCP

TCP is a reliable connection-oriented protocol that can be used in any application where reliability is important.

SCTP

SCTP is a new transport-layer protocol that combines the features of UDP and TCP.

9.2.2 Port Numbers

A transport-layer protocol usually has several responsibilities. One is to create a process-to-process communication; these protocols use port numbers to accomplish this. Port numbers provide end-to-end addresses at the transport layer and allow multiplexing and demultiplexing at this layer, just as IP addresses do the same at the network layer. Table 9.1 gives some common port numbers for the three protocols we discuss in this chapter.

Table 9.1 *Some well-known ports used with three transport protocols*

Port	*Protocol*	*UDP*	*TCP*	*SCTP*	*Description*
7	Echo	√	√	√	Echoes back a received datagram
9	Discard	√	√	√	Discards any datagram that is received
11	Users	√	√	√	Active users
13	Daytime	√	√	√	Returns the date and the time
17	Quote	√	√	√	Returns a quote of the day
19	Chargen	√	√	√	Returns a string of characters
20	FTP-data		√	√	File Transfer Protocol

Table 9.1 *Some well-known ports used with three transport protocols (continued)*

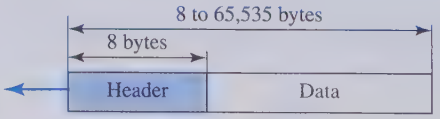
Port	*Protocol*	*UDP*	*TCP*	*SCTP*	*Description*
21	FTP-21		√	√	File Transfer Protocol
23	TELNET		√	√	Terminal Network
25	SMTP		√	√	Simple Mail Transfer Protocol
53	DNS	√	√	√	Domain Name System
67	DHCP	√	√	√	Dynamic Host Configuration Protocol
69	TFTP	√	√	√	Trivial File Transfer Protocol
80	HTTP		√	√	Hypertext Transfer Protocol
111	RPC	√	√	√	Remote Procedure Call
123	NTP	√	√	√	Network Time Protocol
161	SNMP-server	√			Simple Network Management Protocol
162	SNMP-client	√			Simple Network Management Protocol

9.3 USER DATAGRAM PROTOCOL (UDP)

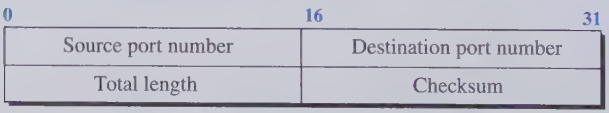
The **User Datagram Protocol (UDP)** is a connectionless, unreliable transport protocol. It does not add anything to the services of IP except for providing process-to-process communication instead of host-to-host communication. If UDP is so powerless, why would a process want to use it? With the disadvantages come some advantages. UDP is a very simple protocol using a minimum of overhead. If a process wants to send a small message and does not care much about reliability, it can use UDP. Sending a small message using UDP takes much less interaction between the sender and receiver than using TCP.

UDP packets, called **user datagrams**, have a fixed-size header of 8 bytes made up of four fields, each of 2 bytes (16 bits). Figure 9.18 shows the format of a user datagram. The first two fields define the source and destination port numbers, respectively. The

Figure 9.18 *User datagram packet format*



a. UDP user datagram



b. Header format

third field defines the total length of the user datagram, header plus data. The 16 bits can define a total length of 0 to 65,535 bytes. However, the total length needs to be less because a UDP user datagram is stored in an IP datagram with the total length of 65,535 bytes. The last field can carry the optional checksum (explained soon).

Example 9.2

The following is the contents of a UDP header in hexadecimal format.

CB84000D001C001C

a. What is the source port number?
b. What is the destination port number?
c. What is the total length of the user datagram?
d. What is the length of the data?
e. Is the packet directed from a client to a server or vice versa?
f. What is the client process?

Solution

a. The source port number is the first four hexadecimal digits $(CB84)_{16}$, which means that the source port number is 52,100.
b. The destination port number is the second four hexadecimal digits $(000D)_{16}$, which means that the destination port number is 13.
c. The third four hexadecimal digits $(001C)_{16}$ define the length of the whole UDP packet as 28 bytes.
d. The length of the data is the length of the whole packet minus the length of the header, or $28 - 8 = 20$ bytes.
e. Because the destination port number is 13 (well-known port), the packet is from the client to the server.
f. The client process is the Daytime (see Table 9.1).

9.3.1 UDP Services

Earlier we discussed the general services provided by a transport-layer protocol. In this section, we discuss what portions of those general services are provided by UDP.

Process-to-Process Communication

UDP provides process-to-process communication using socket addresses, which are a combination of IP addresses and port numbers.

Connectionless Services

As mentioned previously, UDP provides a *connectionless service*. This means that each user datagram sent by UDP is an independent datagram. There is no relationship between the different user datagrams even if they are coming from the same source process and going to the same destination program. The user datagrams are not numbered. Also, unlike TCP, there is no connection establishment and no connection termination. This means that each user datagram can travel on a different path.

One of the ramifications of being connectionless is that the process that uses UDP cannot send a stream of data to UDP and expect UDP to chop them into different, related user datagrams. Instead each request must be small enough to fit into one user datagram. Only those processes sending short messages, messages less than 65,507 bytes (65,535 minus 8 bytes for the UDP header and minus 20 bytes for the IP header), can use UDP.

Flow Control

UDP is a very simple protocol. There is no *flow control*, and hence no window mechanism. The receiver may overflow with incoming messages. The lack of flow control means that the process using UDP should provide for this service, if needed.

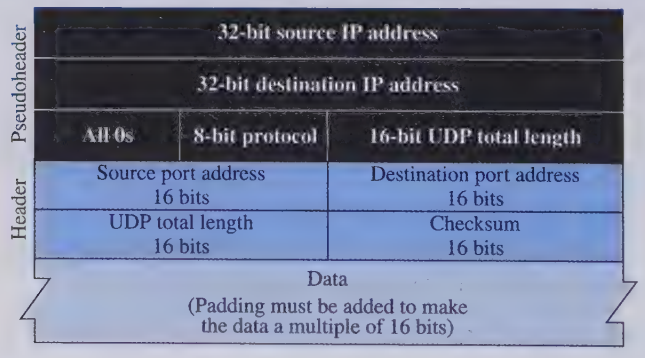
Error Control

There is no *error-control* mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated. When the receiver detects an error through the checksum, the user datagram is silently discarded. The lack of error control means that the process using UDP should provide for this service, if needed.

Checksum

We discuss checksum and its calculation in Appendix F. UDP checksum calculation includes three sections: a pseudoheader, the UDP header, and the data coming from the application layer. The *pseudoheader* is the part of the header of the IP packet (discussed in Chapter 8) in which the user datagram is to be encapsulated with some fields filled with 0s (see Figure 9.19).

Figure 9.19 *Pseudoheader for checksum calculation*



If the checksum does not include the pseudoheader, a user datagram may arrive safe and sound. However, if the IP header is corrupted, it may be delivered to the wrong host.

The protocol field is added to ensure that the packet belongs to UDP, and not to TCP. This means that if a process can use either UDP or TCP, the destination port

number can be the same. The value of the protocol field for UDP is 17. If this value is changed during transmission, the checksum calculation at the receiver will detect it and UDP drops the packet. It is not delivered to the wrong protocol.

Optional Inclusion of Checksum

The sender of a UDP packet can choose not to calculate the checksum. In this case, the checksum field is filled with all 0s before being sent. In the situation where the sender decides to calculate the checksum, but it happens that the result is all 0s, the checksum is changed to all 1s before the packet is sent. In other words, the sender complements the sum 2 times. Note that this does not create confusion because the value of checksum is never all 1s in a normal situation (see Example 9.3).

Example 9.3

What value is sent for the checksum in each of the following hypothetical situations?

a. The sender decides not to include the checksum.
b. The sender decides to include the checksum, but the value of the sum is all 1s.
c. The sender decides to include the checksum, but the value of the sum is all 0s.

Solution

a. The value sent for the checksum field is all 0s to show that the checksum is not calculated.
b. When the sender complements the sum, the result is all 0s; the sender complements the result again before sending. The value sent for the checksum is all 1s. The second complement operation is needed to avoid confusion with the case in part a.
c. This situation never happens because it implies that the value of every term included in the calculation of the sum is all 0s, which is impossible; some fields in the pseudoheader have nonzero values.

Congestion Control

Because UDP is a connectionless protocol, it does not provide congestion control. UDP assumes that the packets sent are small and sporadic and cannot create congestion in the network. This assumption may or may not be true today, when UDP is used for interactive real-time transfer of audio and video.

Encapsulation and Decapsulation

To send a message from one process to another, UDP encapsulates and decapsulates messages.

Queuing

We have talked about ports without discussing the actual implementation of them. In UDP, queues are associated with ports.

At the client site, when a process starts, it requests a port number from the operating system. Some implementations create both an incoming and an outgoing queue associated with each process. Other implementations create only an incoming queue associated with each process.

Multiplexing and Demultiplexing

In a host running a TCP/IP protocol suite, there is only one UDP but possibly several processes that may want to use the services of UDP. To handle this situation, UDP multiplexes and demultiplexes.

Comparison between UDP and Generic Simple Protocol

We can compare UDP with the connectionless simple protocol we discussed earlier. The only difference is that UDP provides an optional checksum to detect corrupted packets at the receiver site. If the checksum is added to the packet, the receiving UDP can check the packet and discard the packet if it is corrupted. No feedback, however, is sent to the sender.

UDP is an example of the connectionless simple protocol we discussed earlier with the exception of an optional checksum added to packets for error detection.

9.3.2 UDP Applications

Although UDP meets almost none of the criteria we mentioned in Section 9.3.1 for a reliable transport-layer protocol, UDP is preferable for some applications. The reason is that some services may have some side effects that are either unacceptable or not preferable. An application designer sometimes needs to compromise to get the optimum. For example, in our daily life, we all know that a one-day delivery of a package by a carrier is more expensive than a three-day delivery. Although time and cost are both desirable features in delivery of a parcel, they are in conflict with each other. We need to choose the optimum.

Example 9.4

A client/server application such as DNS (see Chapter 10) uses the services of UDP because a client needs to send a short request to a server and to receive a quick response from it. The request and response can each fit in one user datagram. Because only one message is exchanged in each direction, the connectionless feature is not an issue; the client or server does not worry that messages are delivered out of order.

Example 9.5

A client/server application such as SMTP (see Chapter 10), which is used in electronic mail, cannot use the services of UDP because a user can send a long e-mail message, which may include multimedia (images, audio, or video). If the application uses UDP and the message does not fit in one single user datagram, the message must be split by the application into different user datagrams. Here the connectionless service may create problems. The user datagrams may arrive and be delivered to the receiver application out of order. The receiver application may not be able to reorder the pieces. This means the connectionless service has a disadvantage for an application program that sends long messages. In SMTP, when one sends a message, one does not expect to receive a response quickly (sometimes no response is required). This means that the extra delay inherent in connection-oriented service is not crucial for SMTP.

Example 9.6

Assume we are downloading a very large text file from the Internet. We definitely need to use a transport layer that provides reliable service. We don't want part of the file to be missing or corrupted when we open the file. The delay created between the deliveries of the parts is not an overriding concern for us; we wait until the whole file is composed before looking at it. In this case, UDP is not a suitable transport layer.

Example 9.7

Assume we are using a real-time interactive application, such as Skype. Audio and video are divided into frames and sent one after another. If the transport layer is supposed to resend a corrupted or lost frame, the synchronizing of the whole transmission may be lost. The viewer suddenly sees a blank screen and needs to wait until the second transmission arrives. This is not tolerable. However, if each small part of the screen is sent using one single user datagram, the receiving UDP can easily ignore the corrupted or lost packet and deliver the rest to the application program. That part of the screen is blank for a very short period of time, which most viewers do not even notice.

Typical Applications

The following shows some typical applications that can benefit more from the services of UDP than from those of TCP.

- UDP is suitable for a process that requires simple request-response communication with little concern for flow and error control. It is not usually used for a process such as FTP that needs to send bulk data (see Chapter 10).
- UDP is a suitable transport protocol for multicasting. Multicasting capability is embedded in the UDP software but not in the TCP software.
- UDP is used for management processes such as SNMP (see Chapter 12).
- UDP is used for some route updating protocols such as Routing Information Protocol (RIP) (see Chapter 8).
- UDP is normally used for interactive real-time applications that cannot tolerate uneven delay between sections of a received message (See Chapter 10).

9.4 TRANSMISSION CONTROL PROTOCOL

Transmission Control Protocol (TCP) is a connection-oriented, reliable protocol. TCP explicitly defines connection establishment, data transfer, and connection teardown phases to provide a connection-oriented service. TCP uses a combination of the **Go-Back-*N* (GBN)** and **Selective-Repeat (SR) protocols** to provide reliability. To achieve this goal, TCP uses checksum (for error detection), retransmission of lost or corrupted packets, cumulative and selective acknowledgments, and timers. In this section, we first discuss the services provided by TCP; we then discuss the TCP features in more detail. TCP is the most common transport-layer protocol in the Internet.

9.4.1 TCP Services

Before discussing TCP in detail, let us explain the services offered by TCP to the processes at the application layer.

Process-to-Process Communication

As with UDP, TCP provides process-to-process communication using port numbers. We have already given some of the port numbers used by TCP in Table 9.1 in Section 9.2.2.

Stream Delivery Service

TCP, unlike UDP, is a stream-oriented protocol. In UDP, a process sends messages with predefined boundaries to UDP for delivery. UDP adds its own header to each of these messages and delivers it to IP for transmission. Each message from the process is called a *user datagram*, and becomes, eventually, one IP datagram. Neither IP nor UDP recognizes any relationship between the datagrams.

TCP, on the other hand, allows the sending process to deliver data as a stream of bytes and allows the receiving process to obtain data as a stream of bytes. TCP creates an environment in which the two processes seem to be connected by an imaginary "tube" that carries their bytes across the Internet. This imaginary environment is depicted in Figure 9.20. The sending process produces (writes to) the stream, and the receiving process consumes (reads from) it.

Figure 9.20 *Stream delivery*

Sending and Receiving Buffers

Because the sending and the receiving processes may not necessarily write or read data at the same rate, TCP needs buffers for storage. There are two buffers, the sending buffer and the receiving buffer, one for each direction. One way to implement a buffer is to use a circular array of 1-byte locations as shown in Figure 9.21. For simplicity, we have shown two buffers of 20 bytes each; normally the buffers are hundreds or thousands of bytes, depending on the implementation. We also show the buffers as the same size, which is not always the case.

Figure 9.21 *Sending and receiving buffers*

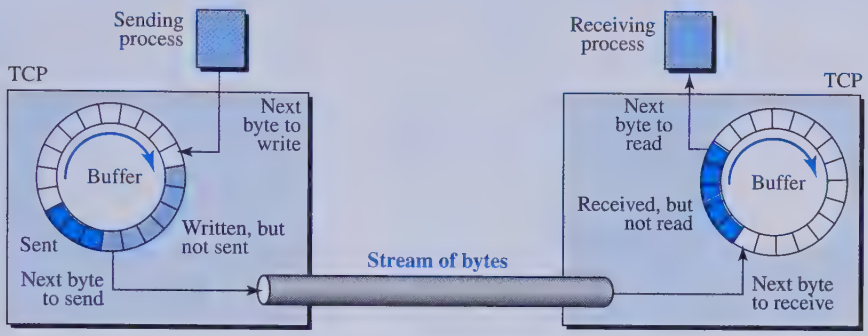


Figure 9.21 shows the movement of the data in one direction. At the sender, the buffer has three types of chambers. The white section contains empty chambers that can be filled by the sending process (producer). The colored area holds bytes that have been sent but not yet acknowledged. The TCP sender keeps these bytes in the buffer until it receives an acknowledgment. The shaded area contains bytes to be sent by the sending TCP. However, TCP may be able to send only part of this shaded section. This could be due to the slowness of the receiving process or congestion in the network. Also note that, after the bytes in the colored chambers are acknowledged, the chambers are recycled and available for use by the sending process. This is why we show a circular buffer.

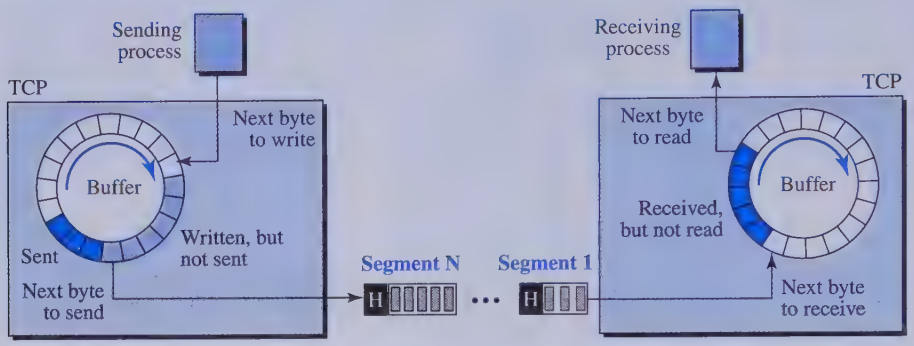
The operation of the buffer at the receiver is simpler. The circular buffer is divided into two areas (shown as white and colored in Figure 9.21). The white area contains empty chambers to be filled by bytes received from the network. The colored sections contain received bytes that can be read by the receiving process. When a byte is read by the receiving process, the chamber is recycled and added to the pool of empty chambers.

Segments

Although buffering handles the disparity between the speed of the producing and consuming processes, we need one more step before we can send data. The network layer, as a service provider for TCP, needs to send data in packets, not as a stream of bytes. At the transport layer, TCP groups a number of bytes together into a packet called a *segment*. TCP adds a header to each segment (for control purposes) and delivers the segment to the network layer for transmission. The segments are encapsulated in an IP datagram and transmitted. This entire operation is transparent to the receiving process. Segments may be received out of order, lost, or corrupted and re-sent. All these are handled by the TCP receiver with the receiving application process unaware of the TCP's activities. Figure 9.22 shows how segments are created from the bytes in the buffers.

Note that segments are not necessarily all the same size. In Figure 9.22, for simplicity, we show one segment carrying 3 bytes and the other carrying 5 bytes. In reality, segments carry hundreds, if not thousands, of bytes.

Figure 9.22 *TCP segments*



Full-Duplex Communication

TCP offers *full-duplex service*, where data can flow in both directions at the same time. Each TCP endpoint then has its own sending and receiving buffer, and segments move in both directions.

Multiplexing and Demultiplexing

Like UDP, TCP performs multiplexing at the sender and demultiplexing at the receiver. However, because TCP is a connection-oriented protocol, a connection needs to be established for each pair of processes.

Connection-Oriented Service

TCP, unlike UDP, is a connection-oriented protocol. When a process at site A wants to send to and receive data from another process at site B, the following three phases occur:

1. The two TCP's establish a logical connection between them.
2. Data are exchanged in both directions.
3. The connection is terminated.

Note that this is a logical connection, not a physical connection. The TCP segment is encapsulated in an IP datagram and can be sent out of order, or lost, or corrupted, and then re-sent. Each may be routed over a different path to reach the destination. There is no physical connection. TCP creates a stream-oriented environment in which it accepts the responsibility of delivering the bytes in order to the other site.

Reliable Service

TCP is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data. We will discuss this feature further in the section on error control (Section 9.4.8).

9.4.2 TCP Features

To provide the services mentioned in Section 9.4.1, TCP has several features that are briefly summarized in this section and discussed later in detail.

Numbering System

Although the TCP software keeps track of the segments being transmitted or received, there is no field for a segment number value in the segment header. Instead, there are two fields, called the *sequence number* and *acknowledgment number*, respectively. These two fields refer to a byte number and not a segment number.

Byte Number

TCP numbers all data bytes (octets) that are transmitted in a connection. Numbering is independent in each direction. When TCP receives bytes of data from a process, TCP stores them in the sending buffer and numbers them. The numbering does not necessarily start from 0. Instead, TCP chooses an arbitrary number between 0 and $2^{32} - 1$ for the number of the first byte. For example, if the number happens to be 1057 and the total data to be sent are 6000 bytes, the bytes are numbered from 1057 to 7056. We will see that byte numbering is used for flow and error control.

The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with an arbitrarily generated number.

Sequence Number

After the bytes have been numbered, TCP assigns a sequence number to each segment that is being sent. The sequence number, in each direction, is defined as follows:

1. The sequence number of the first segment is the initial sequence number (ISN), which is a random number.
2. The sequence number of any other segment is the sequence number of the previous segment plus the number of bytes (real or imaginary) carried by the previous segment.

Example 9.8

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 10,001. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	→	Sequence number:	10,001	**Range:**	10,001	to	11,000
Segment 2	→	Sequence number:	11,001	**Range:**	11,001	to	12,000
Segment 3	→	Sequence number:	12,001	**Range:**	12,001	to	13,000
Segment 4	→	Sequence number:	13,001	**Range:**	13,001	to	14,000
Segment 5	→	Sequence number:	14,001	**Range:**	14,001	to	15,000

The value in the sequence number field of a segment defines the number assigned to the first data byte contained in that segment.

When a segment carries a combination of data and control information (**piggybacking**), it uses a sequence number. If a segment does not carry user data, it does not logically define a sequence number. The field is there, but the value is not valid. However, some segments, when carrying only control information, need a sequence number to allow an acknowledgment from the receiver. These segments are used for connection establishment, termination, or abortion. Each of these segments consume one sequence number as though it carries 1 byte, but there are no actual data. We will elaborate on this issue when we discuss connections.

Acknowledgment Number

As we discussed previously, communication in TCP is full duplex; when a connection is established, both parties can send and receive data at the same time. Each party numbers the bytes, usually with a different starting byte number. The sequence number in each direction shows the number of the first byte carried by the segment. Each party also uses an acknowledgment number to confirm the bytes it has received. However, the acknowledgment number defines the number of the next byte that the party expects to receive. In addition, the acknowledgment number is cumulative, which means that the party takes the number of the last byte that it has received, safe and sound, adds 1 to it, and announces this sum as the acknowledgment number. The term *cumulative* here means that if a party uses 5643 as an acknowledgment number, it has received all bytes from the beginning up to 5642. Note that this does not mean that the party has received 5642 bytes, because the first byte number does not have to be 0.

The value of the acknowledgment field in a segment defines the number of the next byte a party expects to receive. The acknowledgment number is cumulative.

9.4.3 Segment

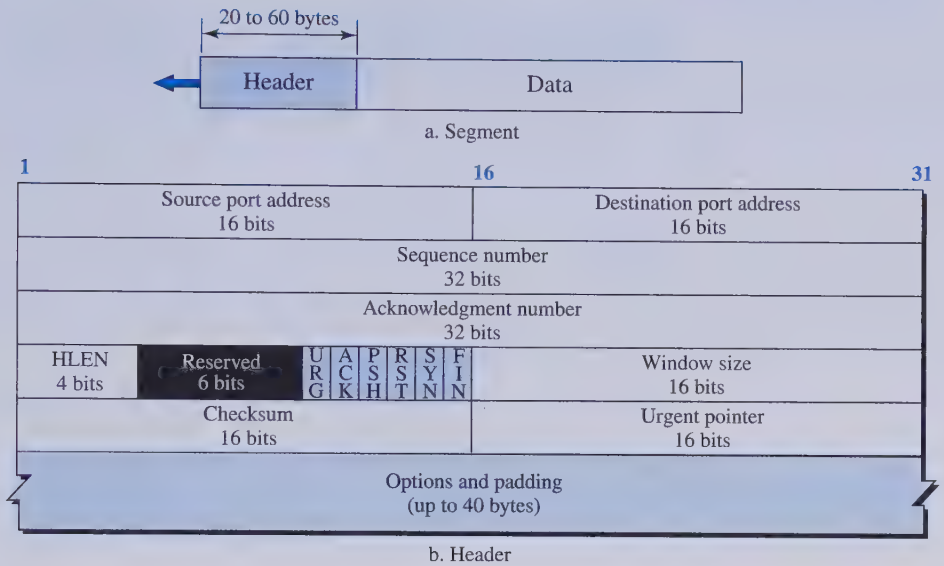
Before discussing TCP in more detail, let us discuss the TCP packets themselves. A packet in TCP is called a **segment**.

Format

The format of a segment is shown in Figure 9.23. The segment consists of a header of 20 to 60 bytes, followed by data from the application program. The header is 20 bytes if there are no options and up to 60 bytes if it contains options. We will discuss some of the header fields in this section. The meaning and purpose of these will become clearer as we proceed through the section.

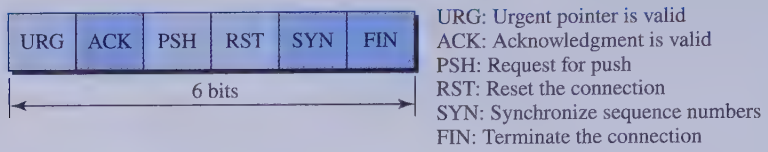
- **Source port address.** This is a 16-bit field that defines the port number of the application program in the host that is sending the segment.
- **Destination port address.** This is a 16-bit field that defines the port number of the application program in the host that is receiving the segment.

Figure 9.23 *TCP segment format*



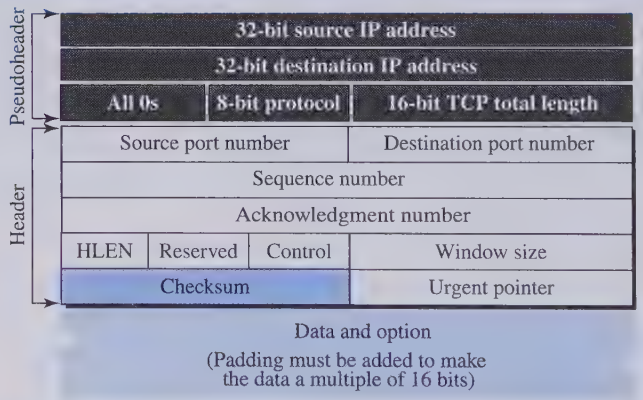
- **Sequence number.** This 32-bit field defines the number assigned to the first byte of data contained in this segment. As we said before, TCP is a stream transport protocol. To ensure connectivity, each byte to be transmitted is numbered. The sequence number tells the destination which byte in this sequence is the first byte in the segment. During connection establishment (discussed in Section 9.4.4) each party uses a random number generator to create an **initial sequence number (ISN)**, which is usually different in each direction.
- **Acknowledgment number.** This 32-bit field defines the byte number that the receiver of the segment is expecting to receive from the other party. If the receiver of the segment has successfully received byte number x from the other party, it returns $x + 1$ as the acknowledgment number. Acknowledgment and data can be piggybacked together.
- **Header length.** This 4-bit field indicates the number of 4-byte words in the TCP header. The length of the header can be between 20 and 60 bytes. Therefore, the value of this field is always between 5 ($5 \times 4 = 20$) and 15 ($15 \times 4 = 60$).
- **Control.** This field defines six different control bits or flags, as shown in Figure 9.24. One or more of these bits can be set at a time. These bits enable flow control, connection establishment and termination, connection abortion, and the mode of data transfer in TCP. A brief description of each bit is shown in Figure 9.24. We will discuss them further when we study the detailed operation of TCP.

Figure 9.24 *Control field*



- **Window size.** This field defines the window size of the sending TCP in bytes. Note that the length of this field is 16 bits, which means that the maximum size of the window is 65,535 bytes. This value is normally referred to as the receiving window (*rwnd*) and is determined by the receiver. The sender must obey the dictation of the receiver in this case.
- **Checksum.** This 16-bit field contains the checksum. The calculation of the checksum for TCP follows the same procedure as the one described for UDP. However, the use of the checksum in the UDP datagram is optional, whereas the use of the checksum for TCP is mandatory. The same pseudoheader, serving the same purpose, is added to the segment. For the TCP pseudoheader, the value for the protocol field is 6. See Figure 9.25.

Figure 9.25 *Pseudoheader added to the TCP datagram*



The use of the checksum in TCP is mandatory.

- **Urgent pointer.** This 16-bit field, which is valid only if the urgent flag is set, is used when the segment contains urgent data. It defines a value that must be added to the sequence number to obtain the number of the last urgent byte in the data section of the segment.

- **Options.** There can be up to 40 bytes of optional information in the TCP header. We will discuss some of the options used in the TCP header later in this section.

Encapsulation

A TCP segment encapsulates the data received from the application layer. The TCP segment is encapsulated in an IP datagram, which in turn is encapsulated in a frame at the data-link layer.

9.4.4 A TCP Connection

TCP is connection-oriented. As discussed before, a connection-oriented transport protocol establishes a logical path between the source and destination. All the segments belonging to a message are then sent over this logical path. Using a single logical pathway for the entire message facilitates the acknowledgment process as well as retransmission of damaged or lost frames. You may wonder how TCP, which uses the services of IP, a connectionless protocol, can be connection-oriented. The point is that a TCP connection is logical, not physical. TCP operates at a higher level. TCP uses the services of IP to deliver individual segments to the receiver, but it controls the connection itself. If a segment is lost or corrupted, it is retransmitted. Unlike TCP, IP is unaware of this retransmission. If a segment arrives out of order, TCP holds it until the missing segments arrive; IP is unaware of this reordering.

In TCP, connection-oriented transmission requires three phases: connection establishment, data transfer, and connection termination.

Connection Establishment

TCP transmits data in full-duplex mode. When two TCPs in two machines are connected, they are able to send segments to each other simultaneously. This implies that each party must initialize communication and get approval from the other party before any data are transferred.

Three-Way Handshaking

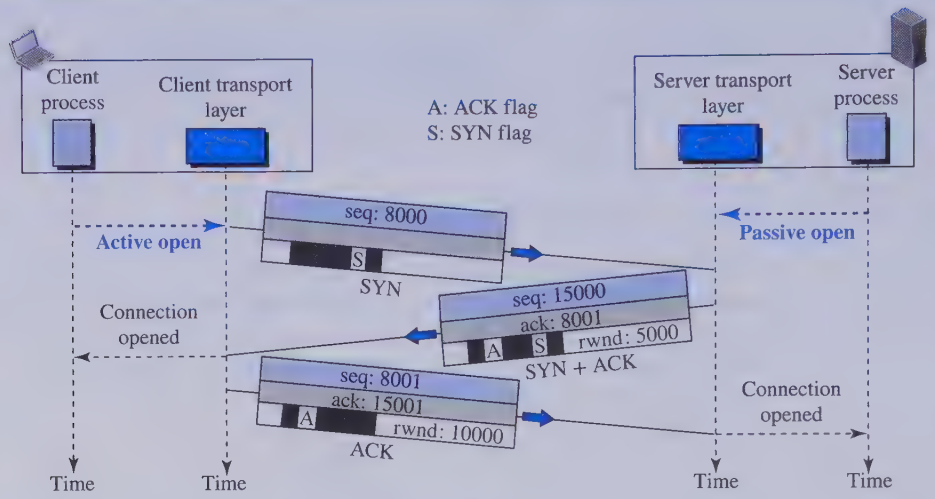
The connection establishment in TCP is called **three-way handshaking**. In our example, an application program, called the client, wants to make a connection with another application program, called the server, using TCP as the transport-layer protocol.

The process starts with the server. The server program tells its TCP that it is ready to accept a connection. This request is called a *passive open*. Although the server TCP is ready to accept a connection from any machine in the world, it cannot make the connection itself.

The client program issues a request for an *active open*. A client that wishes to connect to an open server tells its TCP to connect to a particular server. TCP can now start the three-way handshaking process, as shown in Figure 9.26.

To show the process we use time lines. Each segment has values for all its header fields and perhaps for some of its option fields too. However, we show only the few fields necessary to understand each phase. We show the sequence number, the

Figure 9.26 *Connection establishment using three-way handshaking*



acknowledgment number, the control flags (only those that are set), and window size if relevant. The three steps in this phase are as follows:

1. The client sends the first segment, a SYN segment, in which only the SYN flag is set. This segment is for synchronization of sequence numbers. The client in our example chooses a random number as the first sequence number and sends this number to the server. This sequence number is called the initial sequence number (ISN). Note that this segment does not contain an acknowledgment number. It does not define the window size either; a window size definition makes sense only when a segment includes an acknowledgment. Note that the SYN segment is a control segment and carries no data. However, it consumes one sequence number because it needs to be acknowledged. We can say that the SYN segment carries 1 imaginary byte.

A SYN segment cannot carry data, but it consumes one sequence number.

2. The server sends the second segment, a SYN + ACK segment with two flag bits set as SYN and ACK. This segment has a dual purpose. First, it is a SYN segment for communication in the other direction. The server uses this segment to initialize a sequence number for numbering the bytes sent from the server to the client. The server also acknowledges the receipt of the SYN segment from the client by setting the ACK flag and displaying the next sequence number it expects to receive from the client. Because it contains an acknowledgment, it also needs to define the receive window size, *rwnd* (to be used by the client), as we will see in the flow-control section. Because this segment is playing the role of a SYN segment, it needs to be acknowledged. It, therefore, consumes one sequence number.

A SYN + ACK segment cannot carry data, but it does consume one sequence number.

3. The client sends the third segment. This is just an ACK segment. It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field. Note that the ACK segment does not consume any sequence numbers if it does not carry data, but some implementations allow this third segment in the connection phase to carry the first chunk of data from the client. In this case, the segment consumes as many sequence numbers as the number of data bytes.

If an ACK segment does not carry any data, it does not consume any sequence numbers.

SYN Flooding Attack

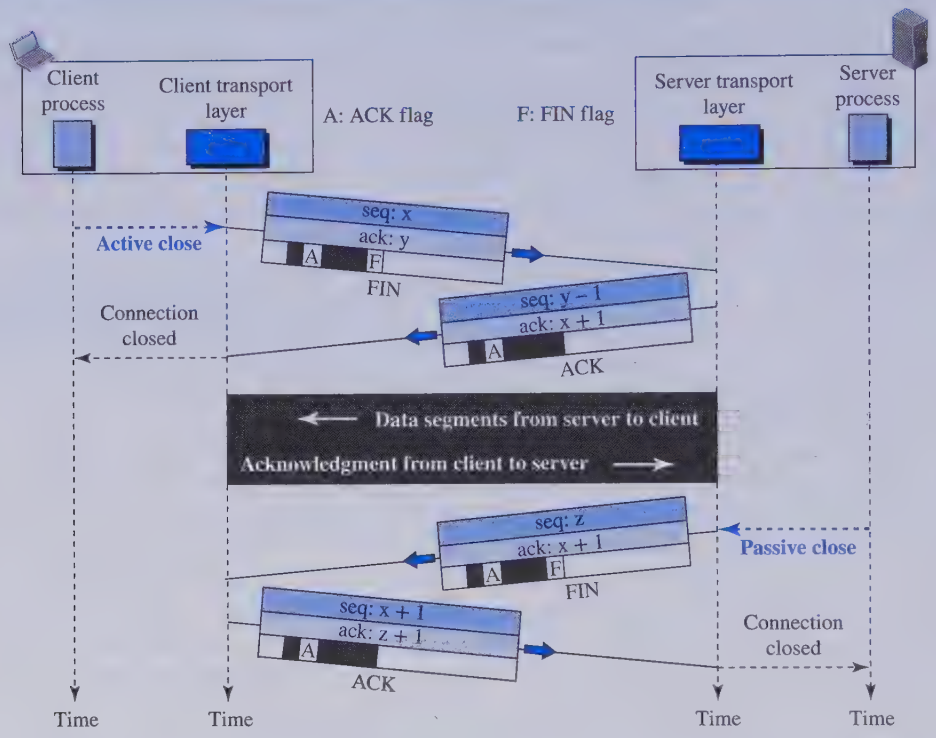
The connection establishment procedure in TCP is susceptible to a serious security problem called a **SYN flooding attack**. This happens when one or more malicious attackers send a large number of SYN segments to a server pretending that each of them is coming from a different client by faking the source IP addresses in the datagrams. The server, assuming that the clients are issuing an active open, allocates the necessary resources, such as creating transfer control block (TCB) tables and setting timers. The TCP server then sends the SYN + ACK segments to the fake clients, which are lost. When the server waits for the third leg of the handshaking process, however, resources are allocated without being used. If, during this short period of time, the number of SYN segments is large, the server eventually runs out of resources and may be unable to accept connection requests from valid clients. This SYN flooding attack belongs to a group of security attacks known as a **denial of service attack**, in which an attacker monopolizes a system with so many service requests that the system overloads and denies service to valid requests.

Some implementations of TCP have strategies to alleviate the effect of a SYN attack. Some have imposed a limit of connection requests during a specified period of time. Others try to filter out datagrams coming from unwanted source addresses. One recent strategy is to postpone resource allocation until the server can verify that the connection request is coming from a valid IP address, by using what is called a **cookie**. SCTP, the new transport-layer protocol that we discuss in Section 9.5 uses this strategy.

Data Transfer

After a connection is established, bidirectional data transfer can take place. The client and server can send data and acknowledgments in both directions. We will study the rules of acknowledgment later in the chapter; for the moment, it is enough to know that data traveling in the same direction as an acknowledgment are carried on the same segment. The acknowledgment is piggybacked with the data. Figure 9.27 shows an example.

Figure 9.27 *Data transfer*



In this example, after a connection is established, the client sends 2000 bytes of data in two segments. The server then sends 2000 bytes in one segment. The client sends one more segment. The first three segments carry both data and acknowledgment, but the last segment carries only an acknowledgment because there is no more data to be sent. Note the values of the sequence and acknowledgment numbers. The data segments sent by the client have the PSH (push) flag set so that the server TCP knows to deliver data to the server process as soon as they are received. We discuss the use of this flag in more detail later. The segment from the server, on the other hand, does not set the push flag. Most TCP implementations have the option to set or not to set this flag.

Pushing Data

We saw that the sending TCP uses a buffer to store the stream of data coming from the sending application program. The sending TCP can select the segment size. The receiving TCP also buffers the data when they arrive and delivers them to the application program when the application program is ready or when it is convenient for the receiving TCP. This type of flexibility increases the efficiency of TCP.

However, there are occasions in which the application program has no need for this flexibility. For example, consider an application program that communicates

interactively with another application program on the other end. The application program on one site wants to send a chunk of data to the application at the other site and receive an immediate response. Delayed transmission and delayed delivery of data may not be acceptable by the application program.

TCP can handle such a situation. The application program at the sender can request a *push* operation. This means that the sending TCP must not wait for the window to be filled. It must create a segment and send it immediately. The sending TCP must also set the push bit (PSH) to let the receiving TCP know that the segment includes data that must be delivered to the receiving application program as soon as possible and not to wait for more data to come. This means to change the byte-oriented TCP to a chunk-oriented TCP, but TCP can choose whether or not to use this feature.

Urgent Data

TCP is a stream-oriented protocol. This means that the data are presented from the application program to TCP as a stream of bytes. Each byte of data has a position in the stream. However, there are occasions in which an application program needs to send *urgent* bytes, some bytes that need to be treated in a special way by the application at the other end. The solution is to send a segment with the URG bit set. The sending application program tells the sending TCP that the piece of data is urgent. The sending TCP creates a segment and inserts the urgent data at the beginning of the segment. The rest of the segment can contain normal data from the buffer. The urgent pointer field in the header defines the end of the urgent data (the last byte of urgent data). For example, if the segment sequence number is 15,000 and the value of the urgent pointer is 200, the first byte of urgent data is the byte 15,000 and the last byte is the byte 15,200. The rest of the bytes in the segment (if present) are nonurgent.

It is important to mention that TCP's urgent mode is neither a priority service nor an out-of-band data service as some people think. Rather, TCP urgent mode is a service by which the application program at the sender side marks some portion of the byte stream as needing special treatment by the application program at the receiver side. The receiving TCP delivers bytes (urgent or nonurgent) to the application program in order, but inform the application program about the beginning and end of urgent data. It is left to the application program to decide what to do with the urgent data.

Connection Termination

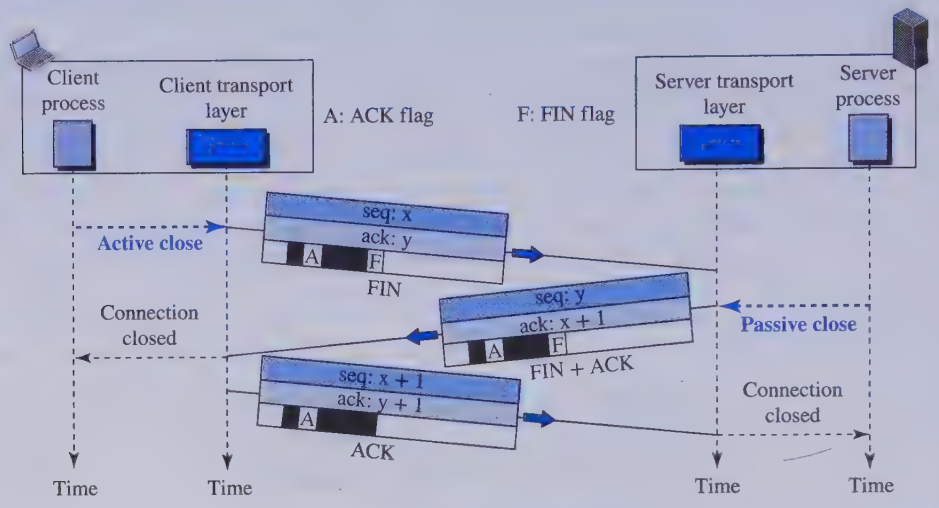
Either of the two parties involved in exchanging data (client or server) can close the connection, although it is usually initiated by the client. Most implementations today allow two options for connection termination: three-way handshaking and four-way handshaking with a half-close option.

Three-Way Handshaking

Most implementations today allow *three-way handshaking* for connection termination, as shown in Figure 9.28.

1. In this situation, the client TCP, after receiving a close command from the client process, sends the first segment, a FIN segment in which the FIN flag is set. Note that a FIN segment can include the last chunk of data sent by the client or it can be

Figure 9.28 *Connection termination using three-way handshaking*



just a control segment as shown in Figure 9.28. If it is only a control segment, it consumes only one sequence number because it needs to be acknowledged.

The FIN segment consumes one sequence number if it does not carry data.

2. The server TCP, after receiving the FIN segment, informs its process of the situation and sends the second segment, a FIN + ACK segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction. This segment can also contain the last chunk of data from the server. If it does not carry data, it consumes only one sequence number because it needs to be acknowledged.
3. The client TCP sends the last segment, an ACK segment, to confirm the receipt of the FIN segment from the TCP server. This segment contains the acknowledgment number, which is one plus the sequence number received in the FIN segment from the server. This segment cannot carry data and consumes no sequence numbers.

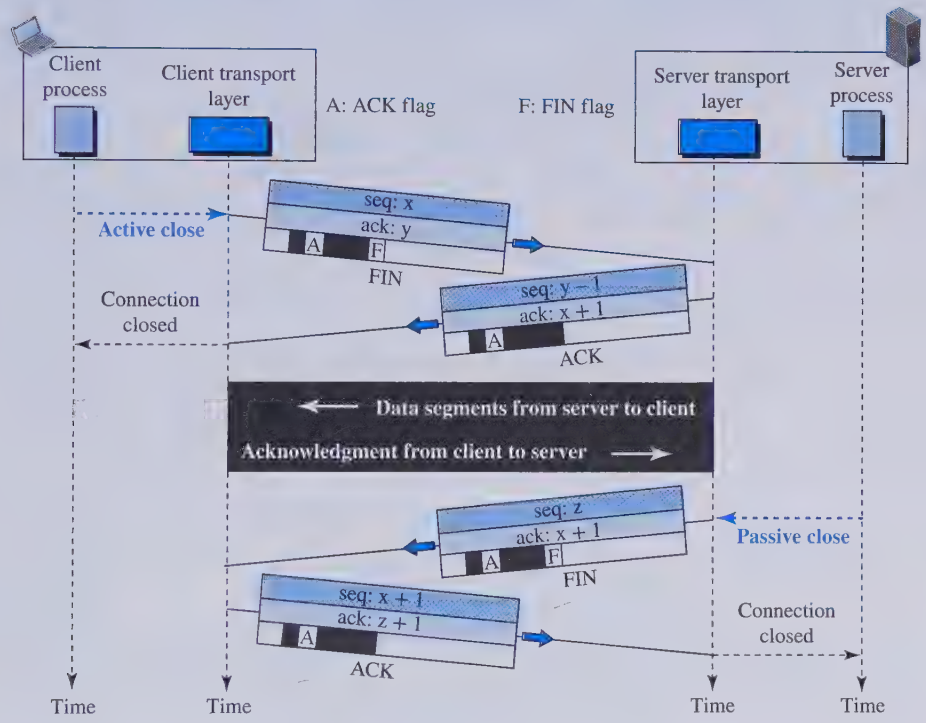
The FIN + ACK segment consumes only one sequence number if it does not carry data.

Half-Close

In TCP, one end can stop sending data while still receiving data. This is called a **half-close**. Either the server or the client can issue a half-close request. It can occur when the server needs all the data before processing can begin. A good example is sorting. When the client sends data to the server to be sorted, the server needs to receive all the data before sorting can start. This means the client, after sending all data, can close the

connection in the client-to-server direction. However, the server-to-client direction must remain open to return the sorted data. The server, after receiving the data, still needs time for sorting; its outbound direction must remain open. Figure 9.29 shows an example of a half-close.

Figure 9.29 *Half-close*



The data transfer from the client to the server stops. The client half-closes the connection by sending a FIN segment. The server accepts the half-close by sending the ACK segment. The server, however, can still send data. When the server has sent all the processed data, it sends a FIN segment, which is acknowledged by an ACK from the client.

After half-closing the connection, data can travel from the server to the client and acknowledgments can travel from the client to the server. The client cannot send any more data to the server.

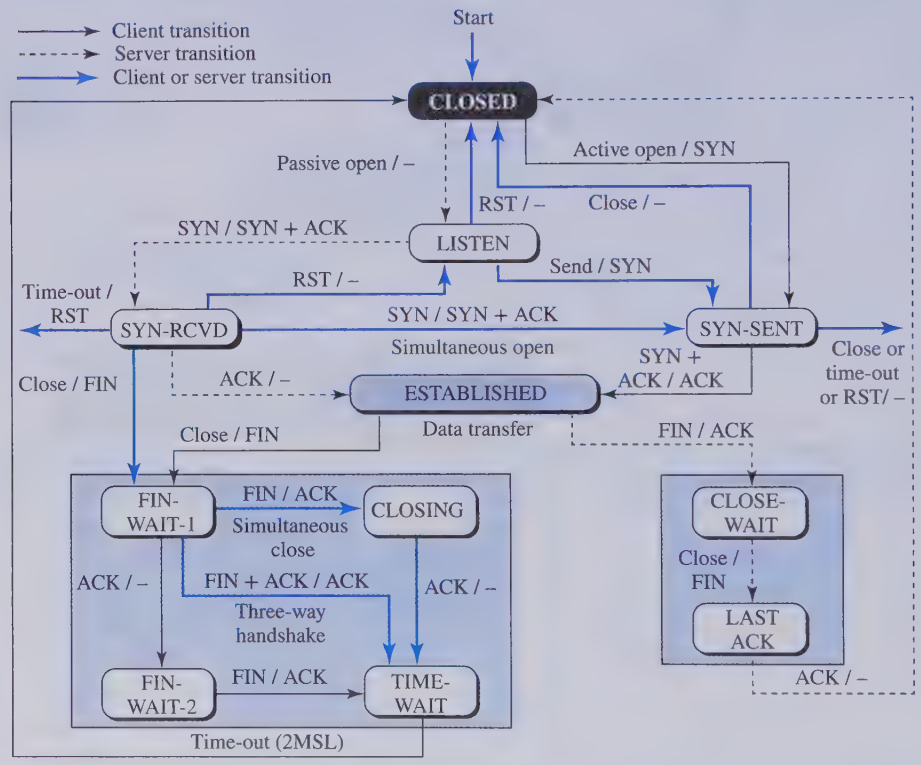
Connection Reset

TCP at one end may deny a connection request, may abort an existing connection, or may terminate an idle connection. All these are done with the RST (reset) flag.

9.4.5 State Transition Diagram

To keep track of all the different events happening during connection establishment, connection termination, and data transfer, TCP is specified as the finite state machine (FSM) as shown in Figure 9.30.

Figure 9.30 *State transition diagram*



The figure shows the two FSMs used by the TCP client and server combined in one diagram. The rounded-corner rectangles represent the states. The transition from one state to another is shown using directed lines. Each line has two strings separated by a slash. The first string is the input, what TCP receives. The second is the output, what TCP sends. The dotted black lines in the figure represent the transition that a server normally goes through; the solid black lines show the transitions that a client normally goes through. However, in some situations, a server transitions through a solid line or a client transitions through a dotted line. The colored lines show special situations. Note that the rounded-corner rectangle marked as ESTABLISHED is in fact two sets of states, a

set for the client and another for the server, that are used for flow and error control. We will discuss some timers mentioned in Figure 9.30, including the 2MSL timer, at the end of the chapter. We use several scenarios based on Figure 9.30 and show the part of the figure in each case.

Table 9.2 shows the list of states for TCP.

Table 9.2 *States for TCP*

State	*Description*
CLOSED	No connection exists
LISTEN	Passive open received; waiting for SYN
SYN-SENT	SYN sent; waiting for ACK
SYN-RCVD	SYN + ACK sent; waiting for ACK
ESTABLISHED	Connection established; data transfer in progress
FIN-WAIT-1	First FIN sent; waiting for ACK
FIN-WAIT-2	ACK to first FIN received; waiting for second FIN
CLOSE-WAIT	First FIN received, ACK sent; waiting for application to close
TIME-WAIT	Second FIN received, ACK sent; waiting for 2MSL time-out
LAST-ACK	Second FIN sent; waiting for ACK
CLOSING	Both sides decided to close simultaneously

Scenarios

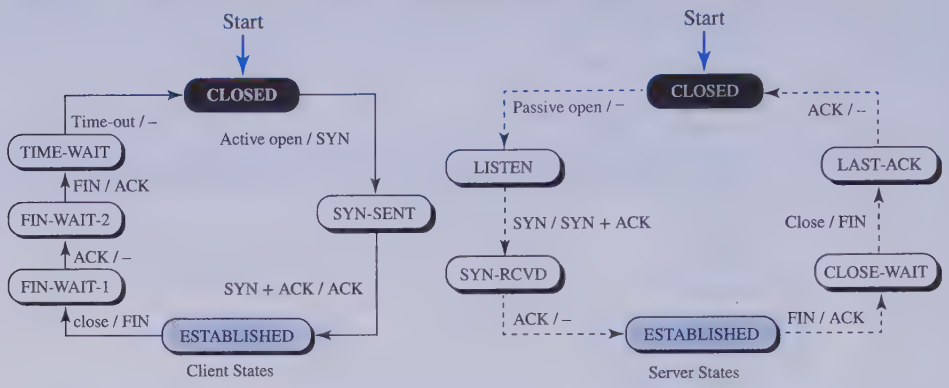
To understand the TCP state machines and the transition diagrams, we go through one scenario in this section.

The state marked as ESTABLISHED in the FSM is in fact two different sets of states that the client and server undergo to transfer data.

A Half-Close Scenario

Figure 9.31 shows the state transition diagram for this scenario. The client process issues an *active open* command to its TCP to request a connection to a specific socket address. TCP sends a SYN segment and moves to the **SYN-SENT** state. After receiving the SYN + ACK segment, TCP sends an ACK segment and goes to the **ESTABLISHED** state. Data are transferred, possibly in both directions, and acknowledged. When the client process has no more data to send, it issues a command called an *active close*. The TCP sends a FIN segment and goes to the **FIN-WAIT-1** state. When it receives the ACK segment, it goes to the **FIN-WAIT-2** state. When the client receives a FIN segment, it sends an ACK segment and goes to the **TIME-WAIT** state. The client remains in this state for 2 MSL seconds (see TCP timers in Section 9.4.10). When the corresponding timer expires, the client goes to the **CLOSED** state.

Figure 9.31 *Transition diagram with half-close connection termination*

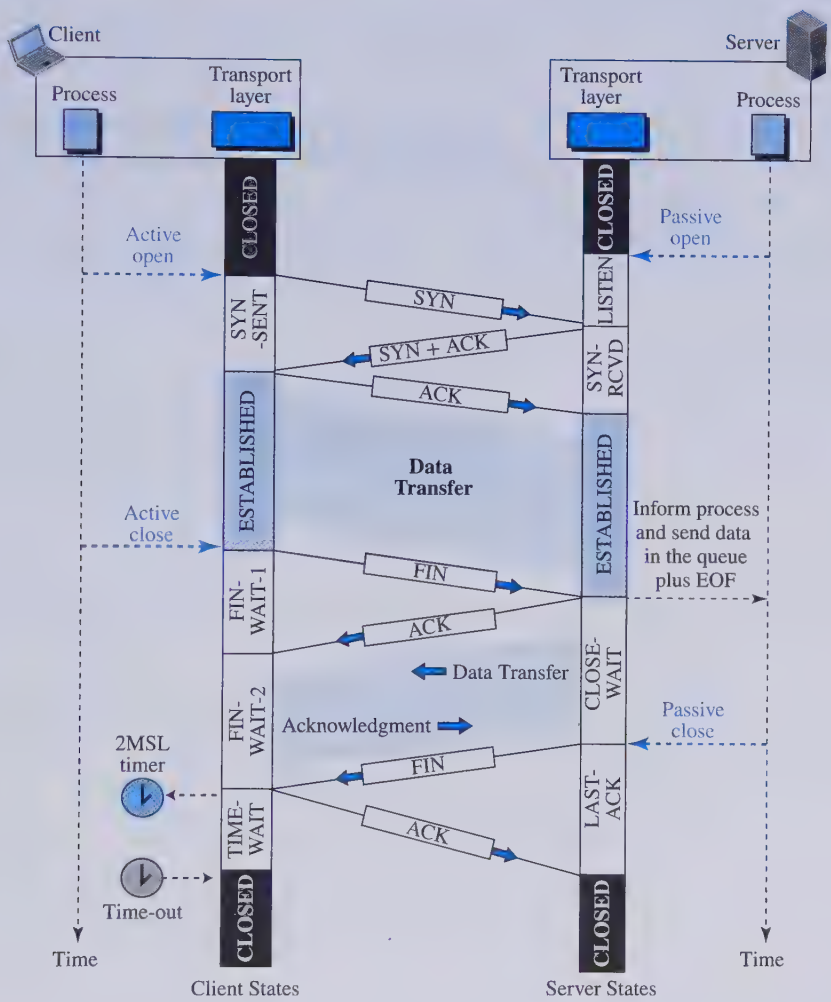


The server process issues a *passive open* command. The server TCP goes to the **LISTEN** state and remains there passively until it receives a SYN segment. The TCP then sends a SYN + ACK segment and goes to the **SYN-RCVD** state, waiting for the client to send an ACK segment. After receiving the ACK segment, TCP goes to the **ESTABLISHED** state, where data transfer can take place. TCP remains in this state until it receives a FIN segment from the client signifying that there are no more data to be exchanged and the connection can be closed. The server, upon receiving the FIN segment, sends all queued data to the server with a virtual end-of-file (EOF) marker, which means that the connection must be closed. It sends an ACK segment and goes to the **CLOSE-WAIT** state, but postpones acknowledging the FIN segment received from the client until it receives a *passive close* command from its process. After receiving the passive close command, the server sends a FIN segment to the client and goes to the **LAST-ACK** state, waiting for the final ACK. When the ACK segment is received from the client, the server goes to the **CLOSE** state. Figure 9.32 shows the same scenario with states over the time line.

9.4.6 Windows in TCP

Before discussing data transfer in TCP and the issues such as flow, error, and congestion control, we describe the TCP windows. TCP uses two windows (send window and receive window) for each direction of data transfer, which means there are four windows for a bidirectional communication. To make the discussion simple, we make an unrealistic assumption that communication is only unidirectional (say from client to server); the bidirectional communication can be inferred using two unidirectional communications with piggybacking.

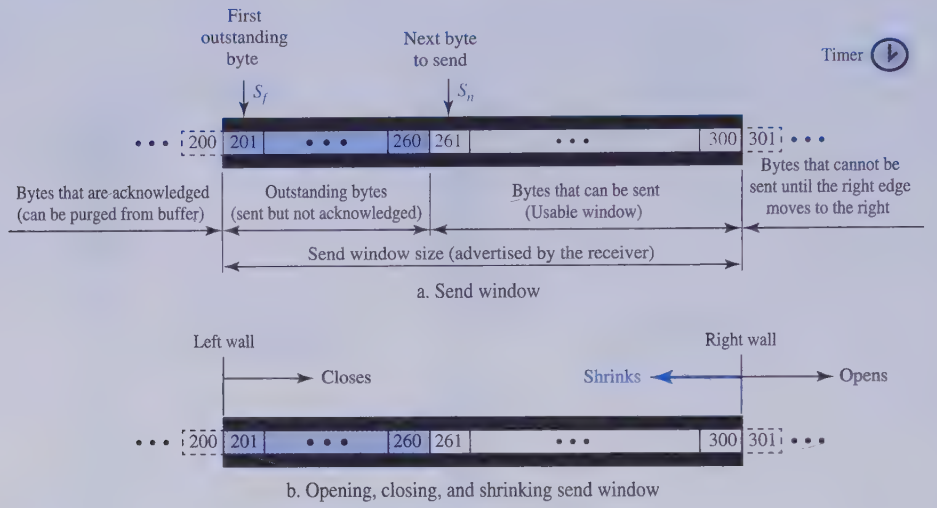
Figure 9.32 *Time-line diagram for a common scenario*



Send Window

Figure 9.33 shows an example of a send window. The window size is 100 bytes, but later we see that the send window size is dictated by the receiver (flow control) and the congestion in the underlying network (congestion control). Figure 9.33 shows how a send window *opens, closes,* or *shrinks.*

Figure 9.33 *Send window in TCP*



The send window in TCP is similar to the one used with the Selective-Repeat protocol, but with some differences:

1. One difference is the nature of entities related to the window. The window size in the SR protocol is the number of packets, but the window size in TCP is the number of bytes. Although actual transmission in TCP occurs segment by segment, the variables that control the window are expressed in bytes.
2. The second difference is that, in some implementations, TCP can store data received from the process and send them later, but we assume that the sending TCP is capable of sending segments of data as soon as it receives them from its process.
3. Another difference is the number of timers. The theoretical SR protocol may use several timers for each packet sent, but as mentioned before, the TCP protocol uses only one timer.

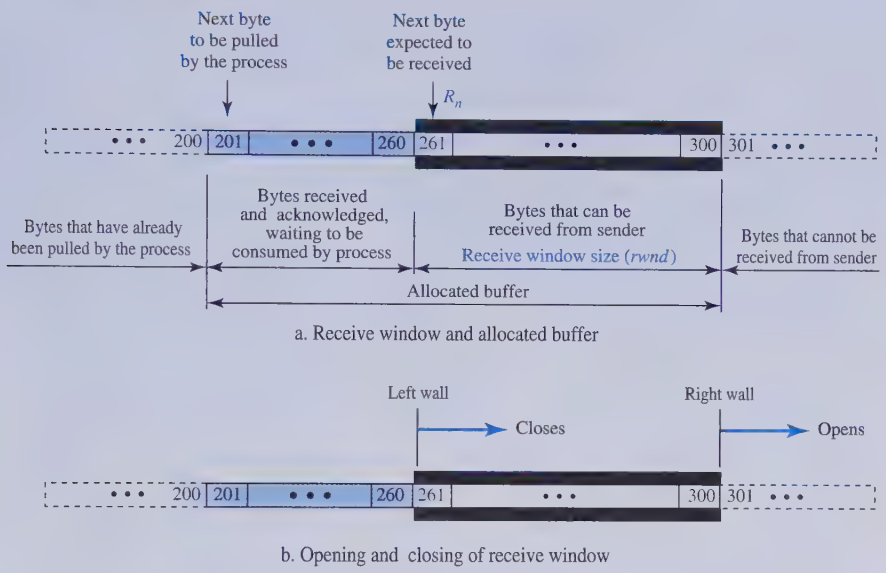
Receive Window

Figure 9.34 shows an example of a receive window. The window size is 100 bytes. The figure also shows how the receive window opens and closes; in practice, the window should never shrink.

There are two differences between the receive window in TCP and the one we used for SR.

1. The first difference is that TCP allows the receiving process to pull data at its own pace. This means that part of the allocated buffer at the receiver may be occupied by bytes that have been received and acknowledged, but are waiting to be pulled by the receiving process. The receive window size is then always smaller or equal to

Figure 9.34 *Receive window in TCP*



a. Receive window and allocated buffer

b. Opening and closing of receive window

the buffer size, as shown in Figure 9.34. The receive window size determines the number of bytes that the receive window can accept from the sender before being overwhelmed (flow control). In other words, the receive window size, normally called *rwnd*, can be determined as:

$$rwnd = \text{buffer size} - \text{number of waiting bytes to be pulled}$$

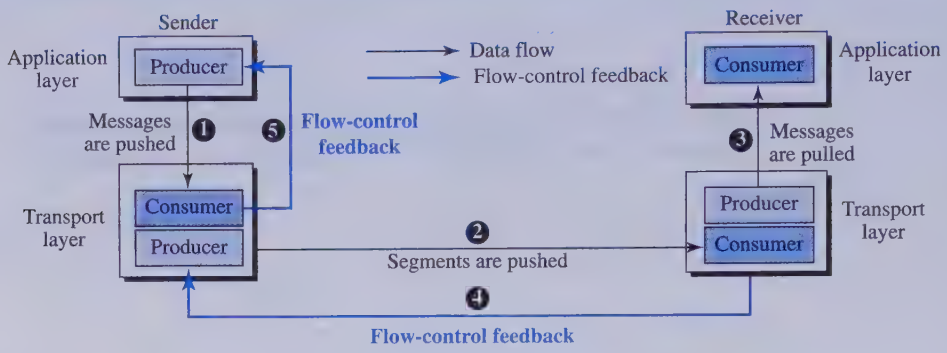
2. The second difference is the way acknowledgments are used in the TCP protocol. Remember that an acknowledgement in SR is selective, defining the uncorrupted packets that have been received. The major acknowledgment mechanism in TCP is a cumulative acknowledgment announcing the next expected byte to receive. The new version of TCP, however, uses both cumulative and selective acknowledgements; we discuss these options on the book website.

9.4.7 Flow Control

As discussed before, *flow control* balances the rate at which a producer creates data with the rate at which a consumer can use the data. TCP separates flow control from error control. In this section we discuss flow control, ignoring error control. We assume that the logical channel between the sending and receiving TCP is error-free.

Figure 9.35 shows unidirectional data transfer between a sender and a receiver; bidirectional data transfer can be deduced from the unidirectional process. The figure shows that data travel from the sending process down to the sending TCP, from the

Figure 9.35 *Data flow and flow-control feedbacks in TCP*



sending TCP to the receiving TCP, and from the receiving TCP up to the receiving process (paths 1, 2, and 3). Flow-control feedbacks, however, are traveling from the receiving TCP to the sending TCP and from the sending TCP up to the sending process (paths 4 and 5). Most implementations of TCP do not provide flow-control feedback from the receiving process to the receiving TCP; they let the receiving process pull data from the receiving TCP whenever it is ready to do so. In other words, the receiving TCP controls the sending TCP; the sending TCP controls the sending process.

Flow-control feedback from the sending TCP to the sending process (path 5) is achieved through simple rejection of data by the sending TCP when its window is full. This means that our discussion of flow control concentrates on the feedback sent from the receiving TCP to the sending TCP (path 4).

Opening and Closing Windows

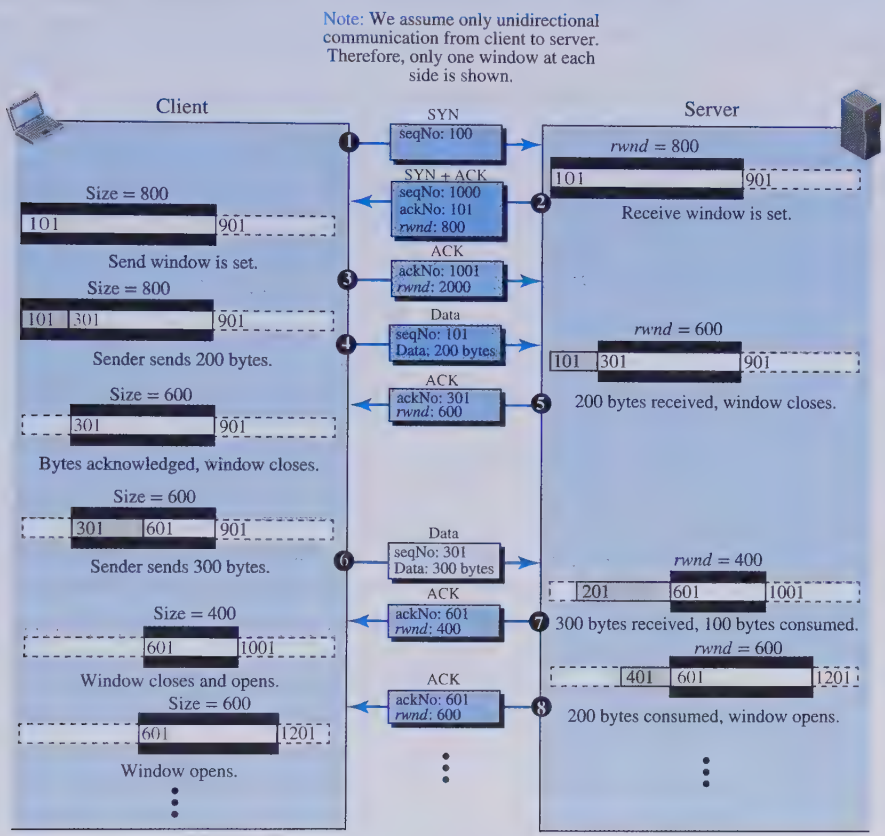
To achieve flow control, TCP forces the sender and the receiver to adjust their window sizes, although the size of the buffer for both parties is fixed when the connection is established. The receive window closes (moves its left wall to the right) when more bytes arrive from the sender; it opens (moves its right wall to the right) when more bytes are pulled by the process. We assume that it does not shrink (the right wall does not move to the left).

The opening, closing, and shrinking of the send window is controlled by the receiver. The send window closes (moves its left wall to the right) when a new acknowledgment allows it to do so. The send window opens (its right wall moves to the right) when the receive window size (*rwnd*) advertised by the receiver allows it to do so (new ackNo + new *rwnd* > last ackNo + last *rwnd*). The send window shrinks in the event this situation does not occur.

A Scenario

We show how the send and receive windows are set during the connection establishment phase and how their situations will change during data transfer. Figure 9.36 shows

Figure 9.36 *An example of flow control*



a simple example of unidirectional data transfer (from client to server). For the time being, we ignore error control, assuming that no segment is corrupted, lost, duplicated, or has arrived out of order. Note that we have shown only two windows for unidirectional data transfer. Although the client defines the server's window size as 2000 in the third segment, we do not shown that window here because the communication is only unidirectional.

Eight segments are exchanged between the client and server:

1. The first segment is from the client to the server (a SYN segment) to request connection. The client announces its initial seqNo = 100. When this segment arrives at the server, it allocates a buffer size of 800 (an assumption) and sets its window to cover the whole buffer (*rwnd* = 800). Note that the number of the next byte to arrive is 101.

2. The second segment is from the server to the client. This is an ACK + SYN segment. The segment uses ackNo = 101 to show that it expects to receive bytes starting from 101. It also announces that the client can set a buffer size of 800 bytes.
3. The third segment is the ACK segment from the client to the server. Note that the client has defined a *rwnd* of size 2000, but we do not use this value in our figure because the communication is only in one direction.
4. After the client has set its window with the size (800) dictated by the server, the process pushes 200 bytes of data. The TCP client numbers these bytes 101 to 300. It then creates a segment and sends it to the server. The segment shows the starting byte number as 101, and the segment carries 200 bytes. The window of the client is then adjusted to show that 200 bytes of data are sent but waiting for acknowledgment. When this segment is received at the server, the bytes are stored, and the receive window closes to show that the next byte expected is byte 301; the stored bytes occupy 200 bytes of buffer.
5. The fifth segment is the feedback from the server to the client. The server acknowledges bytes up to and including 300 (expecting to receive byte 301). The segment also carries the size of the receive window after decrease (600). The client, after receiving this segment, purges the acknowledged bytes from its window and closes its window to show that the next byte to send is byte 301. The window size, however, decreases to 600 bytes. Although the allocated buffer can store 800 bytes, the window cannot open (moving its right wall to the right) because the receiver does not let it.
6. Segment 6 is sent by the client after its process pushes 300 more bytes. The segment defines seqNo as 301 and contains 300 bytes. When this segment arrives at the server, the server stores them, but it has to reduce its window size. After its process has pulled 100 bytes of data, the window closes from the left for the amount of 300 bytes, but opens from the right for the amount of 100 bytes. The result is that the size is only reduced by 200 bytes. The receiver window size is now 400 bytes.
7. In segment 7, the server acknowledges the receipt of data and announces that its window size is 400. When this segment arrives at the client, the client has no choice but to reduce its window again and set the window size to the value of *rwnd* = 400 advertised by the server. The send window closes from the left by 300 bytes and opens from the right by 100 bytes.
8. Segment 8 is also from the server after its process has pulled another 200 bytes. Its window size increases. The new *rwnd* value is now 600. The segment informs the client that the server still expects byte 601, but the server window size has expanded to 600. We need to mention that the sending of this segment depends on the policy imposed by the implementation. Some implementations may not allow advertisement of the *rwnd* at this time; the server then needs to receive some data before doing so. After this segment arrives at the client, the client opens its window by 200 bytes without closing it. The result is that its window size increases to 600 bytes.

Shrinking of Windows

As we said before, the receive window cannot shrink. The send window, on the other hand, can shrink if the receiver defines a value for *rwnd* that results in shrinking the

window. However, some implementations do not allow shrinking of the send window. The limitation does not allow the right wall of the send window to move to the left. In other words, the receiver needs to keep the following relationship between the last and new acknowledgment and the last and new *rwnd* values to prevent shrinking of the send window.

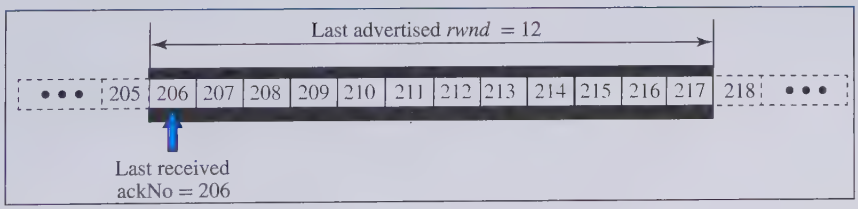
$$\textbf{new ackNo} + \textbf{new } \textit{rwnd} \quad \geq \quad \textbf{last ackNo} + \textbf{last } \textit{rwnd}$$

The left side of the inequality represents the new position of the right wall with respect to the sequence number space; the right side shows the old position of the right wall. The relationship shows that the right wall should not move to the left. The inequality is a mandate for the receiver to check its advertisement. However, note that the inequality is valid only if $S_f < S_n$; we need to remember that all calculations are in modulo 2^{32}.

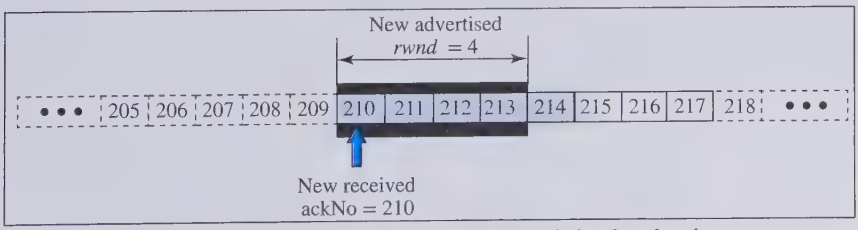
Example 9.9

Figure 9.37 shows the reason for this mandate. Part a of the figure shows the values of the last acknowledgment and *rwnd*. Part b shows the situation in which the sender has sent Bytes 206 to 214. Bytes 206 to 209 are acknowledged and purged. The new advertisement, however, defines the new value of *rwnd* as 4, in which 210 + 4 < 206 + 12. When the send window shrinks, it creates a problem: Byte 214, which has already been sent, is outside the window. The relation discussed before forces the receiver to maintain the right-hand wall of the window to be as shown in part a, because the receiver does not know which of the bytes 210 to 217 has already been sent. One way to prevent this situation is to let the receiver postpone its feedback until enough buffer locations are available in its window. In other words, the receiver should wait until more bytes are consumed by its process to meet the relationship described above.

Figure 9.37 *Example 9.9*



a. The window after the last advertisement



b. The window after the new advertisement; window has shrunk

Window Shutdown

We said that shrinking the send window by moving its right wall to the left is strongly discouraged. However, there is one exception: The receiver can temporarily shut down the window by sending a *rwnd* of 0. This can happen if for some reason the receiver does not want to receive any data from the sender for a while. In this case, the sender does not actually shrink the size of the window, but stops sending data until a new advertisement has arrived. As we will see later, even when the window is shut down by an order from the receiver, the sender can always send a segment with 1 byte of data. This is called probing and is used to prevent a deadlock (see Section 9.4.10 on TCP timers).

Silly Window Syndrome

A serious problem can arise in the sliding window operation when either the sending application program creates data slowly or the receiving application program consumes data slowly, or both. Any of these situations results in the sending of data in very small segments, which reduces the efficiency of the operation. For example, if TCP sends segments containing only 1 byte of data, it means that a 41-byte datagram (20 bytes of TCP header and 20 bytes of IP header) transfers only 1 byte of user data. Here the overhead is 41/1, which indicates that we are using the capacity of the network very inefficiently. The inefficiency is even worse after accounting for the data-link layer and physical-layer overhead. This problem is called the **silly window syndrome**. For each site, we first describe how the problem is created and then give a proposed solution.

Syndrome Created by the Sender

The sending TCP may create a silly window syndrome if it is serving an application program that creates data slowly, for example, 1 byte at a time. The application program writes 1 byte at a time into the buffer of the sending TCP. If the sending TCP does not have any specific instructions, it may create segments containing 1 byte of data. The result is a lot of 41-byte segments that are traveling through an internet.

The solution is to prevent the sending TCP from sending the data byte by byte. The sending TCP must be forced to wait and collect data to send in a larger block. How long should the sending TCP wait? If it waits too long, it may delay the process. If it does not wait long enough, it may end up sending small segments. Nagle found an elegant solution. **Nagle's algorithm** is simple:

1. The sending TCP sends the first piece of data it receives from the sending application program even if it is only 1 byte.
2. After sending the first segment, the sending TCP accumulates data in the output buffer and waits until either the receiving TCP sends an acknowledgment or until enough data has accumulated to fill a maximum-size segment. At this time, the sending TCP can send the segment.
3. Step 2 is repeated for the rest of the transmission. Segment 3 is sent immediately if an acknowledgment is received for segment 2, or if enough data have accumulated to fill a maximum-size segment.

The elegance of Nagle's algorithm is in its simplicity and in the fact that it takes into account the speed of the application program that creates the data and the speed of the network that transports the data. If the application program is faster than the

network, the segments are larger (maximum-size segments). If the application program is slower than the network, the segments are smaller (less than the maximum segment size).

Syndrome Created by the Receiver

The receiving TCP may create a silly window syndrome if it is serving an application program that consumes data slowly, for example, 1 byte at a time. Suppose that the sending application program creates data in blocks of 1 kilobyte, but the receiving application program consumes data 1 byte at a time. Also suppose that the input buffer of the receiving TCP is 4 kbytes. The sender sends the first 4 kbytes of data. The receiver stores it in its buffer. Now its buffer is full. It advertises a window size of zero, which means the sender should stop sending data. The receiving application reads the first byte of data from the input buffer of the receiving TCP. Now there is 1 byte of space in the incoming buffer. The receiving TCP announces a window size of 1 byte, which means that the sending TCP, which is eagerly waiting to send data, takes this advertisement as good news and sends a segment carrying only 1 byte of data. The procedure will continue. One byte of data is consumed, and a segment carrying 1 byte of data is sent. Again we have an efficiency problem and the silly window syndrome.

Two solutions have been proposed to prevent the silly window syndrome created by an application program that consumes data more slowly than they arrive. **Clark's solution** is to send an acknowledgment as soon as the data arrive, but to announce a window size of zero until either there is enough space to accommodate a segment of maximum size or until at least half of the receive buffer is empty. The second solution is to delay sending the acknowledgment. This means that when a segment arrives, it is not acknowledged immediately. The receiver waits until there is a decent amount of space in its incoming buffer before acknowledging the arrived segments. The delayed acknowledgment prevents the sending TCP from sliding its window. After the sending TCP has sent the data in the window, it stops. This kills the syndrome.

Delayed acknowledgment also has another advantage: It reduces traffic. The receiver does not have to acknowledge each segment. However, there also is a disadvantage in that the delayed acknowledgment may result in the sender unnecessarily retransmitting the unacknowledged segments.

The protocol balances the advantages and disadvantages. It now defines that the acknowledgment should not be delayed by more than 500 ms.

9.4.8 Error Control

TCP is a reliable transport-layer protocol. This means that an application program that delivers a stream of data to TCP relies on TCP to deliver the entire stream to the application program on the other end in order, without error, and without any part lost or duplicated.

TCP provides reliability using error control. Error control includes mechanisms for detecting and resending corrupted segments, resending lost segments, storing out-of-order segments until missing segments arrive, and detecting and discarding duplicated segments. Error control in TCP is achieved through the use of three simple tools: checksum, acknowledgment, and time-out.

Checksum

Each segment includes a checksum field, which is used to check for a corrupted segment. If a segment is corrupted, as detected by an invalid checksum, the segment is discarded by the destination TCP and is considered lost. TCP uses a 16-bit checksum that is mandatory in every segment. We discuss checksum calculation in Appendix F.

Acknowledgment

TCP uses acknowledgments to confirm the receipt of data segments. Control segments that carry no data, but consume a sequence number, are also acknowledged. ACK segments are never acknowledged.

ACK segments do not consume sequence numbers and are not acknowledged.

Acknowledgment Type

In the past, TCP used only one type of acknowledgment: cumulative acknowledgment. Today, some TCP implementations also use selective acknowledgment.

Cumulative Acknowledgment (ACK) TCP was originally designed to acknowledge receipt of segments cumulatively. The receiver advertises the next byte it expects to receive, ignoring all segments received and stored out of order. This is sometimes referred to as *positive cumulative acknowledgment*, or ACK. The word *positive* indicates that no feedback is provided for discarded, lost, or duplicate segments. The 32-bit ACK field in the TCP header is used for cumulative acknowledgments, and its value is valid only when the ACK flag bit is set to 1.

Selective Acknowledgment (SACK) More and more implementations are adding another type of acknowledgment called *selective acknowledgment,* or SACK. A SACK does not replace an ACK, but reports additional information to the sender. A SACK reports a block of bytes that is out of order, and also a block of bytes that is duplicated, i.e., received more than once. However, because there is no provision in the TCP header for adding this type of information, SACK is implemented as an option at the end of the TCP header. We discuss this new feature when we discuss options in TCP on the book website.

Generating Acknowledgments

When does a receiver generate acknowledgments? During the evolution of TCP, several rules have been defined and used by several implementations. We give the most common rules here. The order of a rule does not necessarily define its importance.

1. When end A sends a data segment to end B, it must include (piggyback) an acknowledgment that gives the next sequence number it expects to receive. This rule decreases the number of segments needed and therefore reduces traffic.
2. When the receiver has no data to send and it receives an in-order segment (with expected sequence number) and the previous segment has already been acknowledged, the receiver delays sending an ACK segment until another segment arrives or until a period of time (normally 500 ms) has passed. In other words, the receiver

needs to delay sending an ACK segment if there is only one outstanding in-order segment. This rule reduces ACK segments.

3. When a segment arrives with a sequence number that is expected by the receiver, and the previous in-order segment has not been acknowledged, the receiver immediately sends an ACK segment. In other words, there should not be more than two in-order unacknowledged segments at any time. This prevents the unnecessary retransmission of segments that may create congestion in the network.
4. When a segment arrives with an out-of-order sequence number that is higher than expected, the receiver immediately sends an ACK segment announcing the sequence number of the next expected segment. This leads to the *fast retransmission* of missing segments.
5. When a missing segment arrives, the receiver sends an ACK segment to announce the next sequence number expected. This informs the receiver that segments reported missing have been received.
6. If a duplicate segment arrives, the receiver discards the segment, but immediately sends an acknowledgment indicating the next in-order segment expected. This solves some problems when an ACK segment itself is lost.

Retransmission

The heart of the error-control mechanism is the retransmission of segments. When a segment is sent, it is stored in a queue until it is acknowledged. When the retransmission timer expires or when the sender receives three duplicate ACKs for the first segment in the queue, that segment is retransmitted.

Retransmission after RTO

The sending TCP maintains one **retransmission time-out (RTO)** for each connection. When the timer matures, i.e., times out, TCP resends the segment in the front of the queue (the segment with the smallest sequence number) and restarts the timer. Note that again we assume $S_f < S_n$. We will see later that the value of RTO is dynamic in TCP and is updated based on the round-trip time (RTT) of segments. RTT is the time needed for a segment to reach a destination and for an acknowledgment to be received.

Retransmission after Three Duplicate ACK Segments

The previous rule about retransmission of a segment is sufficient if the value of RTO is not large. To expedite service throughout the Internet by allowing senders to retransmit without waiting for a time-out, most implementations today follow the three duplicate ACKs rule and retransmit the missing segment immediately. This feature is called **fast retransmission**. In this version, if three duplicate acknowledgments (i.e., an original ACK plus three exactly identical copies) arrive for a segment, the next segment is retransmitted without waiting for the time-out.

Out-of-Order Segments

TCP implementations today do not discard out-of-order segments. They store them temporarily and flag them as out-of-order segments until the missing segments arrive. Note, however, that out-of-order segments are never delivered to the process. TCP guarantees that data are delivered to the process in order.

Data may arrive out of order and be temporarily stored by the receiving TCP, but TCP guarantees that no out-of-order data are delivered to the process.

FSMs for Data Transfer in TCP

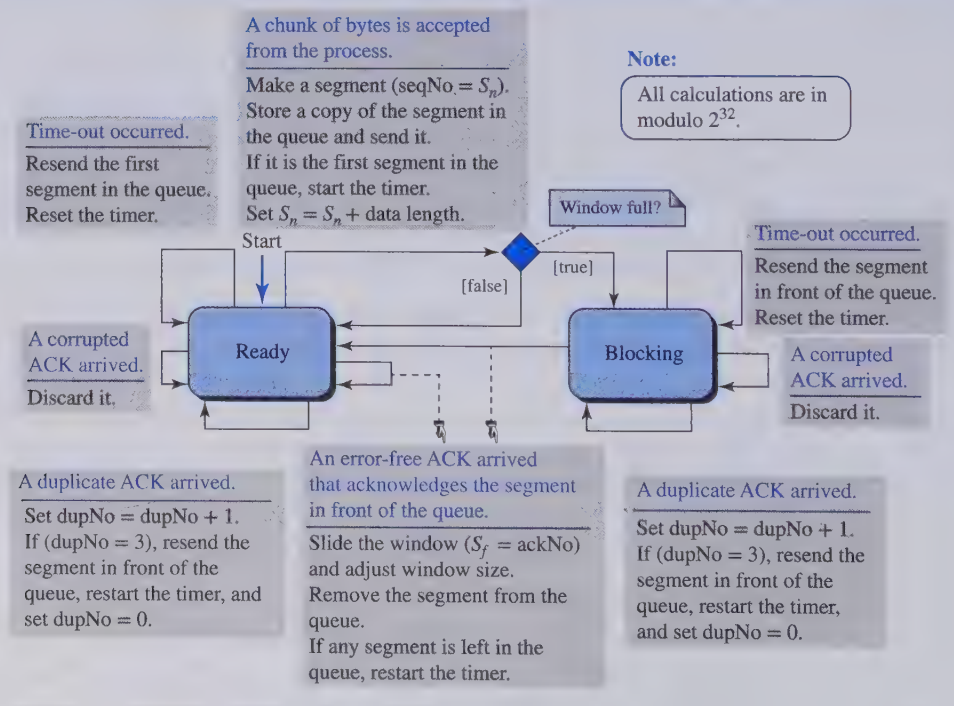
Data transfer in TCP is close to the Selective-Repeat protocol with a slight similarity to GBN. Because TCP accepts out-of-order segments, TCP can be thought of as behaving more like the SR protocol, but because the original acknowledgments are cumulative, it looks like GBN. However, if the TCP implementation uses SACKs, then TCP is closest to the SR protocol.

TCP can best be modeled as a Selective-Repeat protocol.

Sender-Side FSM

Let us show a simplified FSM for the sender side of the TCP protocol similar to the one we discussed for the SR protocol, but with some changes specific to TCP. We assume that the communication is unidirectional and the segments are acknowledged using ACK segments. We also ignore selective acknowledgments and congestion control for the moment. Figure 9.38 shows the simplified FSM for the sender site. Note that the

Figure 9.38 *Simplified FSM for the TCP sender side*



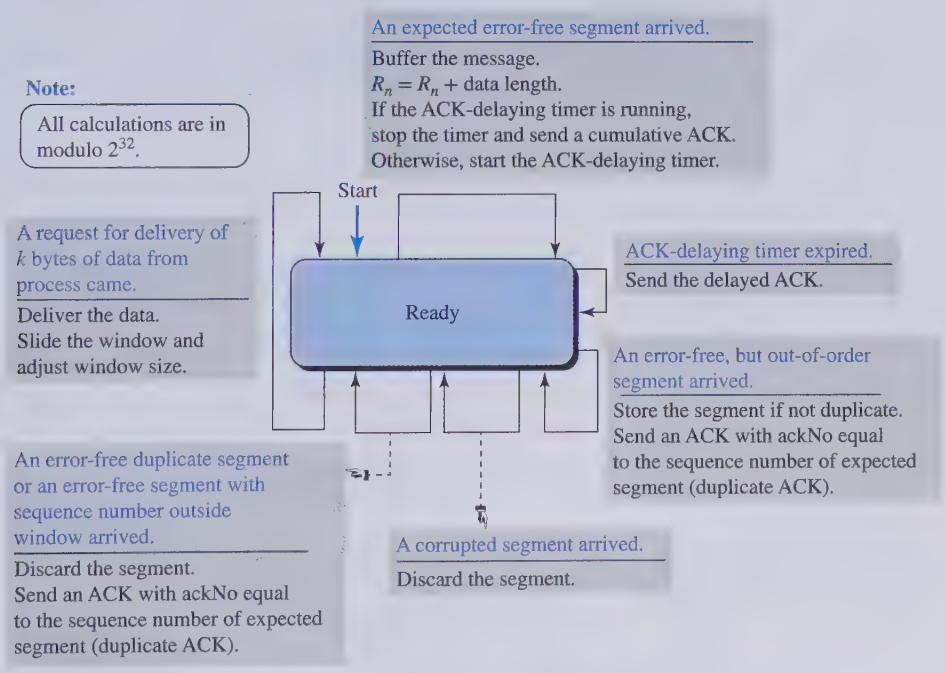
FSM is rudimentary; it does not include issues such as silly window syndrome (Nagle's algorithm) or window shutdown. It defines a unidirectional communication, ignoring all issues that affect bidirectional communication.

There are some differences between the FSM in Figure 9.38 and the one we discussed for an SR protocol. One difference is the fast transmission (three duplicate ACKs). The other is the window size adjustment based on the value of *rwnd* (ignoring congestion control for the moment).

Receiver-Side FSM

Now let us show a simplified FSM for the receiver-side TCP protocol similar to the one we discuss for the SR protocol, but with some changes specific to TCP. We assume that the communication is unidirectional and the segments are acknowledged using ACK segments. We also ignore the selective acknowledgment and congestion control for the moment. Figure 9.39 shows the simplified FSM for the sender. Note that we ignore some issues such as silly window syndrome (Clark's solution) and window shutdown.

Figure 9.39 *Simplified FSM for the TCP receiver side*



Again, there are some differences between this FSM and the one we discussed for an SR protocol. One difference is the ACK delaying in unidirectional communication. The other difference is the sending of duplicate ACKs to allow the sender to implement fast transmission policy.

We also need to emphasize that bidirectional FSM for the receiver is not as simple as the one for SR; we need to consider some policies such as sending an immediate ACK if the receiver has some data to return.

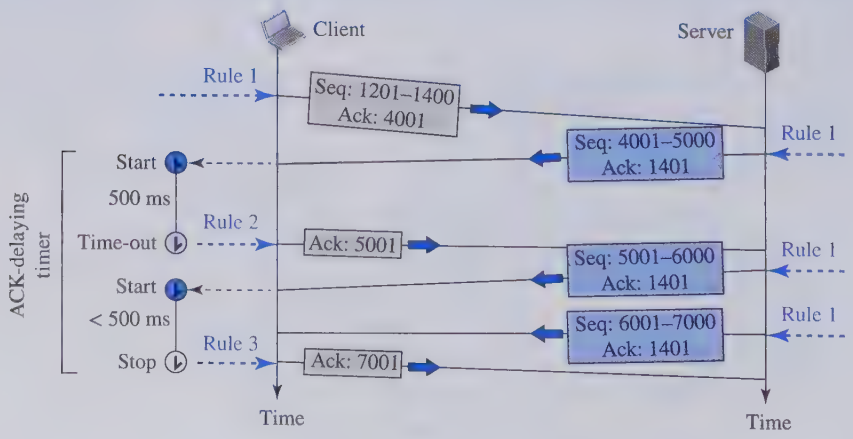
Some Scenarios

In this section we give some examples of scenarios that occur during the operation of TCP, considering only error-control issues. In these scenarios, we show a segment by a rectangle. If the segment carries data, we show the range of byte numbers and the value of the acknowledgment field. If it carries only an acknowledgment, we show only the acknowledgment number in a smaller box.

Normal Operation

The first scenario shows bidirectional data transfer between two systems as shown in Figure 9.40. The client TCP sends one segment; the server TCP sends three. The figure shows which rule applies to each acknowledgment. At the server site, only rule 1 applies. There are data to be sent, so the segment displays the next byte expected. When the client receives the first segment from the server, it does not have any more data to send; it needs to send only an ACK segment. However, according to rule 2, the acknowledgment needs to be delayed for 500 ms to see if any more segments arrive. When the ACK-delaying timer matures, it triggers an acknowledgment. This is because the client has no knowledge if other segments are coming; it cannot delay the acknowledgment forever. When the next segment arrives, another ACK-delaying timer is set. However, before it matures, the third segment arrives. The arrival of the third segment triggers another acknowledgment based on rule 3. We have not shown the RTO timer because no segment is lost or delayed. We just assume that the RTO timer performs its duty.

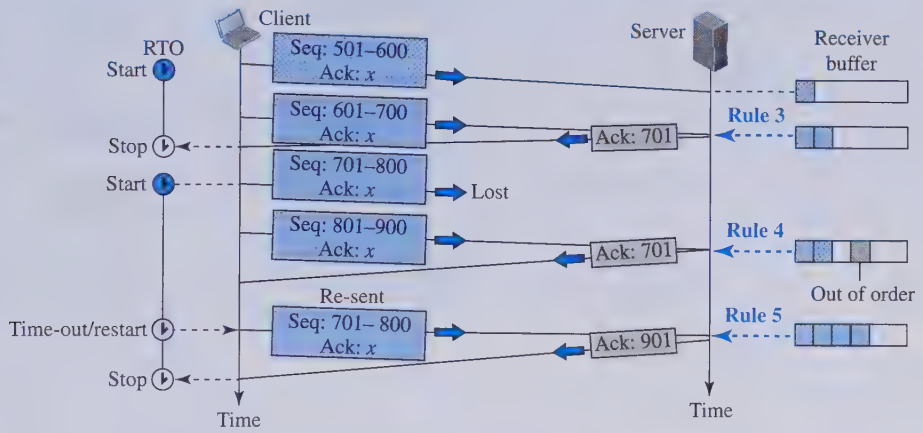
Figure 9.40 *Normal operation*



Lost Segment

In this second scenario, we show what happens when a segment is lost or corrupted. A lost or corrupted segment is treated the same way by the receiver. A lost segment is discarded somewhere in the network; a corrupted segment is discarded by the receiver itself. Both are considered lost. Figure 9.41 shows a situation in which a segment is lost (probably discarded by some router in the network due to congestion).

Figure 9.41 *Lost segment*



We are assuming that data transfer is unidirectional: One site is sending, the other receiving. In our scenario, the sender sends segments 1 and 2, which are acknowledged immediately by an ACK (rule 3). Segment 3, however, is lost. The receiver receives segment 4, which is out of order. The receiver stores the data in the segment in its buffer but leaves a gap to indicate that there is no continuity in the data. The receiver immediately sends an acknowledgment to the sender displaying the next byte it expects (rule 4). Note that the receiver stores bytes 801 to 900, but never delivers these bytes to the application until the gap is filled.

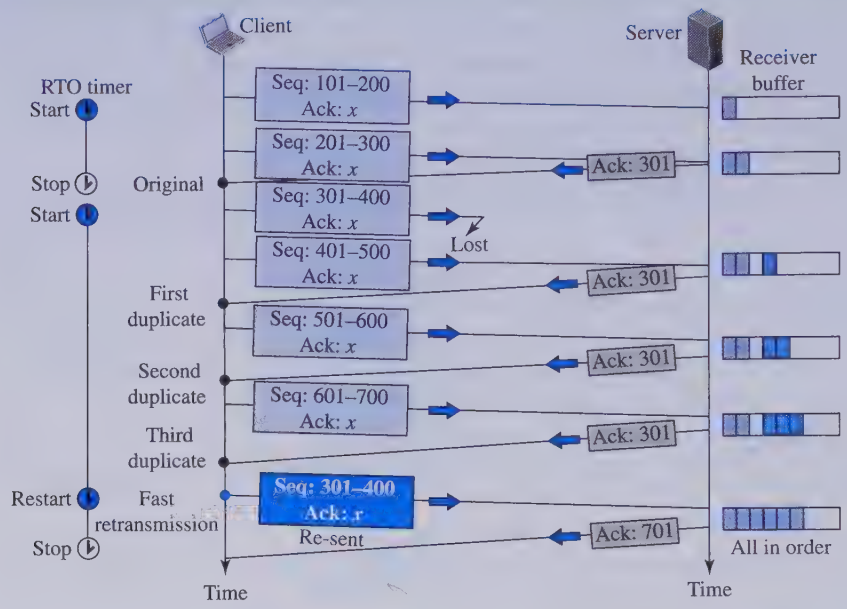
The receiver TCP delivers only ordered data to the process.

The sender TCP keeps one RTO timer for the whole period of connection. When the third segment times out, the sending TCP resends segment 3, which arrives this time and is acknowledged properly (rule 5).

Fast Retransmission

In this scenario, we want to show *fast retransmission*. This scenario is the same as the second one except that the RTO has a larger value (see Figure 9.42).

Figure 9.42 *Fast retransmission*



Each time the receiver receives a subsequent segment, it triggers an acknowledgment (rule 4). The sender receives four acknowledgments with the same value (three duplicates). Although the timer has not matured, the rule for fast retransmission requires that segment 3, the segment that is expected by all these duplicate acknowledgments, be re-sent immediately. After this segment is re-sent, the timer is restarted.

Delayed Segment

The fourth scenario features a delayed segment. TCP uses the services of IP, which is a connectionless protocol. Each IP datagram encapsulating a TCP segment may reach the final destination through a different route with a different delay. Hence, TCP segments may be delayed. Delayed segments sometimes may time out and be re-sent. If the delayed segment arrives after it has been re-sent, it is considered a duplicate segment and discarded.

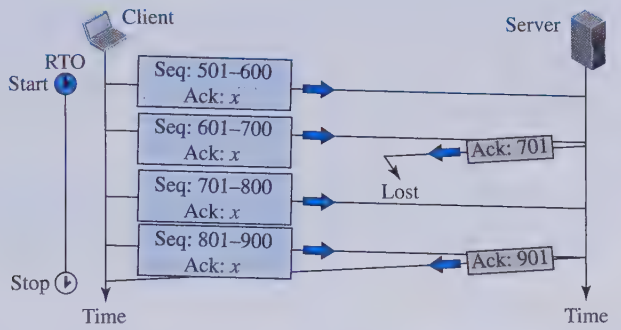
Duplicate Segment

A duplicate segment can be created, for example, by a sending TCP when a segment is delayed and treated as lost by the receiver. Handling the duplicated segment is a simple process for the destination TCP. The destination TCP expects a continuous stream of bytes. When a segment arrives that contains a sequence number equal to an already received and stored segment, it is discarded. An ACK is sent with ackNo defining the expected segment.

Automatically Corrected Lost ACK

This scenario shows a situation in which information in a lost acknowledgment is contained in the next one, a key advantage of using cumulative acknowledgments. Figure 9.43 shows a lost acknowledgment sent by the receiver of data. In the TCP acknowledgment mechanism, the source TCP may not even notice a lost acknowledgment. TCP uses cumulative acknowledgment. We can say that the next acknowledgment automatically corrects the loss of the previous acknowledgment.

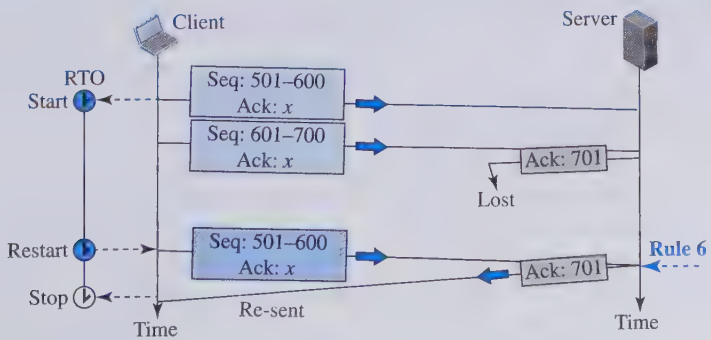
Figure 9.43 *Lost acknowledgment*



Lost Acknowledgment Corrected by Resending a Segment

Figure 9.44 shows a scenario in which an acknowledgment is lost. If the next acknowledgment is delayed for a long time or there is no next acknowledgment (the lost acknowledgment is the last one sent), the correction is triggered by the RTO timer. A duplicate segment is the result. When the receiver receives a duplicate segment, it discards it and resends the last ACK immediately to inform the sender that the segment or segments have been received.

Figure 9.44 *Lost acknowledgment corrected by resending a segment*



Note that only one segment is retransmitted, although two segments are not acknowledged. When the sender receives the retransmitted ACK, it knows that both segments are safe and sound because the acknowledgment is cumulative.

Deadlock Created by Lost Acknowledgment

There is one situation in which loss of an acknowledgment may result in system deadlock. This is the case in which a receiver sends an acknowledgment with *rwnd* set to 0 and requests that the sender shut down its window temporarily. After a while, the receiver wants to remove the restriction; however, if it has no data to send, it sends an ACK segment and removes the restriction with a nonzero value for *rwnd*. A problem arises if this acknowledgment is lost. The sender is waiting for an acknowledgment that announces the nonzero *rwnd*. The receiver thinks that the sender has received this and is waiting for data. This situation is called a **deadlock**; each end is waiting for a response from the other end and nothing is happening. A retransmission timer is not set. To prevent deadlock, a persistence timer was designed that we will study later in Section 9.4.10.

Lost acknowledgments may create deadlock if they are not properly handled.

9.4.9 TCP Congestion Control

TCP uses different policies to handle the congestion in the network. We describe these policies in this section.

Congestion Window

When we discussed flow control in TCP, we mentioned that the size of the send window is controlled by the receiver using the value of *rwnd*, which is advertised in each segment traveling in the opposite direction. The use of this strategy guarantees that the receive window is never overflowed with the received bytes (no end congestion). This, however, does not mean that the intermediate buffers, which are the buffers in the routers, do not become congested. A router may receive data from more than one sender. No matter how large the buffers of a router may be, the router may be overwhelmed with data, which results in some segments sent by a specific TCP sender being dropped. In other words, there is no congestion at the other end, but there may be congestion in the middle. TCP needs to worry about congestion in the middle because if many segments are lost, this may seriously affect the error control. More segment loss means the same segments will need to be sent again, resulting in worsening congestion, and finally the collapse of the communication.

TCP is an end-to-end protocol that uses the service of IP. The congestion in the router is in the IP territory and should be taken care of by IP. However, as we discuss in Chapter 7, IP is a simple protocol with no congestion control. TCP, itself, needs to be responsible for this problem.

TCP cannot ignore congestion in the network; it cannot aggressively send segments to the network. Such aggressiveness would hurt the TCP itself, as we mentioned before. TCP also cannot be very conservative, sending only a small number of segments in each

time interval, because this would mean the available bandwidth of the network would not be utilized. TCP needs to define policies that accelerate the data transmission when there is no congestion and decelerate the transmission when congestion is detected.

To control the number of segments to transmit, TCP uses another variable called a *congestion window*, *cwnd*, whose size is controlled by the congestion situation in the network (as we will explain shortly). The *cwnd* and *rwnd* variables together define the size of the send window in TCP. The first is related to the congestion in the middle (network); the second is related to the congestion at the end. The actual size of the window is the minimum of these two.

$$\textbf{Actual window size} = \textbf{minimum } (\textit{rwnd}, \textit{cwnd})$$

Congestion Detection

Before discussing how the value of *cwnd* should be set and changed, we need to describe how a TCP sender can detect the possible existence of congestion in the network. The TCP sender uses the occurrence of two events as signs of congestion in the network: time-out and receiving three duplicate ACKs.

The first is the *time-out*. If a TCP sender does not receive an ACK for a segment or a group of segments before the time-out occurs, it assumes that the corresponding segment or segments are lost and the loss is due to congestion.

Another event is the receiving of three duplicate ACKs (four ACKs with the same acknowledgment number). Recall that when a TCP receiver sends a duplicate ACK, it is the sign that a segment has been delayed, but sending three duplicate ACKs is the sign of a missing segment, which can be due to congestion in the network. However, the congestion in the case of three duplicate ACKs can be less severe than in the case of time-out. When a receiver sends three duplicate ACKs, it means that one segment is missing, but three segments have been received. The network is either slightly congested or has recovered from the congestion.

We will show later in this section that an earlier version of TCP, called Taho TCP, treated both events (time-out and three duplicate ACKs) similarly, but the later version of TCP, called Reno TCP, treats these two events differently.

A very interesting point in TCP congestion is that the TCP sender uses only one feedback from the other end to detect congestion: ACKs. The lack of regular, timely receipt of ACKs, which results in a time-out, is the sign of strong congestion; the receiving of three duplicate ACKs is the sign of weak congestion in the network.

Congestion Policies

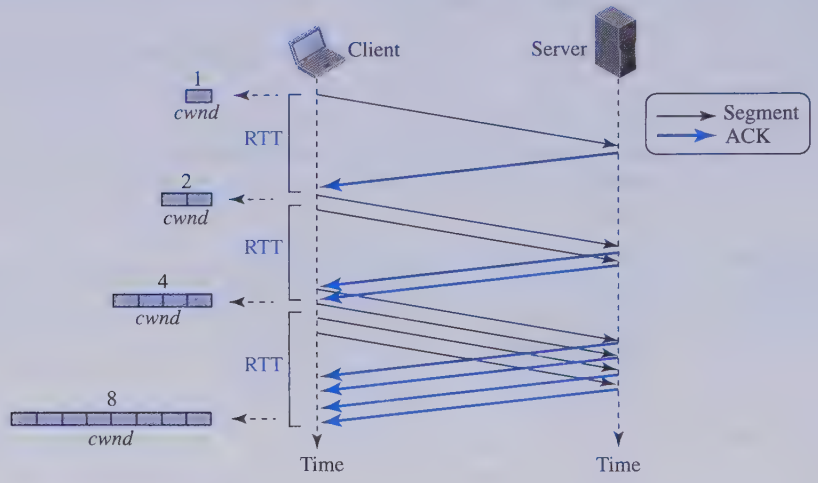
TCP's general policy for handling congestion is based on three algorithms: slow start, congestion avoidance, and fast recovery. We first discuss each algorithm before showing how TCP switches from one to the other in a connection.

Slow Start: Exponential Increase

The **slow-start algorithm** is based on the idea that the size of the congestion window (*cwnd*) starts with one maximum segment size (MSS), but it increases one MSS each time one acknowledgment arrives. As we discussed before, the MSS is a value negotiated during the connection establishment, using an option of the same name.

The name of this algorithm is misleading; the algorithm starts slowly, but grows exponentially. To show the idea, let us look at Figure 9.45. We assume that *rwnd* is much larger than *cwnd*, so that the sender window size always equals *cwnd*. We also assume that each segment is of the same size and carries MSS bytes. For simplicity, we also ignore delayed-ACK policy and assume that each segment is acknowledged individually.

Figure 9.45 *Slow start, exponential increase*



The sender starts with *cwnd* = 1. This means that the sender can send only one segment. After the first ACK arrives, the acknowledged segment is purged from the window, which means there is now one empty segment slot in the window. The size of the congestion window is also increased by 1 because the arrival of the acknowledgment is a good sign that there is no congestion in the network. The size of the window is now 2. After sending two segments and receiving two individual acknowledgments for them, the size of the congestion window now becomes 4, and so on. In other words, the size of the congestion window in this algorithm is a function of the number of ACKs arrived and can be determined as follows:

If an ACK arrives, *cwnd* = *cwnd* + 1.

If we look at the size of *cwnd* in terms of round-trip times (RTTs), we find that the growth rate is exponential in terms of each round-trip time, which is a very aggressive approach:

Start	→	$cwnd = 1 \rightarrow 2^0$
After 1 RTT	→	$cwnd = cwnd + 1 = 1 + 1 = 2 \rightarrow 2^1$
After 2 RTT	→	$cwnd = cwnd + 2 = 2 + 2 = 4 \rightarrow 2^2$
After 3 RTT	→	$cwnd = cwnd + 4 = 4 + 4 = 8 \rightarrow 2^3$

A slow start cannot continue indefinitely. There must be a threshold to stop this phase. The sender keeps track of a variable named *ssthresh* (slow-start threshold). When the size of the window in bytes reaches this threshold, slow start stops and the next phase starts.

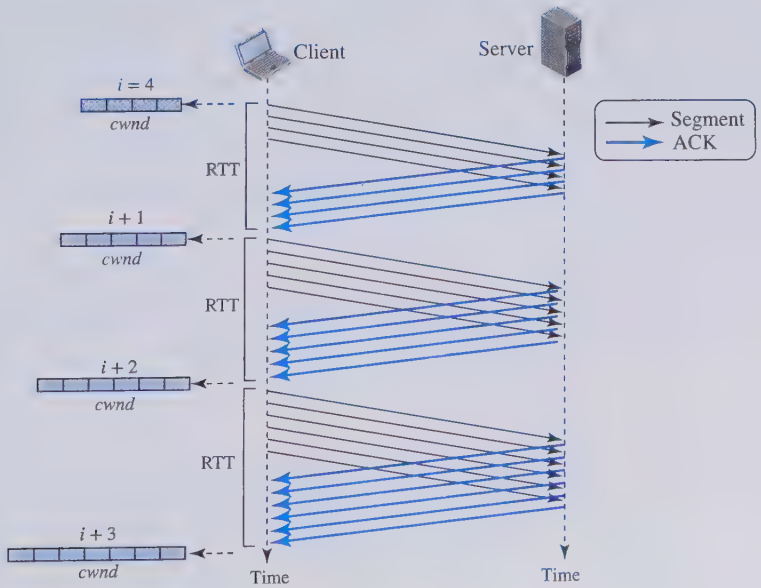
In the slow-start algorithm, the size of the congestion window increases exponentially until it reaches a threshold.

We need, however, to mention that the slow-start strategy is slower in the case of delayed acknowledgments. Remember, for each ACK, *cwnd* is increased by only 1. Hence, if two segments are acknowledged cumulatively, the size of *cwnd* increases by only 1, not 2. The growth is still exponential, but it is not a power of 2. With one ACK for every two segments, it is a power of 1.5.

Congestion Avoidance: Additive Increase

If we continue with the slow-start algorithm, the size of the congestion window increases exponentially. To avoid congestion before it happens, one must slow down this exponential growth. TCP defines another algorithm called **congestion avoidance**, which increases *cwnd* additively instead of exponentially. When the size of the congestion window reaches the slow-start threshold in the case where $cwnd = i$, the slow-start phase stops and the additive phase begins. In this algorithm, each time the whole "window" of segments is acknowledged, the size of the congestion window is increased by 1. A window is the number of segments transmitted during RTT. Figure 9.46 shows the idea.

Figure 9.46 *Congestion avoidance, additive increase*



The sender starts with *cwnd* = 4. This means that the sender can send only four segments. After four ACKs arrive, the acknowledged segments are purged from the window, which means there is now one empty segment slot in the window. The size of the congestion window is also increased by 1. The size of the window is now 5. After sending five segments and receiving five acknowledgments for them, the size of the congestion window now becomes 6. And so on. In other words, the size of the congestion window in this algorithm is also a function of the number of ACKs that have arrived and can be determined as follows:

If an ACK arrives, *cwnd* = *cwnd* + (1/*cwnd*).

In other words, the size of the window increases only 1/*cwnd* portion of MSS (in bytes). In other words, all segments in the previous window should be acknowledged to increase the window 1 MSS byte.

If we look at the size of *cwnd* in terms of round-trip times (RTTs), we find that the growth rate is linear in terms of each round-trip time, which is much more conservative than the slow-start approach.

Start	→	$cwnd = i$
After 1 RTT	→	$cwnd = i + 1$
After 2 RTT	→	$cwnd = i + 2$
After 3 RTT	→	$cwnd = i + 3$

In the congestion-avoidance algorithm, the size of the congestion window increases additively until congestion is detected.

Fast Recovery The **fast-recovery algorithm** is optional in TCP. The old version of TCP did not use it, but the new versions try to use it. It starts when the arrival of three duplicate ACKs is interpreted as light congestion in the network. Like congestion avoidance, this algorithm is also an additive increase, but it increases the size of the congestion window when a duplicate ACK arrives (after the three duplicate ACKs that trigger the use of this algorithm). We can say

If a duplicate ACK arrives, *cwnd* = *cwnd* + (1 / *cwnd*).

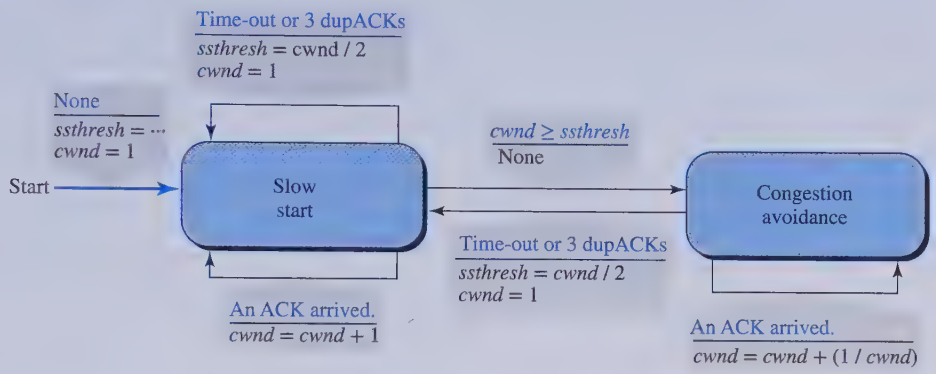
Policy Transition

We discussed three congestion policies in TCP. Now two questions are, "When are each of these policies used?" and "When does TCP move from one policy to another?" To answer these questions, we need to refer to three versions of TCP: Taho TCP, Reno TCP, and New Reno TCP.

Taho TCP

The early TCP, known as Taho TCP, used only two different algorithms in its congestion policy: *slow start and congestion avoidance*. We use Figure 9.47 to show the FSM for this version of TCP. However, we need to mention that we have deleted some small trivial actions, such as incrementing and resetting the number of duplicate ACKs, to make the FSM less crowded and simpler.

Figure 9.47 *FSM for Taho TCP*



Taho TCP treats the two signs used for congestion detection, time-out and three duplicate ACKs, in the same way. In this version, when the connection is established, TCP starts the slow-start algorithm and sets the *ssthresh* variable to a previously agreed upon value (normally multiple of MSS) and the *cwnd* to 1 MSS. In this state, as we said before, each time an ACK arrives, the size of the congestion window is incremented by 1. We know that this policy is very aggressive and exponentially increases the size of the window, which may result in congestion.

If congestion is detected (occurrence of time-out or arrival of three duplicate ACKs), TCP immediately interrupts this aggressive growth and restarts a new slow-start algorithm by limiting the threshold to half of the current *cwnd* and resetting the congestion window to 1. In other words, not only does TCP restart from scratch, but it also learns how to adjust the threshold. If no congestion is detected while reaching the threshold, TCP learns that the ceiling of its ambition is reached; it should not continue at this speed. It moves to the congestion avoidance state and continues in that state.

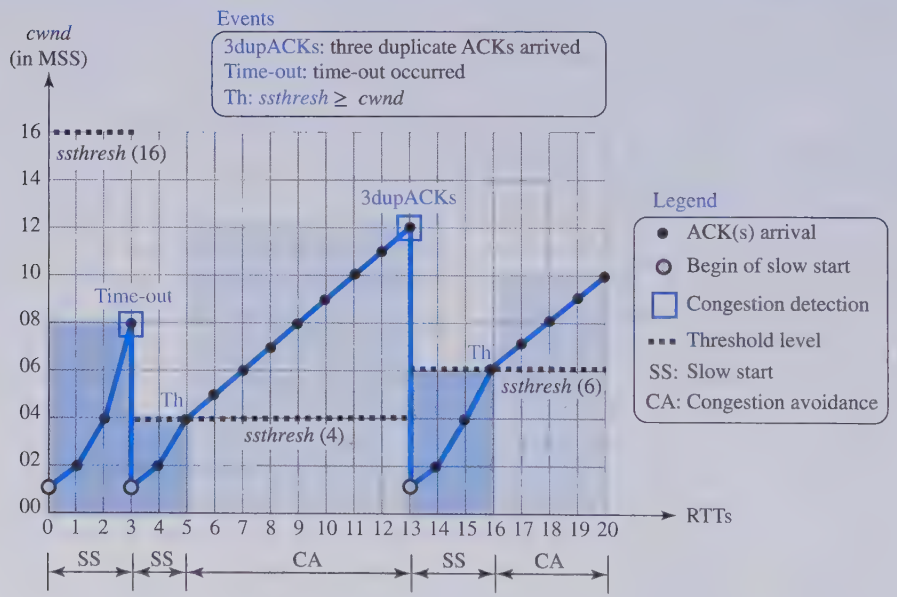
In the congestion-avoidance state, the size of the congestion window is increased by 1 each time a number of ACKs equal to the current size of the window has been received. For example, if the window size is now 5 MSS, five more ACKs should be received before the size of the window becomes 6 MSS. Note that there is no ceiling for the size of the congestion window in this state; the conservative additive growth of the congestion window continues to the end of the data transfer phase unless congestion is detected. If congestion is detected in this state, TCP again resets the value of *ssthresh* to half of the current *cwnd* and moves to the slow-start state again.

Although in this version of TCP the size of *ssthresh* is continuously adjusted in each congestion detection, this does not mean that it necessarily becomes lower than the previous value. For example, if the original *ssthresh* value is 8 MSS and the congestion is detected when TCP is in the congestion avoidance state and the value of the *cwnd* is 20, the new value of the *ssthresh* is now 10, which means it has been increased.

Example 9.10

Figure 9.48 shows an example of congestion control in a Taho TCP version. TCP starts data transfer and sets the *ssthresh* variable to an ambitious value of 16 MSS. TCP begins at the slow-start (SS) state with *cwnd* = 1. The congestion window grows exponentially, but a time-out occurs after the third RTT (before reaching the threshold). TCP assumes that there is congestion in the network. It immediately sets the new *ssthresh* = 4 MSS (half of the current *cwnd,* which is 8) and begins a new slow-start (SA) state with *cwnd* = 1 MSS. The congestion grows exponentially until it reaches the newly set threshold. TCP now moves to the congestion-avoidance (CA) state, and the congestion window grows additively until it reaches *cwnd* = 12 MSS. At this moment, three duplicate ACKs arrive, another indication of the congestion in the network. TCP again halves the value of *ssthresh* to 6 MSS and begins a new slow-start (SS) state. The exponential growth of *cwnd* continues. After RTT 15, the size of *cwnd* is 4 MSS. After sending four segments and receiving only two ACKs, the size of the window reaches the *ssthresh* (6) and the TCP moves to the congestion-avoidance state. The data transfer now continues in the congestion-avoidance (CA) state until the connection is terminated after RTT 20.

Figure 9.48 *Example of Taho TCP*

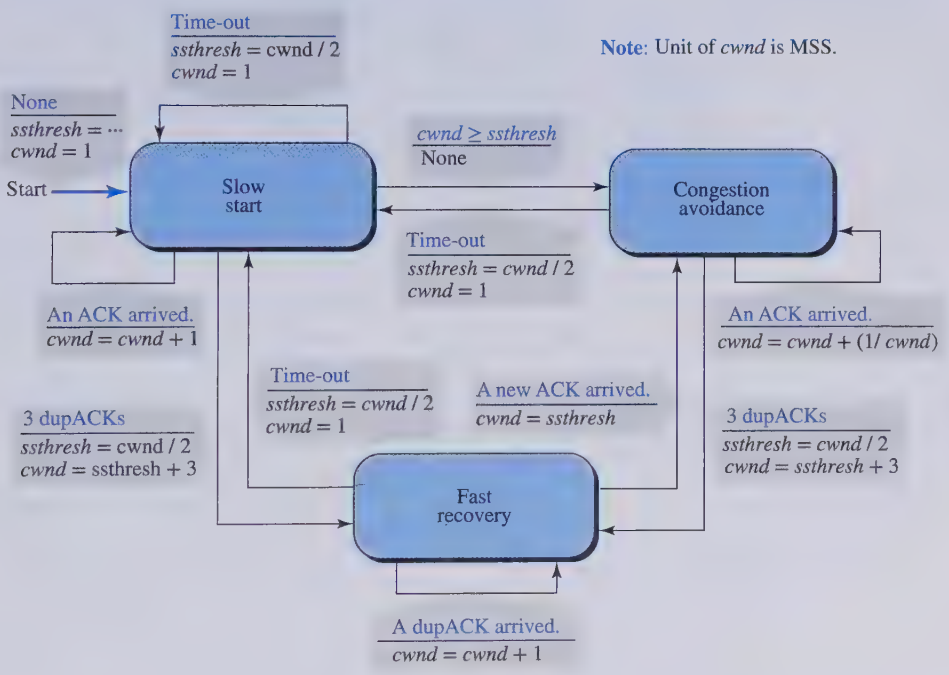


Reno TCP

A newer version of TCP, called the Reno TCP, added a new state to the congestion-control FSM, called the fast-recovery state. This version treated the two signals of congestion, time-out and the arrival of three duplicate ACKs, differently. In this version, if a time-out occurs, TCP moves to the slow-start state (or starts a new round if it is already in this state); on the other hand, if three duplicate ACKs arrive, TCP moves to

the fast-recovery state and remains there as long as more duplicate ACKs arrive. The fast-recovery state is a state somehow between the slow-start and congestion-avoidance states. It behaves like the slow start, in which *cwnd* grows exponentially, but *cwnd* starts with the value of *ssthresh* plus 3 MSS (instead of 1). When TCP enters the fast-recovery state, three major events may occur. If duplicate ACKs continue to arrive, TCP stays in this state, but *cwnd* grows exponentially. If a time-out occurs, TCP assumes that there is real congestion in the network and moves to the slow-start state. If a new (nonduplicate) ACK arrives, TCP moves to the congestion-avoidance state, but deflates the size of *cwnd* to the *ssthresh* value, as though the three duplicate ACKs have not occurred, and transition is from the slow-start state to the congestion-avoidance state. Figure 9.49 shows the simplified FSM for the Reno TCP. Again, we have removed some trivial events to simplify the figure and discussion.

Figure 9.49 *FSM for Reno TCP*

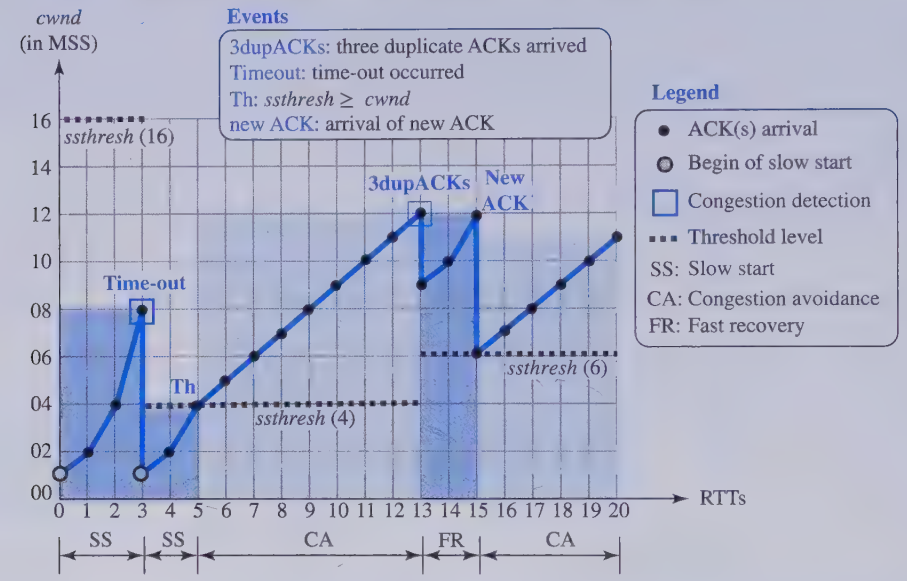


Example 9.11

Figure 9.50 shows the same situation as in Figure 9.48, but in Reno TCP. The changes in the congestion window are the same until RTT 13 when three duplicate ACKs arrive. At this moment, Reno TCP drops *ssthresh* to 6 MSS (same as for Taho TCP), but it sets *cwnd* to a much higher value (*ssthresh* + 3 = 9 MSS) instead of 1 MSS. Reno TCP now moves to the fast recovery state. We assume that two more duplicate ACKs arrive until RTT 15, where *cwnd* grows exponentially.

In this moment, a new ACK (not duplicate) arrives that announces the receipt of the lost segment. Reno TCP now moves to the congestion-avoidance state, but first deflates the congestion window to 6 MSS (the *ssthresh* value) as though ignoring the whole fast-recovery state and moving back to the previous track.

Figure 9.50 *Example of a Reno TCP*



NewReno TCP

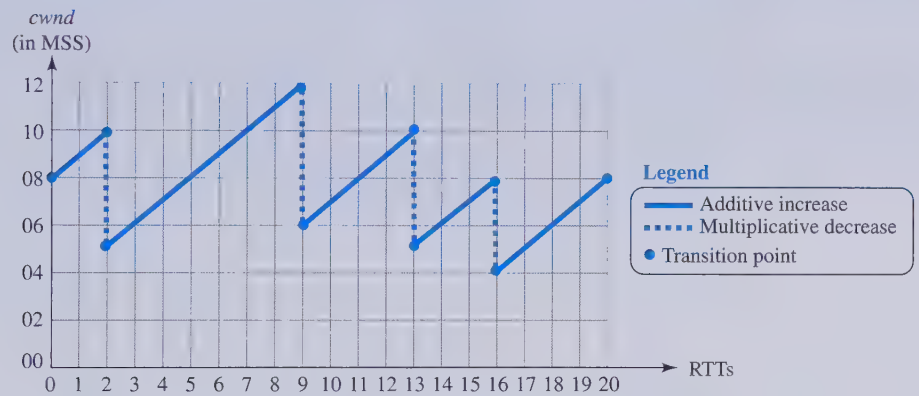
A later version of TCP, called NewReno TCP, made an extra optimization on the Reno TCP. In this version, TCP checks to see if more than one segment is lost in the current window when three duplicate ACKs arrive. When TCP receives three duplicate ACKs, it retransmits the lost segment until a new ACK (not duplicate) arrives. If the new ACK defines the end of the window when the congestion was detected, TCP is certain that only one segment was lost. However, if the ACK number defines a position between the retransmitted segment and the end of the window, it is possible that the segment defined by the ACK is also lost. NewReno TCP retransmits this segment to avoid receiving more and more duplicate ACKs for it.

Additive Increase, Multiplicative Decrease

Out of the three versions of TCP, the Reno version is most common today. It has been observed that, in this version, most of the time the congestion is detected and taken care of by observing the three duplicate ACKs. Even if there are some time-out events, TCP recovers from them by aggressive exponential growth. In other words, in a long TCP connection, if we ignore the slow-start states and short exponential growth during fast

recovery, the TCP congestion window is *cwnd* = *cwnd* + (1/*cwnd*) when an ACK arrives (congestion avoidance) and *cwnd* = *cwnd* / 2 when congestion is detected, as though SS does not exist and the length of FR is reduced to zero. The first is called *additive increase*; the second is called *multiplicative decrease*. This means that the congestion window size, after it passes the initial slow-start state, follows a sawtooth pattern called **additive increase, multiplicative decrease (AIMD)**, as shown in Figure 9.51.

Figure 9.51 *Additive increase, multiplicative decrease (AIMD)*



TCP Throughput

The throughput for TCP, which is based on the congestion window behavior, can be easily found if *cwnd* is a constant (flat line) function of RTT. The throughput with this unrealistic assumption is throughput = *cwnd* / RTT. In this assumption, TCP sends *cwnd* bytes of data and receives acknowledgement for them in RTT time. The behavior of TCP, as shown in Figure 9.51, is not a flat line; it is like saw teeth, with many minimum and maximum values. If each tooth were exactly the same, we could say that the throughput = [(maximum + minimum) / 2] / RTT. However, we know that the value of the maximum is twice the value of the minimum because in each congestion detection the value of *cwnd* is set to half of its previous value. So the throughput can be better calculated as

$$\textbf{Throughput} = (0.75)\, W_{max} / \textbf{RTT}$$

in which W_{max} is the average of window sizes when the congestion occurs.

Example 9.12

If MSS = 10 kbytes and RTT = 100 ms in Figure 9.51, we can calculate the throughput as

$$W_{max} = (10 + 12 + 10 + 8 + 8) / 5 = 9.6 \text{ MSS}$$
$$\text{Throughput} = (0.75 W_{max} / \text{RTT}) = 0.75 \times 960 \text{ kbps} / 100 \text{ ms} = 7.2 \text{ Mbps}$$

9.4.10 TCP Timers

To perform their operations smoothly, most TCP implementations use at least four timers: retransmission, persistence, keepalive, and TIME-WAIT.

Retransmission Timer

To retransmit lost segments, TCP employs one retransmission timer (for the whole connection period) that handles the retransmission time-out (RTO), the waiting time for an acknowledgment of a segment. We can define the following rules for the retransmission timer:

1. When TCP sends the segment in front of the sending queue, it starts the timer.
2. When the timer expires, TCP resends the first segment in front of the queue and restarts the timer.
3. When one or more segments are cumulatively acknowledged, they are purged from the queue.
4. If the queue is empty, TCP stops the timer; otherwise, TCP restarts the timer.

Round-Trip Time (RTT)

To calculate the retransmission time-out (RTO), we first need to calculate the **round-trip time (RTT)**. However, calculating RTT in TCP is an involved process that we explain step by step with some examples.

- **Measured RTT.** We need to find how long it takes to send a segment and receive an acknowledgment for it. This is the measured RTT. We need to remember that the segments and their acknowledgments do not have a one-to-one relationship; several segments may be acknowledged together. The measured round-trip time for a segment is the time required for the segment to reach the destination and be acknowledged, although the acknowledgment may include other segments. Note that in TCP only one RTT measurement can be in progress at any time. This means that if an RTT measurement is started, no other measurement starts until the value of this RTT is finalized. We use the notation RTT_M to stand for measured RTT.

In TCP, there can be only one RTT measurement in progress at any time.

- **Smoothed RTT.** The measured RTT, RTT_M, is likely to change for each round trip. The fluctuation is so high in today's Internet that a single measurement alone cannot be used for retransmission time-out purposes. Most implementations use a smoothed RTT, called RTT_S, which is a weighted average of RTT_M and the previous RTT_S, as shown here:

Initially	→	**No value**
After first measurement	→	$\mathbf{RTT_S = RTT_M}$
After each measurement	→	$\mathbf{RTT_S = (1 - \alpha)\, RTT_S + \alpha \times RTT_M}$

The value of α is implementation-dependent, but it is normally set to 1/8. In other words, the new RTT_S is calculated as 7/8 of the old RTT_S and 1/8 of the current RTT_M.

❑ **RTT Deviation.** Most implementations do not just use RTT_S; they also calculate the RTT deviation, called RTT_D, based on the RTT_S and RTT_M, using the following formulas (the value of β is also implementation-dependent, but is usually set to 1/4):

Initially	→	**No value**
After first measurement	→	$\mathbf{RTT_D = RTT_M / 2}$
After each measurement	→	$\mathbf{RTT_D = (1 - \beta)\, RTT_D + \beta \times |RTT_S - RTT_M|}$

Retransmission Time-out (RTO) The value of RTO is based on the smoothed round-trip time and its deviation. Most implementations use the following formula to calculate the RTO:

Original	→	**Initial value**
After any measurement	→	$\mathbf{RTO = RTT_S + 4 \times RTT_D}$

In other words, take the running smoothed average value of RTT_S and add four times the running smoothed average value of RTT_D (normally a small value).

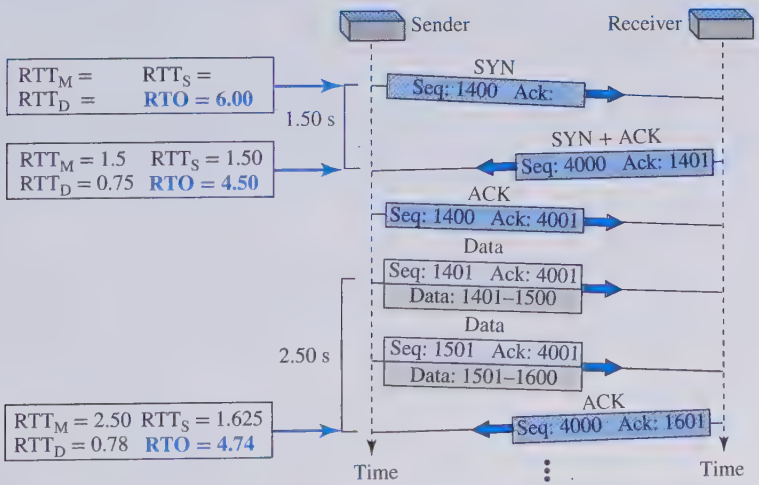
Example 9.13

Let us give a hypothetical example. Figure 9.52 shows part of a connection with the connection establishment and part of the data transfer phases.

1. When the SYN segment is sent, there is no value for RTT_M, RTT_S, or RTT_D. The value of RTO is set to 6.00 s. The following shows the value of these variables at this moment:

$$RTO = 6$$

Figure 9.52 *Example 9.13*



2. When the SYN + ACK segment arrives, RTT_M is measured and is equal to 1.5 s. The following shows the values of these variables:

$$RTT_M = 1.5$$
$$RTT_S = 1.5$$
$$RTT_D = (1.5)/2 = 0.75$$
$$RTO = 1.5 + 4 \times 0.75 = 4.5$$

3. When the first data segment is sent, a new RTT measurement starts. Note that the sender does not start an RTT measurement when it sends the ACK segment, because it does not consume a sequence number and there is no time-out. No RTT measurement starts for the second data segment because a measurement is already in progress. The arrival of the last ACK segment is used to calculate the next value of RTT_M. Although the last ACK segment acknowledges both data segments (cumulative), its arrival finalizes the value of RTT_M for the first segment. The values of these variables are now as follows.

$$RTT_M = 2.5$$
$$RTT_S = (7/8) \times (1.5) + (1/8) \times (2.5) = 1.625$$
$$RTT_D = (3/4) \times (0.75) + (1/4) \times |1.625 - 2.5| = 0.78$$
$$RTO = 1.625 + 4 \times (0.78) = 4.74$$

Karn's Algorithm

Suppose that a segment is not acknowledged during the retransmission time-out period and is therefore retransmitted. When the sending TCP receives an acknowledgment for this segment, it does not know if the acknowledgment is for the original segment or for the retransmitted one. The value of the new RTT is based on the departure of the segment. However, if the original segment was lost and the acknowledgment is for the retransmitted one, the value of the current RTT must be calculated from the time the segment was retransmitted. This ambiguity was solved by Karn. **Karn's algorithm** is simple. Do not consider the round-trip time of a retransmitted segment in the calculation of RTTs. Do not update the value of RTTs until you send a segment and receive an acknowledgment without the need for retransmission.

TCP does not consider the RTT of a retransmitted segment in its calculation of a new RTO.

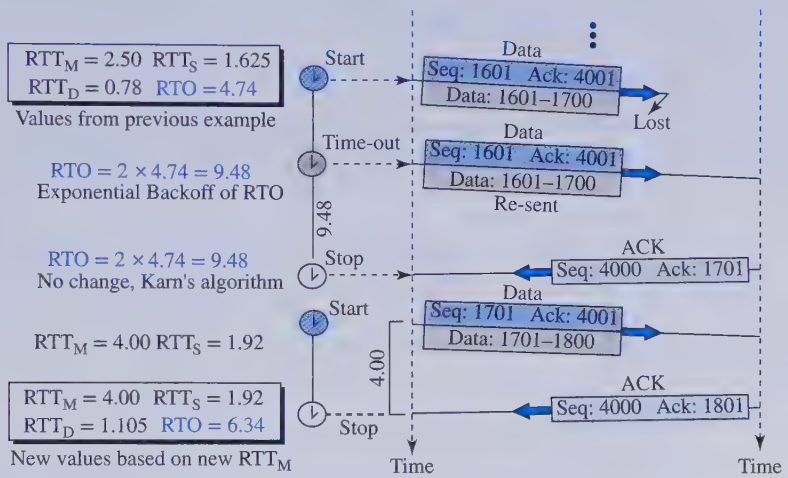
Exponential Backoff

What is the value of RTO if a retransmission occurs? Most TCP implementations use an exponential backoff strategy. The value of RTO is doubled for each retransmission. So if the segment is retransmitted once, the value is 2 times the RTO. If it transmitted twice, the value is 4 times the RTO, and so on.

Example 9.14

Figure 9.53 is a continuation of Example 9.13. There is retransmission and Karn's algorithm is applied.

Figure 9.53 *Example 9.14*



The first segment in the figure is sent, but lost. The RTO timer expires after 4.74 s. The segment is retransmitted, and the timer is set to 9.48, twice the previous value of RTO. This time an ACK is received before the time-out. We wait until we send a new segment and receive the ACK for it before recalculating the RTO (Karn's algorithm).

Persistence Timer

To deal with a zero-window-size advertisement, TCP needs another timer. If the receiving TCP announces a window size of zero, the sending TCP stops transmitting segments until the receiving TCP sends an ACK segment announcing a nonzero window size. This ACK segment can be lost. Remember that ACK segments are not acknowledged nor retransmitted in TCP. If this acknowledgment is lost, the receiving TCP thinks that it has done its job and waits for the sending TCP to send more segments. There is no retransmission timer for a segment containing only an acknowledgment. The sending TCP has not received an acknowledgment and waits for the other TCP to send an acknowledgment advertising the size of the window. Both TCP's might continue to wait for each other forever (a deadlock).

To correct this deadlock, TCP uses a **persistence timer** for each connection. When the sending TCP receives an acknowledgment with a window size of zero, it starts a persistence timer. When the persistence timer goes off, the sending TCP sends a special segment called a *probe*. This segment contains only 1 byte of new data. It has a sequence number, but its sequence number is never acknowledged; it is even ignored in calculating the sequence number for the rest of the data. The probe causes the receiving TCP to resend the acknowledgment.

The value of the persistence timer is set to the value of the retransmission time. However, if a response is not received from the receiver, another probe segment is sent

and the value of the persistence timer is doubled and reset. The sender continues sending the probe segments and doubling and resetting the value of the persistence timer until the value reaches a threshold (usually 60 s). After that the sender sends one probe segment every 60 s until the window is reopened.

Keepalive Timer

A **keepalive timer** is used in some implementations to prevent a long idle connection between two TCPs. Suppose that a client opens a TCP connection to a server, transfers some data, and becomes silent. Perhaps the client has crashed. In this case, the connection remains open forever.

To remedy this situation, most implementations equip a server with a keepalive timer. Each time the server hears from a client, it resets this timer. The time-out is usually 2 h. If the server does not hear from the client after 2 h, it sends a probe segment. If there is no response after 10 probes, each of which is 75 s apart, it assumes that the client is down and terminates the connection.

TIME-WAIT Timer

The TIME-WAIT (2MSL) timer is used during connection termination. The maximum segment life time (MSL) is the amount of time any segment can exist in a network before being discarded. The implementation needs to choose a value for MSL. Common values are 30 s, 1 min, or even 2 min. The 2MSL timer is used when TCP performs an active close and sends the final ACK. The connection must stay for a 2 MSL amount of time to allow TCP to resend the final ACK in case the ACK is lost. This requires that the RTO timer at the other end times out and new FIN and ACK segments are re-sent.

9.4.11 Options

The TCP header can have up to 40 bytes of optional information. Options convey additional information to the destination or align other options. These option are included on the book website for further reference.

TCP options are discussed on the book website.

9.5 SCTP

Stream Control Transmission Protocol (SCTP) is a new transport-layer protocol designed to combine some features of UDP and TCP in an effort to create a better protocol for multimedia communication.

9.5.1 SCTP Services

Before discussing the operation of SCTP, we explain the services offered by SCTP to the application-layer processes.

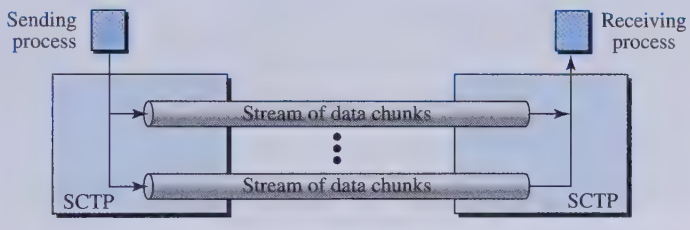
Process-to-Process Communication

SCTP, like UDP or TCP, provides process-to-process communication.

Multiple Streams

We learned that TCP is a stream-oriented protocol. Each connection between a TCP client and a TCP server involves one single stream. The problem with this approach is that a loss at any point in the stream blocks the delivery of the rest of the data. This can be acceptable when we are transferring text; it is not when we are sending real-time data such as audio or video. SCTP allows **multistream service** in each connection, which is called **association** in SCTP terminology. If one of the streams is blocked, the other streams can still deliver their data. Figure 9.54 shows the idea of multiple-stream delivery.

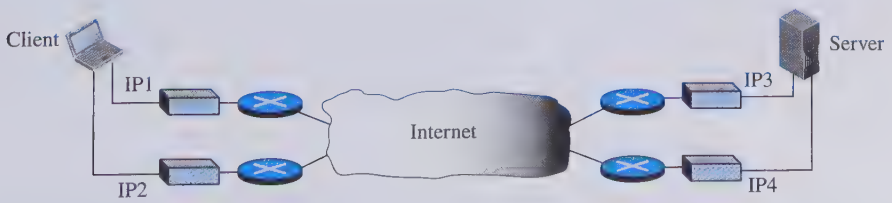
Figure 9.54 *Multiple-stream concept*



Multihoming

A TCP connection involves one source and one destination IP address. This means that even if the sender or receiver is a multihomed host (connected to more than one physical address with multiple IP addresses), only one of these IP addresses per end can be utilized during the connection. An SCTP association, on the other hand, supports **multihoming service**. The sending and receiving host can define multiple IP addresses in each end for an association. In this fault-tolerant approach, when one path fails, another interface can be used for data delivery without interruption. This fault-tolerant feature is very helpful when we are sending and receiving a real-time payload such as Internet telephony. Figure 9.55 shows the idea of multihoming.

Figure 9.55 *Multihoming concept*



In Figure 9.55, the client is connected to two local networks with two IP addresses. The server is also connected to two networks with two IP addresses. The client and the server can make an association using four different pairs of IP addresses. However, note that in the current implementations of SCTP, only one pair of IP addresses can be chosen for normal communication; the alternative is used if the main choice fails. In other words, at present, SCTP does not allow load sharing between different paths.

Full-Duplex Communication

Like TCP, SCTP offers full-duplex service, where data can flow in both directions at the same time. Each SCTP then has a sending and receiving buffer, and packets are sent in both directions.

Connection-Oriented Service

Like TCP, SCTP is a connection-oriented protocol. However, in SCTP, a connection is called an *association*.

Reliable Service

SCTP, like TCP, is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data. We will discuss this feature further in Section 9.5.6 on error control.

9.5.2 SCTP Features

The following shows the general features of SCTP.

Transmission Sequence Number (TSN)

The unit of data in SCTP is a data chunk, which may or may not have a one-to-one relationship with the message coming from the process because of fragmentation (discussed later in Section 9.5.4). Data transfer in SCTP is controlled by numbering the data chunks. SCTP uses a **transmission sequence number (TSN)** to number the data chunks. In other words, the TSN in SCTP plays the analogous role as the sequence number in TCP. TSNs are 32 bits long and randomly initialized between 0 and $2^{32} - 1$. Each data chunk must carry the corresponding TSN in its header.

Stream Identifier (SI)

In SCTP, there may be several streams in each association. Each stream in SCTP needs to be identified using a **stream identifier (SI)**. Each data chunk must carry the SI in its header so that when it arrives at the destination, it can be properly placed in its stream. The SI is a 16-bit number starting from 0.

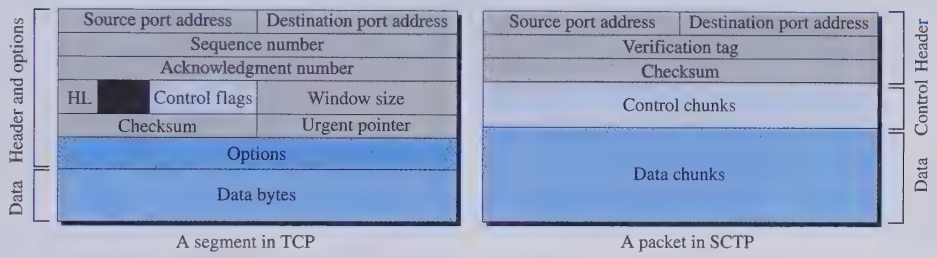
Stream Sequence Number (SSN)

When a data chunk arrives at the destination SCTP, it is delivered to the appropriate stream and in the proper order. This means that, in addition to an SI, SCTP defines each data chunk in each stream with a **stream sequence number (SSN)**.

Packets

In TCP, a segment carries data and control information. Data are carried as a collection of bytes; control information is defined by six control flags in the header. The design of SCTP is totally different: Data are carried as data chunks, and control information is carried as control chunks. Several control chunks and data chunks can be packed together in a packet. A packet in SCTP plays the same role as a segment in TCP. Figure 9.56 compares a segment in TCP and a packet in SCTP. We will discuss the format of the SCTP packet in Section 9.5.4.

Figure 9.56 *Comparison between a TCP segment and an SCTP packet*



In SCTP, we have data chunks, streams, and packets. An association may send many packets, a packet may contain several chunks, and chunks may belong to different streams. To make the definitions of these terms clear, let us suppose that process A needs to send 11 messages to process B in three streams. The first four messages are in the first stream, the second three messages are in the second stream, and the last four messages are in the third stream. Although a message, if long, can be carried by several data chunks, we assume that each message fits into one data chunk. Therefore, we have 11 data chunks in three streams.

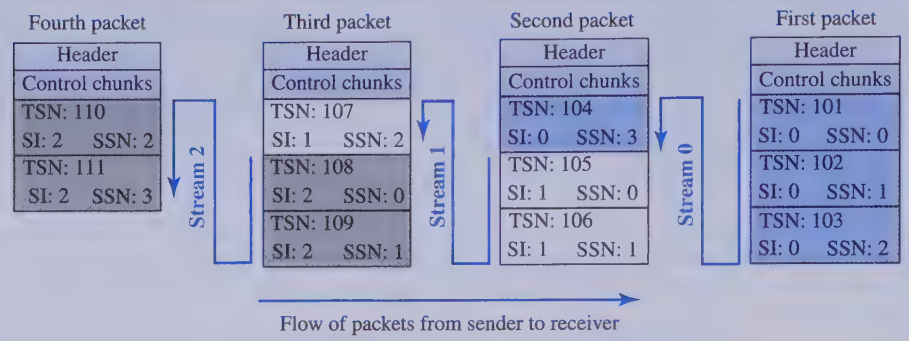
The application process delivers 11 messages to SCTP, where each message is earmarked for the appropriate stream. Although the process could deliver one message from the first stream and then another from the second, we assume that it delivers all messages belonging to the first stream first, all messages belonging to the second stream next, and finally, all messages belonging to the last stream.

We also assume that the network allows only three data chunks per packet, which means that we need four packets, as shown in Figure 9.57.

Data chunks in stream 0 are carried in the first packet and part of the second packet; those in stream 1 are carried in the second and third packet; those in stream 2 are carried in the third and fourth packet.

Note that each data chunk needs three identifiers: TSN, SI, and SSN. TSN is a cumulative number and is used, as we will see later, for flow control and error control. SI defines the stream to which the chunk belongs. SSN defines the chunk's order in a particular stream. In our example, SSN starts from 0 for each stream.

Figure 9.57 *Packets, data chunks, and streams*



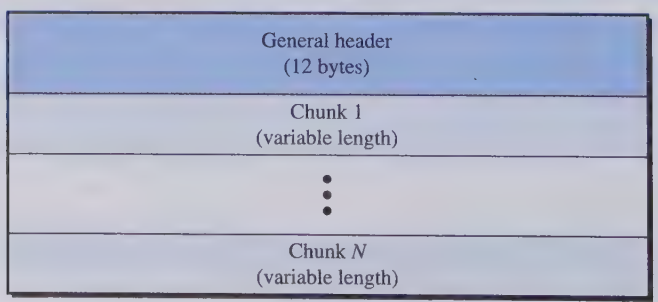
Acknowledgment Number

TCP acknowledgment numbers are **byte-oriented** and refer to the sequence numbers. SCTP acknowledgment numbers are chunk-oriented. They refer to the TSN. A second difference between TCP and SCTP acknowledgments is the control information. Recall that this information is part of the segment header in TCP. To acknowledge segments that carry only control information, TCP uses a sequence number and acknowledgment number (for example, a SYN segment needs to be acknowledged by an ACK segment). In SCTP, however, the control information is carried by control chunks, which do not need a TSN. These control chunks are acknowledged by another control chunk of the appropriate type (some need no acknowledgment). For example, an INIT control chunk is acknowledged by an INIT-ACK chunk. There is no need for a sequence number or an acknowledgment number.

9.5.3 Packet Format

An SCTP packet has a mandatory general header and a set of blocks called chunks. There are two types of chunks: control chunks and data chunks. A control chunk controls and maintains the association; a data chunk carries user data. In a packet, the control chunks come before the data chunks. Figure 9.58 shows the general format of an SCTP packet.

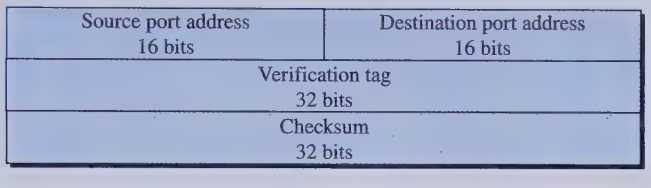
Figure 9.58 *SCTP packet format*



General Header

The *general header* (packet header) defines the endpoints of each association to which the packet belongs, guarantees that the packet belongs to a particular association, and preserves the integrity of the contents of the packet including the header itself. The format of the general header is shown in Figure 9.59.

Figure 9.59 *General header*

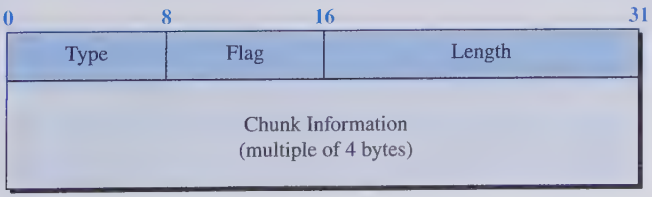


There are four fields in the general header. The source and destination port numbers are the same as in UDP or TCP. The **verification tag** is a 32-bit field that matches a packet to an association. This prevents a packet from a previous association from being mistaken as a packet in this association. It serves as an identifier for the association; it is repeated in every packet during the association. The next field is a checksum. However, the size of the checksum is increased from 16 bits (in UDP, TCP, and IP) to 32 bits in SCTP to allow for the use of the CRC-32 checksum.

Chunks

Control information or user data are carried in chunks. Chunks have a common layout, as shown in Figure 9.60. The first three fields are common to all chunks; the information field depends on the type of chunk. The type field can define up to 256 types of chunks. Only a few have been defined so far; the rest are reserved for future use. The flag field defines special flags that a particular chunk may need. Each bit has a different meaning depending on the type of chunk. The length field defines the total size of the chunk, in bytes, including the type, flag, and length fields. Because the size of the information section is dependent on the type of chunk, we need to define the chunk boundaries. If a chunk carries no information, the value of the length field is 4 (4 bytes). Note that the

Figure 9.60 *Common layout of a chunk*



length of the padding, if any, is not included in the calculation of the length field. This helps the receiver find out how many useful bytes a chunk carries. If the value is not a multiple of 4, the receiver knows there is padding.

Types of Chunks

SCTP defines several types of chunks, as shown in Table 9.3.

Table 9.3 *Chunks*

Type	*Chunk*	*Description*
0	DATA	User data
1	INIT	Sets up an association
2	INIT ACK	Acknowledges INIT chunk
3	SACK	Selective acknowledgment
4	HEARTBEAT	Probes the peer for liveliness
5	HEARTBEAT ACK	Acknowledges HEARTBEAT chunk
6	ABORT	Aborts an association
7	SHUTDOWN	Terminates an association
8	SHUTDOWN ACK	Acknowledges SHUTDOWN chunk
9	ERROR	Reports errors without shutting down
10	COOKIE ECHO	Third packet in association establishment
11	COOKIE ACK	Acknowledges COOKIE ECHO chunk
14	SHUTDOWN COMPLETE	Third packet in association termination
192	FORWARD TSN	For adjusting cumulating TSN

9.5.4 An SCTP Association

SCTP, like TCP, is a connection-oriented protocol. However, a connection in SCTP is called an *association* to emphasize multihoming.

A connection in SCTP is called an association.

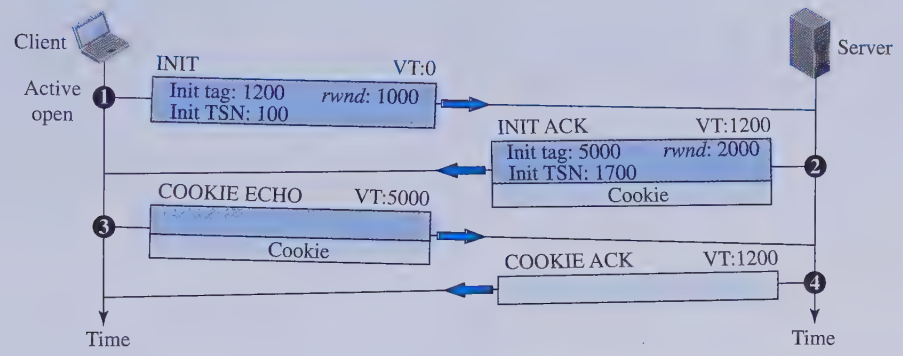
Association Establishment

Association establishment in SCTP requires a *four-way handshake*. In this procedure, a process, normally a client, wants to establish an association with another process, normally a server, using SCTP as the transport-layer protocol. Similar to TCP, the SCTP server needs to be prepared to receive any association (passive open). Association establishment, however, is initiated by the client (active open). SCTP association establishment is shown in Figure 9.61.

The steps, in a normal situation, are as follows:

1. The client sends the first packet, which contains an INIT chunk. The *verification tag* (VT) of this packet (defined in the general header) is 0 because no verification

Figure 9.61 *Four-way handshaking*



tag has yet been defined for this direction (client to server). The INIT tag includes an **initiation tag** to be used for packets from the other direction (server to client). The chunk also defines the **initial TSN** for this direction and advertises a value for *rwnd*. The value of *rwnd* is normally advertised in a SACK chunk; it is done here because SCTP allows the inclusion of a DATA chunk in the third and fourth packets; the server must be aware of the available client buffer size. Note that no other chunks can be sent with the first packet.

2. The server sends the second packet, which contains an INIT ACK chunk. The verification tag is the value of the initial tag field in the INIT chunk. This chunk initiates the tag to be used in the other direction, defines the initial TSN, for data flow from server to client, and sets the server's *rwnd*. The value of *rwnd* is defined to allow the client to send a DATA chunk with the third packet. The INIT ACK also sends a cookie that defines the state of the server at this moment. We will discuss the use of the cookie shortly.
3. The client sends the third packet, which includes a COOKIE ECHO chunk. This is a very simple chunk that echoes, without change, the cookie sent by the server. SCTP allows the inclusion of data chunks in this packet.
4. The server sends the fourth packet, which includes the COOKIE ACK chunk that acknowledges the receipt of the COOKIE ECHO chunk. SCTP allows the inclusion of data chunks with this packet.

Data Transfer

The whole purpose of an association is to transfer data between two ends. After the association is established, bidirectional data transfer can take place. The client and the server can both send data. Like TCP, SCTP supports piggybacking.

There is a major difference, however, between data transfer in TCP and SCTP. TCP receives messages from a process as a stream of bytes without recognizing any boundary between them. The process may insert some boundaries for its peer use, but TCP treats that mark as part of the text. In other words, TCP takes each message and appends

it to its buffer. A segment can carry parts of two different messages. The only ordering system imposed by TCP is the byte numbers.

SCTP, on the other hand, recognizes and maintains boundaries. Each message coming from the process is treated as one unit and inserted into a DATA chunk unless it is fragmented (discussed later). In this sense, SCTP is like UDP, with one big advantage: Data chunks are related to each other.

A message received from a process becomes a DATA chunk, or chunks if fragmented, by adding a DATA chunk header to the message. Each DATA chunk formed by a message or a fragment of a message has one TSN. We need to remember that only DATA chunks use TSNs and only DATA chunks are acknowledged by SACK chunks.

Multihoming Data Transfer

We discussed the multihoming capability of SCTP, a feature that distinguishes SCTP from UDP and TCP. Multihoming allows both ends to define multiple IP addresses for communication. However, only one of these addresses can be defined as the **primary address**; the rest are alternative addresses. The primary address is defined during association establishment. The interesting point is that the primary address of an end is determined by the other end. In other words, a source defines the primary address for a destination.

Data transfer, by default, uses the primary address of the destination. If the primary is not available, one of the alternative addresses is used. The process, however, can always override the primary address and explicitly request that a message be sent to one of the alternative addresses. A process can also explicitly change the primary address of the current association.

A logical question that arises is where to send a SACK. SCTP dictates that a SACK be sent to the address from which the corresponding SCTP packet originated.

Multistream Delivery

One interesting feature of SCTP is the distinction between data transfer and data delivery. SCTP uses TSN numbers to handle data transfer, movement of data chunks between the source and destination. The delivery of the data chunks is controlled by stream identifiers (SIs) and stream sequence numbers (SSNs). SCTP can support multiple streams, which means that the sender process can define different streams and a message can belong to one of these streams. Each stream is assigned an SI that uniquely defines that stream. However, SCTP supports two types of data delivery in each stream: *ordered* (default) and *unordered*. In ordered data delivery, data chunks in a stream use SSNs to define their order in the stream. When the chunks arrive at the destination, SCTP is responsible for message delivery according to the SSN defined in the chunk. This may delay the delivery because some chunks may arrive out of order. In unordered data delivery, the data chunks in a stream have the U flag set, but their SSN field value is ignored. They do not consume SSNs. When an unordered data chunk arrives at the destination SCTP, it delivers the message carrying the chunk to the application without waiting for the other messages. Most of the time, applications use the ordered-delivery service, but occasionally some applications need to send urgent data that must be

delivered out of order (recall the urgent data and urgent pointer facility of TCP). In these cases, the application can define the delivery as unordered.

Fragmentation

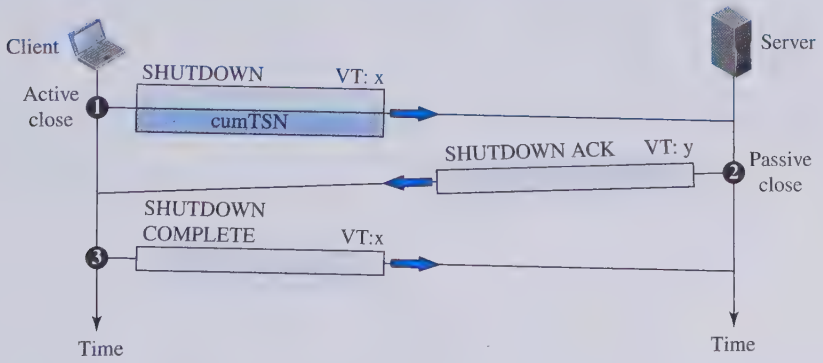
Another issue in data transfer is **fragmentation**. Although SCTP shares this term with IP (see Chapter 7), fragmentation in IP and SCTP belong to different levels: the former at the network layer, the latter at the transport layer.

SCTP preserves the boundaries of the message from process to process when creating a DATA chunk from a message if the size of the message (when encapsulated in an IP datagram) does not exceed the MTU (see Chapter 7) of the path. The size of an IP datagram carrying a message can be determined by adding the size of the message, in bytes, to the four overheads: data chunk header, necessary SACK chunks, SCTP general header, and IP header. If the total size exceeds the MTU, the message needs to be fragmented.

Association Termination

In SCTP, like TCP, either of the two parties involved in exchanging data (client or server) can close the connection. However, unlike TCP, SCTP does not allow a "half-closed" association. If one end closes the association, the other end must stop sending new data. If any data are left over in the queue of the recipient of the termination request, they are sent and the association is closed. Association termination uses three packets, as shown in Figure 9.62. Note that although the figure shows the case in which termination is initiated by the client, it can also be initiated by the server.

Figure 9.62 *Association termination*



9.5.5 Flow Control

Flow control in SCTP is similar to that in TCP. In TCP, we need to deal with only one unit of data, the byte. In SCTP, we need to handle two units of data, the byte and the chunk. The values of *rwnd* and *cwnd* are expressed in bytes; the values of TSN and

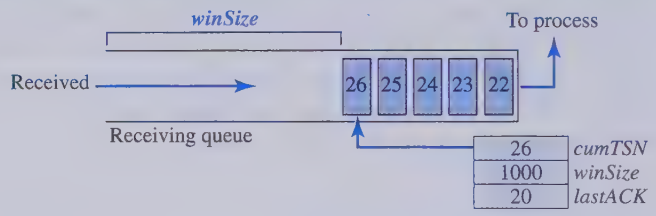
acknowledgments are expressed in chunks. To show the concept, we make some unrealistic assumptions. We assume that there is never congestion in the network and that the network is error-free. In other words, we assume that *cwnd* is infinite and no packet is lost, delayed, or arrives out of order. We also assume that data transfer is unidirectional.

Receiver Site

The receiver has one buffer (queue) and three variables. The queue holds the received data chunks that have not yet been read by the process. The first variable holds the last TSN received, *cumTSN*. The second variable holds the available buffer size, *winSize*. The third variable holds the last cumulative acknowledgment, *lastACK*. Figure 9.63 shows the queue and variables at the receiver site.

1. When the site receives a data chunk, it stores it at the end of the buffer (queue) and subtracts the size of the chunk from *winSize*. The TSN number of the chunk is stored in the *cumTSN* variable.

Figure 9.63 *Flow control, receiver site*



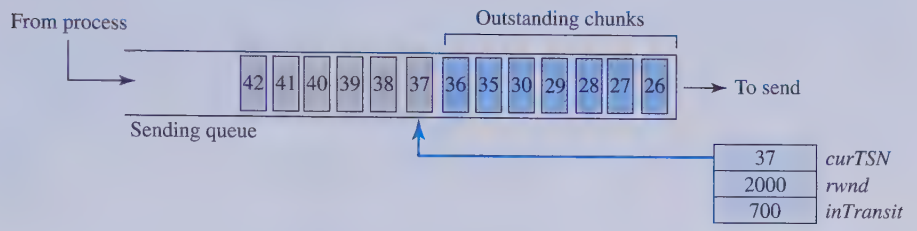
2. When the process reads a chunk, it removes it from the queue and adds the size of the removed chunk to *winSize* (recycling).
3. When the receiver decides to send a SACK, it checks the value of *lastACK;* if it is less than *cumTSN,* it sends a SACK with a cumulative TSN number equal to the *cumTSN*. It also includes the value of *winSize* as the advertised window size. The value of *lastACK* is then updated to hold the value of *cumTSN*.

Sender Site

The sender has one buffer (queue) and three variables: *curTSN*, *rwnd*, and *inTransit*, as shown in Figure 9.64. We assume each chunk is 100 bytes long.

The buffer holds the chunks produced by the process that have either been sent or are ready to be sent. The first variable, *curTSN*, refers to the next chunk to be sent. All chunks in the queue with a TSN less than this value have been sent but not acknowledged; they are outstanding. The second variable, *rwnd*, holds the last value advertised by the receiver (in bytes). The third variable, *inTransit*, holds the number of bytes in transit, bytes sent but not yet acknowledged. The following is the procedure used by the sender.

Figure 9.64 *Flow control, sender site*



1. A chunk pointed to by *curTSN* can be sent if the size of the data is less than or equal to the quantity (*rwnd* – *inTransit*). After sending the chunk, the value of *curTSN* is incremented by 1 and now points to the next chunk to be sent. The value of *inTransit* is incremented by the size of the data in the transmitted chunk.
2. When a SACK is received, the chunks with a TSN less than or equal to the cumulative TSN in the SACK are removed from the queue and discarded. The sender does not have to worry about them anymore. The value of *inTransit* is reduced by the total size of the discarded chunks. The value of *rwnd* is updated with the value of the advertised window in the SACK.

9.5.6 Error Control

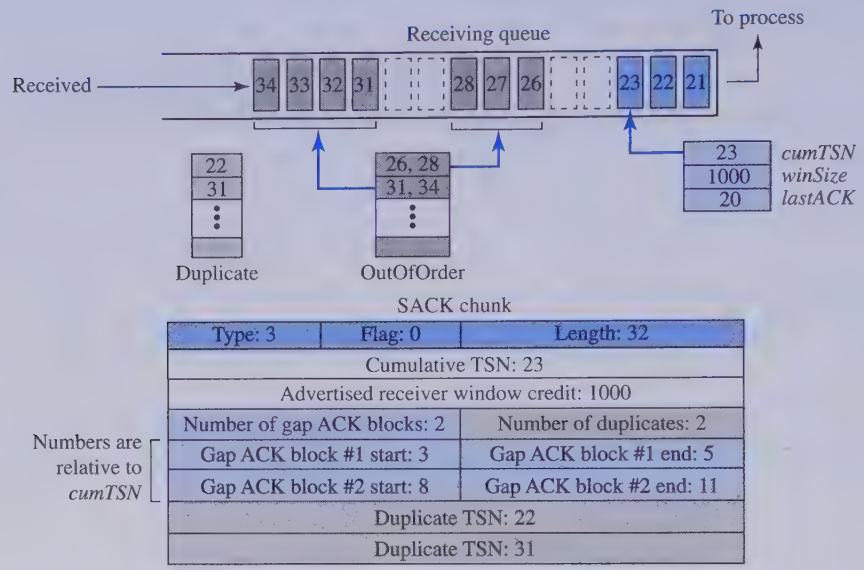
SCTP, like TCP, is a reliable transport-layer protocol. It uses a SACK chunk to report the state of the receiver buffer to the sender. Each implementation uses a different set of entities and timers for the receiver and sender sites. We use a very simple design to convey the concept to the reader.

Receiver Site

In our design, the receiver stores all chunks that have arrived in its queue including the out-of-order ones. However, it leaves spaces for any missing chunks. It discards duplicate messages, but keeps track of them for reports to the sender. Figure 9.65 shows a typical design for the receiver site and the state of the receiving queue at a particular point in time.

The last acknowledgment sent was for data chunk 20. The available window size is 1000 bytes. Chunks 21 to 23 have been received in order. The first out-of-order block contains chunks 26 to 28. The second out-of-order block contains chunks 31 to 34. A variable holds the value of *cumTSN*. An array of variables keeps track of the beginning and the end of each block that is out of order. An array of variables holds the duplicate chunks received. Note that there is no need for storing duplicate chunks in the queue; they will be discarded. The figure also shows the SACK chunk that will be sent to report the state of the receiver to the sender. The TSN numbers for out-of-order chunks are relative (offsets) to the cumulative TSN.

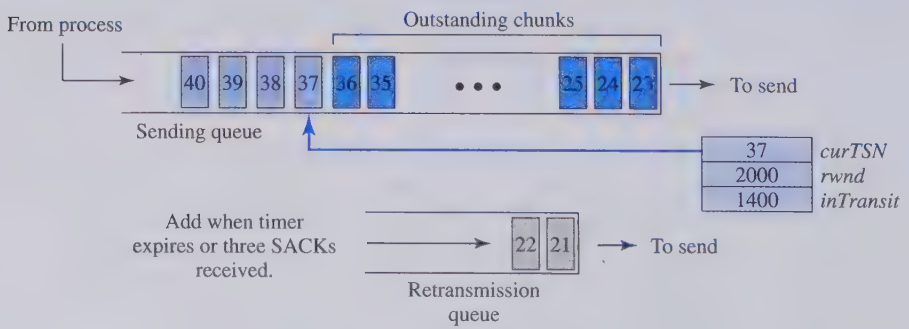
Figure 9.65 *Error control, receiver site*



Sender Site

At the sender site, our design demands two buffers (queues): a sending queue and a retransmission queue. We also use three variables: *rwnd*, *inTransit*, and *curTSN*, as described in Section 9.5.5. Figure 9.66 shows a typical design.

Figure 9.66 *Error control, sender site*



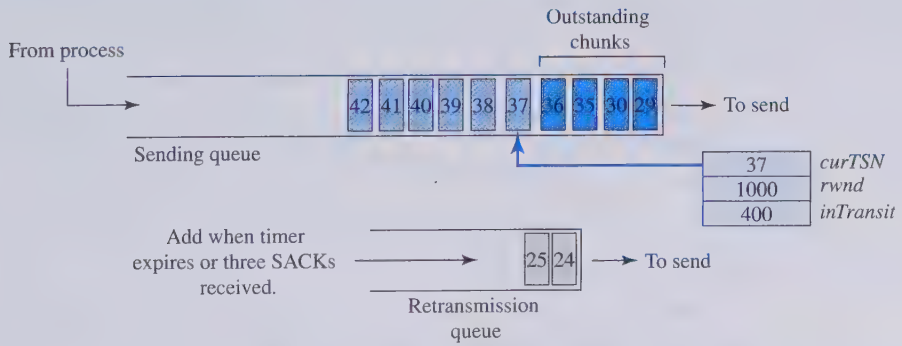
The sending queue holds chunks 23 to 40. Chunks 23 to 36 have already been sent, but not acknowledged; they are outstanding chunks. The *curTSN* points to the

next chunk to be sent (37). We assume that each chunk is 100 bytes, which means that 1400 bytes of data (chunks 23 to 36) are in transit. The sender at this moment has a retransmission queue. When a packet is sent, a retransmission timer starts for that packet. Some implementations use one single timer for the entire association, but we continue with our tradition of one timer for each packet for simplification. When the retransmission timer for a packet expires, or three SACKs arrive that declare a packet as missing (fast retransmission was discussed for TCP), the chunks in that packet are moved to the retransmission queue to be re-sent. These chunks are considered lost, rather than outstanding. The chunks in the retransmission queue have priority. In other words, the next time the sender sends a chunk, it would be chunk 21 from the retransmission queue.

To see how the state of the sender changes, assume that the SACK in Figure 9.65 arrives at the sender site in Figure 9.66. Figure 9.67 shows the new state.

1. All chunks having a TSN equal to or less than *cumTSN* in the SACK are removed from the sending or retransmission queue. They are no longer outstanding or marked for retransmission. Chunks 21 and 22 are removed from the retransmission queue, and chunk 23 is removed from the sending queue.
2. Our design also removes all chunks from the sending queue that are declared in the gap blocks; some conservative implementations, however, save these chunks until a *cumTSN* arrives that includes them. This precaution is needed for the rare occasion when the receiver finds some problem with these out-of-order chunks. We ignore these rare occasions. Chunks 26 to 28 and chunks 31 to 34, therefore, are removed from the sending queue.

Figure 9.67 *New state at the sender site after receiving a SACK chunk*



3. The list of duplicate chunks does not have any effect.
4. The value of *rwnd* is changed to 1000 as advertised in the SACK chunk.
5. We also assume that the transmission timer for the packet that carried chunks 24 and 25 has expired. These move to the retransmission queue, and a new retransmission timer is set according to the exponential backoff rule discussed for TCP.

6. The value of *inTransit* becomes 400 because only four chunks are now in transit. The chunks in the retransmission queue are not counted because they are assumed lost, not in transit.

Sending Data Chunks

An end can send a data packet whenever there are data chunks in the sending queue with a TSN greater than or equal to *curTSN* or if there are data chunks in the retransmission queue. The retransmission queue has priority. However, the total size of the data chunk or chunks included in the packet must not exceed the (*rwnd* − *inTransit*) value and the total size of the frame must not exceed the MTU size. If we assume, in our previous scenario, that our packet can take three chunks (due to the MTU restriction), then chunks 24 and 25 from the retransmission queue and chunk 37, the next chunk ready to be sent in the sending queue, can be sent. Note that the outstanding chunks in the sending queue cannot be sent; they are assumed to be in transit. Note also that any chunk sent from the retransmission queue is also timed for retransmission again. The new timer affects chunks 24, 25, and 37. We need to mention here that some implementations may not allow mixing chunks from the retransmission queue and the sending queue. In this case, only chunks 24 and 25 can be sent in the packet. (The format of the data chunk is on the book website.)

Retransmission

To control a lost or discarded chunk, SCTP, like TCP, employs two strategies: using retransmission timers and receiving three SACKs with the same missing chunks.

- **Retransmission Timers.** SCTP uses a retransmission timer, which handles the retransmission time, the waiting time for an acknowledgment of a segment. The procedures for calculating RTO and RTT in SCTP are the same as we described for TCP. SCTP uses a measured RTT (RTTM), a smoothed RTT (RTTS), and an RTT deviation (RTTD) to calculate the RTO. SCTP also uses Karn's algorithm to avoid acknowledgment ambiguity. Note that if a host is using more than one IP address (multihoming), separate RTOs must be calculated and kept for each path.
- **Four Missing Reports.** Whenever a sender receives four SACKs whose gap ACK information indicates one or more specific data chunks are missing, the sender needs to consider those chunks as lost and immediately move them to the retransmission queue. This behavior is analogous to "fast retransmission" in TCP.

Generating SACK Chunks

Another issue in error control is the generation of SACK chunks. The rules for generating SCTP SACK chunks are similar to the rules used for acknowledgment with the TCP ACK flag. We summarize the rules as follows:

1. When an end sends a DATA chunk to the other end, it must include a SACK chunk advertising the receipt of unacknowledged DATA chunks.
2. When an end receives a packet containing data, but has no data to send, it needs to acknowledge the receipt of the packet within a specified time (usually 500 ms).
3. An end must send at least one SACK for every other packet it receives. This rule overrides the second rule.

4. When a packet arrives with out-of-order data chunks, the receiver needs to immediately send a SACK chunk reporting the situation to the sender.
5. When an end receives a packet with duplicate DATA chunks and no new DATA chunks, the duplicate data chunks must be reported immediately with a SACK chunk.

Congestion Control

SCTP, like TCP, is a transport-layer protocol with packets subject to congestion in the network. The SCTP designers have used the same strategies for congestion control as those used in TCP.

9.6 END-OF-CHAPTER MATERIALS

9.6.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and Requests for Comments (RFCs). The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give information about transport-layer protocols. In particular, we recommend [Com 06], [Pet & Dav 03], [GW 04], [Far 04], [Tan 03], and [Sta 04].

Requests For Comments

The main RFC related to UDP is RFC 768. Several RFCs discuss TCP protocol, including RFC 793, RFC 813, RFC 879, RFC 889, RFC 896, RFC 1122, RFC 1975, RFC 1987, RFC 1988, RFC 1993, RFC 2018, RFC 2581, RFC 3168, and RFC 3782.

9.6.2 Key Terms

additive increase, multiplicative decrease (AIMD)
association
byte-oriented
Clark's solution
client/server paradigm
congestion
congestion avoidance
congestion control
cookie
deadlock
demultiplexing
denial of service attack
ephemeral port number
fast-recovery algorithm
fast retransmission
finite state machine (FSM)
fragmentation
Go-Back-*N* protocol
half-close
initial sequence number (ISN)
initial TSN
initiation tag
Karn's algorithm
keepalive timer
multihoming service
multiplexing
multistream service
Nagle's algorithm
persistence timer
piggybacking
port number

primary address
process-to-process communication
retransmission time-out (RTO)
round-trip time (RTT)
segment
Selective-Repeat (SR) protocol
sequence number
silly window syndrome
sliding window
slow-start algorithm
socket address
Stream Control Transmission Protocol (SCTP)
stream identifier (SI)
stream sequence number (SSN)
SYN flooding attack
three-way handshaking
Transmission Control Protocol (TCP)
transmission sequence number (TSN)
user datagram
User Datagram Protocol (UDP)
verification tag
well-known port number

9.6.3 Summary

The main duty of a transport-layer protocol is to provide process-to-process communication. To define the processes, we need port numbers. The client program defines itself with an ephemeral port number. The server defines itself with a well-known port number. To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages. The transport layer at the source performs multiplexing; the transport layer at the destination performs demultiplexing. Flow control balances the exchange of data items between a producer and a consumer.

UDP is an unreliable and connectionless transport-layer protocol that creates a process-to-process communication, which means it requires little overhead and offers fast delivery. The UDP packet is called a user datagram. UDP has no flow- or error-control mechanism; its only attempt at error control is the checksum. A user datagram is encapsulated in the data field of an IP datagram. UDP uses multiplexing and demultiplexing to handle outgoing and incoming user datagrams. A UDP package can involve five components: a control-block table, a control-block module, input queues, an input module, and an output module.

Transmission Control Protocol (TCP) is another transport layer protocol in the TCP/IP protocol suite. It provides process-to-process, full-duplex, and connection-oriented service. A TCP connection consists of three phases: connection establishment, data transfer, and connection termination. TCP software is normally implemented as a finite state machine (FSM). TCP uses flow control, implemented as a sliding window mechanism, to avoid overwhelming a receiver with data. The TCP window size is determined by the receiver-advertised window size (*rwnd*) or the congestion window size (*cwnd*), whichever is smaller. The bytes of data being transferred in each connection are numbered by TCP. The numbering starts with a randomly generated number. TCP uses error control to provide a reliable service. Error control is handled by checksums, acknowledgment, and time-outs. In modern implementations, a retransmission occurs if the retransmission timer expires or three duplicate ACK segments have arrived. TCP uses congestion control to avoid and detect congestion in the network. The slow start (exponential increase), congestion avoidance (additive increase), and congestion detection (multiplicative decrease) strategies are used for congestion control.

SCTP is a reliable protocol that combines the good features of UDP and TCP. SCTP provides additional services not provided by UDP or TCP, such as multiple-stream and multihoming services. SCTP is a connection-oriented protocol, in which connection is

called an association. SCTP provides flow control, error control, and congestion control. To distinguish between different streams, SCTP uses the sequence identifier (SI). To distinguish between different data chunks belonging to the same stream, SCTP uses the stream sequence number (SSN). The SCTP acknowledgment SACK reports the cumulative TSN, the TSN of the last data chunk received in order, and selective TSN that have been received.

9.7 PRACTICE SET

9.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

9.7.2 Questions

Q9-1. Assume we have a set of dedicated computers in a system, each designed to perform only a single task. Do we still need host-to-host and process-to-process communication and two levels of addressing?

Q9-2. Operating systems assign a process number to every running application program. Can you explain why these process numbers cannot be used instead of port numbers?

Q9-3. Assume you need to write and test a client/server application program on two hosts you have at home.

- **a.** What is the range of port numbers you would choose for the client program?
- **b.** What is the range of port numbers you would choose for the server program?
- **c.** Can the two port numbers be the same?

Q9-4. Assume a new organization needs to create a new server process and allow its customers to access the organization site using that process. How should the port number for the server process be selected?

Q9-5. Can you explain why some transport-layer packets may be received out of order in the Internet?

Q9-6. Can you explain why some transport-layer packets may be lost in the Internet?

Q9-7. Can you explain why some transport-layer packets may be duplicated in the Internet?

Q9-8. Some of the application programs can use the services of two transport-layer protocols (UDP or TCP). When a packet arrives at the destination, how can the computer find which transport layer is involved?

Q9-9. A client residing on a host with IP address 122.45.12.7 sends a message to the corresponding server residing on a host with IP address 200.112.45.90. If the well-known port is 161 and the ephemeral port is 51,000, what are the pair of socket addresses used in this communication?

Q9-10. UDP is a message-oriented protocol. TCP is a byte-oriented protocol. If an application needs to protect the boundaries of its message, which protocol should be used, UDP or TCP?

Q9-11. Assume a private internet, which uses point-to-point communication between the hosts and needs no routing, has totally eliminated the use of the network layer. Can this internet still benefit from the service of UDP or TCP? In other words, can user datagrams or segments be encapsulated in the Ethernet frames?

Q9-12. Assume a private internet uses a protocol suite totally different from the TCP/IP protocol suite. Can this internet still use the service of UDP or TCP as an end-to-end vehicle of message communication?

Q9-13. Can you explain why we need four (or sometimes three) segments for connection termination in TCP?

Q9-14. In TCP, some segment types can be used only for control; they cannot be used to carry data at the same time. Define some of these segments.

Q9-15. In TCP, how do we define the sequence number of a segment (in each direction)? Consider two cases: the first segment and other segments.

Q9-16. In TCP, we have two consecutive segments. Assume the sequence number of the first segment is 101. What is the sequence number of the next segment in each of the following cases?

a. The first segment does not consume any sequence numbers.

b. The first segment consumes 10 sequence numbers.

Q9-17. In TCP, how many sequence numbers are consumed by each of the following segments?

a. SYN **b.** ACK **c.** SYN + ACK **d.** Data

Q9-18. Explain why in TCP each SYN, SYN + ACK, and FIN segment consumes a sequence number, but an ACK segment carrying no data does not consume a sequence number.

Q9-19. Looking at the TCP header (Figure 9.23), we find that the sequence number is 32 bits long, while the window size is only 16 bits long. Does this mean that TCP is closer to the Go-Back-*N* or Select-Repeat protocol in this respect?

Q9-20. The maximum window size of the TCP was originally designed to be 64 kbytes (which means $64 \times 1024 = 65{,}536$ or actually 65,535). Can you think of a reason for this?

Q9-21. What is the maximum size of the TCP header? What is the minimum size of the TCP header?

Q9-22. In TCP, does a SYN segment open a connection in only one direction or in both directions?

Q9-23. In TCP, does a FIN segment close a connection in only one direction or in both directions?

Q9-24. In TCP, what type of flag can totally close the communication in both directions?

Q9-25. Most of the flags can be used together in a segment. Give an example of two flags that cannot be used simultaneously because they are ambiguous.

Q9-26. Assume a client sends a SYN segment to a server. When the server checks the well-known port number, it finds that no process defined by the port number is running. What is the server supposed to do in this case?

Q9-27. Can you explain how TCP, which uses the services provided by the unreliable IP, can provide reliable communication?

Q9-28. We said that TCP provides a connection-oriented service between the two application programs. A connection in this case needs a connection identifier that distinguishes one connection from another. What do you think the unique connection identifier is in this case?

Q9-29. Assume Alice uses her browser to open two connections to the HTTP server running on Bob's server. How can these two connections be distinguished by the TCP?

Q9-30. We used *passive open* and *active open* terms in discussing a connection-oriented communication using TCP. Assume there is a telephone conversation between Alice and Bob. Because a telephone conversation is an example of a connection-oriented communication, assume Alice calls Bob and they talk on the telephone. Who is making a *passive open* connection in this case? Who is making an *active open* connection in this case?

Q9-31. In TCP, can the sender window be smaller, larger, or the same size as the receiver window?

Q9-32. Can you mention some tasks that can be done by one or a combination of TCP segments?

Q9-33. In a TCP segment, what does a sequence number identify?

Q9-34. In a TCP segment, what does an acknowledgment number identify?

Q9-35. Is the use of checksum for error control optional or mandatory in the following?

a. UDP **b.** TCP

Q9-36. Assume a TCP server expects to receive byte 2001, but it receives a segment with sequence number 2200. What is the reaction of the TCP server to this event? Can you justify the reaction?

Q9-37. Assume a TCP client expects to receive byte 2001, but it receives a segment with sequence number 1201. What is the reaction of the TCP client to this event? Can you justify the reaction?

Q9-38. Assume a TCP server is missing bytes 2001 to 3000. The server receives a segment with sequence number 2001 that carries 400 bytes. What is the reaction of the TCP server to this event? Can you justify the reaction?

Q9-39. Assume a TCP server is expecting to receive byte 2401. It receives a segment with the sequence number 2401 that carries 500 bytes. If the server has no data to send at this moment and has not acknowledged the previous segment, what is the reaction of the TCP server to this event? Can you justify the reaction?

Q9-40. Assume a TCP client is expecting to receive byte 3001. It receives a segment with the sequence number 3001 that carries 400 bytes. If the client has no data to sent at this moment and has acknowledged the previous segment, what is the reaction of the TCP client to this event? Can you justify the reaction?

Q9-41. Assume a TCP server is expecting to receive byte 6001. It receives a segment with the sequence number 6001 that carries 2000 bytes. If the server has bytes 4001 to 5000 to send, what should the reaction of the TCP server be to this event? Can you justify the reaction?

Q9-42. The first rule of generating ACKs for TCP is not shown in either Figure 9.38 or 3.60. Can you explain the reason for this?

Q9-43. Which of the six rules we described for ACK generation in TCP can be applied to the case when a server receives a SYN segment from a client?

Q9-44. Which of the six rules we described for ACK generation in TCP can be applied to the case when a client receives a SYN + ACK segment from a server?

Q9-45. Which of the six rules we described for ACK generation can be applied to the case when a client or a server receives a FIN segment from the other end?

Q9-46. In SCTP, a packet is carrying two DATA chunks, each containing 22 bytes of user data. What is the size of each DATA chunk? What is the total size of the packet?

Q9-47. In SCTP, a SACK chunk reports the receipt of three out-of-order data chunks and five duplicate data chunks. What is the total size of the chunk in bytes?

Q9-48. In SCTP, a packet is carrying a COOKIE ECHO message and a DATA chunk. If the size of the cookie is 200 bytes and that of the user data is 20 bytes, what is the size of the packet?

Q9-49. In SCTP, a packet is carrying a COOKIE ACK message and a DATA chunk. If the user data is 20 bytes, what is the size of the packet?

Q9-50. In SCTP, the value of the cumulative TSN in a SACK is 23. The value of the previous cumulative TSN in the SACK was 29. What is the problem?

9.7.3 Problems

P9-1. Compare the range of 16-bit addresses, 0 to 65,535, with the range of 32-bit IP addresses, 0 to 4,294,967,295 (discussed in Chapter 7). Why do we need such a large range of IP addresses, but only a relatively small range of port numbers?

P9-2. Explain why ICANN has divided the port numbers into three groups: well-known, registered, and dynamic.

P9-3. A sender sends a series of packets to the same destination using 5-bit sequence numbers. If the sequence numbers start with 0, what is the sequence number of the 100th packet?

P9-4. Answer the following questions:

a. What is the minimum size of a UDP user datagram?

b. What is the maximum size of a UDP user datagram?

c. What is the minimum size of the application-layer payload data that can be encapsulated in a UDP user datagram?

d. What is the maximum size of the application-layer payload that can be encapsulated in a UDP user datagram?

P9-5. A client uses UDP to send data to a server. The data length is 16 bytes. Calculate the efficiency of this transmission at the UDP level (ratio of useful bytes to total bytes).

P9-6. The following is a dump (contents) of a UDP header in hexadecimal format.

0045DF0000580000

a. What is the source port number?

b. What is the destination port number?
c. What is the total length of the user datagram?
d. What is the length of the data?
e. Is the packet directed from a client to a server, or vice versa?
f. What is the application-layer protocol?
g. Has the sender calculated a checksum for this packet?

P9-7. Compare the TCP header and the UDP header. List the fields in the TCP header that are not part of the UDP header. Give the reason for each missing field.

P9-8. In TCP, if the value of HLEN is 0111, how many bytes of options are included in the segment?

P9-9. What can you say about each of the following TCP segments with the following control field values?

a. 000000 **c.** 010001 **e.** 000010
b. 000001 **d.** 000100 **f.** 010010

P9-10. The control field in a TCP segment is 6 bits. We can have 64 different combinations of bits. List some combinations that you think are normally used.

P9-11. The following is part of a TCP header dump (contents) in hexadecimal format.

E293 0017 00000001 00000000 5002 07FF...

a. What is the source port number?
b. What is the destination port number?
c. What is the sequence number?
d. What is the acknowledgment number?
e. What is the length of the header?
f. What is the type of the segment?
g. What is the window size?

P9-12. To better understand the need for the three-handshake connection establishment, let us go through a scenario. Alice and Bob have no access to telephones or the Internet (think about the old days) to establish their next meeting at a place far from their homes.

a. Suppose that Alice sends a letter to Bob and defines the day and the time of their meeting. Can Alice go to the meeting place and be sure that Bob will be there?
b. Suppose that Bob responds to Alice's request with a letter and confirms the date and time. Can Bob go to the meeting place and be sure that Alice will be there?
c. Suppose that Alice responds to Bob's letter and confirms the same date and time. Can either one go to the meeting and be sure that the other person will be there?

P9-13. To make the initial sequence number a random number, most systems start the counter at 1 during bootstrap and increment the counter by 64,000 every half second. How long does it take for the counter to wrap around?

P9-14. In a TCP connection, the initial sequence number at the client site is 2171. The client opens the connection, sends three segments, the second of which

carries 1000 bytes of data, and closes the connection. What is the value of the sequence number in each of the following segments sent by the client?

a. SYN segment
b. data segment
c. FIN segment

P9-15. In a connection, the value of *cwnd* is 3000 and the value of *rwnd* is 5000. The host has sent 2000 bytes, which have not been acknowledged. How many more bytes can be sent?

P9-16. A client uses TCP to send data to a server. The data consist of 16 bytes. Calculate the efficiency of this transmission at the TCP level (ratio of useful bytes to total bytes).

P9-17. TCP is sending data at 1Mbyte/s. If the sequence number starts with 7000, how long does it take before the sequence number goes back to zero?

P9-18. Eve, the intruder, sends a SYN segment to Bob, the server, using Alice's IP address. Can Eve create a TCP connection with Bob by pretending that she is Alice? Assume that Bob uses a different ISN for each connection.

P9-19. Eve, the intruder, sends a user datagram to Bob, the server, using Alice's IP address. Can Eve, pretending to be Alice, receive the response from Bob?

P9-20. Assume Alice, the client, creates a connection with Bob, the server. They exchange data and close the connection. Now Alice starts a new connection with Bob by sending a new SYN segment. Before Bob responds to this SYN segment, a duplicate copy of the old SYN segment from Alice, which is wandering in the network, arrives at Bob's computer, initiating a SYN + ACK segment from Bob. Can this segment be mistaken by Alice's computer as the response to the new SYN segment? Explain.

P9-21. Assume Alice, the client, creates a TCP connection with Bob, the server. They exchange data and close the connection. Now Alice starts a new connection with Bob by sending a new SYN segment. The server responds with the SYN + ACK segment. However, before Bob receives the ACK for this connection from Alice, a duplicate old ACK segment from Alice arrives at Bob's site. Can this old ACK be confused with the ACK segment Bob is expecting from Alice?

P9-22. In TCP, assume a client has 100 bytes to send. The client creates 10 bytes at a time in each 10 ms and delivers them to the transport layer. The server acknowledges each segment immediately or if a timer times out at 50 ms. Show the segments and the bytes each segment carries if the implementation uses Nagle's algorithm with a maximum segment size (MSS) of 30 bytes. The round-trip time is 20 ms, but the sender timer is set to 100 ms. Does any segment carry the maximum segment size? Is Nagler's algorithm really effective here? Why?

P9-23. As we have explained in the text, the TCP sliding window, when used without new SACK options, is a combination of the Go-Back-*N* and the Selective Repeat protocols. Explain which aspects of the TCP sliding window are close to the Go-Back-*N* and which aspects are close to the Selective Repeat protocol.

P9-24. Although new TCP implementations use the SACK option to report the out-of-order and duplicate range of bytes, explain how old implementations can indicate that the bytes in a received segment are out of order or duplicate.

P9-25. In a TCP connection, assume that the maximum segment size (MSS) is 1000 bytes. The client process has 5400 bytes to send to the server process, which has no bytes to respond (unidirectional communication). The TCP server generates ACKs according to the rules we discussed in the text. Show the time line for the transactions during the slow-start phase, indicating the value of *cwnd* at the beginning, at the end, and after each change. Assume that each segment header is only 20 bytes.

P9-26. The *ssthresh* value for a Taho TCP station is set to 6 MSS. The station now is in the slow-start state with *cwnd* = 4 MSS. Show the values of *cwnd*, *sstresh*, and the state of the station before and after each of following events: four consecutive nonduplicate ACKs arrived followed by a time-out and three nonduplicate ACKs.

P9-27. The *ssthresh* value for a Reno TCP station is set to 8 MSS. The station is now in the slow-start state with *cwnd* = 5 MSS and *ssthresh* = 8 MSS. Show the values of *cwnd*, *sstresh*, and the current and the next state of the station after the following events: three consecutive nonduplicate ACKs arrived, followed by five duplicate ACKs, followed by two nonduplicate ACKs, and followed by a time-out.

P9-28. In a TCP connection, the window size fluctuates between 60,000 and 30,000 bytes. If the average RTT is 30 ms, what is the throughput of the connection?

P9-29. If originally $RRT_s = 14$ ms and α is set to 0.2, calculate the new RTT_s after the following events (times are relative to event 1):

Event 1: 00 ms	Segment 1 was sent.
Event 2: 06 ms	Segment 2 was sent.
Event 3: 16 ms	Segment 1 was timed-out and re-sent.
Event 4: 21 ms	Segment 1 was acknowledged.
Event 5: 23 ms	Segment 2 was acknowledged.

P9-30. An SCTP association is in the **ESTABLISHED** state. It receives a SHUTDOWN chunk. If the host does not have any outstanding or pending data, what does it need to do?

P9-31. An SCTP association is in the **COOKIE-WAIT** state. It receives an INIT chunk; what does it need to do?

P9-32. In SCTP, the following is a dump of a DATA chunk in hexadecimal format.

```
00000015 00000005 0003000A 00000000 48656C6C 6F000000
```

a. Is this an ordered or unordered chunk?
b. Is this the first, the last, the middle, or the only fragment?
c. How many bytes of padding are carried by the chunk?
d. What is the TSN?
e. What is the SI?
f. What is the SSN?
g. What is the message?

P9-33. The following is a dump of an SCTP general header in hexadecimal format.

```
04320017 00000001 00000000
```

a. What is the source port number?
b. What is the destination port number?
c. What is the value of the verification tag?
d. What is the value of the checksum?

P9-34. In SCTP, the state of a receiver is as follows:
a. The receiving queue has chunks 1 to 8, 11 to 14, and 16 to 20.
b. There are 1800 bytes of space in the queue.
c. The value of *lastACK* is 4.
d. No duplicate chunk has been received.
e. The value of *cumTSN* is 5.
Show the contents of the receiving queue and the variables. Show the contents of the SACK message sent by the receiver.

P9-35. In SCTP, the state of a sender is as follows:
a. The sending queue has chunks 18 to 23.
b. The value of *curTSN* is 20.
c. The value of the window size is 2000 bytes.
d. The value of *inTransit* is 200.
If each data chunk contains 100 bytes of data, how many DATA chunks can be sent now? What is the next data chunk to be sent?

P9-36. An SCTP client opens an association using an initial tag of 806, an initial TSN of 14,534, and a window size of 20,000. The server responds with an initial tag of 2000, an initial TSN of 670, and a window size of 14,000. Show the contents of all four packets exchanged during association establishment. Ignore the value of the cookie.

CHAPTER 10

Application Layer

The whole Internet, hardware and software, was designed and developed to provide services at the application layer. The fifth layer of the TCP/IP protocol suite is where these services are provided for Internet users. The other four layers are there to make these services possible.

During the lifetime of the Internet many application protocols have been created and used. Some of these applications were intended for a specific use and have never become standards. Some have been deprecated. Some have been modified or replaced by new ones. Several have survived and become standard applications. New application protocols are being added constantly to the Internet.

This chapter is divided into five sections.

- In the first section, we introduce the nature of services provided by the Internet and introduce two categories of applications: the traditional one, based on the *client/server paradigm*, and the new one, based on the *peer-to-peer paradigm*.
- In the second section, we discuss the concept of the client/server paradigm and how this paradigm provides services to Internet users.
- In the third section, we discuss some predefined or standard applications based on the client/server paradigm. We also discuss some popular applications such as surfing the Web, file transfer, and e-mail.
- In the fourth section, we discuss the concept and protocols in the peer-to-peer paradigm. We introduce some protocols such as Chord, Pastry, and Kademlia. We also mention some popular applications that use these protocols.
- In the fifth section, we show how a new application can be created in the client/server paradigm by writing two programs in the C language, one for a client and one for a server.

10.1 INTRODUCTION

The application layer provides services to the user. Communication is provided using a logical connection, which means that the two application layers assume that there is an imaginary direct connection through which they can send and receive messages. Figure 10.1 shows the idea behind this logical connection.

Figure 10.1 *Logical connection at the application layer*

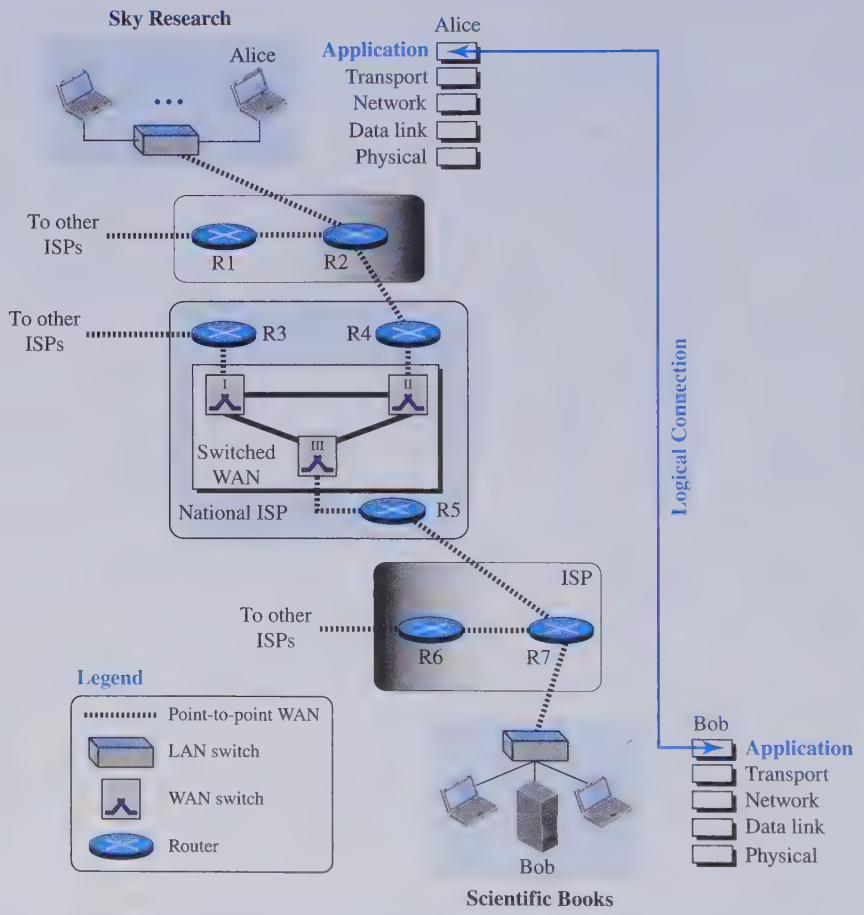


Figure 10.1 shows a scenario in which a scientist, Alice, working at a research company, Sky Research, needs to order a book related to her research from an online bookseller, Scientific Books, whose host is named Bob. A logical connection takes place between the application layer at Alice's computer at Sky Research and the application layer of Bob's computer server at Scientific Books. The communication at the

application layer is logical, not physical. Alice and Bob assume that there is a two-way logical channel between them through which they can send and receive messages. The actual communication, however, takes place through several devices (Alice, R2, R4, R5, R7, and Bob) and several physical channels as shown in Figure 10.1.

10.1.1 Providing Services

All communication networks that started before the Internet were designed to provide services to network users. Most of these networks, however, were originally designed to provide one specific service. For example, the telephone network was originally designed to provide voice service: to allow people all over the world to talk to each other. This telephone network, however, was later used for some other services, such as facsimile (fax), which was enabled by users adding some extra hardware at both ends.

The Internet was originally designed for the same purpose: to provide service to users around the world. The layered architecture of the TCP/IP protocol suite, however, makes the Internet more flexible than other communication networks such as the postal or telephone networks. Each layer in the suite was originally made up of one or more protocols, but new protocols can be added or some protocols can be removed or replaced by the Internet authorities. However, if a protocol is added to each layer, it should be designed in such a way that it uses the service provided by one of the protocols at the lower layer. If a protocol is removed from a layer, care should be taken to change the protocol at the next higher layer that supposedly uses the service of the removed protocol.

The application layer, however, is somehow different from other layers in that it is the highest layer in the suite. The protocols in this layer do not provide services to any other protocol in the suite; they only receive services from the protocols in the transport layer. This means that protocols can be removed from this layer easily. New protocols can be also added to this layer as long as the new protocol can use the service provided by one of the transport-layer protocols.

Because the application layer is the only layer that provides services to the Internet user, the flexibility of the application layer, as just described, allows new application protocols to be easily added to the Internet, which has been occurring during the lifetime of the Internet. When the Internet was created, only a few application protocols were available to users; today we cannot give a number for these protocols because new ones are being added constantly.

Standard and Nonstandard Protocols

To provide a smooth operation of the Internet, the protocols used in the first four layers of the TCP/IP protocol suite need to be standardized and documented. They normally become part of the package that is included in operating systems such as Windows or UNIX. To be flexible, however, the application-layer protocols can be both standard and nonstandard.

Standard Application-Layer Protocols

There are several application-layer protocols that have been standardized and documented by the Internet authority, and we are using them in our daily interaction with the Internet. Each standard protocol is a pair of computer programs that interact with the user

and the transport layer to provide a specific service to the user. We will discuss some of these standard applications in this chapter. In the case of these application protocols, we should know what type of services they provide, how they work, the options that we can use with these applications, and so on. The study of these protocols enables a network manager to easily solve the problems that may occur when using these protocols. The deep understanding of how these protocols work will also give us some ideas about how to create new nonstandard protocols.

Nonstandard Application-Layer Protocols

A programmer can create a nonstandard application-layer program if she can write two programs that provide service to the user by interacting with the transport layer. In this chapter, we show how we can write these types of programs. The creation of a nonstandard (proprietary) protocol that does not even need the approval of the Internet authorities if privately used, has made the Internet so popular in the world. A private company can create a new customized application protocol to communicate with all its offices around the world using the service provided by the first four layers of the TCP/IP protocol suite without using any of the standard application programs. What is needed is to write programs, in one of the computer languages, that use the available services provided by the transport-layer protocols.

10.1.2 Application-Layer Paradigms

It should be clear that to use the Internet we need two application programs to interact with each other: one running on a computer somewhere in the world, the other running on another computer somewhere else in the world. The two programs need to send messages to each other through the Internet infrastructure. However, we have not discussed what the relationship should be between these programs. Should both application programs be able to request services and provide services, or should the application programs just do one or the other? Two paradigms have been developed during the lifetime of the Internet to answer this question: the *client/server paradigm* and the *peer-to-peer paradigm.*

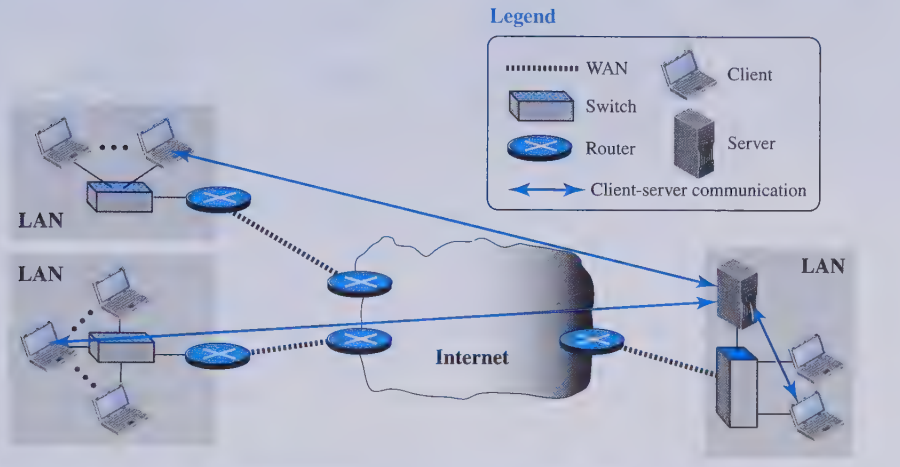
Traditional Paradigm: Client/Server

The traditional paradigm is called the **client/server paradigm**. It was the most popular paradigm until a few years ago. In this paradigm, the service provider is an application program, called the server process; it runs continuously, waiting for another application program, called the client process, to make a connection through the Internet and ask for service. There are normally some server processes that can provide a specific type of service, but there are many clients that request service from any of these server processes. The server process must be running all the time; the client process is started when the client needs to receive service.

The client/server paradigm is similar to some available services out of the territory of the Internet. For example, a telephone directory center in any area can be thought of as a server; a subscriber that calls and asks for a specific telephone number can be thought of as a client. The directory center must be ready and available all the time; the subscriber can call the center for a short period when the service is needed.

Although the communication in the client/server paradigm is between two application programs, the role of each program is totally different. In other words, we cannot run a client program as a server program, or vice versa. In this chapter, when we talk about client/server programming in this paradigm, we show that we always need to write two application programs for each type of service. Figure 10.2 shows an example of a client/server communication in which three clients communicate with one server to receive the services provided by this server.

Figure 10.2 *Example of a client/server paradigm*



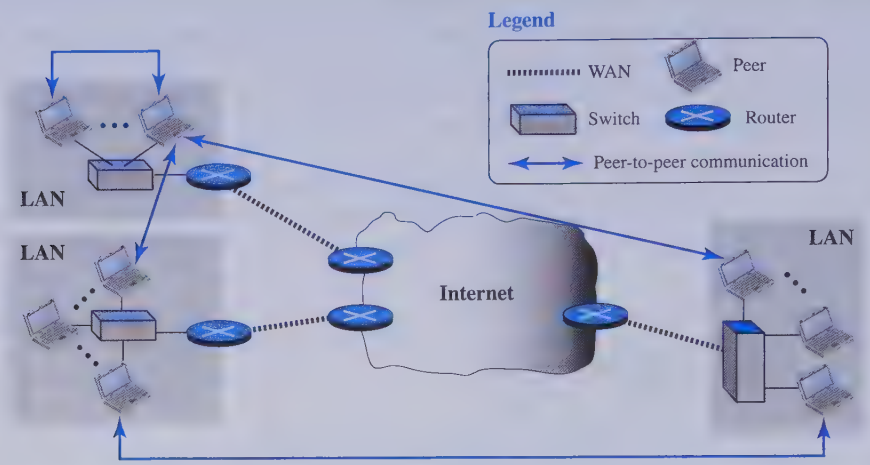
One problem with this paradigm is that the concentration of the communication load is on the shoulder of the server, which means the server should be a powerful computer. Even a powerful computer may become overwhelmed if a large number of clients try to connect to the server at the same time. Another problem is that there should be a service provider willing to accept the cost and create a powerful server for a specific service, which means the service must always return some type of income for the server to encourage such an arrangement.

Several traditional services still use this paradigm, including the **World Wide Web (WWW)** and its vehicle Hypertext Transfer Protocol (HTTP), file transfer protocol (FTP), secure shell (SSH), and e-mail. We discuss some of these protocols and applications later in the chapter.

New Paradigm: Peer-to-Peer

A new paradigm, called the **peer-to-peer (P2P)** paradigm has emerged to respond to the needs of some new applications. In this paradigm, there is no need for a server process to be running all the time and waiting for the client processes to connect. The responsibility is shared between peers. A computer connected to the Internet can provide service at one time and receive service at another time. A computer can even provide and receive services at the same time. Figure 10.3 shows an example of communication in this paradigm.

Figure 10.3 *Example of a peer-to-peer paradigm*



One of the areas that really fits in this paradigm is the Internet telephony. Communication by phone is indeed a peer-to-peer activity; no party needs to be running forever waiting for the other party to call. Another area in which the peer-to-peer paradigm can be used is when some computers connected to the Internet have something to share with each other. For example, if an Internet user has a file available to share with other Internet users, there is no need for the file holder to become a server and run a server process all the time waiting for other users to connect and retrieve the file.

Although the peer-to-peer paradigm has proven to be easily scalable and cost-effective in eliminating the need for expensive servers to be running and maintained all the time, there are also some challenges. The main challenge has been security; it is more difficult to create secure communication between distributed services than between those controlled by some dedicated servers. The other challenge is applicability; it appears that not all applications can use this new paradigm. For example, not many Internet users are ready to become involved, if one day the Web can be implemented as a peer-to-peer service.

There are some new applications, such as BitTorrent, Skype, IPTV, and Internet telephony, that use this paradigm. We will discuss some of these applications later in this chapter.

Mixed Paradigm

An application may choose to use a mixture of the two paradigms by combining the advantages of both. For example, a light-load client/server communication can be used to find the address of the peer that can offer a service. When the address of the peer is found, the actual service can be received from the peer by using the peer-to-peer paradigm.

10.2 CLIENT/SERVER PARADIGM

In a client/server paradigm, communication at the application layer is between two running application programs called **processes**: a *client* and a *server*. A client is a running program that initializes the communication by sending a request; a server is another application program that waits for a request from a client. The server handles the request received from a client, prepares a result, and sends the result back to the client. This definition of a server implies that a server must be running when a request from a client arrives, but the client program needs to be run only when it is needed. This means that if we have two computers connected to each other somewhere, we can run a client process on one of them and the server on the other. However, we need to be careful that the server program is started before we start running the client program. In other words, the lifetime of a server is infinite: It should be started and run forever, waiting for the clients. The lifetime of a client is finite: It normally sends a finite number of requests to the corresponding server, receives the responses, and stops.

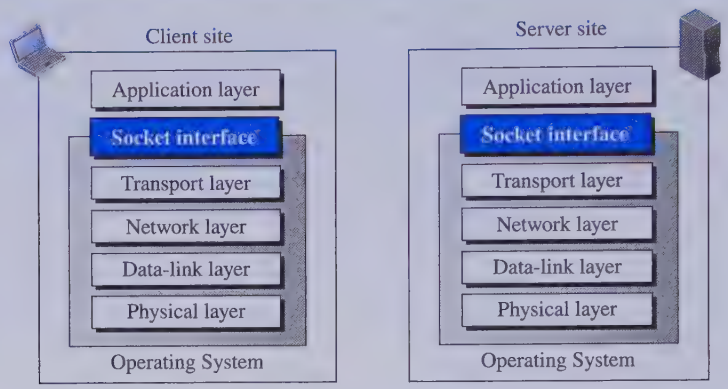
10.2.1 Application Programming Interface

How can a client process communicate with a server process? A computer program is normally written in a computer language with a predefined set of instructions that tells the computer what to do. A computer language has a set of instructions for mathematical operations, a set of instructions for string manipulation, a set of instructions for input/output access, and so on. If we need a process to be able to communicate with another process, we need a new set of instructions to tell the lowest four layers of the TCP/IP protocol suite to open the connection, send and receive data from the other end, and close the connection. A set of instructions of this kind is normally referred to as the **Application Programming Interface (API)**. An interface in programming is a set of instructions between two entities. In this case, one of the entities is the process at the application layer and the other is the *operating system* that encapsulates the first four layers of the TCP/IP protocol suite. In other words, a computer manufacturer needs to build the first four layers of the suite in the operating system and include an API. In this way, the processes running at the application layer are able to communicate with the operating system when sending and receiving messages through the Internet. Several APIs have been designed for communication. Three of them are common: **socket interface**, **transport layer interface** (**TLI**), and **STREAM**. In this section, we briefly discuss only *socket interface*, the most common one, to give a general idea of network communication at the application layer.

Socket interface started in the early 1980s at UC Berkeley as part of a UNIX environment. The socket interface is a set of instructions that provide communication between the application layer and the operating system, as shown in Figure 10.4. It is a set of instructions that can be used by a process to communicate with another process.

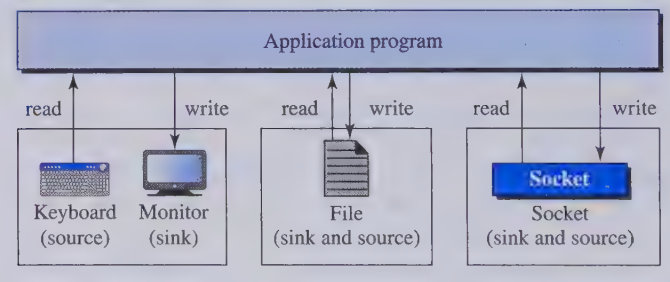
The idea of sockets allows us to use the set of all instructions already designed in a programming language for other sources and sinks. For example, in most computer languages, like C, C++, or Java, we have several instructions that can read and write data to other sources and sinks such as a keyboard (a source), a monitor (a sink), or a

Figure 10.4 *Position of the socket interface*



file (source and sink). We can use the same instructions to read from or write to sockets. In other words, we are adding only new sources and sinks to the programming language without changing the way we send data or receive data. Figure 10.5 shows the idea and compares the sockets with other sources and sinks.

Figure 10.5 *Sockets used the same way as other sources and sinks*

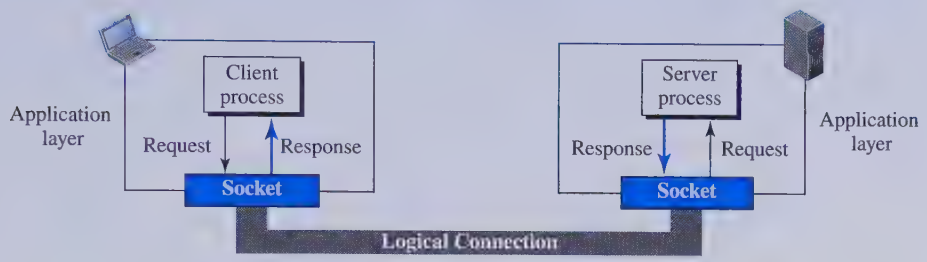


Sockets

Although a socket is supposed to behave like a terminal or a file, it is not a physical entity like them; it is an abstraction. It is a *data structure* that is created and used by the application program.

We can say that, as far as the application layer is concerned, communication between a client process and server process is communication between two sockets, created at two ends, as shown in Figure 10.6. The client thinks that the socket is the entity that receives the request and gives the response; the server thinks that the socket is the one that has a request and needs the response. If we create two sockets, one at each end, and define the source and destination addresses correctly, we can use the available instructions to send and receive data. The rest is the responsibility of the operating system and the embedded TCP/IP protocol.

Figure 10.6 *Use of sockets in process-to-process communication*



Socket Addresses

The interaction between a client and a server is two-way communication. In a two-way communication, we need a pair of addresses: local (sender) and remote (receiver). The local address in one direction is the remote address in the other direction, and vice versa. Because communication in the client/server paradigm is between two sockets, we need a pair of **socket addresses** for communication: a local socket address and a remote socket address. However, we need to define a socket address in terms of identifiers used in the TCP/IP protocol suite.

A socket address should first define the computer on which a client or a server is running. As we discuss in Chapter 7, a computer in the Internet is uniquely defined by its IP address, a 32-bit integer in the current Internet version. However, several client or server processes may be running at the same time on a computer, which means that we need another identifier to define the specific client or server involved in the communication. As we discuss in Chapter 9, an application program can be defined by a port number, a 16-bit integer. This means that a socket address should be a combination of an IP address and a port number as shown in Figure 10.7.

Figure 10.7 *A socket address*

Because a socket defines the end-point of the communication, we can say that a socket is identified by a pair of socket addresses, a local and a remote.

Example 10.1

We can find a two-level address in telephone communication. A telephone number can define an organization, and an extension can define a specific connection in that organization. The

telephone number in this case is similar to the IP address, which defines the whole organization; the extension is similar to the port number, which defines the particular connection.

Finding Socket Addresses

How can a client or a server find a pair of socket addresses for communication? The situation is different for each site.

Server Site

The server needs a local (server) and a remote (client) socket address for communication.

Local Socket Address The local (server) socket address is provided by the operating system. The operating system knows the IP address of the computer on which the server process is running. The port number of a server process, however, needs to be assigned. If the server process is a standard one defined by the Internet authority, a port number is already assigned to it. For example, the assigned port number for a Hypertext Transfer Protocol (HTTP) is the integer 80, which cannot be used by any other process. If the server process is not standard, the designer of the server process can choose a port number, in the range defined by the Internet authority, and assign it to the process. When a server starts running, it knows the local socket address.

Remote Socket Address The remote socket address for a server is the socket address of the client that makes the connection. Because the server can serve many clients, it does not know beforehand the remote socket address for communication. The server can find this socket address when a client tries to connect to the server. The client socket address, which is contained in the request packet sent to the server, becomes the remote socket address that is used for responding to the client. In other words, although the local socket address for a server is fixed and used during its lifetime, the remote socket address is changed in each interaction with a different client.

Client Site

The client also needs a local (client) and a remote (server) socket address for communication.

Local Socket Address The local (client) socket address is also provided by the operating system. The operating system knows the IP address of the computer on which the client is running. The port number, however, is a 16-bit temporary integer that is assigned to a client process each time the process needs to start the communication. The port number, however, needs to be assigned from a set of integers defined by the Internet authority and called the ephemeral (temporary) port numbers. The operating system, however, needs to guarantee that the new port number is not used by any other running client process.

Remote Socket Address Finding the remote (server) socket address for a client, however, needs more work. When a client process starts, it should know the socket address of the server it wants to connect to. We will have two situations in this case.

- Sometimes, the user who starts the client process knows both the server port number and IP address of the computer on which the server is running. This usually occurs in situations when we have written client and server applications and we

want to test them. For example, at the end of this chapter we write some simple client and server programs and we test them using this approach. In this situation, the programmer can provide these two pieces of information when it runs the client program.

- Although each standard application has a well-known port number, most of the time, we do not know the IP address. This happens in situations such as when we need to contact a web page, send an e-mail to a friend, or copy a file from a remote site. In these situations, the server has a name, an identifier that uniquely defines the server process. Examples of these identifiers are URLs, such as www .xxx.yyy, or e-mail addresses, such as xxxx@yyyy.com. The client process should now change this identifier (name) to the corresponding server socket address. The client process normally knows the port number because it should be a well-known port number, but the IP address can be obtained using another client/server application called the Domain Name System (DNS). We will discuss DNS later in Section 10.3.6, but it is enough to know that it acts as a directory in the Internet. Compare the situation with the telephone directory. We want to call someone whose name we know, but whose telephone number can be obtained from the telephone directory. The telephone directory maps the name to the telephone number; DNS maps the server name to the IP address of the computer running that server.

10.2.2 Using Services of the Transport Layer

A pair of processes provide services to the users of the Internet, human or programs. A pair of processes, however, need to use the services provided by the transport layer for communication because there is no physical communication at the application layer. As we discussed in Chapter 9, there are three common transport-layer protocols in the TCP/IP suite: UDP, TCP, and SCTP. Most standard applications have been designed to use the services of one of these protocols. When we write a new application, we can decide which protocol we want to use. The choice of the transport-layer protocol seriously affects the capability of the application processes. In this section, we first discuss the services provided by each protocol to help understand why a standard application uses it or which one we need to use if we decide to write a new application.

UDP

UDP provides connectionless, unreliable, datagram service. Connectionless service means that there is no logical connection between the two ends exchanging messages. Each message is an independent entity encapsulated in a packet called a **datagram**. UDP does not see any relation (connection) between consequent datagrams coming from the same source and going to the same destination.

UDP is not a reliable protocol. Although it may check that the data are not corrupted during the transmission, it does not ask the sender to resend the corrupted or lost datagram. For some applications, UDP has an advantage: It is message-oriented. It gives boundaries to the messages exchanged.

We can compare the connectionless and unreliable service to the regular service provided by the post office. Two entities can exchange several letters between themselves,

but the post office does not see any connection between these letters. For the post office, each letter is a separate entity with its own sender and receiver. If a letter is lost or corrupted during the delivery, the post office is not responsible, although it tries its best.

An application program may be designed to use UDP if it is sending small messages, and the simplicity and speed is more important for the application than reliability. For example, some management and multimedia applications fit in this category.

TCP

TCP provides connection-oriented, reliable, byte-stream service. TCP requires that two ends first create a logical connection between themselves by exchanging some connection-establishment packets. This phase, which is sometimes called handshaking, establishes some parameters between the two ends including the size of the data packets to be exchanged and the size of buffers to be used for holding the chunks of data until the whole message arrives. After the handshaking process, the two ends can send chunks of data in segments in each direction. By numbering the bytes exchanged, the continuity of the bytes can be checked. For example, if some bytes are lost or corrupted, the receiver can request the resending of those bytes, which makes TCP a reliable protocol. TCP also can provide flow control and congestion control. One problem with TCP is that it is not message-oriented; it does not put boundaries on the messages exchanged.

We can compare the connection-oriented and reliable service provided by TCP to the service provided by the telephone company, although only to some extent. If two parties decide to communicate via telephone instead of via the post office, they can create a connection once and exchange words for a period of time. The telephone service is to some extent reliable, because if a person does not understand any of the words pronounced by the other party, he can ask for repetition.

Most of the standard applications that need to send long messages and require reliability may benefit from the service of TCP.

SCTP

SCTP provides a service that is a combination of the two other protocols. Like TCP, SCTP provides a connection-oriented, reliable service, but it is not byte-stream oriented. It is a message-oriented protocol like UDP. In addition, SCTP can provide multistream service by providing multiple network-layer connections.

SCTP is normally suitable for any application that needs reliability and at the same time needs to remain connected, even if a failure occurs in one network-layer connection.

10.3 STANDARD APPLICATIONS

During the lifetime of the Internet, several client/server application programs have been developed. We do not have to redefine them, but we need to understand what they do. For each application, we also need to know the options available to us. The study of these applications and the ways they provide different services can help us to create customized applications in the future.

We have selected six standard application programs in this section. We start with HTTP and the World Wide Web because they are used by almost all Internet users. We then introduce file transfer and electronic mail applications which have high traffic loads on the Internet. Next, we explain remote login and how it can be achieved using the TELNET and SSH protocols. Finally, we discuss DNS, which is used by all application programs to map the application-layer identifier to the corresponding host IP address.

Some other applications, such as Dynamic Host Configuration Protocol (DHCP) or Simple Network Management Protocol (SNMP), will be discussed in Chapter 12.

10.3.1 World Wide Web and HTTP

In this section, we first introduce the World Wide Web (abbreviated as the WWW or Web). We then discuss the Hypertext Transfer Protocol (HTTP), the most common client/server application program used in relation to the Web.

World Wide Web

The idea of the Web was first proposed by Tim Berners-Lee in 1989 at *CERN*, the European Organization for Nuclear Research,[†] to allow several researchers at different locations throughout Europe to access each others' research. The commercial Web started in the early 1990s.

The Web today is a repository of information in which the documents, called *web pages*, are distributed all over the world and related documents are linked together. The popularity and growth of the Web can be related to two terms in the above statement: *distributed* and *linked*. Distribution allows for the growth of the Web. Each web server in the world can add a new web page to the repository and announce it to all Internet users without overloading a few servers. Linking allows one web page to refer to another web page stored in another server somewhere else in the world. The linking of web pages was achieved using a concept called *hypertext*, which was introduced many years before the advent of the Internet. The idea was to use a machine that automatically retrieved another document stored in the system when a link to it appeared in the document. The Web implemented this idea electronically: to allow the linked document to be retrieved when the link was clicked by the user. Today, the term **hypertext**, coined to mean linked text documents, has been changed to **hypermedia**, to show that a web page can be a text document, an image, an audio file, or a video file.

The purpose of the Web has gone beyond the simple retrieving of linked documents. Today, the Web is used to provide electronic shopping and gaming. One can use the Web to listen to radio programs or view television programs whenever one desires without being forced to listen to or view these programs at the time when they are broadcast.

Architecture

The WWW today is a distributed client/server service, in which a client using a browser can access a service using a server. However, the service provided is distributed over many locations called *sites*. Each site holds one or more documents, referred to as web

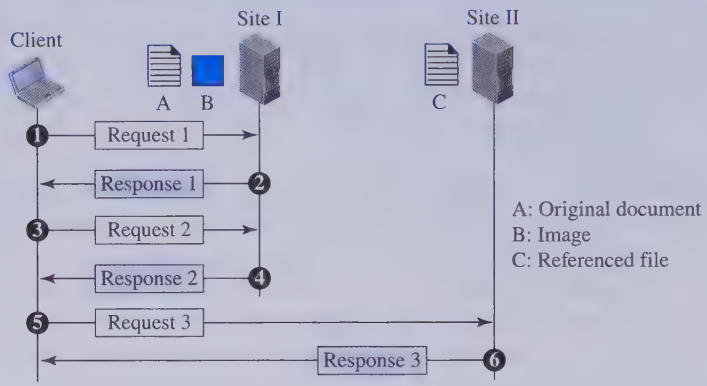
[†]In French: *Conseil Européen pour la Recherche Nucléaire*

pages. Each **web page**, however, can contain some links to other web pages in the same or other sites. In other words, a web page can be simple or composite. A simple web page has no links to other web pages; a composite web page has one or more links to other web pages. Each web page is a file with a name and address.

Example 10.2

Assume we need to retrieve a scientific document that contains one reference to another text file and one reference to a large image. Figure 10.8 shows the situation.

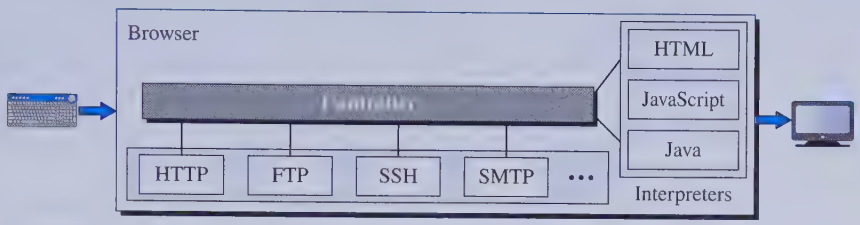
Figure 10.8 *Example 10.2*



The main document and the image are stored in two separate files in the same site (file A and file B); the referenced text file is stored in another site (file C). Because we are dealing with three different files, we need three transactions if we want to see the whole document. The first transaction (request/response) retrieves a copy of the main document (file A), which has references (pointers) to the second and third files. When a copy of the main document is retrieved and browsed, the user can click on the reference to the image to invoke the second transaction and retrieve a copy of the image (file B). If the user needs to see the contents of the referenced text file, she can click on its reference (pointer) invoking the third transaction and retrieving a copy of file C. Note that although files A and B both are stored in site I, they are independent files with different names and addresses. Two transactions are needed to retrieve them. A very important point we need to remember is that files A, B, and C in this example are independent web pages, each with independent names and addresses. Although references to file B or C are included in file A, it does not mean that each of these files cannot be retrieved independently. A second user can retrieve file B with one transaction. A third user can retrieve file C with one transaction.

Web Client (Browser) A variety of vendors offer commercial **browsers** that interpret and display a web page, and all of them use nearly the same architecture. Each browser usually consists of three parts: a controller, client protocols, and interpreters (see Figure 10.9).

Figure 10.9 *Browser*



The controller receives input from the keyboard or the mouse and uses the client programs to access the document. After the document has been accessed, the controller uses one of the interpreters to display the document on the screen. The client protocol can be one of the protocols described later, such as HTTP or FTP. The interpreter can be HTML, Java, or JavaScript, depending on the type of document. Some commercial browsers include Internet Explorer and Firefox.

Web Server The web page is stored at the server. Each time a request arrives, the corresponding document is sent to the client. To improve efficiency, servers normally store requested files in a cache in memory; memory is faster to access than a disk. A server can also become more efficient through multithreading or multiprocessing. In this case, a server can answer more than one request at a time.

Uniform Resource Locator (URL)

A web page, as a file, needs to have a unique identifier to distinguish it from other web pages. To define a web page, we need three identifiers: *host*, *port*, and *path*. However, before defining the web page, we need to tell the browser what client/server application we want to use, which is called the *protocol*. This means we need four identifiers to define the web page. The first is the type of vehicle to be used to fetch the web page; the last three make up the combination that defines the destination object (web page).

- **Protocol.** The first identifier is the abbreviation for the client/server program that we need to access the web page. Although most of the time the protocol is Hypertext Transfer Protocol (HTTP), which we will discuss shortly, we can also use other protocols such as File Transfer Protocol (FTP).
- **Host.** The host identifier can be the IP address of the server or the unique name given to the server. IP addresses can be defined in dotted-decimal notations, as described in Chapter 7 (such as 64.23.56.17); the name is normally the domain name that uniquely defines the host, such as *forouzan.com*, which we discuss with the Domain Name System (DNS) later in Section 10.3.6.
- **Port.** The port, a 16-bit integer, is normally predefined for the client/server application. For example, if HTTP is used for accessing the web page, the well-known port number is 80. However, if a different port is used, the number can be explicitly given.

- **Path.** The path identifies the location and the name of the file in the underlying operating system. The format of this identifier normally depends on the operating system. In UNIX, a path is a set of directory names followed by the file name, all separated by a slash. For example, */top/next/last/myfile* is a path that uniquely defines a file named *myfile*, stored in the directory *last*, which itself is part of the directory *next*, which itself is under the directory *top*. In other words, the path lists the directories from the top to the bottom, followed by the file name.

To combine these four pieces together, the **uniform resource locator (URL)** has been designed; it uses three different separators between the four pieces as shown below:

protocol://host/path	**Used most of the time**
protocol://host:port/path	**Used when port number is needed**

Example 10.3

The URL ***http://www.mhhe.com/compsci/forouzan/*** defines the web page related to one of the authors of this book. The string ***www.mhhe.com*** is the name of the computer in the McGraw-Hill company (the three letters ***www*** are part of the host name and are added to the commercial host). The path is ***compsci/forouzan/***, which defines Forouzan's web page under the directory ***compsci*** (computer science).

Web Documents

The documents in the WWW can be grouped into three broad categories: static, dynamic, and active.

Static Documents **Static documents** are fixed-content documents that are created and stored in a server. The client can get a copy of the document only. In other words, the contents of the file are determined when the file is created, not when it is used. Of course, the contents in the server can be changed, but the user cannot change them. When a client accesses the document, a copy of the document is sent. The user can then use a browser to see the document. Static documents are prepared using one of the several languages: *Hypertext Markup Language* (HTML), *Extensible Markup Language* (XML), *Extensible Style Language* (XSL), and *Extensible Hypertext Markup Language* (XHTML). We discuss these languages in Appendix C.

Dynamic Documents A **dynamic document** is created by a web server whenever a browser requests the document. When a request arrives, the web server runs an application program or a script that creates the dynamic document. The server returns the result of the program or script as a response to the browser that requested the document. Because a fresh document is created for each request, the contents of a dynamic document may vary from one request to another. A very simple example of a dynamic document is the retrieval of the time and date from a server. Time and date are dynamic information in that they change from moment to moment. The client can ask the server to run a program such as the *date* program in UNIX and send the result of the program to the client. Although the *Common Gateway Interface (CGI)* was used to retrieve a dynamic document in the past, today's options include one of the

scripting languages such as *Java Server Pages (JSP)*, which uses the Java language for scripting.

Active Documents For many applications, we need a program or a script to be run at the client site. These are called **active documents**. For example, suppose we want to run a program that creates animated graphics on the screen or a program that interacts with the user. The program definitely needs to be run at the client site where the animation or interaction takes place. When a browser requests an active document, the server sends a copy of the document or a script. The document is then run at the client (browser) site. One way to create an active document is to use *Java applets*, a program written in Java on the server. It is compiled and ready to be run. The document is in bytecode (binary) format. Another way is to use *JavaScripts* but download and run the script at the client site.

Hypertext Transfer Protocol (HTTP)

The **Hypertext Transfer Protocol (HTTP)** is a protocol that is used to define how the client/server programs can be written to retrieve web pages from the Web. An HTTP client sends a request; an HTTP server returns a response. The server uses the port number 80; the client uses a temporary port number. HTTP uses the services of TCP, which, as discussed before, is a connection-oriented and reliable protocol. This means that, before any transaction between the client and the server can take place, a connection needs to be established between them. After the transaction, the connection should be terminated. The client and server, however, do not need to worry about errors in messages exchanged or loss of any message, because the TCP is reliable and will take care of this matter, as we discussed in Chapter 9.

Nonpersistent versus Persistent Connections

The hypertext concept embedded in web page documents may require several requests and responses. If the web pages, objects to be retrieved, are located on different servers, we do not have any other choice than to create a new TCP connection for retrieving each object. However, if some of the objects are located on the same server, we have two choices: to retrieve each object using a new TCP connection or to make a TCP connection and retrieve them all. The first method is referred to as *nonpersistent connections*, the second as *persistent connections*. HTTP, prior to version 1.1, specified *nonpersistent* connections, while *persistent* connections is the default in version 1.1, but it can be changed by the user.

Nonpersistent Connections In a **nonpersistent connection**, one TCP connection is made for each request/response. The steps in this strategy are as follows:

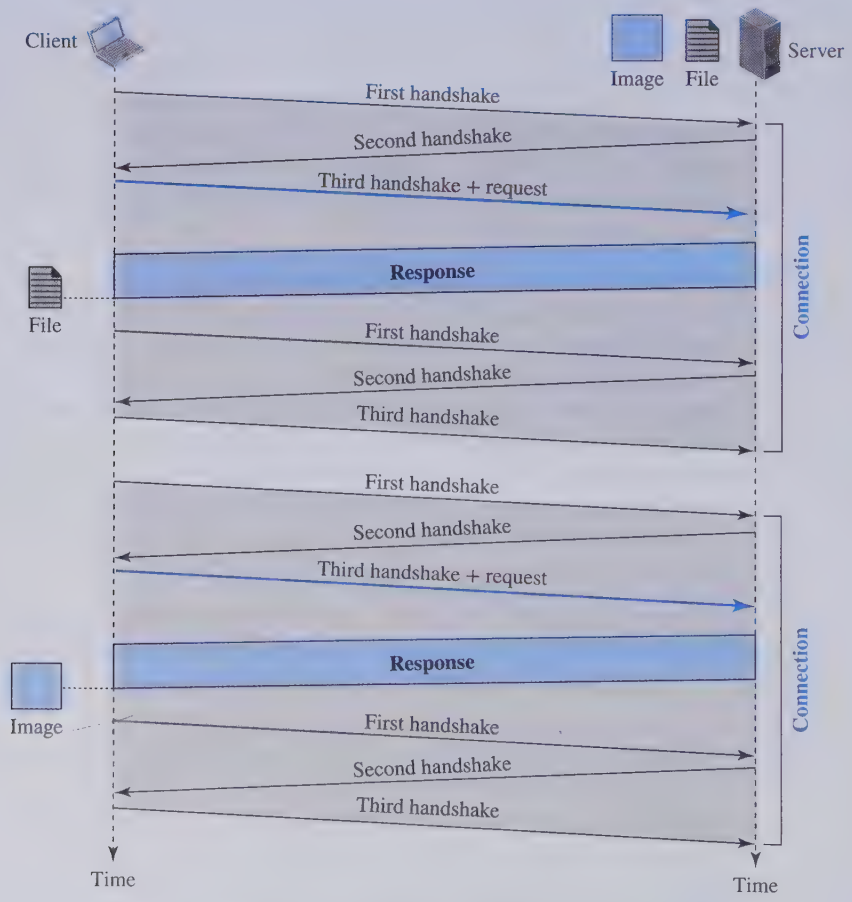
1. The client opens a TCP connection and sends a request.
2. The server sends the response and closes the connection.
3. The client reads the data until it encounters an end-of-file marker; it then closes the connection.

In this strategy, if a file contains links to N different pictures in different files (all located on the same server), the connection must be opened and closed $N + 1$ times. The nonpersistent strategy imposes high overhead on the server because the server needs $N + 1$ different buffers each time a connection is opened.

Example 10.4

Figure 10.10 shows an example of a nonpersistent connection. The client needs to access a file that contains one link to an image. The text file and image are located on the same server. Here we need two connections. For each connection, TCP requires at least three handshake messages to establish the connection, but the request can be sent with the third one. After the connection is established, the object can be transferred. After receiving an object, another three handshake messages are needed to terminate the connection. This means that the client and server are involved in two connection establishments and two connection terminations. If the transaction involves retrieving 10 or 20 objects, the round-trip times spent for these handshakes add up to a big overhead. When we describe the client/server programming at the end of the chapter, we will show that for each connection the client and server need to allocate extra resources such as buffers and variables. This is another burden on both sites, but especially on the server site.

Figure 10.10 *Example 10.4*

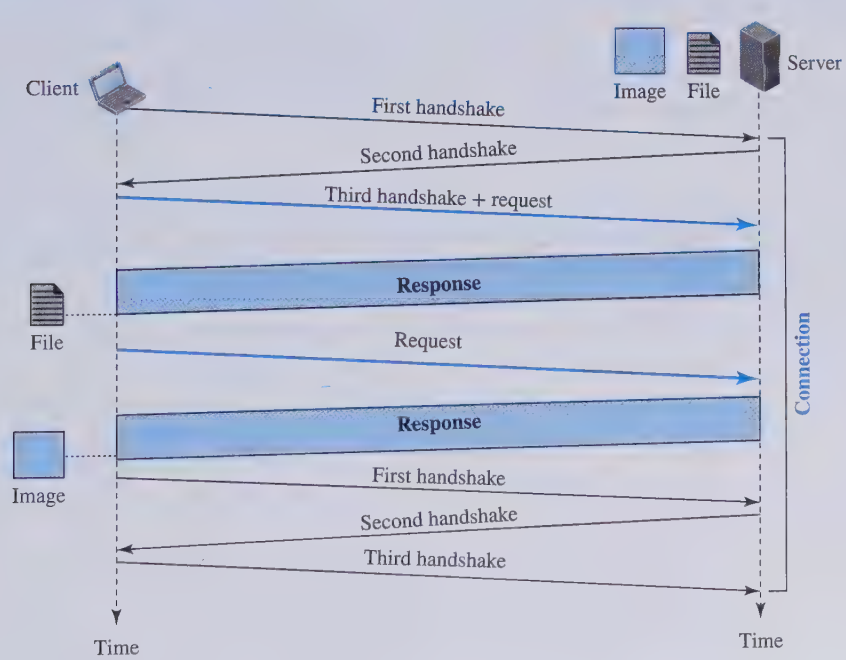


Persistent Connections HTTP version 1.1 specifies a **persistent connection** by default. In a persistent connection, the server leaves the connection open for more requests after sending a response. The server can close the connection at the request of a client or if a time-out has been reached. The sender usually sends the length of the data with each response. However, there are some occasions when the sender does not know the length of the data. This is the case when a document is created dynamically or actively. In these cases, the server informs the client that the length is not known and closes the connection after sending the data so the client knows that the end of the data has been reached. Time and resources are saved using persistent connections. Only one set of buffers and variables needs to be set for the connection at each site. The round-trip time for connection establishment and connection termination is saved.

Example 10.5

Figure 10.11 shows the same scenario as in Example 10.4, but using a persistent connection. Only one connection establishment and connection termination is used, but the request for the image is sent separately.

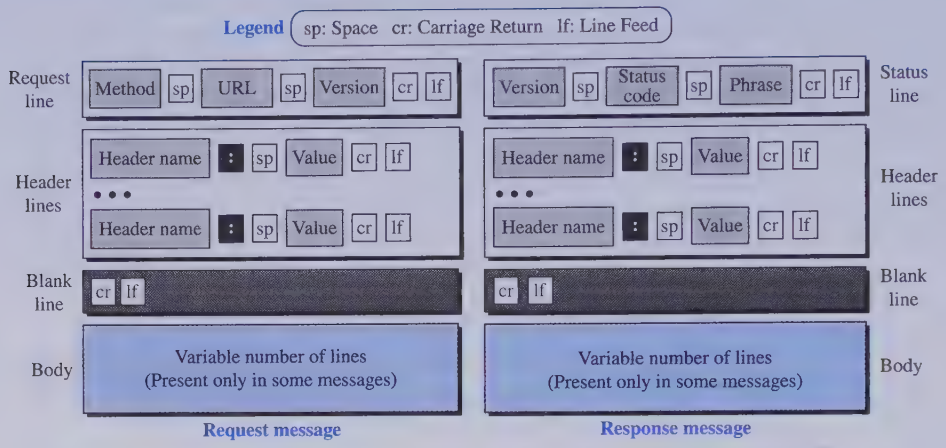
Figure 10.11 *Example 10.5*



Message Formats

HTTP defines the format of the request and response messages, as shown in Figure 10.12. We have put the two formats next to each other for comparison. Each message is made

Figure 10.12 *Formats of the request and response messages*



up of four sections. The first section in the request message is called the *request line*; the first section in the response message is called the *status line*. The other three sections have the same names in the request and response messages. However, the similarities between these sections are only in the names; they may have different contents. We discuss each message type separately.

Request Message As we stated, the first line in a request message is called a request line. There are three fields in this line separated by one space and terminated by two characters (carriage return and line feed) as shown in Figure 10.12. The fields are called *method*, URL, and *version*.

The method field defines the request types. In version 1.1 of HTTP, several methods are defined, as shown in Table 10.1. Most of the time, the client uses the GET method to send a request. In this case, the body of the message is empty. The HEAD method is used when the client needs only some information about the web page from the server, such as the last time it was modified. It can also be used to test the validity of a URL. The response message in this case has only the header section; the body section is empty. The PUT method is the inverse of the GET method; it allows the client to post a new web page on the server (if permitted). The POST method is similar to the PUT method, but it is used to send some information to the server to be added to the web page or to modify the web page. The TRACE method is used for debugging; the client asks the server to echo back the request to check whether the server is getting the requests. The DELETE method allows the client to delete a web page on the server if the client has permission to do so. The CONNECT method was originally created as a reserve method; it may be used by proxy servers. Finally, the OPTIONS method allows the client to ask about the properties of a web page.

Table 10.1 *Methods*

Method	*Action*
GET	Requests a document from the server
HEAD	Requests information about a document but not the document itself
PUT	Sends a document from the client to the server
POST	Sends some information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes the web page
CONNECT	Reserved
OPTIONS	Inquires about available options

The second field, URL, was discussed earlier in this section. It defines the address and name of the corresponding web page. The third field, version, gives the version of the protocol; the most current version of HTTP is 1.1.

After the request line, we can have zero or more *request header* lines. Each header line sends additional information from the client to the server. For example, the client can request that the document be sent in a special format. Each header line has a header name, a colon, a space, and a header value (see Figure 10.12). Table 10.2 shows some header names commonly used in a request. The value field defines the values associated with each header name. The list of values can be found in the corresponding RFCs.

Table 10.2 *Request header names*

Header	*Description*
User-agent	Identifies the client program
Accept	Shows the media format the client can accept
Accept-charset	Shows the character set the client can handle
Accept-encoding	Shows the encoding scheme the client can handle
Accept-language	Shows the language the client can accept
Authorization	Shows what permissions the client has
Host	Shows the host and port number of the client
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Cookie	Returns the cookie to the server (explained later in this section)
If-Modified-Since	Specifies if the file has been modified since a specific date

The body can be present in a request message. Usually, it contains the comment to be sent or the file to be published on the website when the method is PUT or POST.

Response Message The format of the response message is also shown in Figure 10.12. A response message consists of a status line, header lines, a blank line, and sometimes a body. The first line in a response message is called the *status line*. There are three fields in this line separated by spaces and terminated by a carriage return and line feed. The first field defines the version of HTTP, currently 1.1. The status code field defines the status of the request. It consists of three digits. Whereas the codes in the 100 range are only informational, the codes in the 200 range indicate a successful request. The codes in the 300 range redirect the client to another URL, and the codes in the 400 range indicate an error at the client site. Finally, the codes in the 500 range indicate an error at the server site. The status phrase explains the status code in text form.

After the status line, we can have zero or more *response header* lines. Each header line sends additional information from the server to the client. For example, the sender can send extra information about the document. Each header line has a header name, a colon, a space, and a header value. We will show some header lines in the examples at the end of this section. Table 10.3 shows some header names commonly used in a response message.

Table 10.3 *Response header names*

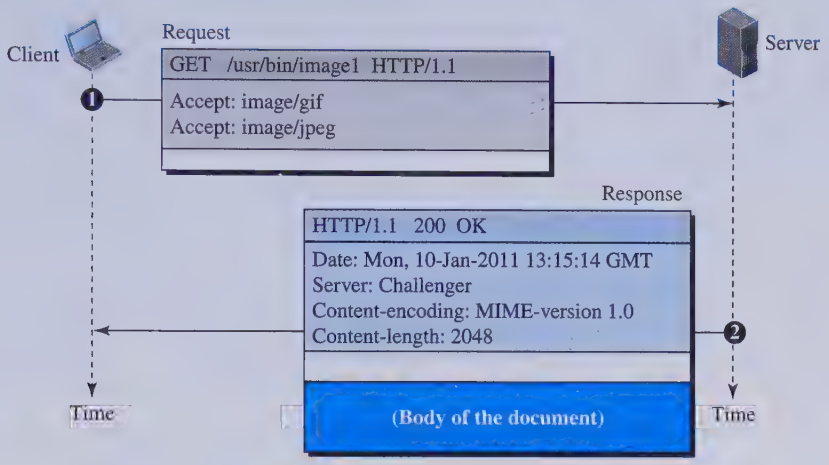
Header	*Description*
Date	Shows the current date
Upgrade	Specifies the preferred communication protocol
Server	Gives information about the server
Set-Cookie	The server asks the client to save a cookie
Content-Encoding	Specifies the encoding scheme
Content-Language	Specifies the language
Content-Length	Shows the length of the document
Content-Type	Specifies the media type
Location	To ask the client to send the request to another site
Accept-Ranges	The server will accept the requested byte-ranges
Last-modified	Gives the date and time of the last change

The body contains the document to be sent from the server to the client. The body is present unless the response is an error message.

Example 10.6

This example retrieves a document (see Figure 10.13). We use the GET method to retrieve an image with the path /usr/bin/image1. The request line shows the method (GET), the URL, and the HTTP version (1.1). The header has two lines that show that the client can accept images in the GIF or JPEG format. The request does not have a body. The response message contains the status line and four lines of header. The header lines define the date, server, content encoding (MIME version, which will be described in Section 10.3.3 on electronic mail), and length of the document. The body of the document follows the header.

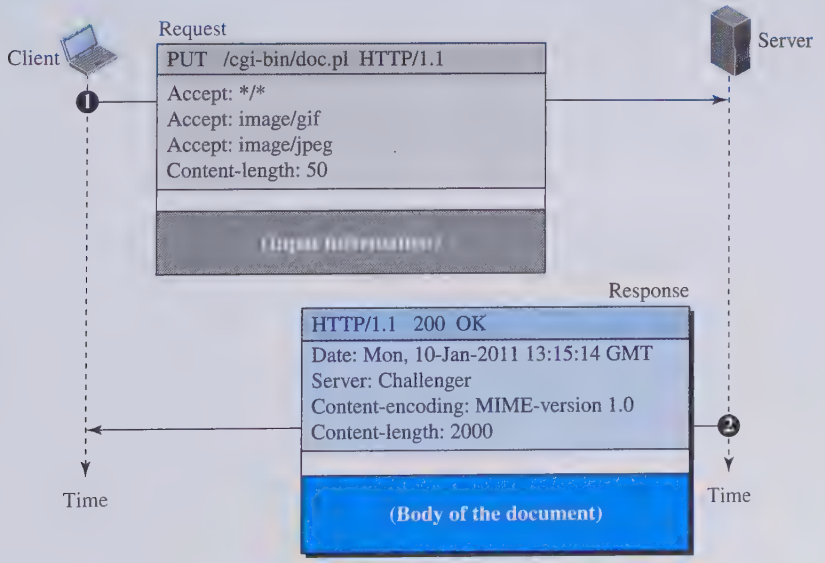
Figure 10.13 *Example 10.6*



Example 10.7

In this example, the client wants to send a web page to be posted on the server. We use the PUT method. The request line shows the method (PUT), URL, and HTTP version (1.1). There are four lines of headers. The request body contains the web page to be posted. The response message contains the status line and four lines of headers. The created document, which is a CGI document, is included as the body (see Figure 10.14).

Figure 10.14 *Example 10.7*



Conditional Request

A client can add a condition in its request. In this case, the server will send the requested web page if the condition is met or inform the client otherwise. One of the most common conditions imposed by the client is the time and date the web page is modified. The client can send the header line *If-Modified-Since* with the request to tell the server that it needs the page only if it is modified after a certain point in time.

Example 10.8

The following shows how a client imposes the modification data and time condition on a request. The status line in the response shows the file was not modified after the defined point in time. The body of the response message is also empty.

GET http://www.commonServer.com/information/file1 HTTP/1.1	**Request line**
If-Modified-Since: Thu, Sept 04 00:00:00 GMT	**Header line**
	Blank line

HTTP/1.1 304 Not Modified	**Status line**
Date: Sat, Sept 06 08 16:22:46 GMT	**First header line**
Server: commonServer.com	**Second header line**
	Blank line
(Empty Body)	**Empty body**

Cookies

The World Wide Web was originally designed as a stateless entity. A client sends a request; a server responds. Their relationship is over. The original purpose of the Web, retrieving publicly available documents, exactly fits this design. Today the Web has other functions that need to remember some information about the clients. Some are listed here:

- Websites are being used as *electronic stores* that allow users to browse through the store, select wanted items, put them in an electronic cart, and pay at the end with a credit card.
- Some websites need to allow access to *registered clients* only.
- Some websites are used as *portals*: The user selects the web pages he wants to see.
- Some websites are just *advertising* agencies.

For these purposes, the **cookie** mechanism was devised.

Creating and Storing Cookies The creation and storing of cookies depend on the implementation; however, the principle is the same.

1. When a server receives a request from a client, it stores information about the client in a file or a string. The information may include the domain name of the client, the contents of the cookie (information the server has gathered about the client such as name and registration number), a timestamp, and other information depending on the implementation.
2. The server includes the cookie in the response that it sends to the client.

3. When the client receives the response, the browser stores the cookie in the cookie directory, which is sorted by the server domain name.

Using Cookies When a client sends a request to a server, the browser looks in the cookie directory to see if it can find a cookie sent by that server. If found, the cookie is included in the request. When the server receives the request, it knows that this is an old client, not a new one. Note that the contents of the cookie are never read by the browser or disclosed to the user. It is a cookie *made* by the server and *eaten* by the server. Now let us see how a cookie is used for the four previously mentioned purposes:

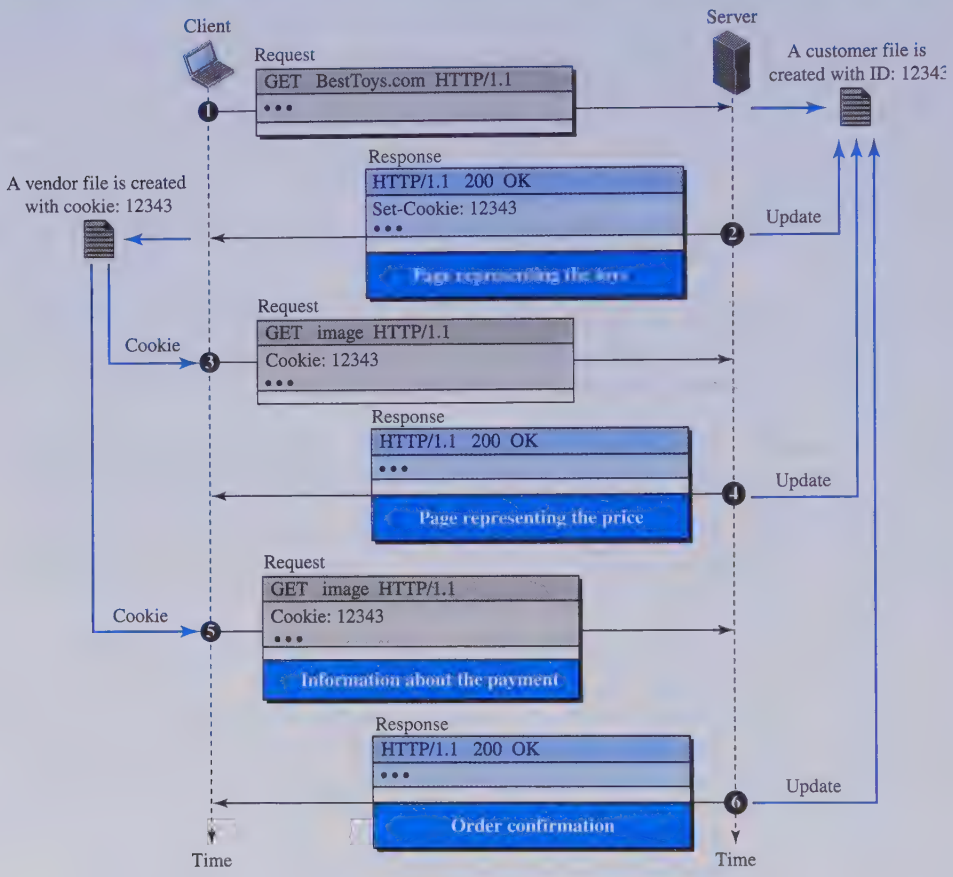
- An *electronic store* (e-commerce) can use a cookie for its client shoppers. When a client selects an item and inserts it in a cart, a cookie that contains information about the item, such as its number and unit price, is sent to the browser. If the client selects a second item, the cookie is updated with the new selection information, and so on. When the client finishes shopping and wants to check out, the last cookie is retrieved and the total charge is calculated.
- The site that restricts access to *registered clients* only sends a cookie to the client when the client registers for the first time. For any repeated access, only those clients that send the appropriate cookie are allowed.
- A web *portal* uses the cookie in a similar way. When a user selects her favorite pages, a cookie is made and sent. If the site is accessed again, the cookie is sent to the server to show what the client is looking for.
- A cookie is also used by *advertising* agencies. An advertising agency can place banner ads on some main website that is often visited by users. The advertising agency supplies only a URL that gives the advertising agency's address instead of the banner itself. When a user visits the main website and clicks the icon of a corporation, a request is sent to the advertising agency. The advertising agency sends the requested banner, but it also includes a cookie with the ID of the user. Any future use of the banners adds to the database that profiles the Web behavior of the user. The advertising agency has compiled the interests of the user and can sell this information to other parties. This use of cookies is very controversial. Hopefully, some new regulations will be devised to preserve the privacy of users.

Example 10.9

Figure 10.15 shows a scenario in which an electronic store can benefit from the use of cookies.

Assume a shopper wants to buy a toy from an electronic store named BestToys. The shopper browser (client) sends a request to the BestToys server. The server creates an empty shopping cart (a list) for the client and assigns an ID to the cart (for example, 12343). The server then sends a response message, which contains the images of all toys available, with a link under each toy that selects the toy if it is being clicked. This response message also includes the Set-Cookie header line whose value is 12343. The client displays the images and stores the cookie value in a file named BestToys. The cookie is not revealed to the shopper. Now the shopper selects one of the toys and clicks on it. The client sends a request, but includes the ID 12343 in the Cookie header line. Although the server may have been busy and forgotten about this shopper, when it receives the request and checks the header, it finds the value 12343 as the cookie. The server knows that

Figure 10.15 *Example 10.9*



the customer is not new; it searches for a shopping cart with ID 12343. The shopping cart (list) is opened and the selected toy is inserted in the list. The server now sends another response to the shopper to tell her the total price and ask her to provide payment. The shopper provides information about her credit card and sends a new request with the ID 12343 as the cookie value. When the request arrives at the server, it again sees the ID 12343, and accepts the order and the payment and sends a confirmation in a response. Other information about the client is stored in the server. If the shopper accesses the store sometime in the future, the client sends the cookie again; the store retrieves the file and has all the information about the client.

Web Caching: Proxy Server

HTTP supports **proxy servers**. A proxy server is a computer that keeps copies of responses to recent requests. The HTTP client sends a request to the proxy server. The proxy server checks its cache. If the response is not stored in the cache, the proxy server

sends the request to the corresponding server. Incoming responses are sent to the proxy server and stored for future requests from other clients.

The proxy server reduces the load on the original server, decreases traffic, and improves latency. However, to use the proxy server, the client must be configured to access the proxy instead of the target server.

Note that the proxy server acts as both server and client. When it receives a request from a client for which it has a response, it acts as a server and sends the response to the client. When it receives a request from a client for which it does not have a response, it first acts as a client and sends a request to the target server. When the response has been received, it acts again as a server and sends the response to the client.

Proxy Server Location

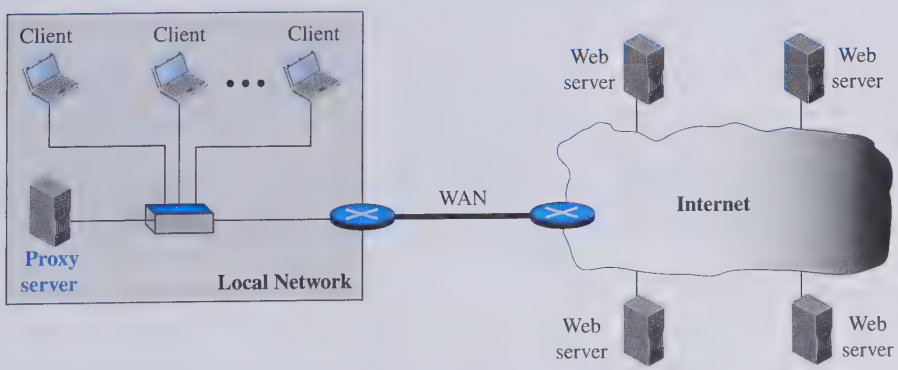
The proxy servers are normally located at the client site. This means that we can have a hierarchy of proxy servers:

1. A client computer can also be used as a proxy server, in a small capacity, that stores responses to requests often invoked by the client.
2. In a company, a proxy server may be installed on the computer LAN to reduce the load going out of and coming into the LAN.
3. An ISP with many customers can install a proxy server to reduce the load going out of and coming into the ISP network.

Example 10.10

Figure 10.16 shows an example of a use of a proxy server in a local network, such as the network on a campus or in a company. The proxy server is installed in the local network. When an HTTP request is created by any of the clients (browsers), the request is first directed to the proxy server. If the proxy server already has the corresponding web page, it sends the response to the client. Otherwise, the proxy server acts as a client and sends the request to the web server in the Internet. When the response is returned, the proxy server makes a copy and stores it in its cache before sending it to the requesting client.

Figure 10.16 *Example 10.10: example of a proxy server*



Cache Update

A very important question is, "How long should a response remain in the proxy server before being deleted and replaced?" Several different strategies are used for this purpose. One solution is to store the list of sites whose information remains the same for a while. For example, a news agency may change its news page every morning. This means that a proxy server can get the news early in the morning and keep it until the next day. Another recommendation is to add some headers to show the last modification time of the information. The proxy server can then use the information in this header to guess how long the information would be valid.

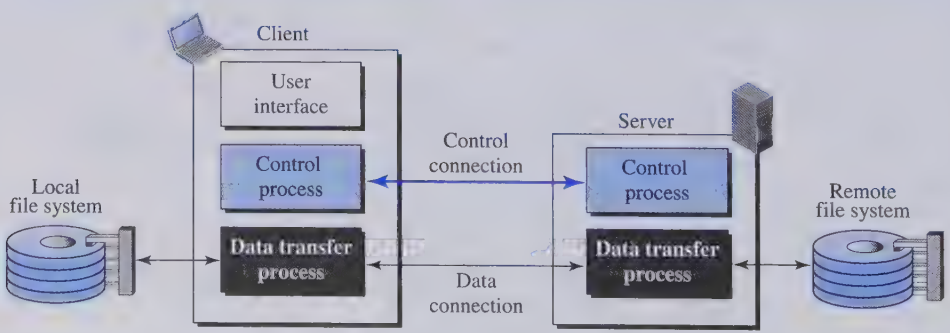
HTTP Security

HTTP per se does not provide security. However, HTTP can be run over the Secure Socket Layer (SSL). In this case, HTTP is referred to as HTTPS. HTTPS provides confidentiality, client and server authentication, and data integrity.

10.3.2 FTP

File Transfer Protocol (FTP) is the standard protocol provided by TCP/IP for copying a file from one host to another. Although transferring files from one system to another seems simple and straightforward, some problems must be dealt with first. For example, two systems may use different file name conventions, have different ways to represent data, and have different directory structures. All these problems have been solved by FTP in a very simple and elegant approach. Although we can transfer files using HTTP, FTP is a better choice to transfer large files or to transfer files using different formats. Figure 10.17 shows the basic model of FTP. The client has three components: user interface, client control process, and the client data transfer process. The server has two components: server control process and server data transfer process. The control connection is made between the control processes. The data connection is made between the data transfer processes.

Figure 10.17 *FTP basic model*



The separation of commands and data transfer makes FTP more efficient. The control connection uses very simple rules of communication. We need to transfer only a line of command or a line of response at a time. The data connection, on the other hand, needs more complex rules due to the variety of data types transferred.

Lifetimes of Two Connections

The two connections in FTP have different lifetimes. The control connection remains connected during the entire interactive FTP session. The data connection is opened and then closed for each file transfer activity. It opens each time commands that involve transferring files are used, and it closes when the file is transferred. In other words, when a user starts an FTP session, the control connection opens. While the control connection is open, the data connection can be opened and closed multiple times if several files are transferred. FTP uses two well-known TCP ports: Port 21 is used for the control connection, and port 20 is used for the data connection.

Control Connection

For control communication, FTP uses the same approach as TELNET (discussed later in Section 10.3.4). It uses the NVT ASCII character set as used by TELNET. Communication is achieved through commands and responses. This simple method is adequate for the control connection because we send one command (or response) at a time. Each line is terminated with a two-character (carriage return and line feed) end-of-line token.

During this control connection, commands are sent from the client to the server and responses are sent from the server to the client. Commands, which are sent from the FTP client control process, are in the form of ASCII uppercase, which may or may not be followed by an argument. Some of the most common commands are shown in Table 10.4.

Table 10.4 *Some common FTP commands*

Command	*Argument(s)*	*Description*
ABOR		Abort the previous command
CDUP		Change to parent directory
CWD	Directory name	Change to another directory
DELE	File name	Delete a file
LIST	Directory name	List subdirectories or files
MKD	Directory name	Create a new directory
PASS	User password	Password
PASV		Server chooses a port
PORT	port identifier	Client chooses a port
PWD		Display name of current directory
QUIT		Log out of the system
RETR	File name(s)	Retrieve files; files are transferred from server to client
RMD	Directory name	Delete a directory
RNFR	File name (old)	Identify a file to be renamed
RNTO	File name (new)	Rename the file
STOR	File name(s)	Store files; file(s) are transferred from client to server
STRU	**F**, **R**, or **P**	Define data organization (**F**: file, **R**: record, or **P**: page)
TYPE	**A**, **E**, **I**	Default file type (**A**: ASCII, **E**: EBCDIC, **I**: image)
USER	User ID	User information
MODE	**S**, **B**, or **C**	Define transmission mode (**S**: stream, **B**: block, or **C**: compressed)

Every FTP command generates at least one response. A response has two parts: a three-digit number followed by text. The numeric part defines the code; the text part defines needed parameters or further explanations. The first digit defines the status of the command. The second digit defines the area in which the status applies. The third digit provides additional information. Table 10.5 shows some common responses.

Table 10.5 *Some common responses in FTP*

Code	*Description*	*Code*	*Description*
125	Data connection open	**250**	Request file action OK
150	File status OK	**331**	User name OK; password is needed
200	Command OK	**425**	Cannot open data connection
220	Service ready	**450**	File action not taken; file not available
221	Service closing	**452**	Action aborted; insufficient storage
225	Data connection open	**500**	Syntax error; unrecognized command
226	Closing data connection	**501**	Syntax error in parameters or arguments
230	User login OK	**530**	User not logged in

Data Connection

The data connection uses the well-known port 20 at the server site. However, the creation of a data connection is different from the control connection. The steps are as follows:

1. The client, not the server, issues a passive open using an ephemeral port. This must be done by the client because it is the client that issues the commands for transferring files.
2. The client sends this port number to the server using the PORT command.
3. The server receives the port number and issues an active open using the well-known port 20 and the received ephemeral port number.

Communication over Data Connection

The purpose and implementation of the data connection are different from those of the control connection. We want to transfer files through the data connection. The client must define the type of file to be transferred, the structure of the data, and the transmission mode. Before sending the file through the data connection, we prepare for transmission through the control connection. The heterogeneity problem is resolved by defining three attributes of communication: file type, data structure, and transmission mode.

Data Structure FTP can transfer a file across the data connection using one of the following interpretations of the structure of the data: *file structure*, *record structure*, or *page structure*. The file structure format (used by default) has no structure. It is a continuous stream of bytes. In the record structure, the file is divided into *records*. This can be used only with text files. In the page structure, the file is divided into pages, with each page having a page number and a page header. The pages can be stored and accessed randomly or sequentially.

File Type FTP can transfer one of the following file types across the data connection: ASCII file or image file.

Transmission Mode FTP can transfer a file across the data connection using one of the following three transmission modes: *stream mode*, *block mode*, or *compressed mode*. The stream mode is the default mode; data are delivered from FTP to TCP as a continuous stream of bytes. In the block mode, data can be delivered from FTP to TCP in blocks. In this case, each block is preceded by a 3-byte header. The first byte is called the *block descriptor*; the next 2 bytes define the size of the block in bytes.

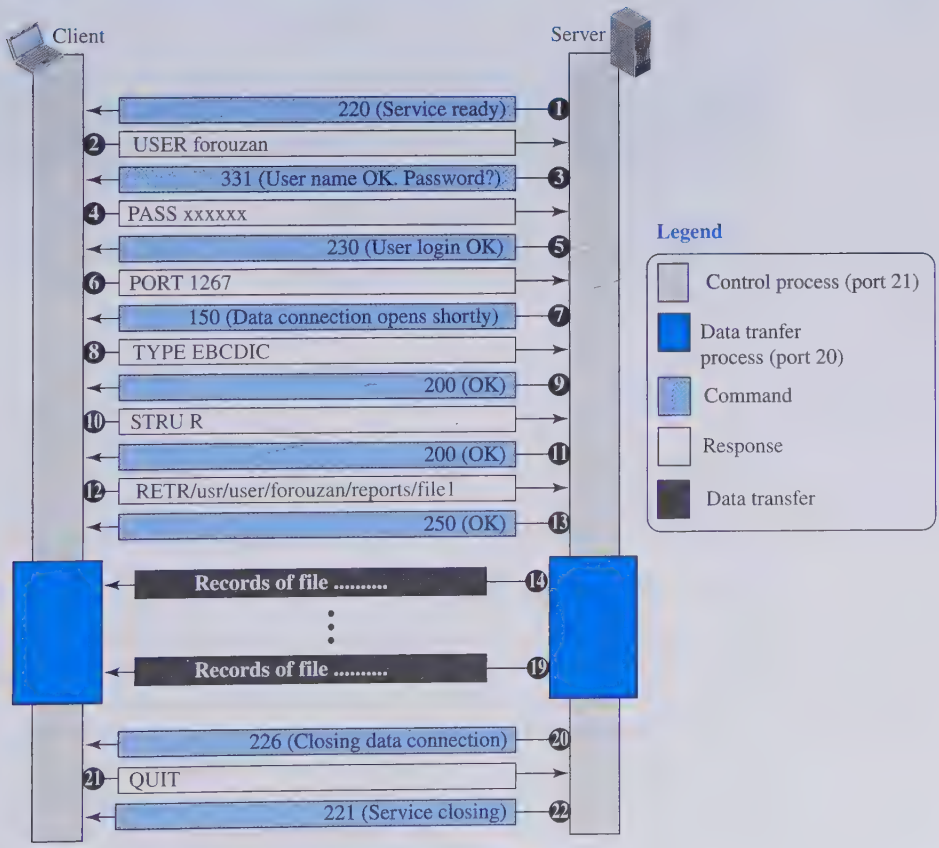
File Transfer

File transfer occurs over the data connection under the control of the commands sent over the control connection. However, we should remember that file transfer in FTP means one of three things: *retrieving a file* (server to client), *storing a file* (client to server), and *directory listing* (server to client).

Example 10.11

Figure 10.18 shows an example of using FTP for retrieving a file. The figure shows only one file to be transferred. The control connection remains open all the time, but the data connection is opened and closed repeatedly. We assume the file is transferred in six sections. After all records

Figure 10.18 *Example 10.11*



have been transferred, the server control process announces that the file transfer is done. Because the client control process has no file to retrieve, it issues the QUIT command, which causes the service connection to be closed.

Example 10.12

The following shows an actual FTP session that lists the directories. The colored lines show the responses from the server control connection; the black lines show the commands sent by the client. The lines in white with black background show data transfer.

```
$ ftp voyager.deanza.fhda.edu
Connected to voyager.deanza.fhda.edu.
220 (vsFTPd 1.2.1)
530 Please login with USER and PASS.
Name (voyager.deanza.fhda.edu:forouzan): forouzan
331 Please specify the password.
Password:*********
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
227 Entering Passive Mode (153,18,17,11,238,169)
150 Here comes the directory listing.
drwxr-xr-x    2    3027    411    4096    Sep 24    2002    business
drwxr-xr-x    2    3027    411    4096    Sep 24    2002    personal
drwxr-xr-x    2    3027    411    4096    Sep 24    2002    school
226 Directory send OK.
ftp> quit
221 Goodbye.
```

Security for FTP

The FTP protocol was designed when security was not a big issue. Although FTP requires a password, the password is sent in plaintext (unencrypted), which means it can be intercepted and used by an attacker. The data transfer connection also transfers data in plaintext, which is insecure. To be secure, one can add a Secure Socket Layer between the FTP application layer and the TCP layer. In this case FTP is called SSL-FTP. We also explore some secure file transfer applications when we discuss SSH later in Section 10.3.5.

10.3.3 Electronic Mail

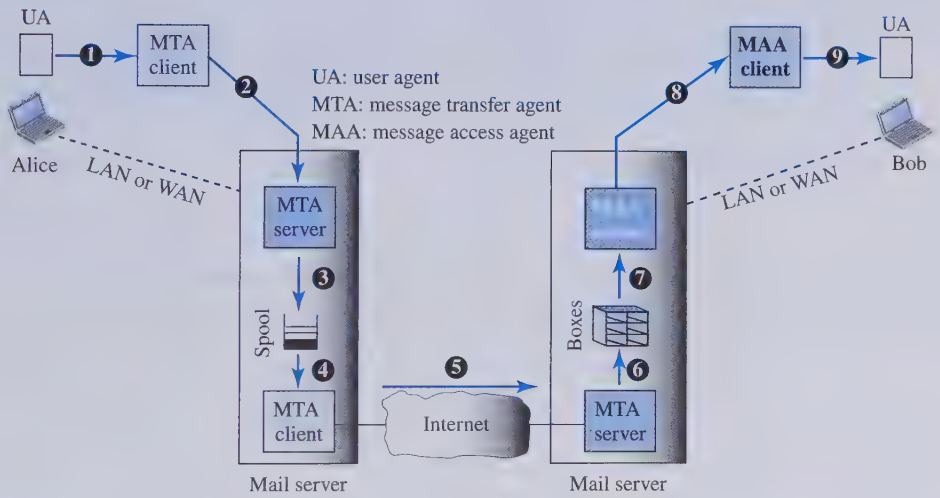
Electronic mail (e-mail) allows users to exchange messages. The nature of this application, however, is different from the other applications discussed so far. In an application such as HTTP or FTP, the server program is running all the time, waiting for a request from a client. When the request arrives, the server provides the service. There is a request, and there is a response. In the case of e-mail, the situation is different. First, e-mail is considered a one-way transaction. When Alice sends an e-mail to Bob, she may expect a response, but this is not a mandate. Bob may or may not respond. If he does respond, it is another one-way

transaction. Second, it is neither feasible nor logical for Bob to run a server program and wait until someone sends an e-mail to him. Bob may turn off his computer when he is not using it. This means that the idea of client/server programming should be implemented in another way: using some intermediate computers (servers). The users run only client programs when they want and the intermediate servers apply the client/server paradigm.

Architecture

To explain the architecture of e-mail, we give a common scenario, as shown in Figure 10.19. Another possibility is the case in which Alice or Bob is directly connected to the corresponding mail server, in which LAN or WAN connection is not required, but this variation in the scenario does not affect our discussion.

Figure 10.19 *Common scenario*



In the common scenario, the sender and the receiver of the e-mail, Alice and Bob, respectively, are connected via a LAN or a WAN to two mail servers. The administrator has created one mailbox for each user where the received messages are stored. A *mailbox* is part of a server hard drive, a special file with permission restrictions. Only the owner of the mailbox has access to it. The administrator has also created a queue (spool) to store messages waiting to be sent.

A simple e-mail from Alice to Bob takes nine different steps, as shown in Figure 10.19. Alice and Bob use three different *agents*: a **user agent** (**UA**), a **message transfer agent** (**MTA**), and a **message access agent** (**MAA**). When Alice needs to send a message to Bob, she runs a UA program to prepare the message and send it to her mail server. The mail server at her site uses a queue (spool) to store messages waiting to be sent. The message, however, needs to be sent through the Internet from Alice's site to Bob's site using an MTA. Here two message transfer agents are needed: one client and one server. Like most client/server programs on the Internet, the server needs to run all the time because it

does not know when a client will ask for a connection. The client, on the other hand, can be triggered by the system when there is a message in the queue to be sent. The user agent at Bob's site allows Bob to read the received message. Bob later uses an MAA client to retrieve the message from an MAA server running on the second server.

There are two important points we need to emphasize here. First, Bob cannot bypass the mail server and use the MTA server directly. To use the MTA server directly, Bob would need to run the MTA server all the time because he does not know when a message will arrive. This implies that Bob must keep his computer on all the time if he is connected to his system through a LAN. If he is connected through a WAN, he must keep the connection up all the time. Neither of these situations is feasible today.

Second, note that Bob needs another pair of client/server programs: message access programs. This is because an MTA client/server program is a *push* program: The client pushes the message to the server. Bob needs a *pull* program. The client needs to pull the message from the server. We discuss more about MAAs shortly.

The electronic mail system needs two UAs, two pairs of MTAs (client and server), and a pair of MAAs (client and server).

User Agent

The first component of an electronic mail system is the **user agent (UA)**. It provides a service to the user to make the process of sending and receiving a message easier. A user agent is a software package (program) that composes, reads, replies to, and forwards messages. It also handles local mailboxes on the user computers.

There are two types of user agents: command-driven and based on the graphical user interface (GUI). Command-driven user agents belong to the early days of electronic mail. They are still present as the underlying user agents. A command-driven user agent normally accepts a one-character command from the keyboard to perform its task. For example, a user can type the character *r*, at the command prompt, to reply to the sender of the message, or type the character *R* to reply to the sender and all recipients. Some examples of command-driven user agents are *mail*, *pine*, and *elm*.

Modern user agents are GUI-based. They contain GUI components that allow the user to interact with the software by using both the keyboard and the mouse. They have graphical components such as icons, menu bars, and windows that make the services easy to access. Some examples of GUI-based user agents are *Eudora* and *Outlook*.

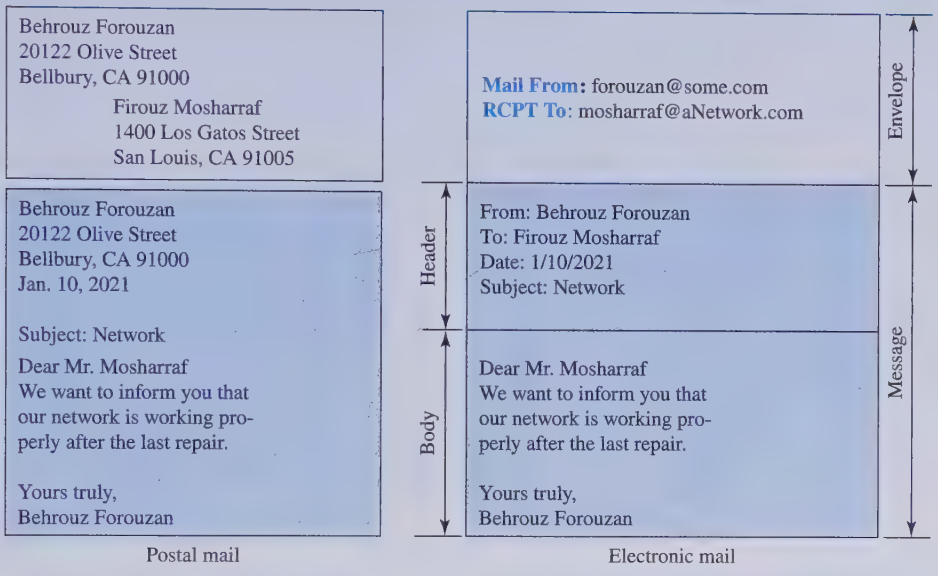
Sending Mail

To send mail, the user, through the UA, creates mail that looks very similar to postal mail. It has an *envelope* and a *message* (see Figure 10.20). The envelope usually contains the sender address, the receiver address, and other information. The message contains the *header* and the *body*. The header of the message defines the sender, the receiver, the subject of the message, and some other information. The body of the message contains the actual information to be read by the recipient.

Receiving Mail

The user agent is triggered by the user (or a timer). If a user has mail, the UA informs the user with a notice. If the user is ready to read the mail, a list is displayed in which each line

Figure 10.20 *Format of an e-mail*

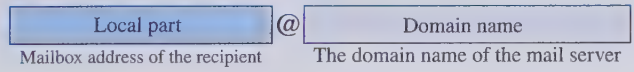


contains a summary of the information about a particular message in the mailbox. The summary usually includes the sender mail address, the subject, and the time the mail was sent or received. The user can select any of the messages and display its contents on the screen.

Addresses

To deliver mail, a mail handling system must use an addressing system with unique addresses. In the Internet, the address consists of two parts: a *local part* and a *domain name*, separated by an @ sign (see Figure 10.21).

Figure 10.21 *E-mail address*



The local part defines the name of a special file, called the user mailbox, where all the mail received for a user is stored for retrieval by the message access agent. The second part of the address is the domain name. An organization usually selects one or more hosts to receive and send e-mail; they are sometimes called *mail servers* or *exchangers*. The domain name assigned to each mail exchanger either comes from the DNS database or is a logical name (for example, the name of the organization).

Mailing List or Group List

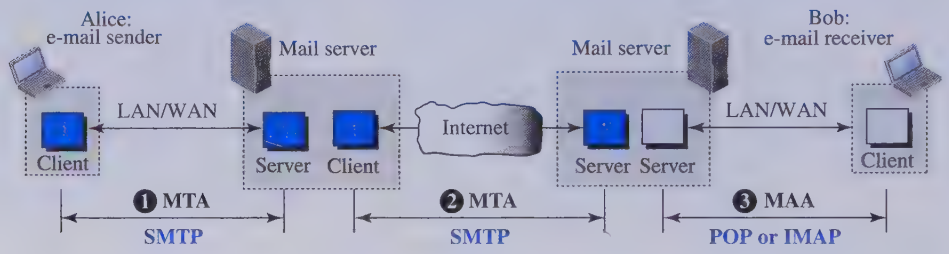
Electronic mail allows one name, an *alias*, to represent several different e-mail addresses; this is called a mailing list. Every time a message is to be sent, the system checks the

recipient's name against the alias database; if there is a mailing list for the defined alias, separate messages, one for each entry in the list, must be prepared and handed to the MTA.

Simple Mail Transfer Protocol (SMTP)

Based on the common scenario (Figure 10.19), we can say that e-mail is one of those applications that needs three uses of client/server paradigms to accomplish its task. It is important that we distinguish these three when we are dealing with e-mail. Figure 10.22 shows these three client/server applications. We refer to the first and the second as message transfer agents (MTAs) and the third as a message access agent (MAA).

Figure 10.22 *Protocols used in electronic mail*



The formal protocol that defines the MTA client and server in the Internet is called **Simple Mail Transfer Protocol (SMTP)**. SMTP is used two times, between the sender and the sender's mail server and between the two mail servers. As we will see shortly, another protocol is needed between the mail server and the receiver. SMTP simply defines how commands and responses must be sent back and forth.

Commands and Responses

SMTP uses commands and responses to transfer messages between an MTA client and an MTA server. The command is from an MTA client to an MTA server; the response is from an MTA server to the MTA client. Each command or reply is terminated by a two-character (carriage return and line feed) end-of-line token.

Commands Commands are sent from the client to the server. The format of a command is shown as

Keyword: argument(s)

It consists of a keyword followed by zero or more arguments. SMTP defines 14 commands, listed in Table 10.6.

Responses Responses are sent from the server to the client. A response is a three-digit code that may be followed by additional textual information. Table 10.7 shows the most common response types.

Table 10.6 *SMTP commands*

Keyword	*Argument(s)*	*Description*
HELO	Sender's host name	Identifies itself
MAIL FROM	Sender of the message	Identifies the sender of the message
RCPT TO	Intended recipient	Identifies the recipient of the message
DATA	Body of the mail	Sends the actual message
QUIT		Terminates the message
RSET		Aborts the current mail transaction
VRFY	Name of recipient	Verifies the address of the recipient
NOOP		Checks the status of the recipient
TURN		Switches the sender and the recipient
EXPN	Mailing list	Asks the recipient to expand the mailing list
HELP	Command name	Asks the recipient to send information about the command sent as the argument
SEND FROM	Intended recipient	Specifies that the mail be delivered only to the terminal of the recipient, and not to the mailbox
SMOL FROM	Intended recipient	Specifies that the mail be delivered to the terminal *or* the mailbox of the recipient
SMAL FROM	Intended recipient	Specifies that the mail be delivered to the terminal *and* the mailbox of the recipient

Table 10.7 *Responses*

Code	*Description*
	Positive Completion Reply
211	System status or help reply
214	Help message
220	Service ready
221	Service closing transmission channel
250	Request command completed
251	User not local; the message will be forwarded
	Positive Intermediate Reply
354	Start mail input
	Transient Negative Completion Reply
421	Service not available
450	Mailbox not available
451	Command aborted: local error
452	Command aborted; insufficient storage
	Permanent Negative Completion Reply
500	Syntax error; unrecognized command
501	Syntax error in parameters or arguments
502	Command not implemented
503	Bad sequence of commands
504	Command temporarily not implemented
550	Command is not executed; mailbox unavailable
551	User not local
552	Requested action aborted; exceeded storage location
553	Requested action not taken; mailbox name not allowed
554	Transaction failed

Mail Transfer Phases

The process of transferring a mail message occurs in three phases: connection establishment, mail transfer, and connection termination.

Connection Establishment After a client has made a TCP connection to the well-known port 25, the SMTP server starts the connection phase. This phase involves the following three steps:

1. The server sends code 220 (service ready) to tell the client that it is ready to receive mail. If the server is not ready, it sends code 421 (service not available).
2. The client sends the HELO message to identify itself, using its domain name address. This step is necessary to inform the server of the domain name of the client.
3. The server responds with code 250 (request command completed) or some other code depending on the situation.

Message Transfer After a connection has been established between the SMTP client and server, a single message between a sender and one or more recipients can be exchanged. This phase involves eight steps. Steps 3 and 4 are repeated if there is more than one recipient.

1. The client sends the MAIL FROM message to introduce the sender of the message. It includes the mail address of the sender (mailbox and the domain name). This step is needed to give the server the return mail address for returning errors and reporting messages.
2. The server responds with code 250 or some other appropriate code.
3. The client sends the RCPT TO (recipient) message, which includes the mail address of the recipient.
4. The server responds with code 250 or some other appropriate code.
5. The client sends the DATA message to initialize the message transfer.
6. The server responds with code 354 (start mail input) or some other appropriate message.
7. The client sends the contents of the message in consecutive lines. Each line is terminated by a two-character end-of-line token (carriage return and line feed). The message is terminated by a line containing just one period.
8. The server responds with code 250 (OK) or some other appropriate code.

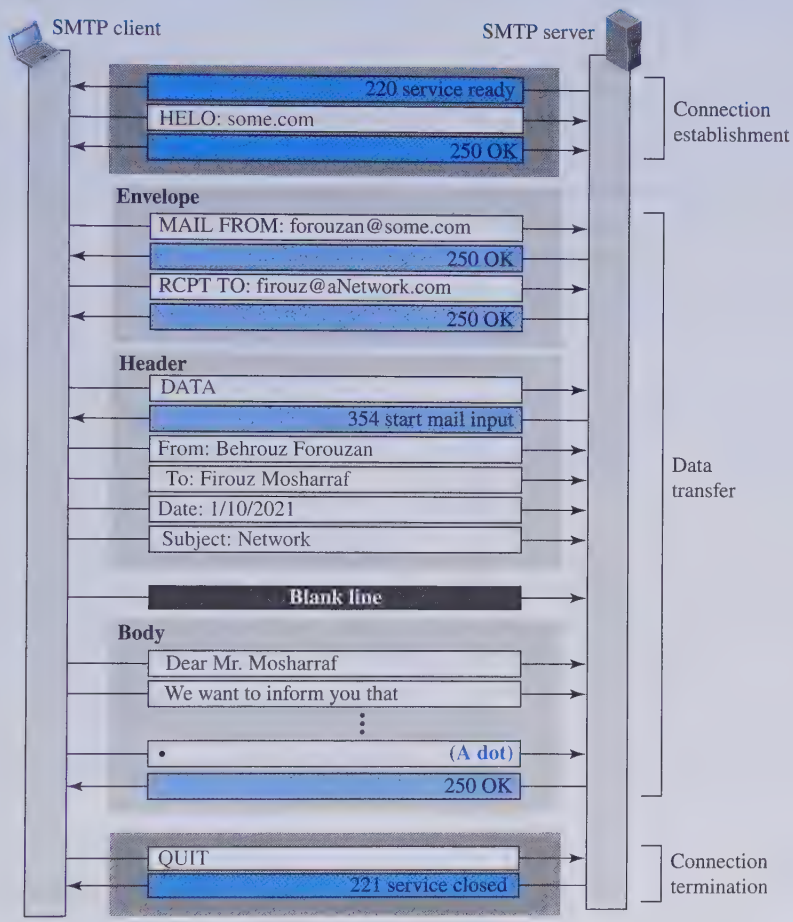
Connection Termination After the message is transferred successfully, the client terminates the connection. This phase involves two steps.

1. The client sends the QUIT command.
2. The server responds with code 221 or some other appropriate code.

Example 10.13

To show the three mail transfer phases, we show all the steps described above using the information depicted in Figure 10.23. In the figure, we have separated the messages related to the envelope, header, and body in the data transfer section. Note that the steps in this figure are repeated 2 times in each e-mail transfer: once from the e-mail sender to the local mail server and once from the local mail server to the remote mail server. The local mail server, after receiving the whole e-mail message, may spool it and send it to the remote mail server at another time.

Figure 10.23 *Example 10.13*



Message Access Agent: POP and IMAP

The first and second stages of mail delivery use SMTP. However, SMTP is not involved in the third stage because SMTP is a *push* protocol; it pushes the message from the client to the server. In other words, the direction of the bulk data (messages) is from the client to the server. On the other hand, the third stage needs a *pull* protocol; the client must pull messages from the server. The direction of the bulk data is from the server to the client. The third stage uses a message access agent.

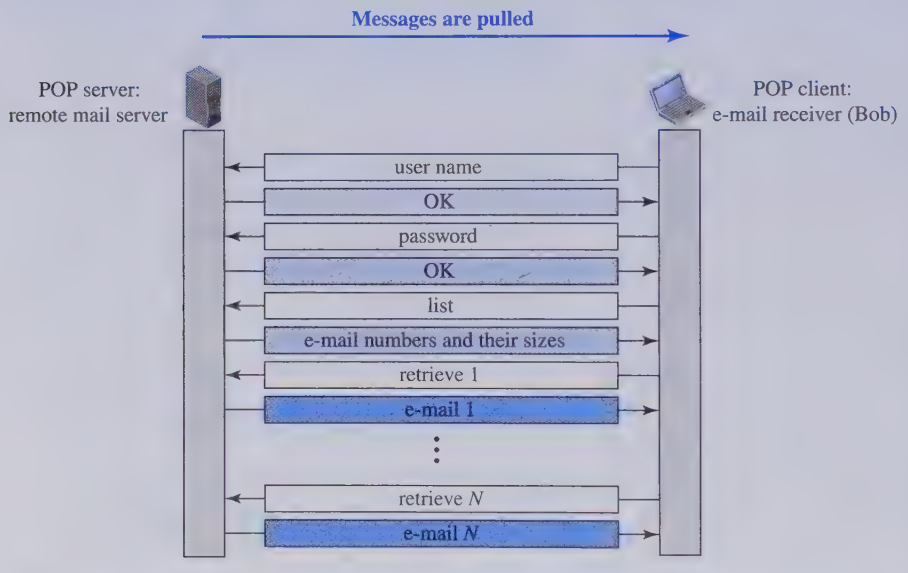
Currently two message access protocols are available: Post Office Protocol, version 3 (POP3), and Internet Mail Access Protocol, version 4 (IMAP4). Figure 10.22 shows the position of these two protocols.

POP3

Post Office Protocol, version 3 (POP3), is simple but limited in functionality. The client POP3 software is installed on the recipient computer; the server POP3 software is installed on the mail server.

Mail access starts with the client when the user needs to download its e-mail from the mailbox on the mail server. The client opens a connection to the server on TCP port 110. It then sends its user name and password to access the mailbox. The user can then list and retrieve the mail messages, one by one. Figure 10.24 shows an example of downloading using POP3. Unlike other figures in this chapter, we have put the client on the right-hand side because the e-mail receiver (Bob) is running the client process to pull messages from the remote mail server.

Figure 10.24 *POP3*



POP3 has two modes: *delete* mode and *keep* mode. In the delete mode, the mail is deleted from the mailbox after each retrieval. In the keep mode, the mail remains in the mailbox after retrieval. The delete mode is normally used when the user is working at her permanent computer and can save and organize the received mail after reading or replying. The keep mode is normally used when the user accesses her mail away from her primary computer (for example, from a laptop). The mail is read but kept in the system for later retrieval and organizing.

IMAP4

Another mail access protocol is **Internet Mail Access Protocol, version 4 (IMAP4)**. IMAP4 is similar to POP3, but it has more features; IMAP4 is more powerful and more complex.

POP3 is deficient in several ways. It does not allow the user to organize her mail on the server; the user cannot have different folders on the server. In addition, POP3 does not allow the user to partially check the contents of the mail before downloading. IMAP4 provides the following extra functions:

- A user can check the e-mail header prior to downloading.
- A user can search the contents of the e-mail for a specific string of characters prior to downloading.
- A user can partially download e-mail. This is especially useful if bandwidth is limited and the e-mail contains multimedia with high bandwidth requirements.
- A user can create, delete, or rename mailboxes on the mail server.
- A user can create a hierarchy of mailboxes in a folder for e-mail storage.

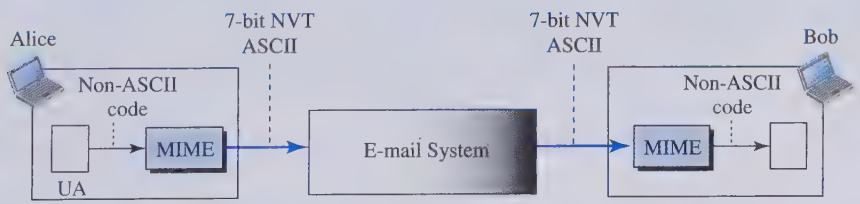
MIME

Electronic mail has a simple structure. Its simplicity, however, comes with a price. It can send messages only in NVT 7-bit ASCII format. In other words, it has some limitations. can only be used for languages that use the Latin alphabet. Also, it cannot be used to send binary files or video or audio data.

Multipurpose Internet Mail Extensions (MIME) is a supplementary protocol that allows non-ASCII data to be sent through e-mail. MIME transforms non-ASCII data at the sender site to NVT ASCII data and delivers it to the client MTA to be sent through the Internet. The message at the receiving site is transformed back to the original data.

We can think of MIME as a set of software functions that transforms non-ASCII data to ASCII data, and vice versa, as shown in Figure 10.25.

Figure 10.25 *MIME*



MIME Headers

MIME defines five headers, as shown in Figure 10.26, which can be added to the original e-mail header section to define the transformation –parameters.

MIME-Version This header defines the version of MIME used. The current version is 1.1.

Content-Type This header defines the type of data used in the body of the message. The content type and the content subtype are separated by a slash. Depending on the subtype, the header may contain other parameters. MIME allows seven different types of data, listed in Table 10.8 (on next page).

Figure 10.26 *MIME header*

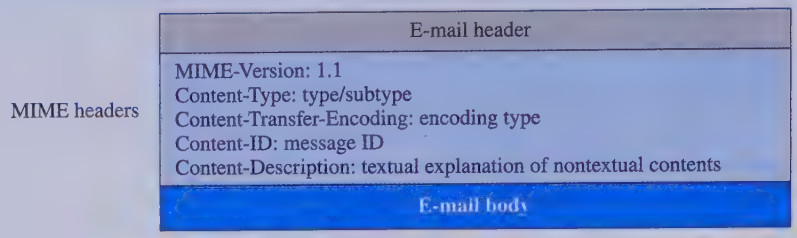


Table 10.8 *Data types and subtypes in MIME*

Type	*Subtype*	*Description*
Text	Plain	Unformatted
	HTML	HTML format (see Appendix C)
Multipart	Mixed	Body contains ordered parts of different data types
	Parallel	Same as above, but no order
	Digest	Similar to Mixed, but the default is message/RFC822
	Alternative	Parts are different versions of the same message
Message	RFC822	Body is an encapsulated message
	Partial	Body is a fragment of a bigger message
	External-Body	Body is a reference to another message
Image	JPEG	Image is in JPEG format
	GIF	Image is in GIF format
Video	MPEG	Video is in MPEG format
Audio	Basic	Single channel encoding of voice at 8 kHz
Application	PostScript	Adobe PostScript
	Octet-stream	General binary data (8-bit bytes)

Content-Transfer-Encoding This header defines the method used to encode the messages into 0s and 1s for transport. The five types of encoding methods are listed in Table 10.9.

Table 10.9 *Methods for Content-Transfer-Encoding*

Type	*Description*
7-bit	NVT ASCII characters with each line less than 1000 characters
8-bit	Non-ASCII characters with each line less than 1000 characters
Binary	Non-ASCII characters with unlimited-length lines
Base64	6-bit blocks of data encoded into 8-bit ASCII characters
Quoted-printable	Non-ASCII characters encoded as an equal sign plus an ASCII code

The last two encoding methods are interesting. In the Base64 encoding, data, as a string of bits, are first divided into 6-bit chunks as shown in Figure 10.27. Each 6-bit section is then converted into an ASCII character according to Table 10.10.

Figure 10.27 *Base64 conversion*

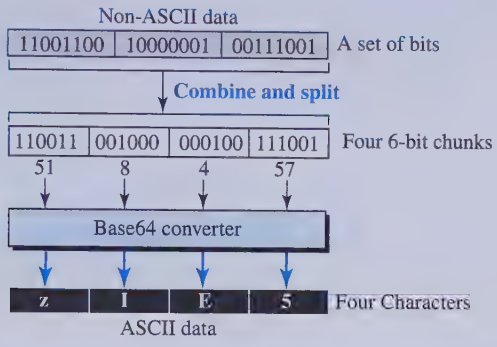


Table 10.10 *Base64 converting table*

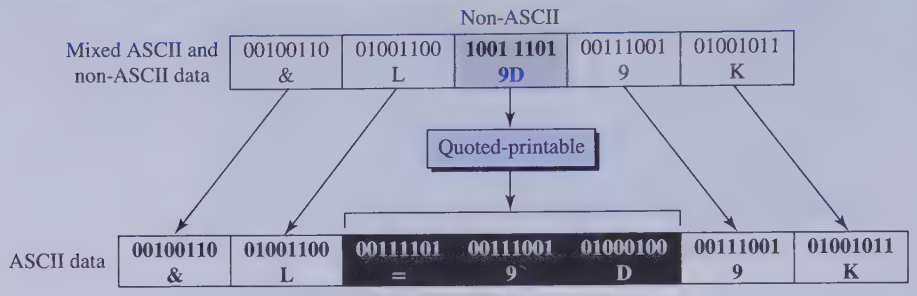
Value	*Code*	*Value*	*Code*	*Value*	*Code*	*Value*	*Code*	*Value*	*Code*	*Value*	*Code*
0	**A**	11	**L**	22	**W**	33	**h**	44	**s**	55	**3**
1	**B**	12	**M**	23	**X**	34	**i**	45	**t**	56	**4**
2	**C**	13	**N**	24	**Y**	35	**j**	46	**u**	57	**5**
3	**D**	14	**O**	25	**Z**	36	**k**	47	**v**	58	**6**
4	**E**	15	**P**	26	**a**	37	**l**	48	**w**	59	**7**
5	**F**	16	**Q**	27	**b**	38	**m**	49	**x**	60	**8**
6	**G**	17	**R**	28	**c**	39	**n**	50	**y**	61	**9**
7	**H**	18	**S**	29	**d**	40	**o**	51	**z**	62	**+**
8	**I**	19	**T**	30	**e**	41	**p**	52	**0**	63	**/**
9	**J**	20	**U**	31	**f**	42	**q**	53	**1**		
10	**K**	21	**V**	32	**g**	43	**r**	54	**2**		

Base64 is a redundant encoding scheme; that is, every 6 bits become one ASCII character and are sent as 8 bits. We have an overhead of 25 percent. If the data -consist mostly of ASCII characters with a small non-ASCII portion, we can use quoted-printable encoding. In quoted-printable encoding, if a character is ASCII, it is sent as is. If a character is not ASCII, it is sent as three characters. The first character is the equal sign (=). The next two characters are the hexadecimal representations of the byte. Figure 10.28 (on next page) shows an example. Here, the third character is non-ASCII because it starts with bit 1. It is interpreted as two hexadecimal digits ($9D_{16}$), which is replaced by three ASCII characters (=, 9, and D).

Content-ID This header uniquely identifies the whole message in a multiple-message environment.

Content-Description This header defines whether the body is image, audio, or video.

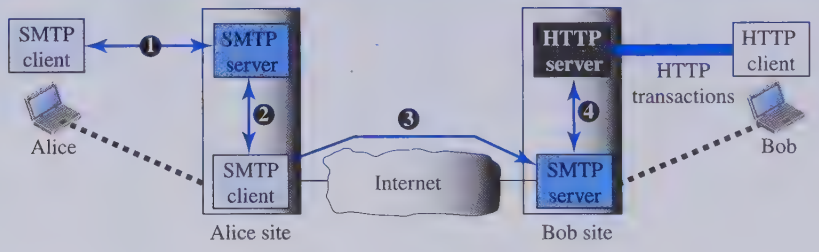
Figure 10.28 *Quoted-printable encoding*



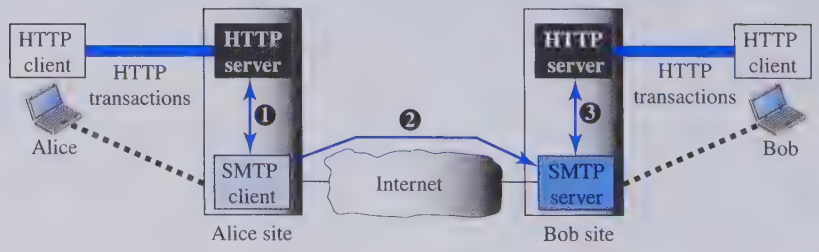
Web-Based Mail

E-mail is such a common application that some websites today provide this service to anyone who accesses the site. The idea is very simple. Figure 10.29 shows two cases.

Figure 10.29 *Web-based e-mail, cases I and II*



Case 1: Only receiver uses HTTP



Case 2: Both sender and receiver use HTTP

Case I

In the first case, Alice, the sender, uses a traditional mail server; Bob, the receiver, has an account on a web server. Mail transfer from Alice's browser to her mail server is done through SMTP. The transfer of the message from the sending mail server to the receiving mail server is still through SMTP. However, the message from the receiving server (the web server) to Bob's browser is done through HTTP. In other words, instead

of using POP3 or IMAP4, HTTP is normally used. When Bob needs to retrieve his -e-mails, he sends a request HTTP message to the website. The website sends a form to be filled in by Bob, which includes the log-in name and the password. If the log-in name and password match, the list of e-mails is transferred from the web server to Bob's browser in HTML format. Now Bob can browse through his received e-mails and then, using more HTTP transactions, can get his e-mails one by one.

Case II

In the second case, both Alice and Bob use web servers, but not necessarily the same server. Alice sends the message to the web server using HTTP transactions. Alice sends an HTTP request message to her web server using the name and address of Bob's mailbox as the URL. The server at the Alice site passes the message to the SMTP client and sends it to the server at the Bob site using SMTP. Bob receives the message using HTTP transactions. However, the message from the server at the Alice site to the server at the Bob site still takes place using SMTP.

E-Mail Security

The protocol discussed in this chapter does not provide any security provisions per se. However, e-mail exchanges can be secured using two application-layer securities designed in particular for e-mail systems. Two of these protocols, *Pretty Good Privacy (PGP)* and *Secure/Multipurpose Internet Mail Extensions (S/MIME)*, are discussed in Chapter 13 after we have discussed basic network security.

10.3.4 TELNET

A server program can provide a specific service to its corresponding client program. For example, the FTP server is designed to let the FTP client store or retrieve files on the server site. However, it is impossible to have a client/server pair for each type of service we need; the number of servers soon becomes intractable. The idea is not scalable. Another solution is to have a specific client/server program for a set of common scenarios, but to have some generic client/server programs that allow a user on the client site to log in to the computer at the server site and use the services available there. For example, if a student needs to use the Java compiler program at her university lab, there is no need for a Java compiler client and a Java compiler server. The student can use a client login program to log in to the university server and use the compiler program at the university. We refer to these generic client/server pairs as **remote login** applications.

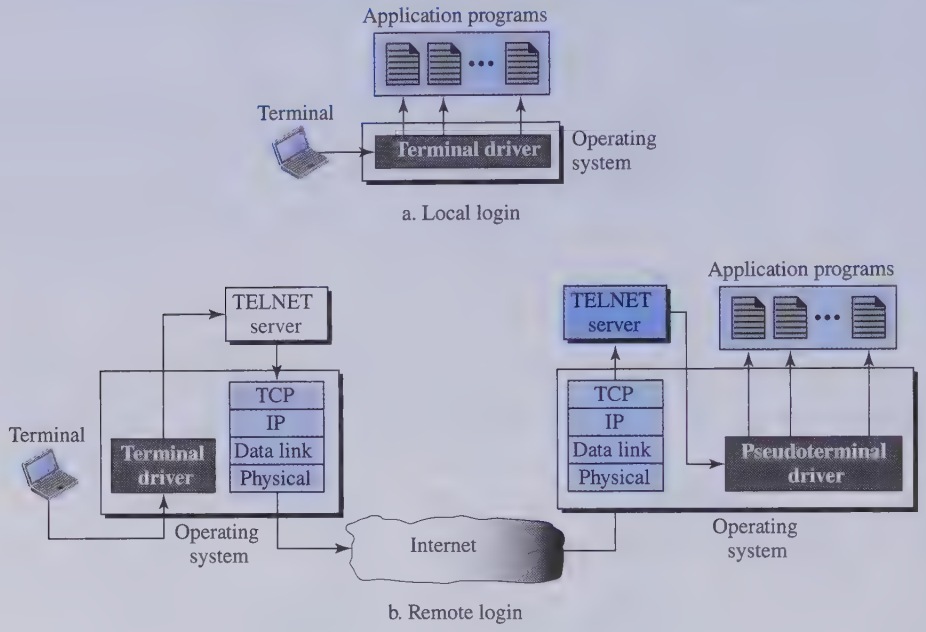
One of the original remote login protocols is **TELNET**, which is an abbreviation for *TErminaL NETwork*. Although TELNET requires a login name and password, it is vulnerable to hacking because it sends all data including the password in plaintext (not encrypted). A hacker can eavesdrop and obtain the login name and password. Because of this security issue, the use of TELNET has diminished in favor of another protocol, Secure Shell (SSH), which we describe in Section 10.3.5. Although TELNET has almost been replaced by SSH, we briefly discuss TELNET here for two reasons:

1. The simple plaintext architecture of TELNET allows us to explain the issues and challenges related to the concept of remote login, which is also used in SSH when it serves as a remote login protocol.
2. Network administrators often use TELNET for diagnostic and debugging purposes.

Local versus Remote Login

We first discuss the concept of local and remote login as shown in Figure 10.30.

Figure 10.30 *Local versus remote login*



When a user logs in to a local system, it is called **local login**. As a user types at a terminal or at a workstation running a terminal emulator, the keystrokes are accepted by the terminal driver. The terminal driver passes the characters to the operating system. The operating system, in turn, interprets the combination of characters and invokes the desired application program or utility.

However, when a user wants to access an application program or utility located on a remote machine, she performs *remote login*. Here the TELNET client and server programs come into use. The user sends the keystrokes to the terminal driver where the local operating system accepts the characters but does not interpret them. The characters are sent to the TELNET client, which transforms the characters into a universal character set called *network virtual terminal* (*NVT*) characters (discussed next) and delivers them to the local TCP/IP stack.

The commands or text, in NVT form, travel through the Internet and arrive at the TCP/IP stack at the remote machine. Here the characters are delivered to the operating system and passed to the TELNET server, which changes the characters to the corresponding characters understandable by the remote computer. However, the characters cannot be passed directly to the operating system because the remote operating system is not designed to receive characters from a TELNET server; it is designed to receive characters

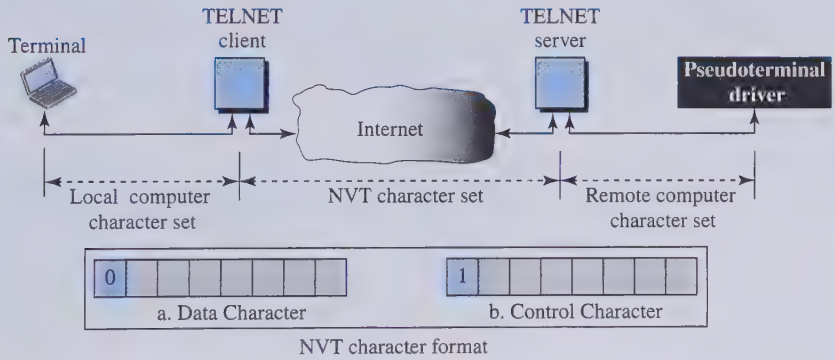
from a terminal driver. The solution is to add a piece of software called a *pseudoterminal driver*, which pretends that the characters are coming from a terminal. The operating system then passes the characters to the appropriate application program.

Network Virtual Terminal (NVT)

The mechanism to access a remote computer is complex. This is because every computer and its operating system accepts a special combination of characters as tokens. For example, the end-of-file token in a computer running the DOS operating system is Ctrl+z, while the UNIX operating system recognizes Ctrl+d.

We are dealing with heterogeneous systems. If we want to access any remote computer in the world, we must first know what type of computer we will be connected to, and we must also install the specific terminal emulator used by that computer. TELNET solves this problem by defining a universal interface called the **network virtual terminal (NVT)** character set. Via this interface, the client TELNET translates characters (data or commands) that come from the local terminal into NVT form and delivers them to the network. The server TELNET, on the other hand, translates data and commands from NVT form into the form acceptable by the remote computer. Figure 10.31 shows the concept.

Figure 10.31 *Concept of NVT*



NVT uses two sets of characters, one for data and one for control. Both are 8-bit bytes as shown in Figure 10.31. For data, NVT normally uses what is called NVT ASCII. This is an 8-bit character set in which the seven lowest-order bits are the same as US ASCII and the highest-order bit is 0. To send control characters between computers (from client to server, or vice versa), NVT uses an 8-bit character set in which the highest-order bit is set to 1.

Options

TELNET lets the client and server negotiate options before or during the use of the service. Options are extra features available to a user with a more sophisticated terminal. Users with simpler terminals can use default features.

User Interface

The operating system (UNIX, for example) defines an interface with user-friendly commands. An example of such a set of commands can be found in Table 10.11.

Table 10.11 *Examples of interface commands*

Command	*Meaning*	*Command*	*Meaning*
open	Connect to a remote computer	**set**	Set the operating parameters
close	Close the connection	**status**	Display the status information
display	Show the operating parameters	**send**	Send special characters
mode	Change to line or character mode	**quit**	Exit TELNET

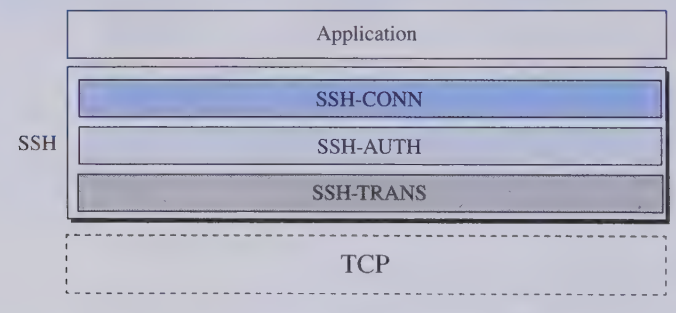
10.3.5 Secure Shell (SSH)

Although **Secure Shell** (**SSH**) is a secure application program that can be used today for several purposes such as remote login and file transfer, it was originally designed to replace TELNET. There are two versions of SSH: SSH-1 and SSH-2, which are totally incompatible. The first version, SSH-1, is now deprecated because of security flaws in it. In this section, we discuss only SSH-2.

Components

SSH is an application-layer protocol with three components, as shown in Figure 10.32.

Figure 10.32 *Components of SSH*



SSH Transport-Layer Protocol (SSH-TRANS)

Because TCP is not a secured transport-layer protocol, SSH first uses a protocol that creates a secured channel on top of the TCP. This new layer is an independent protocol referred to as SSH-TRANS. When the procedure implementing this protocol is called, the client and server first use TCP to establish an insecure connection. Then they exchange several security parameters to establish a secure channel on top of the TCP. We discussed transport-layer security in Chapter 13, but here we briefly list the services provided by this protocol:

1. Privacy or confidentiality of the message exchanged
2. Data integrity, which means that it is guaranteed that the messages exchanged between the client and server are not changed by an intruder

3. Server authentication, which means that the client is now sure that the server is the one that it claims to be
4. Compression of the messages, which improves the efficiency of the system and makes an attack more difficult

SSH Authentication Protocol (SSH-AUTH)

After a secure channel is established between the client and the server, and the server is authenticated for the client, SSH can call another procedure that can authenticate the client for the server. The client authentication process in SSH is very similar to what is done in Secure Socket Layer (SSL), which we discuss in Chapter 13. This layer defines a number of authentication tools similar to the ones used in SSL. Authentication starts with the client, which sends a request message to the server. The request includes the user name, server name, method of authentication, and required data. The server responds with either a success message, which confirms that the client is authenticated, or a failed message, which means that the process needs to be repeated with a new request message.

SSH Connection Protocol (SSH-CONN)

After the secured channel is established and both server and client are authenticated for each other, SSH can call a piece of software that implements the third protocol, SSH-CONN. One of the services provided by the SSH-CONN protocol is multiplexing. SSH-CONN takes the secure channel established by the two previous protocols and lets the client create multiple logical channels over it. Each channel can be used for a different purpose, such as remote login, file transfer.

Applications

Although SSH is often thought of as a replacement for TELNET, SSH is, in fact, a general-purpose protocol that provides a secure connection between a client and server.

SSH for Remote Login

Several free and commercial applications use SSH for remote login. Among them, we can mention PuTTy, by Simon Tatham, which is a client SSH program that can be used for remote login. Another application program is Tectia, which can be used on several platforms.

SSH for File Transfer

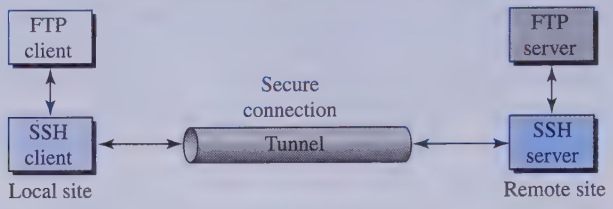
One of the application programs that is built on top of SSH for file transfer is the *Secure File Transfer Program (sftp)*. This application program uses one of the channels provided by the SSH to transfer files. Another common application is called *Secure Copy (scp)*. This application uses the same format as the UNIX copy command, *cp,* to copy files.

Port Forwarding

One of the interesting services provided by the SSH protocol is **port forwarding**. We can use the secured channels available in SSH to access an application program that does not provide security services. Applications such as TELNET and Simple Mail Transfer Protocol (SMTP) can use the services of the SSH port forwarding mechanism,

which creates a tunnel through which the messages belonging to other protocols can travel. For this reason, this mechanism is sometimes referred to as SSH *tunneling*. Figure 10.33 shows the concept of port forwarding for securing the FTP application.

Figure 10.33 *Port forwarding*

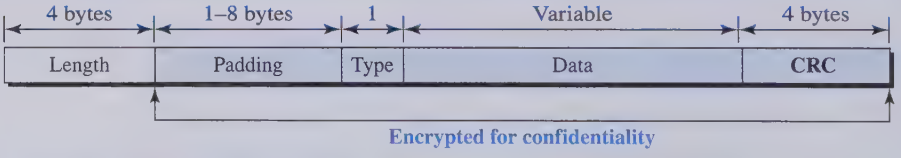


The FTP client can use the SSH client on the local site to make a secure connection with the SSH server on the remote site. Any request from the FTP client to the FTP server is carried through the tunnel provided by the SSH client and server. Any response from the FTP server to the FTP client is also carried through the tunnel provided by the SSH client and server.

Format of the SSH Packets

Figure 10.34 shows the format of packets used by the SSH protocols.

Figure 10.34 *SSH packet format*



The length field defines the length of the packet but does not include the padding. One to eight bytes of padding is added to the packet to make the attack on the security provision more difficult. The *cyclic redundancy check (CRC)* field is used for error detection. The type field designates the type of the packet used in different SSH protocols. The data field is the data transferred by the packet in different protocols.

10.3.6 Domain Name System (DNS)

The last client/server application program we discuss has been designed to help other application programs. To identify an entity, TCP/IP protocols use the IP address, which uniquely identifies the connection of a host to the Internet. However, people prefer to use names instead of numeric addresses. Therefore, the Internet needs to have a directory system that can map a name to an address. This is analogous to the telephone network. A telephone network is designed to use telephone numbers, not names. People

can either keep a private file to map a name to the corresponding telephone number or can call the telephone directory to do so. We discuss how this directory system in the Internet can map names to IP addresses.

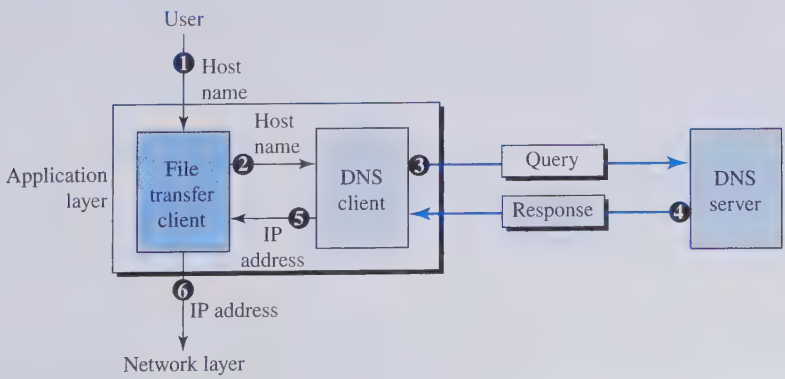
Because the Internet is so huge today, a central directory system cannot hold all the mapping. In addition, if the central computer fails, the whole communication network will collapse. A better solution is to distribute the information among many computers in the world. In this method, the host that needs mapping can contact the closest computer holding the needed information. This method is used by the **Domain Name System (DNS)**. We first discuss the concepts and ideas behind the DNS. We then describe the DNS protocol itself.

Figure 10.35 shows how TCP/IP uses a DNS client and a DNS server to map a name to an address. A user wants to use a file transfer client to access the corresponding file transfer server running on a remote host. The user knows only the file transfer server name, such as *afilesource.com*. However, the TCP/IP suite needs the IP address of the file transfer server to make the connection. The following six steps map the host name to an IP address:

1. The user passes the host name to the file transfer client.
2. The file transfer client passes the host name to the DNS client.
3. Each computer, after being booted, knows the address of one DNS server. The DNS client sends a message to a DNS server with a query that gives the file transfer server name using the known IP address of the DNS server.
4. The DNS server responds with the IP address of the desired file transfer server.
5. The DNS client passes the IP address to the file transfer server.
6. The file transfer client now uses the received IP address to access the file transfer server.

Note that the purpose of accessing the Internet is to make a connection between the file transfer client and server, but before this can happen, another connection needs to be made between the DNS client and DNS server. In other words, we need at least two connections in this case. The first is for mapping the name to an IP address; the second is for transferring files.

Figure 10.35 *Purpose of DNS*



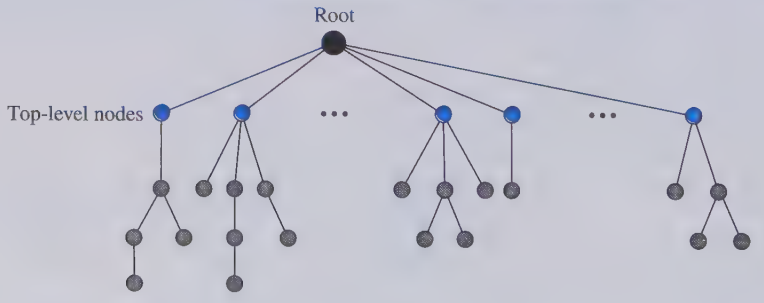
Name Space

To be unambiguous, the names assigned to machines must be carefully selected from a name space with complete control over the binding between the names and IP addresses. In other words, the names must be unique because the addresses are unique. A name space that maps each address to a unique name can be organized in two ways: flat or hierarchical. In a *flat name space*, a name is assigned to an address. A name in this space is a sequence of characters without structure. The names may or may not have a common section; if they do, it has no meaning. The main disadvantage of a flat name space is that it cannot be used in a large system such as the Internet because it must be centrally controlled to avoid ambiguity and duplication. In a *hierarchical name space*, each name is made up of several parts. The first part can define the nature of the organization, the second part can define the name of an organization, the third part can define departments in the organization, and so on. In this case, the authority to assign and control the name spaces can be decentralized. A central authority can assign the part of the name that defines the nature of the organization and the name of the organization. The responsibility for the rest of the name can be given to the organization itself. The organization can add suffixes (or prefixes) to the name to define its host or resources. The management of the organization need not worry that the prefix chosen for a host is taken by another organization because, even if part of an address is the same, the whole address is different. For example, assume two organizations call one of their computers *caesar*. The first organization is given a name by the central authority, such as *first.com*, the second organization is given the name *second.com*. When each of these organizations adds the name *caesar* to the name they have already been given, the end result is two distinguishable names: *ceasar.first.com* and *ceasar.second.com*. The names are unique.

Domain Name Space

To have a hierarchical name space, a **domain name space** was designed. In this design the names are defined in an inverted-tree structure with the root at the top. The tree can have only 128 levels: level 0 (root) to level 127 (see Figure 10.36).

Figure 10.36 *Domain name space*



Label

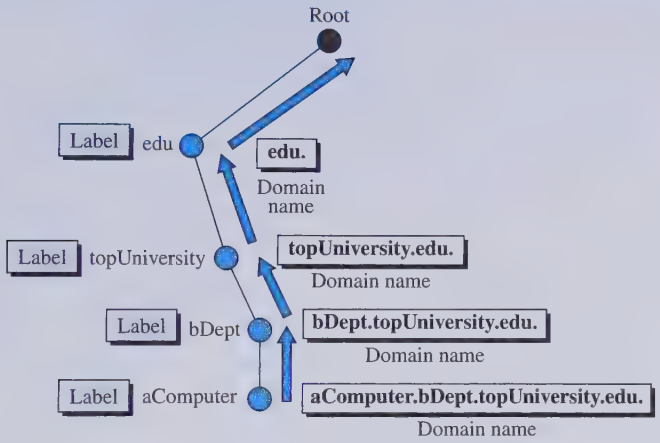
Each node in the tree has a **label**, which is a string with a maximum of 63 characters. The root label is a null string (empty string). DNS requires that children of a node

(nodes that branch from the same node) have different labels, which guarantees the uniqueness of the domain names.

Domain Name

Each node in the tree has a domain name. A full **domain name** is a sequence of labels separated by dots (.). The domain names are always read from the node up to the root. The last label is the label of the root (null). This means that a full domain name always ends in a null label, which means the last character is a dot because the null string is nothing. Figure 10.37 shows some domain names.

Figure 10.37 *Domain names and labels*



If a label is terminated by a null string, it is called a **fully qualified domain name (FQDN)**. The name must end with a null label, but because null means nothing, the label ends with a dot. If a label is not terminated by a null string, it is called a **partially qualified domain name (PQDN)**. A PQDN starts from a node, but it does not reach the root. It is used when the name to be resolved belongs to the same site as the client. Here the resolver can supply the missing part, called the *suffix*, to create an FQDN.

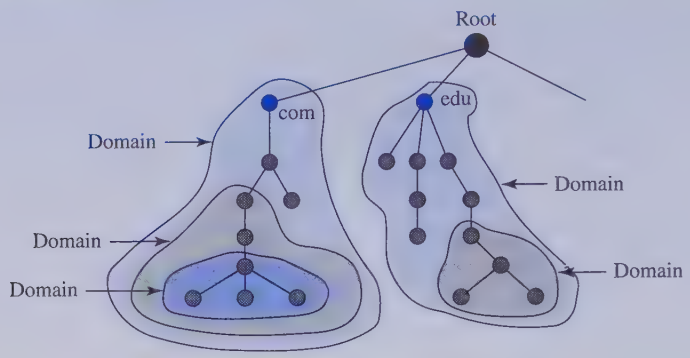
Domain

A **domain** is a subtree of the domain name space. The name of the domain is the name of the node at the top of the subtree. Figure 10.38 shows some domains. Note that a domain may itself be divided into domains.

Distribution of Name Space

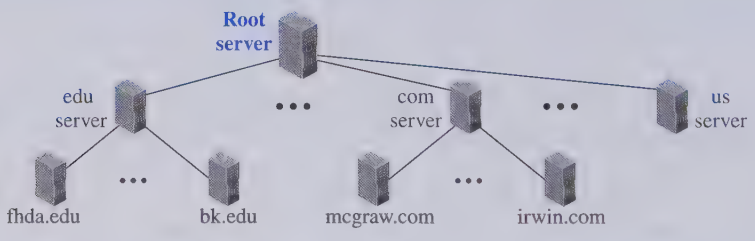
The information contained in the domain name space must be stored. However, it is very inefficient and also not reliable to have just one computer store such a huge amount of information. It is inefficient because responding to requests from all over the world places a heavy load on the system. It is not reliable because any failure makes the data inaccessible.

Figure 10.38 *Domains*



Hierarchy of Name Servers The solution to these problems is to distribute the information among many computers called **DNS servers**. One way to do this is to divide the whole space into many domains based on the first level. In other words, we let the root stand alone and create as many domains (subtrees) as there are first-level nodes. Because a domain created this way could be very large, DNS allows domains to be divided further into smaller domains (subdomains). Each server can be responsible (authoritative) for either a large or small domain. In other words, we have a hierarchy of servers in the same way that we have a hierarchy of names (see Figure 10.39).

Figure 10.39 *Hierarchy of name servers*

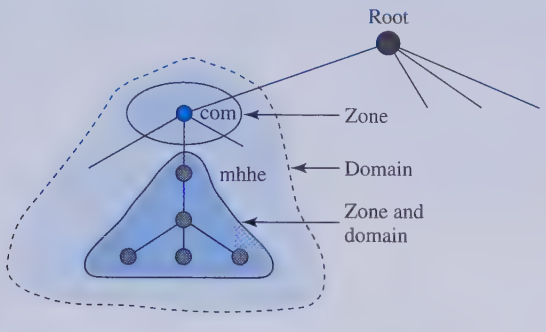


Zone

Because the complete domain name hierarchy cannot be stored on a single server, it is divided among many servers. What a server is responsible for or has authority over is called a zone. We can define a **zone** as a contiguous part of the entire tree. If a server accepts responsibility for a domain and does not divide the domain into smaller domains, the "domain" and the "zone" refer to the same thing. The server makes a database called a zone file and keeps all the information for every node under that domain. However, if a server divides its domain into subdomains and delegates part of its authority to other servers, domain and zone refer to different things. The information about the nodes in the subdomains is stored in the servers at the lower levels, with the original server

keeping some sort of reference to these lower-level servers. Of course, the original server does not free itself from responsibility totally. It still has a zone, but the detailed information is kept by the lower-level servers (see Figure 10.40).

Figure 10.40 *Zone*



Root Server

A **root server** is a server whose zone consists of the whole tree. A root server usually does not store any information about domains but delegates its authority to other servers, keeping references to those servers. There are several root servers, each covering the whole domain name space. The root servers are distributed all around the world.

Primary and Secondary Servers DNS defines two types of servers: primary and secondary. A *primary server* is a server that stores a file about the zone for which it is an authority. It is responsible for creating, maintaining, and updating the zone file. It stores the zone file on a local disk.

A *secondary server* is a server that transfers the complete information about a zone from another server (primary or secondary) and stores the file on its local disk. The secondary server neither creates nor updates the zone files. If updating is required, it must be done by the primary server, which sends the updated version to the secondary.

The primary and secondary servers are both authoritative for the zones they serve. The idea is not to put the secondary server at a lower level of authority but to create redundancy for the data so that if one server fails, the other can continue serving clients. Note also that a server can be a primary server for a specific zone and a secondary server for another zone. Therefore, when we refer to a server as a primary or secondary server, we should be careful to which zone we refer.

A primary server loads all information from the disk file; the secondary server loads all information from the primary server.

DNS in the Internet

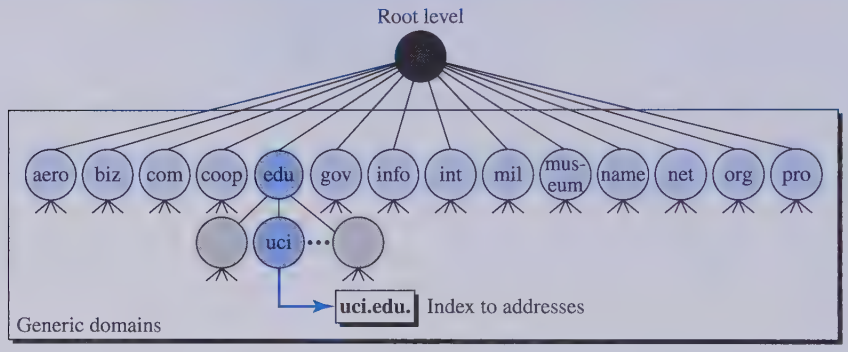
DNS is a protocol that can be used in different platforms. In the Internet, the domain name space (tree) was originally divided into three different sections: generic domains,

country domains, and the inverse domain. However, due to the rapid growth of the Internet, it became extremely difficult to keep track of the inverse domains, which could be used to find the name of a host when given the IP address. The inverse domains are now deprecated (see RFC 3425). We, therefore, concentrate on the first two.

Generic Domains

The **generic domains** define registered hosts according to their generic behavior. Each node in the tree defines a domain, which is an index to the domain name space database (see Figure 10.41).

Figure 10.41 *Generic domains*



Looking at the tree, we see that the first level in the generic domains section allows 14 possible labels. These labels describe the organization types as listed in Table 10.12.

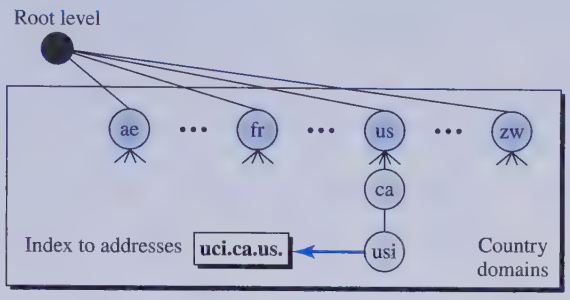
Table 10.12 *Generic domain labels*

Label	*Description*	*Label*	*Description*
aero	Airlines and aerospace	**int**	International organizations
biz	Businesses or firms	**mil**	Military groups
com	Commercial organizations	**museum**	Museums
coop	Cooperative organizations	**name**	Personal names (individuals)
edu	Educational institutions	**net**	Network support centers
gov	Government institutions	**org**	Nonprofit organizations
info	Information service providers	**pro**	Professional organizations

Country Domains

The **country domains** section uses two-character country abbreviations (e.g., us for United States). Second labels can be organizational, or they can be more specific, national designations. The United States, for example, uses state abbreviations as a subdivision of us (e.g., ca.us.). Figure 10.42 shows the country domains section. The address ***uci.ca.us.*** can be translated to University of California, Irvine in the state of California in the United States.

Figure 10.42 *Country domains*



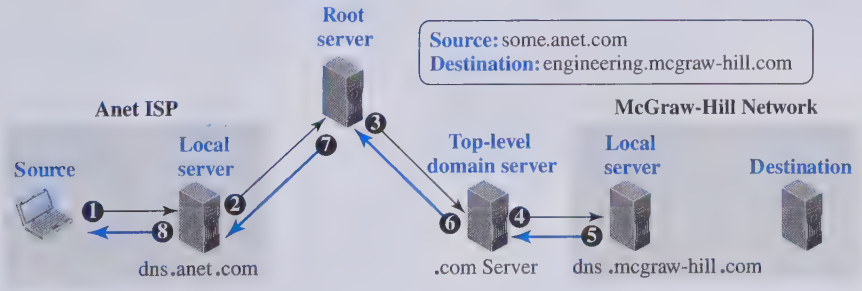
Resolution

Mapping a name to an address is called *name-address resolution.* DNS is designed as a client/server application. A host that needs to map an address to a name or a name to an address calls a DNS client called a **resolver**. The resolver accesses the closest DNS server with a mapping request. If the server has the information, it satisfies the resolver; otherwise, it either refers the resolver to other servers or asks other servers to provide the information. After the resolver receives the mapping, it interprets the response to see if it is a real resolution or an error, and finally delivers the result to the process that requested it. A resolution can be either recursive or iterative.

Recursive Resolution

Figure 10.43 shows a simple example of a recursive resolution. We assume that an application program running on a host named ***some.anet.com*** needs to find the IP address of another host named ***engineering.mcgraw-hill.com*** to which it wants to send a message. The source host is connected to the Anet ISP; the destination host is connected to the McGraw-Hill network.

Figure 10.43 *Recursive resolution*



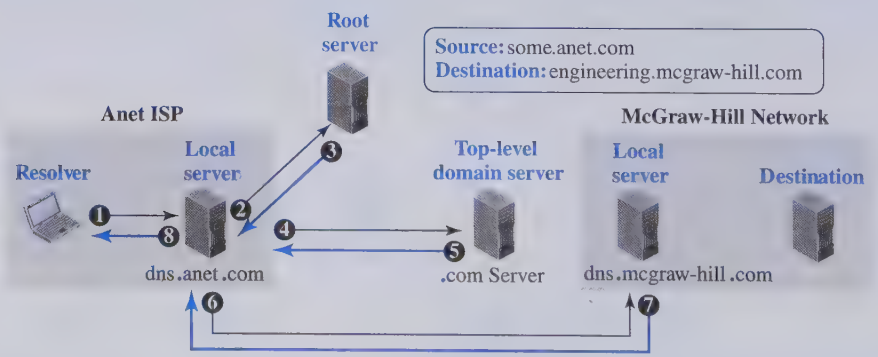
The application program on the source host calls the DNS resolver (client) to find the IP address of the destination host. The resolver, which does not know this address,

sends the query to the local DNS server (for example, *dns.anet.com*) running at the Anet ISP site (event 1). We assume that this server does not know the IP address of the destination host either. It sends the query to a root DNS server, whose IP address is supposed to be known to this local DNS server (event 2). Root servers do not normally keep the mapping between names and IP addresses, but a root server should at least know about one server at each top-level domain (in this case, a server responsible for the *com* domain). The query is sent to this top-level-domain server (event 3). We assume that this server does not know the name-address mapping of this specific destination, but it knows the IP address of the local DNS server in the McGraw-Hill company (for example, *dns.mcgraw-hill.com*). The query is sent to this server (event 4), which knows the IP address of the destination host. The IP address is now sent back to the top-level DNS server (event 5), then back to the root server (event 6), then back to the ISP DNS server, which may cache it for the future queries (event 7), and finally back to the source host (event 8).

Iterative Resolution

In **iterative resolution**, each server that does not know the mapping sends the IP address of the next server back to the one that requested it. Figure 10.44 shows the flow of information in an iterative resolution in the same scenario as the one depicted in Figure 10.43. Normally the iterative resolution takes place between two local servers; the original resolver gets the final answer from the local server. Note that the messages shown by events 2, 4, and 6 contain the same query. However, the mes-sage shown by event 3 contains the IP address of the top-level domain server, the message shown by event 5 contains the IP address of the McGraw-Hill local DNS server, and the message shown by event 7 contains the IP address of the destination. When the Anet local DNS server receives the IP address of the destination, it sends it to the resolver (event 8).

Figure 10.44 *Iterative resolution*



Caching

Each time a server receives a query for a name that is not in its domain, it needs to search its database for a server IP address. Reduction of this search time would increase

efficiency. DNS handles this with a mechanism called *caching*. When a server asks for a mapping from another server and receives the response, it stores this information in its cache memory before sending it to the client. If the same or another client asks for the same mapping, it can check its cache memory and resolve the problem. However, to inform the client that the response is coming from the cache memory and not from an authoritative source, the server marks the response as *unauthoritative*.

Caching speeds up resolution, but it can also be problematic. If a server caches a mapping for a long time, it may send an outdated mapping to the client. To counter this, two techniques are used. First, the authoritative server always adds information to the mapping called *time to live (TTL)*. It defines the time in seconds that the receiving server can cache the information. After that time, the mapping is invalid and any query must be sent again to the authoritative server. Second, DNS requires that each server keep a TTL counter for each mapping it caches. The cache memory must be searched periodically, and those mappings with an expired TTL must be purged.

Resource Records

The zone information associated with a server is implemented as a set of *resource records*. In other words, a name server stores a database of resource records. A *resource record* is a 5-tuple structure:

(Domain Name, Type, Class, TTL, Value)

The domain name field is what identifies the resource record. The value defines the information kept about the domain name. The TTL defines the number of seconds for which the information is valid. The class defines the type of network; we are only interested in the class IN (Internet). The type defines how the value should be interpreted. Table 10.13 lists the common types and how the value is interpreted for each type.

Table 10.13 *Types*

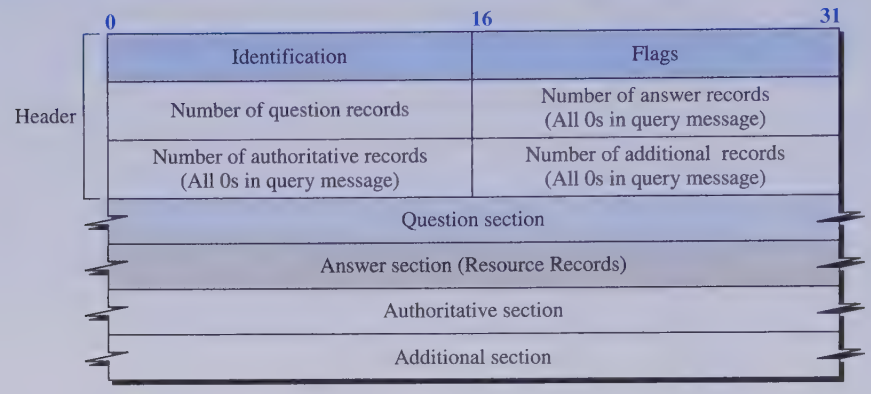
Type	*Interpretation of value*
A	A 32-bit IPv4 address (see Chapter 7)
NS	Identifies the authoritative servers for a zone
CNAME	Defines an alias for the official name of a host
SOA	Marks the beginning of a zone
MX	Redirects mail to a mail server
AAAA	An IPv6 address (see Chapter 7)

DNS Messages

To retrieve information about hosts, DNS uses two types of messages: *query* and *response*. Both types have the same format as shown in Figure 10.45 (on next page).

We briefly discuss the fields in a DNS message. The identification field is used by the client to match the response with the query. The flag field defines whether the message is a query or response. It also includes status of error. The next four fields in the header define the number of each record type in the message. The question section, which is included in the query and repeated in the response message, consists of one or

Figure 10.45 *DNS message*



Note:
The query message contains only the question section.
The response message includes the question section, the answer section, and possibly two other sections.

more question records. It is present in both query and response messages. The answer section consists of one or more resource records. It is present only in response messages. The authoritative section gives information (domain name) about one or more authoritative servers for the query. The additional information section provides additional information that may help the resolver.

Example 10.14

In UNIX and Windows, the *nslookup* utility can be used to retrieve address/name mapping. The following shows how we can retrieve an address when the domain name is given.

```
$nslookup www.forouzan.biz
Name:   www.forouzan.biz
Address: 198.170.240.179
```

Encapsulation

DNS can use either UDP or TCP. In both cases the well-known port used by the server is port 53. UDP is used when the size of the response message is less than 512 bytes because most UDP packages have a 512-byte packet size limit. If the size of the response message is more than 512 bytes, a TCP connection is used. In that case, one of two scenarios can occur:

- If the resolver has prior knowledge that the size of the response message is more than 512 bytes, it uses the TCP connection. For example, if a secondary name server (acting as a client) needs a zone transfer from a primary server, it uses the TCP connection because the size of the information being transferred usually exceeds 512 bytes.

- If the resolver does not know the size of the response message, it can use the UDP port. However, if the size of the response message is more than 512 bytes, the server truncates the message and turns on the TC bit. The resolver now opens a TCP connection and repeats the request to get a full response from the server.

Registrars

How are new domains added to DNS? This is done through a *registrar*, a commercial entity accredited by ICANN. A registrar first verifies that the requested domain name is unique and then enters it into the DNS database. A fee is charged. Today, there are many registrars; their names and addresses can be found at

http://www.intenic.net

To register, the organization needs to give the name of its server and the IP address of the server. For example, a new commercial organization named *wonderful* with a server named *ws* and IP address 200.200.200.5 needs to give the following information to one of the registrars:

Domain name: ws.wonderful.com **IP address:** 200.200.200.5

DDNS

When the DNS was designed, no one predicted that there would be so many address changes. In DNS, when there is a change, such as adding a new host, removing a host, or changing an IP address, the change must be made to the DNS master file. These types of changes involve a lot of manual updating. The size of today's Internet does not allow for this kind of manual operation.

The DNS master file must be updated dynamically. The **Dynamic Domain Name System (DDNS)**, therefore, was devised to respond to this need. In DDNS, when a binding between a name and an address is determined, the information is sent, usually by DHCP to a primary DNS server. The primary server updates the zone. The secondary servers are notified either actively or passively. In active notification, the primary server sends a message to the secondary servers about the change in the zone, whereas in passive notification, the secondary servers periodically check for any changes. In either case, after being notified about the change, the secondary server requests information about the entire zone (called the zone transfer).

To provide security and prevent unauthorized changes in the DNS records, DDNS can use an authentication mechanism.

Security of DNS

DNS is one of the most important systems in the Internet infrastructure; it provides crucial services to Internet users. Applications such as Web access or e-mail are heavily dependent on the proper operation of DNS. DNS can be attacked in several ways including:

1. The attacker may read the response of a DNS server to find the nature or names of sites the user mostly accesses. This type of information can be used to find the user's profile. To prevent this attack, DNS messages need to be confidential.

2. The attacker may intercept the response of a DNS server and change it or create a totally new bogus response to direct the user to the site or domain the attacker wishes the user to access. This type of attack can be prevented using message origin authentication and message integrity.
3. The attacker may flood the DNS server to overwhelm it or eventually crash it. This type of attack can be prevented using the provision against denial-of-service attack.

To protect DNS, the Internet Engineering Task Force (IETF) has devised a technology named *DNS Security (DNSSEC)* that provides message origin authentication and message integrity using a security service called *digital signature*. DNSSEC, however, does not provide confidentiality for the DNS messages. There is no specific protection against the denial-of-service attack in the specification of DNSSEC. However, the caching system protects the upper-level servers against this attack to some extent.

10.4 PEER-TO-PEER PARADIGM

We discussed the client/server paradigm early in this chapter. We also discussed some standard client/server applications. In this section, we discuss the peer-to-peer paradigm. The first instance of peer-to-peer (P2P) file sharing goes back to December 1987 when Wayne Bell created *WWIVnet*, the network component of WWIV (World War Four) bulletin board software. In July 1999, Ian Clarke designed *Freenet*, a decentralized, censorship-resistant distributed data store, aimed to provide freedom of speech through a peer-to-peer network with strong protection of anonymity.

Peer-to-peer gained popularity with Napster (1999–2001), an online music file sharing service created by Shawn Fanning. Although free copying and distributing of music files by the users led to a copyright violation lawsuit against Napster, and eventually closing of the service, it paved the way for peer-to-peer file-distribution models that came later. Gnutella had its first release in March 2000. It was followed by FastTrack (used by the Kazaa), BitTorrent, WinMX, and GNUnet in March, April, May, and November of 2001, respectively.

10.4.1 P2P Networks

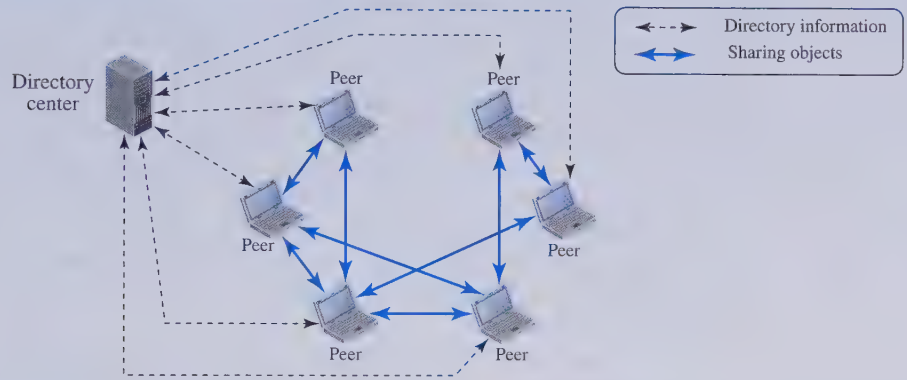
Internet users that are ready to share their resources become peers and form a network. When a peer in the network has a file (for example, an audio or video file) to share, it makes it available to the rest of the peers. An interested peer can connect itself to the computer where the file is stored and download it. After a peer downloads a file, it can make it available for other peers to download. As more peers join and download that file, more copies of the file become available to the group. Because lists of peers may grow and shrink, the question is how the paradigm keeps track of loyal peers and the location of the files. To answer this question, we first need to divide the P2P networks into two categories: centralized and decentralized.

Centralized Networks

In a centralized P2P network, the directory system—listing of the peers and what they offer—uses the client/server paradigm, but the storing and downloading of the files are

done using the P2P paradigm. For this reason, a centralized P2P network is sometimes referred to as a hybrid P2P network. Napster was an example of a centralized P2P. In this type of network, a peer first registers itself with a central server. The peer then provides its IP address and a list of files it has to share. To avoid system collapse, Napster used several servers for this purpose, but we show only one in Figure 10.46.

Figure 10.46 *Centralized network*



A peer, looking for a particular file, sends a query to a central server. The server searches its directory and responds with the IP addresses of nodes that have a copy of the file. The peer contacts one of the nodes and downloads the file. The directory is constantly updated as nodes join or leave the peer.

Centralized networks make the maintenance of the directory simple but have several drawbacks. Accessing the directory can generate huge traffic and slow down the system. The central servers are vulnerable to attack, and if all of them fail, the whole system goes down. The central component of the system was ultimately responsible for Napster's loss of the copyright lawsuit and its closure in July 2001. Roxio brought back the New Napster in 2003; Napster version 2 is now a legal, pay-for-music site.

Decentralized Network

A decentralized P2P network does not depend on a centralized directory system. In this model, peers arrange themselves into an *overlay network*, which is a logical network made on top of the physical network. Depending on how the nodes in the overlay network are linked, a decentralized P2P network is classified as either unstructured or structured.

Unstructured Networks

In an unstructured P2P network, the nodes are linked randomly. A search in an unstructured P2P is not very efficient because a query to find a file must be flooded through the network, which produces significant traffic, and still the query may not be resolved. Two examples of this type of network are Gnutella and Freenet. We discuss the Gnutella network as an example.

Gnutella The Gnutella network is an example of a P2P network that is decentralized but unstructured. It is unstructured in a sense that the directory is randomly distributed between nodes. When node A wants to access an object (such as a file), it contacts one of its neighbors. A neighbor, in this case, is any node whose address is known to node A. Node A sends a *query* message to the neighbor, node W. The query includes the identity of the object (for example, file name). If node W knows the address of node X, which has the object, it sends a *response* message, that includes the address of node X. Node A now can use the commands defined in a transfer protocol such as HTTP to get a copy of the object from node X. If node W does not know the address of node X, it *floods* the request from A to all its neighbors. Eventually one of the nodes in the network responds to the *query* message, and node A can get access to node X. It is worth mentioning here that although flooding in the Gnutella is somehow controlled to prevent huge traffic loads, one of the reasons that Gnutella cannot be scaled well is the flooding.

One of the questions that remains to be answered is whether, according to the process described, node A needs to know the address of at least one neighbor. This is done at the *bootstrap* time, when the node installs the Gnutella software for the first time. The software includes a list of nodes (peers) that node A can record as neighbors. Node A can later use the two messages, called *ping* and *pong*, to investigate whether or not a neighbor is still alive.

As mentioned before, one of the problems with the Gnutella network is the lack of scalability because of flooding. When the number of nodes increases, flooding does not respond very well. To make the query more efficient, the new version of Gnutella implemented a tiered system of *ultra nodes* and *leaves*. A node entering into the network is a leaf, not responsible for routing; nodes that are capable of routing are promoted to ultra nodes. This allows queries to propagate further and improves efficiency and scalability. Gnutella adopted a number of other techniques such as adding *Query Routing Protocol* (*QRP*) and *Dynamic Querying (DQ)* to reduce traffic overhead and make searches more efficient.

Structured Networks

A structured network uses a predefined set of rules to link nodes so that a query can be effectively and efficiently resolved. The most common technique used for this purpose is the *distributed hash table (DHT)*. DHT is used in many applications including Distributed Data Structure (DDS), Content Distributed Systems (CDS), Domain Name System (DNS), and P2P file sharing. One popular P2P file sharing protocol that uses the DHT is BitTorrent. We discuss DHT independently in Section 10.4.2 as a technique that can be used both in structured P2P networks and in other systems.

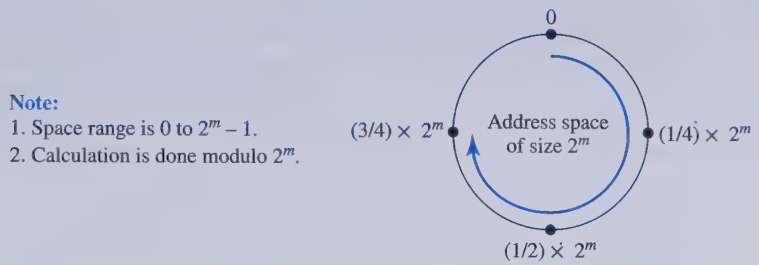
10.4.2 Distributed Hash Table (DHT)

A **distributed hash table** (**DHT**) distributes data (or references to data) among a set of nodes according to some predefined rules. Each peer in a DHT-based network becomes responsible for a range of data items. To avoid the flooding overhead that we discussed for unstructured P2P networks, DHT-based networks allow each peer to have partial knowledge about the whole network. This knowledge can be used to route the queries about the data items to the responsible nodes using effective and scalable procedures that we will discuss shortly.

Address Space

In a DHT-based network, each data item and the peer is mapped to a point in a large address of size 2^m. The address space is designed using modular arithmetic, which means that we can think of points in the address space as distributed evenly on a circle with 2^m points (0 to $2^m - 1$) using the clockwise direction as shown in Figure 10.47. Most of the DHT implementations use $m = 160$.

Figure 10.47 *Address space*



Hashing Peer Identifier

The first step in creating the DHT system is to place all peers on the address space ring. This is normally done by using a *hash* function that hashes the peer identifier, normally its IP address, to an m-bit integer, called a *node ID*.

$$\text{node ID} = \text{hash (Peer IP address)}$$

A hash function is a mathematical function that creates an output from an input. However, DHT uses some of the cryptographic hash functions such as Secure Hash Algorithm (SHA) that are collision resistant, which means that the probability of two inputs being mapped to the same output is very low.

Hashing Object Identifier

The name of the object (for example, a file) to be shared is also hashed to an m-bit integer in the same address space. The result in DHT parlance is called a *key*.

$$\text{key} = \text{hash (Object name)}$$

In the DHT, an object is normally related to the pair (key, value) in which the key is the hash of the object name and the value is the object or a reference to the object.

Storing the Object

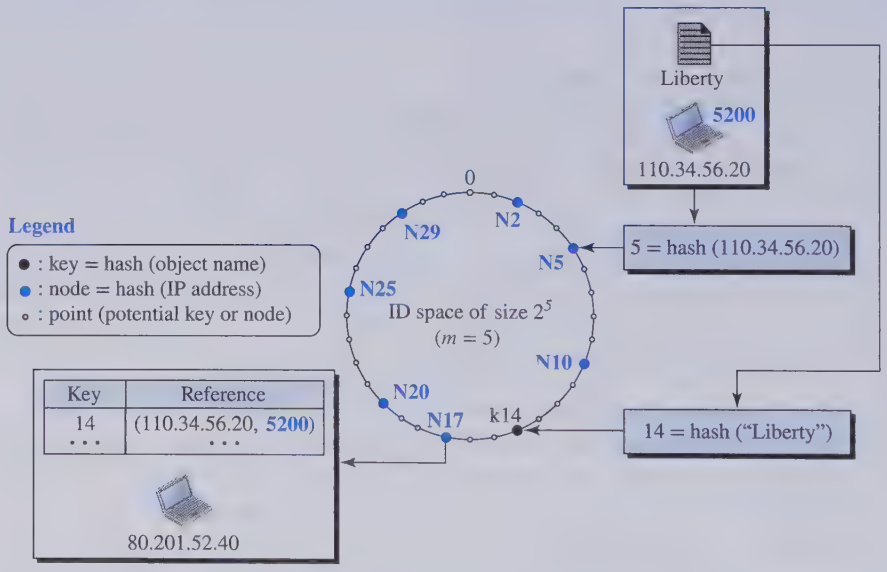
There are two strategies for storing the object: the direct method and the indirect method. In the direct method, the object is stored in the node whose ID is somehow *closest* to the key in the ring. The term *closest* is defined differently in each protocol. This involves the object most likely being transported from the computer that originally owned it. However, most DHT systems use the indirect method due to efficiency. The peer that owns the object keeps the object, but a reference to the object is created and stored in the node whose ID is closest to the key point. In other words, the physical object and the reference to the

object are stored in two different locations. In the direct strategy, we create a relationship between the node ID that stores the object and the key of the object; in the indirect strategy, we create a relationship between the reference (pointer) to the object and the node that stores that reference. In either case, the relationship is needed to find the object if the name of the object is given. In the rest of the section, we use the indirect method.

Example 10.15

Although the normal value of m is 160, for the purpose of demonstration, we use $m = 5$ to make our examples tractable. In Figure 10.48, we assume that several peers have already joined the group. The node N5 with IP address 110.34.56.20 has a file named Liberty that wants to share with its peers. The node makes a hash of the file name, "Liberty," to get the key = 14. Because the *closest* node to key 14 is node N17, N5 creates a reference to file name (key), its IP address, and the port number (and possibly some other information about the file) and sends this reference to be stored in node N17. In other words, the file is stored in N5, the key of the file is k14 (a point in the DHT ring), but the reference to the file is stored in node N17.

Figure 10.48 *Example 10.15*



Routing

The main function of DHT is to route a query to the node that is responsible for storing the reference to an object. Each DHT implementation uses a different strategy for routing, but all follow the idea that each node needs to have a partial knowledge about the ring to route a query to a node that is closest to the responsible node.

Arrival and Departure of Nodes

In a P2P network, each peer can be a desktop or a laptop computer, which can be turned on or off. When a computer peer launches the DHT software, it joins the network; when

the computer is turned off or the peer closes the software, it leaves the network. A DHT implementation needs to have a clear and efficient strategy to handle arrival or departure of the nodes and the effect of this on the rest of the peers. Most DHT implementations treat the failure of a node as a departure.

10.4.3 Chord

There are several protocols that implement DHT systems. In this section, we introduce three of these protocols: Chord, Pastry, and Kademlia. We chose to discuss the Chord protocol because of its simplicity and elegant approach to routing queries. Next we discuss the Pastry protocol because it uses a different approach than Chord and is very close in routing strategy to the Kademlia protocol, which is used in the most popular file-sharing network, BitTorrent.

Chord was published by Stoica *et al.* in 2001. We briefly discuss the main feature of this algorithm here.

Identifier Space

Data items and nodes in Chord are m-bit identifiers that create an identifier space of size 2^m points distributed in a circle in the clockwise direction. We refer to the identifier of a data item as k (for *key*) and the identifier of a peer as N (for *node*). Arithmetic in the space is done modulo 2^m, which means that the identifiers are wrapped from $2^m - 1$ back to 0. Although some implementations use a collision-resistant hash function like SHA-1 with $m = 160$, we use $m = 5$ in our discussion to make the discussion simpler. The closest peer with $N \geq k$ is called the successor of k and hosts the value (k, v), in which k is the key (hash of the data item) and v is the value (information about the peer server that has the object). In other words, a data item such as a file is stored in a peer that owns the data item, but the hash value of the data item, *key*, and the information about the peer, *value*, is stored as the pair (k, v) in the successor of k. This means that the peer that stores the data item and the peer that holds the pair (k, v) are not necessarily the same.

Finger Table

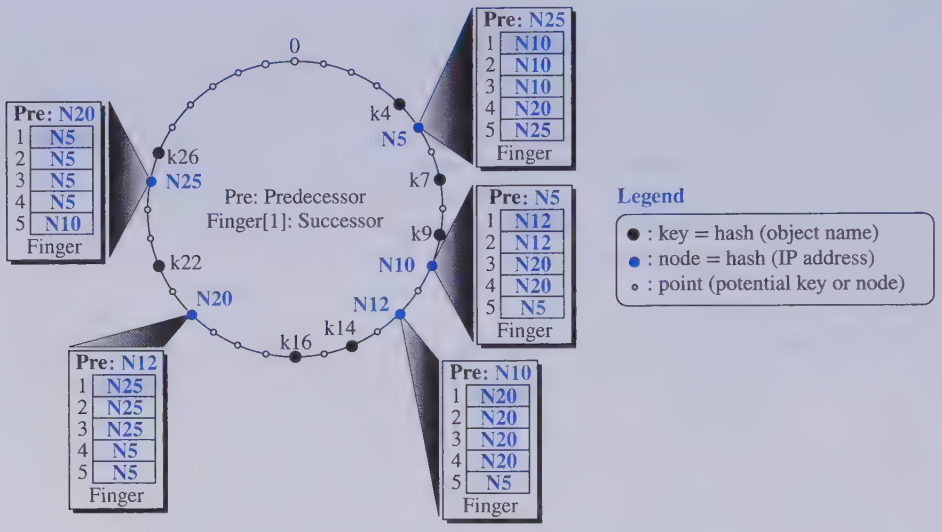
A node in the Chord algorithm should be able to resolve a query: given a key, the node should be able to find the node identifier responsible for that key or forward the query to another node. Forwarding, however, means that each node needs to have a routing table. Chord requires that each node knows about m successor nodes and one predecessor node. Each node creates a routing table, called a *finger table* by Chord, that looks like Table 10.14. Note that the target key at row i is $N + 2^{i-1}$.

Table 10.14 *Finger table*

i	*Target Key*	*Successor of Target Key*	*Information about Successor*
1	$N + 1$	Successor of $N + 1$	IP address and port of successor
2	$N + 2$	Successor of $N + 2$	IP address and port of successor
⋮	⋮	⋮	⋮
m	$N + 2^{m-1}$	Successor of $N + 2^{m-1}$	IP address and port of successor

Figure 10.49 shows only the successor column for a ring with few nodes and keys. Note that the first row ($i = 1$) actually gives the node successor. We have also added the predecessor node ID.

Figure 10.49 *An example of a ring in Chord*



Interface

For its operation, Chord needs a set of operations referred to as the Chord interface. In this section, we discuss some of these operations to give an idea of what is behind the Chord protocol.

Lookup

Probably the most used operation in Chord is the lookup. Chord is designed to let peers share available services between themselves. To find the object to be shared, a peer needs to know the node that is responsible for that object: the peer that stores a reference to that object. We discussed that, in Chord, a peer that is the successor of a set of keys in the ring is the responsible peer for those keys. Finding the responsible node is actually finding the successor of a key. Table 10.15 shows the code for the *lookup* operation.

The lookup function is written using the top-down approach. If the node is responsible for the key, it returns its own ID; otherwise, it calls the function find_successor. The find_successor calls the function find_predecessor. The last function calls the function find_closest_predecessor. The modularity approach allows us to use the three functions in other operations instead of redefining them.

Let us elaborate on the lookup function. If the node is not responsible for the key, the lookup function calls the find_successor function to find the successor of the ID that is passed to it as the parameter. The coding of the successor function can be very simple if we first find the predecessor of the key. The predecessor node can easily help us to

Table 10.15 *Lookup*

```
Lookup (key)
{
    if (node is responsible for the key)
        return (node's ID)
    else
        return find_succesor (key)
}
find_successor (id)
{
    x = find_ predecessor (id)
        return x.finger[1]
}
find_predecessor (id)
{
        x = N                                   // N is the current node
        while (id ∉ (x, x.finger[1]))
        {
        x = x.find_closest_predecessor (id)     // Let x find it
        }
        return x
}
find_closest_predecessor (id)
{
        for (i = m downto 1)
        {
            if (finger [i] ∈ (N, id))           //N is the current node
                return (finger [i])
        }
        return N                                //The node itself is closest predecessor
}
```

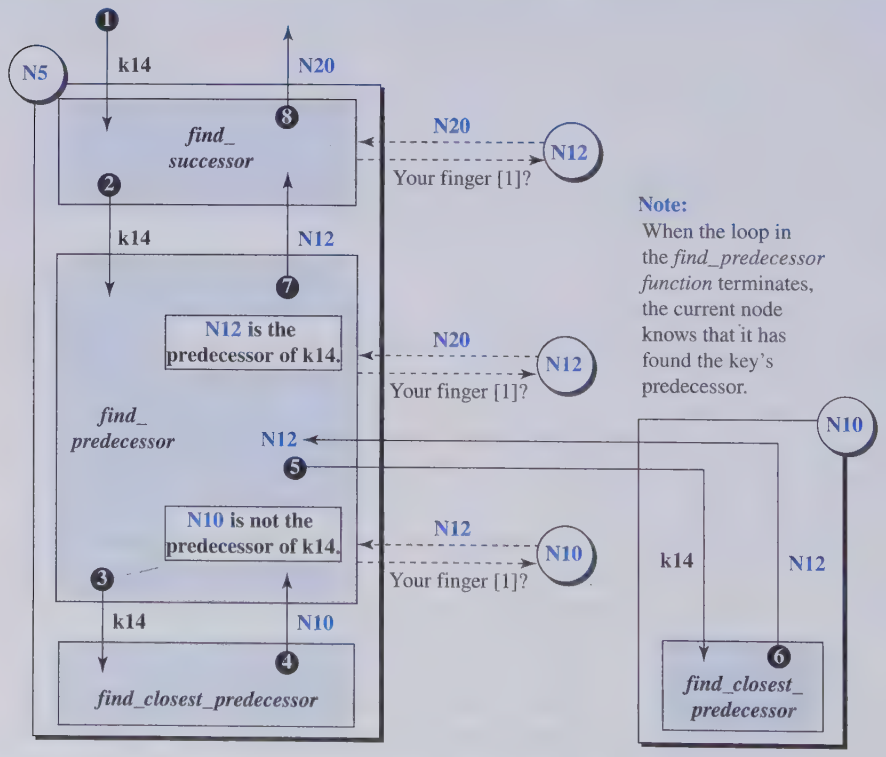
find the next node in the ring because the first finger of the predecessor node (finger [1]) gives us the ID of the successor node. Unfortunately, a node cannot normally find the predecessor of a key by itself; the key may be located far from the node. A node, as we discussed before, has a limited knowledge about the rest of the nodes; the finger table knows only about a maximum of m other nodes (there are some duplicates in the finger table). For this reason, the node needs the help of other nodes to find the predecessor of a key. This can be done using the find_closest_predecessor function as a *remote procedure call* (RPC). A remote procedure call means calling a procedure to be executed at a

remote node and returning the result to the calling node. We use the expression *x.procedure* in the algorithm in which x is the identity of the remote node and *procedure* is the procedure to be executed. The node uses this function to find another node that is closer to the predecessor node than itself. It then passes the duty of finding the predecessor node to the other node. In other words, if node A wants to find node X, it finds node B (closest predecessor) and passes the task to B. Now node B gets control and tries to find X, or passes the task to another node, C. The task is forwarded from node to node until the node that has the knowledge of the predecessor node finds it.

Example 10.16

Assume node N5 in Figure 10.49 needs to find the responsible node for key k14. Figure 10.50 shows the sequence of eight events. After event 4, in which the find_closest_predecessor function returns N10, the find_predecessor function asks N10 to return its finger[1], which is N12. At this moment, N5 finds out that N10 is not the predecessor of k14. Node N5 then asks N10 to find the closest predecessor of k14, which is returned as N12 (events 5 and 6). Node N5 now asks for the finger[1] of node N12, which is returned as N20. Node N5 now checks and sees that N12 is in fact the predecessor of k14. This information is passed to the find_successor function (event 7). N5 now asks for the finger [1] of node N12, which is returned as N20. The search is terminated, and N20 is the successor of k14.

Figure 10.50 *Example 10.16*



Stabilize

Before we discuss how nodes join and leave a ring, we need to emphasize that any change in the ring (such as the joining and arriving of a node or a group of nodes) may destabilize the ring. One of the operations defined in Chord is called *stabilize*. Each node in the ring periodically uses this operation to validate its information about its successor and let the successor validate its information about its predecessor. Node N uses the value of finger[1], S, to ask node S to return its predecessor, P. If the return value, P, from this query is between N and S, this means that there is a node with ID equal to P that lies between N and S. Then node N makes P its successor and notifies P to make node N its predecessor. Table 10.16 shows the stabilize operation.

Table 10.16 *Stabilize*

```
Stabilize ()
{
        P = finger[1].Pre                          //Ask the successor to return its predecessor
        if (P ∈ (N, finger[1]))     finger[1] = P  // P is the possible successor of N
        finger[1].notify (N)                       // Notify P to change its predecessor
}
Notify (x)
{
        if (Pre = null or x ∈ (Pre, N))    Pre = x
}
```

Fix_Finger

Destabilization may change the finger table of up to m nodes. Another operation defined in Chord is called *fix_finger*. Each node in the ring must periodically call this function to maintain its finger table update. To avoid traffic on the system, each node must only update one of its fingers in each call. This finger is chosen randomly. Table 10.17 shows the code for this operation.

Table 10.17 *Fix_Finger*

```
Fix_Finger ()
{
        Generate (i ∈ (1, m])                  //Randomly generate i such as 1 < i ⩽ m
        finger[i] = find_successor (N + 2^(i-1))   // Find value of finger[i]
}
```

Join

When a peer joins the ring, it uses the *join* operation and known ID of another peer to find its successor and set its predecessor to null. It immediately calls the stabilize function to validate its successor. The node then asks the successor to call the *move-key* function that transfers the keys that the new peer is responsible for. Table 10.18 shows the code for this operation.

Table 10.18 *Join*

```
Join (x)
{
        Initialize (x)
        finger[1].Move_Keys (N)
}
Initialize (x)
{
        Pre = null
        if (x = null) finger[1] = N
        else finger[1] = x. Find_Successor (N)
}
Move_Keys (x)
{
        for (each key k)
        {
                if (x ∈ [k, N)) move (k to node x)        // N is the current node
        }
}
```

It is obvious that after this operation, the finger table of the joined node is empty and the finger table of up to m predecessors is out of date. The stabilize and the fix-finger operations that run periodically after this event will gradually stabilize the system.

Example 10.17

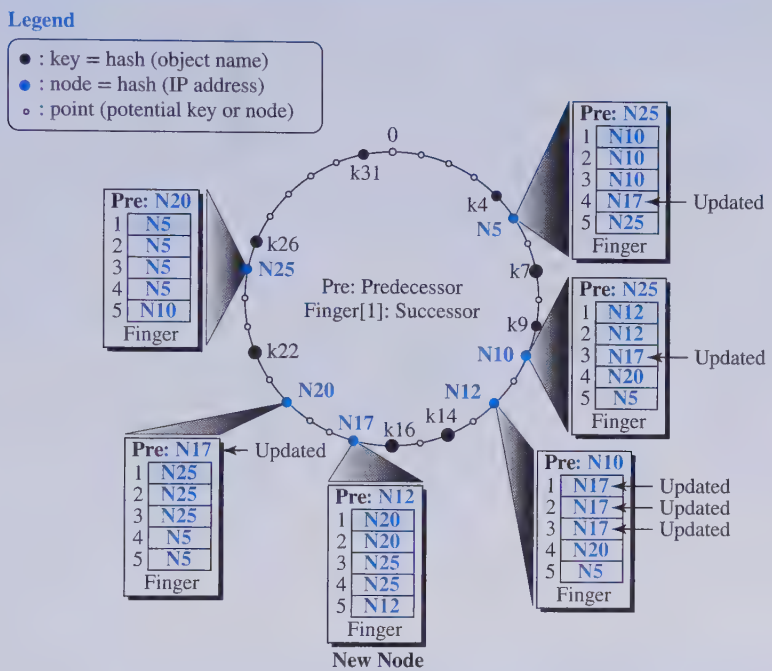
We assume that node N17 joins the ring in Figure 10.49 with the help of N5. Figure 10.51 shows the ring after the ring has been stabilized. The following shows the process:

1. N17 uses the Initialize (5) algorithm to set its predecessor to null and its successor (finger[1]) to N20.
2. N17 then asks N20 to send k14 and k16 to N17 because N17 is now responsible for these keys.
3. In the next time-out, N17 uses stabilize operation to validate its own successor (which is N20) and asks N20 to change its predecessor to N17 (using the notify function).
4. When N12 uses stabilize, the predecessor of N17 is updated to N12.
5. Finally, when some nodes use the fix-finger function, the finger table of nodes N17, N10, N5, and N12 is changed.

Leave or Fail

If a peer leaves the rings or the peer (not the ring) fails, the operation of the ring will be disrupted unless the ring stabilizes itself. Each node exchanges ping and pong messages with neighbors to find out if they are alive. When a node does not receive a pong message in response to its ping message, the node knows that the neighbor is dead.

Figure 10.51 *Example 10.17*



Although the use of *stabilize* and *fix-finger* operations may restore the ring after a leave or failure, the node that detects the problem can immediately launch these operations without waiting for the time-out. One important issue is that the stabilize and fix-finger operations may not work properly if several nodes leave or fail at the same time. For this reason, Chord requires that each node keep track of r successors (the value of r depends on the implementation). The node can always go to the next successor if the previous ones are not available.

Another issue in this case is that the data managed by the node that left or failed is no longer available. Chord stipulates that only one node should be responsible for a set of data and references, but Chord also requires that data and references be duplicated on other nodes in this case.

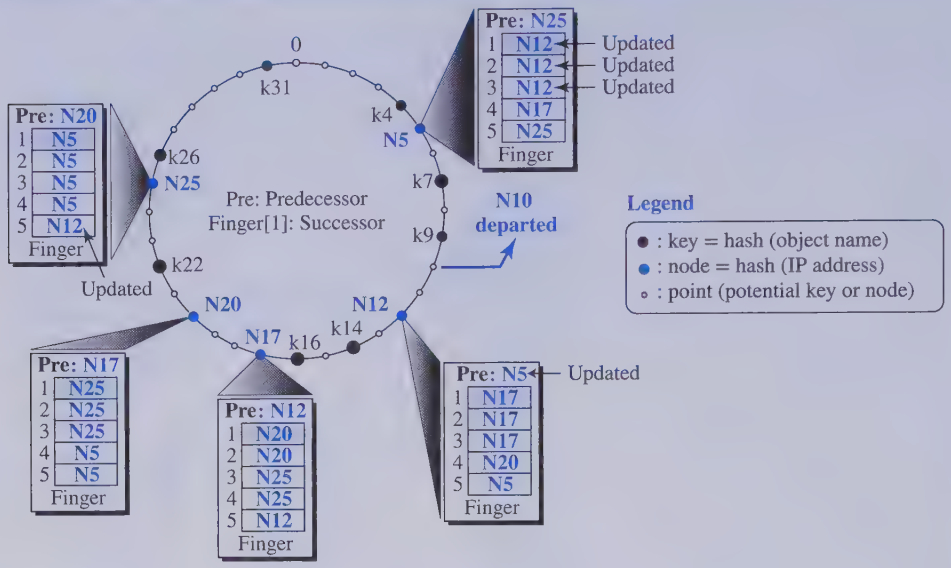
Example 10.18

We assume that a node, N10, leaves the ring in Figure 10.51. Figure 10.52 shows the ring after it has been stabilized.

The following shows the process:

1. Node N5 finds out about N10's departure when it does not receive a pong message to its ping message. Node N5 changes its successor (finger[1]) to N12 (the second in the list of successors).

Figure 10.52 *Example 10.18*



2. Node N5 immediately launches the stabilize function and asks N12 to change its predecessor to N5.
3. Hopefully, k7 and k9, which were under the responsibility of N10, have been duplicated in N12 before the departure of N10.
4. After a few calls of fix-finger, nodes N5 and N25 update their finger tables as shown in Figure 10.52.

Applications

Chord is used in several applications including Collaborative File System (CFS), Con-Chord, and Distributive Domain Name System (DDNS).

10.4.4 Pastry

Another popular protocol in the P2P paradigm is Pastry, designed by Rowstron and Druschel. Pastry uses DHT, as mentioned before, but there are some fundamental differences between Pastry and Chord in the identifier space and routing process, which we describe next.

Identifier Space

In Pastry, like Chord, nodes and data items are m-bit identifiers that create an identifier space of 2^m points distributed uniformly on a circle in the clockwise direction. The common value for m is 128. The protocol uses the SHA-1 hashing algorithm with $m = 128$. However, in this protocol, an identifier is seen as an n-digit string in base 2^b in which b is normally 4 and $n = (m/b)$. In other words, an identifier is a 32-digit number in

base 16 (hexadecimal). In this identifier space, a key is stored in the node whose identifier is numerically closest to the key. This strategy is definitively different from the one used by Chord. In Chord, a key is stored in its successor node; in Pastry, a key may be stored in its successor or predecessor node, the one which is numerically closest to the key.

Routing

A node in Pastry should be able to resolve a query; given a key, the node should be able to find the node identifier responsible for that key or forward the query to another node. Each node in Pastry uses two entities to do so: a *routing table* and a *leaf set*.

Routing Table

Pastry requires that each node keep a routing table with n rows and (2^b) columns. In general, when $m = 128$ and $b = 4$, we have 32 (128/4) rows and 16 columns ($2^{128} = 16^{32}$). In other words, we have a row for each digit in the identifier and a column for each hexadecimal value (0 to F). Table 10.19 shows the outline of the routing table for the general case. In the table for node N, the cell at row i and column j, Table $[i, j]$, gives the identifier of a node (if it exists) that shares the i leftmost digits with the identifier for N, and its $(i + 1)$ th digit has a value of j. The first row, row 0, shows the list of live nodes whose identifiers have no common prefix with N. Row 1 shows a list of live nodes whose identifiers share the leftmost digit with the identifier of node N. Similarly row 31 shows the list of all live nodes that share the leftmost 31 digits with node N; only the last digit is different.

Table 10.19 *Routing table for a node in Pastry*

Common prefix length	*0*	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*A*	*B*	*C*	*D*	*E*	*F*
0																
1																
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
31																

For example, if $N = (574A234B12E374A2001B23451EEE4BCD)_{16}$, then the value of Table [2, D] can be the identifier of a node such as (**57D...**). Note that the leftmost two digits are 57, which are common with the first two digits of N, but the next digit is D, the value corresponding to the Dth column. If there are more nodes with the prefix 57D, the closest one, according to the *proximity metric*, is chosen, and its identifier is inserted in this cell. The proximity metric is a measurement of closeness determined by the application that uses the network. It can be based on the number of hops between the two nodes, the round-trip time between the two nodes, or other metrics.

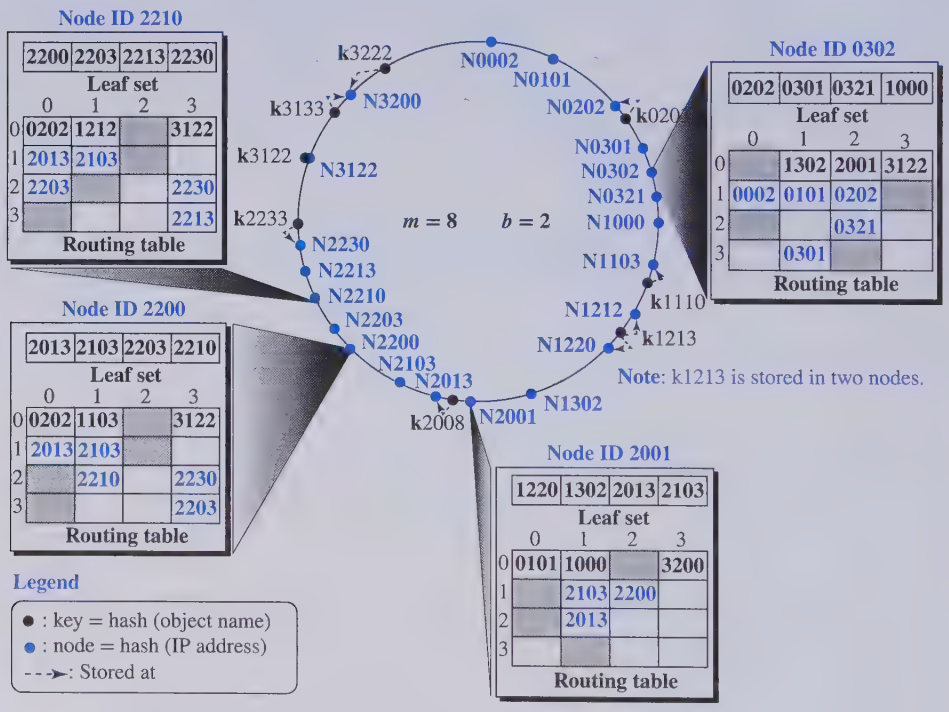
Leaf Set

Another entity used in routing is a set of 2^b identifiers (the size of a row in the routing table) called the *leaf set*. Half of the set is a list of identifiers that are numerically smaller than the identifier of the current node; the other half is a list of identifiers that are numerically larger than the identifier of the current node. In other words, the leaf set gives the identifier of 2^{b-1} live nodes before the current node in the ring and the list of 2^{b-1}nodes after the current node in the ring.

Example 10.19

Let us assume that $m = 8$ bits and b = 2. This means that we have up to $2^m = 256$ identifiers, and each identifier has $m/b = 4$ digits in base $2^b = 4$. Figure 10.53 shows the situation in which there are some live nodes and some keys mapped to these nodes. The key k1213 is stored in two nodes because it is equidistant from them. This provides some redundancy that can be used if one of the nodes fails. Figure 10.53 also shows the routing tables and leaf sets for four selected nodes that are used in the examples described later. In the routing table for node N0302, for example, we have chosen the node 1302 to be inserted in Table [0,1] because we assumed that this node is closest to N0302 according to the proximity metric. We used the same strategy for other entries. Note that one cell in each row in each table is shaded because it corresponds to the digit of the node identifier; no node identifier can be inserted there. Some cells are also empty because there are no live nodes in the network at this moment to satisfy the requirement; when some new nodes join the network, they can be inserted in these cells.

Figure 10.53 *An example of a Pastry ring*



Lookup

As we discussed in the section on Chord, one of the operations used in Pastry is lookup: Given a key, we need to find the node that stores the information about the key or the key itself. Table 10.20 gives the lookup operation in pseudocode. In this algorithm, *N* is the identifier of the local node, the node that receives a message and needs to find the node that stores the key in the message.

Table 10.20 *Lookup*

```
Lookup (key)
{
        if (key is in the range of N's leaf set)
                forward the message to the closest node in the leaf set
        else
                route (key, Table)
}
route (key, Table)
{
        p = length of shared prefix between key and N
        v = value of the digit at position p of the key          // Position starts from 0
        if (Table [p, v] exists)
                forward the message to the node in Table [p, v]
        else
        forward the message to a node sharing a prefix as long as the current node, but
        numerically closer to the key.
}
```

Example 10.20

In Figure 10.53, we assume that node N2210 receives a query to find the node responsible for key 2008. Because this node is not responsible for this key, it first checks its leaf set. The key 2008 is not in the range of the leaf set, so the node needs to use its routing table. Because the length of the common prefix is 1, $p = 1$. The value of the digit at position 1 in the key is $v = 0$. The node checks the identifier in Table [1, 0], which is 2013. The query is forwarded to node 2013, which is actually responsible for the key. This node sends its information to the requesting node.

Example 10.21

In Figure 10.53, we assume that node N0302 receives a query to find the node responsible for key 0203. This node is not responsible for this key, but the key is in the range of its leaf set. The closest node in this set is node N0202. The query is sent to this node and is actually responsible for this node. Node N0202 sends its information to the requesting node.

Join

The process of joining the ring in Pastry is simpler and faster than in Chord. The new node, X, should know at least one node N0, which should be close to X (based on the proximity metric); this can be done by running an algorithm called *Nearby Node Discovery*. Node X sends a *join message* to N0. In our discussion, we assume that N0's identifier has no common prefix with X's identifier. The following steps show how node X makes its routing table and leaf set:

1. Node N0 sends the contents of its row 0 to node X. Because the two nodes have no common prefix, node X uses the appropriate parts of this information to build its

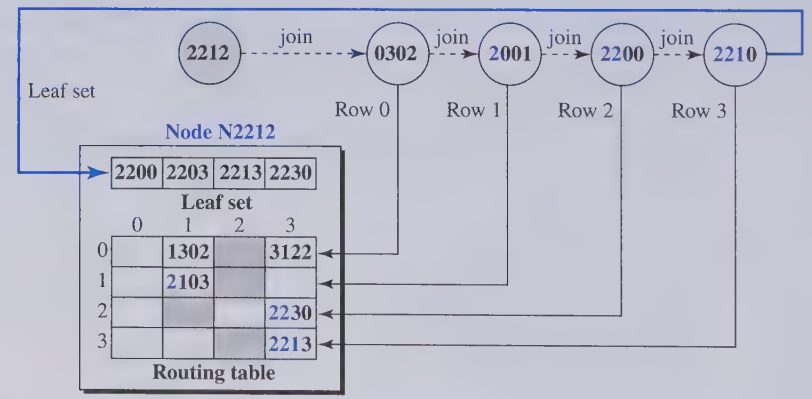
row 0. Node N0 then handles the joining message as a lookup message, assuming that the X identifier is a key. It forwards the join message to a node, N1, whose identifier is closest to X.

2. Node N1 sends the contents of its row 1 to node X. Because the two nodes have one common prefix, node X uses the appropriate parts of this information to build its row 1. Node N1 then handles the joining message as a lookup message, assuming that the X identifier is a key. It forwards the join message to a node, N2, whose identifier is closest to X.
3. The process continues until the routing table of node X is complete.
4. The last node in the process, which has the longest common prefix with X, also sends its leaf set to node X, which becomes the leaf set of X.
5. Node X then exchanges information with nodes in its routing table and leaf set to improve its own routing information and allow those nodes to updates theirs.

Example 10.22

Figure 10.54 shows how a new node X with node identifier N2212 uses the information in four nodes in Figure 10.53 to create its initial routing table and leaf set for joining the ring. Note that the contents of these two tables will become closer to what they should be in the updating process. In this example, we assume that node 0302 is a nearby node to node 2212 based on the proximity metric.

Figure 10.54 *Example 10.22*



Leave or Fail

Each Pastry node periodically tests the liveliness of the nodes in its leaf set and routing table by exchanging probe messages. If a local node finds that a node in its leaf set is not responding to the probe message, it assumes that the node has failed or departed. To replace it in its leaf set, the local node contacts the live node in its leaf set with the largest identifier and repairs its leaf set with the information in the leaf set of that node. Because there is an overlap in the leaf set of close-by nodes, this process is successful.

If a local node finds that a node in its routing table, Table $[i, j]$, is not responsive to the probe message, it sends a message to a live node in the same row and requests the identifier in Table $[i, j]$ of that node. This identifier replaces the failed or departed node.

Application

Pastry is used in some applications including PAST, a distributed file system, and SCRIBE, a decentralized publish/subscribe system.

10.4.5 Kademlia

Another DHT peer-to-peer network is **Kademlia**, designed by Maymounkov and Mazières. Like Pastry, Kademlia routes messages based on the distance between nodes, but the interpretation of the distance metric in Kademlia is different from the one in Pastry, as we will now describe. In this network, the distance between the two identifiers (nodes or keys) is measured as the bitwise exclusive-or (XOR), between them. In other words, if x and y are two identifiers, we have

$$\text{distance}\ (x, y) = x \oplus y$$

The XOR metric has four properties we expect when we use geometric distances between two points:

$x \oplus x = 0$	The distance between a node and itself is zero.
$x \oplus y > 0$ if $x \neq y$	The distance between any two distinct nodes is greater than zero.
$x \oplus y = y \oplus x$	The distance between x and y is the same as between y and x.
$x \oplus z \leq x \oplus y + y \oplus z$	The triangular relationship is satisfied.

Identifier Space

In Kademlia, nodes and data items are m-bit identifiers that create an identifier space of 2^m points distributed on the leaves of a binary tree. The protocol uses the SHA-1 hashing algorithm with $m = 160$.

Example 10.23

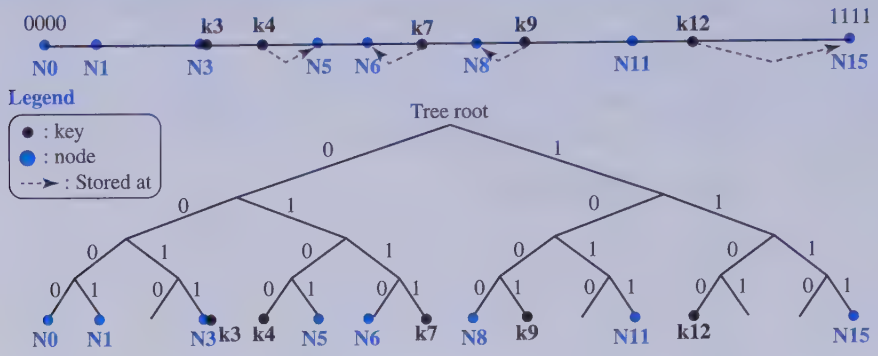
For simplicity, let us assume that $m = 4$. In this space, we can have 16 identifiers distributed on the leaves of a binary tree. Figure 10.55 shows the case with only eight live nodes and five keys.

As Figure 10.55 shows, the key k3 is stored in N3 because $3 \oplus 3 = 0$. Although the key k7 looks numerically equidistant from N6 and N8, it is stored only in N6 because $6 \oplus 7 = 1$ but $6 \oplus 8 = 14$. Another interesting point is that key k12 is numerically closer to N11, but it is stored in N15 because $11 \oplus 12 = 7$, but $15 \oplus 12 = 3$.

Routing Table

Kademlia keeps only one routing table for each node; there is no leaf set. Each node in the network divides the binary tree into m subtrees that do not include the node itself. Subtree i includes nodes that share i leftmost bit (common prefix) with the corresponding node. The routing table is made up of m rows but only one column. In our discussion, we assume that each row holds the identifier of one of the nodes in the corresponding subtree, but later we show that Kademlia allows up to k nodes in each row. The idea is the same as that used by

Figure 10.55 *Example 10.23*



Pastry, but the length of the common prefix is based on the number of bits instead of the number of digits in base 2^b. Table 10.21 shows the routing table.

Table 10.21 *Routing table for a node in Kademlia*

Common prefix length	*Identifiers*
0	Closest node(s) in subtree with common prefix of length 0
1	Closest node(s) in subtree with common prefix of length 1
⋮	⋮
$m - 1$	Closest node(s) in subtree with common prefix of length $m - 1$

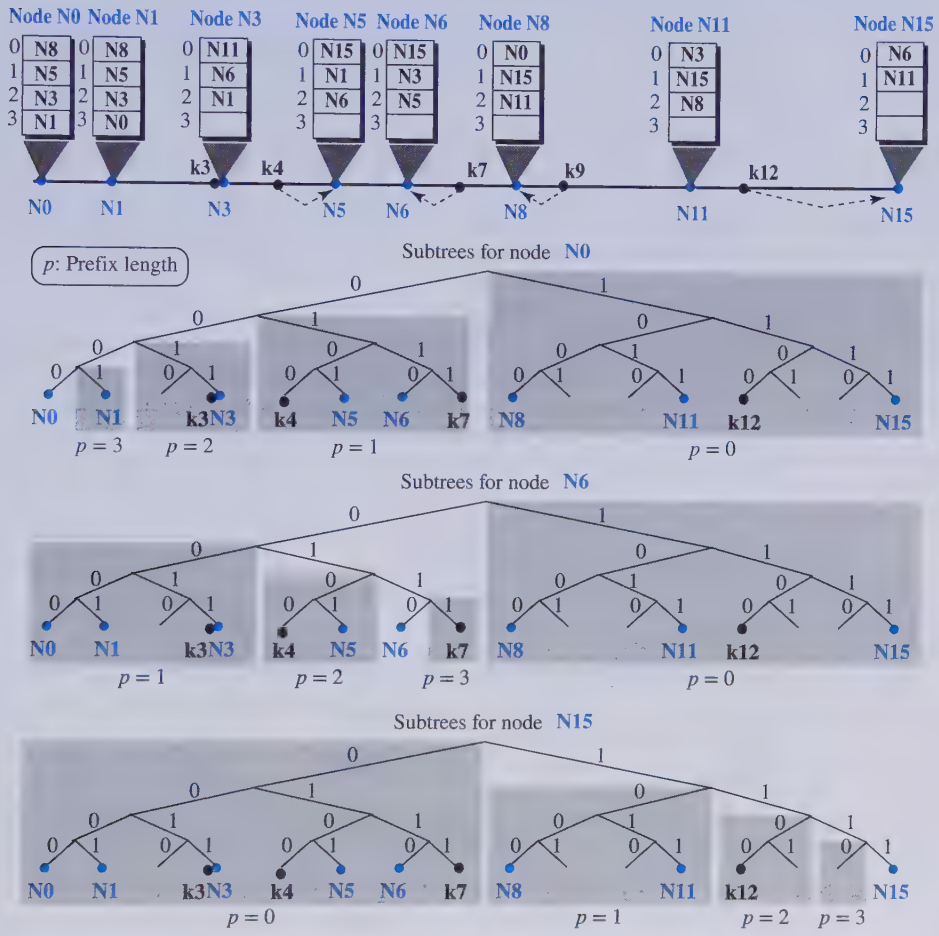
Example 10.24

Let us find the routing table for Example 10.23. To make the example simple, we assume that each row uses only one identifier. Because $m = 4$, each node has four subtrees corresponding to four rows in the routing table. The identifier in each row represents the node that is closest to the current node in the corresponding subtree. Figure 10.56 shows all routing tables, but only three of the subtrees. We have chosen these three, out of eight, to make the figure smaller.

Let us explain how we made the routing table, for example, for node 6, using the corresponding subtrees. The explanations for other nodes are similar.

- **a.** In row 0, we need to insert the identifier of the closest node in the subtree with common prefix length $p = 0$. There are three nodes in this subtree (N8, N11, and N15); however, N15 is the closest to N6 because N6 $\oplus$ N8 = 14, N6 $\oplus$ N11 = 13, and N6 $\oplus$ N15 = 9. N15 is inserted in row 0.
- **b.** In row 1, we need to insert the identifier of the closest node in the subtree with common prefix length $p = 1$. There are three nodes in this subtree (N0, N1, and N3); however, N3 is the closest to N6 because N6 $\oplus$ N0 = 6, N6 $\oplus$ N1 = 7, and N6 $\oplus$ N3 = 5. N3 is inserted in row 1.
- **c.** In row 2, we need to insert the identifier of the closest node in the subtree with common prefix length $p = 2$. There is only one node (N5) in this subtree, which is inserted there.
- **d.** In row 3, we need to insert the identifier of the closest node in the subtree with common prefix length $p = 3$. There is no node in this subtree, so the row is empty.

Figure 10.56 *Example 10.24*



Example 10.25

In Figure 10.56, we assume node N0 $(0000)_2$ receives a lookup message to find the node responsible for k12 $(1100)_2$. The length of the common prefix between the two identifiers is 0. Node N0 sends the message to the node in row 0 of its routing table, node N8. Now node N8 $(1000)_2$ needs to look for the node closest to k12 $(1100)_2$. The length of the common prefix between the two identifiers is 1. Node N8 sends the message to the node in row 1 of its routing table, node N15, which is responsible for k12. The routing process is terminated. The route is N0 → N8 → N15. It is interesting to note that node N15, $(1111)_2$, and k12, $(1100)_2$, have a common prefix of length 2, but row 2 of N15 is empty, which means that N15 itself is responsible for k12.

Example 10.26

In Figure 10.56, we assume node N5 $(0101)_2$ receives a lookup message to find the node responsible for k7 $(0111)_2$. The length of the common prefix between the two identifiers is 2. Node N5

sends the message to the node in row 2 of its routing table, node N6, which is responsible for k7. The routing process is terminated. The route is N5 → N6.

Example 10.27

In Figure 10.56, we assume node N11 $(1011)_2$ receives a lookup message to find the node responsible for k4 $(0100)_2$. The length of the common prefix between the two identifiers is 0. Node N11 sends the message to the node in row 0 of its routing table, node N3. Now node N3 $(0011)_2$ needs to look for the node closest to k4 $(0100)_2$. The length of the common prefix between the two identifiers is 1. Node N3 sends the message to the node in row 1 of its routing table, node N6. Now node N6 $(0110)_2$ needs to look for the node closest to k4 $(0100)_2$. The length of the common prefix between the two identifiers is 2. Node N6 sends the message to the node in row 2 of its routing table, node N5, which is responsible for k4. The routing process is terminated. The route is N11 → N3 → N6 → N5.

K-Buckets

In our previous discussion, we assumed that each row in the routing table lists only one node in the corresponding subtree. For more efficiency, Kademlia requires that each row keeps at least up to k nodes from the corresponding subtree. The value of k is system independent, but for an actual network it is recommended that it be around 20. For this reason, each row in the routing table is referred to as a *k-bucket*. Having more than one node in each row allows the node to use an alternative node when a node leaves the network or fails. Kademlia keeps those nodes in a bucket that has been connected in the network for a long time. It has been proven that the nodes that remain connected for a long time will probably remain connected for a longer time.

Parallel Query

Because there are multiple nodes in a k-bucket, Kademlia allows sending a parallel queries to a nodes at the top of the k-bucket. This reduces the delay if a node fails and cannot answer the query.

Concurrent Updating

Another interesting feature in Kademlia is concurrent updating. Whenever a node receives a query or a response, it updates its k-bucket. If multiple queries to a node receive no response, the node that sent the query can remove the destination node from the corresponding k-bucket.

Join

As in Pastry, a node that needs to join the network needs to know at least one other node. The joining node sends its identifier to the node as though it is a key to be found. The response it receives allows the new node to create its k-buckets.

Leave or Fail

When a node leaves the network or fails, other nodes update their k-buckets using the concurrent process described before.

10.4.6 A Popular P2P Network: BitTorrent

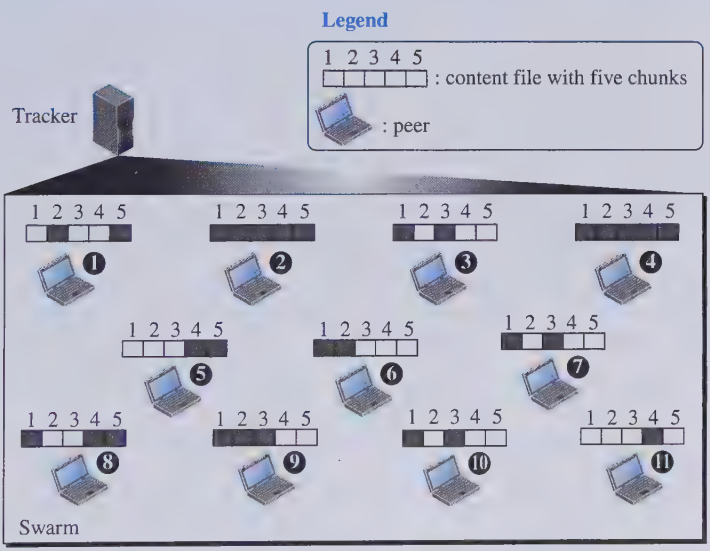
BitTorrent is a P2P protocol, designed by Bram Cohen, for sharing a large file among a set of peers. However, the term *sharing* in this context is different from other file-sharing protocols. Instead of one peer allowing another peer to download the whole file, a

group of peers takes part in the process to give all peers in the group a copy of the file. File sharing is done in a collaborating process called a *torrent*. Each peer participating in a torrent downloads chunks of the large file from another peer that has it and uploads chunks of that file to other peers that do not have it, a kind of *tit for tat*, a trading game played by kids. The set of all peers that takes part in a torrent is referred to as a *swarm*. A peer in a swarm that has the complete content file is called a *seed*; a peer that has only part of the file and wants to download the rest is called a *leech*. In other words, a swarm is a combination of seeds and leeches. BitTorrent has gone through several versions and implementations. We first describe the original one, which uses a central node called a *tracker*. We then show how some new versions eliminate the tracker by using DHT.

BitTorrent with a Tracker

In the original BitTorrent, there is another entity in a torrent, called the tracker, which, as the name implies, tracks the operation of the swarm. Figure 10.57 shows an example of a torrent with seeds, leeches, and the *tracker*.

Figure 10.57 *Example of a torrent*



Note: Peers 2 and 4 are seeds; others are leeches.

In Figure 10.57, the file to be shared, the content file, is divided into five pieces (chunks). Peers 2 and 4 already have all the pieces; other peers have some pieces. The pieces that each peer has are shaded. Uploading and downloading of the pieces will continue. Some peers may leave the torrent; some new peers may join the torrent.

Now assume a new peer wants to download the same content file. The new peer accesses the BitTorrent server with the name of the content file. It receives a metafile, named the torrent file, that contains the information about the pieces in the content

file and the address of the tracker that handles that specific torrent. The new peer now accesses the tracker and receives the addresses of some peers in the torrent, normally called *neighbors*. The new peer is now part of the torrent and can download and upload pieces of the content file. When it has all the pieces, it may leave the torrent or remain in the torrent to help other peers, including the new peers that have joined after it, to get all pieces of the content file. Nothing can prevent a peer from leaving the torrent before it has all the pieces and joining later or not joining again.

Although the process of joining, sharing, and leaving a torrent looks simple, the BitTorrent protocol applies a set of policies to provide fairness, to encourage the peers to exchange pieces with other peers, to prevent overloading a peer with requests from other peers, and to allow a peer to find peers that provide better service.

To avoid overloading a peer and to achieve fairness, each peer needs to limit its concurrent connection to a number of neighbors; the typical value is four. A peer flags a neighbor as unchoked or choked. It also flags the neighbour as interested or uninterested.

In other words, a peer divides its lists of neighbors into two distinct groups: *unchoked* and *choked*. It also divides them into *interested* and *uninterested* groups. The unchoked group is the list of peers that the current peer has concurrently connected to; it continuously uploads and downloads pieces from this group. The choked group is the list of neighbors that the peer is not currently connected to but may connect to in the future.

Every 10 s, the current peer tries a peer in the interested but choked group for a better data rate. If this new peer has a better rate than any of the unchoked peers, the new peer may become unchoked, and the peer with the lowest data rate in the unchoked group may move to the choked group. In this way, the peers in the unchoked group always have the highest data rate among those peers probed. Using this strategy divides the neighbors into subgroups in which those neighbors with compatible data transfer rates will communicate with each other. The idea of tit-for-tat trading strategy described above can be seen in this policy.

To allow a newly joined peer, which does not yet have a piece to share, to also receive pieces from other peers, every 30 s a peer randomly promotes a single interested peer, regardless of its uploading rate, from the choked group and flags it as unchoked. This action is called *optimistic unchoking*.

The BitTorrent protocol tries to provide a balance between the number of pieces each peer may have at each moment by using a strategy called the *rarest-first*. Using this strategy, a peer tries to first download the pieces with the fewest repeated copies among the neighbors. In this way, these pieces are circulated faster.

Trackerless BitTorrent

In the original BitTorrent design, if the tracker fails, new peers cannot connect to the network and updating is interrupted. There are several implementations of BitTorrent that eliminate the need for a centralized tracker. In the implementation that we describe here, the protocol still uses the tracker, but not a central one. The job of tracking is distributed among some nodes in the network. In this section, we show how Kademlia DHT can be used to achieve this goal, but we avoid becoming involved in the details of a specific protocol.

In BitTorrent with a central tracker, the job of the tracker is to provide the list of peers in a swarm when given a metadata file that defines the torrent. If we think of the hash function of metadata as the key and the hash function of the list of peers in a

swarm as the value, we can let some nodes in a P2P network play the role of trackers. A new peer that joins the torrent sends the hash function of the metadata (key) to the node that it knows. The P2P network uses Kademlia protocol to find the node responsible for the key. The responsible node sends the *value*, which is actually the list of peers in the corresponding torrent, to the joining peer. Now the joining peer can use the BitTorrent protocol to share the content file with peers in the list.

10.5 SOCKET INTERFACE PROGRAMMING

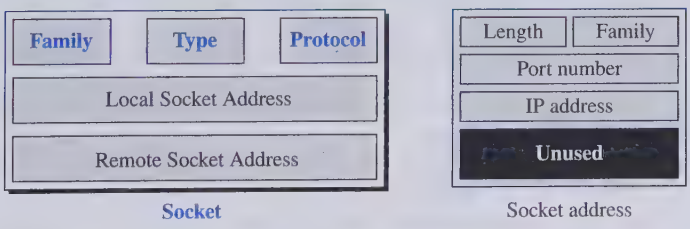
In Section 10.2, we discussed the principle of the client/server paradigm. In Section 10.3, we discussed some standard applications using this paradigm. In this section, we show how to write some simple client/server programs using C, a procedural programming language. We chose the C language in this section for two reasons. First, socket programming traditionally started in the C language. Second, the low-level feature of the C language better reveals some subtleties in this type of programming.

We discussed Application Programming Interface in Section 10.2.1. In this section, we show how this interface is implemented in the C language. The important issue in socket interface is to understand the role of a socket in communication. The socket has no buffer to store data to be sent or received. It is not capable of sending or receiving data. The socket just acts as a reference or a label. The buffers and necessary variables are created inside the operating system.

10.5.1 Data Structure for Socket

The C language defines a socket as a structure (struct). The socket structure is made up of five fields; each socket address itself is a structure made up of five fields, as shown in Figure 10.58. Note that the programmer should not redefine this structure; it is already defined in the header files. We briefly discuss the five fields in a socket structure.

Figure 10.58 *Socket data structure*



- **Family.** This field defines the family protocol (how to interpret the addresses and port number). The common values are PF_INET (for current Internet), PF_INET6 (for next-generation Internet), and so on. We use PF_INET for this section.
- **Type.** This field defines four types of sockets: SOCK_STREAM (for TCP), SOCK_DGRAM (for UDP), SOCK_SEQPACKET (for SCTP), and SOCK_RAW (for applications that directly use the services of IP).

- **Protocol.** This field defines the specific protocol in the family. It is set to 0 for the TCP/IP protocol suite because it is the only protocol in the family.
- **Local socket address.** This field defines the *local socket address*. A socket address is itself a structure made up of the *length* field, the *family* field (which is set to the constant AF_INET for the TCP/IP protocol suite), the port number field (which defines the process), and the IP address field (which defines the host on which the process is running). It also contains an unused field.
- **Remote socket address.** This field defines the remote socket address. Its structure is the same as the local socket address.

10.5.2 Header Files

To be able to use the definition of the socket and all procedures (functions) defined in the interface, we need a set of header files. We have collected all these header files in a file named *headerFiles.h*. This file needs to be created in the same directory as the programs, and its name should be included in all programs.

```
// "headerFiles.h"
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <errno.h>
#include <signal.h>
#include <unistd.h>
#include <string.h>
#include <arpa/innet.h>
#include <sys/wait.h>
```

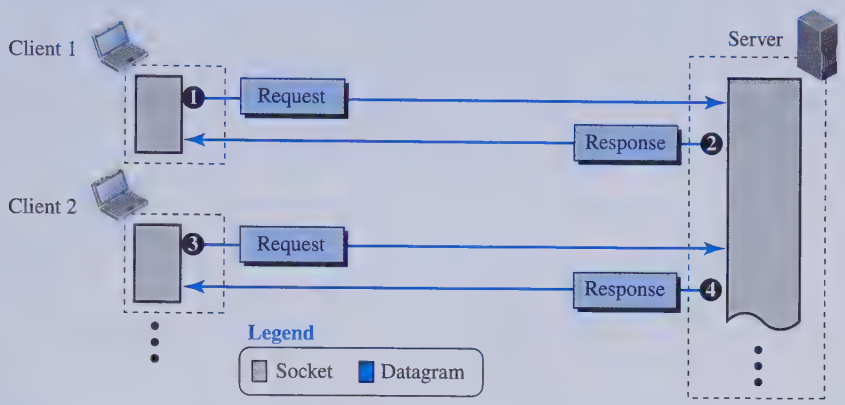
10.5.3 Iterative Communication Using UDP

UDP provides a connectionless server, in which a client sends a request and the server sends back a response.

Sockets Used for UDP

In UDP communication, the client and server use only one socket each. The socket created at the server site lasts forever; the socket created at the client site is closed (destroyed) when the client process terminates. Figure 10.59 shows the lifetime of the sockets in the server and client processes. In other words, different clients use different sockets, but the server creates only one socket and changes only the remote socket address each time a new client makes a connection. This is logical, because the server does know its own socket address, but it does not know the socket address of the clients who need its server; it needs to wait for the client to connect before filling this part of the socket.

Figure 10.59 *Sockets for UDP communication*



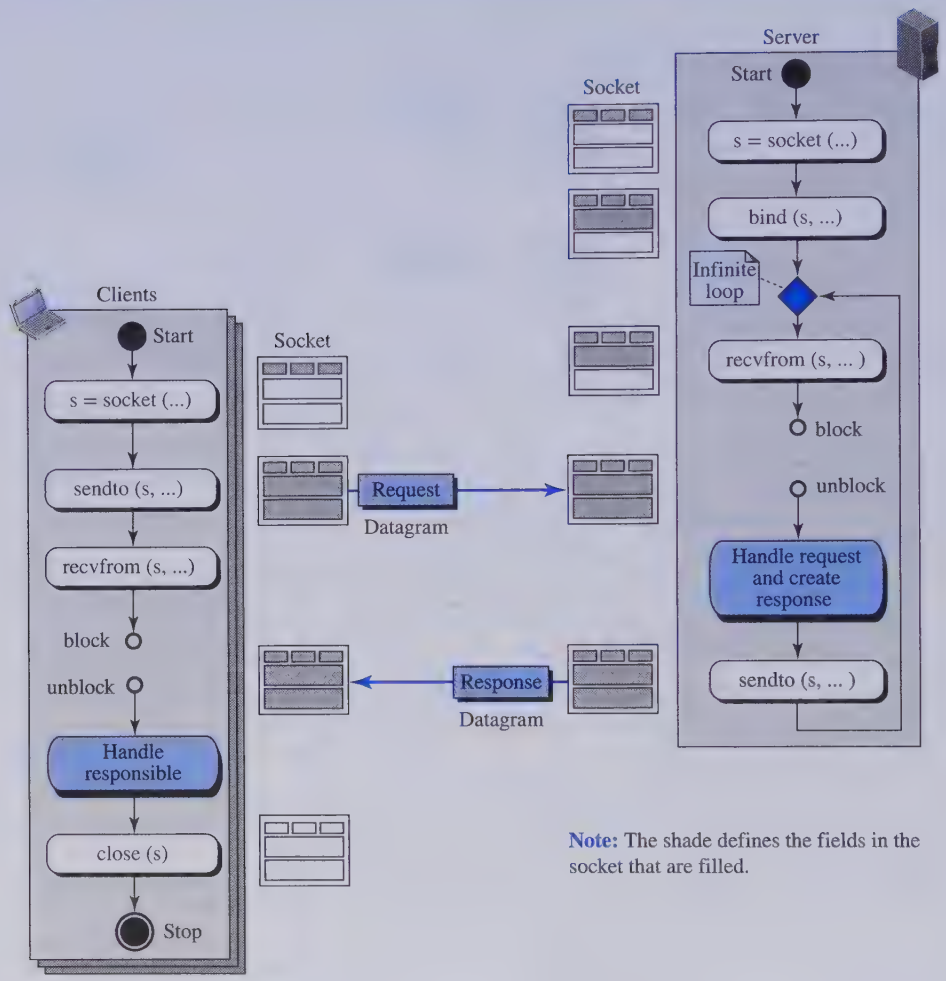
Communication Flow Diagram

Figure 10.60 shows a simplified flow diagram for iterative communication. There are multiple clients, but only one server. Each client is served in each iteration of the loop in the server. Note that there are no connection establishment and connection termination. Each client sends one single datagram and receives one single datagram. In other words, if a client wants to send two datagrams, it is considered as two clients for the server. The second datagram needs to wait for its turn.

Server Process

The server makes a *passive open*, in which it becomes ready for the communication, but it waits until a client process makes the connection. It calls the *socket* procedure to create a socket. The arguments in this procedure call fill the first three fields, but the local and remote socket address fields are still undefined. The server process then calls the *bind* procedure to fill the local socket address field (information comes from the operating system). It then calls another procedure, called *recvfrom*. This procedure, however, blocks the server process until a client datagram arrives. When a datagram arrives, the server process unblocks and extracts the request from the datagram. It also extracts the sender socket address to be used in the next step. After the request is processed and the response is ready, the server process completes the socket structure by filling the remote socket address field with the sender socket address in the received datagram. Now the datagram is ready to be sent. This is done by calling another procedure, called *sendto*. Note that all the fields in the socket should be filled before the server process can send the response. After sending the response, the server process starts a new iteration and waits for another -client to connect. The remote socket address field will be refilled again with the address of a new client (or the same client considered as a new one). The server process is an infinite process; it runs forever. The server socket is never closed unless there is a problem and the process needs to be aborted.

Figure 10.60 *Flow diagram for iterative UDP communication*



Client Process

The client process makes an *active open*. In other words, it starts a connection. It calls the *socket* procedure to create a socket and fill the first three fields. Although some implementations require that the client process also calls the *bind* procedure to fill the local socket address, normally this is done automatically by the operating system, which selects a temporary port number for the client. The client process then calls the *sendto* procedure and provides the information about the remote socket address. This socket address must be provided by the user of the client process. The socket is complete at this moment, and the datagram is sent. The client process now calls the *recvfrom* procedure, which blocks the client process until the response comes back from the server -process. There is no need to extract the remote socket address from this procedure because there is no call to the *sendto* procedure. In

other words, the *recvfrom* procedure at the server site and the client site behave differently. In the server process, the *recvfrom* procedure is called first and the *sendto* next, so the remote socket address for *sendto* can be obtained from *recvfrom*. In the server process, the *sendto* is called before the *recvfrom*, so the remote address should be provided by the program user who knows to which server she wants to connect. Finally the *close* procedure is called to destroy the socket. Note that the client process is finite; after the response has been returned, the client process stops. Although we can design clients to send multiple datagrams, using a loop, each iteration of the loop looks like a new client to the server.

Programming Examples

In this section, we show how to write client and server programs to simulate the standard *echo* application using UDP. The client program sends a short string of characters to the server; the server echoes back the same string to the server. The standard application is used by a computer, the client, to test the liveliness of another computer, the server. Our programs are simpler than the ones used in the standard; we have eliminated some error-checking and debugging details for simplicity.

Echo Server Program

Table 10.22 shows the echo server program using UDP. The program follows the flow diagram in Figure 10.60.

Lines 6 to 11 declare and define variables used in the program. Lines 13 to 16 allocate memory for the server socket address (using the *memset* function) and fill the field of the socket address with default values provided by the transport layer. To insert the port number, we use the *htons* (host to network short) function, which transforms a value in host byte-ordering format to a short value in network byte-ordering format. To insert the IP address, we use the *htonl* (host to network long) function to do the same thing.

Lines 18 to 22 call the socket function in an if-statement to check for error. Because this function returns -1 if the call fails, the programs prints the error message and exits. The *perror* function is a standard error function in C. Similarly, lines 24 to 28 call the bind function to bind the socket to the server socket address. Again, the function is called in an if-statement for error checking.

Lines 29 to 36 use an infinite loop to be able to serve clients in each iteration. Lines 32 and 33 call the *recvfrom* function to read the request sent by the client. Note that this function is a blocking one; when it unblocks, it receives the request message and, at the same time, provides the client socket address to complete the last part of the socket. Line 35 calls the *sendto* function to send back (echo) the same message to the client, using the client socket address obtained in the *recvfrom* message. Note that there is no processing done on the request message; the server just echoes what has been received.

Echo Client Program

Table 10.23 shows the echo client program using UDP. The program follows the flow diagram in Figure 10.60.

Lines 6 to 12 declare and define variables used in the program. Lines 14 to 21 test and set arguments that are provided when the program is run. The first two arguments provide the server name and server port number; the third argument is the string to be echoed. Lines 23 to 26 allocate memory, convert the server name to the server IP address using the

Table 10.22 *Echo server program using UDP*

```
1  // UDP echo server program
2  #include "headerFiles.h"
3  int main (void)
4  {
5     // Declare and define variables
6     int s;                                  // Socket descriptor (reference)
7     int len;                                // Length of string to be echoed
8     char  buffer [256];                     // Data buffer
9     struct sockaddr_in servAddr;            // Server (local) socket address
10    struct sockaddr_in clntAddr;            // Client (remote) socket address
11    int  clntAddrLen;                       // Length of client socket address
12    // Build local (server) socket address
13    memset (&servAddr, 0, sizeof (servAddr));              // Allocate memory
14    servAddr.sin_family =  AF_INET;                        // Family field
15    servAddr.sin_port = htons (SERVER_PORT)                // Default port number
16    servAddr.sin_addr.s_addr =  htonl (INADDR_ANY);        // Default IP address
17    // Create socket
18    if ((s = socket (PF_INET, SOCK_DGRAM, 0)) < 0);
19    {
20          perror ("Error: socket failed!");
21          exit (1);
22    }
23    // Bind socket to local address and port
24    if ((bind (s, (struct sockaddr*) &servAddr, sizeof (servAddr))) < 0);
25    {
26          perror ("Error: bind failed!");
27          exit (1);
28    }
29    for ( ; ; )              // Run forever
30    {
31          // Receive String
32          len = recvfrom (s, buffer, sizeof (buffer), 0,
33                (struct sockaddr*)&clntAddr, &clntAddrLen);
34          // Send String
35          sendto (s, buffer, len, 0, (struct sockaddr*)&clntAddr, sizeof(clntAddr));
36    } // End of for loop
37 } // End of echo server program
```

Table 10.23 *Echo client program using UDP*

```
1  // UDP echo client program
2  #include "headerFiles.h"
3  int main (int argc, char* argv[ ])          // Three arguments to be checked later
4  {
5      // Declare and define variables
6      int  s;                                 // Socket descriptor
7      int  len;                               // Length of string to be echoed
8      char*  servName;                        // Server name
9      int  servPort;                          // Server port
10     char*  string;                          // String to be echoed
11     char  buffer [256 + 1];                 // Data buffer
12     struct  sockaddr_in   servAddr;         // Server socket address
13     // Check and set program arguments
14     if (argc != 3)
15     {
16         printf ("Error: three arguments are needed!");
17         exit(1);
18     }
19     servName = argv[1];
20     servPort = atoi (argv[2]);
21     string  =  argv[3];
22     // Build server socket address
23     memset (&servAddr, 0, sizeof (servAddr));
24     servAddr.sin_family = AF_INET;
25     inet_pton (AF_INET, servName, &servAddr.sin_addr);
26     servAddr.sin_port = htons (servPort);
27     // Create socket
28     if ((s = socket (PF_INET, SOCK_DGRAM, 0)) < 0);
29     {
30         perror ("Error: -Socket failed!");
31         exit (1);
32     }
33     // Send echo string
34     len =  sendto (s, string, strlen (string), 0, (struct sockaddr)&servAddr, sizeof (servAddr));
35     // Receive echo string
36     recvfrom (s, buffer, len, 0, NULL, NULL);
37     // Print and verify echoed string
38     buffer [len] = '\0';
39     printf ("Echo string received: ");
40     fputs (buffer, stdout);
41     // Close the socket
42     close (s);
43     // Stop the program
44     exit (0);
45 } // End of echo client program
```

function inet_pton, which is a function that calls DNS (discussed earlier in Section 10.3.6) and convert the port number to the appropriate byte order. These three pieces of information, which are needed for the sendto function, are stored in appropriate variables.

Line 34 calls the *sendto* function to send the request. Line 36 calls the *recvfrom* function to receive the echoed message. Note that the two arguments in this message are NULL because we do not need to extract the socket address of the remote site; the message already has been sent.

Lines 38 to 40 are used to display the echoed message on the screen for debugging purposes. Note that in line 38 we add a null character at the end of the echoed message to make it displayable by the next line. Finally, line 42 closes the socket and line 44 exits the program.

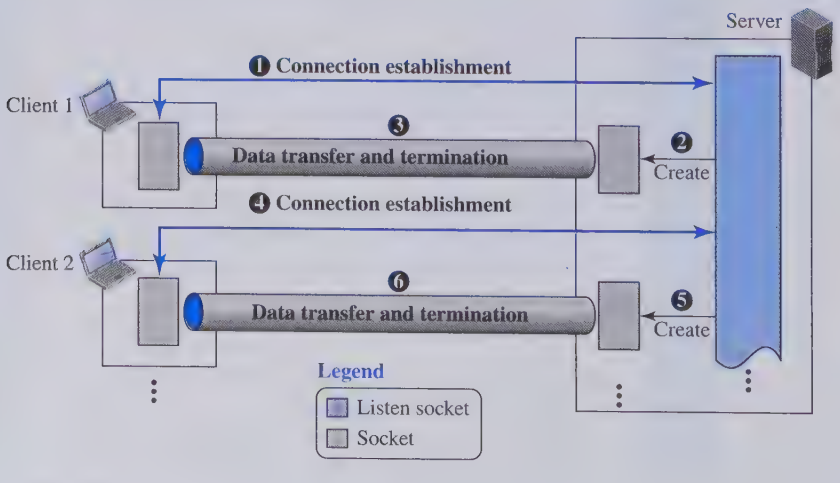
10.5.4 Communication Using TCP

As we described before, TCP is a connection-oriented protocol. Before sending or receiving data, a connection needs to be established between the client and the server. After the connection is established, the two parties can send and receive chunks of data to each other as long as they have data to do so. TCP communication can be iterative (serving a client at a time) or concurrent (serving several clients at a time). In this section, we discuss only the iterative approach.

Sockets Used in TCP

The TCP server uses two different sockets, one for connection establishment and the other for data transfer. We call the first one the *listen socket* and the second the *socket*. The reason for having two types of sockets is to separate the connection phase from the data exchange phase. A server uses a listen socket to listen for a new client trying to establish connection. After the connection is established, the server creates a socket to exchange data with the client and finally to terminate the connection. The client uses only one socket for both connection establishment and data exchange (see Figure 10.61).

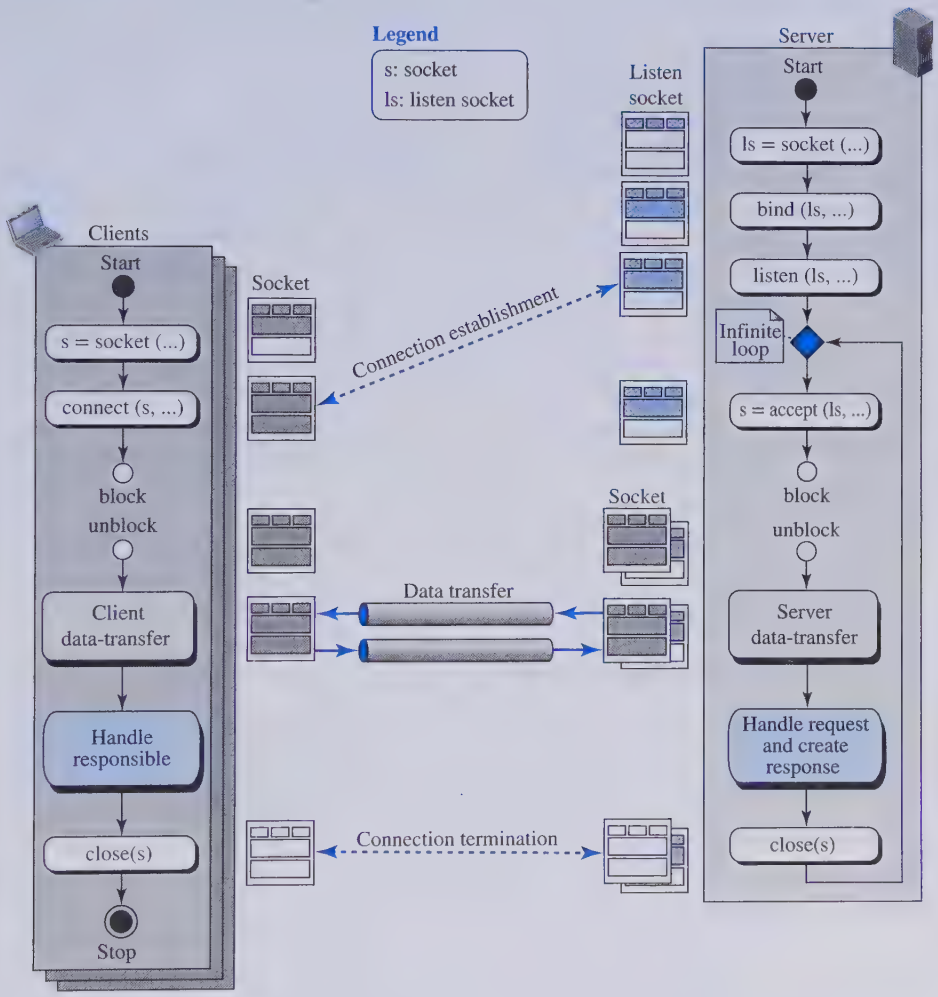
Figure 10.61 *Sockets used in TCP communication*



Communication Flow Diagram

Figure 10.62 shows a simplified flow diagram for iterative communication. There are multiple clients, but only one server. Each client is served in each iteration of the loop. The flow diagram is similar to the one for UDP, but there are differences that we explain for each site.

Figure 10.62 *Flow diagram for iterative TCP communication*



Server Process

In Figure 10.62, the TCP server process, like the UDP server process, calls the *socket* and *bind* procedures, but these two procedures create the listen socket to be used only for the connection establishment phase. The server process then calls the *listen* procedure, to allow the operating system to start accepting the clients, completing the connection

phase, and putting them in the waiting list to be served. This -procedure also defines the size of the connected client waiting list, which depends on the complexity of the server process, but it is normally 5.

The server process now starts a loop and serves the clients one by one. In each iteration, the server process calls the *accept* procedure that removes one client from the waiting list of the connected clients for serving. If the list is empty, the *accept* procedure blocks until there is a client to be served. When the accept procedure returns, it creates a new socket that is the same as the listen socket. The listen socket now moves to the background, and the new socket becomes the active one. The server process now uses the client socket address obtained during the connection establishment to fill the remote socket address field in the newly created socket.

At this time, the client and server can exchange data. We have not shown the specific way in which the data transfer takes place because it depends on the specific client/server pair. TCP uses the *send* and *recv* procedures to transfer bytes of data between them. These two procedures are simpler than the *sendto* and *recvfrom* procedures used in UDP because they do not provide the remote socket address; a connection has already been established between the client and server. However, because TCP is used to transfer messages with no boundaries, each application needs to carefully design the data transfer section. The *send* and *recv* procedures may be called several times to handle a large amount of data transfer. The flow diagram in Figure 10.62 can be considered as a generic one; for a specific purpose, the diagram for the *server data-transfer* box needs to be defined. We will this do for a simple example when we discuss the echo client/server program later in this section.

Client Process

The client flow diagram is similar to the UDP version except that the *client data-transfer* box needs to be defined for each specific case. We do so when we write a specific program later.

Programming Examples

In this section, we show how to write client and server programs to simulate the standard *echo* application using TCP. The client program sends a short string of characters to the server; the server echoes back the same string to the client. However, before we do so, we need to provide the flow diagram for the client and server data-transfer boxes, which is shown in Figure 10.63.

For this special case, because the size of the string to be sent is small (less than a few words), we can do it in one call to the *send* procedure in the client. However, it is not guaranteed that the TCP will send the whole message in one segment. Therefore, we need to use a set of *recv* calls in the server site (in a loop), to receive the whole message and collect them in the buffer to be sent back in one shot. When the server is sending back the echo message, it may also use several segments to do so, which means the *recv* procedure in the client needs to be called as many times as needed.

Another issue to be solved is setting the buffers that hold data at each site. We need to control how many bytes of data we have received and where the next chunk of data is stored. The program sets some variables to control the situation, as shown in Figure 10.64. In each iteration, the pointer (ptr) moves ahead to point to the next bytes to receive, the

Figure 10.63 *Flow diagram for the client and server data-transfer boxes*

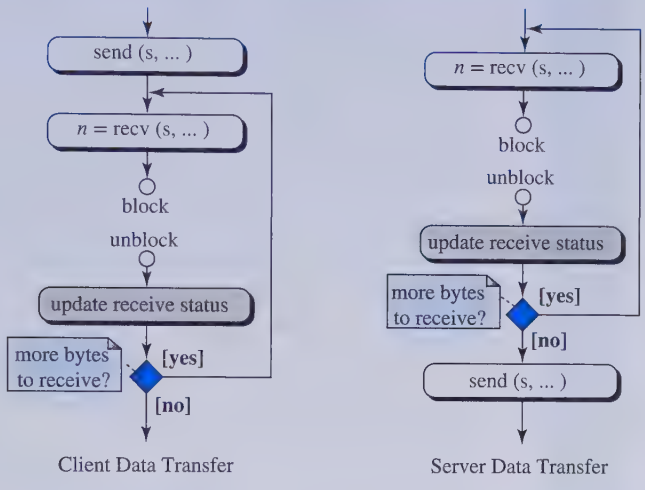
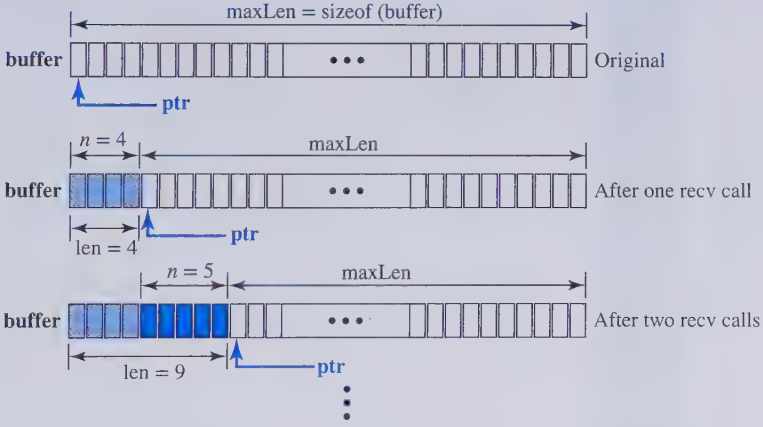


Figure 10.64 *Buffer used for receiving*



length of received bytes (len) is increased, and the maximum number of bytes to be received (maxLen) is decreased.

After the above two considerations, we can now write the server and the client program.

Echo Server Program

Table 10.24 (on next page) shows the echo server program using TCP. The program follows the flow diagram in Figure 10.62. Each shaded section corresponds to one instruction in the layout. The colored sections show the data transfer section of the diagram.

Table 10.24 *Echo server program using the services of TCP*

```
1  // Echo server program
2  #include "headerFiles.h"
3  int main (void)
4  {
5      // Declare and define
6      int ls;                          // Listen socket descriptor -(reference)
7      int s;                           // socket descriptor (reference)
8      char buffer [256];               // Data buffer
9      char* ptr = buffer;              // Data buffer
10     int len = 0;                     // Number of bytes to send or -receive
11     int maxLen = sizeof (buffer);    // Maximum number of bytes to receive
12     int n = 0;                       // Number of bytes for each recv call
13     int waitSize = 16;               // Size of waiting clients
14     struct sockaddr_in serverAddr;   // Server address
15     struct sockaddr_in clientAddr;   // Client address
16     int clntAddrLen;                 // Length of client address
17     // Create local (server) socket address
18     memset (&servAddr, 0, sizeof (servAddr));
19     servAddr.sin_family = AF_INET;
20     servAddr.sin_addr.s_addr = htonl (INADDR_ANY);     // Default IP address
21     servAddr.sin_port = htons (SERV_PORT);             // Default port
22 // Create listen socket
23     if (ls = socket (PF_INET, SOCK_STREAM, 0) < 0);
24     {
25         perror ("Error: Listen socket failed!");
26         exit (1);
27     }
28     // Bind listen socket to the local socket address
29     if (bind (ls, &servAddr, sizeof (servAddr)) < 0);
30     {
31         perror ("Error: binding failed!");
32         exit (1);
33     }
34     // Listen to connection requests
35     if (listen (ls, waitSize) < 0);
36     {
37         perror ("Error: listening failed!");
38         exit (1);
39     }
```

```
40      // Handle the connection
41      for ( ; ; )                              // Run forever
42      {
43          // Accept connections from client
44          if (s = accept (ls, &clntAddr, &clntAddrLen) < 0);
45          {
46              perror ("Error: accepting failed!);
47              exit (1);
48          }
49          // Data transfer section
50          while ((n = recv (s, ptr, maxLen, 0)) > 0)
51          {
52              ptr + = n;              // Move pointer along the buffer
53              maxLen -= n;            // Adjust maximum number of bytes to receive
54              len + = n;              // Update number of bytes received
55          }
56          send (s, buffer, len, 0);          // Send back (echo) all bytes received
57          // Close the socket
58          close (s);
59      } // End of for loop
60  } // End of echo server program
```

Lines 6 to 16 declare and define variables. Lines 18 to 21 allocate memory and construct the local (server) socket address as described in the UDP case. Lines 23 to 27 create the listen socket. Lines 29 to 33 bind the listen socket to the server socket address constructed in lines 18 to 21. Lines 35 to 39 are new in TCP communication. The *listen* function is called to let the operating system complete the connection establishment phase and put the clients in the waiting list. Lines 44 to 48 call the *accept* function to remove the next client in the waiting list and start serving it. This function blocks if there is no client in the waiting list. Lines 50 to 56 code the data transfer section depicted in Figure 10.63. The maximum buffer size, the length of the string echoed, is the same as shown in Figure 10.64.

Echo Client Program

Table 10.25 (on next page) shows the echo client program using TCP. The program follows the outline in Figure 10.62. Each shaded section corresponds to one instruction in the flow diagram. The colored section corresponds to the data transfer section.

The client program for TCP is very similar to the client program for UDP, with a few differences. Because TCP is a connection-oriented protocol, the *connect* function is called in lines 36 to 40 to make connection to the server. Data transfer is done in lines 42 to 48 using the idea depicted in Figure 10.63. The length of data received and the pointer movement are done as shown in Figure 10.64.

Table 10.25 *Echo client program using TCP*

```
1  // TCP echo client program
2  #include "headerFiles.h"
3  int main (int argc, char* argv[ ])          // Three arguments to be checked later
4  {
5       // Declare and define
6       int  s;                                 // Socket descriptor
7       int  n;                                 // Number of bytes in each recv call
8       char* servName;                         // Server name
9       int servPort;                           // Server port number
10      char* string;                           // String to be echoed
11      int  len;                               // Length of string to be echoed
12      char  buffer [256 + 1];                 // Buffer
13      char* ptr = buffer;                     // Pointer to move along the buffer
14      struct sockaddr_in serverAddr;          // Server socket address
15      // Check and set arguments
16      if (argc != 3)
17      {
18           printf ("Error: three arguments are needed!");
19           exit (1);
20      }
21      servName = arg [1];
22      servPort = atoi (arg [2]);
23      string = arg [3];
24      // Create remote (server) socket address
25      memset (&servAddr, 0, sizeof(servAddr));
26      serverAddr.sin_family = AF_INET;
27      inet_pton (AF_INET, servName, &serverAddr.sin_addr);    // Server IP address
28      serverAddr.sin_port = htons (-servPort);                // Server port number
29      // Create socket
30      if ((s = socket (PF_INET, SOCK_STREAM, 0)) < 0);
31      {
32           perror ("Error: socket creation failed!");
33           exit (1);
34      }
35      // Connect  to the server
36      if (connect (sd, (struct sockaddr*)&servAddr, sizeof(servAddr)) < 0);
37      {
38           perror ("Error: connection failed!");
39           exit (1);
40      }
```

```
41      // Data transfer section
42      send (s, string, strlen(string), 0);
43      while ((n = recv (s, ptr, maxLen, 0)) > 0)
44      {
45          ptr + = n;              // Move pointer along the buffer
46          maxLen −= n;            // Adjust the maximum number of bytes
47          len + = n;              // Update the length of string received
48      } // End of while loop
49      // Print and verify the echoed string
50      buffer [len] = '\0';
51      printf ("Echoed string received: ");
52      fputs (buffer, stdout);
53      // Close socket
54      close (s);
55      // Stop program
56      exit (0);
57  } // End of echo client program
```

10.6 END-OF-CHAPTER MATERIALS

10.6.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books and Requests for Comments (RFCs). The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give thorough coverage of materials discussed in this chapter including [Com 06], [Mir 07], [Ste 94], [Tan 03], and [Bar et al. 05].

Requests For Comments

HTTP is discussed in RFCs 2068 and 2109. FTP is discussed in RFCs 959, 2577, and 2585. TELNET is discussed in RFCs 854, 855, 856, 1041, 1091, 1372, and 1572. SSH is discussed in RFCs 4250, 4251, 4252, 4253, 4254, and 4344. DNS is discussed in RFCs 1034, 1035, 1996, 2535, 3008, 3658, 3755, 3757, 3845, 3396, and 3342. SMTP is discussed in RFCs 2821and 2822. POP3 is explained in RFC 1939. MIME is discussed in RFCs 2046, 2047, 2048, and 2049.

10.6.2 Key Terms

active document
application programming interface (API)
browser
Chord
Client/server paradigm
cookie
country domain
datagram
Distributed Hash Table (DHT)
domain
domain name
domain name space
DNS server
Domain Name System (DNS)
dynamic document
Dynamic Domain Name System (DDNS)
File Transfer Protocol (FTP)
fully qualified domain name (FQDN)
generic domain
hypermedia
hypertext
Hypertext Transfer Protocol (HTTP)
Internet Mail Access Protocol, version 4 (IMAPv4)
iterative resolution
Kademlia
label
local login
message access agent (MAA)
message transfer agent (MTA)
Multipurpose Internet Mail Extensions (MIME)
name space
network virtual terminal (NVT)
nonpersistent connection
partially qualified domain name (PQDN)
Pastry
peer-to-peer (P2P) paradigm
persistent connection
port forwarding
Post Office Protocol, version 3 (POP3)\
processes
proxy server
remote login
resolver
root server
Secure Shell (SSH)
Simple Mail Transfer Protocol (SMTP)
socket address
socket interface
static document
STREAM
terminal network (TELNET)
transport layer interface (TLI)
uniform resource locator (URL)
user agent (UA)
web page
World Wide Web (WWW)
zone

10.6.3 Summary

Applications in the Internet are designed using either a client/server paradigm or a peer-to-peer paradigm. In a client/server paradigm, an application program, called a server, provides services and another application program, called a client, receives services. A server program is an infinite program; a client program is finite. In a peer-to-peer paradigm, a peer can be both a client and a server.

The World Wide Web (WWW) is a repository of information linked together from points all over the world. Hypertext and hypermedia documents are linked to one another through pointers. The Hypertext Transfer Protocol (HTTP) is the main protocol used to access data on the WWW.

File Transfer Protocol (FTP) is a TCP/IP client/server application for copying files from one host to another. FTP requires two connections for data transfer: a control connection and a data connection. FTP employs NVT ASCII for communication between dissimilar systems.

Electronic mail is one of the most common applications on the Internet. The e-mail architecture consists of several components such as user agent (UA), main transfer agent (MTA), and main access agent (MAA). The protocol that implements MTA is called Simple Main Transfer Protocol (SMTP). Two protocols are used to implement MAA: Post Office Protocol, version 3 (POP3), and Internet Mail Access Protocol, version 4 (IMAP4).

TELNET is a client/server application that allows a user to log in to a remote machine, giving the user access to the remote system. When a user accesses a remote system via the TELNET process, this is comparable to a time-sharing environment.

The Domain Name System (DNS) is a client/server application that identifies each host on the Internet with a unique name. DNS organizes the name space in a hierarchical structure to decentralize the responsibilities involved in naming.

In a peer-to-peer (P2P) network, Internet users that are ready to share their resources become peers and form a network. Peer-to-peer networks are divided into centralized and decentralized. In a centralized P2P network, the directory system uses the client/server paradigm, but storing and downloading of files are done using the P2P paradigm. In a decentralized network, both the directory system and storing and downloading of files are done using the P2P paradigm.

10.7 PRACTICE SET

10.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

10.7.2 Questions

Q10-1. Assume we add a new protocol to the application layer. What changes do we need to make to other layers?

Q10-2. Explain which entity provides service and which one receives service in the client/server paradigm.

Q10-3. In the client/server paradigm, explain why a server should be run all the time, but a client can be run when it is needed.

Q10-4. Can a program written to use the services of UDP be run on a computer that has installed TCP as the only transport-layer protocol? Explain.

Q10-5. During the weekend, Alice often needs to access files stored on her office desktop, from her home laptop. Last week, she installed a copy of the FTP server process on her desktop at her office and a copy of the FTP client process on her laptop at home. She was disappointed when she could not access her files during the weekend. What could have gone wrong?

Q10-6. Most of the operating systems installed on personal computers come with several client processes, but normally no server processes. Explain the reason.

Q10-7. A new application is to be designed using the client/server paradigm. If only small messages need to be exchanged between the client and the server without the concern for message loss or corruption, what transport-layer protocol do you recommend?

Q10-8. Which of the following can be a source of data?

a. keyboard **b.** monitor **c.** socket

Q10-9. A source socket address is a combination of an IP address and a port number. Explain what each section identifies.

Q10-10. Explain how a client process finds the IP address and the port number to be inserted in a remote socket address.

Q10-11. If an HTTP request needs to run a program at the server site and downloads the result to the client server, the program is an example of which of the following?

a. static document **b.** dynamic document **c.** active document

Q10-12. Assume we design a new client/server application program that requires persistent connection. Can we use UDP as the underlying transport-layer protocol for this new application?

Q10-13. Alice has a video clip that Bob is interested in getting; Bob has another video clip that Alice is interested in getting. Bob creates a web page and runs an HTTP server. How can Alice get Bob's clip? How can Bob get Alice's clip?

Q10-14. When an HTTP server receives a request message from an HTTP client, how does the server know when all headers have arrived and the body of the message is to follow?

Q10-15. In a nonpersistent HTTP connection, how can HTTP inform the TCP protocol that the end of the message has been reached?

Q10-16. Can you find an analogy in our daily life as to when we use two separate connections in communication similar to the control and data connections in FTP?

Q10-17. FTP uses two separate well-known port numbers for control and data connection. Does this mean that two separate TCP connections are created for exchanging control information and data?

Q10-18. FTP uses the services of TCP for exchanging control information and data transfer. Could FTP have used the services of UDP for either of these two connections? Explain.

Q10-19. In FTP, which entity (client or server) starts (actively opens) the control connection? Which entity starts (actively opens) the data transfer connection?

Q10-20. What do you think would happen if the control connection were severed before the end of an FTP session? Would it affect the data connection?

Q10-21. In FTP, if the client needs to retrieve one file from the server site and store one file on the server site, how many control connections and how many data transfer connections are needed?

Q10-22. In FTP, can a server retrieve a file from the client site?

Q10-23. In FTP, can a server get the list of the files or directories from the client?

Q10-24. FTP can transfer files between two hosts using different operating systems with different file formats. What is the reason?

Q10-25. Does FTP have a message format for exchanging commands and responses during control connection?

Q10-26. Does FTP have a message format for exchanging files or a list of directories/files during the file transfer connection?

Q10-27. Can we have a control connection without a data transfer connection in FTP? Explain.

Q10-28. Can we have a data transfer connection without a control connection in FTP? Explain.

Q10-29. Assume we need to download an audio using FTP. What file type should we specify in our command?

Q10-30. Both HTTP and FTP can retrieve a file from a server. Which protocol should we use to download a file?

Q10-31. Are the HELO and MAIL FROM commands both necessary in SMTP? Why or why not?

Q10-32. In Figure 10.20 in the text, what is the difference between the MAIL FROM in the envelope and the FROM in the header?

Q10-33. Alice has been on a long trip without checking her e-mail. She then finds out that she has lost some e-mails or attachments her friends claim they sent to her. What could be the problem?

Q10-34. Assume a TELNET client uses ASCII to represent characters, but the TELNET server uses EBCDIC to represent characters. How can the client log in to the server when character representations are different?

Q10-35. The TELNET application has no commands such as those found in FTP or HTTP to allow the user to do something such as transfer a file or access a web page. In what way can this application be useful?

Q10-36. Can a host use a TELNET client to get services provided by other client/server applications such as FTP or HTTP?

Q10-37. In DNS, which of the following are FQDNs and which are PQDNs?

a. xxx **b.** xxx.yyy.net **c.** zzz.yyy.xxx.edu

Q10-38. In a DHT-based network, assume $m = 4$. If the hash of a node identifier is 18, where is the location of the node in the DHT space?

Q10-39. In a DHT-based network, assume node 4 has a file with key 18. The closest next node to key 18 is node 20. Where is the file stored?

a. in the direct method **b.** in the indirect method

Q10-40. In a Chord network, we have node N5 and key k5. Is N5 the predecessor of k5? Is N5 the successor of k5?

Q10-41. In a Kademlia network, the size of the identifier space is 1024. What is the height of the binary tree (the distance between the root and each leaf)? What is the number of leaves? What is the number of subtrees for each node? What is the number of rows in each routing table?

Q10-42. In Kademlia, assume $m = 4$ and active nodes are N4, N7, and N12. Where is the key k3 stored in this system?

10.7.3 Problems

P10-1. Assume there is a server with the domain name *www.common.com*.

a. Show an HTTP request that needs to retrieve the document **/usr/users/doc**.

The client accepts MIME version 1, GIF, or JPEG images, but the document should not be more than 4 days old.

b. Show the HTTP response to part *a* for a successful request.

P10-2. In HTTP, draw a figure to show the application of cookies in a scenario in which the server allows only the registered customer to access the server.

P10-3. In HTTP, draw a figure to show the application of cookies in a web portal using two sites.

P10-4. In HTTP, draw a figure to show the application of cookies in a scenario in which the server uses cookies for advertisement. Use only three sites.

P10-5. Draw a diagram to show the use of a proxy server that is part of the client network.

a. Show the transactions between the client, proxy server, and the target server when the response is stored in the proxy server.

b. Show the transactions between the client, proxy server, and the target server when the response is not stored in the proxy server.

P10-6. HTTP version 1.1 defines the persistent connection as the default connection. Using RFC 2616, find out how a client or server can change this default situation to nonpersistent.

P10-7. In SMTP, a sender sends unformatted text. Show the MIME header.

P10-8. Answer the following questions for SMTP.

a. A non-ASCII message of 1000 bytes is encoded using base64. How many bytes are in the encoded message? How many bytes are redundant? What is the ratio of redundant bytes to the total message?

b. A message of 1000 bytes is encoded using quoted-printable. The message consists of 90 percent ASCII and 10 percent non-ASCII characters. How many bytes are in the encoded message? How many bytes are redundant? What is the ratio of redundant bytes to the total message?

c. Compare the results of the two previous cases in parts a and b. How much is the efficiency improved if the message is a combination of ASCII and non-ASCII characters?

P10-9. Encode the following message in base64:

01010111 00001111 11110000

P10-10. Encode the following message in quoted-printable:

01001111 10101111 01110001

P10-11. According to RFC 1939, a POP3 session is in one of the following four states: closed, authorization, transaction, or update. Draw a diagram to show these four states and how POP3 moves between them.

P10-12. The POP3 protocol has some basic commands (that each client/server needs to implement). Using the information in RFC 1939, find the meaning and the use of the following basic commands:

a. STAT **b.** LIST **c.** DELE 4

P10-13. POP3 protocol has some optional commands (that a client/server can implement). Using the information in RFC 1939, find the meaning and the use of the following optional commands:

a. UIDL **b.** TOP 1 15 **c.** USER **d.** PASS

P10-14. Using RFC 1939, assume a POP3 client is in the download-and-keep mode. Show the transaction between the client and the server if the client has only two messages of 192 and 300 bytes to download from the server.

P10-15. Using RFC 1939, assume a POP3 client is in the download-and-delete mode. Show the transaction between the client and the server if the client has only two messages of 230 and 400 bytes to download from the server.

P10-16. In FTP, assume a client with user name John needs to store a video clip called *video2* on the directory */top/videos/general* on the server. Show the commands and responses exchanged between the client and the server if the client chooses ephemeral port number 56002.

P10-17. In FTP, a user (Jane) wants to retrieve an EBCDIC file named *huge* from the */usr/users/report* directory using the ephemeral port 61017. The file is so large that the user wants to compress it before it is transferred. Show all the commands and responses.

P10-18. In FTP, a user (Jan) wants to make a new directory called *Jan* under the directory */usr/usrs/letters*. Show all the commands and responses.

P10-19. In FTP, a user (Maria) wants to move a file named *file1* from the */usr/users/report* directory to the directory */usr/top/letters*. Note that this is a case of renaming a file. We first need to give the name of the old file and then define the new name. Show all the commands and responses.

P10-20. In Chord, assume the size of the identifier space is 16. The active nodes are N3, N6, N8, and N12. Show the finger table (only the target-key and the successor column) for node N6.

P10-21. In Chord, assume that the successor of node N12 is N17. Find whether node N12 is the predecessor of any of the following keys.

a. k12 **b.** k15 **c.** k17 **d.** k22

P10-22. In a Chord network using DHT with $m = 4$, draw the identifier space and place four peers with node ID addresses N3, N8, N11, and N13 and three keys with addresses k5, k9, and k14. Determine which node is responsible for each key. Create a finger table for each node.

P10-23. In a Chord network with $m = 4$, node N2 has the following finger-table values: N4, N7, N10, and N12. For each of the following keys, first find if N2 is the predecessor of the key. If the answer is no, find which node (the closest predecessor) should be contacted to help N2 find the predecessor.

a. k1 **b.** k6 **c.** k9 **d.** k13

P10-24. In Pastry, assume the address space is 16 and that $b = 2$. How many digits are in an address space? List some of the identifiers.

P10-25. In a Pastry network with $m = 32$ and $b = 4$, what is the size of the routing table and the leaf set?

P10-26. Show the outline of a routing table for Pastry with address space of 16 and $b = 2$. Give some possible entries for each cell in the routing table of node n21.

P10-27. In a Pastry network using DHT, in which $m = 4$ and $b = 2$, draw the identifier space with four nodes, N02, N11, N20, and N23, and three keys, k00, k12, and k24. Determine which node is responsible for each key. Also show the leaf set and routing table for each node. Although it is unrealistic, assume that the proximity metric between each two nodes is based on numerical closeness.

P10-28. Using the binary tree in Figure 10.55, show the subtree for node N11.

P10-29. Using the routing tables in Figure 10.56, explain and show the route if node N0 receives a lookup message for the node responsible for K12.

P10-30. In a Kademlia network with m = 4, we have five active nodes: N2, N3, N7, and N12. Find the routing table for each active node (with only one column).

CHAPTER 11

Multimedia

Multimedia refers to a number of different integrated media such as text, images, audio, and video that are generated, stored, and transmitted digitally and can be accessed interactively. Multimedia today is a broad subject that cannot be fully discussed in one chapter. In this chapter, we give an overview of multimedia and touch on subjects that are, directly or indirectly, related to multimedia, such as compression or quality of service. This chapter is divided into four sections.

- The first section discusses the general idea behind compression. Although compression is not directly related to the subject of multimedia, multimedia transmission is not possible without first compressing the data. This section describes both lossless and lossy compression.
- The second section discusses the elements of multimedia: text, image, video, and audio. It describes how these elements are represented, encoded, and compressed using the techniques discussed in the first section.
- The third section divides multimedia on the Internet into three categories: streaming stored audio/video, streaming live audio/video, and real-time interactive audio/video. It describes the features and characteristics of each and gives some examples.
- The fourth section concentrates on the real-time interactive category. It introduces the transport-layer protocols used for multimedia applications: RTP and RTCP. The section also describes two protocols that are used in this category for signaling: SIP and H.323. These protocols are used in voice over IP (Internet telephony) and can be used for signaling protocols in future applications.

11.1 COMPRESSION

In this section, we discuss compression, which plays a crucial role in multimedia communication due to the large volume of data exchanged. In compression, we reduce the volume of data to be exchanged. We can divide compression into two broad categories: lossless and lossy. We briefly discuss the common methods used in each category. We included this section to provide the necessary background for those readers that are not familiar with compression techniques, but this section can be skipped if the reader is familiar with compression techniques.

11.1.1 Lossless Compression

In **lossless compression**, the integrity of the data is preserved because the compression and decompression algorithms are exact inverses of each other: No part of the data is lost in the process. Lossless compression methods are normally used when we cannot afford to lose any data. For example, we must not lose data when we compress a text file or an application program. Lossless compression is also applied as the last step in some lossy compression procedures to further reduce the size of the data.

We discuss four lossless compression methods in this section: run-length coding, dictionary coding, Huffman coding, and arithmetic coding.

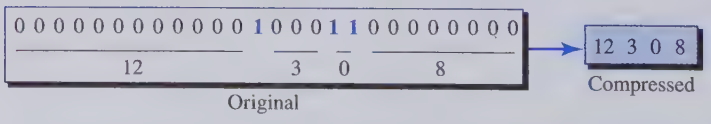
Run-Length Coding

Run-length coding, sometimes referred to as run-length encoding (RLE), is the simplest method of removing redundancy. It can be used to compress data made up of any combination of symbols. The method replaces a repeated sequence, *run*, of the same symbol with two entities: a count and the symbol itself. For example, the following shows how we can compress a string of 17 characters to a string of 10 characters.

AAABBBBCDDDDDDEEE → **3A4B1C6D3E**

A modified version of this method can be used if there are only two symbols in the data, such as a binary pattern made up of 0s and 1s. In this case, we use only the count of one of the symbols that occurs between each occurrence of the other symbol. Figure 11.1 shows a binary pattern in which there are more 0s than 1s. We just show the number of 0s that occur between 1s.

Figure 11.1 *A version of run-length coding to compress binary patterns*



The compressed data can be encoded in binary using a fixed number of bits per digit. For example, using 4 bits per digit, the compressed data can be represented as

1100 0011 0000 1000, in which the rate of compression is 26/16 or almost 1.62 for this example.

Dictionary Coding

There is a group of compression methods based on the creation of a dictionary (array) of strings in the text. The idea is to encode common sequences of characters instead of encoding each character separately. The dictionary is created as the message is scanned, and if a sequence of characters that is an entry in the dictionary is found in the message, the code (index) of that entry is sent instead of the sequence. The one we discuss here was invented by Lempel and Ziv and refined by Welch. It is referred to as Lempel-Ziv-Welch (LZW). The interesting point about this encoding technique is that the creation of the dictionary is dynamic. It is created by the sender and the receiver during the encoding and decoding processes; it is not sent from the sender to the receiver.

Encoding

The encoding process is as follows:

1. The dictionary is initialized with one entry for each possible character in the message (alphabet). At the same time, a buffer, which we call the *string*, is initialized to the first character in the message. The string holds the largest encodable sequence found so far. In the initialization step, only the first character in the message is encodable.
2. The process scans the message and gets the next character in the message.
   a. If the concatenation of the string and the scanned character is in the dictionary, the string is not the largest encodable sequence. The process updates the string by concatenating the character at the end of it and waits for the next iteration.
   b. If the concatenation of the string and the scanned character is not in the dictionary, the largest encodable sequence is the string, not the concatenation of the two. Three actions are taken. First, the process adds the concatenation of the two as the new entry to the dictionary. Second, the process encodes the string. Third, the process reinitializes the string with the scanned character for the next iteration.
3. The process repeats step 2 while there are more characters in the message.

Table 11.1 gives a pseudocode for the encoding process. We have called the next character *char* and the string *S* for simplicity.

Example 11.1

Figure 11.2 show an example of LZW encoding using a text message in which the alphabet is made up of two characters: A and B.

The figure shows how the text "BAABABBBAABBBBAA" is encoded as 1002163670. Note that the buffer PreS holds the string from the previous iteration before it is updated.

Decoding

The decoding process is as follows:

1. The dictionary is initialized as we explained in the encoding process. The first codeword is scanned, and, using the dictionary, the first character in the message is output.

Table 11.1 *LZW encoding*

```
LZWEncoding (message)
{
        Initialize (Dictionary)
        Char = Input (first character)
        S = char                                    // S is the encodable sequence
        while (more characters in message)
        {
                char = Input (next character);
                if ((S + char) is in Dictionary)    // S is not the encodable sequence
                {
                        S = S + char;
                }
                else                                // S is the encodable sequence
                {
        addToDictionary (S + char);
        Output (index of S in Dictionary);
        S = char;
                }
        }
        Output (index of S in Dictionary);
}
```

2. The process then creates a string and sets it to the previous scanned codeword. Now it scans a new codeword.
   a. If the codeword is in the dictionary, the process adds a new entry to the dictionary, which is the string concatenated with the first character from the entry related to the new codeword. It also outputs the entry related to the new codeword.
   b. If the codeword is not in the dictionary (which may happen occasionally), the process concatenates the string with the first character from the string and stores it in the dictionary. It also outputs the result of the concatenation.
3. The process repeats step 2 while there are more codewords in the code.

Table 11.2 shows the simplified algorithm for LZW decoding. We have used C for the codeword and S for the string.

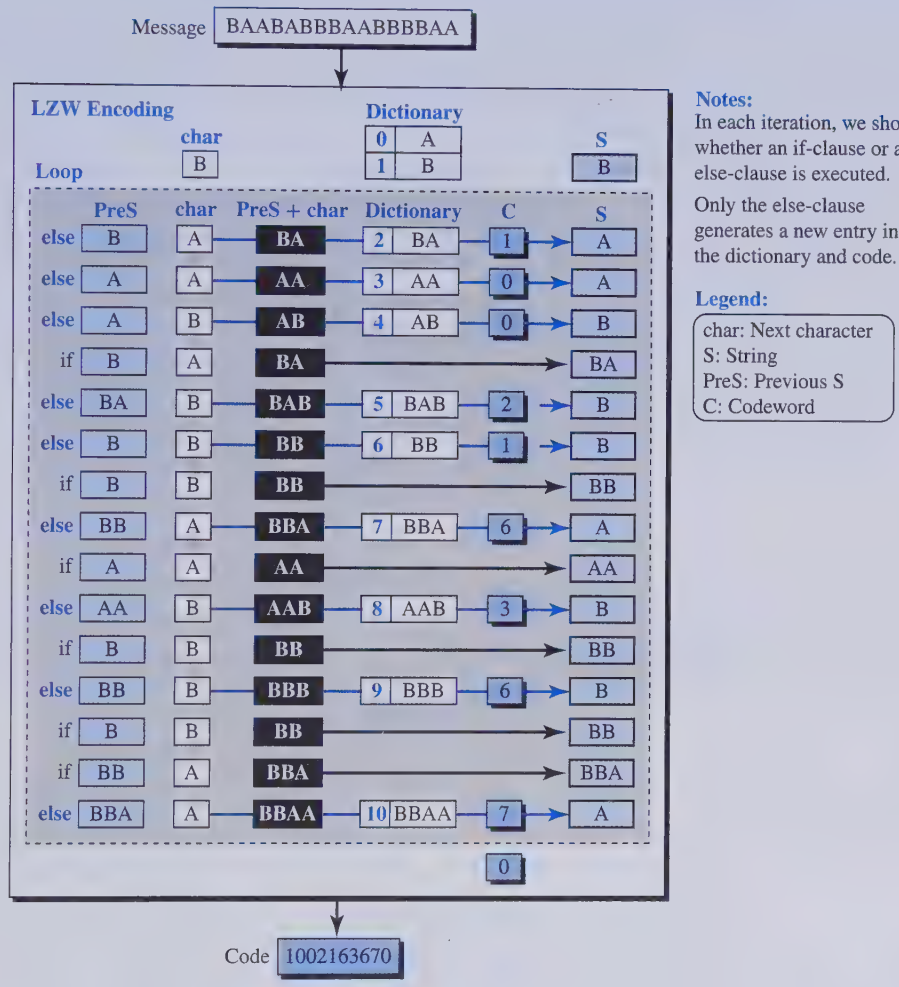
Example 11.2

Figure 11.3 shows how the code in Example 11.1 can be decoded and the original message recovered. The box called PreC holds the codeword from the previous iteration, which is not needed in the pseudocode but is needed here to better show the process. Note that in this example there is only the special case in which the codeword is not in the dictionary. The new entry for the dictionary needs to be made from the string and the first character in the string. The output is also the same as the new entry.

Huffman Coding

When we encode data as binary patterns, we normally use a fixed number of bits for each symbol. To compress data, we can consider the frequency of symbols and the

Figure 11.2 *Example 11.1*



probability of their occurrence in the message. **Huffman coding** assigns shorter codes to symbols that occur more frequently and longer codes to those that occur less frequently. For example, imagine we have a text file that uses only five characters (A, B, C, D, E) with the frequency of occurrence of (20, 10, 10, 30, 30).

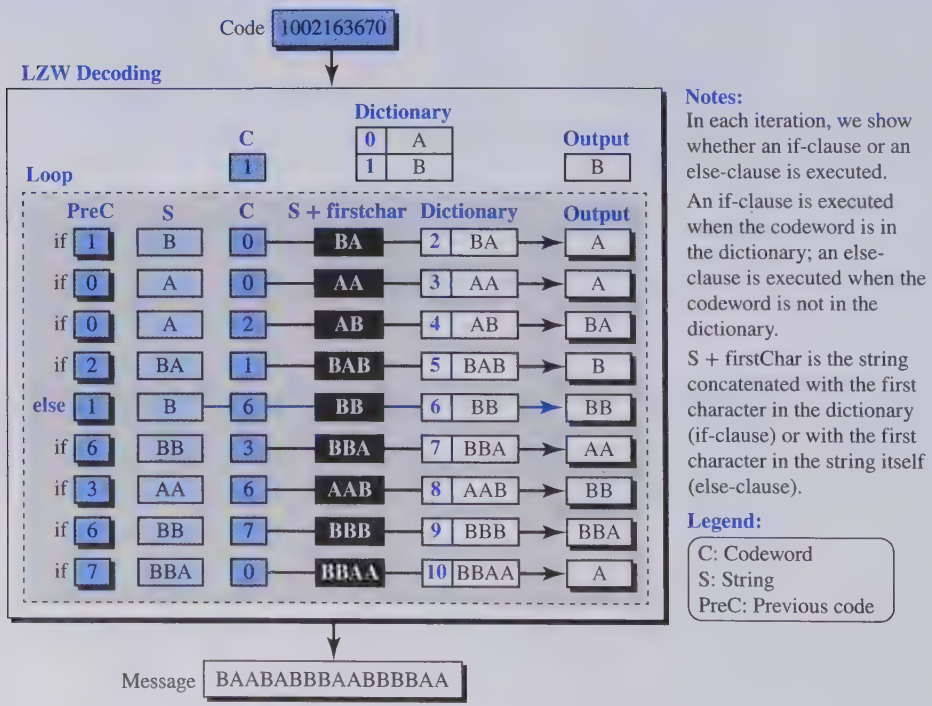
Huffman Tree

To use Huffman coding, we first need to create the Huffman tree. The Huffman tree is a tree in which the leaves of the tree are the symbols. It is made so that the most frequent symbol is the closest to the root of the tree (with the minimum number of nodes

Table 11.2 *LZW decoding*

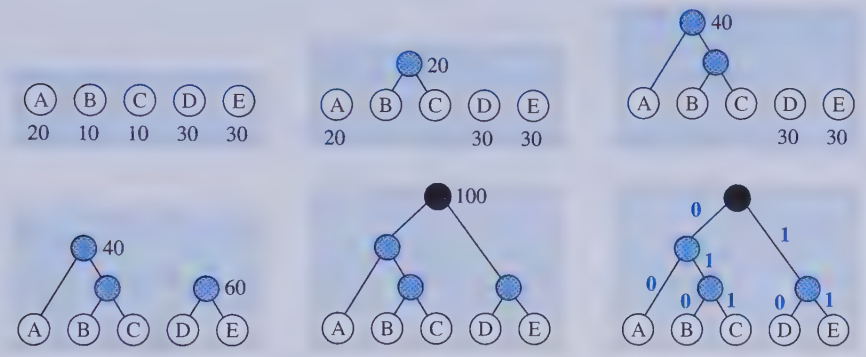
```
LZWDecoding (code)
{
        Initialize (Dictionary);
        C = Input (first codeword);
        Output (Dictionary [C]);
        while (more codewords in code)
        {
                S = Dictionary[C];
                C = Input (next codeword);
                if (C is in Dictionary)                    // Normal case
                {
                        addToDictionary (S + firstSymbolOf Dictionary[C]);
                        Output (Dictionary [C]);
                }
                else                                       // Special case
                {
                        addToDictionary (S + firstSymbolOf (S));
                        Output (S + firstSymbolOf (S));
                }
        }
}
```

Figure 11.3 *Example 11.2*



to the root) and the least frequent symbol is the farthest from the root. Figure 11.4 shows the process.

Figure 11.4 *Huffman tree*



1. We put the entire character set in a row. Each character is now a node at the lowest level of the tree.
2. We select the two nodes with the smallest frequencies and join them to form a new node, resulting in a simple two-level tree. The frequency of the new node is the combined frequencies of the original two nodes. This node, one level up from the leaves, is eligible for combination with other nodes.
3. We repeat step 2 until all the nodes, on every level, are combined into a single tree.
4. After the tree is made, we assign bit values to each branch. Because the Huffman tree is a binary tree, each node has a maximum of two children.

Coding Table

After the tree has been made, we can create a table that shows how each character can be encoded and decoded. The code for each character can be found by starting at the root and following the branches that lead to that character. The code itself is the bit value of each branch on the path, taken in sequence. Table 11.3 shows the character codes for our simple example.

Table 11.3 *Coding Table*

Symbol	*Code*	*Symbol*	*Code*	*Symbol*	*Code*
A	00	C	011	E	11
B	010	D	10		

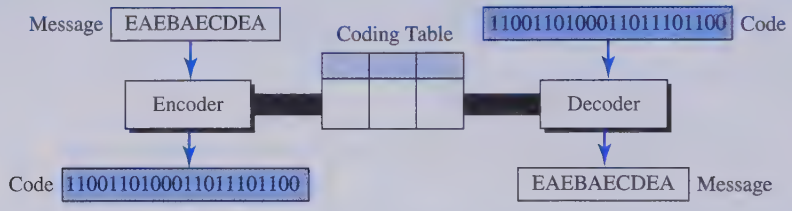
Note these points about the codes. First, the characters with higher frequencies receive a shorter code (A, D, and E) than the characters with lower frequencies (B and C).

Compare this with a code that assigns equal bit lengths to each character. Second, in this coding system, no code is a prefix of another code. The 2-bit codes, 00, 10, and 11, are not the prefixes of any of the two other codes (010 and 011). In other words, we do not have a 3-bit code beginning with 00, 10, or 11. This property makes the Huffman code an *instantaneous* code.

Encoding and Decoding

Figure 11.5 shows how we can encode and decode using Huffman coding. Note that we have achieved compression even with a small message. If we want to send fixed-length codes for a five-character alphabet, we need $\log_2 5 = 2.32$ or 3 bits for each character or 30 bits for the whole message. With Huffman coding we need only 22 bits. The compression ratio is 30/22 or 1.36.

Figure 11.5 *Encoding and decoding in Huffman coding*



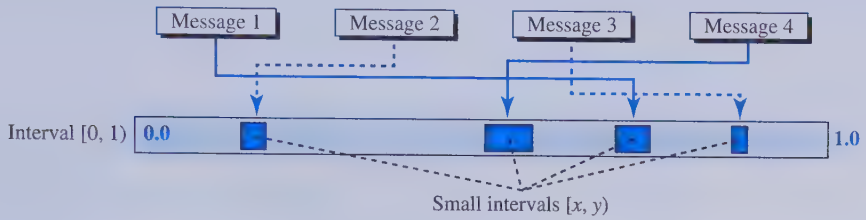
In Huffman coding, no code is the prefix of another code. This means that we do not need to insert delimiters to separate the code for one character from the code for the next. This property of Huffman coding also allows instantaneous decoding. When the decoder has the 2 bits 00, it can immediately decode it as character A; it does not need to see more bits.

One drawback of Huffman coding is that both the encoder and decoder need to use the same encoding table. In other words, the Huffman tree cannot be created dynamically like the dictionary can be in LZW coding. However, if the encoder and decoder use the same set of symbols all the time, the tree can be made and shared once. Otherwise, the table needs to be made by the encoder and then given to the receiver.

Arithmetic Coding

In the previous compression methods, each symbol or sequence of symbols is encoded separately. In **arithmetic coding**, introduced by Rissanen and Langdon in 1981, the entire message is mapped to a small interval inside [0,1). The small interval is then encoded as a binary pattern. Arithmetic coding is based on the fact that we can have an infinite number of small intervals inside the half-open interval [0,1). Each of these small intervals can represent one of the possible messages we can make using a finite set of symbols. Figure 11.6 shows the idea.

Figure 11.6 *Arithmetic coding*



Encoding

To encode a message in arithmetic coding, we first need to assign the probability of occurrence to each symbol. If we have M symbols in the alphabet (including the terminating symbol that we need for decoding), the probabilities are P_1, P_2, $\cdots$, P_M, in which $P_1 + P_2 + \cdots + P_M = 1.0$. Table 11.4 shows the encoding algorithm.

Table 11.4 *Arithmetic encoding*

```
ArithmeticEncoding (message)
{
    currentInterval = [0,1);
    while (more symbols in the message)
    {
        s = Input (next symbol);
        divide currentInterval into subintervals
        subInt = subinterval related to s
        currentInterval = subInt
    }
    Output (bits related to the currentInterval)
}
```

In each iteration of the loop, we divide the current interval into M subintervals, in which the length of each subinterval is proportional to the probability of the corresponding symbol. This is done to uniformly scatter the messages in the interval [0,1). We also preserve the order of the symbols in the new interval in each iteration.

The choice of bits selected for output depends on the implementation. Some implementations use the bits that represent the fractional part of the beginning interval.

Example 11.3

For the sake of simplicity, let us assume that our set of symbols is $S = \{A, B, *\}$, in which the asterisk is the terminating symbol. We assign the probability of occurrence for each symbol as

$$P_A = 0.4 \qquad P_B = 0.5 \qquad P_* = 0.1$$

Figure 11.7 *Example 11.3*

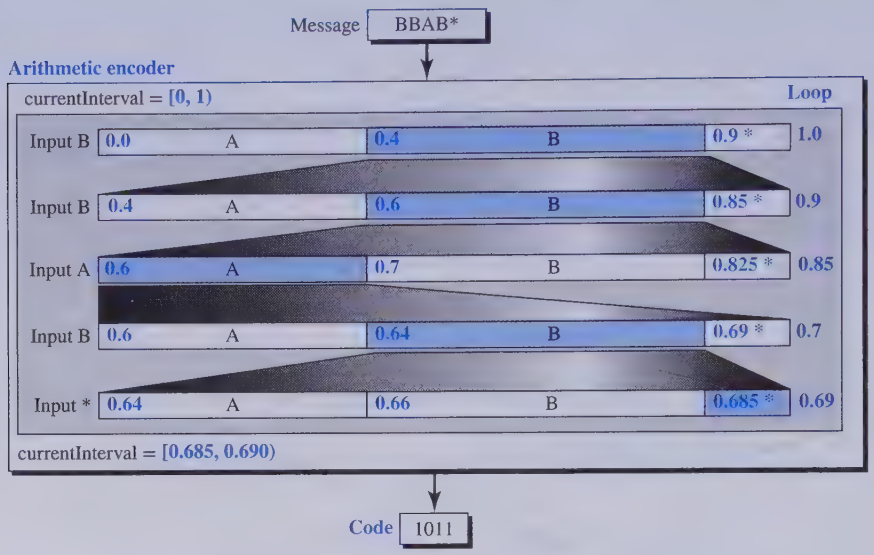


Figure 11.7 shows how we find the interval and the code related to the short message "BBAB*".

We initialize the current interval to [0,1). In each iteration of the loop, we divide the current interval into three subintervals according to the probability of each symbol occurring. We read the first symbol and choose the corresponding subinterval. We then set the current interval to be the chosen interval. The process is repeated until all symbols are input. After reading each symbol, the current interval is reduced until it becomes [0.685, 0.690). We encode the lower bound, 0.685 in binary, which is approximately $(0.1011)_2$, but we keep only the fractional part, $(1011)_2$, as the code. Note that when we change a real number between 0 and 1 to binary, we may get an infinite number of bits. We need to keep enough bits to recover the original message. More than enough bits is not efficient encoding; fewer than enough bits may result in wrong decoding. For example, if we use only 3 bits (101), it represents the real value 0.625, which is outside of the last current interval [0.685, 0.690).

Decoding

Decoding is similar to encoding, but we exit the loop when the terminating symbol is output. This is the reason we need the terminating symbol in the original message. Table 11.5 shows the decoding algorithm.

Example 11.4

Figure 11.8 shows how we use the decoding process to decode the message in Example 11.3. Note that the hand shows the position of the number in the corresponding interval.

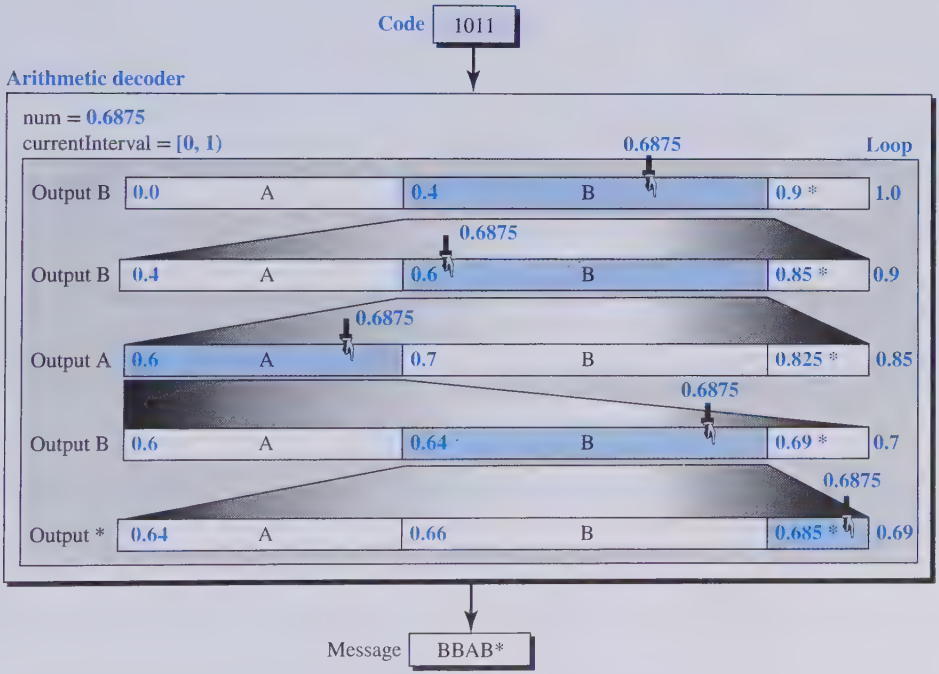
Static versus Dynamic Arithmetic Coding

The literature refers to two versions of arithmetic coding: *static coding* (sometimes called *pure coding*) and *dynamic coding* (sometimes called *interval coding*). The version

Table 11.5 *Arithmetic decoding*

```
ArithmeticDecoding (code)
{
        c = Input (code)
        num = find real number related to code
        currentInterval = [0,1);
        while (true)
        {
                divide the currentInterval into subintervals;
                subInt = subinterval related to num;
                Output (symbol related to subInt);
                if (symbol is the terminating symbol)          return;
                currentInterval = subInt;
        }
}
```

Figure 11.8 *Example 11.4*



we discussed in this section is the first one, static coding. There are two problems with static arithmetic coding. First, if the current interval is very small, we need very high precision arithmetic to encode the message, which results in a lot of 0 bits in the middle of the code. Second, the message cannot be encoded until all symbols are input; this is the reason that we need a terminating symbol for decoding. The new version, the

dynamic arithmetic coding, overcomes these two problems by using a procedure that outputs binary bits immediately after each symbol is read.

11.1.2 Lossy Compression

Lossless compression has limits on the amount of compression. However, in some situations, we can sacrifice some accuracy to increase the compression rate. Although we cannot afford to lose information in text compression, we can afford it when we are compressing images, video, and audio. For example, human vision cannot detect some small distortions that can result from **lossy compression** of an image. In this section, we discuss a few ideas behind lossy compression. In Section 11.2, we show how these ideas can be used in the implementation of image, video, and audio compression.

Predictive Coding

Predictive coding is used when we digitize an analog signal. In Chapter 2, we discussed pulse code modulation (PCM) as a technique that converts an analog signal to a digital signal, using sampling. After sampling, each sample needs to be quantized to create binary values. Compression can be achieved in the quantization step by using *predictive coding*.

In PCM, samples are quantized separately. The neighboring quantized samples, however, are closely related and have similar values. In **predictive coding**, we use this similarity. Instead of quantizing each sample separately, the differences are quantized. The differences are smaller than the actual samples and thus require fewer bits. Many algorithms are based on this principle. We start with the simplest one and move to more sophisticated ones.

Delta Modulation

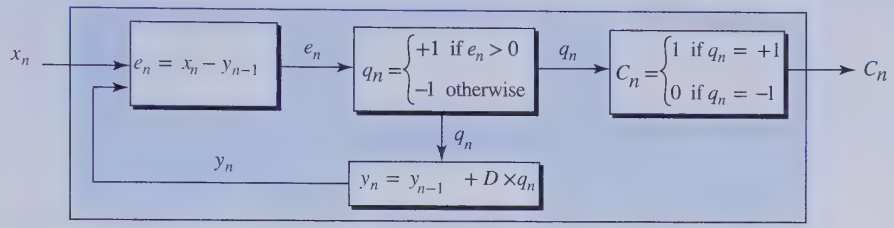
The simplest method in predictive coding is called **delta modulation (DM)**. Let xn represent the value of the original function at sampling interval n, and let y_n be the reconstructed value of x_n. Figure 11.9 shows the encoding and decoding processes in DM. In PCM, the sender quantizes the samples (x_n) and transmits them to the receiver. In DM, the sender quantizes en, the difference between each sample (x_n) and the preceding reconstructed value (y_{n-1}).

The sender then transmits C_n. The receiver reconstructs sample y_n from the received C_n.

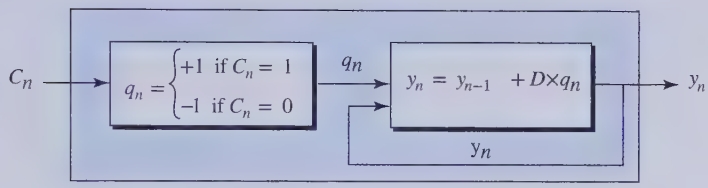
Note that for each sample, PCM needs to transmit several bits. For example, it needs to transmit 3 bits for each sample if the maximum quantized value is 7 (see Chapter 2). DM reduces the number of transmitted bits because it transmits a single bit (1 or 0) for each sample.

We may ask why DM quantizes the difference $x_n - y_{n-1}$ instead of $x_n - x_{n-1}$. The reason is that the second choice makes y vary much faster than x if x is a slow-changing function. Quantizing $x_n - y_{n-1}$ is self-correcting for slow-growing or slow-falling x. Figure 11.10 compares the staircase reconstruction of quantizing $x_n - x_{n-1}$ versus $x_n - y_{n-1}$ for a slow-growing function. (For a slow-falling function, the idea is similar.)

Figure 11.9 *Encoding and decoding in delta modulation*



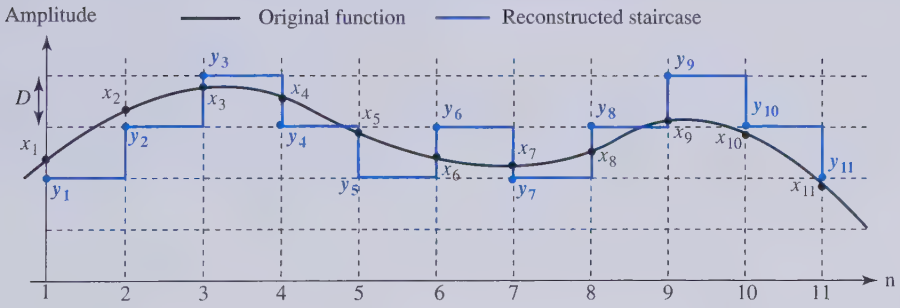
a. Encoder



b. Decoder

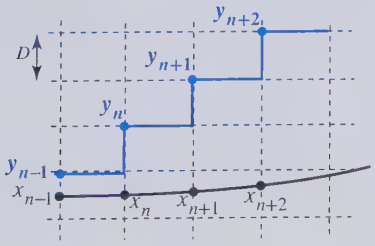
Notes

1. y_n becomes y_{n-1} in the next iteration.
2. y_0 and D are given.

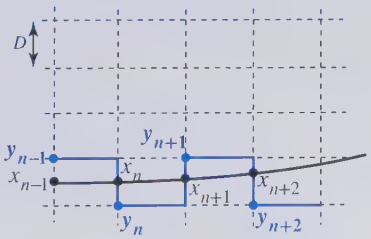


c. Relationship between x_n and y_n

Figure 11.10 *Reconstruction of quantization of $x_n - x_{n-1}$ versus $x_n - y_{n-1}$*



a. Quantizing $q_n = x_n - x_{n-1}$

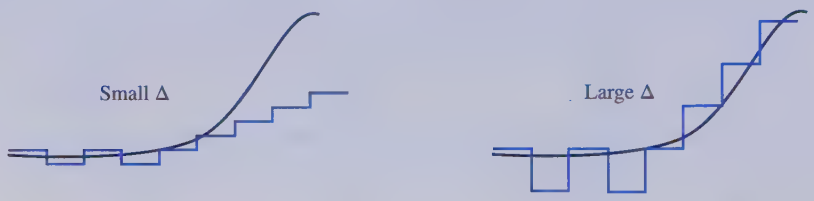


b. Quantizing $q_n = x_n - y_{n-1}$

Adaptive DM

Figure 11.11 shows the role of quantizer Δ on delta modulation. In the region where Δ is relatively small compared to the slope of the original function, the reconstructed staircase cannot catch up with the original function; the result is an error known as *slope overload distortion*. On the other hand, in the region where Δ is relatively large compared to the slope of the original function, the reconstructed staircase continues to oscillate largely around the original function and causes an error known as *granular noise*.

Figure 11.11 *Slope overload and granular noise*



Because most functions have regions with both large and small slopes, selecting a large value or a small value for Δ decreases one type of error but increases the other type. The **adaptive DM (ADM)** is used to solve the problem. In ADM, the value of Δ changes from one step to the next and is calculated as

$$\Delta_n = M_n \Delta_{n-1}$$

where M_n is called the step-size multiplier and is calculated from the values of q_n from a few previous bits. There are many different algorithms for evaluating M_n; one simple algorithm is to increase M_n by a certain percentage if q_n remains the same and decrease it by a certain percentage if q_n changes. The adaptation can be further improved by delaying the coding process to include knowledge about a few future samples in the evaluation of M_n.

Differential PCM

The **differential PCM (DPCM)** is the generalization of delta modulation. In delta modulation, a previously reconstructed sample y_{n-1} is called the *predictor* because it is used to predict the current value. In DPCM, more than one previously reconstructed sample is used for prediction. In this case the difference is evaluated as

$$e_n = x_n - \sum_{i=1}^{N} a_i y_{i-1}$$

where the summation is the predictor, a_i is the *predictor's coefficient* (or weight), and N is the *order of the predictor*. For DM, the order of the predictor is 1 and $a_1 = 1$. The difference is quantized as in DM and sent to the receiver. The receiver reconstructs the current value as

$$y_n = \sum_{i=1}^{N} a_i y_{i-1} + \Delta q_n$$

Predictor coefficients are found by minimizing the cumulative error between the predicted value and the actual value. The optimization uses the *method of square error*, which is beyond the scope of this book.

Adaptive DPCM

Further compression can be achieved by using different coefficients for different regions of the sample or by adjusting the quantizer (Δ) from one step to the next or by doing both. This is the principle behind **adaptive DPCM** (**ADPCM**).

Linear Predictive Coding

In **linear predictive coding** (**LPC**), instead of sending quantized difference signals, the source analyzes the signals and determines their characteristics. The characteristics include frequencies in the sensitive range of frequencies, the power of each frequency, and the duration of each signal. The source then quantizes this information and transmits it to the receiver. The receiver feeds this information into a signal synthesizer to simulate a signal similar to that of the original one. The LPC can achieve a high level of compression. However, this method is normally used by the military for compressing speech. In this case, the synthesized speech, though intelligible, lacks naturalness and quality to identify the speaker.

Transform Coding

In transform coding, a mathematical transformation is applied to the input signal to produce the output signal. The transformation needs to be invertible, to allow the original signal to be recovered. The transformation changes the signal representation from one domain to another (time domain to frequency domain, for example), which results in a reduction of the number of bits in encoding.

We need to emphasize that transformation techniques used in multimedia are lossless per se. However, to achieve the compression goals, another step, quantization, is added to the operation, which makes the whole process lossy.

Discrete Cosine Transform

One of the popular transformations used in multimedia is called **discrete cosine transform** (**DCT**). Although we use two-dimensional DCT in multimedia compression, we first discuss one-dimensional DCT, which is easier to understand.

One-Dimensional DCT In one-dimensional DCT, the transformation is the matrix multiplication of a column matrix **p** (source data) by a square matrix **T** (DCT coefficient). The result is a column matrix **M** (transformed data). Because the square matrix that represents the DCT coefficient is an orthogonal matrix (inverse and transpose are the same), the inverse transformation can be obtained by multiplication of the transformed data matrix by the transpose matrix of the DCT coefficient. Figure 11.12 (on next page) shows the transformation matrix, in which N is the size of matrix **T** and $\mathbf{T}^T$ is the transpose matrix of **T**.

Although we believe the matrix representation of the transformation is easier to understand, the literature also uses two formulas to do so, as shown in Figure 11.13 (on next page).

Figure 11.12 *One-dimensional DCT*

$$\mathbf{M} = \mathbf{T} \times \mathbf{p} \qquad \mathbf{p} = \mathbf{T}^T \times \mathbf{M}$$

a. Transformation b. Inverse transformation

$$\mathbf{T}(m, n) = \mathbf{C}(m) \cos\left[\frac{\pi n(2m+1)}{2N}\right]$$

for $m = 0$ to $N-1$
for $n = 0$ to $N-1$

$$\mathbf{C}(m) = \begin{cases} \sqrt{\frac{1}{N}} & m = 0 \\ \sqrt{\frac{2}{N}} & m > 0 \end{cases}$$

Figure 11.13 *Formulas for one-dimensional forward and inverse transformation*

$$\mathbf{M}(m) = \sum_{n=0}^{N-1} \mathbf{C}(m) \cos\left[\pi n(2m+1)/(2N)\right] \times \mathbf{p}(n) \qquad \text{for } m = 0, \ldots, N-1$$

$$\mathbf{p}(n) = \sum_{m=0}^{N-1} \mathbf{C}(n) \cos\left[\pi m(2n+1)/(2N)\right] \times \mathbf{M}(m) \qquad \text{for } n = 0, \ldots, N-1$$

$$\mathbf{C}(i) = \begin{cases} \sqrt{\frac{1}{N}} & i = 0 \\ \sqrt{\frac{2}{N}} & i > 0 \end{cases}$$

Example 11.5

Figure 11.14 shows the transformation matrix for $N = 4$. As the figure shows, the first row has four equal values, but the other rows have alternate positive and negative values. When each row is multiplied by the source data matrix, we expect that the positive and negative values will result in values close to zero if the source data items are close to each other. This is what we expect from the transformation: to show that only some values in the source data are important and most values are redundant.

Figure 11.14 *Example 11.5*

$$\begin{bmatrix} 203 \\ -2.22 \\ 0.00 \\ -0.16 \end{bmatrix} = \begin{bmatrix} 0.50 & 0.50 & 0.50 & 0.50 \\ 0.65 & 0.27 & -0.27 & -0.65 \\ 0.50 & -0.50 & -0.50 & 0.50 \\ 0.27 & -0.65 & 0.65 & -0.27 \end{bmatrix} \times \begin{bmatrix} 100 \\ 101 \\ 102 \\ 103 \end{bmatrix}$$

$\mathbf{M}$ $\mathbf{T}$ $\mathbf{p}$

a. Transformation

$$\begin{bmatrix} 100 \\ 101 \\ 102 \\ 103 \end{bmatrix} = \begin{bmatrix} 0.50 & 0.65 & 0.50 & 0.27 \\ 0.50 & 0.27 & -0.50 & -0.65 \\ 0.50 & -0.27 & -0.50 & 0.65 \\ 0.50 & -0.65 & 0.50 & -0.27 \end{bmatrix} \times \begin{bmatrix} 203 \\ -2.22 \\ 0.00 \\ -0.16 \end{bmatrix}$$

$\mathbf{p}$ $\mathbf{T}^T$ $\mathbf{M}$

b. Inverse transformation

This example shows how we can transform the sequence of numbers, (100, 101, 102, 103) to another sequence of numbers (203, −2.22, 0.00, −0.16). There are several points we want to mention about the DCT transformation to better explain its properties. First, the transformation is reversible. Second, the transformation matrix is orthogonal ($\mathbf{T}^{-1} = \mathbf{T}^T$), which means that we don't need to use the inverse matrix in calculation of the reverse transformation; the transposed matrix can be used (faster calculation). Third,

the first row of the **M** matrix always is the weighted average of the **p** matrix. Fourth, the other row values in the matrix are very small values (positive or negative) that can be ignored in this case. The very important point about these three values is that they will be the same if we change the four values in the **p** matrix, but keep the same correlation between them. If we change the source data **p** = (7, 8, 9, 10), we can show that the transformed data will be **M** = (17, −2.22, 0.00, −0.16); the first value will change because the average has changed, but the rest of the values will not change because the relationship between the data items has not changed. This is what we expect from the transformation. It removes the redundant data. The last three values in the **p** matrix are redundant; they have a very close relationship with the first value.

Two-Dimensional DCT Two-dimensional DCT is what we need for compressing images, audio, and video. The principle is the same, except that the source data and transformed data are two-dimensional square matrices. To achieve the transformation with the same properties as mentioned for the one-dimensional DCT, we need to use the **T** matrix twice (**T** and $\mathbf{T}^T$). The inverse transformation also uses the **T** matrix twice, but in the reverse order. Figure 11.15 shows the two-dimensional DCT, in matrix format. Figure 11.16 shows the same idea using two formulas.

Figure 11.15 *Two-dimensional DCT*

$$\begin{bmatrix} \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{T} \end{bmatrix} \times \begin{bmatrix} \mathbf{p} \end{bmatrix} \times \begin{bmatrix} \mathbf{T}^T \end{bmatrix} \qquad \mathbf{T}(m, n) = \mathbf{C}(m, n) \cos\left[\frac{m\pi(2n+1)}{2N}\right] \quad \begin{matrix} \text{for } m = 0 \text{ to } N-1 \\ \text{for } n = 0 \text{ to } N-1 \end{matrix}$$

a. Transformation

$$\begin{bmatrix} \mathbf{p} \end{bmatrix} = \begin{bmatrix} \mathbf{T}^T \end{bmatrix} \times \begin{bmatrix} \mathbf{M} \end{bmatrix} \times \begin{bmatrix} \mathbf{T} \end{bmatrix} \qquad \mathbf{C}(m, n) = \begin{cases} \sqrt{\frac{1}{N}} & m = 0 \\ \sqrt{\frac{2}{N}} & m > 0 \end{cases}$$

b. Inverse transformation

Figure 11.16 *Formulas for forward and inverse two-dimensional DCT*

$$\mathbf{M}(m, n) = \frac{2}{N} \mathbf{C}(m)\, \mathbf{C}(n) \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} \mathbf{p}(k, l) \cos\left[\frac{m\pi(2k+1)}{2N}\right] \cos\left[\frac{n\pi(2l+1)}{2N}\right] \quad \begin{matrix} \text{for } m = 0, \ldots, N-1 \\ \text{for } n = 0, \ldots, N-1 \end{matrix}$$

$$\mathbf{p}(k, l) = \frac{2}{N} \mathbf{C}(l)\, \mathbf{C}(k) \sum_{m=0}^{N-1} \sum_{n=0}^{N-1} \mathbf{M}(m, n) \cos\left[\frac{k\pi(2m+1)}{2N}\right] \cos\left[\frac{l\pi(2n+1)}{2N}\right] \quad \begin{matrix} \text{for } k = 0, \ldots, N-1 \\ \text{for } l = 0, \ldots, N-1 \end{matrix}$$

$$\mathbf{C}(u) = \begin{cases} \sqrt{\frac{1}{2}} & u = 0 \\ 1 & u > 0 \end{cases}$$

11.2 MULTIMEDIA DATA

Today, multimedia data consists of *text*, *images*, *video*, and *audio,* although the definition is changing to include futuristic media types.

11.2.1 Text

The Internet stores a large amount of text that can be downloaded and used. One often refers to plaintext, as a linear form, and hypertext, as a nonlinear form, of textual data. Text stored in the Internet uses a character set, such as Unicode, to represent symbols in the underlying language. To store a large amount of textual data, one can compress the text using one of the lossless compression methods we discussed in Section 11.1.1. Note that we need to use lossless compression because we cannot afford to lose any pieces of information when we perform decompression.

11.2.2 Image

In multimedia parlance, an image (or a still image, as it is often called) is the representation of a photograph, a fax page, or a frame in a moving picture.

Digital Image

Before it can be used, an image first must be digitized. *Digitization* in this case means to represent the image as a two-dimensional array of dots, called pixels. Each pixel then can be represented as a number of bits, referred to as the *bit depth*. In a black-and-white image, such as a fax page, the bit depth = 1; each pixel can be represented as a 0-bit (black) or a 1-bit (white). In a gray picture, one normally uses a bit depth of 8 with 256 levels. A color image is normally divided into three channels, with each channel representing one of the three primary colors of red, green, or blue (RGB). In this case, the bit depth is 24 (8 bits for each color). Some representations use a separate channel, called the *alpha* (α) *channel*, to represent the background. In a black-and-white image, this results in two channels; in a color image, this results in four channels.

It is obvious that moving from a black-and-white, to a gray, to a color representation of images tremendously increases the size of the information transmitted on the Internet. This implies that we need to compress images to save time.

Example 11.6

The following shows the time required to transmit an image of 1280×720 pixels using the transmission rate of 100 kbps.

a. Using a black-and-white image with a bit depth of 1,

$$\textbf{Transmission time} = (1280 \times 720 \times 1) / 100{,}000 \approx 9 \text{ s}$$

b. Using a gray image with a bit depth of 8,

$$\textbf{Transmission time} = (1280 \times 720 \times 8) / 100{,}000 \approx 74 \text{ s}$$

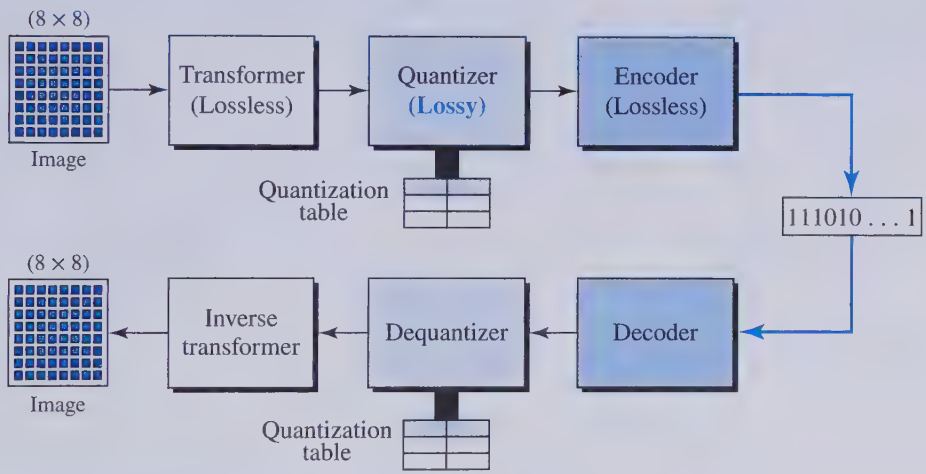
c. Using a color image with a bit depth of 24,

$$\textbf{Transmission time} = (1280 \times 700 \times 24) / 100{,}000 \approx 215 \text{ s}$$

Image Compression: JPEG

Although there are both lossless and lossy compression algorithms for images, in this section we discuss the lossy compression method called *JPEG*. The **Joint Photographic Experts Group (JPEG)** standard provides lossy compression that is used in most implementations. The JPEG standard can be used for both color and gray images. However, for simplicity, we discuss only the grayscale pictures; the method can be applied to each of the three channels in a color image. In JPEG, a grayscale picture is divided into blocks of 8×8 pixels. During compression and decompression, each goes through three steps, as shown in Figure 11.17.

Figure 11.17 *Compression in each channel of JPEG*



The purpose of dividing the picture into blocks is to decrease the number of calculations, because, as we showed in two-dimensional DCT, the number of mathematical operations for each picture is the square of the number of units.

Transformation

JPEG normally uses DCT in the first step in compression and inverse DCT in the last step in decompression. Transformation and inverse transformation are applied on 8×8 blocks. We discussed DCT in Section 11.1.2.

Quantization

The output of DCT transformation is a matrix of real numbers. The precise encoding of these real numbers requires a lot of bits. JPEG uses a quantization step that not only rounds real values in the matrix, but also changes some values to zeros. The zeros can be eliminated in the encoding step to achieve a high compression rate. As previously discussed, the result of DCT transformation defines the weights of different frequencies in the source matrix. Because high frequencies mean sudden changes in the value of pixels, the high frequencies can be eliminated because human vision cannot recognize

them. The quantization step creates a new matrix in which each element, $\mathbf{C}(m, n)$, is defined as

$$\mathbf{C}(m, n) = \textbf{round}\,[\mathbf{M}(m, n) / \mathbf{Q}(m, n)]$$

in which $\mathbf{M}(m, n)$ is an entry in the transformed matrix and $\mathbf{Q}(m, n)$ is an entry in the quantization matrix. The round function first adds 0.5 to a real value and then truncates the value to an integer. This means that 3.7 is rounded to integer 4, but 3.2 is rounded to integer 3.

JPEG has defined 100 quantization matrices, **Q1** to **Q100**, in which **Q1** gives the poorest image quality but the highest level of compression and **Q100** gives the best image quality but the lowest level of compression. It is up to the implementation to choose one of these matrices. Figure 11.18 shows some of these matrices.

Figure 11.18 *Three different quantization matrices*

$$\begin{bmatrix} 80 & 60 & 50 & 80 & 120 & 200 & 255 & 255 \\ 55 & 60 & 70 & 95 & 130 & 255 & 255 & 255 \\ 70 & 65 & 80 & 120 & 200 & 255 & 255 & 255 \\ 70 & 85 & 110 & 145 & 255 & 255 & 255 & 255 \\ 90 & 110 & 185 & 255 & 255 & 255 & 255 & 255 \\ 120 & 175 & 255 & 255 & 255 & 255 & 255 & 255 \\ 245 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 & 255 & 255 & 255 \end{bmatrix}$$

Q10

$$\begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 110 & 103 & 99 \end{bmatrix}$$

Q50

$$\begin{bmatrix} 3 & 2 & 2 & 3 & 5 & 8 & 10 & 12 \\ 2 & 2 & 3 & 4 & 5 & 12 & 12 & 11 \\ 3 & 3 & 3 & 5 & 8 & 11 & 14 & 11 \\ 3 & 3 & 4 & 6 & 10 & 17 & 16 & 12 \\ 4 & 4 & 7 & 11 & 14 & 22 & 21 & 15 \\ 5 & 7 & 11 & 13 & 16 & 12 & 23 & 18 \\ 10 & 13 & 16 & 17 & 21 & 24 & 24 & 21 \\ 14 & 18 & 19 & 20 & 22 & 20 & 20 & 20 \end{bmatrix}$$

Q90

Note that the only phase in the process that is not completely reversible is the quantizing phase. We lose some information here that is not recoverable. As a matter of fact, the only reason that JPEG is called *lossy compression* is because of this quantization phase.

Encoding

After quantization, the values are reordered in a zigzag sequence before being input into the encoder. The zigzag reordering of the quantized values is done to let the values related to the lower frequency feed into the encoder before the values related to the higher frequency. Because most of the higher-frequency values are zeros, this means nonzero values are given to the encoder before the zero values. Figure 11.19 shows the process.

The encoding in this case is a lossless compression using either run-length coding or arithmetic coding.

Example 11.7

To show the idea of JPEG compression, we use a block of gray image in which the bit depth for each pixel is 20. We used a Java program to transform, quantize, and reorder the values in a zigzag sequence. Figure 11.20 shows the encoding.

Example 11.8

As the second example, we have a block that changes gradually; there is no sharp change between the values of neighboring pixels. We still get a lot of zero values, as shown in Figure 11.21.

Figure 11.19 *Reading the table*

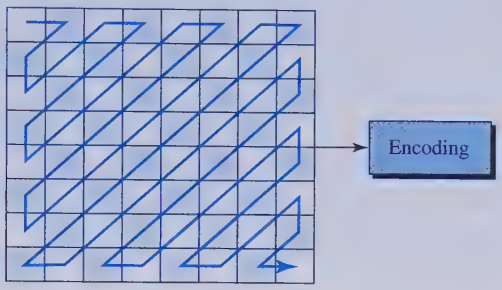


Figure 11.20 *Example 11.7: uniform gray scale*

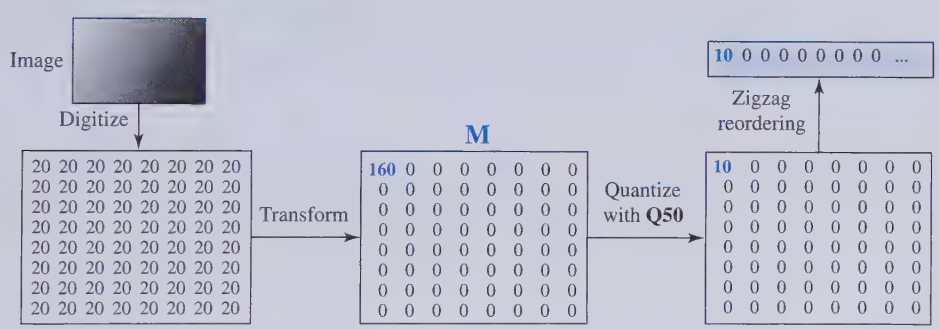


Figure 11.21 *Example 11.8: gradient gray scale*

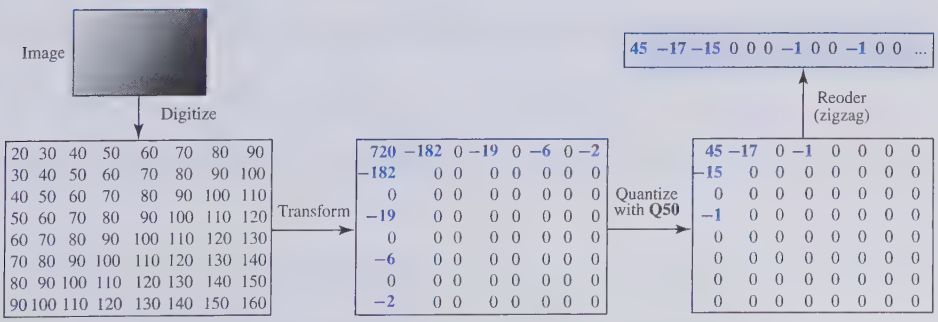


Image Compression: GIF

The JPEG standard uses images in which each pixel is represented as 24 bits (8 bits for each primary color). This means that each pixel can be one of the 2^{24} (16,777,216) complex colors. For example, a *magenta* pixel, which is made up of red and blue components (but contains no green component) is represented as the integer $(FF00FF)_{16}$.

Most simple graphic images do not contain such a large range of colors. The Graphic Interchange Format (GIF) uses a smaller palette (indexed table) of colors with normally $2^8 = 256$ colors. In other words, GIF maps a *true* color with a *palette* color. For example, a magenta pixel can be represented as the integer $(E2)_{16}$ if it is the 226th color in the pallet. This means that GIF reduces the size of the image by a factor of 3 compared with JPEG.

After creating the palette for a particular image, each pixel can be represented by one of the 256 symbols (for example, the two-digit representation of the palette index in hexadecimal). Now we can use one of the lossless compression methods, such as dictionary coding or arithmetic coding, to further compress the image.

11.2.3 Video

Video is composed of multiple frames; each frame is one image. This means that a video file requires a high transmission rate.

Digitizing Video

A video consists of a sequence of frames. If the frames are displayed on the screen quickly enough, we get an impression of motion. The reason is that our eyes cannot distinguish the rapidly flashing frames as individual ones. There is no standard number of frames per second; in North America, 25 frames per second is common. However, to avoid a condition known as *flickering*, a frame needs to be refreshed. The TV industry repaints each frame twice. This means 50 frames need to be sent, or if there is memory at the sender site, 25 frames with each frame repainted from the memory.

Example 11.9

The transmission rate for some video standards are as follows:

a. Color broadcast television takes 720 × 480 pixels per frame, 30 frames per second, and 24 bits per color. The transmission rate without compression is

$$720 \times 480 \times 30 \times 24 = 248{,}832{,}000 \text{ bps} = 249 \text{ Mbps}$$

b. High-definition color broadcast television takes 1920 × 1080 pixels per frame, 30 frames per second, and 24 bits per color: The transmission rate without compression is

$$1920 \times 1080 \times 30 \times 24 = 1{,}492{,}992{,}000 \text{ bps} = 1.5 \text{ Gbps}$$

Video Compression: MPEG

Motion Picture Experts Group (MPEG) is a method to compress video. In principle, a motion picture is a rapid flow of a set of frames, where each frame is an image. In other words, a frame is a spatial combination of pixels, and a video is a temporal

combination of frames that are sent one after another. Compressing video, then, means spatially compressing each frame and temporally compressing a set of frames.

Spatial Compression

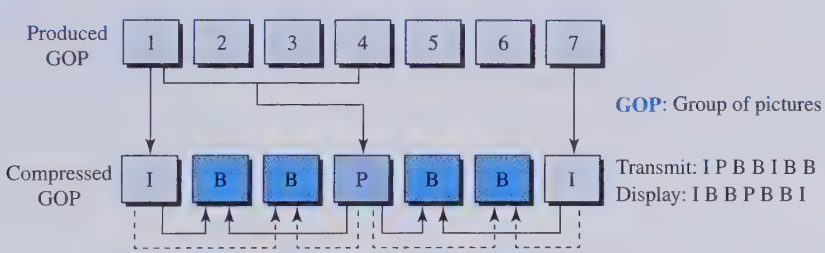
The **spatial compression** of each frame is done with JPEG (or a modification of it). Each frame is a picture that can be independently compressed.

Temporal Compression

In **temporal compression**, redundant frames are removed. When we watch television, we receive 50 frames per second. However, most of the consecutive frames are almost the same. For example, when someone is talking, most of the frame is the same as the previous one except for the segment of the frame around the lips, which changes from one frame to another.

To temporally compress data, the MPEG method first divides a set of frames into three categories: I-frames, P-frames, and B-frames. Figure 11.22 shows how a set of frames (seven in the figure) are compressed to create another set of frames.

Figure 11.22 *MPEG frames*



- **I-frames.** An **intracoded frame (I-frame)** is an independent frame that is not related to any other frame (not to the frame sent before or after). They are present at regular intervals. An **I-frame** must appear periodically to handle some sudden change in the frame that the previous and following frames cannot show. Also, when a video is broadcast, a viewer may tune in at any time. If there is only one I-frame at the beginning of the broadcast, the viewer who tunes in late will not receive a complete picture. I-frames are independent of other frames and cannot be constructed from other frames.
- **P-frames.** A **predicted frame (P-frame)** is related to the preceding I-frame or P-frame. In other words, each P-frame contains only the changes from the preceding frame. P-frames can be constructed only from previous I- or P-frames. P-frames carry much less information than other frame types and carry even fewer bits after compression.
- **B-frames.** A **bidirectional frame (B-frame)** is related to the preceding and following I-frame or P-frame. In other words, each B-frame is relative to the past and the future. Note that a B-frame is never related to another B-frame.

11.2.4 Audio

Audio (sound) signals are analog signals that need a medium in which to travel; they cannot travel through a vacuum. The speed of the sound in the air is about 330 m/s (740 mph). The audible frequency range for normal human hearing is from about 20 Hz to 20 kHz with maximum audibility around 3300 Hz.

Digitizing Audio

To be able to provide compression, audio analog signals are digitized using an analog-to-digital converter. The analog-to-digital conversion consists of two processes: sampling and quantizing. A digitizing process known as pulse code modulation (PCM) was discussed in detail in Chapter 2. This process involves sampling an analog signal, quantizing the sample, and coding the quantized values as streams of bits. Voice signal is sampled at the rate of 8000 samples per second with 8 bits per sample; the result is a digital signal of $8000 \times 8 = 64$ kbps. Music is sampled at 44,100 samples per second with 16 bits per sample; the result is a digital signal of $44{,}100 \times 16 = 705.6$ kbps for monaural and 1.411 Mbps for stereo.

Audio Compression

Both lossy and lossless compression algorithms are used in audio compression. Lossless audio compression allows one to preserve an exact copy of the audio files; it has a small compression ratio of about 2 and is mostly used for archival and editing purposes. Lossy algorithms provide far greater compression ratios (5 to 20) and are used in mainstream consumer audio devices. Lossy algorithms sacrifice a little bit of quality, but substantially reduce space and bandwidth requirements. For example, on a CD, one can fit 1 hour of high-fidelity music, 2 hours of music using lossless compression, or 8 hours of music compressed with a lossy technique.

Compression techniques used for speech and music have different requirements. Compression techniques used for speech must have low latency because significant delays degrade the communication quality in telephony. Compression algorithms used for music must be able to produce high-quality sound with lower numbers of bits. Two categories of techniques are used in audio compressions: *predictive coding* and **perceptual coding**.

Predictive coding

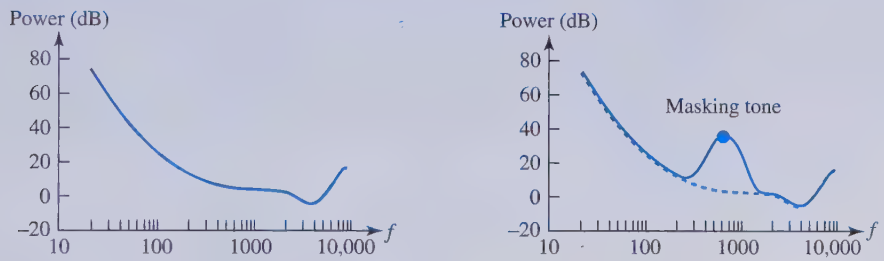
Predictive coding techniques have low latency and therefore are popular in speech coding for telephony where significant delays degrade the communication quality. We discussed several predictive coding methods at the beginning of this chapter: DM, ADM, DPCM, ADPCM, and LPC.

Perceptual Coding

Even at their best, the predictive coding methods cannot sufficiently compress a CD-quality audio for the multimedia application. The most common compression technique used to create CD-quality audio is perceptual coding, which is based on the science of **psychoacoustics**. Algorithms used in perceptual coding first transform the data from time domain to frequency domain; the operations are then performed on the data in the frequency domain. This technique, hence, is also called the *frequency-domain method*.

Psychoacoustics is the study of subjective human perception of sound. Perceptual coding takes advantage of flaws in the human auditory system. The lower limit of human audibility is 0 dB. This is only true for sounds with frequencies of about 2.5 to 5 kHz. The lower limit is less for frequencies between these two frequencies and rises for frequencies outside these ranges, as shown in Figure 11.23a. We cannot hear any frequency whose power is below this curve; thus, it is not necessary to code such a frequency. For example, we can save bits, without loss of quality, by omitting any sound with a frequency of less than 100 Hz if its power is below 20 dB.

Figure 11.23 *Threshold of audibility*



We can save even more using the concepts of **frequency masking** and **temporal masking**. Frequency masking occurs when a loud sound partially or totally masks a softer sound if the frequencies of the two are close to each other. For example, we cannot hear our dance partner in a room where a loud heavy-metal band is performing. In Figure 11.23b, a loud masking tone, around 700 Hz, raises the threshold of the audibility curve between frequencies of about 250 to 1500 Hz. In temporal masking, a loud sound can numb our ear for a short time even after the sound has stopped.

The basic approach to perceptual coding is to feed the audio PCM input into two separate units of the coder simultaneously. The first unit consists of an array of digital bypass filters called an *analysis filter bank*. Using a mathematical tool such as *discrete Fourier transform (DFT)*, the filters break the time-domain input into equally spaced frequency sub-bands. Using the same or a similar mathematical tool such as *fast Fourier transform (FFT)*, the second unit transforms the time-domain input into frequency-domain input and determines the masking frequency for each sub-band. The available bits are then allocated according to the masking property of each sub-band: no bits are allocated to a totally masked sub-band; a small number of bits are allocated to a partially masked sub-band, and a large number of bits are allocated to unmasked sub-bands. The resulting bits are further encoded to achieve more compression.

MP3

One standard that uses perceptual coding is **MP3 (MPEG audio layer 3)**.

11.3 MULTIMEDIA IN THE INTERNET

We can divide audio and video services into three broad categories: *streaming stored audio/video*, *streaming live audio/video*, and *interactive audio/video*. Streaming means a user can listen (or watch) the file after the downloading has started.

11.3.1 Streaming Stored Audio/Video

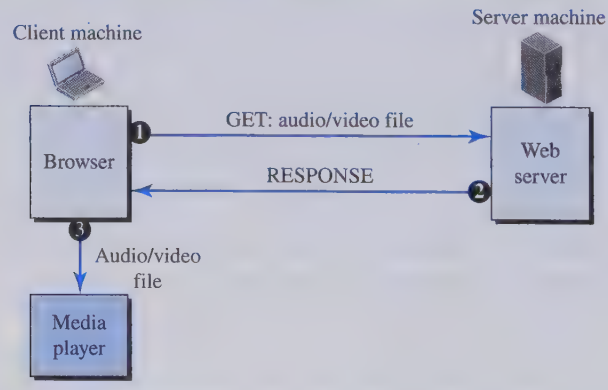
In the first category, streaming stored audio/video, the files are compressed and stored on a server. A client downloads the files through the Internet. This is sometimes referred to as *on-demand audio/video*. Examples of stored audio files are songs, symphonies, books on tape, and famous lectures. Examples of stored video files are movies, TV shows, and music video clips. We can say that streaming stored audio/video refers to *on-demand* requests for compressed audio/video files.

Downloading these types of files from a web server can be different from downloading other types of files. To understand the concept, let us discuss four approaches, each with a different complexity.

First Approach: Using a Web Server

A compressed audio/video file can be downloaded as a text file. The client (browser) can use the services of HTTP and send a GET message to download the file. The web server can send the compressed file to the browser. The browser can then use a help application, normally called a *media player*, to play the file. Figure 11.24 shows this approach.

Figure 11.24 *Using a web server*

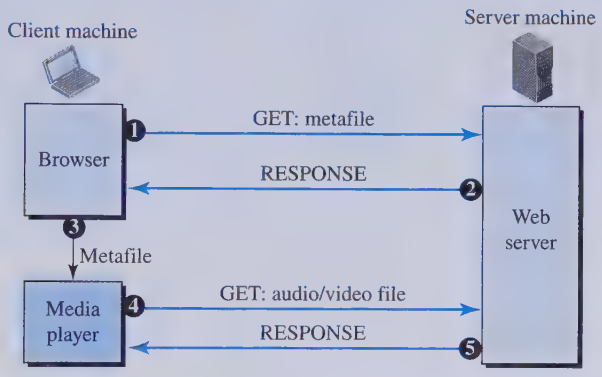


This approach is very simple and does not involve *streaming*. However, it has a drawback. An audio/video file is usually large even after compression. An audio file may contain tens of megabits, and a video file may contain hundreds of megabits. In this approach, the file needs to download completely before it can be played. Using contemporary data rates, the user needs some seconds or tens of seconds before the file can be played.

Second Approach: Using a Web Server with a Metafile

In another approach, the media player is directly connected to the web server for downloading the audio/video file. The web server stores two files: the actual audio/video file and a **metafile** that holds information about the audio/video file. Figure 11.25 shows the steps in this approach.

Figure 11.25 *Using a web server with a metafile*



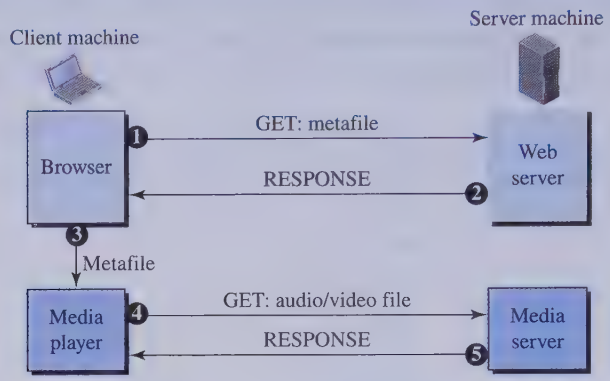
1. The HTTP client accesses the web server using the GET message.
2. The information about the metafile comes in the response.
3. The metafile is passed to the media player.
4. The media player uses the URL in the metafile to access the audio/video file.
5. The web server responds.

Third Approach: Using a Media Server

The problem with the second approach is that the browser and the media player both use the services of HTTP. This is appropriate for retrieving the metafile, but not for retrieving the audio/video file. The reason is that TCP retransmits a lost or damaged segment, which is counter to the philosophy of streaming. We need to dismiss TCP and its error control; we need to use UDP. However, HTTP, which accesses the web server, and the web server itself are designed for TCP; therefore, we need another server, a **media server**. Figure 11.26 shows the concept.

1. The HTTP client accesses the web server using a GET message.
2. The information about the metafile comes in the response.
3. The metafile is passed to the media player.
4. The media player uses the URL in the metafile to access the media server to download the file. Downloading can take place by any protocol that uses UDP.
5. The media server responds.

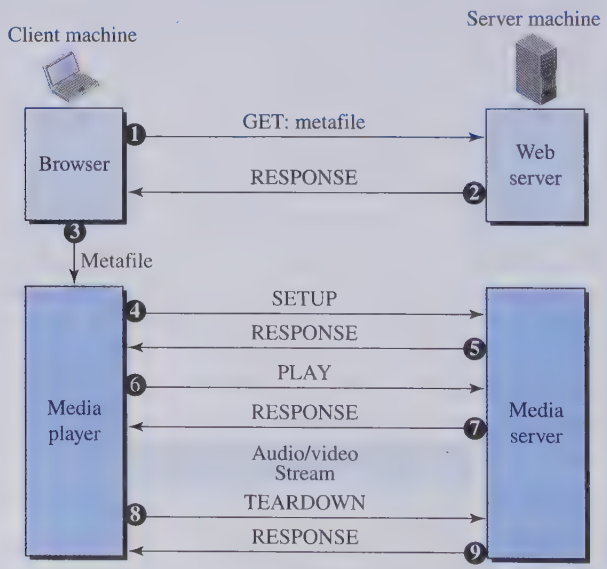
Figure 11.26 *Using a media server*



Fourth Approach: Using a Media Server and RTSP

The **Real-Time Streaming Protocol (RTSP)** is a control protocol designed to add more functionalities to the streaming process. Using RTSP, we can control the playing of audio/video. RTSP is an out-of-band control protocol that is similar to the second connection in FTP. Figure 11.27 shows a media server and RTSP.

Figure 11.27 *Using a media server and RTSP*



1. The HTTP client accesses the web server using a GET message.
2. The information about the metafile comes in the response.

3. The metafile is passed to the media player.
4. The media player sends a SETUP message to create a connection with the media server.
5. The media server responds.
6. The media player sends a PLAY message to start playing (downloading).
7. The audio/video file is downloaded using another protocol that runs over UDP.
8. The connection is broken using the TEARDOWN message.
9. The media server responds.

The media player can send other types of messages. For example, a PAUSE message temporarily stops the downloading; downloading can be resumed with a PLAY message.

Example: Video on Demand (VOD)

Video on Demand (VOD) allows viewers to select a video from a large number of available videos and watch it interactively: pause, rewind, fast forward, etc. A viewer may watch the video in real time or she may download the video to her computer, portable media player, or a device such as a digital video recorder (DVR) and watch it later. Cable TV, satellite TV, and IPTV providers offer both pay-per-view and free content VOD streaming. Many other companies, such as Amazon Prime Video, Netflix, and Hulu, also provide VOD. Internet television is an increasingly popular form of video on demand.

11.3.2 Streaming Live Audio/Video

In the second category, streaming live audio/video, a user listens to broadcast audio and video through the Internet. Good examples of this type of application are Internet radio and Internet TV.

There are several similarities between streaming stored audio/video and streaming live audio/video. They are both sensitive to delay; neither can accept retransmission. However, there is a difference. In the first application, the communication is unicast and on-demand. In the second, the communication is multicast and live. Live streaming is better suited to the multicast services of IP and the use of protocols such as UDP and RTP (discussed later in Section 11.4.2). However, presently, live streaming still uses TCP and multiple unicasting instead of multicasting. There is still much progress to be made in this area.

Example: Internet Radio

Internet radio or web radio is a webcast of audio broadcasting service that offers news, sports, talk, and music via the Internet. It involves a streaming medium that is accessible from anywhere in the world. Web radio is offered via the Internet but is similar to traditional broadcast media: It is noninteractive and cannot be paused or replayed like on-demand services. The largest group of Internet radio providers today includes existing radio stations that simultaneously broadcast their output traditionally and over the Internet. It also includes Internet-only radio stations. In web radio, audio sound is often compressed by MP3 or similar software and the bits are transported over TCP or UDP packets. To prevent jitter, on the user side the bits are buffered and delayed for a few seconds before they are reassembled and played.

Example: Internet Television

Internet television *(ITV)* allows viewers to choose the show they want to watch from a library of shows. The primary models for Internet television are streaming Internet TV or selectable video on an Internet location.

Example: IPTV

Internet protocol television (*IPTV*) is the next-generation technology for delivering real-time and interactive television. Instead of the TV signal being transmitted via satellite, cable, or terrestrial routes, the IPTV signal is transmitted over the Internet. Note that IPTV differs from ITV. Internet TV is created and managed by service providers that cannot control the final delivery; it is distributed via existing infrastructure of the open Internet. IPTV, on the other hand, is highly managed to provide guaranteed quality of service over a complex and expensive network. The network for IPTV is engineered to ensure efficient delivery of large amounts of multicast video traffic and HDTV content to subscribers.

The IP-based platform offers significant advantages, including the ability to integrate television with other IP-based services like high-speed Internet access and VoIP. One way that IPTV operates differently from cable or satellite TV is that in a typical cable or satellite network, all the content constantly flows from the station to each customer. The customer, using a set-top box (a device that connects to a television) selects from the content. In IPTV, content remains in the network, and only the content the customer selects is sent. The advantage is that IPTV requires significantly less bandwidth and therefore allows for the delivery of significantly more content and greater functionality. The disadvantage is that customer's privacy could be compromised because the service provider of IPTV could accurately track down each program watched by each customer.

11.3.3 Real-Time Interactive Audio/Video

In the third category, interactive audio/video, people use the Internet to interactively communicate with one another. The Internet phone, or **voice over IP**, is an example of this type of application. Video conferencing is another example that allows people to communicate visually and orally.

Characteristics

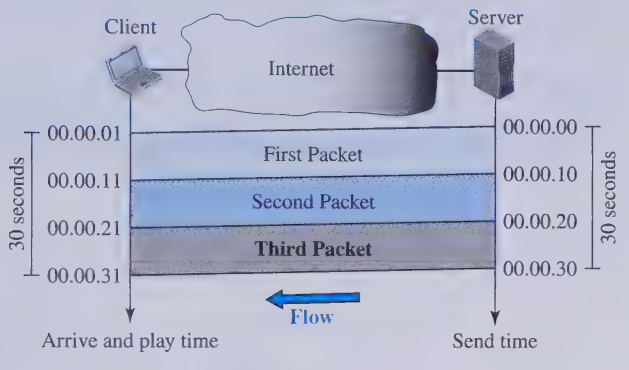
Before discussing the protocols used in this class of applications, we discuss some characteristics of real-time audio/video communication.

Time Relationship

Real-time data on a packet-switched network require the preservation of the time relationship between packets of a session. For example, let us assume that a real-time video server creates live video images and sends them online. The video is digitized and packetized. There are only three packets, and each packet holds 10 s of video information. The first packet starts at 00:00:00, the second packet starts at 00:00:10, and the third packet starts at 00:00:20. Also imagine that it takes 1 s (an exaggeration for simplicity) for each packet to reach the destination (equal delay). The receiver can play back the first packet at 00:00:01, the second packet at 00:00:11, and the third packet at 00:00:21.

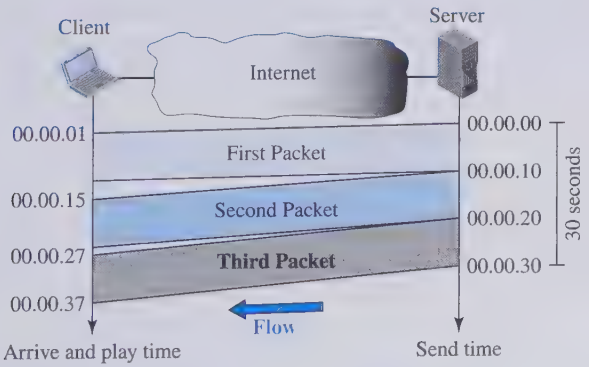
Although there is a 1-s time difference between what the server sends and what the client sees on the computer screen, the action is happening in real time. The time relationship between the packets is preserved. The 1-s delay is not important. Figure 11.28 shows the idea.

Figure 11.28 *Time relationship*



But what happens if the packets arrive with different delays? For example, the first packet arrives at 00:00:01 (1-s delay), the second arrives at 00:00:15 (5-s delay), and the third arrives at 00:00:27 (7-s delay). If the receiver starts playing the first packet at 00:00:01, it will finish at 00:00:11. However, the next packet has not yet arrived; it arrives 4 s later. There is a gap between the first and second packets and between the second and the third as the video is viewed at the remote site. This phenomenon is called **jitter**. Figure 11.29 shows the situation.

Figure 11.29 *Jitter*

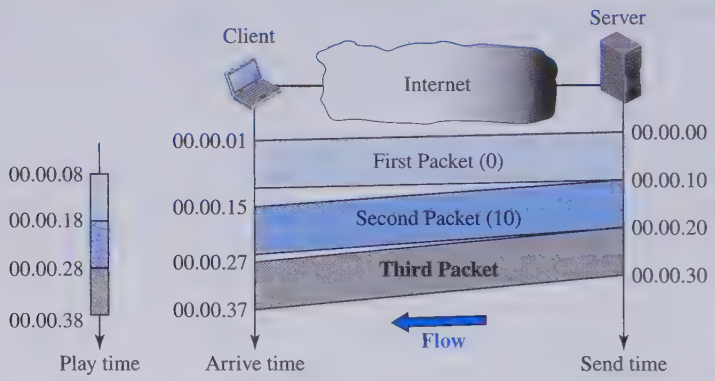


Timestamp

One solution to jitter is the use of a **timestamp**. If each packet has a timestamp that shows the time it was produced relative to the first (or previous) packet, then the receiver can add this time to the time at which it starts the playback. In other words, the receiver knows when each packet is to be played. Imagine the first packet in the previous example has a timestamp of 0, the second has a timestamp of 10, and the third a timestamp of 20. If the receiver starts playing back the first packet at 00:00:08, the second will be played at 00:00:18, and the third at 00:00:28. There are no gaps between the packets. Figure 11.30 shows the situation.

To prevent jitter, we can timestamp the packets and separate the arrival time from the playback time.

Figure 11.30 *Timestamp*



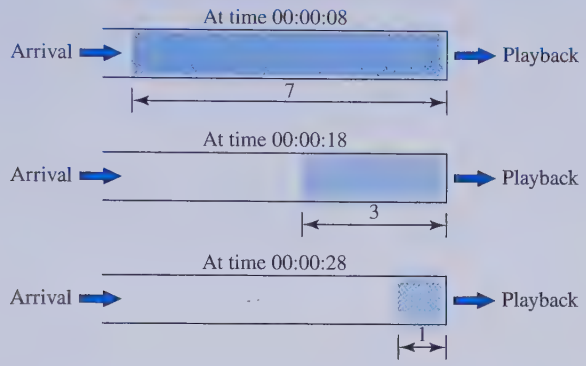
Playback Buffer

To be able to separate the arrival time from the playback time, we need a buffer to store the data until they are played back. The buffer is referred to as a **playback buffer**. When a session begins (the first bit of the first packet arrives), the receiver delays playing the data until a threshold is reached. In the previous example, the first bit of the first packet arrives at 00:00:01, the threshold is 7 s, and the playback time is 00:00:08. The threshold is measured in time units of data. The replay does not start until the time units of data are equal to the threshold value.

Data are stored in the buffer at a possibly variable rate, but they are extracted and played back at a fixed rate. Note that the amount of data in the buffer shrink or expand, but as long as the delay is less than the time to play back the threshold amount of data, there is no jitter. Figure 11.31 shows the buffer at different times for our example.

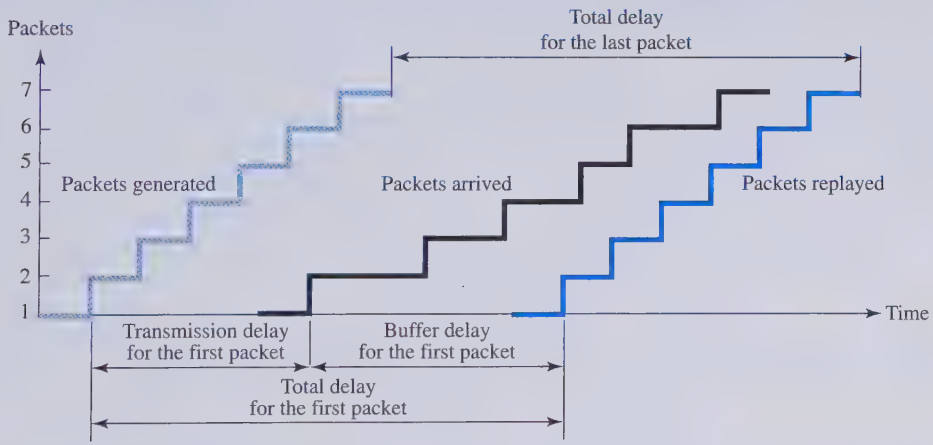
To understand how a playback buffer can actually remove jitter, we need to think about a playback buffer as a tool that introduces more delay in each packet. If the amount of delay added to each packet makes the total delay (the delay in the network and the

Figure 11.31 *Playback buffer*



delay in the buffer) for each packet the same, then the packets are played back smoothly, as though there were no delay. Figure 11.32 shows the idea using the time line for seven packets. Note that we need to select the buffer delay for the first packet in the buffer in such a way that the right two sawtooth curves do not overlap.

Figure 11.32 *The time line of packets*



As Figure 11.32 shows, if the playback time for the first packet is selected properly, then the total delay for all packets should be the same. The packets that have a longer transmission delay should have a shorter waiting in the buffer, and vice versa.

Ordering

In addition to time relationship information and timestamps for real-time traffic, one more feature is needed. We need a *sequence number* for each packet. The timestamp

alone cannot inform the receiver if a packet is lost. For example, suppose the timestamps are 0, 10, and 20. If the second packet is lost, the receiver receives just two packets with timestamps 0 and 20. The receiver assumes that the packet with timestamp 20 is the second packet, produced 20 s after the first. The receiver has no way of knowing that the second packet has actually been lost. A sequence number to order the packets is needed to handle this situation.

Multicasting

Multimedia play a primary role in audio and video conferencing. The traffic can be heavy, and the data are distributed using *multicasting* methods. Conferencing requires two-way communication between receivers and senders.

Translation

Sometimes real-time traffic needs *translation.* A translator is a computer that can change the format of a high-bandwidth video signal to a lower-quality narrow-bandwidth signal. This is needed, for example, for a source creating a high-quality video signal at 5 Mbps and sending to a recipient having a bandwidth of less than 1 Mbps. To receive the signal, a translator is needed to decode the signal and encode it again at a lower quality that needs less bandwidth.

Mixing

If there is more than one source that can send data at the same time (as in a video or audio conference), the traffic is made up of multiple streams. To converge the traffic to one stream, data from different sources can be mixed. A mixer mathematically adds signals coming from different sources to create one single signal.

Example of a Real-Time Application: Skype

Skype (abbreviation of the original project *Sky peer-to-peer*) is a peer-to-peer VoIP application software that was originally developed by Ahti Heinla, Priit Kasesalu, and Jaan Tallinn, who had also originally developed Kazaa (a P2P file-sharing application software). The application allows registered users who have audio input and output devices on their PCs to make free PC-to-PC voice calls to other registered users over the Internet. Skype includes other popular features such as *instant messaging* (*IM*), *short message service* (*SMS*) group chat, file transfer, video conferencing, and SkypeIn and SkypeOut services. Using SkypeIn and SkypeOut services allows registered users to communicate with traditional land-line telephones and mobile phones for a small fee.

Skype is free for PC-to-PC calls, but when a PSTN or a cell phone is involved, Skype offers a fee-based service. There are two modes for involving a PSTN or cell phone in a Skype conversation: SkypeIn and SkypeOut.

SkypeIn service is offered in many countries in the world, and the list is expanding. Wherever the service is offered, it allows the Skype registered user to receive a call from a PSTN or cell phone on her or his computer over the Internet. To use SkypeIn, the user subscribes to an online number for a monthly fee. Using their PSTN or mobile phone, callers can call this number and pay the same standard rate as if they were making a similar call to another PSTN or mobile phone with the same area code. The online number uses the Internet to route the call to the Skype registered user. Except for the price

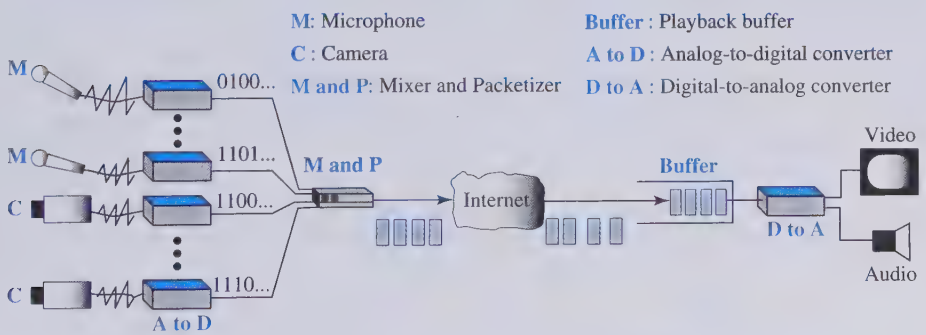
of the online number, the cost of the call is free for the registered user who receives the call. To relieve callers from being charged long-distance rates, the registered user can subscribe to more than one online number. For example, the registered user can get a subscription to one number in the United States and another number in France. SkypeIn service comes with free voice mail.

SkypeOut allows Skype registered users to make phone calls from their PC to any PSTN phone or any cell phone anywhere in the world and pay local rates. To make SkypeOut calls, the user purchases either monthly subscriptions or Skype credit minutes. Using PCs, Skype users dial the phone numbers of PSTN or mobile phones. Skype channels SkypeOut calls to gateways, which then direct the calls to the PSTN or cell phone services. In addition to the monthly subscription rate or Skype credit minute cost, the Skype user pays a small global and local rate for the service. With their subscription, users are able to forward incoming calls to their PSTN or mobile phone. It is important to mention that Skype is not considered as a replacement for telephone service and cannot be used in emergency situations. For example, we cannot use Skype to dial 911 in the United States.

11.4 REAL-TIME INTERACTIVE PROTOCOLS

After discussing the three approaches to using multimedia through the Internet, we now concentrate on the last one, which is the most interesting and involved: real-time interactive multimedia. This application has evoked a lot of attention in the Internet society, and several application-layer protocols have been designed to handle it. Before we discuss the need and rationale for this type of application, let us give a schematic representation in Figure 11.33.

Figure 11.33 *Schematic diagram of a real-time multimedia system*



Although it could have only one microphone and one audio player, today's interactive real-time application is normally made up of several microphones and several cameras. The audio and video information (analog signals) are converted to digital data. The digital data created from different sources are normally mixed and packetized. The

packets are sent to the packet-switched Internet. The packets are received at the destination with different delays (jitter), and some packets may also be corrupted or lost. A playback buffer replays packets based on the timestamp on each packet. The result is sent to a digital-to-analog converter to re-create the audio and video signals. The audio signal is sent to a speaker; the video signal to a display device.

Each microphone or camera at the source site is called a *contributor* and is given a 32-bit identifier called the *contributing source* **(CSRC)** *identifier*. The mixer is also called the synchronizer and is given another identifier called the *synchronizing source* **(SSRC)** *identifier*. We will use these identifiers in the packet later.

11.4.1 Rationale for New Protocols

We discussed the protocol stack for general Internet applications in Chapters 2 to 10. In this section, we want to show why we need some new protocols to handle interactive real-time multimedia applications such as audio and video conferencing.

It is clear that we do not need to change the first three layers of the TCP/IP protocol suite (physical, data-link, and network layers) because these three layers are designed to carry any type of data. The physical layer provides service to the data-link layer, no matter the nature of the bits in a frame. The data-link layer is responsible for node-to-node delivery of the network layer packets no matter what makes up the packet. The network layer is also responsible for host-to-host delivery of a datagram, no matter what is in the datagram, although we need a network layer with a better quality of service for multimedia application.

It looks as if we should worry about only the application and transport layers. Some application-layer protocols need to be designed to encode and compress the multimedia data considering the trade-off between the quality, bandwidth requirement, and the complexity of mathematical operations for encoding and compression. As we describe shortly, it turns out that application-layer protocols that can handle multimedia have some requirements that can be handled by the transport layer instead of being individually handled by each application protocol.

Application Layer

It is clear that we need to develop some application-layer protocols for interactive real-time multimedia because the nature of audio conferencing and video conferencing is different from some applications, such as file transfer and electronic mail, which we discussed in Chapter 10. Several proprietary applications have been developed by the private sector, and more and more applications are appearing in the market every day. Some of these applications, such as MPEG audio and MPEG video, use some standards defined for audio and video data transfer. There is no specific standard that is used by all applications, and there is no specific application protocol that can be used by everyone.

Transport Layer

The lack of a single standard and the general features of multimedia applications discussed so far in this chapter raise some questions about the transport-layer protocol being used for all multimedia applications. The two common transport-layer protocols, UDP and TCP, were developed at a time when no one even thought about the use of multimedia in the Internet. Can we use UDP or TCP as a general transport-layer protocol for

real-time multimedia applications? To answer this question, we first need to think about the requirements for this type of multimedia application and then see if either UDP or TCP can respond to these requirements.

Transport-Layer Requirements for Interactive Real-Time Multimedia

Let us first briefly compose a set of requirements for this type of application.

- **Sender-receiver negotiation.** The first requirement is related to the lack of a single standard for audio and video. We have several standards for audio and video conferencing with different encoding or compression methods. If a sender uses one encoding method and the receiver uses another one, the communication is impossible. The application programs need to negotiate the standards used for audio/video before encoded and compressed data can be transferred.
- **Creation of packet stream.** When we discussed UDP and TCP in Chapter 9, we mentioned that UDP allows the application to packetize its message with clear-cut boundaries before delivering the message to UDP. TCP, on the other hand, can handle streams of bytes without the requirement from the application to put specific boundaries on the chunk of data. In other words, UDP is suitable for those applications that need to send messages with clear-cut boundaries, but TCP is suitable for those applications that send continuous streams of bytes. When it comes to real-time multimedia, we need both features. Real-time multimedia is a *stream of frames* or a *stream chunk* of data in which the chunk or frame has a specific size or boundary, but also there are relationships between frames or chunks. It is clear that neither UDP nor TCP is suitable for handling streams of frames in this case. UDP cannot provide a relationship between frames; TCP provides relationships between bytes, but a byte is much smaller than a multimedia frame or chunk.
- **Source synchronization.** If an application uses more than one source (both audio and video), there is a need for synchronization between the sources. For example, in a teleconferencing that uses both audio and video, such as Skype, the audio and video may be using different encoding and compression methods with different rates. It is obvious that somehow these two types of applications should be synchronized; otherwise, we will see the face of the speaker before hearing what she is saying, or vice versa. It is also possible that there is more than one source for audio or video (using multiple microphones or multiple cameras). Source synchronization is normally done using *mixers*.
- **Error control.** We have already discussed that handling errors (packet corruption and packet loss) need special care in real-time multimedia applications. We showed that we cannot afford to retransmit corrupted or lost packets. We learned that we need to inject extra redundancy in the data to be able to reproduce the lost or corrupted packets without asking for them to be retransmitted. This implies that the TCP protocol is not suitable for real-time multimedia applications.
- **Congestion control.** Like other applications, we need to provide some sort of congestion control in multimedia. If we decide not to use TCP for multimedia (because of retransmission problems), we should somehow implement congestion control in the system.

- **Jitter removal.** We discussed in Section 11.3.3 that one of the problems with real-time multimedia applications is the jitter created at the receiver site, because the packet-switched service provided by the Internet may create uneven delays for different packets in a stream. In the past, audio conferencing was provided by the telephone network, which was originally designed as a circuit-switched network, which is jitter-free. If we gradually move all these applications to the Internet, we need to somehow deal with the jitter. We said in Section 11.3.3 that one of the ways to alleviate jitter is to use playback buffers and timestamping. The playback is implemented at the application layer at the receiver site, but the transport layer should be able to provide the application layer with timestamping and sequencing.
- **Identifying sender.** A subtle issue in multimedia applications, like other applications, is to identify the sender at the application layer. When we use the Internet, the parties are identified by their IP addresses. However, we need to map the IP addresses to something more friendly, as we did with HTTP or electronic mail.

Capability of UDP or TCP to Handle Real-Time Multimedia

After discussing the requirements for real-time multimedia, let us see if either UDP or TCP is capable of handling these requirements. Table 11.6 compares UDP and TCP with respect to these requirements.

Table 11.6 *Capability of UDP or TCP to handle real-time multimedia*

Requirements	*UDP*	*TCP*
1. Sender-receiver negotiation for selecting the encoding type	No	No
2. Creation of packet stream	No	No
3. Source synchronization for mixing different sources	No	No
4. Error control	No	Yes
5. Congestion control	No	Yes
6. Jitter removal	No	No
7. Sender identification	No	No

The first glance at Table 11.6 reveals a very interesting fact: Neither UDP nor TCP can respond to all requirements. However, we should remember that we need a transport-layer protocol to implement the client/server socket; we cannot let the application layer do the job of the transport layer. These means that we probably have three choices:

1. We can use a new transport-layer protocol (such as SCTP, discussed in Chapter 9) that combines the features of UDP and TCP (in particular stream packetizing and multistreaming). This choice is probably the best because SCTP has the combined features of UDP and TCP with additional features of its own. However, SCTP was introduced when there were many multimedia applications. It may become the de facto transport layer in the future.
2. We can use TCP and combine it with another transport facility to compensate for the requirements that cannot be provided by TCP. However, this choice is somewhat difficult because TCP uses a retransmission method that it is not acceptable for real-time applications. Another problem with TCP is that it does not do multicasting.

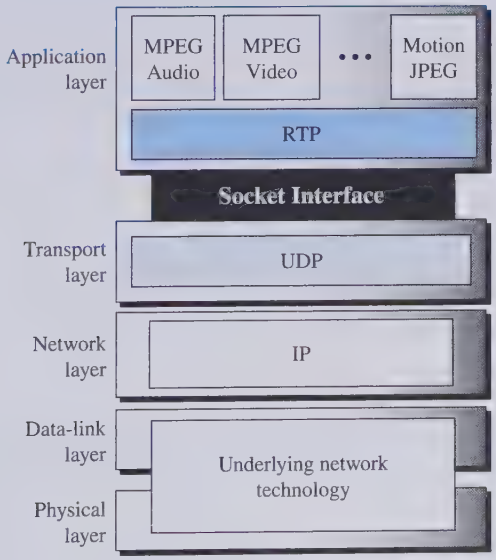
A TCP connection is only a two-party connection; we need multiparty connection for real-time interactive communication.

3. We can use UDP and combine it with another transport facility to compensate for the requirements that cannot be provided by UDP. In other words, we use UDP to provide client/server socket interface but use another protocol that runs at the top of the UDP. This is the current choice for multimedia applications. This transport facility is the Real-Time Transport Protocol (RTP), which we discuss next.

11.4.2 RTP

Real-Time Transport Protocol (RTP) is the protocol designed to handle real-time traffic on the Internet. RTP does not have a delivery mechanism (multicasting, port numbers, and so on); it must be used with UDP. RTP stands between UDP and the multimedia application. The literature and standards treat RTP as the transport protocol (not a transport-layer protocol) that can be thought of as located in the application layer (see Figure 11.34). The data from multimedia applications are encapsulated in RTP, which in turn passes them to the transport layer. In other words, the socket interface is located between RTP and UDP, which implies that we should include the functionality of RTP in client/server programs that we write for each multimedia application. However, some programming languages provide some facilities to make the programming task easier. For example, the C language provides an RTP library and the Java language provides an RTP class for this purpose. If we use the RTP library or the RTP class, we can think that we have separated the applications from the RTP and the RTP has become part of the transport layer.

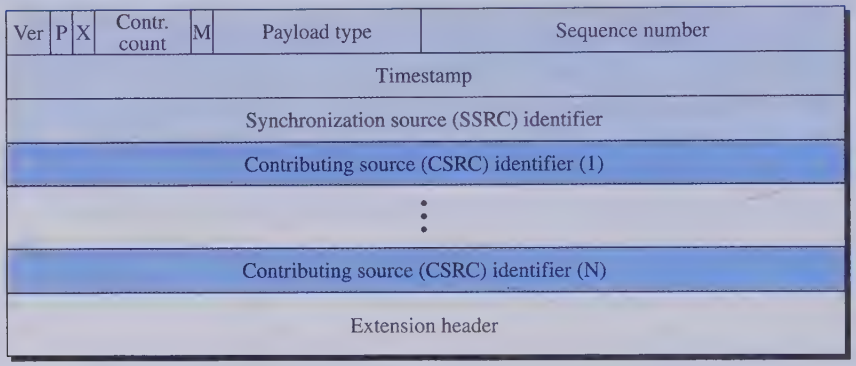
Figure 11.34 *RTP location in the TCP/IP protocol suite*



RTP Packet Format

Before we discuss how RTP can help the multimedia applications, let us discuss its packet format. We can then relate the functions of the fields with the requirements we discussed in Section 11.4.1. Figure 11.35 shows the format of the RTP packet header. The format is very simple and general enough to cover all real-time applications. An application that needs more information adds it to the beginning of its payload.

Figure 11.35 *RTP packet header format*



A description of each field follows.

- **Ver.** This 2-bit field defines the version number. The current version is 2.
- **P.** This 1-bit field, if set to 1, indicates the presence of padding at the end of the packet. In this case, the value of the last byte in the padding defines the length of the padding. Padding is the norm if a packet is encrypted. There is no padding if the value of the P field is 0. The use of this 1-bit field eliminates the need for the length of the RTP data because if there is no padding, the length of the data is the length of the UDP data minus the RTP header. Otherwise, the length of the padding should be subtracted to give the RTP data length.
- **X.** This 1-bit field, if set to 1, indicates an extension header between the basic header and the data. There is no extension header if the value of this field is 0.
- **Contributor count.** This 4-bit field indicates the number of contributing source (CSRC). Note that we can have a maximum of 15 contributors because a 4-bit field only allows a number between 0 and 15. Note that in an audio or video conferencing, each active source (the source that sends data instead of just listening) is called a contributor.
- **M.** This 1-bit field is a marker used by the application to indicate, for example, the end of its data. We said that a multimedia application is a stream of blocks or frames with an end of frame marker. If this bit is set in an RTP packet, it means that the RTP packet carries this marker.

- **Payload type.** This 7-bit field indicates the type of the payload. Several payload types have been defined so far. We list some common applications in Table 11.7. A discussion of the types is beyond the scope of this book.

Table 11.7 Payload types

Type	*Application*	*Type*	*Application*	*Type*	*Application*
0	PCMμ Audio	7	LPC audio	15	G728 audio
1	1016	8	PCMA audio	26	Motion JPEG
2	G721 audio	9	G722 audio	31	H.261
3	GSM audio	10–11	L16 audio	32	MPEG1 video
5–6	DV14 audio	14	MPEG audio	33	MPEG2 video

- **Sequence number.** This field is 16 bits in length. It is used to number the RTP packets. The sequence number of the first packet is chosen randomly; it is incremented by 1 for each subsequent packet. The sequence number is used by the receiver to detect lost or out-of-order packets.
- **Timestamp.** This is a 32-bit field that indicates the time relationship between packets. The timestamp for the first packet is a random number. For each succeeding packet, the value is the sum of the preceding timestamp plus the time the first byte is produced (sampled). The value of the clock tick depends on the application. For example, audio applications normally generate chunks of 160 bytes; the clock tick for this application is 160. The timestamp for this application increases 160 for each RTP packet.
- **Synchronization source (SCRC) identifier.** If there is only one source, this 32-bit field defines the source. However, if there are several sources, the mixer is the synchronization source and the other sources are contributors. The value of the source identifier is a random number chosen by the source. The protocol provides a strategy in case of conflict (two sources start with the same sequence number).
- **Contributing source (CSRC) identifier.** Each of these 32-bit identifiers (a maximum of 15) defines a source. When there is more than one source in a session, the mixer is the synchronization source and the remaining sources are the contributors.

UDP Port

Although RTP is itself a transport-layer protocol, the RTP packet is not encapsulated directly in an IP datagram. Instead, RTP is treated like an application program and is encapsulated in a UDP user datagram. However, unlike other application programs, no well-known port is assigned to RTP. The port can be selected on demand with only one restriction: The port number must be an even number. The next number (an odd number) is used by the companion of RTP, Real-Time Transport Control Protocol (RTCP), which we will discuss in the next section.

RTP uses an even-numbered UDP port.

11.4.3 RTCP

RTP allows only one type of message, one that carries data from the source to the destination. To really control the session, we need more communication between the participants

in a session. Control communication in this case is assigned to a separate protocol called **Real-Time Transport Control Protocol (RTCP)**. We need to emphasize that the RTCP payloads are not carried in RTP packets; RTCP is in fact a sister protocol of RTP. This means that the UDP, as the real transport protocol, sometimes carries RTP payloads and sometimes RTCP payloads as though they belong to different upper-layer protocols.

RCTP packets make an *out-of-band* control stream that provides two-way feedback information between the senders and receivers of the multimedia streams. In particular, RTCP provides the following functions:

1. RTCP informs the sender or senders of multimedia streams about the network performance, which can be directly related to the congestion in the network. Because multimedia applications use UDP (instead of TCP), there is no way to control the congestion in the network at the transport layer. This means that, if it is necessary to control the congestion, it should be done at the application layer. RTCP, as we will see shortly, gives the clues to the application layer to do so. If the congestion is observed and reported by the RTCP, an application can use a more aggressive compression method to reduce the number of packets and, therefore, to reduce congestion, for a trade-off in quality. On the other hand, if no congestion is observed, the application program can use a less aggressive compression method for a better quality service.
2. Information carried in the RTCP packets can be used to synchronize different streams associated with the same source. A source may use two different sources to collect audio or video data. In addition, audio data may be collected from different microphones and video data may be collected from different cameras. In general, two pieces of information are needed to achieve synchronization:
   a. Each sender needs an identity. Although each source may have a different SSRC, RTCP provides one single identity, called a *canonical name (CNAME)* for each source. CNAME can be used to correlate different sources and allow the receiver to combine different sources from the same source. For example, a teleconference may have n senders associated with a session, but we may have m ($m > n$) sources that contribute to the stream. In this system, we have only n CNAMEs, but m SSRCs. A CNAME is in the form of

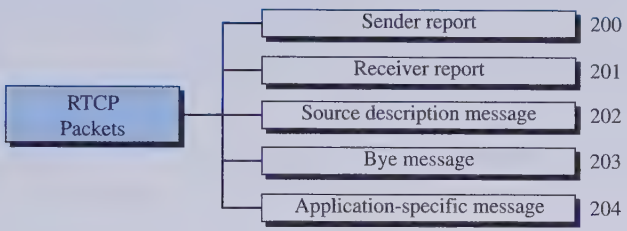
      user@host

      in which *user* is normally the login name of the user and *host* is the domain name of the host.
   b. The canonical name cannot per se provide synchronization. To synchronize the sources, we need to know the absolute timing of the stream, in addition to the relative timing provided by the timestamp field in each RTP packet. The timestamp information in each packet gives the relative time relationship of the bits in the packet to the beginning of the stream; it cannot relate one stream to another. The absolute time, the "wall clock" time as it is sometimes referred to, needs to be sent by RTCP packets to enable synchronization.
3. An RTCP packet can carry extra information about the sender that can be useful for the receiver, such as the name of the sender (beyond canonical name) or captions for a video.

RTCP Packets

After discussing the main functions and purpose of RTCP, let us discuss its packets. Figure 11.36 shows five common packet types. The number next to each box defines the numeric value of each packet. We need to mention that more than one RTCP packet can be packed as a single payload for UDP because the RTCP packets are smaller than RTP packets.

Figure 11.36 *RTCP packet types*



The format and the exact definition of each field is very involved and beyond the scope and space of this book. We briefly discuss the purpose of each packet and relate it to the functions previously described.

Sender Report Packet

The sender report packet is sent periodically by the active senders in a session to report transmission and reception statistics for all RTP packets sent during the interval. The sender report packet includes the following information:

- SSRC of the RTP stream.
- Absolute timestamp, which is the combination of the relative timestamp and the wall clock time, which is the number of seconds elapsed since midnight January 1, 1970. The absolute timestamp allows the receiver to synchronize different RTP packets.
- Number of RTP packets and bytes sent from the beginning of the session.

Receiver Report Packet

The receiver report is issued by passive participants, those that do not send RTP packets. The report informs the sender and other receivers about the quality of service. The feedback information can be used for congestion control at the sender site. A receiver report includes the following information:

- SSRC of the RTP stream for which the receiver report has been generated
- Fraction of packet loss
- Last sequence number
- Interval jitter

Source Description Packet

The source periodically sends a source description packet to give additional information about itself. The packet can include:

- SSRC
- Canonical name (CNAME) of the sender
- Other information such as the real name, the e-mail address, and the telephone number
- Source description packet, may also include extra data, such as captions used for video

Bye Packet

A source sends a bye packet to shut down a stream. It allows the source to announce that it is leaving the conference. Although other sources can detect the absence of a source, this packet is a direct announcement. It is also very useful to a mixer.

Application-Specific Packet

The application-specific packet is a packet for an application that wants to use new applications (not defined in the standard). It allows the definition of a new packet type.

UDP Port

RTCP, like RTP, does not use a well-known UDP port. It uses a temporary port. The UDP port chosen must be the number immediately following the UDP port selected for RTP, which makes it an odd-numbered port.

RTCP uses an odd-numbered UDP port that follows the one selected for RTP.

Bandwidth Utilization

The RTCP packets are sent not only by the active senders, but also by passive receivers, whose numbers are normally greater than the active senders. This means that if the RTCP traffic is not controlled, it may get out of hand. To control the situation, RTCP uses a control mechanism to limit its traffic to the small portion (normally 5 percent) of the traffic used in the session (for both RTP and RTCP). A larger part of this small percentage, x percent, is then assigned to the RTCP packets generated by the passive receiver, a smaller part, $(1 - x)$ percent, is assigned to the RTCP packets generated by the active senders. RTCP protocol uses a mechanism to define the value of x based on the ratio of the passive receiver to the active sender.

Example 11.10

Let us assume that the total bandwidth allocated for a session is 1 Mbps. RTCP traffic gets only 5 percent of this bandwidth, which is 50 kbps. If there are only two active senders and eight passive receivers, it is natural that each sender or receiver gets only 5 kbps. If the average size of the RTCP packet is 5 kbits, then each sender or receiver can send only 1 RTCP packet per second. Note that we need to consider the packet size at the data-link layer.

Requirement Fulfillment

As we promised, let us see how the combination of RTP and RTCP can respond to the requirements of an interactive real-time multimedia application. A digital audio or

video stream, a sequence of bits, is divided into chunks (*blocks* or *frames*, as they are sometimes called). Each chunk has a predefined boundary that distinguishes the chunk from the previous chunk or the next one. A chunk is encapsulated in an RTP packet, which defines a specific encoding (payload type), a sequence number, a timestamp, a synchronization source (SSRC) identifier, and one or more contributing source (CSRC) identifiers.

1. The first requirement, *sender-receiver negotiation*, cannot be satisfied by the combination of the RTP/RTCP protocols. It should be accomplished by some other means. We will see in section 11.4.4 that another protocol (SIP), which is used in conjunction with RTP/RTCP, provides this capability.
2. The second requirement, *creation of a stream of chunks*, is provided by encapsulating each chunk in an RTP packet and giving a sequence number to each chunk. The M field in an RTP packet also defines whether there is a specific type of boundary between chunks.
3. The third requirement, *synchronization of sources*, is satisfied by identifying each source by a 32-bit identifier and using the relative timestamping in the RTP packet and the absolute timestamping in the RTCP packet.
4. The fourth requirement, *error control*, is provided by using the sequence number in the RTP packet and letting the application regenerate the lost packet using forward error correction (FEC) methods.
5. The fifth requirement, *congestion control*, is met by the feedback from the receiver using the *receiver report packets* (RTCP) that notify the sender about the number of lost packets. The sender then can use a more aggressive compression technique to reduce the number of packets sent and therefore alleviate the congestion.
6. The sixth requirement, *jitter removal*, is achieved by the timestamping and sequencing provided in each RTP packet to be used in buffered playback of the data.
7. The seventh requirement, *identification of source*, is provided by using the CNAME included in the source description packets (RTCP) sent by the sender.

11.4.4 Session Initialization Protocol (SIP)

We discussed how to use the Internet for audio/video conferencing in sections 11.4.2 and 11.4.3. Although RTP and RTCP can be used to provide these services, one component is missing: a signaling system required to call the participants.

To understand the issue, let us go back for the moment to the traditional audio conferencing (between two or more people) using the traditional telephone system [public switched telephone network (PSTN)]. To make a phone call, two telephone numbers are needed, that of the caller and that of the callee. We then need to dial the telephone number of the callee and wait for her to respond. The telephone conversation starts after the response of the callee. In other words, regular telephone communication involves two phases: the signaling phase and the audio communication phase.

The signaling phase in the telephone network is provided by a protocol called *Signaling System 7* (*SS7*). The SS7 protocol is totally separate from the voice communication system. For example, although the traditional telephone system uses analog

signals carrying voice over a circuit-switched network, SS7 uses electrical pulses in which each number dialed changes to a series of pulses. Today, SS7 not only provides the calling service, it also provides other services, such as call forwarding and error reporting.

The combination of RTP/RTCP protocols we discussed earlier in Sections 11.4.2 and 11.4.3, is equivalent to the voice communication provided by PSTN; to totally simulate this system over the Internet, we need a signaling system. Our ambition takes us even further. Not only do we want to be able to call our party in an audio or video conference using our computers (PCs), we also want to be able to do so using our telephone set, our mobile phone, our PDAs, and so on. We also need to find our party if she is not sitting at her desk. We need to communicate between a mixture of devices.

The **Session Initiation Protocol (SIP)** is a protocol devised by the Internet Engineering Task Force (IETF) to be used in conjunction with the RTP/SCTP. It is an application-layer protocol, similar to HTTP, that establishes, manages, and terminates a multimedia session (call). It can be used to create two-party, multiparty, or multicast sessions. SIP is designed to be independent of the underlying transport layer; it can run on either UDP, TCP, or SCTP, using the port 5060. SIP can provide the following services:

- It establishes a call between users if they are connected to the Internet.
- It finds the location of the users (their IP addresses) on the Internet, because the users may be changing their IP addresses (think about mobile IP and DHCP).
- It finds out if the users are able or willing to take part in the conference call.
- It determines the users' capabilities in terms of media to be used and the type of encoding.
- It establishes session setup by defining parameters such as port numbers to be used (remember that RTP and RTCP use port numbers).
- It provides session management functions such as call holding, call forwarding, accepting new participants, and changing the session parameters.

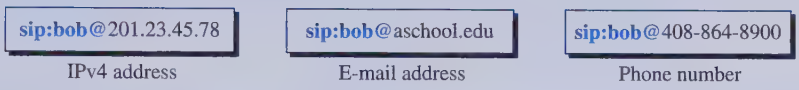
Communicating Parties

One difference that we may have noticed between the interactive real-time multimedia applications and other applications is communicating parties. In an audio or video conference, the communication is between humans, not devices. For example, in HTTP or FTP, the client needs to find the IP address of the server (using DNS) before communication. There is no need to find a person before communicating. In the SMTP, the sender of an e-mail sends the message to the receiver mailbox on an SMTP without controlling when the message will be picked up. In an audio or video conference, the caller needs to find the callee. The callee can be sitting at her desk, can be walking in the street, or can be totally unavailable. What makes the communication more difficult is that the device to which the participant has access at a particular time may have a different capability than the device being used at another time. The SIP protocol needs to find the location of the callee and at the same time negotiate the capability of the devices the participants are using.

Addresses

In a regular telephone communication, a telephone number identifies the sender, and another telephone number identifies the receiver. SIP is very flexible. In SIP, an e-mail address, an IP address, a telephone number, and other types of addresses can be used to identify the sender and receiver. However, the address needs to be in SIP format (also called scheme). Figure 11.37 shows some common SIP formats.

Figure 11.37 *SIP formats*



We have noticed that the SIP address is similar to a URL we have encountered in Chapter 10. In fact, the SIP addresses are URLs that can be included in the web page of the potential callee. For example, Bob can include one of the above addresses as his SIP address, and, if someone clicks on it, the SIP protocol is invoked and calls Bob. Other addresses are also possible, such as those that use the first name followed by the last name, but all addresses need to be in the form *sip:user@address*.

Messages

SIP is a text-based protocol like HTTP. SIP, like HTTP, uses messages. Messages in SIP are divided into two broad categories: requests and responses. The format of both message categories is shown here (note the similarity with HTTP messages as shown in Chapter 10):

Request Messages		Response Messages	
Start line		**Status line**	
Header	**// one or more lines**	**Header**	**// one or more lines**
Blank line		**Blank line**	
Body	**// one or more lines**	**Body**	**// one or more lines**

Request Messages

IETF has originally defined six request messages, but some new request messages have been proposed to extend the functionality of the SIP. We just mention the original six messages as follows:

- **INVITE.** The INVITE request message is used by a caller to initialize a session. Using this request message, a caller invites one or more callees to participate in the conference.
- **ACK.** The ACK message is sent by the caller to confirm that the session initialization has been completed.
- **OPTIONS.** The OPTIONS message queries a machine about its capabilities.
- **CANCEL.** The CANCEL message cancels an already started initialization process, but does not terminate the call. A new initialization may start after the CANCEL message.

- **REGISTER.** The REGISTER message makes a connection when the callee is not available.
- **BYE.** The BYE message is used to terminate the session. Compare the BYE message with the CANCEL message. The BYE message, which can be initiated from the caller or callee, terminates the whole session.

Response Messages

IETF has also defined six types of response messages that can be sent to request messages, but note that there is no relationship between a request and a response message. A response message can be sent to any request message. Like other text-oriented application protocols, the response messages are defined using three-digit numbers. The response messages are briefly described here:

- **Informational responses.** These responses are in the form **SIP 1xx.** (The common ones are 100 trying, 180 ringing, 181 call forwarded, 182 queued, and 183 session progress.)
- **Successful responses.** These responses are in the form **SIP 2xx.** (The common one is 200 OK.)
- **Redirection responses.** These responses are in the form **SIP 3xx.** (The common ones are 301 moved permanently, 302 moved temporarily, 380 alternative service.)
- **Client failure responses.** These responses are in the form **SIP 4xx.** (The common ones are 400 bad request, 401 unauthorized, 403 forbidden, 404 not found, 405 method not allowed, 406 not acceptable, 415 unsupported media type, 420 bad extension, 486 busy here.)
- **Server failure responses.** These responses are in the form **SIP 5xx.** (The common ones are 500 server internal error, 501 not implemented, 503 service unavailable, 504 timeout, 505 SIP version not supported.)
- **Global failure responses.** These responses are in the form **SIP 6xx.** (The common ones are 600 busy everywhere, 603 decline, 604 doesn't exist, and 606 not acceptable.)

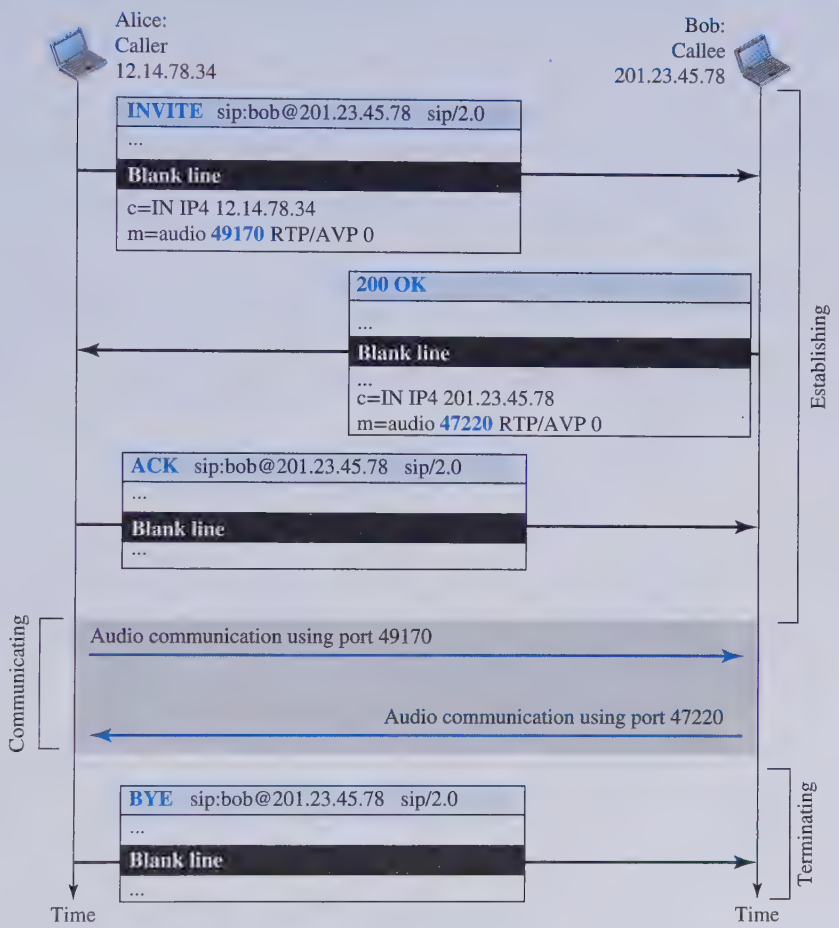
First Scenario: Simple Session

In the first scenario, we assume that Alice needs to call Bob and the communication uses the IP addresses of Alice and Bob as the SIP addresses. We can divide the communication into three modules: establishing, communicating, and terminating. Figure 11.38 shows a simple session using SIP.

Establishing a Session

Establishing a session in SIP requires a three-way handshake. Alice sends an INVITE request message, using UDP, TCP, or SCTP to begin the communication. If Bob is willing to start the session, he sends a response (200 OK) message. To confirm that a reply code has been received, Alice sends an ACK request message to start the audio communication. The establishment section uses two request messages (INVITE and ACK) and one response message (200 OK). We need to say that the INVITE message

Figure 11.38 *SIP simple session*



start line defines the IP address of the receiver and the version of the SIP. We have not included any line in the header, but we will do so shortly. The body of the header uses another protocol, Session Description Protocol (SDP), that defines the syntax (format) and semantic (meaning of each line). We will briefly discuss this protocol later in this section. We just mention that the first line in the body defines the sender of the message; the second line defines the media (audio) and the port number to be used for RTP in the direction from Alice to Bob. The response message defines the media (audio) and the port number to be used for RTP in the Bob to Alice direction. After Alice confirms the establishment of the session with the ACK message request (which does not need a response), the establishing session is finished and the communication can start.

Communicating

After the session has been established, Alice and Bob can communicate using two temporary ports defined in the establishing sessions. The even-numbered ports are used for RTP; RTCP can use the odd-numbered ports that follow (we have shown only the even-numbered ports used for RTP in Figure 11.38).

Terminating the Session

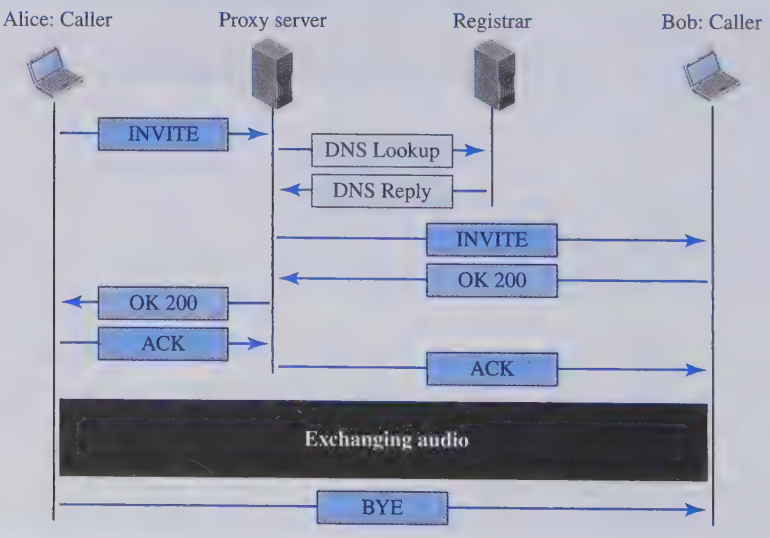
The session can be terminated with a BYE message sent by either party. In Figure 11.38, we have assumed that Alice terminates the session.

Second Scenario: Tracking the Callee

What happens if Bob is not sitting at his terminal? He may be away from his system or at another terminal. He may not even have a fixed IP address if DHCP is being used. SIP has a mechanism (similar to one in DNS) that finds the IP address of the terminal at which Bob is sitting. To perform this tracking, SIP uses the concept of registration. SIP defines some servers as registrars. At any moment, a user is registered with at least one **registrar server**; this server knows the IP address of the callee.

When Alice needs to communicate with Bob, she can use the e-mail address instead of the IP address in the INVITE message. The message goes to a proxy server. The proxy server sends a lookup message (not part of SIP) to some registrar server that has registered Bob. When the proxy server receives a reply message from the registrar server, the proxy server takes Alice's INVITE message and inserts the newly discovered IP address of Bob. This message is then sent to Bob. Figure 11.39 shows the process.

Figure 11.39 *Tracking the callee*



SIP Message Format and SDP Protocol

As we discussed before, the SIP request and response messages are divided into four sections: start or status line, header, a blank line, and the body. Because a blank line needs no more information, let us briefly describe the format of the other sections.

Start Line

The start line is a single line that starts with the message request name, followed by the address of the recipient and the SIP version. For example, the INVITE message request start line has the following start line format:

```
INVITE sip:forouzan@roadrunner.com
```

Status Line

The status line is a single line that starts with the three-digit response code. For example, the 200 response message has the following status line format:

```
200 OK
```

Header

A header, in the request or response message, can use several lines. Each line starts with the line name followed by a colon and space and followed by the value. Some typical header lines are: Via, From, To, Call-ID, Content-Type, Content-Length, and Expired. The *Via* header defines the SIP device through which the message passes, including the sender. The *From* header defines the sender, and the *To* header defines the recipient. The *Call-ID* header is a random number that defines the session. The *Content-Type* defines the type of body of the message, which is normally SPD, which we will describe shortly. The *Content-Length* defines the length of the body of the message in bytes. The *Expired* header is normally used in a REGISTER message to define the expiration of the information in the body. The following is an example of a header in an INVITE message.

```
Via: SIP/2.0/UDP 145.23.76.80
From: sip:alice@roadrunner.com
To: sip:bob@arrowhead.net
Call-ID: 23a345@roadrunner.com
Content-Type: application/spd
Content-Length: 600
```

Body

The body of the message is the main difference we will see between an application such as HTTP and SIP. SIP uses another protocol, called *Session Description Protocol* (SDP), to define the body. Each line in the body is made up of an SDP code followed by an equal sign and then followed by the value. The code is a single character that determines the purpose of the code. We can divide the body into several sections.

The first part of the body is normally general information. The codes used in this section are *v* (for version of SDP) and *o* (for origin of the message).

The second part of the body normally gives information to the recipient for making a decision to take part in the session. The codes used in this section are *s* (subject),

i (information about subject), *u* (for session URL), and *e* (the e-mail address of the person responsible for the session).

The third part of the body gives the technical details to make the session possible. The codes used in this part are *c* (the unicast or multicast IP address that the user needs to join to be able to take part in the session), *t* (the start time and end time of the session, encoded as integers), and *m* (the information about media such as audio, video, the port number, the protocol used).

The following shows an example of a body of an INVITE request message.

```
v=0
o=forouzan 64.23.45.8
s=computer classes
i=what to offer next semester
u=http://www.uni.edu
e=forouzan@roadrunner.com
c=IN IP4 64.23.45.8
t=2923721854 2923725454
```

Putting the Parts Together

Let us put the four sections of a message request together as shown below. The first line is the start line; the next six lines make up the header. The next line (blank line) separates the header from the body, and the last eight lines are the body of the message. We conclude our discussion about the SIP protocol and the auxiliary protocol SPD used by SIP to define the body.

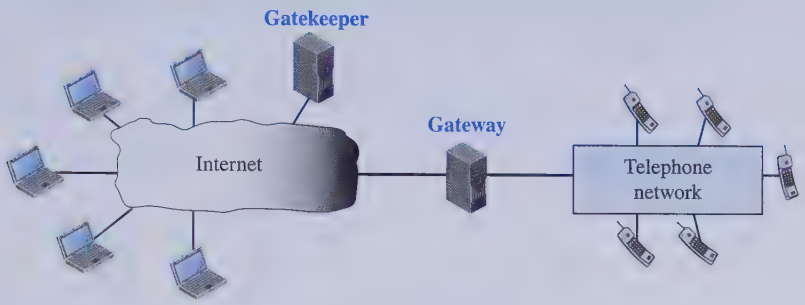
```
INVITE sip:forouzan@roadrunner.com
Via: SIP/2.0/UDP 145.23.76.80
From: sip:alice@roadrunner.com
To: sip:bob@arrowhead.net
Call-ID: 23a345@roadrunner.com
Content-Type: application/spd
Content-Length: 600
// Blank line
v=0
o=forouzan 64.23.45.8
s=computer classes
i=what to offer next semester
u=http://www.uni.edu
e=forouzan@roadrunner.com
c=IN IP4 64.23.45.8
t=2923721854 2923725454
```

11.4.5 H.323

H.323 is a standard designed by the International Telecommunications Union (ITU) to allow telephones on the public telephone network to talk to computers (called *terminals*

in H.323) connected to the Internet. Figure 11.40 shows the general architecture of H.323 for audio, but it can also be used for video.

Figure 11.40 *H.323 architecture*

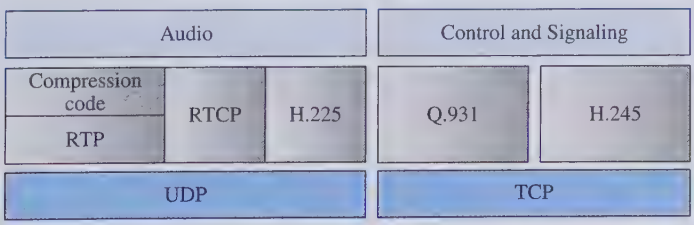


A **gateway** connects the Internet to the telephone network. In general, a gateway is a five-layer device that can translate a message from one protocol stack to another. The gateway here does exactly the same thing. It transforms a telephone network message into an Internet message. The **gatekeeper** server on the local area network plays the role of the registrar server, as we discussed in with SIP in Section 11.4.4.

Protocols

H.323 uses a number of protocols to establish and maintain voice (or video) communication. Figure 11.41 shows these protocols. H.323 uses G.71 or G.723.1 for compression. It uses a protocol named H.245, which allows the parties to negotiate the compression method. Protocol Q.931 is used for establishing and terminating connections. Another protocol, called H.225, or Registration/Administration/Status (RAS), is used for registration with the gatekeeper.

Figure 11.41 *H.323 protocols*



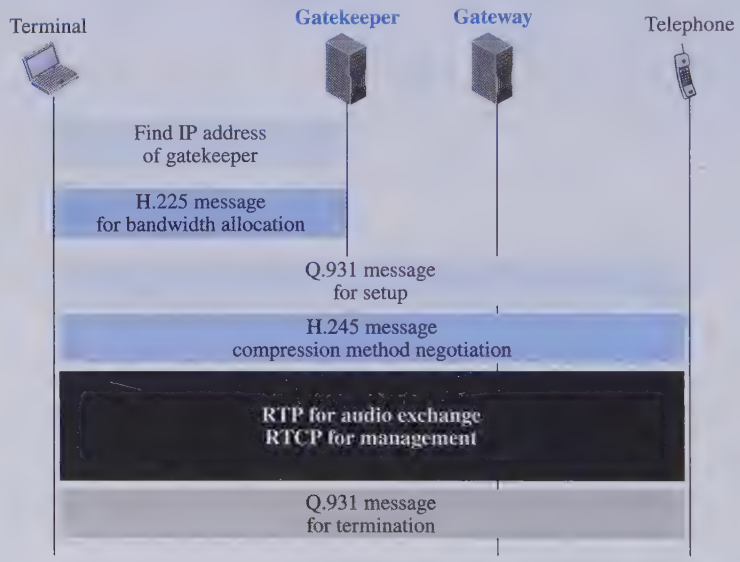
We need to mention that H.323 is a complete set of protocols that cannot be compared with SIP. SIP is only a signaling protocol, which is normally combined with RTP and RTCP to create a complete set of protocols for interactive real-time multimedia

applications, but it can be used with other protocols as well. H.323, on the other hand, is a complete set of protocols that mandates the use of RTP and RTCP.

Operation

Let us use a simple example to show the operation of a telephone communication using H.323. Figure 11.42 shows the steps used by a terminal to communicate with a telephone.

Figure 11.42 *H.323 example*



1. The terminal sends a broadcast message to the gatekeeper. The gatekeeper responds with its IP address.
2. The terminal and gatekeeper communicate, using H.225 to negotiate bandwidth.
3. The terminal, the gatekeeper, gateway, and the telephone communicate using Q.931 to set up a connection.
4. The terminal, the gatekeeper, the gateway, and the telephone communicate using H.245 to negotiate the compression method.
5. The terminal, the gateway, and the telephone exchange audio using RTP under the management of RTCP.
6. The terminal, the gatekeeper, the gateway, and the telephone communicate uses Q.931 to terminate the communication.

11.5 END-OF-CHAPTER MATERIALS

11.5.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give some coverage of multimedia: [Com 06], [Tan 03], and [GW 04].

11.5.2 Key Terms

adaptive DM (ADM)
adaptive DPCM (ADPCM)
arithmetic coding
bidirectional frame (B-frame)
delta modulation (DM)
differential PCM (DPCM)
discrete cosine transform (DCT)
frequency masking
gatekeeper
gateway
H.323
Huffman coding
intracoded frame (I-frame)
jitter
Joint Photographic Experts Group (JPEG)
Lempel-Ziv-Welch (LZW)
linear predictive coding (LPC)
lossless compression
lossy compression
media server
metafile
mixer
Motion Picture Experts Group (MPEG)
MPEG audio layer 3 (MP3)
perceptual coding
playback buffer
predicted frame (P-frame)
predictive coding (PC)
psychoacoustics
Real-Time Streaming Protocol (RTSP)
Real-Time Transport Control Protocol (RTCP)
Real-Time Transport Protocol (RTP)
registrar server
run-length coding
Session Initiation Protocol (SIP)
spatial compression
temporal compression
temporal masking
timestamp
voice over IP

11.5.3 Summary

Multimedia data are normally compressed before transmission. We can divide compression into two broad categories: lossless and lossy compression. In lossless compression, the integrity of the data is preserved because compression and decompression algorithms are exact inverses of each other: No part of the data is lost in the process. Lossy compression cannot preserve the accuracy of data, but we gain the benefit of reducing the size of the compressed data.

Audio/video files can be downloaded for future use (streaming stored audio/video) or broadcast to clients over the Internet (streaming live audio/video). The Internet can also be used for live audio/video interaction. Audio and video need to be digitized before being sent over the Internet. We can use a web server, or a web server with a metafile, or a media server, or a media server and RTSP to download a streaming audio/video file.

Real-time data on a packet-switched network requires the preservation of the time relationship between packets of a session. Gaps between consecutive packets at the

receiver cause a phenomenon called *jitter*. Jitter can be controlled through the use of timestamps and a judicious choice of the playback time.

Real-time multimedia traffic requires both UDP and Real-Time Transport Protocol (RTP). RTP handles timestamping, sequencing, and mixing. Real-Time Transport Control Protocol (RTCP) provides flow control, quality of data control, and feedback to the sources. The Session Initiation Protocol (SIP) is an application-layer protocol that establishes, manages, and terminates multimedia sessions. H.323 is an ITU standard that allows a telephone connected to a public telephone network to talk to a computer connected to the Internet.

11.6 PRACTICE SET

11.6.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

11.6.2 Questions

Q11-1. In dictionary coding, if there are 60 characters in the message, how many times is the loop in the compression algorithm iterated? Explain.

Q11-2. In dictionary coding, should all dictionary entries that are created in the process be used for encoding or decoding?

Q11-3. In an alphabet with 20 symbols, what is the number of leaves in a Huffman tree?

Q11-4. Is the following code an instantaneous one? Explain.

00 01 10 11 001 011 111

Q11-5. Assume a message is made up of four characters (A, B, C, and D) with equal probability of occurrence. Guess what the encoding Huffman table for this message would be. Does encoding here really decrease the number of bits to be sent?

Q11-6. In arithmetic coding, could two different messages be encoded in the same interval? Explain.

Q11-7. In predictive coding, differentiate between DM and ADM.

Q11-8. In predictive coding, differentiate between DPCM and ADPCM.

Q11-9. Compare the number of bits transmitted for each PCM and DM sample for the following maximum quantized values.

a. 12 **b.** 30 **c.** 50

Q11-10. Answer the following questions about predictive coding.

a. What are slope overload distortion and granular noise distortion in DM coding?

b. Explain how ADM coding solves the errors listed in part a.

Q11-11. What is the difference between DM and DPCM?

Q11-12. What is the problem with a speech signal that is compressed using the LPC method?

Q11-13. In transform coding, when a sender transmits the **M** matrix to a receiver, does the sender need to send the **T** matrix used in the calculation? Explain.

Q11-14. In JPEG, do we need less than 24 bits for each pixel if our image is using one or two primary colors? Explain.

Q11-15. In JPEG, explain why the values of the $\mathbf{Q}(m, n)$ in the quantization matrix are not the same. In other words, why is each element in $\mathbf{M}(m, n)$ not divided by one fixed value instead of a different value?

Q11-16. Explain why using Q10 gives a better compression ratio but a poorer image quality than using Q90 (see Figure 11.18).

Q11-17. In JPEG, explain why we need to round the result of division in the quantization step.

Q11-18. Explain why the combination of quantization/dequantization steps in JPEG is a lossy process even though we divide each element of the matrix **M** by the corresponding value in matrix **Q** when quantizing and multiply it by the same value when dequantizing.

Q11-19. In multimedia communication, assume a sender can encode an image using only JPEG encoding, but the potential receiver can only decode an image if it is encoded in GIF. Can these two entities exchange multimedia data?

Q11-20. When we stream stored audio/video, what is the difference between the first approach (Figure 11.24) and the second approach (Figure 11.25)?

Q11-21. When we stream stored audio/video, what is the difference between the second approach (Figure 11.25) and the third approach (Figure 11.26)?

Q11-22. In the fourth approach to streaming audio/video, what is the role of RTSP?

Q11-23. What is the main difference between live audio/video and real-time interactive audio/video?

Q11-24. When we use audio/video on demand, which of these three types of multimedia communication takes place: streaming stored audio/video, streaming live audio/video, or real-time interactive audio/video?

Q11-25. In Figure 11.26, can the web server and media server run on different machines?

Q11-26. In real-time interactive audio/video, what will happen if a packet arrives at the receiver site after the scheduled playback time?

Q11-27. Assume we devise a protocol with the packet size so large that it can carry all chunks of a live or real-time multimedia stream in one packet. Do we still need sequence numbers or timestamps for the chunks? Explain.

Q11-28. Explain why RTP cannot be used as a transport-layer protocol without being run on the top of another transport-layer protocol such as UDP.

Q11-29. Both TCP and RTP use sequence numbers. Do sequence numbers in these two protocols play the same role? Explain.

Q11-30. Can UDP without RTP provide an appropriate service for real-time interactive multimedia applications?

Q11-31. If we capture RTP packets, most of the time we see the total size of the RTP header as 12 bytes. Can you explain this?

Q11-32. Are encoding and decoding of the multimedia data done by RTP? Explain.

Q11-33. Assume an image is sent from the source to the destination using 10 RTP packets. Can the first five packets define the encoding as JPEG and the last five packets as GIF?

Q11-34. UDP does not create a connection. How are different chunks of data, carried in different RTP packets, glued together?

Q11-35. Assume an application program uses separate audio and video streams during an RTP session. How many SSRCs and CSRCs are used in each RTP packet?

Q11-36. Can we say UDP plus RTP is the same as TCP?

Q11-37. Why does RTP need the service of another protocol, RTCP, but TCP does not?

Q11-38. Does SIP need to use the service of RTP? Explain.

Q11-39. We mentioned that SIP is an application-layer program used to provide a signaling mechanism between the caller and the callee. Which party in this communication is the server, and which one is the client?

Q11-40. Assume two parties need to establish IP telephony using the service of RTP. How can they define the two ephemeral port numbers to be used by RTP, one for each direction?

Q11-41. In which situation, a unicast session or a multicast session, can feedback received from an RTCP packet about the session be handled more easily by the sender?

Q11-42. Can the combination of RTP/RTCP and SIP operate in a wireless environment? Explain.

Q11-43. Do some research, and find out if SIP can provide the following services provided by modern telephone sets.

a. caller-ID **b.** call-waiting **c.** multiparty calling

Q11-44. In Internet telephony, explain how a call from Alice can be directed to Bob when he can be in his office or at home?

Q11-45. Do you think H.323 is actually the same as SIP? What are the differences? Make a comparison between the two.

Q11-46. Can H.323 also be used for video?

11.6.3 Problems

P11-1. Given the following message, find the compressed data using run-length coding.

AAACCCCCCBCCCCDDDDDAAAABBB

P11-2. Given the following message, find the compressed data using the second version of run-length coding with the count expressed as a 4-bit binary number.

10000001000001000000000000010000001

P11-3. In dictionary coding, can you easily find the code if the message is one of the following (the message alphabet has only one character)?

a. "A" b. "AA" c. "AAA"
d. "AAAA" e. "AAAAA" f. "AAAAAA"

P11-4. In LZW coding, the message **"AACCCBCCDDAB"** is given.

a. Encode the message. (See Figure 11.2.)

b. Find the compression ratio if we use 8 bits to represent a character and 4 bits to represent a digit (hexadecimal).

P11-5. In LZW coding, the code "0026163301" is given. Assuming that the alphabet is made up of four characters: "A", "B", "C", and "D", decode the message. (See Figure 11.3.)

P11-6. Given the message **"AACCCBCCDDAB"**, in which the probabilities of symbols are $P(A) = 0.50$, $P(B) = 0.25$, $P(C) = 0.125$, and $P(D) = 0.125$, complete the following.

a. Encode the data using Huffman coding.

b. Find the compression ratio if each original character is represented by 8 bits.

P11-7. In Huffman coding, the following coding table is given.

$$A \rightarrow 0 \qquad B \rightarrow 10 \qquad C \rightarrow 110 \qquad D \rightarrow 111$$

Show the original message if the code "001101100111101111111010" is received.

P11-8. Given the message **"ACCBCAAB*"**, in which the probabilities of symbols are $P(A) = 0.4$, $P(B) = 0.3$, $P(C) = 0.2$, and $P(*) = 0.1$, complete the following.

a. Find the compressed data using arithmetic coding with a precision of 10 binary digits.

b. Find the compression ratio if we use 8 bits to represent a character in the message.

P11-9. In arithmetic coding, assume we have received the code 100110011. If we know that the alphabet is made up of four symbols with the probabilities of $P(A) = 0.4$, $P(B) = 0.3$, $P(C) = 0.2$, and $P(*) = 0.1$, find the original message.

P11-10. In predictive coding, assume we have the following sample (x_n).

n	1	2	3	4	5	6	7	8	9	10	11
x_n	13	24	46	60	45	32	30	40	30	27	20

a. Show the encoded message sent if we use delta modulation (DM). Let $y_0 = 10$ and $\Delta = 8$.

b. From the calculated q_n values, what can you say about the given Δ?

P11-11. In predictive coding, we have the following code. Show how we can calculate the reconstructed value (y_n) for each sample if we use delta modulation (DM). We know that $y_0 = 8$ and $\Delta = 6$.

n	1	2	3	4	5	6	7	8	9	10	11
C_n	1	0	0	1	0	1	1	0	0	1	1

P11-12. In predictive coding, assume we have the following sample (x_n). Show the encoded message sent if we use adaptive DM (ADM). Let $y_0 = 10$, $\Delta_1 = 4$, $M_1 = 1$. Also assume that $M_n = 1.5 \times M_{n-1}$ if $q_n = q_{n-1}$ (no change in q_n) and $M_n = 0.5 \times M_{n-1}$ otherwise.

n	1	2	3	4	5	6	7	8	9	10	11
x_n	13	15	15	17	20	20	18	16	16	17	18

P11-13. Assume we have the following code. Show how we can calculate the reconstructed value (y_n) for each sample if we use ADM. Let $y_0 = 20$, $\Delta_1 = 4$, $M_1 = 1$. Also assume that $M_n = 1.5 \times M_{n-1}$ if $q_n = q_{n-1}$ (no change in q_n) and $M_n = 0.5 \times M_{n-1}$ otherwise.

n	1	2	3	4	5	6	7	8	9	10	11
C_n	1	1	1	0	0	1	1	0	0	1	1

P11-14. In one-dimensional DCT, if $N = 1$, the matrix transformation is changed to simple multiplication. In other words, $M = T \times p$, in which T, p, and M are numbers (scalar value) instead of matrices. What is the value of T in this case?

P11-15. In DCT, is the value of $\mathbf{T}(m, n)$ in transform coding always between -1 and 1? Explain.

P11-16. In transform coding, show that a receiver that receives an **M** matrix can create the original **p** matrix.

P11-17. Calculate the **T** matrix for DCT when $N = 1$, $N = 2$, $N = 4$, and $N = 8$.

P11-18. Using one-dimensional DCT encoding, calculate the **M** matrix from the following three **p** matrices (which are given as row matrices but need to be considered as column matrices). Interpret the result.

$\mathbf{p}_1 = [1\ 2\ 3\ 4\ 5\ 6\ 7\ 8]$ $\mathbf{p}_2 = [1\ 3\ 5\ 7\ 9\ 11\ 13\ 15]$ $\mathbf{p}_3 = [1\ 6\ 11\ 16\ 21\ 26\ 31\ 36]$

P11-19. Assume an image uses a palette of size 8 out of the table used by JPEG (GIF uses the same strategy, but the size of the palette is 256), with the combination of the following colors with the indicated level of intensities.

red: 0 and 7 blue: 0 and 5 green: 0 and 4

Show the palette for this situation, and answer the following questions.

a. How many bits are sent for each pixel?

b. What are the bits sent for the following pixels: red, blue, green, black, white, and magenta (red and blue, but no green)?

P11-20. In the first approach to streaming stored audio/video (Figure 11.24), assume that we need to listen to a compressed song of 4 Mbytes (a typical situation). If our connection to the Internet is via a 56-kbps modem, how long will we need to wait before the song can be started (downloading time)?

P11-21. In Figure 11.31, what is the amount of data in the playback buffer at each of the following times?

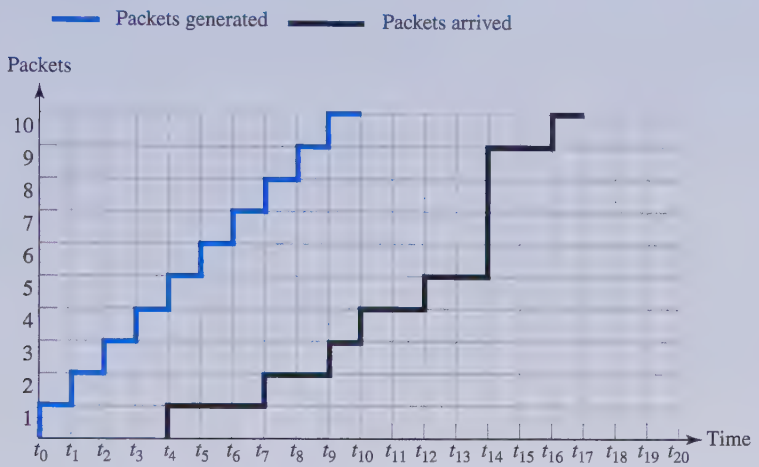
a. 00:00:17 **b.** 00:00:20 **c.** 00:00:25 **d.** 00:00:30

P11-22. Figure 11.43 shows the generated and the arrival time for 10 audio packets. Answer the following questions.

a. If we start our player at t_8, which packets cannot be played?

b. If we start our player at t_9, which packets cannot be played?

Figure 11.43 *Problem P11-22*



P11-23. Given an RTP packet with the first eight hexadecimal digits as $(86032132)_{16}$, answer the following questions.

- **a.** What is the version of the RTP?
- **b.** Is there any padding for security?
- **c.** Is there any extension header?
- **d.** How many contributors are defined in the packet?
- **e.** What is the type of payload carried by the RTP packet?
- **f.** What is the total header size in bytes?

P11-24. In a real-time multimedia communication, assume we have one sender and 10 receivers. If the sender is sending multimedia data at 1 Mbps, how many RTCP packets can be sent by the sender and each receiver in a second? Assume the system allocates 80 percent of the RTCP bandwidth to the receivers and 20 percent to the sender. The average size of each RTCP packet is 1000 bits.

P11-25. Explain why TCP, as a byte-oriented stream protocol, is not suitable for applications such as live or real-time multimedia streaming.

CHAPTER 12

Network Management

Although network management is implemented at the application layer of the TCP/IP protocol suite, we have dedicated one chapter to this issue to be able to discuss it in more detail. Network management plays an important role in the Internet as it becomes larger and larger. The failure of a single device may interrupt the communication from one point of the Internet to the other. In this chapter, we first discuss the areas of network management. We then discuss how one of these areas is implemented at the application layer of the TCP/IP suite.

This chapter is divided into three sections.

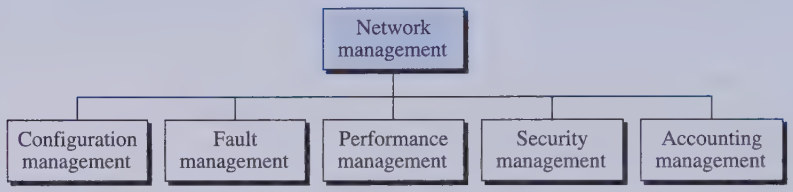
- The first section introduces the concept of network management and discusses five general areas of network management: configuration, fault, performance, security, and accounting. *Configuration management* is related to the status of each entity and its relationship to other entities. *Fault management* is the area of network management that handles issues related to interruptions in the system. *Performance management* tries to monitor and control the network to ensure that it is running as efficiently as possible. *Security management* is responsible for controlling access to the network based on predefined policy. *Accounting management* is the controlling of users' access to network resources through charges.
- The second section discusses Simple Network Management Protocol (SNMP) as a framework for managing devices in an internet using the TCP/IP protocol suite and show how a manager as a host runs an SNMP client and any agents as a router or host runs a server program. The section defines the three components of the management protocol in the Internet. The section also defines Structure of Management Information (SMI) as the language that specifies how data types and objects in SNMP should be identified. Finally, the section introduces Management Information Base (MIB), which designates the objects to be managed in SNMP according to the rules defined in SMI.
- The third section gives a brief discussion of a standard that provides the methods and rules to define data and objects. This section is very brief and only introduces the subject. Part of it is used by SMI in the second section.

12.1 INTRODUCTION

We can define *network management* as monitoring, testing, configuring, and troubleshooting network components to meet a set of requirements defined by an organization. These requirements include the smooth, efficient operation of the network that provides the predefined quality of service for users. To accomplish this task, a network management system uses hardware, software, and humans.

The International Organization for Standardization (ISO) defines five areas of network management: configuration management, fault management, performance management, security management, and accounting management, as shown in Figure 12.1.

Figure 12.1 *Areas of network management*



Although some organizations include other areas, such as cost management, we believe the ISO taxonomy is specific to network management. For example, cost management is a general management area for any management system and not just for network management.

12.1.1 Configuration Management

A large network is usually made up of hundreds of entities that are physically or logically connected to each other. These entities have an initial configuration when the network is set up but can change with time. Desktop computers may be replaced by others; application software may be updated to a newer version; and users may move from one group to another. The *configuration management* system must know, at any time, the status of each entity and its relation to other entities. Configuration management can be divided into two subsystems: *reconfiguration* and *documentation*.

Reconfiguration

Reconfiguration can be a daily occurrence in a large network. There are three types of reconfiguration: *hardware reconfiguration*, *software reconfiguration*, and *user-account reconfiguration*.

Hardware Reconfiguration

Hardware reconfiguration covers all changes to the hardware. For example, a desktop computer may need to be replaced. A router may need to be moved to another part of the network. A subnetwork may be added or removed from the network. All these need the

time and attention of network management. In a large network, there must be specialized personnel trained for quick and efficient hardware reconfiguration. Unfortunately, this type of reconfiguration cannot be automated and must be manually handled case by case.

Software Reconfiguration

Software reconfiguration covers all changes to the software. For example, new software may need to be installed on servers or clients. An operating system may need updating. Fortunately, most software reconfiguration can be automated. For example, an update for an application on some or all clients can be electronically downloaded from the server.

User-Account Reconfiguration

User-account reconfiguration is not simply adding or deleting users on a system. We must also consider each user's privileges, both as an individual and as a member of a group. For example, a user may have both read and write permission with regard to some files, but only read permission with regard to other files. User-account reconfiguration can be, to some extent, automated. For example, in a college or university, at the beginning of each quarter or semester, new students are added to the system. The students are normally grouped according to the courses they take or the majors they pursue. The members of each group have specific privileges; computer science students may need to access a server providing different computer language facilities, while engineering students may need to access servers that provide computer-assisted design (CAD) software.

Documentation

The original network configuration and each subsequent change must be recorded meticulously. This means that there must be documentation for hardware, software, and user accounts.

Hardware Documentation

Hardware documentation normally involves two sets of documents: maps and specifications.

Maps *Maps* track each piece of hardware and its connection to the network. There can be one general map that shows the logical relationships between subnetworks. There can also be a second general map that shows the physical location of each subnetwork. For each subnetwork, then, there is one or more maps that show all pieces of equipment. The maps use some kind of standardization to be easily read and understood by current and future personnel.

Specifications Maps are not enough per se. Each piece of hardware also needs to be documented. There must be a set of *specifications* for each piece of hardware connected to the network. These specifications must include information such as hardware type, serial number, vendor (address and phone number), time of purchase, and warranty information.

Software Documentation

All software must also be documented. *Software documentation* includes information such as the software type, the version, the time installed, and the license agreement.

User-Account Documentation

Most operating systems have a utility that allows *user-account documentation*. The management must make sure that the files with this information are updated and secured. Some operating systems record access privileges in two documents. One shows all files and access types for each user; the other shows the list of users that have access to a particular file.

12.1.2 Fault Management

Complex networks today are made up of hundreds and sometimes thousands of components. Proper operation of the network depends on the proper operation of each component individually and in relation to each other. *Fault management* is the area of network management that handles this issue. An effective fault management system has two subsystems: reactive fault management and proactive fault management.

Reactive Fault Management

A *reactive fault management* system is responsible for detecting, isolating, correcting, and recording faults. It handles short-term solutions to faults.

Detecting Fault

The first step taken by a reactive fault management system is to find the exact location of the fault. A fault is defined as an abnormal condition in the system. When a fault occurs, either the system stops working properly or the system creates excessive errors. A good example of a fault is a damaged communication medium.

Isolating Fault

The next step taken by a reactive fault management system is isolating the fault. A fault, if isolated, usually affects only a few users. After isolation, the affected users are immediately notified and given an estimated time of correction.

Correcting Fault

The next step is correcting the fault. This may involve replacing or repairing the faulty components.

Recording Fault

After the fault is corrected, it must be documented. The record should show the exact location of the fault, the possible cause, the action or actions taken to correct the fault, the cost, and the time it took for each step. Documentation is extremely important for several reasons:

- The problem may reoccur. Documentation can help the present or future administrator or technician solve a similar problem.
- The frequency of the same kind of failure is an indication of a major problem in the system. If a fault happens frequently in one component, the component should be replaced with a similar one or the whole system should be changed to avoid the use of that type of component.
- This statistic is helpful to another part of network management, performance management.

Proactive Fault Management

Proactive fault management tries to prevent faults from occurring. Although this is not always possible, some types of failures can be predicted and prevented. For example, if a manufacturer specifies a lifetime for a component or a part of a component, it is a good strategy to replace it before that time. As another example, if a fault happens frequently at one particular point of a network, it is wise to carefully reconfigure the network to prevent the fault from happening again.

12.1.3 Performance Management

Performance management, which is closely related to fault management, tries to monitor and control the network to ensure that it is running as efficiently as possible. Performance management tries to quantify performance using some measurable quantity, such as capacity, traffic, throughput, or response time. Some protocols, such as SNMP, which is discussed in this chapter, can be used in performance management.

Capacity

One factor that must be monitored by a performance management system is the *capacity* of the network. Every network has a limited capacity, and the performance management system must ensure that it is not used above this capacity. For example, if a LAN is designed for 100 stations at an average data rate of 2 Mbps, it will not operate properly if 200 stations are connected to the network. The data rate will decrease and blocking may occur.

Traffic

Traffic can be measured in two ways: internally and externally. Internal traffic is measured by the number of packets (or bytes) traveling inside the network. External traffic is measured by the exchange of packets (or bytes) outside the network. During peak hours, when the system is heavily used, blocking may occur if there is excessive traffic.

Throughput

We can measure the *throughput* of an individual device (such as a router) or a part of the network. Performance management monitors the throughput to make sure that it is not reduced to unacceptable levels.

Response Time

Response time is normally measured from the time a user requests a service to the time the service is granted. Other factors such as capacity and traffic can affect the response time. Performance management monitors the average response time and the peak-hour response time. Any increase in response time is a very serious condition as it is an indication that the network is working above its capacity.

12.1.4 Security Management

Security management is responsible for controlling access to the network based on predefined policy. In Chapter 13, we will discuss security tools such as encryption and authentication. Encryption allows privacy for users; authentication forces the users to identify themselves.

12.1.5 Accounting Management

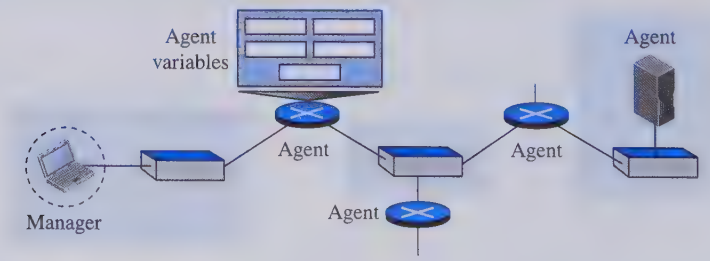
Accounting management is the controlling of users' access to network resources through charges. Under accounting management, individual users, departments, divisions, or even projects are charged for the services they receive from the network. Charging does not necessarily mean cash transfer; it may mean debiting the departments or divisions for budgeting purposes. Today, organizations use an accounting management system for the following reasons:

- It prevents users from monopolizing limited network resources.
- It prevents users from using the system inefficiently.
- Network managers can do short- and long-term planning based on the demand for network use.

12.2 SNMP

Several network management standards have been devised during the last few decades. The most important one is **Simple Network Management Protocol (SNMP)**, used by the Internet. We discuss this standard in this section. SNMP is a framework for managing devices in an internet using the TCP/IP protocol suite. It provides a set of fundamental operations for monitoring and maintaining an internet. SNMP uses the concept of manager and agent. That is, a manager, usually a host, controls and monitors a set of agents, usually routers or servers (see Figure 12.2).

Figure 12.2 *SNMP concept*



SNMP is an application-level protocol in which a few manager stations control a set of agents. The protocol is designed at the application level so that it can monitor devices made by different manufacturers and be installed on different physical networks. In other words, SNMP frees management tasks from both the physical characteristics of the managed devices and the underlying networking technology. It can be used in a heterogeneous internet made up of different LANs and WANs connected by routers made by different manufacturers.

12.2.1 Managers and Agents

A management station, called a *manager*, is a host that runs the SNMP client program. A managed station, called an *agent*, is a router (or a host) that runs the SNMP server program. Management is achieved through simple interaction between a manager and an agent.

The agent keeps performance information in a database. The manager has access to the values in the database. For example, a router can store in appropriate variables the number of packets received and forwarded. The manager can fetch and compare the values of these two variables to see if the router is congested or not.

The manager can also make the router perform certain actions. For example, a router periodically checks the value of a reboot counter to see when it should reboot itself. It reboots itself, for example, if the value of the counter is 0. The manager can use this feature to reboot the agent remotely at any time. It simply sends a packet to force a 0 value in the counter.

Agents can also contribute to the management process. The server program running on the agent can check the environment and, if it notices something unusual, it can send a warning message (called a **Trap**) to the manager.

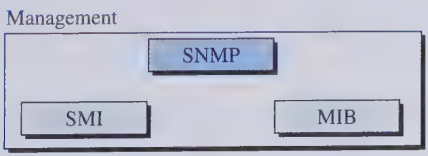
In other words, management with SNMP is based on three basic ideas:

1. A manager checks an agent by requesting information that reflects the behavior of the agent.
2. A manager forces an agent to perform a task by resetting values in the agent database.
3. An agent contributes to the management process by warning the manager of an unusual situation.

12.2.2 Management Components

To do management tasks, SNMP uses two other protocols: **Structure of Management Information (SMI)** and **Management Information Base (MIB)**. In other words, management on the Internet is done through the cooperation of three protocols: SNMP, SMI, and MIB, as shown in Figure 12.3.

Figure 12.3 *Components of network management on the Internet*



Let us elaborate on the interactions between these protocols.

Role of SNMP

SNMP has some very specific roles in network management. It defines the format of the packet to be sent from a manager to an agent, and vice versa. It also interprets the

result and creates statistics (often with the help of other management software). The packets exchanged contain the object (variable) names and their status (values). SNMP is responsible for reading and changing these values.

SNMP defines the format of packets exchanged between a manager and an agent. It reads and changes the status of objects (values of variables) in SNMP packets.

Role of SMI

To use SNMP, we need rules for naming objects. This is particularly important because the objects in SNMP form a hierarchical structure (an object may have a parent object and some child objects). Part of a name can be inherited from the parent. We also need rules to define the types of objects. What types of objects are handled by SNMP? Can SNMP handle simple types or structured types? How many simple types are available? What are the sizes of these types? What is the range of these types? In addition, how are each of these types encoded?

SMI defines the general rules for naming objects, defining object types (including range and length), and showing how to encode objects and values.

We need these universal rules because we do not know the architecture of the computers that send, receive, or store these values. The sender may be a powerful computer in which an integer is stored as 8-byte data; the receiver may be a small computer that stores an integer as 4-byte data.

SMI is a protocol that defines these rules. However, we must understand that SMI only defines the rules; it does not define how many objects are managed in an entity or which object uses which type. SMI is a collection of general rules to name objects and to list their types. The association of an object with the type is not done by SMI.

Role of MIB

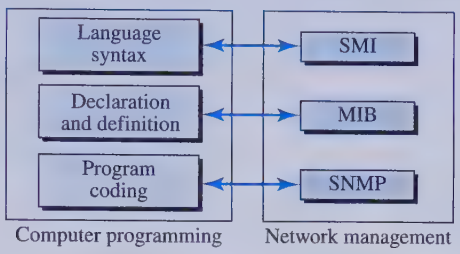
We hope it is clear that we need another protocol. For each entity to be managed, this protocol must define the number of objects, name them according to the rules defined by SMI, and associate a type to each named object. This protocol is MIB. MIB creates a set of objects defined for each entity in a manner similar to that of a database (mostly metadata in a database, names and types without values).

MIB creates a collection of named objects, their types, and their relationships to each other in an entity to be managed.

An Analogy

Before discussing each of these protocols in more detail, let us give an analogy. The three network management components are similar to what we need when we write a program in a computer language to solve a problem. Figure 12.4 shows the analogy.

Figure 12.4 *Comparing computer programming and network management*



Syntax: SMI

Before we write a program, the syntax of the language (such as C or Java) must be predefined. The language also defines the structure of variables (simple, structured, pointer, and so on) and how the variables must be named. For example, a variable name must be 1 to *n* characters in length and start with a letter followed by alphanumeric characters. The language also defines the type of data to be used (integer, real, character, etc.). In programming, the rules are defined by the syntax of the language. In network management, the rules are defined by SMI.

Object Declaration and Definition: MIB

Most computer languages require that objects be declared and defined in each specific program. Declaration and definition create objects using predefined types and allocate memory location for them. For example, if a program has two variables (an integer named *counter* and an array named *grades* of type char), they must be declared at the beginning of the program:

```
int counter;
char grades [40];
```

MIB does this task in network management. MIB names each object and defines the type of the objects. Because the type is defined by SMI, SNMP knows the range and size.

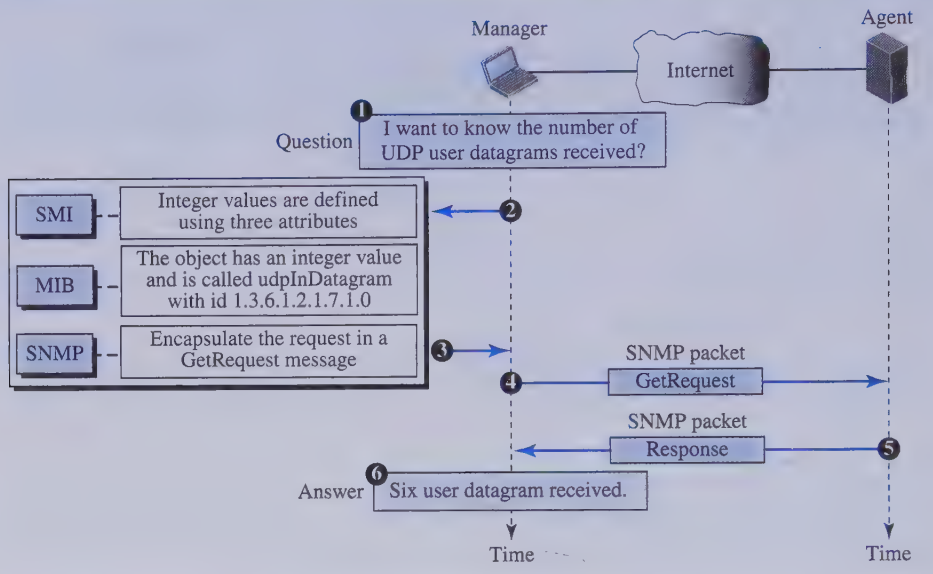
Program Coding: SNMP

After declaration in programming, the program needs to write statements to store values in the variables and change them if needed. SNMP does this task in network management. SNMP stores, changes, and interprets the values of objects already declared by MIB according to the rules defined by SMI.

12.2.3 An Overview

Before discussing each component in more detail, we will show how each of these components is involved in a simple scenario. This is an overview that will be developed later, at the end of the chapter. A manager station (SNMP client) wants to send a message to an agent station (SNMP server) to find the number of UDP user datagrams received by the agent. Figure 12.5 shows an overview of the steps involved.

Figure 12.5 *Management overview*



MIB is responsible for finding the object that holds the number of UDP user datagrams received. SMI, with the help of another embedded protocol, is responsible for encoding the name of the object. SNMP is responsible for creating a message, called a GetRequest message, and encapsulating the encoded message. Of course, things are more complicated than this simple overview, but we first need more details of each protocol.

12.2.4 SMI

The Structure of Management Information, version 2 (SMIv2), is a component for network management. SMI is a guideline for SNMP. It emphasizes three attributes to handle an object: name, data type, and encoding method. Its functions are to:

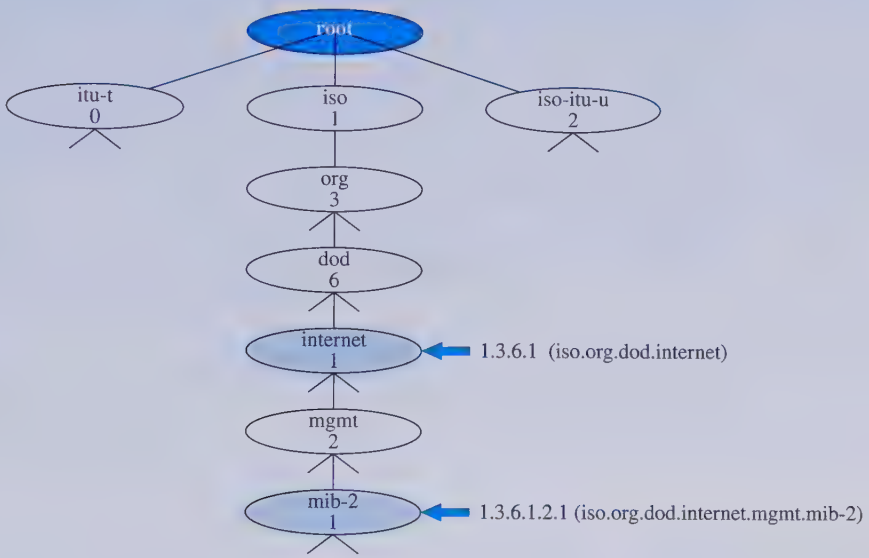
- Name objects
- Define the type of data that can be stored in an object
- Show how to encode data for transmission over the network

Name

SMI requires that each managed object (such as a router, a variable in a router, and a value) have a unique name. To name objects globally, SMI uses an object identifier, which is a hierarchical identifier based on a tree structure (see Figure 12.6).

The tree structure starts with an unnamed root. Each object can be defined using a sequence of integers separated by dots. The tree structure can also define an object using a sequence of textual names separated by dots.

Figure 12.6 *Object identifier in SMI*



The integer-dot representation is used in SNMP. The name-dot notation is used by people. For example, the following shows the same object in two different notations.

The objects that are used in SNMP are located under the *mib-2* object, so their identifiers always start with 1.3.6.1.2.1.

Type

The second attribute of an object is the type of data stored in it. To define the data type, SMI uses Abstract Syntax Notation One (ASN.1) definitions and adds some new definitions. In other words, SMI is both a subset and a superset of ASN.1. (We discuss ASN.1 in Section 12.3).

iso.org.dod.internet.mgmt.mib-2 ↔ **1.3.6.1.2.1**

SMI has two broad categories of data type: *simple* and *structured.* We first define the simple types and then show how the structured types can be constructed from the simple ones.

Simple Type

The **simple data types** are atomic data types. Some of them are taken directly from ASN.1; some are added by SMI. The most important ones are given in Table 12.1. The first five are from ASN.1; the next seven are defined by SMI.

Table 12.1 *Data types*

Type	*Size*	*Description*
INTEGER	4 bytes	An integer with a value between -2^{31} and $2^{31} - 1$.
Integer32	4 bytes	Same as INTEGER.
Unsigned32	4 bytes	Unsigned with a value between 0 and $2^{32} - 1$.
OCTET STRING	Variable	Byte-string up to 65,535 bytes long.
OBJECT IDENTIFIER	Variable	An object identifier.
IPAddress	4 bytes	An IP address made up of four integers.
Counter32	4 bytes	An integer whose value can be incremented from zero to 2^{32}; when it reaches its maximum value, it wraps back to zero.
Counter64	8 bytes	64-bit counter.
Gauge32	4 bytes	Same as Counter32, but when it reaches its maximum value, it does not wrap; it remains there until it is reset.
TimeTicks	4 bytes	A counting value that records time in 1/100ths of a second.
BITS		A string of bits.
Opaque	Variable	Uninterpreted string.

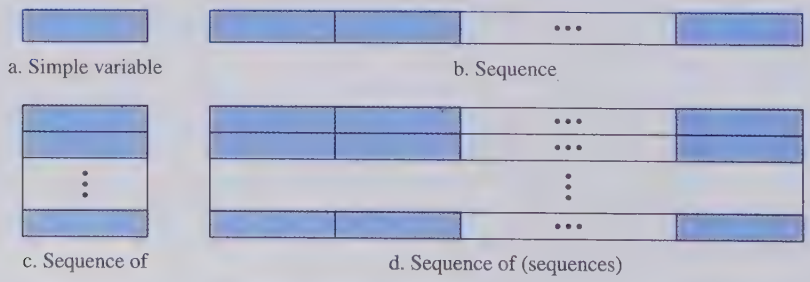
Structured Type

By combining simple and structured data types, we can make new structured data types. SMI defines two **structured data types**: *sequence* and *sequence of*.

- **Sequence.** A *sequence* data type is a combination of simple data types, not necessarily of the same type. It is analogous to the concept of a *struct* or a *record* used in programming languages such as C.
- **Sequence of.** A *sequence of* data type is a combination of simple data types all of the same type or a combination of sequence data types all of the same type. It is analogous to the concept of an *array* used in programming languages such as C.

Figure 12.7 shows a conceptual view of data types.

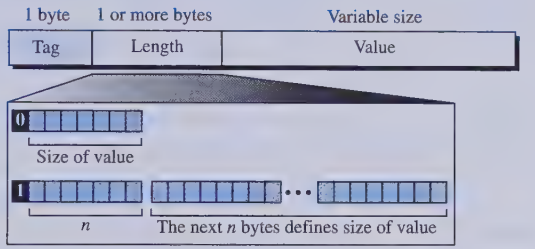
Figure 12.7 *Conceptual data types*



Encoding Method

SMI uses another standard, **Basic Encoding Rules (BER)**, to encode data to be transmitted over the network. BER specifies that each piece of data be encoded in triplet format: tag, length, and value (TLV), as illustrated in Figure 12.8.

Figure 12.8 *Encoding format*



The tag is a 1-byte field that defines the type of data. Table 12.2 shows the data types we use in this chapter and their tags in hexadecimal numbers. The length field is 1 or more bytes. If it is 1 byte, the most significant bit must be 0. The other 7 bits define the length of the data. If it is more than 1 byte, the most significant bit of the first byte must be 1. The other 7 bits of the first byte specify the number of bytes needed to define the length. The value field codes the value of the data according to the rules defined in BER.

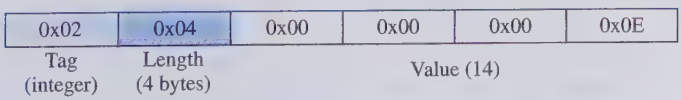
Table 12.2 *Codes for data types*

Data Type	*Tag (Hex)*	*Data Type*	*Tag (Hex)*
INTEGER	**02**	IPAddress	**40**
OCTET STRING	**04**	Counter	**41**
OBJECT IDENTIFIER	**06**	Gauge	**42**
NULL	**05**	TimeTicks	**43**
SEQUENCE, SEQUENCE OF	**30**	Opaque	**44**

Example 12.1

Figure 12.9 shows how to define INTEGER 14. The size of the length field is from Table 12.1.

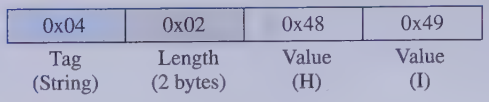
Figure 12.9 *Example 12.1: INTEGER 14*



Example 12.2

Figure 12.10 shows how to define the OCTET STRING "HI."

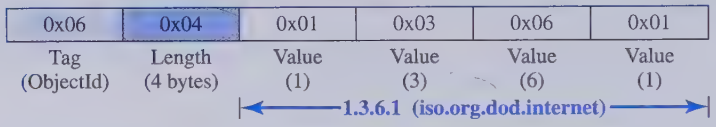
Figure 12.10 *Example 12.2: OCTET STRING "HI"*



Example 12.3

Figure 12.11 shows how to define ObjectIdentifier 1.3.6.1 (iso.org.dod.internet)

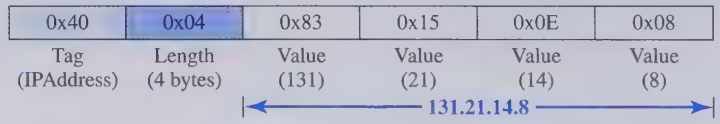
Figure 12.11 *Example 12.3: ObjectIdentifier 1.3.6.1*



Example 12.4

Figure 12.12 shows how to define IPAddress 131.21.14.8.

Figure 12.12 *Example 12.4: IPAddress 131.21.14.8*

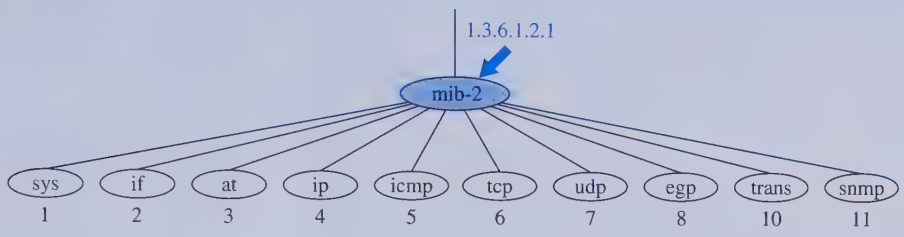


12.2.5 MIB

The Management Information Base, version 2 (MIB2), is the second component used in network management. Each agent has its own MIB2, which is a collection of all the objects that the manager can manage. (See Figure 12.13.)

The objects in MIB2 are categorized under several groups: system, interface, address translation, ip, icmp, tcp, udp, egp, transmission, and snmp (note that group 9 is deprecated). These groups are under the mib-2 object in the object identifier tree. Each group has defined variables and/or tables.

Figure 12.13 *Some mib-2 groups*



The following is a brief description of some of the objects:

- **sys.** This object (*system*) defines general information about the node (system), such as the name, location, and lifetime.
- **if.** This object (*interface*) defines information about all the interfaces of the node including interface number, physical address, and IP address.
- **at.** This object (*address translation*) defines the information about the ARP table.
- **ip.** This object defines information related to IP, such as the routing table and the IP address.
- **icmp.** This object defines information related to ICMP, such as the number of packets sent and received and total errors created.
- **tcp.** This object defines general information related to TCP, such as the connection table, time-out value, number of ports, and number of packets sent and received.
- **udp.** This object defines general information related to UDP, such as the number of ports and number of packets sent and received.
- **egp.** Used for objects related to the operation of EGP.
- **trans.** Used for objects related to the specific method of transmission (future use)
- **snmp.** This object defines general information related to SNMP itself.

Accessing MIB Variables

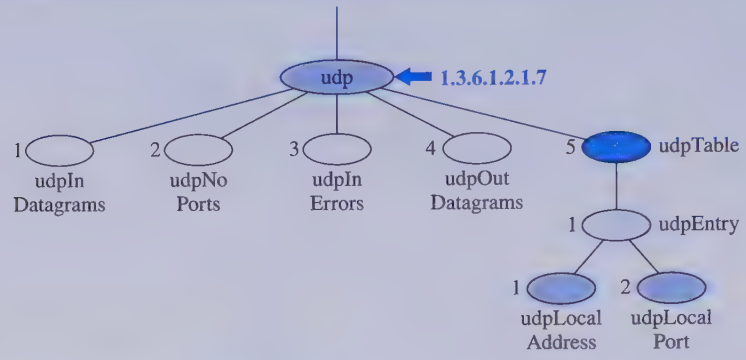
To show how to access different variables, we use the udp group as an example. There are four simple variables in the udp group and one sequence of (table of) records. Figure 12.14 shows the variables and the table. We will show how to access each entity.

Simple Variables

To access any of the simple variables, we use the id of the group (1.3.6.1.2.1.7) followed by the id of the variable. The following shows how to access each variable.

udpInDatagrams	→	**1.3.6.1.2.1.7.1**
udpNoPorts	→	**1.3.6.1.2.1.7.2**
udpInErrors	→	**1.3.6.1.2.1.7.3**
udpOutDatagrams	→	**1.3.6.1.2.1.7.4**

Figure 12.14 *udp group*



However, these object identifiers define the variable, not the instance (contents). To show the instance, or the contents, of each variable, we must add an instance suffix. The instance suffix for a simple variable is simply a zero. In other words, to show an instance of the above variables, we use the following:

udpInDatagrams.0	→	**1.3.6.1.2.1.7.1.0**
udpNoPorts.0	→	**1.3.6.1.2.1.7.2.0**
udpInErrors.0	→	**1.3.6.1.2.1.7.3.0**
udpOutDatagrams.0	→	**1.3.6.1.2.1.7.4.0**

Tables

To identify a table, we first use the table id. The udp group has only one table (with id 5), as illustrated in Figure 12.15. So to access the table, we use the following:

udpTable → **1.3.6.1.2.1.7.5**

However, the table is not at the leaf level in the tree structure. We cannot access the table; we define the entry (sequence) in the table (with id of 1), as follows:

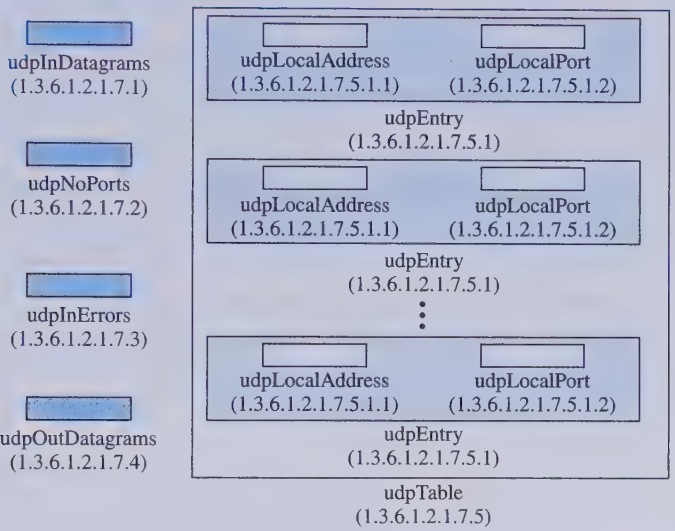
udpEntry → **1.3.6.1.2.1.7.5.1**

This entry is also not a leaf, and we cannot access it. We need to define each entity (field) in the entry.

udpLocalAddress	→	**1.3.6.1.2.1.7.5.1.1**
udpLocalPort	→	**1.3.6.1.2.1.7.5.1.2**

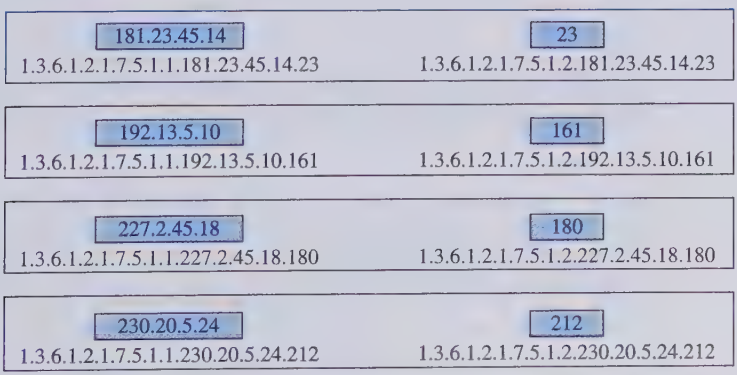
These two variables are at the leaf level of the tree. Although we can access their instances, we need to define *which* instance. At any moment, the table can have several values for each local address/local port pair. To access a specific instance (row) of the table, we add the index to the above ids. In MIB, the indexes of arrays are not integers

Figure 12.15 *udp variables and tables*



(unlike most programming languages). The indexes are based on the value of one or more fields in the entries. In our example, the udpTable is indexed based on both the local address and the local port number. For example, Figure 12.16 shows a table with four rows and values for each field. The index of each row is a combination of two values. To access the instance of the local address for the first row, we use the identifier augmented with the instance index:

Figure 12.16 *Indexes for udpTable*



12.2.6 SNMP Operation

SNMP uses both SMI and MIB in Internet network management. It is an application program that allows:

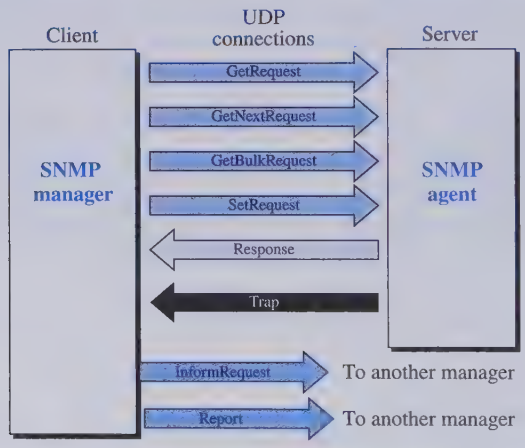
- A manager to retrieve the value of an object defined in an agent.
- A manager to store a value in an object defined in an agent.
- An agent to send an alarm message about an abnormal situation to the manager.

udpLocalAddress.181.23.45.14.23 → **1.3.6.1.2.7.5.1.1.181.23.45.14.23**

PDUs

SNMPv3 defines eight types of protocol data units (or PDUs): *GetRequest*, *GetNext Request*, *GetBulkRequest*, *SetRequest*, *Response*, *Trap*, *InformRequest*, and *Report* (see Figure 12.17).

Figure 12.17 *SNMP PDUs*



GetRequest

The GetRequest PDU is sent from the manager (client) to the agent (server) to retrieve the value of a variable or a set of variables.

GetNextRequest

The GetNextRequest PDU is sent from the manager to the agent to retrieve the value of a variable. The retrieved value is the value of the object following the defined ObjectId in the PDU. It is mostly used to retrieve the values of the entries in a table. If the manager does not know the indexes of the entries, it cannot retrieve the values. However, it can use GetNextRequest and define the ObjectId of the table. Because the first entry has the ObjectId immediately after the ObjectId of the table, the value of the first entry is returned. The manager can use this ObjectId to get the value of the next one, and so on.

GetBulkRequest

The GetBulkRequest PDU is sent from the manager to the agent to retrieve a large amount of data. It can be used instead of multiple GetRequest and GetNextRequest PDUs.

SetRequest

The SetRequest PDU is sent from the manager to the agent to set (store) a value in a variable.

Response

The Response PDU is sent from an agent to a manager in response to GetRequest or GetNextRequest. It contains the value(s) of the variable(s) requested by the manager.

Trap

The **Trap** (also called SNMPv2 Trap to distinguish it from SNMPv1 Trap) PDU is sent from the agent to the manager to report an event. For example, if the agent is rebooted, it informs the manager and reports the time of rebooting.

InformRequest

The InformRequest PDU is sent from one manager to another remote manager to get the value of some variables from agents under the control of the remote manager. The remote manager responds with a Response PDU.

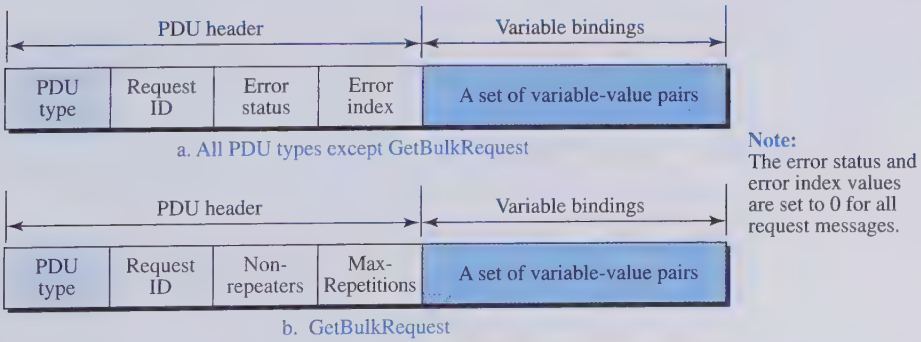
Report

The Report PDU is designed to report some types of errors between managers. It is not yet in use.

Format

The format for the eight SNMP PDUs is shown in Figure 12.18. The GetBulkRequest PDU differs from the others in two areas, as shown in the figure.

Figure 12.18 *SNMP PDU format*



The fields are listed here:

- **PDU type.** This field defines the type of the PDU (see Table 12.3).

Table 12.3 *PDU types*

Type	*Tag (Hex)*	*Type*	*Tag (Hex)*
GetRequest	**A0**	GetBulkRequest	**A5**
GetNextRequest	**A1**	InformRequest	**A6**
Response	**A2**	Trap (SNMPv2)	**A7**
SetRequest	**A3**	Report	**A8**

- **Request ID.** This field is a sequence number used by the manager in a request PDU and repeated by the agent in a response. It is used to match a request to a response.
- **Error status.** This is an integer that is used only in response PDUs to show the types of errors reported by the agent. Its value is 0 in request PDUs. Table 12.4 lists the types of errors that can occur.

Table 12.4 *Types of errors*

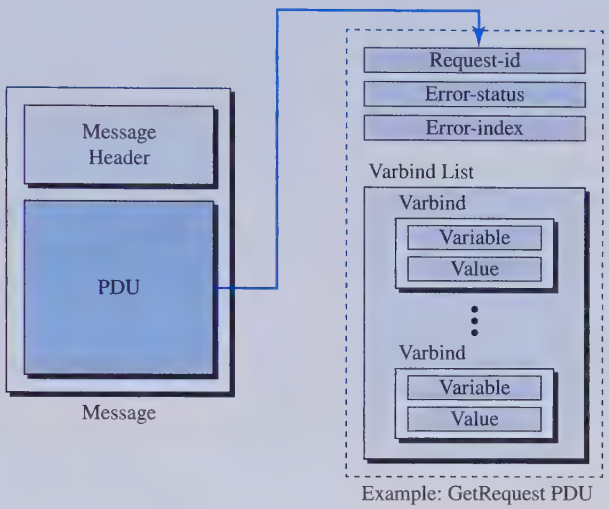
Status	*Name*	*Meaning*
0	noError	No error.
1	tooBig	Response too big to fit in one message.
2	noSuchName	Variable does not exist.
3	badValue	The value to be stored is invalid.
4	readOnly	The value cannot be modified.
5	genErr	Other errors.

- **Nonrepeaters.** This field is used only in GetBulkRequest PDU. The field defines the number of nonrepeating (regular objects) at the start of the variable-value list.
- **Error index.** The error index is an offset that tells the manager which variable caused the error.
- **Max-repetitions.** This field is also used only in GetBulkRequest PDU. The field defines the maximum number of iterations in the table to read all repeating object.
- **Variable-value pair list.** This is a set of variables with the corresponding values the manager wants to retrieve or set. The values are null in request PDUs.

Messages

SNMP does not send only PDUs; it embeds each PDU in a message. A message is made up of a message header followed by the corresponding PDU as shown in Figure 12.19. The format of the message header, which depends on the version and the security provision, is not shown in Figure 12.19. We leave the details to some specific text.

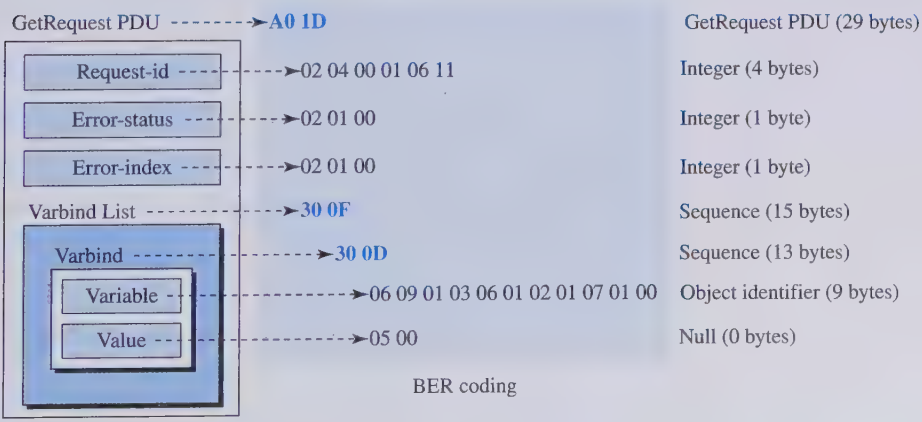
Figure 12.19 *SNMP message*



Example 12.5

In this example, a manager station (SNMP client) uses a message with a GetRequest PDU to retrieve the number of UDP datagrams that a router has received (Figure 12.20).

Figure 12.20 *Example 12.5*



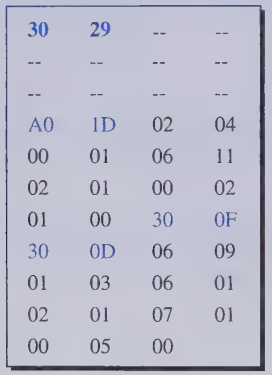
There is only one Varbind sequence. The corresponding MIB variable related to this information is udpInDatagrams with the object identifier 1.3.6.1.2.1.7.1.0. The manager wants to retrieve a value (not to store a value), so the value defines a null entity. The bytes to be sent are shown in hexadecimal representation.

The Varbind list has only one Varbind. The variable is of type 06 and length 09. The value is of type 05 and length 00. The whole Varbind is a sequence of length 0D (13). The Varbind list is also a sequence of length 0F (15). The GetRequest PDU is of length ID (29).

Note that we have intended the bytes to show the inclusion of simple data types inside a sequence or the inclusion of sequences and simple data types inside larger sequences. Note that the PDU itself is like a sequence, but its tag is A0 in hexadecimal.

Figure 12.21 shows the actual message sent. We assume that the message header is made up of 10 bytes. The actual message header may be different. We show the message using rows of 4 bytes. The bytes shown with dashes are the ones related to the message header.

Figure 12.21 *Actual message sent for Example 12.5*



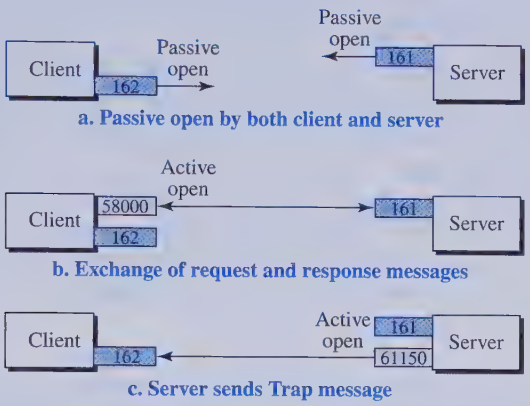
UDP Ports

SNMP uses the services of UDP on two well-known ports, 161 and 162. The well-known port 161 is used by the server (agent), and the well-known port 162 is used by the client (manager).

The agent (server) issues a passive open on port 161. It then waits for a connection from a manager (client). A manager (client) issues an active open using an ephemeral port. The request messages are sent from the client to the server using the ephemeral port as the source port and the well-known port 161 as the destination port. The response messages are sent from the server to the client using the well-known port 161 as the source port and the ephemeral port as the destination port.

The manager (client) issues a passive open on port 162. It then waits for a connection from an agent (server). Whenever it has a Trap message to send, an agent (server) issues an active open, using an ephemeral port. This connection is only one-way, from the server to the client (see Figure 12.22).

Figure 12.22 *Port numbers for SNMP*



The client/server mechanism in SNMP is different from other protocols. Here both the client and the server use well-known ports. In addition, both the client and the server are running infinitely. The reason is that request messages are initiated by a manager (client), but Trap messages are initiated by an agent (server).

Security

SNMPv3 has added two new features to the previous version: security and remote administration. SNMPv3 allows a manager to choose one or more levels of security when accessing an agent. Different aspects of security can be configured by the manager to allow message authentication, confidentiality, and integrity.

SNMPv3 also allows remote configuration of security aspects without requiring the administrator to actually be at the place where the device is located.

12.3 ASN.1

In data communication, when we send a continuous stream of bits to a destination, we somehow need to define the format of the data. If we send a *name* and a *number* in a single message, we need to tell the destination that, for example, the first 12 bits define the name and the next 8 bits define the number. We will have more difficulty when we send a complex data type such as an array or a record. For example, in the case of an array, if we send 2000 bits in a message, we need to tell the receiver that it is an array of 200 numbers each of 10 bits or it is an array of 10 numbers each of 200 bits.

A solution is that we separate the definition of data types from the sequence of bits transmitted through the network. This is done through an abstract language that uses some symbols, key words, and atomic data types and lets us make new data types out of the simple types. The language is called Abstract Syntax Notation One (ASN.1). Note that ASN.1 is a very complex language used in different areas of computer science,

but in this section, we only introduce the language as much as needed for the SNMP protocol.

12.3.1 Language Basics

Before we show how we can define objects and associated values, let us talk about the language itself. The language uses some symbols and some key words and defines some primitive data types. As we said before, SMI uses a subset of these entities in its own language.

Symbols

The language uses a set of symbols, given in Table 12.5. Some of these symbols are single characters, but some are pairs of characters.

Table 12.5 *Symbols used in ASN.1*

Symbol	*Meaning*	*Symbol*	*Meaning*
::=	Defined as or assignment	..	Range
|	Or, alternative, or option	{ }	Start and end of a list
–	Negative sign	[]	Start and end of tag
--	The following is a comment	()	Start and end of a subtype

Key Words

The language has a set of limited key words that can be used. These words can be used in the language only for the purpose for which they have been defined. All words should be in uppercase (Table 12.6).

Table 12.6 *Key words in ASN.1*

Key Word	*Description*
BEGIN	Start of a module
CHOICE	List of alternatives
DEFINITIONS	Definition of a data type or an object
END	End of a module
EXPORTS	Data type that can be exported to other modules
IDENTIFIER	A sequence of non-negative numbers that identifies an object
IMPORTS	Data type defined in an external module and imported
INTEGER	Any positive, zero, or negative integer
NULL	A null value
OBJECT	Used with IDENTIFIER to uniquely define an object
OCTET	Eight-bit binary data
OF	Used with SEQUENCE or SET
SEQUENCE	An ordered list
SEQUENCE OF	An ordered array of data of the same type

SET	An unordered list
SET OF	An array of unordered lists
STRING	A string of data

12.3.2 Data Types

After discussing the symbols and key words used in the language, it is time to define its data types. The idea is similar to what we see in computer languages such as C, C++, or Java. In ASN.1, we have several simple data types such as integer, float, boolean, and char. We can combine these data types to create a new simple data type (with a different name) or to define structured data types such as array or struct. We first define simple data types in ASN.1 and then show how to make a new data type of these data types.

Simple Data Types

ASN.1 defines a set of simple (atomic) data types. Each data type is given a universal tag and has a set of values, as shown in Table 12.7. This is the same idea as used in a computer language when we have some basic data types with predefined ranges of values. For example, in C language, we have the data type *int*, which can take a range of values. Note that the tag in the table is actually the rightmost 5 bits of the tag we defined in Table 12.2.

Table 12.7 *Some simple ASN.1 built-in types*

Tag	*Type*	*Set of values*
Universal 1	BOOLEAN	TRUE or FALSE
Universal 2	INTEGER	Integers (positive, 0, or negative)
Universal 3	BIT STRING	A string of binary digits (bits) or a null set
Universal 4	OCTET STRING	A string of octets or a null set
Universal 5	NULL	Null, single valued
Universal 6	OBJECT IDENTIFIER	A set of values that defines an object
Universal 7	ObjectDescriptor	Human readable text describing an object
Universal 8	EXTERNAL	A type that is not in the standard
Universal 9	REAL	Real numbers in scientific notation
Universal 10	ENUMERATED	A list of integers
Universal 16	SEQUENCE, SEQUENCE OF	Ordered list of types
Universal 17	SET, SET OF	Unordered list of types
Universal 18	NumericString	Digits 0–9 and space
Universal 19	PrintableString	Printable characters
Universal 26	VisibleString	ISO646String
Universal 27	GeneralString	General character string
Universal 30	CHARACTER STRING	Character set

New Data Types

ASN.1 uses Backus–Naur Form (BNF) syntax for defining a new data type from a built-in data type or a previously defined data type:

<new type> ::= **<type>**

where the *new type* must start with a capital letter.

Example 12.6

The following is an example of some new types using built-in types from Table 12.7.

```
Married ::= BOOLEAN
MaritalStatus ::= ENUMERATED {single, married, widowed, divorced}
DayOfWeek ::= ENUMERATED {sun, mon, tue, wed, thu, fri, sat}
Age ::= INTEGER
```

New Subtypes

ASN.1 even allows us to create a subtype whose range is a subrange of a built-in type or a previously defined data type.

Example 12.7

The following shows how we can make three new subtypes. The range of the first is the subset of INTEGER, the range of the second is the subset of REAL, and the range of the third is the subset of DayOfWeek, which we defined in Example 12.6. Note that we use the symbol (..) to define the range and the symbol (|) to define the choice.

```
NumberOfStudents ::= INTEGER (15..40)     -- An integer with the range 15 to 40
Grade ::= REAL (1.0..5.0)                 -- A real number with the range 1.0 to 5.0
Weekend ::= DayOfWeek (sun | sat)         -- A day that can be sun or sat
```

Simple Variables

In a programming language, we can create a variable of a particular type and assign (store) a value in it. In ASN.1, the term *Value Name* is used instead of *variable*, but we use the term *variable*, which is more familiar to programmers. We can create a variable of a particular type and assign a value belonging to the range defined for that type. The following shows the syntax:

<variable> <type> ::= **<value>**

The name of the variable should start with a lowercase letter to distinguish it from the type.

Example 12.8

The following are a few examples of defining some variables and assigning the appropriate value from the range of those types. Note that the first and the third variables are of the built-in

type, the second is of the type defined in Example 12.6, and the last is of a subtype defined in Example 12.7.

```
numberOfComputers INTEGER ::= 2
married Married ::= FALSE
herAge INTEGER ::= 35
classSize NumberOfStudents ::= 22
```

Structured Type

ASN.1 uses a key word SEQUENCE to define a structured data type similar to struct (record) in C language or C++. The SEQUENCE type is an ordered list of variable types. The following shows a new type StudentAccount, which is a sequence of three variables: username, password, and accountNumber.

```
StudentAccount ::= SEQUENCE
{
        userName VisibleString,
        password VisibleString,
        accountNumber INTEGER
}
```

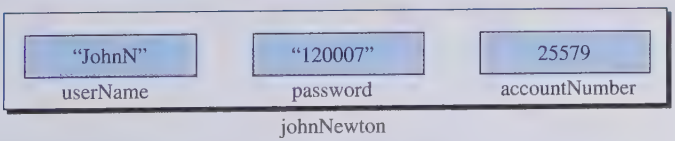
Structure Variables

After defining the new type, we can create a variable out of it and assign values to variable:

```
johnNewton StudentAccount
{
        userName "JohnN",
        password "120007",
        accountNumber 25579
}
```

Figure 12.23 shows the record created from the type definition and value assignments.

Figure 12.23 *Record representing the type definition and value assignments*



We use the key word SEQUENCE OF to define a new type similar to an array in C or C++, which is a composite type in which all components are the same. For example, we can define a forwarding table in a router as SEQUENCE OF Rows in which each Row is itself a sequence made of several variables.

12.3.3 Encoding

After the data has been defined and values are associated with variables, ASN.1 uses one of the encoding rules to encode the message to be sent. We already discussed the Basic Encoding Rules in Section 12.2.4.

12.4 END-OF-CHAPTER MATERIALS

12.4.1 Recommended Reading

For more details about subjects discussed in this chapter, we recommend the following books. The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give thorough coverage of SNMP: [Com 06], [Ste 94], [Tan 03], and [MS 01].

12.4.2 Key Terms

Abstract Syntax Notation One (ASN.1)
Backus–Naur Form (BNF)
Basic Encoding Rules (BER)
Management Information Base (MIB)
object identifier
simple data type
Simple Network Management Protocol (SNMP)
Structure of Management Information (SMI)
structured data type
trap

12.4.3 Summary

The five areas comprising network management are configuration management, fault management, performance management, accounting management, and security management. Configuration management is concerned with the physical or logical changes of network entities. Fault management is concerned with the proper operation of each network component. Performance management is concerned with the monitoring and controlling of the network to ensure the network runs as efficiently as possible. Security management is concerned with controlling access to the network. Accounting management is concerned with controlling user access to network resources through charges.

Simple Network Management Protocol (SNMP) is a framework for managing devices in an internet using the TCP/IP protocol suite. A manager, usually a host, controls and monitors a set of agents, usually routers. SNMP uses the services of SMI and MIB. SMI names objects, defines the type of data that can be stored in an object, and encodes the data. MIB is a collection of groups of objects that can be managed by SNMP. MIB uses lexicographic ordering to manage its variables.

Abstract Syntax Notation One (ASN.1) is a language that defines the syntax and semantics of data. It uses some symbols, key words, simple and structured data types. Part of ASN.1 is used by SMI to define the format of objects and values used in network management.

12.5 PRACTICE SET

12.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

12.5.2 Questions

Q12-1. Which of the following is not one of the five areas of network management defined by ISO?
a. fault **b.** performance **c.** personnel

Q12-2. Which of the following is not part of configuration management?
a. reconfiguration **b.** encryption **c.** documentation

Q12-3. A network manager decides to replace the old router that connects the organization to the Internet with a more powerful one. What area of network management is involved here?

Q12-4. A network manager decides to replace a version of accounting software with a new version. What area of network management is involved here?

Q12-5. Distinguish between reactive fault management and proactive fault management.

Q12-6. If network management does not replace a component whose lifetime has been expired, what area in network management has been ignored?

Q12-7. Distinguish between internal and external data traffic in an organization.

Q12-8. If a student in a college can monopolize access to a piece of software, causing other students to wait for a long time, which area of network management has failed?

Q12-9. Which of the following devices cannot be a manager station in SNMP?
a. router **b.** host **c.** switch

Q12-10. Does an SNMP manager run a client SNMP program or a server SNMP program?

Q12-11. Show how the textual name "iso.org.dod" is numerically encoded in SMI.

Q12-12. Is it possible to have a textual name in SMI as "iso.org.internet"? Explain.

Q12-13. Find the type (simple, sequence, sequence of) of the following objects in SMI.
a. unsigned integer
b. IP address
c. object name
d. list of integers
e. record defining an object name, an IP address, and an integer
f. list of records in which each record is an object name followed by a counter

Q12-14. What is the length of the value field in the following BER encoding?

04 09 48 65 6C 4C ...

Q12-15. Distinguish between SMI and MIB.

Q12-16. What does the *if* object in MIB define? Why does this object need to be managed?

Q12-17. Assume an object identifier in MIB has three simple variables. If the object identifier is x, what is the identifier of each variable?

Q12-18. Can SNMP reference the entire row of a table? In other words, can SNMP retrieve or change the values in the entire row of a table? Explain.

Q12-19. Can an SNMP message reference a leaf node in the MIB tree? Explain.

Q12-20. Assume a manageable object has only three simple variables. How many leaves can be found in the MIB tree for this object?

Q12-21. Assume a manageable object has a table with three columns. How many leaves are there in the MIB tree for this table?

Q12-22. Distinguish between a GetRequest PDU and a SetRequest PDU.

Q12-23. In SNMP, which of the following PDUs are sent from a client SNMP to a server SNMP?

a. GetRequest **b.** Response **c.** Trap

Q12-24. What are the source and destination port numbers when an SNMP message carries one of the following PDUs?

a. GetRequest **b.** Response **c.** Trap **d.** Report

12.5.3 Problems

P12-1. Assume object x has two simple variables: an integer and an IP address. What is the identifier for each variable?

P12-2. Assume object x has only one table with two columns. What is the identifier for each column?

P12-3. Assume object x has two simple variables and one table with two columns. What is the identifier for each variable and each column of the table? We assume that simple variables come before the table.

P12-4. Assume object x has one simple variable and two tables with two and three columns, respectively. What is the identifier for the variable and each column of each table? We assume that the simple variable comes before the tables.

P12-5. Object x has two simple variables. How can SNMP refer to the instance of each variable?

P12-6. Object x has one table with two columns. The table at this moment has three rows with the contents shown. If the table index is based on the values in the first column, show how SNMP can access each instance.

Object *x*

a	aa
b	bb
c	cc

Table

P12-7. One of the objects (groups) that can be managed is the *ip* group with the object identifier (1.3.6.1.2.1.**4**) in which (1.3.6.1.2.1) is the identifier of MIB-2 and (4) defines the ***ip*** group. In an agent, this object has 20 simple variables and three tables. One of the tables is the routing (forwarding) table with the identifier (1.3.6.1.2.1.**4.21**). This table has 11 columns, the first of which is called the *ipRouteDes*, which means the destination IP address. Assume that the indexing is based on the first column. Assume the table has four rows at the moment with the destination IP addresses (201.14.67.0), (123.16.0.0), (11.0.0.0), and (0.0.0.0). Show how SNMP can access all four instances of the second column, called *ipRouteIfIndex*, which defines the interface numbers through which the IP should be sent out.

P12-8. Show the encoding for the INTEGER 1456 using BER.

P12-9. Show the encoding for the OCTET STRING "Hello world." using BER.

P12-10. Show the encoding for the IPAddress 112.56.23.78 using BER.

P12-11. Show the encoding for the object identifier 1.3.6.1.2.1.7.1 (the udpInDatagram variable in udp group) using BER.

P12-12. Using BER, show how we can encode a structured data type made up of an INTEGER of value (2371), an OCTET STRING of value ("Computer"), and an IPAddress of value (185.32.1.5) as shown.

```
SEQUENCE
{
        INTEGER 2371
        OCTET STRING "Computer"
        IP Address 185.32.1.5
}
```

P12-13. Assume a data structure is made up of an INTEGER of value (131) and another structure made up of an IPAddress of value (24.70.6.14) and an OCTETSTRING ("UDP"). Using BER, encode the data structure.

P12-14. Given the code 02040000C738, decode it using BER.

P12-15. Given the code 300C02040000099806040A05030E, decode it using BER.

P12-16. Given the code 300D04024E6F300706030103060500, decode it using BER.

P12-17. Assume a manager needs to know the number of user datagrams an agent has sent out (a udpOutDatagrams counter with the identifier 1.3.6.1.2.1.7.4). Show the code for a Varbind that is sent in a GetRequest message and the code that the agent will send in the Response message if the value of the counter at this moment is 15.

P12-18. Define an SNMP message (see Figure 12.19) using the syntax defined for structured data types in ASN.1.

P12-19. Define a GetRequest PDU (see Figure 12.18) using the syntax defined for structured data types in ASN.1.

P12-20. Define a Response PDU (see Figure 12.18) using the syntax defined for structured data types in ASN.1.

P12-21. Define a VarbindList (see Figure 12.19) using the syntax defined for structured data types in ASN.1.

CHAPTER 13

Cryptography and Network Security

The topic of cryptography and network security is very broad and involves some specific areas of mathematics such as number theory. In this chapter, we try to give a very simple introduction to this topic to prepare the background for more study.

This chapter is divided into seven sections.

- The first section introduces the subject. It describes security goals such as confidentiality, integrity, and availability.
- The second section discusses confidentiality. It first describes symmetric-key ciphers. It then moves to modern symmetric-key ciphers and explains modern block and stream ciphers.
- The third section discusses other aspects of security: message integrity, message authentication, digital signature, entity authentication.
- The fourth section discusses security at the network layer, IPSec. It first describes the two modes of IPSec. It then describes the two versions of the protocol: AS and ESP.
- The fifth section discusses one of the security protocols at the transport layer, SSL. It first describes the SSL architecture: services, algorithms, and parameter generation. It then explains the four protocols that SSL is made up of.
- The sixth section discusses security at the application layer. At this layer, security is provided only for the e-mail application; other applications can use the security at the transport layer, but e-mail, because of its one-way communication cannot do so.
- The seventh section discusses firewalls, a technology that can protect an enterprise from the malicious intension of an intruder. This section describes two versions: packet-filter firewalls and proxy firewalls. The first gives protection only at the network layer; the second provides protection at the application layer.

13.1 INTRODUCTION

We are living in the information age. We need to keep information about every aspect of our lives. In other words, information is an asset that has a value like any other asset. As an asset, information needs to be secured from attacks. To be secured, information needs to be hidden from unauthorized access (*confidentiality*), protected from unauthorized change (*integrity*), and available to an authorized entity when it is needed (*availability*).

During the last three decades, computer networks have created a revolution in the use of information. Information is now distributed. Authorized people can send and retrieve information from a distance using computer networks. Although the three above-mentioned requirements—confidentiality, integrity, and availability—have not changed, they now have some new dimensions. Not only should information be confidential when it is stored, but there should also be a way to maintain its confidentiality when it is transmitted from one computer to another.

In this section, we first discuss the three major goals of information security and see how attacks can threaten these three goals. We then discuss the security services in relation to these security goals. Finally we define two techniques to implement the security goals and prevent attacks.

13.1.1 Security Goals

We first discuss three security goals: confidentiality, integrity, and availability.

Confidentiality

Confidentiality is probably the most common aspect of information security. We need to protect our confidential information. An organization needs to guard against those malicious actions that endanger the confidentiality of its information. Confidentiality not only applies to the storage of information, it also applies to the transmission of information. When we send a piece of information to be stored in a remote computer or when we retrieve a piece of information from a remote computer, we need to conceal it during transmission.

Integrity

Information needs to be changed constantly. In a bank, when a customer deposits or withdraws money, the balance of her account needs to be changed. *Integrity* means that changes need to be done only by authorized entities and through authorized mechanisms. Integrity violation is not necessarily the result of a malicious act; an interruption in the system, such as a power surge, may also create unwanted changes in some information.

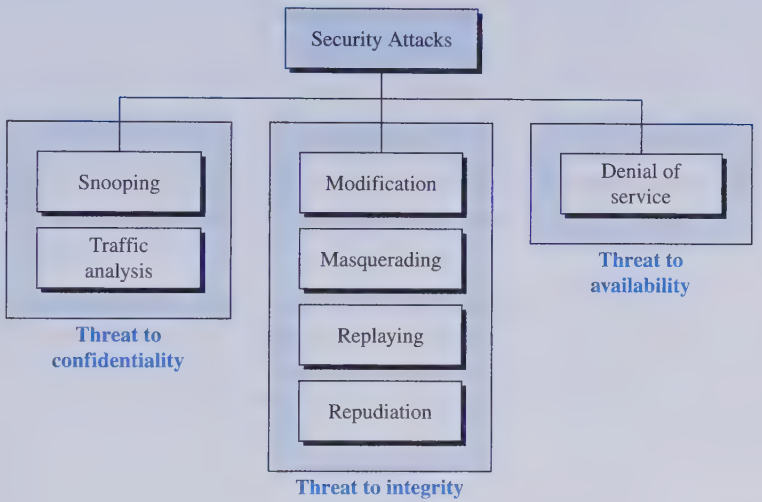
Availability

The third component of information security is *availability*. The information created and stored by an organization needs to be available to authorized entities. Information is useless if it is not available. Information needs to be constantly changed, which means it must be accessible to authorized entities. The unavailability of information is just as harmful for an organization as the lack of confidentiality or integrity. Imagine what would happen to a bank if the customers could not access their accounts for transactions.

13.1.2 Attacks

Our three goals of security—confidentiality, integrity, and availability—can be threatened by security *attacks*. Although the literature uses different approaches to categorizing the attacks, we divide them into three groups related to the security goals. Figure 13.1 shows the taxonomy.

Figure 13.1 *Taxonomy of attacks with relation to security goals*



Attacks Threatening Confidentiality

In general, two types of attacks threaten the confidentiality of information: *snooping* and *traffic analysis*.

Snooping

Snooping refers to unauthorized access to or interception of data. For example, a file transferred through the Internet may contain confidential information. An unauthorized entity may intercept the transmission and use the contents for her own benefit. To prevent snooping, the data can be made nonintelligible to the intercepter by using encipherment techniques, which we discuss later.

Traffic Analysis

Although encipherment of data may make it nonintelligible for the intercepter, she can obtain some other type of information by monitoring online traffic. For example, she can find the electronic address (such as the e-mail address) of the sender or the receiver. She can collect pairs of requests and responses to help her guess the nature of the transaction.

Attacks Threatening Integrity

The integrity of data can be threatened by several kinds of attacks: *modification*, *masquerading*, *replaying*, *and repudiation.*

Modification

After intercepting or accessing information, the attacker modifies the information to make it beneficial to herself. For example, a customer sends a message to a bank to initiate some transaction. The attacker intercepts the message and changes the type of transaction to benefit herself. Note that sometimes the attacker simply deletes or delays the message to harm the system or to benefit from it.

Masquerading

Masquerading, or spoofing, happens when the attacker impersonates somebody else. For example, an attacker might steal the bank card and PIN of a bank customer and pretend that she is that customer. Sometimes the attacker pretends instead to be the receiver entity. For example, a user tries to contact a bank, but another site pretends that it is the bank and obtains some information from the user.

Replaying

Replaying is another attack. The attacker obtains a copy of a message sent by a user and later tries to replay it. For example, a person sends a request to her bank to ask for payment to the attacker, who has done a job for her. The attacker intercepts the message and sends it again to receive another payment from the bank.

Repudiation

This type of attack is different from others because it is performed by one of the two parties in the communication: the sender or the receiver. The sender of the message might later deny that she has sent the message; the receiver of the message might later deny that he has received the message. An example of denial by the sender would be a bank customer asking her bank to send some money to a third party but later denying that she has made such a request. An example of denial by the receiver could occur when a person buys a product from a manufacturer and pays for it electronically, but the manufacturer later denies having received the payment and asks to be paid.

Attacks Threatening Availability

We mention only one attack threatening availability: denial of service.

Denial of Service

Denial of service (DoS) is a very common attack. It may slow down or totally interrupt the service of a system. The attacker can use several strategies to achieve this. She might send so many bogus requests to a server that the server crashes because of the heavy load. The attacker might intercept and delete a server's response to a client, making the client believe that the server is not responding. The attacker may also intercept requests from the clients, causing the clients to send requests many times and overload the system.

13.1.3 Services and Techniques

The International Telecommunications Union, Telecommunication Standardization Sector (ITU-T) defines some security services to achieve security goals and prevent attacks. Each of these services is designed to prevent one or more attacks while maintaining security goals. The actual implementation of security goals needs some techniques. Two techniques are prevalent today: One is very general (cryptography), and one is specific (steganography).

Cryptography

Some security services can be implemented using cryptography. **Cryptography**, a word with Greek origins, means "secret writing." However, we use the term to refer to the science and art of transforming messages to make them secure and immune to attacks. Although in the past *cryptography* referred only to the **encryption** and **decryption** of messages using secret keys, today it is defined as involving three distinct mechanisms: symmetric-key encipherment, asymmetric-key encipherment, and hashing. We will discuss all these mechanisms later in the chapter.

Steganography

Although this chapter is based on cryptography as a technique for implementing security mechanisms, another technique that was used for secret communication in the past is being revived at the present time: steganography. The word **steganography**, with Greek, origins, means "covered writing." *Cryptography* means concealing the contents of a message by enciphering; *steganography* means concealing the message itself by covering it with something else. We leave the discussion of steganography to some books dedicated to this topic.

13.2 CONFIDENTIALITY

We now look at the first goal of security, confidentiality. Confidentiality can be achieved using ciphers. Ciphers can be divided into two broad categories: symmetric-key and asymmetric-key.

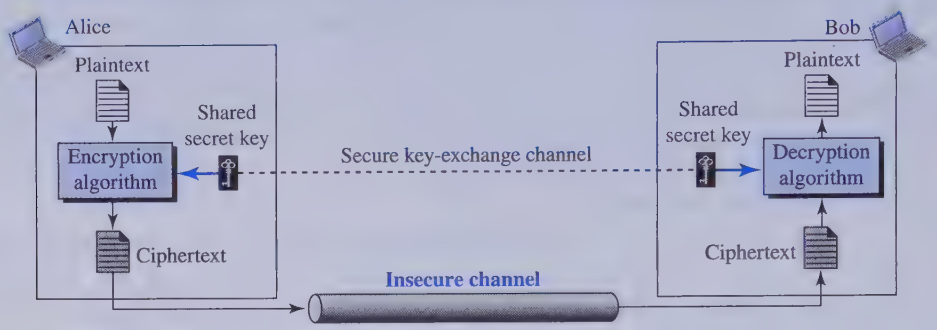
13.2.1 Symmetric-Key Ciphers

A **symmetric-key cipher** uses the same key for both encryption and decryption, and the key can be used for bidirectional communication, which is why it is called *symmetric*. Figure 13.2 shows the general idea behind a symmetric-key cipher.

In Figure 13.2, an entity, Alice, can send a message to another entity, Bob, over an insecure channel with the assumption that an adversary, Eve, cannot understand the contents of the message by simply eavesdropping over the channel.

The original message from Alice to Bob is called **plaintext**; the message that is sent through the channel is called **ciphertext**. To create the ciphertext from the plaintext, Alice uses an **encryption algorithm** and a *shared secret key*.

Figure 13.2 *General idea of a symmetric-key cipher*



To create the plaintext from ciphertext, Bob uses a **decryption algorithm** and the same secret key. We refer to encryption and decryption algorithms as **ciphers**. A *key* is a set of values (numbers) that the cipher, as an algorithm, operates on.

Symmetric-key ciphers are also called secret-key ciphers.

Note that the symmetric-key encipherment uses a single key (the key itself may be a set of values) for both encryption and decryption. In addition, the encryption and decryption algorithms are inverses of each other. If P is the plaintext, C is the ciphertext, and K is the key, the encryption algorithm $E_k(x)$ creates the ciphertext from the plaintext; the decryption algorithm $D_k(x)$ creates the plaintext from the ciphertext. We assume that $E_k(x)$ and $D_k(x)$ are inverses of each other: They cancel the effect of each other if they are applied one after the other on the same input. We have

$$\textbf{Encryption: } C = E_k(P) \qquad \textbf{Decryption: } P = D_k(C)$$

in which, $D_k(E_k(x)) = E_k(D_k(x)) = x$. We need to emphasize that it is better to make the encryption and decryption public but keep the shared key secret. This means that Alice and Bob need another channel, a secured one, to exchange the secret key. Alice and Bob can meet once and exchange the key personally. The secured channel here is the face-to-face exchange of the key. They can also trust a third party to give them the same key. They can create a temporary secret key using another kind of cipher—asymmetric-key ciphers—which will be described later in Section 13.2.2.

Encryption can be thought of as locking the message in a box; decryption can be thought of as unlocking the box. In symmetric-key encipherment, the same key locks and unlocks, as shown in Figure 13.3. Later we show that the *asymmetric-key* encipherment needs two keys, one for locking and one for unlocking.

The symmetric-key ciphers can be divided into traditional ciphers and modern ciphers. Traditional ciphers are simple, character-oriented ciphers that are not secured based on today's standard. Modern ciphers, on the other hand, are complex, bit-oriented

Figure 13.3 *Symmetric-key encipherment as locking and unlocking with the same key*

ciphers that are more secure. We briefly discuss the traditional ciphers to pave the way for discussing more complex modern ciphers.

Traditional Symmetric-Key Ciphers

Traditional ciphers belong to the past. However, we briefly discuss them here because they can be thought of as the components of modern ciphers. To be more exact, we can divide traditional ciphers into substitution ciphers and transposition ciphers.

Substitution Ciphers

A **substitution cipher** replaces one symbol with another. If the symbols in the plaintext are alphabetic characters, we replace one character with another. For example, we can replace letter A with letter D and letter T with letter Z. If the symbols are digits (0 to 9), we can replace 3 with 7 and 2 with 6.

A substitution cipher replaces one symbol with another.

Substitution ciphers can be categorized as either monoalphabetic ciphers or polyalphabetic ciphers.

Monoalphabetic Ciphers In a **monoalphabetic cipher**, a character (or a symbol) in the plaintext is always changed to the same character (or symbol) in the ciphertext regardless of its position in the text. For example, if the algorithm says that letter A in the plaintext is changed to letter D, every letter A is changed to letter D. In other words, the relationship between letters in the plaintext and the ciphertext is one-to-one.

The simplest monoalphabetic cipher is the **additive cipher** (or **shift cipher**). Assume that the plaintext consists of lowercase letters (a to z) and that the ciphertext consists of uppercase letters (A to Z). To be able to apply mathematical operations on the plaintext and ciphertext, we assign numerical values to each letter (lower- or uppercase), as shown in Figure 13.4.

Figure 13.4 *Representation of plaintext and ciphertext characters in modulo 26*

Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Value →	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25

In Figure 13.4 each character (lowercase or uppercase) is assigned an integer in modulo 26. The secret key between Alice and Bob is also an integer in modulo 26. The encryption algorithm adds the key to the plaintext character; the decryption algorithm subtracts the key from the ciphertext character. All operations are done in modulo 26.

In an additive cipher, the plaintext, ciphertext, and key are integers in modulo 26.

Historically, additive ciphers are called shift ciphers because the encryption algorithm can be interpreted as "shift *key* characters down" and the encryption algorithm can be interpreted as "shift *key* characters up." Julius Caesar used an additive cipher, with a key of 3, to communicate with his officers. For this reason, additive ciphers are sometimes referred to as the **Caesar cipher**.

Example 13.1

Use the additive cipher with key = 15 to encrypt the message "hello".

Solution

We apply the encryption algorithm to the plaintext, character by character:

Plaintext: h → 07	Encryption: (07 + 15) mod 26	Ciphertext: 22 → W
Plaintext: e → 04	Encryption: (04 + 15) mod 26	Ciphertext: 19 → T
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: l → 11	Encryption: (11 + 15) mod 26	Ciphertext: 00 → A
Plaintext: o → 14	Encryption: (14 + 15) mod 26	Ciphertext: 03 → D

The result is "WTAAD". Note that the cipher is monoalphabetic because two instances of the same plaintext character (*l*) are encrypted as the same character (*A*).

Example 13.2

Use the additive cipher with key = 15 to decrypt the message "WTAAD".

Solution

We apply the decryption algorithm to the plaintext character by character:

Ciphertext: W → 22	Decryption: (22 − 15) mod 26	Plaintext: 07 → h
Ciphertext: T → 19	Decryption: (19 − 15) mod 26	Plaintext: 04 → e
Ciphertext: A → 00	Decryption: (00 − 15) mod 26	Plaintext: 11 → l
Ciphertext: A → 00	Decryption: (00 − 15) mod 26	Plaintext: 11 → l
Ciphertext: D → 03	Decryption: (03 − 15) mod 26	Plaintext: 14 → o

The result is "hello". Note that the operation is in modulo 26, which means that we need to add 26 to a negative result (for example–15 becomes 11).

Additive ciphers are vulnerable to attacks using exhaustive key searches (brute-force attacks). The key domain of the additive cipher is very small; there are only 26 keys. However, one of the keys, zero, is useless (the ciphertext is the same as the plaintext). This

leaves only 25 possible keys. Eve can easily launch a brute-force attack on the ciphertext. A better solution is to create a mapping between each plaintext character and the corresponding ciphertext character. Alice and Bob can agree on a table showing the mapping for each character. Figure 13.5 shows an example of such a mapping.

Example 13.3

We can use the key in Figure 13.5 to encrypt the message

Figure 13.5 *An example key for a monoalphabetic substitution cipher*

Plaintext →	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
Ciphertext →	N	O	A	T	R	B	E	C	F	U	X	D	Q	G	Y	L	K	H	V	I	J	M	P	Z	S	W

Plaintext:	this message is easy to encrypt but hard to find the key
Ciphertext:	ICFVQRVVNERFVRNVSIYRGAHSLIOJICNHTIYBFGTICRXRS

Polyalphabetic Ciphers In a **polyalphabetic cipher**, each occurrence of a character may have a different substitute. The relationship between a character in the plaintext to a character in the ciphertext is one-to-many. For example, "a" could be enciphered as "D" at the beginning of the text, but as "N" in the middle. Polyalphabetic ciphers have the advantage of hiding the letter frequency of the underlying language. Eve cannot use single-letter frequency statistics to break the ciphertext.

To create a polyalphabetic cipher, we need to make each ciphertext character dependent on both the corresponding plaintext character and the position of the plaintext character in the message. This implies that our key should be a stream of subkeys, in which each subkey depends somehow on the position of the plaintext character that uses that subkey for encipherment. In other words, we need to have a key stream $k = (k_1, k_2, k_3, ...)$ in which k_i is used to encipher the ith character in the plaintext to create the ith character in the ciphertext.

To see the position dependency of the key, let us discuss a simple polyalphabetic cipher called the **autokey cipher**. In this cipher, the key is a stream of subkeys, in which each subkey is used to encrypt the corresponding character in the plaintext. The first subkey is a predetermined value secretly agreed upon by Alice and Bob. The second subkey is the value of the first plaintext character (between 0 and 25). The third subkey is the value of the second plaintext character, and so on.

$$P = P_1P_2P_3 \ldots \qquad C = C_1C_2C_3 \ldots \qquad k = (k_1, P_1, P_2, \ldots)$$

$$\text{Encryption: } C_i = (P_i + k_i) \bmod 26 \qquad \text{Decryption: } P_i = (C_i - k_i) \bmod 26$$

The name of the cipher, *autokey*, implies that the subkeys are automatically created from the plaintext cipher characters during the encryption process.

Example 13.4

Assume that Alice and Bob agreed to use an autokey cipher with initial key value $k_1 = 12$. Now Alice wants to send Bob the message "Attack is today". Enciphering is done character by character. Each character in the plaintext is first replaced by its integer value. The first subkey is added to create the first ciphertext character. The rest of the key is created as the plaintext characters are read. Note that the cipher is polyalphabetic because the three occurrences of "a" in the plaintext are encrypted differently. The three occurrences of "t" are encrypted differently.

Plaintext:	a	t	t	a	c	k	i	s	t	o	d	a	y
P's Values:	00	19	19	00	02	10	08	18	19	14	03	00	24
Key stream:	*12*	*00*	*19*	*19*	*00*	*02*	*10*	*08*	*18*	*19*	*14*	*03*	*00*
C's Values:	12	19	12	19	02	12	18	00	11	7	17	03	24
Ciphertext:	**M**	**T**	**M**	**T**	**C**	**M**	**S**	**A**	**L**	**H**	**R**	**D**	**Y**

Transposition Ciphers

A **transposition cipher** does not substitute one symbol for another; instead it changes the location of the symbols. A symbol in the first position of the plaintext may appear in the tenth position of the ciphertext. A symbol in the eighth position in the plaintext may appear in the first position of the ciphertext. In other words, a transposition cipher reorders (transposes) the symbols.

A transposition cipher reorders symbols.

Suppose Alice wants to secretly send the message "Enemy attacks tonight" to Bob. The encryption and decryption is shown in Figure 13.6. Note that we added an extra character (z) to the end of the message to make the number of characters a multiple of 5.

The first table is created by Alice writing the plaintext row by row. The columns are permuted using a key. The ciphertext is created by reading the second table column by column. Bob does the same three steps in the reverse order. He writes the ciphertext column by column into the first table, permutes the columns, and then reads the second table row by row. Note that the same key is used for encryption and decryption, but the algorithm uses the key in reverse order.

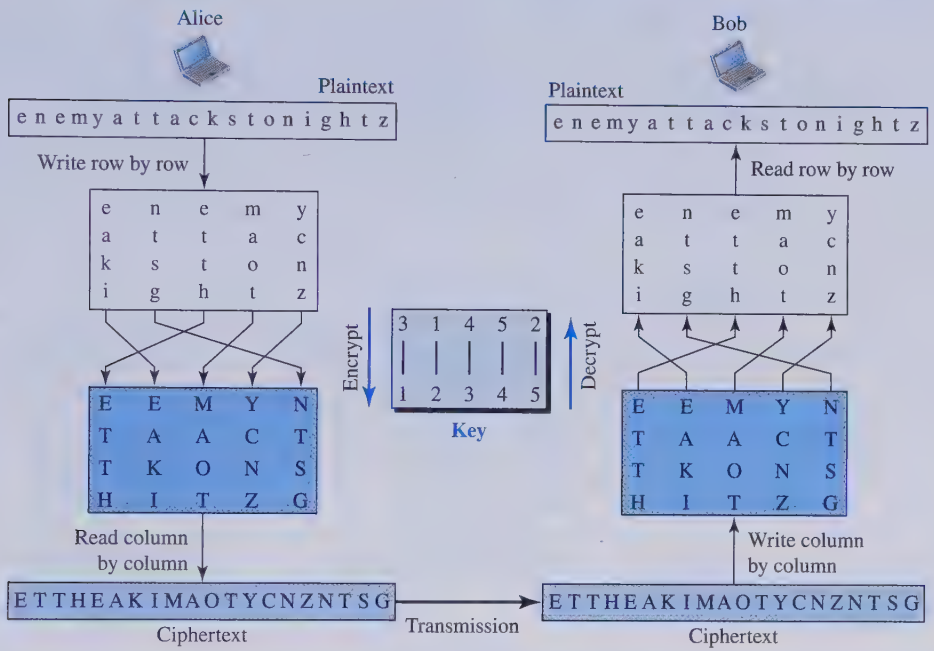
Stream and Block Ciphers

The literature divides the traditional symmetric ciphers into two broad categories: stream ciphers and block ciphers.

Stream Cipher In a **stream cipher**, encryption and decryption are done one symbol (such as a character or a bit) at a time. We have a plaintext stream, a ciphertext stream, and a key stream. Call the plaintext stream P, the ciphertext stream C, and the key stream K.

$$P = P_1P_2P_3, \ldots \qquad C = C_1C_2C_3, \ldots \qquad K = (k_1, k_2, k_3, \ldots)$$
$$C_1 = E_{k1}(P_1) \qquad C_2 = E_{k2}(P_2) \qquad C_3 = E_{k3}(P_3) \ldots$$

Figure 13.6 *Transposition cipher*



Block Ciphers In a **block cipher**, a group of plaintext symbols of size m ($m > 1$) are encrypted together, creating a group of ciphertext of the same size. Based on the definition, in a block cipher, a single key is used to encrypt the whole block even if the key is made up of multiple values. In a block cipher, a ciphertext block depends on the whole plaintext block.

Combination In practice, blocks of plaintext are encrypted individually, but they use a stream of keys to encrypt the whole message block by block. In other words, the cipher is a block cipher when looking at the individual blocks, but it is a stream cipher when looking at the whole message, considering each block as a single unit. Each block uses a different key that may be generated before or during the encryption process.

Modern Symmetric-Key Ciphers

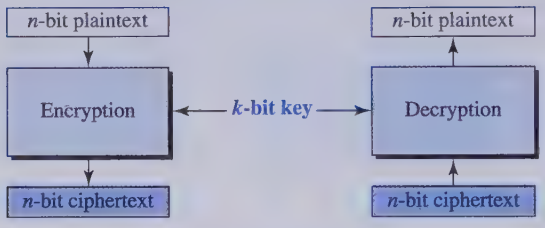
The traditional symmetric-key ciphers that we have studied so far are *character-oriented ciphers*. With the advent of the computer, we need *bit-oriented ciphers*. This is because the information to be encrypted is not just text; it can also consist of numbers, graphics, audio, and video data. It is convenient to convert these types of data into a stream of bits, to encrypt the stream, and then to send the encrypted stream. In addition, when text is treated at the bit level, each character is replaced by 8 (or 16) bits, which means that the number of symbols becomes 8 (or 16) times

larger. Mixing a larger number of symbols increases security. A modern cipher can be either a block cipher or a stream cipher.

Modern Block Ciphers

A symmetric-key *modern block cipher* encrypts an n-bit block of plaintext or decrypts an n-bit block of ciphertext. The encryption or decryption algorithm uses a k-bit key. The decryption algorithm must be the inverse of the encryption algorithm, and both operations must use the same secret key so that Bob can retrieve the message sent by Alice. Figure 13.7 shows the general idea of encryption and decryption in a modern block cipher.

Figure 13.7 *A modern block cipher*

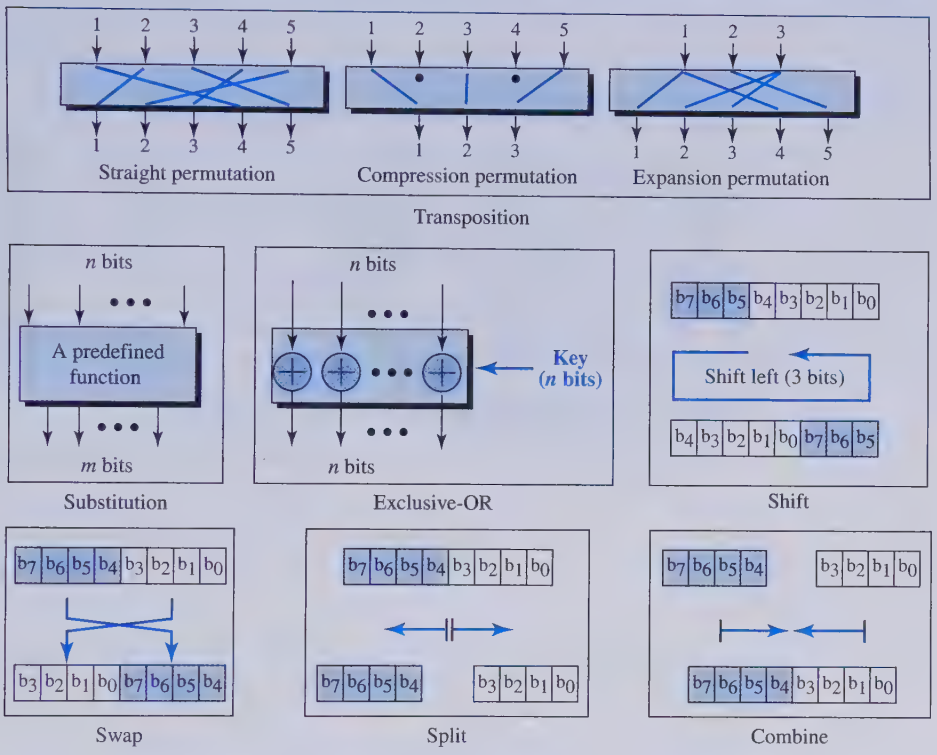


If the message has fewer than n bits, padding must be added to make it an n-bit block; if the message has more than n bits, it should be divided into n-bit blocks and the appropriate padding must be added to the last block if necessary. The common values for n are 64, 128, 256, and 512 bits.

Components of a Modern Block Cipher Modern block ciphers are substitution ciphers when seen as a whole block. However, modern block ciphers are not designed as a single unit. To provide an attack-resistant cipher, a modern block cipher is made up of a combination of transposition units (sometimes called *P-boxes*), substitution units (sometimes called *S-boxes*), and exclusive-OR (XOR) operations, shifting elements, swapping elements, splitting elements, and combining elements. Figure 13.8 shows the components of a modern block cipher.

A **P-box** (permutation box) parallels the traditional transposition cipher for characters, but it transposes bits. We can find three types of P-boxes in modern block ciphers: straight P-boxes, expansion P-boxes, and compression P-boxes. An **S-box** (substitution box) can be thought of as a miniature substitution cipher, but it substitutes bits. Unlike the traditional substitution cipher, an S-box can have a different number of inputs and outputs. An important component in most block ciphers is the *exclusive-OR* operation, in which the output is 0 if the two inputs are the same, and the output is 1 if the two inputs are different. In modern block ciphers, we use n exclusive-OR operations to combine an n-bit data piece with an n-bit key. An exclusive-OR operation is normally the only unit where the key is applied. The other components are normally based on predefined functions.

Figure 13.8 *Components of a modern block cipher*



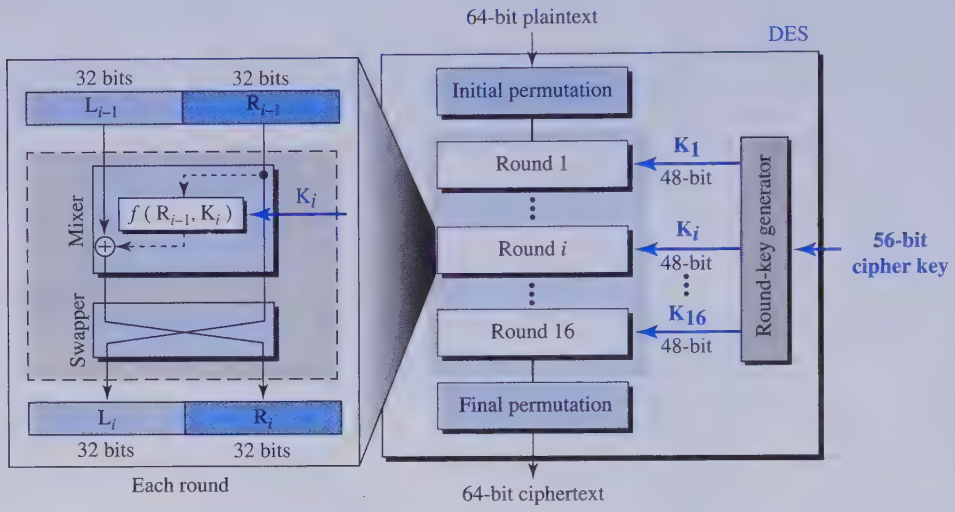
Another component found in some modern block ciphers is the *circular shift operation*. Shifting can be to the left or to the right. The circular left-shift operation shifts each bit in an n-bit word k positions to the left; the leftmost k bits are removed from the left and become the rightmost bits. The *swap operation* is a special case of the circular shift operation where the number of shifted bits $k = n/2$.

Two other operations found in some block ciphers are split and combine. The *split operation* splits an n-bit word in the middle, creating two equal-length words. The *combine operation* normally concatenates two equal-length words, each of size $n/2$ bits, to create an n-bit word.

Data Encryption Standard (DES) As an example of a modern block cipher, let us discuss the **Data Encryption Standard (DES)**. Figure 13.9 shows the elements of DES cipher at the encryption site.

At the encryption site, DES takes a 64-bit plaintext and creates a 64-bit ciphertext; at the decryption site, DES takes a 64-bit ciphertext and creates a 64-bit block of plaintext. The same 56-bit cipher key is used for both encryption and decryption.

Figure 13.9 *General structure of DES*



The initial permutation takes a 64-bit input and permutes them according to a predefined rule. The final permutation is the inverse of the initial permutation. These two permutations cancel the effect of each other. In other words, if the rounds are eliminated from the structures, the ciphertext is the same as the plaintext.

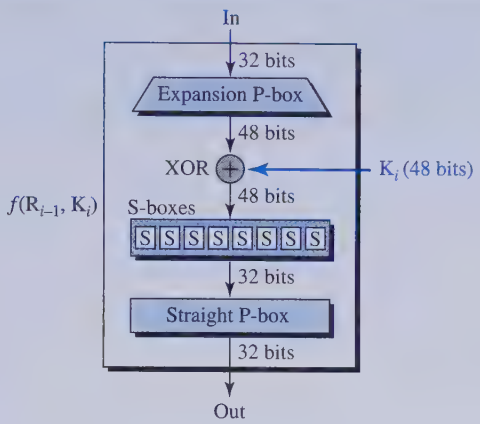
Rounds DES uses 16 rounds. Each round of DES is an invertible (Feistel) transformation, as shown in Figure 13.9. The round takes L_{i-1} and R_{i-1} from the previous round (or the initial permutation box) and creates L_i and R_i, which go to the next round (or final permutation box). Each round can have up to two cipher elements (mixer and swapper). Each of these elements is invertible. The swapper is obviously invertible. It swaps the left half of the text with the right half. The mixer is invertible because of the XOR operation. All noninvertible elements are collected inside the function $f(R_{i-1}, K_i)$.

DES Function The heart of DES is the DES function. The DES function applies a 48-bit key to the rightmost 32 bits (R_{i-1}) to produce a 32-bit output. This function is made up of four sections: an expansion P-box, a whitener (that adds a key), a group of S-boxes, and a straight P-box, as shown in Figure 13.10.

Because R_{i-1} is a 32-bit input and K_i is a 48-bit key, we first need to expand R_{i-1} to 48 bits. This expansion permutation follows a predetermined rule.

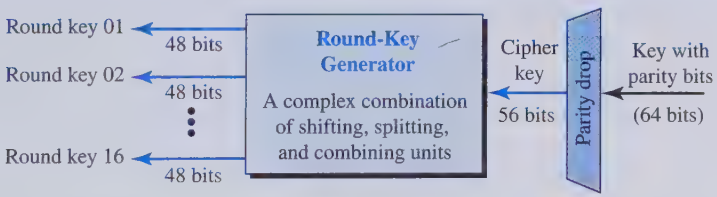
After the expansion permutation, DES uses the XOR operation on the expanded right section and the round key. The S-boxes do the real mixing. DES uses eight S-boxes, each with a 6-bit input and a 4-bit output.The last operation in the DES function is a straight permutation with a 32-bit input and a 32-bit output.

Figure 13.10 *DES function*



Key Generation The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key. However, the cipher key is normally given as a 64-bit key in which 8 extra bits are the parity bits, which are dropped before the actual key-generation process, as shown in Figure 13.11.

Figure 13.11 *Key generation*



Example 13.5

We choose a random plaintext block and a random key and determine (using a program) what the ciphertext block would be (all in hexadecimal):

Plaintext:	Key:	Ciphertext:
123456ABCD132536	**AABB09182736CCDD**	**C0B7A8D05F3A829C**

Example 13.6

To check the effectiveness of DES when a single bit is changed in the input, we use two different plaintexts with only a single bit difference (in a program). The two ciphertexts are completely different without even changing the key.

Although the two plaintext blocks differ only in the rightmost bit, the ciphertext blocks differ in 29 bits.

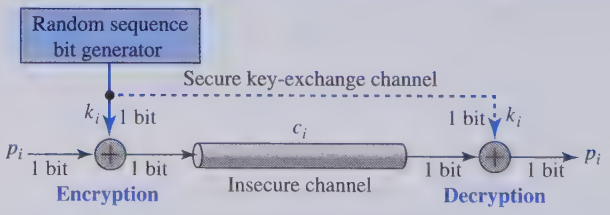
Plaintext:	Key:	Ciphertext:
0000000000000000	22234512987ABB23	4789FD476E82A5F1
Plaintext:	Key:	Ciphertext:
0000000000000001	22234512987ABB23	0A4ED5C15A63FEA3

Modern Stream Ciphers

In addition to modern block ciphers, we can also use modern stream ciphers. The differences between modern stream ciphers and modern block ciphers are similar to the differences between traditional stream and block ciphers, which we explained earlier in this section. In a *modern stream cipher*, encryption and decryption are done r bits at a time. We have a plaintext bit stream $P = p_n \ldots p_2 p_1$, a ciphertext bit stream $C = c_n \ldots c_2 c_1$, and a key bit stream $K = k_n \ldots k_2 k_1$, in which p_i, c_i, and k_i are r-bit words. Encryption is $c_i = E(k_i, p_i)$, and decryption is $p_i = D(k_i, c_i)$. Stream ciphers are faster than block ciphers. The hardware implementation of a stream cipher is also easier. When we need to encrypt binary streams and transmit them at a constant rate, a stream cipher is the better choice to use. Stream ciphers are also more immune to the corruption of bits during transmission.

The simplest and the most secure type of synchronous stream cipher is called the **one-time pad**, which was invented and patented by Gilbert Vernam. A one-time pad cipher uses a key stream that is randomly chosen for each encipherment. The encryption and decryption algorithms each use a single exclusive-OR operation. Based on properties of the exclusive-OR operation, the encryption and decryption algorithms are inverses of each other. It is important to note that in this cipher the exclusive-OR operation is used 1 bit at a time. Note also that there must be a secure channel so that Alice can send the key stream sequence to Bob (Figure 13.12).

Figure 13.12 *One-time pad*



The one-time pad is an ideal cipher. It is perfect. There is no way that an adversary can guess the key or the plaintext and ciphertext statistics. There is no relationship between the plaintext and ciphertext, either. In other words, the ciphertext is a true random stream of bits even if the plaintext contains some patterns. Eve cannot break the cipher unless she tries all possible random key streams, which would be 2^n if the size of the plaintext is n bits. However, there is an issue here. How can the sender and the receiver share a one-time pad key each time they want to communicate? They need to

somehow agree on the random key. So this perfect and ideal cipher is very difficult to achieve. However, there are some feasible, less secured, versions. One of the common alternatives is called a *feedback shift register* (*FSR*), but we leave the discussion of this interesting cipher to the books dedicated to that security topic.

13.2.2 Asymmetric-Key Ciphers

In Section 13.2.1, we discussed symmetric-key ciphers. In this section, we start the discussion of **asymmetric-key ciphers**. Symmetric- and asymmetric-key ciphers will exist in parallel and continue to serve the community. We actually believe that they are complements of each other; the advantages of one can compensate for the disadvantages of the other.

The conceptual differences between the two systems are based on how these systems keep a secret. In symmetric-key cryptography, the secret must be shared between two persons. In asymmetric-key cryptography, the secret is personal (unshared); each person creates and keeps his or her own secret.

In a community of n people, $n(n - 1)/2$ shared secrets are needed for symmetric-key cryptography; only n personal secrets are needed in asymmetric-key cryptography. For a community with a population of 1 million, symmetric-key cryptography would require half a billion shared secrets; asymmetric-key cryptography would require 1 million personal secrets.

Symmetric-key cryptography is based on sharing secrecy; asymmetric-key cryptography is based on personal secrecy.

There are some other aspects of security besides encipherment that need asymmetric-key cryptography. These include authentication and digital signatures. Whenever an application is based on a personal secret, we need to use asymmetric-key cryptography.

Whereas symmetric-key cryptography is based on substitution and permutation of symbols (characters or bits), asymmetric-key cryptography is based on applying mathematical functions to numbers. In symmetric-key cryptography, the plaintext and ciphertext are thought of as a combination of symbols. Encryption and decryption permute these symbols or substitute one symbol for another. In asymmetric-key cryptography, the plaintext and ciphertext are numbers; encryption and decryption are mathematical functions that are applied to numbers to create other numbers.

In symmetric-key cryptography, symbols are permuted or substituted; in asymmetric-key cryptography, numbers are manipulated.

Asymmetric key cryptography uses two separate keys: one private and one public. If encryption and decryption are thought of as locking and unlocking padlocks with keys, then the padlock that is locked with a public key can be unlocked only with the corresponding private key. Figure 13.13 shows that if Alice locks the padlock with Bob's public key, then only Bob's private key can unlock it.

Figure 13.13 *Locking and unlocking in asymmetric-key cryptosystem*

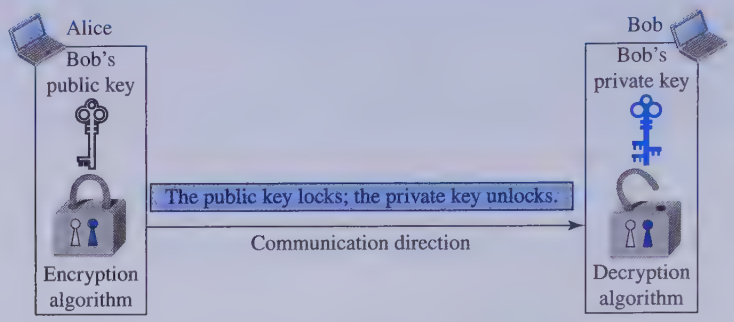


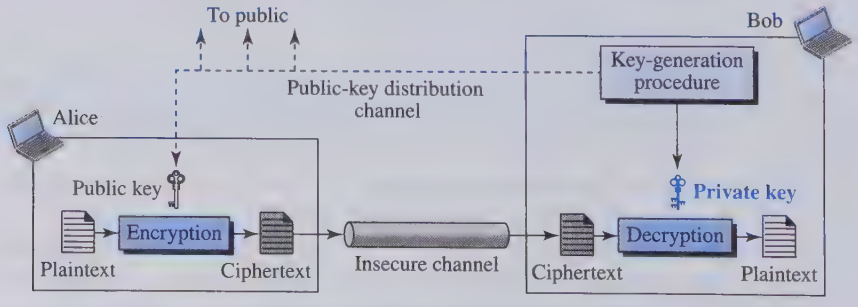
Figure 13.13 shows that, unlike symmetric-key cryptography, there are distinctive keys in asymmetric-key cryptography: a **private key** and a **public key**. Although some books use the term *secret key* instead of *private key*, we use the term *secret key* only for symmetric-key cryptography and the terms *private key* and *public key* for asymmetric-key cryptography. We even use different symbols to show the three keys. In other words, we want to show that a *secret key* is not exchangeable with a *private key*; there are two different types of secrets.

Asymmetric-key ciphers are sometimes called public-key ciphers.

General Idea

Figure 13.14 shows the general idea of asymmetric-key cryptography as used for encipherment and illustrates several important facts. First, it emphasizes the asymmetric nature of the cryptosystem. The burden of providing security is mostly on the shoulders of the receiver (Bob, in this case). Bob needs to create two keys: one private and one public. Bob is responsible for distributing the public key to the community. This can be

Figure 13.14 *General idea of asymmetric-key cryptosystem*



done through a public-key distribution channel. Although this channel is not required to provide secrecy, it must provide authentication and integrity. Eve should not be able to advertise her public key to the community pretending that it is Bob's public key.

Second, asymmetric-key cryptography means that Bob and Alice cannot use the same set of keys for two-way communication. Each entity in the community should create its own private and public keys. Figure 13.14 shows how Alice can use Bob's public key to send encrypted messages to Bob. If Bob wants to respond, Alice needs to establish her own private and public keys.

Third, asymmetric-key cryptography means that Bob needs only one private key to receive all correspondence from anyone in the community, but Alice needs n public keys to communicate with n entities in the community, one public key for each entity. In other words, Alice needs a ring of public keys.

Plaintext/Ciphertext

Unlike in symmetric-key cryptography, plaintext and ciphertext in asymmetric-key cryptography are treated as integers. The message must be encoded as an integer (or a set of integers) before encryption; the integer (or the set of integers) must be decoded into the message after decryption. Asymmetric-key cryptography is normally used to encrypt or decrypt small pieces of information, such as the cipher key for a symmetric-key cryptography. In other words, asymmetric-key cryptography normally is used for ancillary goals instead of message encipherment. However, these ancillary goals play a very important role in cryptography today.

Asymmetric-key cryptography is normally used to encrypt or decrypt small pieces of information.

Encryption/Decryption

Encryption and decryption in asymmetric-key cryptography are mathematical functions applied over the numbers representing the plaintext and ciphertext. The ciphertext can be thought of as $C = f(K_{public}, P)$; the plaintext can be thought of as $P = g(K_{private}, C)$. The decryption function f is used only for encryption; the decryption function g is used only for decryption.

Need for Both

There is a very important fact that is sometimes misunderstood: The advent of asymmetric-key (public-key) cryptography does not eliminate the need for symmetric-key (secret-key) cryptography. The reason is that asymmetric-key cryptography, which uses mathematical functions for encryption and decryption, is much slower than symmetric-key cryptography. For encipherment of large messages, symmetric-key cryptography is still needed. On the other hand, the speed of symmetric-key cryptography does not eliminate the need for asymmetric-key cryptography. Asymmetric-key cryptography is still needed for authentication, digital signatures, and secret-key exchanges. This means that, to be able to use all aspects of security today, we need both symmetric-key and asymmetric-key cryptography. One complements the other.

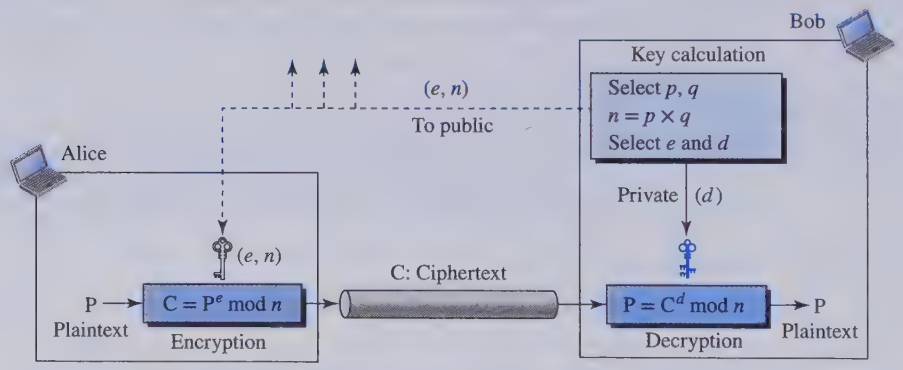
RSA Cryptosystem

Although there are several asymmetric-key cryptosystems, one of the common public-key algorithms is the **RSA cryptosystem**, named for its inventors (Rivest, Shamir, and Adleman). RSA uses two exponents, e and d, where e is public and d is private. Suppose P is the plaintext and C is the ciphertext. Alice uses $C = P^e \bmod n$ to create ciphertext C from plaintext P; Bob uses $P = C^d \bmod n$ to retrieve the plaintext sent by Alice. The modulus n, a very large number, is created during the key generation process.

Procedure

Figure 13.15 shows the general idea behind the procedure used in RSA. Bob chooses two large numbers, p and q, and calculates $n = p \times q$ and $\phi = (p - 1) \times (q - 1)$. Bob then selects e and d such that $(e \times d) \bmod \phi = 1$. Bob advertises e and n to the community as the public key; Bob keeps d as the private key. Anyone, including Alice, can encrypt a message and send the ciphertext to Bob, using $C = (P^e) \bmod n$; only Bob can decrypt the message, using $P = (C^d) \bmod n$. An intruder such as Eve cannot decrypt the message if p and q are very large numbers (she does not know d).

Figure 13.15 *Encryption, decryption, and key generation in RSA*



Example 13.7

For the sake of demonstration, let Bob choose 7 and 11 as p and q and calculate $n = 7 \times 11 = 77$. The value of $\phi(n) = (7 - 1)(11 - 1)$, or 60. If he chooses e to be 13, then d is 37. Note that $e \times d \bmod 60 = 1$. Now imagine that Alice wants to send the plaintext 5 to Bob. She uses the public exponent 13 to encrypt 5. This system is not safe because p and q are small.

Plaintext: 5

$C = 5^{13} = 26 \bmod 77$

Ciphertext: 26

Ciphertext: 26

$P = 26^{37} = 5 \bmod 77$

Plaintext: 5

Example 13.8

Here is a more realistic example calculated using a computer program in Java. We choose a 512-bit p and q and calculate n and $\phi(n)$. We then choose e and calculate d. Finally, we show the results of encryption and decryption. The integer p is a 159-digit number.

$p =$ 96130345313583504574191581280615427909309845594996215822583150879647
94045505647063849125716018034750312098666606492420191808780667421096
063354219926661209

The integer q is a 160-digit number.

$q =$ 12060191957231446918276794204450896001555925054637033936061798321731
48214848376465921538945320917522527322683010712069560460251388714552
4969000359660045617

The modulus $n = p \times q$. It has 309 digits.

$n =$ 11593504173967614968892509864615887523771457375454144775485526137614
78854083263508172768788159683251684688493006254857641112501624145523
39182927162507656772727460097082714127730434960500556347274566628060
09992403710299142447229221577279853172703383938133469268413732762200
09666766718318310883734208234443709 53

$\phi(n) = (p - 1)(q - 1)$ has 309 digits.

Bob chooses $e = 35535$ (the ideal is 65537). He then finds d.

Alice wants to send the message "THIS IS A TEST", which can be changed to a numeric value using the 00–26 encoding scheme (26 is the *space* character).

The ciphertext calculated by Alice is $C = P^e$, as shown in the following:

$\phi(n) =$ 11593504173967614968892509864615887523771457375454144775485526137614
78854083263508172768788159683251684688493006254857641112501624145523
39182927162507656751054233608492916752034482627988117554787657013923
44440571698958172819609822636107546721186461217135910735864061400888
51702653772772644673410662438576641 28

$e =$ 35535

$d =$ 58008302860037763936093661289677917594669062089650962180422866111380
59385282235873170628691003002171085904433840217072986908760061153062
02524959884448047568240966247081485817130463240644077704833134010850
94738529564507193677406119732655742423721761767462077637164207600337
08533328853214470885955136670294831

P = 19070818260818260026190418 19

C = 47530912364622682720636555061054518094237179607049171652323924305445
29606131993285666178434183591141511974112520056829797945717360361012
78218847892741566090480023507190715277185914975188465888632101148354
10336165789846796838676373376577465625079280521148141844044814184430
81277305900469287424855916646210865 6

Bob can recover the plaintext from the ciphertext using $P = C^d$:

P = 19070818260818260026190418 19

The recovered plaintext is "THIS IS A TEST" after decoding.

Applications

Although RSA can be used to encrypt and decrypt actual messages, it is very slow if the message is long. RSA, therefore, is useful for short messages. In particular, we will see that RSA is used in digital signatures and other cryptosystems that often need to encrypt a small message without having access to a symmetric key. RSA is also used for authentication, as we will see later in the chapter.

13.3 OTHER ASPECTS OF SECURITY

The cryptography systems that we have studied so far provide confidentiality. However, in modern communication, we need to take care of other aspects of security, such as integrity, message and entity authentication, non-repudiation, and key management. We briefly discuss these issues in this section.

13.3.1 Message Integrity

There are occasions where we may not even need secrecy but instead must have integrity: The message should remain unchanged. For example, Alice may write a will to distribute her estate upon her death. The will does not need to be encrypted. After her death, anyone can examine the will. The integrity of the will, however, needs to be preserved. Alice does not want the contents of the will to be changed.

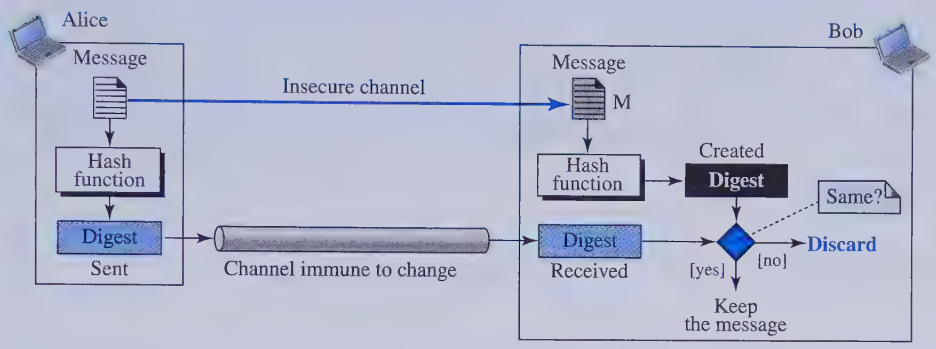
Message and Message Digest

One way to preserve the integrity of a document is through the use of a *fingerprint*. If Alice needs to be sure that the contents of her document will not be changed, she can put her fingerprint at the bottom of the document. Eve cannot modify the contents of this document or create a false document because she cannot forge Alice's fingerprint. To ensure that the document has not been changed, Alice's fingerprint on the document can be compared to Alice's fingerprint on file. If they are not the same, the document is not from Alice. The electronic equivalent of the document and fingerprint pair is the *message* and *digest* pair. To preserve the integrity of a message, the message is passed through an algorithm called a **cryptographic hash function**. The function creates a compressed image of the message, called a **digest**, that can be used like a fingerprint. To check the integrity of a message or document, Bob runs the cryptographic hash function again and compares the new digest with the previous one. If both are the same, Bob is sure that the original message has not been changed. Figure 13.16 shows the idea.

The two pairs (document/fingerprint) and (message/message digest) are similar, with some differences. The document and fingerprint are physically linked together. The message and message digest can be unlinked (or sent separately), and, most importantly, the message digest needs to be safe from change.

The message digest needs to be safe from change.

Figure 13.16 *Message and digest*



Hash Functions

A cryptographic hash function takes a message of arbitrary length and creates a message digest of fixed length. All cryptographic hash functions need to create a fixed-size digest out of a variable-size message. Creating such a function is best accomplished using iteration. Instead of using a hash function with variable-size input, a function with fixed-size input is created and is used a necessary number of times. The fixed-size input function is referred to as a *compression function*. It compresses an n-bit string to create an m-bit string where n is normally greater than m. The scheme is referred to as an *iterated cryptographic hash function*.

Several hash algorithms were designed by Ron Rivest. These are referred to as *MD2*, *MD4*, and *MD5*, where *MD* stands for **Message Digest**. The last version, MD5, is a strengthened version of MD4 that divides the message into blocks of 512 bits and creates a 128-bit digest. It turns out, however, that a message digest of size 128 bits is too small to resist attack.

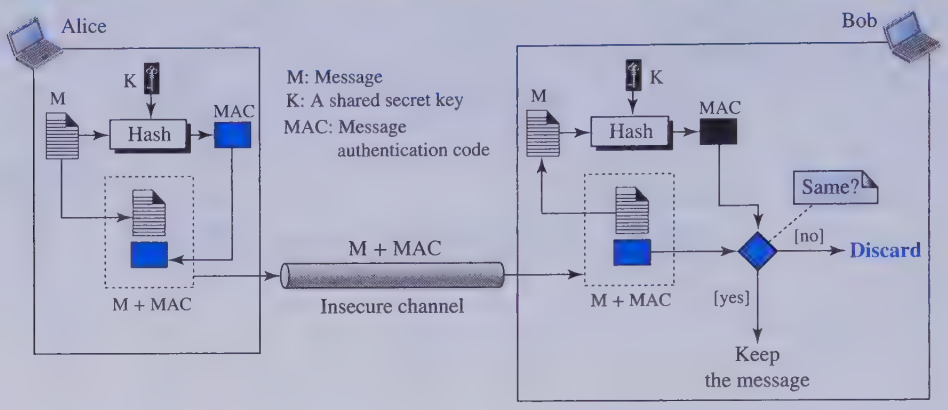
In response to the insecurity of MD hash algorithms, the Secure Hash Algorithm was invented. The **Secure Hash Algorithm (SHA)** is a standard that was developed by the National Institute of Standards and Technology (NIST). SHA has gone through several versions.

13.3.2 Message Authentication

A digest can be used to check the integrity of a message—that the message has not been changed. To ensure the integrity of the message and the data origin authentication—that Alice is the originator of the message, not somebody else—we need to include a secret shared by Alice and Bob (that Eve does not possess) in the process; we need to create a **message authentication code (MAC)**. Figure 13.17 shows the idea.

Alice uses a hash function to create a MAC from the concatenation of the key and the message, $\boldsymbol{h}$ (K + M). She sends the message and the MAC to Bob over the insecure channel. Bob separates the message from the MAC. He then makes a new MAC from the concatenation of the message and the secret key. Bob then compares the newly

Figure 13.17 *Message authentication code*



created MAC with the one received. If the two MACs match, the message is authentic and has not been modified by an adversary.

Note that there is no need to use two channels in this case. Both the message and the MAC can be sent on the same insecure channel. Eve can see the message, but she cannot forge a new message to replace it because Eve does not possess the secret key between Alice and Bob. She is unable to create the same MAC that Alice did.

A MAC provides message integrity and message authentication using a combination of a hash function and a secret key.

HMAC

NIST has issued a standard for a nested MAC that is often referred to as **hashed MAC (HMAC)**. The implementation of HMAC is much more complex than the simplified MAC and is not covered in this text.

13.3.3 Digital Signature

Another way to provide message integrity and message authentication (and some more security services, as we will see shortly) is a digital signature. A MAC uses a secret key to protect the digest; a digital signature uses a pair of private-public keys.

A digital signature uses a pair of private-public keys.

We are all familiar with the concept of a signature. A person signs a document to show that it originated from her or was approved by her. The signature is proof to the recipient that the document comes from the correct entity. When a customer signs a check, the bank needs to be sure that the check is issued by that customer and nobody else. In

other words, a signature on a document, when verified, is a sign of authentication—the document is authentic. Consider a painting signed by an artist. The signature on the art, if authentic, means that the painting is probably authentic.

When Alice sends a message to Bob, Bob needs to check the authenticity of the sender; he needs to be sure that the message comes from Alice and not Eve. Bob can ask Alice to sign the message electronically. In other words, an electronic signature can prove the authenticity of Alice as the sender of the message. We refer to this type of signature as a **digital signature**.

Comparison

Let us begin by looking at the differences between conventional signatures and digital signatures.

Inclusion

A conventional signature is included in the document; it is part of the document. When we write a check, the signature is on the check; it is not a separate document. But when we sign a document digitally, we send the signature as a separate document.

Verification Method

The second difference between the two types of signatures is the method of verifying the signature. For a conventional signature, when the recipient receives a document, she compares the signature on the document with the signature on file. If they are the same, the document is authentic. The recipient needs to have a copy of this signature on file for comparison. For a digital signature, the recipient receives the message and the signature. A copy of the signature is not stored anywhere. The recipient needs to apply a verification technique to the combination of the message and the signature to verify the authenticity.

Relationship

For a conventional signature, there is normally a one-to-many relationship between a signature and documents. A person uses the same signature to sign many documents. For a digital signature, there is a one-to-one relationship between a signature and a message. Each message has its own signature. The signature of one message cannot be used in another message. If Bob receives two messages, one after another, from Alice, he cannot use the signature of the first message to verify the second. Each message needs a new signature.

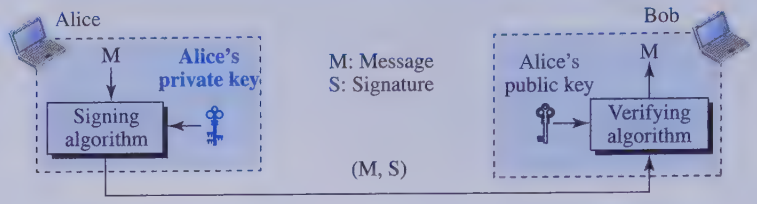
Duplicity

Another difference between the two types of signatures is a quality called *duplicity*. With a conventional signature, a copy of the signed document can be distinguished from the original one on file. With a digital signature, there is no such distinction unless there is a factor of time (such as a timestamp) on the document. For example, suppose Alice sends a document instructing Bob to pay Eve. If Eve intercepts the document and the signature, she can resend it later to get money again from Bob.

Process

Figure 13.18 shows the digital signature process. The sender uses a *signing algorithm* to sign the message. The message and the signature are sent to the receiver. The receiver

Figure 13.18 *Digital signature process*



receives the message and the signature and applies the *verifying algorithm* to the combination. If the result is true, the message is accepted; otherwise, it is rejected.

A conventional signature is like a private "key" belonging to the signer of the document. The signer uses it to sign documents; no one else has this signature. The copy of the signature on file is like a public key; anyone can use it to verify a document, to compare it to the original signature.

In a digital signature, the signer uses her private key, applied to a signing algorithm, to sign the document. The verifier, on the other hand, uses the public key of the signer, applied to the verifying algorithm, to verify the document.

Note that when a document is signed, anyone, including Bob, can verify it because everyone has access to Alice's public key. Alice must not use her public key to sign the document because then anyone could forge her signature.

Can we use a secret (symmetric) key to both sign and verify a signature? The answer is negative for several reasons. First, a secret key is known by only two entities (Alice and Bob, for example). So if Alice needs to sign another document and send it to Ted, she needs to use another secret key. Second, as we will see, creating a secret key for a session involves authentication, which uses a digital signature. We have a vicious cycle. Third, Bob could use the secret key between himself and Alice, sign a document, send it to Ted, and pretend that it came from Alice.

A digital signature needs a public-key system.
The signer signs with her private key; the verifier verifies with the signer's public key.

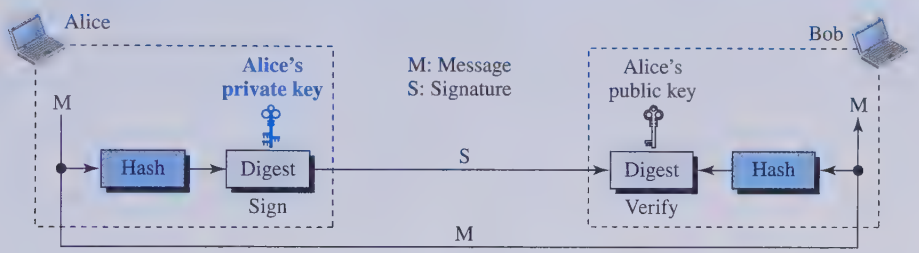
We should make a distinction between private and public keys as used in digital signatures and public and private keys as used in a cryptosystem for confidentiality. In the latter, the public and private keys of the receiver are used in the process. The sender uses the public key of the receiver to encrypt; the receiver uses his own private key to decrypt. In a digital signature, the private and public keys of the sender are used. The sender uses her private key; the receiver uses the sender's public key.

A cryptosystem uses the private and public keys of the receiver;
a digital signature uses the private and public keys of the sender.

Signing the Digest

We said before that the asymmetric-key cryptosystems are very inefficient when dealing with long messages. In a digital signature system, the messages are normally long, but we have to use asymmetric-key schemes. The solution is to sign a digest of the message, which is much shorter than the message. A carefully selected message digest has a one-to-one relationship with the message. The sender can sign the message digest, and the receiver can verify the message digest. The effect is the same. Figure 13.19 shows signing a digest in a digital signature system.

Figure 13.19 *Signing the digest*



A digest is made out of the message at Alice's site. The digest then goes through the signing process using Alice's private key. Alice then sends the message and the signature to Bob.

At Bob's site, using the same public hash function, a digest is first created out of the received message. The verifying process is applied. If authentic, the message is accepted; otherwise, it is rejected.

Services

We discussed several security services in the beginning of the chapter including *message confidentiality*, *message authentication*, *message integrity*, and *non-repudiation*. A digital signature can directly provide the last three; for message confidentiality we still need encryption/decryption.

Message Authentication

A secure digital signature scheme, like a secure conventional signature (one that cannot be easily copied) can provide message authentication (also referred to as data-origin authentication). Bob can verify that the message is sent by Alice because Alice's public key is used in verification. Alice's public key cannot verify the signature signed by Eve's private key.

Message Integrity

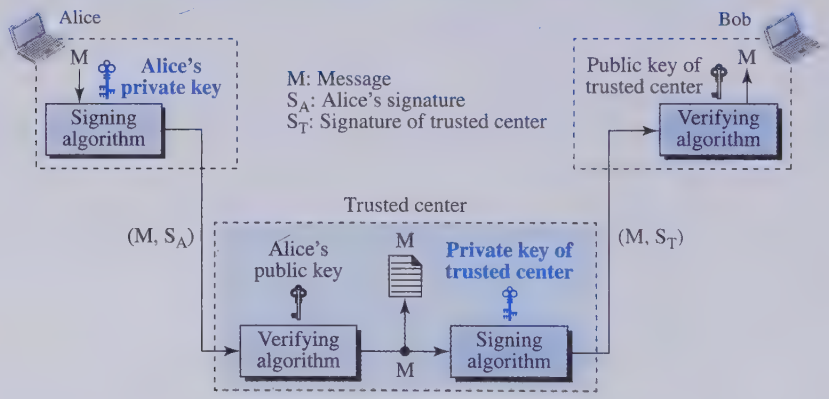
The integrity of the message is preserved if we sign the message or the digest of the message because we cannot get the same digest if any part of the message is changed. The digital signature schemes today use a hash function in the signing and verifying algorithms that preserves the integrity of the message.

Non-repudiation

If Alice signs a message and then denies it, can Bob later prove that Alice actually signed it? For example, if Alice sends a message to a bank (Bob) and asks to transfer $10,000 from her account to Ted's account, can Alice later deny that she sent this message? With the scheme we have presented so far, Bob might have a problem. Bob must keep the signature on file and later use Alice's public key to create the original message to prove the message in the file and the newly created message are the same. This is not feasible because Alice may have changed her private or public key during this time; she may also claim that the file containing the signature is not authentic.

One solution is a trusted third party. People can create an established trusted party among themselves. Later in the chapter, we will see that a trusted party can solve many other problems concerning security services and key exchange. Figure 13.20 shows how a trusted party can prevent Alice from denying that she sent the message.

Figure 13.20 *Using a trusted center for non-repudiation*



Alice creates a signature from her message (S_A) and sends the message, her identity, Bob's identity, and the signature to the center. The center, after checking that Alice's public key is valid, verifies through Alice's public key that the message came from Alice. The center then saves a copy of the message with the sender identity, recipient identity, and a timestamp in its archive. The center uses its private key to create another signature (S_T) from the message. The center then sends the message, the new signature, Alice's identity, and Bob's identity to Bob. Bob verifies the message using the public key of the trusted center.

If in the future Alice denies that she sent the message, the center can show a copy of the saved message. If Bob's message is a duplicate of the message saved at the center, Alice will lose the dispute. To make everything confidential, a level of encryption/decryption can be added to the scheme, as discussed next.

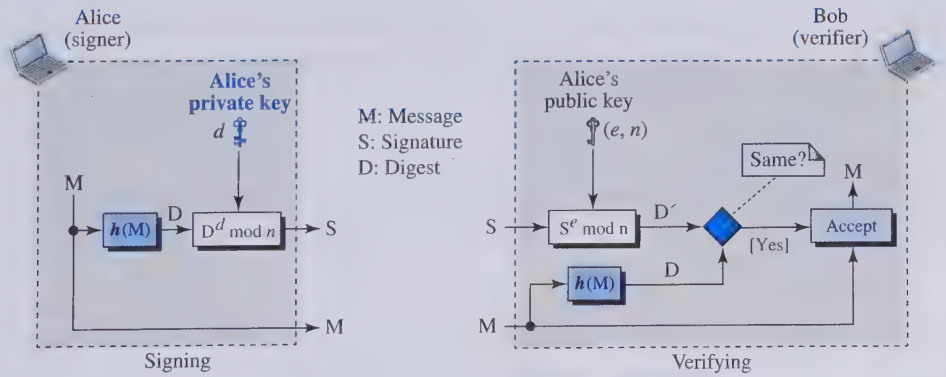
Confidentiality

A digital signature does not provide confidential communication. If confidentiality is required, the message and the signature must be encrypted using either a symmetric-key or an asymmetric-key cipher.

RSA Digital Signature Scheme

Several *digital signature schemes* have evolved during the last few decades. Some of them have been implemented. In this section, we briefly show one of them, RSA. In Section 13.2.2, we discussed how to use the RSA cryptosystem to provide privacy. The RSA idea can also be used for signing and verifying a message. In this case, it is called the *RSA digital signature scheme*. The digital signature scheme changes the roles of the private and public keys. First, the private and public keys of the sender, not the receiver, are used. Second, the sender uses her own private key to sign the document; the receiver uses the sender's public key to verify it. If we compare the scheme with the conventional way of signing, we see that the private key plays the role of the sender's own signature and the sender's public key plays the role of the copy of the signature that is available to the public. Obviously, Alice cannot use Bob's public key to sign the message, because then any other person could do the same. The signing and verifying sites use the same function, but with different parameters. The verifier compares the message and the output of the function for equality in modulo arithmetic. If the result is true, the message is accepted. Figure 13.21 shows the scheme in which the signing and verifying is done on the digest of the message instead of the message itself because the public-key cryptography is not very efficient for use with long messages; the digest is much smaller than the message itself.

Figure 13.21 *The RSA signature on the message digest*



Alice, the signer, first uses an agreed-upon hash function to create a digest from the message, $D = \boldsymbol{h}(M)$. She then signs the digest, $S = D^d \bmod n$. The message and the signature are sent to Bob. Bob, the verifier, receives the message and the signature. He first uses Alice's public exponent to retrieve the digest, $D' = S^e \bmod n$. He then applies

the hash algorithm to the message received to obtain $D = h(M)$. Bob now compares the two digests, D and D′. If they are equal (in modulo arithmetic), he accepts the message.

Digital Signature Standard (DSS)

The **Digital Signature Standard (DSS)** was adopted by NIST in 1994. DSS is a complicated, and more secure, digital signature scheme.

13.3.4 Entity Authentication

Entity authentication is a technique designed to let one party verify the identity of another party. An *entity* can be a person, a process, a client, or a server. The entity whose identity needs to be proven is called the *claimant*; the party that tries to verify the identity of the claimant is called the *verifier*.

Entity versus Message Authentication

There are two differences between *entity authentication and message authentication (data-origin authentication).*

1. Message authentication (or data-origin authentication) might not happen in real time; entity authentication does. In the former, Alice sends a message to Bob. When Bob authenticates the message, Alice may or may not be present in the communication process. On the other hand, when Alice requests entity authentication, there is no real message communication involved until Alice is authenticated by Bob. Alice needs to be online and to take part in the process. Only after she is authenticated can messages be communicated between Alice and Bob. Data-origin authentication is required when an e-mail is sent from Alice to Bob. Entity authentication is required when Alice gets cash from an automatic teller machine.
2. Message authentication simply authenticates one message; the process needs to be repeated for each new message. Entity authentication authenticates the claimant for the entire duration of a session.

Verification Categories

In entity authentication, the claimant must identify herself to the verifier. This can be done with one of three kinds of witnesses: *something known*, *something possessed*, or *something inherent*.

- **Something known.** This is a secret known only by the claimant that can be checked by the verifier. Examples are a password, a PIN, a secret key, and a private key.
- **Something possessed.** This is something that can prove the claimant's identity. Examples are a passport, a driver's license, an identification card, a credit card, and a smart card.
- **Something inherent.** This is an inherent characteristic of the claimant. Examples are conventional signatures, fingerprints, voice, facial characteristics, retinal pattern, and handwriting.

In this section, we only discuss the first type of witness, *something known*, which is normally used for remote (online) entity authentication. The other two categories are normally used when the claimant is personally present.

Passwords

The simplest and oldest method of entity authentication is the use of a *password*, which is something that the claimant *knows*. A password is used when a user needs to access a system's resources (login). Each user has a user identification that is public, and a password that is private. Passwords, however, are very prone to attack. A password can be stolen, intercepted, guessed, and so on.

Challenge-Response

In password authentication, the claimant proves her identity by demonstrating that she knows a secret, the password. However, because the claimant sends this secret, it is susceptible to interception by the adversary. In **challenge-response authentication**, the claimant proves that she *knows* a secret without sending it. In other words, the claimant does not send the secret to the verifier; the verifier either has it or finds it.

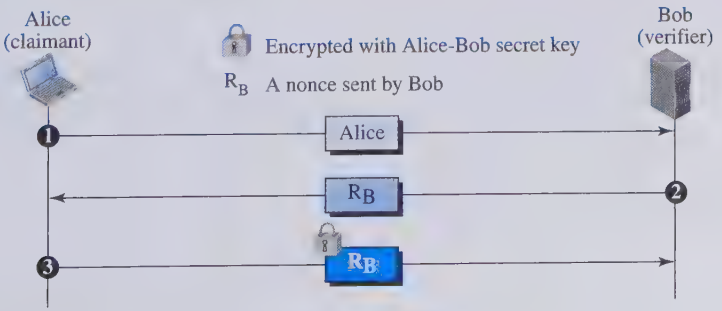
In challenge-response authentication, the claimant proves that she knows a secret without sending it to the verifier.

The *challenge* is a time-varying value such as a random number or a timestamp that is sent by the verifier. The claimant applies a function to the challenge and sends the result, called a *response*, to the verifier. The response shows that the claimant knows the secret.

Using a Symmetric-Key Cipher

Several approaches to challenge-response authentication use symmetric-key encryption. The secret here is the shared secret key, known by both the claimant and the verifier. The function is the encrypting algorithm applied on the challenge. Although there are several approaches to this method, we just show the simplest one to give an idea. Figure 13.22 shows this first approach.

Figure 13.22 *Unidirectional, symmetric-key authentication*



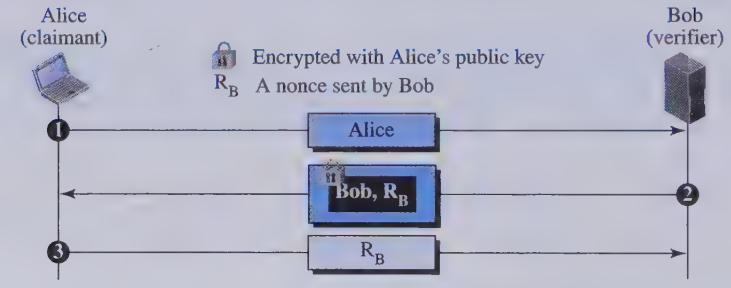
The first message is not part of challenge-response, it only informs the verifier that the claimant wants to be challenged. The second message is the challenge. R_B is the nonce (abbreviation for *number once*) randomly chosen by the verifier (Bob) to challenge the claimant. The claimant encrypts the nonce using the shared secret key known only to the claimant and the verifier and sends the result to the verifier. The verifier decrypts the message. If the nonce obtained from decryption is the same as the one sent by the verifier, Alice is granted access.

Note that in this process, the claimant and the verifier need to keep the symmetric key used in the process secret. The verifier must also keep the value of the nonce for claimant identification until the response is returned.

Using an Asymmetric-Key Cipher

Figure 13.23 shows this approach.

Figure 13.23 *Unidirectional, asymmetric-key authentication*



Instead of a symmetric-key cipher, we can use an asymmetric-key cipher for entity authentication. Here the secret must be the private key of the claimant. The claimant must show that she owns the private key related to the public key that is available to everyone. This means that the verifier must encrypt the challenge using the public key of the claimant; the claimant then decrypts the message using her private key. The response to the challenge is the decrypted message.

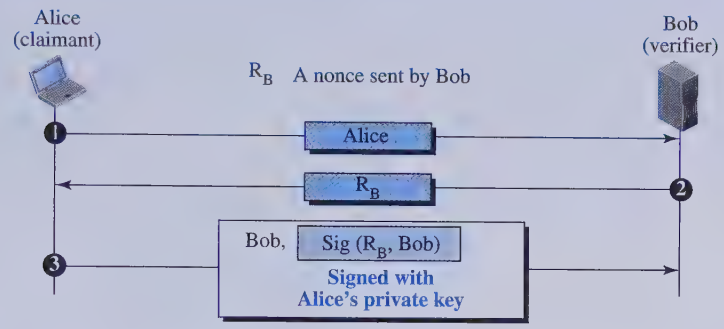
Using Digital Signatures

Entity authentication can also be achieved using a digital signature. When a digital signature is used for entity authentication, the claimant uses her private key for signing. In the first approach, shown in Figure 13.24, Bob uses a plaintext challenge and Alice signs the response.

13.3.5 Key Management

We previously discussed symmetric-key and asymmetric-key cryptography. However, we have not yet discussed how secret keys in symmetric-key cryptography, and public

Figure 13.24 *Digital signature, unidirectional authentication*



keys in asymmetric-key cryptography, are distributed and maintained. This section touches on these two issues.

Symmetric-Key Distribution

Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages. Symmetric-key cryptography, however, needs a shared secret key between two parties.

If Alice needs to exchange confidential messages with N people, she needs N different keys. What if N people need to communicate with each other? A total of $N(N-1)$ keys is needed if we require that two people use two keys for bidirectional communication; only $N(N-1)/2$ keys are needed if we allow a key to be used for both directions. This means that if one million people need to communicate with each other, each person has almost one million different keys; in total, half a trillion keys are needed. This is normally referred to as the N^2 problem because the number of required keys for N entities is close to N^2.

The number of keys is not the only problem; the distribution of keys is another. If Alice and Bob want to communicate, they need a way to exchange a secret key; if Alice wants to communicate with one million people, how can she exchange one million keys with one million people? Using the Internet is definitely not a secure method. It is obvious that we need an efficient way to maintain and distribute secret keys.

Key Distribution Center: KDC

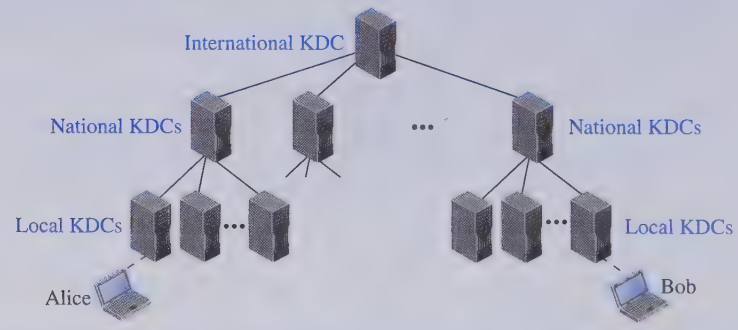
A practical solution is the use of a trusted third party, referred to as a **key distribution center (KDC)**. To reduce the number of keys, each person establishes a shared secret key with the KDC. A secret key is established between the KDC and each member. Now the question is, "How can Alice send a confidential message to Bob?" The process is as follows:

1. Alice sends a request to the KDC stating that she needs a session (temporary) secret key between herself and Bob.
2. The KDC informs Bob about Alice's request.
3. If Bob agrees, a session key is created between the two.

The secret key between Alice and Bob that is established with the KDC is used to authenticate Alice and Bob to the KDC and to prevent Eve from impersonating either of them.

Multiple KDCs When the number of people using a KDC increases, the system becomes unmanageable and a bottleneck can result. To solve the problem, we need to have multiple KDCs. We can divide the world into domains. Each domain can have one or more KDCs (for redundancy in case of failure). Now if Alice wants to send a confidential message to Bob, who belongs to another domain, Alice contacts her KDC, which in turn contacts the KDC in Bob's domain. The two KDCs can create a secret key between Alice and Bob. There can be local KDCs, national KDCs, and international KDCs. When Alice needs to communicate with Bob, who lives in another country, she sends her request to a local KDC; the local KDC relays the request to the national KDC; the national KDC relays the request to an international KDC. The request is then relayed all the way down to the local KDC where Bob lives. Figure 13.25 shows a configuration of hierarchical multiple KDCs.

Figure 13.25 *Multiple KDCs*

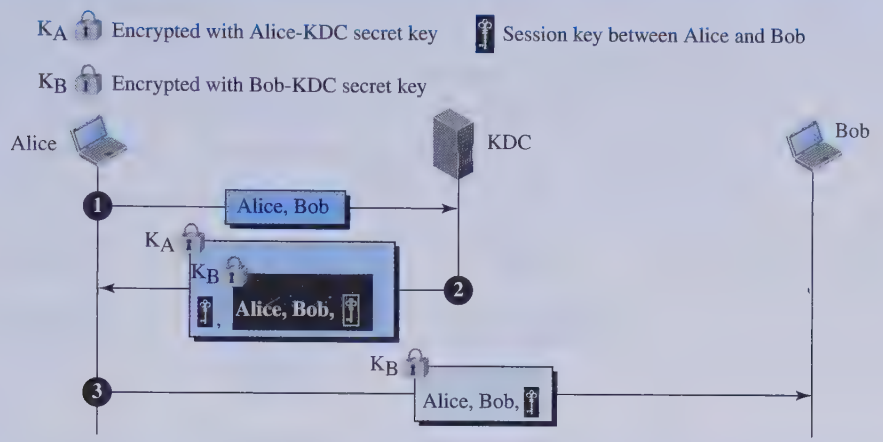


Session Keys A KDC creates a secret key for each member. This secret key can be used only between the member and the KDC, not between two members. If Alice needs to communicate secretly with Bob, she needs a secret key between herself and Bob. A KDC can create a *session key* between Alice and Bob, using their keys with the center. The keys of Alice and Bob are used to authenticate Alice and Bob to the center and to each other before the session key is established. After communication is terminated, the session key is no longer useful.

A session symmetric key between two parties is used only once.

Several different approaches have been proposed to create the session key using ideas discussed in previous sections. We show the simplest approach in Figure 13.26.

Figure 13.26 *Creating a session key using KDC*



Although this approach is very rudimentary, it helps to understand more sophisticated approaches in the literature.

1. Alice sends a plaintext message to the KDC to obtain a symmetric session key between Bob and herself. The message contains her registered identity (the word *Alice* in the figure) and the identity of Bob (the word *Bob* in the figure). This message is not encrypted; it is public. The KDC does not care.
2. The KDC receives the message and creates what is called a **ticket**. The ticket is encrypted using Bob's key (K_B). The ticket contains the identities of Alice and Bob and the session key. The ticket with a copy of the session key is sent to Alice. Alice receives the message, decrypts it, and extracts the session key. She cannot decrypt Bob's ticket; the ticket is for Bob, not for Alice. Note that this message contains a double encryption—the ticket is encrypted, and the entire message is also encrypted. In the second message, Alice is actually authenticated to the KDC, because only Alice can open the whole message using her secret key with KDC.
3. Alice sends the ticket to Bob. Bob opens the ticket and knows that Alice needs to send messages to him using the session key. Note that in this message, Bob is authenticated to the KDC because only Bob can open the ticket. Because Bob is authenticated to the KDC, he is also authenticated to Alice, who trusts the KDC. In the same way, Alice is also authenticated to Bob, because Bob trusts the KDC and the KDC has sent Bob the ticket that includes the identity of Alice.

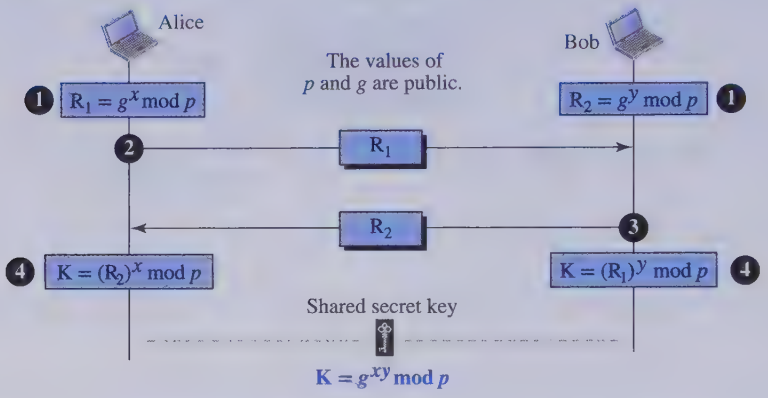
Symmetric-Key Agreement

Alice and Bob can create a session key between themselves without using a KDC. This method of session-key creation is referred to as the *symmetric-key agreement*. Although there are several ways to accomplish this, we discuss only one method, Diffie-Hellman, which shows the basic idea used in more sophisticated (less prone to attack) methods.

Diffie-Hellman Key Agreement

In the **Diffie-Hellman Protocol**, two parties create a symmetric session key without the need of a KDC. Before establishing a symmetric key, the two parties need to choose two numbers p and g. These two numbers have some properties discussed in number theory, but that discussion is beyond the scope of this book. These two numbers do not need to be confidential. They can be sent through the Internet; they can be public. Figure 13.27 shows the procedure.

Figure 13.27 *Diffie-Hellman method*



The steps are as follows:

1. Alice chooses a large random number x such that $0 \leq x \leq p - 1$ and calculates $R_1 = g^x \bmod p$. Bob chooses another large random number y such that $0 \leq y \leq p - 1$ and calculates $R_2 = g^y \bmod p$.
2. Alice sends R_1 to Bob. Note that Alice does not send the value of x; she sends only R_1.
3. Bob sends R_2 to Alice. Again, note that Bob does not send the value of y; he sends only R_2.
4. Alice calculates $K = (R_2)^x \bmod p$. Bob also calculates $K = (R_1)^y \bmod p$.

K is the symmetric key for the session.

$$K = (g^x \bmod p)^y \bmod p = (g^y \bmod p)^x \bmod p = g^{xy} \bmod p$$

Bob has calculated $K = (R_1)^y \bmod p = (g^x \bmod p)^y \bmod p = g^{xy} \bmod p$. Alice has calculated $K = (R_2)^x \bmod p = (g^y \bmod p)^x \bmod = g^{xy} \bmod p$. Both have reached the same value without Bob knowing the value of x and without Alice knowing the value of y.

The symmetric (shared) key in the Diffie-Hellman method is $K = g^{xy} \bmod p$.

Example 13.9

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. Alice chooses $x = 3$ and calculates $R_1 = 7^3 \bmod 23 = 21$. Bob chooses $y = 6$ and calculates $R_2 = 7^6 \bmod 23 = 4$.
2. Alice sends the number 21 to Bob.
3. Bob sends the number 4 to Alice.
4. Alice calculates the symmetric key $K = 4^3 \bmod 23 = 18$. Bob calculates the symmetric key $K = 21^6 \bmod 23 = 18$.

The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7^{18} \bmod 23 = 18$.

Public-Key Distribution

In asymmetric-key cryptography, people do not need to know a symmetric shared key. If Alice wants to send a message to Bob, she only needs to know Bob's public key, which is open to the public and available to everyone. If Bob needs to send a message to Alice, he only needs to know Alice's public key, which is also known to everyone. In public-key cryptography, everyone shields a private key and advertises a public key.

In public-key cryptography, everyone has access to everyone's public key; public keys are available to the public.

Public keys, like secret keys, need to be distributed to be useful. Let us briefly discuss the ways public keys can be distributed.

Public Announcement

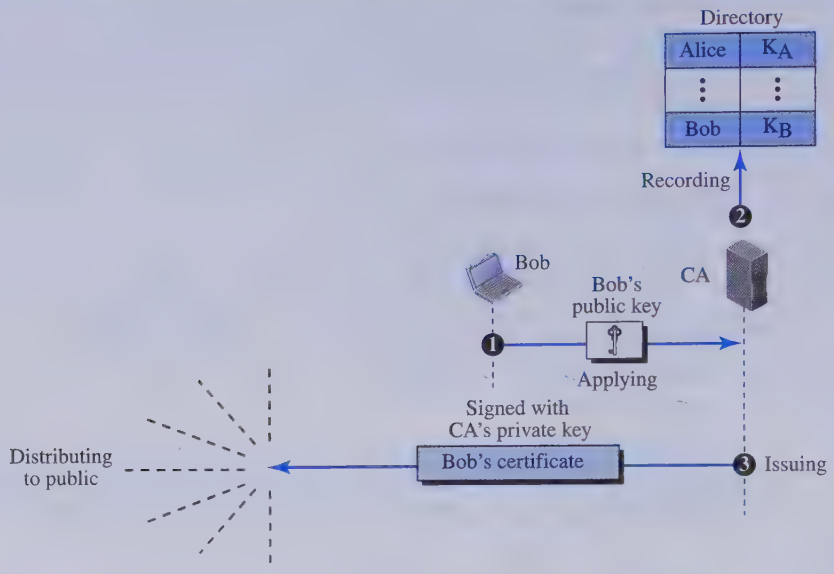
The naive approach is to announce public keys publicly. Bob can put his public key on his website or announce it in a local or national newspaper. When Alice needs to send a confidential message to Bob, she can obtain Bob's public key from his site or from the newspaper, or even send a message to ask for it. This approach, however, is not secure; it is subject to forgery. For example, Eve could make such a public announcement. Before Bob can react, damage could be done. Eve can fool Alice into sending her a message that is intended for Bob. Eve could also sign a document with a corresponding forged private key and make everyone believe it was signed by Bob. The approach is also vulnerable if Alice directly requests Bob's public key. Eve can intercept Bob's response and substitute her own forged public key for Bob's public key.

Certification Authority

The common approach to distributing public keys is to create **public-key certificates**. Bob wants two things; he wants people to know his public key, and he wants no one to accept a forged public key as his. Bob can go to a **certification authority (CA)**, a federal or state organization that binds a public key to an entity and issues a certificate. Figure 13.28 shows the concept.

The CA itself has a well-known public key that cannot be forged. The CA checks Bob's identification (using a picture ID along with other proof). It then asks for Bob's public key and writes it on the certificate. To prevent the certificate itself from being forged, the CA signs the certificate with its private key. Now Bob can upload the signed certificate. Anyone who wants Bob's public key downloads the signed certificate and uses the authority's public key to extract Bob's public key.

Figure 13.28 *Certification authority*



X.509

Although the use of a CA has solved the problem of public-key fraud, it has created a side effect. Each certificate may have a different format. If Alice wants to use a program to automatically download different certificates and digests belonging to different people, the program may not be able to do this. One certificate may have the public key in one format, and another certificate may have it in a different format. The public key may be on the first line in one certificate and on the third line in another. Anything that needs to be used universally must have a universal format. To remove this side effect, the International Telecommunications Union (ITU) has designed **X.509**, a recommendation that has been accepted by the Internet with some changes. X.509 is a way to describe the certificate in a structured way. It uses a well-known protocol called ASN.1

13.4 NETWORK-LAYER SECURITY

We now concentrate on the security at the network layer. At the network layer, security is applied between two hosts, two routers, or a host and a router. The purpose of network-layer security is to protect those applications that use the service of the network layer directly, such as routing protocols. Those applications that use the service of UDP can also benefit from this service because UDP is a connectionless protocol and transport-layer security protocols, as we discuss in Section 13.5, cannot be applied to UDP.

The only network-layer security we discuss here is called IPSec. **IP Security (IPSec)** is a collection of protocols designed by the Internet Engineering Task Force (IETF) to provide security for a packet at the network level. IPSec helps create authenticated and confidential packets for the IP layer.

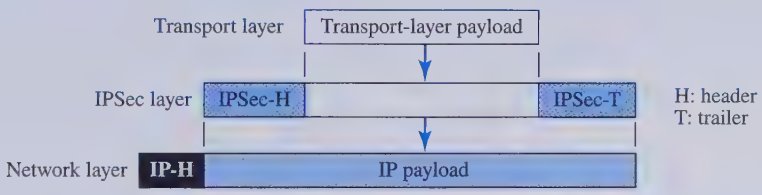
13.4.1 Two Modes

IPSec operates in one of two different modes: transport mode or tunnel mode.

Transport Mode

In **transport mode**, IPSec protects what is delivered from the transport layer to the network layer. In other words, transport mode protects the payload to be encapsulated in the network layer, as shown in Figure 13.29.

Figure 13.29 *IPSec in transport mode*

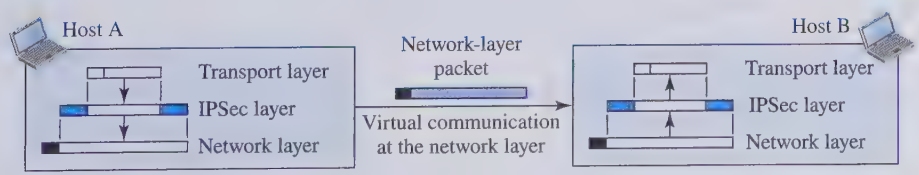


Note that transport mode does not protect the IP header. In other words, transport mode does not protect the whole IP packet; it protects only the packet from the transport layer (the IP-layer payload). In this mode, the IPSec header (and trailer) are added to the information coming from the transport layer. The IP header is added later.

IPSec in transport mode does not protect the IP header; it only protects the payload coming from the transport layer.

Transport mode is normally used when we need host-to-host (end-to-end) protection of data. The sending host uses IPSec to authenticate and/or encrypt the payload delivered from the transport layer. The receiving host uses IPSec to check the authentication and/or decrypt the IP packet and deliver it to the transport layer. Figure 13.30 shows this concept.

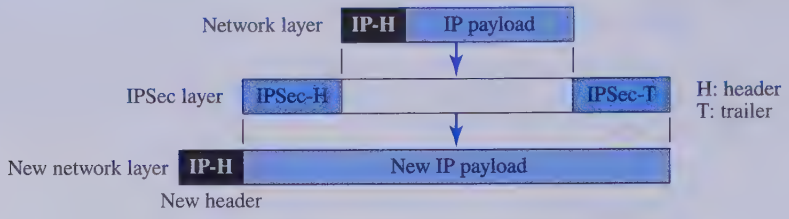
Figure 13.30 *Transport mode in action*



Tunnel Mode

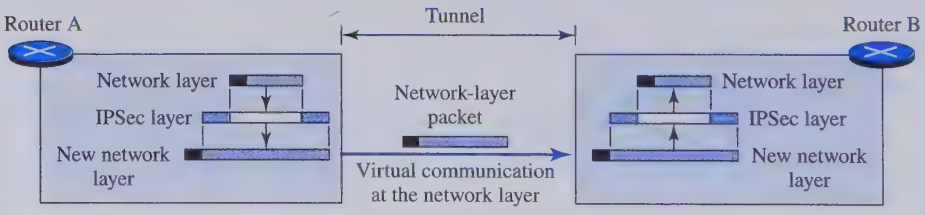
In **tunnel mode**, IPSec protects the entire IP packet. It takes an IP packet, including the header, applies IPSec security methods to the entire packet, and then adds a new IP header, as shown in Figure 13.31.

Figure 13.31 *IPSec in tunnel mode*



The new IP header, as we will see shortly, has different information than the original IP header. Tunnel mode is normally used between two routers, between a host and a router, or between a router and a host, as shown in Figure 13.32. The entire original packet is protected from intrusion between the sender and the receiver, as if the whole packet goes through an imaginary tunnel.

Figure 13.32 *Tunnel mode in action*



IPSec in tunnel mode protects the original IP header.

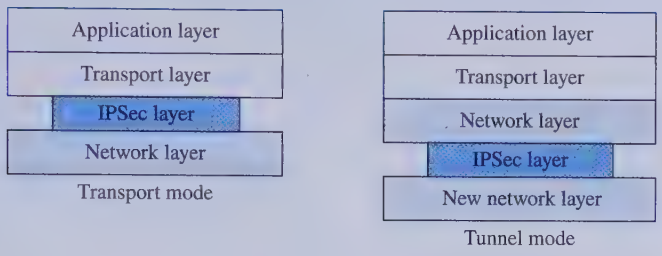
Comparison

In transport mode, the IPSec layer comes between the transport layer and the network layer. In tunnel mode, the flow is from the network layer to the IPSec layer and then back to the network layer again. Figure 13.33 compares the two modes.

13.4.2 Two Security Protocols

IPSec defines two protocols—the Authentication Header (AH) Protocol and the Encapsulating Security Payload (ESP) Protocol—to provide authentication and/or encryption for packets at the IP level.

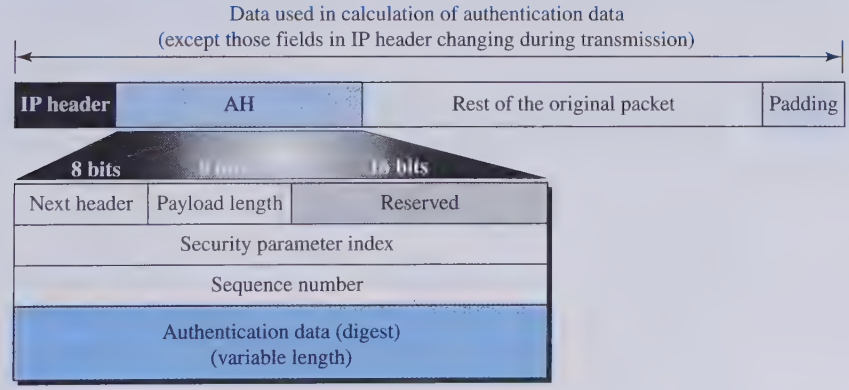
Figure 13.33 *Transport mode versus tunnel mode*



Authentication Header (AH)

The **Authentication Header (AH) Protocol** is designed to authenticate the source host and to ensure the integrity of the payload carried in the IP packet. The protocol uses a hash function and a symmetric (secret) key to create a message digest; the digest is inserted in the authentication header (see MAC). The AH is then placed in the appropriate location, based on the mode (transport or tunnel). Figure 13.34 shows the fields and the position of the authentication header in transport mode.

Figure 13.34 *Authentication Header (AH) protocol*



When an IP datagram carries an authentication header, the original value in the protocol field of the IP header is replaced by the value 51. A field inside the authentication header (the next header field) holds the original value of the protocol field (the type of payload being carried by the IP datagram). The addition of an authentication header follows these steps:

1. An authentication header is added to the payload with the authentication data field set to 0.
2. Padding may be added to make the total length appropriate for a particular hashing algorithm.

3. Hashing is based on the total packet. However, only those fields of the IP header that do not change during transmission are included in the calculation of the message digest (authentication data).
4. The authentication data are inserted in the authentication header.
5. The IP header is added after changing the value of the protocol field to 51.

A brief description of each field follows:

- **Next header.** The 8-bit next header field defines the type of payload carried by the IP datagram (such as TCP, UDP, ICMP, or OSPF).
- **Payload length.** The name of this 8-bit field is misleading. It does not define the length of the payload; it defines the length of the authentication header in 4-byte multiples, but it does not include the first 8 bytes.
- **Security parameter index.** The 32-bit security parameter index (SPI) field plays the role of a virtual circuit identifier and is the same for all packets sent during a connection called a Security Association (discussed in section 13.4.4).
- **Sequence number.** A 32-bit sequence number provides ordering information for a sequence of datagrams. The sequence numbers prevent a playback. Note that the sequence number is not repeated even if a packet is retransmitted. A sequence number does not wrap around after it reaches 2^{32}; a new connection must be established.
- **Authentication data.** Finally, the authentication data field is the result of applying a hash function to the entire IP datagram except for the fields that are changed during transit (e.g., time-to-live).

The AH protocol provides source authentication and data integrity, but not privacy.

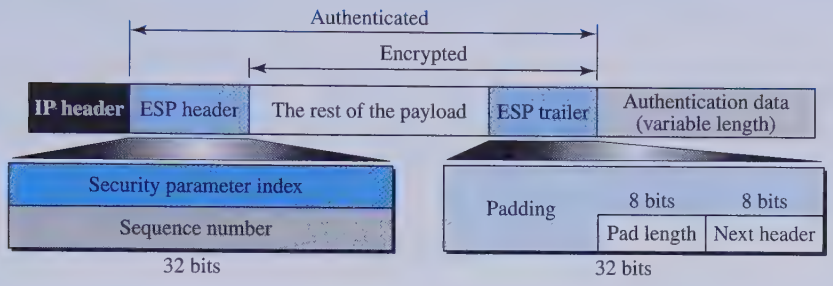
Encapsulating Security Payload (ESP)

The AH protocol does not provide confidentiality, only source authentication and data integrity. IPSec later defined an alternative protocol, **Encapsulating Security Payload (ESP)**, that provides source authentication, integrity, and confidentiality. ESP adds a header and trailer. Note that ESP's authentication data are added at the end of the packet, which makes its calculation easier. Figure 13.35 shows the location of the ESP header and trailer.

When an IP datagram carries an ESP header and trailer, the value of the protocol field in the IP header is 50. A field inside the ESP trailer (the next-header field) holds the original value of the protocol field (the type of payload being carried by the IP datagram, such as TCP or UDP). The ESP procedure follows these steps:

1. An ESP trailer is added to the payload.
2. The payload and the trailer are encrypted.
3. The ESP header is added.
4. The ESP header, payload, and ESP trailer are used to create the authentication data.

Figure 13.35 *Encapsulating Security Payload (ESP)*



5. The authentication data are added to the end of the ESP trailer.
6. The IP header is added after changing the protocol value to 50.

The fields for the header and trailer are as follows:

- **Security parameter index.** The 32-bit security parameter index field is similar to the one defined for the AH protocol.
- **Sequence number.** The 32-bit sequence number field is similar to the one defined for the AH protocol.
- **Padding.** This variable-length field (0 to 255 bytes) of 0s serves as padding.
- **Pad length.** The 8-bit pad-length field defines the number of padding bytes. The value is between 0 and 255; the maximum value is rare.
- **Next header.** The 8-bit next-header field is similar to that defined in the AH protocol. It serves the same purpose as the protocol field in the IP header before encapsulation.
- **Authentication data.** Finally, the authentication data field is the result of applying an authentication scheme to parts of the datagram. Note the difference between the authentication data in AH and ESP. In AH, part of the IP header is included in the calculation of the authentication data; in ESP, it is not.

IPv4 and IPv6

IPSec supports both IPv4 and IPv6. In IPv6, however, AH and ESP are part of the extension header.

AH versus ESP

The ESP protocol was designed after the AH protocol was already in use. ESP does whatever AH does with additional functionality (confidentiality). We actually do not need AH. However, the implementation of AH is already included in some commercial products, which means that AH will remain part of the Internet until these products are phased out.

13.4.3 Services Provided by IPSec

The two protocols, AH and ESP, can provide several security services for packets at the network layer. Table 13.1 shows the list of services available for each protocol.

Table 13.1 *IPSec services*

Services	*AH*	*ESP*
Access control	Yes	Yes
Message authentication (message integrity)	Yes	Yes
Entity authentication (data source authentication)	Yes	Yes
Confidentiality	No	Yes
Replay attack protection	Yes	Yes

Access Control

IPSec provides access control indirectly, using a Security Association Database (SAD), as we will see in Section 13.4.4. When a packet arrives at a destination and there is no Security Association already established for this packet, the packet is discarded.

Message Integrity

Message integrity is preserved in both AH and ESP. A digest of data is created and sent by the sender to be checked by the receiver.

Entity Authentication

The Security Association and the keyed-hash digest of the data sent by the sender authenticate the sender of the data in both AH and ESP.

Confidentiality

The encryption of the message in ESP provides confidentiality. AH, however, does not provide confidentiality. If confidentiality is needed, one should use ESP instead of AH.

Replay Attack Protection

In both protocols, the replay attack is prevented by using sequence numbers and a sliding receiver window. Each IPSec header contains a unique sequence number when the Security Association is established. The number starts from 0 and increases until the value reaches $2^{32} - 1$. When the sequence number reaches the maximum, it is reset to 0 and, at the same time, the old Security Association (see Section 13.4.4) is deleted and a new one is established. To prevent processing duplicate packets, IPSec mandates the use of a fixed-size window at the receiver. The size of the window is determined by the receiver with a default value of 64.

13.4.4 Security Association

Security Association is a very important aspect of IPSec. IPSec requires a logical relationship, called a **Security Association (SA)**, between two hosts. The Security

Association changes the connectionless service provided by IP to a connection-oriented service upon which we can apply security. This section first discusses the idea and then shows how it is used in IPSec.

Idea of Security Association

A Security Association is a contract between two parties; it creates a secure channel between them. Let us assume that Alice needs to unidirectionally communicate with Bob. If Alice and Bob are interested only in the confidentiality aspect of security, they can get a shared secret key between themselves. We can say that there are two SAs between Alice and Bob; one outbound SA and one inbound SA. Each of them stores the value of the key in one variable and the name of the encryption/decryption algorithm in another. Alice uses the algorithm and the key to encrypt a message to Bob; Bob uses the algorithm and the key when he needs to decrypt the message received from Alice. Figure 13.36 shows a simple SA.

Figure 13.36 *Simple SA*

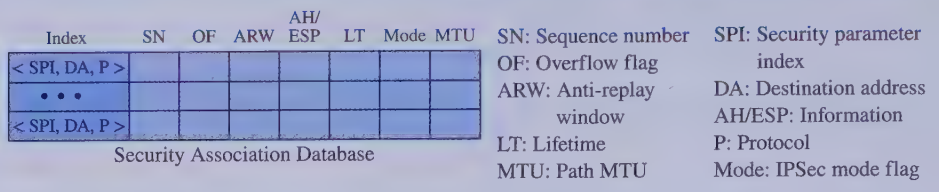
The Security Associations can be more involved if the two parties need message integrity and authentication. Each association needs other data such as the algorithm for message integrity, the key, and other parameters. It can be much more complex if the parties need to use specific algorithms and specific parameters for different protocols, such as IPSec AH or IPSec ESP.

Security Association Database (SAD)

A Security Association can be very complex. This is particularly true if Alice wants to send messages to many people and Bob needs to receive messages from many people. In addition, each site needs to have both inbound and outbound SAs to allow bidirectional communication. In other words, we need a set of SAs that can be collected into a database. This database is called the **Security Association Database (SAD)**. The database can be thought of as a two-dimensional table with each row defining a single SA. Normally, there are two SADs, one inbound and one outbound. Figure 13.37 shows the concept of outbound or inbound SADs for one entity.

When a host needs to send a packet that must carry an IPSec header, the host needs to find the corresponding entry in the outbound SAD to find the information for applying security to the packet. Similarly, when a host receives a packet that carries an IPSec

Figure 13.37 *SAD*



header, the host needs to find the corresponding entry in the inbound SAD to find the information for checking the security of the packet. This searching must be specific in the sense that the receiving host needs to be sure that correct information is used for processing the packet. Each entry in an inbound SAD is selected using a triple index: security parameter index (a 32-bit number that defines the SA at the destination), destination address, and protocol (AH or ESP).

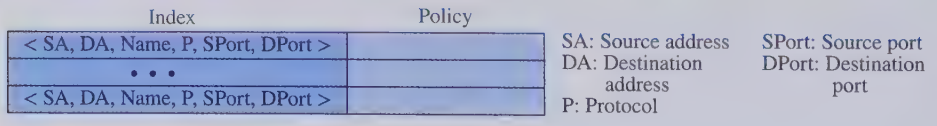
Security Policy

Another important aspect of IPSec is the **Security Policy (SP)**, which defines the type of security applied to a packet when it is to be sent or when it has arrived. Before using the SAD, a host must determine the predefined policy for the packet.

Security Policy Database

Each host that is using the IPSec protocol needs to keep a **Security Policy Database (SPD)**. Again, there is a need for an inbound SPD and an outbound SPD. Each entry in the SPD can be accessed using a sextuple index: source address, destination address, name, protocol, source port, and destination port, as shown in Figure 13.38. The name usually defines a DNS entity. The protocol is either AH or ESP.

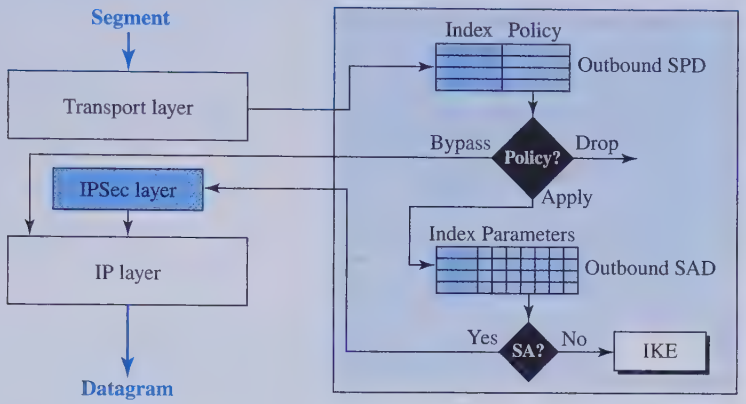
Figure 13.38 *SAD*



Outbound SPD

When a packet is to be sent out, the outbound SPD is consulted. Figure 13.39 shows the processing of a packet by a sender. The input to the outbound SPD is the sextuple index; the output is one of the three following cases: drop (packet cannot be sent), bypass (bypassing security header), and apply (applying the security according to the SAD; if no SAD, creating one).

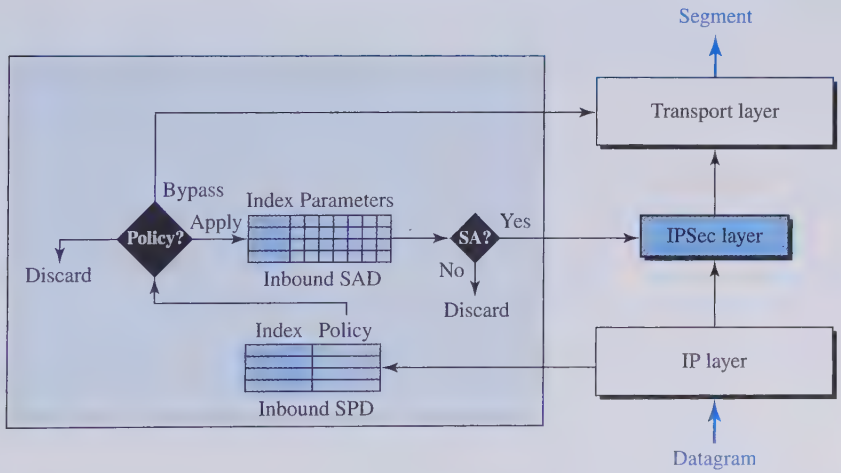
Figure 13.39 *Outbound processing*



Inbound SPD

When a packet arrives, the inbound SPD is consulted. Each entry in the inbound SPD is also accessed using the same sextuple index. Figure 13.40 shows the processing of a packet by a receiver.

Figure 13.40 *Inbound processing*



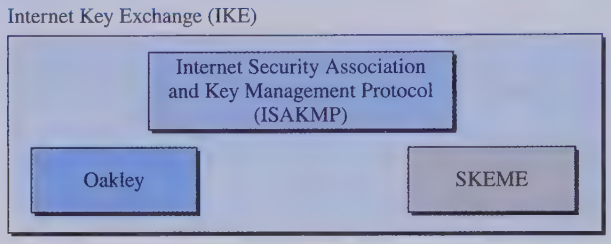
The input to the inbound SPD is the sextuple index; the output is one of the three following cases: discard (drop the packet), bypass (bypassing the security and delivering the packet to the transport layer), and apply (applying the policy using the SAD).

13.4.5 Internet Key Exchange (IKE)

The **Internet Key Exchange (IKE)** is a protocol designed to create both inbound and outbound Security Associations. As we discussed in Section 13.4.4, when a peer needs to send an IP packet, it consults the Security Policy Database (SPD) to see if there is an SA for that type of traffic. If there is no SA, IKE is called to establish one.

IKE is a complex protocol based on three other protocols: Oakley, SKEME, and ISAKMP, as shown in Figure 13.41.

Figure 13.41 *IKE components*



The **Oakley** protocol was developed by Hilarie Orman. It is a key creation protocol. **SKEME**, designed by Hugo Krawcyzk, is another protocol for key exchange. It uses public-key encryption for entity authentication in a key-exchange protocol.

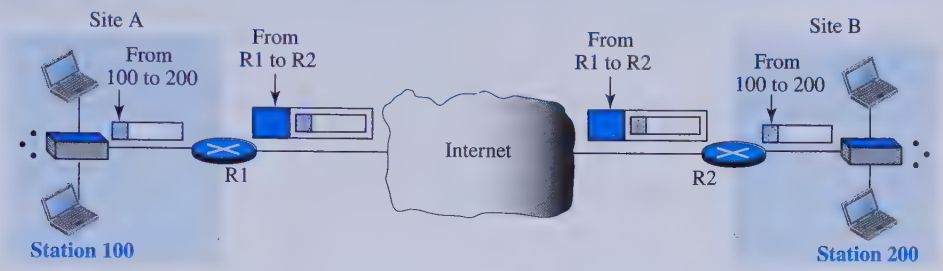
IKE creates SAs for IPSec.

The **Internet Security Association and Key Management Protocol (ISAKMP)** is a protocol designed by the National Security Agency (NSA) that actually implements the exchanges defined in IKE. It defines several packets, protocols, and parameters that allow the IKEs to take place in standardized, formatted messages to create SAs. We leave the discussion of these three protocols for books dedicated to security.

13.4.6 Virtual Private Network (VPN)

One of the applications of IPSec is in *virtual private networks*. A **virtual private network (VPN)** is a technology that is gaining popularity among large organizations that use the global Internet for both intra- and interorganization communication, but require privacy in their intraorganization communication. VPN is a network that is private but virtual. It is private because it guarantees privacy inside the organization. It is virtual because it does not use real private WANs; the network is physically public but virtually private. Figure 13.42 shows the idea of a virtual private network. Routers R1 and R2 use VPN technology to guarantee privacy for the organization. VPN technology uses the ESP protocol of IPSec in the tunnel mode. A private datagram, including the header, is encapsulated in an ESP packet. The router at the border of the sending site uses its own IP address and

Figure 13.42 *Virtual private network*



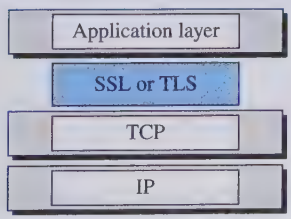
the address of the router at the destination site in the new datagram. The public network (Internet) is responsible for carrying the packet from R1 to R2. Outsiders cannot decipher the contents of the packet or the source and destination addresses. Deciphering takes place at R2, which finds the destination address of the packet and delivers it.

13.5 TRANSPORT-LAYER SECURITY

In fact, security at the transport layer provides security for the application layer that uses the services of TCP (or SCTP) as a connection-oriented protocol. Before the messages of these applications are encapsulated in TCP, they are encapsulated in the security-protocol packets. Those applications that use the services of UDP cannot benefit from these security services because the nature of security requires connection establishment between the two entities. Anther application that cannot benefit from the transport-layer security is electronic mail (e-mail). This application provides one-way connection between the sender and the receiver; we need a special security provision for this application, as discussed in Section 13.6.

Two protocols are dominant today for providing security at the transport layer: the **Secure Sockets Layer (SSL) Protocol** and the **Transport Layer Security (TLS) Protocol**. The latter is actually an IETF version of the former. We discuss SSL in this section; TLS is very similar. Figure 13.43 shows the position of SSL and TLS in the Internet model.

Figure 13.43 *Location of SSL and TLS in the Internet model*



One of the goals of these protocols is to provide server and client authentication, data confidentiality, and data integrity. Application-layer client/server programs, such as HTTP (see Chapter 10), that use the services of TCP can encapsulate their data in SSL packets (HTTPS). If the server and client are capable of running SSL (or TLS) programs, then the client can use the URL *https://...* instead of *http://...* to allow HTTP messages to be encapsulated in SSL (or TLS) packets. For example, credit card numbers can be safely transferred via the Internet for online shoppers.

13.5.1 SSL Architecture

SSL is designed to provide security and compression services to data generated from the application layer. Typically, SSL can receive data from any application-layer protocol, but usually the protocol is HTTP. The data received from the application are compressed (optional), signed, and encrypted. The data are then passed to a reliable transport-layer protocol such as TCP. We discuss SSL in this section.

Services

SSL provides several services on data received from the application layer.

- **Fragmentation.** First, SSL divides the data into blocks of 2^{14} bytes or less.
- **Compression.** Each fragment of data is compressed using one of the lossless compression methods negotiated between the client and server. This service is optional.
- **Message integrity.** To preserve the integrity of data, SSL uses a keyed-hash function to create a MAC.
- **Confidentiality.** To provide confidentiality, the original data and the MAC are encrypted using symmetric-key cryptography.
- **Framing.** A header is added to the encrypted payload. The payload is then passed to a reliable transport-layer protocol.

Key Exchange Algorithms

To exchange an authenticated and confidential message, the client and the server each need a set of cryptographic secrets. However, to create these secrets, one pre-master secret must be established between the two parties. SSL defines several key-exchange methods to establish this pre-master secret.

Encryption/Decryption Algorithms

The client and server also need to agree to a set of encryption and decryption algorithms.

Hash Algorithms

SSL uses hash algorithms to provide message integrity (message authentication). Several hash algorithms have been defined for this purpose.

Cipher Suite

The combination of key exchange, hash, and encryption algorithms defines a **cipher suite** for each SSL session.

Compression Algorithms

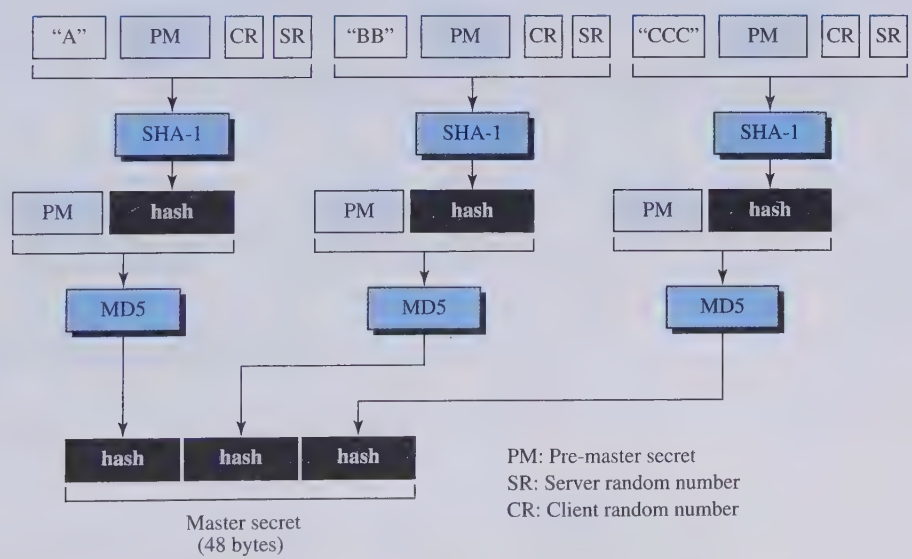
Compression is optional in SSL. No specific compression algorithm is defined. Therefore, a system can use whatever compression algorithm it desires.

Cryptographic Parameter Generation

To achieve message integrity and confidentiality, SSL needs six cryptographic secrets: four keys and two initialization vectors (IVs). The client needs one key for message authentication, one key for encryption, and one IV as the original block in calculation. The server needs the same. SSL requires that the keys for one direction be different from those for the other direction. If there is an attack in one direction, the other direction is not affected. The parameters are generated using the following procedure:

1. The client and server exchange two random numbers; one is created by the client and the other by the server.
2. The client and server exchange one *pre-master secret* using one of the predefined key-exchange algorithms.
3. A 48-byte *master secret* is created from the pre-master secret by applying two hash functions (SHA-1 and MD5), as shown in Figure 13.44.

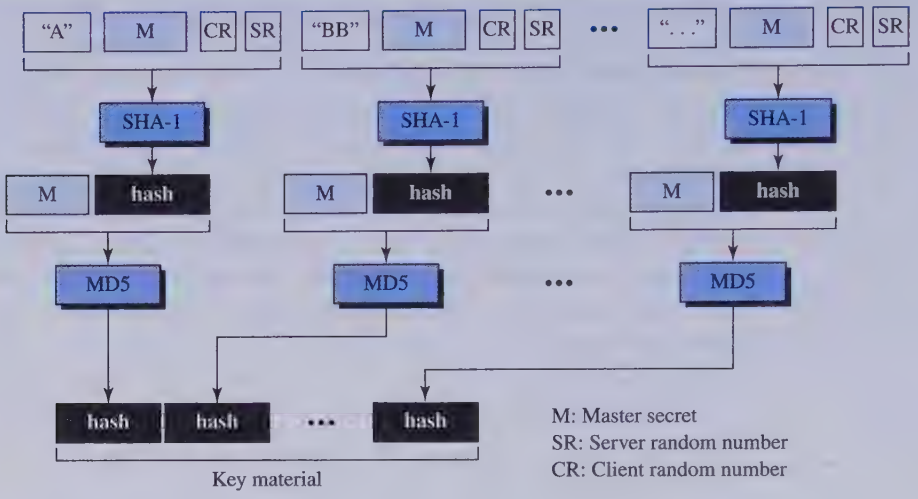
Figure 13.44 *Calculation of master secret from pre-master secret*



Note: "A", "BB", and "CC" are simple text added.

4. The master secret is used to create variable-length *key material* by applying the same set of hash functions and prepending with different constants, as shown in Figure 13.45. The module is repeated until key material of adequate size is created.

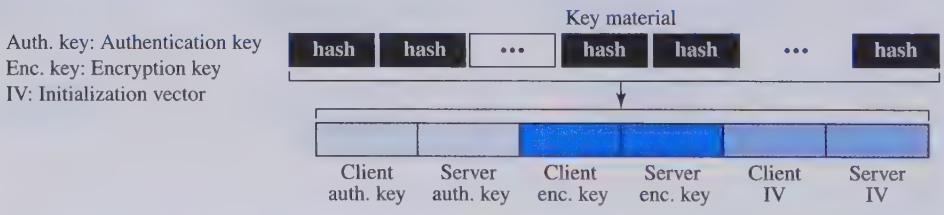
Figure 13.45 *Calculation of key material from master secret*



Note that the length of the key material block depends on the cipher suite selected and the size of keys needed for this suite.

5. Six different secrets are extracted from the key material, as shown in Figure 13.46.

Figure 13.46 *Extractions of cryptographic secrets from key material*



Sessions and Connections

SSL differentiates a *connection* from a *session.* A session is an association between a client and a server. After a session is established, the two parties have common information such as the session identifier, the certificate authenticating each of them (if necessary), the compression method (if needed), the cipher suite, and a master secret that is used to create keys for message authentication encryption.

For two entities to exchange data, the establishment of a session is necessary, but not sufficient; they need to create a connection between themselves. The two entities

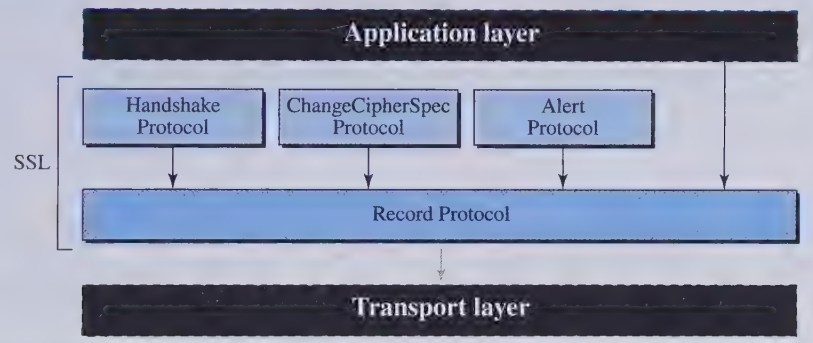
exchange two random numbers and create, using the master secret, the keys and parameters needed for exchanging messages involving authentication and privacy.

A session can consist of many connections. A connection between two parties can be terminated and reestablished within the same session. When a connection is terminated, the two parties can also terminate the session, but it is not mandatory. A session can be suspended and resumed.

13.5.2 Four Protocols

We have discussed the idea of SSL without showing how SSL accomplishes its tasks. SSL defines four protocols in two layers, as shown in Figure 13.47.

Figure 13.47 *Four SSL protocols*



The Record Protocol is the carrier. It carries messages from three other protocols as well as the data coming from the application layer. Messages from the Record Protocol are payloads to the transport layer, normally TCP. The Handshake Protocol provides security parameters for the Record Protocol. It establishes a cipher set and provides keys and security parameters. It also authenticates the server to the client and the client to the server if needed. The ChangeCipherSpec Protocol is used for signaling the readiness of cryptographic secrets. The Alert Protocol is used to report abnormal conditions. We will briefly discuss these protocols in this section.

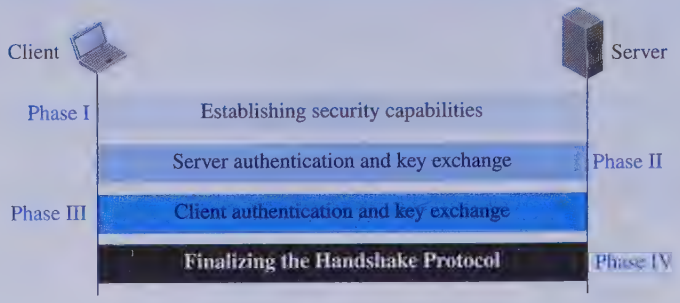
Handshake Protocol

The **Handshake Protocol** uses messages to negotiate the cipher suite, to authenticate the server to the client and the client to the server if needed, and to exchange information for building the cryptographic secrets. The handshaking is done in four phases, as shown in Figure 13.48.

Phase I: Establishing Security Capabilities

In Phase I, the client and the server announce their security capabilities and choose those that are convenient for both. In this phase, a session ID is established and the

Figure 13.48 *Handshake Protocol*



cipher suite is chosen. The parties agree upon a particular compression method. Finally, two random numbers are selected, one by the client and one by the server, to be used for creating a master secret as we saw before. After Phase I, the client and server know the version of SSL, the cryptographic algorithms, the compression method, and the two random numbers for key generation.

Phase II: Server Authentication and Key Exchange

In Phase II, the server authenticates itself if needed. The sender may send its certificate and its public key and may also request certificates from the client. After Phase II, the server is authenticated to the client, and the client knows the public key of the server if required.

Phase III: Client Authentication and Key Exchange

Phase III is designed to authenticate the client. After Phase III, the client is authenticated for the server, and both the client and the server know the pre-master secret.

Phase IV: Finalizing and Finishing

In Phase IV, the client and server send messages to change cipher specifications and to finish the Handshake Protocol.

ChangeCipherSpec Protocol

We have seen that the negotiation of the cipher suite and the generation of cryptographic secrets are formed gradually during the Handshake Protocol. The question now is, "When can the two parties use these parameters or secrets?" SSL mandates that the parties cannot use these parameters or secrets until they have sent or received a special message, the ChangeCipherSpec message, which is exchanged during the Handshake Protocol and defined in the *ChangeCipherSpec Protocol*. The reason is that the issue is not just sending or receiving a message. The sender and the receiver need two states, not one. One state, the pending state, keeps track of the parameters and secrets. The other state, the active state, holds parameters and secrets used by the Record Protocol to sign/verify or encrypt/decrypt messages. In addition, each state holds two sets of values: *read* (inbound) and *write* (outbound).

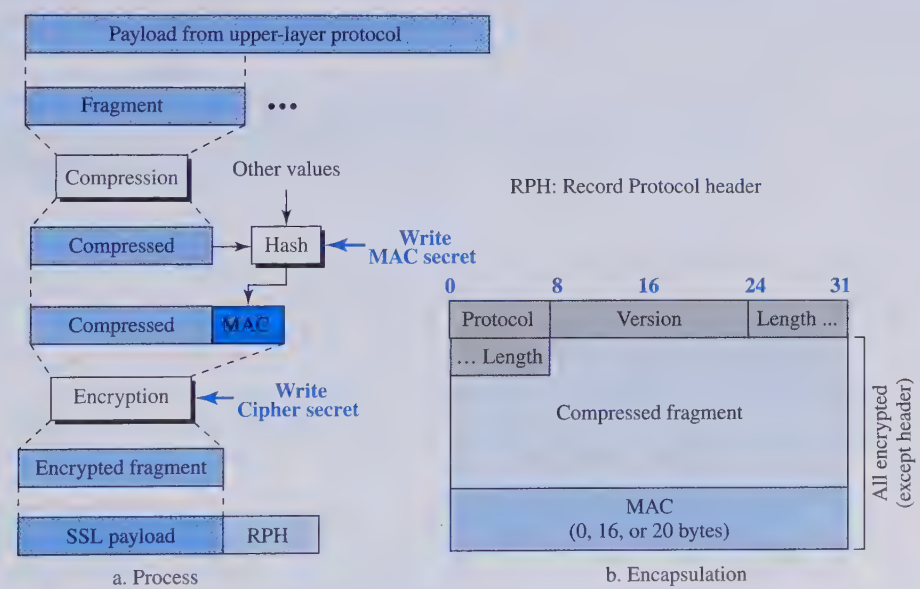
Alert Protocol

SSL uses the *Alert Protocol* for reporting errors and abnormal conditions. It uses only one message that describes the problem and its level (warning or fatal).

Record Protocol

The *Record Protocol* carries messages from the upper layer (Handshake Protocol, ChangeCipherSpec Protocol, Alert Protocol, or application layer). The message is fragmented and optionally compressed; a MAC is added to the compressed message using the negotiated hash algorithm. The compressed fragment and the MAC are encrypted using the negotiated encryption algorithm. Finally, the SSL header is added to the encrypted message. Figure 13.49 shows this process at the sender. The process at the receiver is reversed.

Figure 13.49 *Processing done by the Record Protocol*



13.6 APPLICATION-LAYER SECURITY

This section discusses two protocols providing security services for e-mails: Pretty Good Privacy (PGP) and Secure/Multipurpose Internet Mail Extension (S/MIME).

13.6.1 E-mail Security

Sending an e-mail is a one-time activity. The nature of this activity is different from those we saw in SSL or IPSec. In those protocols, we assume that the two parties create

a session between themselves and exchange data in both directions. In e-mail, there is no session. Alice and Bob cannot create a session. Alice sends a message to Bob; sometime later, Bob reads the message and may or may not send a reply. We discuss the security of a unidirectional message because what Alice sends to Bob is totally independent from what Bob sends to Alice.

Cryptographic Algorithms

If e-mail is a one-time activity, how can the sender and receiver agree on a cryptographic algorithm to use for e-mail security? If there is no session and no handshaking to negotiate the algorithms for encryption/decryption and hashing, how can the receiver know which algorithm the sender has chosen for each purpose?

To solve the problem, the protocol defines a set of algorithms for each operation that the user used in his/her system. Alice includes the names (or identifiers) of the algorithms she has used in the e-mail. For example, Alice can choose DES for encryption/decryption and MD5 for hashing. When Alice sends a message to Bob, she includes the corresponding identifiers for DES and MD5 in her message. Bob receives the message and extracts the identifiers first. He then knows which algorithm to use for decryption and which one for hashing.

In e-mail security, the sender of the message needs to include the names or identifiers of the algorithms used in the message.

Cryptographic Secrets

The same problem for the cryptographic algorithms applies to the cryptographic secrets (keys). If there is no negotiation, how can the two parties establish secrets between themselves? The e-mail security protocols today require that encryption/decryption be done using a symmetric-key algorithm and a one-time secret key sent with the message. Alice can create a secret key and send it with the message she sends to Bob. To protect the secret key from interception by Eve, the secret key is encrypted with Bob's public key. In other words, the secret key itself is encrypted.

In e-mail security, the encryption/decryption is done using a symmetric-key algorithm, but the secret key to decrypt the message is encrypted with the public key of the receiver and is sent with the message.

Certificates

One more issue needs to be considered before we discuss any e-mail security protocol in particular. It is obvious that some public-key algorithms must be used for e-mail security. For example, we need to encrypt the secret key or sign the message. To encrypt the secret key, Alice needs Bob's public key; to verify a signed message, Bob needs Alice's public key. So, for sending a small authenticated and confidential message, two public keys are needed. How can Alice be assured of Bob's public key, and how can Bob be assured of Alice's public key? Each e-mail security protocol has a different method of certifying keys.

13.6.2 Pretty Good Privacy (PGP)

The first protocol discussed in this section is called **Pretty Good Privacy (PGP)**. PGP was invented by Phil Zimmermann to provide e-mail with privacy, integrity, and authentication. PGP can be used to create secure e-mail messages.

Scenarios

Let us first discuss the general idea of PGP, moving from a simple scenario to a complex one. We use the term "Data" to show the message prior to processing.

Plaintext

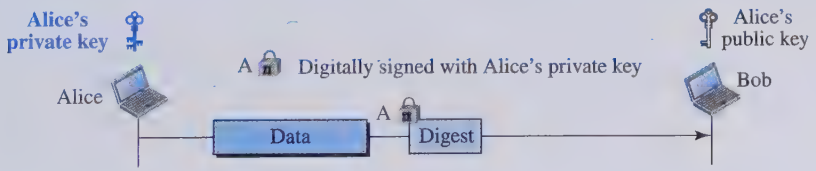
The simplest scenario is to send the e-mail message in plaintext as shown in Figure 13.50. There is no message integrity or confidentiality in this scenario.

Figure 13.50 *A plaintext message*

Message Integrity

Probably the next improvement is to let Alice sign the message. Alice creates a digest of the message and signs it with her private key. Figure 13.51 shows the situation.

Figure 13.51 *An authenticated message*

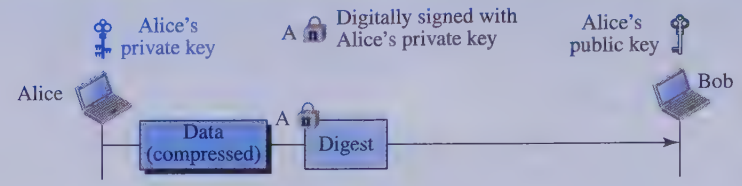


When Bob receives the message, he verifies the message by using Alice's public key. Two keys are needed for this scenario. Alice needs to know her private key; Bob needs to know Alice's public key.

Compression

A further improvement is to compress the message to make the packet more compact. This improvement has no security benefit, but it eases the traffic. Figure 13.52 shows the new scenario.

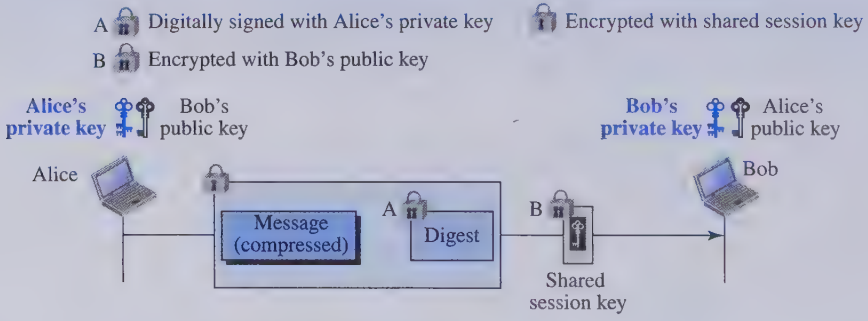
Figure 13.52 *A compressed message*



Confidentiality with One-Time Session Key

Figure 13.53 shows the situation. As we discussed before, confidentiality in an e-mail system can be achieved using conventional encryption with a one-time session key. Alice can create a session key, use the session key to encrypt the message and the digest, and send the key itself with the message. However, to protect the session key, Alice encrypts it with Bob's public key.

Figure 13.53 *A confidential message*



When Bob receives the packet, he first decrypts the session key, using his private key. He then uses the session key to decrypt the rest of the message. After decompressing the rest of the message, Bob creates a digest of the message and checks to see if it is equal to the digest sent by Alice. If it is, then the message is authentic.

Code Conversion

Another service provided by PGP is code conversion. Most e-mail systems allow the message to consist of only ASCII characters. To translate other characters not in the ASCII set, PGP uses Base-64 conversion.

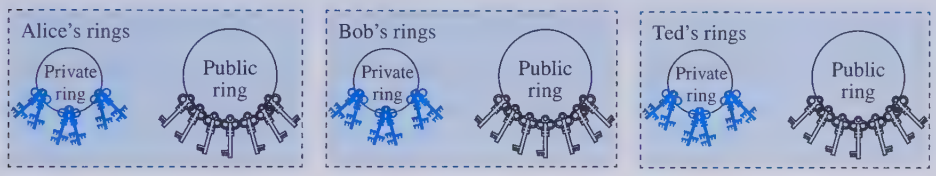
Segmentation

PGP allows segmentation of the message after it has been converted to Radix-64 to make each transmitted unit the uniform size allowed by the underlying e-mail protocol.

Key Rings

In all previous scenarios, we assumed that Alice needs to send a message only to Bob. That is not always the case. Alice may need to send messages to many people; she needs *key rings*. In this case, Alice needs a ring of public keys, with a key belonging to each person with whom Alice needs to correspond (send or receive messages). In addition, the PGP designers specified a ring of private/public keys. One reason is that Alice may wish to change her pair of keys from time to time. Another reason is that Alice may need to correspond with different groups of people (friends, colleagues, and so on). Alice may wish to use a different key pair for each group. Therefore, each user needs to have two sets of rings: a ring of private keys and a ring of public keys of other people. Figure 13.54 shows a community of three people, each having a ring of pairs of private/public keys and, at the same time, a ring of public keys belonging to other people in the community.

Figure 13.54 *Key rings in PGP*



Alice, for example, has several pairs of private/public keys belonging to her and public keys belonging to other people. Note that everyone can have more than one public key. Two cases may arise.

1. Alice needs to send a message to another person in the community.
   a. She uses her private key to sign the digest.
   b. She uses the receiver's public key to encrypt a newly created session key.
   c. She encrypts the message and signed digest with the session key created.
2. Alice receives a message from another person in the community.
   a. She uses her private key to decrypt the session key.
   b. She uses the session key to decrypt the message and digest.
   c. She uses her public key to verify the digest.

PGP Algorithms

PGP defines a set of asymmetric-key and symmetric-key algorithms, cryptography hash functions, and compression methods. We leave the details of these algorithms to the books devoted to PGP. When Alice sends an e-mail to Bob, she defines the algorithm she has used for each purpose.

PGP Certificates and Trusted Model

PGP, like other protocols we have seen so far, uses certificates to authenticate public keys. However, the process is totally different, as we will now explain.

PGP Certificates

In PGP, there is no need for a certification authority (CA); anyone in the ring can sign a certificate for anyone else in the ring. Bob can sign a certificate for Ted, John, Anne, and so on. There is no hierarchy of trust in PGP; there is no tree. The lack of hierarchical structure may result in the fact that Ted may have one certificate from Bob and another certificate from Liz. If Alice wants to follow the line of certificates for Ted, there are two paths: One starts from Bob, and one starts from Liz. An interesting point is that Alice may fully trust Bob, but only partially trust Liz. There can be multiple paths in the line of trust from a fully or partially trusted authority to a certificate. In PGP, the issuer of a certificate is usually called an *introducer*.

In PGP, there can be multiple paths from fully or partially trusted authorities to any subject.

- **Trust and legitimacy.** The entire operation of PGP is based on trust in the introducer, trust in the certificate, and acceptance of the legitimacy of the public keys.
- **Introducer trust levels.** With the lack of a central authority, it is obvious that the ring cannot be very large if every user has to fully trust everyone else. (Even in real life we cannot fully trust everyone that we know.) To solve this problem, PGP allows different levels of trust. The number of levels is mostly implementation dependent, but for simplicity, let us assign three levels of trust to any introducer: *none*, *partial*, and *full*. The introducer trust level specifies the trust levels issued by the introducer for other people in the ring. For example, Alice may fully trust Bob, partially trust Anne, and not trust John at all. There is no mechanism in PGP to determine how to make a decision about the trustworthiness of the introducer; it is up to the user to make this decision.
- **Certificate trust levels.** When Alice receives a certificate from an introducer, she stores the certificate under the name of the subject (certified entity). She assigns a level of trust to this certificate. The certificate trust level is normally the same as the trust level for the introducer that issued the certificate. Assume that Alice fully trusts Bob, partially trusts Anne and Janette, and has no trust in John. The following scenarios can happen.
  1. Bob issues two certificates, one for Linda (with public key K1) and one for Lesley (with public key K2). Alice stores the public key and certificate for Linda under Linda's name and assigns a *full* level of trust to this certificate. Alice also stores the certificate and public key for Lesley under Lesley's name and assigns a full level of trust to this certificate.
  2. Anne issues a certificate for John (with public key K3). Alice stores this certificate and public key under John's name, but assigns a *partial* level for this certificate.
  3. Janette issues two certificates, one for John (with public key K3) and one for Lee (with public key K4). Alice stores John's certificate under his name and Lee's certificate under his name, each with a *partial* level of trust. Note that John now has two certificates, one from Anne and one from Janette, each with a *partial* level of trust.

4. John issues a certificate for Liz. Alice can discard or keep this certificate with a signature trust of *none*.

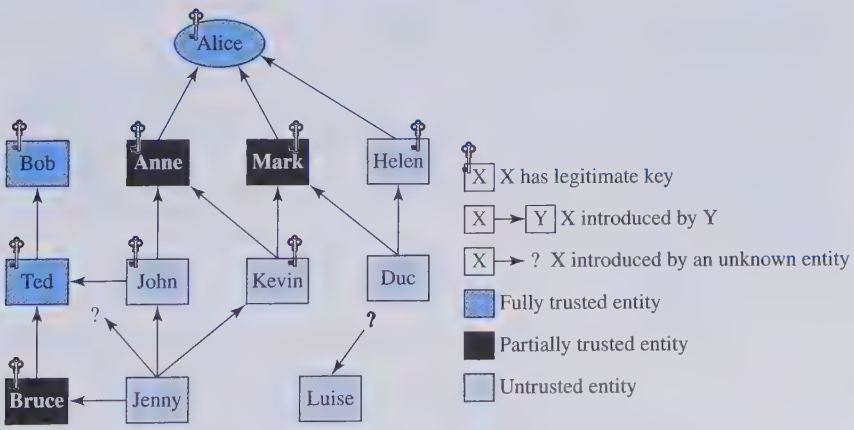
- **Key legitimacy.** The purpose of using trust in the introducer and certificate is to determine the legitimacy of a public key. Alice needs to know how legitimate the public keys of Bob, John, Liz, Anne, and so on are. PGP defines a very clear procedure for determining key legitimacy. The level of the key legitimacy for a user is the weighted trust levels of that user. For example, suppose we assign the following weights to certificate trust levels:
  1. 0 to a nontrusted certificate
  2. 1/2 to a certificate with partial trust
  3. 1 to a certificate with full trust

Then to fully trust an entity, Alice needs one fully trusted certificate or two partially trusted certificates for that entity. For example, Alice can use John's public key in the previous scenario because both Anne and Janette have issued a certificate for John, each with a certificate trust level of 1/2. Note that the legitimacy of a public key belonging to an entity does not have anything to do with the trust level for that person. Although Bob can use John's public key to send a message to him, Alice cannot accept any certificate issued by John because, for Alice, John has a trust level of *none*.

Trust Model in PGP

As Zimmermann has proposed, we can create a trust model for any user in a ring with the user as the center of activity. Such a model can look like the one shown in Figure 13.55. The figure shows the trust model for Alice at some moment.

Figure 13.55 *Trust model*



Let us elaborate on the figure. Figure 13.55 shows that there are three entities in Alice's ring with full trust (Alice herself, Bob, and Ted). The figure also shows three entities with partial trust (Anne, Mark, and Bruce). There are also six entities with no

trust. Nine entities have a legitimate key. Alice can encrypt a message to any one of these entities or verify a signature received from one of these entities (Alice's key is never used in this model). There are also three entities that do not have any legitimate keys with Alice.

Bob, Anne, and Mark have made their keys legitimate by sending their keys by e-mail and verifying their fingerprints by phone. Helen, on the other hand, has sent a certificate from a CA because she is not trusted by Alice and verification on the phone is not possible. Although Ted is fully trusted, he has given Alice a certificate signed by Bob. John has sent Alice two certificates, one signed by Ted and one by Anne. Kevin has sent two certificates to Alice, one signed by Anne and one by Mark. Each of these certificates gives Kevin half a point of legitimacy; therefore, Kevin's key is legitimate. Duc has sent two certificates to Alice, one signed by Mark and the other by Helen. Because Mark is half-trusted and Helen is not trusted, Duc does not have a legitimate key. Jenny has sent four certificates, one signed by a half-trusted entity, two by untrusted entities, and one by an unknown entity. Jenny does not have enough points to make her key legitimate. Luise has sent one certificate signed by an unknown entity. Note that Alice may keep Luise's name in the table in case future certificates for Luise arrive.

- **Web of trust.** PGP can eventually make a **web of trust** among a group of people. If each entity introduces more entities to other entities, the public key ring for each entity gets larger and larger and entities in the ring can send secure e-mail to each other.
- **Key revocation.** It may become necessary for an entity to revoke his or her public key from the ring. This may happen if the owner of the key feels that the key is compromised (stolen, for example) or just too old to be safe. To revoke a key, the owner can send a revocation certificate signed by herself. The revocation certificate must be signed by the old key and disseminated to all the people in the ring that use that public key.

PGP Packets

A message in PGP consists of one or more packets. During the evolution of PGP, the format and the number of packet types have changed. We do not discuss the formats of these packets here.

Applications of PGP

PGP has been extensively used for personal e-mails. It will probably continue to be.

13.6.3 S/MIME

Another security service designed for electronic mail is **Secure/Multipurpose Internet Mail Extension (S/MIME)**. The protocol is an enhancement of the Multipurpose Internet Mail Extension (MIME) protocol.

Cryptographic Message Syntax (CMS)

To define how security services, such as confidentiality or integrity, can be added to MIME content types, S/MIME has defined **Cryptographic Message Syntax (CMS)**. The syntax in each case defines the exact encoding scheme for each content type. The following describe the types of messages and different subtypes that are created from these messages. For details, the reader is referred to RFCs 3369 and 3370.

Data Content Type

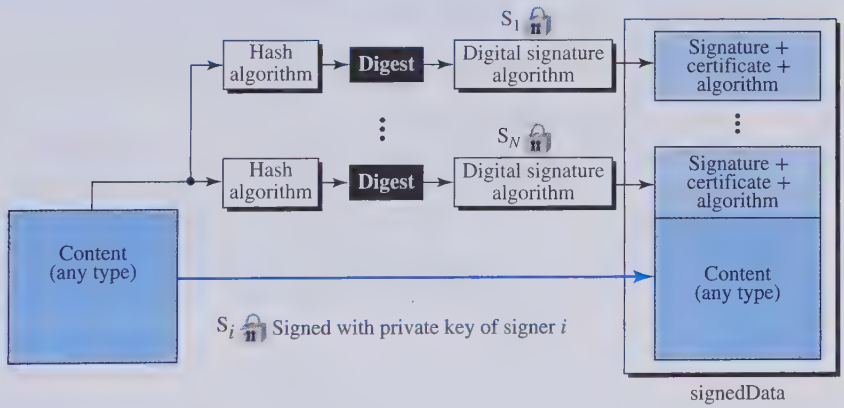
This is an arbitrary string. The object created is called *Data*.

Signed-Data Content Type

This type provides only integrity of data. It contains any data type plus zero or more signature values. The encoded result is an *object* called *signedData*. Figure 13.56 shows the process of creating an object of this type. The following are the steps in the process:

1. For each signer, a message digest is created from the content using the specific hash algorithm chosen by that signer.
2. Each message digest is signed with the private key of the signer.
3. The content, signature values, certificates, and algorithms are then collected to create the *signedData* object.

Figure 13.56 *Signed-data content type*

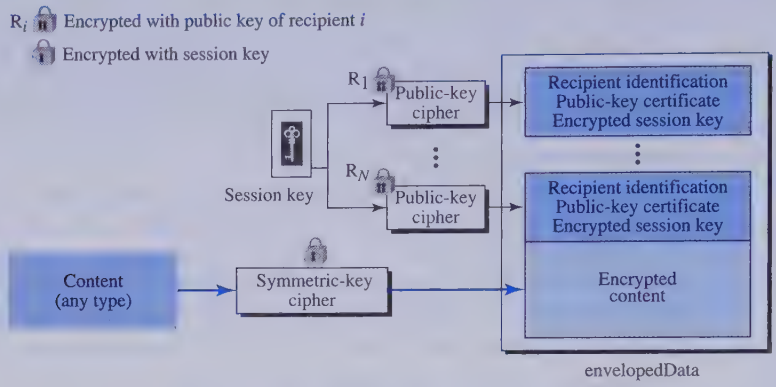


Note that, in this case, the content is not necessarily a personal message. It can be a document whose integrity needs to be preserved. The sender can collect the signatures and then send (or store) them with the message.

Enveloped-Data Content Type

This type is used to provide privacy for the message. It contains any message type plus zero or more encrypted keys and certificates. The encoded result is an *object* called *envelopedData*. Figure 13.57 shows the process of creating an object of this type.

Figure 13.57 *Enveloped-data content type*



1. A pseudorandom session key is created for the symmetric-key algorithms to be used.
2. For each recipient, a copy of the session key is encrypted with the public key of that recipient.
3. The content is encrypted using the defined algorithm and created session key.
4. The encrypted contents, encrypted session keys, algorithm used, and certificates are encoded using Radix-64.

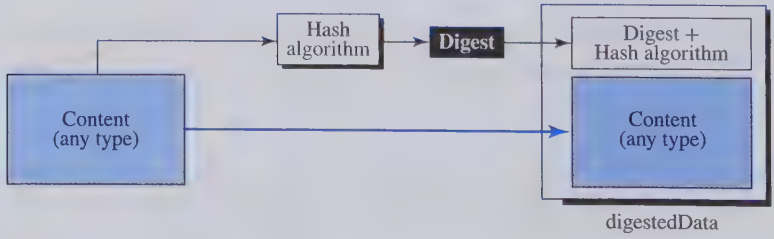
Note that, in this case, we can have one or more recipients.

Digested-Data Content Type

This type is used to provide integrity for the message. The result is normally used as the content for the enveloped-data content type. The encoded result is an *object* called *digestedData*. Figure 13.58 shows the process of creating an object of this type.

1. A message digest is calculated from the content.
2. The message digest, the algorithm, and the content are added together to create the *digestedData* object.

Figure 13.58 *Digested-data content type*



Encrypted-Data Content Type

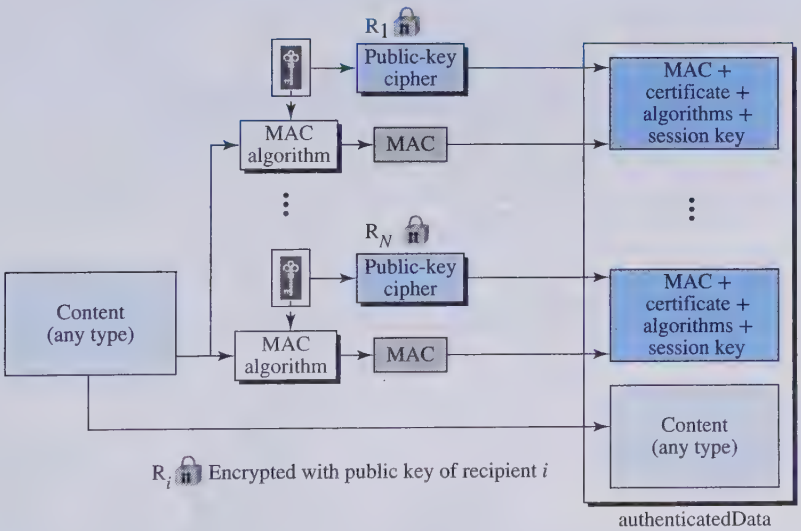
This type is used to create an encrypted version of any content type. Although this looks like the enveloped-data content type, the encrypted-data content type has no recipient. It can be used to store the encrypted data instead of transmitting it. The process is very simple; the user employs any key (normally derived from the password) and any algorithm to encrypt the content. The encrypted content is stored without including the key or the algorithm. The object created is called *encryptedData*.

Authenticated-Data Content Type

This type is used to provide authentication of the data. The object is called *authenticatedData*. Figure 13.59 shows the process.

1. Using a pseudorandom generator, a MAC key is generated for each recipient.
2. The MAC key is encrypted with the public key of the recipient.
3. A MAC is created for the content.
4. The content, MAC, algorithms, and other information are collected to form the authenticatedData object.

Figure 13.59 *Authenticated-data content type*



Key Management

The key management in S/MIME is a combination of key management used by X.509 and PGP. S/MIME uses public-key certificates signed by the certificate authorities defined by X.509. However, the user is responsible for maintaining the web of trust to verify signatures as defined by PGP.

Cryptographic Algorithms

S/MIME defines several cryptographic algorithms. We leave the details of these algorithms to the books dedicated to security in the Internet.

Example 13.10

The following shows an example of an enveloped-data in which a small message is encrypted using triple DES.

```
Content-Type: application/pkcs7-mime; mime-type=enveloped-data
Content-Transfer-Encoding: Radix-64
Content-Description: attachment
name="report.txt";
cb32ut67f4bhijHU21oi87eryb0287hmnklsgFDoY8bc659GhIGfH6543mhjkdsaH23YjBnmN
ybmlkzjhgfdyhGe23Kjk34XiuD678Es16se09jy76jHuytTMDcbnmlkjgfFdiuyu678543m0n3hG
34un12P2454Hoi87e2ryb0H2MjN6KuyrlsgFDoY897fk923jljk1301XiuD6gh78EsUyT23y
```

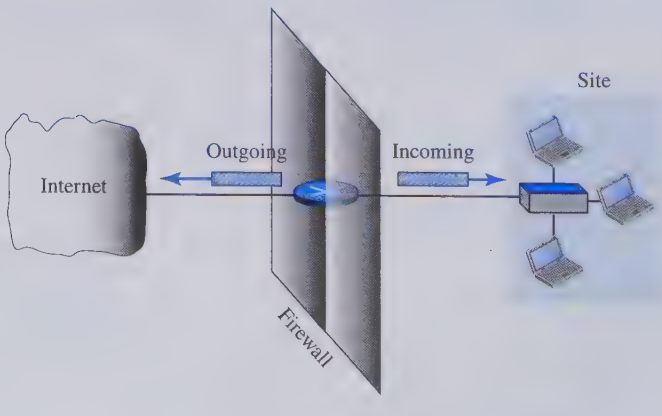
Applications of S/MIME

It is predicted that S/MIME will become the industry choice to provide security for commercial e-mail.

13.7 FIREWALLS

All previous security measures cannot prevent Eve from sending a harmful message to a system. To control access to a system we need firewalls. A **firewall** is a device (usually a router or a computer) installed between the internal network of an organization and the rest of the Internet. It is designed to forward some packets and filter (not forward) others. Figure 13.60 shows a firewall.

Figure 13.60 *Firewall*

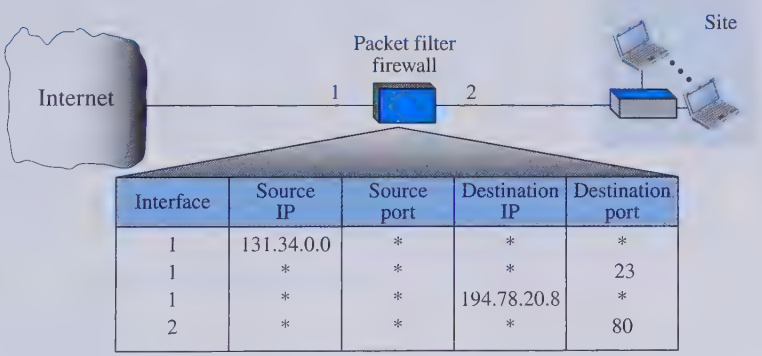


For example, a firewall may filter all incoming packets destined for a specific host or a specific server such as HTTP. A firewall can be used to deny access to a specific host or a specific service in the organization. A firewall is usually classified as a *packet-filter firewall* or a *proxy-based firewall*.

13.7.1 Packet-Filter Firewall

A firewall can be used as a packet filter. It can forward or block packets based on the information in the network-layer and transport-layer headers: source and destination IP addresses, source and destination port addresses, and type of protocol (TCP or UDP). A **packet-filter firewall** is a router that uses a filtering table to decide which packets must be discarded (not forwarded). Figure 13.61 shows an example of a filtering table for this kind of a firewall.

Figure 13.61 *Packet-filter firewall*



Interface	Source IP	Source port	Destination IP	Destination port
1	131.34.0.0	*	*	*
1	*	*	*	23
1	*	*	194.78.20.8	*
2	*	*	*	80

According to Figure 13.61, the following packets are filtered:

1. Incoming packets from network 131.34.0.0 are blocked (security precaution). Note that the * (asterisk) means "any."
2. Incoming packets destined for any internal TELNET server (port 23) are blocked.
3. Incoming packets destined for internal host 194.78.20.8 are blocked. The organization wants this host for internal use only.
4. Outgoing packets destined for an HTTP server (port 80) are blocked. The organization does not want employees to browse the Internet.

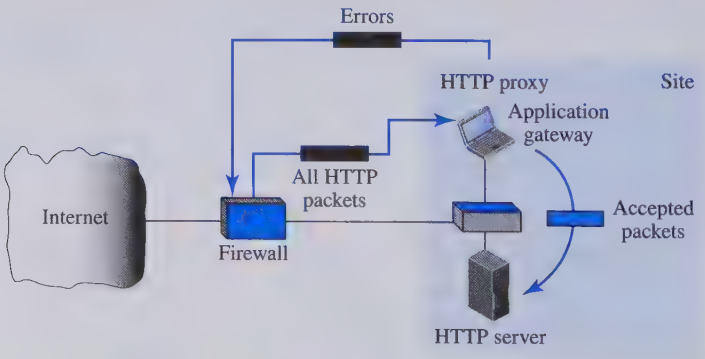
A packet-filter firewall filters at the network or transport layer.

13.7.2 Proxy Firewall

The packet-filter firewall is based on the information available in the network layer and transport layer headers (IP and TCP/UDP). However, sometimes we need to filter a message based on the information available in the message itself (at the application layer). We can do this using a **proxy firewall**. As an example, assume that an organization wants to implement the following policies regarding its web pages: Only those Internet users who have previously established business relations with the company can have access; access to other users must be blocked. In this case, a packet-filter firewall is not feasible because it cannot distinguish between different packets arriving at TCP port 80 (HTTP). Testing must be done at the application level (using URLs).

One solution is to install a proxy computer (sometimes called an **application gateway**), which stands between the customer computer and the corporation computer. When the user client process sends a message, the application gateway runs a server process to receive the request. The server opens the packet at the application level and finds out if the request is legitimate. If it is, the server acts as a client process and sends the message to the real server in the corporation. If it is not, the message is dropped and an error message is sent to the external user. In this way, the requests of the external users are filtered based on the contents at the application layer. Figure 13.62 shows an application gateway implementation for HTTP.

Figure 13.62 *Proxy firewall*



A proxy firewall filters at the application layer.

13.8 END-OF-CHAPTER MATERIALS

13.8.1 Recommended Reading

Books

Several books give thorough coverage of cryptography and network security: [For 08], [Sta 06], [Bis 05], [Mao 04], [Sti 06], [Res 01], [Tho 00], [DH 03], and [Gar 95].

13.8.2 Key Terms

additive cipher
application gateway
asymmetric-key cipher
Authentication Header (AH) Protocol
autokey cipher
block cipher
Caesar cipher
certification authority (CA)
challenge-response authentication
cipher
cipher suite
ciphertext
cryptographic hash function
Cryptographic Message Syntax (CMS)
cryptography
Data Encryption Standard (DES)
decryption
decryption algorithm
denial of service (DoS)
Diffie-Hellman protocol
digest
digital signature
Digital Signature Standard (DSS)
Encapsulating Security Payload (ESP)
encryption
encryption algorithm
firewall
Handshake Protocol
hashed MAC (HMAC)
Internet Key Exchange (IKE)
Internet Security Association and Key Management Protocol (ISAKMP)
IP Security (IPSec)
Key distribution center (KDC)
message authentication code (MAC)
Message Digest (MD)
monoalphabetic cipher
Oakley
one-time pad
P-box
packet-filter firewall
plaintext
polyalphabetic cipher
Pretty Good Privacy (PGP)
private key
proxy firewall
public key
public-key certificate
RSA cryptosystem
S-box
Secure Hash Algorithm (SHA)
Secure Sockets Layer (SSL) Protocol
Secure/Multipurpose Internet Mail Extension (S/MIME)
Security Association (SA)
Security Association Database (SAD)
Security Policy (SP)
Security Policy Database (SPD)
shift cipher
SKEME
steganography
stream cipher
substitution cipher
symmetric-key cipher
ticket
Transport Layer Security (TLS) Protocol
transport mode
transposition cipher
tunnel mode
virtual private network (VPN)
web of trust
X.509

13.8.3 Summary

The three goals of security are confidentiality, integrity, and availability. These goals are threatens by attacks such as snooping, traffic analysis, modification, masquerading, replaying, repudiation, and denial of service. Cryptography is a technique to achieve security goals and is described in this chapter; the other technique, steganography is left for more advanced books on security.

Confidentiality is achieved through asymmetric-key and symmetric-key ciphers. In a symmetric-key cipher, the same key is used for encryption and decryption, and the key can be used for bidirectional communication. We can divide traditional symmetric-key ciphers into two broad categories: substitution ciphers and transposition ciphers. In an asymmetric key cryptography, there are two separate keys: one private and one public. Asymmetric-key cryptography, means that Bob and Alice cannot use the same set of keys for two-way communication.

Discussion of security is not limited to confidentiality; other aspects of security include integrity, message authentication, entity authentication, and key management. Message integrity is achieved using a hashing function to create a digest of the message. Message authentication is achieved using techniques such as message authentication code (MAC) or digital signature. Entity authentication is achieved using either personal identifications, such as password, or techniques such as the challenge-response process. To provide either confidentiality or other aspects of security, we need either secret keys or the combination of private-public keys. The distribution of secret keys can be done by a key distribution center (KDC) or through instantaneous methods such as Diffie-Hellman. The certification of public keys can be done through a certification authority (CA).

IP Security (IPSec) is a collection of protocols designed by the IETF to provide security for a packet at the network level. IPSec operates in transport or tunnel mode. IPSec defines two protocols: Authentication Header (AH) Protocol and Encapsulating Security Payload (ESP) Protocol. IPSec creates a connection-oriented association at the top of the connectionless IP protocol to be able to provide security.

A transport-layer security protocol provides end-to-end security services for applications that use the services of a connection-oriented transport-layer protocol such as TCP. Two protocols are dominant today for providing security at the transport layer: Secure Sockets Layer (SSL) and Transport Layer Security (TLS). We discussed SSL in this chapter; TLS is similar.

Although SSL or TLS can provide security for applications that use the service of connection-oriented protocols such as TCP, the e-mail application is exceptional because the application uses a one-way communication. The Pretty Good Privacy (PGP), invented by Phil Zimmermann, provides e-mail with privacy, integrity, and authentication. Another security service designed for electronic mail is Secure/Multipurpose Internet Mail Extension (S/MIME).

A firewall is a device (usually a router or a computer) installed between the internal network of an organization and the rest of the Internet. It is designed to forward some packets and filter others. A firewall is usually classified as a packet-filter firewall or a proxy firewall.

13.9 PRACTICE SET

13.9.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check their understanding of the materials before continuing with the practice set.

13.9.2 Questions

Q13-1. Which of the following attacks is a threat to confidentiality?
a. snooping **b.** masquerading **c.** repudiationr

Q13-2. Which of the following attacks is a threat to integrity?
a. modification **b.** replaying **c.** denial of service

Q13-3. Which of the following attacks is a threat to availability?
a. repudiation **b.** denial of service **c.** modification

Q13-4. Which of the following words means "secret writing"? Which one means covered writing?
a. cryptography **b.** steganography

Q13-5. When a sealed letter is sent from Alice to Bob, is this an example of using cryptography or steganography for confidentiality?

Q13-6. When a letter is sent from Bob to Alice in a language that only the two can understand, is this an example of cryptography or steganography?

Q13-7. Alice has found a way to write secretly to Bob. Each time, she takes a new text, such as an article from the newspaper, but inserts one or two spaces between the words. A single space means a binary digit 0; a double space means a binary digit 1. Bob extracts the binary digits and interprets them using ASCII code. Is this an example of cryptography or steganography? Explain.

Q13-8. Alice and Bob exchange confidential messages. They share a very large number as the encryption and decryption key in both directions. Is this an example of symmetric-key or asymmetric-key cryptography? Explain.

Q13-9. Alice uses the same key when she encrypts a message to be sent to Bob and when she decrypts a message received from Bob. Is this an example of symmetric-key or asymmetric-key cryptography? Explain.

Q13-10. Distinguish between a substitution cipher and a transposition cipher.

Q13-11. In a cipher, all As in the plaintext have been changed to Ds in the ciphertext and all Ds in the plaintext have been changed to Hs in the ciphertext. Is this a monoalphabetic or polyalphabetic substitution cipher? Explain.

Q13-12. Which cipher can be broken more easily, monoalphabetic or polyalphabetic?

Q13-13. Assume Alice and Bob use an additive cipher in modulo-26 arithmetic. If Eve, the intruder, wants to break the code by trying all possible keys (brute-force attack), how many keys should she try on average?

Q13-14. If we have a single integer key in Example 13.1 and 13.2, how many integer keys do we have in Example 13.3?

Q13-15. Assume we have a plaintext of 1000 characters. How many keys do we need to encrypt or decrypt the message in each of the following ciphers?
a. additive **b.** monoalphabetic **c.** autokey

Q13-16. According to the definitions of stream and block ciphers, find which of the following ciphers is a stream cipher.
a. additive **b.** monoalphabetic **c.** autokey

Q13-17. A permutation block (P-box) in a modern block cipher has five inputs and five outputs. This is a _____ permutation?
a. straight **b.** compression **c.** compression

Q13-18. A permutation block (P-box) in a modern block cipher is an example of a keyless transposition cipher. What does this statement mean? (See Figure 13.8.)

Q13-19. In a modern block cipher, we often need to use a component in the decryption cipher that is the inverse of the component used in the encryption cipher. What is the inverse of each of the following components?
a. swap **b.** shift right **c.** combine

Q13-20. In each round of DES, we have all components defined in Figure 13.8. Which components use a key, and which components do not?

Q13-21. In Figure 13.10, why do we need an expansion P-box? Why can't we use a straight or a compression P-box?

Q13-22. Figure 13.9 shows that DES creates 16 different 48-bit keys, one for each round. Why do we need 16 different keys? Why can't we use the same key in each round?

Q13-23. If the one-time pad cipher (Figure 13.12) is the simplest and most secure cipher, why is it not used all the time?

Q13-24. If Alice and Bob need to communicate using asymmetric-key cryptography, how many keys do they need? Who needs to create these keys?

Q13-25. Why do you think asymmetric-key cryptography is used only with small messages?

Q13-26. In an asymmetric public key cipher, which key is used for encryption? Which key is used for decryption?
a. public key **b.** private key

Q13-27. In RSA, why can't Bob choose 1 as the public key e?

Q13-28. What is the role of the secret key added to the hash function in Figure 13.17 (MAC)? Explain.

Q13-29. Distinguish *message authentication* and *entity authentication*.

Q13-30. Alice signs the message she sends to Bob to prove that she is the sender of the message. Which of the following keys does Alice need to use?
a. Alice's public key **b.** Alice's private key

Q13-31. Alice needs to send a message to a group of 50 people. If Alice needs to use message authentication, which of the following schemes do you recommend?
a. MAC **b.** digital signature

Q13-32. Which of the following services are not provided by digital signature?
a. message authentication **b.** confidentiality **c.** non-repudiation

Q13-33. Assume Alice needs to send a confidential signed document to 100 people. How many keys does Alice need to use to prepare 100 copies if she uses asymmetric-key confidentiality? Explain.

Q13-34. In a club with 50 members, how many secret keys are needed to allow secret messages to be exchanged between any pair of members?

Q13-35. A key distribution center (KDC) is designed to solve the problem of distributing _____ keys.

a. secret **b.** public **c.** private

Q13-36. A certification authority (CA) is designed to solve the problem of distributing _____ keys.

a. secret **b.** public **c.** private

Q13-37. Why does IPSec need a security association?

Q13-38. How does IPSec create a set of security parameters?

Q13-39. What are the two protocols defined by IPSec?

Q13-40. What does AH add to the IP packet?

Q13-41. What does ESP add to the IP packet?

Q13-42. Are both AH and ESP needed for IP security? Why or why not?

Q13-43. What are the two protocols discussed in this chapter that provide security at the transport layer?

Q13-44. What is IKE? What is its role in IPSec?

Q13-45. What is the difference between a session and a connection in SSL?

Q13-46. How does SSL create a set of security parameters?

Q13-47. What are the names of the protocols, discussed in this chapter, that provide security for e-mail?

Q13-48. How does PGP create a set of security parameters?

Q13-49. What is the purpose of the Handshake Protocol in SSL?

Q13-50. What is the purpose of the Record Protocol in SSL?

Q13-51. What is the purpose of a firewall?

Q13-52. What are the two types of firewalls?

Q13-53. What is a VPN, and why is it needed?

Q13-54. How do LANs on a fully private internet communicate?

13.9.3 Problems

P13-1. Define the type of attack in each of the following cases:

a. A student breaks into a professor's office to obtain a copy of the next test.

b. A student gives a check for $10 to buy a used book. Later the student finds out that the check was cashed for $100.

c. A student sends hundreds of e-mails per day to the school using a phony return e-mail address.

P13-2. Use the additive cipher with $k = 10$ to encrypt the plaintext "book". Then decrypt the message to get the original plaintext.

P13-3. Encrypt the message "this is an exercise" using an additive cipher with key = 20. Ignore the space between words. Decrypt the message to get the original plaintext.

P13-4. Atbash was a popular cipher among Biblical writers. In Atbash, "A" is encrypted as "Z", "B" is encrypted as "Y", and so on. Similarly, "Z" is encrypted as "A", "Y" is encrypted as "B", and so on. Suppose that the alphabet is divided into halves and the letters in the first half are encrypted as the letters in the

second, and vice versa. Find the type of cipher and key. Encipher the plaintext "an exercise" using the Atbash cipher.

P13-5. A substitution cipher does not have to be a character-to-character transformation. In a Polybius cipher, each letter in the plaintext is encrypted as two integers. The key is a 5×5 matrix of characters. The plaintext is the character in the matrix; the ciphertext is the two integers (each between 1 and 5) representing the row and column numbers. Encipher the message "An exercise" using the Polybius cipher with the following key:

	1	2	3	4	5
1	z	q	p	f	e
2	y	r	o	g	d
3	x	s	n	h	c
4	w	t	m	i / j	b
5	v	u	l	k	a

P13-6. Alice can use only the additive cipher on her computer to send a message to a friend. She thinks that the message is more secure if she encrypts the message two times, each time with a different key. Is she right? Defend your answer.

P13-7. One of the attacks an intruder can apply to a simple cipher like an additive cipher is called the *ciphertext* attack. In this type of attack, the intruder intercepts the cipher and tries to find the key and eventually the plaintext. One of the methods used in a ciphertext attack is called the *brute-force* approach, in which the intruder tries several keys and decrypts the message until the message makes sense. Assume the intruder has intercepted the ciphertext "UVACLYZLJBYL". Try to decrypt the message by using keys from 1 until a plaintext appears that makes sense.

P13-8. Another method used in a ciphertext attack (see problem P13-7) is called the *statistical* approach, in which the intruder intercepts a long ciphertext and tries to analyze the statistics of the characters in the ciphertext. A simple cipher like the additive cipher does not change the statistics of the characters because encryption is one-to-one. Assume the intruder has intercepted the following ciphertext and the most common character in an English plaintext is the character "e". Use this knowledge to find the key of the cipher and decrypt the ciphertext.

```
XLILSYWIMWRSAJSVWEPIJSVJSYVQMPPMSRHSPPEVWMXMWASVXLQSVILYVVCF
IJSVIXLIWIPPIVVIGIMZIWQSVISJJIVW
```

P13-9. In a transposition cipher, the encryption and decryption keys are often represented as two one-dimensional tables (arrays) and the cipher is represented as a piece of software (a program).

a. Show the array for the encryption key in Figure 13.6. Hint: The value of each element can show the input-column number; the index can show the output-column number.

b. Show the array for the decryption key in Figure 13.6.

c. Explain, given the encryption key, how we can find the decryption key.

P13-10. The circular shift operation is one of the components of the modern block cipher.

a. Show the result of a 3-bit circular left shift on the word $(10011011)_2$.

b. Show the result of a 3-bit circular right shift on the result of part a.

c. Compare the result of part b with the original word in part a to show that shift-right and shift-left operations are inverses of each other.

P13-11. The swap operation is one of the components of the modern block cipher.

a. Swap the word $(10011011)_2$.

b. Swap the word resulting from part a.

c. Compare the results of part a and part b to show that swapping is a self-invertible operation.

P13-12. A very common operation in block ciphers is the XOR operation. Find the result of the following operations. Interpret the results.

a. (01001101) $\oplus$ (01001101)

b. (01001101) $\oplus$ (00000000)

P13-13. Assume you want to write a program to simulate the permutation boxes in Figure 13.8.

a. Show how you represent each box as a table.

b. Show the inversion of each box as a table.

P13-14. Assume we have a keyless substitution box (S-box) with three inputs (x_1, x_2, and x_3) and two outputs (y_1 and y_2). The relation between the inputs and outputs is defined as follows ($\oplus$ means XOR):

$$y_1 = x_1 \oplus x_2 \oplus x_3 \qquad y_2 = x_1$$

What is the output if the input is (110)? What is the output if the input is (001)?

P13-15. Each round in a block cipher should be invertible to make the whole block invertible. Modern block ciphers use two approaches to achieve this. In the first approach, each component is invertible; in the second approach some components are not invertible but the whole round is invertible using what is called a Feistel cipher. This approach is used in DES, as described in the text. The trick in the Feistel cipher is to use the XOR operation as one of the components. To see the point, assume that a round is made up of a noninvertible component, NI, and an XOR operation, as shown in Figure 13.63. Prove that the whole round is invertible, which means that the plaintext can be recovered from the ciphertext. Hint: Use XOR properties ($x \oplus x = 0$ and $x \oplus 0 = x$).

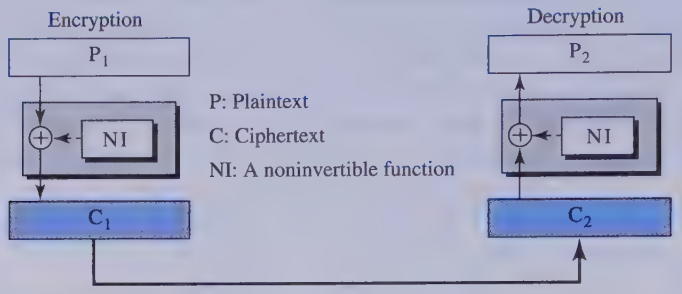
P13-16. In Figure 13.9, we have a swapper in each round. What is the use of this swapper?

P13-17. Host A and host B use IPSec in the transport mode. Can we say that the two hosts need to create a virtual connection-oriented service between them? Explain.

P13-18. When we talk about authentication in IPSec, do we mean *message authentication* or *entity authentication*? Explain.

P13-19. If Alice and Bob are continuously sending messages to each other, can they create a security association once and use it for every packet exchanged? Explain.

Figure 13.63 *Problem P31-15*



P13-20. Can we use SSL with UDP? Explain.

P13-21. Why is there no need for a Security Association with SSL?

P13-22. Compare and contrast PGP and S/MIME. What are the advantages and disadvantages of each?

P13-23. Should the handshaking in SSL occur before or after the three-way handshaking in TCP? Can they be combined? Explain.

P13-24. We defined two security services for e-mail (PGP and S/MIME). Explain why e-mail applications cannot use the services of SSL/TLS and need to use either PGP or S/MIME.

P13-25. Assume Alice needs to send an e-mail to Bob. Explain how the integrity of the e-mail is achieved using PGP.

P13-26. Assume Alice needs to send an e-mail to Bob. Explain how the confidentiality of the e-mail is achieved using PGP.

P13-27. Assume Alice needs to send an e-mail to Bob. Explain how the integrity of the e-mail is achieved using S/MIME.

P13-28. Assume Alice needs to send an e-mail to Bob. Explain how the authentication of the e-mail is achieved using S/MIME.

P13-29. Assume Alice needs to send an e-mail to Bob. Explain how the confidentiality of the e-mail is achieved using S/MIME.

P13-30. When we talk about authentication in SSL, do we mean *message authentication* or *entity authentication*? Explain.

P13-31. When we talk about authentication in PGP (or S/MIME), do we mean *message authentication* or *entity authentication*? Explain.

P13-32. If cryptography algorithms in PGP or S/MIME cannot be negotiated, how can the receiver of the e-mail determine which algorithm has been used by the sender?

APPENDIX A

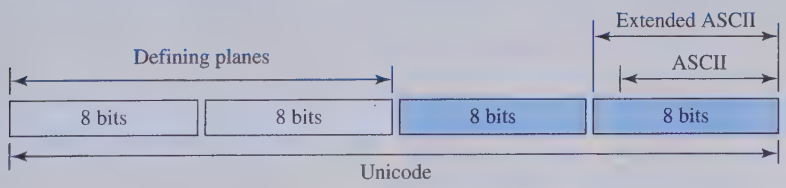
Unicode

Computers use numbers. They store characters by assigning a number for each one. The original coding system was called ASCII (American Standard Code for Information Interchange) and had 128 symbols (0 to 127) each stored as a 7-bit number. ASCII could satisfactorily handle lowercase and uppercase letters, digits, punctuation characters, and some control characters. An attempt was made to extend the ASCII character set to 8 bits. The new code, which was called Extended ASCII, was never internationally standardized.

To overcome the difficulties inherent in ASCII and Extended ASCII, the Unicode Consortium (a group of multilingual software manufacturers) created a universal encoding system to provide a comprehensive character set called **Unicode**.

Unicode was originally a 2-byte character set. Unicode version 3, however, is a 4-byte code and is fully compatible with ASCII and Extended ASCII. The ASCII set, which is now called *Basic Latin*, is Unicode with the most significant 25 bits set to zero. Extended ASCII, which is now called Latin-1, is Unicode with the most significant 24 bits set to zero. Figure A.1 shows how the different systems are compatible.

Figure A.1 *Unicode bytes*



Each character or symbol in this code is defined by a 32-bit number. The code can define up to 2^{32} (4,294,967,296) characters or symbols. The notation uses hexadecimal digits in the following format.

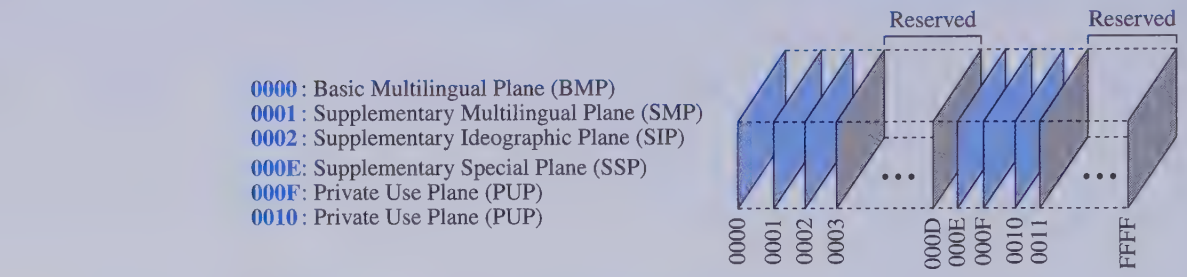
U-XXXXXXXX

Each X is a hexadecimal digit. Therefore, the numbering goes from U-00000000 to U-FFFFFFFF.

A.1 PLANES

Unicode divides the available space codes into planes. The most significant 16 bits define the plane, which means we can have 65,536 planes. Each plane can define up to 65,536 characters or symbols. Figure A.2 shows the structure of Unicode spaces and planes.

Figure A.2 *Unicode planes*



A.1.1 Basic Multilingual Plane (BMP)

Plane $(0000)_{16}$, the basic multilingual plane (BMP), is designed to be compatible with the previous 16-bit Unicode. The most significant 16 bits in this plane are all zeros. The codes are normally shown as U+XXXX with the understanding that XXXX defines only the least significant 16 bits. This plane mostly defines character sets in different languages with the exception of some codes used for control or other special characters.

A.1.2 Other Planes

There are some other (nonreserved) planes, which we briefly describe here.

Supplementary Multilingual Plane (SMP)

Plane $(0001)_{16}$, the supplementary multilingual plane (SMP), is designed to provide more codes for those multilingual characters that are not included in the BMP.

Supplementary Ideographic Plane (SIP)

Plane $(0002)_{16}$, the supplementary ideographic plane (SIP), is designed to provide codes for ideographic symbols, symbols that primarily denote an idea (or meaning) in contrast to a sound (or pronunciation).

Supplementary Special Plane (SSP)

Plane $(000E)_{16}$, the supplementary special plane (SSP), is used for special characters.

Private Use Planes (PUPs)

Planes (000F) and $(0010)_{16}$, private use planes (PUPs), are for private use.

A.2 ASCII

The American Standard Code for Information Interchange (ASCII) is a 7-bit code that was designed to provide code for 128 symbols, mostly in American English. Today, ASCII, or Basic Latin, is part of Unicode. It occupies the first 128 codes in Unicode (00000000 to 0000007F). Table A.1 contains the hexadecimal and graphic codes (symbols). The codes in hexadecimal just define the two least significant digits in Unicode. To find the actual code, we prepend 000000 in hexadecimal to the code.

Table A.1 *ASCII Codes*

Symbol	*Hex*	*Symbol*	*Hex*	*Symbol*	*Hex*	*Symbol*	*Hex*
NULL	00	SP	20	@	40	`	60
SOH	01	!	21	A	41	a	61
STX	02	"	22	B	42	b	62
ETX	03	#	23	C	43	c	63
EOT	04	$	24	D	44	d	64
ENQ	05	%	25	E	45	e	65
ACK	06	&	26	F	46	f	66
BEL	07	'	27	G	47	g	67
BS	08	(	28	H	48	h	68
HT	09	)	29	I	49	i	69
LF	0A	*	2A	J	4A	j	6A
VT	0B	+	2B	K	4B	k	6B
FF	0C	,	2C	L	4C	l	6C
CR	0D	-	2D	M	4D	m	6D
SO	0E	.	2E	N	4E	n	6E
SI	0F	/	2F	O	4F	o	6F
DLE	10	0	30	P	50	p	70
DC1	11	1	31	Q	51	q	71
DC2	12	2	32	R	52	r	72
DC3	13	3	33	S	53	s	73
DC4	14	4	34	T	54	t	74
NAK	15	5	35	U	55	u	75
SYN	16	6	36	V	56	v	76
ETB	17	7	37	W	57	w	77
CAN	18	8	38	X	58	x	78
EM	19	9	39	Y	59	y	79
SUB	1A	:	3A	Z	5A	z	7A

Table A.1 *ASCII Codes (continued)*

Symbol	Hex	Symbol	Hex	Symbol	Hex	Symbol	Hex
ESC	1B	;	3B	[	5B	{	7B
FS	1C	<	3C	\	5C	|	7C
GS	1D	=	3D	]	5D	}	7D
RS	1E	>	3E	^	5E	~	7E
US	1F	?	3F	_	5F	DEL	7F

A.2.1 Some Properties of ASCII

ASCII has some interesting properties that we briefly mention here.

1. The space character $(20)_{16}$ is a printable character. It prints a blank space.
2. The uppercase letters start from $(41)_{16}$. The lowercase letters start from $(61)_{16}$. When compared, uppercase letters are numerically smaller than lowercase letters. This means that in a sorted list based on ASCII values, the uppercase letters appear before the lowercase letters.
3. The uppercase and lowercase letters differ by only 1 bit in the 7-bit code. For example, character *A* is $(1000001)_2$ and character *a* is $(1100001)_2$. The difference is in bit 6, which is 0 in uppercase letters and 1 in lowercase letters. If we know the code for one case, we can easily find the code for the other by adding or subtracting $(20)_{16}$, or we can just flip the sixth bit.
4. The uppercase letters are not immediately followed by lowercase letters. There are some punctuation characters in between.
5. Digits (0 to 9) start from $(30)_{16}$. This means that if you want to change a numeric character to its face value as an integer, you need to subtract $(30)_{16} = 48$ from it.
6. The first 32 characters, $(00)_{16}$ to $(1F)_{16}$, and the last character, $(7F)_{16}$, are nonprintable characters. Character $(00)_{16}$ simply is used as a delimiter to define the end of a character string. Character $(7F)_{16}$ is the delete character used by some programming languages to delete the previous character. The rest of the nonprintable characters are referred to as *control characters* and are used in data communication. Table A.2 gives the description of these characters.

Table A.2 *ASCII Codes*

Symbol	Interpretation	Symbol	Interpretation
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledgment
ACK	Acknowledgment	SYN	Synchronous idle

Table A.2 *ASCII Codes (continued)*

Symbol	*Interpretation*	*Symbol*	*Interpretation*
BEL	Ring bell	ETB	End of transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
DLE	Data link escape		

APPENDIX B

Positional Numbering System

A **positional numbering system** uses a set of symbols. The value that each symbol represents, however, depends on its **face value** and its **place value**, the value associated with the position it occupies in the number. In other words, we have the following.

Symbol value = face value × place value
Number value = sum of symbol values

In this appendix, we discuss only integers, numbers with no fractional part; the discussion of reals, numbers with a fractional part, is similar.

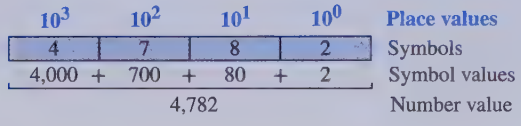
B.1 DIFFERENT SYSTEMS

We first show how integers can be represented in four different systems: base 10, base 2, base 16, and base 256.

B.1.1 Base 10: Decimal

The first positional system we discuss is called the **decimal system**. The term *decimal* is derived from the Latin root *decem* (meaning *ten*). The decimal system uses 10 symbols (0, 1, 2, 3, 4, 5, 6, 7, 8, and 9) with the same face values as the symbols. The place values in the decimal number system are powers of 10. Figure B.1 shows the place values and the symbol values in the integer 4782.

Figure B.1 *An example of a decimal number*

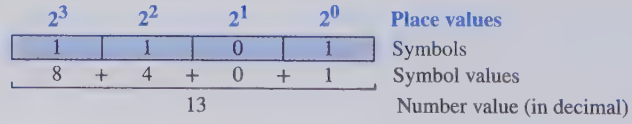


The decimal system uses 10 symbols in which the place values are powers of 10.

B.1.2 Base 2: Binary

The second positional system we discuss is called the **binary system**. The term *binary* is derived from the Latin root *bi* (meaning *two by two*). The binary system uses two symbols (0 and 1) with the same face values as the symbols. The place values in the binary number system are powers of 2. Figure B.2 shows the place values and the symbol values in the binary $(1101)_2$. Note that we use subscript 2 to show that the number is in binary.

Figure B.2 *An example of a binary number*

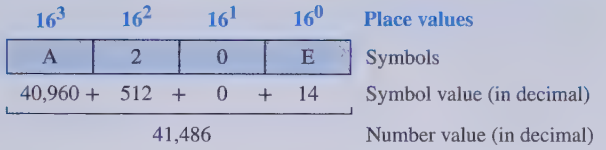


The binary system uses two symbols in which the place values are powers of 2.

B.1.3 Base 16: Hexadecimal

The third positional system we discuss is called the **hexadecimal system**. The term *hexadecimal* is derived from the Greek root hex (meaning 6) and the Latin root *decem* (meaning *ten*). The hexadecimal system uses 16 symbols (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F). The face value of the first 10 symbols are the same as the symbols, but the face values of the symbols A to F are 10 to 15, respectively. The place values in the hexadecimal number system are powers of 16. Figure B.3 shows the place values and the symbol values in the hexadecimal $(A20E)_{16}$. Note that we use subscript 16 to show that the number is in hexadecimal.

Figure B.3 *An example of a hexadecimal number*

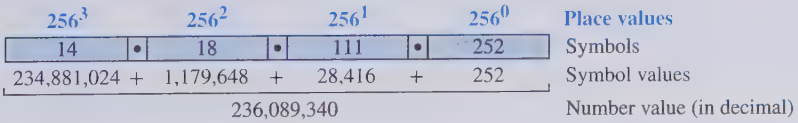


The hexadecimal system uses 16 symbols in which the place values are powers of 16.

B.1.4 Base 256: Dotted-Decimal Notation

The fourth positional system we discuss is base 256, which is called **dotted-decimal notation**. This system is used to represent IPv4 addressing. The place values in this system are powers of 256. However, since using 256 symbols is almost impossible, the symbols in this system are decimal numbers between 0 and 255, with the same face values as the symbols. To separate these numbers from each other, the system uses a dot, as discussed in Chapter 7. Figure B.4 shows the place values and the symbol values of the address (14.18.111.252). Note that we never use more than four symbols in an IPv4 address.

Figure B.4 *An example of a dotted-decimal notation*



Dotted-decimal notation uses decimal numbers (0 to 255) as symbols, but inserts a dot between each symbol.

B.1.5 Comparison

Table B.1 shows how three different systems represent the decimal numbers 0 through 15. For example, decimal 13 is equivalent to binary $(1101)_2$, which is equivalent to hexadecimal D.

Table B.1 *Comparison of three systems*

Decimal	*Binary*	*Hexadecimal*	*Decimal*	*Binary*	*Hexadecimal*
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

B.2 CONVERSION

We need to know how to convert a number in one system to the equivalent number in another system.

B.2.1 Conversion from Any Base to Decimal

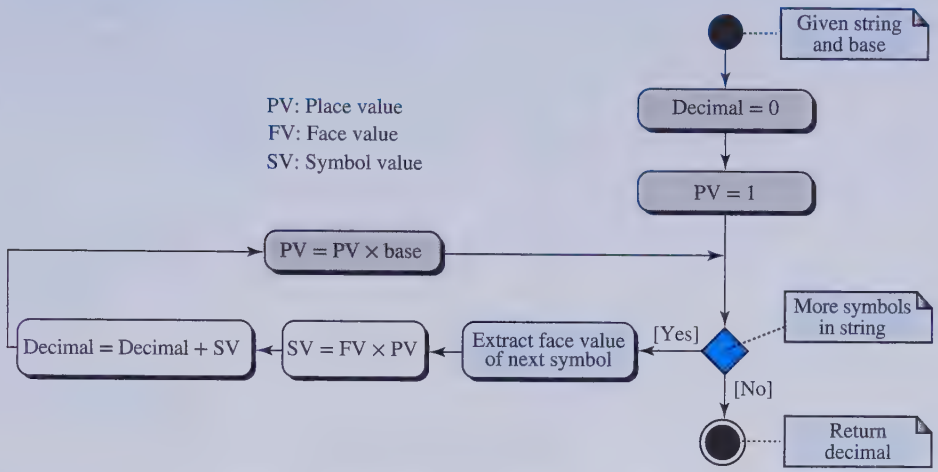
Figures B.2 to B.4 actually show how we can manually convert a number in any base to decimal. However, it is easier to use the algorithm in Figure B.5. The algorithm uses the fact that the next place value is the previous value multiplied by the base (2, 16, or 256). The algorithm is a general one that can be used to convert a string of symbols in a given base to a decimal number. The only section in the algorithm that is different for each base is how to extract the next symbol in the string and find its face value. In the case of base 2, it is simple; the face value can be found by changing the symbol to a numeric value. In the case of base 16, we need to consider the case that the face value of symbol A is 10, the face value of symbol B is 11, and so on. In the case of base 256, we need to extract each string delimited by dots and change the string to its numeric value.

The manual implementation of this algorithm for small numbers can be shown in a few examples.

Example B.1

Show the equivalent of the binary number $(11100111)_2$ in decimal.

Figure B.5 *Algorithm to convert from any base to decimal*



Solution

We follow the algorithm as shown here.

128	64	32	16	8	4	2	1	Place values
1	1	1	0	0	1	1	1	Face values
128	64	32	0	0	4	2	1	Symbol values
231	103	39	7	7	7	3	1	Decimal = 0

The value of decimal is initially set to 0. When the loop is terminated, the value of decimal is 231.

Example B.2

Show the equivalent of the IPv4 address 12.14.67.24 in decimal.

Solution

We follow the algorithm as shown here.

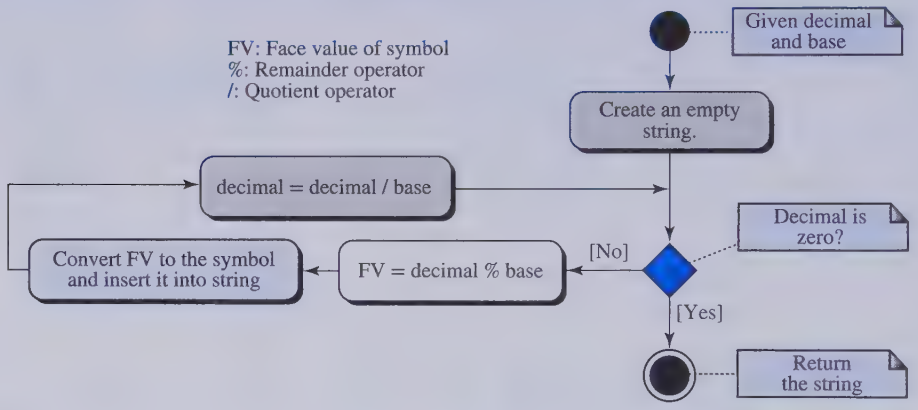
16,777,216		65,536		256		1	Place values
12	.	14	.	67	.	24	Face values
201,326,592		917,504		17,152		24	Symbol values
202,261,272		934,680		17,176		24	Decimal = 0

The value of decimal is initially set to 0. When the loop is terminated, the value of decimal is 202,261,272.

B.2.2 Conversion from Decimal to Any Base

Conversion from a decimal value to any base can be done if we continuously divide the decimal number by the base to find the remainder and the quotient. The remainder is the face value of the next symbol; the quotient is the decimal value to be used in the next iteration. As in the case of inverse conversion, we need to have a separate algorithm to change the face value of a symbol, in the corresponding base, to the actual symbol and insert it in the string representing the converted number.

Figure B.6 *Conversion from decimal to any base*



We can show how we can manually follow the algorithm in a few examples.

Example B.3

Convert the decimal number 25 to its binary equivalent.

Solution

We continuously divide the decimal value by 2 (the base of the binary system) until the quotient becomes 0. In each division we interpret the value of the remainder as the next symbol to be inserted in the hexadecimal string. The down arrow shows the remainder; the left arrow shows the quotient. When the decimal value becomes 0, we stop. The result is the binary string $(11001)_2$.

0	←	1	↑	3	←	6	←	12	←	**25**	Decimal
		↓		↓		↓		↓		↓	
		1		1		0		0		1	Binary

Example B.4

Convert the decimal number 21,432 to its hexadecimal equivalent.

Solution

We continuously divide the decimal value by 16 (the base of the hexadecimal system) until the quotient becomes 0. In each division we interpret the value of the remainder as the next symbol to be inserted in the hexadecimal string. The result is the hexadecimal string $(53B8)_{16}$.

0	←	5	←	83	←	1339	←	**21432**	Decimal
		↓		↓		↓		↓	
		5		3		B		8	Hexadecimal

Example B.5

Convert the decimal number 73,234,122 to base 256 (IPv4 address).

Solution

We continuously divide the decimal value by 256 (the base) until the quotient becomes 0. In each division we interpret the value of the remainder as the next symbol to be inserted in the IPv6 address. We also insert dots as required in the dotted-decimal notation. The result is the IPv4 address 4.93.118.202.

0	←	4	←	1,117	←	286,070	←	**73,234,122**	Decimal
		↓		↓		↓		↓	
		4	.	93	.	118	.	202	IPv4 address

B.2.3 Other Conversions

Conversion from a nondecimal system to another nondecimal system is often easier. We can easily convert a number in binary to hexadecimal by converting a group of 4 bits into 1 hexadecimal digit. We can also convert a hexadecimal digit into a group of 4 bits. We give a few examples to show the process.

Example B.6

Convert the binary number $(1001111101)_2$ to its equivalent in hexadecimal.

Solution

We create groups of 4 bits from the right. We then replace each group with its equivalent hexadecimal digit. Note that we need to add two extra 0s to the last group.

0010	**0111**	**1101**	Binary
↓	↓	↓	
2	7	D	Hexadecimal

The result is $(27D)_{16}$.

Example B.7

Convert the hexadecimal number $(3A2B)_{16}$ to its equivalent in binary.

Solution

We change each hexadecimal digit to its 4-bit binary equivalent.

3	A	2	B	Hexadecimal
↓	↓	↓	↓	
0011	1010	0010	1011	Binary

The result is $(0011\ 1010\ 0010\ 1011)_2$.

Example B.8

Convert the IPv4 address 112.23.78.201 to its binary format.

Solution

We replace each symbol to its equivalent 8-bit binary.

112	•	23	•	78	•	201	IPv4 address
↓		↓		↓		↓	
01110000		00010111		01001110		11001001	Binary

The result is $(01110000\ 00010111\ 01001110\ 11001001)_2$.

APPENDIX C

HTML, CSS, XML, and XSL

This appendix is a very brief introduction to two markup languages and their style counterparts. The appendix is intended to give a high-level introduction to these languages for readers of the book. They are not intended to teach how to write documents in these languages, for which a more detailed text is required.

C.1 HTML

Hypertext Markup Language (**HTML**) is a markup language for creating Web pages. In this text, when we mention HTML, except where otherwise specified, we mean HTML or XHTML. The differences between the two will be cleared up later. The term *markup language* comes from the book publishing industry; before a book is typeset and printed, a copy editor reads the manuscript and puts marks on it. These marks tell the compositor how to format the text. For example, if the copy editor wants a section of a line to be printed in boldface, she draws a wavy line under that section. Similarly, text and other information for a web page are marked by HTML to be interpreted and displayed by a browser. For easy reading, we have set the markup sections of documents in color.

C.1.1 HTML Document

To display a document on a browser, we need to create an HTML document. The document should be unformatted (in standard ASCII) text. Most simple text editors such as Windows Notepad or Macintosh TextEdit create unformatted text, but if we use a word processing software, we should save the result as plaintext. We save the file with an *html* extension, for example, **fileName.html**. We can then open the file from any web browser.

C.1.2 Tags

Tags are the basic element of HTML. Tags are hidden commands that tell the web browsers how to interpret and display text and other content of a web page. Most tags come in pairs: *beginning tag* and *ending tag*.

<tagName> ... **</tagName >**

Note that the tag name is in lowercase and included inside pointed brackets; the ending tag has an extra slash. The content comes between the beginning and ending tags. For example, the pair **** and **** are bold tags:

**** This text will be displayed in bold. ****

Certain tags do not come in pairs. For example, we only put a **
** tag where we want a line break. Such a tag is called an *empty tag* and has a slash written to the right of the tag name:

<tagName/>

Most tags have a set of optional attributes and corresponding values included in the beginning tag:

<tagName attribute = value **attribute** = value ... > content **</tagName>**

Doctype

The doctype declaration is version information that appears as the first line of any HTML document. It refers to a known **Document Type Definition** (**DTD**) that provides the tags and attributes of an HTML document. In any web page, click the right button

and select *View Source*, and you will see the doctype declaration as the first line of the source codes, which looks like

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
"http://www.w3.org/TR/html4/loose.dtd">
```

The following are explanations of different sections of the code:

!DOCTYPE	Doctype declaration (uppercase)
PUBLIC	DTD is a public resource
W3C	Guardian of DTD
HTML 4.01:	Markup language and version
EN	English
http://www.w3.org/TR/html4/loose.dtd	URL for the location of the DTD

Structural Tags

Head and Title

The <head> tag usually comes after the *doctype declaration*. It is followed by important document information. One of the most important pieces of document information is the document title, which comes between the <title> and </title> tags. The title has only informational value and is not displayed by the browser, but it is displayed in the browser title bar. The </head> tag ends the header section of the document. The following shows an example.

```
<head>
        <title> Title of document goes here. </title>
        Other document information
        ...
</head>
```

Body

The actual body of an HTML document is enclosed between <body> and </body> tags. An HTML document is normally organized with the head, title, and body tags as follows:

```
<head>
        <title> Title of document goes here. </title>
        Other document information
        ...
</head>

<body>
        The content of the document goes here.
        ...
</body>
```

Heading
Tags <h*n*> and </h*n*>, with $n = 1, 2, \ldots, 6$, are used to describe the six heading levels in HTML, with h1 the largest and h6 the smallest. Browsers apply a line break after the ending tag.

Paragraph
Tag is used to start a new paragraph; the ending counterpart is used to end the paragraph. Browsers apply a line break after the ending tag.

Line break
The <br/> tag, when placed at the end of a line, forces a line break.

Center Tags
Tags and are used to center a line of a text. Browsers apply a line break after the ending tag.

Blockquote
The <blockquote> and </blockquote> tags are used for marking up a block of text quoted from a person or a source. Normally by default, the text contained within these tags is displayed with left and right indentations. Browsers apply a line break after the ending tag.

Preserve
It is important to know that web browsers, unless specified by special tags, only honor the first space and ignore other white spaces such as carriage return and tab. For example, the nutrition fact table typed as

```
Total fat      5 g
Sodium         15 g
Protein        0 g
```

in a web browser appears as:

Total fat 5 g Sodium 15 g Protein 0 g

However, we may use the <pre> and </pre> tags to preserve white spaces (such as spaces, tabs, and line breaks) in a document. For example, we can make the table appear exactly as typed by placing preserve tags before and after the table.

```
<pre>

            Total fat      5 g
            Sodium         15 g
            Protein        0 g
</pre>
```

List
We can define two types of lists: ordered and unordered. The <ol> and tags are used for an ordered list; the <ul> and tags are used for an unordered list. Each

item in either type of list is enclosed inside <li> and tags. Browsers apply a line break after the tag. Items in an ordered list are numbered, while the items in an unordered list are normally bulleted:

HTML text

```
<ol>
    <li> CIS 20 </li>
    <li> CIS 30 </li>
    <li> CIS 40 </li>
<ol>
```

Appearance in a web browser

1. CIS 20
2. CIS 30
3. CIS 40

Anchor

The special feature of HTML is hypertext links. The links in an HTML document allow the user to navigate from one document to another document. The <a> and tags, called anchor tags or link tags, are used to create a link to another web page. One of the attributes used with the <a> tag is ***href*** (hyperlink reference), whose value is a URL that indicates the link's destination. For example, the following HTML line creates a link to the Behrouz Forouzan web page at McGraw-Hill Publisher.

<a **href** = **"http://www.mhhe.com/forouzan"** > Behrouz Forouzan

Image

We may also include images in a document. The image tag has many attributes. Three are shown in the following tag.

The ***src*** (source) attribute gives the location (URL) where the image is located. The ***alt*** (alternate) attribute defines the text to replace the image if for any reason the image cannot be displayed. The ***align*** attribute defines how the image should be aligned with respect to the text document. An image is not directly embedded in the document; using the above attributes, the browser finds the image and places it where the tag is located.

Text Formatting

Bold and Italic versus Strong and Emphasis

The two most common text formatting tags are bold tags, and , and italic tags, and . The use of these tags, however, is declining in favor of two almost equivalent tags: the strong tags, and , and emphasis tags, and . Just like the bold and italic tags, these tags make the text appear bold or italic, respectively, but also give semantic meaning to the contained text. Unlike the *bold* and *italic* tags, the *strong* and *emphasis* tags indicate how the corresponding words marked by these tags should be spoken by a speech reader.

Small and Big

To increase or decrease the font size by one increment, we use <big> and </big> or <small> and </small> tags. For example, the HTML text

```
This is <small> smaller </small> but that one is <big> bigger </big>
```

appears as "This is smaller but that one is bigger".

Other Formatting

Some other common text formatting tags are listed in the following table.

Strike	<strike>	Strike a line through the text.
Subscript	<sub>	Move the text half a character up.
Superscript	<sup>	Move the text half a character down.
Underline	<u>	Underline the text.

You may want to use the <sub> and <sup> tags with *small* tags:

H <sub> <small> 2 <small/> <sub/> O appears as H_2O

Advisory Tags

Four important advisory tags are abbreviation tags (<abbr> and </abbr>), acronym tags (<acronym> and </acronym>), definition tags (<def> and </def>), and cite tags, (<cite> and </cite>). All four have title attributes and a similar format:

<tagName **title** = "string"> </tagName>

For example, the following abbreviation tags show that DTD is an abbreviation for Document Type Definition.

<abbr **title** = "Document Type Definition"> **DTD** </abbr>

In a browser, when we put the curser over the contained text (DTD in our example) the title (Document Type Definition) is displayed (usually in a tool-tip).

Nesting

We may use two or more tags in nested form. For example, we can nest italic tags inside bold tags:

  This text is in italic bold

Make sure to nest them correctly. The following is the wrong format:

  Wrong Format

The order of nesting can change the appearance of the text. For example, compare the two nested expressions.

HTML Text	Appearance in a web browser
First Second	***First* Second**
First Second	***First* Second**

C.1.3 XHTML

The **Extensible Hypertext Markup Language** (**XHTML**) is almost identical to HTML 4.01, but it also conforms to the restricted syntax of XML. This compliance makes XHTML a structured markup language. A document marked up with XHTML will be a "well-formed" document and, consequently, will be interpreted and displayed by browsers the way the author intended. Some of the most important requirements of XHTML are:

- Elements must be properly nested.
- Elements must always be closed: Normal tags such as paragraph tags ** ** and ** ** must have the beginning and ending tags; the empty tags such as line break must be written as **
** and not as **
**. The slash after *br* indicates closing of the element.
- Elements and attributes must be in lowercase.
- Attribute values must be quoted.
- Documents must have three main parts: DOCTYPE declaration, head section, and body section.

C.2 CSS

Logically, a simple web document is made up of two layers: content and presentation. Although it is possible to keep both layers together, separating them increases flexibility, reduces repetition, and increases efficiency. **Cascading Style Sheets** (**CSS**) were created to separate the document content from document presentation. We may apply styles to elements of an HTML document in three ways: *in-line*, *internal*, or *external*.

C.2.1 In-line Style

We may specify a style to an individual element of an HTML document. For example, the following makes the font size of the contained paragraph 90% and the font color blue.

```
<p style = "font-size: 90%; color: blue" >
The size of the font is 90% and blue
</p>
```

C.2.2 Internal Style Sheet

If we want to specify style rules that apply to a single HTML document, we can enclose the style sheet between **<style>** and **</style>** tags in the head section of the HTML document (such as **body**, **h1**, and so on). The general format of the style sheet rule is

HTML content {**attribute**: value; **attribute**: value; ...}

Note that each attribute is separated from its value by a colon. Attributes are separated by semicolons, and the entire attribute block is placed inside curly brackets {}.

For example, the following internal style sheet applies rules to heading 1 and the body of the document.

```
<head>
    <title> Internal Style sheet </title>
            <style type = "text / css" >
            h1{font-family: mono space; color: green}
            body {font-family: cursive; color: red}
            <style>
</head>
<body>
...
<body>
```

C.2.3 External Style Sheet

To create an external style sheet, we create a text document and place all the desired style rules for each part of the HTML content in that document and save the document with a *css* extension: **fileName.css**. The following is an example of such a document:

```
body {font-size: 10 pt; font-family: Times New Roman; color:
      black; margin-left: 12 pt; margin-right: 12 pt; line-height; 14 pt}

p {margin-left: 24 pt; margin-right: 24 pt}
h1 {font-size: 24 pt; font-family: Book Antiqua; color: red}
h2 {font-size: 22 pt; font-family: Book Antiqua; color: red}
...
h6 {font-size: 12 pt; font-family: Book Antiqua; color: red}
...
a: link {color: red}
a: visited {color: blue}
...
```

Next we link this style sheet to any HTML document by including a <link/> tag in the head section of that document:

<link rel = "style sheet" type = "text/css" href = "URL" />

The ***rel*** (relationship) attribute says that the reference document is a style sheet. The ***type*** attribute identifies the MIME type of the linked resource (text/css), and the ***href*** attribute gives the URL address of the css file.

C.3 XML

The **Extensible Markup Language** (**XML**) is a language that allows users to define a representation of data or a data structure and assign values to each field in the structure.

In other words, XML is a language that allows us to define mark-up elements (our own tags and our own document structure) and create customized markup language. The only restriction is that we need to follow the rules defined in XML. For example, the following shows how we can define a student record with three fields: *name*, *id*, and *birthday*.

```
<?xml version = "1.0"?>
    <student>
        <name> George Brown </name>
        <id> 2345 </id>
        <birthday> 12- 08 - 82 <birthday>
    <student>
```

This is similar to a struct or class in languages like C, C++, or Java.

C.4 XSL

The data defined and initialized to values in an XML document needs another language, a style language, to indicate how the document should be displayed. The **Extensible Style Language** (**XSL**) is the style language of XML, just as CSS is the style language of HTML and XHTML.

APPENDIX D

A Touch of Probability

Probability theory plays a very important role in data communications and networking because this theory is the best way of quantizing uncertainty and the field of data communication is full of uncertainty. For example, when we send a frame, we are not sure how much of it will arrive to the destination uncorrupted. Also when a station tries to access the network, we are not certain how successful it will be.

This appendix is just a review of basic concepts of probability theory that are needed to understand some topics discussed in this book.

D.1 DEFINITION

Although many definitions have been defined for probability, we use the classical one, which is close to our purpose.

The probability of an event A is a number, P [A], that is interpreted as

$$P[A] = N_A / N$$

where N is the total number of possible outcomes (also referred to as the sample space) and N_A is the number of possible outcomes related to event A.

Example D.1

We flip a coin. What is the probability of a head?

Solution

The total number of outcomes is 2 (*head* or *tail*). The number of possible outcomes related to this event is 1 (only *head*). Therefore, we have

$$\mathbf{P[head] = N_{head} / N = 1 / 2}$$

Example D.2

We roll a dice. What is the probability of getting a 5?

Solution

The total number of outcomes is 6 (1, 2, 3, 4, 5, 6). The number of possible outcomes related to this event is 1 (only 5). Therefore, we have

$$\mathbf{P[5] = N_5 / N = 1 / 6}$$

Example D.3

We flip two coins. What is the probability of getting two heads?

Solution

The total number of outcomes is 4 (head-head, head-tail, tail-head, or tail-tail). The number of possible outcomes related to this event is 1 (head-head). Therefore, we have

$$\mathbf{P[head\text{-}head] = N_{head\text{-}head} / N = 1 / 4}$$

D.2 AXIOMS AND PROPERTIES

To be able to use probability theory, we need axioms and properties.

D.2.1 Axioms

To find the probabilities of events, we accept some axioms. Axioms cannot be proved, but they are assumed. The following three axioms are fundamental to probability theory.

Axiom 1

This axiom states that the probability of an event is a non-negative value:

$$\mathbf{P\,[A] \geq 0}$$

Axiom 2

This axiom states that the probability of the sample space is 1. In other words, the probability that one of the possible outcomes occurs is 1:

$$\mathbf{P\,[S] = 1}$$

Axiom 3

This axiom states that if A1, A2, A3,... are disjoint events (the occurrence of one, does not change the probability of the occurrence of the others), then

$$\mathbf{P\,[A1 \text{ or } A2 \text{ or } A3 \text{ or...}] = P[A1] + P[A2] + P[A3] + \cdots}$$

D.2.2 Properties

Accepting the above axioms, a list of properties can be proven. The following is the minimum number of properties we need to understand the rest of this discussion (we leave the proofs to the books on probability):

Property 1

If A is an event and A′ is the complement of that event, then we have

$$\mathbf{P\,[A] = 1 - P[A']}$$

For example, if the probability of getting a 2 when rolling a dice is 1/6, the probability of not getting a 2 is 1 – 1/6 or 5/6.

Property 2

We always have an outcome or

$$\mathbf{P\,[\text{no outcome}] = 0}$$

In other words, if we roll a dice, the probability that none of the numbers shows is 0; that is, a number always shows.

Property 3

If an event is a subset of another event, the probability of the first event is less than or equal to the probability of the second event.

$$\mathbf{\text{If A is a subset of B, then } P[A] \leq P[B]}$$

For example, the probability of getting a 2 or 3 in a roll of a dice, P [2 or 3], is less than the probability of getting 2, 3, or 4, P [2 or 3 or 4].

Property 4

The probability of an event is always between 0 and 1.

$$0 \leq P[A] \leq 1$$

Property 5

If A, B, C,... are independent events, then

$$P[\text{A and B and C and } \ldots] = P[A] \times P[B] \times P[C] \times \cdots$$

If the events are independent (occurrence of the one does not change the probability of the occurrence of the others), then the probability of all events happening together is the product of their probabilities.

D.3 REPEATED TRIALS

So far, we have concentrated on the probability of events in a single trial such as flipping one coin. We are also interested in the probability of events when there is more than one trial. For example, what is the probability of getting one *head* if we flip a coin 10 times? What is the probability of getting five *heads* if we flip the coin 20 times?

Example D.4

Let us find the probability of getting exactly one *head* when we flip a biased coin three times. Assume that the probability of getting a head is p, which means that the probability of not getting a head (getting a tail) is $1 - p$.

Solution

We can get one head either in the first trial, the second trial, or the third trial. However, if a head comes up in one of the trials, it must not come up in the other two trials. We can therefore say that

$$\begin{aligned}&P[\text{only one head in three trials}] = \\ &P[\text{a head in first trial}] \times P[\text{a tail in second trial}] \times P[\text{a tail in third trial}] + \\ &P[\text{a head in second trial}] \times P[\text{a tail in first trial}] \times P[\text{a tail in third trial}] + \\ &P[\text{a head in third trial}] \times P[\text{a tail in first trial}] \times P[\text{a tail in second trial}]\end{aligned}$$

Using the probabilities, we get

$$\begin{aligned}&P[\text{only one head in three trials}] = \\ &p \times (1-p) \times (1-p) + p \times (1-p) \times (1-p) + p \times (1-p) \times (1-p)\end{aligned}$$

Or we can say that

$$P[\text{only one head in three trials}] = 3p \times (1-p)^2$$

Example D.5

We send a small frame of 3 bits. If the probability of each that a bit changes in transmission is 0.10 and each bit is independent, what is the probability that exactly 1 bit changes?

Solution

This problem is similar to Example D.1. We can assume that sending a bit corresponds to flipping a biased coin; a bit can reach the destination with or without change. Therefore, we have

$$\mathbf{P\ [exactly\ 1\ bit\ is\ changed]} = 3\,p \times (1-p)^2 = 3\,(0.1)\,(1-0.1)^2 = 0.243$$

D.3.1 Bernoulli Trials

Bernoulli found the probability of k successful occurrences in n trials. Assuming the probability of success is p and the probability of failure is q (or $1-p$), then

$$\mathbf{P\ [A\ successful\ event\ occurs}\ k\ \mathbf{times\ in}\ n\ \mathbf{trials]} = \mathbf{C}\ (n, k)\ p^k\ q^{n-k}$$

where C (n, k) is the combination of n objects k at a time; its value is

$$\mathbf{C}\ (n, k) = (n!) / [k!\ (n-k)!]$$

Examples D.1 and D.2 are cases of Bernoulli trials with $n = 3$ and $k = 1$.

Example D.6

We send a small frame of 10 bits. If the probability that a bit changes in transmission is 10 percent (0.1) and each bit is independent, what is the probability that exactly 3 bits change?

Solution

Using the result of Bernoulli trials, we can find the probability of 3 bits changing as

$$\mathbf{P\ [exactly\ 3\ bits\ changed]} = \mathbf{C}\ (10, 3)\ p^3 \times (1-p)^7 = 0.057$$

or 5.7 percent, which is much less than the probability of 1 bit being changed.

Example D.7

In Example D.3, what is the probability of no bits changing?

Solution

We can find the probability of no changes

$$\mathbf{P\ [no\ changes]} = \mathbf{C}\ (10, 0)\ p^0 \times (1-p)^{10} = 0.346$$

or 34.6 percent. Note that C (10, 0) here is just 1.

Example D.8

Let us assume that we have a CSMA/CD network with n stations. For any time slot, the probability that a station has a frame to send is p. Now the question is, "What is the probability that a time slot is used successfully (with no collision)?" This probability is important for calculating the efficiency of the network.

Solution

For a successful time slot, one and only one station has a frame to send; all others do not. However, this station can be any of the n stations. This problem is similar to flipping a coin and getting exactly one *head* in n trials. Think of each station as a trial and a successful slot as a head. The probability can be calculated by adding the probability that the first station has a frame to send with the probability that the others do not, $p\ (1-p)^{n-1}$, or the second station has a frame to send with the probability that the others do not, $p\ (1-p)^{n-1}$, and so on. We can use the Bernoulli formula to calculate one success out of n trials.

$$\mathbf{P\ [successful\ slot]} = \mathbf{C}\ (n, 1)\ p\ (1-p)^{n-1} = n\,p\,(1-p)^{n-1}$$

APPENDIX E

Checksum

An error-detection method, which is prevalent in TCP/IP protocol suite is the checksum. We discuss this method in this appendix.

E.1 TRADITIONAL CHECKSUM

Let us first discuss the traditional checksum that has been used in the Internet. We later show some new methods that are different from the traditional one.

E.1.1 Idea

The idea of the traditional checksum is very simple. We show this idea using a simple example.

Example E.1

Suppose our data is a list of five 4-bit numbers that we want to send to a destination. In addition to sending these numbers, we send the sum of the numbers. For example, if the set of numbers is (7, 11, 12, 0, 6), we send (7, 11, 12, 0, 6, **36**), where 36 is the sum of the original numbers. The receiver adds the five numbers and compares the result with the sum. If the two are the same, the receiver assumes no error, accepts the five numbers, and discards the sum. Otherwise, there is an error somewhere and the data are not accepted.

One's Complement Addition

Example E.1 has one major drawback. Each number can be written as a 4-bit word (each is less than 15) except for the sum. One solution is to use **one's complement** arithmetic. In this arithmetic, we can represent unsigned numbers between 0 and $2^n - 1$ using only n bits. If the number has more than n bits, the extra leftmost bits need to be added to the n rightmost bits (wrapping).

Example E.2

In the previous example, the decimal number 36 in binary is $(100100)_2$. To change it to a 4-bit number we add the extra two left-most bits to the right four bits as show below:

$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

Instead of sending 36 as the sum, we can send 6 as the sum (7, 11, 12, 0, 6, **6**). The receiver can add the first five numbers in one's complement arithmetic. If the result is 6, the numbers are accepted; otherwise, they are rejected.

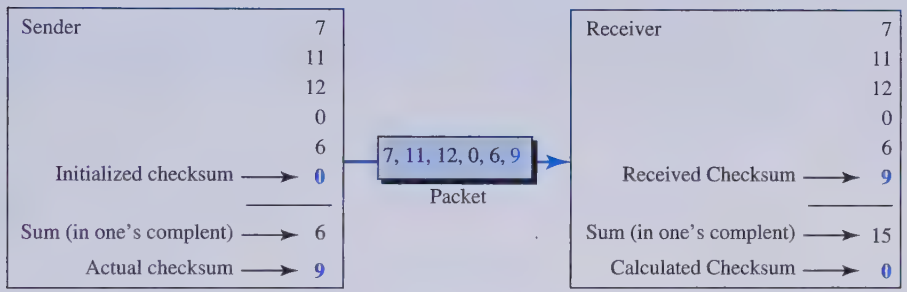
Checksum

We can make the job of the receiver easier if we send the complement of the sum, called the *checksum*. In one's complement arithmetic, the complement of a number is found by completing all bits (changing all 1s to 0s and all 0s to 1s). This is the same as subtracting the number from $2^n - 1$. In one's complement arithmetic, we have two 0s: positive and negative, which are complements of each other. The positive zero has all n bits set to 0; the negative zero has all bits set to 1 (it is $2^n - 1$). If we add a number with its complement, we get a negative zero (a number with all bits set to 1). When the receiver adds all five numbers (including the checksum), it gets a negative zero. The receiver can complement the result again to get a positive zero.

Example E.3

Let us use the idea of checksum in Example E.2. The sender adds all five numbers in one's complement to get the sum = 6. The sender then complements the result to get the checksum = 9, which is 15 − 6. Note that $6 = (0110)_2$ and $9 = (1001)_2$; they are complements of each other. The sender sends the five data numbers and the checksum (7, 11, 12, 0, 6, **9**). If there is no corruption in transmission, the receiver receives (7, 11, 12, 0, 6, **9**) and adds them in one's complement to get 15. The sender complements 15 to get 0. This shows that data have not been corrupted. Figure E.1 shows the process.

Figure E.1 *Process*



E.1.2 Internet Checksum

Traditionally, the Internet has used a 16-bit checksum. The sender and the receiver follow the steps depicted in Table E.1. The sender uses five steps, but the receiver uses only four.

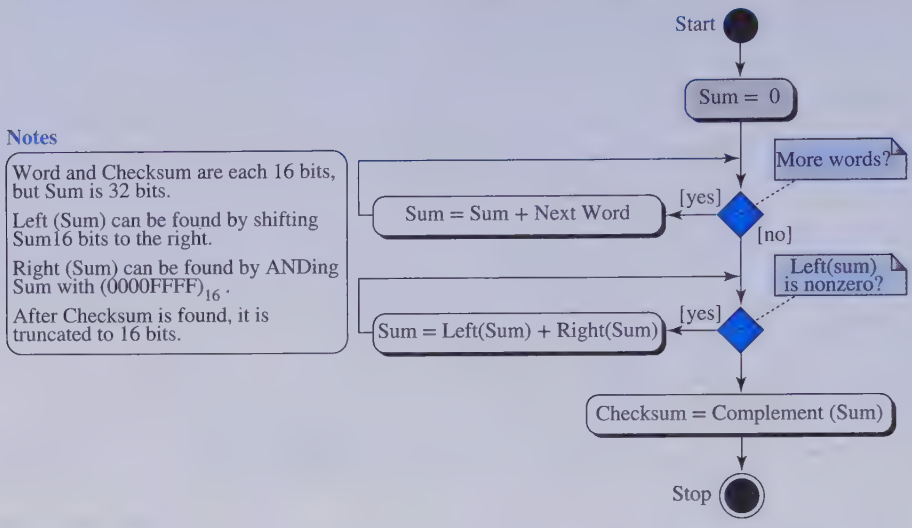
Table E.1 *Procedure to Calculate the Traditional Checksum*

Sender	*Receiver*
1. The message is divided into 16-bit words.	1. The message is divided into 16-bit words.
2. The value of the checksum word is initially set to zero.	2. All words are added using one's complement addition.
3. All words including the checksum are added using one's complement addition.	3. The sum is complemented and becomes the new checksum.
4. The sum is complemented and becomes the checksum.	4. If the value of checksum is 0, the message is accepted; otherwise, it is rejected.
5. The checksum is sent with the data.	

Algorithm

We can use the flow diagram of Figure E.2 to show the algorithm for calculation of the checksum. A program in any language can be easily written based on the algorithm.

Figure E.2 *Algorithm to calculate traditional checksum*



Performance

The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits). However, it is not as strong as the cyclic redundancy check (CRC) in error-checking capability. For example, if the value of one word is incremented and the value of another word is decremented by the same amount, the two errors cannot be detected because the sum and checksum remain the same. Also if the values of several words are incremented but the sum and the checksum do not change, the errors are not detected. Fletcher and Adler have proposed some weighted checksums, in which each word is multiplied by a number (its weight) that is related to its position in the text. This will eliminate the first problem we mentioned. However, the tendency in the Internet, particularly in designing new protocols, is to replace the checksum with a CRC.

E.2 FLETCHER

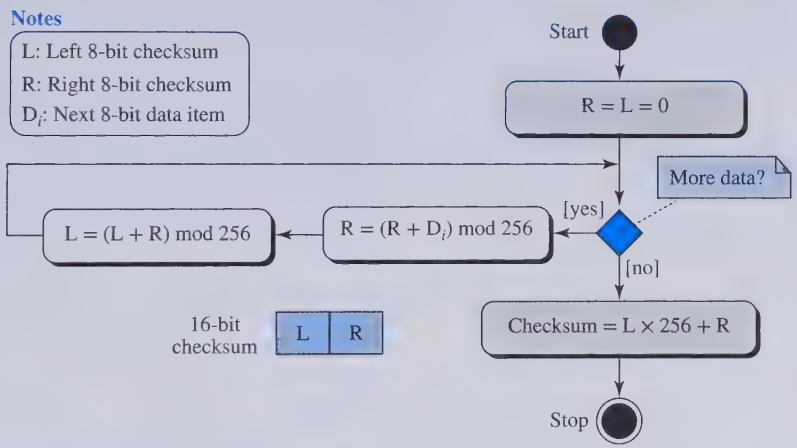
As mentioned before, there is one major problem with the traditional checksum calculation. If two 16-bit items are transposed in transmission, the checksum cannot catch this error. The reason is that the traditional checksum is not weighted: it treats each data item equally. In other words, the order of data item is immaterial to the calculation. The Fletcher checksum was devised to weight each data item according to its position.

Fletcher has proposed two algorithms: 8-bit and 16-bit. The first, 8-bit Fletcher, calculates on 8-bit data items and creates a 16-bit checksum. The second, 16-bit Fletcher, calculates on 16-bit data items and creates a 32-bit checksum.

Eight-Bit Fletcher

The 8-bit Fletcher is calculated over data octets (bytes) and creates a 16-bit checksum. The calculation is done modulo 256 (2^8), which means the intermediate results are divided by 256 and the remainder is kept. The algorithm uses two accumulators, L and R. The first simply adds data items together; the second adds a weight to the calculation. There are many variations of the 8-bit Fletcher algorithm; we show a simple one in Figure E.3.

Figure E.3 *Algorithm to calculate an 8-bit Fletcher checksum*



It can be proved that the accumulator L is a weighted sum of the data items. We have

$$R = D_1 + D_2 + \cdots + D_n$$

$$L = nD_1 + (n-1)D_2 + \cdots + D_n$$

If, for example, D_1 and D_2 are swapped during the transmission, the calculation of L at the receiver is different from the one done at the sender.

As an example, let us calculate the eight-bit Fletcher checksum for the string "Forouzan". We change each character to its equivalent ASCII value and calculate the values of R and L in Table E.2. Note that calculation is done modulo 255, which means if a sum is greater than 256, the number 256 is divided by 256 and only the remainder is used in the calculation.

Table E.2 *Example of an 8-bit Fletcher Checksum*

Byte	D_i	*R = 0*	*L = 0*
F	70	R = 0 + 70 = 70	L = 0 + 70 = 70
o	111	R = 70 +111 = 181	L = 70 + 181 = 251
r	114	R = 181 + 114 = 39	L = 251 + 39 = 34
o	111	R = 39 + 111 = 150	L = 34 + 150 = 184
u	117	R = 150 + 117 = 11	L = 184 + 11 = 195
z	122	R = 11 + 122 = 133	L = 195 + 133 = 72
a	97	R = 133 + 97 = 230	L = 72 + 230 = 46
n	110	R = 230 + 110 = 84	L = 46 + 84 = 130
			Checksum = L × 256 + R = 33,364

The 16-bit checksum in this case is $(8254)_{16}$. Note that the checksum is actually the concatenation of $L = (82)_{16}$ and $R = (54)_{16}$. In other words, when R and L are calculated, L goes to the leftmost byte and R to the rightmost byte.

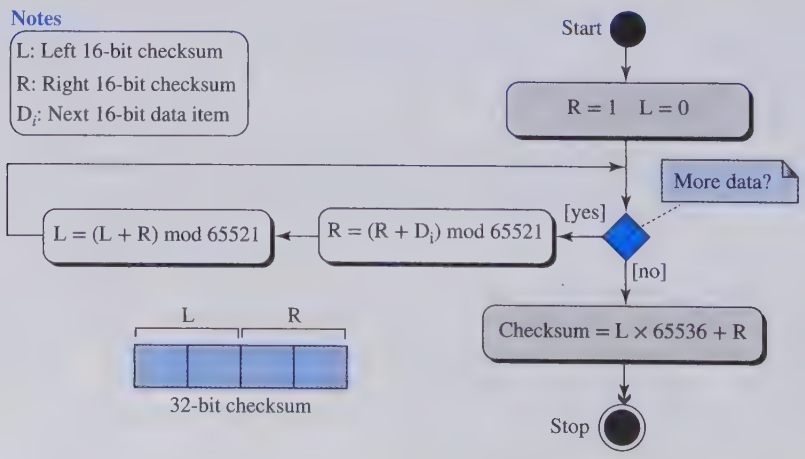
Sixteen-Bit Fletcher

The 16-bit Fletcher checksum is calculated over 16-bit data items and creates a 32-bit checksum. The calculation is done modulo 65536.

E.3 ADLER

The Adler checksum is a 32-bit checksum. Figure E.4 shows a simple algorithm in flowchart form.

Figure E.4 *Algorithm for calculating Adler checksum*



It is similar to the 16-bit Fletcher with three differences. First, calculation is done on single bytes instead of 2 bytes at a time. Second, the modulus is a prime number (65,521) instead of 65,536. Third, L is initialized to 1 instead of 0. It has been proved that a prime modulo has a better detecting capability in some combinations of data.

Let us calculate the Adler checksum for the string "Forouzan". We change each character to its equivalent ASCII value and calculate the values of R and L in Table E.3. The 32-bit checksum in this case is $(0E8A0355)_{16}$. Note that the checksum is actually the concatenation of $L = (0E8A)_{16}$ and $R = (0355)_{16}$.

Table E.3 *Example of Adler Checksum*

Byte	D_i	$R = 1$	$L = 0$
F	70	R = 1 + 70 = 71	L = 0 + 71 = 71
o	111	R = 71 + 111 = 182	L = 71 + 182 = 253
r	114	R = 182 + 114 = 296	L = 253 + 296 = 549
o	111	R = 296 + 111 = 407	L = 549 + 407 = 956
u	117	R = 407 + 117 = 524	L = 956 + 524 = 1,480
z	122	R = 524 + 122 = 646	L = 1480 + 646 = 2,126
a	97	R = 646 + 97 = 743	L = 2126 + 743= 2,869
n	110	R = 743 + 110 = 853	L = 2869 + 853 = 3,722
			Checksum = 3,722 × **65,536** + 853 = 243,925,845

APPENDIX F

Acronyms

2B1Q	two-binary, one-quaternary
4B/5B	four binary, five binary
4D-PAM5	4-dimensional, 5-level pulse amplitude modulation
8B/10B	eight binary, ten binary
8B/6T	eight binary, six ternary
AAL	(ATM) application adaptation layer
AAS	adaptive antenna system
ABM	asynchronous balanced mode
ABR	available bit rate
ACK	acknowledgment
ACL	asynchronous connectionless link
ADM	adaptive DM
ADPCM	adaptive DPCM
ADSL	asymmetric digital subscriber line
AES	Advanced Encryption Standard
AH	authentication header
AIMD	additive increase, multiplicative decrease
AM	amplitude modulation
AMI	alternate mark inversion
AMPS	Advanced Mobile Phone System
ANSI	American National Standards Institute
ANSNET	Advanced Networks and Services Network
AP	access point
API	application programming interface
APS	automatic protection switching
ARP	Address Resolution Protocol
ARPA	Advanced Research Projects Agency
ARPANET	Advanced Research Projects Agency Network
ARQ	automatic repeat request

AS	authentication server
AS	autonomous system
ASCII	American Standard Code for Information Interchange
ASK	amplitude shift keying
ASN.1	Abstract Syntax Notation One
ATM	Asynchronous Transfer Mode
AUI	attachment unit interface
B-frame	bidirectional frame
BECN	backward explicit congestion notification
BER	Basic Encoding Rules
BGP	Border Gateway Protocol
BOOTP	Bootstrap Protocol
BRI	basic rate interface
BSS	basic service set
CA	Certification Authority
CATV	community antenna TV
CBC	cipher-block chaining
CBR	constant bit rate
CBT	Core-Based Tree
CCITT	Consultative Committee for International Telegraphy and Telephony
CCK	complementary code keying
CDMA	code division multiple access
CDPD	cellular digital packet data
CDV	cell delay variation
CGI	common gateway interface
CHAP	Challenge Handshake Authentication Protocol
CIDR	Classless Interdomain Routing
CIR	committed information rate
CLP	cell loss priority
CLR	cell loss ratio
CMS	Cryptographic Message Syntax
CMTS	cable modem transmission system
CPE	customer premises equipment
CRC	cyclic redundancy check
CS	convergence sublayer
CSM	cipher stream mode
CSMA	carrier sense multiple access
CSMA/CA	carrier sense multiple access with collision avoidance
CSMA/CD	carrier sense multiple access with collision detection
CSNET	Computer Science Network

CSRC	contributing source
CTS	clear to send
D-AMPS	digital AMPS
DARPA	Defense Advanced Research Projects Agency
dB	decibel
DC	direct current
DCF	distributed coordination function
DCT	discrete cosine transform
DDNS	Dynamic Domain Name System
DDS	digital data service
DE	discard eligibility
DEMUX	demultiplexer
DES	Data Encryption Standard
DHCP	Dynamic Host Configuration Protocol
DHT	distributed hash table
DiffServ	Differentiated Services
DIFS	distributed interframe space
DISC	disconnect
DNS	Domain Name System
DPCM	differential PCM
DS	Differentiated Services
DSL	digital subscriber line
DSLAM	digital subscriber line access multiplexer
DSS	Digital Signature Standard
DSSS	direct sequence spread spectrum
DTE	data terminal equipment
DVMRP	Distance Vector Multicast Routing Protocol
DWDM	dense wave-division multiplexing
EBCDIC	extended binary coded decimal interchange code
ECB	electronic codebook
EGP	Exterior Gateway Protocol
EIA	Electronic Industries Alliance
ENQ	enquiry frame
ESP	Encapsulating Security Payload
ESS	extended service set
FA	foreign agent
FCC	Federal Communications Commission
FCS	frame check sequence
FDD	frequency division duplex
FDDI	Fiber Distributed Data Interface

FDM	frequency-division multiplexing
FDMA	frequency-division multiple access
FEC	forward error correction
FHSS	frequency-hopping spread spectrum
FIFO	first-in, first-out
FM	frequency modulation
FRMR	frame reject
FQDN	fully qualified domain name
FSK	frequency shift keying
FTP	File Transfer Protocol
GEO	geostationary Earth orbit
GIF	graphical interchange format
GPS	Global Positioning System
GSM	Global System for Mobile Communication
HA	home agent
HDLC	High-level Data-Link Control
HDSL	high bit rate digital subscriber line
HEC	header error check
HFC	hybrid-fiber-coaxial
HMAC	hashed MAC (hashed message authentication code)
HR-DSSS	high-rate direct-sequence spread spectrum
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
Hz	hertz
I-frame	inter-coded frame
IAB	Internet Architecture Board
IANA	Internet Assigned Numbers Authority
iBGP	internal BGP
ICANN	Internet Corporation for Assigned Names and Numbers
ICMP	Internet Control Message Protocol
IEEE	Institute of Electrical and Electronics Engineers
IESG	Internet Engineering Steering Group
IETF	Internet Engineering Task Force
IFDMA	interleaved FDMA
IFS	interframe space
IGMP	Internet Group Management Protocol
IGP	Interior Gateway Protocol
IKE	Internet Key Exchange
ILEC	incumbent local exchange carrier
IMAP	Internet Mail Access Protocol

INTERNIC	Internet Network Information Center
IntServ	Integrated Services
IP	Internet Protocol
IPCP	Internetwork Protocol Control Protocol
IPng	Internet Protocol, new generation
IPSec	IP Security
IPv6	Internet Protocol, version 6
IRTF	Internet Research Task Force
IS-95	Interim Standard 95
ISAKMP	Internet Security Association and Key Management Protocol
ISDN	Integrated Services Digital Network
ISN	initial sequence number
ISO	International Organization for Standardization
ISOC	Internet Society
ISP	Internet service provider
ITM-2000	Internet Mobile Communication 2000
ITU	International Telecommunications Union
ITU-T	ITU, Telecommunication Standardization Sector
IV	initial vector
JPEG	Joint Photographic Experts Group
KDC	key-distribution center
L2CAP	Logical Link Control and Adaptation Protocol
LAN	local area network
LAP	line access procedure
LCP	Link Control Protocol
LEO	low-Earth-orbit
LIS	logical IP subnet
LLC	logical link control
LMI	local management information
LMP	Link Management Protocol
LPC	linear predictive coding
LSA	link-state advertisement
LSP	link-state packet
MA	multiple access
MAA	message access agent
MAC	media access control
MAC	message authentication code
MAN	metropolitan area network
MBONE	multicast backbone
MBS	maximum burst size

MC-CDMA	multicarrier CDMA
MD	Message Digest
MDC	modification detection code
MEO	medium-Earth-orbit
MH	mobile host
MIB	Management Information Base
MID	message identifier
MII	medium independent interface
MILNET	Military Network
MIME	Multipurpose Internet Mail Extensions
MIMO	multiple-input, multiple-output antenna
MLT-3	multiline transmission, 3-level
modem	modulator-demodulator
MOSPF	Multicast Open Shortest Path First
MP3	MPEG audio layer 3
MPEG	Motion Picture Experts Group
MPLS	multiprotocol label switching
MSC	mobile switching center
MSS	maximum segment size
MTA	message transfer agent
MTSO	mobile telephone switching office
MTU	maximum transfer unit
MUX	multiplexer
NAK	negative acknowledgment
NAP	network access point
NAT	Network Address Translation
NAV	network allocation vector
NCP	Network Control Protocol
NIC	Network Information Center
NIC	network interface card
NIST	National Institute of Standards and Technology
NNI	network-to-network interface
NRM	normal response mode
NRZ	nonreturn-to-zero
NRZ-I	nonreturn-to-zero, invert
NRZ-L	nonreturn-to-zero, level
NSA	National Security Agency
NSF	National Science Foundation
NSFNET	National Science Foundation Network
NVT	network virtual terminal

OADM	optical add-drop multiplexer
OC	optical carrier
OFB	output feedback
OSI	Open Systems Interconnection
OSPF	Open Shortest Path First
P/F	poll/final
P2P	peer-to-peer
PAM	pulse amplitude modulation
PAP	Password Authentication Protocol
PC	predictive coding
PCF	point coordination function
PCM	pulse code modulation
PCS	personal communication system
PDU	protocol data unit
PGP	Pretty Good Privacy
PHB	per-hop behavior
PIM	Protocol Independent Multicast
PIM-DM	Protocol Independent Multicast-Dense Mode
PIM-SM	Protocol Independent Multicast-Sparse Mode
PKI	public key infrastructure
PM	phase modulation
PN	pseudorandom noise
PNNI	private network-to-network interface
POP	point of presence
POP3	Post Office Protocol, version 3
POS	packet over SONET
POTS	plain old telephone system
PPM	pulse position modulation
PPP	Point-to-Point Protocol
PQDN	partially qualified domain name
PSK	phase shift keying
PSTN	Public Switched Telephone Network
PVC	permanent virtual circuit
QAM	quadrature amplitude modulation
QoS	quality of service
REJ	reject
RFC	Request for Comment
RIP	Routing Information Protocol
rlogin	remote logging
RNR	Receive Not Ready

ROM	read-only memory
RP	rendezvous point
RPB	reverse path broadcasting
RPF	reverse path forwarding
RPM	reverse path multicasting
RSA	Rivest, Shamir, Adleman
RSVP	Resource Reservation Protocol
RTCP	Real-Time Transport Control Protocol
RTO	retransmission time-out
RTP	Real-Time Transport Protocol
RTS	request to send
RTSP	Real-Time Streaming Protocol
RTT	round-trip time
RZ	return-to-zero
S/MIME	Secure/Multipurpose Internet Mail Extensions
SA	Security Association
SAD	Security Association Database
SAR	segmentation and reassembly
SCCP	signaling connection control part
SCO	synchronous connection-oriented
SCP	server control point
SCTP	Stream Control Transmission Protocol
SDH	Synchronous Digital Hierarchy
SDR	Software Defined Radio
SDSL	symmetric digital subscriber line
SDU	service data unit
SEAL	simple and efficient adaptation layer
SFD	start frame delimiter
SHA	Secure Hash Algorithm
SIFS	short IFS (interframe space)
SIP	Session Initiation Protocol
SKEME	Secure Key Exchange Mechanism
SMI	Structure of Management Information
SMTP	Simple Mail Transfer Protocol
SNMP	Simple Network Management Protocol
SNR	signal-to-noise ratio
SOFDMA	Scalable OFDMA
SONET	Synchronous Optical Network
SP	Security Policy
SP	Simple Protocol

SPD	Security Policy Database
SPE	synchronous payload envelope
SPI	security parameter index
SR	selective-repeat
SREJ	selective reject
SS	spread spectrum
SS7	Signaling System Seven
SSCS	service specific convergence sublayer
SSH	Secure Shell
SSL	Secure Sockets Layer
SSN	stream sequence number
SSRC	synchronization source
STM	synchronous transport module
STP	shielded twisted-pair
STS	synchronous transport signal
SVC	switched virtual circuit
TCAP	transaction capabilities application port
TCB	transmission control block
TCP	Transmission Control Protocol
TCP/IP	Transmission Control Protocol/Internet Protocol
TDD	time-division duplex
TDM	time-division multiplexing
TDMA	time-division multiple access
TELNET	Terminal Network
TFTP	Trivial File Transfer Protocol
TLI	transport-layer interface
TLS	Transport Layer Security
TOS	type of service
TP	transmission path
TRPB	truncated reverse-path broadcasting
TSI	time-slot interchange
TSN	transmission sequence number
TTL	time to live
TUP	telephone user port
UA	user agent
UBR	unspecified bit rate
UDP	User Datagram Protocol
UMTS	Universal Mobile Telecommunication System
UNI	user-to-network interface
URL	uniform resource locator

UTP	unshielded twisted-pair
VBR	variable bit rate
VC	virtual circuit
VCC	virtual circuit connection
VCI	virtual circuit identifier
VDSL	very high bit rate digital subscriber line
VLAN	virtual local area network
VOIP	voice over IP
VP	virtual path
VPI	virtual path identifier
VPN	virtual private network
VT	virtual tributary
WAN	wide area network
WDM	wavelength-division multiplexing
WiMAX	Worldwide Interoperability for Microwave Access
WWW	World Wide Web
XHTML	Extensible Hypertext Markup Language
XML	Extensible Markup Language
XSL	Extensible Style Language

GLOSSARY

10-Gigabit Ethernet The new implementation of Ethernet operating at 10 Gbps.

4-dimensional, 5-level pulse amplitude modulation (4D-PAM5) An encoding scheme used by 1000Base-T.

56K modem A modem technology using two different data rates: one for uploading and one for downloading from the Internet.

56K modem A modem technology using two different data rates: one for uploading and one for downloading from the Internet.

800 Service A telephone service in which the call is free for the *caller*, but it is paid by the *callee*.

900 Service A telephone service in which the call is paid by the *caller* and is normally much more expensive than a normal long-distance call. The reason is that the carrier charges *two* fees: the first is the long-distance toll, and the second is the fee paid to the callee for each call.

Abstract Syntax Notation One (ASN.1) A formal language using abstract syntax for defining the structure of a protocol data unit (PDU).

access point (AP) A central base station in a BSS.

acknowledgment (ACK) A response sent by the receiver to indicate the successful receipt of data.

acknowledgment number In TCP, the number in the acknowledgment field that defines the sequence number of the next byte expected.

active document In the World Wide Web, a document executed at the local site.

adaptive antenna system (AAS) A system that uses multiple antennas on both terminal and base station to increase performance.

adaptive delta modulation (ADM) A delta modulation technique in which the value of delta is adjusted in each step.

adaptive DPCM (ADPCM) A DPCM method in which the value of delta is adjusted in each step.

add/drop multiplexer A SONET device that removes and inserts signals in a path without demultiplexing and re-multiplexing.

additive cipher The simplest monoalphabetic cipher in which each character is encrypted by adding its value with a key.

additive increase In TCP, a congestion control strategy in which the window size is increased by just one segment instead of exponentially.

additive increase, multiplicative decrease (AIMD) Combination of additive increase and multiplicative decrease congestion control methods used in TCP.

address aggregation A mechanism in which the blocks of addresses for several organizations are aggregated into one larger block.

Address Resolution Protocol (ARP) In TCP/IP, a protocol for obtaining the link-layer address of a node when the Internet address is known.

address space The total number of addresses used by a protocol.

ad hoc network A self-configuring network connected by wireless link.

Advanced Encryption Standard (AES) An asynchronous block cipher adapted by NIST to replace DES.

Advanced Mobile Phone System (AMPS) A North American analog cellular phone system using FDMA.

Advanced Network and Services (ANS) A nonprofit organization created by IBM, Merit, and MCI to build a high-speed backbone.

Advanced Network Services NET (ANSNET) The network created by ANS.

Advanced Research Projects Agency (ARPA) The government agency that funded ARPANET. The same agency later funded global Internet.

Advanced Research Projects Agency Network (ARPANET) The packet switching network that was funded by ARPA. It was used for internetworking research.

ALOHA The original random multiple access method in which a station can send a frame any time it has one to send (MA).

alternate mark inversion (AMI) A digital-to-digital bipolar encoding method in which the amplitude representing 1 alternates between positive and negative voltages.

American National Standards Institute (ANSI) A national standards organization that defines standards in the United States.

American Standard Code for Information Interchange (ASCII) A character code developed by ANSI and used extensively for data communication.

amplification When a signal travels through a medium, it loses some of its energy in overcoming the resistance of the medium. To compensate for this loss, we need amplification.

amplitude The strength of a signal, usually measured in volts.

amplitude modulation (AM) An analog-to-analog conversion method in which the carrier signal's amplitude varies with the amplitude of the modulating signal.

amplitude shift keying (ASK) A digital-to-analog conversion method in which the amplitude of the carrier signal is varied to represent binary 0 or 1.

analog data Data that are continuous and smooth and not limited to a specific number of values.

analog leased service A service that offers customers the opportunity to lease a line, sometimes called a *dedicated line,* that is permanently connected to another customer.

analog signal A continuous waveform that changes smoothly over time.

analog switched service The the digital version of an analog switched line. It is a switched digital service that allows data rates of up to 56 kbps.

analog-to-analog conversion The representation of analog information by an analog signal.

analog-to-digital conversion The representation of analog information by a digital signal.

angle of incidence In optics, the angle formed by a light ray approaching the interface between two media and the line perpendicular to the interface.

anycast address An address that defines a group of computers in which the message is sent to the first member in the group.

aperiodic signal A signal that does not exhibit a pattern or repeating cycle.

applet A computer program for creating an active Web document. It is usually written in Java.

application adaptation layer (AAL) A layer in ATM protocol that breaks user data into 48-byte payloads.

application gateway In a proxy firewall, the computer that stands between the customer computer and corporation computer.

application layer The fifth layer in the Internet model; provides access to network resources.

application programming interface (API) A set of declarations, definitions, and procedures followed by programmers to write client-server programs.

area A collection of networks, hosts, and routers all contained within an autonomous system.

arithmetic coding A lossless coding in which the entire message is mapped to a small interval within the interval [0, 1). The small interval is then encoded as a binary pattern.

association A connection in SCTP.

asymmetric DSL (ADSL) A service that provides higher speed (bit rate) in the downstream direction (from the Internet to the resident) than in the upstream direction (from the resident to the Internet).

asymmetric-key cipher A cipher using an asymmetric-key cryptosystem.

asynchronous balanced mode (ABM) In HDLC, a communication mode in which each station can be either primary or secondary.

asynchronous connectionless link (ACL) A link between a Bluetooth master and slave in which a corrupted payload is retransmitted.

Asynchronous Transfer Mode (ATM) A wide area protocol featuring high data rates and equal-sized packets (cells); ATM is suitable for transferring text, audio, and video data.

ATM adaptation layer (AAL) The layer in the ATM protocol that encapsulates the user data.

ATM layer A layer in ATM that provides routing, traffic management, switching, and multiplexing services.

attenuation The loss of a signal's energy due to the resistance of the medium.

audio Recording or transmitting of sound or music.

authentication Verification of the sender of a message.

Authentication Header (AH) Protocol A protocol defined by IPSec at the network layer that provides integrity to a message through the creation of a digital signature by a hashing function.

autoconfiguration A feature in IPv6 that be used to allocate an IPv6 address to a host, but a host can also configure it itself.

autokey cipher A stream cipher in which each subkey in the stream is the same as the previous plaintext character. The first subkey is the secret between two parties.

automatic repeat request (ARQ) An error-control method in which correction is made by retransmission of data.

autonegotiation A Fast Ethernet feature that allows two devices to negotiate the mode or data rate.

autonomous system (AS) A group of networks and routers under the authority of a single administration.

backbone One long cable link connected to all the devices in a network

Backus-Naur Form (BNF) A meta language that specifies which sequences of symbols form a valid term.

band-pass channel A channel that can pass a range of frequencies.

bandwidth The difference between the highest and the lowest frequencies of a composite signal. It also measures the information-carrying capacity of a line or a network.

bandwidth-delay product A measure of the number of bits that can be sent while waiting for news from the receiver.

banyan switch A multistage switch with microswitches at each stage that route the packets based on the output port, represented as a binary string.

Barker sequence A sequence of 11 bits used for spreading.

base header The header in the IP protocol that is available in each packet.

baseband transmission Transmission of digital or analog signal without modulation, using a low-pass channel.

baseline wandering In decoding a digital signal, the receiver calculates a running average of the received signal power. This average is called the baseline. A long string of 0s or 1s can cause a drift in the baseline (baseline wandering) and make it difficult for the receiver to decode correctly.

Basic Encoding Rules (BER) A standard that encodes data to be transferred through a network.

basic service set (BSS) The building block of a wireless LAN as defined by the IEEE 802.11 standard.

Batcher-banyan switch A banyan switch that sorts the arriving packets based on their destination port.

baud The number of signal elements transmitted per second. A signal element consists of one or more bits.

Bayone-Neill-Concelman (BNC) connector A common coaxial cable connector.

beacon frame In Point Coordination Function of Project 802.11, a frame that starts the repetition interval.

Bellman-Ford An algorithm used to calculate routing tables in the distance vector routing method.

bidirectional frame (B-frame) An MPEG frame that is related to the preceding and following I-frame or P-frame.

bipolar encoding A digital-to-digital encoding method in which 0 amplitude represents binary 0 and positive and negative amplitudes represent alternate 1s.

bipolar with 8-zero substitution (B8ZS) A scrambling technique in which a stream of 8 zeros is replaced by a predefined pattern to improve bit synchronization.

bit Binary digit. The smallest unit of data (0 or 1).

bit length The length of a bit in transfer in meter.

bit rate The number of bits transmitted per second.

bit stuffing In a bit-oriented protocol, the process of adding an extra bit in the data section of a frame to prevent a sequence of bits from looking like a flag.

bit-oriented protocol A protocol in which the data frame is interpreted as a sequence of bits.

block cipher A type of cipher in which blocks of plaintext are encrypted one at a time using the same cipher key.

block coding A coding method in which blocks of n bits are encoded into blocks of m bits where $m > n$.

blocking port A port that blocks the frames received by the switch.

Bluetooth A wireless LAN technology designed to connect devices of different functions such as telephones and notebooks in a small area such as a room.

Bootstrap Protocol (BOOTP) The protocol that provides configuration information from a table (file).

Border Gateway Protocol (BGP) An inter-autonomous system routing protocol based on path vector routing.

bridge A network device operating at the first two layers of the Internet model with filtering and forwarding capabilities.

broadband transmission Transmission of signals using modulation of a higher frequency signal. The term implies a wide-bandwidth data combined from different sources.

broadcast address An address that allows transmission of a message to all nodes of a network.

broadcast link A link in which each station receives a sent packet.

broadcasting Transmission of a message to all nodes in a network.

browser An application program that displays a WWW document. A browser usually uses other Internet services to access the document.

BSS-transition mobility In a wireless LAN, a station that can move from one BSS to another but is confined inside one ESS.

bucket brigade attack See *man-in-the middle attack.*

buffer Memory set aside for temporary storage.

burst error Error in a data unit in which two or more bits have been altered.

bursty data Data with varying instantaneous transmission rates.

bus topology A multi-point topology to serve as a backbone to connect all devices in a network.

byte A group of 8 bits. An octet.

byte oriented A text in which the unit of data is a byte.

byte-oriented protocol A protocol in which the data section of the frame is interpreted as a sequence of bytes (characters).

byte stuffing In a byte-oriented protocol, the process of adding an extra byte in the data section of a frame to prevent a byte from looking like a flag.

cable modem A technology in which the TV cable provides Internet access.

cable modem transmission system (CMTS) A device installed inside the distribution hub that receives data from the Internet and passes them to the combiner.

cable TV network A system using coaxial or fiber optic cable that brings multiple channels of video programs into homes.

caching The storing of information in a small, fast memory.

Caesar cipher A shift cipher used by Julius Caesar with the key value of 3.

care of address A temporary IP address used by a mobile host while visiting a foreign network.

carrier extension A feature that defines the minimum length of a frame as 512 bytes

carrier sense multiple access (CSMA) A contention access method in which each station listens to the line before transmitting data.

carrier sense multiple access with collision avoidance (CSMA/CA) An access method in wireless LANs that avoids collision by forcing the stations to send reservation messages when they find the channel is idle.

carrier sense multiple access with collision detection (CSMA/CD) An access method in which stations transmit whenever the transmission medium is available and retransmit when collision occurs.

carrier signal A high frequency signal used for digital-to-analog or analog-to-analog modulation. One of the characteristics of the carrier signal (amplitude, frequency, or phase) is changed according to the modulating data.

Cascading Style Sheets (CSS) A standard developed to use with HTML to define document presentation.

cell A small, fixed-size data unit; also, in cellular telephony, a geographical area served by a cell office.

cell network A network using the cell as its basic data unit.

cellular telephony A wireless communication technique in which an area is divided into cells. A cell is served by a transmitter.

Certification Authority (CA) An agency such as a federal or state organization that binds a public key to an entity and issues a certificate.

Challenge Handshake Authentication Protocol (CHAP) In PPP, a three-way handshaking protocol used for authentication.

challenge-response authentication An authentication method in which the claimant proves that she *knows* a secret without sending it.

channel A communications pathway.

channelization A multiple access method in which the available bandwidth of a link is shared in time.

character-oriented protocol See *byte-oriented protocol.*

checksum A value used for error detection. It is formed by adding data units using one's complement arithmetic and then complementing the result.

chip In CDMA, a number in a code that is assigned to a station.

choke point A packet sent by a router to the source to inform it of congestion.

Chord A P2P protocol that was published by Stoica *et al* in 2001 in which the identifier space is made of 2^m points distributed in a circle in the clockwise direction.

chunk A unit of transmission in SCTP.

cipher A decryption and/or encryption algorithm.

cipher suite A list of possible ciphers.

ciphertext The message after being encrypted.

circuit switching A switching technology that establishes an electrical connection between stations using a dedicated path.

circuit-switched network A network in which circuit-switching technology is used. A good example is the old telephone voice network.

cladding Glass or plastic surrounding the core of an optical fiber; the optical density of the cladding must be less than that of the core.

Clark's solution A solution to prevent the silly window syndrome. An acknowledgment is sent as soon as the data arrive, but announces a window size of zero until either there is enough space to accommodate a segment of maximum size or until half of the buffer is empty.

classful addressing An IPv4 addressing mechanism in which the IP address space is divided into five classes: A, B, C, D, and E. Each class occupies some part of the whole.

classless addressing The new IPv4 addressing that ignored the classes in the classful addressing to make a better use of address space.

Classless Interdomain Routing (CIDR) A technique to do routing using classless addressing.

client A program that can receive services from programs called servers.

client process A running application program on a local site that requests service from a running application program on a remote site.

client-server paradigm A paradigm in which two computers are connected by an internet and each must run a program, one to provide a service and one to request a service.

closed-loop congestion control A method to alleviate congestion after it happens.

coaxial cable A transmission medium consisting of a conducting core, insulating material, and a second conducting sheath.

code In data communication text is represented as a bit pattern. Different sets of bit patterns have been designed to represent text symbols. Each set is called a **code,** and the process of representing symbols is called coding.

code division multiple access (CDMA) A multiple access method in which one channel carries all transmissions simultaneously.

codeword The encoded dataword.

ColdFusion A dynamic web technology that allows the fusion of data items coming from a conventional database.

collision The event that occurs when two transmitters send at the same time on a channel designed for only one transmission at a time; data will be destroyed.

collocated care-of address The care-of address for a mobile host that is acting as a foreign agent.

colon hexadecimal notation In IPv6, an address notation consisting of 32 hexadecimal digits, with every four digits separated by a colon.

committed burst size (Bc) The maximum number of bits in a specific time period that a network must transfer without discarding any frames.

committed information rate (CIR) The committed burst size divided by time.

common carrier A transmission facility available to the public and subject to public utility regulation.

common gateway interface (CGI) A standard for communication between HTTP servers and executable programs. CGI is used in creating dynamic documents.

community antenna TV (CATV) A cable network service that broadcasts video signals to locations with poor or no reception.

compatible address An IPv6 address consisting of 96 bits of zero followed by 32 bits of IPv4.

competitive local exchange carrier (CLEC) The new carriers that can provide services to incumbent local exchange carrier (ILEC).

complementary code keying (CCK) An HR-DSSS encoding method that encodes four or eight bits into one symbol.

composite signal A signal composed of more than one sine wave.

Computer Science Network (CSNET) A network sponsored by the National Science Foundation, originally intended for universities.

concurrent server A server that serves multiple client simultaneously.

congestion Excessive network or internetwork traffic causing a general degradation of service.

congestion avoidance TCP defines an algorithm called **congestion avoidance,** which increases the *cwnd* additively instead of exponentially.

congestion avoidance algorithm An algorithm used by TCP that tries to avoid congestion by slowing down the transmission.

congestion control The mechanism of eliminating or avoiding congestion in a network.

connecting device A device that connects computers or networks.

connection establishment The preliminary setup necessary for a logical connection prior to actual data transfer.

connection-oriented concurrent server A connection-oriented server that can serve many clients at the same time.

connection-oriented service A service for data transfer involving establishment and termination of a connection.

connectionless iterative server A connectionless server that processes one request at a time.

connectionless service A service for data transfer without connection establishment or termination.

constant bit rate (CBR) The data rate of an ATM service class that is designed for customers requiring real-time audio or video services.

constellation diagram A graphical representation of the phase and amplitude of different bit combinations in digital-to-analog modulation.

Consultative Committee for International Telegraphy and Telephony (CCITT) An international standards group now known as the ITU-T.

contention An access method in which two or more devices try to transmit at the same time on the same channel.

contention window In CSMA/CA, an amount of time divided into slots. Each slot is randomly selected by a station for transmission.

controlled access A multiple access method in which the stations consult one another to determine who has the right to send.

convergence sublayer (CS) In ATM protocol, the upper AAL sublayer that adds a header or a trailer to the user data.

cookie A string of characters that holds some information about the client and must be returned to the server untouched.

core The glass or plastic center of an optical fiber.

Core-Based Tree (CBT) In multicasting, a group-shared protocol that uses a center router as the root of the tree.

country domain A subdomain in the Domain Name System that uses two characters as the last suffix.

critical angle In optics, the angle of incidence of a ray that changes its behavior from refraction to reflection.

crossbar switch A switch consisting of a lattice of horizontal and vertical paths. At the intersection of each horizontal and vertical path, there is a crosspoint that can connect the input to the output.

crosspoint The junction of an input and an output on a crossbar switch.

crosstalk The noise on a line caused by signals traveling along another line.

cryptographic hash function A function that creates a much shorter output from an input. To be useful, the function must be resistant to image, preimage, and collision attacks.

Cryptographic Message Syntax (CMS) The syntax used in S/MIME that defines the exact encoding scheme for each content type.

cryptography The science and art of transforming messages to make them secure and immune to attacks.

customer premises equipment (CPE) In WiMAX, the customer premises equipment, or subscriber unit is performing the same job as a modem in wired communication.

cyclic code A linear code in which the cyclic shifting (rotation) of each codeword creates another codeword.

cyclic redundancy check (CRC) A highly accurate error-detection method based on interpreting a pattern of bits as a polynomial.

data The word ***data*** refers to information presented in whatever form is agreed upon by the parties creating and using it.

data communication The exchange of data between two devices via some form of transmission medium such as a wire cable.

data element The smallest entity that can represent a piece of information. A bit.

Data Encryption Standard (DES) A symmetric-key block cipher using rounds of Feistel ciphers and standardized by NIST.

data-link control (DLC) The responsibilities of the data-link layer: flow control and error control.

data-link layer The second layer in the Internet model. It is responsible for node-to-node delivery.

data rate The number of data elements sent in one second.

data-transfer phase The intermediate phase in circuit-switched or virtual-circuit networks in which data transfer takes place.

data transparency The ability to send any bit pattern as data without it being mistaken for control bits.

datagram In packet switching, an independent data unit.

datagram network A packet-switched network in which packets are independent from each other.

dataword The smallest block of data in block coding.

DCF interframe space (DIFS) The period of time a station waits after it is idle before sending a request to send.

deadlock A situation in which a task cannot proceed because it is waiting for an event that will never occur.

decapsulation A process, inverse of encapsulation, that extracts the payload of packets.

decibel (dB) A measure of the relative strength of two signal points.

decryption Recovery of the original message from the encrypted data.

decryption algorithm Algorithm to descramble the ciphertext to create the original plaintext.

default routing A routing method in which a router is assigned to receive all packets with no match in the routing table.

Defense Advanced Research Projects Agency (DARPA) A government organization, which, under the name of ARPA, funded ARPANET and the Internet.

delta modulation An analog-to-digital conversion technique in which the value of the digital signal is based on the difference between the current and the previous sample values.

demodulation The process of separating the carrier signal from the information-bearing signal.

demodulator A device that demodulates a modulated signal to get the original signal.

demultiplexing Inverse of multiplexing. To get the original signal or data from a multiplexed signal or data.

denial of service The only attack on the availability goal that may slow down or interrupt the system.

dense wave-division multiplexing (DWDM) A WDM method that can multiplex a very large number of channels by spacing channels closer together.

destination address The address of the receiver of the data unit.

destination option When the source needs to pass information to the destination only (which means that intermediate routers are not permitted access to this information), the source uses the *destination option* in the datagram.

differential Manchester encoding A digital-to-digital polar encoding method that features a transition at the middle of the bit interval as well as an inversion at the beginning of each 1 bit.

differential PCM (DPCM) DPCM is the generalization of delta modulation in which more than one previously reconstructed sample is used for prediction.

Differentiated Services (DS or DiffServ) A class-based QoS model designed for IP.

Diffie-Hellman protocol A key management protocol that provides a one-time session key for two parties.

digest A condensed version of a document.

digital AMPS (D-AMPS) A second-generation cellular phone system that is a digital version of AMPS.

digital data Data represented by discrete values or conditions.

digital data service (DDS) A digital version of an analog leased line with a rate of 64 Kbps.

digital service Telephone companies are offering digital services to their subscribers. which are less sensitive than analog services to noise and other forms of interference. The two most common ones are switched/56 service and digital data service (DDS).

digital service unit (DSU) Because the line in a switched/56 service is already digital, subscribers do not need modems to transmit digital data. However, they do need another device called a *digital service unit* (DSU).

digital signal A discrete signal with a limited number of values.

digital signal (DS) service A telephone company service featuring a hierarchy of digital signals.

digital signature A security mechanism in which the sender can electronically sign the message and the receiver can verify the message to prove that the message is indeed signed by the sender.

Digital Signature Standard (DSS) The digital signature standard adopted by NIST under FIPS 186.

digital subscriber line (DSL) A technology using existing telecommunication networks to accomplish high-speed delivery of data, voice, video, and multimedia.

digital subscriber line access multiplexer (DSLAM) A telephone company site device that functions like an ADSL modem.

digital-to-analog conversion The representation of digital information by an analog signal.

digital-to-digital conversion The representation of digital data by a digital signal.

digitization Conversion of analog information to digital information.

Dijkstra's algorithm In link-state routing, an algorithm that finds the shortest path to other routers.

direct broadcast address The IP address in a block or subblock (with the suffix set all to 1s). The address is used by a router to send a message to all hosts in the block.

direct current (DC) A zero-frequency signal with a constant amplitude.

direct delivery A delivery in which the final destination of the packet is a host connected to the same physical network as the sender.

direct sequence spread spectrum (DSSS) A service that uses the 2.400–4.835 GHz ISM band. The modulation technique in this specification is PSK at 1 Mbaud/s. The system allows 1 or 2 bits/baud (BPSK or QPSK).

direct sequence spread spectrum (DSSS) distortion A wireless transmission method in which each bit to be sent by the sender is replaced by a sequence of bits called a chip code.

discard eligibility (DE) A bit that identifies a packet that can be discarded if there is congestion in the network.

discrete cosine transform (DCT) A compression technique in which the signal representation of data is changed from a time or space domain to the frequency domain.

discrete multitone technique (DMT) A modulation method combining elements of QAM and FDM.

distance vector The vector used in distance vector routing.

Distance Vector Multicast Routing Protocol (DVMRP) A protocol based on distance vector routing that handles multicast routing.

distance vector routing A routing method in which each router sends its neighbors a list of networks it can reach and the distance to each network.

distortion Any change in a signal due to noise, attenuation, or other influences.

distributed coordination function (DCF) The basic access method in wireless LANs; stations contend with each other to get access to the channel.

distributed database Information stored in many locations.

distributed hash table (DHT) A DHT distributes data (or references to data) among a set of nodes according to some predefined rules. Each peer in a DHT-based network becomes responsible for a range of data items.

distributed interframe space (DIFS) In wireless LANs, a period of time that a station waits before sending a control frame.

distributed processing A strategy in which services provided for the network reside at multiple sites.

distribution hub The regional cable head (RCH) normally serves up to 400,000 subscribers. The RCHs feed the distribution hubs, each of which serves up to 40,000 subscribers

DNS server A computer that holds information about the name space.

domain A subtree of the domain name space.

domain name In the DNS, a sequence of labels separated by dots.

domain name space A method for organizing the name space in which the names are defined in an inverted-tree structure with the root at the top.

Domain Name System (DNS) A TCP/IP application service that converts user-friendly names to IP addresses.

dotted-decimal notation A notation devised to make the IP address easier to read; each byte is converted to its decimal equivalent and then set off from its neighbor by a decimal.

double crossing In mobile IP, double crossing occurs when a remote host communicates with a mobile host that has moved to the same network (or site) as the remote host.

downlink Transmission from a satellite to an earth station.

downloading Retrieving a file or data from a remote site.

dual stack Two protocols (IPv4 and IPv6) on the same station.

dynamic document A Web document created by running a program at the server site.

Dynamic Domain Name System (DDNS) A method to update the DNS master file dynamically.

Dynamic Host Configuration Protocol (DHCP) An extension to BOOTP that dynamically assigns configuration information.

dynamic mapping A technique in which a protocol is used for address resolution.

dynamic routing Routing in which the routing table entries are updated automatically by the routing protocol.

eight-binary, six-ternary (8B6T) encoding A three-level line encoding scheme that encodes a block of 8 bits into a signal of 6 ternary pulses.

eight-binary, ten-binary (8B/10B) encoding A block coding technique in which 8 bits are encoded into a 10-bit code.

electromagnetic spectrum The frequency range occupied by electromagnetic energy.

Encapsulating Security Payload (ESP) A protocol defined by IPSec that provides privacy as well as a combination of integrity and message authentication.

encapsulation The technique in which a data unit from one protocol is placed within the data field portion of the data unit of another protocol.

encrypted security payload An extension that provides confidentiality and guards against eavesdropping.

encryption Converting a message into an unintelligible form that is unreadable unless decrypted.

encryption algorithm An algorithm to convert a message into an unintelligible form that is unreadable unless decrypted.

end-of-option option A one-byte option used for padding at the end of the option field. It, however, can only be used as the last option.

end office A switching office that is the terminus for the local loops.

end system A sender or receiver of data.

entity authentication A technique designed to let one party prove the identity of another party.

ephemeral port number A port number used by the client.

error control The handling of errors in data transmission.

error-reporting message A message that reports problems that a router or a host (destination) may encounter when it processes an IP packet.

escape character A character that is used to change the meaning of the next character.

Ethernet A local area network created by Xerox and has gone through four generations.

extended binary coded decimal interchange code (EBCDIC) An 8-bit character code developed and used by IBM.

extended service set (ESS) A wireless LAN service composed of two or more BSSs with APs as defined by the IEEE 802.11 standard.

Extensible HyperText Markup Language (XHTML) HTML that conforms to the syntax of XML.

Extensible Markup Language (XML) A language that allows users to define representation of data.

Extensible Style Language (XSL) The style language of XML.

extension header An extra header that is used besides the main header for a special purpose

exterior routing Routing between autonomous systems.

extranet A private network that uses the TCP/IP protocol suite to allow authorized access from outside users.

Fast Ethernet Ethernet with a data rate of 100 Mbps.

fast recovery algorithm An algorithm used in TCP when three duplicate ACKs arrive which is interpreted as light congestion.

fast retransmission Retransmission of a segment in the TCP protocol when three duplicate acknowledgments have been received that imply the loss or corruption of that segment.

Federal Communications Commission (FCC) A government agency that regulates radio, television, and telecommunications.

Feistel cipher A class of ciphers consisting of both invertible and noninvertible components.

fiber node The transmission medium from the cable TV office to a box.

fiber-optic cable A high-bandwidth transmission medium that carries data signals in the form of pulses of light. It consists of a thin cylinder of glass or plastic, called the core, surrounded by a concentric layer of glass or plastic called the cladding.

File Transfer Protocol (FTP) In TCP/IP, an application layer protocol that transfers files between two sites.

filtering A process in which a switch makes forwarding decisions.

finite state machine (FSM) A machine that goes through a limited number of states.

firewall A device (usually a router) installed between the internal network of an organization and the rest of the Internet to provide security.

first-in, first-out (FIFO) queue A queue in which the first item in is the first item out.

flag A bit pattern or a character added to the beginning and the end of a frame to separate the frames.

flat name space A name space in which there is no hierarchical structure.

flooding Saturation of a network with a message.

flow control A technique to control the rate of flow of frames (packets or messages).

flow label A 20-bit field that is designed to provide special handling for a particular flow of data.

footprint An area on Earth that is covered by a satellite at a specific time.

foreign agent In mobile IP, the foreign agent is a router or a host attached to the foreign network. The foreign agent receives and delivers packets sent by the home agent to the mobile host.

foreign network The network a mobile host is connected to which is not its home.

forward error correction (FEC) Correction of errors at the receiver without retransmission.

forwarding Placing the packet in its route to its destination.

forwarding port A port which forwards a frame that the switch receives.

four-binary, five-binary (4B/5B) encoding A block coding technique in which a four-bit block is encoded into a 5-bit code.

Fourier analysis The mathematical technique used to obtain the frequency spectrum of an aperiodic signal if the time-domain representation is given.

fragmentation The division of a packet into smaller units to accommodate a protocol's MTU.

frame A group of bits representing a block of data.

frame bursting A technique in CSMA/CD Gigabit Ethernet in which multiple frames are logically connected to each other to resemble a longer frame.

framing Grouping a set of bits or a set of bytes into a unit.

frequency The number of cycles per second of a periodic signal.

frequency masking Frequency masking occurs when a loud sound partially or totally masks a softer sound if the frequencies of the two are close.

frequency modulation (FM) An analog-to-analog modulation method in which the carrier signal's frequency varies with the amplitude of the modulating signal.

frequency shift keying (FSK) A digital-to-analog encoding method in which the frequency of the carrier signal is varied to represent binary 0 or 1.

frequency-division multiple access (FDMA) An access method technique in which multiple sources use assigned bandwidth in a data communication band.

frequency-division multiplexing (FDM) The combining of analog signals into a single signal.

frequency-domain plot A graphical representation of a signal's frequency components.

frequency-hopping spread spectrum (FHSS) A wireless transmission method in which the sender transmits at one carrier frequency for a short period of time, then hops to another carrier frequency for the same amount of time, hops again for the same amount of time, and so on. After N hops, the cycle is repeated.

full-duplex mode A transmission mode in which both parties can communicate simultaneously.

full-duplex switched Ethernet Ethernet in which each station, in its own separate collision domain, can both send and receive.

fully qualified domain name (FQDN) A domain name consisting of labels beginning with the host and going back through each level to the root node.

fundamental frequency The frequency of the dominant sine wave of a composite signal.

gatekeeper In the H.323 standard, a server on the LAN that plays the role of the registrar server.

gateway A device connects the Internet to the telephone network. In general, a gateway is a five-layer device that can translate a message from one protocol stack to another. The gateway here does exactly the same thing. It transforms a telephone network message into an Internet message.

generic domain A subdomain in the domain name system that uses generic suffixes.

geographical routing A routing technique in which the entire address space is divided into blocks based on physical landmasses.

geostationary Earth orbit (GEO) A satellite orbit positioned above the upper Van Allen Belt. A satellite travelling in this orbit looks stationary to the people on Earth.

Gigabit Ethernet Ethernet with one gigabit per second (1000 Mbps) data rate.

Global Positioning System (GPS) An MEO public satellite system consisting of 24 satellites and used for land and sea navigation. GPS is not used for communications.

Global System for Mobile Communication (GSM) A second-generation cellular phone system used in Europe.

Globalstar An LEO satellite system with 48 satellites in six polar orbits with each orbit hosting eight satellites.

Go-Back-N protocol An error-control protocol in which the frame in error and all following frames must be retransmitted.

ground propagation Propagation of radio waves through the lowest portion of the atmosphere (hugging the earth).

group-shared tree A multicast routing feature in which each group in the system shares the same tree.

guard band A bandwidth separating two signals.

guided media Transmission media with a physical boundary.

H.323 A standard designed by ITU to allow telephones on the public telephone network to talk to computers (called terminals in H.323) connected to the Internet.

half-close In TCP, a type of connection termination in which one site stops sending data while it is still receiving data.

half-duplex mode A transmission mode in which communication can be two-way but not at the same time.

Hamming code A method that adds redundant bits to a data unit to detect and correct bit errors.

Hamming distance The number of differences between the corresponding bits in two codewords.

handoff Changing to a new channel as a mobile device moves from one cell to another.

Handshake Protocol The protocol used in connection-oriented networks to establish a connection or to tear down the connection.

harmonics Components of a digital signal, each having a different amplitude, frequency, and phase.

hash function An algorithm that creates a fixed-size digest from a variable-length message.

hashed MAC (HMAC) A MAC based on a hash function such as SHA-1.

hashing A cryptographic technique in which a fixed-length message digest is created from a variable-length message.

head end A cable TV office that receives video signals from broadcasting stations and feeds the signals into coaxial cables.

header Control information added to the beginning of a data packet.

header translation A translation is necessary when the majority of the Internet has moved to IPv6 but some systems still use IPv4. The sender wants to use IPv6, but the receiver does not understand IPv6. In this case, the header format must be totally changed through header translation. The header of the IPv6 packet is converted to an IPv4 header.

hertz (Hz) Unit of measurement for frequency.

hexadecimal colon notation Same as colon hexadecimal notation.

hexadecimal notation The representation of an Ethernet address in hexadecimal notation (6 bytes or 48 bits).

hierarchical routing A routing technique in which the entire address space is divided into levels based on specific criteria.

high-density bipolar 3-zero (HDB3) A scrambling technique in which four consecutive zero-level voltages are replaced with one of the two predefined sequences.

High-level Data Link Control (HDLC) A bit-oriented data-link protocol defined by the ISO.

high-rate direct-sequence spread spectrum (HR-DSSS) A signal generation method similar to DSSS except for the encoding method (CCK).

home address The original address of a mobile host.

home agent Usually a router attached to the home network of the mobile host that receives and sends packets (for the mobile host) to the foreign agent.

home network A network that is the permanent home of the mobile host.

hop count The number of nodes along a route. It is a measurement of distance in routing algorithms.

hop limit In IPv5, the 8-bit hop limit field serves the same purpose as the TTL field in IPv4.

hop-to-hop delivery Transmission of frames from one node to the next.

horn antenna A scoop-shaped antenna used in terrestrial microwave communication.

host A station or node on a network.

host-specific routing A routing method in which the full IP address of a host is given in the routing table.

hostid The part of an IP address that identifies a host.

hub A central device in a star topology that provides a common connection among the nodes.

Huffman coding A statistical compression method using variable-length codes to encode a set of symbols.

hybrid network A network with a private internet and access to the global Internet.

hybrid-fiber-coaxial (HFC) network The second generation of cable networks; uses fiber optic and coaxial cable.

hypermedia Information containing text, pictures, graphics, and sound that is linked to other documents through pointers.

hypertext Information containing text that is linked to other documents through pointers.

HyperText Markup Language (HTML) The computer language for specifying the contents and format of a Web document. It allows additional text to include codes that define fonts, layouts, embedded graphics, and hypertext links.

HyperText Transfer Protocol (HTTP) An application service for retrieving a web document.

image A matrix of pixels (picture elements), where each pixel is a small dot. The number of the pixels depends on the *resolution.* For example, an image can be divided into 1000 pixels or 10,000 pixels. In the second case, there is a better representation of the image (better resolution), but more memory is needed to store the image.

impulse noise A spike (a signal with high energy and very short duration) that comes from power lines, lightning, and so on.

inband signaling Using the same channel for data and control transfer.

incumbent local exchange carrier (ILEC) The carrier that provided services before 1996 and owned the cabling system (local loops).

indirect delivery A delivery in which the source and destination of a packet are in different networks.

induced noise The noise that comes from sources such as motors and appliances.

infrared wave A wave with a frequency between 300 GHz and 400 THz; usually used for short-range communications.

initial sequence number (ISN) In TCP, the random number used as the first sequence number in a connection.

initial TSN The sequence number used by SCTP called the *transmission sequence number (TSN)* to number the data chunks.

inner product A number produced by multiplying two sequences, element by element, and summing the products.

Institute of Electrical and Electronics Engineers (IEEE) A group consisting of professional engineers that has specialized societies whose committees prepare standards in members' areas of specialty.

Integrated Services (IntServ) A flow-based QoS model designed for IP.

interactive audio/video Real-time communication with sound and images.

interautonomous system routing protocol A protocol to handle transmissions between autonomous systems.

interdomain routing Routing among autonomous systems.

interexchange carrier (IXC) The long-distance companies that provide communication services between two customers in different LATAs

initial tag The value of the initial tag field in the INIT chunk

interface The boundary between two pieces of equipment. It also refers to mechanical, electrical, and functional characteristics of the connection. In network programming, a set of procedures available to the upper layer to use the services of the lower layer.

interference Any undesired energy that interferes with the desired signals.

interframe space (IFS) In wireless LANs, a time interval between two frames to control access to the channel.

Interim Standard 95 (IS-95) One of the dominant second-generation cellular telephony standards in North America.

interior routing Routing inside an autonomous system.

interleaved FDMA (IFDMA) The more efficient FDMA used in Universal Mobile Telecommunications System (UMTS).

interleaving In multiplexing, taking a specific amount of data from each device in a regular order.

International Organization for Standardization (ISO) A worldwide organization that defines and develops standards on a variety of topics.

International Telecommunications Union (ITU) An international telecommunication organization.

Internet A global internet that uses the TCP/IP protocol suite.

internet A collection of networks connected by internetworking devices such as routers or gateways.

Internet address A 32-bit or 128-bit network-layer address used to uniquely define a host on an internet using the TCP/IP protocol.

Internet Architecture Board (IAB) The technical adviser to the ISOC; oversees the continuing development of the TCP/IP protocol suite.

Internet Assigned Numbers Authority (IANA) A group supported by the U.S. government that was responsible for the management of Internet domain names and addresses until October 1998.

Internet Control Message Protocol (ICMPv4) The version 4 of a protocol in the TCP/IP protocol suite that handles error and control messages.

Internet Control Message Protocol (ICMPv6) The version 6 of a protocol in the TCP/IP protocol suite that handles error and control messages.

Internet Corporation for Assigned Names and Numbers (ICANN) A private, nonprofit corporation managed by an international board that assumed IANA operations.

Internet draft A working Internet document (a work in progress) with no official status and a six-month lifetime.

Internet Engineering Steering Group (IESG) An organization that oversees the activities of IETF.

Internet Engineering Task Force (IETF) A group working on the design and development of the TCP/IP protocol suite and the Internet.

Internet Group Management Protocol (IGMP) A protocol in the TCP/IP protocol suite that handles multicasting.

Internet Key Exchange (IKE) A protocol designed to create security associations in IPSec.

Internet Mail Access Protocol (IMAP) A complex and powerful protocol to pull e-mail messages from an e-mail server.

Internet Mobile Communication 2000 (ITM-2000) An ITU issued blueprint that defines criteria for third-generation cellular telephony.

Internet model A 5-layer protocol stack that dominates data communications and networking today.

Internet Network Information Center (INTERNIC) An agency responsible for collecting and distributing information about TCP/IP protocols.

Internet Protocol (IP) The network-layer protocol in the TCP/IP protocol suite governing connectionless transmission across packet-switched networks. Two versions commonly in use: IPv4 and IPv6.

Internet Protocol Control Protocol (IPCP) In PPP, the set of protocols that establish and terminate a network layer connection for IP packets.

Internet Protocol, next generation (IPng) Another term for the sixth version of the Internet Protocol, IPv6.

Internet Protocol, version 6 (IPv6) The sixth version of the Internet Protocol.

Internet Research Task Force (IRTF) A forum of working groups focusing on long-term research topics related to the Internet.

Internet Security Association and Key Management Protocol (ISAKMP) A protocol designed by the National Security Agency (NSA) that actually implements the exchanges defined in IKE.

Internet service provider (ISP) A company that provides Internet services.

Internet Society (ISOC) The nonprofit organization established to publicize the Internet.

Internet standard A thoroughly tested specification that is useful to and adhered to by those who work with the Internet. It is a formalized regulation that must be followed.

internetwork (internet) A network of networks.

internetworking Connecting several networks together using internetworking devices such as routers and gateways.

intracoded frame (I-frame) In MPEG, an I-frame is an independent frame that is not related to any other frame (not to the frame sent before or after). They are present at regular intervals.

intranet A private network that uses the TCP/IP protocol suite.

inverse domain A subdomain in the DNS that finds the domain name, given the IP address.

inverse-neighbor advertisement message A message sent in response to the inverse-neighbor-discovery message.

inverse-neighbor solicitation message The message sent by a node that knows the link layer address of a neighbor, but not the neighbor's IP address.

IP datagram The Internetworking Protocol data unit.

IP new generation (IPng) The new version of IP that was a proposal to augment the address space of IPv4 and at the same time redesign the format of the IP packet and revise some auxiliary protocols such as ICMP. It is interesting to know that IPv5 was a proposal, based on the OSI model, that never materialized.

IP Security (IPSec) A collection of protocols designed by the IETF (Internet Engineering Task Force) to provide security for a packet carried on the Internet.

IrDA port A port that allows a wireless keyboard to communicate with a PC.

Iridium A 66-satellite network that provides communication from any Earth site to another.

ISDN user port (ISUP) A user port that can replace TUP to provide services similar to those of an ISDN network.

iterative resolution Resolution of the IP address in which the client may send its request to multiple servers before getting an answer.

iterative server A server that can serve only one client at a time.

ITU Standardization Sector (ITU-T) A standards organization formerly known as the CCITT.

jambo payload An option that can be used to increase the length of the payload in the IP datagram.

jamming signal In CSMA/CD, a signal sent by the first station that detects collision to alert every other station of the situation.

Java An object-oriented programming language.

jitter A phenomenon in real-time traffic caused by gaps between consecutive packets at the receiver caused by uneven delays.

Joint Photographic Experts Group (JPEG) A standard for compressing continuous-tone pictures.

Kademlia A DHT-based P2P network in which the distances between nodes are measured as the XOR of two identifiers.

Karn's Algorithm An algorithm that does not include the retransmitted segments in calculation of round-trip time.

keepalive timer A timer in TCP that checks to see if there is an active process at the other site.

key A set of values that the cipher, as an algorithm, operates on.

key-distribution center (KDC) In secret key encryption, a trusted third party that shares a key with each user.

label An identifier used in connection-oriented service to define the path.

leaky bucket algorithm An algorithm to shape bursty traffic.

least-cost tree In least-cost routing, a tree with the source at the root that spans the whole graph.

Lempel-Ziv-Welch (LZW) A group of compression methods based on dynamic creation of a dictionary (array) of strings in the text, which was invented by Lempel and Ziv and refined by Welch.

limited-broadcast address An address used to broadcast messages only to hosts inside a network (link).

line coding Converting binary data into signals.

line-of-sight propagation The transmission of very high frequency signals in straight lines directly from antenna to antenna.

linear block code A block code in which adding two codewords creates another codeword.

linear predictive coding (LPC) A predictive coding method in which, instead of sending quantized difference signals, the source analyzes the signals and determines their characteristics.

link The physical communication pathway that transfers data from one device to another.

Link Control Protocol (LCP) A PPP protocol responsible for establishing, maintaining, configuring, and terminating links.

link-layer address The address of a device used at the data-link layer (MAC address).

link-layer switch A *switch* operates in both the physical and the data-link layers. As a physical-layer device, it regenerates the signal it receives. As a link-layer device, it can check the MAC addresses (source and destination) contained in the fram.

link local address An IPv6 address that is used if a LAN is to use the Internet protocols but is not connected to the Internet for security reasons.

link local block A sub-block that can be used as a private address in a network.

link-state advertisement (LSA) In OSPF, a method to disperse information.

link-state database In link-state routing, a database common to all routers and made from LSP information.

link-state packet (LSP) In link-state routing, a small packet containing routing information sent by a router to all other routers.

link-state routing A routing method in which each router shares its knowledge of changes in its neighborhood with all other routers.

local access transport area (LATA) After the divestiture of 1984, the United States was divided into more than 200 local-access transport areas (LATAs).

local area network (LAN) A network connecting devices inside a single building or inside buildings close to each other.

local login Logging into a host using the terminal directly connected to the host.

local loop The link that connects a subscriber to the telephone central office.

logical address An address defined in the network layer.

logical link control (LLC) The upper sublayer of the data-link layer as defined by IEEE Project 802.2.

Logical Link Control and Adaptation Protocol (L2CAP) A Bluetooth layer used for data exchange on an ACL link.

logical tunnel The encapsulation of a multicast packet inside a unicast packet to enable multicast routing by non-multicast routers.

long distance company A company that provides long-distance telephone service.

longest mask matching The technique in CIDR in which the longest prefix is handled first when searching a routing table.

loopback address An address used by a host to test its internal software.

loose source route option An option similar to the *strict source route*, but it is less rigid. Each router in the list must be visited, but the datagram can visit other routers as well.

lossless compression A compression method in which the integrity of the data is preserved because compression and decompression algorithms are exact inverses of each other: no part of the data is lost in the process.

lossy compression A compression method in which the loss of some data is sacrificed to obtain a better compression ratio.

low-Earth-orbit (LEO) A polar satellite orbit with an altitude between 500 and 2000 km. A satellite with this orbit has a rotation period of 90 to 120 minutes.

low-pass channel A channel that passes frequencies between 0 and f.

magic cookie In DHCP, the number in the format of an IP address with the value of 99.130.83.99; indicates that options are present.

mail transfer agent (MTA) An SMTP component that transfers the mail across the Internet.

man-in-the-middle attack A key management problem in which an intruder intercepts and sends messages between the intended sender and receiver.

Management Information Base (MIB) The database used by SNMP that holds the information necessary for management of a network.

Manchester encoding A digital-to-digital polar encoding method in which a transition occurs at the middle of each bit interval to provide synchronization.

mapped address An IPv6 address used when a computer that has migrated to IPv6 wants to send a packet to a computer still using IPv4.

mask For IPv4, a 32-bit binary number that gives the first address in the block (the network address) when ANDed with an address in the block.

master secret In SSL, a 48-byte secret created from the *pre-master secret*.

maturity level The phases through which an RFC goes.

maximum transfer unit (MTU) The largest size data unit a specific network can handle.

media access control (MAC) sublayer The lower sublayer in the data-link layer defined by the IEEE 802 project. It defines the access method and access control in different local area network protocols.

media server A server used in streaming audio or video.

medium-Earth-orbit (MEO) A satellite orbit positioned between the two Van Allen belts. A satellite at this orbit takes six hours to circle the earth.

mesh topology A network configuration in which each device has a dedicated point-to-point link to every other device.

message The information (data) to be communicated.

message access agent (MAA) A client-server program that pulls the stored email messages.

message authentication A security measure in which the sender of the message is verified for every message sent.

message authentication code (MAC) An MDC that includes a secret between two parties.

Message Digest (MD) A set of several hash algorithms designed by Ron Rivest and referred to as MD2, MD4, and MD5.

message transfer agent (MTA) An SMTP component that transfers the message across the Internet.

message transport port (MTP) level The physical layer in SS7.

metafile In streaming audio or video, a file that holds information about the audio/video file.

metric A cost assigned for passing through a network.

metropolitan area network (MAN) A network that can span a geographical area the size of a city.

microwave Electromagnetic waves ranging from 2 GHz to 40 GHz.

Military Network (MILNET) A network for military use that was originally part of ARPANET.

minimum Hamming distance In a set of codewords, the smallest Hamming distance between all possible pairs.

mixer A device that mathematically adds signals coming from different sources to create one single signal.

mobile host A host that can move from one network to another.

mobile switching center (MSC) In cellular telephony, a switching office that coordinates communication between all base stations and the telephone central office.

mobile telephone switching office (MTSO) An office that controls and coordinates communi-cation between all of the cell offices and the telephone control office.

modem A device consisting of a modulator and a demodulator. It converts a digital signal into an analog signal (modulation) and vice versa (demodulation).

modification detection code (MDC) The digest created by a hash function.

modular arithmetic Arithmetic that uses a limited range of integers (0 to $n - 1$).

modulation Modification of one or more characteristics of a carrier wave by an information-bearing signal.

modulator A device that modulates a signal to create another signal.

modulus The upper limit in modular arithmetic (n).

monoalphabetic cipher A substitution cipher in which a symbol in the plaintext is always changed to the same symbol in the ciphertext, regardless of its position in the text.

monoalphabetic substitution An encryption method in which each occurrence of a character is replaced by another character in the set.

Motion Picture Experts Group (MPEG) A method to compress videos.

MPEG audio layer 3 (MP3) A standard that uses perceptual coding to compress audio.

multi-carrier CDMA (MC-CDMA) An access method proposed for 4G wireless networks.

multi-user MIMO (MU-MIMO) A more sophisticated version of MIMO in which multiple users can communicate at the same time.

multicast address An address used for multicasting.

multicast backbone (MBONE) A set of internet routers supporting multicasting through the use of tunneling.

multicast Listener Delivery protocol In IPv6, the responsibility of multicast delivery is given to the Multicast Listener Delivery protocol.

Multicast Open Shortest Path First (MOSPF) A multicast protocol that uses multicast link-state routing to create a source-based least-cost tree.

multicast router A router with a list of loyal members related to each router interface that distributes the multicast packets.

multicast routing Moving a multicast packet to its destinations.

multicasting A transmission method that allows copies of a single packet to be sent to a selected group of receivers.

multihoming service A service provided by SCTP protocol in which a host can be connected to more than one network.

multiline transmission, 3-level (MLT-3) encoding A line coding scheme featuring 3 levels of signals and transitions at the beginning of the 1 bit.

multimedia traffic Traffic consisting of data, video, and audio.

multimode graded-index fiber An optical fiber with a core having a graded index of refraction.

multimode step-index fiber An optical fiber with a core having a uniform index of refraction. The index of refraction changes suddenly at the core/cladding boundary.

multiple access (MA) A line access method in which every station can access the line freely.

multiple unicasting Sending multiple copies of a message, each with a different unicast destination address, from one source.

multiple-input multiple-output (MIMO) The standard uses what is called **MIMO** to overcome the noise problem in wireless LANs. The idea is that if we can send multiple output signals and receive multiple input signals, we are in the better position to eliminate noise. Some implementations of this project have reached up to 600 Mbps data rate.

multiple-input multiple-output (MIMO) antenna A branch of intelligent antenna proposed for 4G wireless systems that allows independent streams to be transmitted simultaneously from all the antennas to increase the data rate into multiple folds.

multiplexer (MUX) A device used for multiplexing.

multiplexing The process of combining signals from multiple sources for transmission across a single data link.

multiplicative decrease A congestion avoidance technique in which the threshold is set to half of the last congestion window size, and the congestion window size starts from one again.

Multipurpose Internet Mail Extensions (MIME) A supplement to SMTP that allows non-ASCII data to be sent through SMTP.

multistage switch An array of switches designed to reduce the number of crosspoints.

multistream service A service provided by SCTP that allows data transfer to be carried using different streams.

Nagle's algorithm An algorithm that attempts to prevent silly window syndrome at the sender's site; both the rate of data production and the network speed are taken into account.

name space All the names assigned to machines on an internet.

name-address resolution Mapping a name to an address or an address to a name.

National Science Foundation Network (NSFNET) The network funded by the National Science Foundation.

National Security Agency (NSA) A U.S. intelligence-gathering security agency.

neighbor advertisement message The message sent in response to neighbor solicitation message.

netid The part of an IP address that identifies the network.

network The interconnection of a set of devices capable of communication.

network access point (NAP) A complex switching station that connects backbone networks.

network address An address that identifies a network to the rest of the Internet; it is the first address in a block.

Network Address Translation (NAT) A technology that allows a private network to use a set of private addresses for internal communication and a set of global Internet addresses for external communication.

network allocation vector (NAV) In CSMA/CA, the amount of time that must pass before a station can check for an idle line.

Network Control Protocol (NCP) In PPP, a set of control protocols that allows the encapsulation of data coming from network layer protocols.

Network Information Center (NIC) An agency responsible for collecting and distributing information about TCP/IP protocols.

network interface card (NIC) An electronic device, internal or external to a station, that contains circuitry to enable the station to be connected to the network.

network layer The third layer in the Internet model, responsible for the delivery of a packet to the final destination.

Network Virtual Terminal (NVT) A TCP/IP application protocol that allows remote logging.

network-specific routing Routing in which all hosts on a network share one entry in the routing table.

network-to-network interface (NNI) In ATM, the interface between two networks.

next header An 8-bit field defining the type of first extension header (if present) or the type of the data that follows the base header in the datagram. This field is similar to the protocol field in IPv4.

next-hop routing A routing method in which only the address of the next hop is listed in the routing table instead of a complete list of the stops the packet must make.

no operation option A one-byte option used as a filler between options.

no transition mobility *See* transition mobility.

node An addressable communication device (e.g., a computer or router) on a network.

node-to-node delivery Transfer of a data unit from one node to the next.

noise Random electrical signals that can be picked up by the transmission medium and cause degradation or distortion of the data.

noiseless channel An error-free channel.

noisy channel A channel that can produce error in data transmission.

nonce A large random number that is used once to distinguish a fresh authentication request from a used one.

nonperiodic (aperiodic) signal A signal that has no period; a signal that does not exhibit a pattern or repeating cycle.

nonpersistent connection A connection in which one TCP connection is made for each request/response.

nonpersistent method A random multiple access method in which a station waits a random period of time after a collision is sensed.

nonrepudiation A security aspect in which a receiver must be able to prove that a received message came from a specific sender.

nonreturn to zero (NRZ) A digital-to-digital polar encoding method in which the signal level is always either positive or negative, but never at zero level.

nonreturn-to-zero, invert (NRZ-I) An NRZ encoding method in which the signal level is inverted each time a 1 is encountered.

nonreturn-to-zero, level (NRZ-L) An NRZ encoding method in which the signal level is directly related to the bit value.

normal response mode (NRM) In HDLC, a communication mode in which the secondary station must have permission from the primary station before transmission can proceed.

Nyquist bit rate The data rate based on the Nyquist theorem.

Nyquist theorem A theorem that states that the number of samples needed to adequately represent an analog signal is equal to twice the highest frequency of the original signal.

Oakley A key creation protocol, developed by Hilarie Orman, which is one of the three components of the IKE protocol.

object identifier In MIB, an identifier for an object used in SNMP and some other network management protocols.

omnidirectional antenna An antenna that sends out or receives signals in all directions.

one's complement A representation of binary numbers in which the complement of a number is found by complementing all bits.

one-time pad A cipher invented by Vernam in which the key is a random sequence of symbols having the same length as the plaintext.

Open Shortest Path First (OSPF) An interior routing protocol based on link-state routing.

Open Systems Interconnection (OSI) model A seven-layer model for data communication defined by ISO.

open-loop congestion control Policies applied to prevent congestion.

optical carrier (OC) The hierarchy of fiber-optic carriers defined in SONET.

optical fiber A thin thread of glass or other transparent material to carry light beams.

orbit The path a satellite travels around the earth.

orthogonal FDMA (OFDMA) A new access techniques for 4G. To increase efficiency, capacity, and scalability.

orthogonal-frequency-division-multiplexing (OFDM) A multiplexing method similar to FDM, with all the subbands used by one source at a given time.

orthogonal sequence A sequence with special properties between elements.

out-of-band signaling Using two separate channels for data and control.

P-box A component in a modern block cipher that transposes bits.

p-persistent method A CSMA persistence strategy in which a station sends with probability p if it finds the line idle.

packet Synonym for data unit, mostly used in the network layer.

Packet Internet Groper (PING) An application program to determine the reachability of a destination using an ICMP echo request and reply.

packet switching Data transmission using a packet-switched network.

packet-filter firewall A firewall that forwards or blocks packets based on the information in the network-layer and transport-layer headers.

packet-switched network A network in which data are transmitted in independent units called packets.

packetizing Encapsulating the payload (data received from upper layer) in a network-layer packet at the source and decapsulating the payload from the network-layer packet at the destination.

packet-too-bit message This is a new type of message added to version 6. Since IPv6 does not fragment at the router, if a router receives a datagram that is larger than the maximum transmission unit (MTU) size of the network through which the datagram should pass, two things happen. First, the router discards the datagram. Second, an ICMP error packet—a **packet-too-big message**—is sent to the source.

Pad1 This option is one-byte long and is designed for alignment purposes. Some options need to start at a specific bit of the 32-bit word. If an option falls short of this requirement by exactly one byte.

PadN PadN is similar in concept to Pad1. The difference is that PadN is used when two or more bytes are needed for alignment.

parallel transmission Transmission in which bits in a group are sent simultaneously, each using a separate link.

parity check code An error-detection method using a parity bit.

partially qualified domain name (PQDN) A domain name that does not include all the levels between the host and the root node.

Password Authentication Protocol (PAP) A simple two-step authentication protocol used in PPP.

Pastry A DHT-based P2P network in which the identifiers are n-digit strings in base 2^b.

path attribute In both intra-domain routing protocols (RIP or OSPF), a destination is normally associated with two pieces of information: next hop and cost. The first one shows the address of the next router to deliver the packet; the second defines the cost to the final destination. Interdomain routing is more involved and naturally needs more information about how to reach the final destination. In BGP these pieces are called **path attributes.**

Path MTU Discovery technique A source must use a **Path MTU Discovery technique** to find the smallest MTU supported by any network on the path. The source then fragments using this knowledge.

path vector routing A routing method on which BGP is based; in this method, the ASs through which a packet must pass are explicitly listed.

peak amplitude The maximum amplitude of an analog signal.

peer-to-peer (P2P) paradigm A paradigm in which two peer computers can communicate with each other to exchange services.

peer-to-peer process A process on a sending and a receiving machine that communicates at a given layer.

perceptual coding The most common compression technique used to create CD-quality audio, based on the science of psychoacoustics. Algorithms used in perceptual coding first transform the data from time domain to frequency domain; the operations are then performed on the data in the frequency domain.

performance Performance can be measured in many ways, including transit time and response time. Transit time is the amount of time required for a message to travel from one device to another. Response time is the elapsed time between an inquiry and a response. The performance of a network depends on a number of factors, including the number of users, the type of transmission medium, the capabilities of the connected hardware, and the efficiency of the software.

per-hop behavior (PHB) In the Diffserv model, a 6-bit field that defines the packet-handling mechanism for the packet.

period The amount of time required to complete one full cycle.

periodic signal A signal that exhibits a repeating pattern.

permanent virtual circuit (PVC) A virtual circuit transmission method in which the same virtual circuit is used between source and destination on a continual basis.

persistence timer A timer in TCP that is used to prevent deadlock.

persistent connection A connection in which the server leaves the connection open for more requests after sending a response.

personal communication system (PCS) A generic term for a commercial cellular system that offers several kinds of communication services.

phase The relative position of a signal in time.

phase modulation (PM) An analog-to-analog modulation method in which the carrier signal's phase varies with the amplitude of the modulating signal.

phase shift keying (PSK) A digital-to-analog modulation method in which the phase of the carrier signal is varied to represent a specific bit pattern.

PHY sublayer The transceiver in Fast Ethernet.

physical address See link-layer address.

physical layer The first layer of the Internet model, responsible for the mechanical and electrical specifications of the medium.

physical topology The term refers to the way in which a network is laid out physically. Two or more devices connect to a link; two or more links form a topology. The topology of a network is the geometric representation of the relationship of all the links and linking devices (usually called **nodes**) to one another. There are four basic topologies possible: *mesh*, *star*, *bus*, and *ring*.

piconet A Bluetooth network.

piggybacking The inclusion of acknowledgment on a data frame.

pipelining Sending several packets or frames before news is received concerning previous ones.

pixel A picture element of an image.

plain old telephone system (POTS) The conventional telephone network used for voice communication.

plaintext The message before encryption or after decryption.

playback buffer A buffer that stores the data until they are ready to be played.

point coordination function (PCF) In wireless LANs, an optional and complex access method implemented in an infrastructure network.

point of presence (POP) A switching office where carriers can interact with each other.

point-to-point connection A dedicated transmission link between two devices.

Point-to-Point Protocol (PPP) A protocol for data transfer across a serial line.

poisoned reverse A variation of split horizons. In this method, information received by the router is used to update the routing table and then passed out to all interfaces. However, a table entry that has come through one interface is set to a metric of infinity as it goes out through the same interface.

polar encoding A digital-to-analog encoding method that uses two levels (positive and negative) of amplitude.

policy routing A path vector routing feature in which the routing tables are based on rules set by the network administrator rather than on a metric.

poll In the primary/secondary access method, a procedure in which the primary station asks a secondary station if it has any data to transmit.

poll/final (P/F) bit A bit in the control field of HDLC; if the primary is sending, it can be a poll bit; if the secondary is sending, it can be a final bit.

poll/select An access method protocol using poll and select procedures. See *poll*. See *select*.

polling An access method in which one device is designated as a primary station and the others as the secondary stations. The access is controlled by the primary station.

polyalphabetic cipher A cipher in which each occurrence of a character may have a different substitute.

polyalphabetic substitution An encryption method in which each occurrence of a character can have a different substitute.

polynomial An algebraic term that can represent a CRC divisor.

port address In TCP/IP protocol, an integer that identifies a process (same as port numbers).

port forwarding A service provided by SSH to allow another application to secure channels using SSH.

port number An integer that defines a process running on a host (same as port address).

Post Office Protocol, version 3 (POP3) A popular but simple SMTP mail access protocol.

pre-master secret In SSL, a secret exchanged between the client and server before calculation of the master secret.

preamble The 7-byte field of an IEEE 802.3 frame consisting of alternating 1s and 0s that alert and synchronize the receiver.

predicted frame (P-frame) A predicted frame is related to the preceding I-frame or B-frame. In other words, each P-frame contains only the changes from the preceding frame.

predictive coding (PC) In audio compression, encoding only the differences between the samples.

prefix In an IP address, another name for the common part (similar to the netid).

presentation layer The sixth layer of the OSI model; responsible for translation, encryption, authentication, and data compression.

Pretty Good Privacy (PGP) A protocol invented by Phil Zimmermann to provide e-mail with privacy, integrity, and authentication.

primary address When we use multiple addresses for communication, only one of these addresses can be defined as the *primary address,* which is defined during association establishment. The interesting point is that the primary address of an end is determined by the other end. In other words, a source defines the primary address for a destination.

primary station In primary/secondary access method, a station that issues commands to the secondary stations.

priority queuing A queuing technique in which there are two queues: one for regular packets, the other for the packet with priority.

privacy A security aspect in which the message makes sense only to the intended receiver.

private key In an asymmetric-key cryptosystem, the key used for decryption. In a digital signature, the key is used for signing.

private network A network that is isolated from the Internet.

process A running application program.

process-to-process communication Communication between two running application programs.

process-to-process delivery Delivery of a packet from the sending process to the destination process.

Project 802 The project undertaken by the IEEE in an attempt to solve LAN incompatibility.

propagation delay See propagation time.

propagation speed The rate at which a signal or bit travels; measured by distance/second.

propagation time The time required for a signal to travel from one point to another.

protocol Rules for communication.

Protocol Independent Multicast (PIM) A multicasting protocol family with two members, PIM-DM and PIM-SM; both protocols are unicast-protocol dependent.

Protocol Independent Multicast-Dense Mode (PIM-DM) A source-based routing protocol that uses RPF and pruning/grafting strategies to handle multicasting.

Protocol Independent Multicast-Sparse Mode (PIM-SM) A group-shared routing protocol that is similar to CBT and uses a rendezvous point as the source of the tree.

protocol layering The idea of using a set of protocols to create a hierarchy of rules for handling a difficult task.

protocol suite A stack or family of protocols defined for a complex communication system.

proxy ARP A technique that creates a subnetting effect; one server answers ARP requests for multiple hosts.

proxy firewall A firewall that filters a message based on the information available in the message itself (at the application layer).

proxy server A computer that keeps copies of responses to recent requests.

pruning Stopping the sending of multicast messages from an interface.

pseudoheader Information from the IP header used only for checksum calculation in the UDP and TCP packet.

pseudorandom noise (PN) A pseudorandom code generator used in FHSS.

pseudoternary A variation of AM encoding in which a 1 bit is encoded as zero voltage and a 0 bit is encoded as alternating positive and negative voltage.

psychoacoustics Psychoacoustics is the study of subjective human perception of sound. Perceptual coding takes advantage of flaws in the human auditory system.

public key In an asymmetric-key cryptosystem, the key used for encryption. In digital signature, the key is used for verification.

public key infrastructure (PKI) A hierarchical structure of CA servers.

public-key certificate A certificate that defines the owner of a public key.

public-key cryptography A method of encryption based on a nonreversible encryption algorithm. The method uses two types of keys: The public key is known to the public; the private key (secret key) is known only to the receiver.

pulse amplitude modulation (PAM) A technique in which an analog signal is sampled; the result is a series of pulses based on the sampled data.

pulse code modulation (PCM) A technique that modifies PAM pulses to create a digital signal.

pulse position modulation (PPM) The modulation technique used to modulate an infrared signal.

pulse stuffing In TDM, a technique that adds dummy bits to the input lines with lower rates.

pure ALOHA The original ALOHA that does not uses slots.

quadrature amplitude modulation (QAM) A digital-to-analog modulation method in which the phase and amplitude of the carrier signal vary with the modulating signal.

quality of service (QoS) An issue that refers to a set of technique and mechanism that guarantees the performance of a network.

quantization The assignment of a specific range of values to signal amplitudes.

quantization error Error introduced in the system during quantization (analog-to-digital conversion).

query message The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host. For example, nodes can discover their neighbors. Also, hosts can discover and learn about routers on their network and routers can help a node redirect its messages.

queue A waiting list.

quoted-printable An encoding scheme used when the data consist mostly of ASCII characters with a small non-ASCII portion.

radio wave Electromagnetic energy in the 3-KHz to 300-GHz range.

random access A medium access category in which each station can access the medium without being controlled by any other station.

random access method In a random-access method, each station has the right to the medium without being controlled by any other station. However, if more than one station tries to send, there is an access conflict—*collision*—and the frames will be either destroyed or modified. To avoid access conflict or to resolve it when it happens, each station follows a procedure that answers the following questions:

ranging In an HFC network, a process that determines the distance between the CM and the CMTS.

rate adaptive asymmetrical digital subscriber line (RADSL) A DSL-based technology that features different data rates depending on the type of communication.

raw socket A structure designed for protocols that directly use the services of IP and use neither stream sockets nor datagram sockets.

read-only memory (ROM) Permanent memory with contents that cannot be changed.

Real-Time Streaming Protocol (RTSP) An out-of-band control protocol designed to add more functionality to the streaming audio/video process.

Real-time Transport Control Protocol (RTCP) A companion protocol to RTP with messages that control the flow and quality of data and allow the recipient to send feedback to the source or sources.

Real-time Transport Protocol (RTP) A protocol for real-time traffic; used in conjunction with UDP.

receiver The device that receives the message. It can be a computer, workstation, telephone handset, television, and so on.

record route option The option that is used to record the Internet routers that handle the datagram. The result can be used for debugging and management purposes.

recursive resolution Resolution of the IP address in which the client sends its request to a server that eventually returns a response.

redundancy The addition of bits to a message for error control.

Reed-Solomon A complex, but efficient, cyclic code.

reflection The phenomenon related to the bouncing back of light at the boundary of two media.

reflects When a ray travelling in a denser medium and reaches the boundary of a less dense one, if the incident angle is greater than the critical angle, the ray **reflects** (makes a turn) and travels again in the denser medium.

refraction The phenomenon related to the bending of light when it passes from one medium to another.

refracts When a ray moves from a denser medium to a less dense one, if the incident angle is greater than the critical angle, the ray **reflects** (makes a turn) and travels again in the denser medium.

regional ISP A small ISP that is connected to one or more backbones or international ISPs.

regional office A level of switching in telephone network associated with a region.

registered port A port number, ranging from 1,024 to 49,151, not assigned or controlled by IANA.

registrar An authority to register new domain names.

registrar server In SIP, a server to which a user is registered at each moment.

relay agent For BOOTP, a router that can help send local requests to remote servers.

reliability A QoS flow characteristic; dependability of the transmission. A network is reliable when it does not corrupt, lose, or duplicate a packet.

remote bridge A device that connects LANs and point-to-point networks; often used in a backbone network.

remote logging (rlogin) The process of logging on to a remote computer from a terminal connected to a local computer.

rendezvous point (RP) A router used by PIM to distribute the multicast packets.

rendezvous router A router that is the core or center for each multicast group; it becomes the root of the tree.

rendezvous-point tree A group-shared tree method in which there is one tree for each group.

repeater A device that extends the distance a signal can travel by regenerating the signal.

replay attack The resending of a message that has been intercepted by an intruder.

Request for Comment (RFC) A formal Internet document concerning an Internet issue.

reservation method A control-access method in which a station needs to make a reservation before sending data. Time is divided into intervals. In each interval, a reservation frame precedes the data frames sent in that interval.

resolver The DNS client that is used by a host that needs to map an address to a name or a name to an address.

Resource Reservation Protocol (RSVP) A signaling protocol to help IP create a flow and make a resource reservation to improve QoS.

retransmission time-out (RTO) The expiration of a timer that controls the retransmission of packets.

return-to-zero (RZ) A digital-to-digital encoding technique in which the voltage of the signal is zero for the second half of the bit interval.

reuse factor In cellular telephony, the number of cells with a different set of frequencies.

Reverse Address Resolution Protocol (RARP) A TCP/IP protocol that allows a host to find its Internet address given its physical address.

reverse path broadcasting (RPB) In multicasting, a technique in which it is guaranteed that each destination receives one and only one copy of the packet.

reverse path forwarding (RPF) A technique in which the router forwards only the packets that have traveled the shortest path from the source to the router.

reverse path multicasting (RPM) A technique that adds pruning and grafting to RPB to create a multicast shortest path tree that supports dynamic membership changes.

RGB A coloring method so called because each color is made of a combination of three primary colors: *r*ed, *g*reen, and *b*lue.

ring topology A topology in which the devices are connected in a ring. Each device on the ring receives the data unit from the previous device, regenerates it, and forwards it to the next device.

Rivest, Shamir, Adleman (RSA) See *RSA cryptosystem.*

RJ45 A coaxial cable connector.

roaming In cellular telephony, the ability of a user to communicate outside of his own service provider's area.

root server In DNS, a server whose zone consists of the whole tree. A root server usually does not store any information about domains but delegates its authority to other servers, keeping references to those servers.

round-trip time (RTT) The time required for a datagram to go from a source to a destination and then back again.

route A path traveled by a packet.

router An internetworking device operating at the first three layers of the TCP/IP protocol suite. A router is attached to two or more networks and forwards packets from one network to another.

routing The process performed by a router; finding the next hop for a datagram.

Routing Information Protocol (RIP) A routing protocol based on the distance-vector routing algorithm.

routing table A table containing information a router needs to route packets. The information may include the network address, the cost, the address of the next hop, and so on.

RSA cryptosystem A popular public-key encryption method developed by Rivest, Shamir, and Adleman.

run-length coding A compression method for removing redundancy. The method replaces a repeated sequence, run, of the same symbol with two entities: a count and the symbol itself.

S-box An encryption device made of decoders, P-boxes, and encoders.

sample and hold A sampling method that samples the amplitude of an analog signal and holds the value until the next sample.

sampling The process of obtaining amplitudes of a signal at regular intervals.

sampling rate The number of samples obtained per second in the sampling process.

satellite network A combination of nodes that provides communication from one point on the earth to another.

scatternet A combination of piconets.

scrambling In digital-to-digital conversion, modifying part of the rules in a line coding scheme to create bit synchronization.

secondary station In the poll/select access method, a station that sends a response in answer to a command from a primary station.

secret-key encryption A security method in which the key for encryption is the same as the key for decryption; both sender and receiver have the same key.

Secure Hash Algorithm (SHA) A series of hash function standards developed by NIST and published as FIPS 180. It is mostly based on MD5.

Secure Key Exchange Mechanism (SKEME) A protocol for key exchange, designed by Hugo Krawcyzk, that uses public-key encryption for entity authentication.

Secure/Multipurpose Internet Mail Extensions (S/MIME) An enhancement to MIME designed to provide security for electronic mail.

Secure Shell (SSH) A client-server program that provides secure logging.

Secure Sockets Layer (SSL) protocol A protocol designed to provide security and compression services to data generated from the application layer.

security Protecting data from unauthorized access, damage, and modification.

Security Association (SA) An IPSec protocol that creates a logical connection between two hosts.

Security Association Database (SAD) A two-dimensional table with each row defining a single security association (SA).

security parameter index (SPI) A parameter that uniquely distinguish one security association from the others.

Security Policy (SP) In IPSec, a set of predefined security requirements applied to a packet when it is to be sent or when it has arrived.

Security Policy Database (SPD) A database of security policies (SPs).

segment The packet at the TCP layer. Also, the length of the transmission medium shared by devices.

segmentation The splitting of a message into multiple packets; usually performed at the transport layer.

segmentation and reassembly (SAR) The lower AAL sublayer in the ATM protocol in which a header and/or trailer may be added to produce a 48-byte element.

select In the poll/select access method, a procedure in which the primary station asks a secondary station if it is ready to receive data.

selective-repeat (SR) protocol An error-control protocol in which only the frame in error is resent.

self-synchronization Synchronization of long strings of 1s or 0s through the coding method.

sender The device that sends the data message. It can be a computer, a telephone handset, a video camera, and so on.

sequence number The number that denotes the location of a frame or packet in a message.

serial transmission Transmission of data one bit at a time using only one single link.

server A program that can provide services to other programs, called clients.

server control point (SCP) The point that controls the whole operation of the network.

Session Initiation Protocol (SIP) In voice over IP, an application protocol that establishes, manages, and terminates a multimedia session.

session layer The fifth layer of the OSI model, responsible for the establishment, management, and termination of logical connections between two end users.

setup phase In virtual circuit switching, a phase in which the source and destination use their global addresses to help switches make table entries for the connection.

Shannon capacity The theoretical highest data rate for a channel.

shielded twisted-pair (STP) Twisted-pair cable enclosed in a foil or mesh shield that protects against electromagnetic interference.

shift cipher A type of additive cipher in which the key defines shifting of characters toward the end of the alphabet.

shift register A register in which each memory location, at a time click, accepts the bit at its input port, stores the new bit, and displays it on the output port.

short interframe space (SIFS) In CSMA/CA, a period of time that the destination waits after receiving the RTS.

shortest path tree A routing table formed by using Dijkstra's algorithm.

signal element The shortest section of a signal (timewise) that represents a data element.

signal point (SP) A point to which the user telephone or computer is connected to.

signal rate The number of signal elements sent in one second.

signal-to-noise ratio (SNR) The ratio of average signal power to average noise power.

signal transport port (STP) A node that receive and forward signaling messages.

signaling connection control point (SCCP) A control point is used for special services such as 800-call processing.

signaling System Seven (SS7) The signaling system provided in the telephone network.

silly window syndrome A situation in which a small window size is advertised by the receiver and a small segment sent by the sender.

simple and efficient adaptation layer (SEAL) An AAL layer designed for the Internet (AAL5).

simple bridge A networking device that links two segments; requires manual maintenance and updating.

simple data type Atomic data types.

Simple Mail Transfer Protocol (SMTP) The TCP/IP protocol defining electronic mail service on the Internet.

Simple Network Management Protocol (SNMP) The TCP/IP protocol that specifies the process of management in the Internet.

Simple Protocol The simple protocol we used to show an access method without flow and error control.

simplex mode A transmission mode in which communication is one-way.

sine wave An amplitude-versus-time representation of a rotating vector.

single-bit error Error in a data unit in which only a single bit has been altered.

single-mode fiber An optical fiber with an extremely small diameter that limits beams to a few angles, resulting in an almost horizontal beam.

site local address An IPv6 address for a site having several networks but not connected to the Internet.

SKEME Another protocol for key exchange. It uses public-key encryption for entity authentication in a key-exchange protocol

sky propagation Propagation of radio waves into the ionosphere and then back to earth.

slash notation A shorthand method to indicate the number of 1s in the mask.

sliding window A window that can change its size by sliding over data items.

sliding window protocol A protocol that uses sliding window.

slotted ALOHA The modified ALOHA access method in which time is divided into slots and each station is forced to start sending data only at the beginning of the slot.

slow convergence A RIP shortcoming apparent when a change somewhere in the Internet propagates very slowly through the rest of the Internet.

slow start A congestion-control method in which the congestion window size increases exponentially at first.

socket An end point for a process; two sockets are needed for communication.

socket address A structure holding an IP address and a port number.

socket interface A set of system calls used in client-server paradigm.

Software Defined Radio (SDR) A radio communication system in which traditional hardware components are implemented in software.

source quench A method, used in ICMP for flow control, in which the source is advised to slow down or stop the sending of datagrams because of congestion.

source routing Explicitly defining the route of a packet by the sender of the packet.

source-based tree A tree used for multicasting by multicasting protocols in which a single tree is made for each combination of source and group.

source-to-destination delivery The transmission of a message from the original sender to the intended recipient.

space propagation A type of propagation that can penetrate the ionosphere.

space-division switching Switching in which the paths are separated from each other spatially.

spanning tree A tree with the source as the root and group members as leaves; a tree that connects all of the nodes.

spatial compression Compressing an image by removing redundancies.

spectrum The range of frequencies of a signal.

split horizon A method to improve RIP stability in which the router selectively chooses the interface from which updating information is sent.

split-horizon strategy A solution to instability in which each node sends only part of its table through each interface.

spread spectrum (SS) A wireless transmission technique that requires a bandwidth several times the original bandwidth.

Standard Ethernet The original Ethernet operating at 10 Mbps.

star topology A topology in which all stations are attached to a central device (hub).

start bit In asynchronous transmission, a bit to indicate the beginning of transmission.

state transition diagram A diagram to illustrate the states of a finite state machine.

static document On the World Wide Web, a fixed-content document that is created and stored in a server.

static mapping A technique in which a list of logical and physical address correspondences is used for address resolution.

static routing A type of routing in which the routing table remains unchanged.

stationary host A host that remains attached to one network.

statistical TDM A TDM technique in which slots are dynamically allocated to improve efficiency.

status line In the HTTP response message, a line that consists of the HTTP version, a space, a status code, a space, a status phrase.

steganography A security technique in which a message is concealed by covering it with something else.

stop bit In asynchronous transmission, one or more bits to indicate the end of transmission.

Stop-and-Wait Protocol A protocol in which the sender sends one frame, stops until it receives confirmation from the receiver, and then sends the next frame.

store-and-forward switch A switch that stores the frame in an input buffer until the whole packet has arrived.

straight tip connector A type of fiber-optic cable connector using a bayonet locking system.

STREAM One of the interfaces that have been defined for network programming.

stream cipher A type of cipher in which encryption and decryption are done one symbol (such as a character or a bit) at a time.

Stream Control Transmission Protocol (SCTP) The transport layer protocol designed to combine the features of UDP and TCP.

stream identifier Each stream is assigned a stream identifier (SI) which uniquely defines that stream.

stream sequence number (SSN) In ordered data delivery in SCTP, data chunks in a stream use stream sequence numbers (SSNs) to define their order in the stream.

stream socket A structure designed to be used with a connection-oriented protocol such as TCP.

streaming live audio/video Broadcast data from the Internet that a user can listen to or watch.

streaming stored audio/video Data downloaded as files from the Internet that a user can listen to or watch.

strict source route option An option that can be used by the source to predetermine a route for the datagram as it travels through the Internet.

strong collision Creating two messages with the same digest.

Structure of Management Information (SMI) In SNMP, a component used in network management.

structured data type A complex data type made of some simple or structured data types.

STS multiplexer/demultiplexer A SONET device that multiplexes and demultiplexes signals.

stub link A network that is connected to only one router.

subnet A subnetwork.

subnet address The network address of a subnet.

subnet mask The mask for a subnet.

subnetwork A part of a network.

substitution cipher A cipher that replaces one symbol with another.

suffix The varying part (similar to the hostid) of an IP address.

summary link to AS boundary router LSA An LSA packet that lets a router inside an area know the route to an autonomous boundary router.

summary link to network LSA An LSA packet that finds the cost of reaching networks outside of the area.

supergroup A signal composed of five multiplexed groups.

supernet A network formed from two or more smaller networks.

supernet mask The mask for a supernet.

switch A device connecting multiple communication lines together.

switched/56 service The digital version of an analog switched line that allows data rates of up to 56 kbps.

switched virtual circuit (SVC) A virtual circuit transmission method in which a virtual circuit is created and in existence only for the duration of the exchange.

switching office The place where telephone switches are located.

symmetric-key cipher A cipher using a symmetric-key cryptosystem.

symmetric-key cryptography A cipher in which the same key is used for encryption and decryption.

SYN flooding attack A serious security problem in the TCP connection establishment phase in which one or more malicious attackers send a large number of SYN segments.

synchronous connection-oriented (SCO) link In a Bluetooth network, a physical link created between a master and a slave that reserves specific slots at regular intervals.

Synchronous Digital Hierarchy (SDH) The ITU-T equivalent of SONET.

Synchronous Optical Network (SONET) A standard developed by ANSI for fiber-optic technology that can transmit high-speed data. It can be used to deliver text, audio, and video.

synchronous TDM A TDM technique in which each input has an allotment in the output even when it is not sending data.

synchronous transmission A transmission method that requires a constant timing relationship between the sender and the receiver.

synchronous transport signal (STS) A signal in the SONET hierarchy.

syndrome A sequence of bits generated by applying the error checking function to a codeword.

tandom office If the LATA is geographically large, a call may go through a tandem office (toll office) and the subscriber will pay a fee for the call.

T-lines A hierarchy of digital lines designed to carry speech and other signals in digital forms.

TCP/IP protocol suite A group of hierarchical protocols used in an internet.

teardown phase In virtual circuit switching, the phase in which the source and destination inform the switch to erase their entry.

telecommunications Exchange of information over distance using electronic equipment.

teleconferencing Audio and visual communication between remote users.

Teledesic A system of satellites that provides fiber-optic communication (broadband channels, low error rate, and low delay).

telephone user port (TUP) A port that is responsible for setting up voice calls.

temporal compression An MPEG compression method in which redundant frames are removed.

temporal masking A situation where a loud sound can numb our ears for a short time even after the sound has stopped.

Terminal Network (TELNET) A general purpose client-server program for remote logging.

thermal noise The noise created by the random motion of electrons in a wire, which creates an extra signal not originally sent by the transmitter.

three-way handshake A sequence of events for connection establishment or termination consisting of the request, then the acknowledgment of the request, and then confirmation of the acknowledgment.

throughput The number of bits that can pass through a point in one second.

ticket An encrypted message intended for entity B, but sent to entity A for delivery.

time to live (TTL) The lifetime of a packet.

time-division duplex TDMA (TDD-TDMA) In a Bluetooth network, a kind of half-duplex communication in which the slave and receiver send and receive data, but not at the same time (half-duplex).

time-division multiple access (TDMA) A multiple access method in which the bandwidth is just one time-shared channel.

time-division multiplexing (TDM) The technique of combining signals coming from low-speed channels to share time on a high-speed path.

time-division switching A circuit-switching technique in which time-division multiplexing is used to achieve switching.

time-domain plot A graphical representation of a signal's amplitude versus time.

timestamp A field in the packet related to the absolute or relative time the packed is created or sent.

timestamp option An option that is used to record the time of datagram processing by a router. The time is expressed in milliseconds from midnight.

token A small packet used in the token-passing access method.

token bucket An algorithm that allows idle hosts to accumulate credit for the future in the form of tokens.

token passing An access method in which a token is circulated in the network. The station that captures the token can send data.

topology The structure of a network including physical arrangement of devices.

traffic control A method for shaping and controlling traffic in a wide area network.

traffic shaping A mechanism to improve QoS that controls the amount and the rate of the traffic sent to the network.

trailer Control information appended to a data unit.

transceiver A device that both transmits and receives.

transient link A network with several routers attached to it.

transition mobility In IEEE 802.11, a station with BSS-transition mobility can move from one BSS to another, but the movement is confined inside one ESS. A station with no-transition mobility is either stationary (not moving) or moving only inside a BSS.

transmission capabilities application port (TACP) The signaling connection control point used for special services such as 800-call processing.

Transmission Control Protocol (TCP) A transport-layer protocol in the TCP/IP protocol suite.

Transmission Control Protocol/Internetwork Protocol (TCP/IP) Afive-layer protocol suite that defines the exchange of transmissions across the Internet.

transmission medium The physical path linking two communication devices.

transmission path (TP) In ATM, a physical connection between an end-point and a switch or between two switches.

transmission rate The number of bits sent per second.

transmission sequence number (TSN) A sequence number used by SCTP to number the data chunks. In other words, the TSN in SCTP plays the analogous role as the sequence number in TCP.

transparency The ability to send any bit pattern as data without it being mistaken for control bits.

transparent switch A switch in which the stations are completely unaware of the its existence. If a switch is added or deleted from the system, reconfiguration of the stations is unnecessary. According to the IEEE 802.1d specification, a system equipped with transparent switches must meet some criteria.

transport layer The fourth layer in the Internet and OSI model; responsible for reliable end-to-end delivery and error recovery.

Transport Layer Interface (TLI) A networking API provided by the UNIX system.

Transport Layer Security (TLS) protocol A security protocol at the transport level designed to provide security on the WWW. An IETF version of the SSL protocol.

transport mode Encryption in which a TCP segment or a UDP user datagram is first encrypted and then encapsulated in an IPv6 packet.

transposition cipher A cipher that transposes symbols in the plaintext to create the ciphertext.

Trap In SNMP, a PDU sent from an agent to the manager to report an event.

triangle routing In mobile IP, the less severe inefficiency case that occurs when the remote host communicates with a mobile host that is not attached to the same network (or site) as the mobile host.

triangulation The same as trilateration, but using three angles instead of three distances.

trilateration A two-dimensional method of finding a location given the distances from three different points.

trunk Transmission media that handle communications between offices.

tunnel mode A mode in IPSec that protects the entire IP packet. It takes an IP packet, including the header, applies IPSec security methods to the entire packet, and then adds a new IP header.

tunneling In multicasting, a process in which the multicast packet is encapsulated in a unicast packet and then sent through the network. In VPN, the encapsulation of an encrypted IP datagram in a second outer datagram. For IPv6, a strategy used when two computers using IPv6 want to communicate with each other when the packet must pass through a region that uses IPv4.

twisted-pair cable A transmission medium consisting of two insulated conductors in a twisted configuration.

two-binary, one quaternary (2B1Q) encoding A line encoding technique in which each pulse represents 2 bits.

two-dimensional parity check An error detection method in two dimensions.

type of service (TOS) A criteria or value that specifies the handling of the datagram.

unbalanced configuration An HDLC configuration in which one device is primary and the others secondary.

unguided media Transmission media with no physical boundaries (air).

unicast address An address belonging to one destination.

unicasting The sending of a packet to just one destination.

Unicode The international character set used to define valid characters in computer science.

unidirectional antenna An antenna that sends or receives signals in one direction.

uniform resource locator (URL) A string of characters (address) that identifies a page on the World Wide Web.

unipolar encoding A digital-to-digital encoding method in which one nonzero value represents either 1 or 0; the other bit is represented by a zero value.

unique local unicast block A sub-block that can be privately created and used by a site. The packet carrying this type of address as the destination address is not expected to be routed. This type of address has the identifier 1111 110, the next bit can be 0 or 1 to define how the address is selected.

Universal Mobile Telecommunication System (UMTS) One of the popular 3G technologies using a version of CDMA called Direct Sequence Wideband CDMA (DS-WCDMA).

unshielded twisted-pair (UTP) A cable with wires that are twisted together to reduce noise and crosstalk. See also *twisted-pair cable* and *shielded twisted-pair.*

unspecified bit rate (UBR) The data rate of an ATM service class specifying only best-effort delivery.

uplink Transmission from an earth station to a satellite.

uploading Sending a local file or data to a remote site.

upstream data band The band from the subscriber premises to the Internet that is divided into 6-MHz channels.

user agent (UA) An SMTP component that prepares the message, creates the envelope, and puts the message in the envelope.

user authentication A security measure in which the sender identity is verified before the start of a communication.

user datagram The name of the packet in the UDP protocol.

User Datagram Protocol (UDP) A connectionless TCP/IP transport-layer protocol.

user-to-network interface (UNI) In ATM, the interface between an end point (user) and an ATM switch.

variable bit rate (VBR) The data rate of an ATM service class for users needing a varying bit rate.

verification tag A 32-bit field that matches a packet to an association. This prevents a packet from a previous association from being mistaken as a packet in this association. It serves as an identifier for the association; it is repeated in every packet during the association.

video Recording or transmitting of a picture or a movie.

video band The band that occupies frequencies from 54 to 550 MHz and is used for video communication.

Vigenere cipher A polyalphabetic substitution scheme that uses the position of a character in the plaintext and the character's position in the alphabet.

virtual circuit (VC) A logical circuit made between the sending and receiving computers.

virtual circuit switching A switching technique used in switched WANs.

virtual link An OSPF connection between two routers that is created when the physical link is broken. The link between them uses a longer path that probably goes through several routers.

virtual local area network (VLAN) A technology that divides a physical LAN into virtual workgroups through software methods.

virtual path (VP) A combination of virtual circuits in ATM.

virtual private network (VPN) A technology that creates a network that is physically public, but virtually private.

virtual tributary (VT) A partial payload that can be inserted into a SONET frame and combined with other partial payloads to fill out the frame.

voice over IP A technology in which the Internet is used as a telephone network.

Walsh table In CDMA, a two-dimensional table used to generate orthogonal sequences.

wavelength The distance a simple signal can travel in one period.

wavelengh-division multiplexing (WDM) The combining of modulated light signals into one signal.

web of trust In PGP, the key rings shared by a group of people.

web page A unit of hypertext or hypermedia available on the Web.

weighted fair queuing A packet scheduling technique to improve QoS in which the packets are assigned to queues based on a given priority number.

well-known port number A port number that identifies a process on the server.

wide area network (WAN) A network that uses a technology that can span a large geographical distance.

wide area telephone service (WATS) A telephone service opposite of the 800 service. This service is a less expensive alternative to regular toll calls.

window scale factor An option in TCP that allows for increasing the window size defined in the header.

working group An IETF committee concentrating on a specific Internet topic.

World Wide Web (WWW) A multimedia Internet service that allows users to traverse the Internet by moving from one document to another via links that connect them.

Worldwide Interoperability for Microwave Access (WiMAX) A family of IEEE 802.16 standards to deliver wireless data at the last mile (similar to cable or DLS networks for wired communication).

X.509 A recommendation devised by ITU and accepted by the Internet that defines certificates in a structured way.

YCM A method in coloring in which a color is made of a combination of three other primary colors: yellow, cyan, and magenta.

zero compression A compression that can be applied to colon hexadecimal notation if there are consecutive sections consisting of zeros only. We can remove all the zeros altogether and replace them with a double semicolon.

zone In DNS, what a server is responsible for or has authority over.

REFERENCES

[AL 98] Albitz, P., and Liu, C. *DNS and BIND,* 3rd ed. Sebastopol, CA: O'Reilly, 1998.

[AZ 03] Agrawal, D., and Zeng, Q. *Introduction to Wireless and Mobile Systems.* Pacific Grove, CA: Brooks/Cole Thomson Learning, 2003.

[Bar 02] Barr, T, *Invitation to Cryptology.* Upper Saddle River, NJ: Prentice Hall, 2002.

[Bar et al. 05] Barrett, D. J., Silverman, R. E., and Byrnes, Robert G. *SSH: The Secure Shell: The Definitive Guide.* Sebastopol, CA: O'Reilly, 2005.

[Bel 01] Bellamy, J. *Digital Telephony.* New York: Wiley, 2001.

[Ber 96] Bergman, J. *Digital Baseband Transmission and Recording.* Boston, MA: Kluwer, 1996.

[Bis 03] Bishop, D. *Introduction to Cryptography with Java Applets.* Sebastopol, CA: O'Reilly, 2003.

[Bis 05] Bishop, M. *Introduction to Computer Security.* Reading, MA: Addison-Wesley, 2005.

[Bla 00a] Black, U. *QOS in Wide Area Networks.* Upper Saddle River, NJ: Prentice Hall, 2000.

[Bla 00b] Black, U. *PPP and L2TP: Remote Access Communication.* Upper Saddle River, NJ: Prentice Hall, 2000.

[Bla 03] Blahut, R. *Algebraic Codes for Data Transmission.* Cambridge, UK: Cambridge University Press, 2003.

[BYL 09] Buford, J. F., Yu, H., and Lua, E. K. *P2P Networking and Applications.* San Francisco, CA: Morgan Kaufmann, 2009.

[CBR 03] Cheswick, W., Bellovin, S., and Rubin, A. *Firewalls and Internet Security.* Reading, MA: Addison-Wesley, 2003.

[CD 08] Calvert, K. L., and Donaho, M. J. *TCP/IP Sockets in Java.* San Francisco, CA: Morgan Kaufmann, 2008.

[Cer 89] Cerf, V. *A History of Arpanet, The Interoperability Report,* 1989.

[CHW 99] Crowcroft, J., Handley, M., and Wakeman, I. *Internetworking Multimedia.* San Francisco, CA: Morgan Kaufmann, 1999.

[Com 00] Comer, D. *Internetworking with TCP/IP, vol. 1: Principles, Protocols, and Architecture.* Upper Saddle River, NJ: Prentice Hall, 2000.

[Com 04] Comer, D. *Computer Networks.* Upper Saddle River, NJ: Prentice Hall, 2004.

[Com 06] Comer, Douglas E. *Internetworking with TCP/IP,* vol. 1. Upper Saddle River, NJ: Prentice Hall, 2006.

[Cou 01] Couch, L. *Digital and Analog Communication Systems.* Upper Saddle River, NJ: Prentice Hall, 2001.

[DC 01] Donaho, M. J., and Calvert, K. L. *TCP/IP Sockets: C version.* San Francisco, CA: Morgan Kaufmann, 2001.

[DH 03] Doraswamy, H., and Harkins, D. *IPSec.* Upper Saddle River, NJ: Prentice Hall, 2003.

[Dro 02] Drozdek A. *Elements of Data Compression.* Pacific Grove, CA: Brooks/Cole (Thomson Learning), 2002.

[Dut 01] Dutcher, D. *The NAT Handbook.* New York: Wiley, 2001.

[Far 04] Farrel, A. *The Internet and Its Protocols.* San Francisco: Morgan Kaufmann, 2004.

[FH 98] Ferguson, P., and Huston, G. *Quality of Service.* New York: John Wiley & Sons, Inc., 1998.

[For 03] Forouzan, B. *Local Area Networks.* New York: McGraw-Hill, 2003.

[For 08] Forouzan, B., *Cryptography and Network Security.* New York: McGraw-Hill, 2008.

[For 10] Forouzan, B., *TCP/IP Protocol Suite.* New York: McGraw-Hill, 2010.

[Fra 01] Frankkel, S. *Demystifying the IPSec Puzzle.* Norwood, MA: Artech House, 2001.

[Fre 96] Freeman, R. *Telecommunication System Engineering.* New York: Wiley, 1996.

[Gar 01] Garret, P. *Making, Breaking Codes.* Upper Saddle River, NJ: Prentice Hall, 2001.

[Gar 95] Garfinkel, S. *PGP: Pretty Good Privacy.* Sebastopol. CA: O'Reilly, 1995.

[Gas 02] Gast, M. *802.11 Wireless Networks.* Sebastopol, CA: O'Reilly, 2002.

[Gb et al. 98] Gibson, J. D., Gerger, T., Lookabaugh, T., LindBerg, D., and Baker, R. L. *Digital Compression for Multimedia.* San Francisco: Morgan Kaufmann, 1998.

[GW 04] Garcia, A., and Widjaja, I. *Communication Networks.* New York: McGraw-Hill, 2004.

[Hal 01] Halsall, F. *Multimedia Communication.* Reading, MA: Addison-Wesley, 2001.

[Ham 80] Hamming, R. *Coding and Information Theory.* Upper Saddle River, NJ: Prentice Hall, 1980.

[Har 05] Harol, Elliot R. *Java Network Programming.* Sebastopol, CA: O'Reilly, 2005.

[HM 10] Havaldar, P., and Medioni, G. *Multimedia Systems: Algorithms, Standards, and Industry Practices.* Boston: Course Technology (Cengage Learning), 2010.

[Hsu 03] Hsu, H. *Analog and Digital Communications.* New York: McGraw-Hill, 2003.

[Hui 00] Huitema, C. *Routing in the Internet,* 2nd ed. Upper Saddle River, NJ: Prentice Hall, 2000.

[Izz 00] Izzo, P. *Gigabit Networks.* New York: Wiley, 2000.

[Jam 03] Jamalipour, A. *Wireless Mobile Internet.* New York: Wiley, 2003.

[Jen et al. 86] Jennings, D. M., Landweber, L. M., Fuchs, I. H., Farber, D. H., and Adrion, W. R. "Computer Networking for Scientists and Engineers," *Science* 231, no. 4741 (1986): 943–950.

[KCK 98] Kadambi, J., Crayford, I., and Kalkunte, M. *Gigabit Ethernet.* Upper Saddle River, NJ: Prentice Hall, 1998.

[Kei 02] Keiser, G. *Local Area Networks.* New York: McGraw-Hill, 2002.

[Kes 02] Keshav, S. *An Engineering Approach to Computer Networking.* Reading, MA: Addison-Wesley, 2002.

[Kle 04] Kleinrock, L. *The Birth of the Internet.*

[KMK 04] Kumar, A., Manjunath, D., and Kuri, J. *Communication Network: An Analytical Approach.* San Francisco: Morgan Kaufmann, 2004.

[Koz 05] Kozierock, C. M. *The TCP/IP Guide.* San Francisco: No Starch Press, 2005.

[KPS 02] Kaufman, C., Perlmann, R., and Speciner, M. *Network Security.* Upper Saddle River, NJ: Prentice Hall, 2000.

[KR 05] Kurose, J. and Ross, K. *Computer Networking.* Reading, MA: Addison-Wesley, 2005.

[Lei et al. 98] Leiner, B., Cerf, V., Clark, D., Kahn, R., Kleinrock, L., Lynch, D., Postel, J., Roberts, L., and Wolff, S., "A Brief History of the Internet," *Internet Society,* 1998, http://www.isoc.org/internet/history/brief.shtml.

[Los 04] Loshin, P. *IPv6: Theory, Protocol, and Practice.* San Francisco: Morgan Kaufmann, 2004.

[Mao 04] Mao, W. *Modern Cryptography.* Upper Saddle River, NJ: Prentice Hall, 2004.

[Max 99] Maxwell, K. *Residential Broadband.* New York: Wiley, 1999.

[Mir 07] Mir, N. F. *Computer and Communication Networks.* Upper Saddle River, NJ: Prentice Hall, 2007.

[MOV 97] Menezes, A., Oorschot, P., and Vanstone, S. *Handbook of Applied Cryptography.* New York: CRC Press, 1997.

[Moy 98] Moy, J. *OSPF.* Reading, MA: Addison-Wesley, 1998.

[MS 01] Mauro, D., and Schmidt, K. *Essential SNMP.* Sebastopol, CA: O'Reilly, 2001.

[PD 03] Peterson, L. L., and Davie, B. S. *Computer Networks,* 3rd ed. San Francisco: Morgan Kaufmann, 2003.

[Pea 92] Pearson, J. *Basic Communication Theory.* Upper Saddle River, NJ: Prentice Hall, 1992.

[Per 00] Perlman, R. *Interconnections,* 2nd ed. Reading, MA: Addison-Wesley, 2000.

[PHS 03] Pieprzyk, J., Hardjono, T, and Seberry, J, *Fundamentals of Computer Security.* Berlin: Springer, 2003.

[Pit 06] Pitt, E. *Fundamental Networking in Java.* Berlin: Springer-Verlag, 2006.

[PKA 08] Poo, D., Kiong, D., and Ashok, S. *Object-Oriented Programming and Java.* Berlin: Springer-Verlag, 2008.

[Res 01] Rescorla, E. *SSL and TLS.* Reading, MA: Addison-Wesley, 2001.

[Rhe03] Rhee, M, *Internet Security.* New York: Wiley, 2003.

[Ror 96] Rorabaugh, C. *Error Coding Cookbook.* New York: McGraw-Hill, 1996.

[RR 96] Robbins, K. A., and Robbins, S. *Practical UNIX Programming,* Upper Saddle River, NJ: Prentice Hall, 1996.

[Sau 98] Sauders, S. *Gigabit Ethernet Handbook.* New York: McGraw-Hill, 1998.

[Sch 03] Schiller, J. *Mobile Communications.* Reading, MA: Addison-Wesley, 2003.

[Sch 96] Schneier, B. *Applied Cryptography.* Reading, MA: Addison-Wesley, 1996.

[Seg 98] Segaller, S. Nerds 2.0.1: *A Brief History of the Internet.* New York: TV Books, 1998.

[SFD 04] Stevens, W. R., Fenner, B., and Rudoff, A. M. *UNIX Network Programming: The Sockets Networking API.* Reading, MA: Addison-Wesley, 2004.

[Sna 00] Snader, J. C. *Effective TCP/IP Programming*. Reading, MA: Addison-Wesley, 2000.

[Sal 03] Solomon, D. *Data Privacy and Security.* Berlin: Springer, 2003.

[Spi 74] Spiegel, M. *Fourier Analysis.* New York: McGraw-Hill, 1974.

[Spu 00] Spurgeon, C. *Ethernet.* Sebastopol, CA: O'Reilly, 2000.

[SSS 05] Shimonski, R., Steiner, R., and Sheedy, S. *Network Cabling Illuminated.* Sudbury, MA: Jones and Bartlette, 2005.

[Sta 02] Stallings, W. *Wireless Communications and Networks.* Upper Saddle River, NJ: Prentice Hall, 2002

[Sta 04] Stallings, W. *Data and Computer Communications,* 7th ed. Upper Saddle River, NJ: Prentice Hall, 2004.

[Sta 06] Stallings, W. *Cryptography and Network Security,* 5th ed. Upper Saddle River, NJ: Prentice Hall, 2006.

[Sta 98] Stallings, W. *High Speed Networks.* Upper Saddle River, NJ: Prentice Hall, 1998.

[Ste 94] Stevens, W. R. *TCP/IP Illustrated,* vol. 1. Reading, MA: Addison-Wesley, 1994.

[Ste 95] Stevens, W. R. *TCP/IP Illustrated,* vol. 2. Reading, MA: Addison-Wesley, 1995.

[Ste 96] Stevens, W. *TCP/IP Illustrated,* vol. 3. Upper Saddle River, NJ: Prentice Hall, 2000.

[Ste 99] Stewart, J. W. III. *BGP4: Inter-Domain Routing in the Internet.* Reading, MA: Addison-Wesley, 1999.

[Sti 06] Stinson, D. *Cryptography: Theory and Practice.* New York: Chapman & Hall/CRC, 2006.

[Sub 01] Subramanian, M. *Network Management.* Reading, MA: Addison-Wesley, 2000.

[SW 05] Steinmetz, R., and Wehrle, K. *Peer-to-Peer Systems and Applications.* Berlin: Springer-Verlag, 2005.

[SWE 99] Scott, C., Wolfe, P, and Erwin, M. *Virtual Private Networks.* Sebastopol, CA: O'Reilly, 1998.

[SX 02] Stewart, R. R., and Xie, Q. *Stream Control Transmission Protocol (STCP).* Reading, MA: Addison-Wesley, 2002.

[Tan 03] Tanenbaum, A. S. *Computer Networks,* 4th ed. Upper Saddle River, NJ: Prentice Hall, 2003.

[Tho 00] Thomas, S. *SSL and TLS Essentials.* New York, NY: John Wiley & Sons, 2000.

[WV 00] Warland, J., and Varaiya, P. *High Performance Communication Networks.* San Francisco, CA: Morgan Kaufmann, 2000.

[WZ 01] Wittmann, R., and Zitterbart, M. *Multicast Communication.* San Francisco, CA: Morgan Kaufmann, 2001.

[YS 01] Yuan, R., and Strayer, W. *Virtual Private Network.* Reading, MA: Addison-Wesley, 2001.

[Zar 02] Zaragoza, R. *The Art of Error Correcting Coding.* Reading, MA: Addison-Wesley, 2002.

INDEX

Numerics

1-persistent method, 96
10 Gigabit Ethernet (10 Gbps), 116–117, 126
10GBase-EW, 126
10GBase-LR, 126
10GBase-SR, 126
10GBase-X4, 126
56K modems, 158
64-bit extended unique identifier (EUI-64), 260
100Base-FX, 123
100Base-T4, 122
100Base-TX, 123
800 service, 156
802.1Q (IEEE), 199
802.11 (IEEE), 126, 127
802.11 DSSS (IEEE), 136
802.11 FHSS (IEEE), 135–136
802.11 infrared (IEEE), 136–137
802.11a OFDM (IEEE), 137
802.11b HR-DSSS (IEEE), 137
802.11g (IEEE), 138
802.11n (IEEE), 138
900 services, 156
1000Base-CX, 125
1000Base-LX, 125
1000Base-SX, 125
1000Base-T, 125

A

ABM. *See* asynchronous balanced mode (ABM)
Abstract Syntax Notation One (ASN.1), 627–632
  data types, 629–632
  encoding, 632
  key words, 628–629
  language basics, 628–629
  symbols, 628
access conflict, 89
access control, 680
access method, Ethernet, 121
access point (AP), 127
accuracy, 2
ACK. *See* acknowledgment (ACK)
ACK segment, 373
acknowledgment (ACK), 83, 102, 350, 368, 390, 416
ACL. *See* asynchronous connectionless link (ACL)
active documents, 453
active open, 371
ad hoc architecture, 127
adaptive DM (ADM), 554
adaptive DPCM (ADPCM), 557
additive cipher, 643
additive increase, 401–402
  multiplicative decrease (AIMD), 406–407
address
  aggregation, 218–219, 250–253, 318
  depletion, 213
  field, 82
  mask, 215
  network, 216
  space, 210–211, 258, 501–503
Address Resolution Protocol (ARP), 107, 219, 272
addresses, 471
addressing, 119
  mechanism, 133–135
addressing, IPv4, 210–219
  address aggregation, 218–219
  address depletion, 213
  address mask, 215
  address space, 210–211
  classful, 212–213
  classless, 213–217
  hierarchy in, 211–212
  network address, 216
  notation, 211
  subnetting, 217–218
addressing, IPv6, 257–264
  address space, 258
  anycast address, 258
  compatible address, 262
  global unicast address block, 259–260
  mapped address, 262
  mapping ethernet MAC, 260–261
  mapping EUI-64, 260
  multicast address, 258–259
  unicast address, 258
  unique local unicast block, 262–263
ADSL. *See* asymmetric DSL (ADSL)
Advanced Mobile Phone System (AMPS), 165–166
  bands, 165–166
  transmission, 166
agent advertisement, 241–242
agent discovery, 240
agent solicitation, 242
agents, 611
Alert Protocol, 691
alias, 471–472
ALOHA, 89–91
  pure, 89–92
  slotted, 93–94
alpha (α channel, 460
AM. *See* amplitude modulation (AM)
amplification, 35
amplitude modulation (AM), 45
amplitude shift keying (ASK), 42, 43
AMPS. *See* Advanced Mobile Phone System (AMPS)
analog leased services, 155, 156

analog signals, 31–33
analog switched services, 155–156
analog transmission, 42–47
analog-to-analog conversion, 45–47
analog-to-digital conversion, 41–42
analysis filter bank, 567
angle of incidence, 52
antenna, cellular telephony, 175
anycast address, 258
AP. *See* access point (AP)
aperiodic analog signals, 31
application gateway, 704
application layer, 20
  introduction, 438
  logical connection at, 438
  nonstandard application-layer protocol, 440
  paradigms, 440–442
  security, 691–702
  standard application-layer protocol, 439–440
  standard applications. *See* standard applications, application layer
application programming interface (API), 443–446
  socket interface, 443, 444
  STREAM, 443
  transport layer interface, 443
APs. *See* Authentication Protocols (APs)
architecture, 449–450, 469–470, 686–689
areas, OSPF, 309
arithmetic coding, 551–554
ARP. *See* Address Resolution Protocol (ARP)
AS. *See* autonomous system (AS)
ASK. *See* amplitude shift keying (ASK)
association, in SCTP, 413, 418–421
  establishment, 418–419
  termination, 421
asymmetric DSL (ADSL), 158
asymmetric-key ciphers, 653–658
  applications, 658
  encryption/decryption, 655
  general idea of, 654–655
  plaintext/ciphertext, 655
  RSA cryptosystem, 656–657
asymmetric-key cryptography, 653
asynchronous balanced mode (ABM), 81
asynchronous connectionless link (ACL), 143
AT&T, 152
attacks, security, 639–640
attenuation, 35
audio, 4, 566–567
  compression, 566–567
  digitizing, 566
audio compression
  perceptual coding, 566–567
  predictive coding, 566
authenticate state, 85
authentication
  extension header, 269
  OSPF, 312
Authentication Header (AH) protocol, 677–678
Authentication Protocols (APs), 86–87
autoconfiguration, 263–264
automatic configuration, VLANs, 198–199
automatically corrected lost ACK, 397
autonegotiation, Ethernet, 121–122
autonomous system (AS), 302, 303
availability, 638

B

backbone, 7, 12
backbone area, 309
backoff time, 90
Backus–Naur Form (BNF), 630
bandwidth, 33, 38
bandwidth-delay product, 39
base header, 223
base station (BS), 163
Base64 encoding, 478–479
baseband layer, 141–144
baseband transmission, 34
Basic Encoding Rules (BER), 617
basic service set (BSS), 127
BASK. *See* binary ASK (BASK)
beacon frame, 130
Bellman-Ford equation, 288–289
Berkeley Software Distribution (BSD), 303
BFSK. *See* binary FSK (BFSK)
BGP. *See* Border Gateway Protocol (BGP)
BGP peers, 314
BGP speakers, 314
bidirectional frame (B-frame), 565
big LEOs, 181
binary amplitude shift keying, 43
binary ASK (BASK), 43
binary exponential backoff, 91
binary FSK (BFSK), 44
binary notation, 211
binary PSK (BPSK), 44
bit depth, 460
bit length, 34
bit rate, 33, 158
bit stuffing, 69
bit-oriented ciphers, 647
bit-oriented framing, 68–70
BitTorrent, 442, 518–521
  with tracker, 519–520
  trackerless, 520–521
Blaatand, Harald, 138
block allocation, 216–217
block cipher, 647
block coding, 40–41, 71–72
block mode, 467
Bluetooth, 138–145
  ACL link, 143
  architecture, 138–140
  band, 144
  baseband layer, 141–144
  devices, 140
  frame format, 143–144

group management, 141
L2CAP layer, 140–141
layers, 140–145
modulation, 144–145
multiplexing, 140–141
parked state, 139
piconet, 138–139
primary station, 138–139
radio layer, 144–145
scatternet, 139
SCO link, 143
secondary station, 138–139
segmentation and reassembly, 141
TDD-TDMA, 141–142
Boggs, David, 116
Border Gateway Protocol (BGP), 313–322
external, 313, 314–315
internal, 313, 315–316
messages, 321–322
path attributes, 318–320
performance, 322
route selection, 320–321
border router, 313
BPSK. *See* binary PSK (BPSK)
broadband LEOs, 181
broadband transmission, 34
broadcast address, 107, 119
broadcast link, 65
browsers, 450–451
BS. *See* base station (BS)
BSD. *See* Berkeley Software Distribution (BSD)
BSS. *See* basic service set (BSS)
BSS-transition mobility, 128
buffers, 349, 364–365
burst error, 70
CRC and, 80
bus ring topology, 104
bus topology, 7
byte number, 367
byte-oriented, 416
byte-oriented framing, 67–68
byte-stuffing, 67–68, 85

C

cable modem (CM), 162
cable modem transmission system (CMTS), 162, 163
cable networks, 12
cable TV
amplifier, 160
data transfer, 161–162
drop cable, 160
head end, 160
hybrid fiber-coaxial, 160–161
networks, 52, 160
splitter, 160
tap, 160
traditional, 160
cache update, 464
caching, 494–495
Caesar cipher, 644
care-of address, 238
carrier extension, 124
carrier sense multiple access (CSMA), 89, 94–95
carrier sense multiple access with collision avoidance (CSMA/CA), 89, 101, 129
carrier sense multiple access with collision detection (CSMA/CD), 89, 97–101
collision and abortion in, 98
energy level, 100
Ethernet, 100–101
frame size, 99
throughput, 100
CATV. *See* community antenna TV (CATV)
CCK. *See* complementary code keying (CCK)
CDDI. *See* Copper Distributed Data Interface (CDDI)
cells, 67, 163
cellular telephony, 162–175
access scheme, 174
antenna, 175
base station, 163
first generation (1G), 165–166
fourth generation (4G), 174–175
frequency-reuse principle, 164
GSM, 168–170
handoff, 165
IMT-2000, 173
IS-95, 170–172
mobile station, 182
mobile switching center, 163
modulation, 175
paging, 164
query signal
radio system, 175
receiving, 164
reuse factor, 184
roaming, 165
second generation (2G), 166–172
third generation (3G), 173–174
transmitting, 164
centralized networks, 498–499
certification authority (CA), 673, 674
CF. *See* contention-free (CF) period
Challenge Handshake Authentication Protocol (CHAP), 86–87
challenge-response authentication, 667
using asymmetric-key cipher, 668
using digital signatures, 668
using symmetric-key encryption, 667–668
ChangeCipherSpec Protocol, 690
channel ID (CID), 140
CHAP. *See* Challenge Handshake Authentication Protocol (CHAP)
character-oriented ciphers, 647
character-oriented framing, 67–68
checksum, 80, 359, 360–361, 370, 390
optional inclusion of, 361

chord, 503–510
applications, 510
finger table, 503–504
identifier space, 503
interface, 504
lookup, 504–506
stabilize, 507–510
chunks, 417–418, 426–427
CID. *See* channel ID (CID)
CIDR. *See* classless interdomain routing (CIDR)
cipher suite, 686
ciphers, 642. *See also specific ciphers*
ciphertext, 641, 654
circuit-switched network, 152–153
cladding, 52
Clark's solution, 389
classful addressing, 212–213
classless addressing, 213–217
classless interdomain routing (CIDR), 214
CIDR notation, 258
clear to send (CTS), 129
CLECs. *See* competitive local exchange carriers (CLECs)
client, 343
processes, 443, 530
client protocol, 451
client/server paradigm, 343, 440–441, 443–448
application programming interface, 443–446
using services of transport layer, 447–448
CM. *See* cable modem (CM)
CMS. *See* Cryptographic Message Syntax (CMS)
CMTS. *See* cable modem transmission system (CMTS)
coaxial cable, 50, 51–52
code, 3
codewords, 71
Collaborative File System (CFS), 512
collision avoidance, 130
collision during handshaking, 130
collision elimination, link-layer switch, 194
collocated care-of address, 240
common carriers, 151
Common Gateway Interface (CGI), 452
community antenna TV (CATV), 160
compatible address, 262
competitive local exchange carriers (CLECs), 151
complementary code keying (CCK), 137
components, data communications, 2–3
composite signal, 33
compressed mode, 467
compression, 544
audio, 566–567
image, 561–564
lossless, 544–554
lossy, 554–559
video, 564–565
ConChord, 512
concurrent updating, 518
conditional request, 460
confidentiality, 638, 665, 680
asymmetric-key ciphers, 653–658
attacks threatening, 639
symmetric-key ciphers. *See* symmetric-key cipher
configuration, VLANs, 198–199
automatic, 198–199
manual, 198
semiautomatic, 199
configuration management
reconfiguration, 606–607
congestion, 352, 398
avoidance, 401–402
detection, 399
policies, 399–402
window, 398–399
congestion control, 206, 579
in transport layer, 352, 361, 427
connecting devices, 5, 188–196
categories of, 188
hub, 188–189
link-layer switch, 189–195
router, 195–196
connectionless service, 353–354, 359–360
Ethernet, 117
network layer, 207
connection-oriented service, 207, 354–355, 366, 371–377, 414
connections, 465–468
consumer networks, 301
contention methods, 88–101
contention-free (CF) period, 131
content-transfer-encoding, 478
contributing source (CSRC), 578
control, 369
control field, 82, 370
control frames, 132–133
controlled access, 101–104
polling, 101–102
reservation method, 101
token-passing method, 102–104
controller, 451
convolution coding, 71
cookies, 373, 460
creating and storing, 460–461
used for advertisement, 461
used for electronic store, 461
used in Web portal, 461
used to register client, 461
Copper Distributed Data Interface (CDDI), 104
core, 52
count to infinity, 292
country domain, 492–493
CRC. *See* cyclic redundancy check (CRC)
CRC, Ethernet frame, 117, 118
criteria, network, 5
critical angle, 52
crosstalk, 36
cryptographic hash function, 658
Cryptographic Message Syntax (CMS), 699–701
cryptography, 641
CSMA. *See* carrier sense multiple access (CSMA)

CSMA/CA. *See* carrier sense multiple access with collision avoidance (CSMA/CA)
CSMA/CD. *See* carrier sense multiple access with collision detection (CSMA/CD)
CTS. *See* clear to send (CTS)
cumTSN, 422
cumulative acknowledgment, 390
customer networks, 12
cwnd, 399
cyclic codes, 76–79
cyclic redundancy check (CRC), 77–80, 486

D

DA. *See* destination address (DA)
D-AMPS. *See* digital AMPS (D-AMPS)
data, 2
  Ethernet frame, 117, 118
  multimedia
    audio, 566–567
    image, 560–564
    text, 560
    video, 564–565
data communications
  components, 2–3
  data flow, 4–5
  defined, 2
  message, 3–4
Data Encryption Standard (DES), 649–652
  function, 650–651
  key generation, 651
  rounds, 650
  structure of, 650
data flow, 4–5
data frames, 133
data transfer, 373–375, 419–421
  cable TV for, 161–162
  mobile IP, 244–247
  network, 153
database description message, 311–312
datagram, 105
datagram approach, 207, 253
data-link control (DLC), 66–87, 116
  DLC protocols, 80–87
  error control, 70–80
  framing, 66–70
data-link layer, 20
  broadcast link, 65
  DLC, 66–87
  links, 65
  media access control (MAC), 66, 88–104
  nodes, 65
  overview, 64–65
  Point-to-Point Protocol (PPP), 65, 83–87, 88
datawords, 71
DCF. *See* distributed coordination function (DCF)
DCF interframe space (DIFS), 129
DDNS. *See* Dynamic Domain Name System (DDNS)
DDS. *See* digital data service (DDS)
dead state, 85
deadlock, 398
debugging, 235
decapsulation, 345–346, 361
decentralized networks, 499–500
decibel (dB), 35
decoder, 79
decoding, Ethernet, 120
decryption, 641
decryption algorithms, 642, 686
dedicated line, 156
dedicated point-to-point link, 6
delay, 208–209
  processing, 208
  propagation, 208
  queuing, 208
  total, 209
  transmission, 208
delayed segment, 396
delete mode, POP3, 476
delivery, 2
delta modulation (DM), 42, 554–555
  adaptive, 556
demodulator, 157
demultiplexer, 48, 346
demultiplexing, 346, 347, 362, 366
denial of service (DoS), 640
denial of service attack, 373
DES. *See* Data Encryption Standard (DES)
destination address (DA), 118
destination option, 268
destination port address, 368
destination router, 286
destination unreachable message, 234, 270
DFT. *See* discrete Fourier transform (DFT)
DHT. *See* distributed hash table (DHT)
dial-up service, 12, 156–158
dictionary coding, 545–546
differential PCM (DPCM), 556–557
differentiated services, 221
Diffie-Hellman Protocol, 672–673
DIFS. *See* DCF interframe space (DIFS)
digital AMPS (D-AMPS), 167
  band, 167
  transmission, 167
digital data service (DDS), 156
digital image, 460
digital service unit (DSU), 156
digital services, 156
digital signals, 33–34
digital signature, 660–666
  confidentiality, 665
  conventional signatures *vs.*, 661
  message authentication, 663
  message integrity, 663
  non-repudiation, 664
  process, 661–662
  schemes, 665–666
  services, 663–665
  signing digest, 663

Digital Signature Standard (DSS), 666
digital subscriber line (DSL), 12, 51, 158–159
digital subscriber line access multiplexer (DSLAM), 159
digital transmission, 40–42
digital-to-analog conversion, 42–44
digital-to-digital conversion, 40–41
digitization, 41–42, 460
Dijkstra's algorithm, 295–297, 309
direct connection, Internet, 13
direct sequence spread spectrum (DSSS), 136, 171
discrete cosine transform (DCT), 557
  one-dimensional, 557–559
  two-dimensional, 559
discrete Fourier transform (DFT), 567
distance learning, 324
Distance Vector Multicast Routing Protocol (DVMRP), 324–326
  reverse path broadcasting (RPB), 324, 325
  reverse path forwarding (RPF), 324–325
  reverse path multicasting (RPM), 324, 325–326
distance vectors, 289–291
distance-vector (DV) routing, 288–294
  Bellman-Ford equation, 288–289
  distance vectors, 289–291
distance-vector routing algorithm, 291–294
distortion, 35–36
distributed coordination function (DCF), 128, 129–130
distributed hash table (DHT), 500–503
  address space, 501
  arrival and departure of nodes, 502–503
  hashing object identifier, 501
  hashing peer identifier, 501
  routing, 502
  storing object, 501–502
distribution hubs, 161
distribution system, 127
Distributive Domain Name System (DDNS), 512
divisor, 79–80
DLC. *See* data-link control (DLC)
DLC protocols, 80–87
  high-level data-link control (HDLC), 81–83
  Point-to-Point Protocol (PPP), 83–87
DM. *See* delta modulation (DM)
DNS servers, 490
do not fragment bit, 226
documentation, 607–608
dog-leg routing, 246, 247
domain, 489, 490
domain name, 489–491
domain name space, 488
Domain Name System (DNS), 486–498
  caching, 494–495
  country domain, 492–493
  DDNS, 497
  domain name, 489–491
  encapsulation, 496–497
  generic domains, 492
  in Internet, 491–493
  iterative resolution, 494
  label, 488–489
  messages, 495–496
  name space, 488
  primary server, 491
  purpose of, 487
  recursive resolution, 493–494
  registrars, 497
  resolution, 493–495
  resolver, 493–494
  resource records, 495
  root server, 491
  secondary server, 491
  security of, 497–498
  zone, 490–491
dotted-decimal notation, 211
double crossing, 246
downlink, 177
downloading, 158
downstream data, 161, 162
DPCM. *See* differential PCM (DPCM)
drop line, 7
DSL. *See* digital subscriber line (DSL)
DSLAM. *See* digital subscriber line access multiplexer (DSLAM)
DSSS. *See* direct sequence spread spectrum (DSSS)
DSU. *See* digital service unit (DSU)
dual stack strategy, 273–274
dual-ring topology, 104
duplicate ACK, 391
duplicate segment, 396
DV. *See* distance-vector (DV) routing
DVMRP. *See* Distance Vector Multicast Routing Protocol (DVMRP)
dynamic documents, 452–453
Dynamic Domain Name System (DDNS), 497
dynamic ports, 345
Dynamic Querying (DQ), 500
dynamic/interval coding, 552–554

E

Earth-Centered Earth-Fixed (ECEF) reference frame, 180
eBGP. *See* external BGP (eBGP)
ECEF. *See* Earth-Centered Earth-Fixed (ECEF) reference frame
echo-reply message, 234, 271
echo-request message, 234, 271
e-commerce, 461
edges, 286
EGP. *See* exterior gateway protocol (EGP)
electromagnetic spectrum, 53
electronic mail (e-mail), 468–481
  addresses, 471
  alias, 471–472
  architecture, 469–470
  elm, 470
  Eudora, 470
  format of, 471
  mailbox, 469
  mailing/group list, 471–472

message access agent, 475–477
MIME, 477–480
Outlook, 470
pine, 470
pull program, 470
push program, 470
receiving mail, 470–471
security, 481
sending mail, 470
Simple Mail Transfer Protocol, 472–475
Web-based mail, 480–481
electronic serial number (ESN), 171–172
elm, 470
e-mail security, 481, 691–692
certificates, 692
cryptographic algorithms, 692
cryptographic secrets, 692
encapsulating security payload (ESP), 678–679
encapsulation, 244, 333, 345–346, 361, 371, 496–497
encoder, 78–79
encoding, 632
Fast Ethernet, 122
Gigabit Ethernet, 125
Standard Ethernet, 120
encrypted security payload (ESP), 269
encryption, 230, 641
encryption algorithms, 641, 686
end offices, 150
end-of-option option, 229
energy level, CSMA/CD, 100
engineering.mcgraw-hill.com, 493
entity authentication, 666–668, 680
challenge-response authentication, 667
message authentication *vs.*, 666
passwords, 667
something inherent, 666
something known, 666
something possessed, 666
verification categories, 666–667
ephemeral port number, 343
Ericsson Company, 138
error control, 70–80, 205, 349–352, 360, 389–398, 423–427, 579
block coding, 71–72
checksum, 80
coding, 71
decoder, 79
detection, 71–72
detection *versus* correction, 71
divisor, 79–80
Hamming distance, 73–74
parity-check code, 75–76
redundancy, 70
error correction, 71
error detection, 71–72
error-reporting messages, 232–233, 270–271
ESC. *See* escape character (ESC)
escape character (ESC), 67–68
ESN. *See* electronic serial number (ESN)
ESP. *See* encapsulating security payload (ESP); encrypted security payload (ESP)
ESS. *See* extended service set (ESS)
ESS-transition mobility, 128
establish state, 85
Ethernet, 100–101, 116–126
10 Gigabit Ethernet (10 Gbps), 116–117, 126
addressing, 119
connectionless service, 117
CRC, 117, 118
data, 117, 118
destination address (DA), 117, 118
Fast Ethernet (100 Mbps), 116–117, 121–123
frame format, 117–118
frame length, 118–119
Gigabit Ethernet (1 Gbps), 116–117, 123–126
implementation, 120
preamble, 117, 118
source address (SA), 117, 118
standard Ethernet (10 Mbps), 116–120
start frame delimiter (SFD), 117, 118
transmission of address bits, 119–120
type, 117, 118
unreliable service, 117
Eudora, 470
EUI-64. *See* 64-bit extended unique identifier (EUI-64)
exchangers, 471
exponential backoff, 410–411
exponential increase, 399–401
exposed-station problem, 134–135
extended service set (ESS), 127–128
Extensible Hypertext Markup Language (XHTML), 452
Extensible Markup Language (XML), 452
Extensible Style Language (XSL), 452
extension headers, 268
exterior gateway protocol (EGP), 302
external BGP (eBGP), 313
operation of, 314–315
external link, LSPs, 310

F

Fast Ethernet (100 Mbps), 116–117, 121–123
access method, 121
autonegotiation, 121–122
physical layer, 122–123
fast Fourier transform (FFT), 567
fast retransmission, 391, 395–396
fast-recovery algorithm, 402
fault management, 608–609
FC. *See* frame control (FC)
FCS. *See* frame check sequence (FCS)
FDDI. *See* Fiber Distributed Data Interface (FDDI)
FDM. *See* frequency-division multiplexing (FDM)
feedback shift register (FSR), 653
FFT. *See* fast Fourier transform (FFT)
FHSS. *See* frequency-hopping spread spectrum (FHSS)
Fiber Distributed Data Interface (FDDI), 104
fiber node, 160
fiber-optic cable, 50, 52–53

file structure, 466
file transfer, 467–468
File Transfer Protocol (FTP), 464–468
  basic model, 464
  commands, 465
  control connection, 465–466
  data connection, 466–468
  data structure, 466
  file transfer, 467–468
  file type, 466
  security, 468
  transmission mode, 467
FIN + ACK segment, 376
FIN segment, 376
finger table, 503–504
finite state machine (FSM), 355–356
  for data transfer, 392–394
  receiver-side, 393
  sender-side, 392–393
firewalls, 702–704
  defined, 702
  packet-filter, 703
  proxy, 704
first generation (1G), 165–166
fixed-size framing, 67
flag, 67, 82, 84, 226
flat name space, 488
flickering, 564
flooding, 295
flow control, 205–206, 346–349, 360, 383–389, 421–423
  and error control, 350–352
  handling, 348–349
  pushing/pulling, 347–348
flow label, 207
FM. *See* frequency modulation (FM)
footprint, 176
foreign agent, 239–240, 245
foreign network, 239
forward transmission, IS-95, 170–171
forwarding of IP packets, 247–256
forwarding tables, 304–305
fourth generation (4G), 174–175
four-way handshake, 418, 419
FQDN. *See* fully qualified domain name (FQDN)
fragmentation, 131, 225, 269
fragmentation offset field, 226
frame
  baseband layer, 143–144
  Ethernet, 117–118
  HDLC, 81–82
frame body, 132
frame bursting, 125
frame check sequence (FCS), 82, 85, 132
frame control (FC), 132
frame length, 118–119
frame tagging, 199
framing, 66–70
  bit stuffing, 69
  bit-oriented, 68–70
  byte-stuffing, 67–68
  character-oriented, 67–68
  escape character (ESC), 67–68
  fixed-size, 67
  flag, 67
  PPP, 84–85
  variable-size, 67
frequency, 32
frequency masking, 567
frequency modulation (FM), 45–46
frequency shift keying (FSK), 43–44
frequency-division multiplexing (FDM), 48
frequency-domain method, 566
frequency-domain plot, 32, 33
frequency-hopping spread spectrum (FHSS), 135–136, 144
frequency-reuse principle, 164
FSK. *See* frequency shift keying (FSK)
FSM. *See* finite state machine (FSM)
FSR. *See* feedback shift register (FSR)
full-duplex communication, 366, 414
full-duplex mode, 4–5, 124
full-duplex service, 366
fully qualified domain name (FQDN), 489

G

garbage collection timer, 307
gatekeeper, 595
Gates, Bill, 182
gateway, 595
gateway link (GWL), 181
general header, 417
generic domain, 492
GEO. *See* geostationary Earth orbit (GEO)
geographical routing, 253
geostationary Earth orbit (GEO), 177, 178
geostationary satellites, 178
GetBulkRequest PDU, 623
GetNextRequest PDU, 622
GetRequest PDU, 622
GFSK (FSK with Gaussian bandwidth filtering), 144
Gigabit Ethernet (1 Gbps), 116–117, 123–126, 195
  full-duplex mode, 124
  half-duplex mode, 124–125
  MAC sublayer, 123–124
Global Positioning System (GPS), 170, 178–181
global routing prefix, 259
global routing protocol, 302
Global System for Mobile Communication (GSM), 168–170
  bands, 168
  reuse factor, 170
  transmission, 168
global unicast address block, 259–260
Globalstar, 181
Gnutella network, 500
go-back-*N* (GBN), 363
GPS. *See* Global Positioning System (GPS)
granular noise, 554
graph, 286
Graphic Interchange Format (GIF), 564

group management, L2CAP, 141
group membership messages, 272–273
group-shared tree, 329
GSM. *See* Global System for Mobile Communication (GSM)
guard bands, 48
guided media, 50
GWL. *See* gateway link (GWL)

H

H.323, 594–596
  architecture, 595
  gatekeeper, 595
  gateway, 595
  operation, 596
  protocols, 595–596
half-close, 376–377
half-duplex mode, 4, 124–125
Hamming distance, 73–74
handoff, 165
Handshake Protocol, 689–690
handshaking, collision during, 130
hard handoff, 165
hashed MAC, 660
hashing object identifier, 501
hashing peer identifier, 501
HDLC. *See* high-level data-link control (HDLC)
head end, 160
header checksum, 222–223
header error-correction, 144
header length (HLEN), 221, 368
header translation strategy, 274
hello message, 311
heterogeneous internetwork, 11
heterogenous devices, link-layer switch and, 195
hexadecimal notation, 119, 257
HFC. *See* hybrid fiber-coaxial (HFC) network
hidden-station problem, 130
hierarchical name space, 488
hierarchical protocol, 17
hierarchical routing, 252–253, 302
hierarchical switching, 256
high-level data-link control (HDLC), 81–83
  frames, 81–82
high-rate direct-sequence spread spectrum (HR-DSSS), 137
HLEN. *See* header length (HLEN)
home address, 238
home agent, 239–240, 245
home network, 238–239
home security devices, Bluetooth and, 138
hop count, 303–304
hop-by-hop option, 268
hosts, 5, 451
  mobile, 238–239
  remote, 245
  stationary, 238
HR-DSSS. *See* high-rate direct-sequence spread spectrum (HR-DSSS)
hub, 6, 188–189
Huffman coding, 546–550
Huffman tree, 547–549
hybrid fiber-coaxial (HFC) network, 160–161
hypermedia, 449
hypertext, 449
Hypertext Markup Language (HTML), 452
Hypertext Transfer Protocol (HTTP), 449, 453–464
  conditional request, 460
  cookies. *See* cookies
  message formats, 455–456
  nonpersistent connection, 453–454
  persistent connection, 455
  proxy server, 462–464
  request message, 456–457
  response message, 458–459
  security, 464
Hypertext Transfer Protocol, Secured (HTTPS), 464

I

iBGP. *See* internal BGP (iBGP)
IBM, 104
ICANN ranges, 345
ICMP checksum, 237
ICMPv4. *See* Internet Control Message Protocol version 4 (ICMPv4)
identification field, 226
identifier space, 503, 510–511, 515
IEEE. *See* Institute of Electrical and Electronics Engineers (IEEE)
IEEE 802.11 Project, 126–138
IEEE 802.15 standard, 138
IETF. *See* Internet Engineering Task Force (IETF)
IFDMA. *See* interleaved FDMA (IFDMA)
I-frames (information frames), 81–82
  control field for, 83
IGMP. *See* Internet Group Management Protocol (IGMP)
IGP. *See* interior gateway protocol (IGP)
IKE. *See* Internet Key Exchange (IKE)
ILEC. *See* incumbent local exchange carrier (ILEC)
images, 3, 560–564
  compression, 561–564
  digital, 460
implementation
  10 Gigabit Ethernet, 126
  RIP, 305–307
  Standard Ethernet, 120
impulse noise, 36
IMT-2000. *See* Internet Mobile Communication 2000 (IMT-2000)
IMT-2000 radio interfaces, 173
IMT-DS, 174
IMT-FT, 174
IMT-MC, 174
IMT-SC, 174
IMT-TC, 174
in-band signaling, 153
inbound SPD, 683
incumbent local exchange carrier (ILEC), 151
IND. *See* Inverse-Neighbor-Discovery (IND) protocol
induced noise, 36

industrial, scientific, and medical (ISM) band, 135
information dissemination, 324
information field, 82
information frames (I-frames), 81–82
  control field for, 83
informational messages, 271
InformRequest PDU, 622
infrared waves, 54
infrastructure BSS, 127
initial sequence number (ISN), 368
initial TSN, 419
initiation tag, 419
Institute of Electrical and Electronics Engineers (IEEE), 104, 116, 126, 127
  802.1Q, 199
  802.11, 126, 127
  802.11 DSSS, 136
  802.11 FHSS, 135–136
  802.11 infrared, 136–137
  802.11a OFDM, 137
  802.11b HR-DSSS, 137
  802.11g, 138
  802.11n, 138
  addressing mechanism, 133–135
  architecture, 127–128
  basic service set (BSS), 127
  Bluetooth, 138
  extended service set (ESS), 127–128
  MAC sublayers, 128–133
  physical layer, 135–138
integrity, 638
  attacks threatening, 640
inter-AS routing protocol, 302
interdomain routing protocol, 302, 317
interexchange carriers (IXCs), 151 152
interface numbers, 198
interference, 70
Interim Stand 95 (IS-95), 170–172
  bands and channels, 170
  forward transmission, 170–171
  reverse transmission, 171–172
  synchronization, 170
interior gateway protocol (IGP), 302
inter-LATA services, 151–152
interleaved FDMA (IFDMA), 174
internal BGP (iBGP), 313
  operation of, 315–316
International Organization for Standardization (ISO), 21
International Telecommunication Union (ITU), 594
Internet, 10–12, 64
  accessing, 12–13
  DNS in, 491–493
  multimedia in, 568–577
Internet Control Message Protocol version 4 (ICMPv4), 219, 231–237, 269–273
  destination unreachable message, 270
  echo request and reply, 271
  neighbor solicitation and advertisement, 271–272
  packet too big, 270–272
  parameter problem, 271
  redirection, 272
  time exceeded, 271
Internet Engineering Task Force (IETF), 221
Internet Group Management Protocol (IGMP), 219, 326, 331–333
  encapsulation, 333
  membership information, 332
  messages, 331–332
"Internet in the sky," 182
Internet Key Exchange (IKE), 684
Internet Mail Access Protocol, version 4 (IMAP4), 476–477
Internet Mobile Communication 2000 (IMT-2000), 173
Internet Protocol (IP), 84
Internet Protocol Control Protocol (IPCP), 87
Internet protocol television (IPTV), 572
Internet Protocol version 4 (IPv4)
  addressing, 210–219
  comparison to IPv6, 269
  as connectionless protocol, 220
  datagram, 220–225
  forwarding of IP packets, 247–256
  fragmentation, 221, 225, 226–229
  header checksum, 222–223
  header length, 221
  ICMPv4, 219, 231–237
  IPSec (IP Security), 231
  to IPv6 transition
    dual stack, 273–274
    header translation, 274
    tunneling, 274
  main and auxiliary protocols, 219–229
  maximum transfer unit (MTU), 225–226
  options, 229–231
  service type, 221
  source address, 223
  total length, 221
  as unreliable protocol, 220
  version number, 221
Internet Protocol version 6 (IPv6), 256–273
  address space, 258
  addressing, 257–264
  anycast address, 258
  comparison to IPv4, 269
  compatible address, 262
  destination option, 268
  extension headers, 268
  flow and priority, 267
  fragmentation and reassembly, 267
  global unicast address block, 259–260
  hop-by-hop option, 268
  mapped address, 262
  mapping ethernet MAC, 260–261
  mapping EUI-64, 260
  multicast address, 258–259
  packet format, 265–267
  protocol, 264–269
  transition from IPv4 to
    dual stack, 273–274
    header translation, 274
    tunneling, 274

unicast address, 258
unique local unicast block, 262–263
Internet radio, 571
Internet security
application layer, 691–702
e-mail security, 481, 691–692
network-layer security. *See* IP Security (IPSec)
Internet Security Association and Key Management Protocol (ISAKMP), 684
Internet Service Providers (ISPs), 12, 30, 213, 252, 301
Internet structure, 301
Internet telephony, 442
Internet television, 572
internetwork, 10, 13
interpreter, 451
intersatellite links (ISLs), 181
interval coding, 552–554
intra-AS routing protocol, 302
intracoded frame (I-frame), 565
intradomain routing protocol, 302
intra-LATA services, 151
inverse-neighbor-advertisement message, 272
Inverse-Neighbor-Discovery (IND) protocol, 271
inverse-neighbor-solicitation message, 272
IP. *See* Internet Protocol (IP)
IP addresses, 344
IP new generation (IPng), 256
IP Security (IPSec), 231, 675
Authentication Header protocol, 677–678
encapsulating security payload, 678–679
Internet Key Exchange, 684
protocols, 676–679
Security Association, 680–682
Security Association Database, 681–682
Security Policy, 682–683
Security Policy Database, 682–683
services provided by, 680
transport mode, 675, 677
tunnel mode, 676, 677
virtual private network, 684–685
IP spoofing, 231
IPCP. *See* Internet Protocol Control Protocol (IPCP)
IPSec. *See* IP Security (IPSec)
IPTV, 442
IPv4. *See* Internet Protocol version 4 (IPv4)
IPv6. *See* Internet Protocol version 6 (IPv6)
Iridium, 181
IS-95. *See* Interim Stand 95 (IS-95)
ISDN user port (ISUP), 155
ISLs. *See* intersatellite links (ISLs)
ISM. *See* industrial, scientific, and medical (ISM) band
ISO. *See* International Organization for Standardization (ISO)
ISPs. *See* Internet Service Providers (ISPs)
ISUP. *See* ISDN user port (ISUP)
iterative resolution, 494
iterative UDP communication, 522–528
echo client program using TCP, 533–535
echo client program using UDP, 525, 527–528
echo server program using TCP, 531–533
echo server program using UDP, 525, 526
flow diagram, 523–525
programming examples, 525–528
sockets, 522–523
IXCs. *See* interexchange carriers (IXCs)

J

jamming signal, 100
Java applets, 453
Java Server Pages (JSP), 453
JavaScripts, 453
jitter, 2, 40, 573
removal, 580
Joint Photographic Experts Group (JPEG), 561
encoding, 562–563
quantization, 561–562
transformation, 561
JPEG. *See* Joint Photographic Experts Group (JPEG)
jumbo payload, 268

K

Kademlia, 515–518
concurrent updating, 518
identifier space, 515
join, 518
k-buckets, 518
leave or fail, 518
parallel query, 518
routing table, 515–518
Karn's algorithm, 410
kbps (kilo bits per second), 33
k-buckets, 518
KDC. *See* key distribution center (KDC)
keep mode, POP3, 476
keep-alive messages, 322
keepalive timer, 412
Kepler's law, 176
key, 642
key distribution center (KDC), 669–671
multiple, 670
session keys, 670–671
key exchange algorithms, 686
key management, 668–674
public-key distribution, 673–674
symmetric-key agreement, 671–673
symmetric-key distribution, 669–671
key rings, 695
kilo bits per second (kbps), 33

L

L2CAP. *See* Logical Link Control and Adaptation Protocol (L2CAP)
label, 488–489
LAN. *See* local area network (LAN)
LAN PHY, 126
land unit, 162
LATAs. *See* local access transport areas (LATAs)
latency (delay), 39
layer (module), 15

layered architecture, 17–18
layers, TCP/IP protocol suite, 18–20
  application layer, 20
  data-link layer, 20
  network layer, 20
  physical layer, 20
  transport layer, 20
LCP. *See* Link Control Protocol (LCP)
least-cost routing, 286–288
least-cost trees, 287, 295–297
LEC. *See* local exchange carrier (LEC)
Lempel Ziv Welch (LZW), 545
LEO. *See* low-Earth-orbit (LEO)
line coding, 40
linear block codes, 74
linear predictive coding, 557
link address. *See* link-layer addressing
Link Control Protocol (LCP), 86
link local address, 264
link local block, 263
link-layer addressing, 104–107
  broadcast address, 107
  multicast address, 106
  unicast address, 106
link-layer switch, 17–18, 189–195
  advantages, 194–195
  filtering, 189–190
  transparent switch, 190–194
  vs. router, 195
links, 65
link-state acknowledgment message, 312
link-state advertisement, 309–311
link-state database (LSDB), 294–295, 327
link-state (LS) routing, 294–297
link-state update message, 312
little LEOs, 181
LLC. *See* logical link control (LLC)
local access transport areas (LATAs), 151–152
  inter-LATA services, 151–152
  intra-LATA services, 151
local area network (LAN), 8, 9, 65, 115–146. *See also* wide area network (WAN)
  Bluetooth, 138–145
  Ethernet, 116–126
  WiFi (wireless fidelity), 126–138
local exchange carrier (LEC), 151
local login, 482
local loop, telephone network, 150
local socket address, 446
logical connections, 16
logical link control (LLC), 116
Logical Link Control and Adaptation Protocol (L2CAP), 140–141
logical ring, 103–104
long-distance companies, 151
longest mask matching, 250–252
lookup
  chord, 504–506
  pastry, 512–513
loose source route option, 230
lossless compression, 544–554
  arithmetic coding, 551–554
  dictionary coding, 545–546
  Huffman coding, 546–550
  run-length coding, 544–545
lossy compression, 554–559
  differential PCM, 556–557
  discrete cosine transform, 557–559
  linear predictive coding, 557
  predictive coding, 554–557
  transform coding, 557
low-Earth-orbit (LEO), 177, 181–182
LS. *See* link-state (LS) routing
LS packet (LSP), 295
LSDB. *See* link-state database (LSDB)
LSP. *See* LS packet (LSP)

M

MA. *See* multiple access (MA)
MAC. *See* media access control (MAC); message authentication code (MAC)
MAC address, 198. *See also* link-layer addressing
MAC sublayer, 123–124
  distributed coordination function (DCF), 128, 129–130
  fragmentation, 131
  frame format, 131–132
  frame types, 132–133
  IEEE 802.11 standard, 128–133
  point coordination function (PCF), 128, 130–131
mail servers, 471
mailbox, 469
mailing/group list, 471–472
MAN. *See* metropolitan area network (MAN)
management frames, 132
Management Information Base (MIB), 611, 618–621
  object declaration and definition, 613
  role of, 612
Manchester with 200-Mbaud bandwidth, 122
manual configuration, VLANs, 198
mapped address, 262
masquerading, 640
maximum response code, 273
maximum transfer unit (MTU), 225–226
maximum transmission unit (MTU), 270
Mbps (mega bits per second), 33
McCaw, Craig, 182
MC-CDMA. *See* multicarrier code division multiple access (MC-CDMA)
media access control (MAC), 66, 88–104, 116
  controlled access, 101–104
  random-access, 88–101
  taxonomy of multiple-access protocols, 88
media server, 569–570
medium-Earth-orbit (MEO), 177, 178–181
mega bits per second (Mbps), 33
membership-query message, 273
membership-report message, 273
MEO. *See* medium-Earth-orbit (MEO)

mesh topology, 6, 7
message access agent (MAA), 469, 470, 475–477
  IMAP4, 476–477
  POP3, 476
message authentication, 659–660, 663
  entity authentication *vs.*, 666
message authentication code (MAC), 659–660
Message Digest, 659
message format, 455–456
message integrity, 663, 680
  hash functions, 659
  message digest, 658–659
message transfer, 474
message transfer agent (MTA), 469, 470, 472
message transport part (MTP) level 1, 154
messages, 2, 624–626
metafile, 569
Metcalfe, Robert, 116
metropolitan area network (MAN), 126
MIB. *See* Management Information Base (MIB)
microwaves, 53, 54
MIME. *See* Multipurpose Internet Mail Extensions (MIME)
MIMO. *See* multiple-input multiple-output (MIMO)
mixed notation, 257–258
mixing, 576
mobile hosts, 238–239
mobile IP, 237–247
  agent advertisement, 241–242
  agent discovery, 240
  agent solicitation, 242
  agents, 239–240
  data transfer, 244–247
  double crossing, 246
  inefficiency in, 246
  mobile hosts, 238–239
  registration, 242–244
  stationary hosts, 238
  triangle routing, 246–247
mobile stations (MSs), 162
mobile switching center (MSC), 163
modem, 83–84, 157
modern block ciphers, 648
  circular shift operation, 649
  combine operation, 649
  components of, 648–649
  swap operation, 649
modern stream cipher, 652–653
modularity, 15
modulation
  Bluetooth, 144–145
  cellular telephony, 175
  telephone network, 157
modulator, 157
monitoring devices, Bluetooth and, 138
monoalphabetic ciphers, 643–645
more fragment bit, 226
MOSPF. *See* Multicast Open Shortest Path First (MOSPF)
Motion Picture Experts Group (MPEG), 564–565
  frames, 565
  spatial compression, 565
  temporal compression, 565
MPEG. *See* Motion Picture Experts Group (MPEG)
MPEG audio layer 3 (MP3), 567
MPLS. *See* Multi-Protocol Label Switching (MPLS)
MSC. *See* mobile switching center (MSC)
MSs. *See* mobile stations (MSs)
MTU. *See* maximum transfer unit (MTU); maximum transmission unit (MTU)
multicarrier code division multiple access (MC-CDMA), 174
multicast address, 106, 119, 120, 258–259
Multicast Listener Delivery protocol, 272–273
Multicast Open Shortest Path First (MOSPF), 327
multicast routing, 322–331
  DVMRP, 324–326
  IGMP, 331–333
  MOSPF, 327
  multicasting, 323–324
  PIM, 327–331
  unicasting, 322–323
multicasting, 323–324, 576
multihomed AS, 303
multihoming, 413–414, 420
multilevel ASK, 43
multilink PPP, 84, 87
multimedia, 543
  compression. *See* compression
  data
    audio, 566–567
    image, 560–564
    text, 560
    video, 564–565
  in Internet, 568–577
multiple access (MA), 89
multiple-access protocol, 88
multiple-byte options, 229–230
multiple-input multiple-output (MIMO), 138, 175
multiple-secondary communication, 141–142
multiplexer, 47, 346
multiplexing, 47–49, 86, 346, 347, 362, 366
  IPv4, 222
  L2CAP, 140–141
multipoint/multidrop connection, 5, 6
Multi-Protocol Label Switching (MPLS), 254, 255
Multipurpose Internet Mail Extensions (MIME), 477–480
multistream delivery, 420–421
multistream service, 413
multiuser MIMO (MU-MIMO) antenna, 175

N

Nagle's algorithm, 388–389
NAK. *See* negative acknowledgment number (NAK)
name servers, hierarchy of, 490–491
name space, 488
  distribution of, 489
  domain, 488
  flat, 488
  hierarchical, 488
NAV. *See* network allocation vector (NAV)

NCPs. *See* Network Control Protocols (NCPs)
ND. *See* Neighbor-Discovery (ND) protocol
negative acknowledgment number (NAK), 83, 102
neighbor-advertisement message, 272
neighbor-discovery messages, 271–272
Neighbor-Discovery (ND) protocol, 271
neighbor-solicitation message, 272
network
  criteria, 5
  defined, 5
  local area network (LAN), 8
  performance, 5, 38–50
  physical structures, 5–8
  reliability, 5
  security, 5
  types, 8–13
network address, 216
network allocation vector (NAV), 130
Network Control Protocols (NCPs), 86, 87
network interface card (NIC), 119
network layer, 20, 343
  congestion control, 206
  connectionless service, 207
  connection-oriented service, 207
  delay, 208–209
  error control, 205
  flow control, 205–206
  IP version 4 (IPV4), 210–256
  IP version 6 (IPV6), 256–273
  overview, 203–204
  packet switching, 206–207
  packetizing, 205
  performance, 207–210
  quality of service (QoS), 206
  routing, 205
  security, 206
  services provided, 205–206
  transition from IPV4 TO IPV6, 273–274
  unicast routing. *See* unicast routing
network link, LSPs, 310
network management, 605, 606
  accounting, 610
  capacity, 609
  configuration, 606–608
  documentation, 607–608
  fault, 608–609
  performance, 609
  reconfiguration, 606–607
  response time, 609
  security, 609
  throughput, 609
  traffic, 609
network virtual terminal (NVT), 482, 483
NewReno TCP, 406
NIC. *See* network interface card (NIC)
nodes, 6, 65, 286
node-to-node communication, 66
noise, 36
noiseless channel, 36–37
noisy channel, 37
nonperiodic analog signals, 31
nonpersistent connection, 453–454
nonpersistent method, 96
non-repudiation, 664
nonstandard application-layer protocol, 440
no-operation option, 229
normal response mode (NRM), 81
notation, 211
no-transition mobility, 128
NRM. *See* normal response mode (NRM)
numbering system, 367–368
NVT ASCII, 465
Nyquist bit rate, 36

O

Oakley protocol, 684
odd number of errors, CRC and, 80
OFDM. *See* orthogonal frequency-division multiplexing (OFDM)
OFDMA. *See* orthogonal FDMA (OFDMA)
old route, 306
one-dimensional DCT, 557–559
on-off keying (OOK), 43
OOK. *See* on-off keying (OOK)
open message, 321
Open Shortest Path First (OSPF), 308–313
  algorithm, 313
  areas, 309
  forwarding tables, 309
  implementation, 311–313
  link-state advertisement, 309–311
  messages, 311–312
  metric in, 308
  performance, 313
open state, 85
open system, 21
Open Systems Interconnection (OSI) model, 21–22
  versus TCP/IP, 21–22
opening and closing windows, 384–386
optional attribute, 318
options, 412
  IPv4, 229–231
orbits, satellite, 175–176
ordered data delivery, 420
ordering, 575–576
orthogonal FDMA (OFDMA), 174
orthogonal frequency-division multiplexing (OFDM), 137
OSI model. *See* Open Systems Interconnection (OSI) model
OSPF. *See* Open Shortest Path First (OSPF)
outbound SPD, 682–683
Outlook, 470
out-of-band signaling, 153
out-of-order packets, 350
out-of-order segments, 391–392
overlay network, 499

P

P2P. *See* peer-to-peer
packet, 350, 415–416
  format, 358, 416–418, 486
  loss, 210
  modification, 230–231
  sniffing, 230
packet switching, 206–207
packet-filter firewall, 703
packetizing, 205
packet-too-big message, 270–271
Pad1, 268
padding, 124
PadN, 268
page structure, 466
paging, 164
PAP. *See* Password Authentication Protocol (PAP)
paradigms, application layer, 440–442
  client/server, 440–441, 443–448
  mixed, 442
  peer-to-peer. *See* peer-to-peer (P2P) paradigm
parallel query, 518
parameter problem message, 234, 271
parity bit, 75
parity-check code, 75–76
parked state, 139
partially qualified domain name (PQDN), 489
passive open, 371
Password Authentication Protocol (PAP), 86
passwords, 667
Pastry, 510–515
  application, 515
  identifier space, 510–511
  join, 513–514
  leave or fail, 514–515
  lookup, 512–513
  routing, 511–512
path, 451
path attributes, BGP, 318–320
Path MTU Discovery technique, 269
path-vector algorithm, 300–301
path-vector (PV) routing, 297–301
  spanning trees, 298–300
payload, 85, 144, 222, 223
P-box, 648
PC. *See* personal computer (PC); point controller (PC)
PCF. *See* point coordination function (PCF)
PCM (pulse code modulation), 41–42
peak amplitude, 32
peering points, 12, 301
peer-to-peer (P2P) networks, 498
  centralized, 498–499
  decentralized, 499–500
  structured, 500
  unstructured, 499–500
peer-to-peer (P2P) paradigm, 441–442, 498–521
  BitTorrent, 518–521
  chord, 503–510
  distributed hash table, 500–503
  Kademlia, 515–518
  P2P networks, 498–500
  pastry, 510–515
perceptual coding, 566–567
performance
  BGP, 322
  network, 5, 38–50
  network layer, 207–210
  OSPF, 313
  RIP, 308
performance management, 609
period, 32
periodic analog signals, 31
periodic timer, 306
peripheral devices, Bluetooth and, 138
persistence methods, 96–97
persistence timer, 411–412
persistent connection, 455
personal computer (PC), 158
PGP. *See* Pretty Good Privacy (PGP)
phase, 32
phase modulation (PM), 45, 46–47
phase shift keying (PSK), 42, 44, 137
physical address. *See* link-layer addressing
physical layer
  Fast Ethernet (100 Mbps), 122–123
  Gigabit Ethernet, 125
  IEEE, 135–138
  overview, 20
  signals, 31–34
  transmission media and, 49
physical topology, 6
piconet, 138–139
PIFS. *See* point coordination function interframe space (PIFS)
piggybacking, 82, 83, 368
PIM. *See* Protocol Independent Multicast (PIM)
PIM-DM. *See* Protocol Independent Multicast, Dense Mode (PIM-DM)
pine, 470
ping, 234, 235, 236
plain old telephone system (POTS), 150
plaintext, 641, 655
playback buffer, 574–575
PM. *See* phase modulation (PM)
point controller (PC), 131
point coordination function (PCF), 128, 130–131
point coordination function interframe space (PIFS), 130
point of presence (POP), 152
point-to-point connection, 5, 6
point-to-point link, 65
Point-to-Point Protocol (PPP), 83–87, 88
  byte stuffing, 85–86
  framing, 84–85
  services, 84
point-to-point WAN, 9, 10
poisoned reverse, 294
poison-reverse strategies, 308
policy transition, 402–406

poll function, 102
polling, 101–102
polyalphabetic cipher, 645–646
POP. *See* point of presence (POP)
port, 451
port forwarding, 485–486
port numbers, 343–345, 357–358
  ephemeral, 343
  ICANN ranges, 345
  IP addresses *vs.*, 344
  well-known, 343–344
Post Office Protocol, version 3 (POP3), 476
POTS. *See* plain old telephone system (POTS)
p-persistent method, 96–97
PPM. *See* pulse position modulation (PPM)
PPP. *See* Point-to-Point Protocol (PPP)
PQDN. *See* partially qualified domain name (PQDN)
preamble, 117, 118
predecessor, token-passing method, 102
predicted frame (P-frame), 565
predictive coding, 554–557, 566
  delta modulation, 554–556
prefix in addressing, 211, 259
Pretty Good Privacy (PGP), 481, 693–698
  algorithms, 695
  applications of, 698
  certificates, 695, 696–697
  code conversion, 694
  compression, 693–694
  confidentiality with one-time session key, 694
  key rings, 695
  message integrity, 693
  packets, 698
  plaintext, 693
  segmentation, 694
  trust model in, 697–698
primary address, 420
primary server, 491
primary station, 81, 101, 138–139
principles, protocol layering, 16
private key, 654
proactive fault management, 609
processing delay, 208
process-to-process communication, 20, 342–343, 359, 364, 412
Project 802, 116
propagation delay, 208
propagation time, 95
protocol, 3, 451
protocol data unit (PDU), 622–623
  format, 623–624
Protocol Independent Multicast (PIM), 327–331
  PIM-DM, 328–329
  PIM-SM, 329–331
Protocol Independent Multicast, Dense Mode (PIM-DM), 328–329
protocol layering, 13–16
  advantage of, 15
  logical connections, 16
  principles of, 16
  scenarios, 13–15
provider networks, 12, 301
proximity metric, 512
proxy firewall, 704
proxy server, 462–464
pseudoheader, 360, 370
pseudorange, 180
PSK. *See* phase shift keying (PSK)
psychoacoustics, 566, 567
public announcement, 673
public key, 654
public-key certificates, 673
public-key distribution, 673–674
pull program, 470
pulling, 347–348
pulse code modulation (PCM), 41–42, 554
pulse position modulation (PPM), 136–137
pure ALOHA, 89–92
pure coding, 552–554
push program, 470
pushing, 347–348
pushing data, 374–375
PV. *See* path-vector (PV) routing

Q

QAM. *See* quadrature amplitude modulation (QAM)
QoS. *See* quality of service (QoS)
quadrature amplitude modulation (QAM), 42, 44, 137, 175
quality of service (QoS), 206
query messages, 232, 234–234, 332
Query Routing Protocol (QRP), 500
queuing, 361
queuing delay, 208

R

radio layer, 144–145
radio system, 175
radio waves, 53–54
random-access, 88–101
ranging, 180
RCH. *See* regional cable head (RCH)
real-time interactive audio/video
  mixing, 576
  multicasting, 576
  ordering, 575–576
  playback buffer, 574–575
  time relationship, 572–573
  timestamp, 574
  translation, 576
real-time interactive protocols, 577–596
Real-Time Streaming Protocol (RTSP), 570–571
real-time transmission, 2
Real-Time Transport Control Protocol (RTCP), 583–584
  bandwidth utilization, 586
  packets, 585–586
  requirement fulfillment, 586–587
  UDP port, 586
Real-Time Transport Protocol (RTP), 581
  packet format, 582–583

UDP port, 583
reassembly
datagram, 226
L2CAP, 141
receive not ready (RNR), 83
receive ready (RR), 83
received route, 306
receiver, 3
receiving mail, 470–471
reconfiguration
hardware, 606–607
software, 607
user-account, 607
Record Protocol, 689, 691
record route option, 229
record structure, 466
redirection message, 234, 272
redundancy, 70
reflection, 52
reflects, 52
refraction, 52
regional cable head (RCH), 161
regional offices, 150
registered ports, 345
registrar server, 592
registrars, 497
registration, mobile IP, 242–244
registration reply, 243, 244
registration request, 243–244
REJ. *See* reject (REJ)
reject (REJ), 83
reliability, network, 5
reliable service, 366, 414
remote host, 245
remote login, 481, 482
remote procedure call (RPC), 505–506
remote socket address, 446–447
rendezvous point (RP), 329
Reno TCP, 404–406
renumbering, 264
repeater, 188
vs. router, 195
repetition interval, 130–131
replay, 640, 680
report message, 332
Report PDU, 622
repudiation, 640
request message, 456–457
request to send (RTS), 129
reservation method, 101
resolution, 3, 493–495
resolver, 493–494
resource records, 495
response message, 458–459
Response PDU, 623
response time, 5, 609
retransmission, 391, 426
retransmission time-out (RTO), 391, 409–411
retransmission timer, 408–409, 426
reuse factor, 164
reverse path broadcasting (RPB), 324, 325
versus RPF, 326
versus RPM, 326
reverse path forwarding (RPF), 324–325
versus RPB, 326
reverse path multicasting (RPM), 324, 325–326
versus RPB, 326
reverse transmission, IS-95, 171–172
RGB, 3
ring topology, 8
RIP. *See* Routing Information Protocol (RIP)
RIP-1, 305
RIP-2, 305
RNR. *See* receive not ready (RNR)
roaming, 165
robustness, RIP, 308
root server, 491
rotary telephones, 153
rounds, DES, 650
round-trip time (RTT), 235, 408–409
route selection, BGP, 320–321
route summarization, 218–219
router, 17–18, 195–196, 207, 352
vs. repeater/switch, 195
router link, LSPs, 310
router-advertisement message, 272
router-solicitation message, 271–272
routing, 205, 502, 511–512. *See also* unicast routing
routing algorithms, 288–301
Routing Information Protocol (RIP), 303–308
algorithm, 306
forwarding tables, 304–305
hop count, 303–304
implementation, 305–307
messages, 305
performance, 308
timers in, 306–307
routing table, 515–518
RP. *See* rendezvous point (RP)
RPB. *See* reverse path broadcasting (RPB)
RPC. *See* remote procedure call (RPC)
RPF. *See* reverse path forwarding (RPF)
RPM. *See* reverse path multicasting (RPM)
RR. *See* receive ready (RR)
RSA cryptosystem, 656–657
RTCP. *See* Real-Time Transport Control Protocol (RTCP)
RTS. *See* request to send (RTS)
RTSP. *See* Real-Time Streaming Protocol (RTSP)
RTT. *See* round-trip time (RTT)
run-length coding, 544–545
rwnd, 399

S

SA. *See* Security Association (SA); source address (SA)
SACK. *See* selective acknowledgment (SACK)
SACK chunk, 426–427
satellite network, 175–182
categories, 177
downlink, 177

satellite network (*Continued*)
footprint, 176
frequency band, 177
GEO, 178
geostationary, 178
GPS, 178–181
LEO, 181–182
MEO, 178–181
orbit, 175–176
trilateration, 179
uplink, 177
S-box, 648
SC. *See* sequence control (SC)
scatternet, 139
SCCP. *See* signaling connection control point (SCCP)
SCO. *See* synchronous connection-oriented (SCO)
SCP. *See* service control point (SCP)
scp. *See* Secure Copy (scp)
SDR. *See* Software Defined Radio (SDR)
second generation (2G), 166–172
secondary server, 491
secondary stations, 81, 101, 138–139
Secure Copy (scp), 485
Secure File Transfer Program (sftp), 485
Secure Hash Algorithm (SHA), 659
Secure Shell (SSH), 484–486
components, 484
for file transfer, 485
packet format, 486
port forwarding, 485–486
for remote login, 485
SSH-AUTH, 485
SSH-CONN, 485
SSH-TRANS, 484–485
Secure Sockets Layer (SSL), 464, 468, 685, 686–689
Alert Protocol, 691
architecture, 686–689
ChangeCipherSpec Protocol, 690
cipher suite, 686
compression algorithms, 687
cryptographic parameter generation, 687–688
encryption/decryption algorithms, 686
Handshake Protocol, 689–690
hash algorithms, 686
key exchange algorithms, 686
Record Protocol, 689, 691
services, 686
sessions and connections, 688–689
Secure/Multipurpose Internet Mail Extension (S/MIME), 481, 698–702
applications of, 701
cryptographic algorithms, 701
Cryptographic Message Syntax, 699–701
key management, 701
security, 464, 468, 481, 497–498, 609, 627
application-layer, 691–702
attacks, 639–640
availability, 638
confidentiality. *See* confidentiality
cryptography. *See* cryptography
denial of service, 640
digital signature. *See* digital signature
entity authentication. *See* entity authentication
firewalls, 702–704
goals, 638
home, Bluetooth and, 138
integrity, 638
IPv4 datagrams, 230–231
key management. *See* key management
masquerading, 640
message authentication, 659–660
message integrity, 658–659
modification, 640
network, 5
network layer, 206
replaying, 640
repudiation, 640
services and techniques, 641
snooping, 639
steganography, 641
traffic analysis, 639
transport layer, 685–691
VLANs, 200
Security Association (SA), 680–682
Security Association Database (SAD), 681–682
Security Policy (SP), 682–683
Security Policy Database (SPD), 682–683
segment, 20, 365–366, 368–371
format of, 368, 369
segmentation, L2CAP, 141
select function, 102
selective acknowledgment (SACK), 390
selective reject (SREJ), 83
selective-repeat (SR), 363
semiautomatic configuration, VLANs, 199
sender, 2
sending mail, 470
sequence control (SC), 132
sequence numbers, 350, 367–368, 369
server, 343
processes, 443, 529–530
service control point (SCP), 154
services, 663–665
connectionless. *See* connectionless service
connection-oriented, 354–355, 366, 371–377, 414
IPSec, 680
PPP, 84
SCTP, 412–414
SSL, 686
TCP, 364–366
telephone network, 155–158
UDP, 359–362
session, 314
Session Initialization protocol (SIP), 587–594
addresses, 589
communicating parties, 588
messages, 589–590
registrar server, 592

session keys, 670–671
SetRequest PDU, 623
SFD. *See* start frame delimiter (SFD)
S-frames (supervisory frames), 81–82
  control field for, 83
sftp. *See* Secure File Transfer Program (sftp)
SHA. *See* Secure Hash Algorithm (SHA)
Shannon capacity, 37–38
shared secret key, 641
shielded twisted pair (STP), 50, 123
shift cipher, 643
short interframe space (SIFS), 129
shortest-path tree, 287
shrinking of windows, 386–388
SI. *See* stream identifier (SI)
SIFS. *See* short interframe space (SIFS)
signal impairment, 35–40
  noise, 36
signal points (SPs), 154
signal transport ports (STPs), 154
signaling connection control point (SCCP), 155
signaling network, 153–154
signaling system, telephone networks, 152–153
Signaling System Seven (SS7), 154–155
signals, 31–34
  analog, 31–33
  digital, 33–34
signal-to-noise ratio (SNR), 36, 158
signing digest, 663
silly window syndrome, 388–389
Simple Mail Transfer Protocol (SMTP), 472–475
  commands, 472, 473
  connection establishment, 474
  connection termination, 474
  message transfer, 474
  messages, 624–626
  protocol data unit, 622–623
    format, 623–624
  responses, 472, 473
  security, 627
  UDP ports, 626–627
Simple Network Management Protocol (SNMP), 610–627
  management components, 611–613
  managers and agents, 611
  overview, 613–614
  program coding, 613
  role of, 611–612
simplex mode, 4
sine wave, 31
single errors, CRC and, 80
single-bit error, 70
single-byte options, 229
single-layer protocol, 13
single-secondary communication, 141–142
SIP. *See* Session Initialization protocol (SIP)
SKEME (Secure Key Exchange Mechanism), 684
Skype, 442, 576–577
  SkypeIn, 576–577
  SkypeOut, 577
sliding window, 351–352
slope overload distortion, 554
slotted ALOHA, 93–94
slow start, exponential increase, 399–401
slow-start algorithm, 399
SMI. *See* Structure of Management Information (SMI)
S/MIME. *See* Secure/Multipurpose Internet Mail Extension (S/MIME)
SMTP. *See* Simple Mail Transfer Protocol (SMTP)
SNMP. *See* Simple Network Management Protocol (SNMP)
snooping, 639
SNR. *See* signal-to-noise ratio (SNR)
socket address, 345, 445–447
  local, 446
  remote, 446–447
socket interface, 443
socket interface programming, in C, 521–535
  communication using TCP, 528–535
  header files, 522
  iterative communication using UDP, 522–528
  socket data structure, 521–522
sockets, 443, 444–445
  used for UDP, 522–523
soft handoff, 165, 172
Software Defined Radio (SDR), 175
some.anet.com, 493
something inherent, 666
something known, 666
something possessed, 666
SONET OC-192, 126
source address (SA), 117, 118
source port address, 368
source quench message, 234
source router, 286
source routing, 268
source-based tree, 327
spanning trees, 298–300
spatial compression, 565
spatially shared connection, 6
SPD. *See* Security Policy Database (SPD)
special addresses, 262
split horizon, 293–294, 308
Sprint, 152
SPs. *See* signal points (SPs)
SREJ. *See* selective reject (SREJ)
SS7. *See* Signaling System Seven (SS7)
SSH Authentication Protocol (SSH-AUTH), 485
SSH Connection Protocol (SSH-CONN), 485
SSH Transport-Layer Protocol (SSH-TRANS), 484–485
SSL. *See* Secure Sockets Layer (SSL)
SSL-FTP, 468
SSN. *See* stream sequence number (SSN)
stabilize, chord, 507–510
standard application-layer protocol, 439–440
standard applications, application layer, 448–449
  DNS, 486–498
  electronic mail, 468–481
  File Transfer Protocol, 464–468

standard applications (*Continued*)
Hypertext Transfer Protocol, 453–464
Secure Shell, 484–486
TELNET, 481–484
World Wide Web, 449–453
standard Ethernet (10 Mbps), 116–120
star topology, 6–7, 188
start frame delimiter (SFD), 117, 118
state transition diagram, 378–380
static documents, 452
static/pure coding, 552–554
stationary hosts, 238
steganography, 641
STP. *See* shielded twisted pair (STP)
STPs. *See* signal transport ports (STPs)
STREAM API, 443
stream cipher, 646–647
Stream Control Transmission Protocol (SCTP), 357, 412–427, 448
acknowledgment number, 416
association, 418–421
association in, 413
chunks, 417–418, 426–427
congestion control, 427
connection-oriented service, 414
cumTSN, 422
data transfer, 419–421
error control, 423–427
features, 414–416
flow control, 421–423
four-way handshake, 418, 419
full-duplex communication, 414
general header, 417
multihoming, 413–414, 420
multistream delivery, 420–421
multistream service, 413
ordered delivery, 420
packets, 415–418
primary address, 420
process-to-process communication, 412
reliable service, 414
retransmission, 426
SACK chunk, 426–427
services, 412–414
stream identifier, 414
stream sequence number, 414
transmission sequence number, 414
unordered delivery, 420
verification tag, 418–419
stream delivery service, 364
stream identifier (SI), 414
stream mode, 467
stream sequence number (SSN), 414
streaming live audio/video
internet protocol television, 572
Internet radio, 571
Internet television, 572
streaming stored audio/video
using media server, 569–570
using media server and RTSP, 570–571
using Web server, 568
using Web server with metafile, 569
strict source route option, 230
Structure of Management Information (SMI), 611, 614–618
Basic Encoding Rules, 617
encoding method, 617–618
name, 614–615
object identifier in, 615
role of, 612
syntax, 613
type, 615–616
structured networks, 500
stub AS, 303
subnet identifier, 259
subnetting, 217–218
successor, token-passing method, 102
suffix in addressing, 211
summary link to As border router, LSPs, 310
summary link to network, LSPs, 310
supervisory frames (S-frames), 81–82
control field for, 83
switched WAN, 9–10
switched/56 service, 156
switching office, telephone network, 150
symmetric-key agreement, 671–673
symmetric-key cipher, 641–643
defined, 641
modern, 647–653
substitution ciphers, 643–646
traditional, 643–647
symmetric-key cryptography, 653
symmetric-key distribution
key distribution center, 669–671
SYN + ACK segment, 372–373
SYN flooding attack, 373
SYN segment, 372
synchronization
IS-95, 170
satellite network, 180
synchronizing source (SSRC), 578
synchronous connection-oriented (SCO), 143
syndrome, 76

T

table maintenance, 199
Taho TCP, 402–404
tandem offices, 150
tap, 7
TCAP. *See* transaction capabilities application port (TCAP)
TCP communication, 528–535
flow diagram, 529–530
programming examples, 530–535
server process, 529–530
sockets used in, 528
TCP/IP. *See* Transmission Control Protocol/Internet Protocol (TCP/IP)
TCP/IP protocol suite, 17
layered architecture, 17–18

layers in, 18–20
versus OSI model, 21–22
TDD-TDMA (time-division duplex TDMA), 141–142
TDM. *See* time-division multiplexing (TDM)
TDMA. *See* time-division multiple access (TDMA)
Telecommunications Act of 1996, 151
teleconferencing, 324
Teledesic, 182
telephone network, 12
analog services, 155–156
bandwidth, 157
components, 150
dial-up service, 156–158
digital service, 156
digital subscriber line (DSL), 158–159
LATAs, 151–152
local loop, 150
signaling, 152–155
switching office, 150
trunks, 150
telephone user port (TUP), 155
TELNET, 465
temporal compression, 565
temporal masking, 567
Terminal Network (TELNET), 481–484
local login, 482
network virtual terminal, 482, 483
options, 483
remote login, 481, 482
user interface, 484
terminate state, 85
text, 560
thermal noise, 36
third generation (3G), 173–174
three-dimensional trilateration, 179–180
three-layer protocol, 14
three-node instability, 294
three-way handshaking, 371–373, 375–376
throughput, 38–39, 609
CSMA/CD, 100
network layer, 209–210
pure ALOHA, 92
slotted ALOHA, 94
ticket, 671
time relationship, 572–573
time to live (TTL), 495
time-division multiple access (TDMA), 167–168
time-division multiplexing (TDM), 48–49, 199
time-domain plot, 32, 33
time-exceeded message, 234, 271
timeliness, 2
timeshared connection, 6
timestamp, 574
option, 230
reply, 234
request, 234
time-to-live (TTL), 222, 234, 236
TIME-WAIT timer, 412
TLI. *See* transport layer interface (TLI)
TLS. *See* Transport Layer Security (TLS)
token bus, 104
Token Bus LAN, 104
token management, 103
Token Ring LAN, 104
token-passing method, 102–104
topology
Fast Ethernet, 122
Gigabit Ethernet, 125
TOS. *See* type of service (TOS)
total delay, 209
total length, 221
traceroute program, 234, 235–237
tracert program, 235–236, 237
trackerless BitTorrent, 520–521
traditional cable networks, 160
traffic, 609
traffic analysis, 639
transaction capabilities application port (TCAP), 155
transform coding, 557
transient AS, 303
transit time, 5
transition phases, 85
translation, 576
transmission
address bits, 119–120
baseband, 34
delay, 208
digital, 40–42
media, 49–54
mode, 467
Transmission Control Protocol (TCP), 357, 363–412, 448
ACK segment, 373
acknowledgment, 390
acknowledgment number, 368
additive increase, 401–402
additive increase, multiplicative decrease, 406–407
buffers, 364–365
byte number, 367
capability to handle real-time multimedia, 580–581
checksum, 390
Clark's solution, 389
communication using, 528–535
congestion avoidance, 401–402
congestion detection, 399
congestion policies, 399–402
congestion window, 398–399
connection, 371–377
connection reset, 377
connection termination, 375–377
connection-oriented service, 366
cumulative acknowledgment, 390
data transfer, 373–375
deadlock, 398
defined, 363
delayed segment, 396
demultiplexing, 366
duplicate segment, 396
encapsulation, 371

Transmission Control Protocol (TCP) (*Continued*)
error control, 389–398
exponential backoff, 410–411
fast recovery, 402
fast retransmission, 391, 395–396
features, 366–367
FIN + ACK segment, 376
FIN segment, 376
flow control, 383–389
FSM for data transfer, 392–394
full-duplex communication, 366
half-close, 376–377
Karn's algorithm, 410
keepalive timer, 412
lost segment, 395
multiplexing, 366
Nagle's algorithm, 388–389
NewReno, 406
normal operation, 394
numbering system, 367–368
opening and closing windows, 384–386
options, 412
out-of-order segments, 391–392
persistence timer, 411–412
policy transition, 402–406
process-to-process communication, 364
pushing data, 374–375
receive window, 382–383
reliable service, 366
Reno, 404–406
retransmission, 391
retransmission timer, 408–409
segment, 365–366, 368–371
send window, 381–382
sequence number, 367–368
services, 364–366
shrinking of windows, 386–388
silly window syndrome, 388–389
slow start, 399–401
state transition diagram, 378–380
stream delivery service, 364
SYN + ACK segment, 372–373
SYN flooding attack, 373
SYN segment, 372
Taho, 402–404
three-way handshaking, 371–373, 375–376
throughput, 407
timers, 408–412
TIME-WAIT timer, 412
urgent data, 375
window shutdown, 388
windows in, 380–383
Transmission Control Protocol/Internet Protocol (TCP/IP), 17–20
transmission medium, 3
transmission sequence number (TSN), 414
transparent switch, 190–194
transport layer, 20
acknowledgment, 350
congestion control, 352
connectionless service, 353–354
connection-oriented service, 354–355
decapsulation, 345–346
demultiplexing, 346, 347
encapsulation, 345–346
error control, 349–352
finite state machine, 355–356
flow control, 346–349, 350–352
logical connection at, 342
multiplexing, 346, 347
port numbers, 343–345
process-to-process communication, 342–343
protocols, 356–358
requirements for interactive real-time multimedia, 579–580
sequence numbers, 350
services, 342–356
sliding window, 351–352
SSL protocol, 685, 686–689
TLS protocol, 685
transport layer interface (TLI), 443
Transport Layer Security (TLS), 685
transport mode, IPSec, 675, 677
transposition cipher, 646
Trap PDU, 623
triangle routing, 246–247
triangulation, 179
trilateration, 179
trunks, telephone network, 150
TSN. *See* transmission sequence number (TSN)
TTL. *See* time to live (TTL); time-to-live (TTL)
tunnel mode, IPSec, 676, 677
tunneling strategy, 274
TUP. *See* telephone user port (TUP)
TV broadcasting, 12
twisted-pair cable, 50–51
two-dimensional DCT, 559
two-dimensional trilateration, 179–180
two-node loop, 293
type, Ethernet frame, 117, 118
type of service (TOS), 221

U

UDP. *See* User Datagram Protocol (UDP)
U-frames (unnumbered frames), 81–82
control field for, 83
UML. *See* user mobile link (UML)
UMTS. *See* Universal Mobile Telecommunications System (UMTS)
unguided media, 53–54
unicast address, 106, 119, 120, 258
unicast routing
distance-vector (DV) routing, 288–294
general idea, 286
least-cost routing, 286–288
least-cost trees, 295–297
link-state (LS) routing, 294–297
path-vector (PV) routing, 297–301

unicast routing protocol, 301–322
BGP, 313–322
internet structure, 301–303
OSPF, 308–313
RIP, 303–308
unicasting, 322–323
Unicode, 3
Uniform Resource Locator (URL), 451–452
unique local unicast block, 262–263
Universal Mobile Telecommunications System (UMTS), 174
unnumbered frames (U-frames), 81–82
control field for, 83
unordered data delivery, 420
unreliable service, Ethernet, 117
unshielded twisted pair (UTP), 50, 123
unstructured networks, 499–500
update binding packet, 247
update message, 321
uplink, 177
uploading, 158
upstream databands, 161, 162
U.S. Department of Defense, 178
user agent (UA), 469, 470–472
user datagram, 20
User Datagram Protocol (UDP), 305, 356–357, 447–448
applications, 362–363
capability to handle real-time multimedia, 580–581
checksum, 359, 360–361
congestion control, 361
connectionless services, 359–360
decapsulation, 361
defined, 358
demultiplexing, 362
encapsulation, 361
error control, 360
flow control, 360
generic simple protocol *vs.*, 362
iterative communication using, 522–528
multiplexing, 362
packet format, 358
process-to-process communication, 359
queuing, 361
services, 359–362
user datagrams, 358–359
user datagrams, 358–359
user interface, 484
user mobile link (UML), 181
UTP. *See* unshielded twisted pair (UTP)

V

Van Allen belts, 177
variable-size framing, 67
VER. *See* version number (VER)
verification categories, entity authentication, 666–667
verification tag (VT), 417, 418–419
Verizon, 152
version number (VER), 221
video, 4, 564–565
band, 161, 162
compression, 564–565
conferencing, 572
digitizing, 564–565
Video on Demand (VOD), 571
virtual local area networks (VLANs), 187, 196–200
advantages, 199–200
automatic configuration, 198–199
communication between switches, 199
configuration, 198–199
frame tag, 199
IEEE standard, 199
manual configuration, 198
membership, 198–199
semiautomatic configuration, 199
table maintenance, 199
time-division multiplexing, 199
virtual private network (VPN), 684–685
virtual work groups, 200
virtual-circuit approach, 207, 253
VLANs. *See* virtual local area networks (VLANs)
voice over IP, 572
VPN. *See* virtual private network (VPN)
VT. *See* verification tag (VT)
vulnerable time
ALOHA, 91
CSMA, 95

W

WAN. *See* wide area network (WAN)
WAN PHY, 126
warning packet, 247
WATS. *See* wide area telephone service (WATS)
wavelength, 32
wavelength-division multiplexing (WDM), 48
WDM. *See* wavelength-division multiplexing (WDM)
Web caching, 462–464
Web Client, 450–451
Web documents, 452–453
active documents, 453
dynamic documents, 452–453
static documents, 452
Web page, 450
Web server, 451
Web-based mail, 480–481
weighted graph, 286
well-known attribute, 318
well-known port numbers, 343–344
well-known ports, 345
wide area network (WAN), 8–10, 65, 116. *See also* local area network (LAN)
wide area telephone service (WATS), 156
wideband CDMA (W-CDMA), 174
WiFi (wireless fidelity), 126–138
WiFi Alliance, 127
window, 380–383
window shutdown, 388
window size, 370
wireless communication, 53–54
infrared waves, 54

wireless communication (*Continued*)
microwaves, 53, 54
radio waves, 53–54
wireless Ethernet, 126–138
wireless LAN, 126–138
addressing mechanism, 133–135
basic service set (BSS), 127
extended service set (ESS), 127–128
internetwork, 10
MAC sublayers, 128–133
point-to-point, 9, 10
station types, 128
switched, 9–10
wireless personal area network (WPAN), 138
wireless WAN, 13
World Wide Web (WWW), 441, 449–453
architecture, 449–450
browser, 450–451
hypermedia, 449
hypertext, 449
Uniform Resource Locator, 451–452
Web document, 452–453
WPAN. *See* wireless personal area network (WPAN)

X

X.509, 674
Xerox CP, 87
Xerox Network System (XNS), 303
XNS. *See* Xerox Network System (XNS)

Y

YCM, 3

Z

zero compression, 257
zone, 490–491

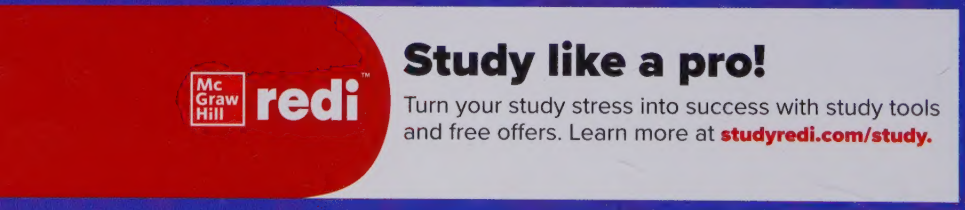
Data Communications and Networking, 6th edition, teaches the principles of networking using TCP/IP protocol suite. It employs a bottom-up approach where each layer in the TCP/IP protocol suite is built on the services provided by the layer below. This edition has undergone a major restructuring to reduce the number of chapters and focus on the organization of TCP/IP protocol suite. It concludes with three chapters that explore multimedia, network management, and cryptography/network security. Technologies related to data communications and networking are among the fastest growing in our culture today, and there is no better guide to this rapidly expanding field than ***Data Communications and Networking***.

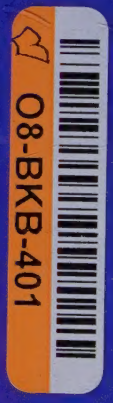
Hallmark Features

- Simple explanation of complex concepts
- More than 500 figures
- More than 400 problems and 300 review questions
- Complete list of key terms and acronyms

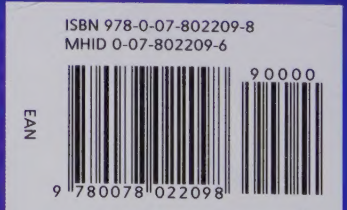
Changes in the Sixth Edition

- Chapters reorganized to focus on the multilayer TCP/IP protocol suite process
- Text streamlined
- One-third of the problems and review questions replaced
- Appendices moved to the book from the Online Learning Center





ISBN 978-0-07-802209-8
MHID 0-07-802209-6



mheducation.com/highered